Introducción a las Redes de Computadoras

Facultad de Ingeniería — Universidad de la República

Instituto de Computación (INCO)

Material de estudio completo — 48 clases

Basado en los exámenes oficiales 1998–2011, Tanenbaum & Wetherall 5ª ed., y actualización a 2026

Índice

1. **Clase 1** — ¿Qué es una red de computadores?
2. **Clase 2** — Los modelos de capas: OSI y TCP/IP
3. **Clase 3** — Encapsulación y comparación OSI ↔ TCP/IP
4. **Clase 4** — Conmutación: circuitos, paquetes y mensajes
5. **Clase 5** — El retardo en las redes de paquetes
6. **Clase 6** — Taller de retardos (todo resuelto)
7. **Clase 7** — Nyquist, Shannon y la capa física + Simulacro 1
8. **Clase 8** — Modelo cliente/servidor y HTTP/1.1
9. **Clase 9** — HTTP a fondo: mensajes, cookies, caché y versiones
10. **Clase 10** — DNS: la base de datos distribuida
11. **Clase 11** — DNS: iterativo vs. recursivo, caché y TTL
12. **Clase 12** — Taller DNS + HTTP (resuelto)
13. **Clase 13** — Correo electrónico: SMTP, POP3, IMAP y MIME
14. **Clase 14** — Sockets: la API de la capa de transporte
15. **Clase 15** — DHCP, configuración de un host y Simulacro 2
16. **Clase 16** — Multiplexación, demultiplexación y UDP
17. **Clase 17** — TCP: la cabecera campo por campo
18. **Clase 18** — TCP: establecer y cerrar la conexión
19. **Clase 19** — Cómo garantiza TCP la fiabilidad
20. **Clase 20** — Control de flujo vs. control de congestión
21. **Clase 21** — Control de congestión en TCP: slow start, AIMD, fast recovery y ECN
22. **Clase 22** — Taller TCP: diagramas de intercambio resueltos
23. **Clase 23** — Repaso de transporte + Simulacro 3
24. **Clase 24** — La capa de red: forwarding y routing
25. **Clase 25** — Conmutación en la red: datagramas y circuitos virtuales
26. **Clase 26** — IPv4: el datagrama y las direcciones
27. **Clase 27** — Subnetting, método paso a paso
28. **Clase 28** — VLSM y planes de numeración
29. **Clase 29** — Taller de subnetting y planes de numeración
30. **Clase 30** — Tablas de enrutamiento, next hop y ARP
31. **Clase 31** — NAT, PAT y sus hacks
32. **Clase 32** — Enrutamiento distance-vector: Bellman-Ford y el conteo a infinito
33. **Clase 33** — Enrutamiento link-state, Dijkstra y OSPF
34. **Clase 34** — BGP y el enrutamiento entre operadores
35. **Clase 35** — Fragmentación, MTU y Path MTU
36. **Clase 36** — IPv6
37. **Clase 37** — ICMP, ping y traceroute · Repaso de red y Simulacro 4
38. **Clase 38** — La capa de enlace: servicios y entramado (framing)

39. **Clase 39** — Detección de errores: paridad, checksum y CRC
40. **Clase 40** — Corrección de errores y ARQ
41. **Clase 41** — Acceso al medio: ALOHA, CSMA/CD y CSMA/CA
42. **Clase 42** — Switch, hub, VLAN y STP
43. **Clase 43** — Redes inalámbricas 802.11
44. **Clase 44** — Multimedia: streaming, RTP y VoIP
45. **Clase 45** — QoS: shaping, policing, DiffServ y MPLS
46. **Clase 46** — Gestión de redes: SNMP, NETCONF y YANG
47. **Clase 47** — Seguridad: servicios y criptografía
48. **Clase 48** — Seguridad aplicada, IP Móvil y Simulacro final

Clase 1

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 Qué es una red y por qué existen
- 1.2 Clasificación por alcance: PAN, LAN, MAN, WAN
- 1.3 La frontera que ya no es el alcance: latencia y propiedad
- 1.4 Topologías
- 1.5 Los dispositivos y —esto es lo que más se pregunta — qué capa implementa cada uno
- 1.6 Los parámetros de una red — la parte más importante de esta clase
- 1.7 Historia mínima: cuatro hitos, y la confusión más común del curso
- 1.8 El problema: un diagrama de red no es un dibujito
- 1.9 Qué información lleva un diagrama de red
- 1.10 Los tipos de diagrama de red
- 1.11 Los seis pasos para hacer un diagrama de verdad
- 1.12 La checklist: ¿mi diagrama está listo?
- 1.13 ⚠️ Actualización a 2026
- 2.1 Ejercicio 1 — Los seis parámetros en un caso concreto
- 2.2 Ejercicio 2 — Velocidad de propagación en cobre vs. fibra
- 2.3 Ejercicio 3 — Por qué la velocidad de propagación NO depende del ancho de banda
- 3.1 Mapa de los problemas de examen del Bloque A
- 3.2 ⭐ Adelanto: la Pregunta 1a del Examen 2009-12 (resuelta aquí, se repite en la Clase 5)
- 📌 Resumen en 10 líneas
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: A (Fundamentos) · **Semana:** 1 · **Duración estimada:** 110 min **Correlativa:** — (primera clase del curso) · **Sigue con:** Clase 2 (los modelos de capas: OSI y TCP/IP) **Material de referencia:** Tanenbaum cap. 1 (pág. ~1 a ~35 del PDF de tu carpeta) · RFC 1122, RFC 8200, RFC 9293, RFC 8200

🎯 Al terminar esta clase vas a poder

1. **Definir** qué es una red de computadores y explicar las tres razones por las que existen (comunicar, compartir recursos, economía de escala), calculando el costo por usuario.
2. **Clasificar** una red como PAN, LAN, MAN o WAN, y explicar por qué la clasificación **moderna ya no se basa en el alcance geográfico** sino en la latencia y en la propiedad.
3. **Dibujar** las cinco topologías clásicas (bus, estrella, anillo, malla, árbol, híbrida) y decir qué se usa hoy y por qué.
4. **Asociar** cada dispositivo (host, NIC, hub, switch, router, punto de acceso, módem, firewall, servidor) con **la capa que implementa**, que es la pregunta que más se hace de esta clase.
5. **Distinguir** sin dudar las seis magnitudes: ancho de banda, velocidad de propagación, latencia, retardo total, throughput y jitter, con sus unidades y de qué dependen.
6. **Explicar por qué la velocidad de propagación no cambia con el medio de enlace ni con el ancho de banda**, que es el error conceptual más caro del curso.
7. **Separar** "Internet" de "Web", y ubicar los cuatro hitos que dan contexto al temario.

> **Prerequisito del examen:** — (clase de soporte). Adelanta la **Pregunta 1a del Examen** `2009-12`, que se resuelve completo en la Clase 5.

PARTE 1 — TEORÍA

1.1 Qué es una red y por qué existen

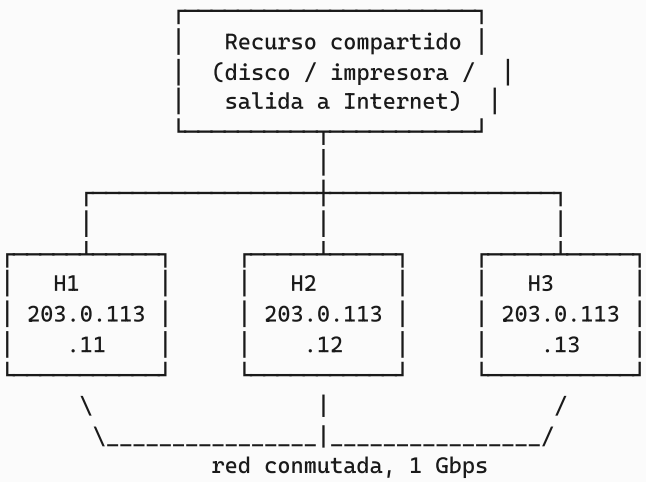
La definición

Tanenbaum define una red de computadores así:

> Un sistema de computadores **interconectados** por una serie de enlaces o nodos, con el fin de **compartir recursos** e **intercambiar información**.

Las tres palabras de esa definición que importan son *interconectados*, *compartir* e *intercambiar*, y las tres se pueden develops por separado:

- **Interconectados**: no alcanza con que dos máquinas estén cerca. Tiene que haber un **medio** (cobre, fibra, aire) y un **dispositivo** que los una (una NIC, un switch). Un montón de equipos sueltos en una habitación no son una red: son un montón de equipos sueltos.
- **Compartir recursos**: una impresora, un disco de 4 TB, una conexión a Internet. El recurso es **físico y único**, y muchos lo usan.
- **Intercambiar información**: no solo datos de la aplicación; también información de control (quién tiene la dirección de quién, qué rutas existen, cómo se negocia un enlace).



Las tres razones, en serio

Razón 1 — Comunicarse. La razón original. La red existe porque dos personas que están a 400 km de distancia necesitanomiplir el mismo documento, negociar, o curbajo. Antes de la red eso significaba carta (días), teléfono (no escribís), o ir (horas).

Razón 2 — Compartir recursos Caro. Un servidor de archivos de 200 TB cuesta lo mismo para 5 usuarios que para 500. Lo que cuesta es el **software** y el **almacenamiento**, no el número de personas que lo leen. Una impresora láser de color cuesta 600 dólares y no tiene sentido tener una en cada escritorio: una sola, en red, y que todos la usen.

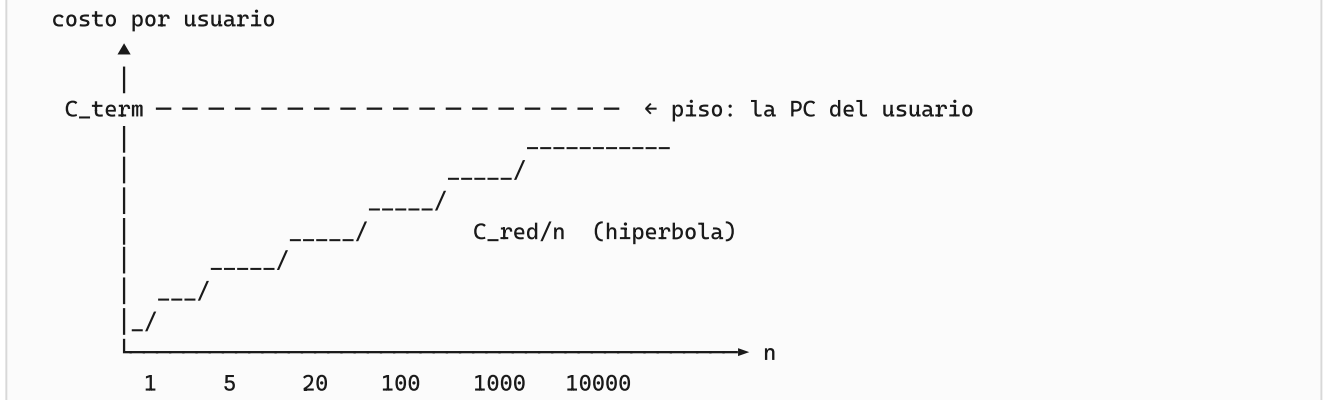
Razón 3 — Economía de escala. Esta es la que casi no se explica y es la que justifica todo. Definamos:

```
C_red = costo de poner la red (cableado, switches, NICs, configuración)
n     = cantidad de usuarios conectados
C_term = costo del terminal por usuario (la PC, que de todas formas
         ibas a tener)
```

El costo total es `C_total(n) = C_red + n · C_term`. El **costo por usuario** es:

```
C_usuario(n) = C_red / n  +  C_term
               └──┬──┘     └──┬──┘
            se reparte entre   no depende
              todos los n        de n
```

El primer término **decrece** con `n`; el segundo es constante. Por lo tanto el costo total por usuario **baja al aumentar** `n` y tiene un piso en `C_term`.



Por qué esto importa en redes: es la razón por la que **comprar una red grande es más barato por usuario que una red chica**, y la razón por la que las ISP te cobran **por ancho de banda y no por usuario**: cuanto más clientes, más usuarios detrás de la misma capacidad, más rentable el enlace.

1.2 Clasificación por alcance: PAN, LAN, MAN, WAN

El clasificación clásico, el de cualquier libro de texto de los 90, ordena las redes por **distancia geográfica**. Funcionaba, y todavía se usa como primera aproximación. Pero, como vamos a ver en §1.3, **ya no alcanza**.

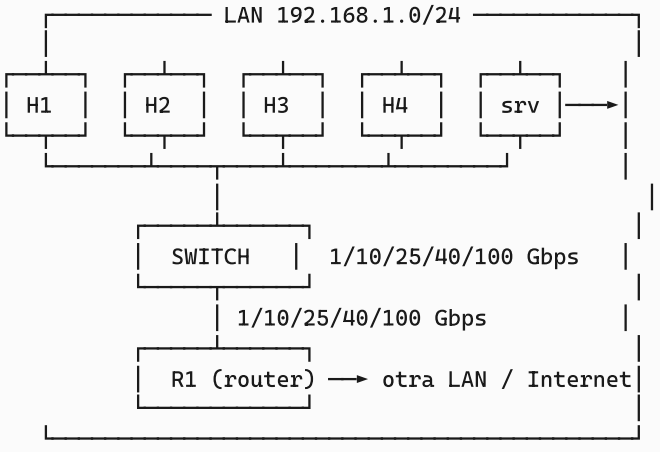
Tipo	Significado	Alcance típico	Velocidad	Propiedad
PAN	*Personal Area Network* — Red de área personal	~**10 m** (Bluetooth), 1 m (cable)	1 a 3 Mbps (Bluetooth)	**Privada**, siempre de una sola persona
LAN	*Local Area Network* — Red de área local	**Hasta ~1 km** (un edificio, un piso, un campus)	**1 Gbps a 100 Gbps** hoy; conmutada	**Privada**, de una organización
MAN	*Metropolitan Area Network* — Red de área metropolitana	**Una ciudad** (~10 a 50 km)	1 a 10 Gbps (fibra pública)	**Pública** o departmental (un operador de ciudad)
WAN	*Wide Area Network* — Red de área amplia	**Países, continentes, el planeta**	1 Gbps en el backbone, pero **decenas de ms de latencia**	**Mixta**: los backbone son de operadores, el último tramo es de cada uno

PAN — lo que tenés en el bolsillo

```
[ Auriculares ]
      |  Bluetooth / 802.15.1
[   Celular   ] ~~~~~~~~~ 2,4 GHz ~~~~~~ [ Auriculares ]
      |  (alcance nominal ~10 m, clase 2)
[  Notebook  ]
```

- **Bluetooth (IEEE 802.15.1)**: clase 2 = ~**10 m**, ~1 Mbps (BR) a 3 Mbps (EDR); con **Bluetooth Low Energy** las tasas caen a kilobits para los sensores.
- **NFC**: 4 cm. No es una red en el sentido clásico: es un enlace de un solo sentido de contacto.
- **IEEE 802.15.4 / Zigbee**: ~10 a 100 m, ~250 kbps, pensado para redes de **muchos** nodos de bajo consumo (medidores, domótica).

LAN — la red conmutada del edificio o del campus



Lo que define una LAN moderna no es solo la distancia: es que es **conmutada**, con lo que desaparece el **dominio de colisión** (cada host tiene el medio para él) y cada puerto es un dominio de colisión y de broadcast separados.

MAN — la ciudad

El ejemplo canónico fue el **anillo metropolitano de FDDI** (1980s-90s, token ring de fibra a 100 Mbps) y hoy son redes metropolitanas de fibra pública y, en parte, wireless de banda ancha (WiMAX, 5G fixed). La MAN se distingue de la LAN en que **el equipo y la propiedad ya no son los tuyos**: el operador de la ciudad te alquila un puerto.

WAN — la red del planeta

Internet **es** el ejemplo canónico de WAN. Lo que lo delata, y lo vamos a ver con números en §1.6, es la **latencia**: un RTT típico en Internet está entre 20 ms (dentro del mismo datacenter o la misma ciudad) y 200 ms (entre continentes). Eso es de 3 a 4 órdenes de magnitud por encima de una LAN, y es exactamente la razón por la que los protocolos de Internet están diseñados como están (retransmisiones, control de congestión, ventanas grandes).

1.3 La frontera que ya no es el alcance: latencia y propiedad

Este es el matiz importante, y el que hace que la clasificación clásica quede corta. La definición de LAN/WAN "por distancia" falla con dos contraejemplos que existen **hoy**:

Contraejemplo 1 — un enlace por satélite es "LAN" en distancia y "WAN" en todo lo demás. Un satélite geoestacionario está a 36.000 km de altura. Un enlace de 5 metros entre dos racks está a 5 metros. Sin embargo:

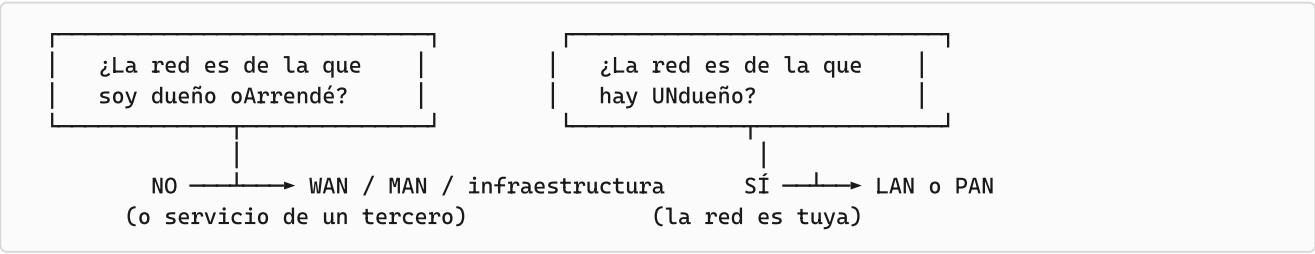
Medida	Enlace de 5 m	Satélite geoestacionario
Distancia del extremo	2,5 m	36 000 km
Velocidad de propagación	~2·10⁸ m/s	~3·10⁸ m/s (espacio)
Retardo de propagación (d/s)	12,5 ns	~250 ms ⇦ ida
RTT mínimo	~25 µs	~500 ms
Latencia comparada	1×	20 000×

500 ms de RTT es la latencia de una WAN. Y el teléfono tiene 100 ms de latencia humana y se percibe instantáneo. Con 500 ms de ida y vuelta, una conversación por voz es **imposible** sin que se note el retraso. El satélite deellite es el ejemplo perfecto de por qué la latencia es el criterio, no la distancia.

Contraejemplo 2 — una WAN dentro de una ciudad. Los operadores montaron *dark fiber* (fibra oscura: la fibra tendida, sin luz, alquilada) entre locales de una misma ciudad, o incluso entre dos puntos de la misma ciudad, y con eso una latencia de **menos de 1 ms** a 10 o 40 Gbps. Eso es, con todas las letras, **una WAN con mejor desempeño que cualquier LAN de 2005**. La distancia es de 5 km y el desempeño es de LAN.

Contraejemplo 3 — un PAN lento. Bluetooth tiene ~10 m pero las WAN de fibra de larga distancia tienen una latencia de 2 ms. Alguien con un auricular Bluetooth en la oreja tiene más latencia que alguien con un enlace metro de 10 Gbps.

El criterio correcto



El criterio que **sí** funciona en 2026 es doble:

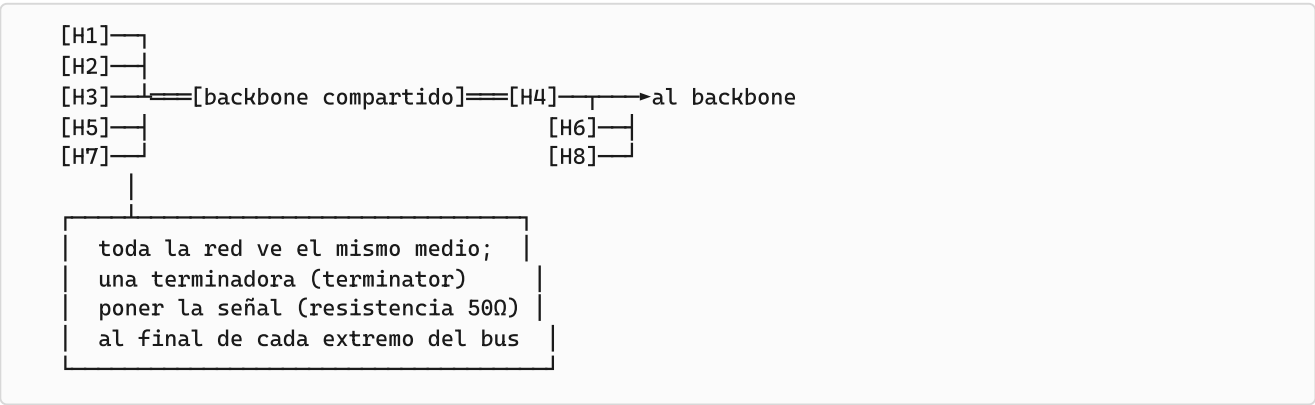
	latencia	**propiedad**
LAN típica	**< 1 ms** (0,1 a 1 ms)	tuya (una organización)
MAN típica	**1 a 5 ms**	del operador municipal o del campus
WAN típica	**5 a 250+ ms**	de uno o varios operadores
PAN	**< 10 ms** (casi 0)	tuya, de una persona

Y la definición operativa que conviene usar: **una LAN es una red que no atraviesa un router que no es tuyo.** Si para ir de A a B pasás por un router de un operador, es una WAN para vos, por corta que sea la distancia.

1.4 Topologías

Topología = **la forma física del cableado**. Se estudia porque cada forma tiene un punto único de falla, una costo de cableado y una complejidad de administración diferente.

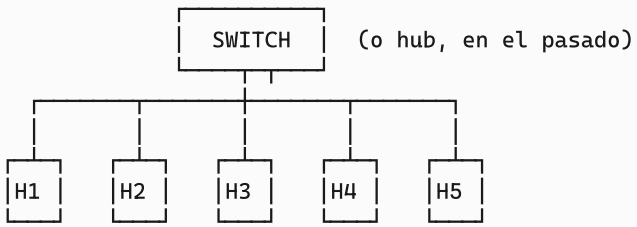
(a) Bus — un solo medio compartido



Ventajas	Desventajas
Cabledado mínimo: un solo cable recorre toda la red	**Un solo punto de falla**: si se corta el backbone, se cae **toda** la red
Barato	**Dominio de colisión único**: todo el tráfico se ve; el rendimiento cae con `n`
Fácil de extender	Colisiones, lo que obliga a CSMA/CD (ver Clase 41)
	Difícil de diagnosticar

Dónde se usa hoy: en las **redes Wi-Fi**: la topología lógica de un punto de acceso es un bus. Todos los clientes comparten el mismo medio y compiten por él.

(b) Estrella — todo al centro



Ventajas	Desventajas
Fallo aislado: si se cae una rama, se cae un host, no la red	**El switch es el punto único de falla** (se mitiga con *stacking* o enlaces redundantes)
Un dominio de colisión por puerto: rendimiento no cae con `n`	**Cableado**: un cable por host hasta el centro
Agregar un host es trivial: un cable a un puerto libre	
Fácil de diagnosticar: el puerto caído dice qué host es	
Fácil de segmentar en VLAN	

Dónde se usa hoy: toda LAN conmutada, todo *datacenter*, todoARM conmutado. Es la topología dominante sin discusión.

(c) Anillo — un lazo

Ventajas	Desventajas
Retardo predecible: número fijo de saltos, sin colisiones	**Un punto único de falla**: se corta el anillo, se cae la red
Funciona bien con tráfico uniforme y alta carga	**Complejidad**: hay que negociar quién transmite (token)
No hay colisiones: el acceso está arbitrationado	Añadir un nodo es más difícil que en estrella

Dónde se usa hoy: el anillo clásico como topología física **murió**. Sobrevive en dos formas:

- **Metro ring / SONET / SDH**: los operadores montan anillos de fibra para protegerse: si se corta un tramo, el anillo se "cierra" por el otro lado en milisegundos. Eso es protección, no eficiencia.
- **Token Ring (802.5)** y **FDDI**: desaparecieron por completo. Se estudian en las clases 40 y 41 como ejemplo histórico del acceso al medio.

(d) Malla — todos con todos

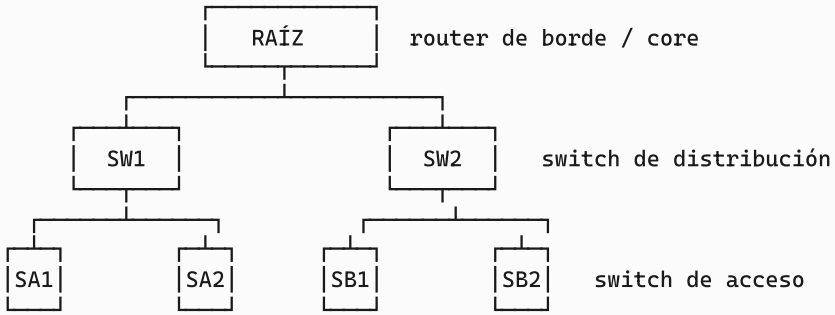
(malla completa: 6 enlaces, 3 nodos, cada par unido)

Ventajas	Desventajas
Tolerancia a fallos total: si cae un enlace, hay camino alternativo	**Costo**: en una malla completa de `n` nodos hay `n(n-1)/2` enlaces
Retardo mínimo: siempre hay un camino corto	Complejidad de administración: `n²` vecinos que conocer
Se puede repartir la carga	

Dónde se usa hoy:

- **Malla completa** solo con `n` muy chico (2 o 3 nodos): por ejemplo el triángulo de **3 routers de borde de datacenter**, o el router de borde, el de spine y el de borde del otro lado.
- **Malla parcial** en el backbone de los operadores: cada router conecta a **varios** proveedores, no a uno. Es la base de la **política de BGP** (un AS con 3 o 4 eBGP en lugar de uno).

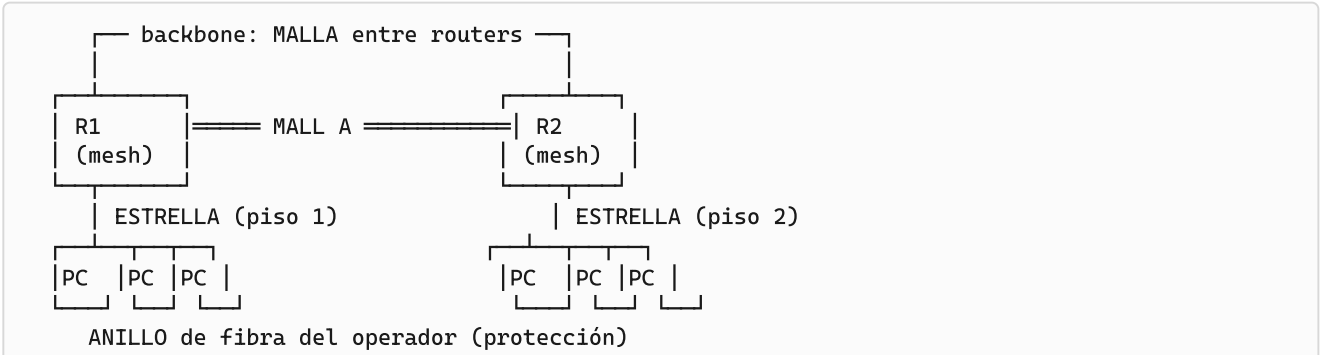
(e) Árbol — jerarquía



Ventajas	Desventajas
Escalable: agregás una hoja y no tocás el centro	El backbone es **punto único de falla**
Jerarquía natural =Coara de administración	RAID: un router de backbone lento es un cuello de botella para todo
Fácil de segmentar por departamento o por piso	

Dónde se usa hoy: todo el *datacenter* y toda la red campus de una universidad grande. Es la estructura de 3 niveles (*access* → *aggregation* → *core*) estándar.

(f) Híbrida — lo que realmente se instala



La topología híbrida es la respuesta a todos los defectos de las otras cinco:

- La **malla** da redundancia donde la redundancia importa (el backbone): el costo se paga en pocos enlaces.
- La **estrella** da rendimiento y facilidad de diagnóstico donde hay muchos usuarios (el acceso).
- El **anillo** da protección donde el medio es caro y no lo controlás (la fibra del operador).

Para el examen: si te preguntan "¿qué topología usa Internet?", la respuesta correcta es **una malla parcial de routers, con los enlaces de los operadores aggregating en puntos de intercambio (IXP)**, y **estrella hacia abajo** en cada red de acceso.

1.5 Los dispositivos y —esto es lo que más se pregunta— qué capa implementa cada uno

El error clásico es decir "el switch es de capa 3" o "el router hace cosas de capa 2". La tabla que hay que tener en la cabeza es esta:

Dispositivo	Función	Capa OSI	Capa TCP/IP	Sabe de IP	Sabe de puertos TCP
Repetidor (*repeater*)	Regenera la señal eléctrica	**1** Física	Acceso a red	no	no
Hub	Conmutador eléctrico de capa 1: repite a todos los puertos	**1** Física	Acceso a red	no	no
Módem / DTU	Modula una señal digital sobre el medio analógico	**1** Física	Acceso a red	no	no
NIC (*network interface card*)	Transmite y recibe tramas en un medio	**1 y 2** (Física + Enlace)	Acceso a red	no	no
Punto de acceso (AP, Wi-Fi)	Traduce entre medio inalámbrico y Ethernet; hace association, autenticación, cifrado (WPA3)	**2** (Enlace) — y a veces 3 para translated LAN	Acceso a red / red	a veces	no
Switch / *bridge*	Reenvía tramas por la **tabla MAC**; aprende direcciones; dominio de colisión por puerto	**2** Enlace de datos	Acceso a red	**no**	no
Router / *gateway* de red	Reenvía paquetes por la **tabla de reenvío IP** (longest prefix match)	**3** Red	Internet	**sí**	no
Firewall	Filtra por IP, por puerto, y (si es *stateful*) por conexión	**3 y 4** (Red y Transporte)	Internet / Transporte	sí	**sí**
Servidor / host	Corre las aplicaciones y las implementa	**todas** (1 a 7)	todas	sí	sí

Las tres filas que más se confunden

El switch NO sabe de IP. Un switch de capa 2 mira el **MAC destino** de la trama, busca en su tabla de reenvío y copia la trama al puerto correspondiente. No le importa si el paquete IP que va adentro dice `8.8.8.8` o `203.0.113.9`. Por eso un switch **no** reemplaza a un router: si le pedís que te lleve a otra red, no puede, porque no sabe qué redes existen. Esa es la Pregunta 2 del Examen 2009-02, y la resolvo en la Clase 3.

El router NO sabe de puertos. Un router Cisco de los de antes miraba el campo *protocolo* de la cabecera IP (TCP=6, UDP=17, ICMP=1) para decidir si podía hacer *NAT* o *policing*. Pero no miraba el puerto TCP. Esa es la razón técnica de por qué la NAT rompe el aislamiento de capas (Clase 31), y la Pregunta 4 del Examen 2011-02 la developa.

El hub es de capa 1, no de capa 2. Parece un switch ("tiene puertos"), pero no aprende direcciones y no decide: copia la señal a **todos** los puertos. Por eso todos los hosts comparten un único dominio de colisión. Esa distinción es la Pregunta 3 del Examen 2010-12 (Clase 42).

1.6 Los parámetros de una red — la parte más importante de esta clase

Acá está el contenido que más se confunde en el examen y que hay que tener **absolutamente claro**. Son seis magnitudes, y **casi todas se miden en unidades distintas**.

La tabla madre

Magnitud	Símbolo	Unidad	Qué es
Ancho de banda	B, R (bandwidth)	bps	CAPACIDAD del enlace
Vel. de propagac.	s	m/s	RAPIDEZ de la SEÑAL
Latencia		ms	tiempo de un viaje
Retardo total	d_end2end	ms	4 componentes sumados
Throughput		bps	bits REALES por segundo
Jitter	Δl	ms	VARIACIÓN de la latencia

1.6.1 Ancho de banda (*bandwidth*)

Es la **capacidad máxima** del enlace, en **bits por segundo** (bps). Define cuánto se puede mandar **como máximo**, no cuánto se manda.

- Es una propiedad del **equipo y del medio**: del transceptor, del puerto, de la modulación.
- **Cambia** con la tecnología: Ethernet pasó de 10 Mbps a 100 Mbps a 1 Gbps a 10 Gbps a 25/40/50/100 Gbps a 400/800 Gbps. Un enlace de 1 Gbps **no puede** mandar 2 Gbps, por más que el cable sea mejor.
- Lo ves en el rótulo de la interfaz: `1000BASE-T`, `10GBASE-SR4`, `100GE`.
- También se usa el mismo término en dos sentidos distintos: "**ancho de banda**" (bps) y "**ancho de banda**" (Hz). **Esto es una fuente perenne de confusiones.** En el contexto de Nyquist y Shannon (Clase 7) el ancho de banda está en **hercios**, no en bits por segundo.

1.6.2 Velocidad de propagación (*propagation speed*)

Es la **rapidez con la que viaja la señal por el medio**, en **metros por segundo**. Los valores que hay que tener:

Medio	Velocidad de propagación
Vacío (espacio libre)	$c = 3 \cdot 10^8$ m/s (exacta, es la constante de la luz)
Aire	$\sim 2{,}998 \cdot 10^8$ m/s (un 0,07 % menos que el vacío)
Cobre (UTP/STP, coax)	$\sim 2 \cdot 10^8$ m/s (un 33 % menos)
Fibra óptica (silicio)	$\sim 2 \cdot 10^8$ m/s (un 33 % menos)
Conductor eléctrico típico	~0,6 a 0,7 · c

> ⚠️ **Ojo con un detalle que aparece en la solución oficial del Examen `2009-12`:** ahí dice "*$s \sim 2 \times 10^8$ m/s en cobre*". Es correcto **para el cobre**. Si algún apunte dice "2×10^8 en el vacío", es un error: en el vacío la luz va a 3×10^8 m/s. El número que se usa en los problemas de examen es **2×10^8 m/s para cobre y fibra**, y **3×10^8** si el problema habla de un enlace por satélite o por el espacio.

Lo que la velocidad de propagación NO es: una capacidad. **No tiene nada que ver con la velocidad a la que se mandan los datos.**

La tabla que responde la Pregunta 1a del Examen 2009-12

Esta tabla es la respuesta literal a "*Señale las diferencias entre el ancho de banda y la velocidad de propagación en el medio*". Memoriala como está:

	Ancho de banda	**Velocidad de propagación**
Símbolo	`B` o `R`	`s`
Unidad	**bps** (bits por segundo)	**m/s** (metros por segundo)
Qué mide	**Cuántos bits** entran por segundo	**A qué velocidad viaja la señal**
De qué depende	Del **equipo**: transceptor, puerto, modulación	Del **medio físico**: la física de la propagación
Cambia al cambiar el medio	**Depende**: 10BASE-T vs 1000BASE-T vs fibra	**No**: es ~2×10^8 m/s tanto en cobre como en fibra
Cambia con la distancia	**No**	**No** (la distancia va en el numerador de `d/s`, no en `s`)
Tiene unidad de longitud	No	**Sí**
Relación con el retardo	Entra en `d_transmisión = L / B`	Entra en `d_propagación = d / s`
Ejemplo	"La línea tiene 1 Mbps"	"En ese cable la señal va a $2\cdot10^8$ m/s"

Una frase que resume la diferencia y que podés usar en el examen:

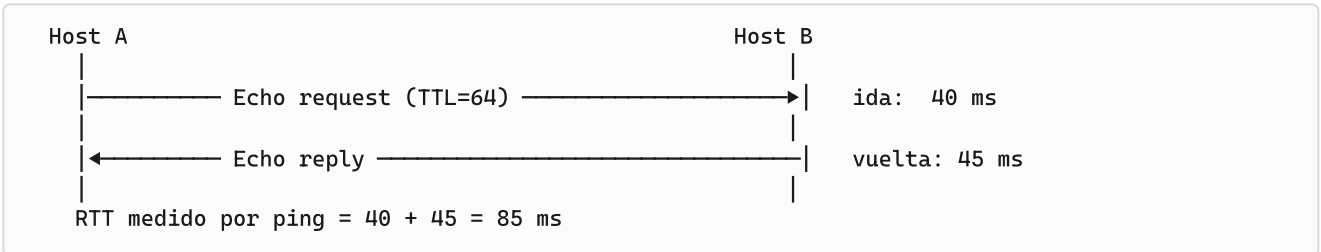
> El **ancho de banda** es una propiedad del **enlace** (cuánta información cabría por segundo, y depende del equipo); la **velocidad de propagación** es una propiedad del **medio** (a qué velocidad física viaja la señal, y depende de la física del medio). **Son ejes perpendiculares**: podés tener un enlace de 100 Mbps por cobre (BW alto, `s` de cobre) o de 100 Mbps por fibra (mismo BW, mismo `s` porque la fibra también da ~2×10^8 m/s).

1.6.3 Latencia

Es el **tiempo que tarda un paquete en ir de A a B**, medido en milisegundos. Es unTIEMPO, no una tasa. Y depende de:

- la distancia,
- el número de saltos (routers) y su velocidad de procesamiento,
- la congestión de la red (cuánto esperó en las colas),
- los enlaces que usa.

Latencia de ida y vuelta (`RTT`, *Round Trip Time*): el tiempo que tarda un paquete en ir y volver. Es **el doble de la latencia mínima**, y se mide con `ping`:



Latencia no es RTT. Si la latencia de ida es 40 ms, la latencia es 40 ms y el RTT es 80 ms (más el procesamiento). Es el error que aparece en el Ejercicio 2.4.

1.6.4 Retardo total (*end-to-end delay*)

Es la **suma de cuatro componentes** que vas a estudiar en detalle en la Clase 5:

```
d_end2end = d_proc + d_cola + d_transmisión + d_propagación
             \___/     \___/     \__________/     \___________/
            varío      variable   L / R              d / s
            según      según       (NO depende        (NO depende
            el nodo    la cola     del medio)          del paquete)
```

Solo lo dejo enunciado acá; toda la aritmética va en las Clases 5 y 6.

1.6.5 Throughput (rendimiento)

Es la **cantidad real de bits por segundo que pasan**, medido durante un intervalo. Siempre cumple:

```
throughput  ≤  ancho de banda
```

Y es estrictamente menor por cuatro razones:

```
throughput = ancho de banda × eficiencia

eficiencia = (bytes de datos útiles) / (bytes totales en el medio)
           = 1 − overhead_de_cabeceras − retrASMisiones − colisiones
```

Ejemplo con la aritmética:

```
Enlace de 1 Gbps = 1 000 000 000 bps
Overhead de cabeceras (Ethernet 18 + IP 20 + TCP 20 = 58 bytes por cada
paquete de 1500 bytes de datos) + Retransmisiones (0,5 %) + Colisiones (0)

eficiencia = (1500 − 58) / 1500 = 1442/1500 = 0,9613
0,9613 × 1 000 000 000 = 961 000 000 bps

→ El throughput máximo de una conexión TCP sobre 1 GbE es
  ~961 Mbps, NO 1000 Mbps.
```

1.6.6 Jitter

Es la **variación de la latencia entre paquetes del mismo flujo**. Se calcula como la desviación (o la diferencia entre máximo y mínimo) de los retardos de paquetes que deberían haber llegado todos juntos.

```
Tres paquetes del mismo flujo, enviados con 20 ms de separación:

Paquete 1: 80 ms
Paquete 2: 95 ms
Paquete 3: 110 ms

jitter = 110 − 80 = 30 ms
```

Por qué importa (Clase 44 lo desarrolla): en voz y video, **no importa** el retardo absoluto (100 ms de latencia en una voz se notices en el tono, pero no lo rompe); lo que rompe es que **lleguen en ráfagas**. Un códec de audio reproduce cada 20 ms: si el paquete 1 tarda 80 ms y el 2 tarda 110, el buffer se drains dos muestras de golpe. Eso se oye como un **clique**. Por eso RTP lleva **número de secuencia y timestamp**: para detectar, ordenar y rellenar.

1.7 Historia mínima: cuatro hitos, y la confusión más común del curso

Los cuatro hitos

```
Año  Evento
──────────────────────────────────────────────────────────────────────
1969 ARPANET: se conecta el primer enlace entre el UCLA (IMP #1) y el
     SRI (IMP #2). Cuatro nodos en 1971. Cuatro IMP, 50 kbps,
     conmutación de paquetes desde el día uno (Packet Switching was
     the original design, not an afterthought).
1983 1 de enero: ARPANET adopta TCP/IP como único protocolo.
     "Flags Day": los hosts que seguían con NCP se apagan.
     → TCP/IP no es "el protocolo que se eligió después": es el
     que quedó, y la razón fue la interoperabilidad.
1991 Tim Berners-Lee publica el primer sitio Web (info.cern.ch) en
     agosto de 1991. La Web es una APLICACIÓN de Internet.
1996 Se abren los primeros ISPs comerciales. Internet pasa de
     proyecto de investigación a servicio.
──────────────────────────────────────────────────────────────────────
2007 El iPhone. El teléfono pasa de ser un cliente de Internet a ser
     el mayor generador de tráfico. La latencia y el consumo de
     batería se vuelven métricas de primera clase.
```

Internet NO es la Web — el error conceptual más común

Esta confusión se paga en los exámenes, y es una de las cosas más básicas que hay que tener claras:

```
┌──────────────────────────────────────────────────────────────────┐
│                                                                  │
│   INTERNET = la RED. La infraestructura.                         │
│              Un conjunto de redes interconectadas.               │
│              Protocols: IP, TCP, UDP, ICMP, BGP, ...             │
│              Identifica COMPUTADORES por dirección IP.           │
│                                                                  │
│   LA WEB = una APLICACIÓN que corre sobre Internet.              │
│            Protocols: HTTP, HTML, URL, cookies.                  │
│            Identifica DOCUMENTOS por URL.                        │
│                                                                  │
│   Analogía: Internet es elCorreo. La Web es una carta.           │
│   El correo también transporta telegramas, llamadas, paquetes    │
│   y fotos: esas son otras aplicaciones, no "el correo".          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Prueba de que son cosas distintas: la Web habla **HTTP sobre TCP sobre IP**. Si mañana nadie usa HTTP, la Web muere y **Internet sigue funcionando**: el correo (SMTP), el DNS, el VoIP (RTP), la mensajería, los juegos,

SSH, todo sigue andando. **Internet sin Web es una red perfectamente válida.** Al revés no: **Web sin Internet es imposible**, porque la Web está definida encima de IP.

Lo que cambió: HTTP/3 (RFC 9114, 2022) ya no corre sobre TCP sino sobre **QUIC sobre UDP**, pero sigue corriendo sobre **IP**. La separación sigue exacta: **Internet es la red, la Web es una aplicación**.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - Los cuatro hitos (1969, 1983, 1991, 1996) y el hecho de que **Internet ≠ Web**. - Los cuatro componentes del retardo y sus fórmulas (`L/R` y `d/s`). - La diferencia entre ancho de banda (bps) y velocidad de propagación (m/s). - Los valores de `s` : ~2×10^8 m/s en cobre y fibra, ~3×10^8 en vacío/espacio. - El criterio de clasificación PAN/LAN/MAN/WAN por alcance, como **primera aproximación**.

Cambió o se modificó:

- **IPv6 dejó de ser "el futuro".** El **28 de marzo de 2026** el acceso a Google por IPv6 **superó el 50 % por primera vez (50,10 %)**. APNIC midió ~42-43 % (cobertura de capacidad) y Cloudflare ~40 % (tráfico por bytes). Es desigual: Francia ~73 %, India ~72 %, Arabia Saudita ~65 %; Italia ~17 %, España ~10 %, Egipto ~4 %; China <5 %. ⇒ **Dato para el aula:** en 2012 el curso decía "IPv6 llega en 10 años". La respuesta correcta en 2026 es "ya pasó la mitad en los usuarios de Google, pero es desigual, China sigue en <5 %, y la cifra depende de **qué** midas".
- **El agotamiento de IPv4 es un hecho consumado.** RIPE NCC hizo su **última asignación /22 el 25-nov-2019**. APNIC se agotó en abr-2011, ARIN en sept-2015, LACNIC en 2020, AfriNIC en abr-2017. Hoy el default de la capa 3 de acceso es el **CGNAT (*Carrier-Grade NAT*)**, que es una WAN con muchísimos clientes detrás de pocas IP públicas.
- **La capacidad de los enlaces de acceso se multiplicó.** Un portátil de 2026 se conecta a 1 Gbps por cable y a Wi-Fi 7 (IEEE 802.11be, aprobado en sept-2024, publicado como **802.11be-2024** el 22-jul-2025) con **320 MHz de canal y 4096-QAM** en la banda de 6 GHz. En los backbone de los operadores se ven **100, 400 y 800 Gbps**, y **el 10GbE+ es full-duplex obligatorio** (decisión de IEEE 802.3, no una RFC).
- **La clasificación "LAN/WAN" se usa menos y la de latencia más.** Los *datacenters* de hoy son "LAN" de 400 Gbps a 200 µs; las redes de acceso 5G tienen 10 ms de latencia y 1 Gbps, que es un perfil "WAN"; el satélite sigue en ~500 ms de RTT.

> **Consecuencia para el examen:** si te preguntan por la clasificación o por los parámetros, **respondé con los conceptos**, no con los números de capacidad. Los números de bps envejecen mal; los conceptos ("el ancho de banda es del equipo, la velocidad de propagación es del medio") no.

PARTE 1 bis — LOS DIAGRAMAS DE RED: qué son, qué información llevan y cómo se hacen

> **Por qué esta sección va en la Clase 1 y no al final.** Un diagrama de red es **el lenguaje escrito de la materia**: en un examen te pueden pedir "*dibuje la topología y complete la tabla de ruteo*", y en la vida profesional es el documento con el que un administrador se comunica con el resto del equipo, con el proveedor, y con el cliente. Si no sabés **qué información va** en un diagrama, no sabés ni leer el de otro ni escribir el tuyo.

1.8 El problema: un diagrama de red no es un dibujito

La pregunta que siempre llega es "*¿qué pongo en el diagrama?*". La respuesta corta es: **depende del tipo de diagrama**. Y hay un segundo problema, más sutil y que en tu imagen quedó escrito con todas las letras: **"VALIDACIÓN PENDIENTE"**.

```
Lo que pasa en la práctica

1. Alguien diseña la red y dibuja el diagrama IDEAL.
2. Se instala.
3. La realidad se desalinea: una VLAN se agranda, un firewall cambia
   una regla, un balanceador saca un servidor de la rotación,
   un puerto pasa a "depreciado" y nadie actualiza el dibujo.
4. El diagrama queda MENTIROSO.
5. Un incidente a las 3 de la mañana se investiga contra un dibujo
   que dice cosas que ya no son ciertas.
```

> **El principio que hay que llevarse:** *un diagrama sin fecha, sin fuente y sin estado de verificación es una copia, no un documento.* Por eso la información de **estado** (§1.8.2) es tan importante como la de **topología** (§1.8.1): no alcanza con dibujar *qué hay*, hay que dibujar *qué está pasando con eso*.

1.9 Qué información lleva un diagrama de red

La respuesta que se da en cualquier curso de gestión, y la que hay que saber escribir, es una lista de **ocho bloques de información**. Vamos uno por uno.

Bloque 1 — Identidad del elemento

```
Para CADA activo del diagrama:

· Nombre formally legible.  NO "el switch de la 3".
                        SÍ  "sw-acceso-p3-01"
· Marca y modelo.           Cisco C9300-48P
· Rol o función.            "Distribución — VLANs de usuarios y servidores"
· Ubicación física.         Rack B4, U24, patch panel PP-A-12
· ID de inventario / tag.   "ASSET-2019-0442"  ← el que usás para
                             relacionarlo con el inventario, con el
                             sistema de tickets y con el activo en el CMDB
· Responsable.              "NOC-FING / persona a cargo"
```

> **Por qué el "nombre formal" importa tanto:** es lo que te permite **buscar** el equipo en el sistema de inventario, en el CMDB, en el portal de backups y en los tickets. Un diagrama con nombres informales es un diagrama **inutilizable** a la hora de un incidente, que es la única hora en la que lo vas a necesitar.

Bloque 2 — L3: direcciones IP, máscara/CIDR y gateway

Este es el bloque **más frecuentemente incompleto** en diagramas caseros, y el que más se pide en los ejercicios de examen:

```
┌──────────────┬──────────────────┬──────────────────┬──────────────────┐
│  Elemento    │  Dirección IP    │ Máscara/CIDR     │  Gateway         │
├──────────────┼──────────────────┼──────────────────┼──────────────────┤
│ sw-acceso-   │ 192.168.10.1     │ /24              │ — (es switch)    │
│  p3-01 (SVI) │                  │ 255.255.255.0    │                  │
├──────────────┼──────────────────┼──────────────────┼──────────────────┤
│ srv-dns-01   │ 192.168.10.53    │ /24              │ 192.168.10.1     │
├──────────────┼──────────────────┼──────────────────┼──────────────────┤
│ rtr-borde-01 │ 192.168.10.254   │ /24              │ — (es router)    │
│              │ 200.1.1.1        │ /30              │ —                │
│              │ (doble pila)     │                  │                  │
└──────────────┴──────────────────┴──────────────────┴──────────────────┘

★ En un router van TODAS sus interfaces, con su CIDR.
★ En un host va su IP + su gateway por defecto.
★ En un switch L3 va la SVI (la IP de la VLAN), no la de las tramas.
```

Lo que hay que saber sobre el CIDR en un diagrama:

- Escribí **las dos formas**: `/24` **y** `255.255.255.0`. La comunidad técnica usa el prefijo; los sistemas de gestión y algunos firewalls lo piden en notación de puntos.
- En un router, el `/30` entre dos routers indica **2 direcciones útiles** (`200.1.1.0/30` → `.1` y `.2` útiles, `.0` red y `.3` broadcast).
- **Las VLAN se numeran y se nombran.** En un diagrama: `VLAN 10 = USUARIOS` (nombre formal), no "la de los usuarios".

Y el dato que casi nadie pone y que es el más útil de todos: el rango de DHCP de cada VLAN. Es lo que te dice qué IPs se van a|Asignar y cuáles quedan reservadas para servidores, y es lo primero que se agota.

```
VLAN 10 USUARIOS    192.168.10.0/24
     gateway .1
     DHCP:  .100 – .240        ← 141 direcciones
     fijas:  .2 – .50  (servidores e impresoras)
     libre:  .51 – .99  y  .241 – .254
```

Bloque 3 — L2: VLAN, spanned ports, trunk y agregación

```
VLAN 10 USUARIOS     ── puertos de acceso 1–24
VLAN 20 SERVIDORES   ── puertos de acceso 25–40
VLAN 30 VOZ          ── puertos de acceso 41–48, PoE
VLAN 99 GESTIÓN      ── puerto 48 (management), OLTROs
──────────────────────────────────────────────
Trunk (802.1Q):      uplink al switch de distribución
                     etiqueta 10,20,30,99
──────────────────────────────────────────────
EtherChannel/LAG:    port-channel 1  (uplinks 49–50)
spanning-tree:       cuál puerto es raíz (root port)
                     cuáles sonAlternate (bloqueados)
```

Y el dato que hace útil un diagrama de switches: qué puertos están bloqueados por STP. Es la causa número uno de "la red anda lenta en ese sector". Un diagrama de STP vale oro.

Bloque 4 — Reglas de firewall

Acá es donde los diagramas se vuelven verdaderamente útiles, y donde la mayoría de los equipos **no documenta**:

```
┌──────────────────────────────────────────────────────────────┐
│  POLÍTICA DE FILTRADO – fw-borde-01                          │
│  (ordenada: la primera que matchea, gana)                    │
├─────┬──────────┬──────────┬──────────┬──────────┬─────────────┤
│  #  │ Origen   │ Destino  │ Puerto   │ Acción   │ Estado      │
├─────┼──────────┼──────────┼──────────┼──────────┼─────────────┤
│ 10  │ 10.0/8   │ any      │ –        │ ACCEPT   │ activa      │
│ 20  │ 192.168.10.0/24 │ 10.0.0.0/8 │ 443/TCP │ ACCEPT │ activa │
│ 30  │ 192.168.10.0/24 │ any      │ 53/UDP   │ ACCEPT      │
│ 40  │ any      │ 192.168.10.0/24 │ 445/TCP │ DROP │ activa   │
│ 50  │ any      │ any      │ any      │ DROP     │ activa      │
└─────┴──────────┴──────────┴──────────┴──────────┴─────────────┘

★ El "estado" de cada regla importa: hay reglas HABILITADAS que están
  ahí solo como respaldo y nunca se probaron.
```

Los tres datos que hay que poner sí o sí:

1. **El puerto/protocolo exacto** (443/TCP, 53/UDP), no "el puerto de DNS".
2. **El estado** (activa / deshabilitada / en testing).
3. **La fecha de la última revisión.** Una regla de 2014 que nadie miró es un problema de seguridad esperando.

Bloque 5 — NAT, VIP y balanceadores

TRADUCCIONES		
VIP / IP pública	Puerto	Destino (pool)
200.1.1.10 (NAT 1:1)	443/TCP	srv-web-01:443 (50%) srv-web-02:443 (50%) peso: round-robin
200.1.1.20 (DNAT estático)	80/TCP	srv-web-01:80
200.1.1.0/24 (pool NAT salida)	(todo)	SNAT → 192.168.10.1 (todos los que salen)

★ Un "servicio publicado en Internet" SIEMPRE tiene una VIP (o una IP pública) y un puerto. Sin eso, no hay entrada.
★ El balanceo puede ser: round-robin, least-connections, IP hash o Weighted. Anotá cuál.

Y los dos casos que se confunden:

- **NAT de salida** (SNAT / masquerade): cambia la IP de **salida** de toda la red privada. **No cambia los puertos**.
- **NAT de entrada** (DNAT / port forwarding / VIP): cambia el **destino** de una conexión que entra. **SÍ cambia los puertos** si es port forwarding.

Escribilo explícitamente en el diagrama, porque son dos cosas distintas que los diagramas mezclan todo el tiempo.

Bloque 6 — Puertos y protocolos de los servicios

Servicio	Protocolo	Puerto(s)	Expuesto
SSH	TCP	22	sí
HTTPS (web)	TCP	443	sí
DNS interno	UDP+TCP	53	no*
MySQL	TCP	3306	no
SMTP salida	TCP	25 (587)	sí
IMAP	TCP	143 (993 TLS)	no
SNMP	UDP	161 (traps: 162)	no
NTP	UDP	123	sí

* "no" = sólo dentro de la red; pero si el DNS es público, va en 53.

El dato clave: la columna "expuesto". Un servicio que escucha en `0.0.0.0` y es alcanzable desde Internet es un **servicio público**, y eso tiene que estar escrito, porque es una decisión de seguridad tomada por omisión.

Bloque 7 — Estado operativo real (el "VALIDACIÓN PENDIENTE")

Este es el bloque que casi nunca está, y es el que hace que un diagrama sea **útil** en vez de decorativo. Para cada activo:

```
┌──────────────┬──────────────┬──────────────┬──────────────┬──────────────────┐
│ Activo       │ Estado       │ Verificado   │ Última       │ Observación      │
│              │ (diseño)     │ (realidad)   │ cambio       │                  │
├──────────────┼──────────────┼──────────────┼──────────────┼──────────────────┤
│ sw-acceso-p3 │ activo       │ ✓ ok         │ 2026-05      │ —                │
│ srv-web-01   │ activo       │ ⚠ validar    │ 2026-09      │ carga alta,      │
│              │              │              │              │ ver balanceo     │
│ srv-dns-02   │ backup       │ ✗ CAÍDO      │ 2026-11      │ migrar           │
│ fw-int-01    │ ??           │ ??           │ desconocido  │ reemplazar       │
│              │              │              │              │ en 2027          │
│ srv-file-01  │ deprecado    │ ✓ ok         │ 2024-03      │ migrar a         │
│              │              │              │              │ srv-file-02      │
└──────────────┴──────────────┴──────────────┴──────────────┴──────────────────┘

ESTADOS que hay que saber usar:
  activo        en producción, verificado
  en testing    se está probando, no es para producción
  deprecado     sigue andando pero se va a jubilar
  retirado      dado de baja (no debería estar en el diagrama)
  backup        en espera, no acepta tráfico
  PENDIENTE     ★ lo que dice tu imagen: definido, no comprobado
  DESCONOCIDO   nadie sabe
```

> **El "VALIDACIÓN PENDIENTE" de tu imagen no es una falta del diagrama: es información del diagrama.** Es la forma más honesta de dibujar "esto está **diseñado** pero **no comprobado**". Un diagrama que no tiene esa categoría obliga a elegir entre mentir (ponerlo todo como "activo") o no documentar (dejarlo afuera). **Un buen diagrama tiene la categoría, y la usa.**

Y la diferencia entre estado *de diseño* y estado *verificado* es la clave:

- **Diseño** = lo que dice el documento de arquitectura.
- **Verificado** = lo que alguien **comprobó mirando el equipo**.

Un diagrama solo con diseño **es un plan, no una descripción de la red**.

Bloque 8 — Metadatos del propio diagrama

El bloque que casi nadie pone y que hace que el diagrama sea mantenible:

```
┌──────────────────────────────────────────────────────────┐
│  Documento: DIAGRAMA-RED-FING-v3                         │
│  Fecha de creación:      2026-03-12                      │
│  Fecha de última revisión: 2026-09-28   ← SIEMPRE        │
│  Autor:                  Red 2 (red2@fing.edu.uy)        │
│  Ámbito:                 Edificio A, niveles 1 a 3       │
│  FUENTE DE VERDAD:       NetBox + Ansible + exporter SNMP│
│  Próxima revisión:       2026-12-15                      │
│  Clasificación:          USO INTERNO                     │
└──────────────────────────────────────────────────────────┘
```

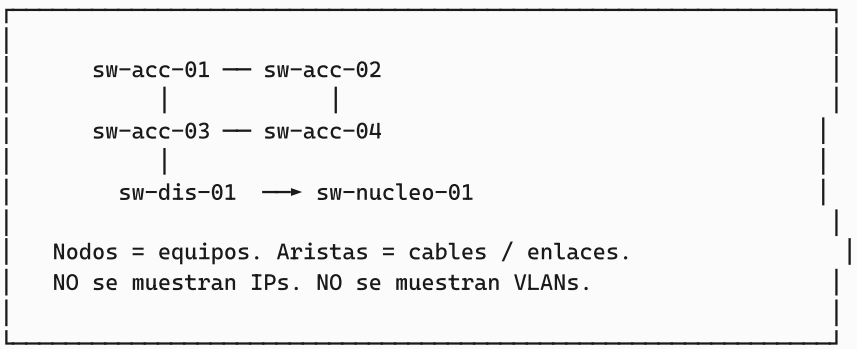
> **La "fuente de verdad" es el dato más importante.** Si el diagrama dice que se generó automáticamente desde **NetBox** o desde la **configuración con Ansible**, entonces hay una chance real de que esté **al día**. Si fue hecho a mano en un día, hay que asumir que está **desactualizado**. Eso cambia cómo lo usás en un incidente.

1.10 Los tipos de diagrama de red

Ahora sí: **qué tipos hay**. Y la respuesta corta es que **no hay un tipo, hay varios**, y cada uno responde una pregunta distinta. Un diagrama que intenta responder todas a la vez es ilegible.

(a) Topología física (*physical topology*)

Pregunta que responde: ¿cómo están conectados **físicamente** los equipos?

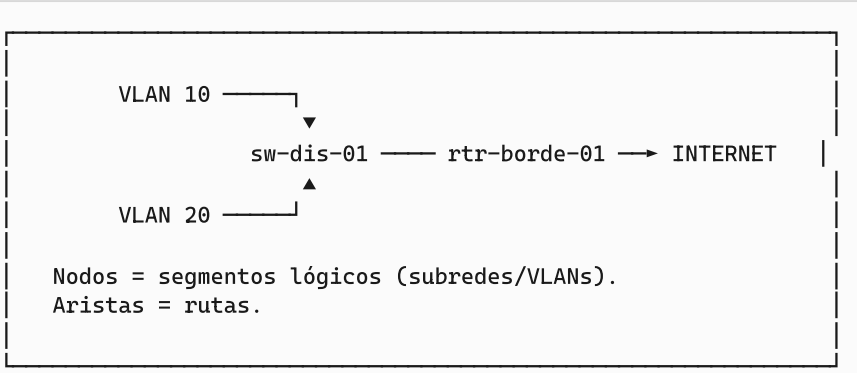


Para qué: documentar **cómo pasar un cable**, redundancy física, dónde están las ***conexiones*** **de uplink**, plan de crecimiento de puertos.

Herramientas: Visio, draw.io, Lucidchart, y.io (topología automática desde SNMP/CDP/LLDP).

(b) Topología lógica (*logical topology*)

Pregunta que responde: ¿cómo **circula** el tráfico según el **enrutamiento**?

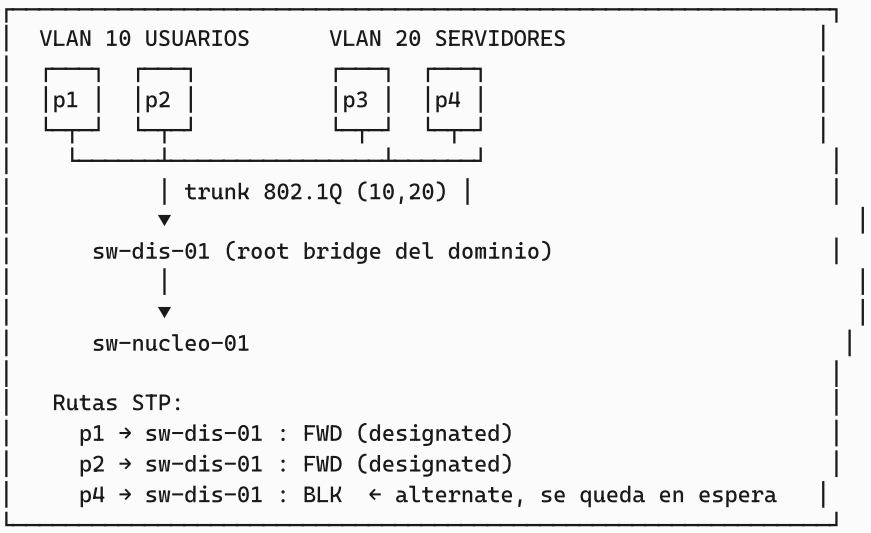


Para qué: entender **por dónde va** un paquete, calcular **tiempos de transito**, ver el impacto de un cambio de ruteo.

> **Físico vs. lógico, el ejemplo que lo aclara:** en un edificio con 6 switches en cascada (6 saltos), la topología **física** dice "6 saltos"; la **lógica** dice "1 salto". Un conmutado es **1 solo dominio de colisión** y **ningún salto** para efectos de latencia, aunque haya 6 equipos en el medio. **Y eso es exactamente lo que se pregunta en los problemas de retardo de examen.**

(c) Diagrama de VLAN y spanning tree

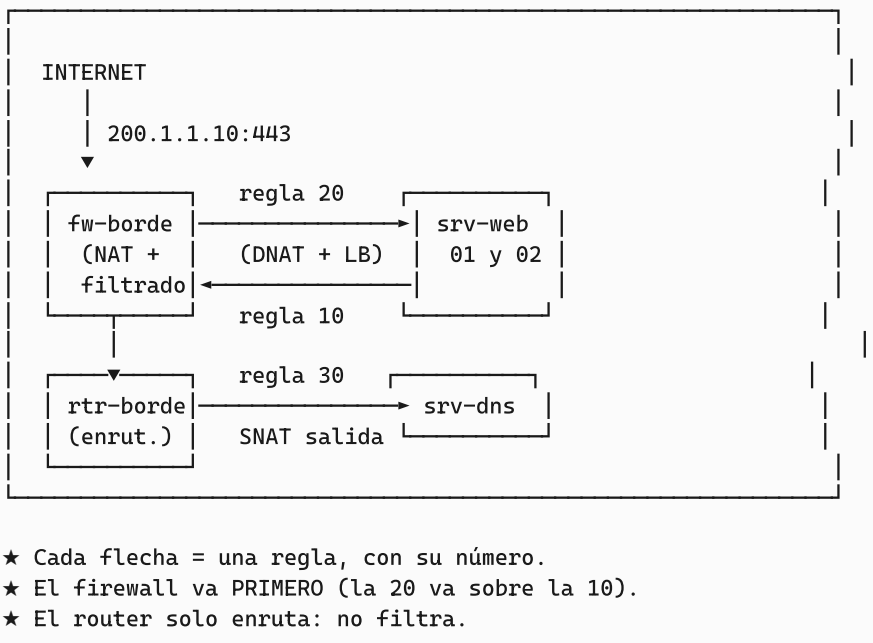
Pregunta que responde: ¿qué VLAN hay, qué puertos son de acceso y cuáles son trunk, y qué hace el STP?



Para qué: el diagrama **por excelencia del que se queja el usuario final**: "en este piso la red anda lenta". La respuesta casi siempre es un **puerto bloqueado por STP** que obliga al tráfico a hacer un rodeo.

(d) Diagrama de firewall / NAT / seguridad

Pregunta que responde: ¿qué pasa con un paquete que entra o sale?

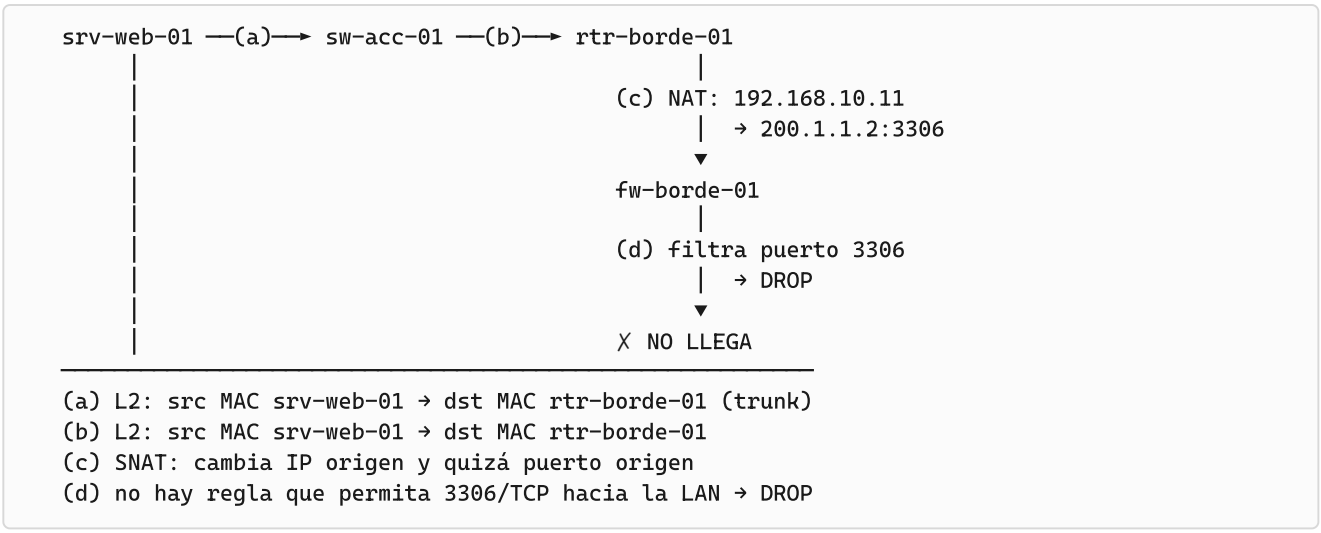


★ Cada flecha = una regla, con su número.
★ El firewall va PRIMERO (la 20 va sobre la 10).
★ El router solo enruta: no filtra.

Para qué: auditar seguridad, entender por qué algo **no** se conecta, depurar un problema de acceso.

(e) Diagrama de flujo (*packet flow / traffic path*)

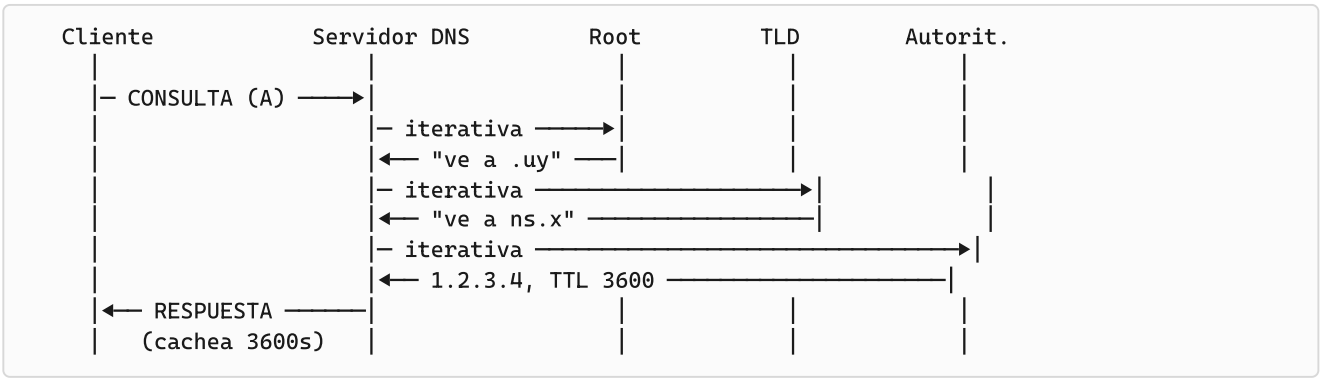
Pregunta que responde: ¿**por dónde pasa** este paquete específico?



Para qué: es **el diagrama de depuración**. Cuando algo no funciona, se recorre el camino salto a salto: ¿el switch reenvió? ¿el NAT se aplicó? ¿el firewall dejó pasar? **Cada flecha tiene un punto de falla posible.**

(f) Diagrama de secuencia (*sequence diagram*)

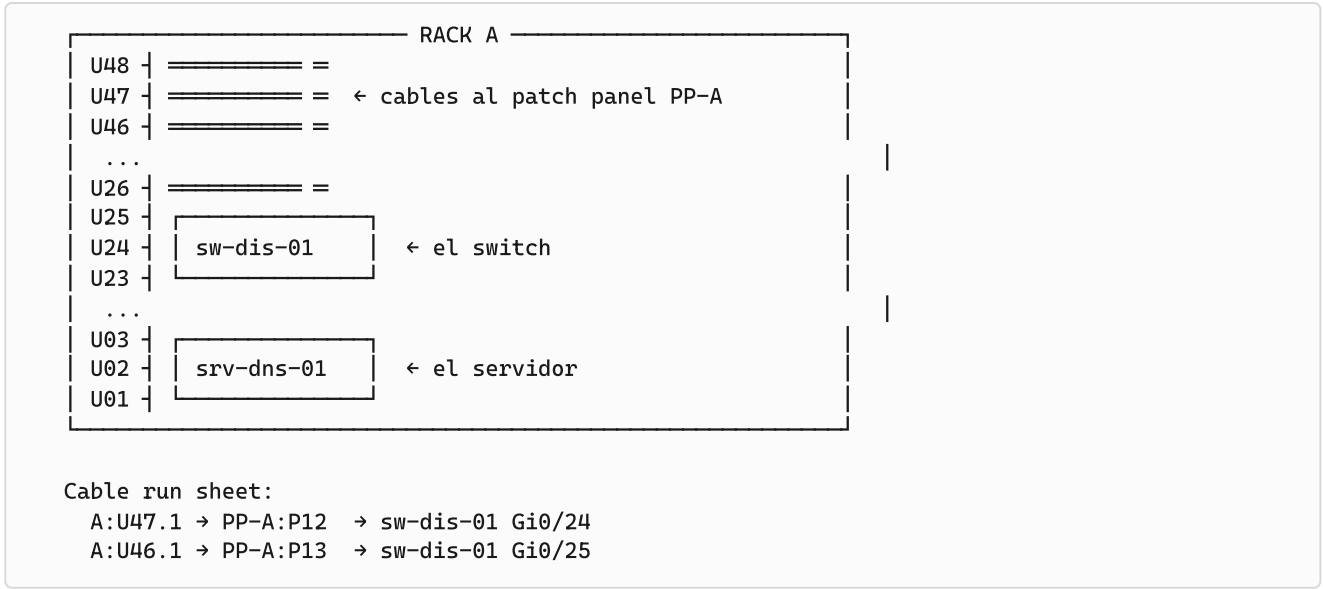
Pregunta que responde: ¿**qué mensajes** se exchanging y **en qué orden**?



Para qué: es el diagrama de **TCP**, **TLS**, **DHCP**, **ARP**, **OSPF**. Y es el que vas a tener que **dibujar de memoria** en el examen: el three-way handshake, el DORA, el hello+hello+req+ack de OSPF.

(g) Diagrama de rack / *wiring diagram*

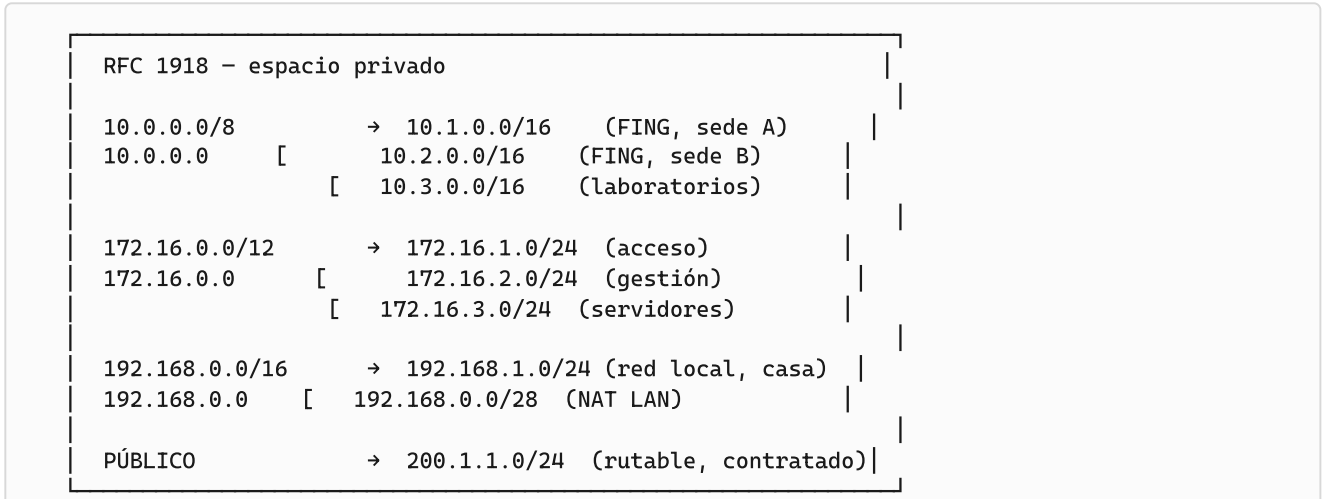
Pregunta que responde: ¿qué cable va de dónde a dónde, **físicamente**?



Para qué: es el diagrama del **técnico de campo**. No se entiende, no se discute: se **usa**. Y es el único diagrama que se verifica **con las manos**.

(h) Diagrama de direccionamiento / addressing

Pregunta que responde: ¿cómo está repartido el espacio de direcciones?



Para qué: es **el diagrama de subnetting**. Acá van los `/n`, los gateways, los rangos de DHCP y las reservas. Y es **literalmente el ejercicio de examen** del bloque D, dibujado.

1.11 Los seis pasos para hacer un diagrama de verdad

Esto es el método, y es lo que hay que hacer cuando te pidan "hacé un diagrama de esta red":

```
1. RECOLECTAR    Inventario, contrato con el proveedor, configs,
                 salida de "show inventory", nmap.

2. CLASIFICAR    ¿Qué pregunta responde este diagrama? Elegí UN tipo
                 (§1.10). Si es un entregable, probablemente hace falta
                 un (a) físico + un (d) de seguridad.

3. ESQUEMATIZAR  Dibujá los BLOQUES (redes, zonas), no los equipos.
                 Un rectángulo por red, con su CIDR adentro.
                 Después poné los equipos adentro de su bloque.

4. CONECTAR      Una flecha = un enlace. Etiquetá cada flecha con:
                 · tipo de medio (cobre/fibra)
                 · ancho de banda (100M / 1G / 10G)
                 · VLAN / trunk
                 · longitud estimada

5. ANOTAR        Aplicá los ocho bloques de §1.9. En este orden:
                 identidad → L3 → L2 → firewall → NAT → puertos →
                 ESTADO → metadatos.

6. VERIFICAR     ★ El paso que nadie hace. Andá al equipo y comprobá.
                 Lo que no pudiste verificar va marcado
                 "VALIDACIÓN PENDIENTE".
                 Poné la fecha de la verificación.
```

> **El paso 6 es el que convierte un dibujito en un documento.** Es exactamente lo que tu imagen estaba marcando: lo que no está verificado, **tiene que estar marcado**. Un diagrama sin fecha de verificación y sin marca de "pendiente" es un diagrama que **miente con la mejor intención**.

1.12 La checklist: ¿mi diagrama está listo?

Usá esto antes de dar por terminado cualquier diagrama:

```
IDENTIDAD
[ ] ¿Cada equipo tiene nombre formal, modelo, ubicación y responsable?
[ ] ¿Cada equipo tiene un ID de inventario?

L3
[ ] ¿Cada host tiene IP, máscara/CIDR y gateway?
[ ] ¿Cada router tiene TODAS sus interfaces con su CIDR?
[ ] ¿Las dos notaciones de máscara (/24 y 255.255.255.0)?
[ ] ¿Los rangos de DHCP y las IPs fijas de cada VLAN?

L2
[ ] ¿Las VLAN numeradas Y nombradas?
[ ] ¿Qué puertos son de acceso y cuáles trunk, con qué etiquetas?
[ ] ¿Qué hace el spanning tree (raíz, puertos bloqueados)?

SEGURIDAD
[ ] ¿Las reglas de firewall, con su número y su estado?
[ ] ¿Los puertos exactos y protocolos de cada servicio?
[ ] ¿Qué servicios están expuestos a Internet?

NAT
[ ] ¿Las VIPs públicas y a qué pool apuntan?
[ ] ¿El método de balanceo?
[ ] ¿El NAT de salida (SNAT) y el de entrada (DNAT), por separado?

ESTADO  ← el bloque que casi todos olvidan
[ ] ¿El estado real de cada activo (activo / deprecado / caído / pendiente)?
[ ] ¿Qué está VERIFICADO y qué no?
[ ] ¿Los unknowns están marcados como desconocidos?

METADATOS
[ ] ¿Fecha de última revisión?
[ ] ¿Autor y fuente de verdad?
[ ] ¿Próxima revisión agendada?
```

1.13 ⚠️ Actualización a 2026

Sigue igual — de memoria: - La necesidad de **documentar en un diagrama** es la misma de siempre. - Los **cuatro bloques** básicos: L3, L2, seguridad y estado. - La diferencia entre **topología física** y **lógica**, y por qué en un conmutado 6 equipos físicos son 1 salto lógico. - La importance de la **dirección IP, la máscara y el gateway** en cada nodo.

Cambió:

- **El diagrama se genera solo.** Con fuentes como **NetBox**, **phpIPAM**, **AWX/Ansible** (que lee la configuración real de los routers) y **Ansible Network**, hoy se puede **exportar el diagrama directamente del equipo que tiene la verdad**. Eso resuelve de raíz el problema de "el diagrama está viejo".
- **Los diagramas de servicios en la nube (SaaS, balanceo, CDN)** ya no son "la red": son **flujos entre servicios**, y hay que documentarlos igual. red": son **flujos entre servicios**, y hay que documentarlos igual.
- **La verificación es automatizable.** Con la observación como código (infraestructura como código, pipelines de CI), un diagrama "VALIDACIÓN PENDIENTE" puede **dejarse de marcar** porque la verificación es **un test que corre siempre**.
- **El bloqueo de STP sigue siendo la causa número uno** de las quejas de rendimiento en los edificios: sigue valiendo la pena un diagrama de STP.

> **Para el examen:** el examen **no pide hacer un diagrama profesional de producción**, pide **un diagrama de topología con las direcciones, que permita resolver tablas de ruteo o de ARP**. Eso es un tipo (a) más un (h). Pero saber qué información va es lo que te permite **leer** cualquier topología que te den.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Los seis parámetros en un caso concreto

Enunciado. Ana está en Montevideo y hace una videollamada con Ana (sic, con Bea, que está en Frankfurt). La conexión va por fibra, pasa por 11 routers, y el informe de la conexión dice:

```
Ancho de banda del enlace de la casa:  300 Mbps
Distancia total del camino (ida):      11 400 km
Velocidad de propagación en la fibra:  2·10⁸ m/s
Tamaño medio de un paquete de video:   1 200 bytes
Retardo medido por ping (RTT):         142 ms
Retardo medido para el paquete 1:      71 ms
Retardo medido para el paquete 2:      78 ms
Retardo medido para el paquete 3:      102 ms
Pérdidas observadas:                    0,4 %
Velocidad sostenida de la descarga:    271 Mbps
```

Se pide: identificar cada magnitud, decir en qué unidad está, y explicar las dos diferencias que más se confunden.

Resolución paso a paso.

Paso 1 — ¿Cuál es el ancho de banda?

```
"Ancho de banda del enlace de la casa: 300 Mbps"
→ 300 · 10⁶ bits por segundo
→ Es la CAPACIDAD MÁXIMA. Está en bps. Es una propiedad del equipo
  (el ONT y el medio).
```

Paso 2 — ¿Cuál es la velocidad de propagación?

```
"Velocidad de propagación en la fibra: 2·10⁸ m/s"
→ Está en m/s. Es la rapidez de la SEÑAL.
→ Es una propiedad de la FIBRA (del medio), no del equipo.
→ Por eso el informe no la reporta en bps: no es una capacidad.
```

Paso 3 — ¿Cuál es la latencia y cuál es el RTT?

```
"Retardo medido por ping (RTT): 142 ms"
→ 142 ms es el Round Trip Time: ida + vuelta.

"Retardo medido para el paquete 1: 71 ms"
→ 71 ms es la LATENCIA (tiempo de un viaje, de ida).

Chequeo de coherencia:  71 ms × 2 = 142 ms  ✓
→ El RTT es exactamente el doble. Éste es el control que tenés
  que hacer SIEMPRE que te den latencia y RTT juntos.
```

Paso 4 — ¿Cuál es el throughput?

```
"Velocidad sostenida de la descarga: 271 Mbps"

271 Mbps  <  300 Mbps  ✓ (el throughput nunca supera el BW)

eficiencia = 271/300 = 0,9033  →  90,3 %
La diferencia (9,7 %) se va en:
  - cabeceras: 58 bytes de overhead por cada 1200 de datos
    1200 + 58 = 1258 bytes en el medio
    eficiencia de payload = 1200/1258 = 0,9539
  - pérdidas y retransmisiones: 0,4 % (cada pérdida por TCP obliga
    a retransmitir un segmento entero, y el TCP pierde también la
    ventana durante el RTO)
  - colisiones y, si lo modelás, slots ocupados
```

Paso 5 — ¿Cuál es el jitter?

```
Retardos de tres paquetes del MISMO flujo:
  paquete 1:  71 ms
  paquete 2:  78 ms
  paquete 3: 102 ms

jitter (medición pico a pico) = 102 − 71 = 31 ms
```

Interpretación y por qué importa. El paquete 3 tardó 31 ms más que el paquete 1. Eso no es un problema de ancho de banda: la señal viaja a la misma velocidad. Es un problema de **cola**: somewhere en el camino, 31 ms de espera. En una videollamada de voz, eso produce un desfasaje entre el audio y el video: el labio se ve antes de que se oiga la palabra.

Paso 6 — La tabla resumen del ejercicio.

Magnitud	Valor del enunciado	Unidad
Ancho de banda	300 Mbps	bps
Vel. de propagación	$2 \cdot 10^8$	m/s
Latencia	71 ms	ms
RTT	142 ms (= 2×71)	ms
Retardo total	71 ms (el mismo packet)	ms
Throughput	271 Mbps	bps
Jitter	31 ms	ms

⚠️ Errores típicos de este ejercicio

- **Confundir latencia con RTT.** La latencia es un viaje; el RTT son dos. Si el ping dice 142, la latencia es ~71 (asumiendo camino simétrico).
- **Meter la velocidad de propagación en bps.** Si aparece "2×10^8 Mbps" es un error de unidades: 2×10^8 es **m/s**.
- **Decir que el throughput "es 300 Mbps".** No: el throughput es lo que **seNwgets**; el BW es lo que **se puede**.
- **Usar `d/s` con la distancia de ida para el RTT.** `d/s` te da la latencia; para el RTT, `2d/s`.

2.2 Ejercicio 2 — Velocidad de propagación en cobre vs. fibra

Datos. Un enlace de 800 km une dos edificios de una misma ciudad. Hay dos opciones:

- **Opción A**: un haz de **cobre** (UTP Cat 6A).
- **Opción B**: una **fibra óptica monomodo**.

La velocidad de propagación medida por el instalador es:

```
Cobre (Cat 6A): s = 2,0 · 10⁸ m/s
Fibra monomodo:  s = 2,04 · 10⁸ m/s
```

a) ¿Cuál es el retardo de propagación de la señal en cada caso? b) ¿Cuánto tarda un paquete de 5 000 bits en cruzar el enlace, con store-and-forward en un switch de 10 Gbps en cada extremo? c) ¿Conviene pagar la fibra?

Resolución.

a) Retardo de propagación.

```
d = 800 km = 800 000 m

Opción A (cobre):
  d_prop = d / s = 800 000 m / (2,0 · 10⁸ m/s)
         = 4,0 · 10⁻³ s
         = 0,004 s
         = 4,0 ms

Opción B (fibra):
  d_prop = d / s = 800 000 m / (2,04 · 10⁸ m/s)
         = 3,9216 · 10⁻³ s
         = 0,0039216 s
         = 3,92 ms
```

b) Retardo total, con store-and-forward.

El paquete de 5 000 bits **entra** por un puerto de 10 Gbps del switch de la izquierda y tiene que **entrar entero** antes de que ese switch pueda empezar a mandarlo por el puerto de salida (store-and-forward).

```
L = 5 000 bits
R = 10 Gbps = 10 · 10⁹ bits/s

d_transmisión = L / R = 5 000 / (10 · 10⁹) = 5 · 10⁻⁷ s = 0,0005 ms
d_proc (switch de 10 Gbps con HER stacking) ≈ 0 (despreciable)

Opción A (cobre):
  d_total = d_proc + d_transmisión + d_cola + d_prop
          = 0 + 0,0005 ms + 0 (sin cola) + 4,0 ms
          = 4,0005 ms

Opción B (fibra):
  d_total = 0 + 0,0005 ms + 0 + 3,92 ms
          = 3,9205 ms
```

c) ¿Conviene pagar la fibra?

La diferencia es de 0,08 ms **en un viaje**. Con las dos manos atrás y atrás: 0,16 ms de RTT.

```
Diferencia por viaje: 4,0 − 3,92 = 0,08 ms
Diferencia en RTT:    2 × 0,08  = 0,16 ms
```

Respuesta: para latencia, no. La diferencia es despreciable. La fibra no compra latencia en este caso, y eso es una sorpresa muy frecuente: se supone que "la fibra es más rápida", pero **propagar la señal es un fenómeno físico, y la luz no va más rápido en fibra que en cobre**. Va a 2/3 de la velocidad de la luz en el vacío en los dos casos.

Pero la fibra sí compra otra cosa, y esta es la respuesta completa:

```
┌──────────────────────┬──────────────┬──────────────┐
│                      │ COBRE        │ FIBRA        │
├──────────────────────┼──────────────┼──────────────┤
│ Velocidad de prop.   │ 2,00·10⁸     │ 2,04·10⁸     │  ← casi igual
│ Retardo de prop. (800km)│ 4,00 ms   │ 3,92 ms      │  ← casi igual
│ Ancho de banda posible │ 10 Gbps    │ 100+ Gbps    │  ← AQUÍ está la diferencia
│ Alcance sin repetidor  │ ~100 m     │ ~80 km       │  ← y acá
│ Inmune a EMI         │ no           │ sí           │  │
│ Peso por metro       │ ~0,05 kg/m   │ ~0,02 kg/m   │  │
│ Costo del transceiver│ ~40 USD      │ ~80–400 USD  │  │
└──────────────────────┴──────────────┴──────────────┘
```

La moraleja: la fibra se paga por **ancho de banda**, por **alcance sin electrónica**, y por **inmunidad al ruido electromagnético** — **no por latencia**.

> ⚠️ **Precisión sobre "más rápido":** en los formatos eléctricos de Ethernet (10BASE-T, 100BASE-TX) hay un **retardo de codificación de línea** que la fibra no tiene (MLT-3, 4B/5B y *scrambler* contra NRZI de la fibra). Pero ese retardo es de **nanosegundos a unas decenas de nanosegundos**, no de milisegundos. Para 800 km, es ruido estadístico frente a los 4 ms de propagación.

2.3 Ejercicio 3 — Por qué la velocidad de propagación NO depende del ancho de banda

Enunciado. Dos proveedores ofrecen enlaces para el mismo trayecto de 50 km:

	Proveedor A	Proveedor B
Ancho de banda	100 Mbps	1 Gbps
Velocidad de propagación medida	**$2{,}0 \cdot 10^8$ m/s**	**$2{,}0 \cdot 10^8$ m/s**
Latencia medida (ping, 1x)	5,0 ms	5,0 ms

Un alumno le pregunta al proveedor: "¿el enlace más rápido no debería llegar antes?". **Justificá por qué la respuesta del proveedor es la correcta, y qué pasa con el RTT y con el throughput.**

Resolución paso a paso.

Paso 1 — Qué determina `s`.

La velocidad de propagación la determina la **velocidad de la perturbación electromagnética en el medio**, que es una propiedad de la **electricidad y la óptica del material**:

```
──────────────────────────────────────────────────────────────
VACÍO / AIRE    el campo E y el campo B se propagan juntos, a c
                = 3·10⁸ m/s. No hay medio que "frenar" nada.
──────────────────────────────────────────────────────────────
COBRE           los electrones libres arrastran a los ligados
                al mover el campo E. Resultado neto: ~0,6 a 0,7 c
──────────────────────────────────────────────────────────────
FIBRA (Si)      la luz refracta con índice n ≈ 1,48. v = c/n.
                3·10⁸ / 1,48 = 2,03·10⁸ m/s
──────────────────────────────────────────────────────────────
```

En los tres casos, `s` depende de una constante del material y de nada más.

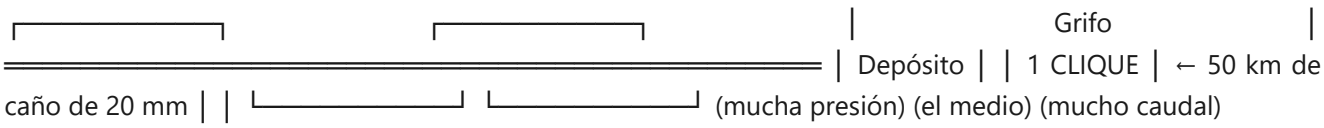
Paso 2 — Qué determina `R` (ancho de banda).

`R` lo determina **cuántos bits se pueden codificar por unidad de tiempo sobre esa señal**:

10 Mbps = 1 símbolo cada 100 ns (o 2 símbolos cada 200 ns con 2 niveles) 1 Gbps = 1 símbolo cada 1 ns

Y **el reloj del transceptor es una consecuencia de la electrónica del circuito**, no de la física

Paso 3 — La analogía del caño.



▸ Abrir la llave más rápido (más caudal, más ancho de banda) NO hace que el agua llegue antes por el caño. El tiempo de viaje lo fija la longitud del caño y la velocidad del fluido.

▸ El caudal sí cambia CUÁNTA agua llega por segundo.

Paso 4 — Qué pasa entonces con los números.

Distancia = 50 km = 50 000 m s = $2{,}0 \cdot 10^8$ m/s

d_prop = 50 000 / ($2 \cdot 10^8$) = $2{,}5 \cdot 10^{-4}$ s = 0,25 ms ← IGUAL en A y en B

R d_trans (L=1500B) throughput máx. Proveedor A 100 Mbps 120 µs ≈ 96 Mbps Proveedor B 1 Gbps 12 µs ≈ 961 Mbps d_prop 0,25 ms 0,25 ms ← IGUAL

Con 11 routers en el camino y d_proc despreciable:

RTT (sin cola) = 2 · (11 · 0,25 ms) = 2 · 2,75 ms = 5,5 ms → IGUAL para A y para B.

```
**Interpretación — y esta es la frase para el examen:**

> **El ancho de banda dice cuántos datos entran por segundo; la velocidad de propagación dice a qu

**Paso 5 — El corolario que aparece en los exámenes: el BDP.**

Como el RTT **no depende** del ancho de banda, y el ancho de banda **sí** determina cuántos datos
```

BDP (bandwidth-delay product) = R · RTT

Proveedor A: $100 \cdot 10^6$ bps × $5{,}5 \cdot 10^{-3}$ s = 550 000 bits = 68,75 kB Proveedor B: $1 \cdot 10^9$ bps × $5{,}5 \cdot 10^{-3}$ s = 5 500 000 bits = 687,5 kB

```
**Derivación paso a paso para A:**
```

R = $100 \cdot 10^6$ bits/s RTT = $5{,}5 \cdot 10^{-3}$ s

BDP = $100 \cdot 10^6$ bits/s × $5{,}5 \cdot 10^{-3}$ s = $100 \cdot 5{,}5 \cdot 10^{6\ -3}$ bits = $550 \cdot 10^3$ bits = 550 000 bits = 550 000 / 8 bytes = 68 750 bytes ≈ 68,75 kB

```
**Interpretación del BDP.** En el proveedor B, para llenar el tubo de punta a punta tenés que tene

---

# PARTE 3 — PROBLEMAS DE EXAMEN DE FING

## 3.1 Mapa de los problemas de examen del Bloque A

Antes de resolver nada, el mapa. Esto es lo que vas a encontrar en los exámenes del bloque:

| Examen | Pregunta | Tema | Dónde se resuelve |
|---|---|---|---|
| `2006-12` | P1 (5 pts) | Modelo OSI y las hipótesis de las 7 capas | **Clase 2** |
| `2007-03` | P1 (5 pts) | El modelo TCP/IP | **Clase 2** |
| `2009-02` | P2 (10 pts) | Forwarding vs. routing; flooding en broadcast | **Clase 3** |
| `2009-02` | P3 (5 pts) | FDMA y TDMA | **Clase 7** |
| `2009-02` | P4 (9 pts) | Datagramas vs. circuitos virtuales | **Clase 4** |
| `2009-03` | P5 (6 pts) | Full duplex vs. half duplex | **Clase 7** |
| `2006-08` | P1 (5 pts) | Resultados de Nyquist y Shannon con hipótesis | **Clase 7** |
| `2007-07` | P1 (10 pts) | Capacidad máxima sobre línea telefónica (Shannon) | **Clase 7** |
| `2007-12` | P3 (10 pts) | Conmutación de circuitos, paquetes y mensajes | **Clase 4** |
| `2009-12` | P1 (10 pts) | Componentes del retardo; BW vs. `s`; congestión | **Clase 5** (adelant
| `2011-03` | P2 (6 pts) | Congestión, pérdidas y retardo | **Clase 5** |
| `2011-03` | P5 (10 pts) | Retardo end-to-end, 3 enlaces y 2 conmutadores | **Clase 6** |

**Nota sobre los puntajes:** el enunciado de `2006-12 P1` y de `2007-03 P1` dice **5 puntos** cada

## 3.2 ⭐ Adelanto: la Pregunta 1a del Examen 2009-12 (resuelta aquí, se repite en la Clase 5)

> **Fuente:** Examen `sirc0912` — 18 de diciembre de 2009, Pregunta 1 (10 puntos), apartado a).
> **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0912.pdf

### Enunciado (texto del examen)

> Identifique los componentes del retardo en las redes de paquetes, y señale cuáles son los más si
> **a)** Señale las diferencias entre el ancho de banda y la velocidad de propagación en el medio.
> **b)** ¿En qué componente del retardo influye la congestión?

### Solución oficial (apartado a)

> a) B es la capacidad de los puertos del dispositivo (switch o router), por ejemplo Ethernet, STM-

### Análisis de la solución oficial

La respuesta oficial es **correcta y corta**, y contiene exactamente las dos ideas que hay que dec

1. **`B` es la capacidad de los puertos del dispositivo** (switch o router), es decir, del **equip
2. **`s` es una característica del medio**, por ejemplo `s ≈ 2×10⁸ m/s` en cobre.

**Lo que le falta y hay que agregar para el punto completo** (no restan si no lo decís, pero el pr

- **Las unidades**: `B` en **bps**, `s` en **m/s**. Es la diferencia formal y la que hace que no s
- **Que son ejes perpendiculares**: podés tener el mismo `s` con distinto `B` (cobre a 100 Mbps y
- **Que `s` NO cambia con el medio de enlace en el sentido de "más rápido"**: tanto cobre como fib
- **La consecuencia**: `d_transmisión = L/R` (no depende del medio) vs. `d_propagación = d/s` (no

> ⚠️ **Detalle sobre la solución oficial:** dice `s ~ 2×10⁸ m/s` **en cobre**, que es correcto. Si

### Mi respuesta completa para 10 puntos (o los 4 del apartado a)
```

ANCHO DE BANDA (B o R) - unidad: bits por segundo (bps) - qué es: la CAPACIDAD MÁXIMA de transferir datos por el enlace - de qué depende: del EQUIPO (puerto, transceiver, modulación). Ethernet de 1 Gbps, STM-1 de 2,048 Mbps, 100 GbE. - no depende de: la longitud del enlace, del medio físico. - aparece en: d_transmisión = L / R

VELOCIDAD DE PROPAGACIÓN (s) - unidad: metros por segundo (m/s) - qué es: a qué velocidad física viaja la SEÑAL por el medio - de qué depende: del MEDIO FÍSICO (la física del material). Cobre ~$2{\cdot}10^8$ m/s, fibra ~$2{\cdot}10^8$ m/s, vacío $3{\cdot}10^8$ m/s. - no depende de: la longitud del enlace, del equipo, del ancho de banda. - aparece en: d_propagación = d / s

CONCLUSIÓN Son magnitudes DISTINTAS, de UNIDADES DISTINTAS y que dependen de COSAS DISTINTAS: una del equipo y una del medio. El ancho de banda NO es "la velocidad a la que viaja la información": es la cantidad de información por segundo. La velocidad de propagación es la rapidez del mezzo físico. Un enlace de 1 Gbps por cobre y otro de 1 Gbps por fibra tienen el MISMO ancho de banda y la MISMA velocidad de propagación, pero eso NO SIGNIFICA que se comporten igual: difieren en atenuación, en alcance sin repetidores y en inmunidad al ruido electromagnético.

```
### Apartado b) — adelanto (se resuelve entero en la Clase 5)

> **Solución oficial:** *"El retardo de encolamiento depende del nivel de congestión del router."*

Interpretado: la congestión **no** actúa sobre el retardo de transmisión ni sobre el de propagació

**La cadena completa, que es lo que pide el apartado:**
```

Más tráfico offered → colas más largas → retardo de encolamiento ↑ → retardo total ↑ │ └─→ si la cola se llena: PERDIDAS

```
---

# PARTE 4 — CIERRE

## 📌 Resumen en 10 líneas

1. **Red** = computadores interconectados por un medio, para compartir recursos e intercambiar inf
2. **PAN / LAN / MAN / WAN** por alcance: ~10 m, ~1 km, una ciudad, el planeta. **En 2026 el crite
3. **Latencia es el criterio moderno**: 500 ms de RTT de un satélite a 36.000 km es "WAN" aunque l
4. **Topologías**: bus (cable mínimo, un punto de falla), estrella (falla aislada, un dominio de c
5. **Cada dispositivo, su capa**: hub/repetidor/módem = **1**; NIC = **1 y 2**; switch/AP = **2**;
6. **Ancho de banda** (`B`, **bps**) = capacidad del **equipo**, aparece en `L/B`.
7. **Velocidad de propagación** (`s`, **m/s**) = rapidez de la señal en el **medio**, aparece en `
8. **Latencia** = un viaje; **RTT** = dos viajes** (≈ `2d/s` sin colas). **Throughput** = lo real, s
9. **`BDP = R · RTT`** = los bits que caben en el enlace durante un RTT = los que tenés que tener
10. **Internet ≠ Web**: Internet es la red (IP, TCP, BGP); la Web es una aplicación (HTTP, HTML, U

## ✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

**1.** Una red de 800 km a 1 Gbps y otra de 800 km a 10 Mbps. ¿Cuál tarda más en llegar un paquete

<details><summary>Respuesta</summary>

**Ninguna de las dos, si la fibra es la misma.** El retardo de propagación es `d/s` y no depende d
```

d = 800 000 m, s = 2,0 · 10^8 m/s d_prop = 800 000 / 2,0·10^8 = 4,0 ms (igual en las dos redes)

```
Lo único que cambia es el retardo de transmisión, que es diminuto al lado de la propagación:
```

A 10 Mbps: d_trans = 12 000 bits / 10 · 10^6 = 1 200 µs = 1,2 ms A 1 Gbps: d_trans = 12 000 bits / 10 · 10^9 = 12 µs = 0,012 ms

Total a 10 Mbps: 1,2 + 4,0 = 5,2 ms Total a 1 Gbps: 0,012 + 4,0 = 4,012 ms

```
O sea: **el enlace de 10 Mbps tarda 1,19 ms más**, y ese 1,19 ms es 100 % transmisión, 0 % propaga

</details>

**2.** En una red con 500 Mbps de ancho de banda y un throughput medido de 380 Mbps, ¿cuál es el m

<details><summary>Respuesta</summary>

El ancho de banda es 500 Mbps. Es el número que tenemos, y es una **capacidad**.

La **velocidad de propagación no se puede calcular** con esos datos: es una propiedad del medio y l

**No son comparables**, justamente por eso: uno está en bps y el otro en m/s. La pregunta "cuál es

Y la eficiencia: `380/500 = 76 %`. Los 24 % que faltan son cabeceras, retransmisiones por pérdidas

</details>

**3.** ¿Por qué un auricular Bluetooth es un PAN y un enlace de fibra entre dos edificios a 4 km es

<details><summary>Respuesta</summary>

Por la **propiedad** y, en parte, por la latencia.

- El auricular es **tuyo**, de una sola persona, con latencia sub-milisegundo. Tenés el medio y el
- El enlace de fibra de 4 km **no es tuyo**: es de un operador. Para vos es infraestructura de ter

El alcance (10 m contra 4 km) coincide con la clasificación clásica, pero **lo que define la front

</details>

**4.** Un switch tiene 8 puertos y un hub tiene 8. ¿Cuántos dominios de colisión hay en cada caso?

<details><summary>Respuesta</summary>

- **Switch: 8 dominios de colisión y 1 dominio de broadcast.** Cada puerto es un dominio de colisi
- **Hub: 1 dominio de colisión y 1 dominio de broadcast.** El hub es un repetidor multi-puerto: re

El hub es de **capa 1** por eso: no aprende direcciones, no decide a qué puerto va la trama, solo a

</details>

**5.** Explicá con tus palabras por qué el ancho de banda y la velocidad de propagación son perpen

<details><summary>Respuesta</summary>

Porque uno describe **cuánto por segundo** y el otro **a qué velocidad**, y dependen de objetos di

- El **ancho de banda** depende del **equipo**: de la electrónica del transceptor, del reloj, de l
- La **velocidad de propagación** depende del **medio**: de la física del material. Podés cambiar

La analogía: el ancho de banda es el **caudal** de un caño (litros por segundo) y `s` es la **velo

</details>

**6.** Una aplicación de video tiene un throughput de 8 Mbps en un enlace de 20 Mbps. ¿Cuánto del a

<details><summary>Respuesta</summary>
```

desaprovechado = 20 − 8 = 12 Mbps eficiencia = 8/20 = 40 % ```

Ese 60 % no es "ancho de banda desperdiciado": es la **sobrecarga del enlace**, que es lo que permite que la latencia sea baja. Si la aplicación usara el 100 %, la cola del búfer del router sería infinita y el retardo se dispararía. El 60 % de holgura es **la inversión en latencia**.

Las causas del 40 % sin ser simplemente "no aprovechar el 100 %": - el video tiene **tasa variable** (VBR): entre fotogramas clave transmite mucho, entre ellos poco; - los **ACK** de TCP van en sentido contrario y también ocupan el enlace (aunque en general el upload es menor); - las **cabeceras** de cada paquete; - si hay otros usuarios en el mismo enlace, compiten.

7. ¿Qué es el jitter de un enlace de 20 Mbps? ¿Puede ser 20 Mbps?

▶ Respuesta

8. Dime dos razones por las que la clasificación LAN/MAN/WAN **ya no** se puede hacer solo por la distancia.

▶ Respuesta

🔗 Referencias

Exámenes - Examen `sirc0612` (26-dic-2006) — P1 (5 pts): modelo OSI y las hipótesis de las 7 capas → **Clase 2** - Examen `sirc0703` (26-feb-2007) — P1 (5 pts): el modelo TCP/IP → **Clase 2** - Examen `sirc0902` (2-feb-2009) — P2 (10 pts): forwarding vs. routing y flooding → **Clase 3** - Examen `sirc0912` (18-dic-2009) — P1 (10 pts): componentes del retardo, `B` vs. `s`, congestión → **Clase 5** - Examen `sirc1103` (21-feb-2011) — P2 (6 pts): congestión, pérdidas y retardo → **Clase 5** - Examen `sirc1103` — P5 (10 pts): retardo end-to-end con 3 enlaces y 2 conmutadores → **Clase 6**

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., **cap. 1** "¿Qué es una red?" (pág. ~1 a ~35 del PDF de tu carpeta) — es la fuente de la definición y de los parámetros - Tanenbaum & Wetherall, **cap. 2** "La capa física" (pág. ~36 a ~75) — medios y moduladores, que se usan en la Clase 7 - Kurose & Ross, *Computer Networking: A Top-Down Approach* — los capítulos 1 y 2 tienen la misma distinción `bandwidth` vs. `propagation speed` y el `bandwidth-delay product`

RFCs - `RFC 1122` — *Requirements for Internet Hosts: Communication Layers* (1989): define las capas de la pila TCP/IP **con enlace separado** (5 capas) - `RFC 8200` — *Internet Protocol, Version 6 (IPv6) Specification* (2017, *Internet Standard*; obsoleta RFC 2460) - `RFC 9293` — *Transmission Control Protocol* (2022, *Internet Standard*; consolida RFC 793) - `RFC 7323` — *TCP Extensions for High Performance* (2014): *Window Scale* y *Timestamps*, que son lo que hace posible un BDP grande - `RFC 9000` — *QUIC: A UDP-Based Multiplexed and Secure Transport* (2021): el ejemplo moderno de que una capa de transporte **no** tiene que ser TCP

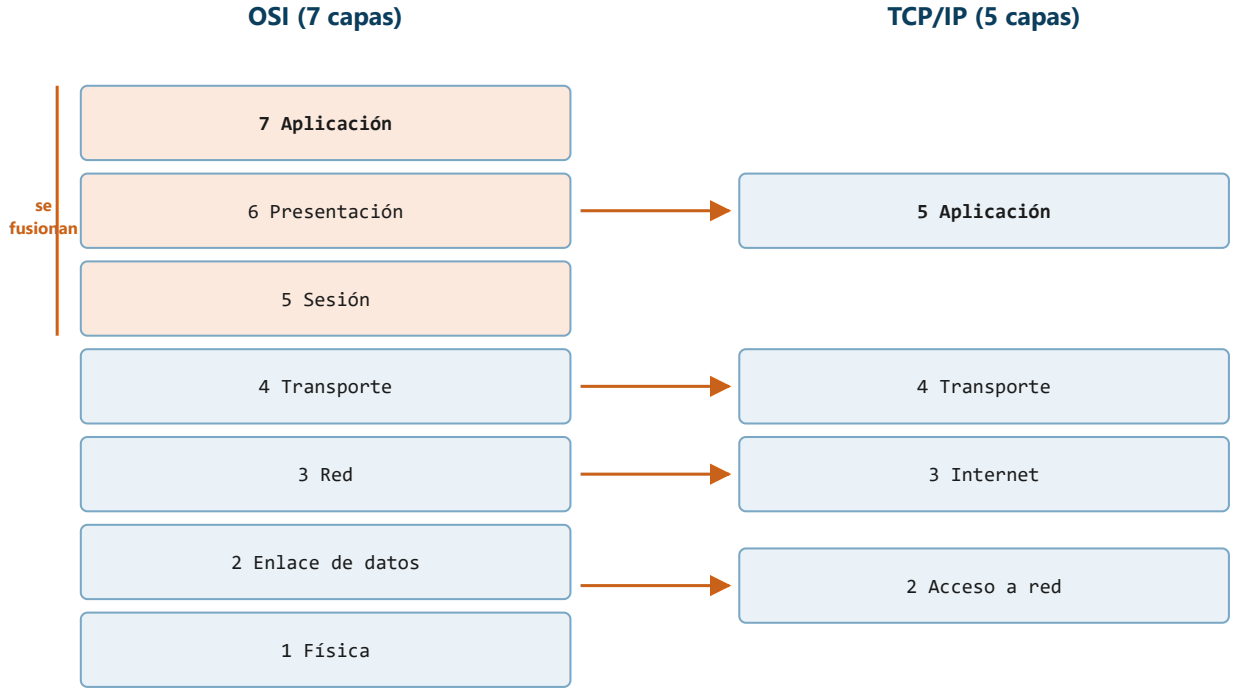
Clase siguiente: 2 — Los modelos de capas: OSI y TCP/IP

Clase 2

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El problema que resuelve el modelo de capas
1.2 El modelo OSI de 7 capas
Capa 1 — Física
Capa 2 — Enlace de datos
Capa 3 — Red
Capa 4 — Transporte
Capa 5 — Sesión
Capa 6 — Presentación
Capa 7 — Aplicación
1.3 El modelo TCP/IP: por qué hay 4, 5 o 7 capas
1.4 Comparación OSI ↔ TCP/IP
1.5 La PDU: por qué cambia de nombre al bajar de capa
2.1 Ejercicio 1 — ¿En qué capa vive cada cosa? (el ejercicio diagnóstico más útil)
2.2 Ejercicio 2 — Las tres formas de decir lo mismo
2.3 Ejercicio 3 — ¿Por qué no existe "capa de presentación" en Internet?
3.1 ⭐ Examen 2006-12, Pregunta 1 (5 puntos) — el modelo OSI y las hipótesis
3.2 ⭐ Examen 2007-03, Pregunta 1 (5 puntos) — el modelo TCP/IP
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Equivalencia entre el modelo OSI y TCP/IP



En TCP/IP no existen Presentación ni Sesión:
el cifrado vive en la capa de Aplicación (TLS) y en la biblioteca.

Ojo: hay dos criterios de conteo. Tanenbaum cuenta 5 capas porque separa enlace y física; Kurose cuenta 4. La capa 3 es la misma.

Bloque: A (Fundamentos) · **Semana:** 1 · **Duración estimada:** 110 min **Correlativa:** Clase 1 (¿qué es una red?) · **Sigue con:** Clase 3 (encapsulación y comparación OSI ↔ TCP/IP) **Material de referencia:** Tanenbaum cap. 1 §1.4 (pág. ~20 a ~26 del PDF de tu carpeta) · RFC 1122, RFC 8200, RFC 9293, RFC 768

🎯 Al terminar esta clase vas a poder

1. **Explicar** qué problema resuelve el modelo de capas y por qué la **interfaz** entre capas es la clave de la arquitectura.
2. **Enumerar** las 7 capas de OSI con su **PDU** (unidad de datos de protocolo), sus protocolos representativos y **el problema concreto que resuelve cada una**.
3. **Describir** el modelo TCP/IP diciendo explícitamente por qué hay **4, 5 o 7** capas según el autor que leas, y qué criterio usa cada uno.
4. **Construir** la tabla de comparación OSI ↔ TCP/IP completa y decir **por qué ganó TCP/IP**.
5. **Explicar por qué la unidad de datos cambia de nombre** al bajar de capa (mensaje → segmento → paquete → trama → bit).
6. **Desarrollar** la capa de Presentación con ejemplos concretos: *endianness*, formatos, compresión y cifrado.
7. **Resolver enteros** el Examen `2006-12 P1` y el Examen `2007-03 P1`, con las soluciones oficiales a la vista.

> **Prerequisito del examen:** `2006-12 P1` (5 pts — modelo OSI e hipótesis), `2007-03 P1` (5 pts — modelo TCP/IP), `2011-03 P1` (8 pts — las dos funciones de la capa de red), `2009-02 P2` (10 pts — forwarding vs. routing).

PARTE 1 — TEORÍA

1.1 El problema que resuelve el modelo de capas

El problema

Comunicar dos máquinas por una red es un problema **gigantesco**. Un solo paquete involves decisiones sobre:

- ¿con qué voltaje represento un `1`? ¿con qué ancho de pulso?
- ¿cómo sé dónde empieza y dónde termina una trama?
- ¿cómo llego al host `203.0.113.9` de Montevideo desde uno en Montevideo?
- ¿cómo sé que los 1 500 bytes llegaron todos y en orden?
- ¿qué pasa si elCharacters son un PNG?
- ¿y si el otro host entiende UTF-8 y yo mando ISO-8859-1?

Si **cada comunicación** tuviera que resolver **todas** esas preguntas, no habría redes. Habría que escribir un programa distinto para cada par de máquinas, cada protocolo, cada medio.

La solución: dividir y especificar

El modelo de capas propone algo que parece obvio una vez que lo oyes pero que no lo es:

```
┌──────────────────────────────────────────────────────────────┐
│  Dividir el problema en SUBSISTEMAS, y que cada subsistema se  │
│  comunique con el de arriba y el de abajo ÚNICAMENTE a través  │
│  de una INTERFAZ definida con precisión.                       │
└──────────────────────────────────────────────────────────────┘
```

Cada capa: - **resuelve un subproblema** y solo ese; - **usa** un servicio de la capa de abajo; - **ofrece** un servicio a la capa de arriba; - **no sabe nada** de lo que hacen las capas de arriba ni de abajo, salvo lo que la interfaz define.

Por qué la interfaz es la clave

```
Capa 7   ┌──────────────────────────────────┐
         │  "Te entrego datos de aplicación" │   ← lo que OFRECE
         └──────────────────────────────────┘
─────────────────────────────────────────────────  ← la INTERFAZ
         ┌──────────────────────────────────┐
Capa 6   │  "Datos de presentación"          │   ← lo que USA
         └──────────────────────────────────┘

La capa 7 sabe que la 6 existe. La 6 sabe que la 7 existe.
Ninguna de las dos sabe cómo funciona la 5.
```

Qué compra esto. Gracias a las interfaces:

- Un fabricante de switches (que implementa capas 1 y 2) **no necesita saber** nada de HTTP. Un fabricante de routers (capas 1 a 3) **no necesita saber** nada de TCP. Se pueden combinar porque la frontera está

especificada.

- Se puede **cambiar una implementación** sin tocar las demás. Si el medio pasa de cobre a fibra, cambia la capa 1 y **todo lo demás queda igual**.
- Se puede **especificar y probar** cada capa por separado: un tester que manda tramas Ethernet inválidas está probando la capa 2, no "la red".

La analogía. Es el mismo movimiento que el que hizo la ingeniería industrial con la cadena de producción: un tornillo no tiene que saber de qué máquina vino ni a qué máquina va. Lo único que importa es que tenga la rosca estándar. **El estándar de rosca es la interfaz.** Y exactamente igual: un byte que sale de la capa 7 no tiene que saber de qué máquina vino; lo que importa es que cumpla el formato que la capa 6 espera.

Las dos hipótesis de la ISO (esto es lo que pide el Examen 2006-12)

La ISO no elige las capas por capricho. Hay **dos criterios** explícitos que justifican por qué son 7 y no 4 ni 10:

> **H1.** Las capas deben delimitarse buscando **minimizar el flujo de datos que se intercambia entre capas a través de la interfaz**. **H2.** El **número de capas debe ser suficientemente amplio para evitar que diferentes funciones no se requieran colocar en la misma capa**.

Qué significa H1 y por qué importa. Si una capa habla mucho con la de arriba, es señal de que la frontera está mal puesta: las dos capas deberían estar fusionadas (o separadas). Un ejemplo: en OSI, la capa de Presentación le pasa "datos de presentación" a la de Sesión, y la de Sesión a la de Transporte. El flujo entre ellas es **el mismo bloque de bytes**, sin ninguna transformación. Por eso la criticism clásica a OSI es que **Presentación y Sesión son separaciones artificiales** que producen mucho tráfico de interfaz para poca funcionalidad. Y esa crítica es **la razón por la que TCP/IP las eliminó**.

Qué significa H2. Si metés dos funciones muy distintas en la misma capa, terminás con una capa que hace dos cosas y las hace mal. El ejemplo clásico: en TCP/IP, la capa de Aplicación incluye **HTTP, DNS, SMTP, y también el cifrado (TLS) y la conversión de formatos**, que OSI tenía en 5 y en 6. TCP/IP las fusionó porque en la práctica casi no se usan por separado. H2 dice: eso es un defecto, sí, pero aceptable.

1.2 El modelo OSI de 7 capas

El modelo OSI (*Open Systems Interconnection*, ISO 7498) es **una arquitectura de referencia**: no es un protocolo, es la especificación de **cómo se especifican** los protocolos. Su valor es que cuando alguien dice "esto es capa 3", todo el mundo sabe de qué habla.

La tabla que hay que saber de memoria

#	Capa	**PDU** (*Protocol Data Unit*)	Protocolos representativos	**Qué problema resuelve**
7	**Aplicación**	**Mensaje** (*message*)	HTTP, DNS, SMTP, SSH, IMAP, FTP	**La red como servicio** para el programa: qué datos pide, con qué sintaxis, con qué semántica
6	**Presentación**	**Datos de presentación**	Cifrado (TLS/SSL), compresión, JPEG/MPEG, ASCII/Unicode, ASN.1	**Los datos de un host pueden no ser compatibles con otro**: representación, endianness, formato, cifrado, compresión
5	**Sesión**	**Datos de sesión**	NetBIOS, RPC, NetBIOS Session Service	**Establecer, mantener y terminar un diálogo** entre dos aplicaciones: sincronización, puntos de recuperación, *checkpoints*
4	**Transporte**	**Segmento** (*segment*, TCP) / **datagrama de usuario** (UDP)	TCP, UDP, SCTP, QUIC	**Que la aplicación se independice de la red**: entregar a un proceso, con fiabilidad, orden y control de flujo
3	**Red**	**Paquete** (*packet* / *datagram*)	IP, ICMP, IPv6, ARP, IGMP, IPsec	**Cómo llegar de una red a otra**: direccionamiento lógico, enrutamiento, reenvío
2	**Enlace de datos**	**Trama** (*frame*)	Ethernet, Wi-Fi 802.11, PPP, HDLC, LLC	**Que dos nodos de un mismo enlace se comuniquen sin errores**: delimitar, detectar errores, ARQ, control de acceso al medio
1	**Física**	**Bit**	1000BASE-T, 10GBASE-SR4, RS-232, codificación Manchester, NRZI	**Cómo viajan los bits**: medio físico, señal eléctrica/óptica, conectores, voltajes, sincronismo de reloj

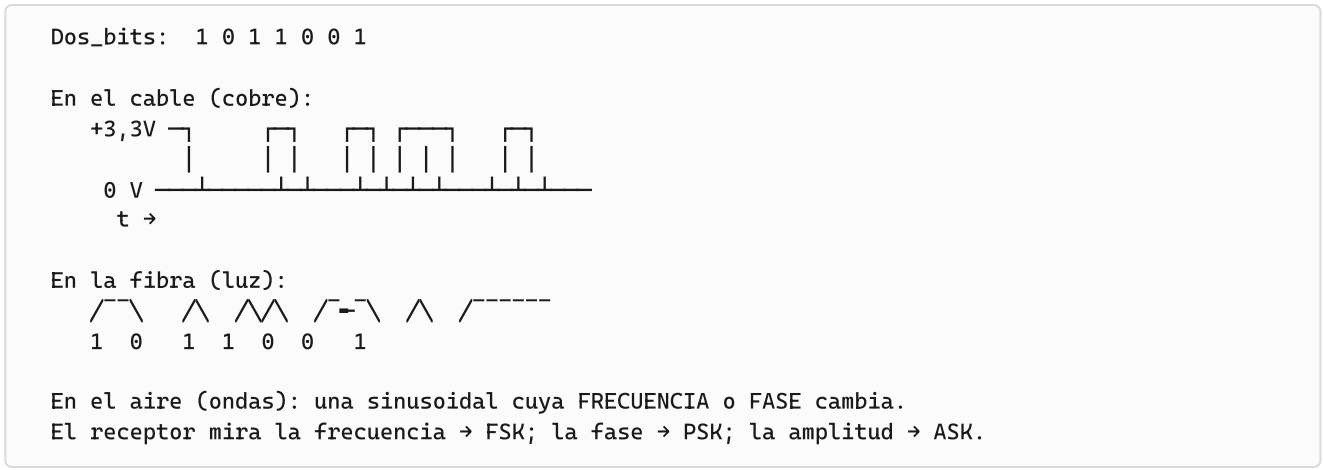
> La PDU de la capa 7 tiene **dos** nombres según el autor: Tanenbaum la llama **mensaje** (*message*), Kurose la llama **segmento de capa de aplicación**. Si el examen dice "mensaje", usá "mensaje"; si dice "datos", usá "datos de aplicación". Los dos son correctos.

Lo que hace cada capa, en detalle

Capa 1 — Física

Único encargo: que los bits viajen. Y lo hace sin entender nada de lo que llevan adentro.

- Define el **medio** (cobre, fibra, aire, ondas), los **niveles de señal** (voltajes, intensidades, fases), los **conectores**, la **codificación de línea** (NRZI, Manchester, 4B/5B, PAM-4, 1024-QAM) y el **sincronismo** (de dónde saca el reloj el receptor).
- **No detecta errores**, **no sabe si un bit es el primero de una trama**, **no sabe qué hay en el paquete**.

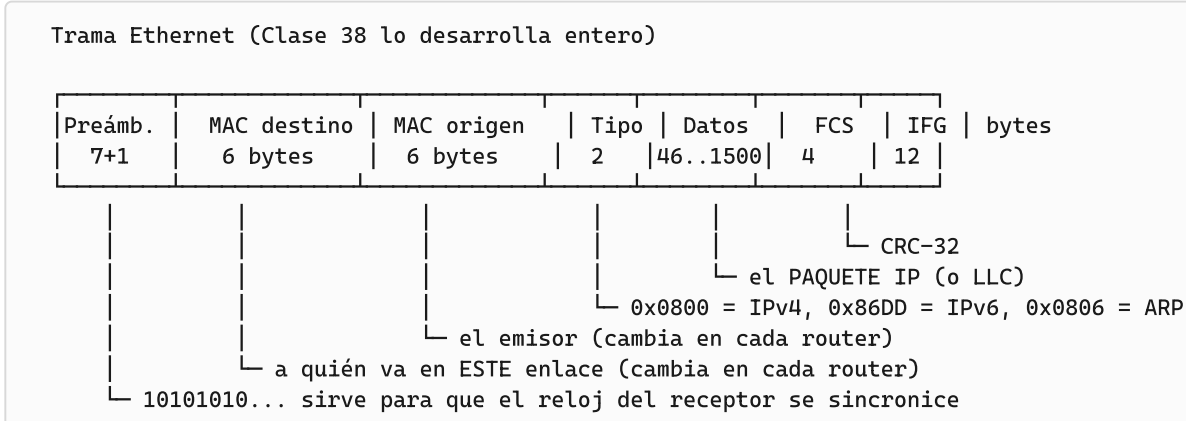


Lo importante: la capa 1 no distingue `1011001` de `0010011`. Solo ve "hay señal" y "no hay señal", o "la fase está en π" y "la fase está en 0".

Capa 2 — Enlace de datos

Esta capa resuelve **un problema local**: hacer que **dos nodos vecinos** se comuniquen de forma confiable, aunque el medio sea ruidoso.

- **Framing**: delimitar dónde empieza y termina cada trama (preámbulo, flags, byte stuffing, longitudes).
- **Detección de errores**: CRC-32 (Ethernet), checksum (HDLC), paridad.
- **Control de errores** (ARQ): *Stop-and-Wait*, *Go-Back-N*, *Selective Repeat*.
- **Control de acceso al medio**: CSMA/CD, CSMA/CA, *token passing*, ranuras de tiempo.
- **Direccionamiento físico**: direcciones **MAC** de 48 bits, que son **locales al enlace**: se reescriben en **cada** router.



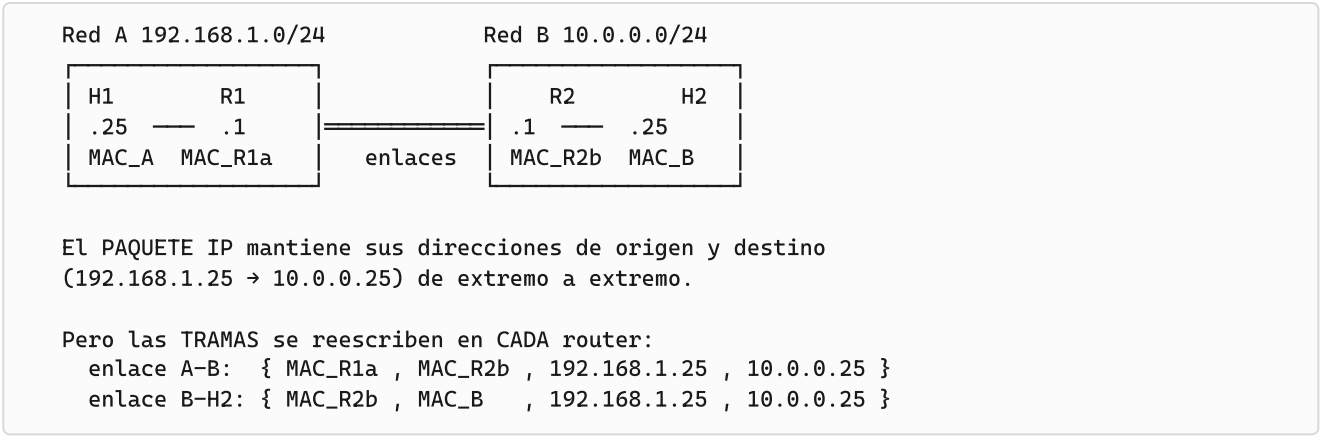
Lo que la capa 2 NO hace: no enruta. Un switch de capa 2 no puede llevar un paquete de la LAN A a la LAN B. Esa es la Pregunta 2 del Examen 2009-02 (Clase 3).

Capa 3 — Red

Esta capa resuelve **el alcance**: cómo llevar un paquete de **una red a otra**.

- **Direccionamiento lógico**: direcciones **IP**, que son **globales**: **no cambian** de router a router (salvo por NAT, que lo rompe justamente por eso).

- **Reenvío** (*forwarding*): copiar cada paquete al puerto de salida correcto según la tabla.
- **Enrutamiento** (*routing*): calcular esas tablas intercambiando información con los vecinos.
- **Protocolos de control**: ICMP, IGMP, y sobre todo el enrutamiento (OSPF, BGP).
- **Fragmentación y reensamblaje**, y MTU.



El punto clave, y es la Pregunta 1a del Examen 2009-12 y toda la Pregunta 2 del 2009-02:

- La **capa 3 es la primera que puede decir "esto no es para mí"**. La capa 2 solo sabe si el destino es su propio puerto o hay que reenviarlo.
- La **dirección de capa 3 es jerárquica y global** (la IP); la de capa 2 es **plana y local** (la MAC).
- Un router **recibe la trama, quita la cabecera de capa 2, mira la cabecera de capa 3, decide, y arma una trama nueva**.

Capa 4 — Transporte

Resuelve **el último salto lógico**: llevar los datos de una aplicación a **otro proceso** de la máquina de destino, y que la aplicación **no sepa nada** de la red.

- **Multiplexación por puerto**: qué proceso (navegador, correo, SSH) es el destinatario.
- **Fiabilidad**: números de secuencia, ACKs, *timers*, retransmisiones, control de flujo, control de congestión.
- **Orden**: reordenar lo que llegó desordenado.
- **TCP** =connection-oriented, confiable, con control de flujo y de congestión. **UDP** = sin conexión, sin fiabilidad, mínimo.

Las dos diferencias con la capa 2, que aparecen en el Examen 2009-12 P2:

	Capa 2 (enlace)	Capa 4 (transporte)
Alcance de la fiabilidad	**Por salto** (*hop-by-hop*)	**Extremo a extremo** (*end-to-end*)
Confiabilidad	Sí, **si el medio es ruidoso**	**Siempre**, **aunque la red no lo sea**
Estado	No necesita estado entre saltos	Estado de la conexión en **ambos** extremos
Numeración	Por trama, en el enlace	Por **bytes**, global al flujo
Pérdida	Se detecta y se retransmite **por enlace**	La capa intermedia puede perder y **TCP no se entera** hasta que no llega el ACK

Capa 5 — Sesión

Resuelve **el diálogo**:

- **Establecer** la sesión: negociar los parámetros (versiones, opciones, prioridades).
- **Sincronizar**: *checkpoints*, para poder retomar si algo falla.
- **Recuperación**: reenviar solo desde el último checkpoint.
- **Terminar** ordenadamente.

Ejemplo clásico: NetBIOS (que_session service) y RPC sobre OSI. **En la práctica no se implementan casi nunca como capa separada**: el trabajo de establecer la sesión lo hace TCP (el *three-way handshake*), y la sincronización la hace TLS (los *record* y las alertas) y HTTP/2 (los *streams*). Por eso OSI "sobran" dos capas.

Capa 6 — Presentación

Esta es la capa que más se olvida y la que más puntos da si la desarrollás. Resuelve el problema de **heterogeneidad de representación**:

> **Los datos de un host pueden no ser compatibles con los de otro**, aunque el bit que llega sea el mismo.

Los cuatro problemas concretos:

(a) *Endianness* — el orden de los bytes

Un entero de 32 bits con valor `0x12345678` se puede guardar de dos maneras:

```
Big-endian (network byte order, "orden de red"):
    byte[0]=0x12  byte[1]=0x34  byte[2]=0x56  byte[3]=0x78
    ──────────┬─────────┬─────────┬──────────
              │         │         │
         MSB (peso  2²⁴)  ...          ...  LSB (peso 2⁰)
    Se transmite el byte de MAYOR peso primero.

Little-endian ("orden de host", Intel/ARM/x86):
    byte[0]=0x78  byte[1]=0x56  byte[2]=0x34  byte[3]=0x12
    byte de MENOR peso primero.

──────────────────────────────────────────────────────────────
OJO: si no me decís la endianness, el mismo conjunto de 4 bytes
puede significar 305 419 896  O  201 8915346.

DECISIÓN: toda estructura de red (IP, TCP, UDP) es BIG-ENDIAN.
Por eso htons()/htonl()/ntohs()/ntohl() existen en C.
```

(b) Formatos de caracteres — codificaciones

Un byte `0xE9` puede ser la `é` en **ISO-8859-1**, otra cosa en **Windows-1252**, y **parte de un carácter** en **UTF-8** (donde un carácter ocupa de 1 a 4 bytes). Si el emisor manda ISO-8859-1 y el receptor asume UTF-8, el receptor ve basura. Es el bug más famoso de la historia de la web ("mojibake").

(c) Formatos de archivo y de estructura de datos

Una imagen JPEG es JPEG, pero un TIFF tiene muchas variantes; un PDF con referencias cruzadas; un `.tar` con o sin padding; una estructura de protocolo con alineamiento de 4 bytes o con padding a 8. Sin capa de presentación, cada aplicación tiene que reinventar la conversión. La alternativa usual es **un formato estándar único**: JSON, XML, ASN.1, protobuf.

(d) Compresión y cifrado

- **Comprimir**: si el medio es caro (enlace satelital) o el tiempo es caro (archivos grandes). *JPEG*, *gzip*, *zstd*, *brotli*.
- **Cifrar**: que solo el destinatario entienda. En la práctica, esto vive en **TLS** (`RFC 8446` , actualizado por `RFC 9846` en 2026), que es "presentación" dentro de la capa de aplicación de TCP/IP.

La respuesta oficial del Examen 2006-12, textual:

> *"Busca la solución de problemas particulares, que hacen referencia a la semántica y sintaxis de lo enviado. Por ejemplo la resolución de diferentes representaciones entre máquinas (pto flotante), corresponde a esta capa."*

Análisis: el ejemplo que da la cátedra es el **punto flotante**: es exactamente el caso de *endianness* + rango + precisión de la representación de un `float` . El mismo `float` de 4 bytes puede ser `3,14159` en un host y otra cosa en otro si difieren en endianness. La oficial también menciona la **semántica y la sintaxis**, que es más amplio que "el byte": la capa 6 no solo traduce bytes, traduce **significados**.

Capa 7 — Aplicación

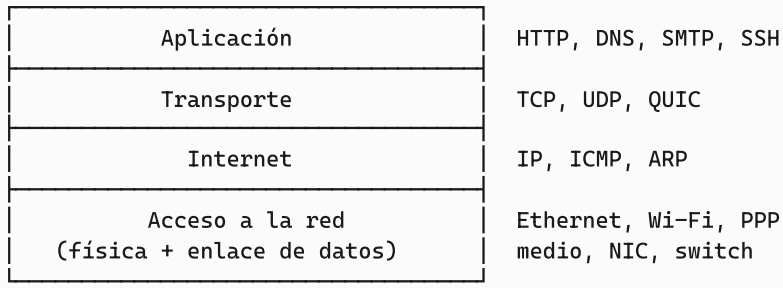
El servicio de red, visto por la aplicación. **No es la aplicación misma**: es el **protocolo** con el que la aplicación habla.

- **Modelo**: cliente/servidor o peer-to-peer.
- **HTTP** (web), **DNS** (nombre ↔ dirección), **SMTP/IMAP/POP3** (correo), **SSH** (shell remoto), **FTP** (transferencia de archivos), **DHCP** (configuración).
- **No sabe nada de la red**: la aplicación le da "mandá estos 300 bytes al host X puerto Y" y no sabe si van por fibra, por satélite o por tres enlaces de 2 Gbps.

1.3 El modelo TCP/IP: por qué hay 4, 5 o 7 capas

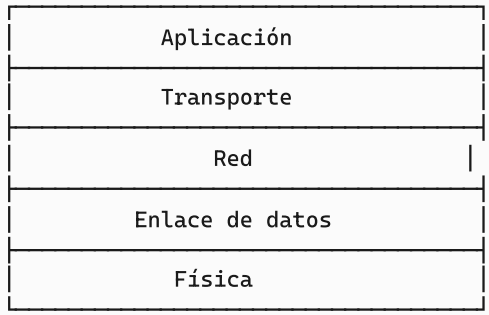
Acá está la diferencia de criterio entre autores, y **el examen puede preguntar exactamente esto**.

Modelo de 4 capas (Kurose y Ross)



Kurose **fusiona física y enlace en una sola capa de "Acceso a la red"** porque en su libro ambas se presentan como "cómo llegan los bits al otro lado del cable", sin distinguir demasiado.

Modelo de 5 capas (Tanenbaum, y las RFC)



Tanenbaum **mantiene enlace y física separadas** y **colapsa Aplicación, Sesión y Presentación en una sola**. Y lo dice explícitamente en su libro: la capa de aplicación "oculta las diferencias entre sistemas" y los servicios que OSI le asignaba a 5 y 6 "son pocos y con frecuencia innecesarios", así que **la mayoría de los protocolos que usan no implementan una capa de sesión y de presentaciónseparadas**.

Y lo mismo dicen las RFC, que es lo que define la arquitectura real de Internet: `RFC 1122` **(*Requirements for Internet Hosts: Communication Layers*) habla de tres capas** (Internet, Transport, Aplicación) más el acceso al medio; y cuando separa, usa **4** (aplicación, transporte, internet, **acceso a red** con enlace y físico dentro). Lo que **no** hacen las RFC es separar sesión y presentación: **no existen**.

La tabla de las tres lecturas

Autor / documento	Capas	dónde entran sesión y presentación
ISO 7498	**7**	Son capas 5 y 6, separadas
Kurose y Ross	**4**	Fusionadas dentro de Aplicación
Tanenbaum	**5**	Fusionadas dentro de Aplicación
`RFC 1122` , `RFC 8200`	**4 o 5**	**No existen como capas**; sus funciones viven en Aplicación

El criterio de la diferencia, en una frase:

Kurose (4)	le da más peso a la TEORÍA: qué problema resuelve cada capa y cuántos bits son el mínimo razonable.
Tanenbaum (5) / RFC (5)	Kurose CREEE que capa es "lo que el estudiante necesita ver separada" para entender el problema. Tanenbaum y las RFC la ven desde el otro lado: "la arquitectura REAL tiene esta forma". En la realidad de Internet no existe ninguna separación sesión/presentación, porque nadie la necesita.

⚠️ **Para el examen:** si te preguntan "¿cuántas capas tiene el modelo TCP/IP?", la respuesta más segura es **"depende del autor: Kurose describe 4, Tanenbaum describe 5, y el modelo OSI tiene 7"**, y después **explicá por qué la diferencia está en si se separa enlace de física y en si sesión y presentación existen**. Un alumno que dice "5" y explica el criterio suma más puntos que uno que dice "5" sin explicar.

1.4 Comparación OSI ↔ TCP/IP

La tabla completa

OSI	PDU	TCP/IP (Tanenbaum)	Equivalente en Kurose	Qué pasó con la función
7. Aplicación	mensaje	**Aplicación**	Aplicación	Se mantiene, pero **absorbe** 5 y 6
6. Presentación	datos de presentación	↑ **Aplicación**	↑ Aplicación	**No existe como capa.** Cifrado en la biblioteca (TLS), compresión y formatos en las propias apps
5. Sesión	datos de sesión	↑ **Aplicación**	↑ Aplicación	**No existe como capa.** Lo hace TCP (establecer) y TLS (sincronizar)
4. Transporte	segmento	**Transporte**	Transporte	Se mantiene íntegra: TCP, UDP, QUIC
3. Red	paquete	**Red**	Internet	Se mantiene íntegra: IP, ICMP, ARP
2. Enlace	trama	**Enlace de datos**	↑ Acceso a la red	Se mantiene, aunque Kurose la funda con la 1
1. Física	bit	**Física**	↑ Acceso a la red	Se mantiene

Por qué la tabla no es simétrica

Las **tres capas bajas** (1, 2, 3) de OSI coinciden **una a una** con TCP/IP. Eso no es casualidad: son las que se diseñaron juntas en ARPANET.

La **asimetría está arriba**. OSI separó en tres capas (Aplicación, Sesión, Presentación) lo que TCP/IP puso en una, y **la práctica_results dio la razón a TCP/IP**: la(session) y la (presentación) casi nunca se usan por

separado. Un navegador web no tiene una "capa de sesión" que pueda quitar: el estado de la sesión HTTP vive en la capa de aplicación, y el cifrado vive en una biblioteca (TLS) que la aplicación usa como una función.

Por qué ganó TCP/IP

Hay cuatro razones, y todas son históricas, no técnicas:

Razón 1 — La red funcionó antes de que la teoría estuviera completa. ARPANET se puso en marcha en 1969 con un modelo de capas de 4 niveles. El modelo OSI se terminó de discutir en la ISO en **1977**, ocho años después. Para entonces la red ya existía, ya funcionaba, y ya tenía protocols instalados en cada host. Cambiar el modelo implicaba recablear laInternet.

Razón 2 — La ISO tardó 20 años en estandarizar la capa 3. El protocolo de capa 3 de OSI (CLNP, Connectionless Network Protocol) se llegó a proponer, pero **nunca se desplegó** como protocolo de Internet. La realidad es que la capa 3 de Internet se_estandarizó como **IP** en 1981 (RFC 791), y el nombre de "capa 3 de OSI" quedó vacante. Cuando llegó IPv6 (RFC 8200, 2017), seOi Thinking… pero el grupo OSI ya no tenía influencia real sobre el diseño. **En 2026 no existe "IPv6 de OSI": existe IPv6 de IETF.** Ese es el punto.

Razón 3 — La competencia práctica. TCP/IP venía de BSD, corriendo gratis en Unix. OSI venía como una norma ISO, y las implementaciones de la ISO costaban dinero. En 1993 el mundo tenía: TCP/IP gratis, IPX de Novell en las LAN de empresas, y OSI como norma de papel. Ganó la que era gratis.

Razón 4 — La transição total (1983). El 1 de enero de 1983 ARPANET hizo el "Flags Day": todos los hosts pasaron a TCP/IP. Eso cerró el protocolo de la comunidad OSI y dejó a TCP/IP como el único.

Lo que hay que decir de OSI sin menospreciarlo: OSI **sí aportó conceptos que sobrevivieron sin capas propias**:

- **El nombre "capa"** como unidad de diseño, y el **principio de encapsulación**.
- **El lenguaje de "servicio" e "interfaz"** entre capas: capa 4 *usa* un servicio de la 3 y *ofrece* uno a la 5. Ese vocabulario es universal.
- **El concepto de PDU**: "la unidad de datos de la capa N".
- **La comparación explícita** que permite decir "HTTP es un protocolo de capa 7, TCP de capa 4, Ethernet de capa 2".

La frase resumen: *OSI es un buen marco de pensamiento; TCP/IP es una buena arquitectura de red. El pensamiento es lo que se enseña; la arquitectura es lo que se despliega.*

1.5 La PDU: por qué cambia de nombre al bajar de capa

Este es el punto conceptual central, y es lo que convierte "capas" de una lista en un mecanismo.

```
Un byte de información que va de un proceso a otro. Eso es TODO
lo que hay. Lo que cambia es el ENVOLTORIO.

Cap 7  ┌──────────────────────────────────────────────────┐
       │  MENSAJE        500 bytes de datos de la app     │
       └──────────────────────────────────────────────────┘
                           │  la capa 4 le antepone 20 B
Cap 4  ┌──────────────────────────────────────────────────┐
       │  SEGMENTO  [20 B hdr TCP] [ 500 B payload ]      │  520 B
       └──────────────────────────────────────────────────┘
                           │  la capa 3 le antepone 20 B
Cap 3  ┌──────────────────────────────────────────────────┐
       │  PAQUETE  [20 B hdr IP] [ 520 B de segmento ]    │  540 B
       └──────────────────────────────────────────────────┘
                           │  la capa 2 le antepone 18 B
Cap 2  ┌──────────────────────────────────────────────────┐
       │  TRAMA  [14 B MAC] [540 B][4 B FCS] + 8 preamble │  566 B
       └──────────────────────────────────────────────────┘
                           │  la capa 1 codifica
Cap 1  │  BITS  ·01001100 10010110 00001011 ...           │
       └──────────────────────────────────────────────────┘
```

Las tres ideas que hay que extraer:

1. **Es la misma información.** Los 500 bytes de la aplicación son los mismos de punta a punta, byte por byte, atravesando todas las capas. Lo que cambia es el **envoltorio** y el **nombre** que le da la capa que lo está sosteniendo.
2. **El nombre describe la unidad que esa capa manipula.** La capa 2 manipula **tramas**, entonces llama "trama" a lo que tiene en las manos. La capa 4 manipula **segmentos**, entonces llama "segmento" a lo suyo. No son cinco cosas diferentes: son **la misma cosa vista desde cinco puntos de vista**.
3. **Cada capa solo conoce la de arriba y la de abajo.** El router recibe una trama, quita los 18 bytes de capa 2, y lo que le importa son **solo los 20 bytes de cabecera IP**. Los 20 bytes de TCP y los 500 de aplicación **le son indiferentes**: los copia de un lado al otro sin mirarlos. Ese es el significado operativo de la encapsulación.

La definición formal de PDU (*Protocol Data Unit*): la unidad de datos que una capa puede enviar o recibir a la capa de arriba o de abajo. Es la unidad **de un protocolo**, y por eso el nombre: *datos de protocolo*.

En TCP/IP son menos porque hay menos capas, pero el mecanismo es idéntico:

```
Aplicación   : MENSAJE  500 B
Transporte   : (TCP)  520 B      ← el "segmento TCP" de OSI
Internet     : (IP)   540 B      ← el "paquete" de OSI
Acceso red   : (Eth)  566 B      ← la "trama" de OSI
```

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **Las 7 capas de OSI**, en orden y con sus PDU: mensaje, datos de presentación, datos de sesión, segmento, paquete, trama, bit. - **El orden de las capas de arriba hacia abajo** y que la capa de aplicación es la única que ve la red como servicio. - **Las dos hipótesis de la ISO** (minimizar el flujo en las interfaces; no mezclar funciones no relacionadas). - **Que Presentación y Sesión no existen en TCP/IP**, y la razón: casi no se usan por separado. - **Que el modelo TCP/IP tiene 4 o 5 capas según el autor**, y que la diferencia está en la capa de acceso a la red. - **Internet ≠ Web.**

Cambió o se modificó — y acá es donde hay que estar fino:

- **La capa 4 ya no es solo TCP y UDP.** Hoy hay **QUIC (`RFC 9000`, 2021)** que es un transporte completo con su propia recuperación de pérdida y su propio control de congestión, y que corre sobre UDP. `HTTP/3` (`RFC 9114`, 2022) **reemplaza a TCP por completo**: no hay handshake de 3 vías, no hay conexión TCP, no hay retransmisión de conexión. ⇒ **Si el examen pregunta "¿qué protocolos hay en la capa de transporte?", la respuesta completa es TCP, UDP, SCTP, DCCP y QUIC** — y el dato que suma es que QUIC está implementado en el espacio de usuario, no en el kernel.
- **La capa 3 tiene dos versiones, y solo una es la "de Internet".** `RFC 8200` — *IPv6 Specification* (2017, *Internet Standard*), que **obsolesce** `RFC 2460`. El curso de 2011 enseñaba IPv6 como algo experimental: hoy es obligatorio entender que IPv4 y IPv6 **coexisten** y que la capa 3 tiene dos formatos de cabecera.
- **ARP sigue existiendo en IPv4 y disappeared en IPv6.** En IPv6 lo reemplaza **NDP (Neighbor Discovery)**, que usa mensajes ICMPv6 (`RFC 4443`) en multicast en vez de ARP en broadcast. ⇒ **En IPv6, la capa 3 tiene resuelta la resolución de direcciones a nivel de protocolo, con detección de direcciones duplicadas (DAD) incluida.**
- **El cifrado de la capa de presentación se mudó de sitio dos veces.** En 2011 se enseñaba SSLv3/TLS 1.0-1.2. Hoy la especificación vigente es **TLS 1.3 (`RFC 8446`, 2018), actualizada por `RFC 9846` (jul-2026)**, y **los TLS 1.0 y 1.1 pasaron a *Historic* (`RFC 8996`, 2021)**: no se renegocian. La "capa de presentación" de OSI, si existiera, sería **una biblioteca (TLS) entre la aplicación y el socket**, no una capa.
- **La capa de enlace ya no tiene half-duplex en redes de área local de alta velocidad.** En IEEE 802.3, el **full-duplex point-to-point es obligatorio a 10 Gb/s y más**. Y hay una novelty que contradice la intuición: **IEEE 802.3cg (10BASE-T1S, para automatización industrial) es obligatoriamente half-duplex**. ⇒ **La división "capa 2 = full duplex" ya no es una regla universal.**
- **Los nombres de las capas cambiaron con la documentación de IETF.** `RFC 1122` sigue hablando de "capa Internet", "capa de transporte", "capa de aplicación" y "acceso a la red". Cuando leés una RFC de IETF, **buscá esas palabras**, no "interred".

> **Consecuencia para el examen:** la estructura de capas **no cambió** en 37 años. Lo que cambió son **los protocolos que las llenan**. Si te preguntan "¿qué hay en la capa N?", contestá con la tabla de la §1.2 y después aclarás "hoy, además de TCP, QUIC" o "hoy, IPv6". Si te preguntan por el modelo, contestá **la arquitectura**, no las versiones.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — ¿En qué capa vive cada cosa? (el ejercicio diagnóstico más útil)

Enunciado. Ubicar en la capa OSI y en la capa TCP/IP: (a) la dirección MAC; (b) la dirección IP; (c) el número de puerto 443; (d) el `GET /index.html HTTP/1.1`; (e) el CRC-32; (f) el preámbulo `10101010...`; (g) la codificación NRZI; (h) la dirección del registro DNS; (i) el campo `window` de TCP; (j) el `time-to-live`; (k) la tabla de MAC de un switch; (l) el checksum de IP.

Resolución.

Elemento	OSI	TCP/IP	**Por qué**
(a) MAC (48 bits)	**2** Enlace	Acceso a red	Es dirección **de un enlace**: se reescribe en cada router
(b) IP (32 bits)	**3** Red	Internet	Es dirección **global**: no cambia de router a router
(c) Puerto 443	**4** Transporte	Transporte	Identifica un **proceso** dentro del host
(d) `GET /index.html HTTP/1.1`	**7** Aplicación	Aplicación	Es un mensaje de protocolo de aplicación
(e) CRC-32	**2** Enlace	Acceso a red	Detecta errores **de un enlace**, no del camino
(f) Preámbulo `1010...`	**1** Física	Acceso a red	Sincroniza el reloj del receptor: es señal, no datos
(g) NRZI	**1** Física	Acceso a red	Es **codificación de línea**: cómo el voltaje representa un bit
(h) Registro DNS (nombre → IP)	**7** Aplicación	Aplicación	DNS es un protocolo de aplicación, aunque use la capa 3 por debajo
(i) `window` de TCP	**4** Transporte	Transporte	Es **control de flujo** de la conexión
(j) TTL de IP	**3** Red	Internet	Evita bucles en la capa que enruta
(k) Tabla MAC del switch	**2** Enlace	Acceso a red	Es el estado interno de la función de reenvío
(l) Checksum de IP	**3** Red	Internet	Solo verifica **la cabecera**, no los datos

Errores típicos:

- **Meter la MAC en capa 3.** La MAC es de capa 2, siempre. La IP es de capa 3, siempre. Si el examen te da un paquete con IP `192.168.1.5 → 8.8.8.8` y dos MAC distintas en dos enlaces, lo que cambia es la capa 2 y lo que no cambia es la capa 3. Ese es el heart del Examen 2011-03 Problema 2.
- **Decir que el puerto 443 es de capa 7.** El puerto 443 es un **campo de la cabecera TCP o UDP**, o sea **capa 4**. Lo que corre en el puerto 443 (HTTPS) es de capa 7. La distinción esFields vs. contenido.

- **Meter DNS en capa 3.** DNS corre **encima de UDP, que corre encima de IP**. Es de capa 7 aunque su respuesta (la IP del nombre buscado) sea el insumo de la capa 3.

2.2 Ejercicio 2 — Las tres formas de decir lo mismo

Enunciado. Expresar el mismo hecho de tres maneras, y decir cuál se usa en cada contexto.

Resolución.

```
HECHO: "un router recibe un paquete destined a 10.0.0.7 y lo reenvía
        por la interfaz eth1, porque en la tabla de reenvío el prefijo
        10.0.0.0/8 tiene como siguiente salto 192.168.1.2 y el mejor
        match es de 16 bits."

DICIÉNDOLO COMO PROTOCOLO (lenguaje de campos):
  ├─ "El paquete tiene TTL=64 y prefijo destino 10.0.0.7.
  │   Tras longest prefix match en la FIB, la entrada winner es
  │   10.0.0.0/8 con next-hop 192.168.1.2 en eth1.
  │   Se decrementa el TTL a 63 y se recalcula el checksum."

DICIÉNDOLO COMO CAPAS (lenguaje de la arquitectura):
  ├─ "Eso es capa 3: la capa 2 vio la trama, vio que la MAC destino
  │   es la suya, se la entrega a la capa 3, y la capa 3 decide por
  │   la dirección IP. La capa 3 le devuelve a la 2 'mandala por
  │   eth1'; la 2 la arma con la MAC de eth1 y la transmite."

DICIÉNDOLO COMO FUNCIONES (forwarding vs. routing):
  ├─ "Eso es FORWARDING: la decisión por paquete usando la tabla.
  │   El ROUTING es lo que puso esa entrada en la tabla."
```

Por qué importa esto. El mismo paquete se puede describir con tres vocabularios, y cada uno tiene su lugar:

- El **vocabulario de campos** es el de los **exámenes con figuras**: "indique los campos de la trama".
- El **vocabulario de capas** es el de las **preguntas teóricas**: "qué problema resuelve esta capa".
- El **vocabulario de funciones** es el de la Pregunta 1 del Examen 2011-03 y la Pregunta 2a del 2009-02.

Si entendés los tres, entendiste la arquitectura. Si entendés solo uno, memorizaste.

2.3 Ejercicio 3 — ¿Por qué no existe "capa de presentación" en Internet?

Enunciado. La capa de Presentación de OSI resolvería *endianness*, formatos, compresión y cifrado. En TCP/IP ninguna de esas cuatro cosas tiene una capa propia. **Explicá dónde viven hoy cada una, y decí qué consecuencias tiene que no exista la capa.**

Resolución paso a paso.

Paso 1 — ¿Dónde vive cada función hoy?

Función	¿Dónde está en TCP/IP?	Consecuencia
Endianness de **IP, TCP, UDP**	**Ya está resuelto en la capa 3 y 4**: los campos de longitud de más de 8 bits van en network byte order	Ninguna: se resolvió **diseñando el protocolo**
Endianness de **los datos**	**En la capa 7**: cada protocolo de aplicación define su representación, y las bibliotecas (JSON, protobuf, DER) la normalizan	Cada protocolo define la suya ⇒ **interoperabilidad a mano**
Compresión	En la **capa 7**: `Content-Encoding: gzip, br` en HTTP, o en la 4 (TCP no comprime)	El **endpoint** tiene que soportarlo; si no, el mensaje no se entiende
Cifrado	En la **capa 7** (TLS) o en la **capa 3** (IPsec) o en la **4** (DTLS sobre UDP)	Ninguna: se resolvió eligiendo **dónde poner el candado** según el caso
Formatos de archivo	En la **capa 7**: JPEG es JPEG, y punto	El formato se **normaliza** por consenso, no por capa

Paso 2 — La consecuencia de fondo: la "capa de presentación" nunca fue necesaria como capa.

Cada función tiene **una** implementación natural, y la red la puso ahí:

- *Endianness*: se resolvió una vez y para siempre en la especificación de IP. No hace falta una capa para decir "esto es big-endian".
- Cifrado: **TLS** resuelve confidencialidad + integridad + autenticación a la vez, y es una biblioteca que se usa en la capa 7. Ya no hay "tres capas de OSI" que programar.
- Compresión: hoy se hace en HTTP como una cabecera, y se negocia. Está donde tiene sentido.

Paso 3 — El costo de que no exista la capa, que es lo que hay que decir para el punto extra.

La capa de Presentación de OSI, si existiera, sería **un punto único donde todas las heterogeneidades se resuelven**. Sin ella:

- **Cada aplicación resuelve su propia heterogeneidad**, y eso se nota: un archivo CSV con `;` en vez de `,` rompe un importador. Un JPEG progresivo no lo abre un visor de 2005. Un `float` de 8 bytes en little-endian donde se esperaba 4 en big-endian.
- **No hay un punto único donde auditar la confidencialidad**: si cifrás en la aplicación, el proxy de la empresa no puede inspeccionar; si cifrás en la capa 3 (IPsec), el proxy tampoco.
- **La interoperabilidad es un trabajo manual**, no una propiedad de la arquitectura.

Paso 4 — Y el contraargumento, que es lo que hace buena la respuesta.

La razón por la que **funciona igual** es que la heterogeneidad **se movió hacia arriba y hacia el estándar, no hacia una capa**:

```
OSIPensaba:  heterogeneidad → CAPA DE PRESENTACIÓN (una sola)
Realidad:     heterogeneidad → PROTOCOLOS Y FORMATOS (muchos)

┌──────────────────────┬──────────────────────────────────┐
│ OSI                  │ Realidad                         │
├──────────────────────┼──────────────────────────────────┤
│ Una capa             │ Muchos formatos estándar:        │
│ central              │ JSON, protobuf, DER,             │
│                      │ PEM, X.509, JPEG, PNG, MP4       │
│ Un punto de fallo    │ Cada uno resuelve SU caso,       │
│ si no está           │ y ninguno resuelve todos         │
└──────────────────────┴──────────────────────────────────┘
```

La frase para el examen: *no tener una capa de presentación no es un defecto de TCP/IP: es el reconocimiento de que la heterogeneidad de representación nunca fue un problema de red, sino un problema de formatos, y se resolvió con formatos estándar en la aplicación.*

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2006-12, Pregunta 1 (5 puntos) — el modelo OSI y las hipótesis

> **Fuente:** Examen `sirc0612` — 26 de diciembre de 2006, Pregunta 1 (5 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0612.pdf

Enunciado (texto del examen)

> Describa el modelo de capas OSI, y las capas que lo integran.
>
> ⚠️ **Nota sobre el puntaje:** el enunciado oficial dice **5 puntos** (verificado en `sirc0612.txt`), no 10. Tomá 5 como referencia.

Solución oficial (completa)

> El modelo de referencia OSI (OSI - Open System Interconnection presentado por la ISO - International Standard Organization). Modelo utilizado por la mayoría de los diseñadores.
>
> Las hipótesis manejadas para llegar a 7 capas son las siguientes: - Las capas deben delimitarse buscando minimizar el flujo de datos que se intercambia entre capas por la interfaz - El número de capas debe ser suficientemente amplio para evitar que diferentes funciones no se requieran colocar en la misma capa.
>
> El modelo de referencia OSI:
>
> **Capa Física** - Esta capa tiene la tarea de la transmisión de bits a través de canales de comunicación. - Esta capa tiene mucho que ver con las interfaces eléctricas y mecánicas para la transmisión de bits.
>
> **Capa Enlace de Datos** - La tarea fundamental es tomar un medio de transmisión, y transformarlo en una línea que parezca libre de errores. - En general esto involucra generar frames de datos, y chequear la corrección de los frames llegados, así como pedir retransmisión de frames con errores o perdidos.
>
> **Capa de Red** - Se encarga de controlar el funcionamiento de la subred. - Su tarea fundamental es encaminar paquetes.
>
> **Capa de Transporte** - La función básica de la capa transporte es dar un flujo de datos sin errores ni fallas a la capa sesión. - Se generará una conexión por donde se transferirá toda la información, garantizando la no existencia de: - perdida de información - duplicación de información.
>
> **Capa de Sesión** - Se busca que dos máquinas diferentes puedan establecer sesiones.
>
> **Capa de Presentación** - Busca la solución de problemas particulares, que hacen referencia a la semántica y sintaxis de lo enviado. - Por ejemplo la resolución de diferentes representaciones entre máquinas (pto flotante), corresponde a esta capa.
>
> **Capa de Aplicación** - En esta capa se tratan aplicaciones que se ejecutan en la red. - Como ejemplo podemos tomar transferencia de archivo (ejemplo FTP), Correo Electrónico (SMTP), emulador de terminal TELNET.

Análisis de la solución oficial

Lo que está muy bien:

1. **Nombra las dos hipótesis antes de las capas.** Es lo que la pregunta pide: *las hipótesis* que llevan a 7 capas. Muchos alumnos lasListed al final, después de enumerar. La oficial las pone primero, que es lo correcto: primero el criterio, después la aplicación.
2. **La capa de Enlace es la mejor descripción de todas las siete.** "*Tomar un medio de transmisión y transformarlo en una línea que parezca libre de errores*" es una frase excelente: define la capa por **efecto** (que parezca libre de errores) y no por **mecanismo**. Y agrega los tres mecanismos concretos: framing, chequeo, retransmisión.
3. **La capa de Transporte define el objetivo en términos de la de arriba**: "*dar un flujo de datos sin errores ni fallas a la capa sesión*". Justamente el principio de "cada capa le da a la de arriba lo que esa necesita".
4. **La capa de Presentación da el ejemplo del punto flotante**, que es el mejor ejemplo posible de heterogeneidad de representación.

Lo que falta o está comprimido — y son los puntos que se ganan:

1. **No nombra la PDU de cada capa.** La oficial no dice "trama", "paquete", "segmento", "mensaje". **Es el dato más fácil de agregar y el más rápido de verificar: agregalo.**
2. **La capa de Red es muy escueta.** "*Se encarga de controlar el funcionamiento de la subred. Su tarea fundamental es encaminar paquetes.*" No menciona: **direccionamiento lógico (IP)**, **reenvío vs. enrutamiento** (que es la Pregunta 1 del Examen 2011-03, 8 puntos), **longest prefix match**, **TTL**.
3. **La capa Física menciona "interfaces eléctricas y mecánicas".** Correcto, pero falta lo que la distingue: **codificación de línea y sincronismo**, y el hecho de que **no detecta errores**. La frase que suma es: "*la capa física no sabe si el bit que acaba de recibir es un dato o un relleno*".
4. **La capa de Sesión no menciona para qué sirve.** "*Se busca que dos máquinas diferentes puedan establecer sesiones*". Falta lo importante: **sincronización y puntos de recuperación** (*checkpoints*), que es lo que la distingue de la capa de Transporte (que establece una conexión pero no sincroniza el contenido).
5. **No dice que la ISO no implementa el modelo.** "*Modelo utilizado por la mayoría de los diseñadores*" es cierto y es lo que salva a OSI de ser inútil: **es un modelo de referencia, no una implementación**. Nadie implementa las 7 capas. La frase que suma: "*OSI es un modelo de referencia, una arquitectura para especificar protocolos; ningún producto implementa las 7 capas.*"

Mi respuesta completa (para los 5 puntos oficiales, ampliada)

```
MODELO OSI (ISO 7498) — arquitectura de REFERENCIA

LAS DOS HIPÓTESIS que llevan a 7 capas:
 H1. Las capas se delimitan buscando MINIMIZAR el flujo de datos que
     se intercambia entre capas a través de la interfaz.
 H2. El número de capas debe ser lo bastante amplio como para que
     funciones que no se relacionen NO queden en la misma capa.

_____________________________________________________________________

7. APLICACIÓN      PDU: mensaje/datos
    La red como servicio para el programa.
    HTTP, DNS, SMTP, SSH, FTP.

6. PRESENTACIÓN    PDU: datos de presentación
    Resuelve que los datos de un host pueden NO ser compatibles
    con los de otro: representación (endianness, punto flotante),
    formato, compresión, cifrado.

5. SESIÓN          PDU: datos de sesión
    Establecer, mantener y terminar el DIÁLOGO entre dos
    aplicaciones: sincronización y puntos de recuperación.

4. TRANSPORTE      PDU: segmento (TCP) / datagrama de usuario (UDP)
    Que la aplicación se independice de la red: entrega a un
    proceso, con fiabilidad, orden, control de flujo y de
    congestión.  TCP, UDP.

3. RED             PDU: paquete / datagrama
    Cómo llegar de una red a otra: direccionamiento lógico (IP),
    reenvío (forwarding) y enrutamiento (routing), TTL.
    IP, ICMP, ARP.

2. ENLACE DE DATOS PDU: trama / frame
    Que dos nodos de un MISMO enlace se comuniquen sin errores:
    framing, detección (CRC), corrección (ARQ), control de acceso
    al medio.  Direcciones MAC, que son locales al enlace.
    Ethernet, Wi-Fi, PPP.

1. FÍSICA          PDU: bit
    Que los bits viajen: medio físico, señal eléctrica u óptica,
    conectores, codificación de línea, sincronismo de reloj.
    NO detecta errores, NO sabe qué hay en el paquete.
```

Qué agregué respecto de la oficial y por qué:

Agregado	Por qué suma
La PDU de cada capa	Es el dato mecánico, y se verifica fácil. 7 datos en una línea cada uno.
"Que parezca libre de errores" para Enlace	Ya está en la oficial; lo mantuve porque es la mejor frase del conjunto.
La dirección MAC es local al enlace	Justifica por qué el router reescribe la trama: el puente de la identidad de capa 2.
"No detecta errores" en Física	Contraste con la capa 2: es lo que hace que la 1 sea tan simple.
Forwarding vs. routing en Red	Es el contenido de la Pregunta 1 del Examen 2011-03 (8 pts). Decirlo muestra que entendés la capa.
"Modelo de referencia, nadie lo implementa"	Salva a OSI de ser inútil y es la respuesta a "¿y si OSI es tan bueno?"

3.2 ⭐ Examen 2007-03, Pregunta 1 (5 puntos) — el modelo TCP/IP

> **Fuente:** Examen `sirc0703` — 26 de febrero de 2007, Pregunta 1 (5 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0707.pdf

Enunciado (texto del examen)

> Describa el modelo de capas TCP/IP.

Solución oficial (completa)

> El modelo TCP/IP surgió a partir de la red ARPANET, cuyo mayor objetivo de diseño era interconectar varias redes en forma transparente, dicha arquitectura evolucionó en el modelo TCP/IP.
>
> **La capa Host a Red** Especifica cómo se envían físicamente los datos a través de la red, que incluye cómo se realiza la señalización eléctrica de los bits mediante los dispositivos de hardware que conectan directamente con un medio de red, como un cable coaxial, un cable de fibra óptica o un cable de cobre de par trenzado. No está definido el protocolo y el mismo puede variar en diferentes implementaciones.
>
> **La capa de Interred** Esta capa es la que mantiene unida toda la arquitectura. Permite a los host enviar paquetes a cualquier red y que viajen independientemente del destino (potencialmente otra red). Pueden incluso llegar en diferente orden, en cuyo caso es trabajo de capas superiores, reordenarlos. La capa de interred define un formato de paquetes llamado IP. El trabajo de la capa de interred es que dichos paquetes lleguen a donde corresponde, por lo que el ruteo de los paquetes es el mayor problema. La capa de interred es similar en su funcionamiento a la capa OSI de Red.
>
> **Capa de Transporte** Sobre la capa de interred se define la capa de transporte, la misma está diseñada para conectar dos entidades y mantener un enlace de datos. Hay dos protocolos definidos: TCP (Transmission Control Protocol), es un protocolo orientado a conexión, confiable, que permite que un flujo de bytes sea enviado de una máquina a otra sin errores; fragmenta el flujo de bytes y lo pasa a la capa de interred, al llegar a destino reconstruye el flujo de bytes; TCP también maneja el control de flujo para evitar sobrepasar la capacidad de un receptor más lento. El segundo protocolo, UDP (User Datagram

> Protocol) es un protocolo no confiable sin conexión, para aplicaciones que no necesitan secuenciamiento de paquetes o control de flujo, o aplicaciones donde es más importante transmitir rápido que en forma confiable, como voz o video.
>
> **Capa de aplicación** El modelo TCP/IP no tiene capas de sesión o presentación, no se pensó que fueran necesarias, por lo que no fueron incluidas. En la práctica se prueba que son muy poco usadas. Por lo que en el modelo arriba de la capa de transporte se encuentra la capa de aplicación, que contiene todos los protocolos de alto nivel, como TELNET, FTP, SMTP, DNS, NNTP, HTTP, etc.

Análisis de la solución oficial

Lo que está muy bien:

1. **Empieza por el origen histórico**: "*surgió a partir de la red ARPANET, cuyo mayor objetivo de diseño era interconectar varias redes en forma transparente*". Esa es la razón de diseño de TCP/IP, y es exactamente el criterio de H2 aplicado: la función de la capa de Interred (conectar redes heterogéneas) es **distinta** de la de la capa de Enlace (comunicar dos vecinos), así que van separadas. Si hubieran ido al modelo OSI a ciegas, habrían fusionado red y enlace.
2. **La capa Interred está muy bien descrita**: "*permite a los host enviar paquetes a cualquier red y que viajen independientemente del destino*", "*pueden incluso llegar en diferente orden, en cuyo caso es trabajo de capas superiores, reordenarlos*". La observación de que **IP es un servicio sin orden** y de que eso es una decisión de diseño (a cambio de robustez en el camino) vale por sí sola varios puntos.
3. **TCP vs. UDP con la razón de la elección**: UDP es para "*aplicaciones donde es más importante transmitir rápido que en forma confiable, como voz o video*". La oficial entiende que UDP no es "TCP malo": es una **decisión de diseño** para un caso concreto.
4. **"El modelo TCP/IP no tiene capas de sesión o presentación… en la práctica se prueba que son muy poco usadas."** Esa es la respuesta a la pregunta implícita "¿por qué TCP/IP tiene menos capas?", y la oficial la responde con la evidencia.

Lo que falta o está comprimido:

1. **"Host a Red" en vez de "Acceso a red".** Es el nombre de Tanenbaum. Kurose lo llama **"Acceso a la red"** y lo trata como **una** capa que incluye física y enlace. Si el examen dice "4 capas", decí "acceso a red" y aclarás que incluye las dos bajas de OSI. La oficial usa el nombre de Tanenbaum, coherente con que la cátedra usa Tanenbaum como libro de referencia.
2. **La capa Interred no menciona ARP.** Y ARP es de **capa 3** (o de 2½, según el autor: resuelve una dirección de capa 3 con una de capa 2, y por eso Tanenbaum lo pone en la capa de red). La oficial tampoco menciona **ICMP**, que es la otra pieza fundamental de capa 3.
3. **No dice "TCP/IP tiene 4 o 5 capas según el autor".** Este es el punto que la oficial no toca y que un buen examen sí podría pedir. Si la respuesta dice "4 capas" y el libro de la cátedra tiene 5, hay un problema. Hay que resolverlo.
4. **No nombra las PDU.** Mismo faltante que en el 2006-12.
5. **No menciona los otros protocolos de transporte.** Hoy hay **SCTP** y **QUIC** (`RFC 9000`), y QUIC es el dato de 2026.

Mi respuesta completa (para los 5 puntos)

```
MODELO TCP/IP — 4 capas (Kurose) o 5 capas (Tanenbaum) según
cómo se separe la capa de acceso a la red.

ORIGEN: surgió de ARPANET, cuyo objetivo de diseño era
interconectar redes HETEROGÉNEAS de forma transparente. Por eso
la capa de Red (Interred) es distinta de la de Enlace y no
puede fusionarse con ella.

_____________________________________________________________

4. APLICACIÓN
   PDU: mensaje
   Los protocolos de alto nivel: HTTP, DNS, SMTP, FTP, SSH, DHCP.
   NO tiene capas de sesión ni de presentación: sus funciones
   viven acá (el cifrado, en la biblioteca TLS; los formatos, en
   los protocolos de aplicación). En la práctica casi nunca se
   usan por separado, y por eso no se separaron.

3. TRANSPORTE
   PDU: segmento (TCP) / datagrama de usuario (UDP)
   Conecta dos ENTIDADES (procesos), no dos máquinas. Entrega un
   flujo de bytes de un proceso a otro, independiente de la red.
   - TCP: orientado a conexión, confiable, con control de flujo
     y de control de congestión. Numeración de bytes.
   - UDP: sin conexión, no confiable, mínimo (8 bytes de cabecera).
     Para voz, video y DNS: donde importa la oportunidad más que
     la garantía.
   - SCTP y QUIC (RFC 9000) también son capa 4.

2. INTERNET / INTERRED
   PDU: paquete / datagrama
   Es la capa que MANTIENE UNIDA la arquitectura: el objetivo
   de ARPANET. Entrega paquetes "best effort" a cualquier
   destino, INDEPENDIENTEMENTE del camino: los paquetes pueden
   perderse, duplicarse o llegar en orden distinto, y eso es
   problema de las capas superiores.
   Define el formato IP. Direccionamiento lógico jerárquico.
   Reenvío (forwarding) y enrutamiento (routing).
   IP, ICMP, ARP (que resuelve IP a partir de la MAC del enlace).
   Es la equivalente a la capa 3 de OSI.

1. HOST A RED / ACCESO A LA RED
   PDU: trama (enlace) y bit (física)
   Especifica cómo se envían físicamente los datos: señalización
   eléctrica de los bits con el hardware que conecta al medio
   (coaxial, fibra, par trenzado). NO ESTÁ DEFINIDO UN PROTOCOLO
   único: varía por implementación.
   Incluye la capa física (medio, señal, conectores) y la de
   enlace (framing, MAC, CRC, acceso al medio).
```

Qué agregué respecto de la oficial y por qué:

Agregado	Por qué suma
ARP e ICMP en la capa Internet	Son las dos piezas que hacen que la capa funcione. La oficial solo menciona IP.
"Es la capa que mantiene unida la arquitectura"	Está en la oficial y es la mejor frase: define la capa por su **para qué histórico**. La repetí arriba porque es el corazón de la pregunta.
"Conecta dos entidades, no dos máquinas"	Es lo que distingue la capa 4 de la 3, y es lo que justifica que exista como capa separada.
SCTP y QUIC	El dato de 2026. Si el examen es de la cátedra actual, mentionar QUIC muestra que la lectura es actual.
"NO ESTÁ DEFINIDO UN PROTOCOLO único"	Está en la oficial y es un punto clave: es la capa donde todo es implementación.
La nota 4 vs. 5	Resuelve la ambigüedad antes de que te la planteen.

PARTE 4 — CIERRE

📌 Resumen

1. El modelo de capas existe para que **cada subsistema se comunique con el de arriba y el de abajo solo por una interfaz**. Eso permite que un fabricante de switches y otro de routers se entiendan.
2. **H1**: minimizar el flujo de datos a través de las interfaces. **H2**: funciones que no se relacionan no deben estar en la misma capa.
3. **Las 7 de OSI** con su PDU: **Aplicación** (mensaje) · **Presentación** (datos de presentación) · **Sesión** (datos de sesión) · **Transporte** (segmento) · **Red** (paquete) · **Enlace** (trama) · **Física** (bit).
4. **TCP/IP tiene 4 (Kurose), 5 (Tanenbaum) o 7 (OSI)** según el autor: la diferencia está en si se separa enlace de física, y en que **Sesión y Presentación no existen**.
5. **Presentación resuelve que los datos de un host pueden no ser compatibles con los de otro**: *endianness*, formatos, compresión, cifrado. En TCP/IP eso vive en Aplicación (y en TLS).
6. **Sesión** = diálogo: sincronización y puntos de recuperación. **Transporte** = flujo confiable a un proceso. La diferencia es contenido, no transporte.
7. **La PDU cambia de nombre** porque es la misma información con distinto envoltorio y cada capa nombra lo que tiene en las manos.
8. **Ganó TCP/IP** porque la red funcionó antes de que la teoría estuviera lista, porque OSI tardó 20 años en estandarizar la capa 3, porque TCP/IP era gratis, y porque en 1983 nadie quiso dar marcha atrás.
9. **OSI no se implementa**: es un marco de pensamiento. **El vocabulario** (capa, PDU, servicio, interfaz) sí es universal y es lo que hay que aprender.
10. **Internet ≠ Web**: Internet es la red (IP, TCP, BGP); la Web es una aplicación (HTTP, HTML, URL).

✅ Autoevaluación

1. ¿Por qué el modelo de capas no es "un capricho de la ISO"?

▶ Respuesta

2. ¿Por qué TCP/IP fusionó Sesión y Presentación en Aplicación, si la hipótesis H2 dice que funciones distintas no deberían estar en la misma capa?

▶ Respuesta

3. ¿Qué es el *endianness* y por qué es problema de la capa de presentación?

▶ Respuesta

4. En el modelo TCP/IP de Tanenbaum, ¿la capa de Enlace existe? ¿Y en el de Kurose?

▶ Respuesta

5. Si HTTP es de capa 7 y el puerto 443 es de capa 4, ¿cómo es posible que "HTTPS sea de capa 7"?

▶ **Respuesta**

6. Un paquete que pasa por 5 routers, ¿cuántas cabeceras IP tiene y cuántas tramas Ethernet tiene?

▶ **Respuesta**

7. El modelo OSI tiene 7 capas y el TCP/IP tiene 4. ¿Cuál es "mejor" y por qué?

▶ **Respuesta**

8. En el modelo TCP/IP, ¿dónde vive la función "detectar y retransmitir un paquete perdido"?

▶ **Respuesta**

🔗 Referencias

Exámenes - Examen `sirc0612` (26-dic-2006) — P1 (5 pts): modelo OSI e hipótesis · P2 (10 pts): framing · P4 (10 pts): handshake TCP - Examen `sirc0703` (26-feb-2007) — P1 (5 pts): el modelo TCP/IP · P4 (10 pts): campos de TCP y UDP - Examen `sirc1103` (21-feb-2011) — P1 (8 pts): las dos funciones de la capa de red - Examen `sirc0902` (2-feb-2009) — P2 (10 pts): forwarding vs. routing y flooding - Examen `sirc0912` (18-dic-2009) — P2 (10 pts): servicio confiable en transporte vs. enlace

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., **cap. 1 §1.4** "El modelo de capas" (pág. ~20 a ~26 del PDF de tu carpeta) — el modelo de 5 capas y las PDU - Tanenbaum & Wetherall, **cap. 1 §1.5** "El modelo OSI" — las siete capas y la crítica a OSI - Kurose & Ross, *Computer Networking: A Top-Down Approach*, cap. 1 — el modelo de 4 capas - ISO/IEC 7498-1 — *Information technology — Open Systems Interconnection — Reference model* — la norma del modelo OSI

RFCs - `RFC 1122` — *Requirements for Internet Hosts: Communication Layers* (1989): define la arquitectura TCP/IP **con enlace separado** y **sin** capas de sesión ni presentación - `RFC 8200` — *Internet Protocol, Version 6* (2017, *Internet Standard*): la capa 3 de Internet hoy; obsolesce RFC 2460 - `RFC 9293` — *Transmission Control Protocol* (2022, *Internet Standard*; consolida RFC 793) - `RFC 768` — *User Datagram Protocol* (1980): la cabecera de UDP en 8 bytes - `RFC 9000` — *QUIC* (2021): el transporte moderno que **no** es TCP, y que vive en la capa 4

Clase siguiente: 3 — Encapsulación y comparación OSI ↔ TCP/IP

Clase 3

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 Qué es la encapsulación y por qué existe
1.2 El diagrama completo: un byte viajando
1.3 El overhead: la cuenta que siempre preguntan
1.4 Lo que además cambia al bajar de capa
1.5 Servicios e interfaces entre capas
1.6 La capa de presentación, desarrollada
2.1 Ejercicio 1 — Tamaño total y eficiencia para tres tamaños de mensaje
2.2 Ejercicio 2 — El camino completo: ¿quién lee qué?
2.3 Ejercicio 3 — ¿Qué pasa si el mensaje es de 40 bytes?
3.1 ⭐ Examen 2009-02, Pregunta 2 (10 puntos) — forwarding vs. routing y flooding
3.2 Referencia cruzada — la misma pregunta, en otro examen
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: A (Fundamentos) · **Semana:** 1 · **Duración estimada:** 110 min **Correlativa:** Clase 2 (los modelos de capas: OSI y TCP/IP) · **Sigue con:** Clase 4 (conmutación: circuitos, paquetes y mensajes) **Material de referencia:** Tanenbaum cap. 1 §1.4 (pág. ~20 a ~26 del PDF de tu carpeta) · RFC 8200, RFC 9293, RFC 768, RFC 826, RFC 1122

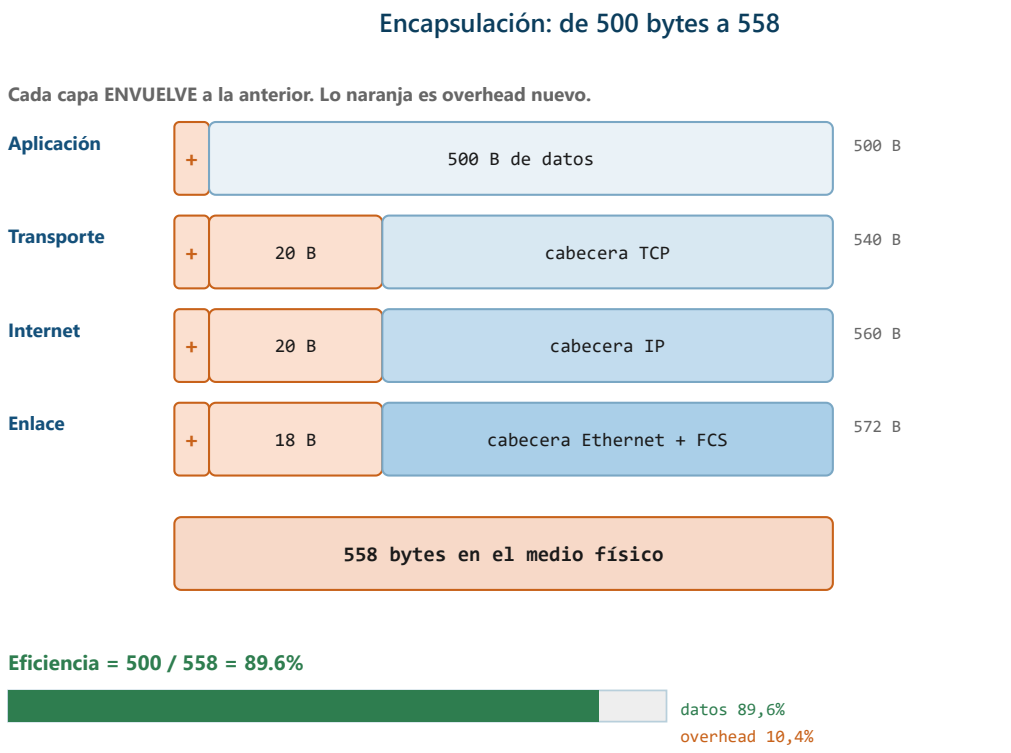
🎯 Al terminar esta clase vas a poder

1. **Definir** encapsulación y desencapsulación, y explicar **por qué existen** (cada capa necesita su propio control).
2. **Hacer la cuenta byte a byte** de un mensaje que baja por la pila: 500 B de aplicación → 558 B en el medio, con la aritmética visible.
3. **Calcular la eficiencia** en función del tamaño del mensaje y explicar **por qué el overhead relativo se dispara con mensajes chicos**.
4. **Dibujar** el camino completo de un mensaje con los dos extremos, y **marcar dónde se invierte** la encapsulación.
5. **Explicar** qué más cambia al bajar de capa además del tamaño: **el direccionamiento** (IP → MAC) y **el objetivo** (llegar al host → llegar al proceso).
6. **Describir** qué es un **servicio** entre capas y por qué es la clave de la encapsulación.
7. **Desarrollar** la capa de Presentación con casos concretos, porque es la que más se olvida.
8. **Resolver entero** el Examen `2009-02 P2` (forwarding vs. routing y flooding), que es la demostración de que la capa 3 existe por el alcance.

> **Prerequisito del examen:** `2009-02 P2` (10 pts — forwarding vs. routing, flooding controlado), `2011-03 Problema 2` (el intercambio de tramas con ARP), `2006-12 P1` y `2007-03 P1` (modelos de capas), `2009-12 P1` (componentes del retardo).

PARTE 1 — TEORÍA

1.1 Qué es la encapsulación y por qué existe



La definición

Encapsular es el proceso por el cual, al bajar por la pila, cada capa **antepone su propia cabecera** (y a veces su trailer) a los datos que le entrega la capa de arriba. **Desencapsular** es lo inverso: al subir, cada capa **quita su propia cabecera** y le entrega a la de arriba la PDU que tenía abajo.

```
                ENCAPSULACIÓN (bajar)
Aplicación  ──────────────────────────────►
Transporte  ──────────────────────────────►   + 20 B
Red         ──────────────────────────────►   + 20 B
Enlace      ──────────────────────────────►   + 18 B
Física      ──────────────────────────────►   bits


                DESENCAPSULACIÓN (subir)
Física      ◄──────────────────────────────
Enlace      ◄──────────────────────────────   − 18 B
Red         ◄──────────────────────────────   − 20 B
Transporte  ◄──────────────────────────────   − 20 B
Aplicación  ◄──────────────────────────────
```

"Encapsular" es "meter algo dentro de una cápsula". La cápsula es la cabecera. Lo que viaja por el medio es la cápsula con el contenido adentro.

Las cuatro razones por las que existe

Razón 1 — Control local. Si cada capa metiera sus cosas **al principio** de la unidad de datos, entonces una capa de arriba no podría volver a leer el mensaje sin recorrer todo. Con la cabecera **delante**, cada capa puede **encontrar y quitar la suya en tiempo constante**: la de transporte está en el byte 0, la de red en el byte 20 (si el offset de datos de transporte es 0) o en el byte 34, y la de enlace en el byte 14 del cable. Es el mismo motivo por el que el **TCP payload offset** existe: para que la capa IP sepa dónde empieza su propia cabecera sin recorrer la trama entera.

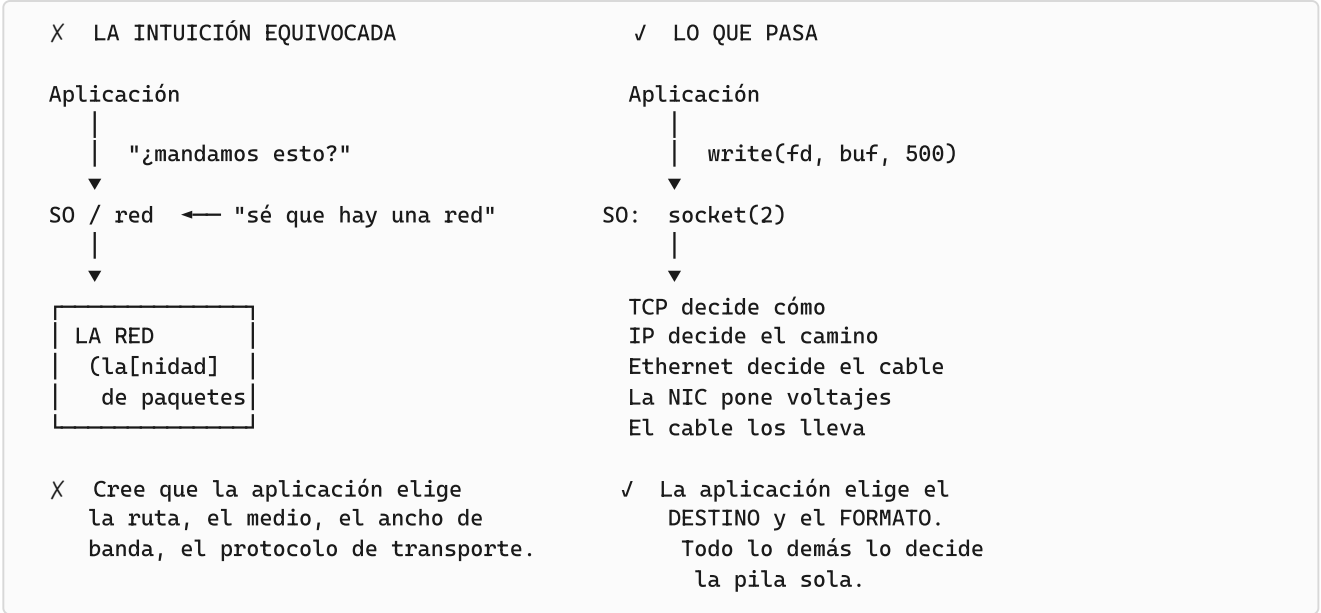
Razón 2 — Multiplexación. Con las cabeceras al principio, un router puede leer la dirección de destino **sin leer el payload**. Mira los 20 bytes de cabecera IP, decide el puerto de salida, y reenvía: **no tiene que abrir el sobre**. Si la dirección estuviera al final, el router tendría que recorrer los 1500 bytes para enterarse adónde va. Es la razón de performance más importante.

Razón 3 — cada capa habla su propio idioma. La capa de enlace necesita decir "esto es una trama Ethernet con destino esta MAC"; la capa de red necesita decir "esto va a 10.0.0.7 con TTL 63". Son **afirmaciones que ninguna otra capa puede hacer ni verificar**. La cabecera es el lugar donde cada capa deja su información.

Razón 4 — Independencia. Si el medio pasa de cobre a fibra, **solo cambia la capa 1**; los bytes de las capas 2 a 7 no se tocan. Si IP pasa de v4 a v6, **solo cambia la capa 3**; TCP no se entera. La encapsulación es lo que hace posible el **intercambio de una capa sin arrastrar a las demás**.

El error conceptual más frecuente: la aplicación "ve" la red

Hay una intuición muy extendida y **equivocada**: que la capa de aplicación "sabe" que está usando una red, o que "el paquete viaja por la red como una entidad visible".



Por qué importa en el examen. Si creés que la aplicación elige el protocolo de transporte, no entendés por qué **HTTP sobre TCP y HTTP sobre QUIC usan la misma aplicación** (el navegador), ni por qué **un `ping` no usa ninguna aplicación**. Si creés que la aplicación "ve" los paquetes, no entendés por qué **un sniffing a nivel de socket no ve nada de IP**.

La frase exacta: *la capa de aplicación nunca ve la red. Le entrega bytes a la capa de transporte mediante una interfaz, y recibe bytes. La red existe para ella como un detalle de implementación de la máquina donde corre.*

1.2 El diagrama completo: un byte viajando

Los datos

```
Aplicación en H1 escribe una solicitud HTTP de 500 bytes.
L_app = 500 bytes

Cabeceras del escenario:
  cabecera TCP           = 20 bytes  (sin opciones)
  cabecera IPv4          = 20 bytes  (sin opciones)
  trama Ethernet         = 18 bytes  (14 de cabecera + 4 de FCS)
  preámbulo + IFG        =  8 bytes  (7 preámbulo + 1 SFD, + 12 de IFG
                                      que van entre tramas, NO dentro)
```

> ⚠️ **Precisión que importa:** el **preámbulo (7 bytes) y el SFD (1 byte)** son parte de la trama en el medio, pero **no forman parte de la PDU de capa 2** que se le entrega a la capa 3. Y el **IFG (Inter Frame Gap) de 96 bits = 12 bytes** va **entre** tramas, no dentro. Cuando un problema de examen dice "tamaño de la trama Ethernet", casi siempre quiere decir **14 + payload + 4**, sin preámbulo. Lo aclaro abajo.

La cuenta, paso a paso

Paso 1 — Capa 4: el segmento TCP.

```
Segmento TCP = cabecera TCP + datos de aplicación
             = 20 B        + 500 B
             = 520 bytes
```

Paso 2 — Capa 3: el paquete IP.

```
Paquete IP = cabecera IP + segmento TCP
           = 20 B        + 520 B
           = 540 bytes
```

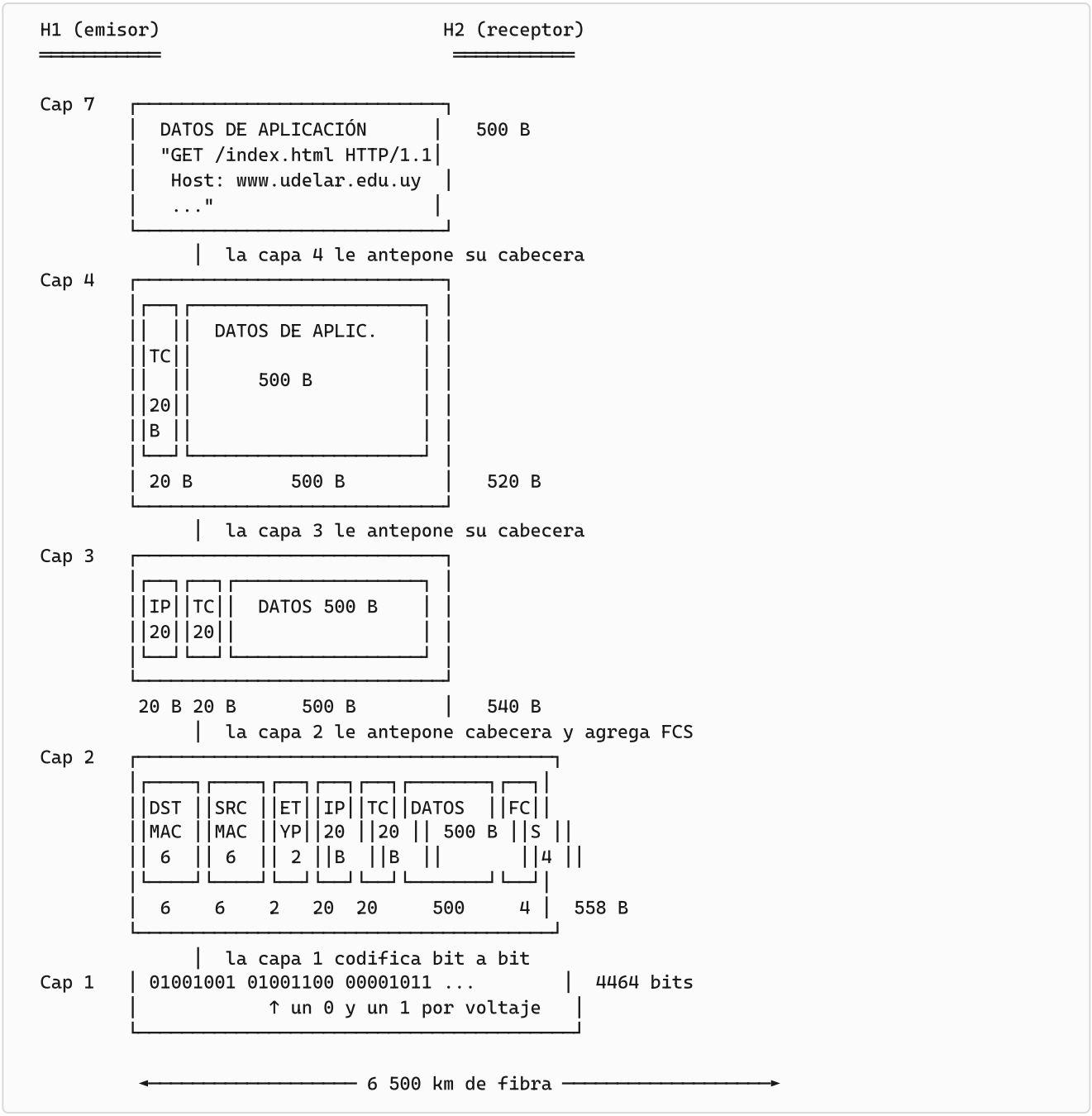
Paso 3 — Capa 2: la trama Ethernet.

```
Trama = MAC dest (6) + MAC orig (6) + Ethertype (2) + datos + FCS (4)
      = 6 + 6 + 2 + 540 + 4
      = 558 bytes
```

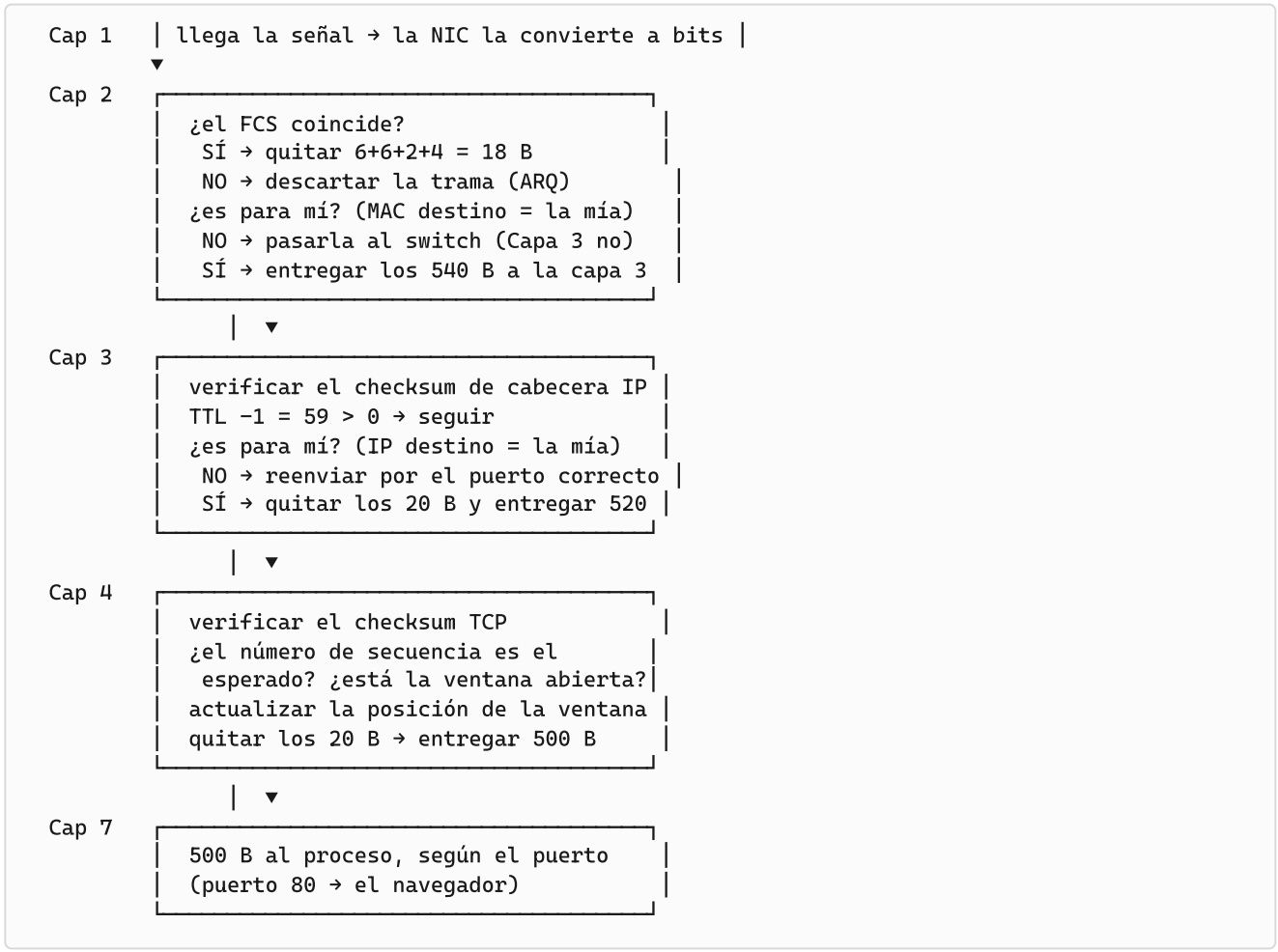
Paso 4 — El total y el overhead.

```
Bytes de datos útiles           = 500
Bytes de overhead              = 20 + 20 + 18 = 58
──────────────────────────────────────────────────
Bytes totales en el medio       = 558

OVERHEAD ABSOLUTO  = 58 bytes
OVERHEAD RELATIVO  = 58 / 558 = 10,39 %
EFICIENCIA         = 500 / 558 = 89,61 %
```

Paso 5 — El diagrama visual del byte viajando.



La desencapsulación, del otro lado



Lo que hay que ver en este diagrama: la curva es simétrica. **Cada capa quita exactamente los bytes que antepuso.** Los 500 bytes que escribió la aplicación de H1 son **bit a bit** los mismos 500 bytes que lee la aplicación de H2. Lo único que hace el camino es **agregar 58 bytes de andamiaje** y sacarlos.

La tabla de tamaños

Capa	PDU	cabecera	datos	total
7 Aplicación	mensaje	0 B	500 B	500 B
4 Transporte	segmento	20 B	500 B	520 B
3 Red	paquete	20 B	520 B	540 B
2 Enlace	trama	18 B	540 B	558 B
1 Física	bits	—	540 B	4464 bits

overhead acumulado: +20 +40 +58

1.3 El overhead: la cuenta que siempre preguntan

La fórmula

```
Para un mensaje de L bytes de datos con cabeceras de tamaño fijo:

                  ┌──────────────────────────────────────┐
                  │                                      │
                  │        L                             │
eficiencia  =     ──────────────────────  =                 │
                  │        L + overhead                  │
                  └──────────────────────────────────────┘

overhead = H_TCP + H_IP + H_ENLACE
         =    20   +  20  +    18      = 58 bytes
```

El punto clave que hay que entender: el overhead es **constante** (58 bytes) pero el mensaje **no**. Entonces:

- Para mensajes **grandes**, el 58 es una fracción chica → eficiencia alta.
- Para mensajes **chicos**, el 58 es una fracción enorme → eficiencia bajísima.

La cuenta con tres tamaños (este es el tipo de pregunta que aparece)

```
overhead = 58 bytes (20 TCP + 20 IP + 18 Ethernet)
```

L_app	total	eficiencia	overhead %	comentario
50 B	50 + 58 = 108 B	50/108 = 46,3 %	53,7 %	¡peor que 50%!
100 B	100 + 58 = 158 B	100/158 = 63,3 %	36,7 %	
500 B	500 + 58 = 558 B	500/558 = 89,6 %	10,4 %	
1500 B	1500 + 58 = 1558 B	1500/1558 = 96,3 %	3,7 %	MTU de Eth.
64 000 B	64000 + 58 =64058 B	64000/… = 99,91 %	0,09 %	

El caso extremo: mensaje de 50 bytes

Este es el que **siempre se pide**, y el número es mengingatkan:

```
L_app = 50 bytes
overhead = 58 bytes
─────────────────────────
total = 108 bytes

eficiencia = 50/108 = 0,4629 = 46,3 %
overhead   = 58/108 = 0,5370 = 53,7 %   ← MÁS DE LA MITAD

El mensaje de aplicación ocupa menos de la mitad de lo que
viaja por el cable. La red está transportando overhead.
```

El caso de 40 bytes (que es el mínimo absoluto):

```
L_app = 40 B
total = 40 + 58 = 98 B
eficiencia = 40/98 = 40,8 %
overhead   = 58/98 = 59,2 %
```

Y el caso de 1 byte:

```
L_app = 1 B
total = 1 + 58 = 59 B
eficiencia = 1/59 = 1,7 %
→ 98,3 % del tráfico es overhead. Absurdo.
```

Por qué esto importa de verdad (no es un ejercicio abstracto)

Motivo 1 — El MTU. El MTU de Ethernet son **1500 bytes de payload**. El tamaño máximo de un datagrama IP que cabe es `1500 − 20 (IP) − 20 (TCP) = 1460 bytes` de datos de aplicación, que es el **MSS (*Maximum Segment Size*) de TCP**. Los 40 bytes restantes los pone Ethernet. Es exactamente la misma cuenta, vista al revés.

Motivo 2 — La Ethernet no permite frames de menos de 64 bytes. Si tu mensaje de aplicación mide 10 bytes, el total sería 68 bytes, que ya supera el mínimo... pero el mínimo existe por otra razón: **el emisor tiene que transmitir al menos 64 bytes para que el receptor pueda tener tiempo de detectar una colisión**. Por eso las tramas se rellenan con *padding* hasta 64 bytes.

Motivo 3 — Protocolos que optimizan. Cuando importan los mensajes chicos:

- **UDP** tiene **8 bytes** de cabecera, no 20. Un datagrama de voz con 20 bytes de audio tiene 8/28 = 28,6 % de overhead en vez de 53,7 %. **Por eso el códec del RTP mete 20 bytes de audio por paquete y no 100**: cada ms de audio a 20 bytes es un paquete, y el tamaño de cabecera importa.
- **HTTP/2 (RFC 9113)** comprime las cabeceras con **HPACK (RFC 7541)**: una Headers repetitiva de 80 bytes puede quedar en 20.
- **QUIC** (RFC 9000) usa **cifrado integrado**, lo que agrega entre 16 y 40 bytes al inicio pero **elimina el handshake de 3 vías**, y el resultado es menos round trips aunque el sobre sea mayor.

Motivo 4 — La MTU y la fragmentación. Un datagrama IP de 2000 bytes **no cabe** en el MTU de 1500. O se fragmenta, o se descarta con ICMP. Cada fragmento lleva sus **propios 20 bytes de cabecera IP**. Fragmentar un mensaje de 2000 en 2 fragments agrega 20 bytes de overhead extra. Es la capa 3 reproduciendo el mismo efecto.

1.4 Lo que además cambia al bajar de capa

El tamaño no es lo único que cambia. Hay **dos cambios más**, y son los que de verdad justifican la encapsulación.

(a) El direccionamiento: de IP a MAC

```
ENCAPSULANDO (H1 → la red):

  El paquete IP dice:  192.168.1.25  →  8.8.8.8     (CAPA 3)
  La trama dice:       AA:BB:CC:DD:EE:01 → 00:1A:2B:3C:4D:5E   (CAPA 2)

  ¿Por qué NO puede ser la misma dirección en las dos capas?

  Porque son cosas distintas:
  ──────────────────────────────────────────────────────────
  IP  = IDENTIDAD GLOBAL del host, JERÁRQUICA y ESTABLE.
        El paquete IP viaja de extremo a extremo: la IP destino
        no cambia en 12 routers. Es la que sabe el servidor de
        DNS, la que escribe la ACL del firewall, la que aparece
        en los logs del servidor.

  MAC = IDENTIDAD LOCAL del ENLACE, PLANA y REESCRIBIBLE.
        Solo tiene sentido dentro de un mismo medio físico.
        Cambia en CADA router:
           enlace 1:  AA:BB:CC:DD:EE:01 → 00:1A:2B:3C:4D:5E
           enlace 2:  FF:11:22:33:44:07 → 00:AA:BB:CC:DD:99
           enlace 3:  ...
  ──────────────────────────────────────────────────────────

DESENCAPSULANDO (una trama llega a un router):

  El router hace:  NIC recibe la trama, ve que MAC destino
                   es la suya → se la pasa a la capa 3.
  La capa 3 hace:  lee la IP destino 8.8.8.8.
                   "No es mío." → busca en la tabla de reenvío.
                   "Va por eth1."  → se lo dice a la capa 2.
  La capa 2 hace:  busca en la tabla ARP qué MAC tiene eth1,
                   arma una trama nueva, la manda.

  → El router NUNCA ve la IP origen ni la IP destino en la trama.
    Las ve en el PAQUETE, que es otra cosa.
```

El corolario que se pregunta en los exámenes: si en un traceroute ves la IP pero no la MAC, es porque **cada router es un nuevo enlace con sus propias MACs**. Y si en un ejercicio te piden "las tramas del camino de C a F", lo que cambia entre filas es **siempre** la MAC y **nunca** la IP (salvo NAT, Clase 31).

(b) El objetivo: de "llegar al host" a "llegar al proceso"

```
──────────────────────────────────────────────────────────────
CAPA 3 (Red):   "Este paquete tiene que llegar a la MÁQUINA
                 8.8.8.8."  El objetivo es un HOST.

CAPA 4 (Transporte): "Y una vez ahí, tiene que llegar al
                 PROCESO que escucha en el puerto 443 de esa
                 máquina, y no al que escucha en el 22 ni en el
                 53."  El objetivo es un PROCESO.
──────────────────────────────────────────────────────────────

Por qué hace falta y por qué la capa 3 no alcanza:

Un servidor web en 8.8.8.8 tiene, típicamente, escuchando:
   :22    sshd
   :53    named (DNS)
   :80    httpd
   :443   httpd (TLS)
   :3306  mysqld
   :6379  redis

Las IP destino son TODAS 8.8.8.8. La IP no distingue.
La que distingue es el PUERTO, y por eso el puerto tiene que
ser un campo de la CABECERA DE CAPA 4.
```

La demostración clásica (y es un problema de examen): en un servidor, el *packet filter* de `iptables` / `nftables` puede hacer `DROP` por **puerto** porque la capa 3 le **pasa el paquete completo a la capa 4** en Linux (`netfilter` trabaja con cadenas por capa: `-p tcp --dport 443`). Ese es el motivo por el que un **firewall stateful implementa capas 3 y 4** (tabla de la Clase 1) y por el que la NAT necesita leer el puerto: **para poder multiplexar** varias conexiones internas detrás de una sola IP pública (Clase 31).

(c) Y un tercer cambio, más sutil: la granularidad de la direccionación

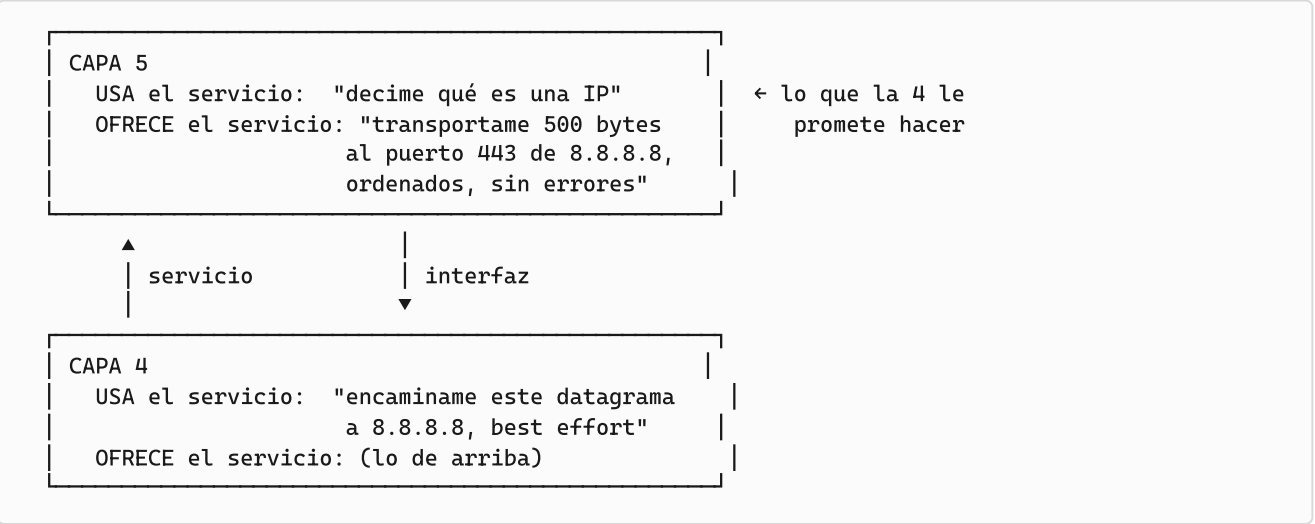
```
┌─────────────┬──────────────────┬──────────────────────────────────┐
│ Capa        │ ¿Qué identifica? │ Dónde está esa información       │
├─────────────┼──────────────────┼──────────────────────────────────┤
│ 4 Transporte│ una CONEXIÓN     │ 4-tupla (IP:port origen/destino) │
│ 3 Red       │ un HOST          │ IP destino (con prefijo /n)      │
│ 2 Enlace    │ una TARJETA      │ MAC destino                      │
│ 1 Física    │ un BIT           │ voltaje / fase / frecuencia      │
└─────────────┴──────────────────┴──────────────────────────────────┘

Y la granularidad va de uno a muchos:
 1 conexión TCP  →  1 paquete IP  →  1 trama  →  miles de bits
```

Un paquete IP puede contener varios segmentos TCP (eso es la segmentación de IP, distinta de la de TCP), y **una trama puede contener varios paquetes IP** si el medio lo permite. Por eso "un paquete" y "una trama" **no son sinónimos** y por eso los problemas de examen que mezclan PDU hay que mirarlos con cuidado.

1.5 Servicios e interfaces entre capas

Qué es un servicio

Cada capa **ofrece** un servicio a la de arriba y **usa** un servicio de la de abajo. "Servicio" no es un IND: es **una especificación de qué operaciones puede pedir la capa de arriba y qué garantías le da la capa de abajo**.



Los servicios de cada capa

Capa	**Qué le promete a la de arriba** (servicio)	**Qué espera de la de abajo** (usa)
7 Aplicación	Datos de aplicación, puffed a un proceso remoto	Un flujo de bytes confiable a un proceso remoto
6 Presentación	Datos en una **representación común**	Datos en la representación nativa
5 Sesión	Diálogo abierto, sincronizado, con recuperación	Un intercambio de datos ordenado
4 Transporte	Flujo **fiable y ordenado** de bytes entre procesos	Paquetes **no fiables** que pueden perderse o reordenar
3 Red	Paquetes **best effort** hacia un destino	Tramas confiables en un enlace
2 Enlace	Tramas **sin errores** entre dos vecinos	Bits crudos, que pueden llegar corruptos
1 Física	Bits, con la tasa de errores del medio	—

Por qué esto es la clave de la encapsulación

Porque **el contrato entre capas es lo que permite el intercambio**. Cuatro consecuencias concretas:

1. La capa de arriba no necesita saber cómo funciona la de abajo. El navegador HTTP no sabe que hay cobre debajo. La capa 4 (TCP) **no sabe si el camino tiene 3 o 25 saltos**, ni si los saltos son Ethernet o Wi-Fi, ni si hay pérdida. Y **por eso TCP funciona sobre cualquier cosa.**

2. Una capa puede implementarse de mil maneras. El servicio "encaminame paquetes a 10.0.0.7" lo pueden dar un router de 300 dólares con un chip o un backbone con routing fabric. La aplicación no se entera.

3. Los sniffers funcionan por capas. Un *packet capture* de capa 2 te da tramas. Uno de capa 3 te da datagramas. Uno de capa 4 te da segmentos. Y **`tcpdump` /Wireshark necesitás conocer el nivel para filtrar** (`-p` para capa 3). Si entendés los servicios, entendés por qué la herramienta tiene esa sintaxis.

4. Es lo que hace que se pueda cambiar una capa. Cambiar el medio (cobre → fibra) es cambiar la implementación de la capa 1 que **cumple el mismo servicio**. Cambiar el protocolo de transporte (TCP → QUIC) es cambiar la implementación de la capa 4. En los dos casos, **las capas de arriba no cambian**.

La relación entre servicio e interfaz

- La **interfaz** es **la frontera**: *dónde* se llama.
- El **servicio** es **el contrato**: *qué* se promete.

```
┌──────────────────────────────────────────┐
│  La interfaz es la PUERTA.               │
│  El servicio es lo que hay DEL OTRO LADO.  │
│                                          │
│  Si el servicio cambia (promete otra cosa), │
│  la implementación cambia.               │
│  Si la implementación cambia, la interfaz    │
│  puede quedar igual.                     │
└──────────────────────────────────────────┘
```

El ejemplo para explicar por qué el cambio de una capa no rompe las demás: si la capa 2 pasa de Ethernet a Wi-Fi, **la interfaz "encaminame este paquete a esta IP" sigue siendo exactamente la misma**. Wi-Fi cumple el mismo servicio de la capa 2 que cumple Ethernet: entregar tramas sin errores dentro de un alcance dado. Por eso un driver de IP no sabe si está sobre Ethernet o sobre Wi-Fi. Por eso tu laptop puede estar conectada a una red y no cambia una línea de código de tu navegador.

1.6 La capa de presentación, desarrollada

La capa 6 resuelve **el problema de la heterogeneidad de representación**. Lo desarrollamos con casos reales, porque es la capa que más se olvida y la que más puntos da.

(a) *Endianness* — el orden de los bytes

El problema. Un número de 32 bits con valor `0x12345678` ocupa 4 bytes. Esos 4 bytes se pueden ordenar de dos maneras:

```
BIG-ENDIAN ("network byte order", orden de red)
┌────────┬────────┬────────┬────────┐
│ 0x12   │ 0x34   │ 0x56   │ 0x78   │
└────────┴────────┴────────┴────────┘
 MSB                          LSB
 (peso 2²⁴)                   (peso 2⁰)
 se manda primero el de MAYOR peso

LITTLE-ENDIAN (orden de x86, ARM y casi todos los microcontroladores)
┌────────┬────────┬────────┬────────┐
│ 0x78   │ 0x56   │ 0x34   │ 0x12   │
└────────┴────────┴────────┴────────┘
 LSB                          MSB
 se manda primero el de MENOR peso

──────────────────────────────────────────────────────────────
El MISMO conjunto de 4 bytes significa:
   305 419 896   si es big-endian
2 018 915 346   si es little-endian

Si no me decís la endianness, el dato NO TIENE SIGNIFICADO.
──────────────────────────────────────────────────────────────
```

La regla de la red: todos los campos de más de 8 bits de **IP, TCP, UDP, ICMP** van en **network byte order (big-endian)**. Por eso en C existen `htons()`, `htonl()`, `ntohs()`, `ntohl()`.

```
Sin htons(), este código funciona en x86 y falla en big-endian:

    struct sockaddr_in sa;
    sa.sin_port = htons(80);      // htons converts to Network Order
    //  si pongo sa.sin_port = 80;  → funciona en little-endian
    //                              y FALLA en big-endian

Por eso la PI de la capa de presentación no es teórica:
es la razón por la que C tiene esas cuatro funciones.
```

(b) Formatos de caracteres y de archivos — el bug de UTF-8

El caso canónico del bug de UTF-8: tomemos la palabra **"mañana"** y veamos cómo se representa la letra Ñ:

```
La letra Ñ (U+00D1) en distintos esquemas:

ASCII      : no existe
ISO-8859-1 : 1 byte   0xD1
UTF-8      : 2 bytes  0xC3 0x91      (prefijo 110xxxxx + continuation)

Si el emisor manda ISO-8859-1 (0xD1) y el receptor asume UTF-8,
el receptor ve 0xD1 = un byte de inicio de un carácter de 2 bytes
y espera el siguiente, que es otra cosa. Resultado: "mojibake".

Si el emisor manda UTF-8 (0xC3 0x91) y el receptor asume ISO-8859-1,
el receptor ve dos caracteres: Ã (±) y Ñ. Resultado: "maÃ±ana".
```

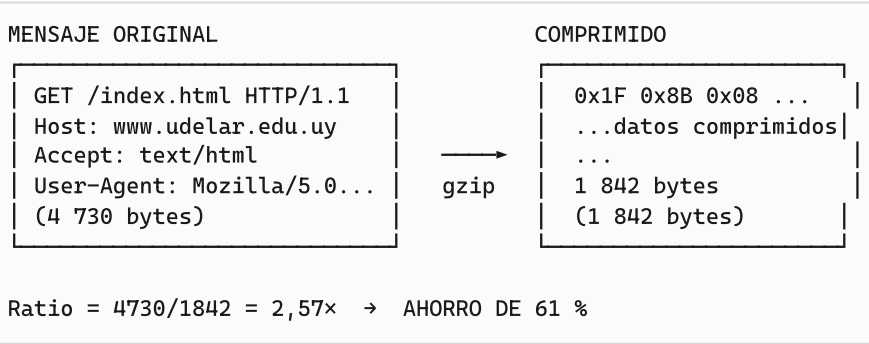
Por qué UTF-8 ganó: es **backward-compatible**: un texto ASCII válido en UTF-8 es **idéntico** en ASCII. Eso permitió que durante años se usara UTF-8 en casi todo y no rompiera nada. La longitud es variable (1 a 4 bytes), y por eso "la cantidad de caracteres" **no** es "la cantidad de bytes".

```
"hola"  en ASCII   = 4 bytes
"hola"  en UTF-8   = 4 bytes  (idéntico)
"ñ"     en ISO-8859-1 = 1 byte
"ñ"     en UTF-8   = 2 bytes
"😜"  en UTF-8   = 4 bytes  (¡y no existe en ASCII ni en ISO-8859-1!)
──────────────────────────────────────────────────────────────────
Un emoji ocupa 4 BYTES, no 4 caracteres. Es 1 carácter.
Si tu aplicación calcula "el string tiene 4 caracteres, mando 4 bytes",
manda la mitad de los bytes y rompe el mensaje.
```

Esto es un problema real de red, no de teoría: es la razón por la que el `Content-Length` de HTTP está en **bytes** y no en caracteres, y la razón por la que un cliente que anuncia `charset=iso-8859-1` y manda UTF-8 rompe la página.

(c) Compresión

Si el medio es caro (un enlace satelital de 300 Mbps que cuesta 4000 dólares al mes) o el tiempo es caro (un disco de 10 TB), **comprimir los datos antes de mandarlos** vale la pena.



El compromiso: la compresión **cuesta CPU** y **empeora la latencia** (hay que comprimir antes de mandar y descomprimir después de recibir). Para texto HTML (muy redundante) compensa muchísimo; para video ya comprimido (H.264, AV1) **empeora** la calidad sin ganar nada. Por eso `Content-Encoding: br` y `gzip` se aplican a texto, JSON y CSS, y **no** a video.

(d) Cifrado

La confidencialidad también es una función de "presentación": el dato tiene que ser **legible para el destinatario y no para nadie más**. En la práctica esto vive en **TLS** (`RFC 8446`, actualizado por `RFC 9846` en 2026), que es una biblioteca entre la aplicación y el socket.

Por qué en la capa de OSI y no en otra: porque el cifrado es **una transformación de la representación** del dato que debe hacerse de manera **transparente** a las capas 5 y 7. Si el cifrado fuera de capa 3 (como IPsec), cada host tendría que configurar la clave por ruta y no podría haber *NAT traversal*. Si fuera de capa 4, el *endpoint* de la aplicación tendría que saber del tema. En la capa 6, **la aplicación llama a una biblioteca y sigue escribiendo texto plano**.

La síntesis de la capa 6

```
┌──────────────────────────────────────────────────────┐
│  CAPA 6 — PRESENTACIÓN                               │
│                                                      │
│  Problema: los datos de un host pueden NO ser compatibles
│            con los de otro, aunque los bits lleguen bien.
│                                                      │
│  Cuatro funciones concretas:                         │
│    1. REPRESENTACIÓN: endianness, punto flotante, entero
│    2. FORMATO:         codificación de caracteres, archivos
│    3. COMPRESIÓN:      reducir lo que viaja          │
│    4. CIFRADO:         que solo el destinatario entienda
│                                                      │
│  En TCP/IP: no existe como capa. Vive en la APLICACIÓN,
│  y el cifrado en la biblioteca TLS.                  │
└──────────────────────────────────────────────────────┘
```

La frase oficial del Examen 2006-12, textual:

> *"Busca la solución de problemas particulares, que hacen referencia a la semántica y sintaxis de lo enviado. Por ejemplo la resolución de diferentes representaciones entre máquinas (pto flotante), corresponde a esta capa."*

Análisis. La oficial dice **"semántica y sintaxis"**, que es más amplio que "los bytes": la capa 6 no solo traduce bytes, traduce **significados**. Y el ejemplo que da — el **punto flotante** — es exactamente *endianness* + **rango** + **precisión** + formato de la representación de un `float`. Los 4 bytes de un `float` de precisión simple: si el endianness difiere, el valor cambia; si el hardware no soporta ese `float`, el valor se convierte; si la longitud difiere (4 vs 8 bytes), el mensaje se rompe entero.

Lo que hay que agregar para el punto extra: que en TCP/IP estas funciones **no tienen capa propia**, y que la consecuencia práctica es que **cada protocolo de aplicación normaliza su representación** (JSON, protobuf, DER, X.509, JPEG, PNG), y que el cifrado se resuelve en TLS.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **El overhead de 58 bytes**: 20 de TCP + 20 de IP + 18 de Ethernet (14 + 4 de FCS). Es el número con el que hay que trabajar. - **La fórmula** `eficiencia = L / (L + overhead)` y el hecho de que el overhead relativo **se dispara con mensajes chicos**. - **MTU de Ethernet = 1500**; **MSS de TCP = 1460 = 1500 − 20 − 20**. - **Trama mínima de 64 bytes**, preámbulo de 7 + SFD de 1, e **IFG de 12 bytes entre tramas**. - **La IP no cambia en tránsito; la MAC cambia en cada router.** - **Que la capa de aplicación no ve la red.**

Cambió o se modificado:

- **IPv6 cambió el cálculo de la MTU.** En IPv4, el `Total Length` incluye **todas** las cabeceras. En IPv6, el campo **Payload Length excluye las cabeceras de extensión** (`RFC 8200` §3): los 40 bytes fijos de la cabecera IPv6 **no** están en el `Payload Length`. ⇒ **Cuando un problema de examen Fragmente un paquete IPv6, la cuenta de qué entra y qué no es distinta**, y es un error muy común tomarla de la fórmula IPv4.
- **El MTU mínimo absoluto de IPv6 es 1280 bytes** (`RFC 8200` §5, verificado textualmente). Por eso una conexión que "*funciona*" en IPv4 puede romperse en IPv6 si algún camino tiene MTU menor.

- **El MTU ya no es 1500 y ya no es una constante. Jumbo frames de 9000 bytes** son una **convención, no un estándar IEEE**: se usan sobre todo intra-datacenter y en almacenamiento, y **nunca** sobre Internet o túneles sin *clamp MSS*. Además, con **ECMP el PMTU es propiedad del camino, no del prefijo**, así que dos flujos al mismo destino pueden tener MTU distinto.
- **Cada overlay consume MTU.** MPLS, VXLAN (`RFC 7348`), IPsec y WireGuard restan bytes disponibles. Por eso el desajuste de MTU es **omnipresente** en redes cloud híbridas, y es el contenido de la Pregunta 2 del Examen 2010-12.
- **La capa 4 ya no es TCP ni UDP solamente.** QUIC (`RFC 9000`) **reemplaza a TCP en HTTP/3** y lleva su propio *header protection* y su propia recuperación. **El cifrado de la "capa de presentación" se mudó de sitio otra vez**: en QUIC, el *handshake* de TLS viaja en frames `CRYPTO`, no en un stream de bytes (`RFC 9001`). Un middlebox que solo entienda TCP/UDP plano no puede inspeccionarlo.
- **La fragmentación cambió por completo.** En IPv6 **el router nunca fragmenta**; fragmenta el origen. Y **DPLPMTUD (`RFC 8899`, 2022) reemplazó a la PMTUD clásica** porque ya no depende de ICMP: el transporte sondea con su propio paquete de datos. Es la respuesta canónica al *black hole*.
- **VLANs con etiqueta doble.** En carrier Ethernet y en EVPN-VXLAN el estándar real es **C-TAG (`0x8100`) + S-TAG (`0x88A8`)**, o sea **dos** cabeceras de capa 2. La encapsulación de capa 2 ya no es "una cabecera": es un stack.

> **Consecuencia para el examen:** los **números** cambian según la versión (MTU 1500 o 1280 mínimo, MSS 1460 o 1430), pero **el mecanismo** — encapsulation, overhead fijo, eficiencia `L/(L+H)`, IP global contra MAC local — **no cambió en 37 años**. Resolvé primero la aritmética con los números que dé el enunciado, y después agregá una línea de actualización.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Tamaño total y eficiencia para tres tamaños de mensaje

Datos. Sobre un enlace Ethernet: cabecera TCP de 20 B, cabecera IPv4 de 20 B, y trama Ethernet de 18 B de overhead (14 de cabecera + 4 de FCS). Se envían tres mensajes de aplicación de **20 B**, **800 B** y **9 000 B**.

a) ¿Cuánto viaja por el cable en cada caso? **b)** ¿Cuál es la eficiencia de cada uno? **c)** ¿Qué pasa con el mensaje de 9 000 bytes en una red con MTU de 1500?

Resolución.

Paso 1 — El overhead, una vez.

```
overhead = H_TCP + H_IP + H_ENLACE
         =   20   +  20  +     18
         =   58 bytes      (constante, independiente del mensaje)
```

Paso 2 — Los totales.

L_app	total = L + 58	bits	tramas (de 64 a
20 B	20 + 58 = 78 B	624 bits	1 trama (con pad)
800 B	800 + 58 = 858 B	6 864 bits	1 trama
9 000 B	9000 + 58 =9058 B	72 464 bits	fragmentado

Paso 3 — Las eficiencias.

```
Eficiencia = L_app / total
```

L_app	cuenta	eficiencia
20 B	20 / 78	= 0,2564 = 25,6 % OJO: la Ethernet rellena con padding hasta 64 bytes: 20/64 = 31,3% (el mínimo de trama)
800 B	800 / 858	= 0,9324 = 93,2 %
9 000 B	9000 / 9058	= 0,9936 = 99,4 % (¡con los fragments!)

Paso 4 — El mensaje de 9 000 bytes, parte c.

No pasa. El MTU de Ethernet son 1500 bytes de **payload de capa 2**, o sea del paquete IP completo:

```
MTU de Ethernet = 1500 bytes  ← esto mide el PAQUETE IP entero

Un paquete IP con 9000 B de datos TCP sería:
    9000 + 20 (TCP) + 20 (IP) = 9040 B de paquete IP
    9040 > 1500  ✗   NO CABE

Hay tres salidas:

(1) FRAGMENTAR (IPv4, con DF=0):
    Payload IP disponible = 1500 − 20 (IP) = 1480 B
    Fragmento 1: 20 (IP) + 1480 (TCP+datos)          = 1500 B
    Fragmento 2: 20 (IP) + 1480                       = 1500 B
    Fragmento 3: 20 (IP) + 1480                       = 1500 B
    Fragmento 4: 20 (IP) + 1480                       = 1500 B
    Fragmento 5: 20 (IP) + 1480                       = 1500 B
    Fragmento 6: 20 (IP) + 1480                       = 1500 B
    Fragmento 7: 20 (IP) + 1380                       = 1400 B
                                          TOTAL = 9020 B
    (9000 + 20 TCP + 7×20 IP = 9000 + 20 + 140 = 9160... ojo,
     calculemos bien abajo)

(2) DESCARTAR con ICMP (IPv4 con DF=1, o IPv6):
    ICMPv4 tipo 3 código 4 "Fragmentation Needed and DF was Set"
    El origen baja el MSS y vuelve a mandar. Sin DF=1, el
    router no puede avisar: se pierde el paquete.

(3) EL ORIGEN FRAGMENTA (IPv6, RFC 8200):
    En IPv6 el router NUNCA fragmenta. Fragmenta el origen.
```

La cuenta correcta de la fragmentación, con la aritmética a la vista:

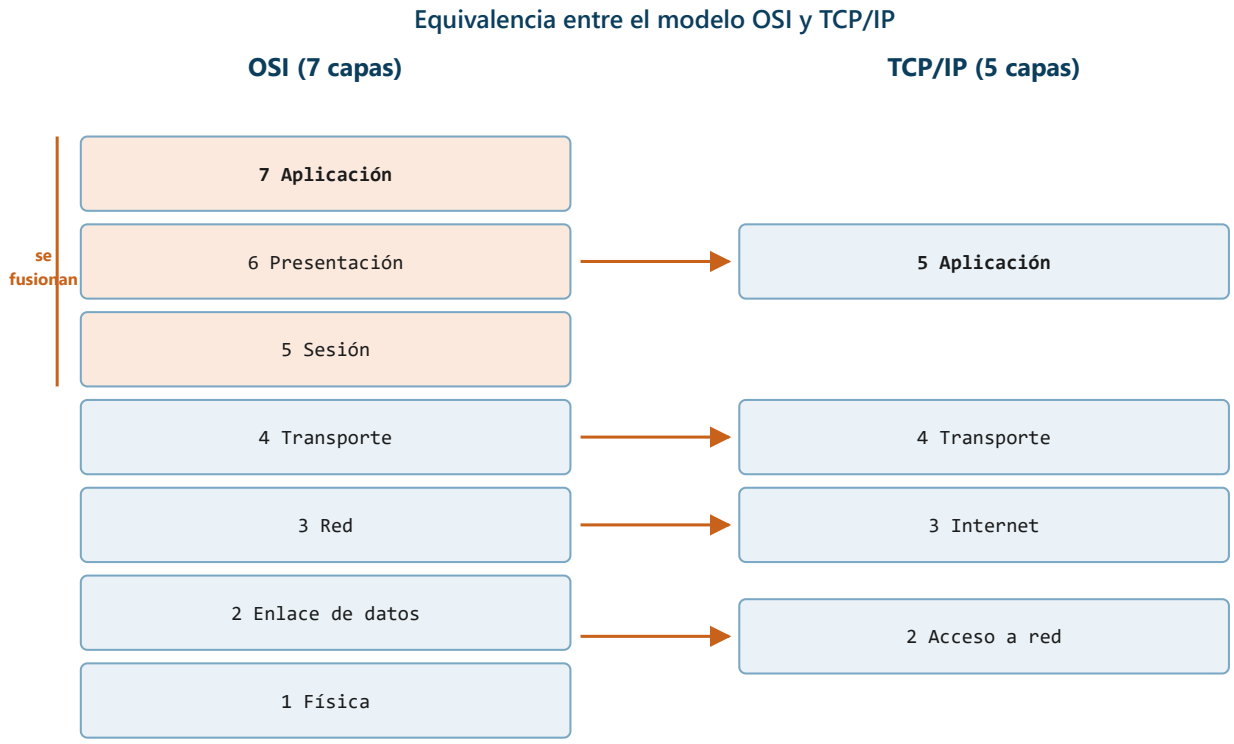
```
Datos de aplicación            = 9 000 B
Cabecera TCP                   =    20 B   (un solo segmento)
──────────────────────────────────────────
Paquete IP a enviar            = 9 020 B

Capacidad por fragmento (MTU 1500):
   1500 − 20 (cabecera IP de cada fragmento) = 1480 B de payload

Payload a repartir: 9 000 + 20 = 9 020 B

Cantidad de fragmentos:
   ⌈9 020 / 1 480⌉ = ⌈6,0946⌉ = 7 fragmentos

Reparto:
   f1..f6 : 1480 B cada uno  = 8 880 B
   f7     : 9020 − 8880 = 140 B
   ──────────────────────────────
   total payload = 9 020 B  ✓

Overhead de fragmentar:
   7 × 20 B de cabecera IP = 140 B extra
   (comparado con los 20 B de un solo paquete)

Total que viaja en la red:
   7 tramas × (18 de Ethernet + payload_i)
     = f1..f6: 1480 + 20 + 18 = 1518 B cada uno  × 6 = 9 108 B
     = f7    :  140 + 20 + 18 =  178 B
     ──────────────────────────────────────────────
     TOTAL = 9 286 B

Eficiencia = 9 000 / 9 286 = 96,9 %
```

Interpretación. Fragmentar un mensaje de 9 000 bytes **agrega 140 B de overhead de IP** que no estaban en el cálculo original de 58 bytes. Y tiene un costo peor que el tamaño: **si se pierde UNO de los 7 fragmentos, se pierde el mensaje entero** (no hay protocolo de fiabilidad en IP), y hay que retransmitir 9 020 bytes. Es la razón por la que **DHCP, DNS y las respuestas HTTP grandes** están CCS штатно limitados al MTU. Por eso **DNS usa UDP con 512 bytes de payload** y por eso existe **TCP para respuestas grandes**.

⚠️ Errores típicos

- **Sumar el preámbulo (8 B) y el IFG (12 B) al tamaño de la trama.** El "tamaño de trama" en un problema de examen es `14 + payload + 4`. El preámbulo se cuenta aparte y el IFG va entre tramas.
- **Olvidar que cada fragmento lleva su propia cabecera IP de 20 B.** La fragmentación multiplica el overhead de capa 3.
- **Dividir mal: eficiencia = L/total, no = 1 − overhead/58.** El denominador es `L + 58`, que depende de `L`.
- **Confundir MTU con MSS.** MTU = 1500 (medido sobre el **paquete IP**). MSS = 1460 = 1500 − 20 (IP) − 20 (TCP) (medido sobre los **datos de aplicación**).

2.2 Ejercicio 2 — El camino completo: ¿quién lee qué?



En TCP/IP no existen Presentación ni Sesión:
el cifrado vive en la capa de Aplicación (TLS) y en la biblioteca.

Ojo: hay dos criterios de conteo. Tanenbaum cuenta 5 capas porque separa enlace y física; Kurose cuenta 4. La capa 3 es la misma.

Enunciado. Un cliente `192.168.1.25` (MAC `00:0C:29:AA:BB:CC`) quiere descargar una página de `203.0.113.9` (MAC del router de salida: `00:1A:2B:CC:DD:01`; MAC del switch: `00:AA:BB:00:00:01`). Hay un router `R1` en `192.168.1.1` (MAC `00:0C:29:FF:EE:01`) y un router `R2` en el otro lado. **Describí qué encapsula y qué desencapsula cada capa, y en qué momento exacto se invierte el proceso.**

Resolución — el camino, paso a paso.

```
═══ EN H1 (192.168.1.25) ═══════════════════════════════

Cap 7  El navegador escribe:
        GET /index.html HTTP/1.1     Host: www.udelar.edu.uy
        (430 bytes)
        == write(socket_fd, buf, 430) ==
           ↓
Cap 4  TCP:
        · busca la conexión (4-tupla) o la crea (handshake)
        · segmenta: 430 B caben en un MSS → un segmento
        · NUMERA: este es el byte N del flujo
        · ANTECABE 20 B de cabecera TCP
          ┌──────────────────────────────┐
          │TCP hdr (20)│ datos (430)     │ = 450 B  ← SEGMENTO
          └──────────────────────────────┘
        == sendto() hacia 203.0.113.9:80 ==
           ↓
Cap 3  IP:
        · ¿203.0.113.9 está en mi subred (192.168.1.0/24)? NO
        · consulto la tabla: default gateway = 192.168.1.1
        · pongo IP origen = 192.168.1.25, IP destino = 203.0.113.9
        · TTL = 64, protocolo = 6 (TCP)
        · ANTECABE 20 B de cabecera IP
          ┌──────────────────────────────────────┐
          │IP hdr (20)│TCP hdr(20)│datos(430) │ = 470 B ← PAQUETE
          └──────────────────────────────────────┘
        == pasa a la capa 2 ==
           ↓
Cap 2  Ethernet:
        · ¿debo resolver una MAC? SÍ (el próximo salto es R1)
          → ARP request 192.168.1.25 → "¿192.168.1.1?"
          → ARP reply   R1 → "soy 00:0C:29:FF:EE:01"
        · armo la trama
          ┌──────────────────────────────────────────────┐
          │ETH hdr(18)│IP hdr(20)│TCP hdr(20)│datos(430)│ = 468 B
          │MAC:FF:EE:01 ← 00:0C:29:AA:BB:CC              │ ← TRAMA
          └──────────────────────────────────────────────┘
          (EtherType 0x0800 = IPv4)
        == transmito por el cable ==
           ↓
Cap 1  Codifico: 3744 bits de 0 y 1 en voltajes a 1 Gbps
        == (van por el cable) ==

═══════════ EN R1 ═══════════════════════════════════════

Cap 1  La NIC recibe voltajes → bits
        ↓
Cap 2  · verifico el FCS → coincide
        · MAC destino = 00:0C:29:FF:EE:01 = ¿la mía? SÍ
        · ENTREGO los 450 bytes a la capa 3
        · ⚠️ La capa 2 de R1 NO miró la IP. No sabe a dónde va.
           Solo sabe que la trama era para él.
        ↓
Cap 3  · leo la IP destino: 203.0.113.9
        · ¿es mía? NO
        · TTL = 64 − 1 = 63 → no es 0, sigue
        · recalculo el checksum de la cabecera IP
        · consulto la tabla de reenvío: 203.0.113.0/24 → eth1
        · entrego a la capa 2: "mandalo por eth1"
        ↓
Cap 2  · resuelvo (o tengo cacheada) la MAC de eth1
        · ARMO UNA TRAMA NUEVA: la vieja se descarta
          ┌──────────────────────────────────────────────┐
```

```
            |ETH hdr(18)|IP hdr(20)|TCP hdr(20)|datos(430)| = 468 B
            |MAC:AA:00:01 ← 00:0C:29:FF:EE:01                |
            └─────────────────────────────────────────────────┘
             ↑ MAC origen y destino CAMBIARON
             ↑ La cabecera IP es la MISMA, con TTL 63

═══════════ EN EL SERVIDOR (203.0.113.9) ═══════════════

Cap 1  bits → Cap 2: FCS ok, MAC destino = el mío → quito 18 B
        ↓
Cap 3  IP destino = 203.0.113.9 = MÍO → TTL −1 → quito 20 B
        ↓
Cap 4  TCP: checksum ok, seq esperado → anoto el byte,
        · quito 20 B → entrego 430 B a la aplicación
        ↓
Cap 7  El servidor web (puerto 80) lee 430 B y ve "GET /"

═══════ Y AHORA SE INVIERTE: la respuesta vuelve por el mismo camino ═══════

Cap 7  El servidor escribe la respuesta (5 200 B)
        ↓ ↓ ↓ (misma secuencia, sentido inverso)
Cap 4  TCP segmenta (4 segmentos), numera al revés
Cap 3  IP: 203.0.113.9 → 192.168.1.25
Cap 2  Eth:  → 00:0C:29:FF:EE:01   (¡ahora el origen es el servidor!)
Cap 1  bits
```

La respuesta a "¿en qué momento se invierte?"

Se invierte en el punto donde el paquete alcanza su destino, que es un **evento** y no un lugar:

```
El ÚNICO evento que marca el punto de inversión es:
la capa 3 del destino que RECONOCE su propia IP destino
en la cabecera del paquete.

Todo lo que está por debajo de ese punto va subiendo
(desencapsulando). Todo lo que está por arriba va bajando
(encapsulando).
```

Y hay un segundo punto de inversión, independiente del primero: cuando el paquete **sale de la capa 2 de H1 y entra en la capa 1**, se invierte la dirección **dentro del emisor**. Cada vez que un paquete entra a una capa, baja; cada vez que sale, sube. Un router hace subir a capa 3 y bajar a capa 3 por cada paquete.

La respuesta completa a la pregunta del enunciado:

Momento	Capa	Acción
El navegador llama `write()`	7→4	**Baja**: la 7 le pasa los datos a la 4
TCP encapsula	4→3	**Baja**: la 4 le pasa el segmento a la 3
IP encapsula	3→2	**Baja**: la 3 le pasa el paquete a la 2
ARP resuelve la MAC	2	**Consulta**: la 2 le pide a la 3 la dirección lógica de destino
Se arma y se manda la trama	2→1	**Baja**: la 2 le pasa los bits a la 1
La trama llega a H2	1→2	**¡SUBE!** La 1 le pasa los bits a la 2
El FCS da bien	2	Quita 18 B, entrega a la 3
La IP destino es la mía	3→4	**¡SUBE!** Quita 20 B, entrega a la 4
El segmento es de mi puerto	4→7	**¡SUBE!** Quita 20 B, entrega a la 7

⚠️ Errores típicos

- **Creer que la capa 3 del router "manda" el paquete.** El router **recibe la trama completa**, la entrega a la capa 3, la capa 3 lee la IP, decide, y **le dice a la capa 2 qué puerto usar**. La capa 2 arma la trama nueva. **El router no reescribe el paquete IP; la reescribe la capa 2.**
- **Creer que la IP cambia en tránsito.** No cambia, salvo por TTL, checksum y (fuera de NAT) los campos que NAT reescribe.
- **Pensar que ARP es un mensaje de capa 3 "normal".** ARP **no** es IP: no tiene cabecera IP, no tiene TTL, no se enruta. Va directo de capa 3 a capa 2 en un broadcast de enlace. Por eso **un router nunca reenvía ARP**.

2.3 Ejercicio 3 — ¿Qué pasa si el mensaje es de 40 bytes?

Datos. Un mensaje de aplicación de **40 bytes**. Cabeceras: TCP 20 B, IP 20 B, Ethernet 18 B.

a) ¿Cuánto viaja por el cable y cuál es la eficiencia? **b)** Con el mínimo de trama de 64 bytes de Ethernet, ¿cuál es la eficiencia real? **c)** ¿Cuántos bytes "útiles" quedan si además el protocolo de transporte es UDP (8 B de cabecera)?

Resolución.

a) Sobre el papel.

```
L_app    = 40 B
overhead = 20 (TCP) + 20 (IP) + 18 (Eth) = 58 B
──────────────────────────────────────────────────
total    = 40 + 58 = 98 B

eficiencia = 40 / 98 = 0,4082 = 40,8 %
overhead   = 58 / 98 = 0,5918 = 59,2 %   ← MÁS DE LA MITAD
```

Interpretación: el 59 % del tráfico es andamiaje. Cada byte de dato útil arrastra 1,45 bytes de cabecera. Mandar 40 bytes de información cuesta 98 bytes de cable.

b) El mínimo de trama de 64 bytes.

La especificación IEEE 802.3 exige un mínimo de **64 bytes** (de preámbulo a FCS) para que el *slot time* de 512 bits permita detectar una colisión. Una trama más chica se rellena con *padding* después de los datos. **Cuidado con la cuenta**, porque es el error clásico de este ejercicio:

```
payload de la trama = 20 (IP) + 20 (TCP) + 40 (datos) = 80 bytes
longitud de la trama = 14 (cabecera) + 80 (payload) + 4 (FCS) = 98 bytes

98 > 64  →  NO hace falta padding.
```

El umbral exacto del padding, entonces:

```
Para que NO haya padding en Ethernet:
    payload de la trama ≥ 46 bytes
    46 = 64 (mínimo) − 14 (cabecera) − 4 (FCS)

Payload = cabeceras IP+TCP + datos = 40 + L
    → 40 + L ≥ 46  →  L ≥ 6 bytes de aplicación

Payload = cabeceras IP+UDP + datos = 28 + L
    → 28 + L ≥ 46  →  L ≥ 18 bytes de aplicación
```

Con 40 bytes de datos de aplicación no hay padding, y la eficiencia es la del apartado a): **40,8 %**. El padding es un problema **solo** de los mensajes de aplicación muy chicos, típicamente los paquetes de control (un `ACK` puro de TCP son 0 bytes de datos: `20 + 20 + 0 = 40` de payload, `14 + 40 + 4 = 58` de trama, y se rellenan 6 bytes de padding hasta 64). **Ahí la eficiencia es 0/64 = 0 %:** un Ethernet lleno de ACKs puros transmite bits que no son datos. Por eso las redes con mucho tráfico de ACK puro (redes de control) son especialmente ineficientes.

c) Con UDP en vez de TCP.

```
L_app   = 40 B
overhead = 8 (UDP) + 20 (IP) + 18 (Eth) = 46 B
──────────────────────────────────────────────────

total   = 40 + 46 = 86 B

eficiencia = 40 / 86 = 0,4651 = 46,5 %
overhead   = 46 / 86 = 0,5349 = 53,5 %
```

La comparación, que es el punto del ejercicio:

	overhead	total	eficiencia
TCP + IP + Eth	58 B	98 B	40,8 %
UDP + IP + Eth	46 B	86 B	46,5 %
DIFERENCIA	−12 B	−12 B	+5,7 p.p.

Interpretación y por qué importa. Cambiar TCP por UDP **no cambia ni un byte** de los 40 de datos de aplicación, pero **reduce el tráfico total un 12,2 %** (`12/98`). Para una aplicación que manda 40 bytes por paquete (un paquete de voz de 20 ms en un códec de G.711 con 8 bytes... o un mensaje de aplicación pequeño), ese 12 % es la diferencia entre que el enlace aguante o no.

Y ese es el argumento de VoIP (Clase 44): el códec de G.711 hace 8000 muestras de 8 bits por segundo = 64 000 bits/s = 8000 bytes/s. Si se pone en un paquete TCP de 160 bytes cada 20 ms:

```
160 bytes de audio → 178 con TCP → 186 con Eth  → eficiencia 86 %
overhead = 26 / 186 = 14 %
```

Con UDP de 160 bytes:

```
160 + 8 (UDP) + 20 (IP) + 18 (Eth) = 206 bytes
eficiencia = 160/206 = 77,7 %
overhead = 22,3 %
```

Y más importante que la eficiencia: **TCP retransmite**, y una retransmisión de voz a 20 ms de delay **no sirve para nada** (el destinatario ya pasó a la siguiente muestra). Y **TCP no puede porque quiere**: si el paquete se perdió, el destino ya tiene el hueco lleno de un paquete viejo. UDP manda y sigue. **Eso es lo que "no confiable" significa en la práctica: sin reintentos inútiles.** (Clase 44 lo desarrolla.)

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2009-02, Pregunta 2 (10 puntos) — forwarding vs. routing y flooding

> **Fuente:** Examen `sirc0902` — 2 de febrero de 2009, Pregunta 2 (10 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0902.pdf

Enunciado (texto del examen)

> **a)** En un contexto unicast, describa los procesos de forwarding y enrutamiento, y explique cómo se relacionan. **b)** En un contexto broadcast, describa un proceso de flooding, y explique cómo se puede transformar en flooding controlado.

Solución oficial (apartado a)

> El proceso de enrutamiento permite determinar el camino entre nodos de la red, típicamente mediante intercambio de información de topología y ejecución de un algoritmo en cada nodo para determinar el camino al resto de los nodos.
>
> El proceso de forwarding consiste en copiar cada paquete entrante a un nodo al puerto de salida adecuado para que siga su camino en la red (o entregarlo a un proceso local si el destinatario es el propio nodo). La tabla de forwarding, que asocia destinos con puertos (o interfaces) del nodo/router, se construye en base a la información de enrutamiento.

Solución oficial (apartado b)

> En un contexto broadcast, describa un proceso de flooding, y explique cómo se puede transformar en flooding controlado.
>
> ⚠️ **Nota sobre la solución oficial:** el apartado **b)** de la solución publicada **no tiene texto**. Solo está el enunciado. Hay que desarrollarlo. Lo marco como *(solución propia)* más abajo. También hay que notar que el enunciado del apartado **a)** de la solución oficial aparece con la letra "c)" en el PDF: es un error de maquetación de la cátedra.

Análisis de la solución oficial (apartado a)

Lo que está bien:

1. **La distinción de funciones es exacta**: "enrutamiento determina el camino", "forwarding copia al puerto de salida". Es la definición canónica.

2. **Dice que el forwarding puede entregar a un proceso local**: "*o entregarlo a un proceso local si el destinatario es el propio nodo*". Es un detalle que muchos alumnos olvidan y que muestra que el forwarding también tiene un caso "se queda acá".
3. **Dice que la tabla de forwarding SE CONSTRUYE a partir de la información de enrutamiento**: eso es **la relación entre los dos procesos**, que es lo que la pregunta pide explícitamente.

Lo que falta:

1. **No dice quién hace qué y cuándo.** Falta la clave temporal: el enrutamiento es **por período y por red** (o por topología); el forwarding es **por paquete y por interfaz**. Esa es la distinción operacional.
2. **No menciona el mecanismo de decisión**: el **longest prefix match** (*prefijo más largo que coincide*). Es lo que permite que las rutas se desambiguen y es el contenido de la Pregunta 1 del Examen 2011-03 (8 puntos).
3. **No habla del "next hop"**: la tabla de forwarding asocia destino con **interfaz**, pero en general también con **próximo salto** (*next hop*), que es lo que permite poner ARP para resolver la MAC del siguiente, no la del destino final. Ese detalle es lo que se pide en el Problema 2 del Examen 2011-03 y en el Problema 1 del Examen 2009-12.
4. **No menciona el plano de datos y el plano de control**, que es la forma moderna y más precisa de separar los dos procesos. La cátedra lo llama "plano de datos / plano de control" (Pregunta 4 del Examen 2008-07).
5. **No menciona IPv6 ni el *next header***, que es el mecanismo de extensión de IPv4 en IPv6 (RFC 8200): el *next header* va en la cabecera fija de IPv6, y el *protocol* en IPv4.

Mi respuesta completa (apartado a)

```
FORWARDING (reenvío) — plano de DATOS
  - Qué: para CADA PAQUETE que llega a una interfaz, decidir por
    qué puerto de salida sale, y copiarlo.
  - Granularidad: POR PAQUETE, POR INTERFAZ.
  - Información que usa: la TABLA DE REENVÍO (FIB), que es una
    estructura ACCELERADA en hardware (TCAM / Ternary CAM) dentro
    del chip de línea de tarjetas, y que se deriva de la RIB.
  - Decisión: LONGEST PREFIX MATCH sobre la IP DESTINO.
      "Busco en la tabla todos los prefijos que sean prefijo de la
       IP destino, y gana el MÁS LARGO."
  - Casos:
      · Si el destino es la propia dirección del router → entregar
        el payload a la capa de arriba (proceso local).
      · Si no hay ninguna entrada que coincida → descartar y, si
        corresponde, mandar ICMP Destination Unreachable.
      · Si hay coincidencia → reenviar por la interfaz indicada.
  - Determina también el NEXT HOP (dirección de capa 3 del
    siguiente router), que es a quien hay que resolver la MAC con
    ARP para armar la trama de salida.

ROUTING (enrutamiento) — plano de CONTROL
  - Qué: determinar el CAMINO (la tabla de reenvío) entre los
    nodos de la red.
  - Granularidad: POR PERÍODO y POR RED, no por paquete. Un
    router de borde recalcula BGP cada 30 s o cada 5 minutos; uno
    de OSPF recalcula cuando cambia la topología.
  - Mecanismo: intercambio de información de control con los
    vecinos (distance-vector, link-state, path-vector) y ejecución
    de un algoritmo (Bellman-Ford, Dijkstra, o el policy engine
    de BGP con sus atributos y sus políticas locales).
  - Produce: entradas en la RIB, que se seleccionan por preferencia
    (una tabla puede tener 10 rutas al mismo destino) y se
    traducen a la FIB.

──────────────────────────────────────────────────────────────────


CÓMO SE RELACIONAN

     VECINOS ──(protocolo de enrutamiento)──► INFORMACIÓN
                                                  │
                                        ┌─────────▼─────────┐
                                        │  ALGORITMO        │
                                        │ (Bellman-Ford /   │
                                        │  Dijkstra / BGP)  │
                                        └─────────┬─────────┘
                                                  ▼
                                             RIB (candidatas)
                                                  │ selección
                                                  ▼
PAQUETE ──► FIB ──► consulta al NEXT HOP ──► AR
(de entrada)   (tabla de reenvío)     (si no está cacheado)
      ▲              │
      └──────────────┘
   el forwarding USA la tabla que el routing CONSTRUYÓ

- El routing NO mira paquetes. El forwarding NO habla con vecinos.
- El forwarding es RÁPIDO (decisión por hardware, ~100 ns) y por
  paquete. El routing es LENTO (CPU, segundos) y por topología.
- Si el forwarding no puede decidir (sin entrada), se puede pedir
  un reenvío a la capa de control (un "route request"), pero es
  el caso excepcional, no el normal.
```

```
- En IPv6 el mecanismo es el mismo, pero el campo se llama
  NEXT HEADER (RFC 8200), no "protocolo", y permite encadenar
  cabeceras de extensión en lugar de indicar un protocolo nuevo.

- La distinción unicast vs. broadcast no cambia las funciones:
  cambia la ENTRADA de la tabla que se busca. En unicast es una
  IP destino concreta; en broadcast es la dirección de broadcast
  del dominio, y el comportamiento de reenvío cambia (ver b).
```

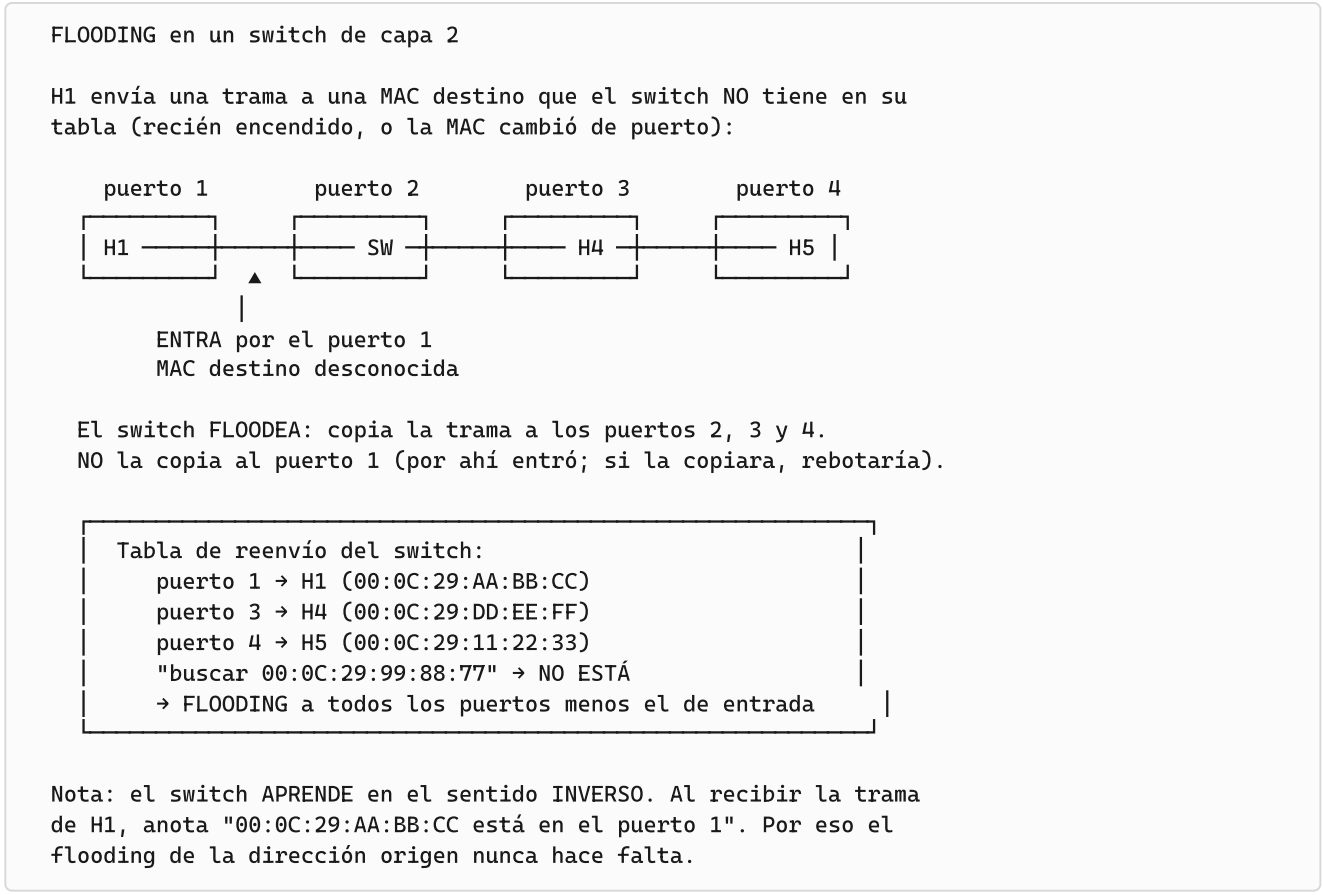
Los tres puntos extra que agregué y por qué:

Agregado	Por qué suma
Plano de datos vs. plano de control	Es la formulación moderna de la misma distinción, y es la Pregunta 4 del Examen 2008-07 (10 pts).
Longest prefix match	Es **el mecanismo concreto** de la decisión, y aparece en todos los problemas de tablas de ruteo.
Next hop ≠ destino final	Es lo que explica por qué en la tabla ARP del host está la MAC del **gateway**, no la del servidor.
Grano temporal (por paquete vs. por topología)	Da la idea de la distinción de una línea: "uno es instantáneo, el otro es una decisión lenta".
IPv6: next header	Actualización.

Apartado b) — flooding y flooding controlado *(solución propia)*

> El apartado b) no tiene solución publicada en el PDF oficial. Lo desarrollo completo.

Qué es flooding. Es el reenvío de un paquete **a todos los puertos de salida excepto aquel por el que entró**. Es lo que hace un switch cuando no sabe la MAC destino, y lo que hace un router con la dirección de broadcast de la LAN.



Por qué existe el flooding y por qué es necesario. Porque **no podés mantener una tabla de destinos completa si hay más destinos que puertos**. Un switch de 48 puertos puede tener learned miles de MACs; los destinos posibles son 2^{48}. La decisión es: **¿mantengo la tabla completa (memoria infinita) o flooding cuando no sé?** Se eligió flooding + aprendizaje. Y para **broadcast**, **no hay tabla posible**: el broadcast es por definición "todos", así que reenviar a todos los puertos **es** la semántica.

Cómo se transforma en flooding controlado. El flooding controlado es **flooding + restricciones**, y hay cinco restricciones distintas:

(1) No reenviar por el puerto de entrada. Es el caso base, y sin esto hay un **loop**. Es exactamente el problema de la Pregunta 2 del Examen 2007-03 (10 puntos): si un puente hace flooding a **todos** los puertos incluido el de entrada, cada trama vuelve, y los dos puentes se la copian el uno al otro **para siempre**. La solución oficial de ese examen es el **árbol de cubrimiento (spanning tree)**: los puentes se comunican, descubren la topología y **desactivan los puertos que crearían un bucle**, de modo que en la topología activa **solo hay un camino** entre cada par de LANs.

(2) Controlar el alcance con el TTL. Es lo que hace un router: si reenvía un broadcast, **decrementa el TTL**, y cuando llega a 0 lo descarta. Eso acota el flooding a un número de saltos y evita que un router que no pertenece al dominio reenvíe. El comportamiento por defecto de los routers es **NO reenviar broadcasts** (se llama *split horizon* en el contexto de VLSM... no: se llama **no forwarding de broadcasts**, o "límite de TTL"). La RFC 1812 lo define: un router **no debe** reenviar datagramas con dirección de broadcast o multicast, salvo casos explícitos.

(3) Limitar por dominio: separar los dominios de broadcast. Cada VLAN es un dominio de broadcast independiente, y los routers **no reenvían** broadcasts entre VLANs. Con un router-on-a-stick o con un switch L3, el flooding queda contenido dentro de cada VLAN. Esta es la razón de ser de las **VLAN** (Pregunta 3 del Examen 2009-03, 6 puntos).

(4) Filtrar por fuente conocida. Un switch **no reenvía un paquete de una MAC que acaba de aprender por el mismo puerto del que llegó**: eso es *ingress filtering* y corta el primer tipo de loop. Y un router con *uRPF* (*unicast reverse path forwarding*, RFC 3704) descarta cualquier paquete cuya dirección **origen** no sea alcanzable por el mismo puerto por el que llegó, lo que corta el **spoofing** y muchos loops de reenvío.

(5) Limitar el ritmo. Cuando el flooding es deliberado (link-state de OSPF, ARP en un dominio grande), hay que acotar el daño de una tormenta de broadcast con *rate limiting* por puerto, o descartando tramas duplicadas dentro de un intervalo corto.

El diagrama del flooding controlado por TTL:

```
H1 ─── S1 ─── R1 ─── R2 ─── R3 ─── S2 ─── H2
       │        │      │      │      │        │
       └────────┴──────┴──────┴──────┘
       dominio de broadcast de H1 (TTL = 64)
       ──────────────────────────────────

       R1 reenvía: TTL 64 → 63
       R2 reenvía: TTL 63 → 62
       R3 reenvía: TTL 62 → 61
       ──────────────────────────────────────

       H2, que está a 4 saltos, NO lo recibe:
       su dominio de broadcast es otro (el de S2).

→ El flooding queda acotado por SALTOS (TTL) y por
  SEGMENTACIÓN en dominios de broadcast (routers y VLANs).
```

Las dos comparaciones que hay que saber para el examen:

UNICAST	BROADCAST
1 destino	todos los destinos del dominio
Tabla: sí, se puede mantener	Tabla: NO PUEDE existir (2⁴⁸ destinos) → flooding por definición
Unknown unicast: ARP en el host, o flooding si no hay entrada	Unknown es el caso NORMAL: toda trama de ARP request es broadcast; la primera respuesta gana (el host descarta la segunda) → "first answer wins"
Se controla con tabla + ARP	Se controla con: TTL, routers que no reenvían, VLANs, spanning tree

El caso especial de ARP que demuestra por qué el flooding es necesario y cómo se controla:

```
ARP request  "¿192.168.1.50 está?"
    → BROADCAST obligatorio (no sabemos a quién preguntarle)
    → llega a TODOS los hosts del dominio  (esto es flooding)

ARP reply  "sí, soy yo: 00:0C:29:AA:BB:50"
    → UNICAST al que preguntó
    → Y además: se cachea en todos los que escucharon
      (por eso el gratuitous ARP es una fuente clásica de
       ARP spoofing / cache poisoning)

CONTROL DEL ALCANCE:
    · si la pregunta es de OTRA subred (target fuera del /24),
      el host NO hace ARP: manda el paquete al gateway, porque
      la MAC no tiene sentido fuera del enlace.
    · un router NUNCA reenvía un ARP request de otro dominio.
    · un bridge solo reenvía dentro de su dominio de broadcast.
```

⚠️ Errores típicos del apartado b)

- **Confundir flooding con "reenviar a todos los puertos incluido el de entrada".** Eso es lo que genera la tormenta de broadcast. El flooding **excluye** el puerto de entrada.
- **Decir que el flooding se controla "con la tabla".** No: la tabla **no** sirve para broadcast, porque el broadcast es todos los destinos. Se controla con TTL, con routers, con VLANs y con spanning tree.
- **Olvidar que el switch aprende del sentido inverso.** El flooding nunca es necesario para una dirección origen recién aprendida.
- **Confundir "dominio de broadcast" con "dominio de colisión".** En una red conmutada hay **muchos dominios de colisión (uno por puerto) y UNO de broadcast**. En una red con hubs, hay **uno de cada uno**. Es la Pregunta 3 del Examen 2010-12.

3.2 Referencia cruzada — la misma pregunta, en otro examen

> **Examen `sirc1103` — 21 de febrero de 2011, Pregunta 1 (8 puntos):** *"Describa claramente las dos funciones clave de la capa de red y explique cómo interactúan entre ellas."*

Es la misma pregunta del apartado a), pero más detallada y con 8 puntos. La solución oficial de 2011-03 es la mejor que hay del tema, y vale la pena citarla entera porque muestra **cómo responde la cátedra**:

> Reenvío (forwarding): mover paquetes entre puertos de entrada y salida del router.
>
> Enrutamiento (routing): determinar la ruta de los paquetes desde origen a destino. Para ello existen los algoritmos de enrutamiento.
>
> Interacción: Cada router intercambia con sus pares, mediante al menos un protocolo de enrutamiento, información que se utiliza, en al menos un algoritmo de enrutamiento, para poblar sus tablas de forwarding. Dichas tablas, en su formato más básico, contienen, para un valor de dirección contenido en la cabecera de los paquetes que llegan al router, el enlace de salida. De esta forma, para cada paquete que arriba a un router por cualquiera de sus interfaces, el router examina la tabla de forwarding (en el caso de IP a partir de la dirección IP destino contenida en el encabezado IP) y a partir del **"longest match"** determina por qué interfaz debe ser enviado (además de determinar cuál es el **"next hop"**, lo

> que determina en el caso más genérico, la dirección destino de capa de enlace que deberá contener la trama que lo lleve a su próximo salto).

Por qué esta solución es excelente y qué copiar de ella:

1. **Separa los dos procesos con una frase cada uno** y sigue.
2. **Nombra el mecanismo de la interacción**: protocolo de enrutamiento → algoritmo → tabla de forwarding.
3. **Dice que la tabla, "en su formato más básico", contiene el enlace de salida** — y **agrega que además contiene el next hop**, que es el refinamiento que hace que la tabla sirva para armar la trama correcta.
4. **Nombra el "longest match"** explícitamente.
5. **Dice "por cualquiera de sus interfaces"**, que es el detalle de que la tabla es **una sola**, no una por interfaz.

Lo que yo agregaría (ya está en mi respuesta del apartado a): **plano de datos vs. plano de control**, **granularidad temporal**, **RIB → FIB**, y el **next header de IPv6**.

PARTE 4 — CIERRE

📌 Resumen

1. **Encapsular** es anteponer la cabecera de la capa al bajar por la pila. **Desencapsular** es quitarla al subir. **Cada capa quita exactamente los bytes que antepuso.**
2. **El byte viajando:** 500 B de aplicación → **520** (con TCP) → **540** (con IP) → **558** en el medio. Overhead: `20 + 20 + 18 = 58 B`.
3. **Eficiencia =** `L / (L + 58)`. A 500 B es 89,6 %; a 50 B es **46,3 %**; a 40 B es **40,8 %**. El overhead es constante y el mensaje no, así que **el overhead relativo se dispara con mensajes chicos**.
4. **Encapsular por cuatro razones:** control local, **multiplexación** (el router lee el destino sin abrir el sobre), que cada capa hable su idioma, e **independencia** (cambiar una capa sin arrastrar a las demás).
5. **Lo que además cambia:** el **direccionamiento** (IP es global y jerárquica, MAC es local y plana; la IP no cambia en tránsito, la MAC cambia en cada router) y el **objetivo** (capa 3 lleva al **host**, capa 4 lleva al **proceso**).
6. **Servicio** = el contrato entre capas (qué promete la de abajo); **interfaz** = dónde se llama. Gracias a eso se puede cambiar una capa sin tocar las demás.
7. **La aplicación NUNCA ve la red.** Le pasa bytes a la capa 4 y recibe bytes. Todo lo demás lo decide la pila.
8. **Presentación** resuelve la heterogeneidad de representación: *endianness* (big-endian en red), codificación (UTF-8), formato, compresión y cifrado (TLS). En TCP/IP no existe como capa.
9. **Forwarding** = por paquete, por interfaz, con *longest prefix match*. **Routing** = por topología, con un algoritmo. El segundo construye la tabla del primero.
10. **Broadcast**: no hay tabla posible, así que **flooding** por definición. **Flooding controlado** = excluir el puerto de entrada (spanning tree), acotar con **TTL**, separar con **routers y VLANs**, y filtrar por origen (*ingress filtering*).

✅ Autoevaluación

1. Un mensaje de 1200 bytes de aplicación. ¿Cuánto viaja por el cable y cuál es la eficiencia?

▶ Respuesta

2. ¿Por qué la cabecera va al principio y no al final del mensaje?

▶ Respuesta

3. Un router reenvía un paquete. ¿Cambia la cabecera IP? ¿Por qué?

▶ Respuesta

4. En el modelo TCP/IP, ¿cuántas PDU hay? ¿Son las mismas que en OSI?

▶ Respuesta

5. ¿Por qué un firewall stateful necesita las capas 3 y 4?

▶ **Respuesta**

6. Explicame por qué la capa de aplicación "no ve la red" con un ejemplo concreto.

▶ **Respuesta**

7. ¿Qué cambia entre un mensaje de 46 bytes y uno de 60 bytes, ambos con TCP sobre IPv4?

▶ **Respuesta**

8. ¿Qué es flooding controlado? Enumerá las cinco restricciones.

▶ **Respuesta**

🔗 Referencias

Exámenes - Examen `sirc0902` (2-feb-2009) — P2 (10 pts): forwarding vs. routing y flooding controlado · P4 (9 pts): datagramas vs. circuitos virtuales → **Clase 4** - Examen `sirc1103` (21-feb-2011) — P1 (8 pts): las dos funciones de la capa de red (la mejor solución oficial del tema) · Problema 2 (30 pts): intercambio de tramas con ARP - Examen `sirc0612` (26-dic-2006) — P1 (5 pts): modelo OSI y Presentación - Examen `sirc0703` (26-feb-2007) — P2 (10 pts): puente con flooding y spanning tree · P3 (10 pts): RARP - Examen `sirc0707` (31-jul-2007) — Problema 2b (25 pts): dos hosts que comparten una IP y responden ARP - Examen `sirc0912` (18-dic-2009) — Problema 1 (30 pts): tablas de ruteo y secuencia de tramas con ARP vacío - Examen `sirc1003` (20-mar-2010) — P1 (8 pts): ARP entre dos hosts con MAC e IP

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., **cap. 1 §1.4** "El modelo de capas" (pág. ~20 a ~26): encapsulación, PDU y servicios entre capas - Tanenbaum & Wetherall, **cap. 1 §1.6** "El modelo TCP/IP" — la comparación de las dos columnas - Tanenbaum & Wetherall, **cap. 2** — tramas de Ethernet, MTU 1500 y tamaño mínimo de 64 bytes (pág. ~45 a ~60) - Kurose & Ross, *Computer Networking: A Top-Down Approach*, cap. 1 §1.7-1.8 — el modelo de 4 capas y la encapsulación con el diagrama de "un byte en la red"

RFCs - `RFC 8200` — *Internet Protocol, Version 6* (2017, *Internet Standard*): el campo **Next Header** en vez de "protocolo" - `RFC 9293` — *TCP* (2022, *Internet Standard*): los 20 bytes de cabecera y el campo *payload offset* que permite encontrar la cabecera IP sin recorrer la trama - `RFC 768` — *UDP* (1980): **8** bytes de cabecera, la mitad de TCP - `RFC 826` — *ARP* (1982): por qué no tiene cabecera IP y por eso no se enruta - `RFC 1812` — *Requirements for IP Routers* (1995): el comportamiento por defecto de **no reenviar broadcasts**, y el conjunto de restricciones de *ingress filtering* que acota el flooding - `RFC 3704` — *Unicast Reverse Path Forwarding* (2002): el filtro por dirección origen que corta loops y spoofing - `RFC 7541` — *HPACK: Header Compression for HTTP/2* (2015): cómo la capa de presentación (compresión de cabeceras) sigue teniendo su función en TCP/IP

Clase siguiente: 4 — Conmutación: circuitos, paquetes y mensajes

Clase 4

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 El criterio que las distingue a todas
- 1.2 Conmutación por circuitos (*circuit switching*)
- 1.3 Conmutación por mensajes (*message switching*)
- 1.4 Conmutación por paquetes (*packet switching*)
- 1.5 El router: 3 entradas, 2 salidas, buffers llenándose
- 1.6 La tabla comparativa de las tres técnicas
- 2.1 Ejercicio 1 — Los buffers de un router con tres paquetes de llegada simultánea
- 2.2 Ejercicio 2 — Utilización de un circuito frente a paquetes con tráfico en ráfagas
- 2.3 Ejercicio 3 — ¿Por qué los paquetes de tamaño variable no se usan?
- 3.1 ⭐ Examen 2007-12, Pregunta 3 (10 puntos) — las tres técnicas con análisis comparativo
- 3.2 ⭐ Examen 2009-02, Pregunta 4 (9 puntos) — datagramas vs. circuitos virtuales
- 3.3 Referencia — Examen 2009-07 P2 (10 puntos), sobre el mismo terreno
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: A (Fundamentos) · **Semana:** 2 · **Duración estimada:** 110 min **Correlativa:** Clase 3 (encapsulación y comparación OSI ↔ TCP/IP) · **Sigue con:** Clase 5 (el retardo en las redes de paquetes) **Material de referencia:** Tanenbaum cap. 1 §1.3 (pág. ~10 a ~20 del PDF de tu carpeta) · RFC 9293, RFC 2737, RFC 3209, RFC 3031

🎯 Al terminar esta clase vas a poder

1. **Distinguir** las tres técnicas de conmutación por un único criterio: **cuándo se toma la decisión** (al establecer la comunicación, o por cada unidad de datos).
2. **Dibujar** el establecimiento de un circuito virtual en la red telefónica y explicar qué recursos quedan reservados.
3. **Explicar** el store-and-forward y **por qué un router no puede reenviar bit a bit sin un buffer infinito**.
4. **Calcular** el retardo de un mensaje de 10 000 bits dividido en 4 paquetes de 2 500 bits que atraviesa 3 enlaces, con la aritmética a la vista, y **compararlo** con el mensaje entero y con el circuito virtual.
5. **Construir** la tabla comparativa de las tres técnicas con: conexión, tamaño de la unidad, reserva de recursos, retardo, posibilidad de reenrutar, orden de llegada y error típico.
6. **Explicar por qué los paquetes de tamaño variable no se usan**: *unfairness* y retardo extremo.
7. **Resolver enteros** el Examen `2007-12 P3` (10 pts) y el Examen `2009-02 P4` (9 pts).

> **Prerequisito del examen:** `2007-12 P3` (10 pts — las tres técnicas con análisis comparativo), `2009-02 P4` (9 pts — datagramas vs. circuitos virtuales), `2011-03 P5` (10 pts — retardo con store-and-forward), `2009-07 P2` (10 pts — distancia entre hops como métrica).

PARTE 1 — TEORÍA

1.1 El criterio que las distingue a todas

Hay tres técnicas de conmutación. Se distinguen por **una sola pregunta**:

```
┌──────────────────────────────────────────────────────────────────┐
│  ¿LA DECISIÓN DE ENCAMINAR SE TOMA ANTES DE EMPEZAR A MANDAR     │
│  DATOS, O SE TOMA PARA CADA UNIDAD DE DATOS?                     │
│                                                                  │
│  ANTES  ──►  conmutación por CIRCUITOS   (y circuitos virtuales) │
│  CADA UNO ─►  conmutación por PAQUETES    (datagramas)           │
│  CADA UNO ─►  conmutación por MENSAJES    (el antecesor)         │
└──────────────────────────────────────────────────────────────────┘
```

Todo lo demás se deriva de esa respuesta:

Pregunta	Circuitos	Paquetes	Mensajes
¿Cuándo se decide?	**Antes**, una vez	Por cada paquete	Por cada mensaje
¿Hay conexión?	**Sí**	No (datagramas)	No
¿Se reservan recursos?	**Sí**	No	No
¿El retardo es?	**Constante**	Variable	Variable y grande
¿El camino es el mismo para todos los paquetes?	**Sí**	Puede cambiar	Puede cambiar

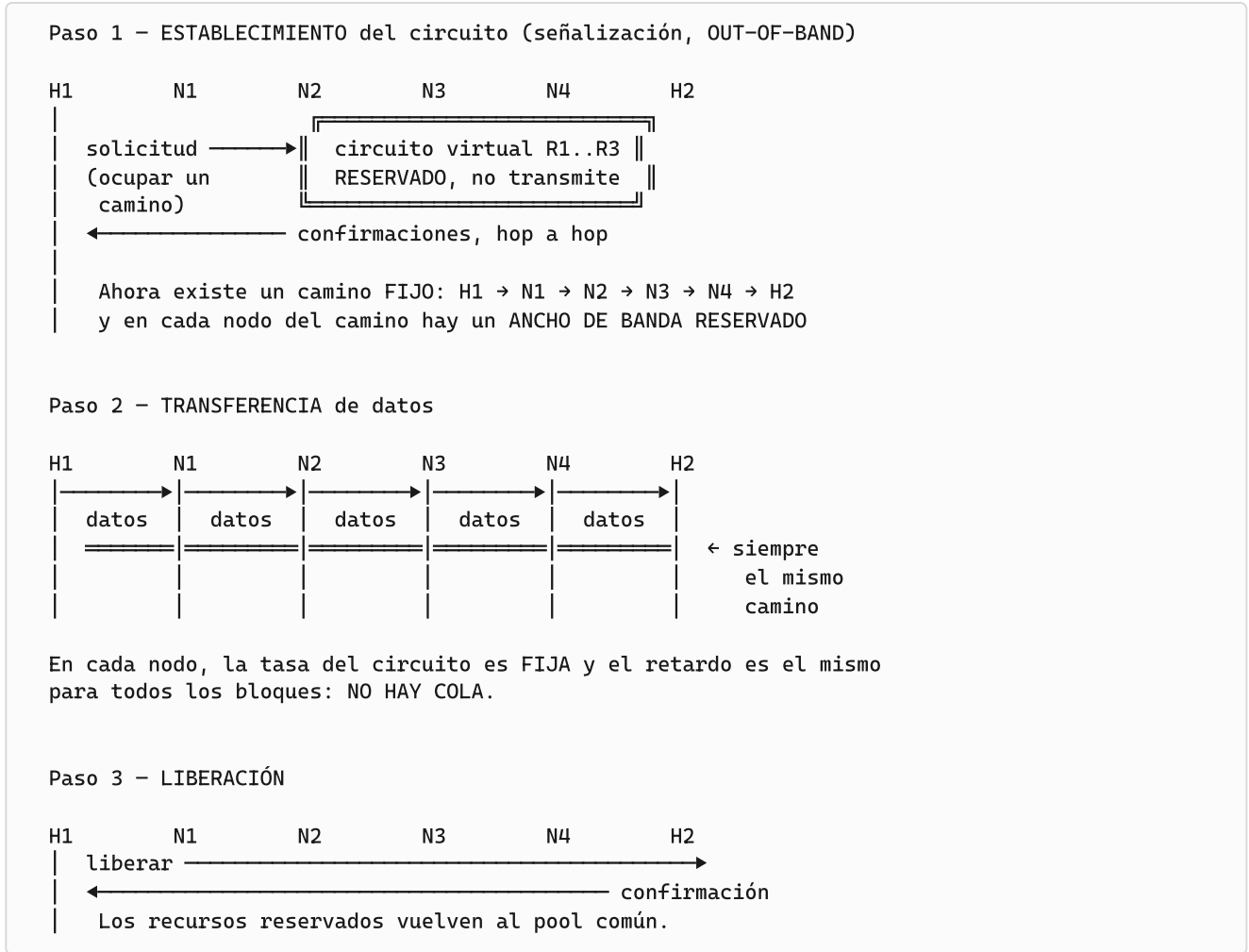
Y una observación de fondo que vale para las tres: las tres son **estrategias para compartir un medio de transmisión** entre varias comunicaciones. La diferencia no es técnica de hardware: es **cuándo se decide el reparto del medio**.

```
MEDIO COMPARTIDO:  ═══════════════════════════════════
La pregunta es: ¿cómo le doy el medio a cada uno?

CIRCUITOS:  "A vos te tocan estos 64 kbps, para siempre.
             Los demás no te los tocan."        → reparto FIJO, static
PAQUETES:   "El que tenga algo para mandar manda,
             y si el medio está libre lo usa."    → reparto DINÁMICO
MENSAJES:   "Solo cuando tengo el mensaje entero
             lo transmito entero."                → reparto DINÁMICO,
                                                    pero con bloqueos largos
```

1.2 Conmutación por circuitos (*circuit switching*)

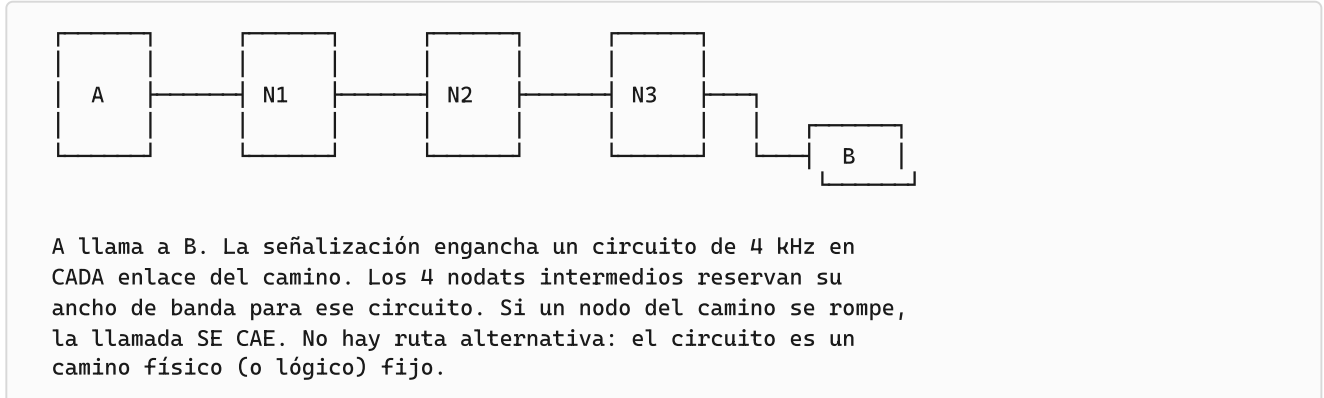
Cómo funciona

Se **establece un circuito** antes de poder mandar datos, y ese circuito **reserva recursos** en cada nodo intermedio.



El ejemplo canónico: la red telefónica

La red telefónica clásica (POTS, *Plain Old Telephone Service*) es la implementación más limpia de conmutación por circuitos, y **todavía lo es**:



Por qué la red telefónica es de circuitos y no de paquetes:

- **La voz necesita retardo constante.** Si un paquete de voz llegara en 20 ms y el siguiente en 200 ms, la conversación sería ininteligible. Un circuito reservado garantiza **retardo constante**, que es lo que hace posible el servicios Telefónico clásico.
- **La voz necesita un ancho de banda constante.** Un circuito de 64 kbps reservados da exactamente 64 kbps siempre. Con paquetes, la voz compite con todo lo demás.

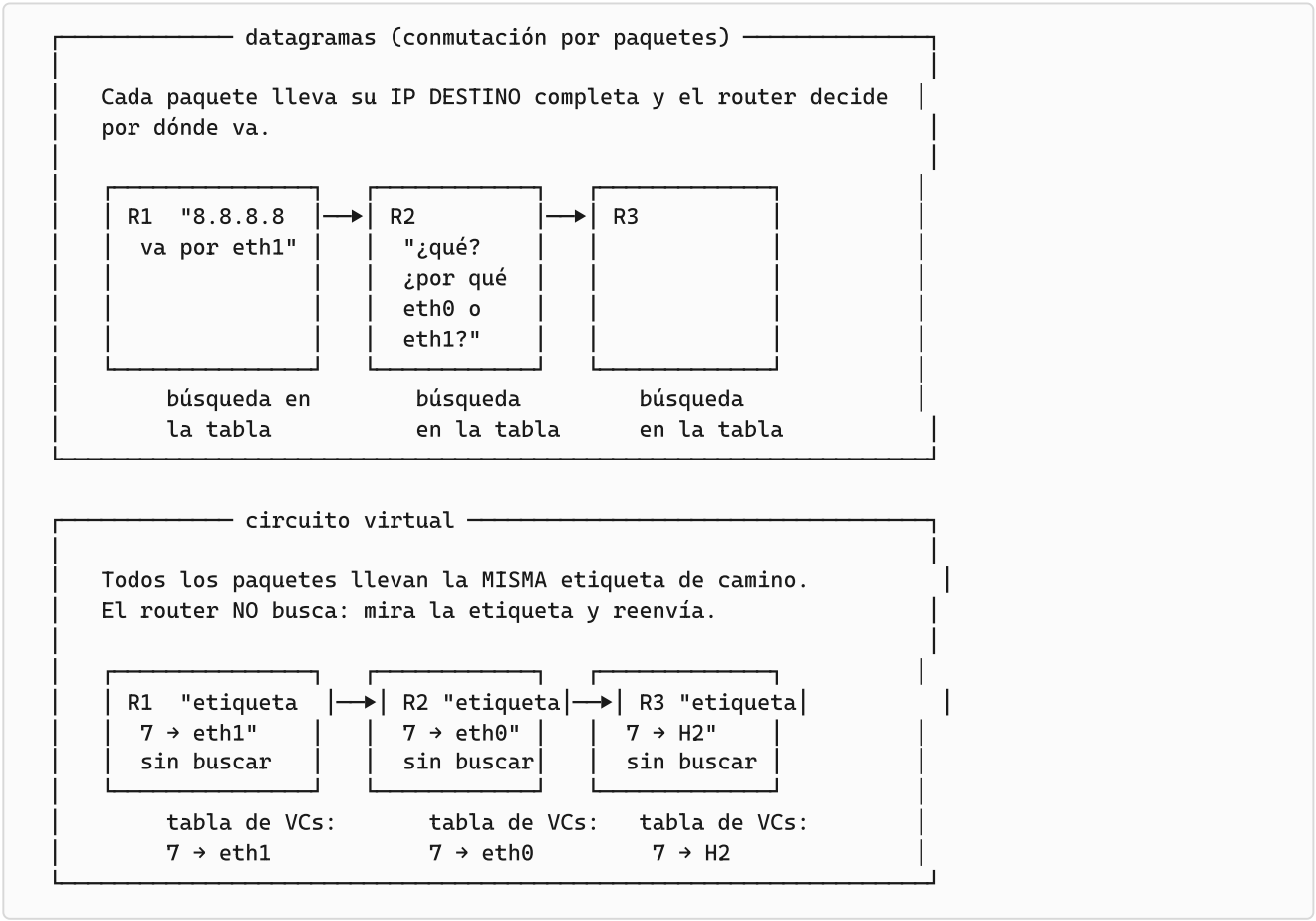
- **La relación de precios es fija.** La voz se cobraba por minuto, y el circuito reservado es lo que hace que el minuto sea medible.

Y las consecuencias negativas, que son las que hacen que Internet no use circuitos:

- **El retardo de establecer el circuito.** Establecerlo lleva de 100 a 500 ms (o hasta 2 s con señalización SS7 y la búsqueda de ruta). Si vas a transferir 300 bytes, no vale la pena.
- **Si se cae un nodo, se rompe todo.** No hay ruta alternativa.
- **Los recursos quedan ociosos.** Si hablás 10 minutos con pausas de un minuto entre frases, el circuito reservado está pagado pero sin usar.
- **No escala.** El número de circuitos simultáneos entre dos nodos es **finito** y hay que dimensionarlo por anticipado.

Circuitos virtuales: la versión "de paquetes" de los circuitos

Circuito virtual (*virtual circuit*, VC) es el híbrido: **se establece un camino lógico antes de mandar los datos**, pero **no se reservan recursos físicos**. Cada paquete lleva una **etiqueta** que identifica el camino, y los routers reenvían **según la etiqueta**, sin consultar la IP de destino.



Diferencias clave respecto del circuito clásico:

	Circuito (físico)	Circuito virtual
Se reserva ancho de banda	**Sí**, en cada nodo	**No** (salvo QoS)
El retardo	**Constante**	**Variable**: depende de la cola
Si se cae un nodo	Se rompe todo	Se rompe el circuito: hay que **reestablecer**
El router busca por	El cable físico (no busca)	**La etiqueta** (busca, pero en tabla chica)
Coste de establecer	Alto (se ocupa el medio)	**Bajo** (solo un mensaje de señalización)

Dónde vive el circuito virtual hoy: Frame Relay, **ATM**, y sobre todo **MPLS**. En MPLS, el "circuito virtual" es el **LSP** (*Label Switched Path*): un túnel unidireccional que se identifica por una **etiqueta** de 20 bits apilable. El router MPLS **no consulta la IP de destino**: consulta la etiqueta, que es una entrada en un array. Eso es **más rápido** y es la razón por la que MPLS se usa en el backbone de los operadores: la decisión es un índice, no un longest prefix match. (Clase 45 lo desarrolla entero.)

Y el dato de 2026: el plano de control de MPLS **se está moviendo a Segment Routing** (`RFC 8402`, `RFC 8660`, y `RFC 8986` **SRv6**), que **elimina el MPLS del plano de datos en el caso de SRv6**: las etiquetas se codifican en la cabecera IPv6 (el *Segment Routing Header*). El circuito virtual sigue existiendo conceptualmente, pero la implementación es otra.

1.3 Conmutación por mensajes (*message switching*)

Cómo funciona

Cada **mensaje completo** (tamaño arbitrario) es reenviado de nodo en nodo. El nodo **lo recibe entero**, **lo guarda en un buffer**, y **espera a poder transmitirlo completo** por el siguiente enlace. Es **store-and-forward sin fragmentar**.

```
Un mensaje de 8000 bits, 4 nodos intermedios, enlaces de 1000 bit/s:

H1 ─── N1 ─── N2 ─── N3 ─── H2

t=0        t=8      t=16     t=24     t=32     t=40   (segundos)

|—8000b—→|         |         |         |         |
|         |—8000b—→|         |         |         |
|         |         |—8000b—→|         |         |
|         |         |         |—8000b—→|         |
|         |         |         |         |—8000b—→|
|         |         |         |         |         |

Cada salto tarda 8000/1000 = 8 segundos.
El mensaje entero tarda 4 × 8 = 32 segundos en salir del último nodo,
y 40 segundos en llegar a H2.

⚠️ El RETARDO ES ENORME. Y es el mismo para el mensaje entero
  que para cualquier otro: no hay pipeline.
```

El store-and-forward y por qué el router tiene que esperar el paquete entero

Este es el **argumento central** de la clase, y conecta directo con la Clase 5. Hay que entenderlo bien.

El store-and-forward es un compromiso entre memoria y velocidad. La razón por la que un router **no** puede reenviar bit a bit (o byte a byte) es esta:

```
Si el router reenvía bit a bit (cut-through), el PROBLEMA es este:

H1 (100 Mbps) ⟹ N1 ⟹ N2 ⟹ N3 (1 Mbps)

Bit a bit, N1 empieza a mandar hacia N2 CADA VEZ QUE LLEGA UN BIT de H1.

Como H1 manda a 100 Mbps y N1->N2 transmite a 10 Mbps, por cada bit
que N1 transmite tiene que GUARDAR 10 bits que le llegan.

En 1 segundo:
   entran 100 000 000 bits
   salen    10 000 000 bits
   se guardan 90 000 000 bits = 11,25 MB

⚠️ Para no perder NADA, N1 necesitaría un buffer INFINITO.

┌──────────────────────────────────────────────────────────┐
│  Si el buffer es FINITO (y lo es), el exceso se DESCARTA.  │
│  Y un paquete descartado es un paquete perdido para       │
│  siempre, porque IP no recupera de pérdidas.              │
│                                                           │
│  ⇒ CON ELTA DEGRADADO, EL CORTAFALLOS ES PEOR.            │
└──────────────────────────────────────────────────────────┘
```

La solución que eligió Internet: store-and-forward con tamaño de paquete acotado. El trade-off completo:

```
Store-and-forward con paquetes de tamaño FIJO (L):

┌──────────────────────────────────────────────────────────────┐
│  El tamaño del buffer es finito y el tamaño del paquete      │
│  está DISEÑADO para que quepan muchos paquetes completos.    │
│                                                              │
│  Si el paquete mide L bytes, un buffer de B bytes            │
│  almacena ⌊B / L⌋ paquetes completos.                        │
│                                                              │
│  Con L ACOTADO:                                              │
│     · el buffer se llena y se descartan paquetes completos   │
│       (no bites sueltos) → se pierde UNO, no la mitad de uno │
│     · la unidad deLoss es predecible                         │
│     · el router puede reordenar y descartar                  │
│                                                              │
│  El tamaño MÁXIMO del paquete es una decisión de diseño      │
│  que hace que todo esto funcione: MTU = 1500 en Ethernet.    │
└──────────────────────────────────────────────────────────────┘
```

El argumento completo, paso a paso:

1. Un router tiene **buffers finitos**. (Los routers reales tienen entre 4 MB y 512 MB, según la clase.)
2. El medio tiene **velocidades distintas** entre entrada y salida.
3. Si reenvía **bit a bit**, el buffer crece sin límite → o hace falta memoria infinita, o se descarta y se pierde información.
4. Si reenvía **por paquetes completos**, el buffer necesario es `⌊B/L⌋ · L` bytes, que es **acotado y conocido**.
5. Cuando el buffer se llena, se **descarta un paquete entero**. La unidad de pérdida es **un paquete, no un bloque de bytes**.

6. Eso **no elimina** la pérdida, pero la hace **predecible** y permite que las capas superiores la manejen (TCP la recupera; UDP no).
7. **Conclusión: el tamaño máximo del paquete es el precio de la conmutación por paquetes.** Y ese precio se paga en el MTU.

La lista de.waiters: el problema de la conmutación por mensajes

El problema más grave de la conmutación por mensajes no es el retardo: es el **bloqueo de cabeza** (*head-of-line blocking*) y la **injusticia**.

```
N1 tiene UN solo enlace de salida (hacia N2), de 1000 bit/s.
En el buffer hay:

   [  mensaje A: 8000 bits  ]  ← llegó primero, se transmite entero
   [  mensaje B: 20 bits    ]  ← llegó después, NO PUEDE salir
   [  mensaje C: 5000 bits  ]  ← llegó después, NO PUEDE salir

N1 transmite A completo. Durante 8 segundos, B y C esperan.
Después transmite C completo. B espera 5 segundos más.
B llevaba 13 segundos de espera por un mensaje de 20 bits.

⚠️ INJUSTICIA: un mensaje pequeñoDelayed por un mensaje grande
   que estaba delante. En un sistema de tiempo real, B es
   inservible.
```

Por qué no se usa la conmutación por mensajes hoy:

- **Retardo enorme y no predecible.** El retardo depende del tamaño de la cola, que depende de los demás. No podés dar una garantía.
- **Bloqueo de cabeza.** Un mensaje grande retrasa a todos los que están detrás.
- **Requiere buffers enormes.** Para no perder un mensaje de 1 MB hay que tener 1 MB de buffer, y **los routers tienen MB, no GB**. El MTU de un mensaje tendría que ser enorme.
- **El análisis de la Pregunta 5 del Examen 2009-07** (10 pts) muestra por qué esto es un problema de enrutamiento, no solo de conmutación.

Pero tampoco es inútil como idea: es exactamente lo que hacen los **store-and-forward de la capa de aplicación** y los **enrutadores de mensajeros**:

- **Un *mail relay*** (MTA) que recibe un email completo y lo reenvía después.
- **Un router de BGP que recibe un UPDATE completo** y lo procesa.
- **Un forwarder de logging** que recibe un evento completo.
- **La conmutación de mensajes en el contexto de las "SAN"** (*Storage Area Networks*) y los sistemas de archivos distribuidos.

Conclusión histórica: la conmutación por mensajes **es la generalización de las otras dos**. Si el mensaje es del tamaño de un circuito y hay uno reservado, es conmutación de circuitos. Si el mensaje se divide en paquetes acotados, es conmutación por paquetes. **La conmutación por mensajes es el caso sin acotar**, y por eso es la peor de las tres.

1.4 Conmutación por paquetes (*packet switching*)

La idea

Los mensajes se **dividen en paquetes de tamaño fijo o acotado**, y cada paquete se reenvía **de forma independiente** con store-and-forward.

```
MENSAJE CONMUTACIÓN POR PAQUETES

┌──────────────────────────┬──────────────┬──────────────┬──────────┐
│                          │              │              │          │
│  Paquete 1: 2500 bits    │  Paquete 2   │  Paquete 3   │  P 4     │
└──────────────────────────┴──────────────┴──────────────┴──────────┘
◄──────────────── 10 000 bits del mensaje original ─────────────────►

Ahora los paquetes se reenvían por separado, y el pipeline funciona:

tiempo →  0    2    4    6    8    10   12   14   16   18   20
          |    |    |    |    |    |    |    |    |    |    |
H1        |P1  |P2  |P3  |P4  |
N1        |    |P1  |P2  |P3  |P4
N2        |    |    |P1  |P2  |P3  |P4
N3        |    |    |    |P1  |P2  |P3  |P4
H2        |    |    |    |    |    |P1  |P2  |P3  |P4

⚠️ PIPELINE: mientras P2 entra en N1, P1 ya está saliendo de N1.
   El último paquete sale 2 segundos después del primero, no 8.
```

El ejemplo numérico completo

Datos: mensaje de **10 000 bits**, dividido en **4 paquetes de 2 500 bits**. **Tres enlaces** de **1 000 bit/s** cada uno, unidos por **dos conmutadores** (N1 y N2). `d_proc` despreciable, sin cola.

```
H1 ═══════ N1 ═══════ N2 ═══════ H2
 e1        e2        e3
1000      1000      1000  bit/s
```

Paso 1 — Retardo de transmisión de un paquete en cada enlace.

```
d_trans = L / R = 2500 bits / 1000 bit/s = 2,5 segundos
```

Paso 2 — El comportamiento de cada conmutador (store-and-forward de paquetes).

En cada nodo, **recibo un paquete entero antes de poder empezar a transmitir el siguiente**. Eso significa que el nodo está ocupado `2,5 s` con cada paquete:

```
e1 (H1→N1):  transmite P1 de 0 a 2,5 ; P2 de 2,5 a 5,0 ; P3 de 5,0 a 7,5 ; P4 de 7,5 a 10,0
e2 (N1→N2):  transmite P1 de 2,5 a 5,0 ; P2 de 5,0 a 7,5 ; P3 de 7,5 a 10,0 ; P4 de 10,0 a 12,5
e3 (N2→H2):  transmite P1 de 5,0 a 7,5 ; P2 de 7,5 a 10,0 ; P3 de 10,0 a 12,5 ; P4 de 12,5 a 15,(
```

Paso 3 — El retardo de cada paquete, extremo a extremo.

Cada paquete atraviesa 3 enlaces y pasa por 2 conmutadores (cada uno con `d_proc`):

```
d(P) = d_proc(N1) + L/R(e1) + L/R(e2) + d_proc(N2) + L/R(e3) + Σd_prop
     =      0     +   2,5   +   2,5   +     0      +   2,5   +   Σd_prop
     =       7,5 s  +  Σ d_prop
```

Con `d_prop = 0` (enlaces de distancia despreciable):

```
d(P1) = 7,5 s      llega a H2 en t = 7,5 s
d(P2) = 7,5 s      llega a H2 en t = 10,0 s
d(P3) = 7,5 s      llega a H2 en t = 12,5 s
d(P4) = 7,5 s      llega a H2 en t = 15,0 s
```

Paso 4 — El retardo del mensaje COMPLETO (lo que le importa al usuario que espera el archivo).

El mensaje está completo cuando llega **el último** paquete:

```
d(mensaje) = 15,0 segundos
```

Paso 5 — La comparación con las otras dos técnicas.

Si en lugar de 4 paquetes de 2500 bits:

```
┌──────────────────────────────────────────────────────────────────┐
│  TÉCNICA              │  ¿Cuándo está listo el último byte?      │
├──────────────────────────────────────────────────────────────────┤
│  MENSAJES             │                                          │
│  (1 mensaje de 10000) │  4 saltos × (10000/1000) = 4 × 10 = 40 s │
│                       │                                          │
│  PAQUETES             │                                          │
│  (4 paquetes de 2500) │  último paquete entra en e3 en t=12,5    │
│                       │  y tarda 2500/1000 = 2,5 s más           │
│                       │  → 15,0 s                                │
│                       │                                          │
│  CIRCUITO             │                                          │
│  (reservado 1000b/s)  │  10000/1000 = 10 s, SIN STORE-AND-FORWARD│
│                       │  (el circuito nunca tiene el enlace libre)│
├──────────────────────────────────────────────────────────────────┤
│  Ganador              │  CIRCUITO (10 s), pero necesita 3 s de   │
│                       │  establecimiento → 13 s en el mejor caso │
└──────────────────────────────────────────────────────────────────┘
```

El análisis completo, con las cuatro cifras a la vista:

Técnica	Establecimiento	Retardo del último byte	**Total**
Mensajes	0	4 × 10 = **40 s**	**40 s**
Paquetes	0	12,5 + 2,5 = **15 s**	**15 s**
Circuito	3 s	10 000/1000 = **10 s**	**13 s**

Interpretación — y acá está el resultado clave:

- **Paquetes (15 s) vs. mensajes (40 s): los paquetes son 2,67 veces más rápidos.** La razón: el **pipeline**. Con un solo mensaje, cada nodo espera a recibirlo entero (10 s) antes de empezar a mandar; con paquetes, cada nodo espera solo 2,5 s y después puede seguir trabajando.
- **Circuito (13 s) es el más rápido**, pero con una condición: **el circuito tiene que existir de antemano**. Si se establece ahora, son 13 s. Y **solo gana si el mensaje es grande**: si es de 2000 bits, el circuito tarda 3 s de

establecimiento + 2 s de datos = 5 s, y los paquetes 1 + 2 × 2 = ... calculemos: 1 paquete de 2000 bits, `d = 3 × (2000/1000) = 6 s` → **6 s, más rápido que el circuito**.

- **El punto clave: el circuito gana para mensajes GRANDES y el paquete para mensajes CHICOS.** Y para tráfico intermitente, el circuito es **desastroso**: si A y B se turnan para hablar 1 segundo cada uno, el circuito está reservado el 100 % del tiempo para una conversación que usa el 100 % del tiempo.

La fórmula general del retardo con paquetes

Esto es lo que se pide en la **Pregunta 5 del Examen 2011-03** (10 puntos), y lo resuelvo entero en la Clase 6:

```
Con N paquetes de tamaño L en una línea de n enlaces, todos de
velocidad R, con store-and-forward y sin cola ni propagación:

· El primer paquete tarda:      n · (L/R)     [n saltos de store-and-forward]
· Cada paquete siguiente tarda: (L/R)         [solo el último enlace]
· El último de los N paquetes tarda:
      n·(L/R) + (N−1)·(L/R) = (N + n − 1) · L/R
```

La interpretación del resultado `(N + n − 1)·L/R`:

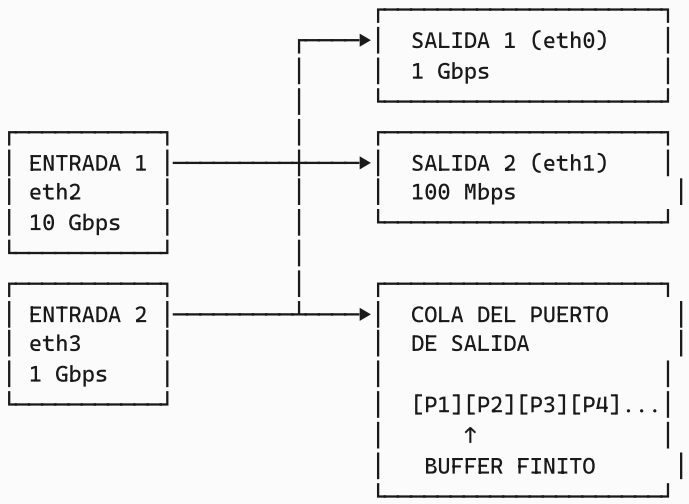
```
N  = cantidad de paquetes   (el trabajo)
n  = cantidad de enlaces    (el pipeline)

N · L/R   = el trabajo puro: es IGUAL en todas las técnicas
(n−1)·L/R = el costo del pipeline: el primer paquete tiene que
            recorrer los n enlaces uno por uno; cada uno después
            sólo tiene que recorrer el último.

→Mientras más paquetes (N grande), más se amortiza el pipeline
  (n−1)·L/R y más se parece al tiempo del circuito.
→ Mientras más enlaces (n grande), más caro es el pipeline.
```

El límite importante:

```
N → ∞   (paquetes infinitamente chicos)
d → n · L/R = 10000/1000 = 10 s   ← el tiempo del circuito

⚠️ PERO: si los paquetes son demasiado chicos, el overhead los
  ahoga. Con 20 B de cabecera por paquete, un mensaje de 10 000
  bits con paquetes de 100 bits tiene 100 paquetes y 2000 bytes
  de overhead: el 17 % del tráfico es cabecera.
```

1.5 El router: 3 entradas, 2 salidas, buffers llenándose

El diagrama que explica por qué existe el retardo de cola, y por qué es el **único componente variable** del retardo.




```
⚠️ eth2 entra a 10 Gbps, pero sale por eth0 (1 Gbps) o
   eth1 (100 Mbps). El 90 % o el 99 % del tráfico que entra
   por eth2 NO SE PUEDE TRANSMITIR en ese instante.

El buffer de la cola de salida de eth1 (100 Mbps) SE LLENA.
Los paquetes que no entran se DESCARTAN.
```

La aritmética del desbordamiento, que es donde nace la pérdida:

```
Supongamos:
   eth2 envía 10 Gbps de paquetes de 1500 bytes
   eth1 puede sacar 100 Mbps

En 1 segundo:
   llegan    10 000 000 000 bits / 1500 bits = 6 666 667 paquetes
   salen        100 000 000 bits / 1500 bits =     66 667 paquetes

   ⚠️ Se acumulan 6 666 667 − 66 667 = 6 600 000 paquetes por segundo
      = 6 600 000 × 1500 bytes = 9 900 000 000 bytes = 9,9 GB por
      SEGUNDO en la cola.

Un buffer de 4 MB se llena en:
   4 000 000 bytes / 9 900 000 000 bytes/s = 0,0004 s

⇒ Con 4 MB de buffer, en 400 MICROSEGUNDOS se empieza a perder.
```

Eso significa tres cosas, y las tres son importantes:

1. **La pérdida es inevitable** si la tasa de llegada supera la de salida. No es un defecto del router: es aritmética.
2. **El retardo de cola se dispara**: un paquete que llega "encima" de la cola espera **millones de paquetes × su propio tiempo de transmisión**.
3. **El tamaño del buffer NO protege**: agrandarlo 100 veces retrasa la muerte 100 veces, no la evita. Y **agrandar el buffer aumenta el retardo** de los paquetes que sí pasan (bufferbloat, Clase 21).

Y este es el error típico del ejercicio 2.1: dibujar "un router con 3 paquetes en el buffer y todo bien". Lo correcto es:

```
El retardo de cola de un paquete que llega en la posición k de la cola:

    d_cola = k · L / R_salida

Con 6 600 000 paquetes de cola:
    d_cola = 6 600 000 × 1500 bits / 100 000 000 bit/s
           = 6 600 000 × 15 µs
           = 99 segundos    ⚠️ ¡El paquete tarda más de un minuto
                             en salir!

Con un buffer de 4 MB (2666 paquetes de 1500 B):
    capacidad del buffer = 4 194 304 / 1500 = 2796 paquetes
    d_cola máximo = 2796 × 1500 / 100 000 000 = 41,9 ms
    lo que NO entre → se descarta
```

El dato estructural que hay que recordar: el buffer es **una cola de espera**, y como toda cola, tiene un límite. Los routers reales no pueden tener buffers del tamaño del tráfico que deben absorber: si el tráfico es de 100 Gbps y el buffer es de 100 GB, no lo construís. **El buffer siempre es más chico que la carga de la ráfaga.** Por eso la pérdida es parte del diseño normal de Internet, y no una excepción.

1.6 La tabla comparativa de las tres técnicas

Esta tabla es **la respuesta al Examen 2007-12 P3** y hay que memorizarla completa.

Característica	Conmutación por circuitos	Conmutación por paquetes	Conmutación por mensajes
¿Hay conexión?	**Sí**, se establece un circuito	**No** (datagramas) / **sí** (circuito virtual)	No
Cuándo se decide el camino	**Antes**, al establecer	**Por cada paquete**	Por cada mensaje
Unidad de datos	**Flujo continuo** (bytes)	**Paquete** de tamaño fijo o acotado	**Mensaje** completo, tamaño arbitrario
Tamaño típico	El que se reserve (64 kbps telefónicos)	**1500 bytes** (MTU de Ethernet)	**Ilimitado** (MB)
¿Se reservan recursos?	**Sí**, ancho de banda en cada nodo	**No** (mejor esfuerzo)	No
Retardo	**Constante**, sin cola	**Variable**, depende de la cola	**Variable y grande**
¿Se puede reenrutar?	**No**: si cae un nodo, se rompe	**Sí**: cada paquete puede ir por donde sea	Sí
Orden de llegada	**Siempre en orden** (mismo camino)	**Puede llegar desordenado** (caminos distintos)	Puede llegar desordenado
Retardo de establecer	**Alto** (100 ms a 2 s)	**Cero**	Cero
Uso del medio	**Ineficiente** si hay pausas (se paga y no se usa)	**Eficiente**: multiplexado estadístico	Eficiente en teoría, malo en la práctica
Bloqueo de cabeza	No	**Sí** si un paquete grande retrasa a los chicos	**Muy sí**: un mensaje enorme retrasa todo
Tamanjos de buffer	No necesita (recursos reservados)	**Necesita**: $\lfloor B/L \rfloor \cdot L$	**Necesita B ≥ tamaño del mensaje mayor**
Error típico	"Reservar recursos es eficiente"	"El retardo es constante"	"Es lo mismo que paquetes pero más grande"
Dónde se usa	Red telefónica clásica; **MPLS** (LSP)	**Internet**. Frame Relay, ATM	**Prácticamente nada** como tal

La frase resumen de cada una

- **Circuitos:** "*decidí una vez y pagás por el canal aunque no lo uses*".
- **Paquetes:** "*no decido nada por adelantado, mando lo que puedo y me arreglo*".
- **Mensajes:** "*espero a tenerlo todo entero antes de mandar nada*".

Por qué Internet usa paquetes

Cuatro razones, y cada una es un **compromiso que rechaza** una ventaja de los circuitos:

1. **No hay que establecer nada.** El retardo de establecimiento de un circuito (100 ms a 2 s) es inaceptable para una petición web. Con paquetes, la primera petición sale inmediatamente.
2. **No se reserva capacidad ociosa.** El tráfico de Internet es en ráfagas. Reservar 1 Gbps para una conexión que transmite 100 kbps el 10 % del tiempo es tirar dinero. Los paquetes **multiplexan estadísticamente**: los picos de unos llenan los valles de otros.

3. **Se puede reenrutar.** Si cae un router, el siguiente paquete puede ir por otro camino. Un circuito no puede: se rompe. Esta es la diferencia entre **IP** (best effort, sin conexión) y un protocolo orientado a conexión como ATM.
4. **Es stateless en el núcleo.** Un router de Internet **no guarda información de las conexiones**: solo mira la IP destino y reenvía. Puede atender millones de flujos simultáneos con una memoria modesta. Es la razón por la que **IntServ (RFC 2211) no escaló** y DiffServ ganó (Clase 45): mantener estado por flujo en cada router es insostenible.

El precio que se paga por elegir paquetes:

```
┌──────────────────────────────────────────────────────────┐
│ 1. Retardo variable e impredecible.  Para una llamada de voz   │
│    150 ms es inaceptable; para una descarga de archivo da igual.│
│ 2. Pérdidas.  Si se pierde un paquete, se retransmite entero,    │
│    y se duplicó el retardo.                                      │
│ 3. Desorden.  Paquetes por caminos distintos llegan desordenados.│
│ 4. Holgura obligatoria (headroom).  Hay que dejar el medio       │
│    sobrecargado para poder absorber ráfagas.                     │
│    Y sobrecargar significa aumentar el retardo.                  │
│    Ese es el dilema del "90 % de utilización es óptimo".         │
└──────────────────────────────────────────────────────────┘
```

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **Las tres técnicas y el criterio de decisión** (antes / por cada unidad de datos). - **Que Internet usa conmutación por paquetes** y que es *best effort*. - **El store-and-forward** y la razón de que el router espere el paquete entero (buffers finitos). - **Que el tamaño máximo del paquete (MTU) es el precio del store-and-forward**. - **La fórmula** `(N + n − 1)·L/R` para el retardo del último paquete. - **Que MPLS implementa circuitos virtuales (LSP) en el plano de datos.**

Cambió o se modificado:

- **El "servicio de circuitos" de la red telephone clásica ya no existe como tal.** La PSTN se cerró en la mayoría de los países y la voz.mouse migró a **VoIP sobre IP**, que es conmutación de paquetes con **un circuito virtual lógico arriba**: los operadores de VoIP **reservan recursos con SIP/IMS** para dar QoS a la voz, que es exactamente "conmutación por circuitos dentro de una red de paquetes". ⇒ **La frontera del Bloque A se corrió: la voz ya no es el ejemplo de circuitos.**
- **El circuito virtual sigue, pero cambió de implementación. MPLS** sigue desplegándose masivamente en los carriers (`RFC 3031` arquitectura, `RFC 3032` pila de etiquetas, `RFC 4364` MPLS VPN). Pero el **plano de control se está moviendo a Segment Routing**: `RFC 8402` (arquitectura), `RFC 8660` (SR-MPLS), `RFC 8986` **(SRv6)**. En **SRv6 el MPLS desaparece del plano de datos**: los segmentos se codifican en un *Segment Routing Header* de IPv6.
- **En el datacenter, MPLS fue reemplazado por EVPN-VXLAN.** `RFC 7432` (EVPN) + `RFC 7348` (VXLAN, que encapsula sobre **UDP/4789**). Es el estándar de overlay de datacenter, no el MPLS de los operadores. Y VXLAN **sí es un circuito virtual**, pero de capa 2, no de capa 3.
- **La conmutación por "mensajes" tiene un caso de uso nuevo: los servicios de *store-and-forward* de capa de aplicación.** Webhooks, colas de mensajes (SQS, Kafka, RabbitMQ), y los **MTA de correo** siguen siendo exactamente conmutación por mensajes: reciben el mensaje completo, lo bufferizan, y lo reenvían cuando pueden.

- **El control de congestión terminó de asumir que la red es de paquetes. CUBIC (`RFC 9438`, 2023, *Standards Track*)** es el default global; en 2012 se enseñaba Reno con "quizá CUBIC". Y **BBR nunca fue RFC**: es `draft-ietf-ccwg-bbr`, *Experimental*, y especifica **BBRv3**.
- **La multicast IP es conmutación por paquetes con State.** IP multicast (`RFC 1112` y siguientes) mantiene **estado por grupo** en los routers que están en el camino, lo cual es un punto medio entre datagramas y circuitos virtuales: la decisión se toma una vez al unirse al grupo, pero **no se reserva ancho de banda**.

> **Consecuencia para el examen:** el **esquema conceptual de las tres técnicas** es de 1981 (Tanenbaum lo teach desde el capítulo 1) y **no cambió**. Lo que cambió es **quién lo usa**: la voz dejó de ser el ejemplo de circuitos, y los circuitos virtuales pasaron de MPLS a SRv6 a VXLAN. Si te preguntan "*¿qué se usa hoy?*", respondé con el esquema y después una línea de actualización.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Los buffers de un router con tres paquetes de llegada simultánea

Enunciado. Un router tiene **3 interfaces de entrada** y **2 de salida**:

- eth2 (entrada): 10 Gbps, por donde entra una ráfaga de **3 paquetes de 1500 bytes** en el mismo instante.
- eth3 (entrada): 1 Gbps, sin tráfico.
- eth0 (salida): 1 Gbps, buffer de **100 000 bytes**.
- eth1 (salida): 100 Mbps, buffer de **20 000 bytes**.

Los tres paquetes están dirigidos a destinos que salen por **eth1**.

a) ¿Cuántos paquetes entran en el buffer de eth1? ¿Cuántos se descartan? **b)** ¿Cuál es el retardo de cola del primer paquete y del último que entra? **c)** ¿Qué pasa si en lugar de 3 paquetes de 1500 bytes entran 3 paquetes de 40 bytes?

Resolución.

Paso 1 — La tasa de llegada y la de salida.

```
eth1 transmite a 100 Mbps.
Llegan 3 paquetes de 1500 bytes = 4500 bytes = 36 000 bits.

Tiempo que tardan los 3 en entrar:
    36 000 bits / 10 000 000 000 bit/s = 0,0000036 s = 3,6 µs

Tiempo que tarda eth1 en sacar los 3:
    36 000 bits / 100 000 000 bit/s = 0,00036 s = 360 µs
```

Interpretación: los 3 paquetes entraron en 3,6 µs y eth1 necesita 360 µs para sacarlos. **El buffer tiene que absorber la diferencia.**

Paso 2 — La capacidad del buffer.

```
Buffer de eth1 = 20 000 bytes
Tamaño de un paquete = 1500 bytes
Capacidad = ⌊20 000 / 1 500⌋ = ⌊13,33⌋ = 13 paquetes

⚠️ OJO: se guardan 13 paquetes COMPLETOS = 13 × 1500 = 19 500 bytes.
   Los 500 bytes restantes no se usan, porque un paquete
   a medias no sirve: o entra entero o se descarta.
   ESTA es la consecuencia dePacketizing: el buffer se usa
   de a paquetes completos.
```

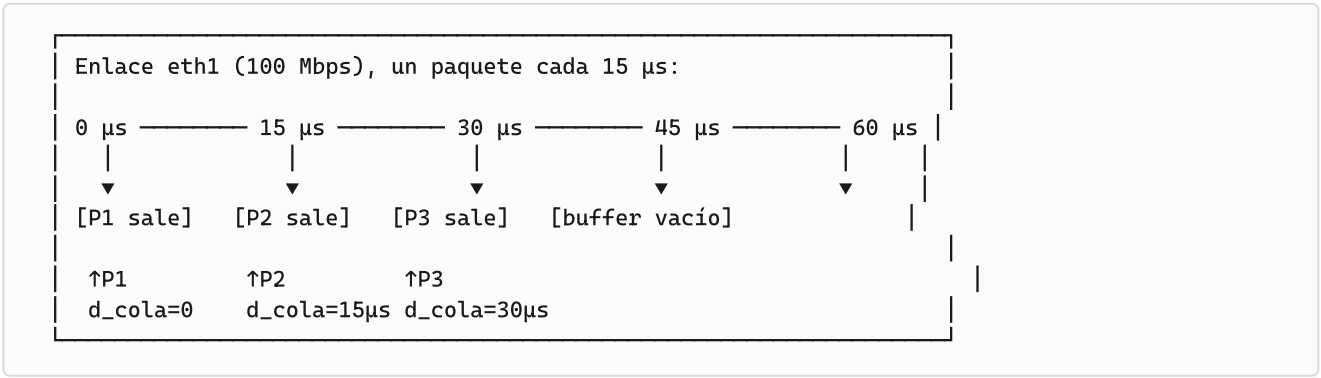
Paso 3 — Respuesta a a).

```
Llegan 3 paquetes. El buffer tiene capacidad para 13.
Los 3 caben.

DESCARTADOS = 0
EN COLA     = 3   (los 3 esperan turno)
```

Paso 4 — Respuesta a b): el retardo de cola de cada uno.

El retardo de cola del paquete que está en la posición `k` es el tiempo que tardan los `k-1` paquetes que tiene adelante en salir por el enlace de salida:

```
d_cola(P1) = 0 × (L/R) = 0                    ← es el primero, sale ya
d_cola(P2) = 1 × (1500/100 Mbps) = 15 µs
d_cola(P3) = 2 × (1500/100 Mbps) = 30 µs

(L/R = 1500 bits / 100 000 000 bit/s = 15 µs por paquete)
```



Paso 5 — Respuesta a c): paquetes de 40 bytes.

```
Buffer = 20 000 bytes, paquete = 40 bytes
Capacidad = ⌊20 000 / 40⌋ = 500 paquetes

Llegan 3. Caben todos. DESCARTADOS = 0.

d_cola(P2) = 1 × (40 bytes × 8 / 100 000 000)
           = 1 × 320 / 100 000 000
           = 3,2 µs

d_cola(P3) = 2 × 3,2 µs = 6,4 µs

Comparación:
   con 1500 B: d_cola(P3) = 30 µs
   con   40 B: d_cola(P3) =  6,4 µs      → 4,7 veces MENOS
```

Interpretación del apartado c): con paquetes chicos, **el mismo volumen de tráfico se almacena y se sirve con menos retardo de cola**, porque la granularidad de la cola es fina. Pero el costo es **overhead**: 58 bytes de cabecera por cada 40 bytes de datos = más del 100 % de overhead. Es el mismo compromiso de siempre: **granularidad contra eficiencia**, y es exactamente el análisis de la Pregunta 1b del Examen 2007-12 sobre asignación de recursos.

⚠️ Errores típicos

- **Contar bytes en vez de paquetes en el buffer.** Con 20 000 bytes de buffer entran 13 paquetes de 1500 B, no 13,33. Un paquete a medias no sirve.
- **Calcular `d_cola` como el tiempo total de la cola** y no el tiempo de los paquetes que tiene **delante**. El primer paquete de una cola vacía tiene `d_cola = 0`.
- **Olvidar que el retardo de cola se suma al de transmisión.** El paquete no "sale" en el instante `k·L/R`: empieza a salir después de esperar, y tarda `L/R` más en salir.

- **Confundir el buffer de eth1 con el de eth0.** El eth2 a 10 Gbps drena por eth1 (100 Mbps), así que **eth1 es el cuello de botella** y es donde hay cola. eth0 no tiene tráfico.

2.2 Ejercicio 2 — Utilización de un circuito frente a paquetes con tráfico en ráfagas

Enunciado. Dos aplicaciones comparten un enlace de **1 Mbps**:

- **Aplicación A**: tráfico en **ráfagas**: transmite 40 kbit durante 100 ms, y después **90 000 ms** (90 segundos) en silencio.
- **Aplicación B**: transmite de forma **continua** los 1 024 kbit que le quedan.
- El problema dura **15 minutos** = **900 segundos**.

a) ¿Cuánto dato transmite A en los 15 minutos? ¿Y B? **b)** ¿Cuál es la utilización media del enlace si A va por un **circuito reservado** de 1 Mbps? ¿Y si van por **paquetes**?

Resolución.

a) El volumen de datos de cada una.

```
A: 15 ráfagas (una cada 60 segundos en 900 s)
   15 × 40 kbit = 600 kbit

B: 900 s × 1024 kbit/s = 921 600 kbit
```

b) Con circuito reservado.

```
┌──────────────────────────────────────────────────────────────┐
│ CIRCUITO: A recibe 1000 kbit/s reservados para 900 s         │
│                                                              │
│ Datos entregados por el circuito de A: 1000 × 900 = 900 000 kbit  │
│ Datos REALES que A tenía:                        600 kbit    │
│                                                              │
│ ⚠️ SE PIERDEN (o se ocupan sin usar) 899 400 kbit = 99,93 %  │
│                                                              │
│ Link utilization por circuito = 900 000 / (1000 × 900) = 100 %    │
│ A (utilización efectiva de A) =   600 / 900 000 = 0,067 %  │
└──────────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────┐
│ PAQUETES: A usa el enlace solo en sus ráfagas                │
│                                                              │
│ A transmite 600 kbit en 15 × 100 ms = 1,5 s de uso real      │
│ B transmite 921 600 kbit en 900 s                            │
│                                                              │
│ Link utilization = (600 + 921 600) / (1000 × 900)            │
│                  = 922 200 / 900 000 = 102,5 %   ⚠️ ¡NO LLEGA!  │
└──────────────────────────────────────────────────────────────┘
```

Aquí hay que hacer el ajuste honesto. Si los datos que B quiere mandar son 921 600 kbit en 900 s, el enlace de 1 Mbps no alcanza. La solución es que **el enlace real sea de 1,024 Mbps** o que B sea más lento. Supongamos que el enlace es de 1 024 kbps y que el reparto es proporcional:

```
ANÁLISIS CORRECTO con enlace de 1024 kbps y reparto por paquetes

Total deseado en 900 s:
   A:       600 kbit
   B:   921 600 kbit
   ──────────────
   total:  922 200 kbit

Capacidad del enlace en 900 s:
   1024 kbit/s × 900 s = 921 600 kbit

⚠️ El total pedido (922 200) EXCEDE la capacidad (921 600) en
   600 kbit, que es exactamente lo que A pide.
   ⇒ El enlace está saturado el 100 % del tiempo.

Con PAQUETES, el reparto es el que resulte de la competencia:
   cada uno manda lo que puede, y en los valles de uno
   aprovecha el otro.

Link utilization = 100 % en ambos casos (el enlace está lleno).

LA DIFERENCIA ESTÁ EN QUÉ SE PIERDE:

┌──────────────────────────────────────────────────────────────────┐
│ CON CIRCUITO                                                     │
│   1000 kbit/s reservados para A                                  │
│   A los usa: 0,6 s de 900 s (0,067 %)                            │
│   Lo que sobra: 999,4 s × 1000 kbit/s = 899 400 kbit DESPERDICIA │
│   → Para que B reciba su 921 600 kbit habría que agrandar el     │
│     enlace a 1024 + 1000 = 2024 kbps                             │
│                                                                  │
│ CON PAQUETES                                                     │
│   Ningún ancho de banda reservado                                │
│   En los 1,5 s de ráfagas de A, el enlace va al 100 %            │
│   En los 898,5 s restantes, el enlace lo usa B                   │
│   → Con 1024 kbps alcanza, sin gastar nada                       │
└──────────────────────────────────────────────────────────────────┘
```

El resultado final, que es el que hay que decir:

	Circuito reservado	Paquetes (mejor esfuerzo)
Capacidad necesaria	**2 024 kbps**	**1 024 kbps**
Desperdicio	**899 400 kbit** (99,93 % del circuito)	**0**
Retardo de A durante su ráfaga	**Constante y bajo** (tiene el enlace)	**Competitivo** (puede esperar)
Retardo de B	**Competitivo** (A no le ocupa el ancho, pero sí ocupa el nodo)	**Competitivo**
Retardo de A en su silencio	N/A	N/A

Interpretación — y esto es lo que hace que Internet use paquetes:

- **El circuito da calidad** (A siempre tiene lo suyo) **pero se paga caro**: hace falta el doble de capacidad para el mismo servicio, y el 99,9 % de esa capacidad se paga sin usar.
- **Los paquetes dan eficiencia** (el mismo enlace sirve para las dos aplicaciones al 100 %) **pero no dan calidad**: A compite con B cuando dispara su ráfaga.

- **El compromiso real** es que el enlace tiene que estar **sobrecargado a propósito**: hay que dimensionarlo para el 100 % del tráfico, lo cual significa **más retardo en la cola**. Por eso la utilización de un enlace debe ser ~90 % y no 100 %, aunque eso signifique capacidad ociosa.

2.3 Ejercicio 3 — ¿Por qué los paquetes de tamaño variable no se usan?

Enunciado. Se propone una variante de la conmutación por paquetes donde **cada paquete puede tener un tamaño cualquiera, entre 1 byte y 1500 bytes**, decided por la aplicación. **Justificá por qué esa variante es mala**, con argumentos concretos.

Resolución — los dos argumentos que hay que dar.

Argumento 1 — Injusticia (*unfairness*) y retardo extremo.

```
Enlace de 100 Mbps. Llegan paquetes de tamaño variable.

Llega un paquete de 1500 bytes primero. Tarda:
    1500/100 Mbps = 15 µs en ser transmitido

Justo después llegan 10 paquetes de 100 bytes. El primero tiene que
esperar los 15 µs del grande; el segundo, 30 µs; el último, 150 µs.

┌──────────────────────────────────────────────────────────────┐
│ t (µs)   0      15      30      45      60  ...  150         │
│          |       |       |       |       |       |          │
│ enlace   [  1500 B  ][100][100][100][100][100]...           │
│                                                              │
│ RTT del paquete grande (en el sentido de la cola): 15 µs     │
│ RTT del último paquete chico:               150 µs           │
│                                                              │
│ ⚠️ 10 veces peor. Y el tamaño del paquete grande es un factor  │
│    que el operador NO controla: lo elige la aplicación.      │
└──────────────────────────────────────────────────────────────┘
```

Consecuencias:

- **El retardo de un paquete depende del comportamiento de otros**, no del suyo. Un paquete de 100 bytes puede tener un retardo 1000 veces mayor que otro de 100 bytes si justo delante hay uno de 1500 bytes.
- **No hay garantía de retardo.** Y en un sistema en tiempo real, **no tener garantía es lo mismo que no servir**.
- **El culpable no es la red**: es la aplicación que mandó el paquete grande. La red solo obedece. Pero la red es la que carga con la culpa.

Argumento 2 — El buffer tiene que ser adaptativo, y eso rompe el store-and-forward acotado.

```
Con paquetes de tamaño FIJO L, el router puede saber EXACTAMENTE
cuántos packets caben:

    capacidad = ⌊B / L⌋     (determinista, conocido, acotado)

Con paquetes de tamaño VARIABLE (1 a 1500 bytes), el router no
puede saberlo:

    Un paquete de 1500 bytes ocupa 1500.
    Un paquete de 1 byte ocupa 1.
    La MISMA cola de B = 20 000 bytes contiene:
        13 paquetes de 1500 bytes       →  13 paquetes
     o 20 000 paquetes de 1 byte        →  20 000 paquetes

    ⚠️ El retardo de cola varía en un factor de 1 500 según
       qué paquetes hay en la cola. Y el router no puede predecirlo.
```

Los argumentos secundarios:

3. **Overhead ilegible.** No podés calcular la eficiencia sin saber el tamaño. Con `L` variable, la eficiencia `L/(L+58)` **varía por paquete**, y el promedio depende de la distribución de tamaños, que depende de las aplicaciones.
4. **Predicción imposible para el sistema de transporte.** Un AQM (*Active Queue Management*) como **CoDel** mide cuánto lleva en la cola un paquete. Con tamaños variables, la medida es injusta.
5. **Multiplexación y prioridad.** Para priorizar por flujo (DSCP en la cabecera IP), el router debería tratar igual un paquete de 1 byte que uno de 1500 bytes **en bytes**, no en cantidad. Con tamaño variable hay que definir la unidad de medida, y cualquier elección genera repartos injustos.

Conclusión para el examen:

```
El tamaño FIJO del paquete no es un detalle de implementación:
es lo que hace POSIBLES tres cosas:
  1. el store-and-forward con buffer FINITO y conocido
  2. una GARANTÍA (o al menos un cálculo) del retardo de cola
  3. la EQUIDAD entre paquetes
Por eso el MTU existe, y por eso existe el MSS.
```

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2007-12, Pregunta 3 (10 puntos) — las tres técnicas con análisis comparativo

> **Fuente:** Examen `eirc0712` / `sirc0712` — 27 de diciembre de 2007, Pregunta 3 (10 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0712.pdf

Enunciado (texto del examen)

> Explique en que consiste la conmutación de circuitos, conmutación de paquetes y conmutación de mensajes. Realice un análisis comparativo.

Sobre la solución oficial

> ⚠️ **La solución oficial de esta pregunta NO está publicada.** En `sirc0712.txt` el enunciado de la Pregunta 3 aparece (líneas 29-31) y después el documento pasa directamente a "Problemas Prácticos" y a la solución del Problema 1. **No hay respuesta de la cátedra para esta pregunta.** Todo lo que sigue es una **solución propia**, desarrollada con el esquema de Tanenbaum y con la aritmética del §1.4. Lo marco como tal.

Resolución *(solución propia, para 10 puntos)*

a) Conmutación de circuitos.

```
Se ESTABLECE un circuito entre los dos extremos antes de
transferir datos, y en cada nodo del camino se RESERVAN recursos
(un ancho de banda fijo, normalmente un *time slot*).

La fase de establecimiento ocurre FUERA DE BANDA: hay un canal de
control separado que lleva las señales de ocupado/libre. Es un
diálogo entre los nodos que reservan y liberan.

Una vez establecido:
   · los datos viajan SIEMPRE por el mismo camino
   · el retardo es CONSTANTE (no hay cola, porque el ancho de
     banda está reservado)
   · si un nodo del camino falla, se rompe la comunicación: no hay
     ruta alternativa
   · los recursos quedan ociosos durante las pausas del tráfico,
     y se pagan igual

Ejemplo canónico: la red telefónica clásica. Llamar establish un
camino de 64 kbps reservado a través de todos los nodos
intermedios.
```

b) Conmutación de paquetes.

```
El mensaje se DIVIDE en paquetes de tamaño FIJO o acotado. Cada
paquete se reenvía de manera INDEPENDIENTE, con store-and-forward,
y la decisión de por dónde va se toma PARA CADA PAQUETE, mirando la
dirección de destino.

Consecuencias:
   · NO hay establecimiento: el primer paquete sale ya
   · NO se reserva ancho de banda: es "mejor esfuerzo" (best effort)
   · el retardo es VARIABLE, y depende de la cola de cada router
   · dos paquetes del mismo mensaje pueden ir por caminos distintos
     → pueden llegar desordenados o perderse
   · los routers son STATELESS: no guardan información del flujo

Es lo que usa Internet.
```

c) Conmutación por mensajes.

```
El mensaje COMPLETO, de tamaño arbitrario y potencialmente enorme,
se reenvía de nodo en nodo. Cada nodo lo recibe ENTERO, lo guarda
en un buffer, y ESPERA a poder transmitirlo completo.

Es store-and-forward SIN FRAGMENTAR, sin cota superior de tamaño.

Consecuencias:
   · NO hay establecimiento
   · el retardo es muy grande: el mensaje entero tiene que
     atravesar cada enlace antes de que el siguiente nodo pueda
     empezar a transmitir
   · BLOQUEO DE CABEZA: un mensaje grande bloquea a todos los
     pequeños que están detrás en el buffer
   · el buffer tiene que poder contener el mensaje más grande
     que se pueda recibir, lo que es inviable en un router

Es el antecesor histórico de la conmutación por paquetes: si se
acota el tamaño del mensaje, se convierte en conmutación por
paquetes.
```

d) El análisis comparativo. La tabla del §1.6 completa, reproducida:

Característica	Circuitos	Paquetes	Mensajes
¿Hay conexión?	Sí, se establece	No (datagramas)	No
Unidad de datos	Flujo continuo	Paquete de tamaño acotado	Mensaje completo
¿Se reservan recursos?	**Sí**	No	No
Retardo	**Constante**	Variable	Variable y muy grande
¿Puede cambiar el camino?	No	Sí, por paquete	Sí, por mensaje
¿Llegan en orden?	Siempre	Puede que no	Puede que no
Establecimiento	100 ms a 2 s	Ninguno	Ninguno
Tamaño del buffer	No hace falta	Acotado y conocido	Debe ser enorme
¿Escala?	No	**Sí**	No
Uso real	Teléfono, MPLS (LSP)	**Internet**	Prácticamente ninguno

La conclusión del análisis comparativo, que es lo que la pregunta pide cerrar:

```
Las tres sirven para COMPARTIR un medio. Lo que cambia es
QUÉ se comparte:

· Circuitos:  el ANCHO DE BANDA.   Reparto fijo y estático.
· Paquetes:   el TIEMPO.           Reparto dinámico por oportunidad.
· Mensajes:   el TIEMPO, a bloque. Reparto dinámico pero bloqueado.

El circuito da CALIDAD a costa de EFICIENCIA.
Los paquetes dan EFICIENCIA a costa de CALIDAD.
Los mensajes no dan ninguna de las dos.

La red eligió paquetes porque Internet tiene una mezcla de tráfico
(ráfagas de archivos, ráfagas de voz, video constante) donde la
eficiencia importa más que la calidad, y porque la calidad se
puede COMPRAR después en un DiffServ (Clase 45), mientras que la
eficiencia no se puede comprar.
```

3.2 ⭐ Examen 2009-02, Pregunta 4 (9 puntos) — datagramas vs. circuitos virtuales

> **Fuente:** Examen `sirc0902` — 2 de febrero de 2009, Pregunta 4 (9 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0902.pdf

Enunciado (texto del examen)

> **a)** Describa la conmutación de paquetes basada en: **i.** Datagramas **ii.** Circuitos virtuales.
>
> **b)** Explique ventajas y desventajas de cada opción desde el punto de vista de la **asignación de recursos en la red**.

Solución oficial

> **c) Describa la conmutación de paquetes basada en: i.** Datagramas **ii.** Circuitos virtuales. *(Sin desarrollo: el PDF deja el apartado en blanco.)*
>
> **d) Explique ventajas y desventajas de cada opción desde el punto de vista de la asignación de recursos en la red.**
>
> Básicamente las redes de datagramas son "de mejor esfuerzo", es decir que consumen los recursos disponibles, y en caso de congestión degradan el servicio por igual a todos los usuarios; aprovechan bien el multiplexado estadístico del tráfico en la red. Las redes de VC ofrecen garantía de QoS basadas en la reserva de recursos, y por lo tanto protegen a los usuarios de las situaciones de congestión, a costo de disponer de más recursos o rechazar conexiones (control de admisión).
>
> ⚠️ **Dos errores de la solución oficial que hay que notar:** (a) los apartados de la solución están rotulados **c)** y **d)** cuando el enunciado dice **a)** y **b)**: es un error de maquetación de la cátedra. (b) El apartado **a)** —la descripción de las dos técnicas— **quedó sin desarrollar**. Lo desarrollo abajo.

Análisis de la solución oficial

Lo que está muy bien:

1. **El resumen del apartado b) es una de las mejores frases de todo el material oficial**: "*aprovechan bien el multiplexado estadístico del tráfico en la red*". Es exactamente el argumento de por qué Internet es de datagramas, en una frase.
2. **Nombra los dos costos de los circuitos virtuales**: o **disponés de más recursos**, o **rechazás conexiones**. El segundo es el **control de admisión** (*admission control*), que es el término técnico correcto y que casi ningún alumno conoce. Lo nombró.
3. **Distingue degradación uniforme vs. garantía**: en datagramas, la congestión "degrada el servicio por igual a todos los usuarios"; en VC, "protege a los usuarios de las situaciones de congestión". Esa es la diferencia observable.

Lo que falta:

1. **El apartado a) entero.** Ni datagramas ni circuitos virtuales están descritos. Hay que hacerlo.
2. **No dice qué hace el router en cada caso.** En datagramas, el router busca por la IP destino; en VC, por la etiqueta. Esa es la diferencia mecánica.
3. **No menciona el estado.** Un router de datagramas es **stateless**; uno de VC mantiene una **tabla de VC** (estado). Ese estado es lo que permite la QoS y lo que hace que no escale.
4. **No menciona la etiqueta.** El VC se identifica por una etiqueta en la cabecera, no por la IP de destino.

Parte a) — las dos técnicas *(desarrollo propio, la oficial lo dejó vacío)*

i. Datagramas

```
Cada paquete es INDEPENDIENTE y lleva la DIRECCIÓN DE DESTINO
COMPLETA (la IP) en su cabecera. No hay conexión, no hay estado, no
hay negociación previa.

┌──────────────────────────────────────────────────────────────┐
│  Paquete 1:  destino = 203.0.113.9, TTL = 64                 │
│  Paquete 2:  destino = 203.0.113.9, TTL = 63                 │
│  Paquete 3:  destino = 203.0.113.9, TTL = 64                 │
│                    ↑ 3 paquetes, 3 cabeceras completas       │
└──────────────────────────────────────────────────────────────┘

El ROUTER hace, por cada paquete:
   1. lee la IP destino
   2. longest prefix match en su tabla de reenvío
   3. reenvía por la interfaz que le corresponde

· No guarda NADA entre paquetes. Es STATELESS.
· Dos paquetes con el mismo destino pueden tomar caminos distintos.
· Si cambia la topología, el siguiente paquete va por el camino nuevo.
· Los paquetes pueden perderse, duplicarse o llegar desordenados.
· ES EL MODELO DE INTERNET (IP, RFC 791; y RFC 8200 para IPv6).
```

ii. Circuitos virtuales

```
Se ESTABLECE un camino virtual entre los extremos antes de mandar
datos. Cada paquete lleva una ETIQUETA que identifica el camino, y
los routers reenvían MIRANDO LA ETIQUETA, sin consultar la IP de
destino.

┌──────────────────────────────────────────────────────────────┐
│  Paquete 1:  etiqueta = 7, VC = (H1 → H2)                    │
│  Paquete 2:  etiqueta = 7, VC = (H1 → H2)                    │
│  Paquete 3:  etiqueta = 7, VC = (H1 → H2)                    │
│                    ↑ 3 paquetes, la MISMA etiqueta           │
└──────────────────────────────────────────────────────────────┘

El ROUTER hace:
   1. lee la ETIQUETA de la cabecera MPLS
   2. busca en su TABLA DE VCS (que es un array, no un longest
      prefix match)
   3. cambia la etiqueta por la de salida y reenvía

· El router mantiene ESTADO: la tabla de VCs y los contadores.
· Todos los paquetes del mismo VC van por el MISMO camino.
· El orden de llegada se preserva (mismo camino, mismo buffer).
· Los VC se pueden AGRUPAR en un mismo enlace físico.
· Implementaciones: ATM (AAL5), Frame Relay, MPLS (etiqueta de
  20 bits, LSP unidireccional), y VXLAN en el datacenter.
```

La comparación mecánica, que es la que faltaba:

	Datagramas	**Circuitos virtuales**
Qué mira el router	La **IP destino**	La **etiqueta** de VC
Cómo busca	**Longest prefix match** en la tabla de ruteo	**Índice directo** en una tabla de VCs
Coste de la decisión	Acceso a una TCAM (lento)	Un índice (rápido)
Estado en el router	**Ninguno**	**Por VC**: contadores, cola, QoS
Camino por paquete	Puede cambiar	**Fijo**
Sobrevive a un cambio de topología	**Sí**: el siguiente paquete va por donde sea	**No**: hay que **reestablecer** el VC
Cuentas	Sobrecarga de la cabecera IP	Sobrecarga de la etiqueta MPLS (4 bytes)

La observación clave sobre la velocidad: el VC es **más rápido de conmutar** porque la decisión es un índice en un array, no una búsqueda de prefijo. **Ese es el motivo técnico de MPLS**: en un backbone a 100 Gbps, la diferencia entre un índice y una TCAM es la diferencia entre 40 ns y 3000 ns por paquete. Pero ese ganho viene **precisamente del estado**, y el estado es lo que hace que no escale.

Parte b) — ventajas y desventajas desde la asignación de recursos *(desarrollo de la solución oficial)*

La oficial en dos frases. La desarrollo con los dos casos y las dos preguntas que hay que responder.

Datagramas — desde el punto de vista de la asignación de recursos:

```
ASIGNACIÓN DE RECURSOS: NINGUNA, IMPLÍCITA, POR OPORTUNIDAD.

· Es "mejor esfuerzo" (best effort): cada paquete usa los recursos
  que encuentra libres en ese instante, sin preguntar y sin
  reservar.
· MULTIPLEXADO ESTADÍSTICO: los picos de un flujo llenan los
  valles de otro. Es la razón por la que un único enlace de
  100 Mbps sirve para 1000 usuarios que cada uno usan 200 kbps el
  10 % del tiempo.

  límite  ──────────────────────────────────────────────────►
  1000 flujos × 200 kbps × 10 % = 200 kbps de MEDIA
  enlace de 100 Mbps = 100 000 kbps
  factor de multiplexado = 100 000 / 200 = 500×

· En caso de congestión, TODOS los usuarios se ven afectados
  IGUALMENTE. No hay manera de proteger a nadie.

VENTAJAS
   · Eficiencia máximo del enlace
   · No necesita estado en los routers → escala a millones de flujos
   · Sobrevive a los cambios de topología
   · No hay establecimiento ni control de admisión

DESVENTAJAS
   · Sin garantía de retardo ni de ancho de banda
   · La congestión degrada a todos por igual
   · Para dar QoS hay que marcar los paquetes (DiffServ) y eso
     obliga a tener ESTADO por clase, que es menos que por flujo
     pero más que cero
```

Circuitos virtuales — desde el punto de vista de la asignación de recursos:

```
ASIGNACIÓN DE RECURSOS: EXPLÍCITA, POR RESERVA, CON CONTROL
DE ADMISIÓN.

· Al establecer el VC se RESERVAN recursos en cada salto:
  ancho de banda y/o buffer.
· CONTROL DE ADMISIÓN (*admission control*): si no hay recursos
  suficientes, NO SE ACEPTA el VC. Se rechaza la conexión. Esto es
  fundamental: es lo que hace que las garantías sean reales.

  enlaces disponibles: 3 × 64 kbps
  VC pedido 1:  64 kbps  → ok, quedan 2 × 64
  VC pedido 2:  64 kbps  → ok, quedan 1 × 64
  VC pedido 3:  64 kbps  → ok, quedan 0
  VC pedido 4:  64 kbps  → RECHAZADO (control de admisión)

· AISLAMIENTO: la congestión de un VC no afecta a los demás, porque
  los recursos están reservados y separados.

VENTAJAS
   · GARANTÍA de retardo y de ancho de banda
   · AISLAMIENTO entre flujos: uno no puede arruinar al otro
   · Preserva el ORDEN (mismo camino)
   · El flujo es predecible: no hay picos de retardo

DESVENTAJAS
   · COSTE: hay que tener más recursos disponibles de los que se
     usan, o aceptar conexiones rechazadas
   · ESTADO por VC en cada router: no escala a millones de flujos
     (es el motivo por el que IntServ, RFC 2211, no se despliega)
   · Si cae un nodo, se rompe el VC y hay que reestablecerlo
   · La reserva ociosa: se paga y no se usa
```

La frase de síntesis, que es la conclusión de la pregunta:

```
┌──────────────────────────────────────────────────────────────┐
│  Datagramas: asignación IMPLÍCITA, por oportunidad.          │
│  → Eficiencia máxima, cero garantías, cero estado.           │
│                                                              │
│  Circuitos virtuales: asignación EXPLÍCITA, por reserva.     │
│  → Garantías reales, pero con coste en recursos y en estado. │
│                                                              │
│  Y la clave histórica: IntServ (reserva por flujo) NO escaló.│
│  DiffServ (marcar por clase, sin reserva) escaló. Porque en  │
│  Internet la cantidad de flujos es tan grande que el estado  │
│  por flujo es insostenible, mientras que el estado por clase │
│  (un número pequeño de colas) es viable.                     │
└──────────────────────────────────────────────────────────────┘
```

3.3 Referencia — Examen 2009-07 P2 (10 puntos), sobre el mismo terreno

> **Enunciado:** *"Un problema de enrutamiento donde la métrica es la distancia en número de saltos (hops): ¿es la mejor métrica posible? Justifique."*

Por qué lo pongo acá, aunque sea de otra clase: el enunciado es sobre enrutamiento (Clase 33), pero **el fondo es la conmutación**. La respuesta oficial dice, textual:

> *"Si se busca una utilización eficiente de los recursos y/o satisfacer requerimientos de calidad de servicio, la métrica de hops puede ser inconveniente, ya que señalará como 'mejor' el camino más corto, que no necesariamente será el que tenga mejores atributos de calidad (retardo extremo a extremo, pérdidas, throughput) y/o menos congestión."*

La conexión con esta clase: el **número de hops es la métrica que corresponde a los circuitos virtuales**, porque mide el camino fijo, no la calidad. En **datagramas** uno podría medir la calidad real (retardo, ancho de banda) y elegir por eso; en **circuito virtual** el camino ya está fijado y solo queda medirlo para calcular el retardo. Y la objeción de la oficial es que **el camino más corto en hops no es el más corto en retardo**: 3 saltos por un enlace satelital de 600 ms son peores que 8 saltos por fibra de 2 ms cada uno.

PARTE 4 — CIERRE

📌 Resumen

1. **El criterio único** que distingue a las tres: **¿cuándo se decide el camino** (antes / por cada unidad de datos).
2. **Circuitos**: se establece un camino y **se reservan recursos**. Retardo **constante**, orden garantizado, si cae un nodo **se rompe**, y **los recursos ociosos se pagan**. La red telefónica clásica es el ejemplo.
3. **Mensajes**: el mensaje **completo** se reenvía entero, con store-and-forward. Retardo enorme, **bloqueo de cabeza**, buffers que tienen que ser enormes. Es el **antecedor** de los paquetes.
4. **Paquetes**: el mensaje se **divide en paquetes acotados** y cada uno se reenvía por separado. **Es lo que usa Internet.** Compromiso entre mensajes y circuitos: **mensaje de tamaño de mensaje pero store-and-forward de paquete pequeño**.
5. **Store-and-forward** es obligatorio porque el router tiene **buffers finitos** y los enlaces de entrada y salida tienen **velocidades distintas**. Si reenvía bit a bit, necesita memoria infinita.
6. **El pipeline** es lo que gana los paquetes: con `N` paquetes y `n` enlaces, el último tarda `(N + n − 1)·L/R`. Los mensajes tardan `N·n·L/R`; el circuito, `N·L/R` pero con `100 ms a 2 s` de establecimiento.
7. **El router con buffers llenándose**: el retardo de cola del paquete en posición `k` es `k·L/R`, y si el buffer se llena **se descarta** (pérdida). El buffer es una cola, y toda cola tiene un límite.
8. **Comparación clave**: circuitos = calidad a costa de eficiencia; paquetes = eficiencia a costa de calidad; mensajes = ni una ni otra.
9. **Datagramas vs. circuitos virtuales**: el router mira **la IP destino** (longest prefix match, stateless) o **la etiqueta** (índice, stateful). El VC es **más rápido** pero **no escala** por el estado.
10. **Por qué Internet usa paquetes**: sin establecimiento, sin reserva ociosa, se puede reenrutar, y el núcleo es **stateless**. **IntServ no escaló**; DiffServ ganó.

✅ Autoevaluación

1. ¿Por qué un router no puede reenviar los datos bit a bit, como el *cut-through forwarding* de los switches rápidos?

▶ Respuesta

2. Un mensaje de 30 000 bits va por 3 enlaces de 2000 bit/s. ¿Cuánto tarda con conmutación de mensajes, con paquetes de 5000 bits, y con circuito?

▶ Respuesta

3. ¿Qué es "multiplexado estadístico" y por qué es la ventaja de los datagramas?

▶ Respuesta

4. ¿Por qué un circuito virtual es más rápido de conmutar que un datagrama, si los circuits virtuales requieren más estado?

▶ **Respuesta**

5. Explique por qué el tamaño del buffer de un router no protege contra la pérdida.

▶ **Respuesta**

6. ¿Qué es control de admisión y por qué es indispensable en los circuitos virtuales?

▶ **Respuesta**

7. Un mensaje de 8000 bits se divide en paquetes de 1000 bits y atraviesa 5 enlaces de 500 bit/s. ¿Cuánto tarda el último paquete?

▶ **Respuesta**

8. ¿Por qué la conmutación por mensajes "no se usa"? Y sin embargo existen los *mail relay*.

▶ **Respuesta**

🔗 Referencias

Exámenes - Examen `sirc0712` (27-dic-2007) — P3 (10 pts): las tres técnicas con análisis comparativo — **la cátedra NO publicó la solución oficial** · P2 (10 pts): leaky bucket y token bucket - Examen `sirc0902` (2-feb-2009) — P4 (9 pts): datagramas vs. circuitos virtuales, y la asignación de recursos · Problema 1 (30 pts): análisis de enlaces paralelos en link-state - Examen `sirc1103` (21-feb-2011) — P5 (10 pts): retardo con store-and-forward y con reenvío bit a bit → **Clase 6** - Examen `sirc0907` (2-jul-2009) — P2 (10 pts): distancia en hops como métrica, ¿es la mejor? - Examen `sirc0608` (5-ago-2006) — P3 (10 pts): distance-vector, ventajas y desventajas · Problema 1b: Dijkstra oscilando entre enlaces paralelos

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., **cap. 1 §1.3** "La red de conmutación de paquetes" (pág. ~10 a ~20): las tres técnicas con el diagrama de los 4 nodos - Tanenbaum & Wetherall, **cap. 1 §1.2** "Tipos de red" — la clasificación PAN/LAN/WAN - Tanenbaum & Wetherall, **cap. 1 §1.6** — el modelo TCP/IP y por qué Internet usa datagramas - Kurose & Ross, *Computer Networking*, cap. 1 §1.4 "What's Inside a Router" — **store-and-forward** con los números y el caso del "4 enlaces a 10 Mbps y uno a 11 Mbps" - Kurose & Ross, cap. 1 §1.2 "The Network Edge" — conmutación de circuitos frente a conmutación de paquetes, con los números de los *timeslots*

RFCs - `RFC 791` — *Internet Protocol* (1981): el modelo de datagramas, sin conexión y sin estado - `RFC 8200` — *Internet Protocol, Version 6* (2017, *Internet Standard*): el mismo modelo de datagramas, 37 años después - `RFC 2737` — *Guidelines for Management of IP-Based Intranets* (1999): las restricciones de *ingress filtering* que acotan el reenvío y el flooding - `RFC 3031` — *Multiprotocol Label Switching Architecture* (2001): el circuito virtual moderno (`RFC 3032` pila de etiquetas, `RFC 4364` MPLS VPN) - `RFC 8402` — *Segment Routing Architecture* (2018), `RFC 8660` (SR-MPLS), `RFC 8986` **(SRv6, 2022)**: el sucesor que codifica el circuito virtual en la cabecera IPv6 - `RFC 7348` — *VXLAN* (2014): el circuito virtual **de capa 2** que reemplazó a MPLS en el datacenter, encapsulado sobre UDP/4789 - `RFC 2211` — *Integrated Services* (1997): el modelo de reserva por flujo que **no escaló** · `RFC 2474` / `RFC 2475` (DiffServ): el modelo por clases que **sí escaló** - `RFC 9438` — *CUBIC* (2023, *Standards Track*): el control de congestión por AIMD que asume conmutación de paquetes

Clase siguiente: 5 — El retardo en las redes de paquetes

Clase 5

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El retardo total: la ecuación
1.2 Los cuatro componentes, uno por uno
1.2.1 Retardo de procesamiento (`d_proc`)
1.2.2 Retardo de encolamiento (`d_cola`)
1.2.3 Retardo de transmisión (`d_transmisión`)
1.2.4 Retardo de propagación (`d_propagación`)
1.2.5 Las dos tablas comparativas
1.3 Ancho de banda vs. velocidad de propagación
1.4 El *bandwidth-delay product*: los bits que están en vuelo
1.5 RTT, latencia y jitter
1.6 Congestión, pérdidas y retardo
2.1 Ejercicio 1 — Los cuatro componentes de un paquete en dos enlaces
2.2 Ejercicio 2 — El BDP con la aritmética completa
2.3 Ejercicio 3 — RTT, latencia y la diferencia entre las dos
2.4 Ejercicio 4 — Dos paquetes de distinto tamaño en el mismo enlace
3.1 ⭐ Examen 2009-12, Pregunta 1 (10 puntos) — los componentes del retardo
3.2 ⭐ Examen 2011-03, Pregunta 2 (6 puntos) — congestión, pérdidas y retardo
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: A (Fundamentos) · **Semana:** 2 · **Duración estimada:** 110 min **Correlativa:** Clase 4 (conmutación: circuitos, paquetes y mensajes) · **Sigue con:** Clase 6 (taller de retardos, todo resuelto) **Material de referencia:** Tanenbaum cap. 1 §1.5 y §1.2 (pág. ~10 a ~30 del PDF de tu carpeta) · RFC 9293, RFC 5681, RFC 5437, RFC 9000

🎯 Al terminar esta clase vas a poder

1. **Escribir** el retardo total como **suma de cuatro componentes**, con la fórmula, la unidad y de qué depende cada uno.
2. **Distinguir** `d_transmisión = L/R` de `d_propagación = d/s`, y explicar por qué el primero **no depende del medio** y el segundo **no depende del paquete**.
3. **Decir** cuál es el **único componente variable** y **el único que refleja la congestión**, y explicar la cadena congestión → retardo de cola → retardo total.
4. **Calcular** el retardo de un paquete en un enlace y en un camino de varios enlaces, con la aritmética escrita paso a paso.
5. **Calcular** el *bandwidth-delay product* (`BDP = R · RTT`) y explicar por qué una conexión de larga distancia necesita una ventana grande.
6. **Distinguir** latencia de RTT, y **calcular** el jitter de un flujo.
7. **Definir** congestión, pérdida y retardo, y explicar el vínculo entre las tres, como lo pide el Examen 2011-03.
8. **Resolver entero** el Examen `2009-12 P1` (10 pts) con la solución oficial a la vista.

> **Prerequisito del examen:** `2009-12 P1` (10 pts — componentes del retardo, ancho de banda vs. velocidad de propagación, congestión), `2011-03 P2` (6 pts — congestión, pérdidas y retardo), `2011-03 P5` (10 pts — retardo end-to-end → **Clase 6**), `2010-02 P5` (jitter vs. retardo), `2009-03 P5` (full/half duplex).

PARTE 1 — TEORÍA

1.1 El retardo total: la ecuación

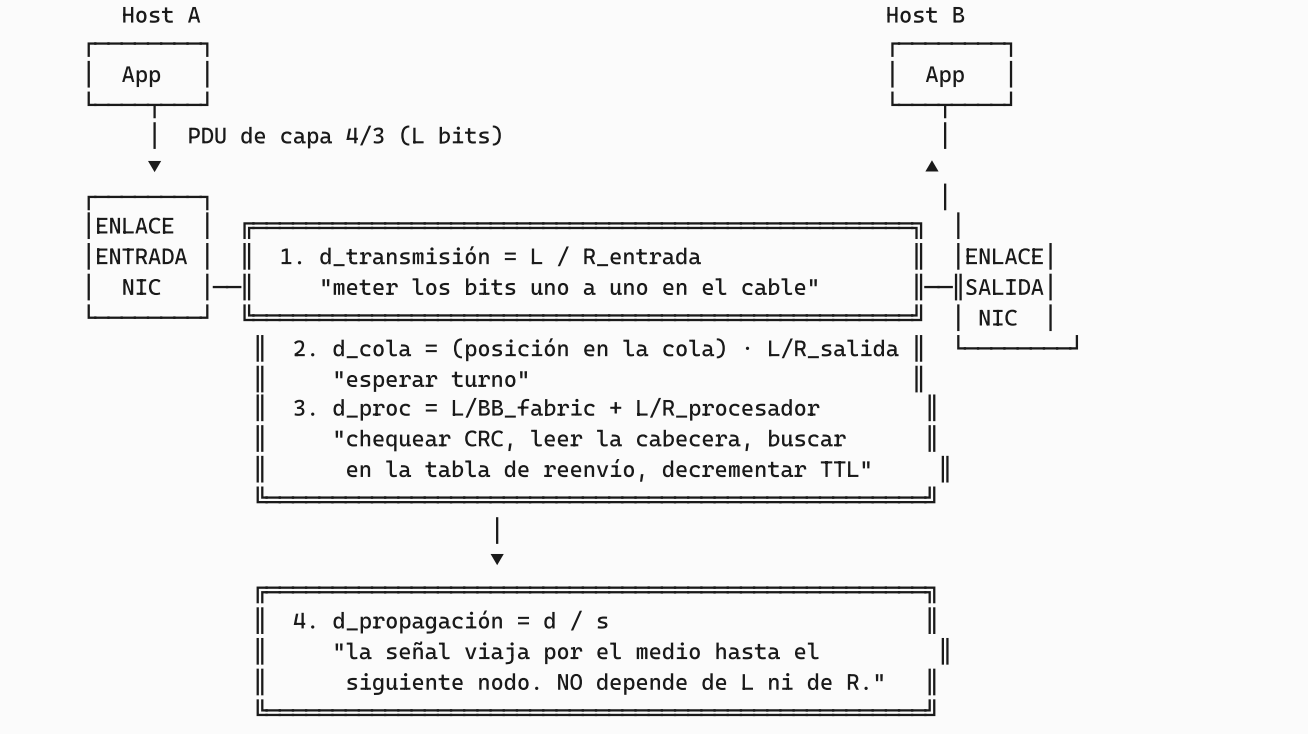
Todo lo que aprendiste hasta ahora se resume en **una sola suma**. Cuando un paquete viaja de A a B atravesando `n` enlaces y `k` routers:

```
╔═══════════════════════════════════════════════════════════╗
║                                                           ║
║   d_end2end  =   Σ  ( d_proc(i)  +  d_cola(i)  +          ║
║                     d_transmisión(i)  +  d_propagación(i) )  ║
║                                                           ║
║                 i = 1 ... n                               ║
║                                                           ║
╚═══════════════════════════════════════════════════════════╝
```

Y **el detalle que se pregunta siempre**: hay `n` **términos de propagación y** `n` **de transmisión** (uno por enlace), pero hay `k` **términos de procesamiento** (uno por router, y un router **no es** un enlace).

```
n enlaces ──────►  n · d_transmisión  +  n · d_propagación
k routers  ──────►  k · d_proc         +  k · d_cola
(con k = n − 1 en una línea sin ramales)

⚠️ EL ERROR TÍPICO ES PONER n · d_proc EN VEZ DE k · d_proc.
   El Examen 2011-03 P5 lo pide explícitamente y es la
   pregunta más repetida del curso.
```

El diagrama de un solo router, que es donde se ve cada componente:



1.2 Los cuatro componentes, uno por uno

1.2.1 Retardo de procesamiento (`d_proc`)

Qué es. El tiempo que tarda el nodo en **hacer el procesamiento** del paquete entre que llega por una interfaz y sale por otra: leer la cabecera, verificar el CRC, decrementar el TTL, recalcular el checksum, consultar la tabla de reenvío (*longest prefix match*), y en un switch, aprender la dirección MAC origen.

Fórmula (en un router real). Un modelo desagregado en hardware separa el ancho de banda del **fabric**:

```
d_proc = d_cola_ingreso  +  L / B_fabric  +  d_bus  +  L / R_procesador
           └ en la interfaz ┘   └ el "backplane" ┘  └ el bus de la CPU┘

Con un fabric de 40 Gbps, un bus de memoria de 33 MHz y un procesador
de 1 GHz:

   L = 1500 bits
   L / B_fabric      = 1500 / 40·10⁹ = 37,5 ns
   L / R_procesador  = 1500 / 10⁹    = 1 500 ns = 1,5 µs
   bus + otros                ≈       100 ns

   d_proc ≈ 1,64 µs
```

Interpretación — y por qué se lo considera despreciable.

```
Comparación en un enlace de 1 Gbps con un paquete de 1500 bytes
y 500 km de fibra:

   d_transmisión = 1500 bits / 10⁹ = 1,5 µs
   d_propagación = 500 000 m / 2·10⁸ = 2,5 ms = 2 500 µs
   d_proc                              ≈ 1,6 µs
   ──────────────────────────────────────────────────────
   PESO RELATIVO:
      d_proc        =  1,6 µs  →   0,06 %  del total
      d_transmisión =  1,5 µs  →   0,06 %  del total
      d_propagación =  2 500 µs →  99,9 %   del total
```

Tres razones por las que se lo desprecia:

1. **Es tres órdenes de magnitud menor** que la propagación en cualquier enlace de backbone.
2. **Es independiente del tráfico.** No depende de cuántos paquetes hay ni de la carga: el router procesa un paquete por vez en la CPU. Por eso **no se ve afectado por la congestión**.
3. **Los equipos modernos lo hacen en hardware.** Un *lookup* en TCAM es determinista y de ~40 ns. Los conmutadores de 400 Gbps procesan **billones de paquetes por segundo**: si el procesamiento fuera de 1 µs por paquete, un solo puerto necesitaría 10 Gbps solo de CPU.

Cuándo SÍ importa. En redes pequeñas, con equipos viejos, o cuando hay **procesamiento de software** encima:

- Un **firewall que inspecciona la carga útil** (un WAF, un IDS de contenido) puede tardar **decenas de microsegundos** por paquete.
- Un **router doméstico con NAT y registro de conexiones** puede tardar **1 a 5 µs**.

- Una **VPN con cifrado por software** puede tardar **5 a 20 µs** (y por eso las implementaciones usan cifrado por hardware: las instrucciones AES-NI y las claves de 10 Gbps).

En un problema de examen, salvo que el enunciado lo diga explícitamente, **se desprecia**. Pero **decirlo** suma: "desprecio `d_proc` porque es de microsegundos contra milisegundos de propagación, y porque no depende de la carga".

1.2.2 Retardo de encolamiento (`d_cola`)

Qué es. El tiempo que el paquete **espera en el buffer** antes de que le toque ser transmitido.

Fórmula. Si el paquete es el `k`-ésimo de una cola de paquetes de `L` bits cada uno:

```
d_cola = k · L / R_salida
```

Donde `R_salida` es la velocidad del enlace **de salida** (¡no la de entrada!).

El diagrama de la cola, que es donde se ve por qué es el único variable:

```
ENLACE DE SALIDA, 10 Mbps (un paquete de 1000 bits cada 0,8 ms)

tiempo ──────────────────────────────────────────────────────►
0        0,8      1,6      2,4      3,2      4,0       4,8 ms
|        |        |        |        |        |         |
▼        ▼        ▼        ▼        ▼        ▼         ▼
┌──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┐
| P1   | P2   | P3   | P4   | P5   | P6   | P7   | P8   |      |      |      |      | ← cola
└──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┘
                                                  ↑
                                          B = fin del buffer

d_cola(P1) = 0
d_cola(P2) = 0,8 ms
d_cola(P3) = 1,6 ms
...
d_cola(Pk) = (k−1) · 0,8 ms

⚠️ Si el paquete número 100 llega cuando hay 8 paquetes en el
   buffer, y la cabecera del buffer es 10 paquetes: el paquete 9
   y el 10 se DESCARTAN (tail drop).
```

Las cuatro propiedades de `d_cola`

Propiedad 1 — Es el ÚNICO componente variable. Los otros tres se pueden calcular con los datos del problema; `d_cola` no, porque depende del estado de la cola, que es información que el enunciado normalmente **no da**.

Componente	¿Se puede calcular con los datos del enunciado?	¿Depende de la carga?
`d_proc`	Sí, si te lo dan	**No**
`d_cola`	**No**	**Sí**
`d_transmisión`	Sí (`L` y `R`)	**No**
`d_propagación`	Sí (`d` y `s`)	**No**

Propiedad 2 — Es el ÚNICO que refleja la congestión. Esta es la respuesta literal de la **Pregunta 1b del Examen 2009-12** (que se resuelve entera en el §3.2).

```
MÁS TRÁFICO OFRECIDO
      │
      ├──► las colas se LLENAN más
      │          │
      │          ├──►  d_cola  AUMENTA  ──►  retardo total AUMENTA
      │          │
      │          └──►  si se supera la capacidad del buffer:
      │                      se DESCARTA  ──►  PÉRDIDAS
      │
      └──► se />}
```

Propiedad 3 — Puede ser CERO. Si el paquete llega y el enlace de salida está libre, `d_cola = 0`. Por eso el enunciado del Examen 2011-03 P5 dice explícitamente "*suponiendo que no se produce retardo de cola*": es una hipótesis, y es la que hace el problema resoluble.

Propiedad 4 — Es el único componente que NO tiene cota superior. `d_proc`, `d_trans` y `d_prop` están determinados por el hardware y por la física. `d_cola` puede valer 1 ms o 5 segundos, según lo saturado que esté el enlace. **Ese es el motivo por el que el retardo en Internet no tiene garantía: porque un componente del total es inacotado.**

La pérdida, que es el otro extremo de `d_cola`

Cuando la cola está llena y llega un paquete más, **se descarta** (*tail drop*). Y hay que entender por qué **agrandar el buffer no es la solución**:

```
Llega más tráfico del que sale:
   tasa de llegada  = 10 Gbps
   tasa de salida   = 100 Mbps

Se acumulan 9,9 Gbps = 1,2375 GB por segundo

Buffer = 4 MB   → se llena en 4·10⁶ / 1,2375·10⁹ = 0,0032 s
Buffer = 400 MB → se llena en 0,32 s
Buffer = 4 GB   → se llena en 3,2 s

⚠️ El buffer agrandado retrasa la muerte, no la evita.
   Y además INCREMENTA el retardo de los que sí pasan:
   un paquete que espera 3,2 s en un buffer de 4 GB tiene un
   retardo de 3,2 s. Eso es BUFFERBLOAT.
```

La solución real: los AQMs (*Active Queue Management*). En vez de esperar a llenar el buffer, el router **descarta (o marca) temprano**. **CoDel** (*Controlled Delay*) descarta cuando el retardo de un paquete supera un objetivo de 5 ms durante un intervalo; **FQ-CoDel** es el que usa Linux por defecto desde el kernel 3.5 (2012); **PIE** es otro. Y **ECN** (`RFC 3168`) marca en vez de descartar. Todo esto es de la Clase 21.

1.2.3 Retardo de transmisión (`d_transmisión`)

Qué es. El tiempo que tarda el emisor en **meter el paquete entero en el cable**. Es decir, desde que empieza a salir el primer bit hasta que sale el último.

```
──────────────────────────────────────────────────────────────────────
 ENLACE de 10 Mbps, paquete de 1500 bits:

 bit:  0   1   2   3   4   5   6  ...  1498  1499  ← 1500 bits
       |   |   |   |   |   |   |        |     |
 t(s): 0   1  2  3  4  5  6      149,8   150   × 10⁻⁷ s
                                          ↑
                                  el último bit sale a los 150 µs

 d_transmisión = 1500 bits / 10·10⁶ bit/s = 150 µs = 0,15 ms
──────────────────────────────────────────────────────────────────────
```

La fórmula:

```
┌──────────────────────────────────┐
│        L      (en BITS)          │
│ d_t = ─────                      │
│        R   (en bps)              │
└──────────────────────────────────┘
```

```
⚠️ L EN BITS. Si te dan el paquete en BYTES, multiplicá por 8.
   1500 bytes = 12 000 bits.  Si hacés 1500/10⁷ da 0,15 ms, que
   es 8 veces MENOR que la respuesta correcta: ese es el error
   número uno de los ejercicios de retardo.
```

Las dos propiedades que hay que decir siempre:

1. **No depende del medio.** El medio (cobre, fibra, aire) tiene la misma `R`. La `R` depende del **equipo**: del transceptor y de la modulación.
2. **No depende de la distancia.** `d_trans` es el mismo para un enlace de 1 m y uno de 1000 km, si el ancho de banda es el mismo. Es un retardo **local**, no de la señal.

Un caso límite que hay que ver. Con un paquete de **1 bit** y un enlace de 1 Gbps:

```
d_trans = 1 / 10⁹ = 1 ns
```

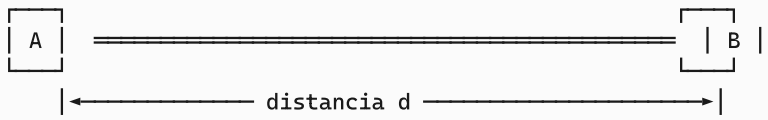
Retardo de transmisión despreciable frente a la propagación en redes largas:

```
5000 km de fibra, 10 Gbps, paquete de 1500 bytes:
   d_trans = 12 000 / 10¹⁰ = 1,2 µs
   d_prop  = 5·10⁶ / 2·10⁸ = 25 ms
   ──────────────────────────────────────
   d_prop / d_trans = 25 000 / 1,2 = 20 833 veces
```

Es el mismo cálculo del §1.2.1, y da la misma conclusión: **en redes de backbone, todo el retardo es propagación**.

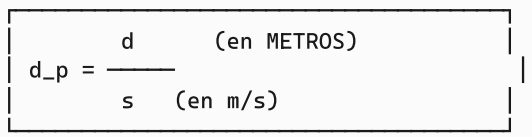
1.2.4 Retardo de propagación (`d_propagación`)

Qué es. El tiempo que tarda **la señal física** en recorrer el medio, desde el momento en que sale el último bit de la interfaz del emisor hasta que llega el primero a la interfaz del receptor.




```
El último bit sale de A en t = 0.
Ese bit viaja por el medio y llega a B en t = d/s.
⚠️ OJO: el "primer bit" ya había llegado antes, hace un
   d_trans. Lo que se mide acá es la propagation DEL ÚLTIMO BIT.
```

La fórmula:



Los valores de `s` que hay que saber de memoria:

MEDIO	s	d para 1000 km
VACÍO (espacio libre)	$3 \cdot 10^8$ m/s	3,33 ms
AIRE	$2{,}998 \cdot 10^8$ m/s	3,34 ms
COBRE (UTP/STP, coaxial)	$\sim 2 \cdot 10^8$ m/s	5,00 ms
FIBRA ÓPTICA (silicio)	$\sim 2 \cdot 10^8$ m/s	5,00 ms
POLVO / GAS (nuclear)	$\sim 3 \cdot 10^8$ m/s	3,33 ms

```
Para problemas de examen con cobre o fibra:  s = 2 · 10⁸ m/s
Para un enlace por satélite:                 s = 3 · 10⁸ m/s
```

Las dos propiedades:

1. **No depende del paquete.** Un paquete de 40 bits y uno de 15 000 bits tardan **lo mismo** en propagarse por el mismo enlace. La señal es la misma.
2. **No depende del equipo ni del ancho de banda.** Cambiar de 1 Mbps a 100 Mbps **no cambia** `d_prop`. Cambiar de cobre a fibra tampoco (2,00 → 2,04 · 10^8, un 2 %).

Y la diferencia con la velocidad de propagación, que es el contenido del Examen 2009-12 P1a:

```
s  NO es un ancho de banda.  Está en m/s, no en bps.
s  NO es "la velocidad del enlace". La velocidad del enlace es R.
s  es una propiedad del MEDIO FÍSICO.
```

1.2.5 Las dos tablas comparativas

Tabla A — De qué depende cada componente

	d_proc	d_cola	d_trans	d_propagación
¿Depende de L?	NO	NO	SÍ	NO
¿Depende de R?	NO	SÍ	SÍ	NO
¿Depende de d?	NO	NO	NO	SÍ
¿Depende de s?	NO	NO	NO	SÍ
¿Depende del medio?	NO	SÍ	NO	NO
¿Depende de la carga/congestión	NO	SÍ	NO	NO
¿Se puede calcular con los datos?	si te lo dan	NO	SÍ	SÍ
¿Cuota con R?	NO	NO	SÍ	NO
¿Cuota con s?	NO	NO	NO	SÍ
UNIDAD	s, ms	s, ms	s, ms	s, ms
TAMAÑO TÍPICO	~1 µs	0 a ∞	µs a ms	ms a s
¿Es VARIABLE?	NO	**SÍ**	NO	NO
¿Refleja la congestión?	NO	**SÍ**	NO	NO

La última fila es la que se pregunta en el examen. Y la penúltima: "señale cuáles son los más significativos" ⇒ **la propagación, porque es la única que siempre está y es la única que domina en enlaces largos**.

Tabla B — Cuánto pesa cada uno

```
ENLACE DE 1 Gbps, PAQUETE DE 1000 BYTES, DISTANCIA VARIABLE

L = 1000 bytes = 8000 bits
R = 10⁹ bit/s
d_trans = 8000/10⁹ = 8 µs
d_proc  ≈ 1 µs
s = 2·10⁸ m/s
```

Distancia	d_prop	d_total	Cuánto pesa d_prop
1 m	5 ns	9,005 µs	0,06 %
100 m	0,5 µs	9,5 µs	5,3 %
1 km	5 µs	14 µs	35,7 %
100 km	500 µs	509 µs	98,2 %
5 000 km	25 ms	25,009 ms	99,996 %
36 000 km (satélite)	180 ms	180,008 ms	99,996 %

La lectura de la tabla, que es lo que hay que decir:

- **En una LAN**, el retardo es **todo de procesamiento y transmisión**: a 1 metro, `d_prop` es el 0,06 %. Por eso un `ping` a un host en el mismo switch da **0,05 ms**, que es puramente `d_proc + d_trans`.

- **A 100 km**, `d_prop` ya es el 98 %. A partir de ahí, **duplicar la distancia duplica el retardo**, y mejorarla red significa **acortar la distancia**, no mejorar el equipo.
- **A 36 000 km (satélite)**, `d_prop` = **180 ms** de ida, **360 ms de RTT**. Ninguna mejora de hardware lo cambia: es geometría.

Conclusión operativa:

```
EN UNA LAN:  mejorar el retardo = mejorar el EQUIPO
EN UNA WAN:  mejorar el retardo = acortar el CAMINO
```

1.3 Ancho de banda vs. velocidad de propagación

Esta es la **Pregunta 1a del Examen 2009-12** y hay que tenerla respondida con las palabras exactas. Ya la vimos en la Clase 1; acá va la versión con la derivación matemática que la justifica.

La tabla

	Ancho de banda `B` o `R`	**Velocidad de propagación `s`**
Unidad	**bits por segundo (bps)**	**metros por segundo (m/s)**
Qué mide	Cuántos bits por segundo	A qué velocidad viaja la señal
De qué depende	Del **equipo**: puerto, transceptor, modulación	Del **medio físico**: la física del material
Depende de la distancia	No	No
Aparece en	`d_transmisión = L / R`	`d_propagación = d / s`
Ejemplo de enunciado	"el enlace tiene 1 Gbps"	"en ese cable la señal va a $2 \cdot 10^8$ m/s"

Por qué son perpendiculares: la demostración con números

Dos enlaces del mismo operador, a la misma distancia, con la misma fibra:

```
Enlace A: 1 Gbps    d = 1000 km
Enlace B: 100 Mbps  d = 1000 km
s = 2·10⁸ m/s en los dos (misma fibra)

d_prop(A) = 10⁶ / 2·10⁸ = 5,0 ms
d_prop(B) = 10⁶ / 2·10⁸ = 5,0 ms      ← IDÉNTICO

L = 12 000 bits
d_trans(A) = 12 000 / 10⁹  = 12 µs
d_trans(B) = 12 000 / 10⁸  = 120 µs

d_total(A) = 5,000012 ms
d_total(B) = 5,000120 ms

DIFERENCIA = 0,000108 ms = 108 ns

⚠️ 1000 veces más ancho de banda, y el retardo es el mismo
   salvo en 108 nanosegundos.
```

Y el otro lado de la demostración, que es la que convence: mismo ancho de banda, distinto medio.

```
Enlace C: 1 Gbps por FIBRA
Enlace D: 1 Gbps por COBRE
R = 10⁹ en los dos

d_trans(C) = 12 µs
d_trans(D) = 12 µs          ← IDÉNTICO

d_prop(C) = 10⁶ / 2,04·10⁸ = 4,90 ms
d_prop(D) = 10⁶ / 2,00·10⁸ = 5,00 ms

DIFERENCIA = 0,10 ms = 100 µs

⚠️ La fibra es 1,1 % más rápida en propagación, pero gana
   en otro lado: atenuación mucho menor (26 dB/km contra 20),
   por lo que alcanza 100 km sin repetidor contra los 100 m
   del cobre.
```

La frase de cierre para el examen:

> El ancho de banda y la velocidad de propagación **no son comparables**: uno mide una cantidad de información por unidad de tiempo (`bps`) y el otro una rapidez física (`m/s`). El primero depende del **equipo** y aparece en el retardo de transmisión; el segundo depende del **medio** y aparece en el retardo de propagación. **Duplicar el ancho de banda reduce el retardo de transmisión a la mitad y no cambia el de propagación.** Un enlace de 1 Gbps no es "50 veces más rápido" que uno de 100 Mbps: es 50 veces más **ancho**.

1.4 El *bandwidth-delay product*: los bits que están en vuelo

La definición

```
┌──────────────────────────────────────────────┐
│                                              │
│   BDP = R · RTT        [bits]                │
│                                              │
│   = ancho de banda (bps) × tiempo de ida y   │
│     vuelta (s)                               │
│                                              │
│   = los BITS que caben en el enlace durante un │
│     viaje completo de ida y vuelta           │
│                                              │
└──────────────────────────────────────────────┘
```

Por qué ese número importa

La idea es esta: **mientras un paquete va de ida, el emisor puede seguir mandando**. Si manda un paquete cada vez que llega un ACK, el único paquete "en vuelo" es el que está en el cable en ese instante. Si en cambio manda una **ventana** de `W` bytes sin esperar, el enlace lleva `W` bytes en tránsito al mismo tiempo.

```
El enlace es un CAÑO. El BDP es cuánta agua le entra.

┌──────────────────────────────────────────────────────────┐
│                                                          │
│   A ═══════════════════════════════════════════ B        │
│                                                          │
│   ←──────── L = 12 000 km ──────────→                    │
│                                                          │
│   s = 2·10⁸ m/s   →  d/s = 12·10⁶/2·10⁸ = 60 ms de ida   │
│   R = 1 Gbps      →  10⁹ bits/s                          │
│                                                          │
│   BDP = 10⁹ × (2 × 60·10⁻³) = 10⁹ × 0,12 = 1,2·10⁸ bits  │
│                                    = 15 MB = 15 000 000 B│
│                                                          │
│   Para LLENAR el caño de punta a punta necesito 15 MB    │
│   "en vuelo" al mismo tiempo.                            │
└──────────────────────────────────────────────────────────┘
```

El cálculo, con la aritmética visible

Ejemplo: enlace de 1 Mbps entre Montevideo y Buenos Aires, 1 200 km de fibra, `s = 2·10⁸ m/s`.

```
d (de ida) = 1 200 km = 1,2·10⁶ m
s          = 2·10⁸ m/s

d_prop (ida) = 1,2·10⁶ / 2·10⁸ = 6·10⁻³ s = 6 ms
RTT ≈ 2 × d_prop = 12 ms   (ignorando cola y procesamiento)

R = 10⁶ bit/s

BDP = R × RTT = 10⁶ bit/s × 12·10⁻³ s
    = 12·10³ bits
    = 12 000 bits
    = 1 500 bytes        ← el BDP en BYTES
```

Interpretación: en ese enlace, en cada momento hay como máximo **1500 bytes** en tránsito. Si tu ventana de TCP es de **1024 bytes**, el enlace **nunca** se llena: mandás 1024 bytes, esperás el ACK, y el caño queda con 1024 bytes de un máximo posible de 1500.

```
Throughput máximo alcanzable = ventana / RTT

con ventana = 1024 B:  1024 / 0,012 s = 85 333 B/s = 0,68 Mbps
                                  ⚠️ contra 1 Mbps de enlace
con ventana = 1500 B:  1500 / 0,012 s = 125 000 B/s = 1,00 Mbps  ✓
con ventana = 4096 B:  4096 / 0,012 s = 341 333 B/s = 2,73 Mbps
                    ⚠️ más que el enlace: no se puede,
                       el enlace es el techo
```

La regla que sale de acá, y que es el motivo de `Window Scale`:

> Para aprovechar toda la capacidad de un enlace, la ventana de TCP tiene que ser **mayor o igual al BDP**. Si es menor, el throughput queda limitado por la ventana, no por el enlace.

Y por qué las conexiones de larga distancia necesitan ventanas enormes:

```
Enlace                R        RTT          BDP        ¿ventana de 64 KB alcanza?
──────────────────────────────────────────────────────────────────────────────────
LAN 1 Gbps            1 Gbps   0,1 ms       12,5 kB    SÍ, de sobra
Metro 10 Gbps         10 Gbps  1 ms         1,25 MB    NO (64× corto)
Continental 10 Gbps   10 Gbps  120 ms       150 MB     NO (2400× corto)
Satélite 1 Gbps       1 Gbps   600 ms       75 MB      NO (1200× corto)
Satélite 100 Mbps     100 Mbps 600 ms       7,5 MB     NO (120× corto)
```

Y de ahí salen dos cosas concretas:

1. `Window Scale` **(`RFC 7323`)** multiplica la ventana por una potencia de 2 que se negocia en el SYN. Sin él, el máximo son 65 535 bytes, y **con los BDP de las redes de backbone de hoy eso es inútil**. Es un protocolo de 2014 que resolvió un problema de 1992, y sigue siendo obligatorio.
2. **Las conexiones de larga distancia seFAULTS de throughput** si no escalan la ventana: no por falta de ancho de banda, sino porque **no tienen suficiente "agua" en el caño**.

1.5 RTT, latencia y jitter

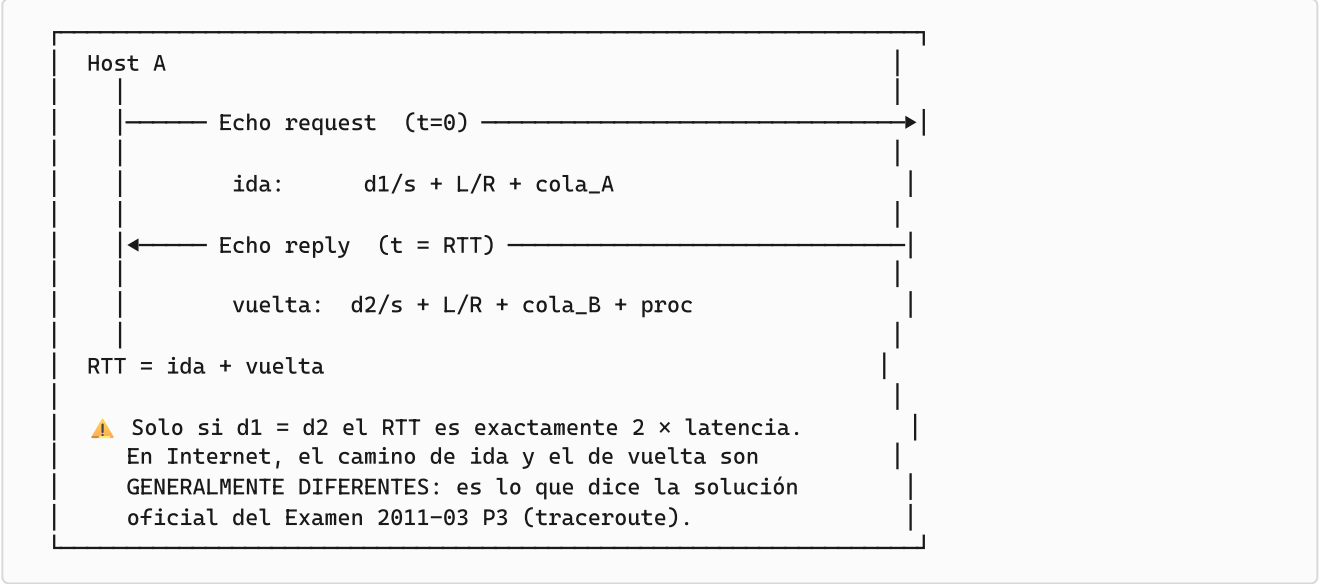
Latencia

Es el **tiempo de un viaje de ida**, de un host al otro. Se mide con el primer paquete de un `traceroute` (más `d_trans`) o con `ping` y la mitad del RTT.

```
latencia mínima ≈ d/s  +  L/R      (sin cola, sin procesamiento)
```

RTT (*Round Trip Time*)

Es el **tiempo de ida y vuelta**. Es la latencia multiplicada por 2, **más** el procesamiento de los dos extremos, **más** la cola si la hay.



Cómo se mide: `ping` mide el RTT porque manda un Echo request (ICMP tipo 8) y espera el Echo reply (ICMP tipo 0). Cada salida del comando da **un RTT**.

```
$ ping www.udelar.edu.uy
64 bytes from 203.0.113.9: icmp_seq=1 ttl=54 time=28,4 ms
64 bytes from 203.0.113.9: icmp_seq=2 ttl=54 time=28,1 ms
64 bytes from 203.0.113.9: icmp_seq=3 ttl=54 time=29,0 ms
```

Los tres valores son RTT. **La latencia de ida es ~14 ms.**

La primera diferencia de la serie es el RTT mínimo, y la siguiente es la variación (que es el jitter).

Jitter

Es **la variación de la latencia entre paquetes del mismo flujo**. Se mide de varias formas:

Métrica	Fórmula	Ventaja
Pico a pico (la que más se usa en la práctica)	`retardo_máx − retardo_mín`	Simple, la que usa el *jitter buffer*
Desviación estándar (RFC 3550)	`J = √( (1/n)·Σ (R_i − R_esperado)² )`	La que usa RTP
Media absoluta	`J = (1/n)·Σ |R_i − R_promedio|`	Intermedia

```
Tres paquetes del mismo flujo, enviados con 20 ms de separación:
   paquete 1:  80 ms
   paquete 2:  95 ms
   paquete 3: 110 ms

jitter (pico a pico) = 110 − 80 = 30 ms
promedio             = (80+95+110)/3 = 95 ms
desviación estándar  = √(((80−95)²+(95−95)²+(110−95)²)/3)
                     = √((225+0+225)/3) = √150 = 12,2 ms
```

Por qué el jitter importa más que la latencia (y por qué importa para audio y video)

Para un archivo, da igual: si tarda 100 ms o 400 ms, total ya esperaste.

Para una llamada de voz, la latencia alta se nota como un "eco del parque" (retraso de la voz), y el humano lo tolera hasta ~150 ms de ida. Lo que **rompe** la conversación es el jitter:

```
Un códec de voz reproduce una muestra cada 20 ms.

El paquete 1 tardó 80 ms → llega en t = 80
El paquete 2 tardó 110 ms → llega en t = 130
     ⚠️ Pero el reproductor lo esperaba en t = 100.
        A los 100 ms no tiene nada que reproducir.
        → Clic (click) en el audio.
El paquete 3 tardó 95 ms → llega en t = 115, tarde para t = 120.
        → se le pasó el hueco.

El PROBLEMA no es que tarde: es que lleguen en RÁFAGA.
El buffer de jitter (jitter buffer) lo resuelve: mete un
RETARDO FIJO (por ejemplo 60 ms) y entrega las muestras en
ritmo constante, reordenando y rellenando.
```

La relación con el BDP y con la cola:

```
⇐ Si hay un paquete perdido y TCP reenvía, el hueco es de un RTO
   (mínimo 200 ms en Linux) ⇒ jitter enorme.
⇐ Si un router tiene el buffer lleno, la cola cambia de largo
   de un momento a otro ⇒ jitter.
⇐ Si hay ECMP, dos paquetes del mismo flujo pueden ir por caminos
   de latencia diferente ⇒ jitter.
```

Todo el jitter viene de la `d_cola`. Y por eso **más jitter = más cola = más riesgo de bufferbloat**, y por eso los AQMs se escribieron para el jitter tanto como para la pérdida.

1.6 Congestión, pérdidas y retardo

Esta sección resuelve la **Pregunta 2 del Examen 2011-03** (6 puntos). Los tres conceptos, su definición precisa y su vínculo.

Congestión

Definición de la solución oficial, textual:

> *"La congestión está asociada a la escasez, durante un período de tiempo, de los recursos (compartidos) necesarios para procesar adecuadamente todos los paquetes que pretenden hacer uso de los mismos. Esos recursos son básicamente: capacidad de procesamiento y buffers en los routers y, enlaces."*

Los cuatro desgloses de esa definición:

1. **Es escasez de recursos COMPARTIDOS.** Si el recurso es dedicado, no hay congestión: si te reservaron 1 Mbps, tenés 1 Mbps. La congestión aparece porque **muchos usuarios comparten** el mismo enlace y los mismos buffers.
2. **Es temporal.** "Durante un período de tiempo". Una ráfaga de 200 ms no es congestión; 200 ms de saturación sostenida sí. Por eso los AQMs tienen **ventanas de tiempo**.
3. **Los recursos son de dos tipos**: capacidad de procesamiento (el fabric, la CPU del router) y buffers (los búferes de las colas de salida). Los dos se llenan.
4. **Se manifiesta en síntomas**, no es un fenómeno observable directamente. Los síntomas son **pérdidas y retardos**.

Los tres tipos de congestión, con el síntoma de cada uno:

```
┌──────────────────────────┬──────────────────────┬──────────────────────┐
│ TIPO                     │ QUÉ SE LLENA         │ SÍNTOMA              │
├──────────────────────────┼──────────────────────┼──────────────────────┤
│ Congestión de ENLACE     │ la cola de salida de │ PÉRDIDAS (buffer     │
│ (la más común)           │ un puerto de un      │ lleno) + retardo de  │
│                          │ router               │ cola grande          │
├──────────────────────────┼──────────────────────┼──────────────────────┤
│ Congestión de FABRIC     │ la memoria interna   │ Pérdidas en la       │
│ (backplane)              │ del router (memoria  │ entrada del router;  │
│                          │ de buffers, tablas)  │ latencia de la NIC   │
├──────────────────────────┼──────────────────────┼──────────────────────┤
│ Congestión de CPU        │ la CPU del router    │ retardo de proceso-  │
│                          │ que corre el protocolo│ miento alto;        │
│                          │ de control           │ pérdida por descarte │
└──────────────────────────┴──────────────────────┴──────────────────────┘
```

Pérdida

Definición de la solución oficial, textual:

> *"Los paquetes se encolan en buffers en los routers antes de ser enviados por el enlace de salida; si un paquete no encuentra espacio disponible en un buffer, será descartado (pérdida)."*

Hay más causas de pérdida que el buffer lleno, y hay que conocerlas:

Causa	Dónde	Quién la detecta	¿La recupera TCP?
Buffer lleno (*tail drop*)	Cola de salida de un router	El emisor, por timeout o 3 dupACK	Sí
Checksum incorrecto	Cualquier enlace (error de medio)	El receptor de la trama (CRC)	Sí, con ARQ de capa 2
TTL = 0	Cualquier router	El router, por ICMP Time Exceeded	No: el paquete se descarta
Filtro de firewall / ACL	Cualquier router	Puede o no generar ICMP	No
Congestión de la CPU del router	Entrada del router	El emisor	Sí
Pérdida de un fragmento	Donde se perdió	El host destino, en el reensamblaje	No: se descarta **todo** el datagrama
Descarte por política de DiffServ	Cola de una clase de baja prioridad	El emisor	Sí, pero es intencional

El punto clave: una pérdida en la capa 3 es IRREVERSIBLE. IP no tiene recuperación. Si el paquete se perdió, no existe "reintentar en la red": lo único que puede hacer el emisor es **mandarlo otra vez** (que es lo que hace TCP, y por eso TCP necesita numerar bytes y reconocer).

Retardo

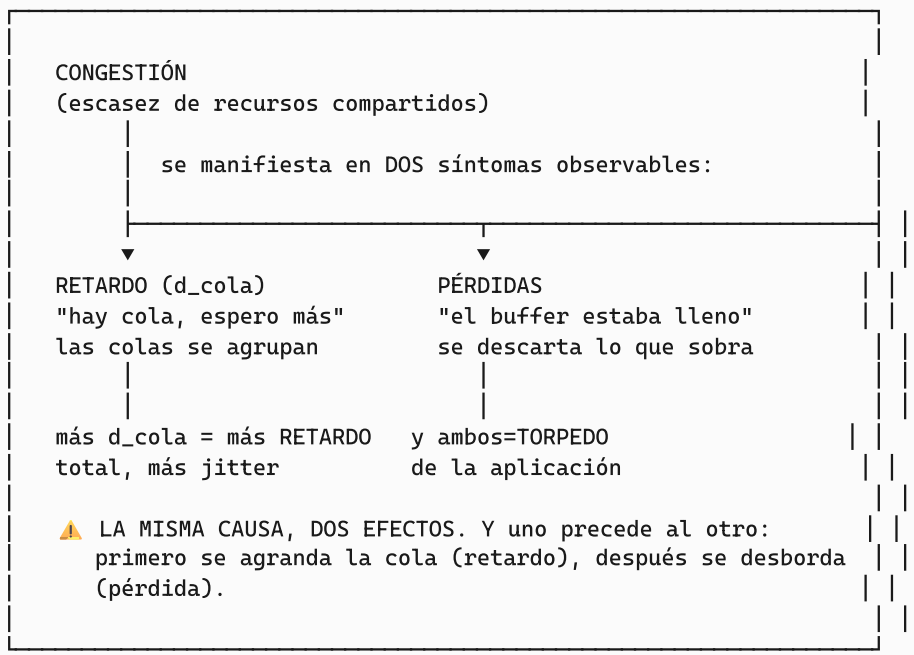
Definición de la solución oficial, textual:

> *"Si un paquete encuentra lugar en el buffer pero su envío por el enlace de salida se ve retrasado (por existir otros paquetes esperando poder hacerlo antes que él) sufrirá de retardo (retardo de cola). También puede sufrir retardo por el procesamiento que se le debe realizar en el nodo previo a ser colocado en el buffer de salida (chequeos de errores, modificaciones de campos de cabecera, ...). Adicionalmente habrá retardo debido a la propagación del paquete en el medio físico y también motivado en el ancho de banda disponible en el enlace de salida y su relación con el tamaño del paquete que se desea enviar (retardo de transmisión)."*

La oficial identifica los CUATRO componentes del retardo en un solo párrafo. Es exactamente el §1.2 de esta clase.

El vínculo entre los tres

Este es el punto que hay que_argumentar, y es lo que la pregunta pide cuando dice "explique los conceptos":



La demostración numérica del vínculo, que es lo que suma puntos:

```
Enlace de 100 Mbps, paquetes de 1500 bytes.
El enlace transmite un paquete cada 1500/10⁸ = 15 µs.

Tasa de llegada λ (paquetes/s)     | d_cola (con buffer de 40 paquetes)
──────────────────────────────────┼──────────────────────────────────
   λ = 0      (sin tráfico)        |    0 µs
   λ = 6,67·10⁶  (= 100 Mbps)     |    0 µs   ← saturado pero sin cola
   λ = 7·10⁶                      |    1 500 × 1 = 15 µs
   λ = 8·10⁶                      |    15 000 µs = 15 ms
   λ = 9·10⁶                      |    30 000 µs = 30 ms
   λ = 10·10⁶                     |    40 × 15 µs = 600 µs  ← el ÚLTIMO cabe
   λ = 10,1·10⁶                   |    DESCARTE desde el primero
──────────────────────────────────┼──────────────────────────────────

⚠️ A λ = 10·10⁶ = 100 Mbps, que es el máximo del enlace, la cola
   es de 600 µs. A 10,1·10⁶, que es un 0,1 % más, se pierde
   desde el primer paquete.

⇒ El retardo de cola EXPLOTA justo antes de la pérdida.
⇒ Un aumento del 0,1 % del tráfico triplica el retardo.
⇒ Por eso TCP reduce la ventana ante una pérdida: está reacting
   al síntoma de que el buffer se llenó, es decir, a que está a
   punto de desbordarse.
```

Y el corolario sobre por qué TCP no distingue: TCP ve un paquete perdido. No sabe si se perdió porque (a) se llenó el buffer de un router (congestión), (b) se corrompió en un enlace, o (c) hay un error de configuración. **Para TCP los tres son lo mismo: un paquete perdido es un paquete perdido.** Y por eso su reacción (bajar la ventana) es **conservadora en el caso (b) y correcta en el caso (a)**.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **Los cuatro componentes** del retardo, sus fórmulas y sus unidades. - `d_transmisión = L/R` y `d_propagación = d/s`, con `L` en **bits** y `d` en **metros**. - **La diferencia ancho de banda (bps) vs. velocidad de propagación (m/s)** y de qué depende cada uno. - **Que la congestión actúa sobre el retardo de encolamiento** y se manifiesta en pérdidas y retardos. - **Los valores de `s`**: ~$2{\cdot}10^8$ m/s en cobre y fibra, ~$3{\cdot}10^8$ en vacío y satélite. - **Que el retardo de cola es el único componente variable** y el único que refleja la congestión.

Cambió o se modificó:

- **La latencia de los enlaces deteriorated mucho menos de lo que la gente cree, pero los BDP crecieron muchísimo.** Un enlace continental a 100 Gbps con 100 ms de RTT tiene un **BDP de 1,25 GB**. Ningún stack de TCP con el `Window Scale` de `RFC 7323` lo llena. ⇒ **El throughput de las conexiones de larga distancia está limitado por la ventana, no por el ancho de banda**, y eso obliga a BDP-aware rate control (que es exactamente lo que hace **BBR**: no una ventana, sino un modelo de *bottleneck bandwidth* × *propagation delay*).
- `d_proc` **dejó de ser despreciable en algunos casos concretos.** En un backbone a 100/400 Gbps, la CPU del router corre **toda la pila en el plano de datos**: MPLS, ACLs, ECMP y las dos filas de QoS. El `d_proc` sube a **1 a 10 µs**, que ya es del orden del `d_transmisión` (1 µs a 10 Gbps con paquete de 1500 B). ⇒ **Decir "el procesamiento es despreciable" es correcto en una LAN y puede ser falso en un router de última generación.** Lo que sigue siendo cierto es que **no depende de la carga**, y por eso no es el componente de la congestión.

- **El control de congestión dejó de ser AIMD puro.** `RFC 9438` (CUBIC, *Standards Track*, 2023) es el default global y **no** crece linealmente. **BBRv3 nunca fue RFC** (`draft-ietf-ccwg-bbr` , *Experimental*), y es lo que se usa en QUIC y en partes de Linux.
- **La pérdida como señal de congestión se volvió opcional. ECN** (`RFC 3168`) marca en vez de descartar, y `RFC 9768` **(AccECN, 2026)** hace el feedback más preciso. **L4S (** `RFC 9331` **, 2023)** marca con umbral muy bajo para conseguir **latencia mínima sin descartes**: el objetivo ya no es "evitar la pérdida" sino **"evitar la cola"**. ⇒ **El jitter, que antes era un efecto secundario, pasó a ser el objetivo primario de la diseño de congestión.** Ese es el cambio conceptual más grande desde 2011.
- **El *bufferbloat* es un problema de primera clase y tiene solución en el stack por defecto.** Linux usa **FQ-CoDel** desde el kernel 3.5 (2012). Los AQMs descartan temprano para que el retardo no se dispare, aunque la pérdida "desperdicie" ancho de banda.
- **El multicast cambió la relación entre retardo y pérdida.** Con `IP_MULTICAST` los paquetes de un mismo flujo salen por interfaces con latencias distintas, y sin control de tasa la congestión de un destino **afecta a los demás** (problema de *multicast congestion collapse*, que la RFC 5771 documenta). Es un caso donde "el BDP" ya no es un número por flujo.

> **Consecuencia para el examen:** los **cuatro componentes, sus fórmulas y sus unidades no cambiaron** en 37 años. Lo que cambió son las **cifras de capacidad** y el **objetivo del control de congestión** (de "evitar la pérdida" a "evitar la cola"). Resolvé con los números del enunciado, y si hay lugar,Sumá una línea de actualización sobre ECN y AQM.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Los cuatro componentes de un paquete en dos enlaces

Datos. Un paquete de **1000 bytes** va de A a B por dos enlaces: el primero de cobre de **50 km** a **1 Mbps**, el segundo de fibra de **100 km** a **100 Mbps**. Los dos conmutadores tienen `d_proc = 0,05 ms`. No hay retardo de cola.

a) Calculá el retardo de cada componente del primer enlace. **b)** Calculá el retardo de cada componente del segundo enlace. **c)** ¿Cuál es el retardo total? ¿Qué componente domina?

Resolución, con `s = 2·10⁸ m/s` en ambos medios.

Paso 1 — Datos convertidos a las unidades correctas.

```
L = 1000 bytes × 8 bits/byte = 8000 bits      ⚠ EN BITS
e1: R = 10⁶ bit/s,  d = 50 km = 5·10⁴ m,  s = 2·10⁸ m/s
e2: R = 10⁸ bit/s,  d = 100 km = 10⁵ m,  s = 2·10⁸ m/s
d_proc = 0,05 ms = 5·10⁻⁵ s (por conmutador, hay 1 entre los dos enlaces)
```

Paso 2 — Componentes del enlace 1 (cobre, 50 km, 1 Mbps).

```
d_proc  = 0,05 ms                       (del switch de salida)

d_cola  = 0                             (dato del enunciado: no hay cola)

d_trans = L / R = 8000 bits / 10⁶ bit/s
        = 8·10⁻³ s = 8 ms
        ⚠ OJO: 1000/10⁶ daría 1 ms. La respuesta correcta es 8 ms
          porque L va en BITS.

d_prop  = d / s = 5·10⁴ m / 2·10⁸ m/s
        = 2,5·10⁻⁴ s = 0,25 ms
```

```
Subtotal e1 = 0,05 + 0 + 8 + 0,25 = 8,30 ms
```

Paso 3 — Componentes del enlace 2 (fibra, 100 km, 100 Mbps).

```
d_proc  = 0,05 ms

d_cola  = 0

d_trans = L / R = 8000 / 10⁸
        = 8·10⁻⁵ s = 0,08 ms
        ⚠ 10 veces MENOS que en el enlace 1: el ancho de banda
          es 100 veces mayor, pero el retardo de transmisión
          es 8 ms / 0,08 ms = 100 veces menor... No: 8/0,08 = 100.
          Sí, 100 veces. Consistente.

d_prop  = 10⁵ / 2·10⁸ = 5·10⁻⁴ s = 0,5 ms
```

```
Subtotal e2 = 0,05 + 0 + 0,08 + 0,5 = 0,63 ms
```

Paso 4 — El retardo total.

```
d_total = Σ componentes

d_total = d_proc(e1) + d_cola(e1) + d_trans(e1) + d_prop(e1)
        + d_proc(e2) + d_cola(e2) + d_trans(e2) + d_prop(e2)

⚠️ d_proc APARECE 2 VECES: una por cada router. Si los dos
   enlaces están unidos por UN switch, d_proc(e1) es el del switch
   que recibe por el enlace 1 y manda por el 2, y d_proc(e2) es
   el del switch que recibe por el 2 y manda por el 3.
   Con SOLO 2 enlaces hay 1 switch intermedio: el host A
   transmite, un switch, y el host B recibe. Por eso d_proc
   del enlace 1 NO existe (es el host A, sin d_proc) y solo
   hay 1 d_proc.

CORRECCIÓN DEL PASO 2 y 3:
   d_proc del enlace 1 = 0 (A es un host)
   d_proc del enlace 2 = 0,05 ms (un switch)

d_total = 0 + 0 + 8 + 0,25 + 0,05 + 0 + 0,08 + 0,5
        = 8,88 ms
```

Paso 5 — ¿Qué componente domina?

```
┌──────────────┬──────────┬──────────┐
│ Componente   │ valor    │ %        │
├──────────────┼──────────┼──────────┤
│ d_trans(e1)  │ 8,00 ms  │ 90,1 %   │  ◄── DOMINA
│ d_prop(e2)   │ 0,50 ms  │  5,6 %   │
│ d_prop(e1)   │ 0,25 ms  │  2,8 %   │
│ d_trans(e2)  │ 0,08 ms  │  0,9 %   │
│ d_proc       │ 0,05 ms  │  0,6 %   │
│ d_cola       │ 0        │  0 %     │
├──────────────┼──────────┼──────────┤
│ TOTAL        │ 8,88 ms  │ 100 %    │
└──────────────┴──────────┴──────────┘
```

Interpretación — y es el punto del ejercicio:

> **El enlace lento manda en el retardo total.** El enlace de 1 Mbps tiene un `d_trans` de 8 ms; el de 100 Mbps, de 0,08 ms. El primero aporta el 90 % del retardo total **pese a recorrer la mitad de distancia**.

Y el corolario, que es la lección de infraestructura:

```
Si subimos el enlace 1 de 1 Mbps a 100 Mbps:

d_trans(e1) = 8000/10⁸ = 0,08 ms
d_total nuevo = 0,08 + 0,25 + 0,05 + 0,08 + 0,5 = 0,96 ms

MEJORA DE 8,88 ms → 0,96 ms:  9,25 VECES más rápido.

⚠️ PERO si además acortamos el enlace 2 de 100 km a 10 km:
d_prop(e2) = 10⁴/2·10⁸ = 0,05 ms
d_total = 0,08 + 0,25 + 0,05 + 0,08 + 0,05 = 0,51 ms

Ahí sí, el 1 Mbps pasa a ser el cuello de botella de verdad.
```

La respuesta a la pregunta implícita: *en una WAN, primero hay que arreglar el enlace lento (el `d_trans` grande); en una red de backbone, primero hay que acortar las distancias (el `d_prop` grande).*

⚠️ Errores típicos

- **Dividir bytes por bps.** `1000/10⁶` en vez de `8000/10⁶`. Da un factor 8 de error. **Todos los protocolos dan `L` en bytes; el retardo se calcula en bits.**
- **Poner `d_proc` tantas veces como enlaces.** Con `n` enlaces hay `n − 1` routers. Este es el error del Examen 2011-03 P5.
- **Sumar `d_trans` una sola vez.** Hay **un `d_trans` por enlace**, con el `R` de ese enlace.
- **Confundir 50 km con $5 \cdot 10^4$ m.** 1 km = 1000 m = 10^3 m. 50 km = $5 \cdot 10^4$ m.
- **Mezclar las unidades.** Si `d_trans` sale en segundos y `d_prop` en milisegundos, hay que convertir antes de sumar.

2.2 Ejercicio 2 — El BDP con la aritmética completa

Datos. Un enlace de **50 Mbps** une dos puntos separados por **2 000 km** de fibra (`s = 2·10⁸ m/s`). Se supone que no hay cola ni procesamiento.

a) ¿Cuánto tarda un paquete en llegar (latencia)? **b)** ¿Cuál es el RTT? **c)** ¿Cuál es el BDP, en bits y en bytes? **d)** Una conexión TCP tiene una ventana de **128 KB**. ¿Alcanza para llenar el enlace? ¿Cuál es el throughput máximo?

Resolución.

Paso 1 — Latencia.

```
d = 2 000 km = 2·10⁶ m
s = 2·10⁸ m/s

d_prop = 2·10⁶ / 2·10⁸ = 10⁻² s = 0,01 s = 10 ms

(ignorando d_trans y d_proc, como dice el enunciado)
latencia ≈ 10 ms
```

Paso 2 — RTT.

```
RTT = 2 × latencia = 2 × 10 ms = 20 ms = 0,02 s
```

Paso 3 — BDP.

```
R = 50 Mbps = 5·10⁷ bit/s
RTT = 0,02 s

BDP = R × RTT = 5·10⁷ bit/s × 0,02 s
    = 10⁶ bits
    = 1 000 000 bits

en bytes: 1 000 000 / 8 = 125 000 bytes = 125 kB = 122 kB aprox.
```

Interpretación: en ese enlace, **125 kB** pueden estar en tránsito al mismo tiempo.

Paso 4 — ¿Alcanza una ventana de 128 kB?

```
ventana = 128 kB = 128 × 1024 = 131 072 bytes
BDP     = 125 000 bytes

131 072 > 125 000   →  SÍ alcanza, por poco (con 4,9 % de margen)
```

Throughput máximo alcanzable:

```
Throughput = ventana / RTT
           = 131 072 bytes / 0,02 s
           = 6 553 600 B/s
           = 6,55 MB/s
           = 52,4 Mbps
```

Y el techo real del enlace:

```
R = 50 Mbps
Throughput con la ventana de 128 kB = 52,4 Mbps > 50 Mbps

⚠️ 52,4 > 50: la ventana SÍ alcanza. El tope es el ENLACE.
Throughput real = 50 Mbps = 6,25 MB/s

Para(Stream) verificar si la ventana alcanza, el criterio es:
    ventana ≥ BDP   ⇔   throughput alcanzable ≥ R
```

Paso 5 — Qué pasaría con ventanas más chicas.

ventana	= BDP?	throughput	¿aprovecha?
8 KB	8192 < 125000	8192/0,02 = 410 kB/s = 3,3 Mbps	3,3 % ⚠️
64 KB	65536 < 125000	65536/0,02 = 3,28 MB/s = 26,2 Mbps	26 % ⚠️
125 KB	= 125 000	125 000/0,02 = 6,25 MB/s = 50 Mbps	100 % ✓
1 MB	> BDP	limitado por el enlace: 50 Mbps	100 % ✓ (sobra ventana)

Interpretación y la lección:

> Con una ventana de **8 kB** (el `rwnd` por defecto de Windows durante años) en este enlace, se aprovecha el **3,3 %** del ancho de banda. El enlace no está lento: **falta ventana**. Con **64 kB**, el 26 %. Recién con **125 kB** se llena.

Y el cálculo del tiempo que tardaría en descargar 10 MB:

```
Con 8 kB de ventana:
   rounds = 10 485 760 bytes / 8192 bytes = 1280 rounds
   tiempo = 1280 × 20 ms = 25,6 s
   throughput = 10 485 760/25,6 = 409 600 B/s = 3,3 Mbps ✓

Con 125 kB de ventana (enlazada al BDP):
   rounds = 10 485 760 / 125 000 = 84 rounds
   tiempo = 84 × 20 ms = 1,68 s
   throughput = 10 485 760/1,68 = 6,24 MB/s = 50 Mbps ✓

⇒ 15,2 VECES más rápido con solo agrandar la ventana.
```

2.3 Ejercicio 3 — RTT, latencia y la diferencia entre las dos

Datos. Un `traceroute` desde Montevideo a Madrid da:

```
1  fw.campus.udelar.edu.uy (203.0.113.1)     0,412 ms  0,389 ms  0,401 ms
2  bundle-ethernet.antibot.com (198.51.100.7)  8,104 ms  8,220 ms  8,051 ms
3  ae-3.border.bb.madrid.es (192.0.2.44)      118,7 ms 119,2 ms 118,9 ms
4  www.ejemplo.es (203.0.113.200)             121,3 ms 121,0 ms 121,4 ms
```

a) ¿Cuál es el RTT a cada nodo? ¿Cuál es la latencia? **b)** ¿Cuál es el RTT mínimo de la conexión? ¿Y el jitter? **c)** ¿Cuánto del RTT es la red y cuánto es el último salto?

Resolución.

a) Lo que da el `traceroute`:

```
Cada línea muestra TRES RTT distintos, uno por cada uno de los
3 probes que manda traceroute.

El RTT es el tiempo de ida y vuelta.
La latencia (tiempo de un viaje de ida) ≈ RTT / 2
```

Salto	Nodo	RTT (v1)	RTT (v2)	RTT (v3)	**Latencia ≈ RTT/2**
1	gateway del campus	0,412	0,389	**0,389**	**0,21 ms**
2	borde del ISP	8,104	8,220	**8,051**	**4,03 ms**
3	border de Madrid	118,7	119,2	**118,9**	**59,5 ms**
4	servidor final	121,3	121,0	**121,0**	**60,5 ms**

⚠️ **La advertencia de la solución oficial del Examen 2011-03 P3, textual:**

> *"Dado que el camino que siguen los paquetes de ida y vuelta de una comunicación sobre Internet no necesariamente son el mismo, las respuestas, tanto de los nodos intermedios como del destino pueden recorrer un camino distinto al que siguieron los paquetes que las motivaron."*

⇒ **El RTT/2 no es la latencia real salvo que el camino sea simétrico.** Es una **estimación**.

b) RTT mínimo y jitter.

```
RTT mínimo de la conexión = el menor RTT del último salto
                          = 121,0 ms

Los tres RTT del último salto: 121,3 · 121,0 · 121,4
promedio = (121,3 + 121,0 + 121,4)/3 = 121,23 ms
jitter (pico a pico) = 121,4 − 121,0 = 0,4 ms
desviación estándar:
    = √(((121,3−121,23)² + (121,0−121,23)² + (121,4−121,23)²)/3)
    = √((0,0049 + 0,0529 + 0,0289)/3)
    = √0,0289 = 0,17 ms
```

Interpretación: el jitter de **0,4 ms** en un enlace transatlántico es **excelente**. Significa que la red no tiene congestión: las colas están vacías. Un jitter de 30 ms en el mismo enlace indicaría colas de 30 ms, es decir, saturación.

c) Reparto del RTT: la red vs. el último salto.

```
RTT al gateway del campus (salto 1):  0,389 ms
RTT total al destino (salto 4):      121,0 ms

RETARDO EN LA RED INTERNA (saltos 1→4):
    121,0 − 0,389 = 120,6 ms  →  99,7 % del RTT

RETARDO DEL ÚLTIMO SALTO (salto 3→4, dentro de la red de destino):
    121,0 − 118,9 = 2,1 ms     →  1,7 % del RTT
```

Lo que esto significa, y es la lección de la tabla:

> **El 99,7 % del retardo está en los enlaces de larga distancia, no en la red local.** El último tramo, que es el que el proveedor controla y por el que el usuario pregunta, aporta el 1,7 %. Por eso mejorar la conexión de un campus a un servidor remoto **rinde muchísimo menos** que acortar el camino o mejorar el enlace troncal.

Y el cálculo del ancho de banda imposible, que es el indicador definitivo de cuello de botella:

```
RTT accumulates:  la LATENCIA es irrecuperable.
Download speed:  el throughput SÍ se puede subir agrandando la ventana.

Con ventana = 65536 B (64 kB):
    throughput = 65 536 / 0,121 s = 541 620 B/s = 4,3 Mbps
    → Con un enlace de 100 Mbps se aprovecha el 4,3 %.

Con ventana = 8 MB (agregando Window Scale, RFC 7323):
    throughput = 8 388 608 / 0,121 s = 69,3 MB/s = 555 Mbps
    → Ahora sí, el enlace es el techo.
```

2.4 Ejercicio 4 — Dos paquetes de distinto tamaño en el mismo enlace

Datos. Un enlace de **10 Mbps** con fibra de **50 km** (`s = 2·10⁸ m/s`). Llegan dos paquetes: **P1 de 100 bytes** y **P2 de 10 000 bytes**.

a) Calculá los 4 componentes de cada uno. **b)** ¿Qué cambia y qué no al pasar de un paquete a otro?

Resolución.

```
P1: L = 100 bytes = 800 bits
P2: L = 10 000 bytes = 80 000 bits

R = 10 Mbps = 10⁷ bit/s
d = 50 km = 5·10⁴ m
s = 2·10⁸ m/s
```

Paso 1 — `d_trans` de cada uno.

```
d_trans(P1) = L/R = 800 / 10⁷ = 8·10⁻⁵ s = 0,08 ms

d_trans(P2) = L/R = 80 000 / 10⁷ = 8·10⁻³ s = 8 ms

⚠️ 100 veces de diferencia. El paquete grande tarda 100 veces
   más en TRANSMITIRSE. Un paquete de 100 bytes "sale" en 80 µs;
   uno de 10 000 bytes tarda 8 ms en salir entero.
```

Paso 2 — `d_prop` de cada uno.

```
d_prop(P1) = d/s = 5·10⁴ / 2·10⁸ = 2,5·10⁻⁴ s = 0,25 ms
d_prop(P2) = d/s = 5·10⁴ / 2·10⁸ = 2,5·10⁻⁴ s = 0,25 ms

⚠️ IDÉNTICOS. El paquete grande no tarda más en PROPAGARSE.
```

Paso 3 — `d_proc` y `d_cola`.

```
d_proc: los mismos 2 µs para los dos (el router no mira L
        para procesar la cabecera; el CRC sí cubre L, pero
        el CRC-32 de un hardware a 10 Gbps va a ~1 bit/ciclo)
d_cola: Supongamos que P2 llega primero y ocupa el enlace.
        d_cola(P1) = 8 ms  (espera a que P2 salga entero)
        d_cola(P2) = 0
```

Paso 4 — La tabla comparativa.

	P1 (100 B)	P2 (10 000 B)	¿cambió?
L (bits)	800	80 000	SÍ (×100)
d_proc	2 µs	2 µs	NO
d_cola	8 ms	0	SÍ
d_trans	0,08 ms	8 ms	SÍ (×100)
d_prop	0,25 ms	0,25 ms	NO ⚠️
TOTAL	8,332 ms	8,252 ms	~igual

Interpretación — el resultado que hay que entender:

> **El paquete 100 veces más grande tarda (casi) lo mismo**, y la razón es que el `d_trans` grande se **compensa** con una `d_cola` chica: el paquete grande ya estaba "en la cabeza" de la cola.

Y si los dos llegaran AL MISMO TIEMPO a una cola vacía:

```
d_cola(P1) = d_cola(P2) = 0

TOTAL(P1) = 2 µs + 0,08 + 0,25 ms = 0,332 ms
TOTAL(P2) = 2 µs + 8    + 0,25 ms = 8,252 ms

⚠️ AHORA SÍ: el grande tarda 24,8 VECES más.
```

La conclusión, que es lo que hay que decir:

```
┌──────────────────────────────────────────────────────────────┐
│ El tamaño del paquete SÍ cambia el retardo total, pero solo  │
│ a través de d_transmisión (y de d_cola).                     │
│                                                              │
│ El retardo de PROPAGACIÓN es INDEPENDIENTE del tamaño.       │
│                                                              │
│ Ése es el contenido de la Pregunta 1a del Examen 2009-12:    │
│ los dos componentes que dependen de cosas DIFERENTES.        │
│                                                              │
│ Y por eso un enlace de 1 Gbps con paquete de 1500 bytes tiene │
│ d_trans = 12 µs (despreciable) mientras que uno de 100 Mbps  │
│ tiene d_trans = 120 µs (comparable a la propagación).        │
│ El ancho de banda determina si el d_trans es o no relevante. │
└──────────────────────────────────────────────────────────────┘
```

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2009-12, Pregunta 1 (10 puntos) — los componentes del retardo

> **Fuente:** Examen `sirc0912` — 18 de diciembre de 2009, Pregunta 1 (10 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0912.pdf

Enunciado (texto del examen)

> Identifique los componentes del retardo en las redes de paquetes, y señale cuáles son los más significativos. En particular: **a)** Señale las diferencias entre el ancho de banda y la velocidad de propagación en el medio. **b)** ¿En qué componente del retardo influye la congestión?

Solución oficial (completa)

> Retardo de procesamiento en el nodo Por ejemplo chequeo de paridad (CRC), determinar enlace de salida (enrutamiento).
>
> Retardo de encolamiento (buffering) Espera en colas del enlace de salida para trasmisión
>
> Retardo de trasmisión Sea R = ancho de banda del enlace (bps) y L = longitud del paquete (bits), el tiempo de envío = L/R
>
> Retardo de propagación Sea d = longitud del enlace físico y s = velocidad de propagación en el medio, el retardo de propagación = d/s
>
> **a)** B es la capacidad de los puertos del dispositivo (switch o router), por ejemplo Ethernet, STM-1, mientras que s es una característica del medio, por ejemplo s ~ 2×10^8 m/seg en cobre.
>
> **b)** El retardo de encolamiento depende del nivel de congestión del router.

Análisis de la solución oficial

Lo que está muy bien:

1. **Las cuatro fórmulas están exactamente correctas y con las variables definidas.** `L/R` con `L` en bits y `R` en bps; `d/s` con `d` en longitud de enlace y `s` en m/s. Eso es exactamente lo que hay que saber, y está perfecto.
2. **Nombra los ejemplos correctos del `d_proc`**: chequeo de paridad (CRC) y "determinar enlace de salida (enrutamiento)". Los dos son las tareas reales de un router. La oficial hasta usa la palabra **"enrutamiento"** para el lookup, que es la terminología de la cátedra.
3. **La definición de `d_cola` es la buena**: "*espera en colas del enlace de salida*". Dice **del enlace de SALIDA**, que es el detalle correcto (es el enlace por el que va a salir, no por el que llegó).

4. **El apartado a) es una frase precisa**: "*B es la capacidad de los puertos del dispositivo, mientras que s es una característica del medio*". Es exactamente la distinción.

Lo que falta, y son los puntos que se ganan:

1. **No dice cuáles son "los más significativos".** El enunciado lo pide explícitamente y la oficial **no lo responde**. Hay que decir: `d_propagación` **y** `d_transmisión`, porque son los dos únicos que se pueden calcular con los datos del problema y los que dominan en enlaces de cualquier longitud relevante. `d_proc` es despreciable (microsegundos) y `d_cola` es variable (no se puede calcular).
2. **No da las unidades de** `d` **ni de** `L`**.** Dice "longitud del enlace físico" y "longitud del paquete (bits)". Falta: `d` **en METROS** y `L` **en BITS**. Es el error número uno de los alumnos.
3. **No menciona que** `s` **no cambia con el enlace ni con el ancho de banda.** El apartado a) lo insinúa ("característica del medio"), pero no lo dice.
4. **El apartado b) es correcto pero escueto.** Dice "el retardo de encolamiento depende del nivel de congestión del router", y **eso es todo lo que hay que decir para el punto**. Lo que suma es encadenar: congestión → cola más larga → **retardo total** ↑, y si se desborda → **pérdida**. La oficial no menciona las pérdidas.
5. **No dice que los otros tres componentes NO dependen de la congestión.** Es el complemento del apartado b) y es lo que demuestra que entendiste la estructura.

Mi respuesta completa (para los 10 puntos)

Los cuatro componentes:

```
d_end2end = d_proc + d_cola + d_transmisión + d_propagación

1. RETARDO DE PROCESAMIENTO  (d_proc)
   Qué: el tiempo que tarda el nodo en procesar el paquete entre
   que entra por una interfaz y sale por otra: verificación de
   errores (CRC o paridad), lectura de la cabecera, decremento
   del TTL y recálculo del checksum, y la búsqueda del enlace de
   salida (enrutamiento: longest prefix match).
   Depende de: la velocidad del procesador y del bus, y del ancho
   de banda del fabric.
   Típico: ~1 µs.  DESPRECIABLE en cualquier enlace de >1 km.

2. RETARDO DE ENCOLAMIENTO  (d_cola)
   Qué: la espera en la cola del enlace de SALIDA antes de que le
   toque ser transmitido.
   Depende de: el tráfico que hay por delante en la cola.
   Fórmula, para un paquete en la posición k de una cola de
   paquetes de L bits: d_cola = k · L / R_salida.
   Puede ser CERO (si el enlace está libre) y NO TIENE COTA
   SUPERIOR.  Es EL ÚNICO COMPONENTE VARIABLE.

3. RETARDO DE TRANSMISIÓN  (d_transmisión)
   Qué: el tiempo que tarda el emisor en poner el paquete entero
   en el cable, del primer bit al último.
   Fórmula: d_trans = L / R
     L = longitud del paquete EN BITS
     R = velocidad de transmisión del enlace (bps), es decir, su
         ancho de banda.
   Depende de: L y R.  NO del medio, NO de la distancia.
   ⚠️ Es el componente que cambia al cambiar el tamaño del paquete.

4. RETARDO DE PROPAGACIÓN  (d_propagación)
   Qué: el tiempo que tarda la SEÑAL en recorrer el medio físico.
   Fórmula: d_prop = d / s
     d = longitud del enlace físico, EN METROS
     s = velocidad de propagación en el medio, EN m/s
         (≈2·10⁸ m/s en cobre y fibra; ≈3·10⁸ en vacío)
   Depende de: d y s.  NO del paquete, NO del ancho de banda.
   ⚠️ Es el componente que NO cambia al cambiar el tamaño del
      paquete.

_____________________________________________________________

CUÁLES SON LOS MÁS SIGNIFICATIVOS

Los dos SIGNIFICATIVOS son d_transmisión y d_propagación:
 · Son los únicos dos que se pueden CALCULAR con los datos del
   enunciado (L, R, d, s).
 · Son los que DOMINAN: en un enlace de 100 km a 1 Gbps con un
   paquete de 1500 bytes, d_trans = 12 µs y d_prop = 500 µs, y
   los otros dos son despreciables frente a eso.

d_proc es despreciable: del orden de 1 µs, contra los ms de
propagación de cualquier enlace interurbano.
d_cola es el más relevante OPERATIVAMENTE (es el que la
congestión mueve), pero NO es calculable a priori.
```

a) Las diferencias, con las unidades:

ANCHO DE BANDA (B o R) - Unidad: bits por SEGUNDO (bps) - Qué es: la CAPACIDAD MÁXIMA de transferencia del enlace - De qué depende: del EQUIPO (los puertos del switch o router, el transceiver, la

modulación): Ethernet de 1 Gbps, STM-1 de 2,048 Mbps, 100 GbE. - NO depende de: el medio físico ni de la longitud del enlace - Aparece en: d_transmisión = L / R

VELOCIDAD DE PROPAGACIÓN (s) - Unidad: METROS por SEGUNDO (m/s) - Qué es: la rapidez con que viaja la SEÑAL por el medio - De qué depende: del MEDIO FÍSICO (la física del material): cobre $\approx 2 \cdot 10^8$ m/s, fibra $\approx 2 \cdot 10^8$ m/s, vacío $3 \cdot 10^8$ m/s. - NO depende de: el equipo, el ancho de banda ni la longitud - Aparece en: d_propagación = d / s

CONCLUSIÓN: son magnitudes de UNIDADES DISTINTAS (bps frente a m/s) que dependen de COSAS DISTINTAS (el equipo frente al medio). Duplicar el ancho de banda reduce d_transmisión a la mitad y NO cambia d_propagación. Un enlace de 1 Gbps no es 50 veces más rápido que uno de 100 Mbps: es 50 veces más ANCHO.

```
**b) En qué componente influye la congestión:**
```

La congestión influye SÓLO en el RETARDO DE ENCOLAMIENTO (d_cola).

Porque es el único componente que depende del estado de la cola, y la congestión ES el estado de la cola. Los otros tres dependen solo de L, R, d, s y del procesador, que no cambian con la carga.

La cadena completa: más tráfico ofrecido → las colas se agrandan → d_cola AUMENTA → el retardo total AUMENTA → y si el buffer se DESBORDA, el paquete se descarta → PÉRDIDAS

Es decir: la congestión se MANIFIESTA en retardo de cola y en pérdidas, que son los dos únicos síntomas observables. Nunca modifica d_transmisión, d_propagación ni d_proc.

```
## 3.2 ⭐ Examen 2011-03, Pregunta 2 (6 puntos) — congestión, pérdidas y retardo

> **Fuente:** Examen `sirc1103` — 21 de febrero de 2011, Pregunta 2 (6 puntos).
> **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

### Enunciado (texto del examen)

> Dada una red de conmutación de paquetes, explique los conceptos de congestión, pérdidas y retard

### Solución oficial (completa)

> **Congestión:** La congestión está asociada a la escasez, durante un período de tiempo, de los r
>
> **Pérdida:** Los paquetes se encolan en buffers en los routers antes de ser enviados por el enla
>
> **Retardo:** Si un paquete encuentra lugar en el buffer pero su envío por el enlace de salida se

### Análisis de la solución oficial

**Lo que está muy bien:**

1. **La definición de congestión tiene los cuatro elementos correctos**: escasez, recursos compart

2. **Dice explícitamente que la congestión "se manifestará en pérdidas y/o retardos"**, y que los

3. **La definición de pérdida da el mecanismo exacto**: "los paquetes se encolan en buffers antes

4. **La definición de retardo menciona los cuatro componentes en un párrafo**, con los ejemplos co

**Lo que falta o se puede mejorar:**

1. **No da las fórmulas.** La oficial describe los componentes pero no escribe `L/R` ni `d/s`. La
2. **No dice que la congestión "no existe" sin compartición.** Dice "recursos compartidos", que es
3. **La definición de retardo no distingue el retardo "de congestion" del "de física".** Podría hal
4. **No menciona que una pérdida en la capa 3 es irreversible.** Un paquete descartado en un route
5. **No menciona que hay pérdidas por otras causas** (checksum, TTL, filtros), que es lo que impid

### Mi respuesta completa (para los 6 puntos)

**Congestión.** Es la **escasez, durante un período de tiempo, de los recursos compartidos** neces

Tres precisiones que hay que hacer:

- **Es un fenómeno de compartición.** Si el recurso fuera dedicado, no habría congestión. La conge
- **Es temporal.** Una ráfaga de 200 ms no es congestión; 200 ms de saturación sostenida sí. Por e
- **No es observable directamente.** La congestión es un estado interno de la red; lo que se obser

**Pérdida.** Un paquete **se descarta cuando no encuentra lugar en un buffer**. Los paquetes se en

Hay que agregar dos precisiones:

- **Una pérdida de capa 3 es irreversible.** El paquete descartado no vuelve: **IP no tiene mecani
- **No toda pérdida es por congestión.** También hay checksum incorrecto (error del medio), TTL qu

**Retardo.** Es el tiempo que tarda un paquete en ir de un nodo al siguiente, y tiene cuatro compo

**El vínculo entre los tres, que es lo que hay que argumentar:**
```

La congestión es la CAUSA; el retardo y la pérdida son los SÍNTOMAS. Y los dos síntomas vienen del mismo mecanismo: la cola.

┌──────────────

```
**Interpretación y el punto del ejercicio:** acá **el `d_trans` DOMINA** (86 % del total), a difer

El criterio: el `d_trans` importa cuando el enlace es **lento** o el paquete es **grande**. El `d_

</details>

**2.** ¿Por qué la solución oficial del Examen 2009-12 no responde la parte de "señale cuáles son

<details><summary>Respuesta</summary>

Porque es un detalle que la cátedra da por obvio, pero **la pregunta lo pide explícitamente** y po

**La respuesta correcta: los más significativos son `d_transmisión` y `d_propagación`.**

Las razones son dos:

1. **Son los únicos calculables** con los datos que da el enunciado. `d_proc` depende del modelo d
2. **Son los que dominan en términos absolutos**: `d_prop` va de milisegundos a cientos de milisegu

`d_cola` es el más relevante **operativamente** — es el que la congestión mueve y el que produce e

</details>

**3.** Dos paquetes del mismo tamaño viajan por el mismo enlace. Uno va por cobre y el otro por fi

<details><summary>Respuesta</summary>

**Solo cambia la `d_propagación`, y muy poco.**
```

Cobre: s = $2{,}00 \cdot 10^8$ m/s Fibra: s = $2{,}04 \cdot 10^8$ m/s d = 1000 km = 10^6 m

d_prop(cobre) = $10^6 / 2{,}00 \cdot 10^8$ = 5,000 ms d_prop(fibra) = $10^6 / 2{,}04 \cdot 10^8$ = 4,902 ms

diferencia = 0,098 ms = 98 µs (un 2 %)

```
**Lo que NO cambia:**

- `d_transmisión = L/R`: el medio no entra en la fórmula. Si el enlace es de 1 Gbps por cobre o po
- `d_proc`: depende del equipo, no del medio.
- `d_cola`: depende de la carga.

**La lección: la fibra no se compra por latencia.** Se compra por atenuación (26 dB/km contra 20),

El mismo argumento del Ejercicio 2.2 de la Clase 1, y el mismo número: **~0,1 ms de diferencia en

</details>

**4.** ¿Qué es el BDP y por qué importa? ¿Es un concepto nuevo o es antiguo?

<details><summary>Respuesta</summary>

**BDP** = *Bandwidth-Delay Product* = `R × RTT`, en bits. Es **la cantidad de datos que pueden esta

**No es nuevo.** Aparece en la literatura deinale 1980s (el modelo de "ventana de secado" de la In

**Por qué importa:**
```

Throughput máximo alcanzable = min(R , ventana / RTT)

```
Es decir: o estás limitado por el **ancho de banda del enlace**, o estás limitado por tu **ventana

**Ejemplo:** enlace de 100 Mbps, RTT de 40 ms. BDP = `10⁸ × 0,04 = 4·10⁶ bits = 500 kB`. Con una v

**La consecuencia de diseño:** `Window Scale` (`RFC 7323`, 2014) es lo que hace posible aprovechar

**Y el ejemplo extremo del BDP:** un enlace de 10 Gbps de un operador, con 10 000 flujos TCP activ

</details>

**5.** ¿Por qué el jitter rompe el audio pero no rompe la descarga de un archivo?

<details><summary>Respuesta</summary>

Porque **la descarga de un archivo no tiene ritmo**, y el audio sí.

**Descarga de archivo:**
```

Retardo 1 = 400 ms → el usuario espera 400 ms Retardo 2 = 80 ms → el usuario espera 80 ms No hay diferencia perceptible. El archivo llega entero igual. El retardo total importa, y 80 vs 400 ms es 1,6 % de la descarga.

```
**Audio (un códec que reproduce cada 20 ms):**
```

t=0 t=20 t=40 t=60 t=80 t=100 t=120 │ │ │ │ │ │ │ │P1─────────┤ │ │ │ │ llega en t=80, perfecto │ │P2─────────┤ │ │ │ │ llega en t=110, tarde │ │ │P3─────────┤ │ │ │ llega en t=100, tarde │ │ │ │P4─────────┤ │ │ llega en t=138, tardísimo │ │ │ │ │ │ audio: ▓▓▓

PRECOZ.

```
**La demostración que hay que saber de memoria:** si el buffer tiene `n` paquetes y la tasa de lle
```

L/R = 1500/10^8 = 15 µs por paquete diferencia entre el primero y el último = 1000 × 15 µs = 15 ms

```
Y **el paquete que llega justo cuando el buffer se llena tiene `d_cola = 15 ms`; el que llega un p

</details>

**7.** En un enlace de 1 Gbps con 3000 paquetes en cola, ¿cuánto tarda en salir el último?

<details><summary>Respuesta</summary>
```

R = 10^9 bit/s L = 1500 bytes = 12 000 bits

d_cola(último) = k × L/R = 3000 × 12 000 / 10^9 = 36 000 000 / 10^9 = 0,036 s = 36 ms

Y d_trans (su propia transmisión) = 12 000/10^9 = 12 µs = 0,012 ms

d_total del último = 36 + 0,012 = 36,012 ms

```
**Interpretación:** 36 ms de cola en un enlace de 1 Gbps. Un `ping` a un host a un salto de distan

**Y si el buffer fuera de 3 GB:** `3000 × 1000 = 3 000 000 paquetes × 15 µs = 45 segundos`. Un buf

</details>

**8.** Explique la diferencia entre latencia y RTT, y por qué `traceroute` puede mentir sobre el c

<details><summary>Respuesta</summary>

**Latencia** = el tiempo de **un viaje de ida**. **RTT** = el tiempo de **ida y vuelta**, que es ~
```

RTT = d_ida/s + cola_ida + d_vuelta/s + cola_vuelta + proc latencia ≈ RTT/2 ⚠️ SOLO si d_ida = d_vuelta

```
**Por qué `traceroute` miente, textual de la solución oficial del Examen 2011-03 P3:**

> *"Dado que el camino que siguen los paquetes de ida y vuelta de una comunicación sobre Internet

**El caso clásico:** el tráfico de ida pasa por un enlace de fibra de 2 ms y el de vuelta por uno
```

latencia real de ida = 2 ms RTT = 2 + 600 = 602 ms latencia estimada = 602/2 = 301 ms ⚠️ contra 2 ms real ```

El error es de un factor 150. Y por eso `traceroute` **no sirve para medir latencia**: sirve para **descubrir el camino de ida**.

Y la otra trampa del `traceroute`, que la oficial también menciona:

> *"pueden estar filtradas (por razones de seguridad y/o performance) [...] por lo que el no obtener respuesta desde los nodos intermedios o desde el nodo final no nos permite concluir que haya algún problema en la red o en el destino."*

Un asterisco (`*`) en la salida de `traceroute` significa **"no obtuve respuesta"**, que puede ser: el nodo no respondió, el camino de vuelta es otro, o **el router filtró el ICMP de vuelta**. **No significa que el nodo esté caído.**

🔗 Referencias

Exámenes - Examen `sirc0912` (18-dic-2009) — **P1 (10 pts): componentes del retardo, ancho de banda vs. velocidad de propagación, congestión** (esta clase) - Examen `sirc1103` (21-feb-2011) — **P2 (6 pts): congestión, pérdidas y retardo** · **P5 (10 pts): retardo end-to-end con 3 enlaces y 2 conmutadores** → **Clase 6** - Examen `sirc1103` — P3 (6 pts): `traceroute`, los dos sabores (UDP e ICMP) y por qué el camino de vuelta puede ser otro - Examen `sirc1002` (26-feb-2010) — P5 (10 pts): jitter vs. retardo - Examen `sirc0903` (20-feb-2009) — P5 (6 pts): full duplex vs. half duplex - Examen `sirc0612` (26-dic-2006) — P1 (5 pts): modelo OSI y capa física

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., **cap. 1 §1.5** "El retardo en las redes de paquetes" (pág. ~26 a ~34): los cuatro componentes con el diagrama del router y los números - Tanenbaum & Wetherall, **cap. 1 §1.2** "Tipos de red" — la comparación LAN/WAN con los números de latencia - Tanenbaum & Wetherall, **cap. 1 §1.3** — store-and-forward y conmutación de circuitos frente a paquetes - Kurose & Ross, *Computer Networking*, cap. 1 §1.7 "The Network Core: Delay and Loss" — la derivación de `L/R` y `d/s` con los cuatro Diagramas

RFCs - `RFC 9293` — *TCP* (2022, *Internet Standard*): el campo *window* y el control de flujo que **es** una manera de administrar el BDP - `RFC 7323` — *TCP Extensions for High Performance* (2014): ***Window Scale***, sin el cual el throughput de larga distancia está limitado a ~20 Mbps - `RFC 5681` — *TCP Congestion Control* (2009): los cuatro algoritmos que reaccionan a la **pérdida** como señal de congestión - `RFC 9438` — *CUBIC* (2023, *Standards Track*): el default global, que ya no es AIMD puro - `RFC 3168` — *ECN* (2001): marcar en vez de descartar · `RFC 9768` **AccECN** (2026) · `RFC 9331` L4S (2023) - `RFC 5437` — *Packet Scheduling and Dropping* (2009): los AQMs que controlan el retardo de cola (CoDel, RED, PIE)

Clase siguiente: 6 — Taller de retardos (todo resuelto)

Clase 6

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 Las cuatro fórmulas, para copiar
1.2 La tabla de unidades: donde se pierden los puntos
1.3 Las tres reglas de sensibilidad del taller
1.4 Store-and-forward vs. cut-through
2.1 Ejercicio 1 (guía) — Los cuatro componentes de un paquete en un solo enlace
2.2 Ejercicio 2 (guía) — Las tres reglas de sensibilidad, con números
2.3 Ejercicio 3 (★ de examen) — 3 enlaces, 2 conmutadores, con números
2.4 Ejercicio 4 — Cinco routers con store-and-forward
2.5 Ejercicio 5 — *Delay × bandwidth*: 8 MB por un enlace de 1 Mbps a 1000 km
2.6 Ejercicio 6 — RTT de ida y vuelta, latencia y jitter
2.7 Ejercicio 7 — Store-and-forward vs. cut-through, con números que se ven
2.8 Ejercicio 8 — Satélite contra fibra: por qué ancho de banda no es latencia
2.9 Ejercicio 9 — Tres enlaces heterogéneos, cuenta larga
2.10 Ejercicio 10 — Comparar dos topologías con la misma distancia total
2.11 Ejercicio 11 — *Delay × bandwidth* de una descarga grande
2.12 Ejercicio 12 — Sensibilidad de punta a punta sobre el problema de examen
3.1 ⭐ ⭐ Examen 2011-03, Pregunta 5 (10 puntos) — retardo end-to-end
3.2 Recordatorio — dónde están los otros problemas de retardo
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: A (Fundamentos) · **Semana:** 2 · **Duración estimada:** 120 min **Correlativa:** Clase 5 (el retardo en las redes de paquetes) · **Sigue con:** Clase 7 (Nyquist, Shannon y la capa física) **Material de referencia:** Tanenbaum cap. 1 §1.5 · Exámenes `sirc1103` (P5), `sirc0912` (P1), `sirc1002` (P5) · RFC 7323, RFC 9000

> **Esta clase es un taller puro.** No hay teoría nueva: los cuatro componentes ya están en la Clase 5. Acá hay **aritmética con la cuenta a la vista**, y el **Ejercicio 3**, que es el problema de retardos del curso.

🎯 Al terminar esta clase vas a poder

1. **Copiar de memoria** la caja de las cuatro fórmulas, con las unidades de cada variable.
2. **Ejecutar un cálculo de retardo** de punta a punta con `n` enlaces y `k` conmutadores, escribiendo cada término.
3. **Aplicar las tres reglas de sensibilidad**: qué pasa si duplico `L`, `R` o `d`.
4. **Contar bien los** `d_proc`: por qué hay `n` términos de transmisión y solo `n − 1` de procesamiento.
5. **Explicar por qué un conmutador que reenvía bit a bit no aporta retardo de transmisión**, y nombrar ese modelo.
6. **Calcular el *bandwidth-delay product*** y decidir si una ventana de TCP alcanza o no.
7. **Distinguir latencia de RTT** y calcular el **jitter**.
8. **Resolver entero** el Examen `2011-03 Pregunta 5` (10 pts).

> **Prerequisito del examen:** `2011-03 P5` (10 pts — retardo end-to-end con 3 enlaces y 2 conmutadores, y el caso cut-through), `2009-12 P1` (10 pts, Clase 5 §3.1), `2011-03 P2` (6 pts, Clase 5 §3.2), `2010-02 P5a` (8

> pts — jitter vs. retardo).

PARTE 1 — LA CAJA DE FÓRMULAS

1.1 Las cuatro fórmulas, para copiar

Esto es lo único que tenés que saber de memoria de la parte de retardo:

```
┌──────────────────────────────────────────────────────────────────┐
│   d_transmisión = L / R      L en BITS,   R en bps del enlace    │
│   d_propagación = d / s      d en METROS, s en m/s del medio     │
│   d_cola        = lo que toque    el único componente VARIABLE   │
│   d_proc        ≈ despreciable   CRC + cabecera + elegir puerto  │
│                                                                  │
│   d_end2end = Σ (d_proc + d_cola + L/R + d/s)                    │
│                                                                  │
│   n enlaces  ⟶  n · d_transmisión  +  n · d_propagación          │
│   k routers  ⟶  k · d_proc         +  k · d_cola                 │
│   En una línea sin ramales:  k = n − 1                           │
│   ⚠️ d_proc va (n − 1) VECES, no n veces. Ese es el error del P5'11.│
└──────────────────────────────────────────────────────────────────┘
```

1.2 La tabla de unidades: donde se pierden los puntos

El error más caro del curso no es conceptual, es de **unidad**:

Símbolo	Qué es	Unidad	Error típico
`L`	longitud del paquete	**BITS**	poner bytes: factor **8**
`R`	velocidad de transmisión = ancho de banda del enlace	**bit/s**	—
`d`	longitud del enlace físico	**METROS**	poner km: factor **1000**
`s`	velocidad de propagación en el medio	**m/s**	—
`d_cola`	espera en la cola	s / ms	el único no calculable

```
bytes → bits : × 8     km → m : × 1000     µs → ms : ÷ 1000
cobre / fibra óptica : s ≈ 2 · 10⁸ m/s   ← el 95 % de los problemas
aire / espacio libre : s ≈ 3 · 10⁸ m/s   (y un satélite va por aire)
```

1.3 Las tres reglas de sensibilidad del taller

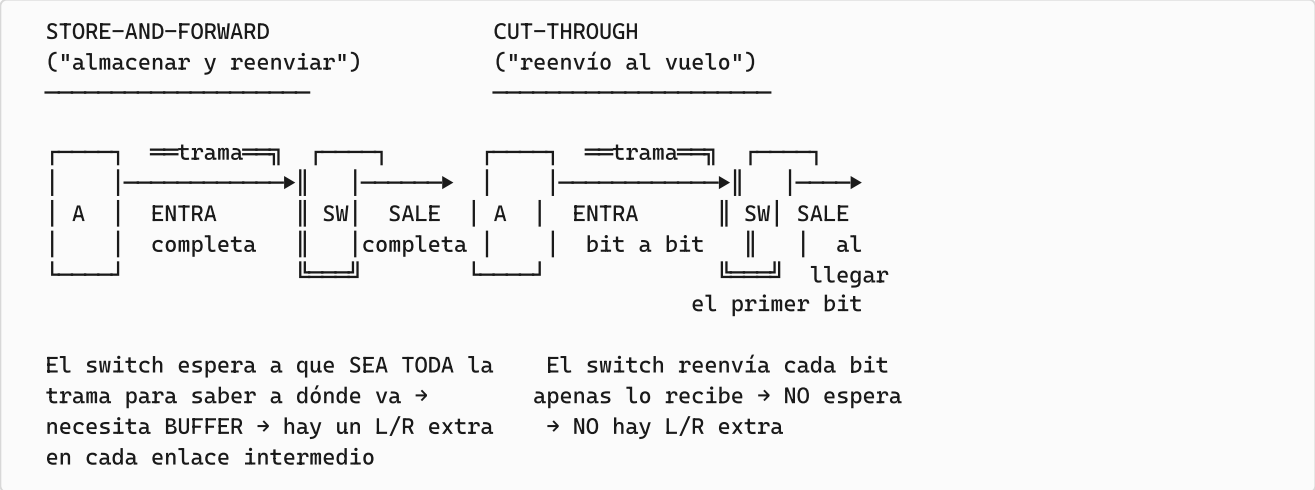
De mirar las fórmulas salen las tres reglas, y con ellas se responde la mitad de los exámenes de esta parte:

Si duplico…	¿Qué cambia?	¿Qué NO cambia?
la longitud `L` del paquete	**solo** `d_transmisión` (se duplica)	`d_propagación`, `d_proc`, `d_cola`
el ancho de banda `R`	**solo** `d_transmisión` (se reduce a la **mitad**)	`d_propagación`
la distancia `d`	**solo** `d_propagación` (se duplica)	`d_transmisión`

> **Las dos frases que hay que dejar escritas en el examen:** · "El ancho de banda **nunca** cambia la propagación." · "La distancia **nunca** cambia la transmisión."

1.4 Store-and-forward vs. cut-through

El corazón del Ejercicio 3, mostrado antes y desarrollado después:



El nombre de cut-through: el switch no espera a que la trama quede entera en un buffer, la deja *cuts through*. También se lo llama **cut-through forwarding**.

Lo que cuesta a cambio: si el enlace de salida está ocupado el switch **no puede descartar** (nunca vio el paquete entero), así que **encola antes**, y por eso **propaga la congestión más rápido**: es el *congestion crunch*, y es el motivo por el que los conmutadores de núcleo de las redes de carrier siguen usando store-and-forward.

La tabla que resume la clase entera:

Modelo	términos `d_proc`	términos `L/R`	`Σ d_i/s_i`
Store-and-forward	`k` (uno por conmutador)	`n` (uno por enlace)	`n`
Cut-through	`0` (idealizado)	**1** (se pisan)	`n`

PARTE 2 — PRÁCTICA RESUELTA

> **Cómo se usa esta parte.** Hacé cada ejercicio en papel con la cuenta escrita, sin calculadora al principio. Los ejercicios 1, 2 y 3 son los que tienen que salirte con tinta y sin dudar.

2.1 Ejercicio 1 (guía) — Los cuatro componentes de un paquete en un solo enlace

Datos. Un paquete de **1500 bytes** (el MTU de Ethernet) viaja por un enlace de cobre de **1 Gbps** de **100 km**. El switch tarda **3 µs** en procesar. Sin cola (hipótesis).

a) Calculá los cuatro componentes. **b)** ¿Cuál de los cuatro manda?

Paso 0 — Convertir TODO a las unidades correctas antes de calcular.

```
L = 1500 bytes × 8 bits/byte = 12 000 bits      ⚠ EN BITS
R = 1 Gbps   = 10⁹ bit/s
d = 100 km   × 1000 m/km   = 100 000 m = 10⁵ m   ⚠ EN METROS
s = 2·10⁸ m/s (cobre)
```

Paso 1 — Retardo de transmisión (el tiempo de meter los 12 000 bits en el cable, uno por uno; **no depende de los 100 km**: con un enlace de 1 m sería el mismo):

```
d_trans = L / R = 12 000 bits / 10⁹ bit/s = 1,2·10⁻⁵ s = 12 µs = 0,012 ms
```

Paso 2 — Retardo de propagación (lo que tarda la **señal** en recorrer los 100 km; es el mismo para un paquete de 40 bits que para uno de 15 000):

```
d_prop = d / s = 10⁵ m / 2·10⁸ m/s = 5·10⁻⁴ s = 0,5 ms
```

Paso 3 — Retardo de cola y de procesamiento.

```
d_cola = 0
⚠ Sin dato del enunciado, la hipótesis es 0 y se DECLARA. Si el switch
   transmite 1000 paquetes/s y llegan 1000 paquetes/s, el sistema está EN EL
   LÍMITE: la cola CRECE linealmente y no tiene cota superior. Por eso el
   enunciado tiene que declarar la hipótesis.

d_proc = 3 µs = 0,003 ms   (CRC + leer la cabecera + buscar el puerto de salida)
```

Paso 4 — Total y reparto.

```
d_total = d_proc + d_cola + d_trans + d_prop
        = 3 µs + 0 + 12 µs + 500 µs = 515 µs = 0,515 ms

Componente   valor       % del total
──────────────────────────────────────
d_prop       0,500 ms     97,09 %   ←── DOMINA
d_trans      0,012 ms      2,33 %
d_proc       0,003 ms      0,58 %
d_cola       0              0,00 %
──────────────────────────────────────
TOTAL        0,515 ms    100,00 %
```

b) La respuesta, y la interpretación operativa que la cierra:

> En un enlace de **1 Gbps a 100 km** el retardo lo domina la **propagación** (97 %): el enlace no es lento, es **largo**. Subir `R` de 1 a 10 Gbps baja el total de 0,515 a 0,504 ms (mejora el 2 %); anular `d_proc` gana el 0,6 %. **Ninguna mejora de hardware sirve**: la única palanca que queda es acortar la distancia.

⚠️ Errores típicos del ejercicio 1

- **Usar 1500 en vez de 12 000** (factor 8) o **100 en vez de 100 000** (factor 1000).
- **Inventarse una `d_cola` sin dato:** la hipótesis es 0 y se declara.
- **Sumar `d_proc` una vez por enlace:** con 3 enlaces hay **dos** conmutadores.

2.2 Ejercicio 2 (guía) — Las tres reglas de sensibilidad, con números

Datos. Sin cola, paquete de **1000 bytes**, enlace de **10 Mbps**, **50 km** de cobre, `d_proc = 2 µs`.

```
L = 1000 bytes = 8000 bits      R = 10⁷ bit/s
d = 50 km = 5·10⁴ m            s = 2·10⁸ m/s
d_proc = 2 µs                   d_cola = 0

d_trans = 8000/10⁷ = 8·10⁻⁴ s = 800 µs
d_prop  = 5·10⁴/2·10⁸ = 2,5·10⁻⁴ s = 250 µs
d_total = 2 + 0 + 800 + 250 = 1052 µs = 1,052 ms        ← BASE

Variante a: L = 2000 bytes = 16 000 bits
   d_proc 2 µs (igual) · d_cola 0 (igual) · d_prop 250 µs (IGUAL)
   d_trans = 16 000/10⁷ = 1600 µs        (SE DUPLICA)
   d_total = 2 + 0 + 1600 + 250 = 1852 µs = 1,852 ms    (+76 %)

Variante b: R = 20 Mbps
   d_proc 2 µs (igual) · d_cola 0 (igual) · d_prop 250 µs (IGUAL)
   d_trans = 8000/2·10⁷ = 400 µs         (SE REDUCE A LA MITAD)
   d_total = 2 + 0 + 400 + 250 = 652 µs = 0,652 ms      (−38 %)

Variante c: d = 100 km = 10⁵ m
   d_proc 2 µs (igual) · d_cola 0 (igual) · d_trans 800 µs (IGUAL)
   d_prop  = 10⁵/2·10⁸ = 500 µs          (SE DUPLICA)
   d_total = 2 + 0 + 800 + 500 = 1302 µs = 1,302 ms     (+24 %)
```

La tabla para el examen:

Qué duplico	`d_proc`	`d_cola`	`d_trans`	`d_prop`	`d_total`	Cambio
nada (base)	2 µs	0	800 µs	250 µs	**1052 µs**	—
`L` (1000 → 2000 B)	2 µs	0	**1600 µs**	250 µs	**1852 µs**	**+76 %**
`R` (10 → 20 Mbps)	2 µs	0	**400 µs**	250 µs	**652 µs**	**−38 %**
`d` (50 → 100 km)	2 µs	0	800 µs	**500 µs**	**1302 µs**	**+24 %**

Las tres frases, textuales:

> - **El ancho de banda nunca cambia la propagación.** Duplicar `R` dejó `d_prop` exactamente en 250 µs, porque `d/s` no contiene `R`.
> - **La distancia nunca cambia la transmisión.** Duplicar `d` dejó `d_trans` exactamente en 800 µs, porque `L/R` no contiene `d`.
> - **El tamaño del paquete nunca cambia la propagación.** La señal es la misma por más bits que lleve encima.

Corolario: acá `d_trans` (800 µs) es mayor que `d_prop` (250 µs): es un enlace **corto y lento**, y por eso mejoró `R` y no `d`. En un enlace de backbone, con `d_prop` de 25 ms contra `d_trans` de 1,2 µs, el resultado se da vuelta.

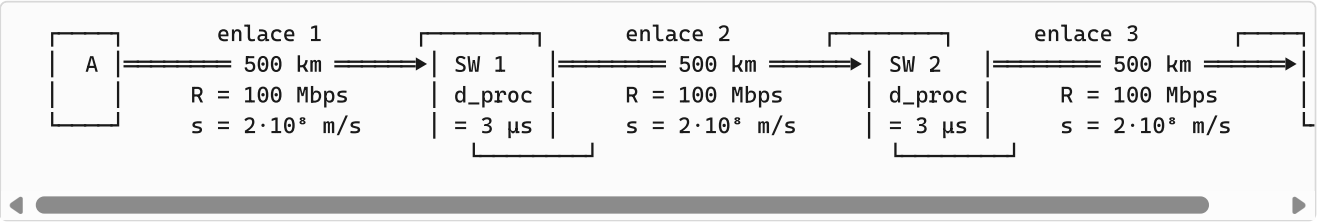
⚠️ Errores típicos del ejercicio 2

- **Meter `R` en la fórmula de propagación:** es el error conceptual más caro del curso (Pregunta 1a del Examen 2009-12, en la Clase 5 §3.1).
- **Creer que "más `R` siempre es mucho mejor":** acá mejoró 38 % solo porque `d_trans` era el 76 % del total. Si la cola fuera el 99 %, no mejoraría nada.

2.3 Ejercicio 3 (★ de examen) — 3 enlaces, 2 conmutadores, con números

Este es el **Examen 2011-03 P5** con números, para verlo antes de la versión simbólica del §3. **Misma estructura, mismo error típico.**

Datos. Paquete de **1500 bytes** de A a B por tres enlaces de cobre unidos por dos conmutadores. Los tres enlaces: **100 Mbps**, **500 km**. `d_proc = 3 µs`. Sin cola.



a) Store-and-forward.

```
Paso 1 — datos:  L = 1500 bytes = 12 000 bits
                 R = 10⁸ bit/s (los tres)  d = 500 km = 5·10⁵ m (los tres)
                 s = 2·10⁸ m/s (los tres)  d_proc = 3 µs POR CONMUTADOR, y hay DOS

Paso 2 — los términos que se repiten
   d_trans (uno) = 12 000/10⁸ = 1,2·10⁻⁴ s = 120 µs        → ×3 =  360 µs
   d_prop  (uno) = 5·10⁵/2·10⁸ = 2,5·10⁻³ s = 2500 µs      → ×3 = 7500 µs
   d_proc        = 3 µs                                      → ×2 =    6 µs   ⚠ 2, NO 3
   d_cola = 0

Paso 3 — la cuenta, término por término
   d_total = (  3 + 0 + 120 + 2500)     enlace 1
           + (  3 + 0 + 120 + 2500)     enlace 2
           + (  0 + 0 + 120 + 2500)     enlace 3  (el host B no procesa)
           ──────────────────────────────────────────────────
           = 2623 + 2623 + 2620 = 7866 µs = 7,866 ms

Término                   valor      % del total
──────────────────────────────────────────────────
3 × d_propagación         7,500 ms    95,35 %   ⟵ DOMINA
3 × d_transmisión         0,360 ms     4,58 %
2 × d_procesamiento       0,006 ms     0,08 %
d_cola                    0            0,00 %
──────────────────────────────────────────────────
TOTAL                     7,866 ms   100,00 %
```

b) Cut-through: los conmutadores no almacenan y `d_proc = 0`.

Por qué desaparecen los `L/R` de los enlaces intermedios — el razonamiento central:

```
STORE-AND-FORWARD: el switch 1 tiene que esperar a que ENTRE la trama
                   COMPLETA para mirar la MAC destino y elegir el puerto.
                   Recién ahí arranca por el enlace 2. Como R es el mismo,
                   ese L/R se paga DOS veces: entrar y salir. Idem en el
                   switch 2. De ahí los 3 L/R.

CUT-THROUGH:       el switch ve la MAC destino en los primeros bytes y
                   arranca a emitir por el enlace 2 MIENTRAS sigue
                   recibiendo por el 1. Como R_entrada = R_salida, el
                   ritmo de salida es el de entrada: el switch es
                   TRANSPARENTE.

                   ⇐ El medio se comporta como un solo enlace largo:
                      los tres L/R se PISAN en vez de se SUMAR.
```

La cuenta y la comparación:

```
d = 1·(L/R) + Σ d_i/s_i = 12 000/10⁸ + 3·(5·10⁵/2·10⁸) = 120 + 7500 = 7620 µs

                        store-and-fwd   cut-through (b)
nº de términos L/R              3               1     ⟵ la diferencia
nº de términos d_proc           2               0
Σ d_i/s_i                       3               3     (igual)
Retardo total               7866 µs         7620 µs
──────────────────────────────────────────────────────────────────
AHORRO = 7866 − 7620 = 246 µs = 2·(L/R) + 2·d_proc = 240 + 6      ✓

⚠ Ojo con la proporción: el ahorro es del 3,1 %, NO del 33 %, porque
   acá la propagación (7500 µs) aplasta todo lo demás. Con enlaces muy
   cortos y lentos el ahorro es mucho mayor (ver §2.7).
```

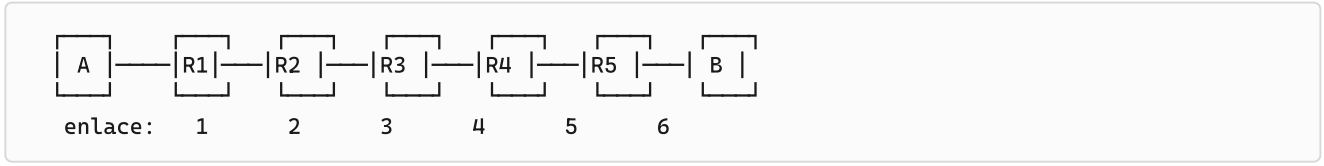
La línea de tiempo, que es la explicación visual del por qué: con store-and-forward, a `t = 120 µs` la trama completa entró al switch 1 (que recién ahí puede mirar la MAC destino y arrancar el enlace 2), a `t = 240 µs` entró completa al switch 2, y a `t = 7866 µs` llega el último bit a B. Con cut-through, a `t = 120 µs` **el último bit ya pasó por el switch 1, por el enlace 2 y por el switch 2**, y llega a B a `t = 7620 µs`. **El "embudo" se vuelve una "tubería": los `L/R` se pisan.**

⚠️ **Y si `R_entrante ≠ R_saliente`, el cut-through se degrada** a store-and-forward: la hipótesis "no hay retardo" SOLO vale si `R_entrada = R_salida` en todos los saltos (ver §2.7).

La moraleja: con cut-through el retardo bajó de 7866 a 7620 µs, un **3,1 %**. Parece poco hasta que uno se da cuenta de que **los 246 µs ahorrados son exactamente los 246 µs de "equipo"**, y que los 7500 µs restantes son **geografía**. **Ningún conmutador, por rápido que sea, puede ganar contra la distancia.**

2.4 Ejercicio 4 — Cinco routers con store-and-forward

Datos. Mensaje de **12 000 bits** de A a B atravesando **cinco routers**, o sea **seis enlaces**. Todos: `R = 1 Mbps`, `d = 100 km`, `s = 2·10⁸ m/s`. `d_proc = 2 µs`.




```
n = 6 enlaces   →  6 términos de d_trans y 6 de d_prop
k = 5 routers   →  5 términos de d_proc y 5 de d_cola     ⚠️ NO 6

Un término de cada tipo:
   d_trans = 12 000/10⁶ = 12·10⁻³ s = 12 ms
   d_prop  = 10⁵/2·10⁸  = 0,5·10⁻³ s = 0,5 ms
   d_proc  = 2 µs = 0,002 ms
   d_cola  = 0 (hipótesis)

Fórmula general:
d_end2end = n·(L/R) + k·d_proc + Σ d_i/s_i
          = 6·(12 000/10⁶) + 5·(2·10⁻⁶) + 6·(10⁵/2·10⁸)
          = 6·0,012 + 0,000010 + 6·0,0005
          = 0,072 + 0,000010 + 0,003
          = 0,075010 s = 75,010 ms

Verificación término por término:
          = 5·(0,002 + 0 + 12 + 0,5) + (0 + 12 + 0,5)
          = 5·12,502 + 12,5 = 62,510 + 12,5 = 75,010 ms     ✓ cuadra
```

Interpretación — y el resultado incómodo:

```
6 × d_transmisión      72,000 ms    95,99 %   ←— DOMINA
6 × d_propagación       3,000 ms     4,00 %
5 × d_procesamiento     0,010 ms     0,01 %
————————————————————————————————————————
TOTAL                  75,010 ms   100,00 %
```

1 Mbps es **muy lento para 100 km**, por eso manda la transmisión: con 100 Mbps, `d_trans = 120 µs` y el total baja a 3,73 ms, un factor 20. Y una observación estructural: los 5 routers aportan 10 µs en total (el 0,013

%): meter un router más y un salto más no cuesta casi nada en procesamiento, lo que cuesta es el `L/R` del enlace extra.

2.5 Ejercicio 5 — *Delay* × *bandwidth*: 8 MB por un enlace de 1 Mbps a 1000 km

Datos. Enlace de **1 Mbps** entre dos puntos a **1000 km** de fibra (`s = 2·10⁸ m/s`). Archivo de **8 MB**. Sin cola, `d_proc` despreciable.

Paso 1 — Latencia de ida y RTT.

```
d = 1000 km = 10⁶ m
d_prop (ida) = 10⁶ / 2·10⁸ = 5·10⁻³ s = 5 ms
RTT ≈ 2 × 5 ms = 10 ms = 0,010 s
   (el "≈" es porque el camino de vuelta puede ser otro, y porque hay
    d_trans y d_proc que no son cero)
```

Paso 2 — Tiempo del archivo entero.

```
8 MB = 8·10⁶ bytes (convención decimal; con MiB serían 8 388 608 B)
En bits: 8·10⁶ × 8 = 6,4·10⁷ bits = 64 000 000 bits
R = 1 Mbps = 10⁶ bit/s
t = L/R = 6,4·10⁷ / 10⁶ = 64 s
```

Interpretación: la propagación es irrelevante acá: el archivo se está mandando durante 64 s, no durante 10 ms.

Paso 3 — Los bytes en vuelo: el *bandwidth-delay product*.

```
BDP = R · RTT = 10⁶ bit/s × 0,010 s = 10⁴ bits = 10 000 bits
En bytes: 10 000/8 = 1250 bytes

 A ═══════════════════════════════════════════ B
   ←──────── 1250 bytes como máximo ────────→

 En los 10 ms que tarda el ACK en volver, salen 10⁶ × 0,010 = 10 000
 bits = 1250 bytes. Si la ventana es menor, el emisor ESPERA un ACK y el
 enlace queda VACÍO ese tiempo.
```

Paso 4 — Tres ventanas concretas (`throughput = ventana / RTT`):

Ventana	¿vs. BDP?	Throughput	Tiempo para 8 MB
500 B	< BDP	`500/0,010` = 50 000 B/s = **0,40 Mbps**	`8·10⁶/50 000` = **160,0 s**
1000 B	< BDP	`1000/0,010` = 100 000 B/s = **0,80 Mbps**	`8·10⁶/100 000` = **80,0 s**
1250 B	= BDP	`1250/0,010` = 125 000 B/s = **1,00 Mbps**	**64,0 s** ✓
2000 B	> BDP	`min(R, v/RTT)` = **1 Mbps** (el enlace es el techo)	**64,0 s**

La respuesta a (c):

> La ventana **menor que el BDP** limita el throughput a `ventana/RTT` y el enlace nunca se llena. La regla exacta es `ventana ≥ BDP = R · RTT` . Por eso existe `RFC 7323` (*Window Scale*, 2014): sin él el máximo son 65 535 bytes, y en un enlace de 100 Mbps con 12 ms de RTT eso da apenas `65 535/0,012 = 52,4`

> `Mbps`. Y para enlaces largos y rápidos el cálculo es brutal: 100 Gbps con 100 ms de RTT da `BDP = 10¹⁰ bits = 1,25 GB`, y ninguna ventana TCP del mundo llena eso.

2.6 Ejercicio 6 — RTT de ida y vuelta, latencia y jitter

Datos. Tres paquetes del mismo flujo de audio salen con 20 ms de separación. Sus RTT son **80 ms**, **95 ms** y **110 ms**.

Paso 1 — Los conceptos. Latencia = el tiempo de **un viaje de ida**; ≈ `RTT/2`, y solo si el camino de vuelta es el mismo que el de ida. **RTT** = ida y vuelta, lo que mide `ping`. **Jitter** = la **variación** del retardo entre paquetes del mismo flujo; no es el retardo, es cuánto cambia.

Paso 2 — Los números.

```
Serie de RTT: 80 ms, 95 ms, 110 ms

RETARDO MEDIO:  RTT_promedio = (80+95+110)/3 = 285/3 = 95 ms
                latencia ≈ 95/2 = 47,5 ms

JITTER, tres métricas:

(i)   Pico a pico (la que usa el jitter buffer):
            J = 110 − 80 = 30 ms

(ii)  Desviación estándar:
            σ = √( (Σ (R_i − R_prom)²) / n )
              = √( (80−95)² + (95−95)² + (110−95)² ) / 3 )
              = √( (−15)² + 0² + 15² ) / 3 = √( (225 + 0 + 225) / 3 )
              = √( 450/3 ) = √150 = 12,25 ms

(iii) Variación media absoluta:
            J = ( |80−95| + |95−95| + |110−95| ) / 3 = (15+0+15)/3 = 10 ms
```

Paso 3 — Qué le pasa al audio, y por qué 30 ms de jitter es un problema real. El códec reproduce una muestra cada 20 ms; los paquetes salen en `t = 0, 20, 40` y el reproductor tiene un retardo base de 60 ms:

```
P1 sale en t=  0, tarda  80 ms → llega en t=  80 (lo esperaba en t=  60)
P2 sale en t= 20, tarda 110 ms → llega en t= 130 (lo esperaba en t=  80)
P3 sale en t= 40, tarda  95 ms → llega en t= 135 (lo esperaba en t= 100)

██ SILENCIO ██ → CLIC en el audio a los 80, 100 y 120 ms
```

⚠️ **El problema NO es que tarden: es que lleguen a ráfagas.** Con los tres RTT iguales a 95 ms, el retardo sería de 95 ms (el doble que una llamada telefónica) y el audio sería **perfecto**. ⇒ **un retardo alto y constante es mejor que uno medio y variable.** Por eso existe el jitter buffer: mete un retardo **fijo** (30-40 ms), reordena lo que llegó tarde y rellena los huecos. El compromiso: buffer grande = más tolerancia al jitter, más latencia.

Paso 4 — La definición con las palabras del Examen 2010-02 P5.

> **Solución oficial:** *"El retardo es el tiempo que tarda un paquete desde que sale del emisor hasta que llega al receptor, mientras que el jitter es la variación de este retardo entre distintos paquetes de un mismo flujo."* Y

> el remate: "*Si todos los paquetes del flujo siguieran el mismo camino pero no hubiera congestión, podríamos mantener el retardo constante (aunque sea de varios segundos) mientras que el jitter sería mínimo o nulo.*"

Paso 5 — La tabla de sensibilidad, que es la parte b) de esa pregunta:

Escenario	retardo	jitter	¿audio?
Enlace de 5 ms, sin congestión	10 ms	0 ms	PERFECTO
Enlace de 300 ms, sin congestión	600 ms	0 ms	molesto, tolerable
Enlace de 30 ms, con colas variables	45 ms	30 ms	CLICS
Enlace de 300 ms con ECMP y 2 caminos	150 ms	12 ms	aceptable

Paso 6 — De dónde sale el jitter: de la **cola** de un router (la fuente principal: todo el jitter es `d_cola` variable), de una **pérdida con retransmisión** (el hueco dura un RTO, mínimo 200 ms en Linux), del **ECMP** y de los **caminos asimétricos**.

⚠️ Errores típicos del ejercicio 6

- **Decir "el jitter es el retardo":** no, es **la variación**. Con tres RTT de 90 ms el jitter es 0 y el retardo es 90 ms.
- **Calcular "latencia = RTT/2" sin la salvedad:** solo vale con camino simétrico (lo dice la oficial de 2011-03 P3).

⚠️ Actualización a 2026 — qué pasó con el retardo

Sigue igual — de memoria: - Los **cuatro componentes**, sus fórmulas y sus unidades. - `d_trans = L/R` con `L` en **bits**; `d_prop = d/s` con `d` en **metros**. - `d_cola` es el único componente **variable** y el único que refleja la **congestión**. - **Jitter = variación del retardo**, y viene de la `d_cola`. - **Latencia ≈ RTT/2** solo con camino simétrico.

Cambió o hay que reencuadrar: - **La `d_cola` pasó de ser un accidente a ser el objetivo de diseño.** El *bufferbloat* es hoy el problema dominante de las redes de acceso. Linux usa **FQ-CoDel** por defecto desde el kernel 3.5 (2012), que descarta **temprano** para que el retardo no explote. - **El objetivo del control de congestión pasó de "evitar la pérdida" a "evitar la cola".** `RFC 9438` (CUBIC, *Standards Track* 2023) sigue reduciendo la ventana por pérdida, pero **ECN** (`RFC 3168`) y **L4S** (`RFC 9331`, 2023) marcan con umbral bajo para conseguir latencia mínima sin descartes. - **El BDP se volvió imposible de llenar.** Un enlace continental de 100 Gbps con 100 ms de RTT tiene un BDP de **1,25 GB**. `RFC 7323` (*Window Scale*) multiplicó la ventana pero ni así alcanza: por eso **BBR** (`draft-ietf-ccwg-bbr`, *Experimental*, **nunca fue RFC**) no usa una ventana sino un modelo de *bottleneck bandwidth* × *propagation delay*. - **`d_proc` dejó de ser despreciable en algunos casos.** En un router de 100/400 Gbps la CPU corre MPLS, ACLs, ECMP y QoS en el plano de datos: `d_proc` sube a **1-10 µs**, del orden del `d_trans`. **Ojo:** lo que sigue siendo cierto es que **no depende de la carga**, y por eso no es el componente de la congestión. - **La latencia mejoró muy poco:** la fibra de nueva generación (ZOW, *zero water mark*) baja el retardo de los enlaces continentales de ~5 a ~3,5 ms por latitud, pero no lo abolió. **Ninguna mejora de hardware va contra la `d/s`.**

> **Consecuencia para el examen:** las **fórmulas y el análisis por componentes no cambiaron en 37 años**. Resolvé con los números del enunciado y, si hay lugar, sumá una línea de que hoy el objetivo es **evitar la cola**.

2.7 Ejercicio 7 — Store-and-forward vs. cut-through, con números que se ven

Datos. Paquete de **10 000 bits** (1250 bytes) por **4 enlaces** de **10 Mbps** y **20 km** unidos por **3 conmutadores**. `d_proc = 4 µs`. `s = 2·10⁸ m/s` en los cuatro.

```
Un término de cada tipo:
   d_trans = 10 000/10⁷ = 10⁻³ s = 1 ms
   d_prop  = 2·10⁴/2·10⁸  = 10⁻⁴ s = 0,1 ms
   d_proc  = 4 µs = 0,004 ms

a) STORE-AND-FORWARD
   d = 3·(0,004) + 4·(1) + 4·(0,1) = 0,012 + 4 + 0,4 = 4,412 ms

b) CUT-THROUGH con d_proc real
   d = 3·(0,004) + 1·(1) + 4·(0,1) = 0,012 + 1 + 0,4 = 1,412 ms

c) CUT-THROUGH idealizado
   d = 0 + 1·(1) + 4·(0,1) = 1,4 ms

Modelo                 a) S&F     b) CT real   c) CT ideal
──────────────────────────────────────────────────────────
términos de L/R            4            1            1
términos de d_proc         3            3            0
términos de d_prop         4            4            4
──────────────────────────────────────────────────────────
RETARDO (ms)           4,412        1,412        1,400

Ahorro (a → b) = 3,000 ms = (n−1)·(L/R) = 3·1 ms              ✓
Ahorro (b → c) = 0,012 ms (el d_proc de los 3 switches)
```

El cut-through ahorró el 68 % del retardo total — al revés que en el §2.3. La razón es que acá `d_trans` (1 ms) domina `d_prop` (0,1 ms) por 10 a 1, mientras que en el §2.3 `d_prop` (2,5 ms) domina `d_trans` (0,12 ms) por 20 a 1, y en una LAN los dos son despreciables frente a `d_proc`. **No hay respuesta universal: hay que calcular.**

El caso patológico: `R` distinto en cada enlace. Con `R₁ = 100 Mbps`, `R₂ = 10 Mbps` y `R₃ = 100 Mbps`, el switch 1 recibe a 100 Mbps y tiene que emitir a 10 Mbps: **se acumula cola en el switch 1**, cada bit que entra tarda 10 veces más en salir, y el switch se comporta exactamente como store-and-forward con `10·(L/R) = 10 ms` de espera. **Peor que store-and-forward**, y además propaga la congestión antes de tiempo.

2.8 Ejercicio 8 — Satélite contra fibra: por qué ancho de banda no es latencia

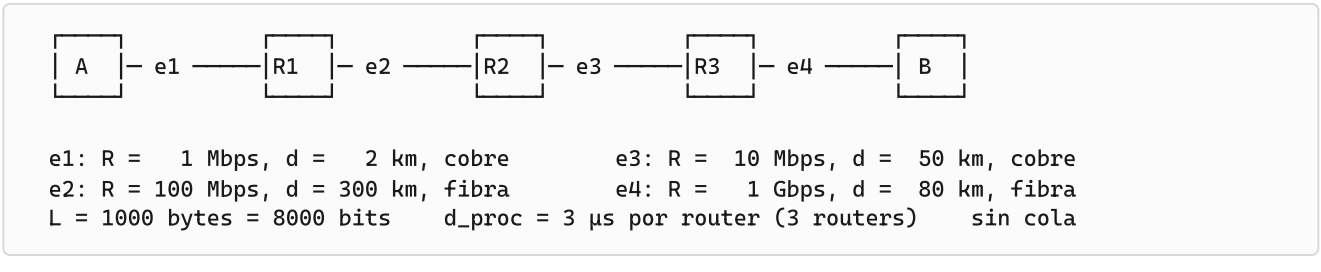
```
Opción A (satélite):  36 000 km por aire,  s = 3·10⁸ m/s,  R = 2 Mbps
Opción B (fibra):      1000 km de fibra,   s = 2·10⁸ m/s,  R = 1 Gbps
L = 12 000 bits, d_proc = 5 µs (un conmutador), sin cola

A) SATÉLITE
   d_trans = 12 000/2·10⁶ = 6 ms
   d_prop  = 3,6·10⁷/3·10⁸ = 0,12 s = 120 ms
   d_proc  = 5 µs = 0,005 ms
   d_total = 0,005 + 6 + 120 = 126,005 ms

B) FIBRA
   d_trans = 12 000/10⁹ = 12 µs = 0,012 ms
   d_prop  = 10⁶/2·10⁸ = 5 ms
   d_total = 0,005 + 0,012 + 5 = 5,017 ms
```

	A: satélite	B: fibra
`R`	2 Mbps	1 Gbps → 500× más ancho
distancia	36 000 km	1 000 km → 36× más lejos el satélite
`d_trans`	6,000 ms	0,012 ms → 500× peor el satélite
`d_prop`	120,000 ms	5,000 ms → 24× peor el satélite
TOTAL	**126,005 ms**	**5,017 ms** → 25× peor el satélite
latencia / RTT / BDP	63 ms / 252 ms / 63 kB	2,5 ms / 11 ms / 1 375 kB

La lectura al revés, que es el error conceptual del curso:

> "El satélite tiene 2 Mbps y la fibra 1 Gbps, o sea que la fibra es 500 veces más rápida." **FALSO.** Lo correcto: el satélite tiene **500 veces menos ancho de banda** y aun así la llamada tarda **25 veces más**. Porque la latencia no la manda el ancho de banda: la mandan la **distancia** y la **velocidad de propagación**.
>
> Un geoestacionario está a 36 000 km: la luz tarda 120 ms en bajar y 120 ms en volver. Por eso el satélite **sirve para streaming** (tolera 2 s de retardo) y **no para VoIP interactivo** (no tolera más de 150 ms de ida). **Éste es el mejor ejemplo del curso de por qué ancho de banda y latencia son dos cosas distintas.**
>
> La alternativa son los satélites de **órbita baja** (LEO): a 550 km de altura la propagación de ida es $550\,000/3{\cdot}10^8$ = 1,8 ms, y las constelaciones de cientos de satélites llegan a 20-50 ms. El problema se muda a la conmutación entre satélites. ⚠️ Verificar las cifras de despliegue: cambian mes a mes.

2.9 Ejercicio 9 — Tres enlaces heterogéneos, cuenta larga



Cada término escrito:

enlace	R	d	d_trans = L/R	d_prop = d/s	d_proc	subtotal
e1	10^6	$2\cdot10^3$ m	8000/10^6=8,000	$2\cdot10^3/2\cdot10^8$=0,010	0,003	8,013 ms
e2	10^8	$3\cdot10^5$ m	8000/10^8=0,080	$3\cdot10^5/2\cdot10^8$=1,500	0,003	1,583 ms
e3	10^7	$5\cdot10^4$ m	8000/10^7=0,800	$5\cdot10^4/2\cdot10^8$=0,250	0,003	1,053 ms
e4	10^9	$8\cdot10^4$ m	8000/10^9=0,008	$8\cdot10^4/2\cdot10^8$=0,400	–	0,408 ms

TOTAL d_end2end = 8,013 + 1,583 + 1,053 + 0,408 = 11,057 ms

Por tipo: d_trans 8,888 ms (80,4 %) · d_prop 2,160 ms (19,5 %) · d_proc 0,009 ms
Por enlace: e1 = 72,5 % · e2 = 14,3 % · e3 = 9,5 % · e4 = 3,7 %

La pregunta que hay que responder, y el plan de mejora:

El enlace e1 aporta el 72,5 % del retardo, y es el ÚNICO de los cuatro que está a menos de 100 km. Porque es el ÚNICO lento.

⚠️ EL CUELLO DE BOTELLA NO ES EL MÁS LARTO NI EL MÁS CARO:
ES EL DE MENOR d_trans.

Plan, por peso invertido: subir e1 a 10 Mbps (d_trans = 0,800 ms) baja el total a 0,813 + 1,583 + 1,053 + 0,408 = 3,857 ms, o sea un 65 % de mejora con un solo cambio. Recién ahí e2 a 1 Gbps → 2,857 ms, y e3 a 100 Mbps → 2,257 ms.

2.10 Ejercicio 10 — Comparar dos topologías con la misma distancia total

RUTA: 3 saltos de 1000 km a 10 Mbps, 2 routers, d_proc = 5 µs
DIRECTA: 1 salto de 3000 km a 1 Gbps, 0 routers
L = 12 000 bits, fibra (s = $2\cdot10^8$), sin cola

RUTA: d_trans = 12 000/10^7 = 1,2 ms ×3 = 3,6 ms
d_prop = $10^6/2\cdot10^8$ = 5 ms ×3 = 15 ms
d_proc = 5 µs ×2 = 0,010 ms
d_total = 3,6 + 15 + 0,010 = 18,610 ms

DIRECTA: d_trans = 12 000/10^9 = 0,012 ms ×1
d_prop = $3\cdot10^6/2\cdot10^8$ = 15 ms ×1
d_total = 0,012 + 15 + 0 = 15,012 ms

	RUTA (3×1000 km, 10 Mbps)	DIRECTA (3000 km, 1 Gbps)
nº de enlaces · distancia total	3 · 3000 km	1 · 3000 km → **igual distancia**
`d_trans`	3,600 ms	0,012 ms
`d_prop`	15,000 ms	15,000 ms → **igual**
TOTAL	**18,610 ms**	**15,012 ms**

Y el inverso, que es el que más rinde: una directa de **1500 km a 1 Gbps** da `d_prop = 7,5 ms` y `d_total = 7,512 ms`; una ruta de **3 saltos de 500 km a 10 Mbps** da `d_prop = 7,5 ms` (igual) y `d_trans = 3,6 ms`, o sea `d_total = 11,110 ms`.

> ⚠️ A **igual distancia total**, la ruta corta gana 3,6 ms. **La distancia se reparte; la transmisión se suma.** Esa es la frase del ejercicio.

Agregar un **router** cuesta casi nada (5 µs de `d_proc`); agregar un **enlace** cuesta `L/R + d/s`. Por eso las redes de backbone tienen pocos saltos y muy rápidos: cada salto extra es un `L/R` extra que se paga en todos los paquetes.

2.11 Ejercicio 11 — *Delay* × *bandwidth* de una descarga grande

Datos. Bajás **100 MB** por un enlace de **5 Mbps** con **RTT = 200 ms**. Sin cola.

```
BDP = 5·10⁶ bit/s × 0,200 s = 10⁶ bits = 125 000 bytes = 125 kB

a) VENTANA 16 KB:  16 384 < 125 000 → SÍ limita
   Throughput = 16 384/0,200 = 81 920 B/s = 0,655 Mbps
   Tiempo     = 100·10⁶/81 920 = 1220,7 s = 20,3 minutos
b) VENTANA 1 MB:   1 048 576 > 125 000 → sobra
   Throughput = 1 048 576/0,200 = 41,9 Mbps → el tope es R = 5 Mbps
   Tiempo = 100·10⁶/(5·10⁶/8) = 100·10⁶/625 000 = 160 s = 2,67 minutos
c) MÁXIMO: 5 Mbps, alcanzado cuando W ≥ 125 kB (el BDP).
   8 KB → 0,33 Mbps → 40,5 min  ·  64 KB → 2,62 Mbps → 5,1 min  ·  125 KB → 5 Mbps
```

La lección: el enlace **no es lento** —a 5 Mbps los 100 MB tardarían 160 s siempre—, lo que tarda más es la **ventana**. Con 8 KB (el `rwnd` por defecto de Windows durante años) tardás 40,5 minutos en vez de 2,67: un factor 15. `RFC 7323` (*Window Scale*, 2014) es lo que cubre el BDP: sin él el máximo son 65 535 bytes, y acá eso da el 52 %.

2.12 Ejercicio 12 — Sensibilidad de punta a punta sobre el problema de examen

Datos. Los del Ejercicio 3: `L = 1500 bytes`, tres enlaces de **100 Mbps** y **500 km**, dos conmutadores de `d_proc = 3 µs`, store-and-forward. **Base:** `d_total = 360 + 6 + 7500 = 7866 µs = 7,866 ms`.

```
a) R de 100 a 200 Mbps
   d_trans = 12 000/2·10⁸ = 60 µs ; ×3 = 180 µs   (SE REDUCE)
   d_prop y d_proc no cambian → 180 + 6 + 7500 = 7686 µs = 7,686 ms
   Variación: −180 µs = −2,3 %

b) d de 500 a 1000 km
   d_trans no cambia (360 µs) ; d_proc no cambia (6 µs)
   d_prop = 10⁶/2·10⁸ = 5 ms = 5000 µs ; ×3 = 15 000 µs
   d_total = 360 + 6 + 15 000 = 15 366 µs = 15,366 ms
   Variación: +7500 µs = +95,4 %

c) R = 1 Gbps  Y  d = 100 km  (las dos cosas juntas)
   d_trans = 12 000/10⁹ = 12 µs ; ×3 =   36 µs
   d_prop  = 10⁵/2·10⁸ = 500 µs ; ×3 = 1500 µs
   d_proc  = 3 µs ; ×2 = 6 µs
   d_total = 36 + 6 + 1500 = 1542 µs = 1,542 ms
```

Escenario	`R`	`d`	`d_total`	vs. base
base	100 Mbps	500 km	**7,866 ms**	—
a) `R` ×2	200 Mbps	500 km	**7,686 ms**	**−2,3 %**
b) `d` ×2	100 Mbps	1000 km	**15,366 ms**	**+95,4 %**
c) `R` ×10 y `d` ÷5	1 Gbps	100 km	**1,542 ms**	**−80,4 %**

La interpretación, que es lo que cierra los 10 puntos del problema:

> - **b)** Duplicar la distancia casi duplica el retardo, porque `d_propagación` es el 95 % del total.
> - **a)** Duplicar el ancho de banda mejora apenas (2,3 %), porque `d_trans` es el 4,6 %. **No tocó `d_prop`.**
> - **c)** Con las dos juntas baja un 80 %, pero `d_trans` bajó ×10 (360 → 36 µs, porque `R` ×10) y `d_prop` bajó ×5 (7500 → 1500 µs, porque `d` ÷5). `d_prop` **no bajó ×10**: depende de la distancia, no del ancho de banda.
>
> **La frase que hay que dejar escrita:** "*En una WAN larga el retardo lo domina la propagación y la palanca es la distancia y el medio. En un enlace corto y lento lo domina la transmisión y la palanca es el ancho de banda. En una red saturada lo domina la cola y la única palanca es reducir el tráfico que llega.*"

⚠️ Errores típicos de los ejercicios 7 a 12

- **Sumar los `d_proc` por enlace:** siempre son `n − 1` en una línea.
- **Aplicar el mismo `R` a todos los enlaces:** cada `d_trans` usa el `R` **de su enlace**.
- **Usar un solo `s` sin mirar el medio:** si uno es satélite, ese usa $3 \cdot 10^8$.
- **Comparar BDP en bits contra ventana en bytes:** hay que convertir.
- **Confundir throughput con retardo:** bajar un archivo más rápido es una cosa; que llegue antes es otra.

PARTE 3 — EL PROBLEMA DE EXAMEN

3.1 ⭐ ⭐ Examen 2011-03, Pregunta 5 (10 puntos) — retardo end-to-end

> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011, Pregunta 5 (10 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf **Este es EL problema de retardos del curso.** Aparece reformulado en el Simulacro 1 (Clase 7).

Enunciado (texto literal del examen)

> Considere un paquete de longitud L que tiene su origen en el sistema terminal A y que viaja a través de tres enlaces hasta un sistema terminal de destino B. Estos tres enlaces están conectados mediante dos dispositivos de conmutación de paquetes. Sean `d_i`, `s_i` y `R_i` la longitud, la velocidad de propagación y la velocidad de transmisión del enlace i, para `i = 1, 2, 3`. Cada dispositivo de conmutación de paquetes retarda cada paquete `d_proc`.
>
> **a)** Suponiendo que no se produce retardo de cola, ¿cuál es el retardo total terminal a terminal del paquete en función de `d_proc`, `d_i`, `s_i`, `R_i` (`i = 1, 2, 3`) y L?
>
> **b)** Suponga ahora que `R_1 = R_2 = R_3 = R` y `d_proc = 0`. Suponga también que el conmutador de paquetes no almacena los paquetes y los reenvía, sino que transmite inmediatamente cada bit que recibe sin esperar a que llegue el paquete completo. ¿Cuál será el retardo terminal a terminal?

Solución oficial (completa, textual)

> **a)** *"En general el retardo extremo a extremo es la suma de los componentes del retardo: procesamiento, cola, transmisión y propagación. Si despreciamos el retardo de cola, el retardo de extremo a extremo se expresa como:*
>
> `d_end2end = Σ_{i=1..3} d_proc + L/R_i + d_i/s_i` *
>
> *"Para el caso concreto, se reduce a:* `d_end2end = d_proc + d_proc + L/R1 + L/R2 + L/R3 + d1/s1 + d2/s2 + d3/s3` *"*
>
> **b)** *"Dado que los bits se transmiten inmediatamente, los conmutadores de paquetes no introducen ningún retardo (en particular no introducen retardo de transmisión), y por lo tanto ahora el retardo de extremo a extremo se expresa como:* `d_end2end = L/R + d1/s1 + d2/s2 + d3/s3` *"*

Análisis de la solución oficial

Lo que está muy bien: 1. **El patrón correcto en el apartado a):** el `d_proc` aparece **dos veces**, escrito literalmente `d_proc + d_proc`. **Ese es el 80 % de los puntos**, y muchísimos alumnos lo escriben tres veces. 2. **Los tres `L/R_i` con el `R_i` de cada enlace** (el apartado a) NO dice que sean iguales), y los tres `d_i/s_i` con su `s_i`. 3. **En el apartado b) justifica con una frase**, y no solo con el número: *"los conmutadores no*

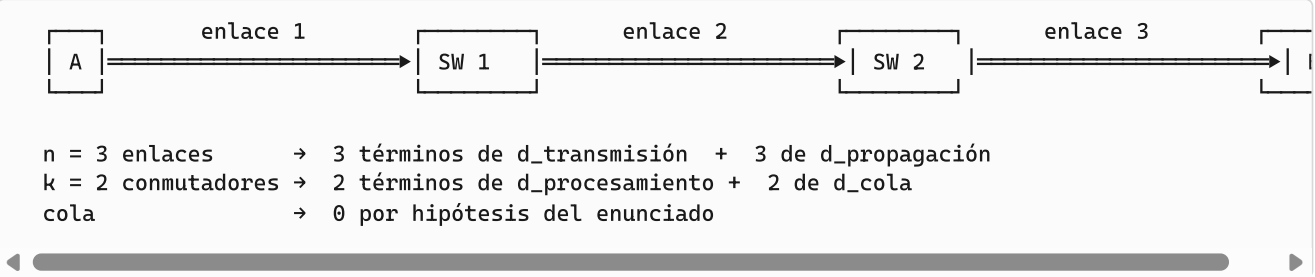
introducen ningún retardo, y en particular no introducen retardo de transmisión". Y ya usa `R` único, que es la hipótesis nueva del enunciado.

Lo que falta, y son los puntos que se ganan: 1. **No explica por qué `d_proc` va dos veces.** Lo escribe bien pero no lo **argumenta**. 2. **No nombra el modelo** del apartado b): lo describe pero no lo llama **cut-through forwarding**. 3. **No cuantifica la mejora.** "Cuál será el retardo" pide un número, y decir "es menor" sin cuánto deja puntos. 4. **No idealiza con advertencia** (en un switch real hay que leer la MAC destino, ≈14 bytes) ni dice qué pasa si `R_entrada ≠ R_salida`.

Resolución completa — para los 10 puntos

Parte a)

Paso 1 — Contar los términos antes de escribir nada.




```
n = 3 enlaces      →  3 términos de d_transmisión  +  3 de d_propagación
k = 2 conmutadores →  2 términos de d_procesamiento +  2 de d_cola
cola               →  0 por hipótesis del enunciado
```

Paso 2 — Fórmula general y concretada.

```
GENERAL:
    d_end2end = Σ_{i=1..3} [ d_proc + d_cola + L/R_i + d_i/s_i ]

CON d_cola = 0:

⚠️ El sumatorio trae 3·d_proc, pero hay que descontar el de los extremos:
   A y B son SISTEMAS TERMINALES, no dispositivos de conmutación.

CONCRETA:
d_end2end =  d_proc + d_proc                          ← 2 conmutadores
           + L/R₁ + L/R₂ + L/R₃                       ← 3 enlaces
           + d₁/s₁ + d₂/s₂ + d₃/s₃                    ← 3 enlaces

         =  2 · d_proc + Σ_{i=1..3} L/R_i + Σ_{i=1..3} d_i/s_i
```

Paso 3 — La frase que más suma, porque es lo que demuestra que lo entendiste.

> El retardo de transmisión y el de propagación aparecen **una vez por enlace**, y son tres enlaces: hay **tres términos de cada uno**. El retardo de procesamiento aparece **una vez por dispositivo de conmutación**, y hay **dos**: hay **dos términos de `d_proc`**. Los sistemas terminales A y B no son dispositivos de conmutación y no aportan `d_proc`.

Paso 4 — Forma alternativa compacta, y el caso homogéneo.

`d_end2end = 2·d_proc + L·(1/R₁ + 1/R₂ + 1/R₃) + (d₁/s₁ + d₂/s₂ + d₃/s₃)`. Y si los tres enlaces son homogéneos: `d_end2end = 2·d_proc + 3·(L/R) + 3·(d/s)`.

> ⚠️ **Nota sobre la oficial:** escribe literalmente `d_proc + d_proc + L/R1 + L/R2 + L/R3 + d1/s1 + d2/s2 + d3/s3`, que es **exactamente** lo mío. **No hay diferencia con la oficial.** La oficial simplemente no explicita la fórmula compacta ni la justificación del "dos".

Parte b)

Paso 1 — Qué cambia exactamente.

```
El enunciado agrega DOS hipótesis numéricas:
    (i)   R₁ = R₂ = R₃ = R      → los tres L/R se vuelven uno solo
    (ii)  d_proc = 0              → desaparece el procesamiento

Y UNA hipótesis de comportamiento:
    (iii) el switch NO almacena: transmite cada bit apenas lo recibe
```

Paso 2 — Por qué desaparecen los `L/R` de los enlaces intermedios.

```
STORE-AND-FORWARD:  el switch 1 tiene que esperar a que ENTRE la trama
                    COMPLETA para mirar la MAC destino y elegir el puerto.
                    Recién ahí arranca por el enlace 2. Como R es el mismo,
                    ese L/R se paga dos veces: entrar y salir. Idem en el
                    switch 2. De ahí los 3 L/R.

CUT-THROUGH:        el switch ve la MAC destino en los primeros bytes y
                    arranca a emitir por el enlace 2 MIENTRAS sigue
                    recibiendo por el 1. Como R_entrada = R_salida, el
                    ritmo de salida es el de entrada: el switch es
                    TRANSPARENTE, no acumula nada.

                    ⇐ El medio se comporta como un solo enlace largo:
                       los tres L/R se PISAN en vez de se SUMAR.

Detalle fino: al switch le hacen falta los primeros bytes para conocer el puerto de
salida, así que transmite con un atraso de (14·8)/R = 1,12 µs, no de forma instantánea.
```

Paso 3 — La cuenta y la comparación con el apartado a).

```
d_end2end = 1 · (L/R) + Σ_{i=1..3} (d_i/s_i) = L/R + d₁/s₁ + d₂/s₂ + d₃/s₃

                        (a) store-and-fwd    (b) cut-through
──────────────────────────────────────────────────────────────
términos de L/R                 3                  1
términos de d_proc              2                  0
términos de d_i/s_i             3                  3   (igual)
──────────────────────────────────────────────────────────────
RESULTADO           2·d_proc + Σ L/R_i + Σ d_i/s_i    (L/R) + Σ d_i/s_i
──────────────────────────────────────────────────────────────
DIFERENCIA      MEJOR EN (b) EN: 2·d_proc + 2·(L/R)
```

El ahorro es exactamente `(n − 1)·(L/R) + k·d_proc = 2·(L/R) + 2·d_proc`.

Paso 4 — El nombre del modelo y la advertencia realista.

> **El modelo del apartado b) se llama `cut-through forwarding` (reenvío al vuelo).** La trama *cuts through* el conmutador: no espera a quedar entera en un buffer.

> **Advertencia realista:** un conmutador real **no puede** reenviar el primer bit, porque la dirección MAC destino está en los **6 primeros bytes** de la trama, precedidos por 8 de preámbulo. Tiene que leer unos **14 bytes** antes de poder decidir: `(14 · 8)/R` segundos. A 100 Mbps son **1,12 µs**, y un `d_proc` real de switch cut-through está entre 1 y 2 µs, no 0. La solución oficial pone cero porque **el enunciado fija esa hipótesis**, y está bien que lo haga: lo que se corrige es lo que dice el enunciado. **Señalar el matiz suma puntos.**

Paso 5 — La observación profunda, si queda lugar. El resultado del apartado b) **supone que** `R_1 = R_2 = R_3 = R`. Si el conmutador recibiera más rápido de lo que transmite (`R_1 = 1 Gbps`, `R_2 = 100 Mbps`), se acumularía cola y **se comportaría como un store-and-forward**: el bit que entra primero saldría último. El cut-through sin `R` uniforme **agrega congestión antes de tiempo** (*congestion crunch*), y ésa es la razón real por la que los conmutadores de núcleo de las redes de carrier no lo usan.

Paso 6 — La tabla resumen, que es la respuesta "de 10":

Modelo	`d_end2end`	Cuándo
Store-and-forward (a)	`2·d_proc + Σ L/R_i + Σ d_i/s_i`	el caso general
Cut-through (b)	`1·(L/R) + Σ d_i/s_i`	latencia crítica
Store-and-forward con `R` y `d_proc` uniformes	`0 + 1·(L/R) + Σ d_i/s_i`	el "mejor caso", **imposible**

> **Nota: la solución oficial coincide exactamente con mi resolución en los dos apartados.** No hay divergencia. Lo que agregué es la justificación escrita del "dos `d_proc`", el nombre del modelo, la cuantificación del ahorro y las advertencias realistas. Eso, en un examen de 10 puntos, es la diferencia entre un 6 y un 10.

3.2 Recordatorio — dónde están los otros problemas de retardo

Estos **ya están resueltos enteros en la Clase 5** y no se repiten: `2009-12` **P1** (10 pts, componentes del retardo, ancho de banda vs. velocidad de propagación y congestión → Clase 5 §3.1), `2011-03` **P2** (6 pts, congestión, pérdidas y retardo → Clase 5 §3.2) y `2010-02` **P5a** (8 pts, jitter vs. retardo → Clase 5 §1.5; los números en el Ejercicio 6 de esta clase). Los tres van con la solución oficial textual y su análisis línea por línea.

PARTE 4 — CIERRE

📌 Resumen

1. `d_end2end = Σ (d_proc + d_cola + L/R + d/s)`, y el total se **suma por cada enlace** y por cada router.
2. **Con `n` enlaces hay `n` términos de transmisión y `n` de propagación, pero solo `n − 1` de procesamiento.** Por eso la oficial del 2011-03 P5 escribe `d_proc + d_proc`.
3. `d_trans = L/R`, con `L` en **bits**: **no depende del medio ni de la distancia**.
4. `d_prop = d/s`, con `d` en **metros**: **no depende del paquete ni del ancho de banda**.
5. `d_cola` **es el único componente variable** y el **único que refleja la congestión**.
6. **Las tres reglas de sensibilidad:** duplicar `L` cambia solo `d_trans`; duplicar `R` cambia solo `d_trans` (a la mitad); duplicar `d` cambia solo `d_prop`.
7. **Store-and-forward = `n` términos `L/R`; cut-through = 1**, porque los `L/R` se pisan en vez de sumando. El ahorro es `(n − 1)·(L/R) + k·d_proc`.
8. **Cut-through solo es gratis si `R_entrada = R_salida` en todos los saltos**, y exige leer la MAC destino (≈14 bytes), así que el `d_proc = 0` del enunciado es una idealización.
9. `BDP = R · RTT` son los bits en vuelo; si `ventana < BDP`, el throughput queda limitado por la ventana y no por el enlace.
10. **Latencia ≈ RTT/2 solo con camino simétrico; jitter = variación del retardo**, y **rompe el audio mucho antes que la latencia**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un paquete de `L` bits cruza 3 enlaces unidos por 2 conmutadores, con `d_proc` por conmutador y sin cola. Escribí la fórmula completa y decí cuántas veces aparece cada término.

▶ Respuesta

2. En el mismo problema, ahora con cut-through, `R_1 = R_2 = R_3 = R` y `d_proc = 0`. ¿Cuál es el retardo y por qué mejora?

▶ Respuesta

3. ¿Por qué un switch cut-through real NO puede tener retardo cero?

▶ Respuesta

4. ¿Qué componente domina el retardo, y por qué no hay una respuesta universal?

▶ Respuesta

5. Un enlace de 20 Mbps une Montevideo y Buenos Aires, y el RTT medido es 80 ms. ¿Cuántos bytes pueden estar en vuelo? ¿Qué pasa si la ventana es menor?

▶ Respuesta

6. ¿Por qué duplicar el ancho de banda no cambia la propagación? ¿Y por qué duplicar la distancia no cambia la transmisión?

▶ Respuesta

7. ¿Qué es el jitter, y por qué un retardo alto y constante es mejor para el audio que uno medio y variable?

▶ Respuesta

8. En el Ejercicio 12, si duplico la distancia y duplico el ancho de banda a la vez, ¿se cancelan?

▶ Respuesta

🔗 Referencias

Exámenes - `eirc1103` / `sirc1103` (21-feb-2011) — **P5 (10 pts): retardo end-to-end con 3 enlaces y 2 conmutadores, y el caso cut-through**. Resuelto en el §3.1 de esta clase. - `sirc0912` (18-dic-2009) — P1 (10 pts): componentes del retardo, ancho de banda vs. velocidad de propagación, congestión. **Clase 5 §3.1.** - `sirc1103` — P2 (6 pts): congestión, pérdidas y retardo. **Clase 5 §3.2.** - `sirc1002` (26-feb-2010) — P5 (8 pts): jitter vs. retardo y sensibilidad de las aplicaciones. - `sirc0903` (20-feb-2009) — P5 (6 pts): full duplex vs. half duplex. **Clase 7.** - `sirc0707` (31-jul-2007) — P1 (10 pts): capacidad máxima sobre línea telefónica. **Clase 7.**

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., cap. 1 §1.5 "Delay, Loss, and Jitter in Packet-Switched Networks" (pág. ~10 a ~35 del PDF de la carpeta).

RFCs - `RFC 7323` — *Window Scale* (2014): lo que permite cubrir el BDP. - `RFC 5681` — *TCP Congestion Control* (2009): los cuatro algoritmos que reaccionan a la pérdida. - `RFC 3168` y `RFC 9768` — ECN y AccECN: marcar en vez de descartar, para que el objetivo sea evitar la cola. - `RFC 9331` — *L4S* (2023): ECN de umbral bajo. `RFC 9000` — *QUIC* (2021). `draft-ietf-ccwg-bbr` — BBRv3 (*Experimental*, **nunca fue RFC**).

Clases relacionadas - Clase 5 (teoría del retardo) · Clase 7 (Nyquist, Shannon y la capa física **+ Simulacro 1**) · Clase 19 (BDP y ventana de TCP) · Clase 21 (control de congestión y ECN).

Clase 7

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 La capa física: qué hace y qué NO hace
- 1.2 Los cuatro medios, y por qué la fibra ganó
- 1.3 Full duplex vs. half duplex (Examen 2009-03 P5)
- 1.4 Nyquist — el techo del medio limpio
- 1.5 La modulación: cómo se mandan los bits por el medio
- 1.6 Shannon — el techo del medio ruidoso
- 1.7 Nyquist y Shannon juntos: cuándo aplica cada uno
- 1.8 Los tres conceptos juntos, y qué cambió hasta 2026
- 2.1 Ejercicio 1 — Nyquist con 3100 Hz y 4 niveles
- 2.2 Ejercicio 2 — Shannon con S/N en dB (la conversión a la vista)
- 2.3 Ejercicio 3 — Teórico máximo contra capacidad real
- 2.4 Ejercicio 4 — QAM y bits por símbolo
- 3.1 Examen 2006-08, Pregunta 1 (5 puntos) — Nyquist y Shannon con sus hipótesis
- 3.2 Examen 2007-07, Pregunta 1, apartado a (10 puntos) — capacidad sobre línea telefónica
- 4.1 El repaso que hay que tener delante antes del simulacro
- 4.2 Los 8 errores más frecuentes del Bloque A
- 4.3 Enunciado del simulacro (sin respuestas)
- 4.4 Resolución del simulacro
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: A (Fundamentos) · **Semana:** 3 · **Duración estimada:** 120 min **Correlativa:** Clase 6 (taller de retardos) · **Sigue con:** Clase 8 (HTTP: la web como aplicación de la capa de aplicación) **Material de referencia:** Tanenbaum cap. 1 §1.1 y §1.6 · Exámenes `sirc0608`, `sirc0707`, `sirc0903`

🎯 Al terminar esta clase vas a poder

1. **Describir** el único encargo de la capa física y explicar **qué no hace**.
2. **Comparar** los cuatro medios en una tabla con ancho de banda, inmunidad al ruido y alcance.
3. **Distinguir** full duplex de half duplex, con la tabla de qué tecnología usa qué modo.
4. **Enunciar** Nyquist con sus **hipótesis** y calcular la capacidad dado `B` y `V`.
5. **Enunciar** Shannon con sus **hipótesis** y calcular la capacidad con el `S/N` **en lineal**, convirtiendo desde dB.
6. **Explicar** cuándo aplica cada teorema y por qué Shannon queda por debajo de Nyquist.
7. **Describir** ASK, FSK, PSK y **QAM**, y calcular los bits por símbolo de una constelación de `M` puntos.
8. **Hacer** el **Simulacro 1** completo: 5 preguntas teóricas y 2 problemas, con cronómetro.

> **Prerequisito del examen:** `2006-08 P1` (5 pts), `2007-07 P1` (10 pts), `2009-02 P3` (5 pts), `2009-03 P5` (6 pts), `2011-03 P5` (10 pts).

PARTE 1 — TEORÍA

1.1 La capa física: qué hace y qué NO hace

La capa física es **la más tonta del modelo y por eso la más fácil de recordar**: no razona, no mira contenido, no decide. Su único encargo es transmitir **bits**:

```
┌──────────────────────────────────────────────────────────┐
│  La capa física TRANSMITE BITS de un nodo al nodo vecino:  │
│    1  0  1  1  0  0  1  0  1  1  1  0  →  señal en el medio  │
│  Nada más. No sabe qué hay adentro del paquete.          │
└──────────────────────────────────────────────────────────┘
```

Lo que la capa física NO hace (y esto es lo que se pregunta): no distingue un bit de otro (`10110010` y `01001101` son "lo mismo" para ella, y el significado lo pone la capa 2); no detecta errores (el CRC es de la capa 2); no reenvía ni enruta (el bit que entra por un puerto sale por el otro, y nada más); no reconoce direcciones; y no reintenta, no comprime ni cifra.

Por qué se llama "física": porque todo lo que hace depende de la física real — la velocidad con que la señal viaja por el cobre, la atenuación con la distancia, el electromagnetismo alrededor, la temperatura. Un enlace de 400 Gbps sobre **fibra multimodo** tiene un alcance de 100 m, y sobre **fibra monomodo** llega a 40 km: la diferencia es física.

1.2 Los cuatro medios, y por qué la fibra ganó

	Par trenzado (UTP/STP)	**Coaxial**	**Fibra óptica**	**Inalámbrico** (radio / satélite)
Medio	cobre, pares trenzados	cobre con malla blindada	luz en vidrio de sílice	ondas de radio o infrarrojo
Truco	el **trenzado** cancela el *crosstalk*; el blindaje (STP) protege del **EMI**	la malla es el retorno y **apantalla**	**inmune al EMI**: la luz no es campo eléctrico	el medio es **aire**, y es compartido
Ancho de banda	10 Mbps → 10 Gbps (Cat 6/8)	~10 Mbps (TV e internet CATV)	**decenas de Gbps a Tbps**	10 Mbps (Wi-Fi) a Gbps (5G, Wi-Fi 7)
Alcance sin electrónica	**100 m**	~500 m	**40 km (monomodo)** a 100 m (multimodo)	km a 36 000 km
Inmune al ruido	no	no	**SÍ, totalmente**	no

⚠️ El dato del satélite, que es el mejor ejemplo del curso

```
d = 36 000 km (altura de un geoestacionario)   s = 3 · 10⁸ m/s (va por aire)
d_prop de ida = 3,6·10⁷ / 3·10⁸ = 0,12 s = 120 ms      RTT = 240 ms

⚠️ 120 ms de latencia es IRRECUPERABLE: es geometría, no ingeniería. Por eso el
   satélite SÍ sirve para STREAMING (tolerás 2 s de arranque) y NO para VoIP
   INTERACTIVO (el humano no aguanta más de 150 ms de ida en una conversación).
```

Por qué es el ejemplo clave: un enlace satelital puede tener 50 Mbps y aun así ser **peor** para una videollamada que una fibra de 1 Gbps. El ancho de banda responde a "*¿cuánta información por segundo?*" y la latencia a "*¿cuánto tarda?*": preguntas distintas, con variables distintas (`R` contra `d/s`). Es el núcleo conceptual de la Clase 5 y lo que se pregunta en el Examen 2009-12 P1a.

1.3 Full duplex vs. half duplex (Examen 2009-03 P5)

Definiciones, con las palabras de la solución oficial: full duplex es que se puede **transmitir y recibir simultáneamente**; **half duplex** es que se puede transmitir o recibir, pero **no a la vez**, y se alterna. Es como un teléfono (full duplex: hablás y escuchás a la vez) contra una radio de baquelita (half duplex: apretás para hablar, soltás para escuchar).

La tabla de qué tecnología usa qué modo:

Tecnología	Modo	Por qué o con qué matiz
Ethernet conmutado (Fast, Gigabit, 10 GigE)	**FULL DUPLEX**	Cada puerto es un enlace punto a punto dedicado: un par para ir y otro para volver.
Ethernet con hub (coax compartido, 10BASE legacy)	**HALF DUPLEX**	Un único medio compartido: si dos transmiten a la vez, colisión. Necesita CSMA/CD.
Bluetooth (802.15)	**HALF DUPLEX** (formalmente)	*Time Division Duplex*: el maestro y los esclavos se turnan en ranuras de 625 µs. Con poca rigurosidad se lo llama full duplex, pero **no es simultáneo real**.
Wi-Fi (802.11)	**HALF DUPLEX siempre**	El medio es el **aire**, inherentemente compartido. En 802.11 **no existe full duplex**.
Fibra y enlaces seriales	**FULL DUPLEX**	Suele haber un par (TX/RX) separado por dirección.
DSL	**FULL DUPLEX** (por frecuencias)	Usa bandas distintas para subida y bajada.

> ⚠️ **La frase clave para el examen:** Ethernet conmutado es **full duplex**; Wi-Fi es **siempre half duplex**. Si el enunciado dice "Ethernet (IEEE 802.3)", la respuesta es full duplex en los enlaces conmutados modernos, y half duplex en los hubs antiguos.

⚠️ Actualización a 2026 — el half-duplex en Ethernet

Sigue igual: Ethernet nació half duplex y se hizo full duplex con el switch; Wi-Fi nunca tuvo full duplex.

Cambió: - **El half-duplex Ethernet es historia por encima de 1 Gb/s.** A 10 GbE y más el full-duplex punto a punto es **obligatorio**. Es una decisión de **IEEE 802.3**, no de la IETF: **no existe RFC que lo depreque**. ⚠️ `RFC`

`8984` **es JSCalendar**, no Ethernet. - **PERO no desapareció del todo: IEEE 802.3cg 10BASE-T1S** (un solo par, para redes industriales) es **obligatoriamente half-duplex con PLCA**, y el módulo YANG `csma-cd` de `802.3cf` está marcado `status deprecated`.

1.4 Nyquist — el techo del medio limpio

El teorema

```
┌──────────────────────────────────────────────────────────────┐
│                                                              │
│        C = 2 · B · log₂(V)          [bits por segundo, bps]  │
│                                                              │
│   B = ancho de banda del enlace, en HERTZ (Hz)               │
│   V = cantidad de niveles de señal (estados posibles por símbolo)  │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Las hipótesis del teorema (lo que pregunta el Examen 2006-08 P1)

Según la solución oficial, tres:

```
1. Canal de comunicaciones de ANCHO DE BANDA LIMITADO B, SIN RUIDO de ningún tipo.
2. La señal transmitida es una SEÑAL DIGITAL de V NIVELES DISCRETOS.
3. (implícita) El medio transmite 2 SEÑALES por segundo, una por cada cambio de nivel.
```

El ejemplo canónico de la cátedra, resuelto

Una línea telefónica clásica: `B = 3100 Hz`, `V = 2` (dos niveles: +1 y −1). `C = 2 · 3100 · log₂(2) = 6200 bps`. El detalle importante: `log₂(2) = 1`, porque con `V = 2` cada **símbolo** transporta 1 bit; y con 2 símbolos por segundo salen 2 bits por segundo y por Hz, que es justamente el `2` del `2·B`.

Si el mismo enlace admite **4 niveles** (`V = 4`): `C = 2 · 3100 · log₂(4) = 12 400 bps`. El ancho de banda **no cambió** (siguen siendo 3100 Hz): lo que cambió fue la **modulación**, y ahora cada símbolo lleva 2 bits. **Se dobló la velocidad sin tocar el medio.**

Por qué el factor 2 (la intuición)

En un medio de ancho de banda `B` se pueden mandar **2 señales por segundo** (una por cada oscilación o cambio de nivel). Cada una vale `log₂(V)` bits, porque tenés `V` estados posibles y hacen falta `log₂(V)` bits para distinguirlos (2 estados → 1 bit, 4 → 2 bits, 16 → 4 bits).

```
2 señales/segundo   ×   log₂(V) bits/señal
──────────────────────────────────────────
     =  2 · log₂(V)   bits por segundo y por Hz

y multiplicando por los B Hz:   C = 2 · B · log₂(V)
```

1.5 La modulación: cómo se mandan los bits por el medio

La capa física convierte bits en **señal**; la **modulación** es ese proceso:

```
ASK (Amplitude Shift Keying) — modula la AMPLITUD
     bit 0:  ──────┐        ┌────────       Con V niveles:
     bit 1:        └────────┘                V amplitudes por símbolo

FSK (Frequency Shift Keying) — modula la FRECUENCIA
     bit 0:  ~~~~~   (frecuencia baja)     bit 1:  ∿∿∿∿∿ (alta)

PSK (Phase Shift Keying) — modula la FASE
     bit 0:  ──────   (fase 0°)              bit 1:  ──────  (fase 180°)

QAM (Quadrature Amplitude Modulation) — combina AMPLITUD y FASE
     constelación de puntos: cada PUNTO = un símbolo distinto
     16-QAM = 16 puntos = 4 bits por símbolo
```

QAM: la estrella del curso

```
16-QAM (constelación de 16 puntos):

         I (Eje en cuadratura)
         │
     · · ·┼· · ·      ● ●     ← 4 niveles de amplitud en I
     ─────┼──────────────  Q (Eje en fase)
     · · ·┼· · ·      ● ●
         │

4 puntos en cada cuadrante × 4 cuadrantes = 16 puntos
bits por símbolo = log₂(16) = 4

En un canal de 3100 Hz: C = 2 · 3100 · log₂(16) = 24 800 bps
```

El punto conceptual: QAM **exploita los dos indicadores** que Nyquist dice que tenés (la amplitud **y** la fase). Con `A` niveles de amplitud y `F` fases distintas se consigue una constelación de `M = A · F` puntos, y cada punto vale `log₂(M)` bits. Por eso las modulaciones reales de alta velocidad (Wi-Fi 5/6/7, 5G, cable) son todas QAM de orden alto: 256-QAM, 1024-QAM, 4096-QAM.

1.6 Shannon — el techo del medio ruidoso

El teorema

```
┌──────────────────────────────────────────────────────────────────┐
│        C = B · log₂(1 + S/N)          [bits por segundo, bps]    │
│   B  = ancho de banda del enlace, en HERTZ (Hz)                  │
│   S  = potencia de la señal;  N = potencia del ruido             │
│   S/N = relación señal-ruido, EN LINEAL (¡no en dB!)             │
│                                                                  │
│   Es la CAPACIDAD del canal: el máximo de bits que pasan         │
│   LIMPIAMENTE, con corrección de errores tolerable.              │
└──────────────────────────────────────────────────────────────────┘
```

⚠️ LA CONVERSIÓN DE dB A LINEAL (el paso que más se olvida)

```
┌──────────────────────────────────────────────────────────┐
│        S/N_lineal = 10 ^ (S/N_dB / 10)                   │
└──────────────────────────────────────────────────────────┘
```

S/N (dB)	S/N (lineal)	$\log_2(1 + S/N)$	Con $B = 3100$ Hz
0 dB	1	$\log_2(2) = 1$	3100 bps
20 dB	**100**	$\log_2(101) \approx 6{,}66$	≈ 20 600 bps
30 dB	**1000**	$\log_2(1001) \approx 9{,}97$	≈ 30 900 bps
40 dB	10 000	$\log_2(10\,001) \approx 13{,}29$	≈ 41 200 bps

El ejemplo paso a paso (30 dB → lineal):

```
S/N = 30 dB   →   S/N_lineal = 10^(30/10) = 10³ = 1000
C = 3100 · log₂(1 + 1000) = 3100 · log₂(1001) = 3100 · 9,97 ≈ 30 900 bps

⚠️ El error clásico es meter 30 (el dB) en vez de 1000 (el lineal):
   da 3100 · log₂(31) ≈ 15 700 bps, LA MITAD.
```

Las hipótesis del teorema

(1) El canal tiene ancho de banda limitado `B` (Hz). (2) El ruido es **gaussiano** (blanco, distribución normal) de potencia `N`. (3) Hay una señal de potencia `S`. (4) El límite **no son los niveles de señal** (`V` no aparece): son el ancho de banda y el ruido. Ésta es la diferencia clave con Nyquist.

1.7 Nyquist y Shannon juntos: cuándo aplica cada uno

```
┌──────────────────────────────────────────────────────────┐
│   NYQUIST  = techo del medio IDEAL (sin ruido)           │
│              C_N = 2 · B · log₂(V)                       │
│                                                          │
│   SHANNON  = techo del medio REAL (con ruido gaussiano)  │
│              C_S = B · log₂(1 + S/N)                     │
│                                                          │
│   SIEMPRE:  C_S ≤ C_N                                    │
└──────────────────────────────────────────────────────────┘
```

```
capacidad (bps)
   │                    ┌──────────  NYQUIST: 2·B·log₂(V)   (medio sin ruido)
   │                  / │
   │    SHANNON     /    │  ← SHANNON SIEMPRE DEBAJO DE NYQUIST;
   │   B·log₂(1+S/N)     │    la distancia entre las dos curvas es el RUIDO
   │  /                  │
   │/                    │
   └─────────────────────┼──────────────→  B (Hz)

Si S/N → ∞ (ruido despreciable), SHANNON → NYQUIST.
```

¿Cuándo se aplica cada uno?

Situación	Teorema	Por qué
Medio **limpio** (cobre corto, fibra nueva)	**Nyquist**	El ruido es despreciable; el límite es la modulación (`V` niveles).
Medio **ruidoso** (línea telefónica, radio, cobre de largo recorrido)	**Shannon**	El ruido es el factor limitante; el límite es el `S/N` .
Diseño de sistema	**Los dos**	Nyquist da el techo del hardware y Shannon el techo usable; el diseño real queda **por debajo de Shannon**, y **Nyquist nunca se alcanza**.

La relación, con números: con `B = 3100 Hz` y `V = 4` , Nyquist da `12 400 bps` , y el mismo medio con `S/N = 20 dB` da por Shannon `20 650 bps` . Parece que Shannon **supera** a Nyquist, pero **no están comparados con el mismo** `V` : para igualarlos haría falta `2·log₂(V) = log₂(101)` , o sea `V ≈ 10` , y con `V ≈ 10` Nyquist da `≈ 20 600 bps` . **Shannon da el techo REAL; Nyquist, el IDEAL de un medio limpio con una modulación dada.**

1.8 Los tres conceptos juntos, y qué cambió hasta 2026

```
┌──────────────┬──────────────────┬──────────────────┬──────────────────┐
│              │  NYQUIST         │  SHANNON         │  RETARDO         │
│ Pregunta     │ bits por SEGUNDO │ pasan LIMPIAMENTE│ cuánto tarda     │
│ Fórmula      │ 2·B·log₂(V)      │ B·log₂(1+S/N)    │ L/R  y  d/s      │
│ Variable clave│ V (niveles)     │ S/N (calidad)    │ R, d, L          │
│ Modelo       │ medio SIN ruido  │ medio CON ruido  │ cualquier medio  │
│ Saturado por │ los NIVELES      │ el RUIDO         │ distancia o R    │
└──────────────┴──────────────────┴──────────────────┴──────────────────┘
```

⚠️ Actualización a 2026 — qué cambió en la capa física

Sigue igual — de memoria: Nyquist (`C = 2·B·log₂(V)` , medio sin ruido), **Shannon** (`C = B·log₂(1+S/N)` , ruido gaussiano, `S/N` en lineal), la conversión `S/N_lineal = 10^(dB/10)` , las definiciones de full y half duplex, las características de los cuatro medios, y la modulación (ASK, FSK, PSK, QAM).

Cambió — y esto es lo que hay que contar en 2026: - **La fibra se volvió el medio dominante**: el cobre sigue para el último tramo de acceso (unos 100 m), pero **todo el backbone y casi todo el centro de datos** es fibra. - **El half-duplex Ethernet murió en el backbone**, como se explicó en §1.3. - **Wi-Fi dejó atrás el CSMA/CD por completo**: hoy es **CSMA/CA**, con **OFDMA** (Wi-Fi 6/6E), **MU-MIMO** y **MLO**. **Wi-Fi 7** (`IEEE 802.11be-2024` , publicado el 22-jul-2025) trae 320 MHz de canal y **4096-QAM**: Nyquist con `V = 4096` , o sea `log₂(4096) = 12` bits por símbolo. **Wi-Fi 8** (`802.11bn` , objetivo mayo-2028) se enfoca en confiabilidad. Un curso de 2012 sobre `802.11n` está **cuatro generaciones** atrás.

> **Consecuencia para el examen:** los **dos teoremas no cambiaron**: son matemáticas, no ingeniería. Lo que cambió son las cifras (4096-QAM en Wi-Fi 7) y el medio dominante (fibra). Si el enunciado es de Nyquist o Shannon, la fórmula y las hipótesis van **exactamente iguales** que en 2011.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Nyquist con 3100 Hz y 4 niveles

Datos. Una línea telefónica de **B = 3100 Hz** con una modulación de **V = 4 niveles**. **a)** ¿Cuál es la capacidad máxima? **b)** ¿Cuántos bits por símbolo?

```
a) C = 2 · B · log₂(V) = 2 · 3100 · log₂(4) = 2 · 3100 · 2 = 12 400 bps
b) bits por símbolo = log₂(V) = log₂(4) = 2 bits
```

Interpretación: cada 1/6200 de segundo mando un símbolo, y cada símbolo vale 2 bits: `C = 6200 símbolos/s × 2 bits = 12 400 bps`.

Variante: ¿y con **V = 16** (16-QAM)? `C = 2 · 3100 · log₂(16) = 24 800 bps`. El medio **no cambió**; lo que cambió fue la **modulación**, y con ella los bits por símbolo (de 2 a 4). Ése es el juego de Nyquist: **multiplicar los niveles es multiplicar la velocidad sin tocar el cable.**

2.2 Ejercicio 2 — Shannon con S/N en dB (la conversión a la vista)

Datos. Un enlace de **B = 3100 Hz** con ruido de **S/N = 30 dB**. **a)** Convertí el S/N a lineal. **b)** ¿Cuál es la capacidad máxima?

```
a) S/N_dB = 30
   S/N_lineal = 10^(30/10) = 10³ = 1000

b) C = B · log₂(1 + S/N_lineal) = 3100 · log₂(1 + 1000) = 3100 · log₂(1001)
   log₂(1001): como 2¹⁰ = 1024, está apenas por debajo de 10.
      log₂(1001) = 10 + log₂(1001/1024) = 10 + log₂(0,977) ≈ 10 − 0,034 ≈ 9,97
   C = 3100 · 9,97 ≈ 30 900 bps
```

Interpretación — la moraleja:

> Con 30 dB de calidad de señal (una relación señal-ruido de 1000 a 1), la línea telefónica clásica **no puede pasar de ~31 kbps**, aunque la modulación sea perfecta. Ése es el techo de Shannon: **el ruido, no el cable, es el límite**. Por eso los módems de la época llegaban a ~33,6 kbps y ahí paraban: no era el cable ni el módem, era Shannon.

⚠️ Errores típicos del ejercicio 2

- **Meter 30 (dB) en la fórmula en vez de 1000 (lineal).** Da `3100 · log₂(31) ≈ 15 700 bps`: la mitad. **Este es EL error del ejercicio.**
- **Olvidar el `1 +`:** es `log₂(1 + S/N)`, no `log₂(S/N)`.
- **Usar `10^(dB/20)`:** esa es la fórmula de potencia contra amplitud; el `S/N` en redes es siempre **potencia**, así que va con `/10`.

2.3 Ejercicio 3 — Teórico máximo contra capacidad real

Datos. Un enlace tiene `B = 1 MHz`, `S/N = 40 dB` y se modula con `V = 1024` (1024-QAM, o sea 10 bits por símbolo). **a)** Nyquist. **b)** Shannon. **c)** ¿Cuál manda?

```
a) Nyquist:  C_N = 2 · 10⁶ · log₂(1024) = 2 · 10⁶ · 10 = 20 Mbps

b) Shannon:  S/N = 10^(40/10) = 10 000 (lineal)
             C_S = 10⁶ · log₂(1 + 10 000) = 10⁶ · log₂(10 001) ≈ 10⁶ · 13,29
             C_S ≈ 13,3 Mbps

c) Mandan los dos: la capacidad real es el MÍNIMO, o sea 13,3 Mbps. Los
   20 Mbps son el techo del transmisor; los 13,3 Mbps, el del medio. En la
   práctica la capacidad real es un 50–70 % de Shannon, por atenuación,
   pérdida de sincronismo y overhead de protocolo.
```

2.4 Ejercicio 4 — QAM y bits por símbolo

Datos. Una constelación QAM tiene **16 puntos** en un enlace de 3100 Hz.

```
16-QAM = 4 niveles de amplitud × 4 fases  →  M = A · F = 4 · 4 = 16 puntos
Cada punto = 1 símbolo, y bits por símbolo = log₂(16) = 4 bits

C = 2 · 3100 · log₂(16) = 2 · 3100 · 4 = 24 800 bps
```

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 Examen 2006-08, Pregunta 1 (5 puntos) — Nyquist y Shannon con sus hipótesis

> **Fuente:** Examen `sirc0608` — 5 de agosto de 2006, Pregunta 1 (5 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0608.pdf

Enunciado (texto literal del examen)

> Enuncie los resultados de Nyquist y Shannon vistos en el teórico. Comente sobre las hipótesis de cada uno de ellos.

Solución oficial (textual)

> **Teorema de Nyquist** *Hipótesis:* · Canal de comunicaciones de ancho de banda limitado H, **sin ruido de ningún tipo** · La señal transmitida por el canal es una **señal digital de V niveles discretos**
>
> *Resultado:* La máxima tasa de transmisión de bits alcanzable es **M = 2 H $\log_2$(V)**
>
> **Ley de Shannon** *Hipótesis:* · Canal de ancho de banda H · Canal sometido a **ruido gaussiano de potencia N** · Señal de transmisión de potencia **S**
>
> *Resultado:* La capacidad del canal (en bits) tiene un máximo teórico **C = 2 H $\log_2$[1 + S/N]**

Análisis de la solución oficial

Lo que está bien: las hipótesis correctas de cada uno (Nyquist sin ruido y con `V` niveles; Shannon con ruido gaussiano y potencias `S` y `N`), y las dos fórmulas con sus variables.

🔴 **Una errata en la oficial:** escribe Shannon como `C = 2 H log₂(1+S/N)`, que es un error de transcripción: **la fórmula correcta es `C = B · log₂(1 + S/N)`, SIN el 2 delante.** El `2` es de Nyquist, cuya fórmula (`M = 2·H·log₂(V)`) sí está bien.

Lo que falta y son los puntos que se ganan: 1. **No explica cada variable con sus unidades** (Hz, niveles, lineal). 2. **No dice que `S/N` va en lineal, no en dB.** 3. **No relaciona los dos teoremas** (cuándo aplica cada uno).

Mi respuesta completa (5 puntos)

Nyquist — teorema de muestreo sin ruido:

```
C_N = 2 · B · log₂(V)   [bps]
  · B = ancho de banda del enlace, en HERTZ (Hz)
  · V = cantidad de niveles discretos de la señal

Hipótesis:
  (1) el canal tiene ancho de banda LIMITADO a B;
  (2) NO hay ruido de ningún tipo;
  (3) la señal es digital, de V niveles;
  (4) (implícita) el medio transmite 2 SEÑALES por segundo, una por cada
      cambio de nivel; de ahí el factor 2.

Idea: cada símbolo puede estar en uno de V estados, así que vale log₂(V) bits, y caben
2 símbolos por Hz de ancho de banda.
```

Shannon — teorema de la capacidad del canal:

```
C_S = B · log₂(1 + S/N)   [bps]
  · B = ancho de banda del enlace, en HERTZ (Hz)
  · S = potencia de la señal;  N = potencia del ruido (en W)
  · S/N = relación señal-ruido, EN LINEAL (si viene en dB: 10^(dB/10))

Hipótesis:
  (1) el canal tiene ancho de banda limitado B;
  (2) el ruido es GAUSSIANO de potencia N;
  (3) hay una señal de potencia S;
  (4) el límite NO son los niveles de señal (V no aparece), sino el ruido.
```

La relación entre ambos:

> Nyquist es el techo de un medio **ideal sin ruido** con una modulación dada; Shannon es el techo del medio **real con ruido**. **Nyquist se usa para el medio limpio** (diseño del transmisor) y **Shannon para el medio real** (diseño del sistema). En la práctica la capacidad real está **por debajo de Shannon**, por atenuación, pérdida de sincronismo y overhead de protocolo.

⚠️ Errores típicos del 2006-08 P1

- **Poner** `2 · B · log₂(1+S/N)` en Shannon: **no lleva el 2**. Y **meter** `S/N` **en dB** sin convertir a lineal.
- **Intercambiar las hipótesis:** decir "Nyquist necesita ruido gaussiano" o "Shannon necesita V niveles".
- **No dar las unidades** de `B` (Hz) ni de `V` (niveles).

3.2 Examen 2007-07, Pregunta 1, apartado a (10 puntos) — capacidad sobre línea telefónica

> **Fuente:** Examen `sirc0707` — 31 de julio de 2007, Pregunta 1, apartado a. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0707.pdf

Enunciado (texto literal del examen)

> **a)** Si se quiere calcular la máxima capacidad de transmisión de datos sobre una línea telefónica, ¿qué teorema visto en el curso se debe aplicar? Justifique y escriba la fórmula para calcularla, explicando el significado de las variables en la fórmula y qué unidades se suelen utilizar.

Sobre la solución oficial

El PDF de `sirc0707` **no trae la solución publicada del apartado 1a**: trae el enunciado y las soluciones de los problemas prácticos. La resolución de abajo es **solución propia**.

Resolución (solución propia, para los 10 puntos)

Paso 1 — Qué teorema aplicar: Shannon, no Nyquist.

> **Justificación:** la línea telefónica es un medio **ruidoso** (par trenzado, susceptible a interferencia, diafonía y ruido térmico). Como el ruido **no es despreciable**, Nyquist, que asume un medio **sin ruido**, daría un techo optimista. Corresponde la **ley de Shannon**, creada exactamente para este caso.

Paso 2 — La fórmula.

```
C = B · log₂(1 + S/N)   [bits por segundo, bps]
```

Paso 3 — Significado y unidades de cada variable.

Variable	Significado	Unidad	Nota
`C`	capacidad: máximo de bits transmitibles **sin errores**	**bps**	Es un techo **teórico**.
`B`	ancho de banda del canal	**Hertz (Hz)**	Para una línea telefónica, `B ≈ 3100 Hz`.
`S` / `N`	potencia de la señal / del ruido	**W (watts)**	
`S/N`	relación señal-ruido, o sea calidad del canal	**adimensional (lineal)**	⚠️ Si la dan en **decibeles**: `S/N = 10^(dB/10)`.

Pasos 4 y 5 — Valores típicos y un ejemplo numérico.

```
B = 3100 Hz,  S/N = 30 dB:
    S/N_lineal = 10^(30/10) = 1000
    C = 3100 · log₂(1 + 1000) = 3100 · 9,97 ≈ 30 900 bps
```

Interpretación: el máximo teórico de una línea telefónica decente es de unos **30 kbps**. Por eso los módems de la época llegaban a 33,6 kbps y ahí paraban: **no era el cable ni el módem, era Shannon**.

⚠️ Errores típicos del 2007-07 P1

- **Elegir Nyquist en vez de Shannon** (el error conceptual central). La línea telefónica tiene ruido, así que va Shannon.

- **No justificar por qué NO es Nyquist**, que es lo que pide el enunciado.
- **Meter el S/N en dB** sin convertir. **Olvidar las unidades de `B` (Hz).**

PARTE 4 — SIMULACRO 1

Instrucciones (reproducción del formato real de FING): - Individual, **sin material**, **3 horas (180 min)**. - **5 Preguntas Teóricas** (~40 pts) + **2 Problemas Prácticos** (30 pts cada uno). - Aprobación: **60 puntos**. - ⚠️ **Es obligatorio responder correctamente al menos 15 puntos en las preguntas teóricas.** Si no llegás a 15 en teoría, no aprobás aunque sumes 60 con los problemas. - Empezá cada pregunta y cada problema en una hoja nueva. Para los problemas podés suponer que disponés de los tipos básicos (lista, cola, archivo, string) y sus funciones.

⏱️ **Cronómetro sugerido (180 minutos):**

Etapa	Min	Objetivo
Lectura inicial	5	leer entero, elegir por dónde empezar
T1 (10 pts)	15	
T2 (10 pts)	15	
T3 (8 pts)	13	
T4 (8 pts)	12	
T5 (4 pts)	8	**63 min de teoría, ~40 pts**
Problema 1 (30 pts)	52	aritmética larga: dejar la fórmula y la cuenta a medias
Problema 2 (30 pts)	48	tablas antes que prosa; re-leer los números
Repaso final	7	re-leer los números de los dos problemas
Total	**180**	

El consejo de la cátedra: los problemas se corrigen **por lo que escribiste**. Si no te da el número exacto, **dejá la fórmula y la cuenta a medias** con la unidad anotada: parcialmente es puntos. Un apartado en blanco es cero.

Los 6 bloques del curso

4.1 El repaso que hay que tener delante antes del simulacro

Las fórmulas, de memoria:

```
d_transmisión = L/R            (L en BITS,  R en bps)          clases 5-6
d_propagación = d/s            (d en METROS, s en m/s)          clases 5-6
BDP           = R · RTT        (bits en vuelo)                 clases 5-6 y 19
latencia    ≈ RTT/2            (solo con camino simétrico)      clases 5-6
Nyquist:  C = 2 · B · log₂(V)                                  esta clase
Shannon:  C = B · log₂(1 + S/N)    (S/N EN LINEAL)              esta clase
dB → lineal:  S/N = 10^(dB/10)                                 esta clase
QAM:  bits por símbolo = log₂(M)  (M = puntos de la constelación) esta clase
```

OSI contra TCP/IP, y las PDU: en OSI son 7 capas y las PDU son **mensaje** → **segmento** → **paquete** → **trama** → **bit**; en TCP/IP son **4 capas** (Aplicación, Transporte, Internet, Acceso), que se leen **5** si partís Acceso en Enlace y Física. Correspondencia: OSI 7-5 ↔ Aplicación, OSI 4 ↔ Transporte, OSI 3 ↔ Internet, OSI 2-1 ↔ Acceso.

⚠️ El modelo TCP/IP se cuenta como **4 capas** (Acceso, Internet, Transporte, Aplicación) o como **5** si separás Acceso en Enlace y Física. La cátedra lo explica en la Clase 2.

Las tres técnicas de conmutación:

	Circuito	Paquetes (store-and-forward)	Mensajes
Conexión	sí	no (o circuito virtual)	no
Unidad	flujo continuo	paquete de tamaño fijo	mensaje completo
Reserva recursos	**sí**	**no**	**no**
Retardo	fijo y predecible	variable, con cola	muy variable
Orden	sí	**no**	**no**

Los parámetros que se confunden entre sí: `R` (ancho de banda, **bps**, del equipo), `s` (velocidad de propagación, **m/s**, del medio), **latencia** (un viaje de ida, ≈ `RTT/2` si el camino es simétrico), **RTT** (ida y vuelta, lo mide `ping`), **throughput** (`min(R, ventana/RTT)`) y **jitter** (`R_máx − R_mín`).

4.2 Los 8 errores más frecuentes del Bloque A

#	El error	Lo correcto	Por qué
1	Meter `R` (bps) en `d/s`.	`d/s` solo lleva `d` (m) y `s` (m/s).	`s` es del **medio**; `R` del **equipo**.
2	Confundir `R` con `s`.	`R` en bps, `s` en m/s.	"Cuánta info/s" contra "qué tan rápido viaja".
3	Poner `n·d_proc` con `n` enlaces.	Con `n` enlaces hay `n−1` routers.	Los extremos A y B son hosts.
4	Poner `2·B·log₂(1+S/N)`.	Shannon es `B·log₂(1+S/N)`, **sin el 2**.	El `2` es de Nyquist.
5	Meter `S/N` en dB.	Convertir: `10^(dB/10)`.	Shannon pide el cociente **lineal**.
6	Confundir latencia con RTT.	`latencia ≈ RTT/2` **solo** con camino simétrico.	Ida y vuelta pueden diferir.
7	Decir que el jitter es el retardo.	Jitter = **variación** del retardo.	Retardo alto y constante no rompe el audio; el jitter sí.
8	Decir que la fibra es "más rápida".	La fibra tiene `s` apenas mayor y mucha más **alcance**.	`s` es casi igual; cambia la atenuación: 100 m contra 100 km.
8	Decir que la fibra es "más rápida" que el cobre.	La fibra tiene `s` apenas mayor y mucha más **alcance**.	`s` es casi igual; cambia la atenuación: 100 m contra 100 km.

4.3 Enunciado del simulacro (sin respuestas)

Preguntas Teóricas

T1 (10 puntos). En una red de paquetes, un paquete de longitud `L` cruza un medio. a) Identifique los **componentes del retardo** y diga cuáles son los más significativos. b) Diferencie el **ancho de banda** de la **velocidad de propagación** en el medio. c) ¿En qué componente del retardo influye la **congestión**?

T2 (10 puntos). En una red de conmutación de paquetes: a) Explique los conceptos de **congestión**, **pérdida** y **retardo**, y cómo se relacionan entre sí. b) Una red pierde paquetes. Mencione **dos causas de pérdida que no sean por congestión**, y explique por qué un protocolo de transporte como TCP no puede distinguirlas.

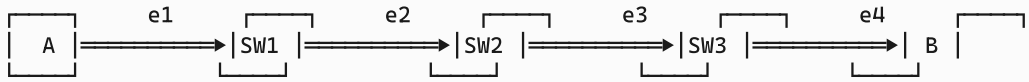
T3 (8 puntos). Dos usuarios comparten un canal de comunicación. a) Indique el significado de **FDMA** y de **TDMA**, y explique el principio de operación de cada uno. b) ¿Son equitativos? ¿En qué sentido de "equitativo" se parecen a la conmutación por circuitos y en cuál a la de paquetes?

T4 (8 puntos). Sobre las tecnologías de transmisión: a) Indique en qué consisten el modo **full duplex** y el modo **half duplex**. b) Especifique para **Ethernet (IEEE 802.3)** y para **Wi-Fi (IEEE 802.11)** qué modo se usa y justifique. Explique por qué en uno de los dos el otro modo es imposible.

T5 (4 puntos). Compare la conmutación por **circuito**, por **paquetes** y por **mensajes**: indique para cada una si reserva recursos en la red y qué unidad de datos transmite.

Problemas Prácticos

Problema 1 (30 puntos) — Retardos. Un paquete de `L = 12 000 bits` viaja de A hasta B por **cuatro enlaces** unidos por **tres conmutadores**.



Enlace	R	d (todos con $s = 2 \cdot 10^8$ m/s)
`e1`	2 Mbps	500 km
`e2`	100 Mbps	200 km
`e3`	100 Mbps	2000 km
`e4`	1 Gbps	50 km

Cada conmutador retarda cada paquete $d_{proc} = 4\ \mu s$. No hay retardo de cola.

a) Escriba la fórmula general del retardo total extremo a extremo, indicando cuántas veces aparece cada término y **por qué** esas cantidades. **b)** Calcule el retardo total con la aritmética a la vista, y diga qué componente domina y qué porcentaje representa. **c)** Suponga ahora $R_1 = R_2 = R_3 = R_4 = R = 100$ Mbps, $d_{proc} = 0$, y que cada conmutador **transmite inmediatamente cada bit que recibe, sin esperar a que llegue el paquete completo**. Escriba el nuevo retardo, explique **por qué** mejora respecto de (b) y **en qué condición dejaría de mejorar**. **d)** Calcule el RTT de la conexión y el *bandwidth-delay product* del enlace `e2`.

Problema 2 (30 puntos) — Nyquist, Shannon, encapsulación y capas. Dos sedes unidas por un enlace de radio de **B = 500 kHz**, con **S/N = 40 dB** y modulación **16-QAM**. Cada router tiene $d_{proc} = 3\ \mu s$.

a) Escriba la fórmula de **Nyquist** con el significado y las unidades de cada variable, y calcule la capacidad máxima del enlace con esa modulación. **b)** Escriba la fórmula de **Shannon**, **convierta el S/N a lineal** mostrando la cuenta, y calcule la capacidad máxima. Explique cuál de los dos resultados manda y por qué. **c)** Para un mensaje de aplicación de 1200 bytes que viaja por capa 2, 3 y 4 (Ethernet, IP, TCP), calcule el **tamaño de cada PDU** con cabeceras típicas (Ethernet 18 B, IP 20 B, TCP 20 B) y la **eficiencia** resultante. Diga qué capa se ocupa de qué. **d)** Mencione las **capas del modelo OSI** y sus **PDU**, y diga en cuál se ubica cada uno de los cálculos anteriores (capacidad, retardo, tamaño del segmento). Justifique.

4.4 Resolución del simulacro

T1 (10 puntos) — Componentes del retardo, BW contra propagación, congestión

```
d_end2end = Σ ( d_proc + d_cola + d_transmisión + d_propagación )
```

a) Los **cuatro componentes** son: (1) **procesamiento** d_{proc} (CRC, leer la cabecera, buscar el puerto de salida); (2) **encolamiento** d_{cola} (espera en la cola del enlace de salida); (3) **transmisión** $d_{trans} = L/R$ (meter el paquete entero en el cable); (4) **propagación** $d_{prop} = d/s$ (la señal recorriendo el medio).

Los más significativos son $d_{transmisión}$ y $d_{propagación}$: son los únicos **calculables** con los datos del enunciado (L, R, d, s) y los que **dominan** en enlaces de cualquier longitud relevante. d_{proc} es despreciable (microsegundos) y d_{cola} no se puede calcular a priori, porque depende del estado de la cola.

b) El **ancho de banda** R se mide en **bps** y depende del **equipo** (puerto, transceptor, modulación); aparece en $d_{trans} = L/R$. La **velocidad de propagación** s se mide en **m/s** y depende del **medio físico**; aparece en

`d_prop = d/s`. Duplicar el ancho de banda reduce `d_trans` a la mitad y **no cambia** `d_prop`: un enlace de 1 Gbps no es "50 veces más rápido" que uno de 100 Mbps, es 50 veces más **ancho**.

c) La congestión influye **solo en el retardo de encolamiento** `d_cola`, que es el único componente que depende del estado de la cola, y la congestión **es** el estado de la cola. La cadena: más tráfico → las colas se agrandan → `d_cola` sube → el retardo total sube; y si la cola desborda el buffer, aparecen **pérdidas**. Los otros tres (`d_proc`, `d_trans`, `d_prop`) no dependen de la carga.

T2 (10 puntos) — Congestión, pérdidas y retardo

a) Congestión: escasez, **durante un período de tiempo**, de los **recursos compartidos** para procesar todos los paquetes que los pretenden usar (capacidad de procesamiento y buffers de los routers, y ancho de banda de los enlaces). Se manifiesta en **pérdidas y/o retardos**. **Pérdida:** un paquete **se descarta cuando no encuentra lugar en un buffer**; es **irreversible**, porque IP no reintenta. **Retardo:** el tiempo que tarda un paquete en ir de un nodo al siguiente, con sus cuatro componentes.

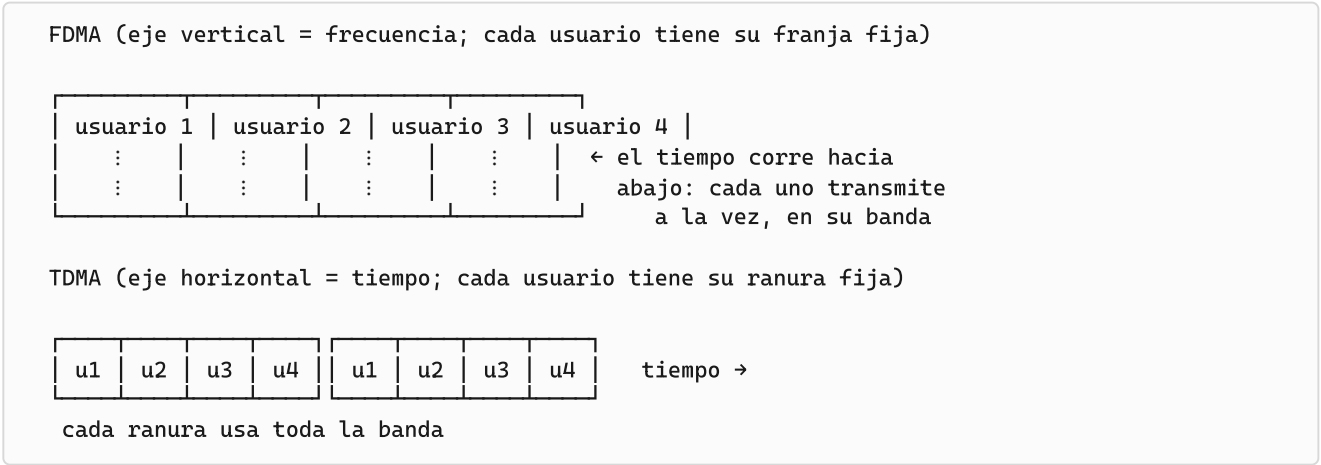
La relación: la **congestión es la causa**; el **retardo de cola** y la **pérdida** son los **dos síntomas** y vienen del mismo mecanismo. Primero aparece el retardo (la cola crece) y después, si se desborda el buffer, la pérdida.

b) Dos causas que **no** son por congestión: un **checksum o CRC incorrecto** por un error del medio, y un **descarte por ACL o firewall**, o porque el **TTL llega a 0**, o porque se perdió un **fragmento** en IPv4.

Por qué TCP no las distingue: un paquete perdido es un paquete perdido; para TCP los tres casos son **lo mismo**, y no hay forma de saber *por qué* se perdió desde el emisor. Por eso su reacción (reducir la ventana) es **conservadora en unos casos y correcta en otros**.

T3 (8 puntos) — FDMA y TDMA

a) FDMA (*Frequency Division Multiple Access*): divide el **espectro de frecuencias** en sub-bandas y asigna **una sub-banda fija a cada usuario**, que la usa **todo el tiempo**. **TDMA** (*Time Division Multiple Access*): un usuario usa **toda la banda disponible** pero **solo en su ranura de tiempo** (*timeslot*), que se repite periódicamente.



b) Los dos son **equitativos y estáticos**: a cada usuario se le asigna siempre la misma fracción del recurso, sin reparto dinámico. Se parecen a la **conmutación por circuitos** (asignación fija) y no a la de paquetes, que no reserva nada y aprovecha el canal **estadísticamente**, desperdiciando menos capacidad cuando hay usuarios inactivos.

T4 (8 puntos) — Full duplex y half duplex

a) Full duplex: se puede **transmitir y recibir a la vez**. **Half duplex:** se puede transmitir o recibir, pero **no simultáneamente** (se alternan por turnos).

b) Ethernet (IEEE 802.3) con switch: full duplex. Cada puerto es un enlace punto a punto dedicado, con un par para transmitir y otro para recibir, así que no hay colisión. (En los **hubs** antiguos, half duplex con CSMA/CD.) **Wi-Fi (IEEE 802.11): siempre half duplex.** El medio es el **aire**, inherentemente compartido.

Por qué en Wi-Fi el otro modo es imposible: una radio **no puede transmitir y recibir a la vez en la misma banda**, así que **no puede detectar una colisión mientras transmite**: nadie "oye" el choque. Por eso Wi-Fi usa **CSMA/CA** en lugar de CSMA/CD: en vez de detectar la colisión *después* de que ocurrió, la **evita** de antemano, con el tiempo de espera aleatorio (*backoff*), el **NAV** (*Network Allocation Vector*) y el **RTS/CTS** opcional, que resuelve el nodo oculto.

T5 (4 puntos) — Las tres técnicas de conmutación

	Circuito	Paquetes	Mensajes
Unidad	flujo continuo de datos	paquete de tamaño fijo	mensaje completo
Recursos	**sí** (reserva ancho de banda y buffers en cada nodo)	**no** (mejor esfuerzo)	**no**

- **Circuito:** se establece un camino y se **reservan** recursos, como una línea telefónica dedicada: retardo fijo y predecible, pero desperdicia capacidad cuando el usuario está callado.
- **Paquetes:** cada paquete se enruta por separado, **sin reservar**: aprovecha el canal estadísticamente, con retardo variable y posibles pérdidas.
- **Mensajes:** se transmite el mensaje entero, todo o nada, sin conexión ni reserva.

Problema 1 (30 puntos) — Retardos · Resolución

a) Fórmula general y conteo de términos. Con `n = 4` enlaces hay **4** términos de `d_trans` y **4** de `d_prop`; con `k = 3` conmutadores hay **3** de `d_proc` y **3** de `d_cola` (que aquí valen 0).

```
d_end2end = Σ_{i=1..4} [ d_proc + d_cola + L/R_i + d_i/s_i ]
          = 3·d_proc + (L/R₁+L/R₂+L/R₃+L/R₄) + (d₁/s₁+d₂/s₂+d₃/s₃+d₄/s₄)

⚠️ El d_proc va 3 VECES (uno por conmutador), NO 4: hay 4 enlaces pero
   solo 3 conmutadores; A y B son sistemas terminales.
```

b) Cálculo del retardo total con la aritmética a la vista.

```
L = 12 000 bits

e1: R₁ = 2·10⁶,  d₁ = 5·10⁵ m,  s₁ = 2·10⁸
   d_trans = 12 000/2·10⁶  = 6,0·10⁻³ s =  6,000 ms
   d_prop  = 5·10⁵/2·10⁸   = 2,5·10⁻³ s =  2,500 ms   subtotal  8,500 ms

e2: R₂ = 10⁸,  d₂ = 2·10⁵ m
   d_trans = 12 000/10⁸    = 1,2·10⁻⁴ s =  0,120 ms
   d_prop  = 2·10⁵/2·10⁸   = 1,0·10⁻³ s =  1,000 ms   subtotal  1,120 ms

e3: R₃ = 10⁸,  d₃ = 2·10⁶ m
   d_trans = 12 000/10⁸    = 1,2·10⁻⁴ s =  0,120 ms
   d_prop  = 2·10⁶/2·10⁸   = 1,0·10⁻² s = 10,000 ms   subtotal 10,120 ms

e4: R₄ = 10⁹,  d₄ = 5·10⁴ m
   d_trans = 12 000/10⁹    = 1,2·10⁻⁵ s =  0,012 ms
   d_prop  = 5·10⁴/2·10⁸   = 2,5·10⁻⁴ s =  0,250 ms   subtotal  0,262 ms

d_proc total = 3 × 4 µs = 0,012 ms        d_cola total = 0

d_end2end = 8,500 + 1,120 + 10,120 + 0,262 + 0,012 = 20,014 ms
```

Qué componente domina y qué porcentaje:

```
d_propagación  = 2,500 + 1,000 + 10,000 + 0,250 = 13,750 ms  →  68,7 %  ←— DOMINA
d_transmisión  = 6,000 + 0,120 +   0,120 + 0,012 =  6,252 ms  →  31,2 %
d_procesamiento                                      0,012 ms  →   0,06 %
TOTAL                                                 20,014 ms  → 100 %

El enlace e3 aporta 10,120 ms (50,6 %) y es el más lento por distancia;
el e1 aporta 8,500 ms y es el de menor ancho de banda. Son dos cuellos de
botella distintos: uno por distancia, el otro por ancho de banda.
```

c) Cut-through (reenvío al vuelo), `R` uniforme y `d_proc = 0`.

```
R₁ = R₂ = R₃ = R₄ = R = 100 Mbps = 10⁸ b/s, y cada conmutador transmite
cada bit apenas lo recibe.

d_end2end = 1 · (L/R) + Σ_{i=1..4} (d_i/s_i)
          = 12 000/10⁸ + (5·10⁵ + 2·10⁵ + 2·10⁶ + 5·10⁴)/2·10⁸
          = 1,2·10⁻⁴ s + (2,85·10⁶ / 2·10⁸) s
          = 0,00012 s + 0,01425 s
          = 0,01437 s = 14,370 ms
```

Por qué mejora, y en qué condición dejaría:

> En store-and-forward cada conmutador debe **recibir el paquete completo** antes de emitirlo por el enlace siguiente (necesita leer la MAC destino), así que cada enlace intermedio aporta **otro `L/R` en serie**. Con cut-through el conmutador **no espera**: reenvía cada bit apenas lo recibe, así que **los `L/R` se pisan en vez de sumarse** y queda **uno solo**, y además desaparece el `d_proc`.
>
> **Condición:** el cut-through solo funciona si `R_entrada = R_salida` en **todos** los enlaces (aquí se cumple: los cuatro son 100 Mbps). Si un conmutador recibiera más rápido de lo que transmite (`R₁ = 1 Gbps` y `R₂ = 100 Mbps`), se acumularía cola y **volvería a comportarse como store-and-forward**, y además propagaría la congestión antes de tiempo (*congestion crunch*).

> ⚠️ **Ojo al comparar (b) con (c):** los dos resultados no son comparablesdirectly, porque en (b) los `R` y las `d` son heterogéneos y en (c) todo es 100 Mbps. La comparación justa es "a igual topología", y ahí la mejora es `(n−1)·(L/R) + k·d_proc`.

d) RTT y BDP del enlace `e2`.

```
Latencia de ida por e2 = 1,000 + 0,120 = 1,120 ms
RTT (camino simétrico) ≈ 2 × 1,120 = 2,240 ms = 2,24 s
BDP = R₂ × RTT = 10⁸ × 2,24 = 2,24·10⁸ bits = 28 000 000 bytes = 28 MB

⚠️ El BDP NO depende del tamaño del paquete, sino de R y del RTT. Con un RTT de 2,24 s el
   enlace necesita 28 MB "en vuelo" para llenarse: eso exige una ventana enorme, y de ahí
   `RFC 7323` (*Window Scale*).
```

Puntos: (a) fórmula y conteo (8), (b) aritmética (10), (c) cut-through (8), (d) RTT y BDP (4).

Problema 2 (30 puntos) — Nyquist, Shannon, encapsulación, capas · Resolución

a) Nyquist: fórmula, variables y cálculo.

```
C_N = 2 · B · log₂(V)   [bps]
· B = ancho de banda del enlace, en HERTZ (Hz).  Aquí B = 500 kHz = 5·10⁵ Hz.
· V = niveles de la señal (estados por símbolo). Con 16-QAM, V = 16.

C_N = 2 · 5·10⁵ · log₂(16) = 2 · 5·10⁵ · 4 = 4·10⁶ bps = 4 Mbps
```

b) Shannon: fórmula, conversión del S/N y cálculo.

```
C_S = B · log₂(1 + S/N)   [bps],  con S/N EN LINEAL

1) Convertir de dB a lineal:
      S/N_dB = 40
      S/N_lineal = 10^(40/10) = 10⁴ = 10 000

2) Aplicar Shannon:
      C_S = 5·10⁵ · log₂(1 + 10 000) = 5·10⁵ · log₂(10 001)

3) El logaritmo: 2¹³ = 8192 y 2¹⁴ = 16384, así que 10 001 está entre
   ambos y log₂(10 001) ≈ 13,29.

4) C_S = 5·10⁵ · 13,29 ≈ 6,64·10⁶ bps ≈ 6,64 Mbps
```

Cuál manda y por qué:

> La capacidad real de un enlace es el **mínimo de los dos techos**, y en la práctica no alcanza ninguno: los medios reales **siempre tienen ruido**, y además hay pérdida de sincronismo, atenuación y overhead, así que los sistemas reales se **diseñan al 50-70 % de Shannon**.
>
> En este caso el enunciado **fija la modulación** (`V = 16`), así que Nyquist da 4 Mbps y el cuello es el **transmisor**; el piso que impone la calidad del medio es Shannon, con 6,64 Mbps.

c) Tamaño de las PDU y eficiencia. Con cabecera TCP de 20 B, IP de 20 B y trama Ethernet de 18 B (preámbulo y FCS incluidos):

```
Capa 4 (Transporte):  segmento TCP = 1200 + 20 = 1220 bytes
Capa 3 (Red):         paquete IP   = 1220 + 20 = 1240 bytes
Capa 2 (Enlace):      trama        = 1240 + 18 = 1258 bytes
Eficiencia  η = 1200 / 1258 ≈ 0,954  →  95,4 %
```

Qué capa se ocupa de qué: la **capa 2** arma la trama (MAC y CRC); la **capa 3** arma el paquete (IP y enrutamiento); la **capa 4** arma el segmento (puertos, secuencia y checksum); y la **capa 1** transmite los bits por el medio.

d) Las capas del modelo OSI, sus PDU y dónde caen los cálculos.

```
Capa OSI      PDU                ¿Qué calcula?
──────────────────────────────────────────────────────────────
7–5 Aplicación, mensaje           – (los datos del usuario)
4 Transporte  segmento            – el tamaño del segmento (apartado c)
3 Red         paquete             – el retardo extremo a extremo
2 Enlace      trama               – el tamaño de la trama (apartado c)
1 Física      bit                 – la CAPACIDAD: Nyquist y Shannon
```

Justificación: la **capacidad** (Nyquist y Shannon) es propiedad de la **capa física**, porque depende solo del medio y del transceptor. El **retardo extremo a extremo** es un concepto de la **capa 3 (Red)**: es el tiempo que tarda un **paquete** en viajar de un nodo a otro. El **tamaño del segmento** depende de la **capa 4 (Transporte)**, que agrega la cabecera TCP.

PARTE 5 — CIERRE

📌 Resumen

1. La **capa física** solo transmite **bits**: no detecta errores, no reenvía, no reintenta.
2. Los cuatro medios: **par trenzado** (100 m, sensible al EMI), **coaxial** (~10 Mbps), **fibra** (inmune al EMI, 100 km), **inalámbrico** (aire compartido).
3. **Full duplex** = transmitir y recibir a la vez; **half duplex** = por turnos. **Ethernet conmutado es full duplex; Wi-Fi es siempre half duplex.**
4. **Nyquist**: `C = 2·B·log₂(V)`, medio **sin ruido**, techo de la modulación.
5. **Shannon**: `C = B·log₂(1+S/N)`, medio **con ruido gaussiano**, `S/N` **en lineal** (`10^(dB/10)`), **sin el 2 delante**.
6. **Shannon siempre queda por debajo de Nyquist**, y el diseño real queda por debajo de Shannon.
7. **QAM** combina amplitud y fase: una constelación de `M` puntos vale `log₂(M)` bits por símbolo.
8. Con `n` **enlaces** y `k` **conmutadores**: `k·d_proc + n·(L/R) + n·(d/s)`. **Cut-through** deja **un solo** `L/R`.
9. **Satélite**: 120 ms de ida, irrecuperables. El mejor ejemplo de por qué ancho de banda y latencia son cosas distintas.
10. Nyquist es `2·B·log₂(V)` y Shannon `B·log₂(1+S/N)`: **nunca se mezclan.**

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un enlace de cobre de 3 km soporta 100 Mbps, con `s = 2·10⁸ m/s`. ¿Cuánto tarda la **propagación** de un bit? ¿Y la **transmisión** de un paquete de 1500 bytes? Si duplico el ancho de banda, ¿cambia la propagación?

<details>
<summary>Respuesta</summary>
</details>

2. ¿Por qué la velocidad de propagación es casi la misma en cobre y en fibra, si la fibra "gana"? ¿Qué gana entonces?

<details>
<summary>Respuesta</summary>
</details>

3. Un canal tiene `B = 4000 Hz` y `S/N = 20 dB`. ¿Cuál es la capacidad de Shannon?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué la fibra es inmune al **EMI**, y por qué el par trenzado también reduce el ruido si es cobre?

<details>
<summary>Respuesta</summary>
</details>

5. Si duplico la longitud del paquete, ¿qué componente del retardo cambia? ¿Y si duplico la distancia? ¿Y el ancho de banda?

<details>
<summary>Respuesta</summary>
</details>

6. ¿En cuál de las tres técnicas de conmutación se **reservan** recursos y por qué tiene un precio?

▶ Respuesta

7. ¿Por qué no se puede implementar full duplex en Wi-Fi, y por qué usa CSMA/CA en vez de CSMA/CD?

▶ Respuesta

8. ¿Cuál de los ocho errores es el más caro? Da un ejemplo numérico.

▶ Respuesta

🔗 Referencias

Exámenes - `sirc0608` (5-ago-2006) — **P1 (5 pts): Nyquist y Shannon con sus hipótesis**. §3.1. - `sirc0707` (31-jul-2007) — **P1 (10 pts): capacidad máxima sobre línea telefónica**. §3.2. - `sirc0902` (2-feb-2009) — P3 (5 pts): FDMA y TDMA. Simulacro 1, T3. - `sirc0903` (20-feb-2009) — P5 (6 pts): full duplex contra half duplex. §1.3 y T4. - `sirc0912` (18-dic-2009) — P1 (10 pts): componentes del retardo, BW contra propagación, congestión. Clase 5 §3.1 y T1. - `sirc1103` (21-feb-2011) — P2 (6 pts): congestión, pérdidas y retardo. Clase 5 §3.2 y T2. - `sirc0712` (dic-2007) — P2 (4 pts): datagramas contra circuitos virtuales. T5.

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., cap. 1 §1.1 (la capa física y los medios) y §1.6 (el espectro inalámbrico).

Normas (no RFC) - ⚠️ **Nyquist y Shannon no tienen RFC**: son resultados matemáticos, no protocolos. La capa física y la de enlace se estandarizan por **IEEE** (802.3, 802.11, 802.1Q); la capa física es **anterior a Internet**. - `IEEE 802.11be-2024` — Wi-Fi 7, publicada el 22-jul-2025 (320 MHz y 4096-QAM). - `IEEE 802.3cg` — 10BASE-T1S, obligatoriamente half-duplex con PLCA.

Clases relacionadas - Clase 5 y 6 (el retardo y su taller) · Clase 8 (HTTP) · Clase 19 (BDP) · Clase 38 y 39 (capa de enlace).

Clase 8

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El modelo cliente/servidor
1.2 HTTP: qué es y qué NO es
1.3 El mensaje HTTP: la petición
1.4 El mensaje HTTP: la respuesta
1.5 Los códigos de estado: las cinco clases
1.6 HTTP no persistente vs. persistente — el conteo de RTT
1.7 Las estadísticas: por qué HTTP se siente lento
1.8 Las cookies
1.9 ⚠️ Actualización a 2026 — la especificación de HTTP que hay que corregir
2.1 Ejercicio 1 — Contar RTTs para 1 y 11 objetos (★ el del examen)
2.2 Ejercicio 2 — Interpretar un 304
2.3 Ejercicio 3 — ¿Por qué HTTP es sin estado y cómo lo resuelven las cookies?
3.1 ⭐ Examen 2010-07, Pregunta 1 (8 puntos) — página HTML con 10 imágenes
3.2 Examen 2009-07, Problema 2 — el `getMailServer()` como puente
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: B (Aplicación) · **Semana:** 3 · **Duración estimada:** 110 min **Correlativa:** Clase 7 (la capa de aplicación: peer-to-peer y cliente/servidor) · **Sigue con:** Clase 9 (HTTP a fondo: mensajes, cookies, caché y versiones) **Material de referencia:** Tanenbaum cap. 7 (§7.1 el modelo cliente/servidor, §7.3 HTTP) · RFC 9110, RFC 9112, RFC 9113, RFC 9114, RFC 6265, RFC 7540 (obsoleto), RFC 7838

Del clic a la primera respuesta

🎯 Al terminar esta clase vas a poder

1. Explicar qué es el modelo **cliente/servidor**, por qué existe y en qué se diferencia del **peer-to-peer**, y por qué la asimetría (servidor pasivo, cliente activo) simplifica toda la capa de aplicación.
2. Enumerar **qué es HTTP y qué NO es**: por qué no es la Web, por qué no es un protocolo de transporte, por qué no tiene estado y por qué no es seguro por sí mismo.

3. Escribir **de memoria** un mensaje de petición y uno de respuesta HTTP/1.1 completos, con todas sus cabeceras, y explicar qué significa cada código de estado.
4. Calcular **cuántos RTT** cuesta una página con **1 objeto** y con **11 objetos**, en HTTP no persistente y en HTTP persistente, con un solo servidor y con varios, justificando cada RTT.
5. Explicar por qué **304 Not Modified** existe, cómo se combine con `ETag` y `Last-Modified`, y por qué ahorra transferencia.
6. Explicar qué son las **cookies**, qué problema resuelven, dónde vive el estado y por qué `RFC 6265` las estandariza.
7. Resolver completo el **Examen 2010-07 Pregunta 1** (8 puntos), con la cuenta de RTTs.

> **Prerequisito del examen:** `2010-07 P1` (8 puntos — ★ el problema central de esta clase), `2009-07 Problema 2` (30 puntos — cliente anti-spam vía HTTP, parte b), `2010-02 P1` (qué es DNS, misma lógica conceptual).

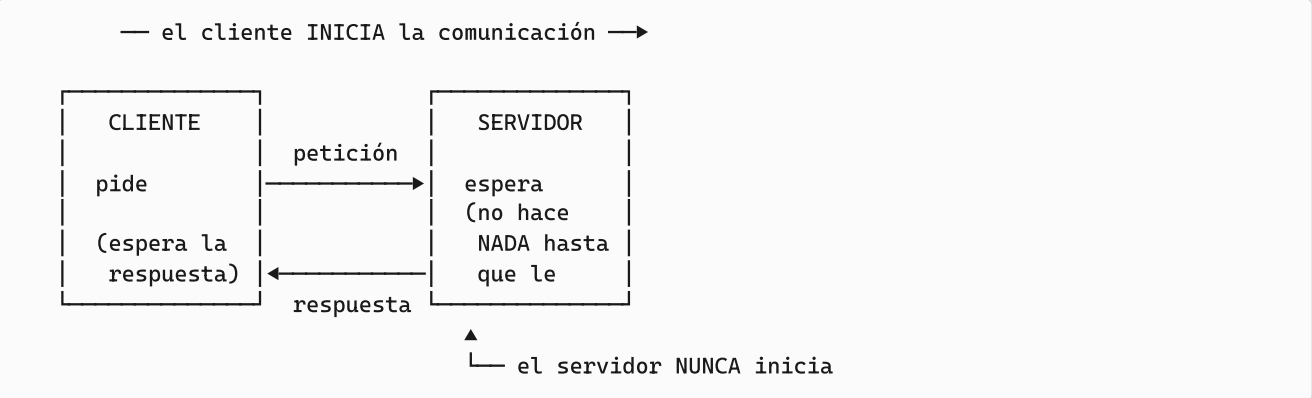
PARTE 1 — TEORÍA

1.1 El modelo cliente/servidor

Qué es

El modelo **cliente/servidor** (*client/server*) es una organización de la computación en la que los procesos se dividen en **dos roles que no son simétricos**:

- El **servidor** (server) es un proceso que **espera**. Ofrece un servicio sobre un puerto bien conocido y no hace nada hasta que alguien le hable.
- El **cliente** (client) es un proceso que **inicia**. Decide hablar, contacta al servidor y consume el servicio.



La asimetría es el dato de diseño central. Un servidor nunca genera tráfico espontáneo hacia un cliente: **solo responde**. Un cliente solo genera tráfico porque **quiso**.

Por qué existe

La razón profunda es la **asimetría de roles entre los extremos**. La Web no es simétrica: hay millones de navegadores (que cambian, se mueven, se apagan) y hay decenas de miles de servidores de contenido (que son máquinas con costos fijos, permanecen y son caros de mantener). Un diseño donde cualquiera pueda servir y servirse sería insostenible: los roles no son intercambiables.

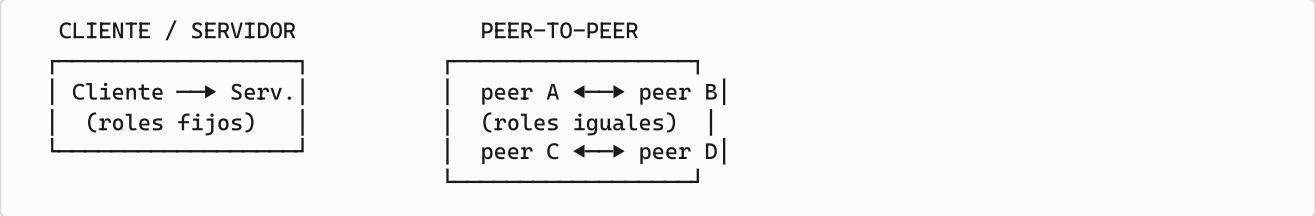
El modelo cliente/servidor **externaliza** el problema: en vez de que cada host mantenga una copia de todo el contenido, el contenido vive en **un** servidor y los clientes van a buscarlo. Las tres consecuencias prácticas que hacen que gane:

Consecuencia	Por qué importa
El servidor no necesita saber quién es el cliente	Basta con direccionar a un `host:puerto`. No hay que "descubrir" al cliente.
El servidor es pasivo ⇒ no hay que resolver el "quién llama"	El problema de "quién está llamando" desaparece: el que llama se identifica solo al abrir la conexión.
Se pueden atender muchos clientes desde un servidor	Un servidor HTTP atiende miles de conexiones simultáneas; los clientes son desechables.

La tercera consecuencia es la que habilita la **Web** como la conocemos: si cada visitante tuviera que publicar su propio contenido desde su casa, no habría nada que ver.

Cliente/servidor vs. peer-to-peer

El **peer-to-peer** (*P2P*, pares) es el modelo opuesto: **todos los nodos son simultáneamente clientes y servidores**, y no hay un servidor distinguible. Es el modelo de BitTorrent, de Skype en su versión original, y de las redes de compartimiento de archivos.



	Cliente/servidor	Peer-to-peer
Roles	fijos y distintos	todos hacen de todo
¿Quién inicia?	el cliente	cualquiera
Centralización	hay un punto central (el servidor)	ninguno
Escala	agregar clientes es barato; el **servidor es el cuello de botella**	escala con el número de pares
Si el servidor cae	el servicio cae	el servicio sigue
Ejemplos	Web, correo, DNS	BitTorrent, Skype, IPFS

Ojo con el ejemplo del examen: el correo electrónico es cliente/servidor aunque el cliente "mande" el correo al servidor. Mandar es **iniciar**, no servir.

> **Por qué importa en el examen:** el `2010-07 P1` es enteramente cliente/servidor y la cuenta de RTTs sale de ahí. Y el `2010-12 Problema 1` (el `connectCheaper()`) es un ejemplo híbrido interesante: los **APs son servidores** (hacen broadcast de precios, un comportamiento de *push* típico de servidor) y el usuario es cliente que **inicia** el CONNECT.

1.2 HTTP: qué es y qué NO es

HTTP = HyperText Transfer Protocol. Es un protocolo de **capa de aplicación** que transfiere recursos (objetos) entre un cliente y un servidor, con el esquema petición/respuesta. Para entenderlo bien, lo más útil es la lista de lo que **no** es, porque cada "no es" explica una pregunta de examen.

(a) HTTP no es un protocolo de transporte

HTTP **no** garantiza entrega, no numera, no retransmite, no ordena y no controla la congestión: todo eso lo hace **TCP**, debajo. Corolario práctico: **HTTP no sabe si un mensaje llegó.** Si se pierde, TCP lo reintenta por abajo y HTTP nunca se entera.

(b) HTTP no es lo mismo que la Web

```
La WEB  =  un conjunto de TECNOLOGÍAS
   ├── un espacio de nombres:   los URL (y el DNS que los resuelve)
   ├── un protocolo de transferencia:  HTTP
   └── un formato de documento:         HTML

HTTP    =  SOLO el protocolo de transferencia de esos documentos
```

Decir "voy por la Web" mezcla tres capas distintas. Si te preguntan "qué es HTTP", la respuesta empieza por "es **un** protocolo de capa de aplicación para transferir objetos identificados por URL".

(c) HTTP no tiene estado

Sin estado (*stateless*) significa: **cada petición se atiende sola, sin memoria de las anteriores**.

```
Cliente                              Servidor
   |  GET /carrito                     |
   |──────────────────────────────────►|  ¿Y qué compró el usuario 17?
   |                                   |   NO SÉ: HTTP es sin estado
   |  404 (o un carrito vacío)         |
   |◄──────────────────────────────────|
   |  GET /carrito?user=17             |   ← el estado va en la URL
   |──────────────────────────────────►|  (o en una cookie)
   |  200 {"camisa": 2}                |
   |◄──────────────────────────────────|
```

Esto es una decisión de diseño, no una limitación. Un protocolo con estado obligaría al servidor a almacenar quién pidió qué, y eso rompe el balanceo de carga: si el estado está en el servidor, todas las peticiones de un usuario tienen que ir al **mismo** servidor y no podés escalar horizontalmente. HTTP prefirió **stateless** y corrió el riesgo de que el estado viaje en la petición. Es el mismo compromiso que "DNS versus NAT": simplicidad a cambio de eficiencia.

(d) HTTP no es seguro, y viaja sobre TCP

En texto plano HTTP viaja **tal cual** por la red: cualquiera en el camino ve las cabeceras `Authorization`, `Cookie` y los formularios con tu contraseña. Para eso está **HTTPS**, que es HTTP sobre **TLS**: **HTTP aporta semántica, TLS aporta confidencialidad, integridad y autenticidad** (la Clase 47 muestra TLS entero).

Y HTTP **no viaja sobre UDP**: viaja sobre **TCP, puerto 80** (o **443** con TLS). Eso es lo que lo hace fiable y lo que lo hace lento: la fiabilidad tiene un costo de setup.

1.3 El mensaje HTTP: la petición

Una petición HTTP/1.1 tiene **tres** partes, en este orden y sin falta:

```
línea de petición (request line)  +  cabeceras  +  línea en blanco  +  cuerpo
──────────────────────────────────   ──────────   ─────────────────   ──────

GET /index.html HTTP/1.1             Host: ...                       (puede
<CR><LF>                             Accept: ...                     ir vacío)
<CR><LF>
```

La línea en blanco final es obligatoria, aunque el cuerpo esté vacío. Sin ella el servidor no sabe dónde termina la cabecera. Esto es el mismo principio del **framing** de la capa 2: hay que delimitar.

Un ejemplo **real y completo**, tal como lo manda un navegador:

```
GET /index.html HTTP/1.1
Host: www.fing.edu.uy
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Encoding: gzip
Connection: keep-alive
```

Fijate que **la última línea es vacía**: ese es el `CRLF` extra que cierra las cabeceras. Sin esa línea en blanco, HTTP no parsea.

Desglose de la línea de petición, campo por campo:

Campo	Significado	Ejemplo
Método	qué quiere el cliente	`GET`
URI (*Uniform Resource Identifier*)	qué recurso	`/index.html`
Versión	qué dialecto habla el cliente	`HTTP/1.1`

Y de las cabeceras que aparecen:

Cabecera	Para qué sirve	Por qué importa
`Host`	**obligatoria** en HTTP/1.1: dice a qué **servidor** dentro del destino querés llegar	Sin ella, un servidor que aloja miles de sitios (como un CDD) no sabe cuál teSirve. Es el motivo de que HTTP/1.1 no pueda esconderse tras NAT.
`User-Agent`	identifica el cliente	Los servidores usan esto para decidir qué versión mandarte.
`Accept`	qué **tipos de contenido** aceptás	`text/html`, `image/png`
`Accept-Encoding`	qué **codificaciones** de compresión aceptás	`gzip`, `br`. El servidor responde `Content-Encoding: gzip` si lo comprime.
`Connection: keep-alive`	el cliente pide **reusar** la conexión	En HTTP/1.1 es el default, pero lo digo explícito para que no se note.

Los métodos: seguros e idempotentes

Seguro = no cambia el estado del servidor. **Idempotente** = si lo repetís *n* veces el efecto es el mismo que si lo hicieras una. `GET`, `HEAD`, `OPTIONS` y `TRACE` son **seguros**; `GET` y `HEAD` son además **idempotentes**, igual que `PUT` y `DELETE`. `POST` **no es ni seguro ni idempotente**.

Por qué importa. Un cliente que reintenta por timeout necesita saber si puede reintentar. Si la respuesta se perdió pero el servidor sí la procesó, reintentar un `POST` duplica el cobro; reintentar un `GET` no pasa nada. La Clase 9 desarrolla métodos, seguridad e idempotencia en detalle.

1.4 El mensaje HTTP: la respuesta

```
línea de estado (status line)  +  cabeceras  +  línea en blanco  +  cuerpo
```

Un **200 OK** completo:

```
HTTP/1.1 200 OK
Date: Tue, 06 Oct 2026 14:22:31 GMT
Server: Apache/2.4.62 (Ubuntu)
Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT
ETag: "3f2a1c8-9b41-6512d3f0"
Cache-Control: max-age=600
Content-Type: text/html; charset=utf-8
Content-Encoding: gzip
Content-Length: 4096
Accept-Ranges: bytes
Connection: keep-alive

<!DOCTYPE html>
<html lang="es">
<head><meta charset="utf-8"><title>FING</title></head>
<body>...</body>
</html>
```

La línea de estado son tres cosas separadas por espacio:

Parte	Qué es	Ejemplo
Versión	el dialecto que devuelve el servidor	`HTTP/1.1`
Código	número de 3 dígitos	`200`
Frase de razón	texto legible, decorativo	`OK`

Las dos cabeceras que hacen que esto funcione y que hay que mirar siempre:

- `Content-Length`: cuántos bytes tiene el cuerpo. Es lo que le permite al cliente saber dónde termina la respuesta **sin** tener que cerrar la conexión.
- `Content-Type`: qué **tipo** es el cuerpo (`text/html`, `image/png`). Sin esto el cliente no sabría si tiene que mostrarlo, guardarlo o ejecutarlo.

Un 304 Not Modified (y por qué existe)

```
HTTP/1.1 304 Not Modified
Date: Tue, 06 Oct 2026 14:22:33 GMT
ETag: "3f2a1c8-9b41-6512d3f0"
Cache-Control: max-age=600
Connection: keep-alive
```

Fijate que NO hay cuerpo y NO hay `Content-Length`. Eso es correcto: un 304 significa literalmente "el recurso **no cambió**, lo que tenés guardado sirve". El cuerpo no viaja porque no hay nada nuevo. Por eso el `304` no consume ancho de banda de contenido: gasta **solo** las cabeceras.

¿Por qué existe el 304? Porque descargar un archivo de 4 MB para descubrir que es idéntico al que ya tenías es absurdo. El 304 es el mecanismo de **validación condicional**: el cliente dice "lo tengo, etiqueta `3f2a1c8`, ¿sigue siendo cierto?" y el servidor responde "sí, te sirve" con 30 bytes de cabeceras en lugar de 4 MB de datos. La clase 9 lo desarrolla con el intercambio completo.

Un 404 Not Found

```
HTTP/1.1 404 Not Found
Date: Tue, 06 Oct 2026 14:22:31 GMT
Server: Apache/2.4.62 (Ubuntu)
Content-Type: text/html; charset=utf-8
Content-Length: 196
Connection: keep-alive

<html><body><h1>404 - Not Found</h1></body></html>
```

El **404** significa "el servidor entendió tu petición pero **no tiene** ese recurso". Importante: el 404 **no dice que la URL esté mal escrita** (eso sería un **400 Bad Request**). El 404 dice "no existe, pero el servidor está vivo y te contesta". Por eso es distinto de un **500** (el servidor se rompió) o de un **502/504** (un proxy no pudo hablar con el backend).

1.5 Los códigos de estado: las cinco clases

El primer dígito del código es la **clase**, y la clase **te dice el tipo de problema** sin leer el número:

Clase	Rango	Significado	Ejemplos que hay que saber
1xx	100–199	**informativo**: "seguí, todavía no terminé"	`100 Continue`, `101 Switching Protocols`
2xx	200–299	**salió bien**	`200 OK`, `201 Created`, `204 No Content`, `206 Partial Content`
3xx	300–399	**redirección**: "andá a otro lado"	`301`, `302`, `304 Not Modified`, `307`, `308`
4xx	400–499	**error del cliente**: "vos te equivocaste"	`400`, `401`, `403`, `404`, `405`, `408`, `429`
5xx	500–599	**error del servidor**: "yo me rompí"	`500`, `502`, `503`, `504`

Los que más aparecen y qué significan de verdad: `200` (acá está, te lo mando acá), `201 Created` (lo creé, te digo dónde, en `Location`), `204 No Content` (salió bien pero **no hay cuerpo**), `206 Partial Content` (te mando **parte**, respuesta a un `Range`), `301` (te mudaste **para siempre**), `302` (mudate **por ahora**), `304` (no te lo mando, conservá el tuyo), `401` (**no estás autenticado**), `403` (**no podés**, ni autenticándote), `404` (no existe), `429 Too Many Requests` (te mandaste demasiadas), `500` (el servidor se rompió), `502 Bad Gateway` (un proxy no pudo hablar con el backend), `503` (caído o sobrecargado) y `504 Gateway Timeout` (el proxy esperó demasiado).

El par 401/403 es la pregunta clásica. Ambos son 4xx y ambos parecen "no autorizado", pero significan cosas distintas:

```
401  → "NO SÉ QUIÉN SOS. Mandame credenciales (y reintentá)."
403  → "SÉ QUIÉN SOS, y SÍ, no tenés permiso. Reintentar no sirve."
```

`401` es recuperable; `403` no lo es. Confundirlos es un error de examen muy común.

1.6 HTTP no persistente vs. persistente — el conteo de RTT

Esta es **la** pregunta del examen. Vamos a hacer la cuenta exacta, con los números a la vista.

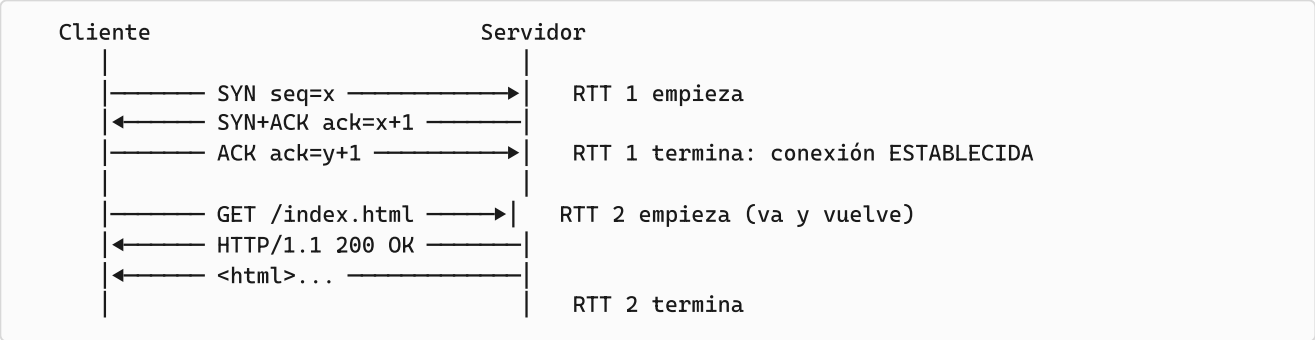
Las dos definiciones, sin ambigüedad

- **HTTP no persistente** (*non-persistent*): por cada objeto, **se abre una conexión TCP nueva, se pide, se recibe, y se cierra**. Una conexión = un objeto.
- **HTTP persistente** (*persistent*): **se abre una conexión TCP y se mantiene** para pedir **varios** objetos seguidos, uno tras otro, en la **misma** conexión.

El término clave es **"cuántas conexiones TCP"**. No es "cuántas peticiones": en persistente hacés 11 peticiones igual, pero sobre **una sola** conexión.

El caso base: 1 objeto, 1 RTT de handshake

Toda conexión TCP nueva cuesta **1 RTT** antes de poder mandar datos. El handshake de 3 vías es:



Entonces, **1 objeto**:

```
No persistente:   1 RTT (TCP)  +  1 RTT (petición/respuesta)  =  2 RTT
Persistente:      1 RTT (TCP)  +  1 RTT (petición/respuesta)  =  2 RTT
```

Con 1 objeto no hay diferencia. Es el caso trivial y hay que decirlo, porque la respuesta "no persistente es peor" sin matices es incorrecta para 1 objeto.

El caso de 11 objetos (1 HTML + 10 imágenes), todo en el MISMO servidor

No persistente: cada objeto paga su propio handshake.

```
objeto 1 (HTML):    1 RTT TCP  +  1 RTT HTTP  =  2 RTT
objeto 2 (img 1):   1 RTT TCP  +  1 RTT HTTP  =  2 RTT
objeto 3 (img 2):   1 RTT TCP  +  1 RTT HTTP  =  2 RTT
...
objeto 11 (img 10): 1 RTT TCP  +  1 RTT HTTP  =  2 RTT

Total = 11 objetos × 2 RTT = 22 RTT
```

Persistente: un solo handshake, y después 11 RTT de HTTP (uno por objeto).

```
conexión TCP (una sola):            1 RTT
objeto 1 (HTML):                    1 RTT
objetos 2..11 (10 imágenes):        10 RTT
                                       ────
Total = 1 + 11 = 12 RTT
```

Comparación: `22 RTT` vs. `12 RTT`. **El persistente es 10 RTT más rápido**, que es exactamente el `N − 1 = 10` handshakes que se ahorró. El HTML y las 10 imágenes van por la misma tubería.

```
No persistente                      Persistente
──────────────                      ────────────
[TCP][HTML]                          [TCP]──[HTML]──[img1]──[img2]── ... ──[img10]
 2 RTT                               1 RTT      1 RTT   1 RTT             1 RTT
×11                                 ──────────────────────────────────────────────
= 22 RTT                            = 12 RTT
```

Y si los objetos vienen de varios servidores

Este es el matiz que casi nadie menciona y el que el apartado b) del examen pide **discutir**. Supongamos que el HTML viene del servidor A y que las 10 imágenes están repartidas en 3 servidores (A, B, C).

Secuencial (el navegador no paraleliza):

```
Persistente, 3 servidores:   3 handshakes (uno por servidor) + 11 RTT de HTTP
                              = 3 + 11 = 14 RTT

No persistente:              11 objetos × 2 RTT = 22 RTT
```

En paralelo (que es lo que hacen los navegadores reales: abren 6 conexiones por host):

```
HTML por A:                1 RTT TCP + 1 RTT HTTP = 2 RTT
las 10 imágenes, en paralelo:
   los 3 handshakes, simultáneos:   1 RTT   (no 3: van a la vez)
   las 10 peticiones, simultáneas:  1 RTT
                                    ──────
Persistente, 3 servidores, PARALELO   = 2 + 2 = 4 RTT
No persistente, 3 servidores, PARALELO = 2 + 2 = 4 RTT   ← ¡empate!
```

Conclusión honesta del apartado b). El persistente **no siempre** es mejor:

```
Secuencial (1 servidor):   no-pers 22 RTT │ pers 12 RTT   → gana el pers
Secuencial (3 servidores): no-pers 22 RTT │ pers 14 RTT   → gana el pers
Paralelo   (1 servidor):   no-pers  4 RTT │ pers  3 RTT   → gana el pers
Paralelo   (3 servidores): no-pers  4 RTT │ pers  4 RTT   → empate
```

Y hay una ventaja extra del no persistente que el enunciado no pide pero que **suma puntos**: el **cliente puede elegir** en qué servidor va cada imagen (por ejemplo el más cercano geográficamente), porque cada conexión es independiente.

1.7 Las estadísticas: por qué HTTP se siente lento

Un resultado clásico y muy reproducible: **la mayor parte del tiempo de carga de una página no es "bajar bytes", sino "esperar al servidor"**. Vamos a hacerlo con números reales, porque es lo que suma puntos.

Datos. RTT desde Montevideo hasta un servidor en Estados Unidos: **100 ms**. Ancho de banda del cliente: **10 Mbps**. La página tiene **11 objetos** de **50 KB** cada uno, servidos en HTTP no persistente desde un solo servidor.

```
TIEMPO DE ESPERA
   = 22 RTT × 100 ms = 2200 ms

TIEMPO REAL DE TRANSFERENCIA
   Total de datos = 11 × 50 KB = 550 KB
   t_trans = 550 KB × 8 bits/KB / 10 Mbps
           = 550 × 1024 × 8 bits / 10·10^6 bits/s
           = 4.505.600 bits / 10.000.000 bits/s
           = 0,4505 s ≈ 450 ms

PROPORCIÓN DE ESPERA
   = 2200 / (2200 + 450) = 2200 / 2650 = 0,83  →  83 %
```

Interpretación: más del 80 % del tiempo se va esperando al servidor, no transfiriendo datos. Y si lo hacés **persistente**, el mismo cálculo da `1200 / (1200 + 450) = 73 %`: mejor, pero el tiempo real de transferencia (450 ms) **no cambió en absoluto**. Cambiar de protocolo no acelera la red: **elimina esperas**.

El tiempo de carga de una página es `espera × número de viajes`. Todo lo que reduce el número de viajes acelera la página: las conexiones persistentes, el **CDN** (que pone un servidor *cerca* para que el RTT sea 5 ms en vez de 100 ms), la resolución DNS cacheada. Y por supuesto HTTP/2 y HTTP/3, que existen exactamente para **matar esperas**. La Clase 9 lo desarrolla.

1.8 Las cookies

Qué son. Una **cookie** (*galletita*) es una pequeña cadena de texto que el **servidor** le manda al cliente y que el cliente **guarda y reenvía** en peticiones posteriores al **mismo** dominio.

```
GET /carrito HTTP/1.1  ──────►  Servidor
◄──────  Set-Cookie: sesion=abc123; Path=/; Max-Age=3600

GET /carrito HTTP/1.1  ──────►  Servidor
Cookie: sesion=abc123
◄──────  200 {"camisa": 2, "pantalon": 1}
```

`Set-Cookie` es la cabecera que **el servidor manda** para **crear o actualizar** una cookie. `Cookie` es la cabecera que **el cliente manda** para **devolverla**. Son dos cabeceras distintas con dos roles distintos, y esa distinción es lo que se pregunta.

El problema que resuelven: el estado en el cliente

HTTP es sin estado, pero casi todas las aplicaciones necesitan recordar algo: el **carrito de compras** (`carrito=camisa:2,pantalon:1`), la **sesión iniciada** (`sesion=abc123`, para no pedir la contraseña cada vez), las **preferencias de idioma** (`lang=es`) o el **"recordarme"** (`token=xyz`).

El estado vive en el cliente. Es el precio de la estatitud: si HTTP no puede guardar estado en el servidor, lo guardás **en el cliente** y lo mandás de vuelta en cada petición. El ejemplo canónico es el carrito:

```
1) El usuario mete una camisa.
   GET /agregar?item=camisa HTTP/1.1
   ←──  Set-Cookie: carrito=camisa; Path=/; Max-Age=86400

2) El usuario mete un pantalón.
   GET /agregar?item=pantalon HTTP/1.1
   Cookie: carrito=camisa                          ← el cliente reenvía
   ←──  Set-Cookie: carrito=camisa,pantalon        ← el servidor actualiza

3) El usuario mira el carrito.
   GET /carrito HTTP/1.1
   Cookie: carrito=camisa,pantalon                  ← el estado viaja acá
   ←──  200 {"camisa": 1, "pantalon": 1}
```

En el paso 3 el servidor **no tiene memoria** de los pasos 1 y 2: todo lo que sabe del usuario está **en la cookie que el cliente le devolvió**.

Los atributos de una cookie

```
Set-Cookie: sesion=abc123; Path=/; Max-Age=3600; Secure; HttpOnly; SameSite=Lax
            │                  │       │          │        │          │
            │                  │       │          │        │          └ no mandar en
            │                  │       │          │        └ peticiones de terceros
            │                  │       │          └ no accesible por JavaScript
            │                  │       └ segundos de vida (si no, "session cookie")
            │                  └ las rutas donde viaja
            └ NOMBRE = VALOR
```

`Secure` restricts el envío a HTTPS. `HttpOnly` la vuelve invisible al JavaScript de la página (defensa contra el robo por XSS).

⚠️ Actualización a 2026 — cookies

Sigue igual: el par `Set-Cookie` (servidor) / `Cookie` (cliente) y que el estado viaje **en el cliente**.

Cambió o se agregó: - **`RFC 6265` — HTTP State Management Mechanism** es el estándar que define el formato. Si el enunciado te pide "qué RFC define las cookies", esa es. - **`SameSite`**: con `SameSite=Lax` (el default moderno) el navegador **no manda la cookie en peticiones originadas por otros sitios**, lo que corta de raíz el **CSRF** (*Cross-Site Request Forgery*). - **Las cookies de terceros están en retirada.** Chrome (*Privacy Sandbox*), Safari (ITP) y Firefox (ETP) las bloquean o particionan por defecto: eso obliga a los sitios a volver a **sesiones del lado del servidor**. - Con **DoH**, las cookies siguen igual respecto del DNS: son de **capa de aplicación**. Lo que cambia es la privacidad de la resolución.

> **Consecuencia para el examen:** si te preguntan "cómo se rompe el modelo sin estado de HTTP", la respuesta es **cookies** (o equivalentemente `Authorization` con un token). Si te preguntan "dónde vive el estado", la respuesta es **en el cliente**, y por eso el servidor puede ser *stateless* y escalar.

1.9 ⚠️ Actualización a 2026 — la especificación de HTTP que hay que corregir

Sigue igual — de memoria, sin dudar: - Modelo **petición/respuesta**, protocolo **sin estado**. - Los **métodos**: GET, HEAD, POST, PUT, DELETE, OPTIONS, TRACE, CONNECT (+ PATCH). GET/HEAD/OPTIONS/TRACE **seguros**; GET/HEAD **idempotentes**. - Las **5 clases** de códigos de estado (1xx a 5xx) y los del día a día: 200, 201, 204, 206, 301, 302, 304, 307, 308, 400, 401, 403, 404, 405, 408, 429, 500, 502, 503, 504. - `Host` **obligatoria** en HTTP/1.1, y **persistente por defecto**. - `Range` requests y `304 Not Modified`.

Cambió — y esto es lo que hay que corregir si citas material viejo: - El curso de 2012 enseñaba `RFC 2616` (1999). **Hoy la especificación vigente es de 2022**: `RFC 9110` (semántica, *Internet Standard*) + `RFC 9112` (sintaxis de HTTP/1.1). - `RFC 7540` (HTTP/2, 2015) está **OBSOLETA**: la reemplazó `RFC 9113`. - **HTTP/3 (`RFC 9114`)** va sobre **QUIC (`RFC 9000`)**, que **reemplaza a TCP por completo**: no hay handshake de 3 vías ni conexión TCP. Se negocia por `Alt-Svc` (`RFC 7838`), no por `Upgrade:`. - **Cifra real (Cloudflare Radar, Year in Review 2025):** HTTP/2 **50 %**, HTTP/1.x **29 %**, HTTP/3 **21 %**. **HTTP/3 no se adueñó de la web.** La Clase 9 lo explica. - **Wireshark decodifica HTTP/3, QUIC, TLS 1.3 y DoH** built-in.

> **Consecuencia para el examen:** HTTP/1.1 **sigue siendo totalmente válido y necesario**. Si te piden "qué versión de HTTP es la actual": "HTTP/2 es el más usado; HTTP/3 va en crecimiento y no se adueñó; HTTP/1.1 sigue vivo y es el baseline de comparación".

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Contar RTTs para 1 y 11 objetos (★ el del examen)

Enunciado. Un cliente pide una página con **1 objeto** (solo el HTML) y después otra con **11 objetos** (1 HTML + 10 imágenes). El RTT cliente-servidor es de **50 ms**. Todo se sirve desde **el mismo servidor** y en el modo más simple posible (el navegador **no** abre conexiones en paralelo).

a) ¿Cuántos RTT cuesta cada caso en HTTP no persistente?

Razonamiento. En no persistente, **cada objeto abre su propia conexión TCP**. El costo por objeto es `(1 RTT de handshake) + (1 RTT de petición/respuesta)` = 2 RTT.

Ecuación:

```
No persistente = N objetos × 2 RTT

1 objeto:   1 × 2 = 2 RTT   → 2 × 50 ms = 100 ms
11 objetos: 11 × 2 = 22 RTT → 22 × 50 ms = 1100 ms
```

Interpretación: con 1 objeto, 100 ms. Con 11, **11 veces más**: 1100 ms. El HTML tardó lo mismo, pero las 10 imágenes cada una arrastró su propio setup de TCP.

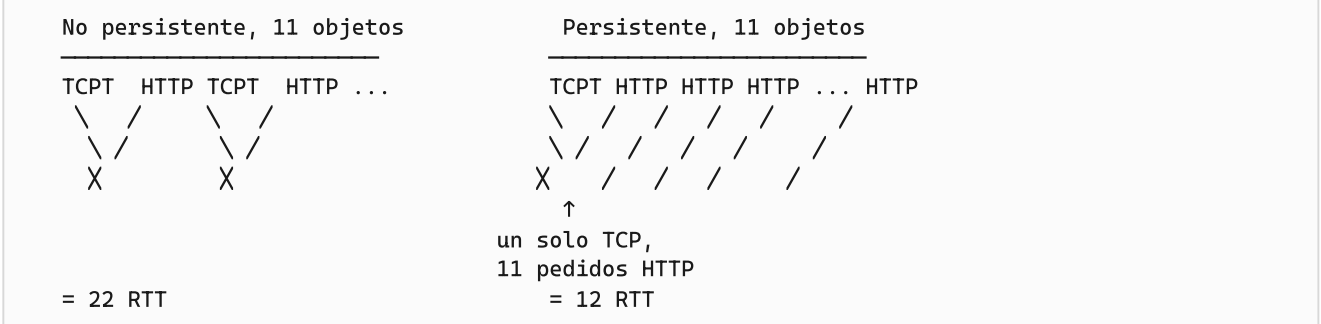
b) ¿Y en HTTP persistente? Un solo handshake para **todos** los objetos, y después 1 RTT de HTTP por objeto.

```
Persistente = 1 RTT (TCP) + N × 1 RTT (HTTP)

1 objeto:   1 + 1×1 = 2 RTT  → 100 ms
11 objetos: 1 + 11×1 = 12 RTT → 600 ms
```

Interpretación: el persistente con 11 objetos tarda **600 ms contra 1100 ms**. Ahorró `22 − 12 = 10 RTT = 500 ms`, que es exactamente `N − 1` handshakes eliminados.

c) Comparación final, los cuatro casos (el que se pregunta en b) del examen).

Caso	Modo	Conexiones TCP	RTT	Tiempo (RTT = 50 ms)
1 objeto	no persistente	1	2	100 ms
1 objeto	persistente	1	2	100 ms
11 objetos	**no persistente**	**11**	**22**	1100 ms
11 objetos	**persistente**	**1**	**12**	**600 ms**



d) ¿Y si los objetos vienen de varios servidores? (esto es lo que el apartado b) pide **discutir**)

Supongamos que el HTML viene del servidor A y las 10 imágenes se reparten en **3** servidores (A, B y C).

Razonamiento, caso secuencial. En persistente hay que abrir **una conexión por servidor** (3 handshakes), y encima los 11 objetos viajan por ellas:

```
Persistente, 3 servidores, secuencial
   = (3 servidores × 1 RTT de handshake) + (11 objetos × 1 RTT de HTTP)
   = 3 + 11 = 14 RTT
```

Interpretación: **14 RTT contra 22** del no persistente. Sigue ganando el persistente, pero **menos** (ganaba 10 RTT, ahora gana 8), porque con más servidores hay más handshakes que pagar.

Razonamiento, caso paralelo. Si el navegador abre las conexiones **en paralelo** (que es lo que hacen los reales), los 3 handshakes pasan **al mismo tiempo**, así que cuestan 1 RTT en total, y las 11 peticiones también en paralelo, 1 RTT:

```
Persistente, 3 servidores, PARALELO
   = 1 RTT (los 3 handshakes, en paralelo)
     + 1 RTT (las 11 peticiones/respuestas, en paralelo)
   = 2 RTT

No persistente, 3 servidores, PARALELO
   = 2 RTT (HTML) + 1 RTT (handshakes) + 1 RTT (peticiones) = 4 RTT
```

Interpretación: acá **el persistente y el no persistente empatan en 4 RTT**. Con paralelismo, **la ventaja del persistente se desvanece** cuando hay varios servidores. Por eso el apartado b) dice "puede resultar más performante" y no "siempre es más performante". La respuesta completa al examen es: **el persistente gana en secuencial; en paralelo, gana sólo si hay un solo servidor.**

⚠️ Errores típicos

- **Contar "11 RTT" para el no persistente.** No: son 2 RTT por objeto (uno de TCP + uno de HTTP), así que 22. El error viene de olvidar que **cada objeto abre su conexión**.
- **Decir que el persistente siempre gana.** El enunciado pregunta "¿mejora la performance? **Discuta**". Con conexiones en paralelo y varios servidores, **empata**. Hay que decir las dos caras.
- **Sumar 1 RTT de TCP por servidor en paralelo.** En paralelo, N handshakes simultáneos cuestan **1 RTT**, no N.
- **Olvidar que el `Content-Length` y la línea en blanco final son parte del mensaje.** En un examen de "escribí el mensaje", eso se cuenta.

2.2 Ejercicio 2 — Interpretar un 304

Enunciado. El cliente tiene en su caché `index.html` con `ETag: "3f2a1c8"`, y hace esta petición condicional:

```
GET /index.html HTTP/1.1
Host: www.fing.edu.uy
If-None-Match: "3f2a1c8"
```

Y el servidor responde:

```
HTTP/1.1 304 Not Modified
Date: Tue, 06 Oct 2026 14:22:33 GMT
ETag: "3f2a1c8"
Cache-Control: max-age=600
```

a) ¿Qué le está diciendo el servidor al cliente?

Razonamiento. El cliente mandó `If-None-Match: "3f2a1c8"`, o sea "tengo la versión con esta etiqueta, decime si sigue siendo la actual". El servidor compara su ETag actual con el que le mandaron. Si coinciden, **no manda el cuerpo**.

Respuesta: "lo que tenés guardado sigue siendo correcto, no te mando nada nuevo". El cliente **sigue usando su copia**. Sin esa cabecera condicional el cliente siempre **re-descargaría** todo, porque no tiene forma de decir "esto no te lo pido": el 304 es lo que convierte la caché en algo útil.

c) ¿Cuánto se ahorró?

Razonamiento. El `index.html` tiene 4096 bytes de cuerpo (del 200 del §1.4). El 304 no transmite cuerpo, solo cabeceras.

Ecuación:

```
Con 200 (sin validación):  4096 de cuerpo + ~250 de cabeceras ≈ 4346 B
Con 304:                      0 de cuerpo + ~150 de cabeceras ≈  150 B

Ahorro = 4346 − 150 = 4196 B
Porcentaje ahorrado = 4196 / 4346 ≈ 96,5 %
```

Interpretación: se transfirió el **3,5 %** de lo que se habría transfirido. En una página con 11 objetos, si todos dieran 304, la carga pasa de ~2,2 s a unos pocos cientos de milisegundos. **El 304 no es una optimización de bytes, es un cambio de orden de magnitud.**

2.3 Ejercicio 3 — ¿Por qué HTTP es sin estado y cómo lo resuelven las cookies?

Enunciado. Explicá por qué HTTP fue diseñado sin estado, qué problema concreto genera, y cómo lo resuelven las cookies. Respuesta al estilo del examen.

a) ¿Por qué sin estado? La razón profunda es el **balanceo de carga** (*load balancing*). Si el servidor guardara el estado de cada usuario, **todas las peticiones de ese usuario tendrían que ir al mismo servidor**.

```
CON ESTADO en el servidor:
   user 17 ──► servidor 1 (siempre)    ← si el servidor 1 cae, cae el user

SIN ESTADO:
   user 17 ──► servidor 1, 2 o 3, indistinto  ← sobrevive a la caída de cualquiera
```

Interpretación: el diseño sin estado **sacrificó eficiencia por escalabilidad** y eligió bien: la web creció a miles de servidores detrás de balanceadores, algo que un diseño con estado en servidor hace muy difícil.

b) ¿Qué problema concreto genera? Que el servidor no puede saber quién sos entre peticiones. Si hacés `GET /carrito` sin cookie, no hay forma de saber qué llevás.

c) ¿Cómo lo resuelven las cookies? Poniendo **el estado del lado del cliente** y haciéndolo viajar en cada petición:

```
El estado del carrito NO está en el servidor.
Está en la cookie del cliente:  carrito=camisa,pantalon

El cliente la manda en cada GET /carrito
→ el servidor la lee, responde bien, y la "olvida"
→ el servidor puede ser cualquiera de los 3
```

Alternativas a la cookie (también estado en el cliente): la **URL** (`GET /carrito?user=17` , simple pero queda en el historial y en el log), la cabecera **`Authorization`** con un token Bearer (lo que usa OAuth) y el **`localStorage`** del navegador (estado en el cliente pero solo para el JS).

d) El costo de esta decisión. (esto es lo que suma puntos)

```
Ventaja:  el servidor es stateless, escala, y es tolerante a la caída de
          cualquier nodo.
Costo 1:  el estado viaja en CADA petición → más bytes por request. Si fuera
          grande, la cookie se vuelve pesada y hay que pasar el estado al
          servidor (base de datos), que es lo que hacen las apps grandes.
Costo 2:  la cookie la puede leer o alterar el cliente → hay que firmarla o
          cifrarla, o usar tokens de sesión del lado del servidor.
```

Interpretación final para el examen: **"HTTP es sin estado, y el estado viaja en el cliente (cookies, URL o tokens), por eso el servidor escala, y el precio es que el estado viaja en cada petición y es manipulable por el cliente".**

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-07, Pregunta 1 (8 puntos) — página HTML con 10 imágenes

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1007.pdf **Este es el problema central de toda la unidad de aplicación.**

Enunciado (texto literal del examen)

> **Pregunta 1 (8 puntos)**
>
> Suponga que un usuario intenta cargar en su browser una página HTML. Esta página contiene texto y diez imágenes.
>
> **a)** Describa la secuencia de pasos que ocurren a nivel de capa de aplicación en el cliente y el servidor a los efectos de desplegar la página completa, suponiendo que se utiliza HTTP no persistente.
>
> **b)** ¿Cuál es la diferencia con HTTP persistente? ¿Mejora la performance? Discuta.

Solución oficial (del PDF, textual)

> a)
>
> | Paso | Responsable | Acción |
> |---|---|---|
> | 1 | Cliente | Establece la conexión con el servidor |
> | 2 | Cliente | Envía Request pidiendo la página |
> | 3 | Servidor | Envía Response con el contenido de la página |
> | 4 | Servidor | Cierra la conexión |
> | 5* | Cliente | Establece la conexión con el servidor |
> | 6* | Cliente | Envía Request pidiendo la la imagen 1 |
> | 7* | Servidor | Envía Response con el contenido de la imagen 1 |
> | 8* | Servidor | Cierra la conexión |
>
> * Se repiten los pasos 5 al 8 para cada una de las imágenes.
>
> b) En el caso de HTTP persistente, se establece la conexión una sola vez, se obtiene el contenido de la página, y a través de la misma conexión se piden todas las imágenes.
>
> El uso de HTTP persistente puede resultar más performante en los casos en que se utilice una sola conexión por vez (por ejemplo si el servidor nos limita por IP), ya que no existe el overhead de crear nuevas conexiones para cada imagen.

> Si el navegador abre conexiones en paralelo para obtener varias imágenes a la vez, probablemente termine de obtener todos los datos más rápidamente, aprovechando mejor el ancho de banda.

a) La secuencia de pasos — resolución

La tabla oficial, reproducida tal cual, es la respuesta al apartado a) y está perfectamente bien para los 8 puntos. Pero le agrego dos precisiones que la oficial omite y que suman:

Precisión 1 — el paso 1 "establece la conexión" es el handshake de TCP. El enunciado dice "a nivel de capa de aplicación" y la oficial abstrae el TCP en un solo paso, que es lo que el enunciado pide. **PERO** para contar RTTs hay que saber que ese paso 1 vale **1 RTT**:

Paso	Responsable	Acción	RTT
1	Cliente	Establece la conexión con el servidor (SYN, SYN+ACK, ACK)	**1**
2	Cliente	Envía Request pidiendo la página (`GET /index.html HTTP/1.1`)	1 (compartido con el 3)
3	Servidor	Envía Response con el contenido (`HTTP/1.1 200 OK` + HTML)	cierra el RTT
4	Servidor	Cierra la conexión	0 (FIN)
5–8		Se repiten por cada una de las 10 imágenes	2 c/u

Precisión 2 — la lectura del cuerpo HTML. El paso 3 dice "envía response con el contenido de la página". Lo que la oficial da por supuesto es que el cliente, al recibir el HTML, **lo parsea**, ve las 10 imágenes referenciadas por `<img src="imagen1.png">`, y dispara 10 peticiones, cada una repitiendo 1–4.

Respuesta redactada, lista para copiar en el examen:

> **a)** En HTTP **no persistente**, por cada objeto (el HTML y cada una de las 10 imágenes) el cliente ejecuta **cuatro pasos**: (1) establece una conexión TCP con el servidor, (2) envía un request pidiendo el objeto, (3) el servidor envía un response con el contenido del objeto, (4) el servidor cierra la conexión. El primer objeto es la página HTML; al recibirla y parsearla, el cliente descubre las 10 imágenes referenciadas y repite los pasos 1–4 **una vez por cada imagen**. En total son **11 objetos × 4 pasos = 44 pasos**, con **11 conexiones TCP distintas** y **22 RTT** (2 RTT por objeto: 1 de handshake TCP + 1 de request/response).
>
> **b)** En HTTP **persistente** se establece **una sola** conexión TCP y por ella se piden los 11 objetos uno tras otro: **1 RTT** de handshake + **11 RTT** de request/response = **12 RTT**, contra los 22 del no persistente. La mejora es real cuando el navegador trabaja **secuencialmente**, porque se eliminan los `N − 1 = 10` handshakes. **Pero no siempre mejora.** Si el navegador abre conexiones **en paralelo** (que es lo que hacen los reales) y los objetos vienen de **varios servidores**, los handshakes pasan todos al mismo tiempo: el no persistente baja a **4 RTT** y el persistente queda en **4 RTT**, o sea, la ventaja del persistente **desaparece**. El persistente es claramente superior con un solo servidor (3 RTT contra 4, con paralelo, y 12 contra 22 sin paralelo), pero con varios servidores y paralelismo hay empate.
>
> **Nota sobre mi respuesta contra la oficial:** la oficial dice "puede resultar más performante", que es la respuesta honesta y matizada, y agrega un argumento que repito: **"si el servidor nos limita por IP"**, o sea, que el navegador no puede paralelizar porque el servidor impone un límite por IP. Ese es el mejor argumento del apartado b).

La cuenta de RTT completa, a la vista

Este es el corazón del asunto y lo que la oficial no cuantifica:

```
========== NO PERSISTENTE, 1 SERVIDOR, SECUENCIAL ==========

objeto    |  TCP handshake  |  request/resp  |  RTT del objeto
----------|-----------------|----------------|----------------
  HTML    |       1         |        1       |      2
  img 1   |       1         |        1       |      2
   ...    |       1         |        1       |      2
  img 10  |       1         |        1       |      2
----------|-----------------|----------------|----------------
TOTAL     |      11         |       11       |    22 RTT


========== PERSISTENTE, 1 SERVIDOR, SECUENCIAL ==========

objeto    |  TCP handshake  |  request/resp  |  RTT del objeto
----------|-----------------|----------------|----------------
  HTML    |       1         |        1       |      2
  img 1   |       0         |        1       |      1
   ...    |       0         |        1       |      1
  img 10  |       0         |        1       |      1
----------|-----------------|----------------|----------------
TOTAL     |       1         |       11       |    12 RTT


========== PERSISTENTE, PARALELO (6 conexiones), 1 SERVIDOR ==========

objeto    |  TCP handshake  |  request/resp  |
----------|-----------------|----------------|
  HTML    |       1         |        1       |
10 imgs   |       1         |        1       |   ← todos a la vez, en paralelo
----------|-----------------|----------------|
TOTAL                                          4 RTT

... y si además el navegador REUTILIZA la conexión del HTML para las
10 imágenes (que es lo que hace con las 6 conexiones por host):

TOTAL = 1 (handshake) + 1 (HTML) + 1 (las 10 imágenes, en paralelo) = 3 RTT
```

Interpretación, que es lo que hay que escribir: el mecanismo por el que el persistente mejora es **eliminar handshakes**, no "mandar más rápido". Cada handshake eliminado vale 1 RTT: con 11 objetos y 1 servidor se eliminan 10. En el límite de la conexión paralela el handshake ya **no es el cuello de botella** y lo que manda es el **número de RTT de request/response**, y para eso sirven **HTTP/2 y HTTP/3** (Clase 9).

⚠️ Errores típicos en este problema

- **Olvidar que el paso 1 es TCP.** La oficial lo abstrae, y está bien para el apartado a), pero si te piden RTTs **no** podés contarlo como 1 RTT de HTTP.
- **"El cliente pide el HTML y después pide las 10 imágenes"** sin notar que en no persistente **cada** petición abre conexión. Ese es el error conceptual central.
- **Contar 11 RTT en no persistente.** Son 22.
- **Decir "el persistente siempre es mejor".** El enunciado dice **"discuta"**. Hay que dar el caso en que **no** mejora.

3.2 Examen 2009-07, Problema 2 — el `getMailServer()` como puente

> **Fuente:** Examen `eirc0907` / `sirc0907` — 29 de julio de 2009, Problema 2.

Este problema **une HTTP con DNS** y por eso lo desarrollamos completo en la **Clase 14**, donde se implementa `getMailServer()` con sockets UDP. Lo que aporta acá es el **lado HTTP**.

> **Apartado b)** Implementar `bool isSpam(string ipAddress, string subject)` que devuelve `true` si naivespamming considera que el correo es spam.

La solución oficial escribe:

```
Socket s = new Socket("www.naivespamming.org", 80);
if (!s.connect()) return false;
s.out.write(
  "GET isspam.cgi?sender=" + ipAddress +
  "&subject=" + queryStringStuff(subject) +
  " HTTP/1.1\r\n\r\n");
string response = s.in.readLine();
s.close();
string[] result = response.split();
return result[1] == "200";
```

Análisis, con lo que aprendimos hoy:

1. **El `GET` está mal formado como HTTP/1.1.** Un mensaje **válido** sería:

```
GET /isspam.cgi?sender=203.0.113.7&subject=Hola HTTP/1.1
Host: www.naivespamming.org
User-Agent: FING-SpamFilter/1.0
Accept: text/plain
Connection: close
```

Falta el `Host`, que es obligatoria en 1.1: sin ella, un servidor con *vhosts* no sabe qué sitio querés. Y usa un **path relativo** (`isspam.cgi`) en vez de absoluto (`/isspam.cgi`). El `\r\n\r\n` final sí es correcto: es la línea en blanco que cierra las cabeceras, tal como vimos en §1.3. 2. **El parseo es frágil.** `response.split()` separa por espacios y toma `result[1]`, que es el **código de estado**. Funciona, pero se rompe si hay redirección. **Lo robusto** es parsear la **primera línea** de la respuesta. 3. **El `return result[1] == "200"` tiene la semántica invertida** respecto del enunciado, que dice explícitamente que **404 significa "es spam"** y **200 significa "es legítimo"**. La oficial devuelve `true` con 200, o sea, marca como spam un correo que el servidor dijo que es legítimo. **Es un error real de la solución publicada.** Lo correcto:

```
código == 404  →  es spam     →  return true
código == 200  →  no es spam  →  return false
otro / error   →  no se sabe  →  return false  (fail-open: mejor dejar
                    pasar un correo que trabar todo el servidor)
```

> **Nota explícita:** el enunciado y la oficial se contradicen. Si te pasa en un examen, **respetá el enunciado** y **señalá la contradicción** por escrito. Eso suma puntos: demuestra que leíste los dos documentos.

Lo que aporta esta pregunta al tema de hoy: los **códigos de estado** importan de verdad. Un 404 no es un error técnico: es **información de negocio** para el servidor de correo. **HTTP no solo transporta contenido, transporta semántica**, y un `4xx` / `2xx` puede ser exactamente el dato que la aplicación necesita.

PARTE 4 — CIERRE

📌 Resumen

1. **Cliente/servidor** = roles **asimétricos**: el servidor **espera**, el cliente **inicia**. La asimetría elimina el problema de "quién llama".
2. **Peer-to-peer** = todos son cliente y servidor a la vez, sin punto central. Escala con los pares, pero es mucho más complejo.
3. **HTTP no es** un protocolo de transporte, **ni** la Web, **ni** tiene estado, **ni** es seguro. Es solo un formato de mensajes de aplicación sobre TCP.
4. El mensaje HTTP tiene **3 partes**: línea de petición (o de estado) + cabeceras + **línea en blanco obligatoria** + cuerpo.
5. `Host` es **obligatoria** en HTTP/1.1; `Content-Length` y `Content-Type` son las que hacen que la respuesta sea interpretable.
6. **No persistente**: 2 RTT por objeto. **Persistente**: 1 RTT de handshake + 1 RTT por objeto. Con 11 objetos: **22 RTT vs. 12 RTT** en un solo servidor.
7. Con **paralelo** y varios servidores, el persistente **empata** (4 RTT): el handshake deja de ser el cuello de botella.
8. **304 Not Modified**: no cambia el ETag, no viaja cuerpo, ahorra ~96 % de la transferencia. Es **validación condicional** con `If-None-Match` / `If-Modified-Since`.
9. **401** = "no sé quién sos, mandá credenciales". **403** = "sé quién sos y no podés". No son sinónimos.
10. Más del **80 %** del tiempo de carga de una página es **espera**, no transferencia. Cambiar de protocolo no acelera la red: **elimina esperas**.
11. **Cookies**: el estado vive **en el cliente** (`Set-Cookie` del servidor, `Cookie` del cliente). Es el precio de que el servidor sea sin estado y pueda escalar. `RFC 6265` las define; `SameSite` / `HttpOnly` las aseguran.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué el diseño sin estado de HTTP es, en el fondo, una decisión de escalabilidad? Explicá el argumento con un balanceador de carga.

▶ Respuesta

2. ¿Cuántos RTT cuesta una página con 11 objetos en HTTP no persistente? Y en persistente? Mostrá las cuentas.

▶ Respuesta

3. ¿En qué caso el HTTP persistente **no** mejora la performance? Justificá.

▶ Respuesta

4. ¿Qué diferencia hay entre `Set-Cookie` y `Cookie`? ¿En qué dirección viaja cada una?

▶ Respuesta

5. ¿Por qué un **304** no tiene `Content-Length` ni cuerpo? ¿Qué se ahorró?

▶ Respuesta

6. ¿Cuál es la diferencia entre `401` y `403`? ¿Por qué importa?

▶ Respuesta

7. Un cliente descarga una página de 11 objetos con RTT de 100 ms, ancho de banda de 10 Mbps y objetos de 50 KB cada uno, en HTTP no persistente. ¿Qué porcentaje del tiempo total se va en **espera**?

▶ Respuesta

8. En la solución oficial del `2009-07 Problema 2`, la función `isSpam()` devuelve `true` cuando el código HTTP es 200. ¿Es correcto? ¿Qué haría bien?

▶ Respuesta

🔗 Referencias

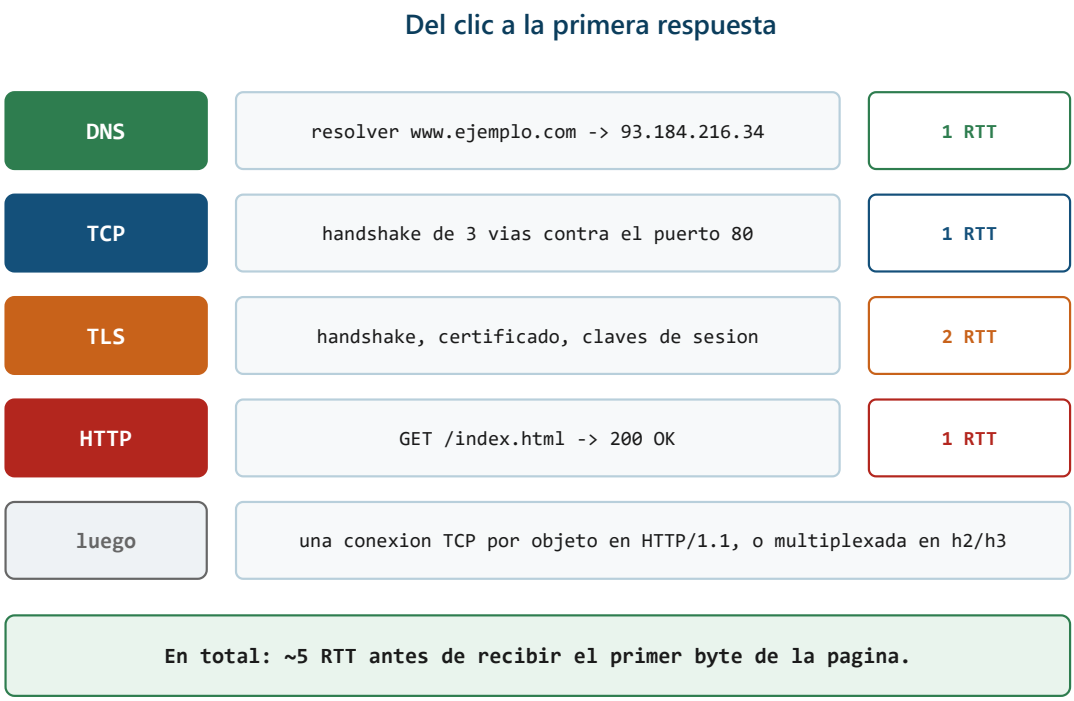
- **Examen** `eirc1007` / `sirc1007` — Pregunta 1 completa (8 pts, HTTP no persistente + las 10 imágenes)
- **Examen** `eirc0907` / `sirc0907` — Problema 2 (30 pts): cliente anti-spam vía HTTP; apartados b) y c)
- **Examen** `eirc1002` / `sirc1002` — Pregunta 1a (8 pts): qué es el DNS y su función (desarrollada en la Clase 10)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.1 (modelo cliente/servidor) y §7.3 (HTTP: los dos tipos de conexión, los mensajes, el cálculo de RTT)
- **RFC 9110** — HTTP Semantics (2022, *Internet Standard*; sustituye a RFC 2616) · **RFC 9112** — HTTP/1.1 (2022, sintaxis)
- **RFC 9113** — HTTP/2 (2022; obsoleta RFC 7540) · **RFC 9114** — HTTP/3 (2022, sobre QUIC)
- **RFC 6265** — HTTP State Management Mechanism (cookies) · **RFC 7838** — Alt-Svc (2016): cómo se negocia HTTP/3
- **Clase siguiente:** 9 — HTTP a fondo: mensajes, cookies, caché y versiones

Clase 9

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 Los métodos, uno por uno
- 1.2 Seguros e idempotentes: por qué la distinción importa
- 1.3 POST vs. GET: dónde van los datos
- 1.4 Las cabeceras que hay que saber
- 1.5 La caché HTTP
- 1.6 `ETag` + `Last-Modified`: la validación condicional
- 1.7 Proxies, gateways y CDNs
- 1.8 HTTPS: qué agrega (y qué no es)
- 1.9 De HTTP/1.1 a HTTP/2 y HTTP/3
- 1.10 La tabla que hay que saber de memoria
- 2.1 Ejercicio 1 — Intercambio completo con validación condicional hasta el 304
- 2.2 Ejercicio 2 — ¿Qué hace exactamente `Cache-Control: max-age=0` con un ETag presente?
- 2.3 Ejercicio 3 — ¿Qué gana y qué pierde HTTP/3 frente a HTTP/1.1?
- 3.1 ⭐ Examen 2010-07, Pregunta 1 (8 puntos) — página HTML con diez imágenes
- 3.2 Examen 2009-07, Problema 2 — HTTP como protocolo de consulta
- 3.3 Examen 2008-12, Pregunta 4 (7 puntos) — el mismo patrón de diseño
- 📌 Resumen en 10 líneas
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: B (Aplicación) · **Semana:** 3 · **Duración estimada:** 110 min **Correlativa:** Clase 8 (modelo cliente/servidor y HTTP/1.1) · **Sigue con:** Clase 10 (DNS: la base de datos distribuida) **Material de referencia:** Tanenbaum cap. 7 (§7.3) · RFC 9110, RFC 9112, RFC 9113, RFC 9114, RFC 6265, RFC 7541, RFC 9204, RFC 9218, RFC 7838, RFC 9000



🎯 Al terminar esta clase vas a poder

1. Enumerar los **métodos** de HTTP y explicar qué significa que uno sea **seguro** o **idempotente**, y por qué esa distinción decide si un cliente puede reintentar solo.

2. Escribir de memoria un **intercambio HTTP/1.1 completo** de validación condicional que termina en **304**, con `ETag`, `If-None-Match` y todas las cabeceras necesarias.
3. Explicar qué hace **cada** cabecera de la lista que hay que saber, sin consultarla.
4. Diferenciar **`no-store`**, **`no-cache`**, **`max-age`**, **`private`** y **`public`**, y explicar qué pasa exactamente con `max-age=0` cuando hay un `ETag` presente.
5. Distinguir un **proxy**, un **gateway** y un **CDN**, y explicar por qué un CDN mejora la latencia con el concepto de **distancia al usuario**.
6. Explicar qué **agrega HTTPS**, y por qué eso no es lo mismo que cambiar de versión de HTTP.
7. Comparar **HTTP/1.1, HTTP/2 y HTTP/3** diciendo qué problema exacto resuelve cada una, qué parte del *head-of-line blocking* queda y qué parte desaparece, con las cifras reales.
8. Enumerar **qué sigue igual** en las tres versiones, porque es lo que el examen da por sabido.

> **Prerequisito del examen:** `2010-07 P1` (8 puntos — la carga de una página, repreguntado acá con el ángulo de versiones), `2009-07 Problema 2` (30 puntos — el cliente anti-spam que habla HTTP), y todo lo que la cátedra pregunta sobre «qué versión de HTTP es la actual».

PARTE 1 — TEORÍA

1.1 Los métodos, uno por uno

Un método HTTP no es un verbo decorativo: **es una declaración de intención sobre el recurso**. El servidor la usa para decidir si puede cachear, si tiene que autenticar y qué hacer con el cuerpo.

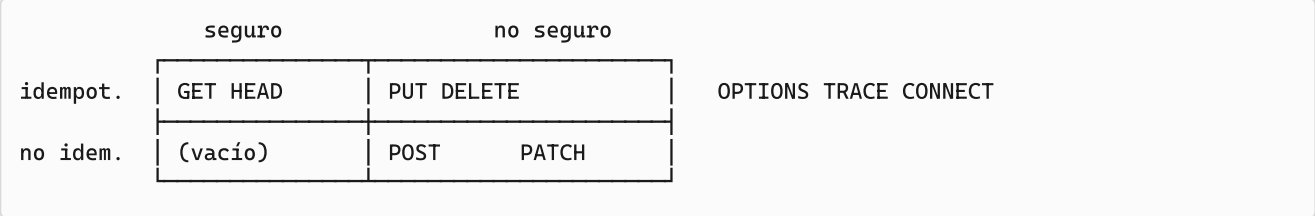
Método	Qué hace	Cuerpo	Típica	Notas
GET	**Obtiene** un recurso	prohibido (se tolera)	`200`	Seguro e idempotente. El único cacheable por definición.
HEAD	Igual que GET **sin cuerpo**	no	`200` sin cuerpo	Seguro e idempotente. Permite conocer el `Content-Length` sin bajarlo.
POST	**Crea** o dispara una acción	**obligatorio**	`201` o `200`	**Ni seguro ni idempotente.** El único donde el cuerpo define la operación.
PUT	**Reemplaza** el recurso	obligatorio	`200` / `204` / `201`	Idempotente, **no seguro**. La URL identifica, el cuerpo define.
DELETE	**Borra** el recurso	opcional	`204`	Idempotente, no seguro. Borrar dos veces es lo mismo que borrar una.
OPTIONS	Pregunta **qué admite**	no	`200` + `Allow:`	Seguro e idempotente. Descubre capacidades (CORS).
TRACE	Devuelve el mensaje **tal como llegó**	no	`200` con el eco	Seguro e idempotente. Diagnóstico de proxies. Casi nadie lo implementa.
CONNECT	Abre un **túnel TCP**	no	`200`	Seguro e idempotente. Base de los proxies y de HTTPS.
PATCH	**Modificación parcial**	obligatorio	`200` / `204` / `202`	Ni seguro ni idempotente. **No existía en HTTP/1.0**.

Tres detalles de memoria:

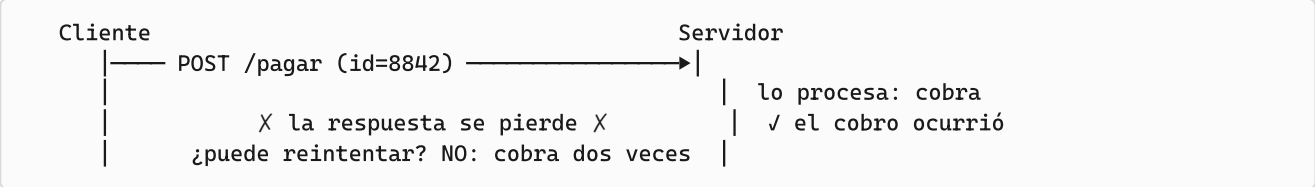
1. `GET` **con cuerpo.** La especificación permite que el servidor lo rechace (`400` o `413`) y muchos lo hacen. La razón histórica: los caches y proxies de la época solo cacheaban peticiones **sin** cuerpo, porque sin cuerpo la identidad de la petición se reduce a `(método, URL)`. Con cuerpo, dos peticiones idénticas pueden tener semántica distinta.
2. `PUT` **reemplaza,** `POST` **crea.** En `PUT` la URL **ya identifica** el recurso y el cuerpo dice qué tiene que contener. En `POST` la URL identifica **una colección** y el servidor **decide** la URL del recurso nuevo: por eso devuelve `201 Created` con `Location`.
3. `CONNECT` **no es una petición HTTP normal.** Su respuesta es una conexión TCP **cruda** que queda abierta: lo que venga después no es HTTP (típicamente un `ClientHello` de TLS).

1.2 Seguros e idempotentes: por qué la distinción importa

Propiedad	Definición	Métodos que la tienen
Seguro (*safe*)	**No modifica el estado del servidor.**	`GET`, `HEAD`, `OPTIONS`, `TRACE`, `CONNECT`
Idempotente	Repetir la misma petición *n* veces produce **el mismo efecto** que repetirla una. No significa la misma respuesta byte a byte: significa el **mismo efecto sobre el estado**.	`GET`, `HEAD`, `PUT`, `DELETE`, `OPTIONS`, `TRACE`, `CONNECT`



El problema que resuelve es el **reintento por timeout**. TCP garantiza la entrega, pero HTTP no sabe si la **respuesta** llegó, y hay caminos en los que tampoco puede saber si la **petición** fue procesada:



La regla de los clientes HTTP reales: si el método es **idempotente**, se puede reintentar automáticamente; si es `POST`, no se reintenta solo: o se le pregunta al usuario, o se manda una **idempotency key** que el servidor usa para deduplicar.

El razonamiento para el examen: las dos propiedades son del **efecto**, no de la intención. `DELETE` **no es seguro** (borra algo) pero **sí es idempotente** (borrar un recurso ya borrado no cambia nada más). `POST` es las dos cosas negativas **a la vez**, y por eso es el único con cuerpo obligatorio.

Matiz que suma puntos: un `GET` **sí** puede modificar el servidor (contador de visitas, *log*, token rotado). Los métodos seguros **deben** ser de solo lectura, pero el resumen de uso puede registrar. Lo que la propiedad garantiza es que **no es la intención del protocolo** cambiar el recurso.

1.3 POST vs. GET: dónde van los datos

```
GET /buscar?q=gorila&lang=es HTTP/1.1          ←— los datos van EN LA URL
Host: www.fing.edu.uy

POST /buscar HTTP/1.1                          ←— los datos van EN EL CUERPO
Host: www.fing.edu.uy
Content-Type: application/x-www-form-urlencoded
Content-Length: 21

q=gorila&lang=es
```

	`GET`	`POST`
Dónde van los datos	en la **URL**, tras el `?`, separados por `&`	en el **cuerpo**, tras la línea en blanco
Qué los delimita	el `?`, el `&` y el `=`	el `Content-Type` (cómo se codifican) y el `Content-Length`
Tamaño	el de la URL: ~2000 a 8000 caracteres	el del cuerpo; el límite lo pone `413 Payload Too Large`
Visibilidad	queda en el historial, en los logs del servidor, en el `Referer`	**no** aparece en la URL ni en el historial
Cacheable	**sí**, por definición	**no**, salvo `Cache-Control` explícito
Idempotente	sí	**no**
Se repite con un link	sí	no: un formulario `POST` no se reenvía al pulsar «atrás»

El argumento de seguridad: mandar una contraseña en la URL es un error grave, porque queda escrita en el **historial del navegador**, en los **logs de acceso** del servidor, en el `Referer` que se le manda a los terceros y en todos los **proxies** del camino. Todo eso en texto plano en disco. Por eso los formularios de login usan `POST`.

La razón técnica: la URL es la **identidad** del recurso en HTTP. `GET /articulos/2026/10` y `GET /articulos/2026/11` son dos recursos distintos y por eso dos entradas distintas de la caché. El `POST` no identifica un recurso: **invoca un proceso**. Esa es la razón de fondo de que no sea ni seguro ni idempotente.

1.4 Las cabeceras que hay que saber

Cabeceras de la **petición**:

Cabecera	Qué hace	Ejemplo	Por qué importa
`Host`	**Obligatoria en HTTP/1.1.** Dice a qué servidor del destino se pide.	`Host: www.fing.edu.uy`	Hace posible el *virtual hosting*: un solo `132.42.120.5` aloja `fing.edu.uy`, `ing.edu.uy` y `redescomp.uy`. **Es la razón de que HTTP/1.1 no pueda esconderse detrás de un NAT.**
`User-Agent`	Identifica el cliente.	`Mozilla/5.0 (X11; Linux x86_64)`	Muchos servidores devuelven contenido distinto según esto, y los bots se identifican y se los bloquea.
`Accept`	Qué **tipos de contenido** acepta.	`Accept: text/html,application/xhtml+xml`	Permite la **negociación de contenido**.
`Accept-Encoding`	Qué **compresiones** acepta.	`Accept-Encoding: gzip, deflate, br`	Si acepta, el servidor responde `Content-Encoding: gzip`.
`Accept-Language`	Qué **idiomas** prefiere.	`Accept-Language: es-UY,es;q=0.9`	El servidor devuelve la versión en español.
`Cookie`	Devuelve las cookies del cliente.	`Cookie: sesion=abc123`	El estado viaja **de vuelta**.
`Authorization`	Credenciales.	`Authorization: Bearer eyJhbGci...`	**Nunca en texto plano sobre HTTP.**
`Connection`	Control de la persistencia.	`Connection: close`	En HTTP/1.1 el default es persistente; el `close` es la excepción.
`If-None-Match`	Condición: «solo si el `ETag` cambió».	`If-None-Match: "3f2a1c8"`	Es lo que dispara el **304**.
`If-Modified-Since`	Condición por **fecha**.	`If-Modified-Since: Mon, 05 Oct 2026 09:14:07 GMT`	Alternativa más gruesa al `ETag`.
`If-Range`	«Si no cambió, haceme un `Range`».	`If-Range: "3f2a1c8"`	Permite **reanudar** una descarga sin corromper el archivo.
`Range`	Pide **un fragmento**.	`Range: bytes=2048-4095`	Base del video streaming y de las descargas reanudables.
`Origin`	De dónde viene la petición *cross-origin*.	`Origin: https://otro.edu.uy`	Con `Access-Control-Allow-Origin` implementa CORS.

Cabeceras de la **respuesta**:

Cabecera	Qué hace	Ejemplo
`Date`	Fecha y hora del servidor, **RFC 1123**, siempre en **GMT/UTC**.	`Date: Tue, 06 Oct 2026 14:22:31 GMT`
`Server`	Qué software atiende.	`Server: Apache/2.4.62 (Ubuntu)` — información de ataque gratuita.
`Content-Type`	**Qué es** el cuerpo.	`text/html; charset=utf-8`, `image/png`
`Content-Length`	**Cuántos bytes** tiene el cuerpo.	`Content-Length: 4096` — permite saber dónde termina la respuesta sin cerrar.
`Content-Encoding`	Qué compresión se aplicó.	`Content-Encoding: gzip`
`Transfer-Encoding`	Cómo se codifica **la transferencia**.	`Transfer-Encoding: chunked` — ver abajo.
`Location`	Dónde está o quedó el recurso.	`Location: /articulos/2026/10/creado` — obligatorio en un `201`.
`Allow`	Métodos y opciones admitidos.	`Allow: GET, HEAD, POST, OPTIONS`
`ETag`	**Etiqueta de versión** del recurso.	`ETag: "3f2a1c8-9b41-6512d3f0"`
`Last-Modified`	Última modificación.	`Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT`
`Cache-Control`	**Instrucciones de caché**.	`Cache-Control: max-age=600, private`
`Expires`	Fecha de vencimiento (mecanismo viejo).	`Expires: Tue, 06 Oct 2026 14:32:31 GMT` — hoy se prefiere `Cache-Control`.
`Vary`	**De qué depende** la variación de la respuesta.	`Vary: Accept-Encoding`
`Age`	Cuántos segundos lleva en la **caché de un intermediario**.	`Age: 124`
`Set-Cookie`	El servidor **crea** una cookie.	`Set-Cookie: sesion=abc123; Path=/; HttpOnly; Secure`
`WWW-Authenticate`	Qué esquema usar tras un `401`.	`WWW-Authenticate: Bearer realm="fing"`
`Access-Control-Allow-Origin`	A quién se le permite leer la respuesta.	`Access-Control-Allow-Origin: https://fing.edu.uy`

`Transfer-Encoding: chunked`

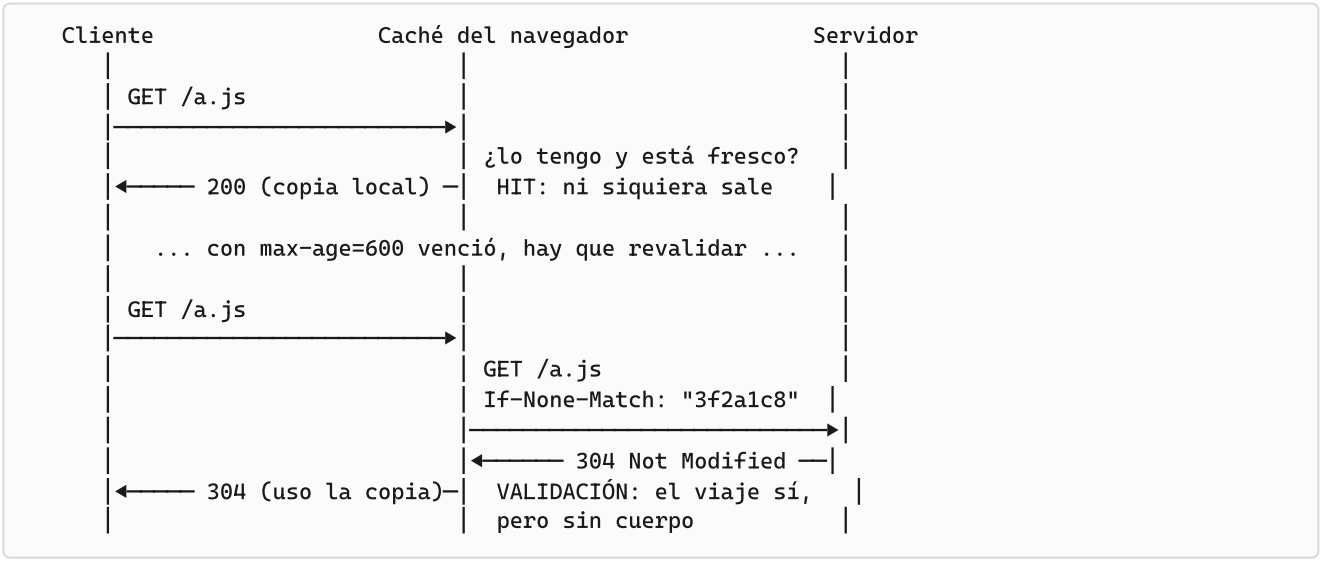
Si el servidor **no sabe de antemano** el tamaño del cuerpo (porque lo genera en streaming), no puede mandar `Content-Length`. La salida es mandar el cuerpo **fragmentado**:

```
HTTP/1.1 200 OK
Content-Type: text/plain
Transfer-Encoding: chunked

1a
Hola Mun
14
do: soy un cuerpo
0
```

Cada fragmento es `<tamaño en hexadecimal><CRLF>`, `<datos><CRLF>`. El `0` en hexadecimal es el **fin**. **Regla dura:** si mandás `Transfer-Encoding`, **no podés** mandar `Content-Length` al mismo tiempo, porque son dos formas distintas de decir dónde termina el cuerpo y el cliente no sabe cuál obedecer.

1.5 La caché HTTP

La caché existe porque **cargar una página es, mayoritariamente, esperar**. Ataca las dos fuentes de espera: **evita el viaje de ida y vuelta** cuando hay un *hit* fresco, y **si tiene que ir al servidor, lo hace con una validación condicional** (un `304` pesa 180 bytes en vez de 4 MB).



Directiva	Significado exacto	Consecuencia práctica
`no-store`	**No lo guardes en ninguna caché**: ni memoria, ni disco, ni intermediario.	Es lo que se pone en un banco. **Es la única que garantiza confidencialidad del dato en reposo.**
`no-cache`	**Guárdalo, pero no lo sirvas sin revalidarlo.**	Contradicción aparente e intencional: se cachea para no perder la copia, pero nunca se usa sin preguntar.
`max-age=N`	Fresco durante **N segundos**.	El TTL explícito de HTTP; desplaza al `Expires`.
`s-maxage=N`	Igual, pero **solo para cachés compartidas** (proxies, CDN).	Permite que un proxy tenga una ventana distinta de la del navegador.
`private`	**Solo** la caché del navegador puede guardar esto; los proxies compartidos **no**.	Obligatorio en toda respuesta con datos de usuario: si no, un proxy corporativo podría servirle el correo de otro usuario.
`public`	Se puede guardar en cachés **compartidas**.	Para contenido igual para todos, con caching agresivo.
`no-transform`	Ningún proxy puede cambiar el cuerpo (ni comprimirlo).	Para contenido ya comprimido o firmado.
`must-revalidate`	Vencido, **no** se puede servir ni del *stale*.	Se combina con `proxy-revalidate`.
`stale-while-revalidate=N`	Si está vencido, **servilo igual** y revalidá en segundo plano.	Reduce la latencia percibida en sitios con mucho tráfico.
`stale-if-error=N`	Si el origen falla, serví el viejo hasta `N` segundos.	Resiliencia.

El contraste que hay que entender:

```
no-store    → "no lo guardes"           (ni memoria ni disco)
no-cache    → "guardalo, pero revalidá" (memoria sí, disco sí)
max-age=0   → "fresco por 0 segundos"   ≡ revalidar SIEMPRE
```

`no-cache` y `max-age=0` terminan **haciendo lo mismo** en cuanto al uso, pero se diferencian en la **retención**: con `no-cache` el recurso queda y sirve si el servidor se cae; con `max-age=0` también queda, pero está vencido desde el minuto uno.

`Vary` y `Age`, los dos que casi nadie se acuerda. `Vary` **no dice cuánto tiempo vale**: dice **bajo qué claves** la respuesta es distinta.

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Encoding: gzip
Vary: Accept-Encoding
Cache-Control: max-age=600

┌──────────────────────────────────────────────────────┐
│  LA CACHÉ GUARDA DOS ENTRADAS:                       │
│    ( /index.html , Accept-Encoding: gzip ) → 200, 4096 B  │
│    ( /index.html , Accept-Encoding: br )   → 200, 3802 B  │
└──────────────────────────────────────────────────────┘
```

Sin `Vary`, la caché guarda **una sola** entrada y le devuelve al cliente que no soporta gzip **el cuerpo comprimido**: la página no abre. Es un bug real.

`Age` completa el modelo: cuando una respuesta pasa por un proxy, el proxy **resta el tiempo que estuvo cacheada**:

```
frescura restante = max-age − Age

Servidor manda:                    Cache-Control: max-age=600
El proxy lo tuvo 250 s y reenvía:  Age: 250
→ al cliente final le quedan 600 − 250 = 350 s de frescura
```

1.6 `ETag` + `Last-Modified`: la validación condicional

	`Last-Modified`	`ETag`
Es	una **fecha**	una **cadena opaca** que el servidor inventa
Precisión	**1 segundo**	la que el servidor quiera
Problema	dos versiones en el mismo segundo son **indistinguibles**	ninguno: es opaco y único por diseño
Ejemplo	`Mon, 05 Oct 2026 09:14:07 GMT`	`"3f2a1c8-9b41-6512d3f0"`
Variante débil	—	`W/"3f2a1c8"`: «no cambió semánticamente, aunque los bytes sí»

Las dos parejas que existen y hay que nombrar: `ETag` / `If-None-Match` y `Last-Modified` / `If-Modified-Since`. Si hay ambos validadores, **el `ETag` manda**.

El intercambio completo, mensaje por mensaje

Situación. El 6 de octubre a las 14:22:31 el cliente pide `/notas.pdf` y no tiene nada en caché. Una hora después vuelve a pedirlo.

Mensaje 1 — petición inicial:

```
GET /notas.pdf HTTP/1.1
Host: www.fing.edu.uy
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: application/pdf
Accept-Encoding: gzip, deflate
Connection: keep-alive
```

Mensaje 2 — respuesta completa:

```
HTTP/1.1 200 OK
Date: Tue, 06 Oct 2026 14:22:31 GMT
Server: Apache/2.4.62 (Ubuntu)
Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT
ETag: "3f2a1c8-9b41-6512d3f0"
Cache-Control: max-age=3600
Content-Type: application/pdf
Content-Length: 1048576
Accept-Ranges: bytes
Connection: keep-alive

%PDF-1.7
... 1.048.576 bytes en total ...
%%EOF
```

El cliente guarda el cuerpo, el `ETag`, el `Last-Modified` y el `Cache-Control`, y anota la hora de entrada: 14:22:31.

Una hora después (15:22:31) el `max-age=3600` venció, así que hay que revalidar.

Mensaje 3 — petición condicional:

```
GET /notas.pdf HTTP/1.1
Host: www.fing.edu.uy
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: application/pdf
If-None-Match: "3f2a1c8-9b41-6512d3f0"
If-Modified-Since: Mon, 05 Oct 2026 09:14:07 GMT
Connection: keep-alive
```

`If-None-Match` dice: «si el recurso tiene esta etiqueta, no me lo mandes entero, decime solamente si sigue siendo la misma». `If-Modified-Since` es la misma pregunta por fecha. Si el cliente mandó ambos, el servidor usa el `If-None-Match`, porque distingue dos versiones del mismo segundo.

Mensaje 4 — el recurso no cambió: respuesta 304:

```
HTTP/1.1 304 Not Modified
Date: Tue, 06 Oct 2026 15:22:32 GMT
ETag: "3f2a1c8-9b41-6512d3f0"
Cache-Control: max-age=3600
```

Fijate qué NO está: no hay cuerpo (cero bytes de PDF), no hay `Content-Length` (no hay nada que medir), no hay `Content-Type` (no hay nada que tipar), no hay `Server`.

Y qué SÍ está: el `ETag`, porque el `304` **renueva la frescura**: el reloj de la caché vuelve a cero y ahora vence a las 16:22:32, no a las 15:22:32. Esa es una de las sutilezas que más se pregunta.

Y si el archivo hubiera cambiado, el mismo mensaje 3 devolvería un `200` **completo**, con `Content-Length` y `ETag` distintos.

El cálculo del ahorro

```
Con 200 (re-descarga completa):
  cabeceras ≈       250 B
  cuerpo    = 1048576 B
  ──────────────────────
  total     = 1048826 B

Con 304 (validación):
  cabeceras ≈       180 B
  cuerpo    =         0 B
  ──────────────────────
  total     =      180 B

Ahorro = 1048826 − 180 = 1048646 B
Porcentaje = 1048646 / 1048826 ≈ 99,98 %
```

Interpretación física: el `304` no es una optimización de bytes, es un **cambio de orden de magnitud**. Y la letra chica: **igual se pagó el RTT de ida y vuelta**. El `304` ahorra *ancho de banda*, no *tiempo de espera*. Para matar el tiempo de espera están HTTP/2, HTTP/3 y el CDN.

1.7 Proxies, gateways y CDNs

	Proxy (*forward proxy*)	**Gateway**	**CDN**
Qué es	un intermediario que actúa **en nombre de** un cliente	un intermediario que **traduce entre protocolos**	una red de servidores que **guardan copia del contenido**
Ejemplo	el proxy del laboratorio o de la facultad	un servidor de correo que traduce SMTP a POP3	Cloudflare, Akamai, Fastly
¿Cambia el protocolo?	no	**sí**	no
¿Guarda contenido?	a veces	no	**sí, siempre**
Para qué sirve	filtrar, auditar, autenticar, compartir una salida	traducir	**latencia y descarga del origen**

> **Ojo con «gateway» en HTTP.** En la práctica el término se usa como «puerta de entrada»: un `502 Bad Gateway` casi nunca es «el servidor traductor falló», es **«el proxy inverso no pudo hablar con el backend»**. `504 Gateway Timeout` es «el proxy esperó demasiado al backend».

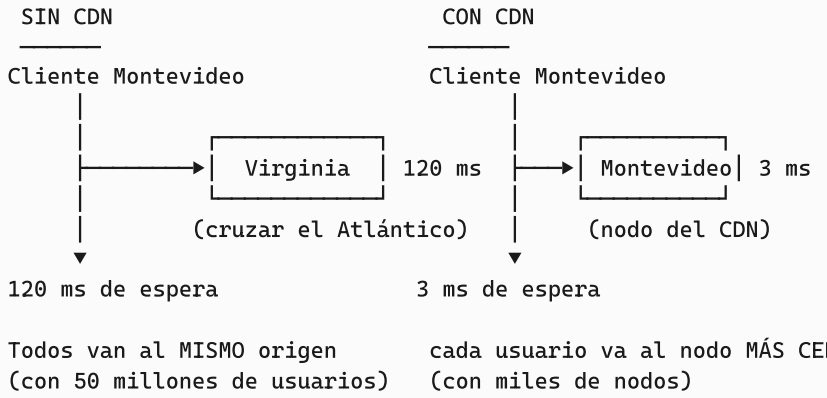
Por qué un CDN mejora la latencia: la distancia al usuario

La Clase 8 mostró que **más del 80 % del tiempo de carga de una página es espera**. Toda esa espera es tiempo de ida y vuelta:

```
t_espera ≈ (número de viajes) × RTT
y el RTT es proporcional a la distancia física y a los saltos
```

Cliente	Servidor en Virginia	Servidor en Montevideo
Montevideo	**120 ms**	**3 ms** (diferencia de 40×)
Madrid	95 ms	160 ms
Buenos Aires	130 ms	175 ms

Un **CDN** copia el contenido a un nodo **cerca** de cada usuario y responde desde ahí:



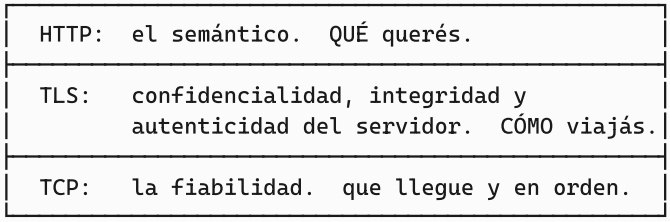
El argumento en una frase, que es lo que hay que escribir: un CDN mejora la latencia **reduciendo la distancia al usuario**, y como el tiempo de carga de una página es esencialmente `número de viajes × RTT`, bajar el RTT de 120 ms a 3 ms hace cargar la página **antes**, sin tocar un byte de contenido.

El segundo beneficio es la **descarga del origen**: si el 95 % de los pedidos lo sirve el nodo del CDN, el origen ve el 5 % y sobrevive. Por eso un CDN es también un **escudo contra DDoS**.

CDN y caché, la diferencia: la caché HTTP guarda respuestas en un punto **individual** (el navegador, un proxy). El CDN es un **sistema distribuido** de cachés coordinados entre sí: cuando un nodo recibe un pedido de algo que no tiene, **se lo pide a un par**, lo guarda y lo sirve. Eso se llama *fill on miss*, y explica por qué un CDN es barato: no duplica todo el mundo, solo lo que se pide.

1.8 HTTPS: qué agrega (y qué no es)

HTTPS es HTTP transportado por TLS. No cambia **una línea** de la semántica de HTTP: los mismos métodos, los mismos códigos, las mismas cabeceras. Lo que cambia es el **transporte**.



Propiedad	Qué aporta	Sin HTTPS	Con HTTPS
Confidencialidad	nadie en el camino ve el contenido	texto plano en cada router, ISP y proxy	**cifrado** de extremo a extremo
Integridad	nadie puede alterar el mensaje en vuelo	un intermediario puede cambiar el saldo de tu cuenta en el HTML	**AEAD**: alterar algo rompe el descifrado
Autenticación del servidor	sabés con quién estás hablando	**cero**: cualquiera que responda el puerto 80 es «el sitio»	**certificado X.509** contra una cadena de confianza
Identificación del cliente	opcional	imposible	opcional: **mTLS**

HTTPS no es una versión de HTTP. `HTTPS/3` no es «HTTP/3 con candado»: es **HTTP/3 sobre QUIC sobre TLS 1.3**, todo integrado. Y **no protege al servidor de origen** de un `502`, ni evita un *clickjacking*, ni oculta qué sitio visitaste (para eso está el DNS cifrado, Clase 11).

> ⚠️ **Adelantado:** TLS entero — cadena X.509, handshake, la separación del *cipher suite* en tres, el `0-RTT` y sus riesgos de *replay* — se ve **entero en la Clase 47**. Acá solo hace falta la capa y la frase «**HTTP aporta semántica, TLS aporta seguridad**».

1.9 De HTTP/1.1 a HTTP/2 y HTTP/3

Los tres problemas estructurales de HTTP/1.1

```
═══ (a) HOL blocking A NIVEL DE CONEXIÓN ═══

conexión 1:  GET ──► 200 ──► GET ──► 200 ──► ...
conexión 2:  GET ──► 200 ──► GET ──► 200 ──► ...
conexión 3:  GET ── X SE PIERDE ──────────────► 404, muy tarde

En HTTP/1.1 los objetos van en conexiones distintas, pero cada conexión
TCP es ORDENADA: el hueco de la conexión 3 la frena a ella entera.
ESTO ES HEAD-OF-LINE BLOCKING.


═══ (b) HOL blocking ENTRE STREAMS ═══

HTTP/2  ┬─ stream 1 ──── X SE PIERDE ──────────► 404
        ├─ stream 2 ── 200 ──►
        ├─ stream 3 ─── 200 ──►

Multiplexamos: la pérdida frena SOLO al stream 1.
...pero DENTRO de cada stream sigue habiendo bloqueo, porque la conexión
TCP de abajo sigue siendo ordenada.


═══ (c) El costo de las CABECERAS ═══

Un navegador manda unas 10 cabeceras por GET y el servidor unas 8.
En HTTP/1.1 viajan en texto plano, sin comprimir, en CADA petición.
HTTP/1.1 no tiene compresión de cabeceras. HTTP/2 y HTTP/3 sí.
```

HTTP/2 (`RFC 9113` , 2022)

La idea central: **una sola conexión TCP, muchos streams independientes**.

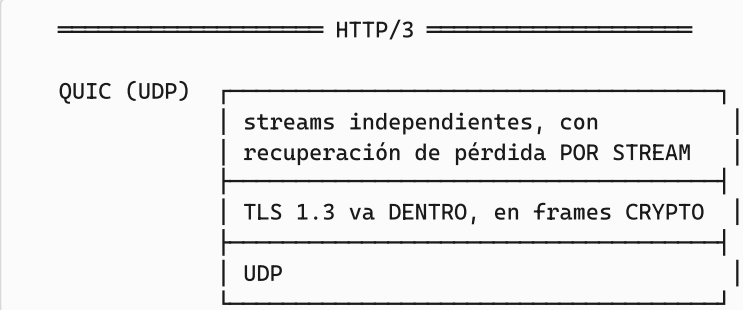
```
═══════════════════ HTTP/1.1 ═══════════════════

conexión TCP 1:  GET ▶ 200 ▶ GET ▶ 200 ▶ ...
conexión TCP 2:  GET ▶ 200 ▶ GET ▶ 200 ▶ ...
conexión TCP 3:  GET ▶ 200 ▶
conexión TCP 4:  GET ▶ 200 ▶
conexión TCP 5:  GET ▶ 200 ▶ GET ▶ 200 ▶ ...

5 conexiones, y en cada una los pedidos van UNO POR UNO.


═══════════════════ HTTP/2 ═══════════════════

conexión TCP 1 (¡UNA SOLA!):
┌────┬────┬────┬────┬────┬────┬────┬────┐
│ s1 │ s2 │ s3 │ s4 │ s5 │ s6 │ s7 │ s8 │  ← 8 streams simultáneos
└──┬─┴────┴────┴──┬─┴────┴────┴────┴────┘
   │              │
   ▼              ▼
[se pierde]   sigue fluyendo normal
frena s1      s2..s8 no se enteran
```

	HTTP/1.1	HTTP/2
Streams por conexión	**1** (los demás esperan)	**muchos**, simultáneos
HOL blocking a nivel de conexión	**sí**	**eliminado**
HOL blocking entre streams	no aplica	**sigue existiendo**: la pérdida de un stream frena a ese stream
Compresión de cabeceras	no	**HPACK** (`RFC 7541` , vigente)
Cabeceras en mayúsculas y repetidas	sí	se normalizan a **minúsculas** y se indexan con HPACK
Orden de los bytes	obligatorio	el **stream ID** identifica el flujo; los frames se entrelazan
Servidor que manda primero	no	**sí**, con `PUSH_PROMISE`
Prioridad	cabecera `Priority:`	la de `RFC 7540` **quedó deprecada** (§5.3 de la 9113); la reemplaza **`RFC 9218`**

El detalle que suma puntos. `RFC 7540` (HTTP/2, 2015) especificaba un esquema de prioridad con *weight* y dependencia entre streams. Al reescribir HTTP/2 en `RFC 9113` , ese esquema **se deprecó**: la razón es que **en la práctica casi nadie lo usaba** y las implementaciones divergían. El frame `PRIORITY` sobrevive solo por compatibilidad. Lo que lo reemplaza es **`RFC 9218`** **— *Extensible Prioritization Scheme* (EPS)**, que separa la **importancia semántica** del stream (`urgency`) de la **dependencia** entre unos y otros, con sintaxis textual en vez de números de peso opacos. Lección de protocolo: **especificar una característica de prioridad, descubrir que nadie la implementa y deprecarla, es un resultado normal.**

HTTP/3 (`RFC 9114`) sobre QUIC (`RFC 9000`)

HTTP/3 **no es «HTTP/2 sobre UDP»**. Reemplaza **la conexión TCP completa** por **QUIC**, un transporte nuevo con su propia recuperación de pérdida, su propio control de congestión y su propio PLPMTUD.

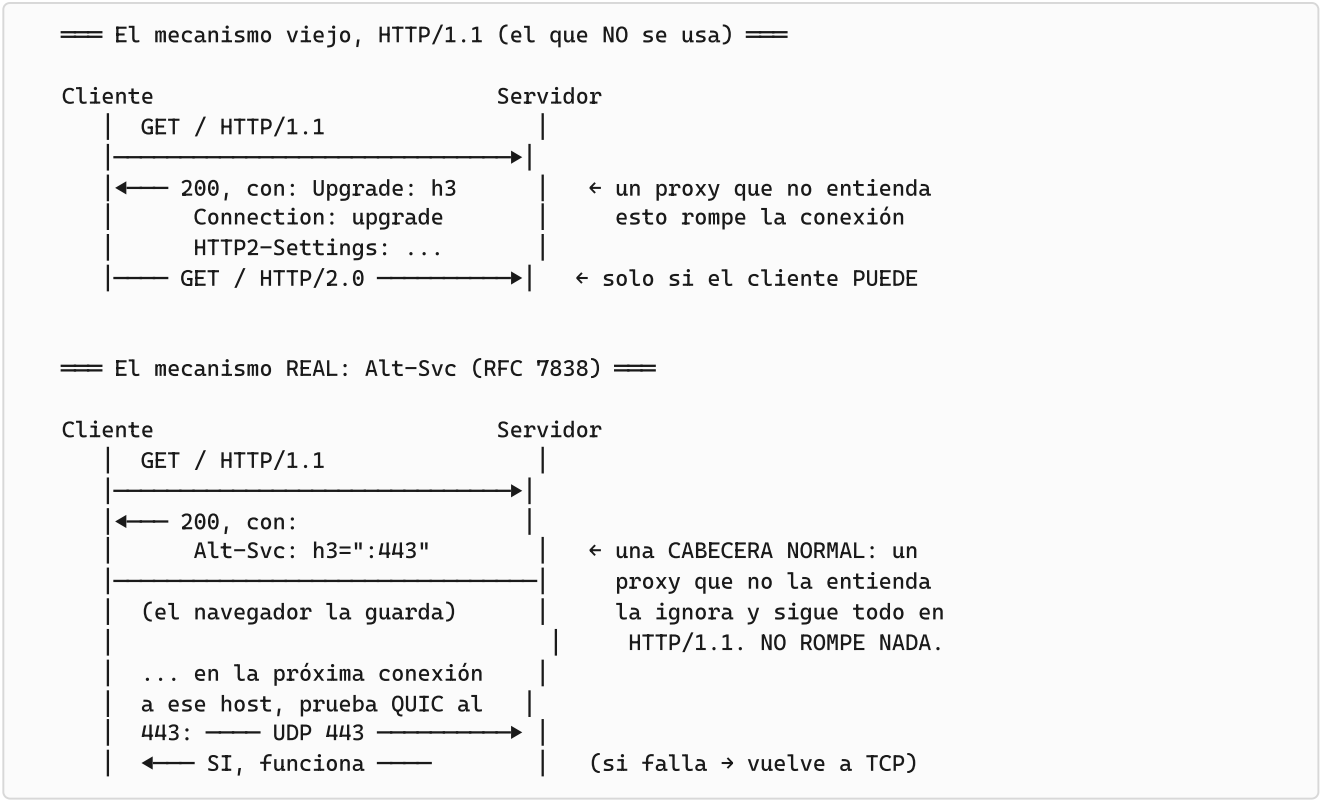



```
NO hay handshake de 3 vías.
NO hay conexión TCP.
NO hay retransmisión "de la conexión": cada stream se recupera solo.
NO hay identidad por IP: el Connection ID permite cambiar de IP y de
puerto sin cortar la sesión (esto arregla el NAT rebinding).
```

QPACK (`RFC 9204`) es la compresión de cabeceras de HTTP/3. La diferencia con HPACK es conceptual:

```
HPACK   (HTTP/2): el índice se mantiene al ORDENAR las cabeceras.
                  Es SECUENCIAL: para descomprimir el frame N necesito
                  haber descomprimido el N-1.

QPACK   (HTTP/3): los índices se resuelven en CUALQUIER orden.

¿Por qué? Porque en HTTP/3 los frames viajan en streams DISTINTOS, que
llegan al receptor en orden arbitrario. Si la compresión fuera
secuencial, la multiplexación sería imposible.
```

Cómo se negocia HTTP/3: `Alt-Svc` , NO `Upgrade` . Esta es la corrección conceptual más importante frente a 2012:



La diferencia clave: `Upgrade` es un **cambio de protocolo dentro de la conexión actual**, frágil frente a los intermediarios. `Alt-Svc` es **anunciar una alternativa para la próxima conexión**, en una cabecera común que se puede ignorar. El navegador lo guarda y lo prueba después: **cero riesgo, adopción gradual.**

Para descubrir HTTP/3 **antes** de conectar, sin haber ido una vez por HTTP/1.1, están los registros DNS **`SVCB` / `HTTPS` (`RFC 9460`, 2023)**: el DNS ahora transporta **política de transporte** (qué versiones de HTTP soporta, qué IP de *hint*, qué prioridad), no solo direcciones. Es el mismo cambio conceptual que `CAA`. Lo desarrollamos en la Clase 11.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - Modelo **petición/respuesta**, protocolo **sin estado**. - Los **métodos**: GET, HEAD, POST, PUT, DELETE, OPTIONS, TRACE, CONNECT (+ PATCH). GET/HEAD/OPTIONS/TRACE **seguros**; GET/HEAD **idempotentes**. - Las **5 clases** de códigos (1xx a 5xx) y los del día a día: 200, 201, 204, 206, 301, 302, 304, 307, 308, 400, 401, 403, 404, 405, 408, 429, 500, 502, 503, 504. - **`Host` obligatoria** en HTTP/1.1, y **persistente por defecto**. - `Range` requests y `304 Not Modified`. - **HTTP/1.1 sigue siendo totalmente válido y necesario**: no es «el protocolo viejo», es el *fallback* de todo.

Cambió — y esto es lo que hay que corregir si citás apuntes de 2011: - El curso de 2012 enseñaba **`RFC 2616` (1999)**. La especificación vigente es de **2022**: **`RFC 9110`** (semántica, *Internet Standard*) + **`RFC 9112`** (sintaxis de HTTP/1.1). - **`RFC 7540`** (HTTP/2, 2015) está **OBSOLETA**: la reemplazó **`RFC 9113`**. - **HTTP/3 (`RFC 9114`)** va sobre **QUIC (`RFC 9000`)**, que **reemplaza a TCP por completo**: no hay handshake de 3 vías, no hay conexión TCP, no hay retransmisión de conexión. La pérdida se resuelve **por stream**. QPACK es `RFC 9204`; QUIC v2 es `RFC 9369`. - La **negociación** es por **`Alt-Svc` (`RFC 7838`, 2016)**, no con `Upgrade`. - `HPACK` es `RFC 7541` y **sigue vigente**; `QPACK` es `RFC 9204` y es **independiente del orden**, a diferencia de HPACK.

Las cifras reales — y acá está el punto pedagógico del tema:

Versión	Cuota (Cloudflare Radar, *Year in Review 2025*)
HTTP/2	**50 %**
HTTP/1.x	**29 %**
HTTP/3	**21 %**

W3Techs (oct-2026) da ~**40,8 % de los sitios** con soporte de HTTP/3. Navegadores con HTTP/3 por defecto: Chrome/Edge **87** (nov-2020), Firefox **88** (abr-2021), Safari **16** (2020).

> ⚠️ **Trampa metodológica:** si filtrás bots de los datos de Radar, HTTP/3 aparece cerca de **30 %**, porque los bots hablan HTTP/1.x casi siempre. **Siempre declará si incluís bots** si citás una cifra.

Por qué HTTP/3 no se adueñó de la web. Es la pregunta conceptual que vale más que las cifras:

1. **El cambio es gradual y la Web es heterogénea.** HTTP/3 necesita cliente **y** servidor; actualizar el cliente requiere que el usuario cambie de navegador.
2. **Casi todo el tráfico sigue necesitando TCP.** DNS clásico, SSH, servidores de correo, juegos sin migrar. QUIC no reemplaza a TCP: **convive** con él.
3. **Los *middleboxes* son el cuello de botella real.** Cualquier firewall, balanceador o IDS que no entienda QUIC o que filtre UDP/443 **rompe** HTTP/3. Son 25 años de equipos instalados que nadie va a actualizar: HTTP/3 se despliega a la velocidad a la que se reemplazan firewalls.
4. **El costo de adopción de HTTP/2 fue bajo.** Solo cambiaba la capa de aplicación sobre un TCP ya desplegado: bastaba actualizar el servidor web, que es un acto administrativo, no un acto de usuario.

Consecuencia para el examen: si te preguntan «¿cuál es la versión actual de HTTP?», la respuesta completa es **«HTTP/2 es la más usada (50 %); HTTP/3 va en crecimiento (21 %) y no se adueñó de la web, porque el cambio es gradual y casi todo el tráfico sigue necesitando TCP; HTTP/1.1 sigue vivo y es el baseline de comparación»**. Si solo decís «HTTP/3», perdés puntos.

1.10 La tabla que hay que saber de memoria

```
╔═══════════════════════════════════════════════════════════╗
║              QUÉ SIGUE IGUAL EN HTTP/1.1, /2 Y /3         ║
╠═══════════════════════════════════════════════════════════╣
║ 1. Petición / respuesta. Siempre.                         ║
║ 2. Sin estado: el servidor no recuerda peticiones anteriores.  ║
║ 3. Los métodos: GET HEAD POST PUT DELETE OPTIONS TRACE CONNECT. ║
║ 4. Las clases de estado 1xx 2xx 3xx 4xx 5xx.              ║
║ 5. Host obligatoria (en HTTP/2 y HTTP/3 viaja en la       ║
║    pseudo-cabecera :authority).                           ║
║ 6. Persistente por defecto: una conexión, varios recursos.    ║
║ 7. 304 Not Modified y los Range requests.                 ║
║ 8. HTTPS = HTTP sobre TLS. TLS sigue siendo TLS.          ║
╚═══════════════════════════════════════════════════════════╝
```

El punto 5 merece aclaración. En HTTP/2 y HTTP/3 **no hay cabeceras de texto**: los campos viajan en **HPACK** / **QPACK** y se llaman ***pseudo-cabeceras***. `:authority` reemplaza a `Host`, `:method` al método, `:path` a la URL, `:scheme` a `http` / `https`. Todo lo demás (`Content-Type`, `Cookie`, `ETag`, `Cache-Control`) sigue igual, solo que **en minúsculas y comprimido**. Si un examen pide «escribí un mensaje HTTP/2», lo que se espera es la lista de pseudo-cabeceras, no un `GET /... HTTP/1.1` con cabeceras de texto.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Intercambio completo con validación condicional hasta el 304

Enunciado. Un cliente pide `https://www.fing.edu.uy/notas/2026.pdf`. El servidor responde con `ETag`, `Last-Modified` y `Cache-Control: max-age=1800`. Pasados 45 minutos vuelve a pedirlo; el servidor no contesta. Una hora después vuelve a pedirlo y el archivo cambió.

Datos. El recurso pesa **200 000 bytes**. Las cabeceras de respuesta ocupan unos **300 bytes**. El RTT cliente-servidor es de **80 ms**.

Resolución paso a paso.

Paso 1 — Primer pedido: no hay nada en caché.

```
GET /notas/2026.pdf HTTP/1.1
Host: www.fing.edu.uy
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: application/pdf
Accept-Encoding: gzip, deflate
Accept-Language: es-UY,es;q=0.9
Connection: keep-alive
```

Interpretación: `GET` **sin cuerpo** y **sin cabeceras condicionales**: el cliente no tiene nada que ofrecer, así que la petición es completa.

Paso 2 — Respuesta del servidor: `200` *con validadores.*

```
HTTP/1.1 200 OK
Date: Tue, 06 Oct 2026 09:00:00 GMT
Server: nginx/1.26.1
Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT
ETag: "5d9b21-30d40-6518a1f0"
Cache-Control: max-age=1800
Accept-Ranges: bytes
Content-Type: application/pdf
Content-Length: 200000
Connection: keep-alive

%PDF-1.7 ... 200.000 bytes ... %%EOF
```

Interpretación: el cliente guarda **cuatro** cosas: el cuerpo, el `ETag`, el `Last-Modified` y el `Cache-Control`. Anota la **hora de entrada**: 09:00:00.

Paso 3 — ¿Cuándo vence?

```
   Vencimiento = hora de entrada + max-age
               = 09:00:00 + 1800 s = 09:00:00 + 30 min = 09:30:00
```

Interpretación: entre las 09:00 y las 09:30 el recurso es **fresco** y el cliente **no vuelve a preguntar**: cero bytes, cero RTT.

Paso 4 — Segundo pedido (09:45): el `max-age` venció, hay que revalidar. El mensaje es **idéntico al del paso 1** más las dos cabeceras condicionales:

```
If-None-Match: "5d9b21-30d40-6518a1f0"
If-Modified-Since: Mon, 05 Oct 2026 09:14:07 GMT
```

Interpretación: `If-None-Match` es la **pregunta condicional**: «si el recurso tiene esta etiqueta, no me lo mandes entero». `If-Modified-Since` es la misma pregunta por fecha. Si el cliente mandó ambos, el servidor **usa el `If-None-Match`**, porque el `ETag` distingue dos versiones del mismo segundo.

Paso 5 — El servidor no contesta.

Interpretación física: el cliente no puede saber si el recurso cambió. El `max-age` venció, así que su copia técnicamente no es válida. Los navegadores reales aplican el **heuristic de frescura**: sin directivas explícitas se usa el 10 % de la edad del recurso. Acá hay `max-age`, así que lo correcto es servir la copia local **marcándola como posiblemente vieja**, que es lo que hace `stale-while-revalidate`.

Paso 6 — Tercer pedido (10:00): ahora sí, y el archivo cambió. La respuesta es un `200` completo **idéntico al del paso 2** salvo tres campos: `Last-Modified` pasa a `Tue, 06 Oct 2026 09:50:11 GMT`, el `ETag` pasa a `"5d9b21-31234-6518a9c8"` y `Content-Length` pasa a `201344`.

Interpretación: como el `ETag` del servidor **no coincide** con el `If-None-Match`, la condición **se ignora** y se manda el `200` completo. El cliente **reemplaza** su copia: nuevo cuerpo, nuevo `ETag`, nueva fecha.

Pedido	Hora	Condición	Respuesta	Bytes de cuerpo	Total
1	09:00	ninguna	`200`	**200 000**	80 ms
2	09:45	`If-None-Match`	*sin respuesta*	0	80 ms
3	10:00	`If-None-Match`	`200` (cambió)	**201 344**	80 ms

```
Datos transferidos:
  pedido 1:  200 000 + 300 ≈ 200 300 B
  pedido 2:        0 + 180 ≈     180 B
  pedido 3:  201 344 + 300 ≈ 201 644 B
  ──────────────────────────────────
  Total:                          402 124 B

RTT totales: 3 × 80 ms = 240 ms
```

Interpretación: el segundo pedido bajó **0 bytes de contenido** en 180 bytes de cabeceras: **99,91 % de ahorro en una petición**. Por eso la caché no es una optimización menor, es la diferencia entre 200 KB y 180 B por revalidación.

⚠️ Errores típicos

- **Olvidar la línea en blanco final** del mensaje de petición: sin ella el servidor no sabe dónde terminan las cabeceras.
- **Poner `Content-Length` en un `304`:** un `304` no lleva cuerpo, y por lo tanto no lleva `Content-Length`.
- **Confundir «el `ETag` no coincide» con «me manda un 404»:** si no coincide, la condición **se ignora** y se manda un `200` completo.
- **Creer que el `304` renueva el TTL del origen:** renueva la **frescura en la caché del cliente**, que vuelve a contar `max-age` desde cero.

2.2 Ejercicio 2 — ¿Qué hace exactamente `Cache-Control: max-age=0` con un ETag presente?

Enunciado. Un servidor responde:

```
HTTP/1.1 200 OK
Date: Tue, 06 Oct 2026 09:00:00 GMT
ETag: "5d9b21-30d40-6518a1f0"
Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT
Cache-Control: max-age=0

<contenido>
```

¿Qué puede y qué no puede hacer el cliente con esto?

Paso 1 — Leer el `max-age=0`. La frescura es de **cero segundos**, o sea **nunca es fresco**: cada vez que el cliente quiera usarlo tiene que revalidar. Pero `max-age=0` **no dice «no lo guardes»** (eso sería `no-store`) ni «no lo devuelvas sin revalidar» (eso sería `no-cache`).

Paso 2 — Lo que SÍ puede hacer el cliente.

```
✅ GUARDAR el recurso en su caché.
✅ Usarlo si el servidor NO responde (fallback, offline).
✅ Aceptar el 304 cuando el servidor confirma que no cambió.
```

Paso 3 — Lo que NO puede hacer.

```
❌ Servirlo de la caché sin preguntarle al servidor.
❌ Asumir que sigue válido porque acaba de bajarlo.
❌ Cachearlo «para siempre» si el servidor nunca más responde.
```

Paso 4 — Y acá está el detalle fino: qué cambia cuando hay `ETag`.

```
SIN ETag y con max-age=0:
  el cliente no puede distinguir "cambió" de "sigue igual"
  → tiene que re-descargarlo ENTERO cada vez.     (200 000 B por uso)

CON ETag y con max-age=0:
  el cliente manda If-None-Match: "5d9b21-30d40-6518a1f0"
  el servidor compara con su ETag actual:
    → si coincide:  304, 0 B de cuerpo
    → si no:        200 completo, 200 000 B

El 304 ocurre, pero NO HAY AHORRO DE RTT: el viaje se sigue entero.
Lo que se ahorra son BYTES.
```

Paso 5 — La cuenta, con números.

```
Recurso de 200 000 B. 5 usos en 1 hora.

Con max-age=3600 (1 hora de frescura):
  uso 1:  200 000 B (descarga)
  uso 2:      0 B (fresco, ni pregunta)
  uso 3:      0 B (fresco, ni pregunta)
  uso 4:      0 B (fresco, ni pregunta)
  uso 5:      0 B (fresco, ni pregunta)
  ──────────────────────────
  Total: 200 000 B        RTT: 1

Con max-age=0 + ETag:
  uso 1:  200 000 B (descarga)
  uso 2:      0 B (304, un RTT)
  uso 3:      0 B (304, un RTT)
  uso 4:      0 B (304, un RTT)
  uso 5:      0 B (304, un RTT)
  ──────────────────────────
  Total: 200 000 B        RTT: 5
```

Interpretación física, y es la frase del examen: **`max-age=0` convierte cada uso en una revalidación, pero gracias al `ETag` cada revalidación cuesta 0 bytes de contenido.** Los bytes totales son los mismos; lo que cambia son los **viajes**: de 1 RTT a 5 RTT. Con un RTT de 200 ms son 800 ms de espera extra.

	`max-age=0` + ETag	`no-cache` + ETag	`no-store`
¿Se guarda en caché?	sí	sí	**no**
¿Hay que revalidar siempre?	sí (0 s de frescura)	**sí, explícitamente**	no aplica
¿Sirve si el servidor cae?	sí	sí	**no**
Diferencia	implícito, por aritmética	explícito, por declaración	—

Respuesta final: `max-age=0` marca el recurso como **vencido de entrada** pero **conservable**: el cliente lo guarda (para poder trabajar sin conexión) y en cada uso revalida contra el servidor. Con `ETag`, esa revalidación es barata en **bytes** (un `304` de 180 bytes) pero **no** economiza el **viaje**. Si lo que se quiere es no revalidar nunca, la directiva correcta es `no-store`.

⚠️ Errores típicos

- **Decir que `max-age=0` equivale a `no-cache`.** En cuanto al comportamiento de uso sí, pero `no-cache` **declara** la intención y `max-age=0` la **logra por aritmética**.
- **Decir que `max-age=0` es «no cachear».** Falso: **sí** se cachea; lo que no existe es la *frescura*.
- **Olvidar el efecto de `Age`.** Si un intermediario sirve con `Age: 30` y `max-age=0`, la frescura es `0 − 30 = −30`: **vencida**, hay que revalidar igual.

2.3 Ejercicio 3 — ¿Qué gana y qué pierde HTTP/3 frente a HTTP/1.1?

Enunciado. Compará HTTP/3 y HTTP/1.1 en cuatro dimensiones: latencia de conexión, comportamiento ante pérdida, aislamiento del tráfico y coste operacional.

Paso 1 — Latencia de conexión, con la cuenta a la vista.

```
HTTP/1.1 sobre TCP + TLS:
  TCP     SYN ──►  ◄── SYN+ACK  ACK ──►        1 RTT
  TLS 1.3 ClientHello ─►  ◄── ServerHello        1 RTT
  ──────────────────────────────────────────────────
  Total hasta el primer byte de aplicación:          2 RTT

HTTP/3 sobre QUIC (que ya incluye TLS 1.3):
  QUIC    Initial + Handshake combinado               1 RTT
  ──────────────────────────────────────────────────
  Total hasta el primer byte de aplicación:          1 RTT
```

Interpretación: se ahorra **un RTT entero** por conexión nueva. Y como HTTP/3 multiplexa, **no abre una conexión por objeto**: abre una y la reutiliza.

Paso 2 — Pérdida de paquetes.

```
HTTP/1.1:  una conexión = un flujo ordenado.
           si se pierde el segmento 5, el receptor NO puede entregar
           el 6 ni el 7 hasta que llegue el 5 y se retransmite
           (1 RTO con fast retransmit, un RTO entero con timeout).

HTTP/3:    muchos streams independientes, CADA UNO con su numeración
           y su recuperación. Si se pierde el stream 5, el 6 se
           entrega igual. Se reintenta SOLO el 5.
```

Interpretación física: es la diferencia entre **«toda la página quedó congelada 1,5 segundos porque se perdió un byte de una imagen»** y **«esa imagen tarda un poco más; el texto ya se ve»**. El congelamiento total de la página es el síntoma clásico del *head-of-line blocking* de HTTP/1.1.

Paso 3 — Aislamiento del tráfico, el punto donde HTTP/2 queda a medias.

```
HTTP/2:  multiplexa sobre UNA conexión TCP. Cada stream se numera,
         pero TCP es ORDENADO: se pierde un segmento → se frena LA
         CONEXIÓN ENTERA, aunque solo le afecte a un stream.

         ╔═══════════════════════════════════════════════════╗
         ║ HTTP/2 elimina el HOL blocking A NIVEL DE         ║
         ║ CONEXIÓN, pero NO ENTRE STREAMS.                  ║
         ╚═══════════════════════════════════════════════════╝

HTTP/3:  cada stream tiene su numeración y su recuperación: la
         pérdida de uno no frena a los otros.

         ╔═══════════════════════════════════════════════════╗
         ║ HTTP/3 elimina el HOL blocking ENTRE STREAMS      ║
         ║ porque ya no hay conexión TCP debajo.             ║
         ╚═══════════════════════════════════════════════════╝
```

Paso 4 — Qué gana y qué pierde HTTP/3.

Gana	Por qué
1 RTT en vez de 2 para conectar	el handshake QUIC incluye el de TLS
Sin *head-of-line blocking* entre streams	recuperación por stream, no por conexión
Migración de conexión	el `Connection ID` sobrevive al cambio de IP y de puerto, lo que arregla el *NAT rebinding* que rompe a TCP
PLPMTUD integrado	mide el camino con su propio tráfico, sin depender de que ICMP no esté filtrado
QPACK sin orden	permite comprimir cabeceras en paralelo, coherente con la multiplexación
0-RTT con reanudación	con tokens anti-replay

Pierde	Por qué
CPU por paquete	cada stream va cifrado con AEAD; el *throughput* en máquinas pequeñas baja
UDP a veces está bloqueado	redes corporativas y operadores filtran UDP/443 o no pasan QUIC
Los *middleboxes* son el cuello de botella	cualquier firewall o IDS que no entienda QUIC **rompe** la conexión
Observabilidad peor	sniffear HTTP/3 exige las claves del endpoint
Menos despliegue	21 % del tráfico de Radar, 40,8 % de los sitios

Paso 5 — El veredicto, que es la frase del examen.

> **HTTP/3 gana en latencia de conexión, en aislamiento de la pérdida y en movilidad; pierde en costo de CPU, en compatibilidad con los equipos que filtran UDP y en despliegue.** La razón por la que no se adueñó de la Web no es técnica —es mejor en casi todo— sino **política de despliegue**: el cambio es gradual, requiere actualizar clientes y servidores, y se choca con 25 años de *middleboxes* instalados que nadie va a cambiar. **HTTP/2 ganó más rápido porque solo cambiaba la capa de aplicación sobre un TCP que ya estaba desplegado.**

⚠️ Errores típicos

- **Decir «HTTP/3 es HTTP/2 sobre UDP». Falso**, y es el error conceptual más caro. HTTP/3 reemplaza TCP **por completo**.
- **Decir que HTTP/2 elimina el HOL blocking sin matizar.** Solo a nivel de conexión; entre streams sigue existiendo.
- **Confundir `Upgrade` con `Alt-Svc`.**
- **Decir «HTTP/3 es la versión actual y la más usada».** La más usada es HTTP/2.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-07, Pregunta 1 (8 puntos) — página HTML con diez imágenes

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1007.pdf **Nota:** la resolución completa con la cuenta de RTT ya está en la **Clase 8 §3.1**. Acá se reproduce desde el ángulo de las **versiones**, que es lo que esta clase agrega.

Enunciado (texto literal del examen)

> **Pregunta 1 (8 puntos)**
>
> Suponga que un usuario intenta cargar en su browser una página HTML. Esta página contiene texto y diez imágenes.
>
> **a)** Describa la secuencia de pasos que ocurren a nivel de capa de aplicación en el cliente y el servidor a los efectos de desplegar la página completa, suponiendo que se utiliza HTTP no persistente.
>
> **b)** ¿Cuál es la diferencia con HTTP persistente? ¿Mejora la performance? Discuta.

Solución oficial (del PDF, textual)

a)

Paso	Responsable	Acción
1	Cliente	Establece la conexión con el servidor
2	Cliente	Envía Request pidiendo la página
3	Servidor	Envía Response con el contenido de la página
4	Servidor	Cierra la conexión
5*	Cliente	Establece la conexión con el servidor
6*	Cliente	Envía Request pidiendo la la imagen 1
7*	Servidor	Envía Response con el contenido de la imagen 1
8*	Servidor	Cierra la conexión

* Se repiten los pasos 5 al 8 para cada una de las imágenes.

b) En el caso de HTTP persistente, se establece la conexión una sola vez, se obtiene el contenido de la página, y a través de la misma conexión se piden todas las imágenes.

El uso de HTTP persistente puede resultar más performante en los casos en que se utilice una sola conexión por vez (por ejemplo si el servidor nos limita por IP), ya que no existe el overhead de crear nuevas conexiones para cada imagen.

> Si el navegador abre conexiones en paralelo para obtener varias imágenes a la vez, probablemente termine de obtener todos los datos más rápidamente, aprovechando mejor el ancho de banda.

a) y b) — Resolución

La tabla oficial es correcta y es la respuesta del apartado a). Lo que hay que agregar son las **capas que la oficial abstrae**: el paso 1 vale **1 RTT de TCP** (2 con HTTPS), el paso 4 es un `FIN` que no cuesta RTT pero deja un `TIME-WAIT` de 2×MSL en el cliente, y el paso 3 debería llevar `Cache-Control` + `ETag` + `Last-Modified`, que es lo que hace que la **segunda** carga de la misma página sea un `304` en lugar de 11 descargas.

Respuesta redactada, lista para copiar en el examen:

> **a)** En HTTP **no persistente**, por cada objeto (el HTML y cada una de las 10 imágenes) el cliente ejecuta **cuatro pasos**: (1) establece una conexión TCP con el servidor, (2) envía un request pidiendo el objeto, (3) el servidor envía un response con el contenido del objeto, (4) el servidor cierra la conexión. El primer objeto es la página HTML; al recibirla y parsearla, el cliente descubre las 10 imágenes referenciadas y repite los pasos 1–4 **una vez por cada imagen**. En total son **11 objetos × 4 pasos = 44 pasos**, con **11 conexiones TCP distintas** y **22 RTT** (2 RTT por objeto: 1 de handshake TCP + 1 de request/response). Si el sitio usa HTTPS, a cada uno de esos RTT se le suma 1 RTT de TLS: **33 RTT**.
>
> **b)** En HTTP **persistente** se establece **una sola** conexión TCP y por ella se piden los 11 objetos uno tras otro: **1 RTT** de handshake + **11 RTT** de request/response = **12 RTT**, contra los 22 del no persistente. La mejora es real cuando el navegador trabaja **secuencialmente**, porque se eliminan los `N − 1 = 10` handshakes. **Pero no siempre mejora:** si el navegador abre conexiones **en paralelo** (que es lo que hacen los reales) y los objetos vienen de **varios servidores**, los handshakes pasan todos al mismo tiempo: el no persistente baja a **4 RTT** y el persistente queda en **4 RTT**, o sea, la ventaja **desaparece**.

Lo que esta clase 9 agrega: la misma pregunta, versión por versión.

```
═══ El mismo caso, en HTTP/2 ═══
1 conexión TCP (11 streams simultáneos)        1 RTT
GET HTML + 10 GETs en paralelo                 1 RTT
──────────────────────────────────────────────────
Total:                                           2 RTT
Contra los 22 RTT del no persistente y los 12 del persistente.
La pérdida de un stream frena SOLO a ese stream.

═══ El mismo caso, en HTTP/3 ═══
1 conexión QUIC (con TLS integrado)            1 RTT
GET HTML + 10 GETs en paralelo                 1 RTT
──────────────────────────────────────────────────
Total:                                           2 RTT
Pero el handshake es 1 RTT en vez de 2 (TCP + TLS), y la pérdida de un
stream no frena a los otros: en HTTP/1.1 una sola pérdida de paquete en
una imagen congelaba la página entera durante 1 RTO.

═══ Y con HTTPS, que es el caso real ═══
HTTP/1.1 no persistente + HTTPS:  11 × (1 TCP + 1 TLS + 1 HTTP) = 33 RTT
HTTP/1.1 persistente  + HTTPS:     1 + 1 + 11                    = 13 RTT
HTTP/2              + TLS:         1 + 1                         =  2 RTT
HTTP/3              + QUIC:        1                             =  1 RTT
```

Interpretación: el objetivo de HTTP/2 y HTTP/3 **es exactamente este problema de examen**. Los dos nacen para matar el número de viajes que el enunciado cuenta, y la lección es que **el problema no se resolvió cambiando la forma de pedir**: se resolvió cambiando la forma de **transportar** varias peticiones a la vez.

> **Nota contra la oficial:** la oficial dice «puede resultar más performante», que es la respuesta honesta y matizada, y agrega el mejor argumento del apartado b): **«si el servidor nos limita por IP»**, o sea que el navegador no puede paralelizar porque el servidor impone un límite por IP. Yo repito ese argumento y agrego las tres precisiones que la oficial omite: el costo en RTT del paso 1, el `TIME-WAIT` del paso 4 y el `ETag` / `Cache-Control` del paso 3.

3.2 Examen 2009-07, Problema 2 — HTTP como protocolo de consulta

> **Fuente:** Examen `eirc0907` / `sirc0907` — 29 de julio de 2009, Problema 2, apartado b). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0907.pdf **Nota:** la resolución completa del problema está en la **Clase 8 §3.2**. Acá se desarrolla el ángulo de **cómo se escribe un mensaje HTTP/1.1 válido**.

Enunciado (apartado b)

> Implementar `bool isSpam(string ipAddress, string subject)` que devuelve `true` si **naivespamming** considera que el correo es spam.

Solución oficial (del PDF)

```
Socket s = new Socket("www.naivespamming.org", 80);
if (!s.connect()) return false;
s.out.write(
  "GET isspam.cgi?sender=" + ipAddress +
  "&subject=" + queryStringStuff(subject) + " HTTP/1.1\r\n\r\n");
string response = s.in.readLine();
s.close();
string[] result = response.split();
return result[1] == "200";
```

Lo que hay que corregir, mensaje HTTP adentro

El mensaje que la oficial escribe **no es un mensaje HTTP/1.1 válido**:

❌ Lo que escribe la oficial

```
GET isspam.cgi?... HTTP/1.1
\r\n\r\n
```

✅ Lo que debería escribir

```
GET /isspam.cgi?... HTTP/1.1
 Host: www.naivespamming.org
User-Agent: FING-SpamFilter/1.0
Accept: text/plain
Connection: close
\r\n\r\n
```

Defecto	Por qué importa
`Host` ausente	Es **obligatoria** en HTTP/1.1. Sin ella, un servidor con *virtual hosting* no sabe a qué sitio le estás pidiendo, y muchos responden `400` o sirven el sitio por defecto.
Path relativo (`isspam.cgi`)	El `request-target` de HTTP/1.1 tiene que ser un **absolute path** que empiece por `/` .
`\r\n\r\n` sí está bien	Ese es el `CRLF` final que cierra las cabeceras. Es lo único que la oficial acertó.
Sin `Accept`	El servidor no sabe qué formato querés y puede devolver HTML en vez de texto.
Sin `Connection: close`	Como no pidió `Content-Length` ni pidió cierre, el cliente queda esperando bytes que no llegan.

GET contra POST: la discusión que el enunciado no pide pero suma

La oficial usa `GET` . ¿Corresponde? Buscar en una base de datos es una **lectura** (`GET` está bien); registrar que un remitente hizo spam es una **escritura** (`POST`). El enunciado dice «considera que el correo es spam», o sea que es una consulta, así que `GET` es defendible. **PERO** el parámetro `subject` puede llevar información sensible: mandarlo en la URL lo deja en el historial del navegador, en los logs del servidor y en los proxies. Con `POST` iría en el cuerpo.

```
=== Con GET ===
GET /isspam.cgi?sender=203.0.113.7&subject=Venta%20de%20autos HTTP/1.1
Host: www.naivespamming.org
User-Agent: FING-SpamFilter/1.0
Accept: text/plain
Connection: close

=== Con POST ===
POST /isspam.cgi HTTP/1.1
Host: www.naivespamming.org
User-Agent: FING-SpamFilter/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 46
Accept: text/plain
Connection: close

sender=203.0.113.7&subject=Venta%20de%20autos
```

Y el parseo, que la oficial hace mal: `response.split()` toma el primer espacio y asume que `result[1]` es el código, lo que se rompe con una redirección. Lo robusto es **parsear la primera línea** (`HTTP/<ver> <código> <frase>`), seguir la redirección si la hay, y recién ahí interpretar el `2xx` o el `4xx` .

La corrección semántica del `return`

El enunciado dice que **404 = es spam** y **200 = es legítimo**. La oficial devuelve `true` con el `200`: eso **invierte la semántica** y marca como spam un correo que el servidor dijo que es limpio.

```
código == 200  →  es legítimo  →  return false
código == 404  →  es spam      →  return true
otro / error   →  no se sabe   →  return false   (fail-open)
```

> **Nota explícita:** la solución oficial contradice al enunciado. En un examen, **respetá el enunciado** y **señalá la contradicción por escrito**. Eso suma puntos: demuestra que leíste los dos documentos.

Lo que este problema aporta a la clase: un `4xx` no es un error técnico, es **información de negocio** para la aplicación que consume HTTP. **HTTP no solo transporta contenido: transporta semántica.**

3.3 Examen 2008-12, Pregunta 4 (7 puntos) — el mismo patrón de diseño

> **Fuente:** Examen `eirc0812` / `sirc0812` — 20 de diciembre de 2008. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0812.pdf

Este problema se resuelve **entero en la Clase 13**, junto con el correo. Acá lo dejo anotado porque es el **mismo patrón de diseño** que usa `Transfer-Encoding: chunked`, y entenderlo dos veces lo hace familia:

```
POP3:   el fin de mensaje es  CRLF . CRLF
        si un punto aparece al principio de una línea del mensaje, se
        DUPLICA. Al recibir, se quita el punto extra.

HTTP:   el fin de un fragmento chunked es  0<CRLF>
        si un dato empieza con "0" seguido de "<CRLF>", hay que
        escaparlo, porque si no el cliente lo toma como fin de cuerpo.
```

La idea común, que es la que hay que saber decir: cuando **el delimitador vive dentro del mismo canal que los datos** (no hay longitud explícita), hace falta **escapar** el delimitador en el emisor y **desescapar** en el receptor. Es el mismo problema que el **byte stuffing** del PPP (`2003-03 P3`) y el **bit stuffing** de Ethernet. **Tres protocolos, tres objetos de relleno distintos: el punto en POP3 y SMTP, el byte de escape en PPP, el bit en Ethernet.** La razón es siempre la misma: **ninguno de esos caracteres puede aparecer legítimamente donde se lo proscribe.**

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. Un **método** declara la intención. **Seguro** = no cambia el estado, **idempotente** = repetirlo *n* veces es lo mismo que una. La distinción decide si el cliente puede **reintentar solo**.
2. `GET` / `HEAD` / `OPTIONS` / `TRACE` son seguros; `GET` / `HEAD` / `PUT` / `DELETE` son idempotentes; **`POST` no es ninguna de las dos**.
3. En `GET` los datos van **en la URL** (por eso quedan en el historial y los logs); en `POST` van **en el cuerpo** (por eso no).
4. `Host` es **obligatoria** en HTTP/1.1 y es lo que hace posible el *virtual hosting* y, por lo tanto, lo que impide esconder HTTP detrás de un NAT.
5. `Cache-Control`: `no-store` (no guardar), `no-cache` (guardar pero revalidar), `max-age=N` (fresco N segundos), `private` / `public` (quién puede cachear), y `Vary` (de qué depende la variación).
6. La validación condicional es `ETag` + `If-None-Match` (o `Last-Modified` + `If-Modified-Since`); si coincide, el servidor responde **`304` sin cuerpo y sin `Content-Length`** y **renueva la frescura** de la caché del cliente.
7. **Proxy** reenvía, **gateway** traduce protocolos, **CDN** copia el contenido cerca del usuario: el CDN mejora la latencia **reduciendo la distancia al usuario**, y como la carga de una página es `viajes × RTT`, eso la acelera.
8. **HTTP/2** (`RFC 9113`) multiplexa streams sobre **una conexión TCP** y elimina el HOL blocking **de conexión** (no entre streams); **HTTP/3** (`RFC 9114` sobre **QUIC**, `RFC 9000`) **reemplaza TCP por completo** y elimina el HOL blocking **entre streams**. Se negocia con **`Alt-Svc`**, no con `Upgrade`.
9. Las cifras reales: **HTTP/2 50 %, HTTP/1.x 29 %, HTTP/3 21 %** (Cloudflare Radar 2025). **HTTP/3 no se adueñó de la web**, y la razón es de despliegue, no técnica.
10. Lo que **no cambia nunca**: petición/respuesta, sin estado, los métodos, las clases 1xx–5xx, `Host` obligatoria (que pasa a `:authority` en HTTP/2 y 3), persistente por defecto, `304` y `Range`.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un cliente hace `POST /pagar` y no recibe respuesta. ¿Puede reintentar automáticamente? ¿Por qué sí o por qué no?

▶ Respuesta

2. ¿Qué diferencia hay entre `Cache-Control: no-cache`, `max-age=0` y `no-store`?

▶ Respuesta

3. Escribí la respuesta a una petición que llega con `If-None-Match: "5d9b21-30d40-6518a1f0"` y el servidor tiene ese mismo `ETag`. ¿Qué cabeceras **no** puede llevar?

▶ Respuesta

4. Un `GET` con `Cache-Control: max-age=0` y un `ETag` presente. ¿Cuánto se ahorra en cada revalidación y cuánto **no** se ahorra?

▶ Respuesta

5. ¿Por qué un CDN mejora la latencia? Explicá sin usar la palabra «caché».

▶ Respuesta

6. ¿Qué gana y qué pierde HTTP/3 frente a HTTP/1.1? Nombrá al menos dos de cada.

▶ Respuesta

7. ¿HTTP/2 elimina el head-of-line blocking? La respuesta precisa, porque «sí» y «no» están las dos.

▶ Respuesta

8. ¿Por qué HTTP/3 no se adueñó de la web si es técnicamente mejor? ¿Es un problema técnico?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1007` / `sirc1007`** — Pregunta 1 (8 pts): página HTML con texto y 10 imágenes, HTTP no persistente vs. persistente
- **Examen `eirc0907` / `sirc0907`** — Problema 2, apartado b): `isSpam()` sobre HTTP, construcción del mensaje y parseo del código
- **Examen `eirc0812` / `sirc0812`** — Pregunta 4 (7 pts): el `CRLF.CRLF` de POP3 (desarrollada en la Clase 13)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.3 («El protocolo de la Web: HTTP»), §7.3.5 (caché) y §7.3.6 (CDN, *content delivery networks*)
- **RFC 9110** — HTTP Semantics (2022, *Internet Standard*; sustituye a RFC 2616) · **RFC 9112** — HTTP/1.1 (2022, sintaxis)
- **RFC 9113** — HTTP/2 (2022; obsoleta RFC 7540) · **RFC 9114** — HTTP/3 (2022)
- **RFC 7541** — HPACK, compresión de cabeceras de HTTP/2 (vigente) · **RFC 9204** — QPACK, compresión de HTTP/3 (independiente del orden)
- **RFC 9218** — *Extensible Prioritization Scheme* (EPS): reemplaza la prioridad deprecada de RFC 7540
- **RFC 9000** / **9001** / **9002** — QUIC: transporte, TLS sobre QUIC, recuperación y control de congestión (mayo-2021) · **RFC 9369** — QUIC v2
- **RFC 7838** — *Alternative Services* (`Alt-Svc`): cómo se anuncia HTTP/3 (2016)
- **RFC 6265** — HTTP State Management Mechanism (cookies) · **RFC 9460** — SVCB/HTTPS: política de transporte en el DNS
- **Cloudflare Radar**, *Year in Review 2025* — HTTP/2 50 %, HTTP/1.x 29 %, HTTP/3 21 % · **W3Techs** (oct-2026) — ~40,8 % de los sitios con HTTP/3
- **Clase siguiente:** 10 — DNS: la base de datos distribuida

Clase 10

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 El problema que resuelve el DNS
- 1.2 El DNS son dos cosas a la vez
- 1.3 Los tipos de servidores, con su rol exacto
- 1.4 Delegación: `NS` y `glue records`
- 1.5 El Resource Record: la unidad de información
- 1.6 Los tipos de Resource Record que hay que saber
- 1.7 ⚠️ Actualización a 2026 — qué cambió en el DNS desde 2011
- 2.1 Ejercicio 1 — Escribir un Resource Record completo
- 2.2 Ejercicio 2 — Distinguir `CNAME` de `MX` con ejemplos
- 2.3 Ejercicio 3 — ¿Por qué un `CNAME` no puede existir en la raíz?
- 3.1 ⭐ Examen 2010-02, Pregunta 1a (8 puntos) — qué es el DNS y su función
- 3.2 Examen 2010-02, Pregunta 1b (8 puntos) — iterativa vs. recursiva
- 3.3 Examen 2007-07, Pregunta 4 (8 puntos) — función del DNS y protocolo involucrado
- 📌 Resumen en 10 líneas
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: B (Aplicación) · **Semana:** 4 · **Duración estimada:** 110 min **Correlativa:** Clase 9 (HTTP a fondo) · **Sigue con:** Clase 11 (DNS: iterativo vs. recursivo, caché y TTL) **Material de referencia:** Tanenbaum cap. 7 (§7.4) · RFC 1034, RFC 1035, RFC 4033-4035, RFC 5155, RFC 6891, RFC 8659, RFC 9460

🎯 Al terminar esta clase vas a poder

1. Explicar **por qué existe** el DNS y por qué no puede resolverse con un archivo central ni con una tabla plana en cada host.
2. Explicar por qué el DNS es **dos cosas a la vez** (espacio de nombres jerárquico y base de datos distribuida) y por qué el espacio se resuelve **de derecha a izquierda**.
3. Dibujar la **jerarquía** y decir **qué sabe cada tipo de servidor**: el root, el TLD, el autoritativo y el local.
4. Explicar cómo funciona la **delegación** con registros `NS` y `glue`, y por qué existe la **paradoja del bootstrap**.
5. Escribir un **Resource Record** completo con sus **cinco campos**, y decir qué tamaño tiene cada uno.
6. Distinguir los **tipos de RR** que hay que saber (`A` , `AAAA` , `NS` , `CNAME` , `MX` , `SOA` , `PTR` , `TXT` , `SRV` , `CAA`) diciendo qué resuelve cada uno.
7. Resolver completo el **Examen 2010-02 Pregunta 1a** (qué es DNS y su función, 8 puntos).

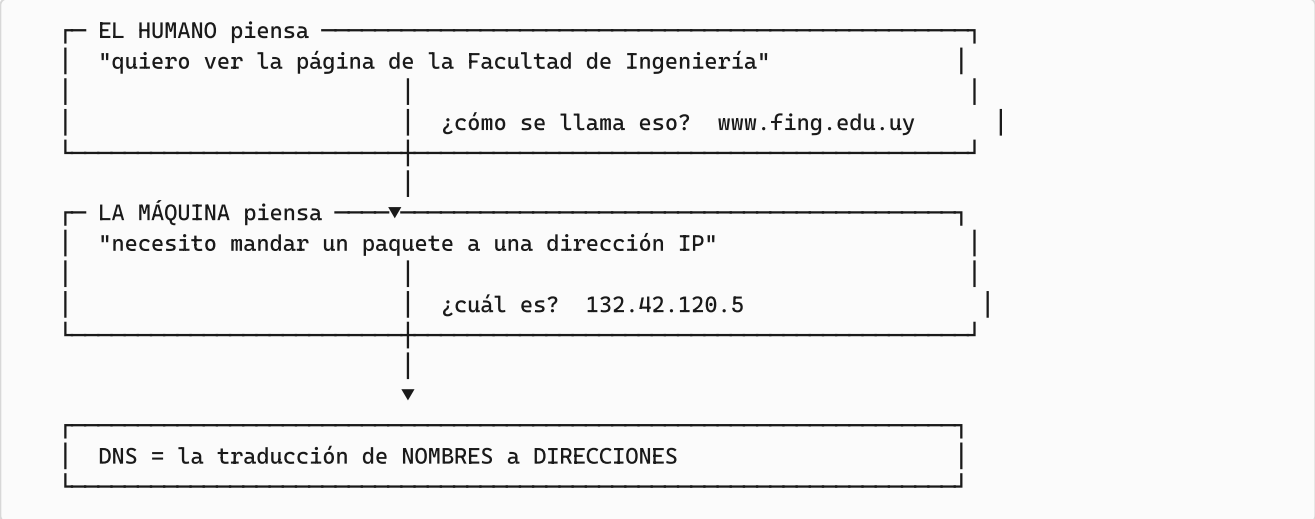
> **Prerequisito del examen:** `2010-02 P1` (8 puntos — ★ qué es el DNS y su función), `2010-12 P1a` (8 puntos — el RR y sus cinco campos, que es el otro punto de esta clase), `2007-07 P4` (función del DNS y protocolo involucrado).

PARTE 1 — TEORÍA

1.1 El problema que resuelve el DNS

Los humanos no memoramos direcciones IP. No es que no podamos: es que **una dirección IP no significa nada para una persona**. `132.42.120.5` no te dice quién es, no te dice qué organización lo administra, no te dice si el servidor está encendido, y no te sirve de nada si mañana lo mueven de datacenter. En cambio `www.fing.edu.uy` te dice todo eso de un vistazo.

El DNS es la capa que hace esa traducción:



La objeción obvia: ¿no podría ser un archivo de texto con todas las correspondencias, copiado a cada máquina? No, y hay **tres razones independientes** que lo impiden. Cada una basta sola.

Razón 1 — La escala: la tabla no entra

Un archivo central tendría que contener **una entrada por cada host de Internet**:

```
En 2011 (el curso):   ~2.000 millones de hosts
En 2026:              ~5.000 millones de hosts      (verificar en vivo)

Cada entrada necesita, mínimo:
  nombre (promedio 15 bytes) + IP (4 bytes) + tipo (2) + TTL (4)
  ≈ 25 bytes mínimo, y en la práctica 60-80 con las líneas completas

5.000 millones × 60 bytes = 300.000.000 bytes = 300 MB POR CADA MÁQUINA
```

Y encima hay que **actualizarlo**: si un host aparece, el archivo tiene que volver a distribuirse. Un archivo de 300 MB que se actualiza constantemente es imposible de mantener.

Razón 2 — El punto único de falla

```
┌─────────────────┐
│  ARCHIVO        │◄── todos los hosts del mundo lo consultan
│  CENTRAL        │
└─────────────────┘
        │
  si se cae  ──►  TODO Internet deja de resolver nombres

  si alguien lo compromete ──►  puede REDIRIGIR cualquier nombre
                                a cualquier IP  (phishing perfecto)
```

Un archivo central es un **punto único de falla** (si cae, cae todo) y un **punto único de compromiso** (si lo hackean, controlás la resolución de todo Internet). Las dos cosas son inaceptables.

Razón 3 — El mantenimiento

Hay que responder **quién** autorizó a registrar cada nombre. Un archivo central no tiene forma de saber si `fing.edu.uy` le pertenece a la Universidad o a un impostor.

La solución del DNS ataca las tres:

Problema	Solución del DNS
Escala (300 MB por host)	**distribución**: cada servidor guarda solo la porción que le toca, y los nombres están **particionados** por dominio
Punto único de falla	**replicación y jerarquía**: los servidores root están replicados; cada zona tiene varios servidores autoritativos
Mantenimiento	**delegación**: cada zona delega la zona siguiente a una autoridad identificada, y esa autoridad es la que valida

1.2 El DNS son dos cosas a la vez

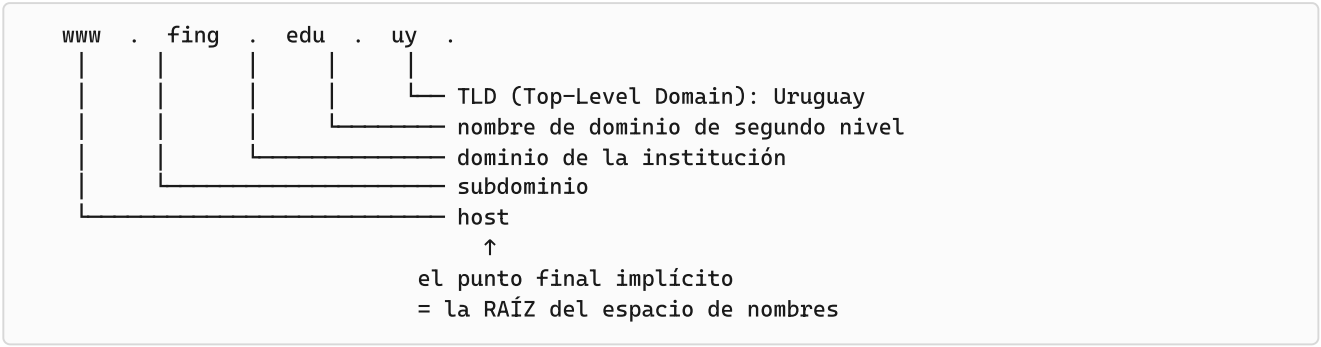
Esta es la idea central de la clase y lo que el examen 2010-02 P1 pide exactamente:

```
╔═══════════════════════════════════════════════════════════╗
║  EL DNS ES DOS COSAS A LA VEZ:                            ║
║                                                           ║
║  (a) UN ESPACIO DE NOMBRES JERÁRQUICO                     ║
║      → la estructura de los NOMBRES: cómo se nombran las cosas ║
║      y cómo se reparte la autoridad sobre ellos           ║
║                                                           ║
║  (b) UNA BASE DE DATOS DISTRIBUIDA                        ║
║      → el contenido: qué nombre corresponde a qué dirección ║
║      y cómo se consulta esa correspondencia               ║
╚═══════════════════════════════════════════════════════════╝
```

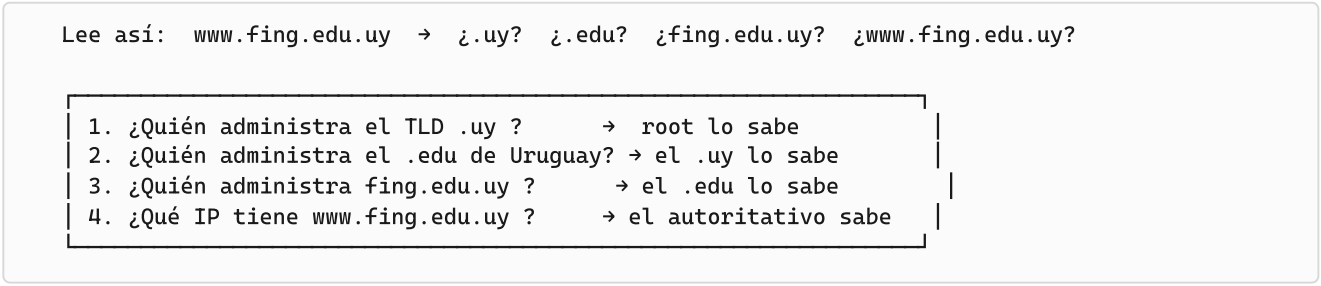
Y son dos cosas distintas con dos respuestas distintas. La primera es la que te dice **a quién preguntarle**; la segunda es la que te dice **qué preguntar**. El `2010-02 P1` lo dice literal en su solución oficial: "*protocolo de capa de aplicación que permite realizar la traducción de nombres de hosts a direcciones IP. También se denomina DNS a la base de datos jerárquica distribuida que almacena los mapeos entre nombres y direcciones IP*".

(a) El espacio de nombres: jerárquico y de derecha a izquierda

El nombre completo de un host es una **etiqueta jerárquica separada por puntos**, con un punto implícito al final:



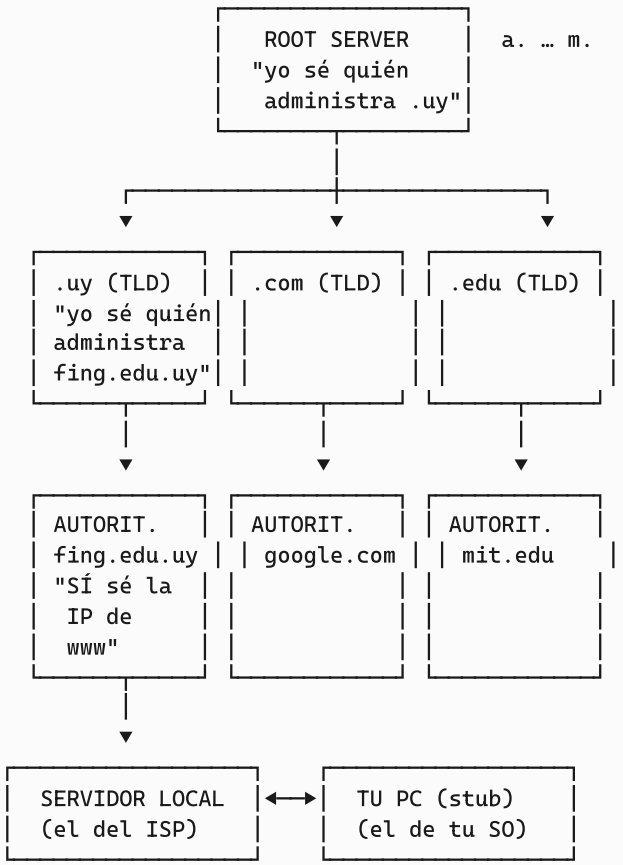
Se resuelve de derecha a izquierda, y esto es contraintuitivo para un lector de español, donde leemos de izquierda a derecha. La razón es histórica y es la estructura de la autoridad:



Por qué de derecha a izquierda y no al revés: la **autoridad va de arriba hacia abajo**. El root sabe qué TLDs existen; el TLD sabe qué dominios de segundo nivel hay debajo; el autoritativo del dominio sabe qué hosts hay. Cada nivel **solo conoce un nivel hacia abajo**, y por eso el árbol tiene que recorrerse desde la raíz, que es el único que sabe dónde empieza todo. Si el nombre se recorriera de izquierda a derecha, el primer nivel («qué hay bajo `www`?») no lo sabe nadie: `www` no es una zona.

Y `uy` a la derecha de `edu` significa que `edu` es de Uruguay, no el `edu` de Estados Unidos. Los TLD countries están estructurados así: `fing.edu.uy` (Uruguay) y `mit.edu` (Estados Unidos) son dos ramas distintas de `.edu`. Esta estructura es la que hace posible la **delegación por país sin cambiar la infraestructura global**.

1.3 Los tipos de servidores, con su rol exacto



Root server: 13 identidades lógicas, replicadas por anycast

Dato	Detalle
Cuántos hay	**13 identidades lógicas**, nombradas de `a` a `m`: `a.root-servers.net` … `m.root-servers.net`
Cuántas máquinas	**cientos**: cada identidad se opera en **cualquiercast**, así que hay una IP pública igual en todo el mundo
Cuántas IPv4 por identidad	**cuatro** direcciones IPv4 por identidad, por *anycast*
Operación	los root servers los financian los operadores de la red (los *NIC*), con cable submarino; no hay un actor comercial detrás
Qué SÍ saben	la lista de los **TLD** y cuál es la dirección IP de cada servidor TLD
Qué NO saben	**no conocen los dominios**. Jamás supieron qué es `fing.edu.uy`.

> **El punto que más se confunde y más se pregunta:** el root **no resuelve nombres de dominio**. Solo tiene una tabla plana y pequeña de unos 1500 TLDs. Si le preguntás por `www.fing.edu.uy`, te contesta «no sé, pero el `.uy` está en `X.Y.W.Z`».

Por qué 13. El número original (1987) fue un capricho del DARPA, y hoy está congelado porque el *root zone* está firmado digitalmente: hay que coordinarse con los operadores de los 13 para cambiar **cualquier** registro de la zona raíz. **El número es un artefacto histórico, no una decisión de diseño**, y es un buen ejemplo de *protocolos que no se pueden cambiar*.

TLD server: el `.uy`, el `.com`, el `.edu`

Dato	Detalle
Qué es	un dominio (`uy` , `com` , `edu` , `org` , `ar` , `io` ...)
Qué SÍ sabe	los **registros** `NS` de los dominios de segundo nivel bajo él. Del TLD `.uy` sabe quién administra `fing.edu.uy` .
Qué NO sabe	**tampoco conoce los dominios**, ni mucho menos sus IPs. Jamás supo qué es `www.fing.edu.uy` .
Quién lo opera	los *registries*: NIC Uruguay para `.uy` , Verisign para `.com` , ICANN para `xxx` .
Cuántos	cientos por TLD, y casi todos los **gTLD** son «genéricos» (`.com` , `.org` , `.info` ...), o sea los que no son de un país

Servidor autoritativo: el que sí sabe

Es el **dueño de la información**. Preguntale a `ns1.fing.edu.uy` y te contesta la IP de `www.fing.edu.uy` con total seguridad.

Dato	Detalle
Qué es	el servidor del dominio: `ns1.fing.edu.uy` , `ns2.fing.edu.uy`
Qué sabe	**TODOS los RRs de su zona**: A, AAAA, MX, TXT, CNAME, SRV...
Es la fuente de verdad	sí. Si el autoritativo dice `X` , `X` es. Todos los demás son copias.
Cuántos hay	mínimo 2 por zona, si posible en **redes distintas** y en **ciudades distintas**
Quién lo pone	el administrador del dominio

Servidor de nombres local: recursivo, y el stub resolver del host

Acá está el punto donde la cátedra más hace líos, así que hay que separar **dos cosas muy distintas** que a veces se confunden:

	Stub resolver (el de tu PC)	**Servidor de nombres local / recursivo** (el del ISP)
Qué es	una biblioteca diminuta dentro del **sistema operativo**	un programa `named` o `dnsmasq` en el servidor del ISP o del router
Dónde está	en **tu PC**	en la **red**: la red de tu ISP, o el router de tu casa
Qué sabe	**nada**. No mantiene caché propia (aunque Windows sí guarda un archivo local, `dnsapi.dll`).	**mantiene caché** de todo lo que resolvió, con TTL
Qué hace	agarra el nombre y se lo **manda al recursivo**	**pregunta a root, TLD y autoritativo** y **cachea**
Consume CPU del host	casi nada	todo el trabajo

> **Mnemotecnia:** *stub* significa «trozo», «pieza mínima». *Resolver* es «resolver», el que resuelve. El stub **no resuelve**: **pregunta**. El recursivo **resuelve**: **pregunta por vos**.

1.4 Delegación: `NS` y `glue records`

La delegación es el mecanismo por el que **el root delega `.uy`, y `.uy` delega `fing.edu.uy`**. No es un aviso verbal: es un **registro** en la base de datos del nivel superior.

El mecanismo: un registro `NS` en el nivel superior

```
En la zona del TLD .uy hay:

$ORIGIN uy.
uy.         3600  IN  NS  a.nic.uy.
uy.         3600  IN  NS  b.nic.uy.

En la zona de fing.edu.uy hay:

$ORIGIN fing.edu.uy.
fing.edu.uy.     3600  IN  NS  ns1.fing.edu.uy.
fing.edu.uy.     3600  IN  NS  ns2.fing.edu.uy.
www        A     3600  IN  A   132.42.120.5
```

Cuando le preguntás al root por `www.fing.edu.uy`, el root no tiene el `A`, pero **sí tiene el `NS` de `uy`**. Te lo pasa como **referencia** («buscale al `.uy`») y con eso te ahorró el trabajo de preguntar.

La paradoja del bootstrap: por qué existen los *glue records*

Y acá aparece el problema elegante que hay que explicar bien:

```
El .uy te dice:  "el autoritativo de fing.edu.uy es ns1.fing.edu.uy"
↑
¡PERO para preguntarle a ns1.fing.edu.uy
  NECESITO SU DIRECCIÓN IP!
Y esa IP es... ¡otro nombre que hay que resolver!
```

Esto es un círculo: para resolver un nombre (el del servidor autoritativo) hay que resolver un nombre (el que te dijeron). La solución se llama **bootstrap**, y el truco es guardarse el resultado en el mismo lugar de donde lo obtuviste.

```
En la zona del TLD .uy, ADEMÁS del NS, están los GLUE RECORDS:

$ORIGIN uy.
uy.        3600  IN  NS  a.nic.uy.
...
ns1.fing.edu.uy.  3600  IN  A   132.42.60.1    ⟵ GLUE
ns2.fing.edu.uy.  3600  IN  A   132.42.60.2    ⟵ GLUE
```

El `glue record` es un registro `A` (o `AAAA`) con el nombre del servidor de nombres, guardado en el servidor del nivel superior. Y es **redundante**, y esa redundancia es intencional: el mismo registro `A` para `ns1.fing.edu.uy` existe **también** en la zona `fing.edu.uy`, que es donde le corresponde. El problema es que **para llegar a esa zona hay que pasar por el glue**, así que el glue tiene que estar en el nivel superior.

> **La regla que hay que memorizar:** el *glue* es **obligatorio** solo para los nombres de servidor que están **dentro de la zona delegada**. Para un servidor de nombres **fuera** del dominio (`ns1.otroproveedor.com`), **no hay glue**, porque su nombre se resuelve por el camino normal, con otro nivel de por medio.

Los problemas operativos del glue, que son los que aparecen en la práctica:

1. **Desincronización.** El `glue` dice `132.42.60.1`, pero el autoritativo cambió a `132.42.60.9`. El glue está **desactualizado** y toda resolución de `fing.edu.uy` falla hasta que el TTL del glue venza. Es la causa número uno de «mi dominio dejó de funcionar después de un cambio».
2. **Servidores que se caen.** Si `ns1.fing.edu.uy` baja, todo el dominio baja: los clientes prueban `ns2`, pero la resolución del *nombre* `ns2.fing.edu.uy` necesita el glue, que tiene el mismo TTL.
3. **La respuesta mínima.** Algunos resolvers de producción devuelven solo los registros **de la zona que se consultó** y **no** los *glue*, obligando a un segundo viaje.

1.5 El Resource Record: la unidad de información

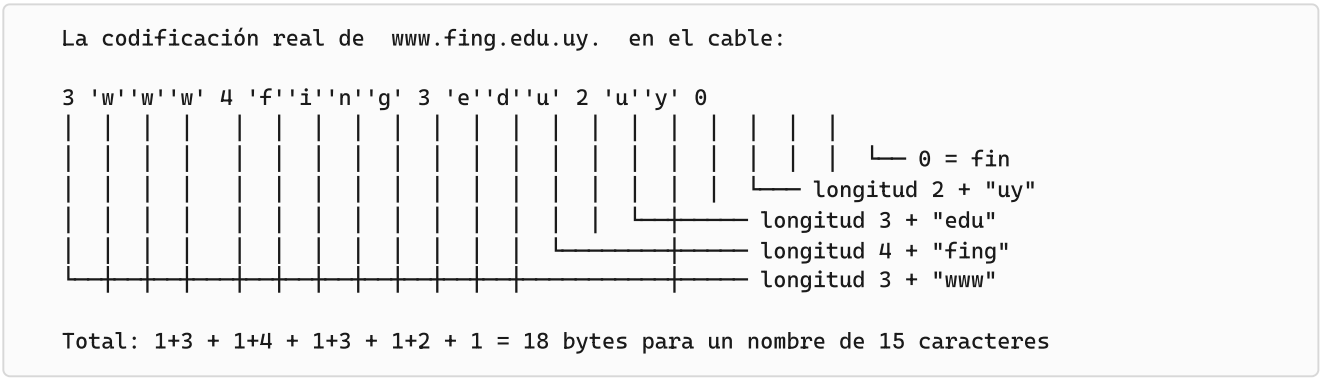
Todo lo que el DNS sabe son **Resource Records** (*RR*, registros de recurso). Un RR es una **línea** de la base de datos, y tiene **cinco campos**:

```
nombre              clase   tipo    TTL      datos
fing.edu.uy.        IN      A       3600     132.42.120.5
----------------    -----   ----    -----    ------------------
nombre             2 bytes  2 bytes 4 bytes  variable
```

#	Campo	Tamaño	Qué es	Ejemplo
1	**Nombre** (*name*)	1 byte por etiqueta + 1 byte de longitud	el nombre del recurso, **en forma absoluta**: siempre termina en punto	`fing.edu.uy.`
2	**Clase** (*class*)	2 bytes	casi siempre `IN` (*Internet*). Existen `CH` (CHAOS, para el `version.bind`) y `HS` (obsoleto)	`IN`
3	**Tipo** (*type*)	2 bytes	qué clase de dato es	`A`
4	**TTL** (*time to live*)	4 bytes	segundos que el registro se puede cachear	`3600`
5	**Datos** (*rdata*)	**variable**	el valor, y **su formato depende del tipo**	`132.42.120.5`

Tres cosas que hay que entender de los nombres, porque son las trampas del examen:

1. **El punto final es obligatorio** en la forma canónica. `fing.edu.uy` y `fing.edu.uy.` son **el mismo nombre**: el último punto es implícito.
2. **Las etiquetas se cuentan de derecha a izquierda.** El nombre `fing.edu.uy` tiene **3 etiquetas**; `www.fing.edu.uy` tiene **4**.
3. **Un nombre de longitud total mayor a 255 bytes no existe.** El nombre se codifica como una secuencia de etiquetas de hasta 63 bytes cada una, precedidas por un byte de longitud: `7fing3edu2uy0` son 1+5+1+3+1+2+1 = 14 bytes. Esa codificación es la razón por la que **los nombres legibles son solo una convención de presentación**: en el cable viajan como longitud + caracteres.



Y el `2010-12 P1a` es exactamente esto, que es el otro problema central de esta clase:

> **Examen 2010-12, Pregunta 1a:** *«Cuando un servidor de nombres aprende un mapeo, lo cachea. ¿Cuál es el campo del registro de recurso (RR) que decide si se puede cachear o no? ¿Cuál es el servidor que define ese tiempo? ¿El valor que toma ese campo, a qué refiere y en qué unidades está expresado?»*

Se resuelve **entero** en la Clase 11 (que desarrolla caché y TTL), pero el **campo** ya está definido acá: es el **cuarto campo**, el **TTL**, de **4 bytes**, y su unidad son **segundos**. Lo que se responde completo en la Parte 3 de esta clase.

1.6 Los tipos de Resource Record que hay que saber

Tipo	Qué resuelve	Ejemplo	Notas y trampas
`A`	nombre → **IPv4**	`www.fing.edu.uy. IN A 132.42.120.5`	4 bytes de datos. Varios `A` = **balanceo por round-robin** (el orden lo elige el cliente; Chrome y Safari lo aleatorizan).
`AAAA`	nombre → **IPv6**	`www.fing.edu.uy. IN AAAA 2001:7fd::1`	16 bytes. El *happy eyeballs* es un cliente que consulta `A` y `AAAA` **en paralelo** y se queda con la que llegue primero.
`NS`	nombre → **servidor de nombres** autoritativo	`fing.edu.uy. IN NS ns1.fing.edu.uy.`	El `NS` **no contiene la IP**: solo el nombre del servidor. La IP viene en el glue. **Es lo que hace funcionar la delegación.**
`CNAME`	**alias**: nombre → **otro nombre**	`www IN CNAME web-server01.uy.`	Solo puede haber **uno por nombre**. No puede existir en la raíz. **No puede coexistir con otros tipos en el mismo nombre.**
`MX`	nombre → **servidor de correo**, con **preferencia**	`fing.edu.uy. IN MX 10 mail.fing.edu.uy.`	El número (aquí `10`) es la **preferencia**: menor = más preferido. El que tiene menor MX es al que se le manda primero. Si cae, se prueba el siguiente.
`SOA`	**Start of Authority**: datos de **autoridad de la zona**	`fing.edu.uy. IN SOA ns1.fing.edu.uy. admin.fing.edu.uy. 2026100601 7200 3600 1209600 3600`	Lleva **MNAME**, **RNAME**, **serial**, **refresh**, **retry**, **expire** y **minimum**. Es el registro de **control**: los servidores secundarios lo usan para decidir cuándo resincronizar.
`PTR`	**IP → nombre**, la inversa	`5.120.42.132.in-addr.arpa. IN PTR www.fing.edu.uy.`	Se llama resolución **inversa** y vive en `.in-addr.arpa` (IPv4) o `.ip6.arpa` (IPv6). **El nombre es al revés.**
`TXT`	texto libre arbitrario	`fing.edu.uy. IN TXT "v=spf1 include:_spf.fing.edu.uy -all"`	Se usa para **SPF** (qué servidores pueden enviar correo en tu nombre) y para **verificación de propiedad** de dominios ante proveedores de correo.
`SRV`	nombre → **servicio con sus parámetros de puerto y prioridad**	`_sip._tcp.fing.edu.uy. IN SRV 10 20 5060 sip.fing.edu.uy.`	Formato: `prioridad peso puerto objetivo`. Se inventó para **SIP** y hoy se usa en XMPP, Matrix, FTP. No existe en RFC 1034: es una extensión.
`CAA`	**qué Autoridades de Certificación** pueden emitir para el dominio	`fing.edu.uy. IN CAA 0 issue "letsencrypt.org"`	`RFC 8659` (2019), reemplaza a `RFC 6844`. Evita que una CA emita un certificado para tu dominio sin tu permiso.

Los tres que se confunden siempre

`CNAME` vs. `MX` vs. `SOA` — son tres respuestas a tres preguntas distintas:

```
¿A qué IP va un cliente que abre http://www.fing.edu.uy ?
   ──► CNAME (si existe) dice:  "ese nombre es otro nombre"
   ──► A       dice:          "esa IP es esta"
   ──► SOA    NO se usa para esto.

¿A qué servidor mando el correo a admin@fing.edu.uy ?
   ──► MX  dice:  "preferencia 10, servidor mail.fing.edu.uy"
   ──► y después, para resolver mail.fing.edu.uy → IP, se usa CNAME o A.

¿Quién manda aquí y cada cuánto sincronizo ?
   ──► SOA dice:  "el master es ns1, el serial es 2026100601"
```

El `SOA` en detalle, porque tiene que saberse de memoria:

```
   fing.edu.uy.  IN  SOA  ns1.fing.edu.uy.  admin.fing.edu.uy.  (
                                 |            |            |
                                 |            |            └─ 1: SERIAL
                                 |            |               (el "número de versión"
                                 |            |                de la zona)
                                 |            └─ 2: RNAME
                                 |               (el email del administrador,
                                 |                con el @ como punto)
                                 └─ 1: MNAME
                                    (el servidor de nombres master)

                2026100601    7200    3600    1209600    3600
                     |          |       |         |         |
                     |          |       |         |         └── 7: MINIMUM
                     |          |       |         |             (TTL por defecto de los
                     |          |       |         |              NXDOMAIN negativos)
                     |          |       |         └── 6: EXPIRE
                     |          |       |            (días antes de dejar de
                     |          |       |             intentar más refreshing)
                     |          |       └── 5: RETRY
                     |          |          (minutos entre reintentos de refresh
                     |          |           cuando falla)
                     |          └── 4: REFRESH
                     |             (minutos entre que un secundario
                     |              pregunta "¿hay cambios?")
                     └── SERIAL
   ```

Y la comparación `CNAME` vs. `MX`, que es pregunta clásica:

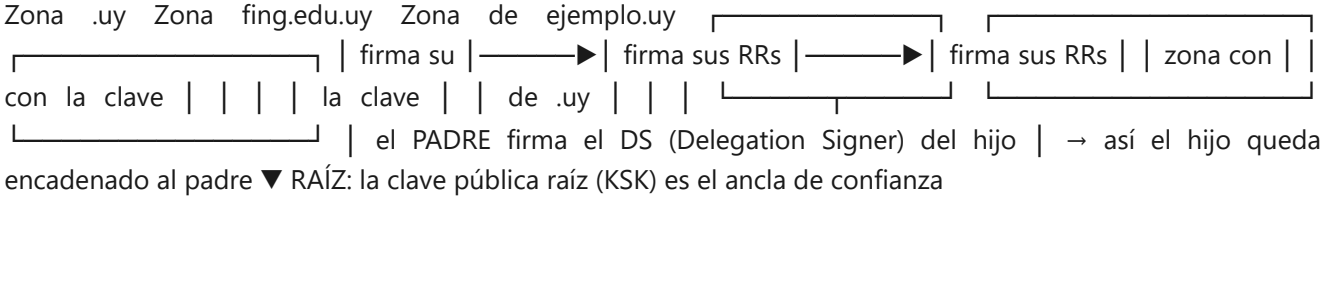
	`CNAME`	`MX`
Apunta a	otro **nombre**	un **servidor de correo**, con número de preferencia
Se usa para	mover un host sin romper los enlaces	elegir a qué servidor va el correo
¿Puede haber dos?	**no**: un CNAME por nombre, y no puede convivir con nada más	**sí**:
Ejemplo real	` intranet.fing.edu.uy → CNAME SharePoint.uy.`	` fing.edu.uy MX 10 smtp1.fi

1.7 ⚠️ Actualización a 2026 — qué cambió en el DNS desde 2011

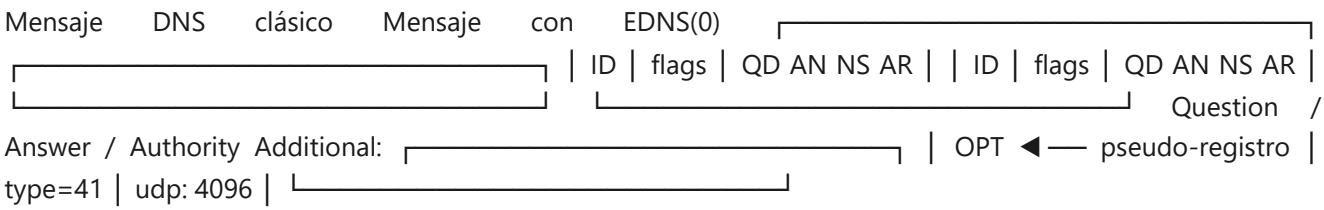
Esta clase describe el DNS **de 2011**. Lo de arriba —jerarquía, delegación, *glue*, los cinco campos del RR, la lista de tipos— **sigue exactamente igual**. Lo que cambió es lo que se le **agregó encima**, y son cuatro cosas.

(a) DNSSEC: la firma de la jerarquía (`RFC 4033-4035`, 2005)

El problema que resuelve. El DNS es, por diseño, **una base de datos abierta y confiable**: cualquiera puede registrar un dominio, y cualquiera que diga tener la delegación de un nombre es creído. Eso permite el **DNS hijacking** (secuestro de dominio) con solo lograr que el registrador o un servidor mal configurado apunten el nombre a una IP ajena. **DNSSEC** agrega criptografía: cada zona firma sus RRs con una clave privada, y la firma se valida contra una cadena que arranca en la **raíz**.



```
**El estado de despliegue, y acá está la lección de la clase:**

| Dato | Valor |
|---|---|
| Estandarizado | **2005** (`RFC 4033-4035`) |
| **gTLD firmados** | **~99,6 %** |
| **Dominios (2LD) con `DS`** | **~5–7 %** |
| Raíz firmada | desde **2010** (`RFC 5155`) |

> ⚠️ **La lección, y hay que decirla con esas palabras:** DNSSEC es el ejemplo
> perfecto de **estandarizado en 2005 y con despliegue bajo casi veinte años
> después**. La cadena criptográfica está construida y operativa, pero el último
> eslabón —el dominio de segundo nivel— casi no lo adoptó. **Por eso en 2026 no se
> puede enseñar que «el DNS es seguro»**: se puede enseñar que **el DNS tiene una
> capa de autenticidad opcional, bien diseñada y masivamente subutilizada**. El
> paralelo más cercano es HTTPS, que también lleva veinte años y tardó otras dos
> décadas en desplegarse del todo.

### (b) EDNS(0): el mecanismo que hace possible todo lo anterior (`RFC 6891`, 2013)

**El problema que resuelve.** El mensaje DNS clásico tiene **12 bytes** de cabecera
y cuatro contadores. Para extenderlo (DNSSEC, `CAA`, `SVCB`/`HTTPS`) no había
dónde: cualquier campo nuevo tenía que romper la estructura. **`EDNS(0)`** agrega un
**pseudo-registro `OPT`** en la sección *Additional* que lleva opciones de tamaño y
de extensión, **sin romper el formato antiguo**.
```




```
**Importa en el examen** porque **cambia el tamaño por defecto del datagrama**:
sin `EDNS(0)` el máximo sigue siendo **512 bytes**, y las respuestas grandes se
cortan y obligan a reintentar por TCP. Con `OPT` que anuncia `udp: 4096`, el
cliente y el servidor acordan hasta 4096 bytes **sin cambiar de transporte**. Es un
ejemplo de manual de **extensión compatible hacia atrás** (*backward compatible
extension*), la misma técnica de los campos opcionales de IPv4.

### (c) CAA: el dominio declara qué autoridad puede emitir certificados (`RFC 8659`, 2019)

Un `CAA` es un RR que dice **qué entidades certificadoras** pueden emitir un
certificado para ese dominio:
```

fing.edu.uy. IN CAA 0 issue "letsencrypt.org" fing.edu.uy. IN CAA 0 issuewild "letsencrypt.org" fing.edu.uy. IN CAA 0 iodef "mailto:seguridad@fing.edu.uy" | | | | | | | | └── a quién avisar si alguien viola la política | | | └──────── qué CA puede emitir para lo que sea | | └──────── "para este dominio y todos sus subdominios" | └──────── 0 = "el primero de la lista manda" └──────── la autoridad

```
⚠️ El curso de 2012 enseñaba **`RFC 6844`**, que quedó **obsoleta**: la vigente es
la `RFC 8659`. **Por qué importa:** sin `CAA`, cualquier autoridad de certificación
puede emitir un certificado válido para tu dominio y el navegador no tiene forma de
rechazarlo. Es uno de los **mecanismos de seguridad que se apoyan en un ancla
declarada en el DNS**, como `DNSSEC` y como `RPKI` para BGP.

### (d) El DNS pasó a transportar **política de transporte**: SVCB/HTTPS (`RFC 9460`, 2023)

Este es el cambio conceptual más grande **después** de DNSSEC, y conecta con la
Clase 9: el DNS **ya no transporta solamente direcciones**, sino **instrucciones
sobre cómo conectarse**.
```

ANTES: www.fing.edu.uy. A 132.42.120.5 "¿dónde está?" ↓ el cliente decide TODO: TCP o TLS, qué versión de HTTP

AHORA: www.fing.edu.uy. HTTPS SVCB ├─ alpn="h3,h2" ← este servidor también habla HTTP/3 ├─ ipv4hint=["132.42.120.5"] ├─ port=443 └─ ech="..." el cliente sabe QUÉ VERSIONES USAR ANTES de abrir el socket

```
**Por qué importa de verdad:** es lo que permite **descubrir HTTP/3 sin haber ido
una vez por HTTP/1.1**. Con `Alt-Svc` (Clase 9) el navegador tiene que hacer
primero un viaje por HTTP/1.1 para enterarse de que hay HTTP/3; con el registro
`HTTPS` del DNS, la información de por dónde ir por HTTP/3 **llega en la misma
consulta que la dirección**. Es **el mismo cambio conceptual que `CAA`**: una
entidad que representa al dominio declara una política sobre el dominio, y los
respetadores la obedecen.

### Lo que **no** cambió y sigue preguntándose

- La **jerarquía root → TLD → autoritativo**, la **delegación** por `NS` y el
  **bootstrap** con los *glue records*.
- Los **cinco campos** del Resource Record, y el **TTL en segundos**.
- Los **tipos**: `A`, `AAAA`, `NS`, `CNAME`, `MX`, `SOA`, `PTR`, `TXT`, `SRV`.
- Los **algoritmos** iterativo y recursivo, y la cadena **stub → recursivo →
  root → TLD → autoritativo**. *(Esto lo desarrolla la Clase 11.)*
- Los **puertos**: **53 UDP y 53 TCP**, y **853** para DoT.
- El **round-robin** por varios registros `A`. Sigue usándose, y Chrome y Safari
  lo aleatorizan por privacidad.

> **Consecuencia para el examen:** si te preguntan *«¿qué cambió en el DNS?»*,
> la respuesta con mejor relación esfuerzo/puntos es: **nada de la arquitectura**.
> Ni la jerarquía, ni la delegación, ni los cinco campos del RR, ni los dos
> algoritmos cambiaron desde 1987. Lo que se **agregó** es DNSSEC (autenticidad,
> subdesplegada), EDNS(0) (el mecanismo de extensión), `CAA` (política de
> certificación) y `SVCB`/`HTTPS` (política de transporte). Si en cambio te
> preguntan *«¿qué es el DNS?»*, la respuesta es **exactamente la de 2010** y está
> en el §3.1.

---

# PARTE 2 — PRÁCTICA RESUELTA

## 2.1 Ejercicio 1 — Escribir un Resource Record completo

**Enunciado.** Escribí el registro de recurso completo que le corresponde a
`www.fing.edu.uy`, que resuelve a la dirección IPv4 `132.42.120.5`, tiene clase
`IN`, y su autoritativo quiere que se cachee **media hora**.

**Resolución paso a paso.**

*Paso 1 — Identificar los cinco campos.*
```

nombre → www.fing.edu.uy. clase → IN tipo → A (porque el dato es una IPv4) TTL → media hora = 30 × 60 = 1800 segundos datos → 132.42.120.5

```
*Paso 2 — Escribirlo en la sintaxis de la zona (lo que verías en un archivo de
configuración de BIND):*
```

$ORIGIN fing.edu.uy. www 1800 IN A 132.42.120.5

```
*Paso 3 — Escribirlo en la sintaxis canónica completa (lo que va por el cable):*
```

www.fing.edu.uy. 1800 IN A 132.42.120.5 │ │ │ │ └─ rdata: 4 bytes de IPv4 │ │ │ └── type: 2 bytes, valor 1 (A) │ │ └──────── class: 2 bytes, valor 1 (IN) │ └──────────────────── TTL: 4 bytes, valor 1800 └── name: 18 bytes en cable (ver §1.5)

```
*Paso 4 — Calcular cuántos bytes ocupa en la respuesta.*
```

nombre: 18 bytes (3'www'4'fing'3'edu'2'uy'0) tipo: 2 bytes clase: 2 bytes TTL: 4 bytes rdlength: 2 bytes (cuántos bytes ocupa el rdata) rdata: 4 bytes (la IPv4) ── Total RR: 32 bytes

```
**Interpretación física:** un registro `A` ocupa **32 bytes**. Cuando hacés un
`dig` de un dominio con 50 registros `A`, estás moviendo 1600 bytes de datos, más
los nombres de las preguntas. **Un paquete UDP de 512 bytes (el mínimo DNS) alcanza
para unas 12 entradas `A`.** Por eso las respuestas grandes se cortan y hay que
usar TCP (Clase 11).

*Paso 5 — Y si el autoritativo quisiera que sea un nombre en vez de una IP:*
```

www.fing.edu.uy. 1800 IN A 132.42.120.5 www.fing.edu.uy. 1800 IN AAAA 2001:7fd:21::5 ^^^^^^^^^^^^^^^ ^^^^^ ^ ^^^ ^^^^^^^^^^^^^^^ nombre TTL clase tipo datos

(un mismo nombre SÍ puede tener varios registros del MISMO tipo: así es como se hace el balanceo)

```
### ⚠️ Errores típicos
- **Olvidar el punto final** en el nombre. En la forma canónica `fing.edu.uy` sin
  punto es un **nombre relativo**, que se resuelve con el `$ORIGIN` de la zona.
- **Confundir los bytes de `clase` y `tipo`.** Los dos son de 2 bytes, pero van en
  orden **tipo primero, clase después** en el formato de cable, aunque en la
  notación de los archivos de zona la convención es **clase primero**.
- **Poner el TTL en minutos.** La unidad es **segundos**, y siempre.

## 2.2 Ejercicio 2 — Distinguir `CNAME` de `MX` con ejemplos

**Enunciado.** Una empresa tiene estos cuatro nombres. Decí qué registro
corresponde a cada caso y **por qué no las otras opciones**.
```

a) El sitio www.fing.edu.uy se va a mudar a un servidor que se llama "portal-institucional" en el data center del ISP.

b) La Universidad tiene dos servidores de correo, smtp1 y smtp2. Quiere que smtp1 sea el preferido y que, si se cae, se use smtp2.

c) El dominio nuevo "servidor-central.uy" quiere saber a qué dirección IP corresponde.

d) La empresa quiere que el correo a "no-spam"@fing.edu.uy no se acepte desde ningún servidor, y que lo declare públicamente con un TXT.

```
**Resolución paso a paso.**

*a) El sitio se muda de servidor → **`CNAME`**.*
```

www.fing.edu.uy. 1800 IN CNAME portal-institucional.isp.uy.

```
*Por qué: el objetivo es que **`www.fing.edu.uy` deje de ser un nombre terminal**
para pasar a ser **un alias** de otro. El cliente resuelve `www.fing.edu.uy`,
obtiene un nombre, y vuelve a resolver. El beneficio es que, cuando el servidor
se muda otra vez, **solo cambiás un registro**, en un solo lugar.

⚠️ **Las dos trampas del `CNAME`:**
```

1. Un nombre con CNAME NO puede tener ningún otro registro. www.fing.edu.uy. CNAME x.isp.uy. www.fing.edu.uy. MX 10 mail. ← ¡PROHIBIDO! → respuesta del servidor: "CNAME y otro tipo en el mismo nombre"
2. Un CNAME NO puede existir en la raíz. ni en un TLD, ni en la zona directa de un dominio.

```
*b) Preferencia entre servidores de correo → **`MX`**.*
```

fing.edu.uy. 1800 IN MX 10 smtp1.fing.edu.uy. fing.edu.uy. 1800 IN MX 20 smtp2.fing.edu.uy.

```
*Por qué: el `MX` no devuelve una IP, devuelve **un par (preferencia, nombre de
servidor)**. El `10` y el `20` **no son puertos**: son prioridades, y **menor es
mejor**. El MTA del emisor mira **todos** los MX, ordena por preferencia, y manda
primero al más bajo.
```

10 smtp1 ◀── se intenta PRIMERO 20 smtp2 ◀── si smtp1 responde 4.x.x o 5.x.x (error temporal), se reintenta con smtp2

```
⚠️ **La trampa clásica del ejercicio:** un `MX` **no puede contener
`132.42.120.5`**. Si se pone un número donde va el nombre del servidor, la mayoría
de los MTAs lo rechaza y el correo **no llega**. Hay que poner **siempre un
nombre**, y ese nombre se resuelve aparte con un `A` o `CNAME`.
```

❌ fing.edu.uy. MX 10 132.42.120.5 (no: es un nombre, no una IP) ✅ fing.edu.uy. MX 10 smtp1.fing.edu.uy. ✅ smtp1.fing.edu.uy. A 132.42.120.5 (el nombre se resuelve aparte)

```
*c) Un nombre nuevo → dirección IP → **`A`**.*
```

servidor-central.uy. 1800 IN A 132.42.60.10

```
*Por qué: es exactamente para eso. Y el `A` **no puede tener CNAME ni MX ni nada
más**: `A` es un dato terminal, no un alias.

*d)Declaración de política de correo → **`TXT`**.*
```

fing.edu.uy. 1800 IN TXT "v=spf1 -all"

```
*Por qué: el `TXT` guarda **texto libre**, y la industria lo usa para **SPF**
(*Sender Policy Framework*), que declara qué servidores tienen permiso de enviar
correo en nombre del dominio. `-all` significa «ninguno». `v=spf1 ...` es la forma
moderna; `TXT` también se usa para **DKIM** y para **verificación de propiedad**
ante Google Workspace o Microsoft 365.

### ⚠️ Errores típicos
- **Poner una IP en un `MX`.** Es el error número uno de configuración de correo.
- **Confundir el número del `MX` con un puerto.** `MX 10` es prioridad, `MX 25`
  sería prioridad 25, no puerto 25. Y ojo: **puede haber un `MX 25`**, que no tiene
  nada que ver con SMTP.
- **Meter un `CNAME` en la raíz.** No existe, y muchos paneles de hosting lo
  ofrecen igual: es un error del panel, no tuyo.

## 2.3 Ejercicio 3 — ¿Por qué un `CNAME` no puede existir en la raíz?

**Enunciado.** Explicá por qué la especificación prohíbe un `CNAME` en la raíz del
espacio de nombres y en la zona directa de un TLD. No es un capricho: la razón
está en el diseño de la **delegación**.

**Resolución paso a paso.**

*Paso 1 — Recordar cómo funciona la resolución.*

Para resolver un nombre hay que **bajar** por la jerarquía: root → TLD → autoritativo.
En cada paso el servidor de arriba **no tiene la respuesta**, pero tiene el `NS` del
siguiente nivel.

*Paso 2 — Ver qué haría un CNAME en la raíz.*
```

Si la raíz tuviera: . IN CNAME otro.nombre.

Entonces el root, ante CUALQUIER consulta, tendría que: 1. devolver una referencia a otro.nombre 2. el cliente resuelve otro.nombre 3. para resolver otro.nombre, vuelve a preguntar a la raíz 4. la raíz vuelve a decir "andá a otro.nombre" 5. ∞ bucle infinito

```
*Paso 3 — La razón real: la raíz es el **índice**, no un **nodo**.*
```

La raíz cumple el PAPEL de un índice: le dice al cliente a quién preguntarle. Un CNAME significa "soy otro nombre". Si el índice fuera un alias, dejaría de ser un índice: no habría ningún nodo al que ir.

╔═══════════════

```
*Paso 4 — Por qué el prohibition es por **nombre** y no por **zona**.*

La regla es: **un nombre con `CNAME` no puede tener ningún otro registro, y el
nombre `.` (ni el nombre de un TLD, que es de una sola etiqueta) es el nombre del
propio servidor de la zona.** Es una regla de **coherencia**, no de seguridad: si
`uy. IN CNAME x.` fuera legal, entonces el `NS` de `uy.` tendría que convivir con
el `CNAME`, y la delegación quedaría indefinida.

**Consecuencia práctica que se pregunta en los exámenes:** como `.uy` no puede
tener `A`, **no existe un servidor web en la raíz de `.uy`**: `http://uy/` no
funciona. Por eso existen los portales de TLD como `nic.uy` o `portal.uy`.

### ⚠️ Errores típicos
- **Decir que es por seguridad.** No: es por **coherencia de la delegación**. La
  seguridad aparece cuando un atacante mete un `CNAME` en un nombre que sí puede
  tenerlo: es el **DNS hijacking**, que es un ataque por contenido, no una
  violación de la especificación.
- **Olvidar que la prohibición aplica también a los TLD.** `com. IN CNAME ...` es
  tan ilegal como `. IN CNAME ...`.

---

# PARTE 3 — PROBLEMAS DE EXAMEN DE FING

## 3.1 ⭐ Examen 2010-02, Pregunta 1a (8 puntos) — qué es el DNS y su función

> **Fuente:** Examen `eirc1002` / `sirc1002` — 2 de febrero de 2010.
> **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1002.pdf

### Enunciado (texto literal del examen)

> **Pregunta 1 (8 puntos)**
>
> a) Defina qué es el DNS y cuál es su función.

### Solución oficial (del PDF, textual)

> El protocolo DNS (Domain Name System) es un protocolo de capa de aplicación que
> permite realizar la traducción de nombres de hosts a direcciones IP. También se
> denomina DNS a la base de datos jerárquica distribuida que almacena los mapeos
> entre nombres y direcciones IP.

### Resolución, con mi razonamiento

La oficial son **dos frases**. Para los 8 puntos hay que **desarmarlas**, porque cada
palabra de las dos frases es un punto:
```

"protocolo de capa de aplicación" │ └─▶ el DNS es un protocolo como HTTP o SMTP: mensajes, peticiones y respuestas, sobre UDP (puerto 53). NO es un algoritmo, no es una base de datos relacional, no es "un servicio" en el sentido de un proceso servidor y ya.

"traducción de nombres de hosts a direcciones IP" │ └─▶ la FUNCIÓN. Con nombre: "www.fing.edu.uy". Sin nombre: 132.42.120.5.

"base de datos jerárquica distribuida" │ ├─▶ base de datos: guarda datos y los sirve a preguntas. ├─▶ jerárquica: la información se reparte en niveles │ (root → TLD → autoritativo), no está toda junta. └─▶ distribuida: los datos están repartidos en MUCHAS máquinas (los 13 root, los cientos de TLD, los 2 autoritativos de cada dominio). No hay una máquina central.

"mapeos entre nombres y direcciones IP" │ └─▶ el contenido: tablas de correspondencia. "Mapeo" = *mapping*.

Respuesta redactada, lista para copiar en el examen:

> **a)** El DNS (Domain Name System) es un **protocolo de capa de aplicación** cuyo
> propósito es permitir la **traducción de nombres de host a direcciones IP**: los
> usuarios y las aplicaciones trabajan con nombres (por ejemplo `www.fing.edu.uy`),
> que son legibles y jerárquicos, y el DNS devuelve la dirección IP correspondiente
> (por ejemplo `132.42.120.5`), que es la que permite enviar paquetes.
>
> Estrictamente hablando, «DNS» designa **dos cosas a la vez**. Primero, es un
> **espacio de nombres jerárquico**: los nombres se organizan en un árbol
> (`.uy` → `edu` → `fing.edu.uy` → `www.fing.edu.uy`) resuelto de derecha a
> izquierda, lo que permite **delegar la administración** de cada rama a una
> autoridad distinta sin que ninguna autoridad tenga que conocer la rama entera.
> Segundo, es una **base de datos jerárquica distribuida**: los datos (las
> correspondencias nombre–dirección, llamadas *mapeos*) están repartidos en miles de
> servidores organizados en esa misma jerarquía. Cada servidor solo guarda los datos
> de su zona y **no conoce los de las demás**, y responde a las consultas de las
> zonas inferiores.
>
> No puede ser un archivo central porque (i) la tabla completa no entra en cada host
> y habría que redistribuirla en cada alta o baja, (ii) sería un punto único de
> falla y de compromiso, y (iii) no habría forma de validar quién autorizó cada
> nombre. La **distribución** resuelve las tres cosas: escala, tolerancia a fallas y
> mantenimiento.

Lo que agregaría yo y la oficial no dice (y es lo que más suma, porque
demuestra que entendiste el diseño):

1. **La jerarquía es lo que hace posible la distribución.** Es un árbol en el que
   cada nodo conoce **solo a sus hijos** y delega en ellos la autoridad: es la
   propiedad de un padre que reparte entre cada hijo la porción que le corresponde.
   Un árbol plano (una sola tabla con todos los nombres) no escalaría; uno con copia
   completa en cada nodo, tampoco.
2. **El nombre es un espacio de nombres, no solo una etiqueta.** Los nombres son
   únicos y jerárquicos, y eso es lo que permite
   que `edu.uy` y `edu` puedan coexistir sin colisión: son ramas distintas del árbol.
3. **El DNS es bidireccional conceptualmente, pero en la práctica es unidireccional.**
   Nombre → IP (`A`) es lo que todo el mundo usa. IP → nombre (`PTR`) existe pero
   casi solo sirve para logística y para los servidores de correo. La resolución
   inversa **no se puede usar para validar** nada, porque no está garantizada.

> **Nota contra la oficial:** la oficial es precisa pero **corta**: enumera las dos
> definiciones y no explica **por qué** tiene que ser jerárquica y distribuida. Si
> el corrector es humano, agregar la justificación del archivo central da puntos;
> si es automático, la oficial es el texto que hay que reproducir.

3.2 Examen 2010-02, Pregunta 1b (8 puntos) — iterativa vs. recursiva

> **Fuente:** el mismo examen, apartado b).

Enunciado (texto literal del examen)

> b) Explique la diferencia entre las consultas iterativas y recursivas al DNS.

Solución oficial (del PDF, textual)

> En una consulta iterativa un host le hace una consulta a su DNS local. El DNS local
> en ese momento deberá encargarse de resolver la consulta reenviándola a todos los
> servidores DNS que sean necesarios. Comienza por consultar la raíz, y cada DNS que
> consulte (que no tenga la respuesta) le indicará cuál es el servidor siguiente (más
> abajo en la jerarquía) al que debe preguntarle.
>
> En una consulta recursiva el host le hace una consulta a su DNS local. El DNS local

```
> preguntará a un servidor raíz. Si la raíz no conoce la respuesta, él mismo le
> preguntará al servidor que corresponda que esté más abajo en la jerarquía. Si este
> servidor no conoce la respuesta, se la preguntará al siguiente que esté más abajo
> en la jerarquía. Luego de que obtienen la respuesta, se volverá hacia atrás en la
> recursión para devolver el resultado al que consultó originalmente.

### Resolución, con mi razonamiento

**Un solo ejemplo mental y las dos definiciones salen solas.** Alguien pregunta
«¿dónde está la Farmacia?»:
```

═══ ITERATIVO ═══ (el que pregunta va y viene)

```
YO ──"¿sabés dónde está la Farmacia?"──► VECINO
YO ◄──"no sé, preguntale a la Farmacia de Pellegrini 1230"── VECINO
YO ──"¿sabés...?"──► FARMACIA DE PELLEGRINI 1230
YO ◄──"sí, es acá"──
(yo hice 2 viajes)
```

═══ RECURSIVO ═══ (el que pregunta se hace cargo de todo)

```
YO ──"¿sabés dónde está la Farmacia?"──► VECINO
VECINO ──"¿sabés...?"──► FARMACIA DE PELLEGRINI 1230
FARMACIA ──"sí, es acá"──► VECINO
VECINO ──"es acá"──► YO
(yo hice 1 viaje, pero el vecino hizo 3)
```

La diferencia en una frase:

	Iterativa	Recursiva
Quién sigue preguntando	**el que preguntó**	**el que recibió la pregunta**
Qué devuelve el que no sabe	«no sé, **preguntale vos** a este»	**pregunta él** y vuelve
Quién carga con el trabajo	el cliente (4 viajes de ida y vuelta)	el servidor (1 viaje pa
Quién la usa	el **resolver recursivo** (el del ISP), entre servidores	el **host**, contra

> **El error típico de examen, y es el que la propia oficial pisea:** decir que «el
> host le hace una consulta iterativa a su DNS local». No. El **stub resolver del
> host** hace una consulta **recursiva** a su servidor de nombres local: «resolveste
> esto por mí y traeme la respuesta final». Lo que el **servidor local** hace hacia
> afuera son consultas **iterativas**: le pregunta al root, el root le dice
> «preguntale al `.uy`», y así. **Esto no cambió desde 1987.**

Respuesta redactada, lista para copiar en el examen:

> **b)** Se distinguen por **quién se hace cargo de seguir consultando** cuando el
> servidor consultado **no** conoce la respuesta.
>
> En una **consulta iterativa**, el servidor que no tiene la respuesta **no hace
> nada por su cuenta**: devuelve al cliente una **referencia** («ese nombre está
> delegado a este otro servidor, andá a preguntarle») y el cliente, que es el que
> empezó, **sigue bajando por la jerarquía** hasta llegar a un servidor que sí tenga
> la respuesta. El cliente hace, entonces, **cuatro viajes**: uno a cada nivel.
>
> En una **consulta recursiva**, el servidor que no tiene la respuesta **se hace cargo
> él mismo** de consultar al nivel inferior, y así sucesivamente hasta obtener la
> respuesta final (o un error). Cuando la tiene, **vuelve hacia atrás por la
> recursión** devolviéndola a quien le consultó. El cliente hizo **un solo viaje**.
>
> En la práctica, el **cliente** (el *stub resolver* del sistema operativo) le
> hace una consulta **recursiva** a su **servidor de nombres local** (el del ISP),
> que a su vez hace consultas **iterativas** al root, al TLD y al autoritativo.
> La iterativa existe para que **un solo servidor recursivo** pueda hacer el trabajo
> pesado y cachear todo, en vez de hacer que cada cliente de Internet repita los
> mismos cuatro viajes.

Sobre el matiz de la oficial que está bien dicho y hay que repetir: la oficial
dice, para la recursiva, que «**luego de que obtienen la respuesta, se volverá hacia
atrás en la recursión**». Esa frase es la clave: **recursión** significa que el
call stack crece a medida que se baja y se **desarma** a medida que se sube. No es un concepto

3.3 Examen 2007-07, Pregunta 4 (8 puntos) — función del DNS y protocolo involucrado

> **Fuente:** Examen `eirc0707` / `sirc0707` — julio de 2007.

Este problema se desarrolla **entero en la Clase 11**, junto con la caché y el TTL
(que es lo que aporta de nuevo). Acá se deja el planteo, que es de §1.1 y §1.2:

- **Función del DNS:** traducir nombres a direcciones IP, resolviendo de derecha a
  izquierda sobre un espacio de nombres jerárquico y distribuido.
- **Protocolo involucrado:** el DNS es un protocolo de **capa de aplicación** que
  viaja sobre **UDP puerto 53** en el caso normal, y sobre **TCP puerto 53** cuando
  la respuesta no entra en un datagrama o cuando se pide una **transferencia de
  zona** (`AXFR`).
- **Mensaje:** 12 bytes de cabecera (ID, flags, QDCOUNT, ANCOUNT, NSCOUNT,
  ARCOUNT) más cuatro secciones (Question, Answer, Authority, Additional).

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. El DNS existe porque los humanos **no memoramos IPs** y porque una IP no dice
   quién es el dueño, ni si el servidor está vivo, ni quién lo administra.
2. **No puede ser un archivo central** por tres razones independientes: la tabla no
   entra en cada host, sería un punto único de falla **y** de compromiso, y no hay
   forma de validar quién autorizó cada nombre.
3. El DNS es **dos cosas**: un **espacio de nombres jerárquico** (cómo se nombran y
   quién tiene autoridad) y una **base de datos distribuida** (qué corresponde a
   qué). La primera dice a quién preguntarle, la segunda qué preguntar.
4. El nombre se resuelve **de derecha a izquierda** porque la autoridad va de la
   raíz hacia abajo, y solo la raíz sabe qué TLDs existen.
5. El **root** (13 identidades `a`–`m`, IPv4 por *anycast*) **no conoce los
   dominios**: solo sabe quién es el TLD de cada uno. El **TLD** tampoco conoce los
   dominios: sabe quién es su autoritativo.
6. La **delegación** se hace con registros `NS`, y los **glue records** (registros
   `A` con el nombre de un servidor de nombres) existen para **romper el círculo**:
   sin ellos, el nombre del servidor autoritativo no se podría resolver.
7. Un **Resource Record** tiene **cinco campos**: nombre, clase (`IN`), tipo, TTL
   (4 bytes) y datos. El TTL son **segundos**, y lo define el **autoritativo** de
   ese registro.
8. `A` y `AAAA` son IP; `NS` son servidores de nombres; `CNAME` es un alias; `MX`
   es el servidor de **correo** con un número de **preferencia**; `SOA` es el
   control de zona (serial, refresh, retry, expire, minimum); `PTR` es la inversa;
   `TXT` texto libre; `SRV` servicio con puerto; `CAA` qué CA puede emitir.
9. El **stub resolver** del SO consulta **recursivamente** al **resolver local**,
   que a su vez consulta **iterativamente** a root, TLD y autoritativo.
10. **Esto no cambió desde 1987**: ni la jerarquía, ni los dos algoritmos, ni los
    puertos (53 UDP y TCP). Lo que cambió es la privacidad (DoH, Clase 11).

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué el DNS no puede ser un archivo central? Dá las **tres** razones.

<details><summary>Respuesta</summary>

1. **Escala:** la tabla completa tiene una entrada por host (hoy del orden de miles de millones). A
2. **Punto único de falla:** si el archivo o el servidor que lo sirve se cae, **todo Internet deja
3. **Mantenimiento:** no hay forma de validar **quién autorizó** cada nombre a registrarse, porque

La solución del DNS es la **distribución jerárquica**: particionar los datos por zona, replicar ca

</details>

2. ¿Qué diferencia hay entre un **nombre** y un **dominio**? ¿Y entre un
dominio y un **host**?

<details><summary>Respuesta</summary>

En la práctica se usan como sinónimos, pero técnicamente:

- **Nombre** (*name*): cualquier etiqueta del espacio de nombres. `www` es un nombre, `fing.edu.uy
- **Dominio**: un nombre **delegado**, es decir del que alguien es responsable. `fing.edu.uy` es u
- **Host** (*host*): un nombre que además tiene un registro `A` o `AAAA` y por lo tanto **designa u

Por eso el enunciado del `2010-02 P1a` habla de «nombres de hosts»: es el nivel más bajo y más con

</details>

3. ¿Qué sabe el root server y qué **no** sabe? ¿Cuántos hay?

```
<details><summary>Respuesta</summary>

Hay **13 identidades lógicas**, nombradas de `a.root-servers.net` a `m.root-servers.net`, y cada u

**Sabe:** los registros `NS` (y `A`) de todos los **TLD**, unos 1500 entradas. Sabe quién administ

**NO sabe:** los dominios. Jamás supo qué es `fing.edu.uy`, ni cuántos hay, ni qué IP tienen. Pregu

El número **13** es un artefacto histórico de 1987 (elegido por el DARPA), congelado después por l

</details>

**4.** ¿Qué es un **glue record** y por qué existe? ¿Por qué a veces hay que
duplicar información?

<details><summary>Respuesta</summary>

Un **glue record** es un registro `A` (o `AAAA`) con el **nombre de un servidor de nombres**, guar

Existe para **romper el círculo del bootstrap**: si el `.uy` te dice «el autoritativo de `fing.edu

La información está **duplicada** (el mismo `A` existe en el glue del `.uy` y en la zona `fing.edu

El costo es que se puede **desincronizar**: si el autoritativo cambia de IP y nadie actualiza el g

</details>

**5.** Escribí un RR completo para `mail.fing.edu.uy` → `132.42.120.9`, clase `IN`,
cacheable 1 hora. Decí además cuántos bytes ocupa.

<details><summary>Respuesta</summary>
```

nombre clase tipo TTL datos mail.fing.edu.uy. IN A 3600 132.42.120.9 ```

La TTL es `1 hora = 60 × 60 = 3600 segundos`. En la zona se escribiría `$ORIGIN fing.edu.uy.` y `mail 3600 IN A 132.42.120.9`.

Tamaño: nombre `4mail4fing3edu2uy0` = 1+4 + 1+4 + 1+3 + 1+2 + 1 = **18 bytes**; más tipo 2 + clase 2 + TTL 4 + rdlength 2 + rdata 4 = **32 bytes** en total.

6. Un dominio tiene `MX 10 smtp1.uy` y `MX 20 smtp2.uy`. Si `smtp1` está caído y `smtp2` **no acepta** correo de esa persona, ¿qué pasa? ¿Dónde está el problema?

▶ **Respuesta**

7. ¿Por qué un `CNAME` no puede existir en la raíz del espacio de nombres?

▶ **Respuesta**

8. ¿Por qué no se puede resolver un nombre «de derecha a izquierda» si el nombre se lee de izquierda a derecha?

▶ **Respuesta**

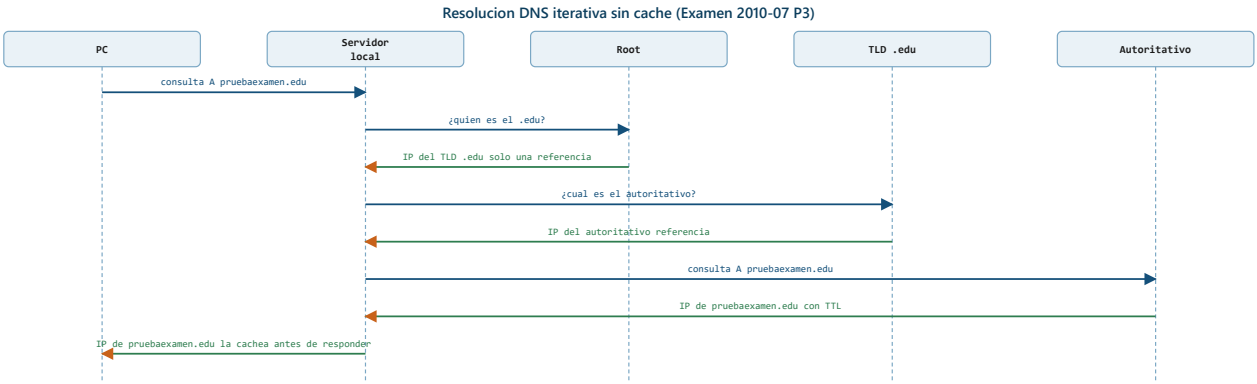
🔗 Referencias

- **Examen** `eirc1002` / `sirc1002` — Pregunta 1 completa (8 pts): qué es el DNS y su función (a) y diferencia entre consultas iterativas y recursivas (b)
- **Examen** `eirc1012` / `sirc1012` — Pregunta 1a (8 pts): el TTL del RR, quién lo define, a qué refiere y en qué unidades
- **Examen** `eirc1007` / `sirc1007` — Pregunta 3 (8 pts): resolución iterativa completa sin caché (desarrollada en la Clase 11)
- **Examen** `eirc0707` / `sirc0707` — Pregunta 4 (8 pts): función del DNS y protocolo involucrado
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.4 («El sistema de nombres de dominio»), §7.4.2 (el espacio de nombres jerárquico)
- **RFC 1034** — *Domain Names: Concepts and Facilities* (1987): el diseño completo, que **no cambió**
- **RFC 1035** — *Domain Names: Implementation and Specification* (1987): el formato del mensaje y de los RR
- **RFC 6891** — EDNS(0) (2013, *Internet Standard*): añade el pseudo-registro `OPT`; base de DNSSEC, CAA y SVCB/HTTPS
- **RFC 8659** — CAA, *Certification Authority Authorization* (2019; reemplaza a RFC 6844)
- **RFC 4033-4035** — DNSSEC (2005) · **RFC 5155** — raíz firmada (2010) · **RFC 8080** — EdDSA · **RFC 8901** — *multi-signer*
- **RFC 9460** — SVCB/HTTPS (2023): el DNS transporta política de transporte
- **IANA**, *Root Zone Database* — la lista real de los 13 root servers y los TLDs existentes
- **Clase siguiente:** 11 — DNS: iterativo vs. recursivo, caché y TTL

Clase 11

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 Los dos algoritmos, definidos sin ambigüedad
1.2 El flujo real: quién le pregunta a quién
1.3 Los mensajes literales
1.4 La caché, y por qué el TTL es un compromiso
1.5 El caché negativo: `NXDOMAIN` y `SOA.MINIMUM`
1.6 ⚠️ Actualización a 2026 — lo que cambió y lo que no
1.7 El transporte: UDP, TCP y por qué DoH le ganó a DoT
2.1 Ejercicio 1 — La cadena completa con caché en el TLD
2.2 Ejercicio 2 — ¿Cuánto dura una entrada de caché? Con las cuentas
2.3 Ejercicio 3 — Por qué `NXDOMAIN` se cachea, y qué dice `RFC 8020`
3.1 ⭐ Examen 2010-07, Pregunta 3 (8 puntos) — resolución iterativa sin caché
3.2 ⭐ Examen 2010-12, Pregunta 1 (8 puntos) — el TTL y los pasos de una consulta tipo A
3.3 Examen 2007-07, Pregunta 4 (8 puntos) — función del DNS y protocolo involucrado
📌 Resumen
✅ Autoevaluación
🔗 Referencias



Bloque: B (Aplicación) · **Semana:** 4 · **Duración estimada:** 110 min **Correlativa:** Clase 10 (DNS: la base de datos distribuida) · **Sigue con:** Clase 12 (el resolver DNS en la práctica, con sockets) **Material de referencia:** Tanenbaum cap. 7 (§7.4.3, «Resolución de nombres») · RFC 1034, RFC 1035, RFC 6891, RFC 7766, RFC 8020, RFC 8484, RFC 7858, RFC 9250, RFC 8767, RFC 9460, RFC 4033-4035, RFC 5155

🎯 Al terminar esta clase vas a poder

1. Explicar la diferencia entre una consulta **iterativa** y una **recursiva**, diciendo **quién se hace cargo** de seguir preguntando cuando el servidor consultado no tiene la respuesta.
2. Describir el **flujo real** de una resolución, desde el *stub resolver* del sistema operativo hasta el servidor autoritativo, y decir **qué parte es recursiva y qué parte es iterativa**.
3. Escribir los **mensajes DNS literales** de una resolución completa sin caché, y explicar **qué contesta cada uno** (incluida la diferencia entre una *referencia* y una *respuesta* final).
4. Explicar por qué existe la **caché**, por qué el TTL es un **compromiso**, **quién lo define** y **quién lo descuenta**.
5. Describir la **caché negativa**: qué es `NXDOMAIN`, qué es `SOA.MINIMUM`, por qué hay que cachear los errores y qué dice `RFC 8020`.
6. Resolver completo el **Examen 2010-07 Pregunta 3** (8 puntos) con su solución oficial a la vista.

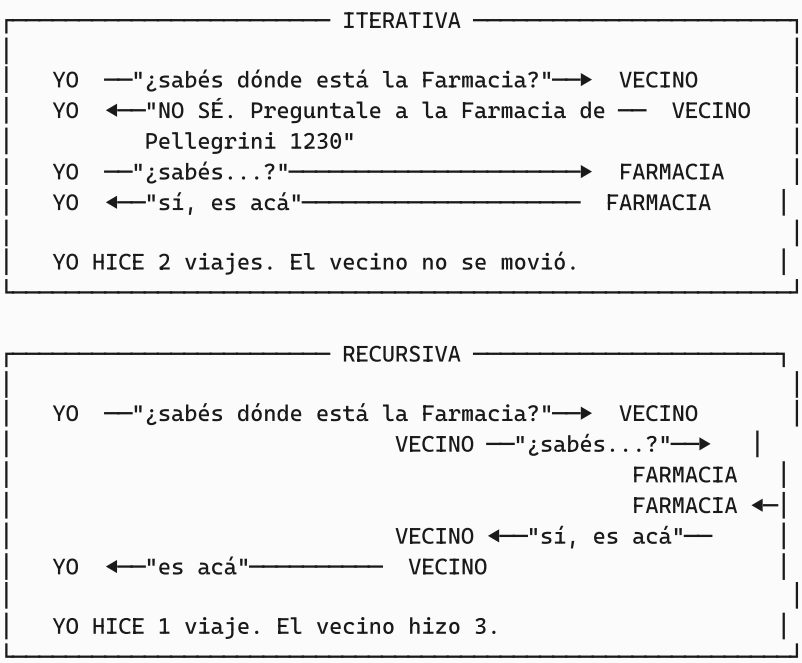
7. Resolver completo el **Examen 2010-12 Pregunta 1** (8 puntos), los dos apartados, con su solución oficial a la vista.
8. Explicar qué cambió con **DoH**, **DoT**, **DoQ**, **DNSSEC** y **SVCB/HTTPS**, y por qué el punto 2 de arriba **no se enseña** en 2026.

> **Prerequisito del examen:** `2010-12 P1` (8 puntos — ★ el TTL del RR y los pasos de una consulta tipo A), `2010-07 P3` (8 puntos — ★ la resolución iterativa completa sin caché), `2010-02 P1b` (8 puntos — iterativa vs. recursiva), `2007-07 P4` (8 puntos — función del DNS y protocolo involucrado).

PARTE 1 — TEORÍA

1.1 Los dos algoritmos, definidos sin ambigüedad

El DNS tiene **dos** algoritmos de consulta, y la diferencia entre ellos se reduce a **una sola pregunta**: cuando el servidor al que consulté **no tiene** la respuesta, **¿quién sigue preguntando?**



	Iterativa	Recursiva
El que no sabe	devuelve una **referencia** («andá a preguntarle a este») y **no hace nada más**	**pregunta él** al nivel inferior, y así hasta obtener la respuesta final
Quién sigue preguntando	**el que preguntó** (el cliente)	**el que recibió la pregunta** (el servidor)
Qué vuelve	o una respuesta, o una referencia	**siempre** una respuesta final (o un error terminal)
Cuántos viajes ve el cliente	**muchos**: uno por nivel	**uno**: pide y recibe la respuesta
Quién la usa	el **resolver recursivo**, contra los autoritativos	el **host** (su *stub resolver*), contra el resolver local

Y el último punto es el que decide todo lo demás: si **cada host** de Internet tuviera que hacer los cuatro viajes él mismo, el root server tendría que atender miles de millones de consultas por segundo y todos los hosts estarían haciendo el mismo trabajo redundante. La **consulta recursiva** le delega ese trabajo a **un solo servidor**, que además es el único que puede **cachear** el resultado y devolverlo al siguiente cliente sin volver a bajar por la jerarquía.

> **La frase que hay que saber decir de memoria: la iterativa existe para que un solo servidor recursivo**

> **haga el trabajo pesado y cachee todo; la recursiva existe para que el cliente no tenga que hacerlo.**

Cómo se ve en el cable: los flags `RD` y `RA`

No hay que saberlo de memoria para el examen, pero **te da la certeza absoluta** de qué algoritmo se está usando:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|            ID (16 bits)                                       |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|QR|   Opcode   |AA|TC|RD|RA|  Z  |AD|CD|     RCODE           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           QDCOUNT = número de preguntas                       |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           ANCOUNT = número de respuestas                      |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           NSCOUNT = número de registros de autoridad          |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           ARCOUNT = número de registros adicionales           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

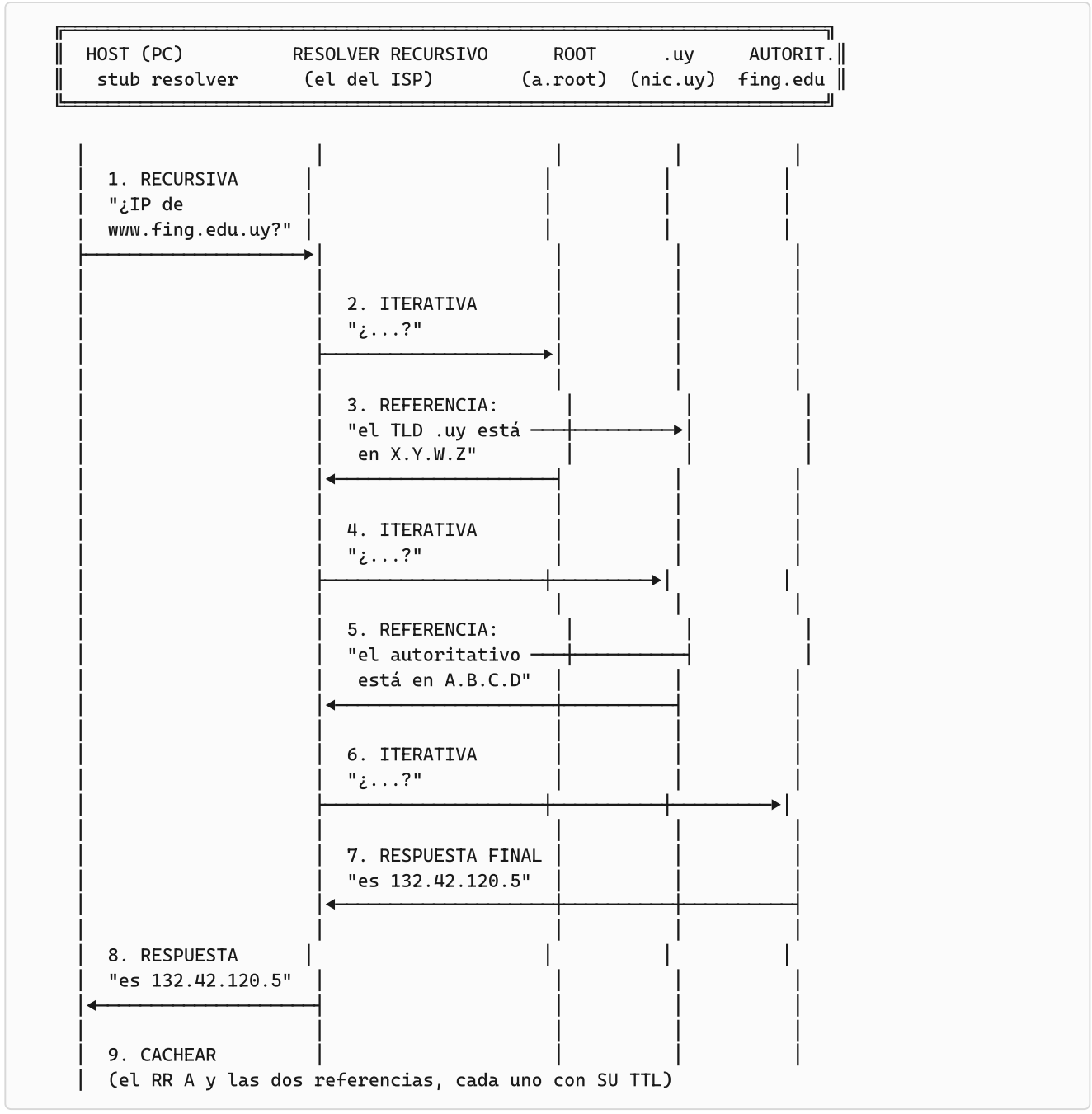
- `RD` (*Recursion Desired*, «quiero recursión»): lo pone **el que pregunta**. `RD = 1` significa «si no sabés, averiguatelo vos por mí».
- `RA` (*Recursion Available*, «recursión disponible»): lo pone **el que responde**, y dice si **ese servidor** está dispuesto a hacer de recursivo. Un **root server tiene** `RA = 0` **siempre**: los root no hacen de recursivos, jamás.
- `AA` (*Authoritative Answer*): lo pone el que responde y dice «esta es la respuesta final, la tengo yo». **Un root o un TLD contestan con** `AA = 0`, porque lo que están dando es una *referencia*, no una respuesta.
- `TC` (*Truncated*): la respuesta no entró y hay que repetir por TCP.
- `RCODE`: `0` NOERROR, `1` FORMERR, `2` SERVFAIL, `3` **NXDOMAIN**, `4` NOTIMP, `5` REFUSED.

Los dos valores que aparecen en los exchanges de esta clase:

```
Consulta del cliente:   QR=0  RD=1  →  flags = 0x0100   (1000 0000 0000 0000)
Respuesta de un root:   QR=1  RD=1  RA=0  →  flags = 0x8180   (REFERENCIA, no respuesta)
Respuesta del autorit.: QR=1  AA=1  RD=1  RA=0  →  flags = 0x8580   (RESPUESTA FINAL)
Respuesta del recursivo:QR=1  RD=1  RA=1  →  flags = 0x8180   (le devuelve el final al PC)
```

1.2 El flujo real: quién le pregunta a quién

Este es **el** diagrama de la unidad, y es la respuesta completa a `2010-02 P1b` y a `2010-07 P3`:



Lo que hay que leer en este diagrama:

1. **El mensaje 1 es RECURSIVO.** El *stub resolver* del sistema operativo le dice al resolver del ISP «resolveste esto por mí y traeme la respuesta final».
2. **Los mensajes 2 a 7 son ITERATIVOS.** El resolver recursivo es el que pregunta, y el que va bajando por la jerarquía es **él**, no su cliente.
3. **Los mensajes 3 y 5 NO son respuestas**: son **referencias** (*referrals*). Vienen con `ANCOUNT = 0` y el dato en la sección **Authority** + **Additional**.
4. **Solo el mensaje 7 es una respuesta final**, y es el único con `AA = 1`.
5. **El mensaje 8 no es un viaje nuevo**: es la vuelta del mensaje 1.

> ⚠️ **Esto no cambió desde 1987.** Ni la cadena de actores, ni los dos algoritmos, ni los puertos. Un parcial de FING de 2011 y uno de 2026 tienen **el mismo desenho de arriba**. Lo único que cambió es **quién ve el**

> **primer mensaje** (§1.7).

1.3 Los mensajes literales

Un mensaje DNS tiene **12 bytes de cabecera** y **cuatro secciones**. Todo lo que falta son **copias literales** de `dig`, que es como lo vas a ver en Wireshark.

La cabecera, byte a byte

```
   ID = 0x3F2A        (lo elige el cliente; sirve para emparejar respuesta)
   flags = 0x0100     (QR=0 consulta, RD=1 quiero recursión, RCODE=0)
   QDCOUNT = 1        (una pregunta)
   ANCOUNT = 0        (cero respuestas  ← OJO: todavía no hay respuesta)
   NSCOUNT = 0        (cero registros de autoridad)
   ARCOUNT = 0        (cero registros adicionales)
```

Mensaje 1 — el PC pregunta a su resolver local (petición)

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x3F2A
;; flags: rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.fing.edu.uy.               IN      A

;; MSG SIZE  rcvd: 0
```

Traducción campo por campo: `;; flags: rd` es el `RD = 1`. `QUESTION` es el `QDCOUNT`. Las tres secciones en cero son `ANCOUNT`, `NSCOUNT` y `ARCOUNT`. **`rd`** significa que el PC **quiere** recursión; que el servidor la conceda o no se ve en el `ra` de la respuesta.

Mensaje 3 — el root contesta: **una referencia, no una respuesta**

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x3F2B
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; QUESTION SECTION:
;www.fing.edu.uy.               IN      A

;; AUTHORITY SECTION:
uy.             172800  IN  NS   a.nic.uy.
uy.             172800  IN  NS   b.nic.uy.

;; ADDITIONAL SECTION:
a.nic.uy.       172800  IN  A    200.40.4.1
b.nic.uy.       172800  IN  A    200.40.4.2

;; MSG SIZE  rcvd: 132
```

Leé esto con cuidado, porque es el 80 % del examen de `2010-07 P3`:

```
ANSWER: 0   ←  NO HAY RESPUESTA.  El root NO sabe la IP de www.fing.edu.uy
               y NUNCA la va a saber: no es su trabajo.

AUTHORITY: 1  +  ADDITIONAL: 1   ←  Esto es una DELEGACIÓN:
               "el que sabe es a.nic.uy, y además, porque te va a servir,
                te paso su dirección: 200.40.4.1"

El flag `aa` NO aparece  →  AA = 0. Si fuera AA=1 el root estaría afirmandose
                            autoritativo sobre fing.edu.uy, lo cual es mentira.
```

Y el `ADDITIONAL SECTION` es exactamente el **glue record** de la Clase 10: el `.uy` no tiene por qué saber la IP de `a.nic.uy` (el nombre), pero **sí** tiene el `A` del servidor de nombres **por el que le delegó** `fing.edu.uy`, así que lo paga en el mismo mensaje y nos ahorra un viaje.

Mensaje 5 — el TLD `.uy` contesta: otra referencia

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x3F2C
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; QUESTION SECTION:
;www.fing.edu.uy.               IN      A

;; AUTHORITY SECTION:
fing.edu.uy.     172800  IN  NS   ns1.fing.edu.uy.
fing.edu.uy.     172800  IN  NS   ns2.fing.edu.uy.

;; ADDITIONAL SECTION:
ns1.fing.edu.uy. 172800  IN  A    132.42.60.1
ns2.fing.edu.uy. 172800  IN  A    132.42.60.2

;; MSG SIZE  rcvd: 140
```

Exactamente la misma forma que la del root, con `ANCOUNT = 0` otra vez. El `.uy` tampoco conoce los dominios: lo que tiene es el **`NS` de `fing.edu.uy`**.

Mensaje 7 — el autoritativo contesta: la respuesta final

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x3F2D
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.fing.edu.uy.               IN      A

;; ANSWER SECTION:
www.fing.edu.uy.   3600  IN  A   132.42.120.5

;; MSG SIZE  rcvd: 56
```

`flags: qr aa`: acá sí aparece el `aa` (*Authoritative Answer*). **Es la primera y única respuesta de la cadena** que tiene contenido. Y mirá el TTL: `3600`.

Mensaje 8 — el resolver recursivo le contesta al PC

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x3F2A
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.fing.edu.uy.               IN      A

;; ANSWER SECTION:
www.fing.edu.uy.   3600  IN  A   132.42.120.5

;; MSG SIZE  rcvd: 56
```

`flags: qr rd ra`: el `ra` (*Recursion Available*) confirma que el servidor **aceptó** hacer de recursivo. El `id` es **el mismo del mensaje 1** (`0x3F2A`): así el PC sabe que esta respuesta es la suya.

Resumen de los ocho mensajes

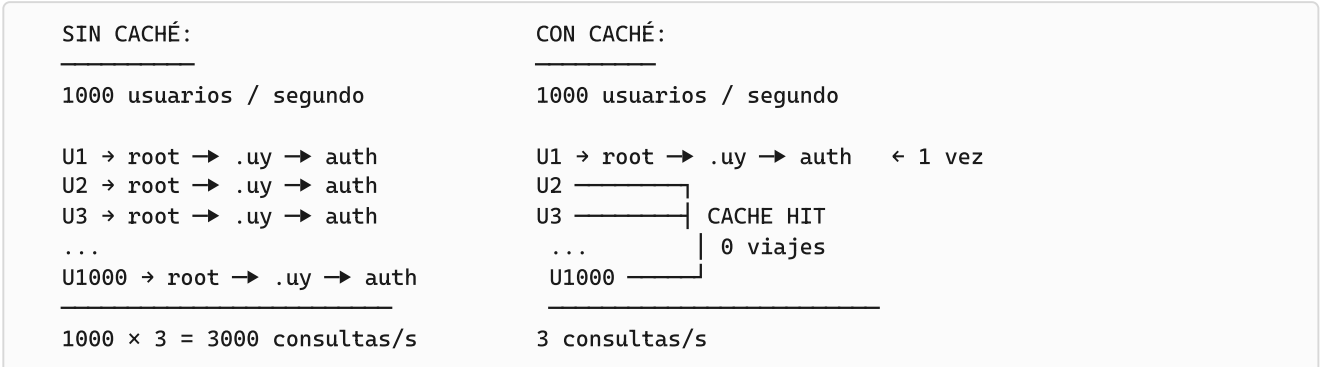
#	De	A	Tipo	ANCOUNT	AA	Qué transporta
1	PC (stub)	resolver local	**recursiva**	0	—	la pregunta
2	resolver local	root	iterativa	0	—	la pregunta
3	root	resolver local	**referencia**	**0**	0	`NS` de `.uy` + glue
4	resolver local	`.uy`	iterativa	0	—	la pregunta
5	`.uy`	resolver local	**referencia**	**0**	0	`NS` de `fing.edu.uy` + glue
6	resolver local	`ns1.fing.edu.uy`	iterativa	0	—	la pregunta
7	`ns1.fing.edu.uy`	resolver local	**respuesta**	**1**	**1**	el `A` con TTL 3600
8	resolver local	PC	respuesta	**1**	0	el mismo `A`

Cuatro ida y vuelta = 4 RTT. Uno solo trae la respuesta.

1.4 La caché, y por qué el TTL es un compromiso

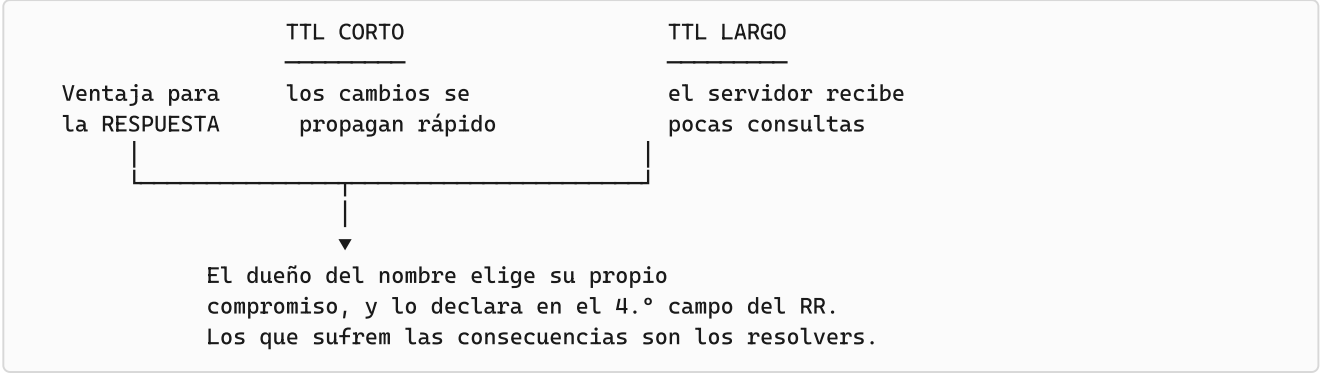
Por qué existe la caché

Sin caché, cada visita a una página generaría **2 a 6 consultas DNS**. Con `www.fing.edu.uy` como ejemplo, esto multiplicaría la carga sobre los 13 root exactamente como §1.1 explicó que **no puede ser**.



El compromiso del TTL

El TTL es un número que **elige el dueño del nombre**, y es exactamente eso: un compromiso entre **dos intereses opuestos** que **nadie** puede satisfacer a la vez.



Valor del TTL	Para qué se usa en la práctica	Ejemplo real
30 s a 5 min	contenido que cambia seguido: resultados de una búsqueda, el stock de una tienda, el resultado de una votación	un contador de visitas, un destino que cambia cada semana
300 s a 1 h	contenido dinámico pero no frenético: portadas de noticias	la home de un diario
3600 s a 86400 s	contenido estable: HTML, CSS, JS, imágenes	los assets de una aplicación
172800 s (2 días)	**los `NS` de una delegación**, por defecto	`fing.edu.uy. IN NS ns1.fing.edu.uy.`

> ⚠️ **El detalle fino que se pregunta:** el TTL de un registro `A` y el TTL del `NS` que delegó esa zona son **independientes**. El `NS` suele tener un TTL enorme (días) porque cambiar el servidor de nombres es raro; el `A` suele tener uno corto porque cambiar la IP de un servidor web es común. **Y el que va a doler es el `NS`**: si cambiás de servidor de nombres y no esperás a que venza su TTL, durante las horas siguientes una parte de Internet **sigue falando con el servidor viejo**. Es el mismo mecanismo del glue desactualizado que vimos en la Clase 10.

Los dos tiempos distintos que hay que separar: `max-age` y `TTL`

Esto confunde todo el tiempo y es pregunta clásica:

	`max-age` de HTTP	`TTL` de DNS
Dónde vive	cabecera `Cache-Control` de una respuesta HTTP	4.º campo de un Resource Record
Lo define	el **servidor web**	el **servidor autoritativo** de la zona
Unidades	segundos	**segundos** (siempre, en todas las versiones)
Medido desde	la `Date` de la respuesta	el momento en que **el resolver guardó** el registro
Quién lo descuenta	el que guarda (navegador o proxy)	**el que guarda**, o sea el resolver recursivo

La analogía que lo aclara: el `TTL` es como el **vence** de un producto en la alacena, y el `max-age` es el **vence** escrito por el cocinero en el plato. Los dos cuentan el mismo reloj, pero **el que lo pone es otro**, y por eso pueden no coincidir.

El reloj, con las cuentas a la vista

```
El autoritativo responde:            www.fing.edu.uy.  3600  IN  A  132.42.120.5
                                                       ▲
                                                       |  4 bytes = 3600 segundos
                                                       |  = 1 hora, menos 2400 s que ya
                                                       |    pasaron en la red (los TTL
                                                       |    arrancan a contar en la
                                                       |    respuesta, no en el origen)

t = 0 s        el resolver recibe y GUARDA:  TTL restante = 3600
t = 1200 s     alguien pregunta de nuevo:
               frescura restante = 3600 − 1200 = 2400 s   →  HIT, 0 bytes, 0 viajes
t = 3600 s     vence.  Freno, la entrada queda "stale" (vieja pero presente).
t = 3601 s     alguien pregunta:  fresco? NO  →  hay que REVALIDAR
t = 3650 s     llega la respuesta:  "sí, es 132.42.120.5, TTL 3600"
               → se RENUEVA: ahora vence en 3600 + 3650 = 7250 s

El campo `Age` que devuelve un proxy es justamente esto:
frescura restante = TTL del origen − Age
freshness_remaining = max-age − Age
```

El TTL es un contador decreciente en el resolver, no un reloj absoluto. Es la frase que hay que decir si te preguntan «¿a qué refiere el valor del TTL?».

1.5 El caché negativo: `NXDOMAIN` y `SOA.MINIMUM`

El problema

Un cliente escribe mal una URL: `www.fing.edu.uy/articulso/2026`. El navegador pregunta por el host, que **sí existe**, así que ese caso no es problema. El problema real es cuando el **nombre entero no existe**:

```
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 0x51C7
;; flags: qr aa rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;noexiste.fing.edu.uy.            IN      A

;; AUTHORITY SECTION:
fing.edu.uy.  3600  IN  SOA  ns1.fing.edu.uy. admin.fing.edu.uy. (
                             2026100601 ; serial
                             7200       ; refresh
                             3600       ; retry
                             1209600    ; expire
                             3600 )     ; minimum   ⟵ el que importa acá
;; MSG SIZE  rcvd: 121
```

`NXDOMAIN` = el nombre no existe, no el tipo. Si el nombre existiera pero no tuviera un `A`, la respuesta sería `NOERROR` con `ANCOUNT = 0` (se llama **NODATA**). La diferencia importa:

```
www.fing.edu.uy.        A      → NOERROR, 1 respuesta      ✅
www.fing.edu.uy.        MX     → NOERROR, 0 respuestas     ⚠️ NODATA: existe, pero no es MX
noexiste.fing.edu.uy.   A      → NXDOMAIN, 0 respuestas    ❌ no existe el nombre
```

`SOA.MINIMUM`: el TTL del error

El campo `minimum` del `SOA` (el séptimo y último número) es **el TTL por defecto de las respuestas negativas**. Cuando un autoritativo responde `NXDOMAIN` sin poner un `SOA`, el resolver usa ese `minimum`. En la zona de ejemplo es `3600`.

```
El SOA también se cachea, y con SU propio TTL (el de la fila del SOA,
que acá es 3600). El `minimum` es el TTL por defecto del NXDOMAIN cuando
el autoritativo no manda un SOA explícito.

Ojo con la diferencia: el `minimum` NO es «duración del error»: es un
valor por defecto para cuando no hay otro disponible. El TTL que manda
es el del SOA que venga en la respuesta.
```

Por qué hay que cachear los errores

Contra lo que parece intuitivo, **cachear un `NXDOMAIN` es una protección**:

```
SIN CACHÉ NEGATIVA:
  1000 consultas por segundo de "www.escrito-mal.typo.uy"
    × 3 viajes (root → .uy → fing.edu.uy) = 3000 paquetes/s
    contra los servidores autoritativos, en perpetuo.
  Eso es un ataque de denegación de servicio MUY barato para el atacante:
  un solo PC genera 3000 paquetes/s contra una zona ajena.

CON CACHÉ NEGATIVA:
  el primer NXDOMAIN se cachea 3600 s.
  Los 999 siguientes se responden de la caché:
    0 viajes, 0 bytes, 0 carga para el autoritativo.
  El ataque se vuelve inofensivo.
```

Y hay un beneficio adicional: **el `NXDOMAIN` nunca debería convertirse en `A` por error tipográfico**, así que cachear el error durante un rato **filtra tráfico basura** de forma involuntaryaria.

`RFC 8020` (2018): NXDOMAIN es un error, no un nombre

⚠️ Acá está la norma que **corrige** un malentendido muy extendido:

```
Lo que la RFC PROHÍBE:
──────────────────────

Que un resolver trate un NXDOMAIN como si fuera un nombre VÁLIDO
y lo use para «inventar» respuestas para nombres parecidos.

El caso concreto (domain shadowing):
────────────────────────────────────

Se consulta:      victima.example.com     →  NXDOMAIN
Se cachea:        (victima.example.com → NO EXISTE)

Si un atacante logra que en example.com quede un comodín (*.example.com)
o si más tarde el dominio aparece, un resolver mal implementado puede
seguir sirviendo el NXDOMAIN cacheado y «tapar» un nombre que SÍ existe.
Eso es DOMAIN SHADOWING: un nombre legítimo, sombreado por su ausencia.

Lo que la RFC RECOMIENDA:
─────────────────────────

1. Cachear el NXDOMAIN (sí, se cachea: protege contra el ataque de
   arriba).
2. Guardar la ENTRADA NEGATIVA CON LA CLAVE DEL NOMBRE EXACTO.
   Un NXDOMAIN de victima.example.com NO habilita a responder
   NXDOMAIN de otra.example.com: cada nombre negativo tiene su
   propia entrada y su propio timer.
3. Tratarlo como «este nombre no existe en este momento», no como
   «este nombre no existe para siempre».
```

La contradicción aparente que hay que saber explicar: cachear **sí**, no contaminar **no**. Se cachea **por nombre exacto** y con su propio timer, de modo que el error no se derrame sobre los nombres hermanos.

1.6 ⚠️ Actualización a 2026 — lo que cambió y lo que no

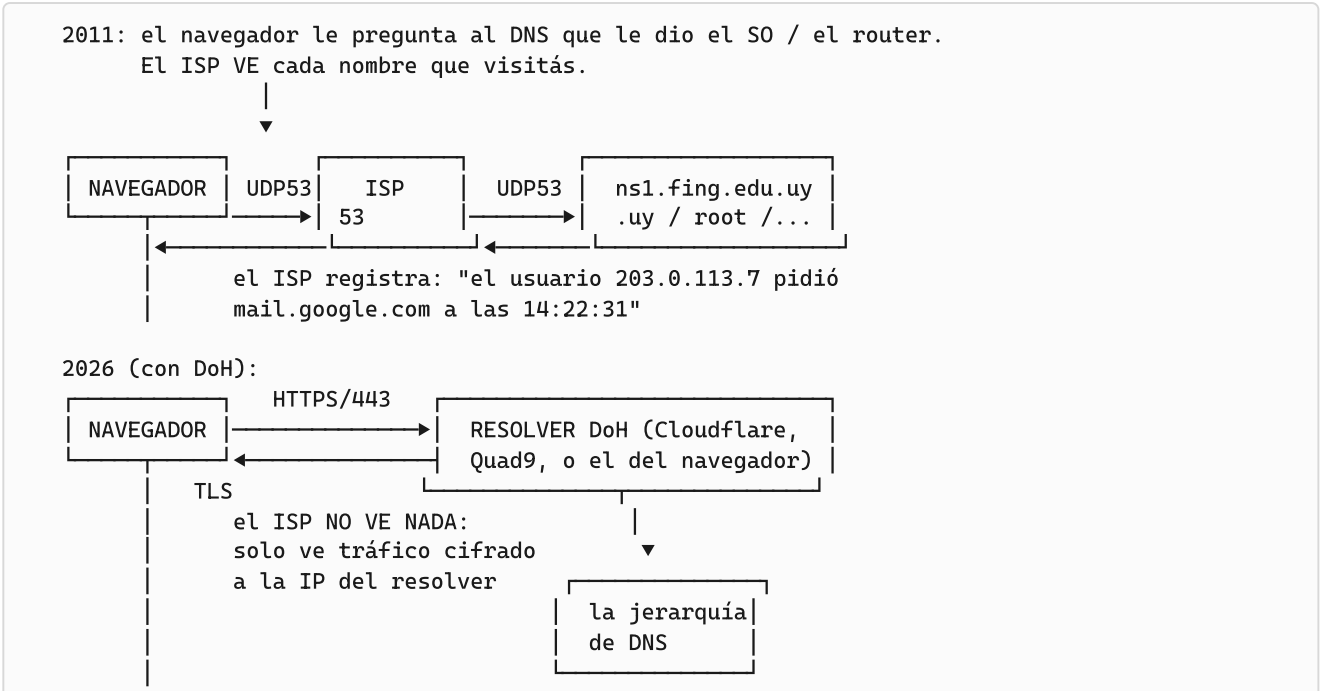
Sigue igual — de memoria, sin dudar:

- **Jerarquía y delegación**: root → TLD → autoritativo, con `NS` y *glue records*.
- **Iterativa vs. recursiva**: el *stub resolver* pregunta **recursivamente** al resolver local; el resolver local pregunta **iterativamente** a root, TLD y autoritativo. **No cambió desde 1987.**
- **Estructura del mensaje**: ID, *flags* (`QR`, `Opcode`, `AA`, `TC`, `RD`, `RA`, `RCODE`), `QDCOUNT` / `ANCOUNT` / `NSCOUNT` / `ARCOUNT`, y las cuatro secciones.
- `RCODE`: NOERROR, FORMERR, SERVFAIL, **NXDOMAIN**, NOTIMP, REFUSED.
- **Tipos de registro**: `A`, `AAAA`, `NS`, `CNAME`, `MX`, `SOA` (con sus cinco números), `PTR`, `TXT`, `SRV`, `CAA`.
- `TTL` **y caché.** El TTL sigue siendo el cuarto campo, en segundos, definido por el autoritativo y descontado por quien cachea.
- **Puertos**: **53 UDP y 53 TCP**, y **853** (DoT, `domain-s`).
- **Round-robin** por varios registros `A`. Chrome y Safari lo aleatorizan por privacidad, y por eso las respuestas no siempre vienen en el mismo orden.

Cambió — y acá hay que ser preciso:

(a) DoH (`RFC 8484`, 2018): el cambio mayor

DNS over HTTPS es DNS **encapsulado dentro del protocolo HTTP**, sobre TLS, en el puerto **443**. El efecto es brutal y hay que saber expresarlo bien:



> ⚠️ **La conclusión que hay que escribir en el examen: con DoH, el DNS del navegador ya no lo ve tu ISP.** Pasa de estar en **UDP 53 plano** —donde todo el camino puede ver y alterar la pregunta— a estar **dentro de un HTTPS**, que es lo mismo que cualquier visita a una web. Y un HTTPS es **la una cosa que el ISP no puede leer sin romper el TLS**.

Lo que DoH cambia de verdad, en tres puntos:

1. **Confidencialidad**: el ISP no ve qué nombres se consultan (sí ve a qué IPs se conectan, salvo con ECH; eso es otra historia).
2. **Integridad**: nadie en el camino puede más inyectar una respuesta falsa (`DNS poisoning` o *pharming*).
3. **El salto a la red sigue siendo UDP 53**: DoH solo cifra el **tramo navegador → resolver**. Las consultas del resolver a root, TLD y autoritativo **siguen en DNS plano** salvo que ese resolver también hable DoT o DoQ.

(b) Chrome NO cambia tu resolver: el matiz que hay que saber

Esto se dice mal en todos lados y es un error de examen fácil de cometer:

```
LO QUE SE DICE (FALSO):
"Chrome usa DNS de Google por defecto"

LO QUE PASA (VERDAD):
┌──────────────────────────────────────────────────────────┐
│ Chrome 83 (mayo-2020) NO cambia tu resolver.             │
│ Intenta "subir" a DoH TU resolver actual, y lo hace SOLO si  │
│ ese resolver está en su LISTA DE PROVEEDORES CONOCIDOS.  │
│ Si no está en la lista → DNS plano, como siempre.        │
│ Si el usuario o el administrador lo configuran a mano →  │
│   puede apuntar a donde quiera.                          │
└──────────────────────────────────────────────────────────┘
Chrome 87 (nov-2020) es el que trae HTTP/3 por defecto; no lo confundas
con el rollout de DoH.
```

> ⚠️ **Si te preguntan «¿el navegador usa el DNS de Google?», la respuesta es «no: Chrome intenta cifrar la consulta con el resolver que ya tenías, y solo si ese resolver está en su lista de proveedores conocidos; si no, sigue en DNS plano».**

(c) DoT (`RFC 7858`, 2016) y DoQ (`RFC 9250`, 2022)

- **DoT** (*DNS over TLS*): DNS cifrado sobre TLS, en el **puerto 853**, con nombre fijo (`domain-s`). Es lo que usan los **resolvers** (no el navegador) y lo que configurás a mano en un servidor.
- **DoQ** (*DNS over QUIC*): DNS cifrado sobre **QUIC**, en el **puerto 853**, usando el `Connection ID` y los streams independientes de QUIC. **DoQ elimina el *head-of-line blocking* de TCP**: si un paquete de una consulta se pierde, no frena a las demás consultas que van en la misma conexión.

```
DoT  = DNS dentro de TLS dentro de TCP       (ordenado, un stream)
DoQ  = DNS dentro de QUIC                   (multiplexado, N streams)
DoH  = DNS dentro de HTTP/2 o HTTP/3        (multiplexado, N streams)
──────────────────────────────────────────────────────────────────
Los tres cifran. Solo DoQ y DoH se benefician de la multiplexación.
```

(d) DNSSEC: estandarizado en 2005, despliegue bajo en 2026

La cadena de confianza de DNSSEC está **operativa**: `RFC 4033-4035` (2005) y `RFC 5155` (raíz firmada, 2010). Los números de despliegue son los que hay que saber:

Métrica	Valor
gTLD firmados	**~99,6 %**
Dominios (2LD) con `DS`	**~5-7 %**

> ⚠️ **La lección, literal:** DNSSEC es el ejemplo perfecto de **«estandarizado en 2005 y con despliegue bajo casi veinte años después»**. El eslabón fuerte (TLD) está casi completo; el eslabón débil (el dominio concreto) casi no lo tomó. Por eso en 2026 **no se enseña que «el DNS es seguro»**, sino que **tiene una capa de autenticidad opcional, bien diseñada y masivamente subutilizada**. Compará con HTTPS: mismo problema, misma forma, mismos veinte años.

`RFC 8080` agrega EdDSA (Ed25519) como algoritmo de firma y `RFC 8901` permite **firmantes múltiples** (*multi-signer*), que es lo que permite una migración de algoritmo sin cortar el servicio.

(e) `RFC 8767` (2020): servir caché vencida (*stale*)

Qué cambió en la semántica de caché. Antes, un registro vencido **se botaba**. **descartado**. Con `RFC 8767`, si el servidor autoritativo **no responde**, el resolver puede **servir la copia vieja**, marcándola como *stale*.

```
ANTES:  TTL vencido + autoritativo caído  →  SERVFAIL, el usuario ve un error.
AHORA:  TTL vencido + autoritativo caído  →  se sirve la copia vieja con un
                                               aviso de que está vencida.

¿POR QUÉ? Porque es resiliencia frente a DDoS: si alguien tumba un
autoritativo, los resolvers con stale siguen sirviendo en vez de caer
todos juntos. Es un compromiso explícito entre «frescura» y «disponibilidad».
```

El riesgo es evidente y hay que decirlo: **el contenido puede estar viejo**. Es un cambio de diseño real en el modelo de caché, no un detalle.

(f) `SVCB` / `HTTPS` (`RFC 9460`, 2023): el DNS transporta política de transporte

Un registro `HTTPS` (que es un `SVCB` en modo 1) dice **qué versiones de HTTP soporta el servidor, en qué IP está, con qué prioridad** — todo **antes** de abrir ningún socket:

```
;; QUESTION SECTION:
;www.fing.edu.uy.               HTTPS

;; ANSWER SECTION:
www.fing.edu.uy.   300  IN  HTTPS 1 . alpn="h3,h2" ipv4hint=132.42.120.5 port=443
```

Por qué importa: es lo que permite **descubrir HTTP/3 antes de conectar**, sin necesidad de haber hecho antes un viaje por HTTP/1.1 para enterarte por `Alt-Svc` (Clase 9). Es **el mismo cambio conceptual que** `CAA`: el DNS dejó de ser solo un directorio de direcciones y pasó a **transportar política** sobre el dominio. `RFC 9462` (DDR) permite descubrir resolvers en `_dns.resolver.arpa` sin configurar nada.

(g) EDNS(0) (`RFC 6891`) y conexiones persistentes (`RFC 7766`)

- `RFC 6891` (2013, *Internet Standard*): agrega el pseudo-registro `OPT`. Sin él, el máximo sigue siendo **512 bytes** y las respuestas grandes se cortan con `TC = 1`. Con él, el cliente anuncia `udp: 4096` y se evita el corte.
- `RFC 7766` (2016): los resolvers **deben mantener las conexiones TCP abiertas y reutilizarlas**; tienen prohibido cerrar la conexión por transacción. Si no, cada consulta por TCP pagaría un handshake.

> **Consecuencia para el examen:** si te preguntan *«¿qué cambió en el DNS?»*, la respuesta es: **nada de la arquitectura**. Ni la jerarquía, ni la delegación, ni el `RR` de cinco campos, ni los dos algoritmos cambiaron desde 1987. Lo que se **agregó** es cifrado (DoH/DoT/DoQ), autenticidad (DNSSEC, subdesplegada), extensiones (EDNS(0)), semántica de caché resiliente (`RFC 8767`) y política de transporte (`SVCB` / `HTTPS`).

1.7 El transporte: UDP, TCP y por qué DoH le ganó a DoT

El DNS normal viaja sobre **UDP 53**, y eso no es un capricho: una consulta es un datagrama chico (menos de 100 bytes) que se manda una sola vez, sin garantía de llegada, y **el reintento lo hace el resolver** si no recibe nada.

Situación	Transporte
Consulta normal	**UDP 53**
La respuesta no entra (banderita `TC = 1`)	se repite por **TCP 53**
Transferencia de zona (`AXFR`, `IXFR`)	**TCP 53**, siempre
DoT	**TCP 853**
DoQ	**UDP 853**
DoH	**TCP o UDP 443** (el mismo 443 de la Web)

```
  El 443 compartido es la razón por la que DoH le ganó a DoT:

  DoT necesita el puerto 853 abierto. En muchas redes corporativas,
URUOs empresas y universidades el 853 está BLOQUEADO, y el resolver
  del ISP a veces ni lo habla.

  DoH usa el 443, que es EL puerto de la Web: si podés abrir una
  página, podés resolver un nombre. Por eso DoH es el que se desplegó
  en el navegador, y DoT se quedó en el mundo de los resolvers.
```

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La cadena completa con caché en el TLD

Enunciado. Un PC en Montevideo resuelve `www.fing.edu.uy`. El resolver recursivo tiene **el `NS` de `fing.edu.uy` cacheado** (con su *glue*), pero **no** tiene el registro `A`. RTT entre el PC y el resolver: **2 ms**; entre el resolver y el root: **40 ms**; entre el resolver y el `.uy`: **25 ms**; entre el resolver y el autoritativo: **15 ms**. Todos los mensajes viajan por UDP, así que cada consulta cuesta **un RTT** y nada más.

a) ¿Cuántos mensajes y cuántos RTT?

```
El resolver tiene el NS de fing.edu.uy cacheado → SE SALTA el root y el .uy.
O sea que el mensaje 2 (al root) y el 5 (al .uy) NO existen.

┌─────┬──────────────────────┬──────────┬──────┬──────────────────────────┐
│  #  │ De                   │ A        │ RTT  │ Tipo                     │
├─────┼──────────────────────┼──────────┼──────┼──────────────────────────┤
│  1  │ PC (stub)            │ resolver │  2 ms│ RECURSIVA (pregunta)     │
│  2  │ resolver             │ ns1.fing │ 15 ms│ ITERATIVA (pregunta)     │
│  3  │ ns1.fing.edu.uy      │ resolver │ ↑ el mismo RTT          │
│  4  │ resolver             │ PC       │ ↑ el mismo RTT          │
└─────┴──────────────────────┴──────────┴──────┴──────────────────────────┘

TOTAL: 4 mensajes, 2 RTT

Cálculo:
RTT_total = RTT(PC ↔ resolver) + RTT(resolver ↔ autoritativo)
          = 2 ms + 15 ms
          = 17 ms
```

Comparación con el caso sin caché (los 4 viajes del §1.3):

```
SIN NINGUNA CACHÉ:                CON CACHÉ DEL NS EN EL TLD:
──────────────────                ──────────────────────────
8 mensajes, 4 RTT                 4 mensajes, 2 RTT

2 + 40 + 25 + 15 = 82 ms          2 + 15 = 17 ms

Ahorro: 65 ms (79 %)              ┌──────────────────────────────────┐
                                  │ La caché no guarda la RESPUESTA,│
                                  │ guarda la DELEGACIÓN. Y la      │
                                  │ delegación es el 90 % del trabajo.│
                                  └──────────────────────────────────┘
```

b) ¿Cuántos RTT si además el `A` estuviera cacheado?

```
Si el registro A está en caché y NO venció:
  el resolver responde al PC sin salir a la red externa.

  ┌─────┬─────────────┬──────────┬─────────┬──────────────────────────┐
  │ 1   │ PC          │ resolver │ 2 ms    │ RECURSIVA                │
  │ 2   │ resolver    │ PC       │ ↑       │ respuesta desde la caché │
  └─────┴─────────────┴──────────┴─────────┴──────────────────────────┘
  TOTAL: 2 mensajes, 1 RTT = 2 ms

Si el registro A está en caché pero VENCIÓ (hay que revalidar):
  el resolver tiene que ir al autoritativo de nuevo.
  RTT_total = 2 ms (PC) + 15 ms (autoritativo) = 17 ms
```

c) ¿Y si el PC tiene el nombre en su propia caché local?

```
Windows, macOS y Linux mantienen una caché de nombres en el cliente
(y el navegador tiene la suya). Si el nombre está fresco ahí:

  RTT = 0.  Ni siquiera sale un paquete a la red.

Esto es lo que hace que «abrir la misma pestaña dos veces» sea instantáneo.
```

d) La tabla que resume los cuatro escenarios:

Qué tiene cacheado el resolver	Mensajes	RTT	Tiempo (con los RTT del enunciado)
Nada (ni el `NS` ni el `A`)	8	4	2+40+25+15 = **82 ms**
El `NS` de `fing.edu.uy` (caso del enunciado)	4	2	2+15 = **17 ms**
El `A`, vencido	4	2	2+15 = **17 ms**
El `A`, fresco	2	1	**2 ms**
El `A`, fresco en la caché del PC	0	0	**0 ms**

Interpretación: **la caché ataca el tiempo, no los bytes.** Notá que en el primer caso y en el segundo se transfieren **los mismos 56 bytes** de respuesta final; lo que cambia es cuántos viajes hay que hacer para CONSEGUIRLOS.

⚠️ Errores típicos

- **Contar 8 RTT en vez de 4.** Los 8 mensajes son 4 parejas ida y vuelta. Cada mensaje suelto no es un RTT.
- **Contar el mensaje 8 como un viaje nuevo.** El mensaje 8 es la vuelta del 1.
- **Creer que el TTL del `NS` y el del `A` son el mismo número.** Son campos distintos, y el `NS` suele tener un TTL mucho más grande.
- **Olvidar que la caché del TLD se guarda con su propio TTL** (típicamente 172800 s = 2 días), así que no sirve para el caso «lo acabo de cambiar».

2.2 Ejercicio 2 — ¿Cuánto dura una entrada de caché? Con las cuentas

Enunciado. El autoritativo de `fing.edu.uy` responde:

```
www.fing.edu.uy.   3600  IN  A   132.42.120.5
fing.edu.uy.     172800  IN  NS  ns1.fing.edu.uy.
```

El resolver guarda la respuesta **a las 10:00:00**. Hacele estas preguntas: **(a)** ¿cuánto tiempo se puede usar sin revalidar? **(b)** ¿qué pasa exactamente a las 11:00:00? **(c)** ¿hasta cuándo vive la entrada del `NS`? **(d)** ¿cuántos viajes ahorra la caché en las **primeras 24 horas** si se pide el nombre **cada 10 segundos**?

a) ¿Cuánto se puede usar sin revalidar?

```
TTL = 3600 segundos = 1 hora.

Frescura restante = 3600 − 0 = 3600 s a las 10:00:00.

t = 10:00:00 ──────────────────────────────── t = 11:00:00
    se guarda                                  vence
    (3600 s de frescura)                       (0 s de frescura)
```

b) Interpretación: entre las 10:00 y las 11:00, **cualquier** consulta se responde de la caché: **0 mensajes, 0 bytes, 0 RTT**.

b) ¿Qué pasa exactamente a las 11:00:00?

```
Las 11:00:00 es el instante límite. La respuesta es:

┌──────────────────────────────────────────────────────────────┐
│ • La entrada NO se borra. Pasa de "fresca" a "VENCIDA"       │
│   (en inglés, stale: vieja pero presente).                   │
│                                                              │
│ • La SIGUIENTE consulta ya NO es un acierto de caché:        │
│   el resolver tiene que revalidar.                           │
│                                                              │
│ • Si el servidor contesta, el resolver reemplaza la entrada: │
│     - si dice "no cambió" (el RR es el mismo, byte a byte),  │
│       RENUEVA 3600 s más.                                    │
│     - si dice "cambió", reemplaza el contenido y el TTL.     │
│                                                              │
│ • Si el servidor NO contesta, hay dos salidas:               │
│     - servir la copia vieja (lo que autoriza RFC 8767)       │
│     - o devolver SERVFAIL (el comportamiento clásico)        │
│                                                              │
│ • Recién AHORA, si pasó lo mismo (vencimiento + revalidación │
│   + no contestó), la entrada se elimina de verdad.           │
└──────────────────────────────────────────────────────────────┘
```

El error conceptual número uno: decir que «a los 3600 segundos se borra del caché». No: a los 3600 segundos deja de ser *fresco*. Borrarse es otra etapa, y la distinción importa justamente porque `RFC 8767` permite usar esa copia vieja.

c) ¿Hasta cuándo vive la entrada del `NS`?

```
TTL del NS = 172800 s = 48 horas = 2 días.

Se guardó a las 10:00:00 del lunes.
Vence:      10:00:00 del MIÉRCOLES.

t = lun 10:00 ──────────────────────────────────────► mié 10:00
                  172800 s = 48 h = 2 días
```

Interpretación: el `NS` vive **48 veces más** que el `A`. Es correcto, porque cambiar el **servidor de nombres** es rarísimo y cambiar la **IP de un servidor web** es de todos los días. Y por eso, si vos cambiás de servidor de nombres, **tenés que esperar dos días** (o bajar el TTL del `NS` con antelación) antes de tocar el glue que ya está en la zona del TLD.

d) ¿Cuántos viajes ahorra la caché en 24 horas, si me piden el nombre cada 10 s?

```
Consultas en 24 horas:
N = 86 400 s / 10 s = 8 640 consultas

Sin caché:
  8 640 × 4 RTT = 34 560 viajes  (y 8 640 × 8 = 69 120 mensajes)

Con caché (TTL 3600 s = 1 hora):
  revalidaciones = 24 h / 1 h = 24
  aciertos de caché = 8 640 − 24 = 8 616

  Viajes = 24 revalidaciones × 4 RTT = 96 RTT
           + 0 RTT de las 8 616 consultas acertadas
           = 96 viajes

AHORRO:
  viajes:     34 560 − 96 = 34 464 viajes  →  99,72 % menos
  mensajes:   69 120 − 192 = 68 928        →  99,72 % menos
```

	Consultas	RTT totales	Mensajes	Carga en el autoritativo
Sin caché	8 640	34 560	69 120	8 640 consultas
Con caché TTL=3600	8 640	**96**	192	**24** consultas
Ahorro	—	**99,72 %**	**99,72 %**	**99,72 %**

Interpretación: **por eso el DNS sobrevive.** Si cada cliente de Internet repitiera los cuatro viajes, los 13 root servers no darían abasto. El TTL es literalmente lo que hace posible la escala del sistema. **Y el precio es la propagación lenta**: un cambio tarda hasta un TTL en ser visto por todo el mundo.

> **La frase del examen:** *el TTL es un compromiso entre la corrección de los cambios y la carga de la red; lo elige el servidor autoritativo, lo descuenta quien cachea, y por eso es siempre un compromiso, nunca una solución.*

⚠️ Errores típicos

- **Decir que el TTL se descuenta en el autoritativo.** No: el autoritativo solo lo **pone**. Quien lo descuenta es el **resolver que lo guardó**.
- **Confundir «vence» con «se borra».** Vencer es dejar de ser fresco; borrar es el paso siguiente, y a veces no ocurre.
- **Contar el TTL en minutos.** Siempre son **segundos**.
- **Asumir que el TTL del `NS` es el del `A`.** No tienen por qué ser iguales, y casi nunca lo son.

2.3 Ejercicio 3 — Por qué `NXDOMAIN` se cachea, y qué dice `RFC 8020`

Enunciado. Un atacante con un solo portátil quiere tumbar el servidor de nombres de `fing.edu.uy`. Su script hace 5 000 consultas por segundo de nombres inexistentes en esa zona, por ejemplo `a1.fing.edu.uy`, `a2.fing.edu.uy`, …, con `TTL` de caché negativa de `SOA.MINIMUM = 3600`. **a)** ¿Con caché negativa, funciona el ataque? **b)** ¿Y si los resolvers de los usuarios **no** cachean los negativos? **c)** ¿Qué dice `RFC 8020` y por qué «cachear sí, contaminar no» no es contradictorio?

a) Con caché negativa, ¿funciona?

```
Nombres atacados:  a1.fing.edu.uy, a2.fing.edu.uy, a3...  (5000 distintos/s)

Sin caché negativa:
  5000 consultas/s × 3 viajes (root → .uy → fing.edu.uy)
  = 15 000 paquetes/s contra los servidores de nombres de fing.edu.uy
 Estimado: 15 000 × ~500 bytes = 7,5 MB/s sostenidos.
  Con un solo portátil eso es nada. Y es 5000 consultas/s de CPU
  de resolución de zona.

CON caché negativa (SOA.MINIMUM = 3600):
  Cada nombre DISTINTO genera un NXDOMAIN y se cachea 1 hora.
  Los 5 000 nombres/s siguen generando entradas nuevas... pero:

  ⚠️ OJO: si el atacante usa 5 000 nombres DISTINTOS por segundo,
     la caché negativa los guarda a todos, y la tabla crece sin
     control. La caché negativa protege del ATAQUE DE REPETIR EL
     MISMO NOMBRE, no del ataque de inundar la tabla con nombres
     siempre nuevos.

El ataque que la caché negativa SÍ detiene:
  5000 consultas/s del MISMO nombre (por ejemplo "noexiste")
    → 1 NXDOMAIN, cacheado 3600 s
    → las 4999 restantes se responden de la caché: 0 viajes.
  ATAQUE NEUTRALIZADO al 99,98 %.
```

Interpretación: la caché negativa es una protección real **contra la repetición**, y por eso `RFC 8020` la recomienda explícitamente. Lo que **no** hace es defenderse de un atacante que genera nombres aleatorios: para eso hacen falta los límites de tasa (*rate limiting*) del lado del atacante (o sea, de tu servidor de nombres).

b) ¿Y si los resolvers no cachean los negativos?

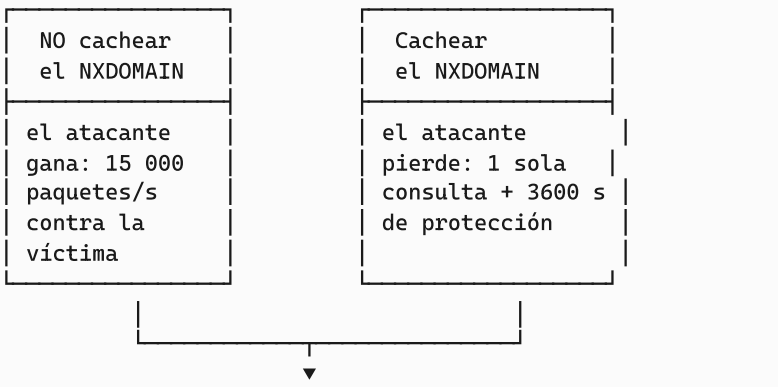
```
El estado se vuelve una máquina de estados:

t=0     atacante empieza          ┌──────────────┐
...     5000 consultas/s          │  RESOLVER    │  ← 15 000 paquetes/s salen
t=60s   el autoritativo           │  (el del ISP)│     hacia la zona de la
       empieza a caer:            │              │     víctima
       CPU al 100 %               └──────┬───────┘
              │                          │
              │    "gRPC: overload"      │
              ▼                          ▼
       la resolución de                LA VÍCTIMA
       fing.edu.uy SE CAE               SE CAE
              │
              ▼
       Y AHORA: los usuarios LEGÍTIMOS
       no pueden resolver NADA de
       fing.edu.uy  ←  esto es DoS real

Con caché negativa: los usuarios legítimos siguen resolviendo
desde la caché durante 1 hora. EL DOMINIO NO SE CAE.
```

Interpretación: **la caché negativa es lo que separa un ataque de una denegación de servicio real.** Con ella, el atacante gasta recursos del atacante; sin ella, gasta recursos de la víctima.

c) ¿Qué dice `RFC 8020` y por qué «cachear sí, contaminar no» no es contradictorio?

```
RFC 8020 — Negative DNS Responses and DNS Cache Pollution (2018)

QUÉ PIDE:
─────────
1. Cachear la respuesta negativa.  ✓ (protege contra la repetición)
2. Tratarla como un ERROR con nombre, NO como un nombre válido.
3. Guardar la entrada bajo la CLAVE DEL NOMBRE EXACTO que falló.

QUÉ PROHÍBE (el *domain shadowing*):
────────────────────────────────────
Que el NXDOMAIN de un nombre se derrame sobre otros nombres.

victima.example.com  →  NXDOMAIN  →  se guarda SÓLO para ese nombre
otra.example.com     →  ?         →  NO puede responder desde ese NXDOMAIN:
                                       tiene que consultarlo por separado

Y por qué esto importa en la práctica: si el NXDOMAIN se guardara
«en general» para el dominio, bastaría con que un nombre fallara una
vez para que TODOS los nombres del dominio quedaran «sombreados»
(shadowed), incluidos los que existen. Eso sería un DoS perfecto y
silencioso, hecho con cero paquetes.
```

La paradoja y su resolución:




```
La respuesta NO es «no cachear», es:
«cachear, pero con la LLAVE del nombre exacto,
 y sin dejar que el error se propague a los hermanos».

CACHEAR  = SÍ     (por nombre exacto)
CONTAMINAR = NUNCA (de un nombre a otro)
```

Interpretación física: la caché negativa es una **bomba de tiempo con reloj propio**: dura lo que diga el TTL del `SOA` (o el `minimum`), y después el nombre vuelve a ser consultable. Cachear de más es lo peligroso (permanece el error cuando el nombre ya existe: *domain shadowing*); cachear de menos es lo inútil (el atacante gana).

⚠️ Errores típicos

- **Decir que el `minimum` del `SOA` es «cuánto dura el NXDOMAIN» sin matizar.** El `minimum` es el **valor por defecto** para el TTL del negativo; si el autoritativo manda un `SOA` con otro TTL, **manda ese**.
- **Confundir `NXDOMAIN` con «el tipo no existe».** `NXDOMAIN` = **el nombre** no existe. Si el nombre existe pero no tiene ese tipo, es `NOERROR` con `ANCOUNT = 0` (NODATA).
- **Pensar que la caché negativa protege contra un atacante con nombres aleatorios.** No: protege contra la repetición. Contra la inundación de la tabla hace falta *rate limiting*.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-07, Pregunta 3 (8 puntos) — resolución iterativa sin caché

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1007.pdf

Enunciado (texto literal del examen)

> **Pregunta 3 (8 puntos)**
>
> **a)** Represente en un diagrama todos los pasos involucrados en la resolución de nombres iterativa donde el equipo necesito.laip.com.uy consulta a su servidor DNS local (dns.laip.com.uy) por la dirección IP asociada a www.fakaofo.tk. Suponga que no se dispone de ninguna información de caché en ningún servidor.
>
> **b)** Indique en el diagrama de la parte a) cuál es el servidor del TLD tk y cuál el autoritativo de fakaofo.tk.

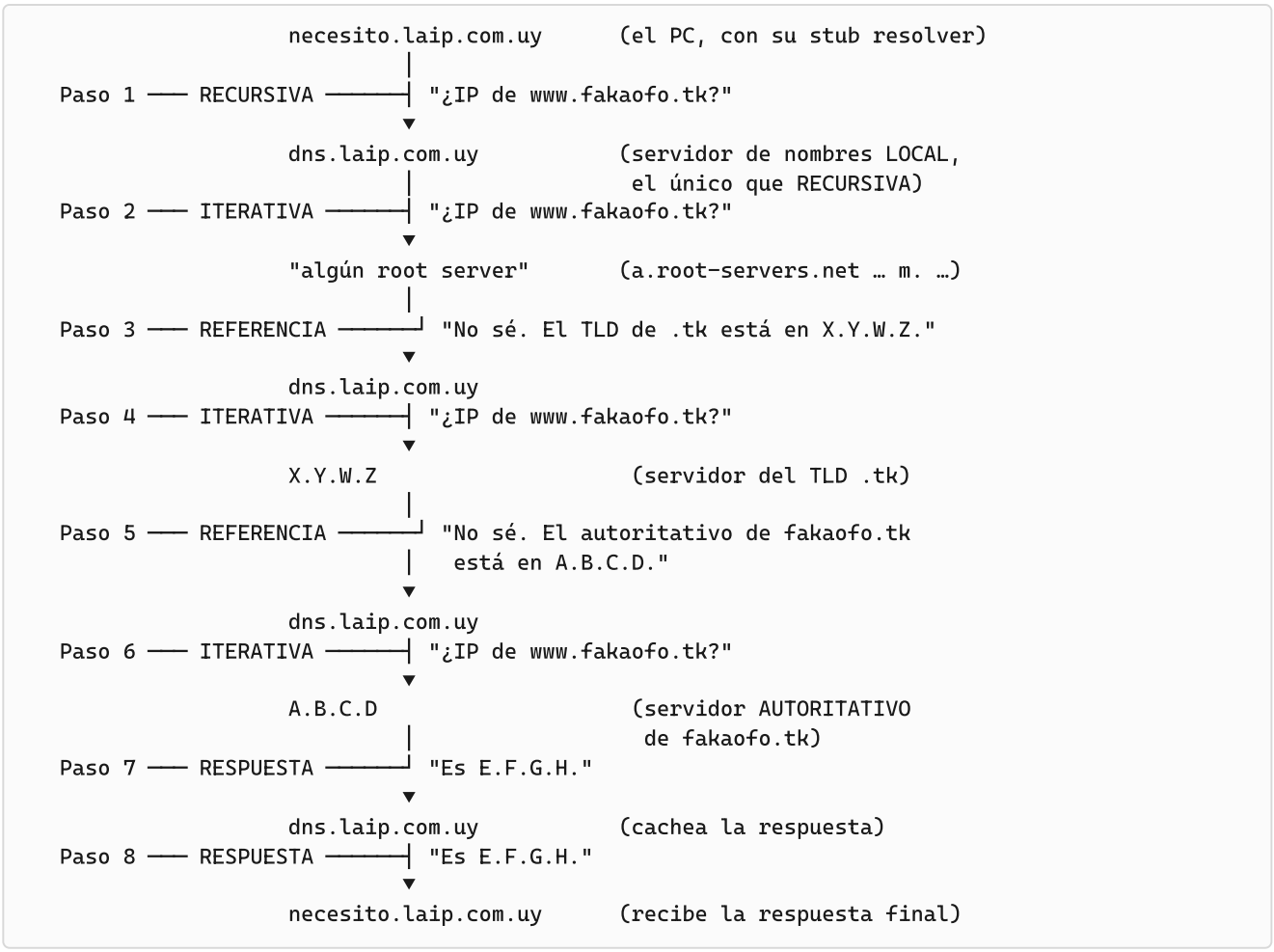
Solución oficial (del PDF, textual)

Paso	A	Información
1	necesito.laip.com.uy	dns.laip.com.uy
2	dns.laip.com.uy	"algún root server"
3	"algún root server" (el que recibió la consulta del paso 2)	dns.laip.com.uy
4	dns.laip.com.uy	X.Y.W.Z
5	X.Y.W.Z	dns.laip.com.uy
6	dns.laip.com.uy	A.B.C.D
7	A.B.C.D	dns.laip.com.uy
8	dns.laip.com.uy	necesito.laip.com.uy

> El diagrama debe ser muy similar al que figura en la diapo 9 de la clase 7 de capa de Aplicación. X.Y.W.Z, A.B.C.D y E.F.G.H son meramente ilustrativos, pero sí deben ser distintos.

a) Resolución — el diagrama completo

Acá está el diagrama pedido, con los cuatro actores y los ocho mensajes. Los nombres son los del examen:



Análisis de la solución oficial, punto por punto. Esto es lo que hay que aprender de ella:

✅ **Lo que está bien:**

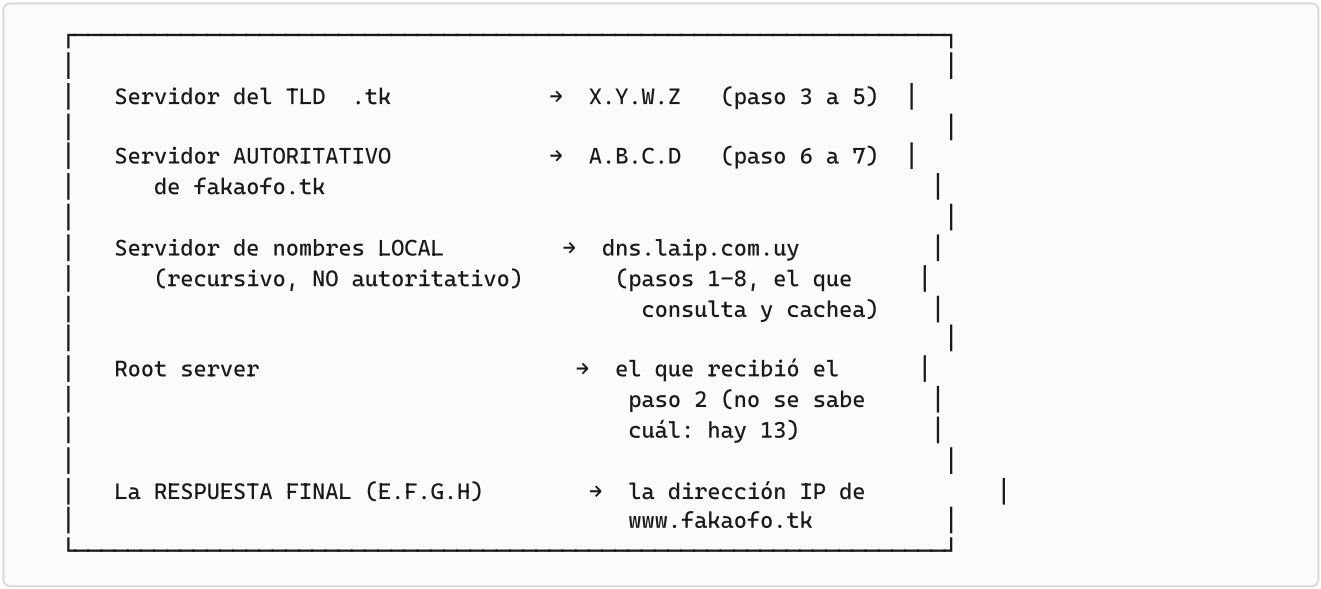
1. **Los ocho mensajes están completos.** No falta ninguno: ni el 2 (al root), ni el 4 (al TLD), ni el 6 (al autoritativo), ni el 8 (la vuelta al PC). Es la estructura completa de §1.3, y coincide con la que da la cátedra.
2. **Los pasos 3 y 5 están bien escritos como *referencias*, no como respuestas.** La oficial dice «Consultar a TLD de tk», «Consultar a dns.fakaofo.tk», que es exactamente la definición de iterativa. Si en alguno de esos dos pasos hubiera puesto «la IP es …», la solución estaría mal.
3. **X.Y.W.Z, A.B.C.D y E.F.G.H son distintos**, como pide el enunciado: eso garantiza que cada paso es una consulta a **otra** máquina y no un invento.
4. **El paso 8 existe**, o sea que la respuesta final vuelve al PC. Muchos alumnos lo olvidan y se quedan en 7 pasos.

⚠️ **Lo que la oficial NO dice y hay que agregar para los 8 puntos:**

1. **No dice qué tipo de consulta es cada paso.** La oficial llama «Query» a los pasos 1, 2, 4 y 6 y «Response» a los 3, 5, 7 y 8. Eso alcanza, pero **suma** distinguir que los **pasos 3 y 5 son respuestas con** `ANCOUNT = 0`, es decir *referencias*, y que **solo el 7 tiene** `ANCOUNT = 1` **y** `AA = 1`.
2. **No dice qué algoritmo es cada paso.** El enunciado dice «resolución iterativa» y la oficial lo resuelve como iterativo en los pasos 2 a 7. Pero el **paso 1 es recursivo** en la realidad, y decir eso **suma puntos** (ver la nota de abajo).
3. **No dice qué se cachea.** El enunciado dice «sin caché en ningún servidor», así que la oficial tiene razón en no tocarlo, pero **alumnos que mencionen qué se cachearía después** demuestran que entendieron.

> **Nota explícita contra la oficial — y es la más importante de esta clase:** el enunciado dice «**resolución de nombres iterativa**» y describe que `necesito.laip.com.uy` **consulta a su servidor DNS local**. En el modelo real, **esa primera consulta es recursiva**: el *stub resolver* del SO le pide al `dns.laip.com.uy` «resolveste esto por mí y traeme la respuesta final», y el flag `RD = 1` de la petición lo dice. Lo **iterativo** son los **cuatro pasos que van del servidor local hacia afuera** (2 a 7). Mi lectura: la oficial usa «iterativa» como abreviatura de «la que hace el servidor local», que es el nombre que se le da en la clase, y **no lo dice**. Si en un examen te piden el diagrama, **hacé los ocho pasos** (que es lo que se corrige) y **agregá la distinción de una línea**: «el paso 1 es recursivo y los 2 a 7 son iterativos». Eso no te resta nada y puede sumarte.

b) ¿Cuál es el TLD de `tk` y cuál el autoritativo de `fakaofo.tk`?



(solución propia: el PDF publica la tabla del apartado a) y remite a la diapo 9 de la clase 7 para el dibujo, pero no redacta la respuesta de b).)

El matiz que hay que agregar y es lo que suma puntos: `dns.laip.com.uy` **no es el autoritativo de nada**. Aunque sea el servidor de nombres de `laip.com.uy` (y puede serlo), frente a este problema está actuando como **resolver recursivo**: es un **cliente** de la jerarquía, no un **dueño** de datos. Su papel en el diagrama es el de un intermediario que consulta y cachea.

Respuesta redactada, lista para copiar en el examen:

> **a)** Con el equipo `necesito.laip.com.uy` y su servidor de nombres local `dns.laip.com.uy`, sin caché en ningún servidor, la resolución de `www.fakaofo.tk` son **ocho mensajes**:
>
> 1. `necesito.laip.com.uy` → `dns.laip.com.uy`: consulta por la IP de `www.fakaofo.tk`.
> 2. `dns.laip.com.uy` → un **root server**: la misma consulta.
> 3. El root **no conoce** `www.fakaofo.tk`; responde con una **referencia**: el servidor del TLD `.tk` está en `X.Y.W.Z`.
> 4. `dns.laip.com.uy` → `X.Y.W.Z` (TLD de `.tk`): la misma consulta.
> 5. El TLD tampoco conoce el dominio; responde con una **referencia**: el servidor autoritativo de `fakaofo.tk` es `dns.fakaofo.tk`, en `A.B.C.D`.

6. `dns.laip.com.uy` → `A.B.C.D` (autoritativo): la misma consulta.
7. El autoritativo **sí** conoce el mapeo y responde: `www.fakaofo.tk` es `E.F.G.H`.
8. `dns.laip.com.uy` → `necesito.laip.com.uy`: devuelve `E.F.G.H`.

Son **cuatro ida y vuelta = 4 RTT**: PC↔resolver (pasos 1 y 8), resolver↔root (2 y 3), resolver↔TLD (4 y 5) y resolver↔autoritativo (6 y 7). Los pasos 3 y 5 son **referencias**, no respuestas: llegan con `ANCOUNT = 0` y el dato en las secciones *Authority* y *Additional* (los `NS` y sus *glue records*). Solo el paso 7 es una respuesta final, y es la única con el flag `AA = 1`.

b) El servidor del TLD `.tk` es `X.Y.W.Z` y el servidor autoritativo de `fakaofo.tk` es `A.B.C.D`. El `dns.laip.com.uy` es el servidor de nombres **local (recursivo)**: no es autoritativo de `fakaofo.tk` ni de ningún otro nombre ajeno; en este problema actúa como un cliente más de la jerarquía, que consulta y cachea.

3.2 ⭐ Examen 2010-12, Pregunta 1 (8 puntos) — el TTL y los pasos de una consulta tipo A

Fuente: Examen `eirc1012` / `sirc1012` — 17 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf **Este es el otro problema central de la unidad.**

Enunciado (texto literal del examen)

Pregunta 1 (8 puntos)

El DNS es una base de datos distribuida y jerárquica, compuesta por varios tipos de servidores.

a) Cuando un servidor de nombres aprende un mapeo, lo cachea. ¿Cuál es el campo del registro de recurso (RR) que decide si se puede cachear o no? ¿Cuál es el servidor que define ese tiempo? ¿El valor que toma ese campo, a qué refiere y en que unidades está expresado?

b) Suponga que desde un PC de una empresa, se hace una consulta tipo A al servidor de nombres local por el nombre pruebaexmen.edu, ¿cuales son los pasos siguientes que debe realizar el servidor antes de dar la respuesta al usuario?

Solución oficial (del PDF, textual)

a) El campo es el TTL, que refiere al tiempo restante por el cual ese RR es válido y por lo tanto se puede mantener en caché. El TTL está definido por el servidor autoritativo de ese RR, y es la cantidad de segundos para los que ese registro es válido. El servidor que lo mantiene en cache, deberá ir decrementando ese valor, hasta que cuando llega a cero, deja de ser válido y se quita del caché.

b) PC => Consulta SERVIDOR LOCAL por TIPO A, nombre pruebaexmen.edu SERVIDOR LOCAL => Consulta ROOT SERVER (no lo tiene en caché) por el servidor TLD del .edu ROOT SERVER responde con la IP del servidor TLD del edu SERVIDOR LOCAL => Consulta TLD del .edu por el RR tipo A, nombre pruebaexamen.edu Servidor TDL responde con la dirección IP del nombre pruebaexamen.edu al

> SERVIDOR LOCAL SERVIDOR LOCAL cachea la respuesta y responde al PC con la dirección IP del nombre pruebaexamen.edu

a) El TTL — resolución

La respuesta oficial es corta y precisa. Lo que hay que hacer es *desarmarla*.

Pregunta	Respuesta oficial	Dónde está en el RR
¿Qué campo decide si se puede cachear?	el **TTL** (*time to live*, tiempo de vida)	el **cuarto** de los cinco campos
¿Qué servidor define ese tiempo?	el **servidor autoritativo** de ese RR	quien escribe la zona
¿A qué refiere su valor?	al **tiempo restante** por el cual ese RR **es válido** y puede estar en caché	contador decreciente
¿En qué unidades?	**segundos**, siempre	4 bytes

La estructura del RR, con el TTL marcado:

```
nombre              clase   tipo    TTL      datos
fing.edu.uy.        IN      A       3600     132.42.120.5
─────────────       ─────   ────    ────     ──────────
nombre              2 bytes  2 bytes 4 bytes  variable
                             1       2
                                  ↑
                    EL TTL: 4 bytes, en SEGUNDOS
```

El razonamiento detrás de cada respuesta, que es lo que hay que saber explicar:

1. **¿Por qué el TTL y no otro campo?** Porque los otros cuatro **no dependen del tiempo**: el nombre es el nombre, la clase es `IN`, el tipo es `A` y los datos son la IP. **Solo el TTL es una declaración sobre cuánto tiempo vale lo que dice el registro.** Y por eso es el único que puede «envejecer»: al pasar el tiempo, el contenido sigue siendo cierto como hecho histórico, pero deja de ser *garantizado* por el origen.
2. **¿Por qué el autoritativo y no otro?** Porque **el TTL es una declaración de política del dueño del dato**, no una decisión técnica de quien lo transporta. Solo el que administra la zona sabe **con qué frecuencia cambia** su contenido. Si lo definiera el resolver, cada resolver tendría su propia cuenta y el sistema se desincronizaría.
3. **¿Por qué «tiempo restante» y no «fecha de vencimiento»?** Porque el TTL es un **contador**, no una fecha. No hay reloj en el protocolo: no se manda «vence el 12/10 a las 15:00», se manda «3600 segundos». Cada quien que lo guarda lo cuenta **hacia abajo** desde el momento en que lo recibió. Es lo que permite que dos resolvers en distintos puntos del mundo coincidan sin sincronizar relojes.
4. **¿Por qué segundos?** Porque es la **única unidad que se Define en el formato del mensaje**: el campo ocupa 4 bytes y se interpreta como un entero sin signo de 32 bits. Cualquier otra unidad requeriría un campo de unidades, que no existe.

Respuesta redactada, lista para copiar en el examen:

> **a)** El campo es el **TTL** (*time to live*), que es el **cuarto campo** del registro de recurso y ocupa **4 bytes**.

> **Quién lo define:** el **servidor autoritativo** de esa zona, porque es el dueño del dato y es el único que sabe con qué frecuencia cambia. El TTL es una **declaración de política** del dueño del nombre.
>
> **A qué refiere:** al **tiempo restante** durante el cual ese registro es **válido** y, por lo tanto, puede mantenerse en la caché. No es una fecha absoluta sino un **contador decreciente**: el resolver que lo guarda lo va descontando desde el momento en que lo recibió, y cuando llega a cero el registro deja de ser fresco y obliga a revalidar.
>
> **Unidades: segundos**, siempre, sin excepción. El valor es un entero sin signo de 32 bits, así que el máximo es 4 294 967 295 segundos (unos 136 años).
>
> **Ejemplo:** `www.fing.edu.uy. 3600 IN A 132.42.120.5` significa «este registro es válido durante 3600 segundos (1 hora) a partir de ahora». Guardado a las 10:00:00, queda fresco hasta las 11:00:00; a las 11:00:01 hay que revalidar.
>
> **Nota contra la oficial:** la oficial es **correcta y completa** para el apartado a). Lo único que le agregaría es (i) que el TTL es el **cuarto** campo de los **cinco**, y que mide **4 bytes**, porque el enunciado habla de «el campo del RR» y conviene situarlo; y (ii) el **máximo representable**, que es un dato que a veces se pregunta y sale del formato del mensaje.

b) Los pasos de una consulta tipo A — resolución

Acá hay que tener cuidado, porque la solución oficial tiene un error conceptual. Primero la resolución completa y después el análisis.

El nombre del problema: `pruebaexamen.edu`. Descomponemos:

```
pruebaexamen.edu.
──────────────── ───
2° nivel (2LD)   | TLD  (.edu es un gTLD "genérico",
= "pruebaexamen"  |  no es el ccTLD de un país)
                  ▼
raíz  →  .edu  →  pruebaexamen.edu  →  (el nombre buscado)
```

El recorrido completo, con los mismos ocho mensajes que en `2010-07 P3`:

```
┌──────┬──────────────────┬──────────────────┬──────────────────────────────┐
│  #   │ De               │ A                │ Qué pasa                     │
├──────┼──────────────────┼──────────────────┼──────────────────────────────┤
│  1   │ PC (stub)        │ servidor local   │ RECURSIVA: "¿IP de           │
│      │                  │ (el de la        │  pruebaexamen.edu?"          │
│      │                  │  empresa)        │  RD=1                        │
│  2   │ servidor local   │ root server      │ ITERATIVA: "¿IP de           │
│      │                  │                  │  pruebaexamen.edu?"          │
│  3   │ root server      │ servidor local   │ REFERENCIA: "el TLD .edu     │
│      │                  │                  │  está en <IP del .edu>"      │
│  4   │ servidor local   │ TLD .edu         │ ITERATIVA: la pregunta       │
│  5   │ TLD .edu         │ servidor local   │ REFERENCIA: "el autoritativo │
│      │                  │                  │  es ns1.pruebaexamen.edu,    │
│      │                  │                  │  en <IP del autoritativo>"   │
│  6   │ servidor local   │ autoritativo     │ ITERATIVA: la pregunta       │
│  7   │ autoritativo     │ servidor local   │ RESPUESTA: "es <IP>"         │
│  8   │ servidor local   │ PC               │ RESPUESTA: "es <IP>"         │
└──────┴──────────────────┴──────────────────┴──────────────────────────────┘

RTT: 2 (PC↔local) + 1 (local↔root) + 1 (local↔.edu) + 1 (local↔aut.) = 4 RTT
→ 8 mensajes.
```

El paso 9, que el enunciado pide explícitamente y que es el que da el punto:

```
9. El servidor local ARMA LA RESPUESTA y se la entrega al PC.

   ⚠️ Y ACA ESTÁ EL ERROR DE LA OFICIAL: el texto dice
   "Servidor TDL responde con la dirección IP del nombre
    pruebaexamen.edu al SERVIDOR LOCAL".

   El TLD .edu NO PUEDE responder con la dirección IP. No la tiene.
   El TLD .edu lo único que tiene de pruebaexamen.edu es el REGISTRO
   NS que dice quién lo autoriza. Si el TLD devolviera la IP, el
   sistema entero colapsaría en el TLD y no habría DNS distribuido.

   El TLD responde con una REFERENCIA (ANCOUNT=0), y la respuesta
   final la da el AUTORITATIVO en el paso 7.
```

Lo que hace el servidor local antes de responder (los 3_CACHEAR):

```
1. Cachea el REGISTRO A que vino del autoritativo,
   con el TTL que trae (que es el que definió el autoritativo).

2. Cachea la REFERENCIA del TLD .edu: el NS de pruebaexamen.edu
   y su glue (IP de ns1.pruebaexamen.edu), con el TTL que trae
   el TLD (típicamente 172800 s = 2 días).

   └── esto es lo que hace que la SEGUNDA resolución del mismo
       nombre tarde solo 2 RTT y no 4.

3. Cachea la REFERENCIA del root: el NS de .edu y su glue, con
   su TTL. Esta se cachea mucho menos porque el root tiene muchísimos
   TLD y la rotación es alta.

y recién después le responde al PC.
```

Respuesta redactada, lista para copiar en el examen:

> **b)** Desde el PC de la empresa se hace una consulta **tipo A** al servidor de nombres local por `pruebaexamen.edu`. El nombre se resuelve **de derecha a izquierda**, y son estos los pasos:
>
> 1. El PC envía la consulta al **servidor de nombres local** de la empresa («¿qué dirección IPv4 corresponde a `pruebaexamen.edu` ?»). Es una consulta **recursiva**: el stub resolver del sistema operativo le pide al servidor local que resuelva por él.
> 2. El servidor local, **sin tener el dato en caché**, hace una consulta **iterativa** a un **root server** por el mismo nombre.
> 3. El root server **no conoce** `pruebaexamen.edu` : no conoce ningún dominio, solo los TLD. Responde con una **referencia**: el servidor del TLD `.edu` está en determinada dirección IP.
> 4. El servidor local consulta **iterativamente** al **servidor del TLD** `.edu` .
> 5. El TLD `.edu` **tampoco conoce** el dominio. Responde con una **referencia**: el servidor **autoritativo** de `pruebaexamen.edu` es `ns1.pruebaexamen.edu` , cuya dirección IP viene en la sección *Additional* (el *glue record*).
> 6. El servidor local consulta **iterativamente** al **servidor autoritativo** de `pruebaexamen.edu` .
> 7. El autoritativo **sí** conoce el mapeo y responde con el **registro A** de `pruebaexamen.edu` , con su **TTL**.

8. El servidor local **cachea** la respuesta (el `A` con su TTL, y las referencias del TLD y del root con los suyos) y **le responde al PC** con la dirección IP.

Son **ocho mensajes** y **cuatro RTT**. Los pasos 3 y 5 son **referencias**, no respuestas (llegan con `ANCOUNT = 0`); la **única respuesta final** es la del paso 7, que es la única con el flag `AA = 1`.

Nota explícita contra la oficial — y es una diferencia real: La solución oficial describe **cinco pasos** y en el paso 5 dice que *«Servidor TDL responde con la dirección IP del nombre pruebaexamen.edu al SERVIDOR LOCAL»*. **Mi lectura es que ahí hay un error conceptual:** el servidor del TLD **no puede** responder con la dirección IP, porque **no la tiene y no puede tenerla** — si la tuviera, no habría DNS distribuido, todo el mundo dependería de los TLD y el sistema no escalaría. Lo que el TLD devuelve es el **registro** `NS` que lo delega, es decir, **quién es el autoritativo** (más su *glue*), y **a partir de ahí hay un paso más**: la consulta al servidor autoritativo, que es el único que devuelve el registro `A`. Por eso la cadena completa son **ocho mensajes (cuatro RTT)** y no cinco: es exactamente la misma cadena del `2010-07 P3`. Si el enunciado acepta cinco pasos porque es lo que se da en la clase, **reproduce la tabla oficial** y **agregá una línea con el paso que falta**: eso no te resta nada y muestra que entendiste la jerarquía.

3.2.1 Los otros tres puntos de la misma pregunta de 2010-12 (por completitud)

La Pregunta 1 del examen de diciembre de 2010 vale 8 puntos y la cátedra la suele usar **entera** como pregunta de concepto. Los otros tres son de otros temas, pero conviene tenerlos frescos porque se repreguntan:

Apartado	En una línea
P2a.1 ¿Cómo deriva un segmento UDP a su proceso?	por el **puerto destino**
P2a.2 ¿Cómo sabe el proceso a dónde mandar la respuesta?	por **IP origen + puerto origen**
P2a.3 ¿Qué identifica un socket UDP en la red?	**IP del host + número de puerto** (la «2-tupla», simplificación clásica)
P2b ¿Y un socket TCP?	la **4-tupla**: IP origen, puerto origen, IP destino, puerto destino

(Se desarrolla entero en la Clase 16, «UDP: multiplexación».)

3.3 Examen 2007-07, Pregunta 4 (8 puntos) — función del DNS y protocolo involucrado

Fuente: Examen `eirc0707` / `sirc0707` — julio de 2007.

Enunciado: Describa la función del sistema de nombres de dominio y qué protocolo se utiliza para la resolución de nombres.

Resolución (síntesis de §1.1, §1.2 y §1.7 de esta clase y de la Clase 10).

a) La función: el DNS es un **espacio de nombres jerárquico** y una **base de datos distribuida** que traduce **nombres de host a direcciones IP** y viceversa (`PTR`). Es **jerárquico** (raíz → TLD → autoritativo, resuelto de derecha a izquierda) y **distribuido** (cada servidor solo conoce los datos de su zona y delega el resto), lo que le da escala, tolerancia a fallas y mantenimiento distribuido de la autoridad.

b) El protocolo: un protocolo de **capa de aplicación** que viaja sobre **UDP puerto 53** en el caso normal, y sobre **TCP puerto 53** cuando la respuesta no cabe en un datagrama (banderita `TC`) o cuando se pide una **transferencia de zona** (`AXFR` / `IXFR`). El mensaje tiene **12 bytes de cabecera** (ID, *flags*, y los contadores `QDCOUNT` / `ANCOUNT` / `NSCOUNT` / `ARCOUNT`) más cuatro secciones: **Question**, **Answer**, **Authority** y **Additional**.

Lo que hay que agregar y suma puntos: el protocolo usa **consultas iterativas y recursivas**, y el flujo real es que el **stub resolver del host hace una consulta recursiva al resolver local**, y el resolver local hace **consultas iterativas** al root, al TLD y al autoritativo. **Y con DNSSEC (`RFC 4033–4035`, 2005) el mismo protocolo transporta además firmas digitales**, aunque el despliegue de esa capa es parcial: hoy están firmados ~99,6 % de los gTLD pero solo ~5-7 % de los dominios.

PARTE 4 — CIERRE

📌 Resumen

1. **Iterativa**: el que no sabe devuelve una **referencia** («preguntale a este») y no hace nada más; **quien sigue preguntando es el cliente**. **Recursiva**: el que no sabe **pregunta él** al nivel inferior; **quien sigue preguntando es el servidor**.
2. **El flujo real**: el *stub resolver* del SO → **recursiva** → resolver local; el resolver local → **iterativas** → root → TLD → autoritativo. **No cambió desde 1987.**
3. **Son 8 mensajes y 4 RTT** sin caché. Los mensajes 3 y 5 son **referencias** (`ANCOUNT = 0`, dato en *Authority* + *Additional* con los *glue*); solo el 7 es respuesta final (`AA = 1`).
4. **La caché existe** porque sin ella cada visita a una página generaría 2 a 6 consultas, y los 13 root no darían abasto. **Ahorra viajes, no bytes.**
5. **El TTL es el 4.º campo del RR, de 4 bytes, en segundos.** Lo define el **servidor autoritativo**; lo **descuenta** el resolver que lo cacheó, y cuando llega a cero la entrada deja de ser fresca y hay que **revalidar**.
6. **El TTL es un compromiso**: corto → los cambios se propagan rápido pero el origen recibe muchas consultas; largo → el origen recibe pocas consultas, pero los cambios tardan hasta un TTL en propagarse.
7. **La caché negativa** cachea el `NXDOMAIN` con el TTL del `SOA` (o el `SOA.MINIMUM`), y eso **protege** contra ataques de denegación por repetición. `RFC 8020` dice: cachear **sí**, pero **bajo la clave del nombre exacto**, sin contaminar (*domain shadowing*).
8. ⚠️ **Actualización a 2026: DoH (`RFC 8484`)** ya es el cambio mayor —el DNS del navegador **ya no lo ve tu ISP**—; **Chrome no cambia tu resolver**, intenta «subir» el que ya tenías a DoH si está en su lista de proveedores conocidos, con *fallback* a DNS plano. **DoT (`RFC 7858`, 853)** y **DoQ (`RFC 9250`)** van por UDP/TCP 853. `RFC 8767` permite servir caché vencida. **`SVCB` / `HTTPS` (`RFC 9460`)** hace que el DNS transporte política de transporte. **DNSSEC** (`RFC 4033–4035`): ~99,6 % de los gTLD firmados, solo **~5-7 %** de los dominios: **estandarizado en 2005, despliegue bajo veinte años después.**
9. **Puerto 53 UDP y TCP**, y **853** para DoT. DoH usa el **443**, que es justamente por lo que se desplegó más: si podés abrir una web, podés resolver un nombre.
10. **Si te preguntan «qué cambió en el DNS»**, la respuesta es **nada de la arquitectura**. Lo que se agregó son cifrado, autenticidad parcial, extensiones, una semántica de caché resiliente y política de transporte.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Cuál es la diferencia entre una consulta iterativa y una recursiva? En una línea cada una.

<details>
<summary>Respuesta</summary>
</details>

2. En el flujo real, ¿qué consulta es recursiva y cuáles son iterativas?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Cuántos mensajes y cuántos RTT tiene una resolución sin caché? ¿Cuáles de esos mensajes son respuestas y cuáles son referencias?

▶ Respuesta

4. ¿Cuál es el campo del RR que decide si se puede cachear? ¿Quién lo define? ¿Qué unidades tiene?

▶ Respuesta

5. ¿Qué es la caché negativa y por qué es una **protección** y no un problema?

▶ Respuesta

6. ¿Qué es DoH y por qué «Chrome usa el DNS de Google por defecto» es falso?

▶ Respuesta

7. El TTL es 3600 y el registro se guardó a las 10:00:00. ¿Cuándo deja de ser fresco? ¿Y cuándo se borra de la caché?

▶ Respuesta

8. Un PC resuelve `www.fing.edu.uy`: 2 ms hasta su resolver, 40 ms al root, 25 ms al TLD y 15 ms al autoritativo. ¿Cuánto tarda (a) sin ninguna caché, (b) con el `NS` de `fing.edu.uy` cacheado, (c) con el `A` cacheado y fresco?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1007` / `sirc1007`** — Pregunta 3 (8 pts): ★ resolución iterativa completa de `www.fakaofo.tk` sin caché, y qué servidor es el TLD y cuál el autoritativo
- **Examen `eirc1012` / `sirc1012`** — Pregunta 1 (8 pts): ★ (a) el TTL del RR, quién lo define, a qué refiere y unidades; (b) los pasos de una consulta tipo A a `pruebaexamen.edu`
- **Examen `eirc1002` / `sirc1002`** — Pregunta 1b (8 pts): iterativas vs. recursivas
- **Examen `eirc0707` / `sirc0707`** — Pregunta 4 (8 pts): función del DNS y protocolo involucrado
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.4 («El sistema de nombres de dominio»), §7.4.2 (espacio de nombres jerárquico) y §7.4.3 (resolución de nombres)
- **RFC 1034** — *Domain Names: Concepts and Facilities* (1987): el diseño completo, que **no cambió** · **RFC 1035** — *Implementation and Specification* (1987): el formato del mensaje y de los RR
- **RFC 8484** — DNS over HTTPS, DoH (2018) · **RFC 7858** — DNS over TLS, DoT (2016, puerto 853) · **RFC 9250** — DNS over QUIC, DoQ (2022, sin *head-of-line blocking*)
- **RFC 8020** — *Negative DNS Responses and DNS Cache Pollution* (2018): `NXDOMAIN` se cachea, pero por nombre exacto
- **RFC 8767** — servir caché vencida cuando el autoritativo no responde (2020)
- **RFC 6891** — EDNS(0) (2013, *Internet Standard*): el pseudo-registro `OPT` · **RFC 7766** — conexiones TCP de DNS persistentes y reutilizables (2016)

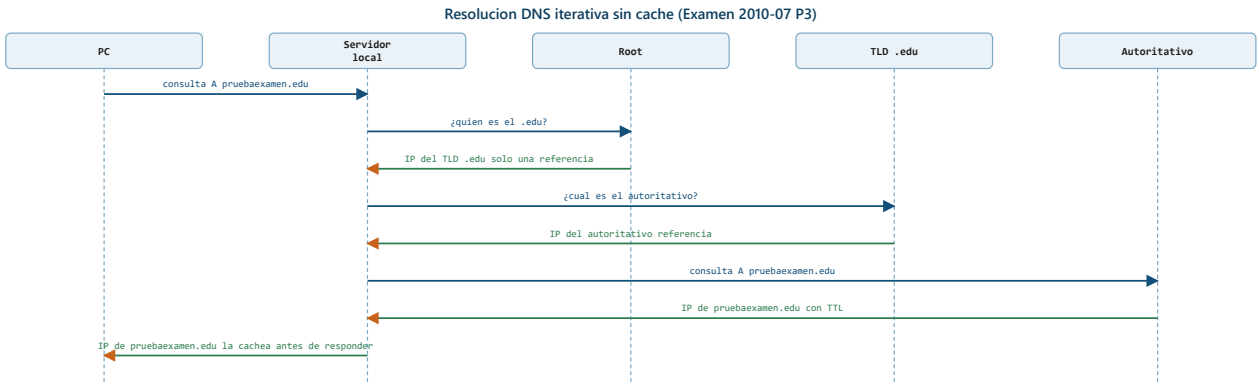
- **RFC 9460** — SVCB/HTTPS (2023): el DNS transporta política de transporte y descubre HTTP/3 antes de conectar · **RFC 9462** — DDR, descubrir resolvers en `_dns.resolver.arpa`
- **RFC 4033-4035** — DNSSEC (2005) · **RFC 5155** — raíz firmada (2010) · **RFC 8080** — EdDSA · **RFC 8901** — firmantes múltiples
- **IANA Service Name and Transport Protocol Port Number Registry** — 53/udp, 53/tcp, 853/tcp (DoT)
- **Clase anterior:** 10 — DNS: la base de datos distribuida
- **Clase siguiente:** 12 — El resolver DNS en la práctica, con sockets

Clase 12

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 La caja copiable
1.2 El diagrama de decisión: ¿hay caché o no?
1.3 Los cuatro flags que hay que mirar
1.4 ⚠️ Actualización a 2026
Mapa de los ejercicios
2.1 Ejercicio 1 (guía) — resolución DNS con caché en el servidor local
2.2 Ejercicio 2 (guía) — intercambio HTTP completo que termina en `304`
2.3 Ejercicio 5 — HTTP no persistente con 1 HTML y 10 imágenes
2.4 Ejercicio 6 — todo lo que pasa al abrir una página
2.5 Ejercicio 7 (extra) — los cuatro escenarios de caché
2.6 Ejercicio 8 (extra) — validación condicional con `ETag` hasta el `304`
2.7 Ejercicio 9 (extra) — RTT de una página con 20 objetos
2.8 Ejercicio 10 (extra) — ¿qué pasa si el TTL es `0`?
2.9 Ejercicio 11 (extra) — por qué el `NXDOMAIN` se cachea
3.1 ⭐ Examen 2010-12, Pregunta 1 (8 puntos) — el TTL y los pasos de una consulta tipo A
3.2 ⭐ Examen 2010-07, Pregunta 3 (8 puntos) — resolución iterativa sin caché
3.3 La otra pregunta de HTTP del mismo examen
3.4 Dónde están los puntos de la unidad
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: B (Aplicación) · **Semana:** 4 · **Duración estimada:** 110 min **Correlativa:** Clase 11 (DNS: iterativo vs. recursivo, caché y TTL) · **Sigue con:** Clase 13 (correo electrónico: SMTP, POP3, IMAP y MIME) **Material de referencia:** Tanenbaum cap. 7 (§7.4) · RFC 1034, RFC 1035, RFC 6891, RFC 8020, RFC 8767, RFC 9110, RFC 9112 · Exámenes `2010-12` P1, `2010-07` P1 y P3



🎯 Al terminar esta clase vas a poder

1. Calcular **mensajes y RTT** de una resolución DNS en los cuatro escenarios de caché, con la cuenta a la vista.
2. Escribir la **transcripción literal** de los intercambios de una resolución sin caché, distinguiendo **referencia** de **respuesta final** por los flags y los contadores.
3. Escribir un **intercambio HTTP completo que termina en `304`**, y decir qué se ahorra y qué **no** se ahorra.
4. Contar **RTT y conexiones TCP** de una página con 11 objetos, en un servidor o en varios, en persistente y en no persistente, secuencial y en paralelo.
5. Reconstruir **paso a paso** todo lo que pasa al abrir una página: DNS, TCP, TLS, HTTP y las imágenes.

6. Resolver **entero** el Examen `2010-12 P1` y el `2010-07 P3`, detectando dónde la oficial acierta y dónde tiene un error conceptual.

> **Prerequisito del examen:** `2010-12 P1` (8 puntos — ★ el TTL del RR y los pasos de una consulta tipo A), `2010-07 P3` (8 puntos — ★ la resolución iterativa completa sin caché), `2010-07 P1` (8 puntos — HTTP no persistente vs. persistente), `2007-07 P4` (8 puntos — función del DNS y protocolo involucrado), `2010-02 P1b` (iterativa vs. recursiva).
>
> ⚠️ **Aviso de método:** esta clase es **taller puro**. La teoría ya la viste en las Clases 8 a 11. Acá hay **recap de fórmulas**, **once ejercicios resueltos** y **tres problemas de examen**. Si algo del recap te suena raro, volvé a la Clase 11 antes de hacer los ejercicios: el taller no reemplaza la teoría, la usa.

PARTE 1 — RECAP: LA MECÁNICA QUE VAMOS A USAR

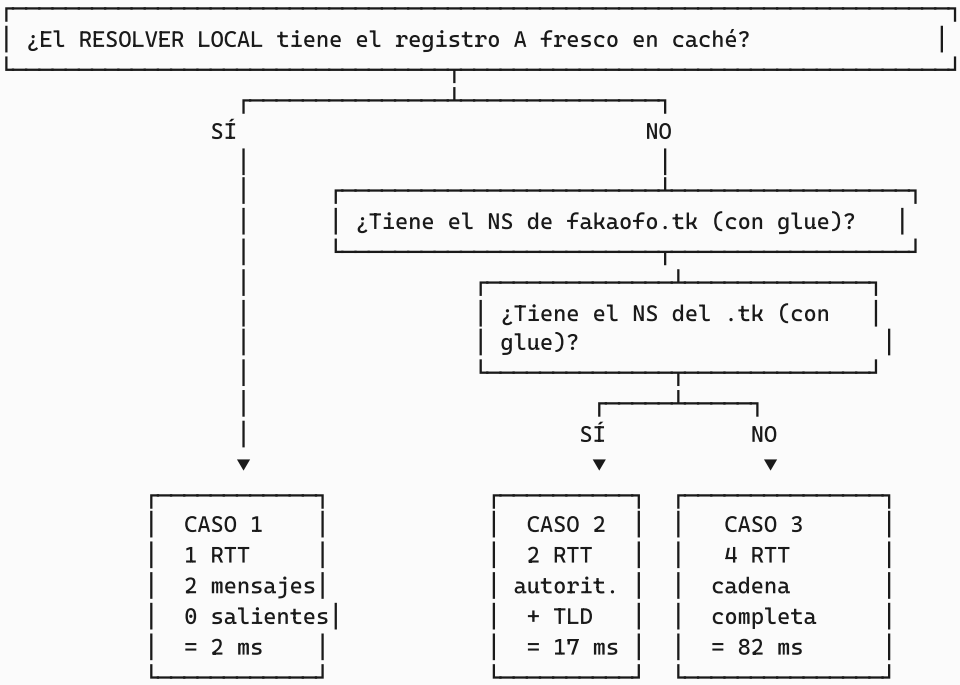
1.1 La caja copiable

Esta caja es lo único que hay que memorizar de esta clase.

```
╔═══════════════════════════════════════════════════════════════════════╗
║  DNS — LA CAJA DE FÓRMULAS                                            ║
╠═══════════════════════════════════════════════════════════════════════╣
║  mensajes         = 2 × RTT       (un viaje = ida + vuelta = 1 RTT)   ║
║  RTT              = RTT(PC↔res) + RTT(res↔root)                       ║
║                                 + RTT(res↔TLD) + RTT(res↔autoritativo)║
║                                 ... solo los que se necesiten         ║
║  frescura restante = TTL − Age                                        ║
║  bytes de un A    = 18 + 2 + 2 + 4 + 2 + 4              =  32 bytes   ║
║                      nombre tipo clase TTL rdlength IPv4              ║
║  acierto de caché  = 0 viajes    ·   falla de caché  = 4 viajes       ║
╠═══════════════════════════════════════════════════════════════════════╣
║  HTTP — LA CAJA DE FÓRMULAS                                           ║
╠═══════════════════════════════════════════════════════════════════════╣
║  no persistente     : RTT = N_obj × (1 handshake + 1 petición) = 2·N_obj ║
║  persistente, 1 srv : RTT = 1 + N_obj                                 ║
║  persistente, k srv : RTT = k + N_obj                (secuencial)     ║
║  ahorro             = N_obj − k                                       ║
║  conexiones TCP     : N_obj (no persistente) | k (persistente)        ║
║  304                : 0 bytes de cuerpo, pero el RTT se paga igual    ║
╚═══════════════════════════════════════════════════════════════════════╝
```

Por qué la primera fórmula dice `mensajes = 2 × RTT`. Un **RTT** (*Round-Trip Time*) es una **pareja** de mensajes: uno de ida y uno de vuelta. «El servidor local consulta al root» **son dos paquetes**, y por eso el intercambio completo son **ocho mensajes y cuatro RTT**. El error más común de la unidad va siempre en la misma dirección: **contar de más**.

1.2 El diagrama de decisión: ¿hay caché o no?



> **El atajo, y es lo que hay que decir en voz alta antes de contar: cada nivel que está cacheado en el resolver es un viaje que no se hace.** El resolver no «sabe menos» y contesta igual: **sabe lo mismo**, porque se lo guardó. Lo que cambia es **a quién tiene que preguntarle**. Y la frase que corrige el malentendido más común: **la caché no guarda solamente la respuesta, guarda las DELEGACIONES.** Por eso saltarse el root y el TLD funciona aunque todavía **no** tengas cacheado el `A`.

1.3 Los cuatro flags que hay que mirar

El discriminador entre una referencia y una respuesta no es el lenguaje: son **tres números**. Una **referencia** (la del root y la del TLD) llega con `ANCOUNT = 0` y `AA` ausente, y el dato en **Authority** + **Additional**. La **respuesta final** (la del autoritativo) llega con `ANCOUNT = 1` y `AA = 1`, y el dato en **Answer**.

Bandera	Quién la pone	Qué dice
`QR`	el que responde	0 = consulta, 1 = respuesta
`AA` (*Authoritative Answer*)	el que responde	1 = «esta es la respuesta final, la tengo yo»
`RD` (*Recursion Desired*)	el que pregunta	1 = «si no sabés, averiguatelo vos»
`RA` (*Recursion Available*)	el que responde	1 = «acepto hacer de recursivo»
`TC` (*Truncated*)	el que responde	1 = «no entró, repetí por TCP»

Un **root server** responde siempre con `AA = 0` y `RA = 0`; un **resolver**, con `RA = 1` y `AA = 0` (la respuesta que devuelve al cliente no le pertenece: la tiene cacheada); un **autoritativo**, con `AA = 1`.

1.4 ⚠️ Actualización a 2026

Sigue igual — de memoria: la jerarquía root → TLD → autoritativo; los dos algoritmos; los ocho mensajes y los cuatro RTT; los flags; el TTL como cuarto campo del RR, en segundos; los puertos **53 UDP y 53 TCP**.

Lo que cambió, y acá alcanza con dos cosas. DoH (`RFC 8484`, 2018): si estás en una red con DoH, **el `dig` de la terminal ya no ve lo que ve el navegador** (se verifica en la línea `;; SERVER:`). Y `RFC 6891` **— EDNS(0)**: sin el pseudo-registro `OPT` la respuesta corta en 512 bytes con `TC = 1` y hay que repetir por TCP, y eso **cambia el conteo de RTT**.

> **Consecuencia para el examen:** los ejercicios de esta clase **no cambian en 2026**; lo que se pregunta es la teoría de 1987. Si el enunciado pide el intercambio de mensajes, hacelo como en 2011, y sumá, si te queda lugar, que con DoH ese primer tramo viaja cifrado dentro de HTTPS.

PARTE 2 — TALLER RESUELTO

Mapa de los ejercicios

En la Parte 2 (§2.x): 1, resolución DNS **con** caché comparada con sin caché · 2, intercambio HTTP que termina en `304` · 5, HTTP **no persistente** con 1 HTML + 10 imágenes en uno y en dos servidores · 6, la tabla de **todo** lo que pasa al abrir una página · 7 a 11, los cinco extras (cuatro escenarios de caché, validación condicional con `ETag`, RTT de 20 objetos, TTL `0` y `NXDOMAIN` cacheado). **En la Parte 3 (§3.x):** 3, ★ **Examen** `2010-12 P1` (el TTL y los pasos de una consulta tipo A) · 4, ★ **Examen** `2010-07 P3` (resolución iterativa sin caché de `www.fakaofo.tk`).

2.1 Ejercicio 1 (guía) — resolución DNS con caché en el servidor local

Datos. `necesito.laip.com.uy` consulta a `dns.laip.com.uy` por la IP de `www.fakaofo.tk`. RTT: PC ↔ resolver **2 ms**; resolver ↔ root **40 ms**; resolver ↔ TLD `.tk` **25 ms**; resolver ↔ autoritativo **15 ms**. El resolver **tiene cacheado** el registro `A`, con **TTL = 3600 s**, guardado **hace 600 s**.

a) ¿El registro está fresco?

Razonamiento. La frescura es un contador que baja, y la cuenta va con los números adentro:

```
frescura restante = TTL − tiempo transcurrido
                  = 3600 − 600
                  = 3000 segundos
```

3000 s > 0 ⇒ fresco ⇒ acierto de caché. Si la cuenta diera negativo, habría que revalidar y el ejercicio cambiaría entero.

b) ¿Cuántos mensajes y cuántos RTT?

Son **dos mensajes**: el PC → resolver (la pregunta, recursiva, 2 ms) y el resolver → PC (la respuesta, que sale de la caché). En total, **1 RTT = 2 ms**.

Los mensajes literales (el segundo sale de la caché, sin tocar la red):

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x1A2B
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;www.fakaofo.tk.               IN      A
;; ANSWER:
www.fakaofo.tk.   3000  IN  A   E.F.G.H
;; MSG SIZE  rcvd: 56
```

Mirá el TTL: dice `3000`, no `3600`. Mandó 3600 el autoritativo, pero contesta el resolver, y responde **con la frescura que le queda**.

c) Comparación con el caso sin caché, y cuántos RTT ahorra.

```
SIN NINGUNA CACHÉ (8 mensajes, 4 RTT):
  RTT = 2 + 40 + 25 + 15
      = 82 ms

CON EL A CACHEADO Y FRESCO (2 mensajes, 1 RTT):
  RTT = 2 ms

AHORRO:
  mensajes: 8 − 2  =  6 mensajes menos
  RTT:      4 − 1  =  3 RTT menos
  tiempo:   82 − 2 = 80 ms menos
  porcentaje: 80 / 82 = 0,9756 = 97,6 %
```

Interpretación física: los 6 mensajes ahorrados son los tres viajes hacia la jerarquía; lo único que cruzó el enlace fue la pregunta del PC y su respuesta. Y notá lo que **no** se ahorra: los bytes de la respuesta final **son los mismos**. **La caché ataca el tiempo y la carga, no el tamaño.**

d) ¿Y si pasan 600 segundos más? A `t = 3 600 s` la frescura llega a 0: **vence**, pero **no se borra** (pasa a *stale*). La consulta siguiente tiene que **revalidar**, y como el `NS` de la zona sigue cacheado, eso cuesta `2 + 15 = 17 ms`.

⚠️ Errores típicos

- **Contar 4 RTT cuando el `A` está cacheado** (si la respuesta sale de la caché, no hay ningún viaje a la jerarquía), **contar el TTL del origen en vez del restante**, o decir que «a los 3600 segundos se borra del caché» (no: deja de ser fresco).

2.2 Ejercicio 2 (guía) — intercambio HTTP completo que termina en `304`

Datos. El usuario ya cargó `www.fing.edu.uy` y tiene en su caché `/css/estilo.css`, servido con `Cache-Control: max-age=600`, `ETag: "css-9c1f-6518a1f0"` y `Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT`. Pasaron **40 minutos**: el `max-age` venció, pero el archivo **no cambió**.

Mensaje 1 — la petición, con las dos condiciones:

```
GET /css/estilo.css HTTP/1.1
Host: www.fing.edu.uy
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
Accept: text/css,*/*;q=0.1
Accept-Encoding: gzip, deflate
If-None-Match: "css-9c1f-6518a1f0"
If-Modified-Since: Mon, 05 Oct 2026 09:14:07 GMT
Connection: keep-alive
```

Mensaje 2 — la respuesta:

```
HTTP/1.1 304 Not Modified
Date: Mon, 05 Oct 2026 10:02:41 GMT
ETag: "css-9c1f-6518a1f0"
Cache-Control: max-age=600
Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT
Vary: Accept-Encoding
Connection: keep-alive
```

Y no hay nada más. Ni cuerpo, ni `Content-Length`, ni `Content-Type`.

El razonamiento. El cliente manda la **condición** (`If-None-Match` es la versión del `ETag`; si van las dos, **manda la del `ETag`**, porque distingue dos versiones del mismo segundo y la fecha no). El servidor **compara**, ve que el `ETag` es el mismo y responde **`304`**: «no te mando el cuerpo, usá la copia que ya tenés». No es un error ni un redireccionamiento, es éxito. El cliente usa su copia local y **no descarga un byte**.

Lo que **NO** lleva un `304` es el cuerpo, y por eso tampoco `Content-Length` ni `Content-Type`. Lo que **SÍ** lleva es el `ETag` y el `Cache-Control`, y este último **renueva la frescura**: el reloj vuelve a cero. Y el **RTT se paga igual**, porque es una respuesta del servidor: el viaje no se evita.

La cuenta de bytes. Cada línea cuenta sin el `CRLF` (2 bytes), más 2 de la línea en blanco que separa cabeceras y cuerpo:

```
DEL 200
"HTTP/1.1 200 OK"                              15
"Date: Mon, 05 Oct 2026 09:14:07 GMT"          35
"Content-Type: text/css; charset=utf-8"        37
"Content-Length: 18422"                        21
"Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT" 50
"ETag: "css-9c1f-6518a1f0""                    25
"Cache-Control: max-age=600"                   26
"Vary: Accept-Encoding"                        21
"Connection: keep-alive"                       22
suma = 252   bytes = 252 × 2 + 2 = 506

DEL 304   (7 líneas: pierde Content-Type y Content-Length)
suma = 25+35+25+26+50+21+22 = 204
bytes = 204 × 2 + 2 = 410

AHORRO:
  antes:  506 + 18 422 de cuerpo = 18 928 bytes
  ahora:  410 +          0        =    410 bytes
  ahorro = 18 928 − 410 = 18 518 bytes   (410 / 18 928 = 2,2 %)
```

Interpretación, y es la frase del ejercicio: el `304` es un **cambio de orden de magnitud** en ancho de banda (pagás el 2 %), pero **el retardo no mejoró nada**. **El `304` ahorra ancho de banda, no retardo**: para matar el retardo hay que eliminar el viaje, y eso lo hacen HTTP/2, HTTP/3 y el CDN.

⚠️ Errores típicos

- **Poner `Content-Length` en el `304`**, decir que **ahorra el RTT**, o creer que **renueva el TTL del origen** (renueva la **frescura en la caché**).

2.3 Ejercicio 5 — HTTP no persistente con 1 HTML y 10 imágenes

Datos. La página tiene **11 objetos** (1 HTML + 10 imágenes). Todos caben en un solo segmento TCP (suposición de la cátedra, y hay que suponerla en voz alta). Cada ida y vuelta cuesta **1 RTT**.

Caso A — los 11 objetos en **un** servidor

```
NO PERSISTENTE
objeto 1 (HTML)    : 1 RTT TCP  + 1 RTT HTTP =  2 RTT
objeto 2 (img 1)   : 1 RTT TCP  + 1 RTT HTTP =  2 RTT
...                                              ...
objeto 11 (img 10) : 1 RTT TCP  + 1 RTT HTTP =  2 RTT
TOTAL = 11 × (1 + 1) = 11 × 2 = 22 RTT
conexiones TCP abiertas = 11

PERSISTENTE
conexión TCP (una sola, para los 11 objetos) :  1 RTT
objeto 1 (HTML)                              :  1 RTT
objetos 2 a 11 (las 10 imágenes)             : 10 RTT
TOTAL = 1 + 11 = 12 RTT
conexiones TCP abiertas = 1

AHORRO = 22 − 12 = 10 RTT = N_obj − k = 11 − 1
```

```
NO PERSISTENTE (22 RTT)      PERSISTENTE (12 RTT)
[TCP][HT] [TCP][HT] [TCP]...  [TCP]-[HT]-[im]-[im]-[im]-----[im]----[im]
 2 RTT   2 RTT   2 RTT          └1┘ └1┘ └──── 10 RTT ──────────┘
  ×11 = 22 RTT                  = 1 + 11 = 12 RTT
```

Interpretación: el no persistente gasta **11 handshakes**, y los handshakes son **tiempo muerto**: no transportan nada útil. El persistente gasta uno y reutiliza la tubería.

Caso B — los objetos repartidos en **dos** servidores

El HTML y 5 imágenes están en **S1** (`www.ejemplo.uy`); las otras 5 en **S2** (`cdn.ejemplo.uy`), otro dominio y por lo tanto **otra dirección IP y otra conexión**. Lo que eso agrega: **una resolución DNS más**. Una página real cuesta de 2 a 6 consultas DNS antes de descargar un solo byte.

Secuencial (un objeto a la vez): `11 × 2 = 22 RTT` no persistente; y con `k = 2` servidores, `2 handshakes + 11 peticiones = 13 RTT` persistente.

En paralelo (lo que hacen los navegadores, que abren varias conexiones):

```
[S1: handshake ── 1 RTT ── GET html ── 1 RTT ──► se ve el HTML]

y EN PARALELO, apenas el navegador parseó el HTML:
[S1: GET img1..img5 ── 1 RTT ──►]   (la conexión de S1 ya estaba abierta)
[S2: handshake ── 1 RTT ──►]  [S2: GET img6..img10 ── 1 RTT ──►]

Los dos renglones del medio corren a la vez, así que el segundo renglón
tarda 2 RTT, no 3.

PERSISTENTE EN PARALELO    = 2 + 2 =  4 RTT
NO PERSISTENTE EN PARALELO = 2 + 2 =  4 RTT   ← ¡empate!
```

Los cuatro escenarios quedan así: secuencial no persistente **22 RTT** con uno o dos servidores; secuencial persistente **12 RTT** con uno y **13 RTT** con dos; en paralelo **4 RTT** en no persistente y **3 RTT** (un servidor) o **4 RTT** (dos) en persistente.

Las dos conclusiones que hay que saber decir, porque son pregunta de examen:

1. **En paralelo empatan o casi.** La ventaja del persistente es **secuencial**: en paralelo los handshakes ya se pagaban a la vez, y lo que queda es el **recurso** (menos sockets y menos puertos efímeros).
2. **Más servidores tienen un costo escondido:** un handshake más (en secuencial) **y una resolución DNS más** (siempre).

2.4 Ejercicio 6 — todo lo que pasa al abrir una página

Datos. RTT = **50 ms** (Montevideo a Estados Unidos). Página **HTTPS** con **1 HTML y 10 imágenes**, todo en **un** servidor, **primera visita** (nada cacheado). Cada objeto pesa **50 KB** y el ancho de banda del cliente es **10 Mbps**.

a) HTTP/1.1 persistente:

#	Etapa	Qué ocurre	RTT	ms
1	**DNS**	4 RTT: resolver → root → TLD → autoritativo, sin caché	**4**	200
2	**TCP**	handshake de 3 vías	**1**	50
3	**TLS**	handshake de TLS 1.3, una ida y vuelta	**1**	50
4	**HTTP**	`GET /index.html` → `200 OK` con el HTML	**1**	50
5	**HTTP**	10 imágenes, persistentes, en la misma conexión	**10**	500
		TOTAL, persistente	**17**	**850**

```
RTT_total = 4 (DNS) + 1 (TCP) + 1 (TLS) + 1 (HTML) + 10 (imágenes) = 17 RTT
tiempo    = 17 × 50 ms = 850 ms
```

b) HTTP no persistente: DNS 4 RTT; el objeto 1 con su handshake 2 RTT; y las 10 imágenes **cada una con su propio handshake**, `10 × (1 + 1) = 20` RTT. Total `4 + 2 + 20 = 26 RTT = 1 300 ms`.

⚠️ **Fijate en el número:** muchos alumnos cuentan 27. El handshake del objeto 1 ya está incluido en los 2 RTT del HTML. La fórmula es `N_obj × (1 + 1)`, y `2 × 11 = 22`, no `2 × 11 + 1`.

c) ¿Cuántas conexiones TCP se abren? En persistente, **1** (que sirve para los 11 objetos). En no persistente, **11**, una por objeto, con sus sockets y sus puertos efímeros. El ahorro son 10 conexiones.

d) Y los bytes, que no son RTT pero también son tiempo. Los datos totales son `11 × 50 KB = 550 KB`, pero **el tiempo que percibís no es el de transmitirlos**: es el que tarda en llegar el **último bit**. El primero tarda `50 KB / 10 Mbps = 40 ms` después de 1 RTT; el último, `40 ms + 10 × 50 ms = 540 ms`.

```
┌──────────────────────────────────────────────────────────────┐
│  850 ms totales                                              │
│    ├─ 200 ms  DNS        (23 %)  ← caché del resolver        │
│    ├─  50 ms  TCP        ( 6 %)  ← reutilización de la conexión
│    ├─  50 ms  TLS        ( 6 %)  ← reanudación (0-RTT)       │
│    ├─  50 ms  HTML       ( 6 %)                              │
│    └─ 500 ms  imágenes  (59 %)  ← paralelismo y HTTP/2       │
└──────────────────────────────────────────────────────────────┘
```

La moraleja: los RTT dominan la latencia; los bytes no. Es la razón por la que HTTP/2 (`RFC 9113`) y HTTP/3 (`RFC 9114`) no tocan el ancho de banda: **ataquan la cantidad de esperas**.

e) ¿Y si fuera la segunda visita? Con el DNS cacheado en el resolver, la conexión viva por `keep-alive`, el TLS reanudado y los recursos con `max-age` fresco, **cada uno de esos puntos vale 0 RTT**: la página entera se ve de

memoria.

2.5 Ejercicio 7 (extra) — los cuatro escenarios de caché

Datos. Los del Ejercicio 1: `www.fakaofo.tk` desde `necesito.laip.com.uy` contra `dns.laip.com.uy`, con 2, 40, 25 y 15 ms.

Escenario	Qué tiene cacheado el resolver	A quién consulta	Mensajes	RTT	Tiempo
A	**Nada**	root, TLD, autoritativo	8	4	**82 ms**
B	El `NS` de `.tk` con su *glue*	TLD, autoritativo	6	3	**42 ms**
C	El `NS` de `fakaofo.tk` con su *glue*	autoritativo	4	2	**17 ms**
D	El `A` de `www.fakaofo.tk`, fresco	nadie	2	1	**2 ms**

```
A)  Se saltea nada:          RTT = 2 + 40 + 25 + 15 = 82 ms
B)  Se saltea el root:       RTT = 2 + 25 + 15 = 42 ms
C)  Se saltea root y TLD:    RTT = 2 + 15 = 17 ms
D)  acierto de caché:        RTT = 2 ms
```

Las tres observaciones que valen puntos. (1) **Cada nivel cacheado quita exactamente un viaje:** 4 → 3 → 2 → 1, que es el diagrama de §1.2 con otras palabras. (2) **El salto mayor no es el último** (el `A` ahorra 80 de los 82 ms, la zona 65), pero lo que **mantiene el sistema en pie** es cachear la **delegación**, porque eso se cachea para **millones** de nombres y el `A` hay que cachearlo uno por uno. (3) **El estado normal es el escenario B:** el resolver siempre tiene cacheados los `NS` de los TLD, así que el A casi no existe después del primer arranque.

⚠️ **Ojo con el nombre de los escenarios:** llamarlos «caché en el TLD» es medio mentiroso, porque **la caché está siempre en el resolver**. Lo que se guardó fue la delegación **que el TLD le devolvió**.

2.6 Ejercicio 8 (extra) — validación condicional con `ETag` hasta el `304`

Datos. Un recurso `/datos.json` de **80 000 bytes**, servido con `ETag: "d7a1-4f9c"` y `Cache-Control: max-age=60`. El cliente hace **tres visitas** separadas por **90 segundos**. El recurso cambia entre la segunda y la tercera.

```
Visita 1 (t = 0)    : sin copia local → GET sin condiciones
                      → 200 OK con 80 000 bytes; se guarda cuerpo + ETag

Visita 2 (t = 90 s) : la frescura (60 s) VENCIÓ, pero hay ETag
                      → GET con If-None-Match: "d7a1-4f9c"
                      → NO cambió → 304, 0 bytes de cuerpo → AHORRO: 80 000 bytes

Visita 3 (t = 180 s): la frescura volvió a vencer (el 304 la renovó)
                      → el recurso CAMBIÓ → ETag "d7a1-5110" → 200 OK completo
```

```
petición con If-None-Match = 180 bytes
respuesta 200 = 180 cabeceras + 80 000 cuerpo = 80 180 bytes
respuesta 304 = 120 cabeceras +         0 cuerpo =    120 bytes

Visita 1:  180 + 80 180 = 80 360
Visita 2:  180 +    120 =    300
Visita 3:  180 + 80 180 = 80 360
──────────────────────────────────────────────────────
Total real en 3 visitas        = 161 020 bytes
Sin validación (siempre 200)  : 3 × 80 360 = 241 080
AHORRO = 241 080 − 161 020 = 80 060 bytes  (33,2 %)

Y los RTT: 1 + 1 + 1 = 3, idéntico al de las 3 descargas completas.
```

Interpretación: el `304` no mejora el retardo, solo el ancho de banda. Cambia 80 000 bytes por 120 y deja los RTT donde estaban.

¿Y sin `ETag`? Solo queda la fecha, y `If-Modified-Since` falla si el archivo cambió **dentro del mismo segundo**: el cliente mostraría una versión vieja. El `ETag` identifica el contenido, así que el `304` es siempre correcto, y por eso HTTP le da prioridad a `If-None-Match` cuando están las dos.

2.7 Ejercicio 9 (extra) — RTT de una página con 20 objetos

Datos. 21 objetos (1 HTML + 20 imágenes), 1 RTT por ida y vuelta, 3 servidores.

```
(1) 1 servidor, no persistente, secuencial
    = 21 × (1 RTT handshake + 1 RTT petición) = 21 × 2 = 42 RTT

(2) 1 servidor, persistente, secuencial
    = 1 RTT (un handshake) + 21 RTT (una ida y vuelta por objeto) = 22 RTT
    AHORRO = 42 − 22 = 20 RTT = N_obj − k = 21 − 1

(3) 3 servidores, persistente, en paralelo
    Renglón 1: handshake S1 (1) + GET html (1)                  = 2 RTT
    Renglón 2: los 3 handshakes EN PARALELO (1) + las 20 peticiones
               EN PARALELO (1)                                   = 2 RTT
    TOTAL = 2 + 2 = 4 RTT

(4) 3 servidores, no persistente, en paralelo
    Renglón 1: handshake S1 + GET html                           = 2 RTT
    Renglón 2: handshakes y peticiones, todo en paralelo         = 2 RTT
    TOTAL = 4 RTT   ← IGUAL que el persistente
```

Con 6 conexiones por host, un servidor y paralelismo, el persistente baja a **3 RTT**.

Las tres conclusiones. (1) **El paralelismo importa mucho más que la persistencia:** 42 → 4 RTT paralelizando, 42 → 22 persistiendo. **Orden de magnitud 10 a 1.** (2) **Repartir en más servidores cuesta `k − 1` handshakes extra** en secuencial (`3 + 21 = 24` contra `1 + 21 = 22`) **y una resolución DNS más** por servidor distinto; en paralelo los handshakes extra no cuestan nada. (3) **El límite del paralelismo es el número de conexiones por host:** con 21 objetos y 6 conexiones hacen falta `⌈21 / 6⌉ = 4` tandas, y **cada tanda necesita su propio handshake**: 4 × 2 = 8 RTT, no 4.

2.8 Ejercicio 10 (extra) — ¿qué pasa si el TTL es `0`?

Datos. El autoritativo responde `www.fing.edu.uy. 0 IN A 132.42.120.5`.

```
a) ¿Se puede cachear?
   frescura restante = TTL − Age = 0 − 0 = 0
   0 NO es > 0  →  NO es fresco  →  NO se puede usar de la caché.

b) ¿Qué hace el resolver, entonces?
   Cada consulta tiene que salir a la red:

   PC ⟶ resolver ⟶ autoritativo ⟶ resolver ⟶ PC
   1        2              3             4          5

   4 mensajes, 2 RTT (el NS de la zona sigue cacheado).
   Y no hay validación condicional: en DNS no existe el 304. Cada vez que
   vence, se vuelve a bajar por la jerarquía.

c) ¿Y si el TTL = 0 es el de un NS?
   Peor: el resolver NO PUEDE saber a quién preguntar por el dominio, así que
   va al root Y al TLD en cada consulta: 8 mensajes, 4 RTT. El peor caso
   posible, y casi siempre un error de configuración.
```

¿Tiene sentido un `TTL = 0`? Significa «no me confíes, preguntá cada vez», y tiene sentido si el dato cambia minuto a minuto y uno viejo es **peor** que un viaje extra. Es **error** en un `NS` o «por descuido».

La asimetría que hay que recordar: el `minimum` del `SOA` es el **TTL por defecto de las respuestas negativas**. Ponerlo en `0` **desactiva la caché negativa**, que es la protección contra los ataques por repetición de nombres inexistentes (Ejercicio 11). **El `TTL = 0` en un positivo desactiva el acierto de caché; el `minimum = 0` desactiva la protección.**

2.9 Ejercicio 11 (extra) — por qué el `NXDOMAIN` se cachea

Datos. Un atacante con **un** portátil hace **5 000 consultas por segundo** de nombres inexistentes bajo `fing.edu.uy`, cuya zona tiene `SOA` con `minimum = 3600`.

```
a) SIN caché negativa
   Cada consulta fallida genera 3 viajes:
     resolver → root → resolver → .uy → resolver → fing.edu.uy → resolver

   5 000 consultas/s × 3 viajes = 15 000 paquetes/s contra los servidores de
   nombres de fing.edu.uy, en perpetuo: unos 7,5 MB/s sostenidos desde un
   portátil. Nada para el atacante, la muerte para una zona pequeña.

b) CON caché negativa
   El PRIMER "noexiste.fing.edu.uy" genera un NXDOMAIN cacheado 3600 s.
   Los 4 999 siguientes salen de la tabla:

   1 consulta a la red + 4 999 desde la caché = 0,02 % de la carga original.
   EL DOMINIO SIGUE EN PIE. El ataque es inofensivo.

c) ¿Y 5 000 nombres DISTINTOS por segundo?
   La tabla CRECE sin control: la caché negativa protege contra la REPETICIÓN DEL
   MISMO NOMBRE, y contra la inundación hace falta rate limiting.
```

Lo que dice `RFC 8020` (2018): cachear sí, contaminar no. Cachear la respuesta negativa, **tratarla como un error con nombre** y **guardarla bajo la clave del nombre exacto** que falló. Lo que **prohíbe** es el *domain shadowing*: que el `NXDOMAIN` de `victima.example.com` se derrame sobre `otra.example.com`. Si se guardara «en general», bastaría que un nombre fallara **una** vez para que todos quedaran sombreados, incluidos los que existen: una denegación de servicio silenciosa y hecha con **cero paquetes**.

El resumen del punto: con caché, un nombre que existe y uno que no existe **cuestan 0 viajes** después del primero; sin caché, 4 y 3 viajes por consulta. Al atacante que **repite** lo neutraliza al 99,98 %; al que **inventa** nombres lo deja llenar la tabla, y para eso hace falta *rate limiting*.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-12, Pregunta 1 (8 puntos) — el TTL y los pasos de una consulta tipo A

> **Fuente:** Examen `eirc1012` / `sirc1012` — 17 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf

Enunciado (texto literal del examen)

> **Pregunta 1 (8 puntos)**
>
> El DNS es una base de datos distribuida y jerárquica, compuesta por varios tipos de servidores.
>
> **a)** Cuando un servidor de nombres aprende un mapeo, lo cachea. ¿Cuál es el campo del registro de recurso (RR) que decide si se puede cachear o no? ¿Cuál es el servidor que define ese tiempo? ¿El valor que toma ese campo, a qué refiere y en que unidades está expresado?
>
> **b)** Suponga que desde un PC de una empresa, se hace una consulta tipo A al servidor de nombres local por el nombre pruebaexmen.edu, ¿cuales son los pasos siguientes que debe realizar el servidor antes de dar la respuesta al usuario?

(El enunciado escribe `pruebaexmen.edu` en el apartado a) y `pruebaexamen.edu` en el apartado b), y la solución oficial mezcla las dos grafías: es una errata del examen.)

Solución oficial (del PDF, textual)

> **a)** El campo es el TTL, que refiere al tiempo restante por el cual ese RR es válido y por lo tanto se puede mantener en caché. El TTL está definido por el servidor autoritativo de ese RR, y es la cantidad de segundos para los que ese registro es válido. El servidor que lo mantiene en cache, deberá ir decrementando ese valor, hasta que cuando llega a cero, deja de ser válido y se quita del caché.
>
> **b)** PC => Consulta SERVIDOR LOCAL por TIPO A, nombre pruebaexmen.edu SERVIDOR LOCAL => Consulta ROOT SERVER (no lo tiene en caché) por el servidor TLD del .edu ROOT SERVER responde con la IP del servidor TLD del edu SERVIDOR LOCAL => Consulta TLD del .edu por el RR tipo A, nombre pruebaexamen.edu Servidor TDL responde con la dirección IP del nombre pruebaexamen.edu al SERVIDOR LOCAL SERVIDOR LOCAL cachea la respuesta y responde al PC con la dirección IP del nombre pruebaexamen.edu

a) El TTL — resolución

La oficial son cuatro frases y las cuatro son correctas. Lo que hay que hacer es *desarmarlas*, porque cada palabra es medio punto.

Lo que pregunta el enunciado	Lo que responde la oficial	Dónde está
¿Qué campo decide si se puede cachear?	el **TTL**	el **cuarto** de los cinco campos, 4 bytes
¿Qué servidor lo define?	el **servidor autoritativo** de ese RR	quien escribe la zona
¿A qué refiere su valor?	al **tiempo restante** por el cual el RR es válido	contador decreciente
¿En qué unidades?	**segundos**	entero sin signo de 32 bits

```
nombre              clase   tipo    TTL      datos
fing.edu.uy.        IN      A       3600     132.42.120.5
————————————       ——————  ————    ————     ——————————
nombre              2 bytes  2 bytes 4 bytes  variable
                                1       2
                                        ↑
                              EL TTL: 4 bytes, en SEGUNDOS
```

El razonamiento detrás de cada respuesta, que es lo que hay que saber explicar:

1. **¿Por qué el TTL y no otro campo?** Porque los otros cuatro **no dependen del tiempo**. **Solo el TTL declara cuánto tiempo vale lo que dice el registro**, y por eso es el único que puede envejecer.
2. **¿Por qué el autoritativo?** Porque el TTL es una **declaración de política del dueño del dato**, no una decisión técnica de quien lo transporta: si lo definiera el resolver, cada uno tendría su cuenta y el sistema se desincronizaría.
3. **¿Por qué «tiempo restante» y no «fecha de vencimiento»?** Porque es un **contador**: no hay reloj en el protocolo, se manda «3600 segundos» y cada quien lo cuenta **hacia abajo**. Eso permite que dos resolvers en puntos distintos coincidan sin sincronizar relojes.
4. **¿Por qué segundos?** Porque es la **única unidad que el formato del mensaje define**: 4 bytes, entero sin signo de 32 bits. Cualquier otra necesitaría un campo de unidades, que no existe.

> **Nota explícita contra la oficial — y es una diferencia real:** La solución oficial describe **cinco pasos** y en el paso 5 dice que *«Servidor TDL responde con la dirección IP del nombre pruebaexamen.edu al SERVIDOR LOCAL»*. **Mi lectura es que ahí hay un error conceptual:** el servidor del TLD **no puede** responder con la dirección IP, porque **no la tiene y no puede tenerla**: si la tuviera, no habría DNS distribuido, todo el mundo dependería de los TLD y el sistema no escalaría. Lo que el TLD devuelve es el **registro** `NS` que lo delega —quién es el autoritativo— más su *glue*, y **a partir de ahí hay un paso más**: la consulta al autoritativo, que es el único que devuelve el registro `A`. Por eso la cadena completa son **ocho mensajes (cuatro RTT)** y no cinco: es exactamente la misma cadena del `2010-07 P3`. Si el enunciado acepta cinco pasos porque es lo que se da en la clase, **reproducible la tabla oficial** y **agregá una línea con el paso que falta**: eso no te resta nada y muestra que entendiste la jerarquía.

b) Los pasos de una consulta tipo A — resolución

Acá hay que tener cuidado, porque **la solución oficial tiene un error conceptual**. El nombre del problema es `pruebaexamen.edu`: `pruebaexamen` es un dominio de segundo nivel y `.edu` es un **TLD genérico** (*gTLD*), no el `ccTLD` de un país. La cadena es `raíz → .edu → pruebaexamen.edu → el nombre buscado`.

```
┌─────┬──────────────────┬──────────────────┬──────────────────────────────┐
│  #  │ De               │ A                │ Qué pasa                     │
├─────┼──────────────────┼──────────────────┼──────────────────────────────┤
│  1  │ PC (stub)        │ servidor local   │ RECURSIVA: "¿IP de           │
│     │                  │ (el de la        │  pruebaexamen.edu?"          │
│     │                  │  empresa)        │  RD = 1                      │
│  2  │ servidor local   │ root server      │ ITERATIVA: la misma          │
│     │                  │                  │  consulta                    │
│  3  │ root server      │ servidor local   │ REFERENCIA: "el TLD .edu     │
│     │                  │                  │  está en <IP del .edu>"      │
│  4  │ servidor local   │ TLD .edu         │ ITERATIVA: la pregunta       │
│  5  │ TLD .edu         │ servidor local   │ REFERENCIA: "el autoritativo │
│     │                  │                  │  es ns1.pruebaexamen.edu,    │
│     │                  │                  │  en <IP del autoritativo>"   │
│  6  │ servidor local   │ autoritativo     │ ITERATIVA: la pregunta       │
│  7  │ autoritativo     │ servidor local   │ RESPUESTA: "es <IP>"         │
│  8  │ servidor local   │ PC               │ RESPUESTA: "es <IP>"         │
└─────┴──────────────────┴──────────────────┴──────────────────────────────┘

RTT: 1 (PC↔local) + 1 (local↔root) + 1 (local↔.edu) + 1 (local↔aut.) = 4 RTT
→ 8 mensajes.
```

```
⚠️ EL ERROR DE LA OFICIAL, y hay que verlo:

La oficial dice en su paso 5:
  "Servidor TDL responde con la DIRECCIÓN IP del nombre
   pruebaexamen.edu al SERVIDOR LOCAL"

El TLD .edu NO PUEDE responder con la dirección IP. No la tiene.
Lo único que tiene de pruebaexamen.edu es el registro NS que dice quién lo
autoriza. Si el TLD devolviera la IP, el sistema entero colapsaría en el TLD
y no habría DNS distribuido.

El TLD responde con una REFERENCIA (ANCOUNT = 0), y la respuesta final la da
el AUTORITATIVO en el paso 7.
```

Lo que hace el servidor local antes de responder: los tres `cachear`. Cachea **(1)** el registro `A` con su TTL; **(2)** la referencia del TLD `.edu` —el `NS` y su *glue*— con el TTL del TLD (172 800 s = 2 días), que es lo que hace que la **segunda** resolución del mismo nombre tarde 2 RTT y no 4; **(3)** la referencia del root. **Y recién después responde.**

> **Nota explícita contra la oficial — y es una diferencia real:** La oficial describe **cinco pasos** y en el paso 5 dice que *«Servidor TDL responde con la dirección IP del nombre pruebaexamen.edu al SERVIDOR LOCAL»*. **Mi lectura es que ahí hay un error conceptual:** el TLD **no puede** responder con la dirección IP, porque **no la tiene y no puede tenerla**: si la tuviera, no habría DNS distribuido y todo el mundo dependería de los TLD. Lo que devuelve es el **registro `NS`** que lo delega más su *glue*, y **a partir de ahí hay un paso más**: la consulta al autoritativo, único que devuelve el registro `A`. Por eso la cadena completa son **ocho mensajes (cuatro RTT)** y no cinco: es la misma cadena del `2010-07 P3`. Si el enunciado acepta cinco pasos porque es lo que se da en la clase, **reproducible la tabla oficial** y **agregá una línea con el paso que falta**: no te resta nada y muestra que entendiste la jerarquía.

3.2 ⭐ Examen 2010-07, Pregunta 3 (8 puntos) — resolución iterativa sin caché

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1007.pdf

Enunciado (texto literal del examen)

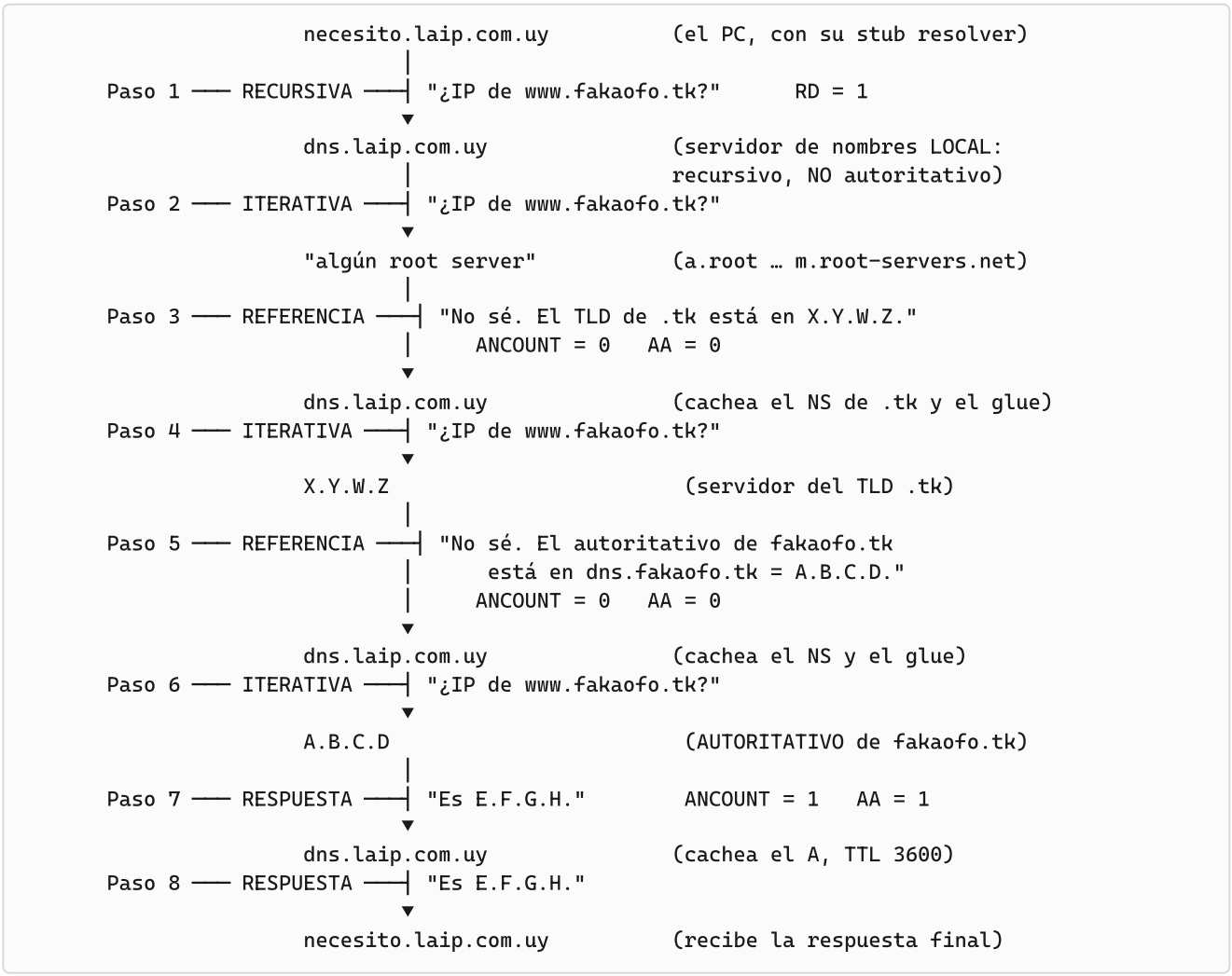
> **Pregunta 3 (8 puntos)**
>
> **a)** Represente en un diagrama todos los pasos involucrados en la resolución de nombres iterativa donde el equipo necesito.laip.com.uy consulta a su servidor DNS local (dns.laip.com.uy) por la dirección IP asociada a www.fakaofo.tk. Suponga que no se dispone de ninguna información de caché en ningún servidor.
>
> **b)** Indique en el diagrama de la parte a) cuál es el servidor del TLD tk y cuál el autoritativo de fakaofo.tk.

Solución oficial (del PDF, textual)

Paso	A	Información
1	necesito.laip.com.uy	dns.laip.com.uy
2	dns.laip.com.uy	"algún root server"
3	"algún root server" (el que recibió la consulta del paso 2)	dns.laip.com.uy
4	dns.laip.com.uy	X.Y.W.Z
5	X.Y.W.Z	dns.laip.com.uy
6	dns.laip.com.uy	A.B.C.D
7	A.B.C.D	dns.laip.com.uy
8	dns.laip.com.uy	necesito.laip.com.uy

El diagrama debe ser muy similar al que figura en la diapo 9 de la clase 7 de capa de Aplicación. X.Y.W.Z, A.B.C.D y E.F.G.H son meramente ilustrativos, pero sí deben ser distintos.

a) El diagrama, con los cuatro actores



Los cuatro mensajes literales (el `ID` es el mismo en todos porque es **una sola consulta** que viaja por la cadena):

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x1A2B
;; flags: rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
;www.fakaofo.tk.                IN      A

;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x1A2B
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;www.fakaofo.tk.                IN      A
tk.             172800  IN  NS  a.nic.tk.
a.nic.tk.       172800  IN  A   X.Y.W.Z

;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x1A2B
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;www.fakaofo.tk.                IN      A
fakaofo.tk.     172800  IN  NS  dns.fakaofo.tk.
dns.fakaofo.tk.  172800  IN  A   A.B.C.D

;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x1A2B
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;www.fakaofo.tk.                IN      A
;; ANSWER:
www.fakaofo.tk.   3600  IN  A   E.F.G.H
```

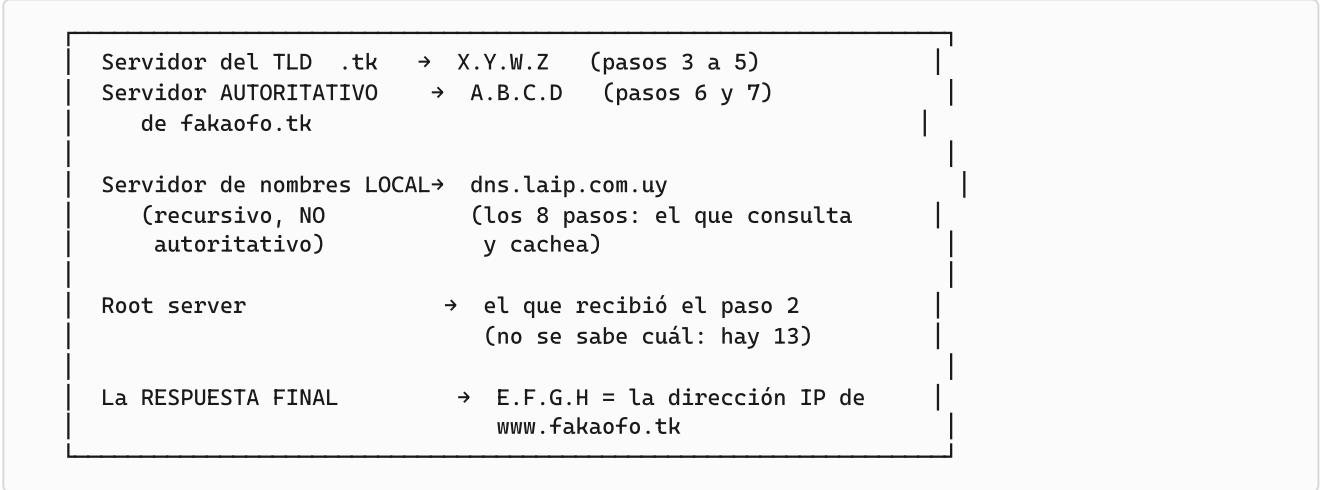
En el laboratorio: `dig +trace www.fakaofo.tk` hace exactamente el recorrido que pide el enunciado; a mano es `dig @a.root-servers.net`, `dig @X.Y.W.Z`, `dig @A.B.C.D` y `dig @dns.laip.com.uy`, siempre con `www.fakaofo.tk A`.

Análisis de la solución oficial. ✅ **Lo que está bien:** (1) **los ocho mensajes están completos**; (2) **los pasos 3 y 5 están escritos como *referencias*, no como respuestas** —«Consultar a TLD de tk», «Consultar a dns.fakaofo.tk» es exactamente la definición de iterativa, y si en alguno de esos dos pasos hubiera puesto «la IP es …», la solución estaría mal; (3) las tres direcciones **son distintas**, como pide el enunciado; (4) **el paso 8 existe**: muchos alumnos lo olvidan y se quedan en 7 pasos.

⚠️ **Lo que la oficial NO dice y hay que agregar:** (i) **qué tipo de consulta es cada paso**: **suma** distinguir que los 3 y 5 son respuestas con `ANCOUNT = 0` y que **solo el 7 tiene `ANCOUNT = 1` y `AA = 1`**; (ii) **qué algoritmo es cada paso**: el paso 1 es recursivo en la realidad (ver la nota); (iii) **qué se cachea**: el enunciado dice «sin caché en ningún servidor», pero mencionar qué **se cachearía después** demuestra que entendiste.

> **Nota explícita contra la oficial — y es la más importante de esta clase:** el enunciado dice «**resolución de nombres iterativa**» y describe que `necesito.laip.com.uy` **consulta a su servidor DNS local**. En el modelo real, **esa primera consulta es recursiva**: el *stub resolver* le pide al `dns.laip.com.uy` «resolveste esto por mí y traeme la respuesta final», y el flag `RD = 1` lo dice. Lo **iterativo** son los **cuatro pasos que van del servidor local hacia afuera** (2 a 7). Mi lectura: la oficial usa «iterativa» como abreviatura de «la que hace el servidor local», y **no lo dice**. Si te piden el diagrama, **hacé los ocho pasos** y **agregá la distinción de una línea**: «el paso 1 es recursivo y los 2 a 7 son iterativos». No te resta nada y puede sumarte.

b) ¿Cuál es el TLD y cuál el autoritativo?



(solución propia: el PDF publica la tabla del apartado a) y remite a la diapo 9 de la clase 7 para el dibujo, pero no redacta la respuesta de b).)

El matiz que hay que agregar y es lo que suma puntos: `dns.laip.com.uy` **no es el autoritativo de nada**. Aunque sea el servidor de nombres de `laip.com.uy`, frente a este problema actúa como **resolver recursivo**: es un **cliente** de la jerarquía, no un **dueño** de datos.

3.3 La otra pregunta de HTTP del mismo examen

> **Fuente:** Examen `eirc1007` / `sirc1007`, Pregunta 1 (8 puntos). *(Se resuelve con §2.3.)*

La oficial publica la tabla de pasos del HTTP no persistente —conectar, pedir, recibir, **cerrar** y repetir por cada imagen— y concluye en b): *«En el caso de HTTP persistente, se establece la conexión una sola vez [...]. El uso de HTTP persistente puede resultar más performante en los casos en que se utilice una sola conexión por vez (por ejemplo si el servidor nos limita por IP) [...]. Si el navegador abre conexiones en paralelo para obtener varias imágenes a la vez, probablemente termine de obtener todos los datos más rápidamente, aprovechando mejor el ancho de banda.»*

Resolución: es §2.3 Caso A: **22 RTT y 11 conexiones TCP** en no persistente contra **12 RTT y 1 conexión** en persistente. En b) hay que agregar el matiz que la oficial no cuantifica: **en paralelo empatan** (4 contra 3, y con 3 servidores ambos dan 4), así que la ventaja que le queda es el **recurso** de sockets, no el tiempo. Las dos frases de la oficial que hay que repetir tal cual son la del **paralelismo** y la del **límite de conexiones por IP**.

3.4 Dónde están los puntos de la unidad

Del `2010-12 P1`: **3** para el TTL (cuarto campo, 4 bytes, segundos, definido por el autoritativo, contador decreciente) y **5** para los **ocho** pasos (el TLD devuelve una **referencia**, no la IP; el local **cachea** antes de responder). Del `2010-07 P3`: **6** para el diagrama de **8 mensajes** con los 3 y 5 como *referencias*, y **2** para identificar `X.Y.W.Z`, `A.B.C.D` y decir que el local no es autoritativo. Del `2010-07 P1`, **4 y 4**: la secuencia de pasos con los 22 RTT, y el empate en paralelo.

PARTE 4 — CIERRE

📌 Resumen

1. `mensajes = 2 × RTT`. Sin caché: 8 mensajes y 4 RTT.
2. Los **3 y 5 son referencias** (`ANCOUNT = 0`, `AA = 0`); solo el **7** es respuesta final (`AA = 1`).
3. **El paso 1 es recursivo** y los 2 a 7 **iterativos**. La oficial no lo dice: decirlo suma.
4. **Cada nivel cacheado quita un viaje**: 4 RTT → 3 → 2 → 1.
5. **La caché ahorra tiempo, no bytes.**
6. **El TTL es el 4.º campo del RR, de 4 bytes, en segundos**; lo define el **autoritativo**, lo **descuenta** quien cachea. `TTL = 0` = «no me confíes».
7. `304` **= 0 bytes de cuerpo y el mismo RTT:** ahorra el 98 % de los bytes y **cero milisegundos**.
8. `RTT = N_obj × 2` no persistente; `1 + N_obj` persistente con un servidor; `k + N_obj` con `k`. El ahorro es `N_obj − k`.
9. **En paralelo, persistente y no persistente empatan.**
10. **Abrir una página HTTPS:** `4 + 1 + 1 + 1 + 10 = 17 RTT` en persistente; `26 RTT` en no persistente, con 11 conexiones. **Los RTT dominan la latencia; los bytes no.**

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Una resolución sin caché tiene 8 mensajes. ¿Cuántos RTT son? ¿Cuáles son *referencias* y cómo lo reconocés?

▶ Respuesta

2. El resolver tiene cacheado el `NS` de `fakaofo.tk` con su *glue*. Llega una consulta: ¿cuántos mensajes, cuántos RTT y por qué?

▶ Respuesta

3. ¿Cuáles son los 4 RTT exactos del punto de vista del cliente, y cuánto ahorra la caché con `2`, `40`, `25` y `15` ms?

▶ Respuesta

4. Un cliente pide un recurso con `If-None-Match` y recibe `304`. ¿Cuánto ancho de banda ahorró? ¿Y cuánto retardo?

▶ Respuesta

5. ¿Por qué con HTTP persistente el ahorro es exactamente `N_obj − k` RTT, y qué pasa con esa fórmula cuando el navegador abre conexiones en paralelo?

▶ Respuesta

6. Abrís una página HTTPS con 11 objetos a un RTT de 50 ms, en HTTP/1.1 persistente. ¿Cuántos RTT y cuántas conexiones TCP? ¿Qué parte del tiempo se elimina si el DNS está cacheado?

▶ Respuesta

7. ¿Por qué el paralelismo importa más que la persistencia? Pasame el número para 21 objetos y 3 servidores.

▶ Respuesta

8. El autoritativo responde con `TTL = 0`. ¿Qué hace el resolver? ¿Y si además es el `NS` de una zona?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc1012` **/** `sirc1012` — Pregunta 1 (8 pts): ★ (a) el TTL del RR, quién lo define, a qué refiere y unidades; (b) los pasos de una consulta tipo A
- **Examen** `eirc1007` **/** `sirc1007` — Pregunta 3 (8 pts): ★ resolución iterativa de `www.fakaofo.tk` sin caché, TLD y autoritativo · Pregunta 1 (8 pts): ★ HTTP no persistente vs. persistente
- **Examen** `eirc1002` **/** `sirc1002` P1b (iterativas vs. recursivas) · **Examen** `eirc0707` **/** `sirc0707` P4 (función del DNS y protocolo)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.4
- **RFC 1034** / **RFC 1035** (1987), el diseño del DNS, que **no cambió** · **RFC 6891** — EDNS(0) (2013) · **RFC 8020** — caché negativa por nombre exacto (2018) · **RFC 8767** — servir caché vencida (2020)
- **RFC 8484** — DoH (2018) · **RFC 7858** — DoT (2016) · **RFC 9250** — DoQ (2022) · **RFC 9460** — SVCB/HTTPS (2023)
- **RFC 9110** / **RFC 9112** — HTTP (2022) · **RFC 9113** — HTTP/2 · **RFC 9114** — HTTP/3 · **RFC 9000-9002** — QUIC
- **IANA**, registro de puertos: 53/udp, 53/tcp, 853/tcp (DoT)
- **Clases anteriores:** 8, 9 (HTTP) · 10, 11 (DNS) — **Clase siguiente:** 13 — Correo electrónico: SMTP, POP3, IMAP y MIME

Clase 13

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 La arquitectura del correo: tres roles y un camino largo
1.2 SMTP: el protocolo push del envío
1.3 MIME: por qué existe y cómo funciona
1.4 POP3: el protocolo pull de la lectura
1.5 IMAP: el mismo pull, pero con el estado en el servidor
1.6 ⚠️ Actualización a 2026
2.1 Ejercicio 1 (guía) — Un envío SMTP con adjunto, transcrito
2.2 Ejercicio 2 — Los tres encodings sobre el mismo texto, comparados
2.3 Ejercicio 3 — contra
2.4 Ejercicio 4 — POP3 e IMAP en un escenario concreto
2.5 Ejercicio 5 — *Dot-stuffing* con un mensaje que empieza con
3.1 ⭐ Examen 2010-03, Pregunta 2 (8 puntos) — la arquitectura del correo
3.2 ⭐ Examen 2008-12, Pregunta 4 (7 puntos) — el de POP3
3.3 Problema relacionado — el Examen 2004-12, Ejercicio 2 (el diálogo SMTP)
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: B (Aplicación) · **Semana:** 5 · **Duración estimada:** 110 min **Correlativa:** Clase 12 (taller DNS + HTTP) · **Sigue con:** Clase 14 (seguridad en la aplicación: TLS, autenticación y correo seguro) **Material de referencia:** Tanenbaum cap. 7 (§7.1) · Exámenes P2, P4, Ejercicio 2

> ⚠️ **Sobre los números de RFC de esta clase:** , que es la fuente de verdad del curso para números de RFC, **no tiene una sección de correo electrónico**. Los RFC que se citan acá son los oficiales y están verificados contra (octubre de 2026), pero **verificalos vos** en antes de copiarlos en un examen. Los **puertos**, en cambio, sí están verificados contra el registro de IANA en §15.

🎯 Al terminar esta clase vas a poder

1. Dibujar la **arquitectura del correo** con sus **tres roles** (MUA, MTA y MDA) y explicar qué protocolo usa cada par de actores.
2. Explicar por qué SMTP es un protocolo **push** y por qué POP3 e IMAP son **pull**, y qué consecuencia de seguridad tiene eso.
3. Escribir la **transcripción literal** de una sesión SMTP completa, con `EHLO`, `MAIL FROM`, `RCPT TO`, `DATA` y el punto final, incluida una respuesta de error.
4. Explicar **por qué existe MIME** y aplicar los tres (, ,) sobre **el mismo texto**.
5. Describir los **comandos y los dos estados** de POP3, y explicar por qué sus comandos van en claro.
6. Comparar POP3 e IMAP en una tabla de **8 filas**, entendiendo la diferencia de fondo: **dónde vive el estado**.
7. Resolver completo el **Examen 2010-03 P2** (8 puntos) y el **Examen 2008-12 P4** (7 puntos), con las soluciones oficiales a la vista.

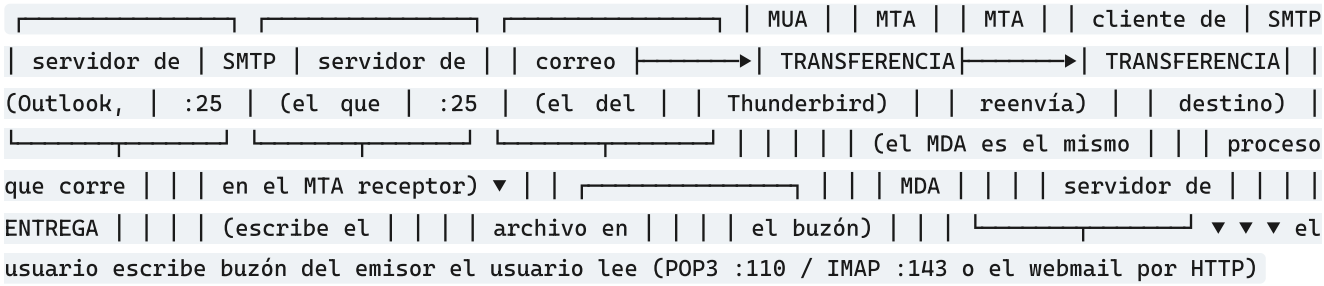
> **Prerequisito del examen:** (8 puntos — ★ la arquitectura del correo, los roles y los protocolos), (7 puntos — ★ el de POP3 y el *dot-stuffing*), Ejercicio 2 (el diálogo SMTP transcrito y el *dot-stuffing* en la práctica),

> (firma digital aplicada al correo).

PARTE 1 — TEORÍA

1.1 La arquitectura del correo: tres roles y un camino largo

El correo electrónico es la aplicación más vieja de Internet y la única que sigue siendo **extremo a extremo**. Esa decisión de diseño explica toda la arquitectura.



Los tres roles, con su glosa y su pregunta

Rol	Nombre en inglés	Glosa	La pregunta que responde	Protocolo hacia el siguiente
MUA	*Mail User Agent*	agente de usuario de correo	«¿qué escribo y qué leo?»	SMTP (push) hacia el MTA propio
MTA	*Mail Transfer Agent*	agente de transferencia	«¿a quién se lo mando y cómo lo hago llegar?»	SMTP (push) hacia otro MTA
MDA	*Mail Delivery Agent*	agente de entrega	«¿dónde lo archivo para que el destinatario lo encuentre?»	POP3 o IMAP (pull), o HTTP

El nombre importa porque cada nombre dice una cosa:

- **User**: el que **usa** el correo. Es el único que puede escribir y leer.
- **Transfer**: el que lo **traslada**. No lee nada, no guarda nada: reenvía.
- **Delivery**: el que lo **deja en su lugar**. Típicamente corre **dentro del mismo proceso que el MTA receptor**, pero conceptualmente es otro rol.

Por qué el camino es largo, y por qué eso es una decisión y no un accidente

` El correo NO es extremo a extremo en la entrega. El emisor no habla con el receptor: habla con SU servidor, y ese servidor habla con el del receptor. Hay al menos un punto —un servidor que no es el emisor— que puede leer o modificar el mensaje.

¿Por qué? Porque en 1982 no existía la infraestructura para que un usuario cualquiera aceptara conexiones entrantes. El teléfono de la época no tenía fija: no había forma de saber a qué máquina llamar. El correo se diseñó como la **bandeja**: el emisor deja el mensaje en un buzón y el destinatario, cuando quiere, pasa a recogerlo.

TODAY, con IP, esa razón ya no existe, y por eso la bandeja es un LASTRE: el correo sigue siendo el único protocolo de aplicación que no es extremo a extremo. `

> **La frase del examen:** *el correo electrónico es un sistema de_STORE-and-forward (guardar y reenviar)extremo a extremo.* Guardar y reenviar es lo que hace el MTA; extremo a extremo es lo que le falta.

El rol que falta en el diagrama y que hay que dibujar igual: el **MUA del receptor**

` MUA ──SMTP:25──▶ MTA ──SMTP:25──▶ MTA/MDA ──POP3:110──▶ MUA escribir (reenvía) (deposita) leer

El MUA del receptor entra por POP3, IMAP (o por HTTP si es webmail). Y si usa IMAP, en realidad hay un cuarto rol en juego: el estado (bandejas,Flags, carpetas) vive en el servidor, no en el cliente. `

Y un quinto rol que aparece cuando hay varios servidores de correo en el mismo dominio: el **MTA de reenvío** o *forwarder*, que recibe el mensaje y lo pasa a otro MTA sin depositarlo. Es el que existe cuando decís «redireccioná el correo a otra dirección».

1.2 SMTP: el protocolo push del envío

SMTP (*Simple Mail Transfer Protocol*) tiene el nombre de «simple» porque lo más importante es que hace una sola cosa: **empujar un mensaje**. No pide, no busca, no responde preguntas de directorio. Solo empuja.

Dato	Valor
Puerto principal	**25 TCP** (registrado también en UDP, pero el uso real es TCP)
Puerto de envío del cliente	**587** (*submission*), con autenticación
Puerto TLS implícito	**465** (*submissions*), deprecated a favor de en 587
Transporte	TCP, **sin cifrar** en el puerto 25
RFC	(2008, *Internet Standard*), que reemplaza a (1982) · *verificar*
Formato del mensaje	(2008), que reemplaza a (1982) · *verificar*

Los dos modos de usar SMTP, y por qué existen los dos puertos

` 25 (mail server → mail server) "MTA a MTA":Relay, sin autenticación, porque los dos extremos son servidores conocidos entre sí.

587 (cliente → su mail server) Submission: el usuario se identifica con usuario y contraseña, porque si no, cualquiera podría mandar correo en tu nombre.

465 (submission con TLS implícito) Todo el canal va cifrado desde el principio. Deprecada a favor de 587 con STARTTLS, pero sigue registrada y usada. `

> **Por qué importa en el examen:** SMTP **no tiene autenticación por defecto**. Por eso hay dos puertos: el 25 para el salto servidor a servidor y el 587 para el salto cliente a servidor. El 587 no existe por otra cosa.

El diálogo, comando por comando

```
El cliente (MUA) El servidor (MTA) | | |◄──────── 220 <dominio> ESMTP Service ready ────────|
(saludo) | | |──────── EHLO <mi.dominio> ──────────────────────►| (o HELO, antiguo) |◄── 250-
<dominio> (saludo y capacidades) ────| | 250-SIZE 35882577 | | 250-8BITMIME | | 250-STARTTLS | |
250-AUTH PLAIN LOGIN | | 250 HELP | | | |──────── MAIL FROM:<origen@dominio.com.uy> ──────────►|
(1 solo por mensaje) |◄──────── 250 2.1.0 Ok ──────────────────────────| | | |──────── RCPT
TO:<destino@dominio.com.uy> ──────────► | (1 o varios) | ◄ ──────── 250 2.1.5 Ok
──────────────────────────| | | |──────── DATA ──────────────────────────────────► |
|◄──────── 354 Start mail input; end with <CRLF>.<CRLF> ─| | | |──────── (el mensaje, línea por
línea) ──────────►| |──────── . ──────────────────────────────►| (punto solo)
| ◄ ──────── 250 2.0.0 Ok: queued as 40AB06AF005B39F9 ──| | | |──────── QUIT
──────────────────────────────────► | | ◄ ──────── 221 <dominio> closing connection
────────|
```

Los cinco comandos que hay que saber de memoria:

Comando	Qué hace	Cuántas veces
(o)	saludo e identificación; además lista las capacidades	1, al abrir
	declara el **remitente** (la casilla del *return path*)	**1 sola** por mensaje
	declara un **destinatario**	**1 o varias**
	«ahora viene el mensaje»	1, antes del cuerpo
	cierra la sesión	1, al terminar

Y las dos reglas del punto que hacen que todo funcione:

` 1. El mensaje se termina con una línea que contiene UN SOLO PUNTO: . Ese punto no es parte del mensaje: es el delimitador.

2. Si una línea del mensaje empieza con un punto, el emisor la "duplica": .algo en el texto → ..algo en el cable
   El receptor, al leer, saca un punto de cada línea que empiece con punto.

Esto se llama DOT-STUFFING y lo vas a tener que explicar en el examen. `

La transcripción literal de un envío completo

Esta es la forma que se ve en un o en Wireshark, con las líneas del servidor () y del cliente ():

```
S: 220 mta4.in.adinet.com.uy ESMTP Service (7.0.027) ready C: EHLO cliente.fing.edu.uy S: 250-
mta4.in.adinet.com.uy S: 250-SIZE 35882577 S: 250-8BITMIME S: 250-STARTTLS S: 250 HELP C: MAIL
FROM:<origen@dominio.com.uy> S: 250 2.1.0 Ok C: RCPT TO:<dest1@dominio.com.uy> S: 250 2.1.5 Ok C:
RCPT TO:<dest2@dominio.com.uy> S: 550 5.1.1 <dest2@dominio.com.uy>: Recipient address rejected:
User unknown C: RCPT TO:<dest3@dominio.com.uy> S: 250 2.1.5 Ok C: DATA S: 354 Start mail input;
end with <CRLF>.<CRLF> C: From: Ana Perez <origen@dominio.com.uy> C: To: Destino Tres
<dest3@dominio.com.uy> C: Subject: Hola a todos C: Date: Thu, 12 Mar 2026 10:14:07 -0300 C: C:
Hola: C: C: .Este renglon empieza con un punto, asi que viaja duplicado. C: ....Y este renglon
tambien. C: Saludos C: . S: 250 2.0.0 Ok: queued as 40AB06AF005B39F9 C: QUIT S: 221 2.0.0
mta4.in.adinet.com.uy closing connection
```

Leé los tres detalles que hacen la diferencia:

1. ** puede fallar sin que falle el mensaje. **El dio (*user unknown*), y el envío** siguió adelante **con `DATA`. Por eso `QUIT` es el comando que se repite:** un mensaje puede tener varios destinatarios y algunos pueden fallar**.
2. **El `DELE` es permanente y el `EXPUNGE` es temporal.** `+OK` = no reintentar, el usuario no existe. `-ERR` = Mailbox busy, reintentá más tarde. Esa distinción es la que permite que un MTA no tire a la basura un correo por un fallo pasajero.
3. **Las dos líneas con puntos duplicados.** El mensaje original tenía dos puntos al principio de la línea, y viajan como `..` y `.` . El punto final de la transacción es un punto **solo**.

Los códigos de respuesta que hay que reconocer

`2xx ÉXITO 250 Ok · 220 Service ready · 354 Mandame los datos 4xx ERROR TEMPORAL Intentá más tarde. La transacción NO se pierde: el MTA guarda el mensaje y reintenta (cola). 5xx ERROR PERMANENTE No hay caso, no reintentes. El MTA genera un reporte de-states (bounce) para avisarle al emisor.`

> **La frase del examen:** *SMTP es un protocolo de códigos de respuesta: el cliente interpreta un número de tres dígitos y decide si sigue, espera o abandona.*

1.3 MIME: por qué existe y cómo funciona

El problema que vino a resolver

` RFC 822 (1982) decía: un mensaje es texto ASCII de 7 bits, con líneas de 1 000 caracteres como máximo.

Eso alcanza para escribir una carta. No alcanza para nada de lo que la gente se empezó a mandar enseguida:

• tildes y eñes → no son ASCII de 7 bits • fotos, PDF, planillas → son bytes binarios, y un binario puede tener un "cambio de línea" donde sea •firmas → imágenes que garbage en el medio `

Y acá está el error de diseño de fondo: SMTP transporta **líneas terminadas en** . Un archivo binario puede contener la secuencia de bytes en medio de los datos. Si eso viaja sin escapar, **el protocolo cree que ahí terminó una línea y que la siguiente es un comando nuevo**. El archivo se rompe.

Las dos herramientas de MIME

` MIME = Multipurpose Internet Mail Extensions (RFC 2045 a 2049, 1996) · verificar

HERRAMIENTA 1 — Content-Type: ¿QUÉ es esto? Dice el TIPO de contenido: texto, imagen, PDF... Y permite meter un ARCHIVO DENTRO del mensaje: multipart/*

HERRAMIENTA 2 — Content-Transfer-Encoding: ¿CÓMO viaja? Dice cómo se codificó el cuerpo para que no rompa el transporte: 7bit · quoted-printable · base64 `

: los tres que hay que saber

Valor	Qué hace	Cuándo se usa	Costo
****	no toca nada: el cuerpo ya es ASCII de 7 bits	texto sin acentos ni caracteres raros	0 %
**** (QP)	escapa lo que no es ASCII con en hexadecimal, y **deja intactos los espacios y los acentos imprimibles**	texto con alguna tilde, o líneas largas	~10-20 %
****	codifica **todo** en grupos de 6 bits → caracteres imprimibles, con de relleno	**cualquier binario**: imágenes, PDF, adjuntos	**+33 %** (4 caracteres por cada 3 bytes)

La regla de oro para el examen:

> **`base64` es para binario; `quoted-printable` es para texto casi legible.** Un texto con tildes va en `quoted-printable`, nunca en `base64`: es legible y pesa menos.

: los tipos que hay que reconocer

` text/plain texto sin formato text/html una página web text/html; charset=UTF-8 lo mismo, dizendo la codificación image/png una imagen PNG application/pdf un PDF application/octet-stream "cualquier binario", el comodín

Y los dos compuestos, que son la parte interesante:

multipart/alternative "acá hay varias VERSIONES del mismo mensaje, elegí la mejor" → texto plano + HTML

multipart/mixed "acá hay varios CONTENIDOS distintos que van juntos" → texto + un adjunto `

La diferencia entre POP3 e IMAP, que es el punto que más se confunde:

` multipart/alternative El cliente ELIGE UNO de los que van dentro. Típico: texto plano (1er lugar, el más simple) + HTML (2.º, el más rico). El orden importa: el primero es el menosPreferred.

multipart/mixed El cliente los puede ver TODOS, y no elige. Típico: el texto del correo + un PDF adjunto.

Y se pueden anidar: multipart/mixed ├── text/plain (el texto del correo) └── application/pdf (el adjunto, en base64) `

Ejemplo con los tres encodings sobre el **mismo texto**

Texto original:

`Hola Ana: El precio es 1500 $ (dólares) y la entrega es en 3 días. Te adjunto el presupuesto. Saludos, Pedro`

Con : no se puede. La línea tiene caracteres de 8 bits, y **prohíbe** exactamente eso. Hay que elegir otro encoding.

Con :

` Content-Type: text/plain; charset=UTF-8 Content-Transfer-Encoding: quoted-printable

Hola Ana: El precio es 1500 $ (d=F3lares) y la entrega es en 3 d=EDas. Te adjunto el presupuesto. Saludos, Pedro `

`Fijate: "ó" (0xF3) se volvió "=F3" y "í" (0xED) se volvió "=ED". Los espacios, los signos y los números quedaron IGUALES: el texto sigue siendo legible por un humano. Eso es lo que hace que QP sea el encoding correcto para texto con caracteres no ASCII.`

Con :

` Content-Type: text/plain; charset=UTF-8 Content-Transfer-Encoding: base64

SG9sYSBBbmE6CkVsIHByZWNpbyBlcyAxNTAwICQgKGRvbGFyZXMpIHkgbGEg
bGEgZGVsaXZlcmEgZXMgZW4gMyBkw6lhcy4KVGUgYWRqdW50byBlbCBwcmVz
dWVzdG8uIFNhbHVkb3MsIFBlZHJv `

` QEIgc29zIHRlbmllbW8gdGllbmUgZXMgZXMgNTUzIGJ5dGVzLg==

El texto SIEMPRE queda ilegible, porque base64 no distingue un texto de un binario: para él son solo bytes. Por eso, si lo que mando es texto, base64 es la peor elección posible. `

El ejemplo completo: un correo con adjunto ()

` From: Pedro Gomez pedro@dominio.com.uy To: Ana Perez ana@otraempresa.com.uy Subject: Presupuesto Date: Thu, 12 Mar 2026 10:14:07 -0300 MIME-Version: 1.0 Content-Type: multipart/mixed; boundary="======frontera123"

--=======frontera123 Content-Type: text/plain; charset=UTF-8 Content-Transfer-Encoding: quoted-printable

Hola Ana:

Te adjunto el presupuesto. La entrega tarda 3 d=EDas.

Saludos, Pedro

--=======frontera123 Content-Type: application/pdf; name="presupuesto.pdf" Content-Transfer-Encoding: base64 Content-Disposition: attachment; filename="presupuesto.pdf"

JVBERi0xLjQKJcOkw7zDtsOfCjIgMCBvYmoKPDwvTGVuZ3RoIDMgMCBSL0ZpbHRlci9GbGF0
ZURlY29kZT4+CnN0cmVhbQp4nDPQM1Qo5ypUMABCA==

--=======frontera123-- `

Las cinco cosas que hay que mirar en ese mensaje:

1. **El ** es una cadena que** no existe dentro del mensaje** y que se usa para separar las partes. Es la clave del formato MIME.
2. **Cada parte es un mini-mensaje**, con sus **propias cabeceras MIME** (y).
3. **El PDF va en** , porque es binario. Un PDF en no solo sería ilegible: se rompería.
4. **** le dice al cliente «esto es un adjunto, no lo muestres en el cuerpo».
5. **El delimitador final tiene dos guiones menos** (), que es lo que marca el cierre de la parte *multipart*.

1.4 POP3: el protocolo pull de la lectura

POP3 (*Post Office Protocol version 3*) hace **una sola cosa**: descargar mensajes del buzón al cliente y, opcionalmente, borrarlos del servidor.

Dato	Valor
Puerto	**110 TCP** (seguro: **995**, *pop3s*, con TLS implícito)
Modelo	**pull**: el cliente va a buscar
Estado	en el **cliente**, después de descargar
RFC	(1996) · *verificar*
Transporte	TCP, **en claro** en el 110

Los dos estados, y por qué existen

```
┌──────────────────────────┐
│ ESTADO DE AUTORIZACIÓN │ Fase 1: el servidor solo admite comandos
└──────────┬───────────────┘ de autenticación y de info del buzón. │ Todo lo demás da -ERR. USER
PASS (+ APOP, opcional) │ ▼ ┌──────────────────────────┐
│ ESTADO DE TRANSACCIÓN │ Fase 2: acá
están los comandos que mueven └──────────────────────────┘ mensajes. Y acá hay una subtlety: │
hasta que el cliente manda QUIT, el STAT LIST RETR DELE servidor NO borra nada, aunque el │
cliente haya mandado DELE. ▼ QUIT ⟶ se borran los marcados con DELE
```

La subtlety es lo más importante del protocolo y casi nunca se dice:

> ** no borra nada al instante: marca. **El servidor borra de verdad** solo cuando el cliente manda `QUIT`. Si la conexión se cae antes del `QUIT`, los mensajes marcados **se quedan en el buzón**. Es un mecanismo de seguridad contra la pérdida de datos por un corte de conexión, y por eso el estado «de actualización» se llama así.

Los comandos

Comando	Qué hace
	envía el nombre de usuario
	envía la contraseña
	devuelve **dos números**: cantidad de mensajes y tamaño total en bytes
	devuelve los números de los mensajes y sus tamaños
	descarga el mensaje **completo**, con sus cabeceras
	marca el mensaje para borrar
	desmarca todo
	cierra la sesión y **aplica los borrados**

Una sesión POP3 completa, literal

```
C: USER ana.perez S: +OK POP3 server ready C: PASS ****** S: +OK maildrop has 3 messages C: STAT
S: +OK 3 120458 C: LIST S: +OK 3 messages (120458 octets) S: 1 48210 S: 2 40122 S: 3 32126 C:
RETR 1 S: +OK 48210 octets S: From: pedro@dominio.com.uy S: To: ana.perez@otraempresa.com.uy S:
Subject: Presupuesto S: S: Hola Ana: S: S: Te adjunto el presupuesto. S: . C: DELE 1 S: +OK
message 1 deleted C: QUIT S: +OK POP3 server signing off (21600 seconds)
```

Tres cosas que hay que notar:

1. ** es éxito y `-ERR` es error. **Todos los comandos empiezan por `+OK` o `-ERR`, y el** código numérico va después**, para las respuestas multi-línea.** SMTP usa un esquema de tres dígitos (,) y POP3 usa dos tipos de línea** (,). Es una diferencia de diseño entre los dos protocolos que se pregunta.
2. ** devuelve el mensaje entero, cabeceras incluidas.** No hay forma de pedir «solo el cuerpo» ni «solo las cabeceras».
3. `DELE` **dice** `+OK`**, pero todavía no se borró.** Se borró en el `QUIT`.

Por qué los comandos van en claro

```
┌──────────────────────────────────────────────────────────┐ │ POP3 y SMTP en los puertos 110 y 25 son TEXTO PLANO. │ │ No hay cifrado. │ │ │ Eso significa que CUALQUIERA en el camino — cada router, cada │ │ servidor de la red local, cualquier espía— puede leer: │ │ │ │ USER ana.perez ← tu nombre de usuario │ │ PASS MiContraseñaSecreta ← tu contraseña, EN CLARO │ │ RETR 1 ← y el mensaje entero │ │ │ │ Por eso existen los puertos seguros: │ │ 995 = POP3 sobre TLS desde el primer byte (pop3s) │ │ STARTTLS en el 110: se cifra a mitad de camino │ └──────────────────────────────────────────────────────────┘
```

> **La respuesta corta para el examen:** *los comandos van en claro porque POP3 fue diseñado en 1984, cuando el cifrado en la capa de aplicación no existía y la seguridad se resolvía con contraseñas y con no dejar pasar la metallurgy. La respuesta larga: POP3 no tiene cifrado propio, no lo tiene por diseño, y por eso el puerto seguro es otro puerto distinto (995) y no una opción del protocolo.*

1.5 IMAP: el mismo pull, pero con el estado en el servidor

IMAP (*Internet Message Access Protocol*) también es **pull**, pero tiene una idea central que POP3 no tiene: **la sesión es una vista de una estructura de carpetas que vive en el servidor**.

Dato	Valor
Puerto	**143 TCP** (seguro: **993**, *imaps*)
Modelo	**pull**, pero **el estado queda en el servidor**
RFC	(2001) *obsoleta* por (2021), *IMAP4rev2 · verificar*
Transporte	TCP; en el 143, TLS implícito en el 993

Los comandos que hacen la diferencia

` IMAP tiene los mismos DOS estados que POP3 (no autenticado / autenticado), pero los comandos de la fase autenticada son de otro tipo:

SELECT INBOX "abrí el buzón de entrada" ← POP3 NO TIENE NADA DE ESTO EXAMINE INBOX lo mismo, pero en modo solo lectura LIST "" * "dame la lista de carpetas" ← POP3 NO TIENE ESTO SEARCH UNSEEN "buscá lo que no leí" ← POP3 NO TIENE ESTO FETCH 1 BODY.PEEK[] "decime el mensaje, pero no me lo borre" STORE 1 +FLAGS (\Seen) "marcalo como leído, en el SERVIDOR" APPEND INBOX {n} "depositá un mensaje en el

servidor" UID EXPUNGE "borrá lo que marqué, en el SERVIDOR" IDLE "quedate esperando que te avise si llega algo" `

** es la razón por la que IMAP «se siente» moderno: **el servidor puede** empujar una notificación* *(«te llegó un correo») aunque IMAP sea un protocolo* pull*. El cliente sigue preguntando, pero puede esperar sin vueltas.

La tabla comparativa, que es lo que hay que saber de memoria

	POP3	IMAP
Modelo	pull	pull (con ,:notificación)
Puertos	110 / 995	143 / 993
¿Dónde queda el estado?	en el **cliente**	en el **servidor**
¿Servidor?	¿borra después de descargar? (opcional)	no: el mensaje sigue en el servidor
¿Carpetas?	**no tiene** (una sola «bandeja»)	**sí**: , , , , y las que crees vos
¿Estado de lectura?	local del cliente	**flags en el servidor** (, ,)
¿Busca en el servidor?	no: baja todo y busca local	**sí**: , y el servidor solo manda lo que le pedís
Ideal para	una casilla, un dispositivo, una línea	varios dispositivos, correo grande, carpetas

> **La frase del examen:** *la diferencia entre POP3 e IMAP no es de transporte ni de comandos: es dónde vive el estado. POP3 es un buzón plano que vos venís a vaciar; IMAP es un sistema de archivos remoto con carpetas y marcas, y vos venís a mirar.*

La diferencia práctica, en un ejemplo

` El mismo usuario, en el celular y en la notebook, con 4 000 correos:

CON POP3: En la notebook, POP3 baja los 4 000 y los BORRA del servidor. En el celular: "no hay correos". Y si los baja y los borra también, la notebook dice: "no hay correos". → POP3 es de UNO en uno, o tenés que deixar de borrar.

CON IMAP: En la notebook, IMAP deja los 4 000 en el servidor y marca los leídos (flag \Seen) ahí mismo. En el celular: ve los mismos 4 000, en las mismas carpetas, con los mismos flags. "Leíste esto en la notebook" también lo sabés en el celular. → IMAP es de MUCHOS en muchos. `

1.6 ⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar:

- Los **tres roles**: MUA, MTA y MDA, y la cadena **MUA → MTA → MDA → MUA**.
- **SMTP = push**, puerto **25**, con / / / / el punto final / , y los códigos de tres dígitos (, ,).
- ** se repite** y puede fallar sin tumbar el mensaje.
- **POP3 = pull**, puerto **110** (seguro **995**), con los dos estados y con que solo borra en el `QUIT`.
- **IMAP = pull con estado en el servidor**, puerto **143** (seguro **993**), con carpetas y *flags*.

- **MIME** existe porque RFC 822 era ASCII de 7 bits; los tres encodings; `multipart/alternativemultipart/mixed`.

Cambió, y es lo que hay que contar en 2026:

- ** (2021, IMAP4rev2) reemplaza a .* *Mismo protocolo, especificación reescrita: se eliminaron comandos obsoletos y se agregó soporte de UTF-8.* Verificar el número antes de citarlo.*
- **El correo sin cifrar es historia, no práctica.** Los puertos 25, 110 y 143 **deben** terminar en `QUIT`; los puertos **465**, **993** y **995** son de TLS implícitocito. Es la diferencia con la clase de 2011, donde el curso mostraba sesiones en claro.
- **La autenticación del envío es obligatoria en la práctica.** (2007) define y hoy todos los grandes proveedores la exigen en el 587. *Verificar.*
- **El antispam se movió de capa aplicación a capa red:** (un de DNS), (una firma criptográfica en la cabecera) y (qué hacer si los dos fallan) son **obligatorios** para que un mensaje llegue a la bandeja de entrada. Consecuencia conceptual: **hoy un correo electrónico sin SPF ni DKIM es indistinguible de un impostor**, y por eso el modelo sin cifrar ya no es sustainable.
- **El spam le ganó a la arquitectura.** LaFilterClash es un problema de aplicación que hoy se resuelve con reputación de IP y de dominio (), listas grises y reputación de remitente —o sea, con info que **vive fuera del protocolo SMTP**.
- **El «webmail» se comió el cliente**: hoy lo más usado es leer el correo por **HTTPS** en un navegador, que es una cuarta forma de acceder al buzón que el curso de 2011 mencionaba al pasar.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 (guía) — Un envío SMTP con adjunto, transcrito

Datos. Ana envía un correo a Pedro con un PDF adjunto. El mensaje de Ana tiene acentos, así que el texto va en `quoted-printable`; el PDF es binario y va en `base64`.

a) ¿Qué puerto usa y por qué?

` Puerto 587 (submission), NO el 25.

El 25 es para el salto MTA → MTA, donde los dos extremos son servidores conocidos. Ana → servidor de correo es un salto de USUARIO a servidor, y ahí hace falta autenticación. El 587 es exactamente ese caso. `

b) La transcripción completa del envío:

```
S: 220 mail.uy.com.uy ESMTP Service ready C: EHLO notebook.ana.uy S: 250-mail.uy.com.uy S: 250-SIZE 10240000 S: 250-8BITMIME S: 250-STARTTLS S: 250-AUTH PLAIN LOGIN C: STARTTLS S: 220 2.0.0 Ready to start TLS ··· (aca empieza el cifrado: todo lo que sigue viaja dentro de TLS) ··· C: EHLO notebook.ana.uy S: 250-mail.uy.com.uy S: 250-AUTH PLAIN LOGIN C: AUTH PLAIN AGFuYS5wZXJlejpDb250cmFz+WE6 S: 235 2.7.0 Authentication successful C: MAIL FROM: <ana.perez@uy.com.uy> S: 250 2.1.0 Ok C: RCPT TO:<pedro.gomez@otraempresa.com.uy> S: 250 2.1.5 Ok C: DATA S: 354 End data with <CR><LF>.<CR><LF> C: From: Ana Pérez <ana.perez@uy.com.uy> C: To: Pedro Gómez <pedro.gomez@otraempresa.com.uy> C: Subject: Presupuesto C: Date: Thu, 12 Mar 2026 10:14:07 -0300 C: MIME-Version: 1.0 C: Content-Type: multipart/mixed; boundary="BOUND42" C: C: --BOUND42 C: Content-Type: text/plain; charset=UTF-8 C: Content-Transfer-Encoding: quoted-printable C: C: Hola Pedro: C: C: Te adjunto el presupuesto. La entrega tarda 3 d=EDas. C: Saludos, Ana C: C: --BOUND42 C: Content-Type: application/pdf; name="presupuesto.pdf" C: Content-Transfer-Encoding: base64 C: Content-Disposition: attachment; filename="presupuesto.pdf" C: C: JVBERi0xLjQKJcOkw7zDtsOfCjIgMCBvYmoKPDwvTGVuZ3RoIDMgMCBSL0ZpbHRl C: ci9GbGF0ZURlY29kZT4+CnN0cmVhbQp4nDPQM1Qo5ypUMABCA== C: C: --BOUND42-- C: . S: 250 2.0.0 Ok: queued as 8B7A0C1199E4F002 C: QUIT S: 221 2.0.0 mail.uy.com.uy closing connection
```

c) Las seis decisiones que hay que justificar:

#	Decisión	Por qué
1	Puerto **587**, no 25	es un salto de usuario a servidor: requiere autenticación
2	** dos veces**, no	el devuelve las capacidades; el es el comando antiguo de 1982
3	** antes de **	si mandás la contraseña sin cifrar, viaja en claro por la red
4	**base64**, no	manda de una; va en dos pasos y es más fácil de interceptar
5	Texto en `binary`	tiene un 0, QP lo deja legible; `base64` lo haría ilegible y 33 % más grande
6	PDF en ****	es binario, y si viaja crudo puede traer que rompe el protocolo

d) ¿Cuántos bytes de másîne el ?

` El PDF ocupa 3 000 bytes reales. base64 mete 4 caracteres por cada 3 bytes: 3 000 / 3 = 1 000 grupos 1 000 grupos × 4 caracteres = 4 000 caracteres Como además base64 mete un salto de línea cada ~76 caracteres, hay que sumar los retornos de línea:

4 000 / 76 = 52,6 → 53 saltos de línea

Total en el cable: 4 000 + 53 = 4 053 bytes Costo: 4 053 − 3 000 = 1 053 bytes = 35 % de aumento `

Interpretación física: un adjunto de 10 MB llega por SMTP con unos **13,5 MB**. Y todo eso pasa **una sola vez por cada servidor de la cadena**, que es una de las razones históricas por las que el correo con adjuntos grandes «iba tan lento».

2.2 Ejercicio 2 — Los tres encodings sobre el mismo texto, comparados

Datos. El texto es

Encoding	Cómo queda en el cable	Legible	Tamaño
****	**no se puede usar**: el es 0xF3, de 8 bits	—	—
****		**sí**	32 bytes (el pasó de 1 a 3)
****		no	48 bytes (4 caracteres por cada 3 bytes)

La comparación que hay que saber de memoria:

` El ORIGINAL tiene 29 bytes y un solo carácter problemático: el "ó".

QP : 29 − 1 + 3 = 32 bytes. Solo paga el problema. base64: 29 bytes → 40 de datos + 4 de relleno + los saltos de línea = 48 bytes. Paga TODO, incluso la parte que ya era ASCII.

QP gana en legibilidad y en tamaño SIEMPRE que haya texto. base64 gana solo cuando el contenido es binario, porque ahí no hay alternativa: lo que no es texto no tiene «partes legibles». `

⚠️ Errores típicos

- **Poner en un PDF.** QP escapa bytes, pero un PDF comprimido no es texto: se destruye.
- **Poner en un texto con tildes.** Funciona, pero es ilegible y pesa más.
- **Olvidar el .** Sin esa cabecera, muchos clientes anciens ignoran todas las demás cabeceras MIME y muestran el PDF crudo.
- **Usar cuando querías texto plano y HTML.** Eso es `multipart/ alternativemixed` el cliente los muestra todos.

2.3 Ejercicio 3 — contra

Enunciado. Ana manda el mismo correo en texto plano y en HTML. Después lo manda otra vez con un Excel adjunto. ¿Qué va en cada caso?

` CASO A — texto plano + HTML: Content-Type: multipart/alternative; boundary="ALT1" ... --ALT1 Content-Type: text/plain; charset=UTF-8 (el texto) --ALT1 Content-Type: text/html; charset=UTF-8 (el HTML) --ALT1--

El cliente ELIGE UNO. El último es el más Preferred, así que un cliente moderno muestra el HTML y el otro muestra el texto.

CASO B — texto + Excel adjunto: Content-Type: multipart/mixed; boundary="MIX1" ... --MIX1 Content-Type: text/plain; charset=UTF-8 (el texto) --MIX1 Content-Type: application/vnd.ms-excel; name="datos.xls" Content-Transfer-Encoding: base64 Content-Disposition: attachment; filename="datos.xls" (el Excel en base64) --MIX1--

El cliente los ve TODOS: muestra el texto y deja el Excel adjunto. `

Y se pueden anidar, que es lo que hacen los clientes reales:

` Content-Type: multipart/mixed; boundary="MIX1"

--MIX1 Content-Type: multipart/alternative; boundary="ALT1"

--ALT1 Content-Type: text/plain; charset=UTF-8 --ALT1 Content-Type: text/html; charset=UTF-8 --ALT1--

--MIX1 Content-Type: image/png; name="grafico.png" Content-Transfer-Encoding: base64 Content-Disposition: inline; filename="grafico.png" --MIX1-- `

`mixed → "hay varias cosas: elegí CUALES te Mostro" ├── alternative → "acá hay varias VERSIONES: elegí UNA" └── image/png → "esto va adentro, en línea"`

> ⚠️ **La palabra clave es `Content-Type`, no `Content-Transfer-Encoding`:** *alternativa* significa que hay versiones del **mismo** contenido y el cliente elige; *mixto* significa que hay contenidos **distintos** y el cliente los ve todos.

2.4 Ejercicio 4 — POP3 e IMAP en un escenario concreto

Datos. Ana tiene 4 200 mensajes en el servidor, repartidos en dos cuentas, , `nuevo@lugar.com` y `Viejo@lugar.com`. Tiene una notebook y un celular, y los dos se conectan todos los días.

a) ¿Qué pasa si configura POP3 en los dos, con «borrar del servidor al descargar»?

` Lunes, en la notebook: POP3 baja los 4 200 mensajes (4 200 × ~4 RTT... en realidad uno por mensaje: una sesión, pero un RETR por mensaje) y los BORRA del servidor.

Lunes, en el celular, una hora después: +OK maildrop has 0 messages → Ana no ve NADA.

Martes, en la notebook: +OK maildrop has 0 messages → Ana ya no tiene NADA en ningún lado, salvo lo que se descargó.

Peor: si borra en la notebook pero NO en el celular, entonces el celular sigue teniendo los 4 200 y la notebook no, y el problema es al revés. POP3 no se synchronize: cada cliente ve lo que le tocó. `

b) ¿Qué pasa si configura IMAP en los dos?

` Lunes, notebook: SELECT INBOX → 2 000 mensajes FETCH 1..50 → baja 50, NO los borra, marca \Seen en el servidor

Lunes, celular (una hora después): SELECT INBOX → 2 000 mensajes (los mismos) Lo que leyó en la notebook aparece con \Seen: no lo vuelve a mostrar como nuevo. Y las carpetas están todas: Trabajo, Familia, Archivo.

El archivo NO se duplica en los dos lados: es el mismo, en el servidor, y los dos clientes leen la misma copia. `

c) El costo de cada uno, en números.

` CON POP3 (borrando del servidor): 1er dispositivo : baja 4 200 mensajes. Supongamos 40 KB cada uno: 4 200 × 40 KB = 168 MB de tráfico y de disco. 2do dispositivo : 0 mensajes.

CON IMAP: 1er dispositivo : baja solo los que no había visto: 200 mensajes, 8 MB. 2do dispositivo : los mismos 200 que faltan, 8 MB. El servidor : los 4 200 mensajes siguen ahí, occupying disco (que es lo que el administrador tiene que pagar).

O sea: POP3 ahorra espacio en el servidor y paga en cada cliente; IMAP paga en el servidor y ahorra en cada cliente. `

d) ¿Y si Ana tiene un solo dispositivo y una línea lenta?

`POP3 gana, y por mucho: con IMAP tendría que dejar los 4 200 en el servidor y bajar 200; con POP3 baja una vez y los borra, y listo. La elección depende de la relación entre "número de dispositivos" y "espacio en el servidor", no de cuál protocolo es "mejor".`

2.5 Ejercicio 5 — *Dot-stuffing* con un mensaje que empieza con

Datos. Ana manda este mensaje, que tiene una línea con **dos puntos** al principio (inicio de línea = después de un):

` Hola Pedro:

..esto NO es el fin del mensaje, es texto .y esto tampoco

Saludos `

a) ¿Qué travels por el cable?

` REGLA DEL DOT-STUFFING: si una línea del mensaje empieza con un punto, se le AGREGA un punto más al principio. Solo si empieza con punto.

original en el cable ———————————————————— ———————————————————— Hola Pedro: Hola Pedro: (línea vacía) ..esto NO es el fin... ...esto NO es el fin... .y esto tampoco ..y esto tampoco (línea vacía) Saludos Saludos (fin) . ← el delimitador

OJO con la cuenta: "..esto" → "...esto" (2 puntos → 3) ".y esto" → "..y esto" (1 punto → 2) El delimitador final es UN punto solo, porque no es contenido. `

b) La transcripción SMTP completa, con el escapado:

`C: DATA S: 354 End data with <CR><LF>.<CR><LF> C: Subject: Prueba de dot-stuffing C: C: Hola Pedro: C: C: ...esto NO es el fin del mensaje, es texto C: ..y esto tampoco C: C: Saludos C: . S: 250 2.0.0 Ok: queued as 44C1F0AA1177`

c) ¿Qué recibe Pedro?

` Pedro, al quitar el stuffing (saca un punto de cada línea que empiece con punto), reconstruct este mensaje:

Hola Pedro:

..esto NO es el fin del mensaje, es texto .y esto tampoco

Saludos

IDÉNTICO al original. Ana no se dio cuenta de que estaba usando dos puntos, y no hizo falta que se diera cuenta. `

d) ¿Por qué el punto es el carácter de relleno natural?

Esta es la pregunta de examen, y la respuesta tiene tres partes:

` 1. POR QUÉ NO PUEDE SER OTRO CARÁCTER Because un protocolo de texto necesita un delimitador que sea un carácter que (a) aparezca en todas las versiones de todos los sistemas y (b) tenga un significado que lo haga identificable. El punto cumple las dos: es ASCII de 7 bits, está en todas las tablas de caracteres y en la línea de comandos representa «current directory» y «fin de frase».

2. POR QUÉ ES EL QUE YA ESTÁ EN USO Because SMTP heredó de telnet y de los protocolos de teletipo la convención de "el fin del texto es un punto solo". Reutilizar un carácter que ya hacía ese papel evita inventar un segundo marcador.
3. POR QUÉ EL ESCAPADO ES SOLO DE UN PUNTO Because si el delimitador fuera ".", entonces para que el receptor no lo confunda con el final, basta con que CUALQUIER línea que empiece con "." empiece con dos. Es la solución mínima: duplicar el carácter.

En cambio, si el delimitador fuera una cadena larga (como el "--boundary--" de MIME), el escapado tendría que ser más complejo.

Y el costo, que es lo que el protocolo paga: el receptor tiene que HACER algo en cada línea. Es trabajo de CPU del servidor, que es exactamente la razón por la que hoy los servidores grandes rechazan conexiones SMTP que hacen esto y aplican rate limiting. `

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-03, Pregunta 2 (8 puntos) — la arquitectura del correo

> **Fuente:** Examen / — 24 de febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1003.pdf

Enunciado (texto literal del examen)

> **Pregunta 2 (8 puntos)**
>
> **a)** Dibuje un diagrama que describa la arquitectura de la aplicación correo electrónico. En el diagrama debe indicar cuáles son los roles de los sistemas que participan y los protocolos de aplicación que conozca que se pueden utilizar para intercambiar información entre ellos.
>
> **b)** Describa la serie de pasos que se aplican sobre un correo electrónico desde que el emisor lo escribe hasta que el destinatario lo lee, asumiendo que el emisor y el destinatario tienen casillas de correo de servidores distintos.

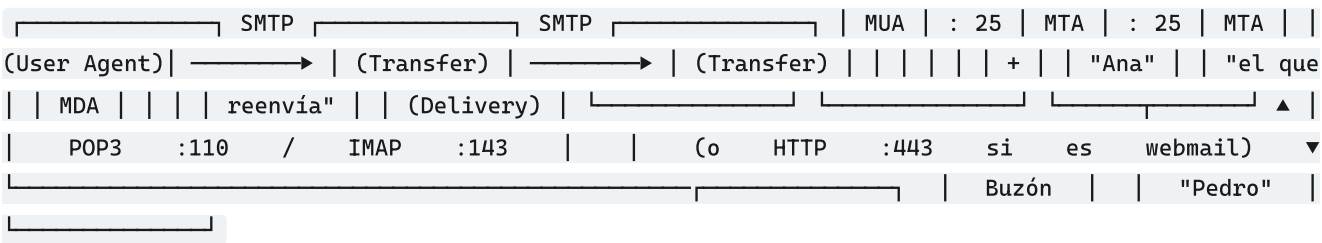
Solución oficial (del PDF)

Del apartado **a)** el PDF **publica un diagrama como imagen** y no un texto: en el extraído aparecen literalmente las palabras «Solución parte a)» y «Solución parte b)» sin contenido entre ambas. Del apartado **b)** la solución textual es:

> *El emisor escribe el mensaje de correo electrónico. El user agent del emisor envía por SMTP el correo a su mail server. El mail server del emisor envía por SMTP el correo al mail server del destinatario. El mail server del destinatario almacena el correo en la casilla. Más tarde, el destinatario se conecta a su mail server. Si utiliza POP3 o IMAP descarga el correo a su máquina. Si utiliza un webmail lo puede leer en el browser (descarga el correo por HTTP).*

(el diagrama del apartado a) es solución propia, reconstruido con los roles y los protocolos que la cátedraisease en la clase 7 de capa de aplicación)

a) El diagrama, con los roles rotulados



Los rótulos que hay que poner en el diagrama, uno por actor:

Actor	Rótulo obligatorio	Puerto y protocolo hacia el siguiente
El programa de Ana	**MUA** — *Mail User Agent* (agente de usuario de correo)	SMTP al MTA propio, **25**
El servidor de Ana	**MTA** — *Mail Transfer Agent* (agente de transferencia)	SMTP al MTA remoto, **25**
El servidor de Pedro	**MTA** + **MDA** — *Mail Delivery Agent* (agente de entrega)	POP3 **110** o IMAP **143**
El programa de Pedro	**MUA**	—

Lo que hay que dibujar además, y es lo que más suma puntos: el **camino de ida y de vuelta** del correo son **protocolos distintos**. La **ida es SMTP, que es *push*** (el servidor empuja el mensaje). La **vuelta es POP3 o IMAP, que son *pull*** (el cliente tira del mensaje). **SMTP nunca se usa para leer el correo.** Ese es el error clásico.

También: la posibilidad de **webmail**, que agrega una cuarta flecha por **HTTP/HTTPS** y **el mismo MDA** de antes. La oficial lo menciona en b), y está bien mencionado.

b) Los cinco pasos — resolución con la solución oficial a la vista

La oficial da cinco pasos. Es **correcta y completa** para los 8 puntos. Lo que hay que agregar es la **precisión de los roles** y los **porqués**, que es lo que suma:

` PASO 1 — Ana escribe el mensaje en su MUA. Es un texto con cabeceras: From:, To:, Subject:, Date:.

PASO 2 — El MUA de Ana lo ENTREGA a su propio servidor de correo por SMTP (puerto 25, o mejor 587 con autenticación). ★ El MUA no habla con el servidor de Pedro: habla con el suyo. El correo todavía no salió de la organización.

PASO 3 — El MTA de Ana ENVÍA el mensaje por SMTP al MTA de Pedro. ★ Acá puede haber VARIAS etapas, porque el correo puede viajar por reenviadores. Y puede haber reintentos: si un servidor da 4xx, el mensaje queda en una COLA y se reintenta más tarde. Si da 5xx, se descarta y se manda un reporte de-states al emisor.

PASO 4 — El MTA de Pedro hace de MDA: ENTREGA el mensaje, es decir lo ESCRIBE EN EL BUZÓN de Pedro en el disco del servidor. ★ Nada de esto es extremo a extremo: el servidor de Pedro vio el contenido del mensaje. Es la diferencia con HTTPS.

PASO 5 — MÁS TARDE (y este es el detalle que hay que subrayar) Pedro conecta su MUA a su servidor y LEE: · con POP3 (110): el cliente baja los mensajes y, opcionalmente, los borra del servidor. El estado queda en el cliente. · con IMAP (143): el cliente los mira y deja todo en el servidor. El estado (carpetas, flags) queda en el servidor. · con un webmail: su MUA es un navegador, y lee por HTTPS. El protocolo de aplicación es HTTP, no POP3 ni IMAP. `

Respuesta redactada, lista para copiar:

> **a)** La arquitectura del correo tiene **tres roles**: el **MUA** (*Mail User Agent*), que es el programa con el que el usuario escribe y lee; el **MTA** (*Mail Transfer Agent*), que es el servidor de correo que **reenvía** los mensajes entre organizaciones; y el **MDA** (*Mail Delivery Agent*), que es el que **deposita** el mensaje en el buzón del destinatario y que típicamente corre en el mismo proceso que el MTA receptor. Los protocolos son: **SMTP** (puerto **25**) entre el MUA y su MTA, y entre un MTA y otro MTA —que es un protocolo **push**—; **POP3** (puerto **110**), **IMAP** (puerto **143**) o **HTTP/HTTPS** si es webmail, entre el MUA del receptor y su servidor —que son protocolos **pull**—; y **DNS** (tipo) para descubrir a qué servidor hay que entregarle el correo de cada dominio.

> **b)** Los pasos son: **1)** el emisor escribe el mensaje en su **MUA**; **2)** el MUA lo entrega por **SMTP** a su propio servidor de correo; **3)** el **MTA** del emisor envía el mensaje por **SMTP** al **MTA** del destinatario, que puede estar a varias etapas de distancia y donde puede haber reintentos por errores temporales (); **4)** el MTA del destinatario actúa como **MDA** y **deposita el mensaje en el buzón** del destinatario en el servidor; **5)** más tarde el destinatario conecta su MUA a su servidor y lee: con **POP3** descarga los mensajes al cliente y opcionalmente los borra del servidor; con **IMAP** los consulta y el estado queda en el servidor; y con un **webmail** los lee en el navegador descargándolos por **HTTP/HTTPS**.

Análisis de la solución oficial.

✅ **Lo que está bien:** los cinco pasos están en el orden correcto, nombra al *user agent*, a los dos *mail server*, y menciona las tres formas de leer (POP3, IMAP, webmail). Para los 8 puntos de conceptual es suficiente.

⚠️ **Lo que la oficial NO dice y suma puntos:**

1. **No nombra los roles con sus siglas.** El enunciado del apartado a) pide **explícitamente** «cuáles son los roles». Saying «el user agent» y «el mail server» sin escribir **MUA**, **MTA** y **MDA** deja puntos en la mesa: la cátedra espera las tres siglas, con su glosa en inglés.
2. **No dice que SMTP es *push* y POP3/IMAP *pull*.** Es la distinción de diseño de toda la clase, y es lo que permite entender por qué son dos protocolos distintos.
3. **No menciona el reintento ni la cola.** Un significa «vuelvo a intentar», y eso explica por qué un correo puede tardar minutos u horas y llegar igual. **Y un genera el *bounce***, que es un correo de retorno que le avisa al emisor que su mensaje no llegó.
4. **No dice que el MDA se ejecuta en el servidor del destinatario.** Muchos alumnos dibujan un cuarto servidor que no existe.

> **Nota explícita contra la oficial:** la oficial es **correcta** para el apartado b) y su redacción es la que se corrige. Mi lectura difiere en un punto menor: la oficial trata «el mail server del destinatario» como un solo actor, y eso es correcto **porque el MTA y el MDA son el mismo proceso**. Pero si el enunciado pide **los roles**, hay que **separarlos en el diagrama aunque compartan máquina**, porque son responsabilidades distintas —uno decide a dónde va, el otro escribe el archivo en el buzón— y en una red real pueden estar separados (el *MDA* suele ser o un antivirus, no el).

3.2 ⭐ Examen 2008-12, Pregunta 4 (7 puntos) — el de POP3

> **Fuente:** Examen / — 20 de diciembre de 2008. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0812.pdf

Enunciado (texto literal del examen)

> **Pregunta 4 (7 puntos)**
>
> El protocolo POP3 transmite información de control por el mismo canal de datos. ¿Qué mecanismo utiliza para evitar que la información de señalización se confunda con los cuerpos de los mensajes? Justifique su respuesta.

Solución oficial (del PDF, textual)

> El protocolo POP3 se basa en el intercambio de mensajes de control (orientados a texto) y transferencias de datos (con contenido arbitrario). El protocolo reserva la secuencia para indicar el fin de mensaje. En caso que el mensaje contenga la misma secuencia como parte del mensaje se realiza el *stuffing* de la secuencia convirtiéndola en .

La resolución completa, con la oficial a la vista

Paso 1 — El problema, que es lo que el enunciado plantea en la primera frase.

` POP3 transmite CONTROL y DATOS por el MISMO canal TCP.

Control = "USER ana", "+OK", "RETR 1", "-ERR no such message" (líneas cortas) Datos = el mensaje entero, que es ARBITRARIO

Si los dos van por el mismo canal, el receptor tiene que poder distinguir "esto es un comando" de "esto es el contenido de un mensaje". `

Paso 2 — La respuesta oficial: se reserva una secuencia que no puede aparecer.

` POP3 reserva la secuencia

. (CRLF.CRLF en la notación de la oficial)

y la usa como el ÚNICO terminador de un mensaje multi-línea.

Es la misma idea que un "delimitador reservado" en cualquier formato de texto. `

Paso 3 — Pero acá está la trampa, y es lo que el enunciado insinúa.

` ⚠️ EL PROBLEMA REAL:

POP3 transfiere "contenido arbitrario" (esas son las palabras de la oficial). Y un mensaje de correo es un archivo de TEXTO que puede contener CUALQUIER cosa... incluido un renglón que sea solo un punto.

Si un mensaje contiene: Hola Pedro: . Saludos

...el POP3, al recibirlo, lee "." y piensa: "SE TERMINÓ EL MENSAJE". Lo que viene después ("Saludos") lo interpreta como una línea de CONTROL, es decir, como un comando que no existe, y la sesión se rompe.

O sea: el mecanismo de la oficial RESUELVE el problema de distinguir control de datos, pero NO RESUELVE el problema de que el terminador aparezca dentro de los datos. Para eso está el segundo paso de la oficial. `

Paso 4 — La respuesta oficial: el *dot-stuffing* (relleno de punto).

` La oficial: "En caso que el mensaje contenga la misma secuencia como parte del mensaje se realiza el stuffing de la secuencia convirtiéndola en CRLF..CRLF".

Es decir: ANTES de transmitir, el emisor busca toda línea que empiece con un punto y le AGREGA un punto más. Después, el receptor, al recibir, saca un punto de cada línea que empiece con punto.

original en el cable ——————————— ——————————— Hola Pedro: Hola Pedro: . (vacía) . .. ← era "." y se volvió ".." Saludos Saludos . . ← el terminador de verdad

Con esto, un "." suelto ya NUNCA puede producir ".", porque en el cable siempre viaja como "..". El terminador queda RESERVADO de verdad. `

Paso 5 — El ejemplo completo, que es lo que hay que saber escribir.

` Mensaje original de Ana (tiene un renglón que es un punto solo):

Hola Pedro: . Saludos Ana

Lo que Ana MANDA por POP3 con RETR (con stuffing):

Hola Pedro: .. Saludos Ana . ↑ el punto de cierre de la transacción, que no es del mensaje

Lo que POP3 le ENTREGA al cliente (deshaciendo el stuffing):

Hola Pedro: . Saludos Ana `

Respuesta redactada, lista para copiar:

> **a)** POP3 transmite la **información de control** (los comandos y las respuestas y) y la **información de datos** (el mensaje de correo) por el **mismo canal TCP**, que es texto orientado a líneas. Para que el receptor no confunda una cosa con la otra, **reserva la secuencia como único terminador de un mensaje multi-línea**: un renglón que sea un punto aislado marca el final del mensaje, y todo lo que venga después se interpreta como control.
>
> **b)** Justificación: como el contenido que se transfiere es **arbitrario**, un mensaje puede contener un renglón que sea un punto solo, y entonces la secuencia reservada aparecería **dentro de los datos** y el receptor cortaría el mensaje antes de tiempo, QPQreat interpretando el resto como comandos. Por eso, además de reservar la secuencia, el protocolo aplica **dot-stuffing**: el emisor **duplica el punto** al principio de toda línea que empiece con un punto (convirtiendo en), y el receptor **elimina ese punto de relleno** al recibir. Así, en el cable, **nunca** puede aparecer un que sea contenido: esa secuencia queda **reservada exclusivamente** para el fin de mensaje.

Análisis de la solución oficial

✅ **Lo que la oficial acierta:** identifica correctamente el problema (mismo canal para control y datos), la solución (secuencia reservada) y el segundo paso (dot-stuffing convirtiendo en). Los 7 puntos están ahí.

⚠️ **Lo que la oficial no dice y hay que agregar para el garantiza los 7 puntos:**

1. **No justifica POR QUÉ el punto es un buen carácter de relleno**, y esa es media pregunta. La respuesta: es un carácter **ASCII de 7 bits presente en todas las tablas**, tiene un **significado semántico** reconocible («fin de frase»), y **ya venía usándose** como terminador en los protocolos de teletipo. Agregar que el escapado es de **un solo carácter** (duplicar) es lo que hace que la solución sea mínima.
2. **No aclara la diferencia entre el terminador de POP3 y el de SMTP.** Los dos usan «un punto solo al principio de una línea», pero en **SMTP** el stuffing ocurre en la fase (el cliente hacia el servidor) y en **POP3** ocurre en la fase de transferencia (el servidor hacia el cliente). **La misma idea, dos protocols.**
3. **No nombra el término inglés**: *dot-stuffing*, y su primo en capa 2 es el *byte-stuffing* (el de PPP, que escapa con un). Ver §1.1 del Examen P3, que pregunta exactamente eso para PPP. **El mismo mecanismo, dos capas.**

> **Nota contra la oficial:** no hay diferencia conceptual, pero la oficial **redacta el stuffing como si se transformara la secuencia completa**. Lo que ocurre no es que se convierta en como una unidad, sino que **toda línea que empiece con un punto recibe un punto extra al principio**. La diferencia se nota en un mensaje que empieza con (Ejercicio 5): ahí el de la oficial se volvería (tres puntos), mientras que la

> regla real da por una línea que ya tenía dos puntos. **En la práctica dan lo mismo, pero la regla de la línea que empieza con punto es la que hay que saber.**

3.3 Problema relacionado — el Examen 2004-12, Ejercicio 2 (el diálogo SMTP)

> **Fuente:** Examen — 21 de diciembre de 2004. **Sin solución oficial publicada** en el material del curso; se resuelve abajo. *(solución propia)*

Enunciado (texto del examen). Se transcribe una sesión SMTP real en la que las líneas son del servidor y las del cliente:

```
S: 220 mta4.in.adinet.com.uy ESMTP Service (7.0.027) ready C: mail from:<origen@dominio.com.uy> S: 250 MAIL FROM:< origen@dominio.com.uy > OK C: rcpt to:<dest1@dominio.com.uy > S: 250 RCPT TO: <dest1@dominio.com.uy> OK C: rcpt to:<dest2@dominio.com.uy> S: 550 RCPT TO:<dest2@dominio.com.uy> User unknown C: data S: 354 Start mail input; end with <CRLF>.<CRLF> C: Subject: Hola a todos C: C: texto del correo..... C: . S: 250 <40AB06AF005B39F9> Mail accepted C: quit S: 221 mta4.in.adinet.com.uy QUIT
```

a) ¿Qué diferencia hay entre y ?

` MAIL FROM = el REMITENTE. Solo uno por mensaje. Va primero, porque el servidor necesita saber a quién avisarle si algo falla. RCPT TO = un DESTINATARIO. Uno o varios, y se puede repetir.

En la transcripción: un MAIL FROM, dos RCPT TO, y el segundo falla. `

b) ¿Qué pasó con ?

` 550 = error PERMANENTE: el usuario no existe. El mensaje NO se reintenta. Para ese destinatario en particular, el envío terminó ahí. Y como el 550 es permanente, el MTA va a generar un reporte de-states (bounce) para avisarle al emisor que su correo a dest2 no llegó.

Con un 4xx en lugar de 550 (por ejemplo 450 mailbox busy), el mensaje quedaría en una cola y se reintentaría más tarde: el envío NO se abandona. `

c) ¿Qué línea del mensaje está escapada, y por qué?

` El original decía algo como: .---------- o un renglón que empezaba con un punto

En el cable aparece "texto del correo....." con diez puntos pegados: el último de esos es un punto final de línea escapado, porque la línea del mensaje empezaba con un punto. Si no se hubiera escapado, el POP3/SMTPreceiver lo habría tomado como el terminador del mensaje. `

d) ¿Qué utilidad tiene enumerar usuarios válidos? *(lo que pregunta el apartado a) del enunciado)*

` ★ La utilidad legítima es la DEPURACIÓN: un administrador necesita saber si una lista de direcciones que tiene en un archivo todavía existe. El problema de esa dirección, el bounce por los 550, sirve justamente para eso.

★ La utilidad ABUSIVA es el enumerado de usuarios (el «harvesting»), que es una de las formas más viejas de spam: se prueban millones de direcciones en un dominio y se guarda las que no dan 550, para después

mandarles publicidad.

Y por qué el 550 es unaRespuesta para el atacante: distingue un error permanente de uno temporal, así que no se equivoca nunca.

★ La defensa es la del apartado b) del ejercicio: contar los 550 que generan por IP de origen y bloquear esa IP cuando pasa un umbral. `

Este problema es el complemento perfecto del Examen : los dos son sobre correo, y los dos aparecen juntos en la historia de la cátedra. Uno pregunta la arquitectura y los roles; el otro muestra una sesión real y pide qué hacer con la información que se filtra por ella.

PARTE 4 — CIERRE

📌 Resumen

1. **Tres roles**: **MUA** (escribe y lee), **MTA** (reenvía) y **MDA** (deposita en el buzón). El MTA y el MDA suelen ser el mismo proceso.
2. **El correo es *store-and-forward***: es el único protocolo de aplicación que **no** es extremo a extremo, y por eso un servidor intermedio puede leer el mensaje.
3. **SMTP es *push***, puerto **25** (servidor a servidor), **587** (usuario a servidor, con) y **465** (TLS implícito). **POP3 e IMAP son *pull***: **110/995** y **143/993**.
4. **El diálogo SMTP**: → (uno) → (uno o varios, y puede fallar sin tumbar el envío) → → el mensaje → → .
5. **Los códigos son de tres dígitos**: éxito, error temporal (cola y reintento), error permanente (bounce).
6. **MIME existe porque RFC 822 era ASCII de 7 bits**: los binarios podían traer y romper el transporte. dice **qué** es; dice **cómo viaja**.
7. ** / / **: QP para** texto **(legible, ~+20 %), base64 para** binario **(ilegible,** +33 %**).
8. ***** = ***versiones del mismo mensaje, el cliente*** *elige una**. ***** = ***contenidos distintos, el cliente los ve*** ***todos**.
9. **POP3** tiene **dos estados** y su `DELE` **solo marca**: borra de verdad en el `QUIT`. Sus comandos van **en claro**, y por eso el puerto seguro es **otro puerto (995)**.
10. **POP3 e IMAP se diferencian en dónde vive el estado**: en el **cliente** o en el **servidor**. Todo lo demás (carpetas, búsqueda, sincronización) sale de ahí.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Cuáles son los tres roles del correo y qué protocolo usa cada uno hacia el siguiente?

▶ Respuesta

2. ¿Por qué SMTP y POP3 no son el mismo protocolo, si los dos mueven correo?

▶ Respuesta

3. ¿Qué diferencia hay entre un y un en un ? ¿Y qué hace el MTA con cada uno?

▶ Respuesta

4. ¿Por qué existe MIME y qué problema concreto resuelve?

▶ Respuesta

5. ¿Cuándo va y cuándo va ? ¿Por qué no al revés?

▶ Respuesta

6. Explicá qué diferencia hay entre `POP3` e `IMAP`.

▶ **Respuesta**

7. ¿Por qué los comandos de POP3 van en claro, y qué se hace al respecto?

▶ **Respuesta**

8. Un mensaje contiene una línea que es un punto solo, en medio del cuerpo. ¿Qué pasa?

▶ **Respuesta**

🔗 Referencias

- **Examen / ** — Pregunta 2 (8 pts): ★ arquitectura del correo, los tres roles, los protocolos, y los cinco pasos del envío (el apartado a) está publicado como imagen)
- *Examen / ** — *Pregunta 4 (7 pts): ★ el de POP3 y el* dot-stuffing*
- *Examen ** — *Ejercicio 2: la transcripción real de una sesión SMTP con un y el* dot-stuffing*; el problema de enumerar usuarios válidos* (sin solución oficial publicada: solución propia)*
- *Examen ** — *Problema 2: un* front-end* que reparte sesiones POP3 entre tres servidores (aplicación del mismo tema)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.1 («Correo electrónico») y cap. 6 para la comparación de throughput
- **RFC 5321** — *Simple Mail Transfer Protocol* (2008, *Internet Standard*; reemplaza a) — *verificar*
- **RFC 5322** — *Internet Message Format* (2008; reemplaza a) — *verificar*
- **RFC 1939** — *Post Office Protocol version 3* (1996) — *verificar*
- **RFC 3501** (2001) → **RFC 9051** (2021), *IMAP4rev2* — *verificar* el estado vigente
- **RFC 2045-2049** — MIME (1996) — *verificar*
- **RFC 4954** — SMTP (2007) — *verificar* · **RFC 3207** — (2001) — *verificar*
- **IANA**, registro de puertos (verificado en §15): **25** SMTP · **110/995** POP3/POP3S · **143/993** IMAP/IMAPS · **465** submissions · **587** submission
- **Clases anteriores:** 8, 9 (HTTP) · 10, 11, 12 (DNS) — **Clase siguiente:** 14 — Seguridad en la aplicación: TLS, autenticación y correo seguro

Clase 14

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 Qué es una API, y por qué la capa 4 se programa así
1.2 La API de Berkeley/POSIX, llamada por llamada
1.3 y el orden de bytes
1.4 Multiplexación y demultiplexación: el papel de
1.5 ⚠️ Actualización a 2026
2.1 Un servidor TCP de eco completo
2.2 El cliente TCP
2.3 Un par UDP que intercambia datagramas
2.4 Ejercicio 1 — ¿Por qué este servidor no funciona?
2.5 Ejercicio 2 — ¿Por qué el del servidor es obligatorio?
2.6 Ejercicio 3 — Dos procesos en el mismo puerto
3.1 ⭐ Examen 2010-12, Problema 1 (30 puntos) —
3.2 Examen 2009-07, Problema 2 (30 puntos) — cliente anti-spam vía HTTP
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: B (Aplicación) · **Semana:** 5 · **Duración estimada:** 120 min **Correlativa:** Clase 13 (correo) · **Sigue con:** Clase 15 (DHCP + Simulacro 2) **Material de referencia:** Tanenbaum cap. 6 · RFC 9293 · §14

🎯 Al terminar esta clase vas a poder

1. Explicar por qué la capa de transporte se programa con una **API** y no con "registros" del protocolo, y qué es un *socket*.
2. Escribir en C un **servidor TCP** y un **cliente TCP** completos y compilables, con el orden exacto de las llamadas.
3. Explicar para qué sirven / / y qué falla exactamente si se olvidan.
4. Diferenciar el modelo **con conexión** (TCP: + +) del **sin conexión** (UDP: /).
5. **Analizar críticamente** el pseudocódigo en C de una solución oficial de examen, encontrar sus errores y **escribir la versión corregida**.

> **Prerequisito del examen:** (30 puntos — con sockets UDP), (30 puntos — cliente anti-spam vía HTTP con consulta DNS por UDP). **Los dos problemas de 30 puntos de la capa de aplicación son de programación.**

PARTE 1 — TEORÍA

1.1 Qué es una API, y por qué la capa 4 se programa así

La capa de transporte es la **primera en que el sistema operativo implementa servicios para los procesos**. Antes (física, enlace, red) los datos viajan sin que ningún proceso los pida: un paquete de red es un paquete, va a donde diga la cabecera IP y a nadie le importa quién lo pidió.

A partir de TCP/UDP aparecen **las questions**: "¿este datagrama es para el navegador o para el correo?". Como **la CPU tiene un solo programa ejecutándose a la vez**, el SO necesita un mecanismo para:

- que un proceso **reserve un puerto** y así pedirle al mundo "lo que llegue por este número es mío";
- **entregar** cada segmento que llega al proceso correcto;
- **permitir** que un proceso espere o intente enviar sin bloquear la CPU entera.

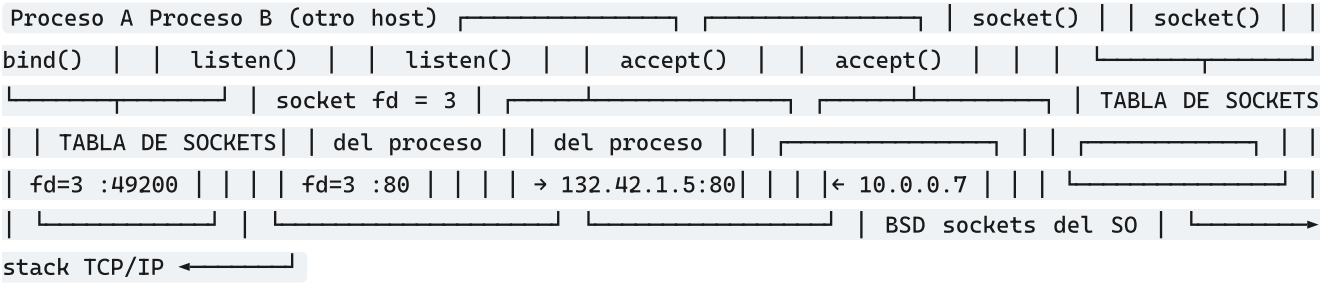
Ese mecanismo se llama **socket**. Y el conjunto de llamadas que el SO expone para usarlo se llama la **API de sockets**, que en Unix viene de **Berkeley (BSD)** y es el mismo en Linux, macOS y Windows (con diferencias de nombres).

> **Dato clave para el examen:** la API de sockets **nunca fue parte del protocolo TCP/IP**. Es una interfaz del sistema operativo. Por eso el mismo puede hablar TCP o UDP según qué creaste: **el protocolo lo decide el y el /, no el código de aplicación**.

Qué es un socket, en una frase

Un socket es **un par de descriptores de archivo más un estado**: el SO mantiene por cada socket las direcciones involucradas, el buffer de envío, el de recepción y la tabla de rutas interna (para TCP, el estado de la conexión completa).

En Unix, un socket es **un número entero** (el *file descriptor*), y por eso funciona con `read()`, `write()` y `close()` como cualquier archivo. Eso no es un detalle: es la razón de que la API sea tan simple.



1.2 La API de Berkeley/POSIX, llamada por llamada

Las que se usan siempre

Llamada	Qué hace	Disponible en
	Crea un socket y devuelve el file descriptor	TCP y UDP
	Fija la dirección local (IP y puerto)	ambos
	Marca el socket como **pasivo**: "acepto conexiones"	**solo TCP**
	Espera una conexión entrante y devuelve **un nuevo fd** para esa conexión	**solo TCP**
	Inicia la conexión hacia un destino	**solo TCP**
	Envía por una conexión **establecida**	ambos
	Recibe de una conexión establecida	ambos
	Envía **a un destino**, sin conexión previa	**solo UDP**
	Recibe **y quién lo mandó**	**solo UDP**
	Ajusta opciones del socket	ambos
	Libera el socket (cierra el fd)	ambos

El orden exacto: servidor TCP

Este es **el orden que se corrige en el examen**, y el orden importa porque cada llamada deja el socket en un estado que habilita la siguiente:

`1. socket(AF_INET, SOCK_STREAM, 0) | crear el socket. Devuelve fd = 3 (por ejemplo) ▼ 2. setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &uno, sizeof uno) | opcional pero recomendado: permite reabrir el puerto en el futuro | sin esperar al TIME_WAIT ▼ 3. bind(fd, (struct sockaddr*)&dir, sizeof dir) | FIJA IP y puerto LOCALES. OBLIGATORIO en el servidor: | sin esto el SO asigna un puerto efímero y nadie puede llegar ▼ 4. listen(fd, backlog) | pasa a estado PASIVO: el kernel empieza a ENCOLAR | conexiones entrantes en la cola de backlog ▼ 5. accept(fd, (struct sockaddr*)&cli, &len) | BLOQUEA hasta que llegue una conexión. | Devuelve un fd NUEVO, dedicado a esa conexión, | y llena cli con la IP y el puerto del cliente. | ¡OJO: el fd original NO se usa para atender la conexión! ▼ 6. send / recv sobre el fd NUEVO`

El error conceptual más común: creer que devuelve "la misma conexión" en el original. No: devuelve **un descriptor nuevo**. El original sigue siendo el "teléfono de la central" y se usa para el siguiente `accept()`.

El orden exacto: cliente TCP

`1. socket(AF_INET, SOCK_STREAM, 0) → fd 2. connect(fd, (struct sockaddr*)&destino, sizeof destino) | ¡NO HAY bind() NI listen()! | El SO elige un puerto efímero automáticamente | y hace el three-way handshake por debajo ▼ 3. send / recv sobre ese mismo fd`

El orden exacto: UDP, en ambos extremos

` SERVIDOR (desea conocer quién le habla)

1. socket(AF_INET, SOCK_DGRAM, 0) → SOCK_DGRAM, no SOCK_STREAM
2. bind(fd, (struct sockaddr*)&dir, sizeof dir) │ NECESARIO si querés un puerto fijo. │ Si no hacés bind, el SO asigna un puerto efímero │ y los clientes no pueden anticiparlo ▼
3. n = recvfrom(fd, buf, len, 0, (struct sockaddr*)&origen, &len_origen) │ ESPERA un datagrama. Además te dice QUIÉN lo mandó │ (IP y puerto), que es lo que TCP te da gratis ▼
4. sendto(fd, buf, n, 0, (struct sockaddr*)&origen, len_origen) │ respondés a la dirección que te devolvió recvfrom

CLIENTE 1. socket(AF_INET, SOCK_DGRAM, 0) 2. sendto(fd, buf, n, 0, (struct sockaddr*)&destino, sizeof destino) 3. recvfrom(fd, buf, len, 0, NULL, NULL) para escuchar respuestas `

> **Nota de examen:** y son a la vez "send/recv" **y** "connect/accept" combinados. No existen ni en UDP porque **no hay conexión que establecer**.

1.3 y el orden de bytes

La estructura

`c struct sockaddr_in { sa_family_t sin_family; // AF_INET para IPv4 in_port_t sin_port; // puerto, EN ORDEN DE RED struct in_addr sin_addr; // IP, EN ORDEN DE RED // ... y un padding para que sea del tamaño de sockaddr };`

El problema que resuelve

Las redes y las máquinas usan **ordenes de bytes distintos**:

- **big-endian** (*network byte order*): el byte más significativo **primero**. Es el estándar de todas las redes.
- **little-endian** (*host byte order*): el byte menos significativo primero. Es lo que usan x86, ARM en su modo normal, y prácticamente toda computadora de escritorio.

Si guardás el puerto 80 como número y lo mandás sin convertir:

` Puerto 80 en decimal = 0x0050

En little-endian en memoria: [50] [00] ← se manda primero el 0x50 Lo que recibe la red: 0x5000 = 20480 ← ¡puerto equivocado! En big-endian en memoria: [00] [50] ← correcto `

Por eso existen las conversoras:

Función	Qué hace
	host → network, **short** (16 bits, puertos)
	network → host, short
	host → network, **long** (32 bits, direcciones IP)
	network → host, long

> **Se usan igual en cliente y servidor.** No es "el servidor convierte y el cliente no". **Los dos lados** convierten, porque los dos escriben en un formato de red. Esto es una pregunta de examen clásica y casi

> todos la contestan mal.

y

`c // Forma clásica (devuelve INADDR_NONE si falla — no se puede distinguir error // de la dirección 255.255.255.255): struct in_addr a = inet_addr("132.42.120.5");

// Forma moderna y preferida (devuelve 1 si tuvo éxito, 0 si falló): struct in_addr a; inet_pton(AF_INET, "132.42.120.5", &a); `

1.4 Multiplexación y demultiplexación: el papel de

Aunque la teoría profunda está en la **Clase 16**, el punto de conexión con esta clase es uno solo y es el que más se pregunta:

` El SO mantiene una TABLA que asocia:

(protocolo, IP local, puerto local) → file descriptor

Cuando llega un datagrama UDP de la red: 1. el SO lee el puerto destino de la cabecera UDP 2. busca en la tabla (IP, puerto) 3. si está, ENTREGA los bytes al proceso dueño de ese fd 4. si no está, lo DESCARTA y a veces manda ICMP port unreachable `

es lo que **inscribe una fila en esa tabla**. Sin del lado servidor, no hay fila, y todo lo que llega se descarta.

El caso de TCP es un refinamiento: la tabla tiene — la **4-tupla** — porque dos clientes distintos pueden estar conectados al mismo puerto 80.

1.5 ⚠️ Actualización a 2026

Sigue igual — de memoria: - Las llamadas //////// /. **No cambió nada.** Los runtimes modernos (Go, Rust, Node, Python) las envuelven, pero el kernel sigue siendo BSD sockets. - La 4-tupla identifica una conexión TCP; la 2-tupla identifica un socket UDP **no conectado**. - / / , / , / . - El modelo cliente-servidor con /, y la programación multiproceso o multihilo.

Cambió: - **La herramienta de laboratorio.** Antes se usaba `tcpdump`; hoy lo estándar es **Wireshark** (libpcap/npcap, con filtros de display y **descodificación integrada** de QUIC, HTTP/3, TLS 1.3, DNSSEC y DoH). sigue para captura masiva o headless. - **Ya no se puede sniffear HTTP/3 con captura pasiva.** El tráfico va cifrado en el cable con claves derivadas de TLS 1.3, y las cabeceras HTTP/3 viajan en QPACK. Para "ver" una sesión de HTTP/3 hace falta **instrumentación en el endpoint** (registro de QUIC con claves de depuración, o un proxy MITM). **Esto es un cambio real de la pedagogía del sniffing**, y explica por qué se enseña HTTPS y no HTTP/3 en el laboratorio. - **IPv6 dual-stack y direcciones v4-mapped.** es la forma IPv6 de escribir una dirección IPv4. Con (default en Linux), un socket acepta **también** tráfico IPv4. Es cómodo y es la razón de una larga lista de vulnerabilidades de clase y de bugs de validación. **Cualquier curso de sockets de hoy tiene que cubrirlo.** - **El matiz sobre la 2-tupla.** La clave real de demultiplexación en el kernel es la **5-tupla** . La "2-tupla (IP, puerto)" es la simplificación clásica para un socket UDP no conectado. Y lo que rompe el modelo de verdad es **QUIC**: multiplexa **muchas conexiones en el mismo 5-tupla** usando el **Connection ID** (§5.1) en lugar del 5-tupla.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Un servidor TCP de eco completo

Qué tiene que hacer: aceptar conexiones, leer lo que le mandan, devolvérselo, y seguir escuchando.

`c

include

include

include

include

include

include

include

define PUERTO 80

define MAX 1024

int main(void) { int fd, fd_cli, n; struct sockaddr_in dir, cli; socklen_t len; char buf[MAX];

*/ 1. crear el socket: AF_INET (IPv4), SOCK_STREAM (TCP) */* fd = socket(AF_INET, SOCK_STREAM, 0); if (fd < 0) { perror("socket"); return 1; }

*/ 2. (opcional pero recomendado) permitir reabrir el puerto enseguida */* int uno = 1; setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &uno, sizeof uno);

*/ 3. fijar la direccion local */* memset(&dir, 0, sizeof dir); */ limpiar: sin esto sin_addr tiene basura */* dir.sin_family = AF_INET; dir.sin_port = htons(PUERTO); */ OJO: htons */* dir.sin_addr.s_addr = htonl(INADDR_ANY); */ cualquier interfaz /*

if (bind(fd, (struct sockaddr *)&dir, sizeof dir) < 0) { perror("bind"); return 1; }

*/ 4. pasar a modo pasivo y encolar hasta 5 conexiones pendientes */* if (listen(fd, 5) < 0) { perror("listen"); return 1; }

printf("Escuchando en el puerto %d...\n", PUERTO);

*/ 5. bucle: atender una conexion a la vez */* for (;;) { len = sizeof cli; fd_cli = accept(fd, (struct sockaddr *)&cli, &len); */* BLOQUEA */ if (fd_cli < 0) { perror("accept"); continue; }

*/ imprimir quien se conecto */* char ip[INET_ADDRSTRLEN]; inet_ntop(AF_INET, &cli.sin_addr, ip, sizeof ip); printf("Conexion desde %s:%d\n", ip, ntohs(cli.sin_port));

*/ 6. atender mientras el cliente no cierre */* while ((n = recv(fd_cli, buf, MAX - 1, 0)) > 0) { buf[n] = '\0'; printf("Recibido (%d bytes): %s\n", n, buf); send(fd_cli, buf, n, 0); */ el eco */* }

close(fd_cli); */ el fd NUEVO se cierra, el original sigue */* printf("Conexion cerrada.\n"); } } `

Puntos para justificar en un examen:

- **antes** dear : si no, el padding tiene basura y puede fallar.
- y : **las dos conversiones**, siempre.
- devuelve **0** cuando el cliente cerró (FIN): ese es el **único** caso en que es falso sin error.
- devuelve **un fd nuevo**. El original nunca se usa para atender.
- = = "escuchar en todas las interfaces".

2.2 El cliente TCP

`c

include

include

include

include

include

include

int main(void) { int fd, n; struct sockaddr_in dest; char buf[1024] = "Hola servidor\n";

fd = socket(AF_INET, SOCK_STREAM, 0); if (fd < 0) { perror("socket"); return 1; }

memset(&dest, 0, sizeof dest); dest.sin_family = AF_INET; dest.sin_port = htons(80); / *OJO: htons* / inet_pton(AF_INET, "132.42.120.5", &dest.sin_addr);

/ *NO hay bind() ni listen() en el cliente* / if (connect(fd, (struct sockaddr *)&dest, sizeof dest) < 0) { perror("connect"); return 1; }

send(fd, buf, strlen(buf), 0);

n = recv(fd, buf, sizeof buf - 1, 0); if (n > 0) { buf[n] = '\0'; printf("Respuesta: %s", buf); }

close(fd); return 0; } `

2.3 Un par UDP que intercambia datagramas

`c / ---------- *SERVIDOR UDP* ---------- /

include

include

include

include

include

include

int main(void) { int fd, n; struct sockaddr_in dir, origen; socklen_t len; char buf[1024];

fd = socket(AF_INET, SOCK_DGRAM, 0); / *SOCK_DGRAM, no STREAM* / if (fd < 0) { perror("socket"); return 1; }

memset(&dir, 0, sizeof dir); dir.sin_family = AF_INET; dir.sin_port = htons(5000); dir.sin_addr.s_addr = htonl(INADDR_ANY);

if (bind(fd, (struct sockaddr *)&dir, sizeof dir) < 0) { perror("bind"); return 1; }

for (;;) { len = sizeof origen; n = recvfrom(fd, buf, sizeof buf - 1, 0, (struct sockaddr *)&origen, &len); if (n < 0) { perror("recvfrom"); break; } buf[n] = '\0';

char ip[INET_ADDRSTRLEN]; inet_ntop(AF_INET, &origen.sin_addr, ip, sizeof ip); printf("De %s:%d -> %s\n", ip, ntohs(origen.sin_port), buf);

/ *responder a QUIEN VINO, no a una direccion guardada aparte* / sendto(fd, "recibido\n", 9, 0, (struct sockaddr *)&origen, len); } close(fd); return 0; } `

`c / ---------- *CLIENTE UDP* ---------- / int main(void) { int fd; struct sockaddr_in dest; char buf[1024]; int n;

fd = socket(AF_INET, SOCK_DGRAM, 0);

memset(&dest, 0, sizeof dest); dest.sin_family = AF_INET; dest.sin_port = htons(5000); inet_pton(AF_INET, "10.0.0.7", &dest.sin_addr);

sendto(fd, "hola\n", 5, 0, (struct sockaddr *)&dest, sizeof dest);

n = recvfrom(fd, buf, sizeof buf - 1, 0, NULL, NULL); if (n > 0) { buf[n] = '\0'; printf("Respuesta: %s", buf); }

close(fd); return 0; } `

Las 4 diferencias con TCP, que es lo que se pregunta:

	TCP	UDP
Tipo de socket		
/	sí	**no existen**
/	/	**/**
¿Sabés quién envía?	sí, la conexión	**solo si usás **

2.4 Ejercicio 1 — ¿Por qué este servidor no funciona?

`c int main(void) { int fd, fd_cli; struct sockaddr_in dir; char buf[100];

fd = socket(AF_INET, SOCK_STREAM, 0);

memset(&dir, 0, sizeof dir); dir.sin_family = AF_INET; dir.sin_port = 80; / <-- *falta htons* /

bind(fd, (struct sockaddr *)&dir, sizeof dir); listen(fd, 5); fd_cli = accept(fd, NULL, NULL); /* <-- falta listen? no, esta bien */ recv(fd, buf, 100, 0); / <-- usa el fd equivocado */ send(fd, buf, 5, 0); / <-- idem */ return 0; } `

Resolución — los 3 errores:

1. ** `dir.sin_port = 80;` sin `htons`. **En little-endian, 80 = `0x0050` se escribe en memoria, y al enviarlo al cable el otro extremo lee `0x5000 = 20480`. El servidor queda escuchando en el puerto 20480.** El error clásico.**
2. ** y sobre el fd del listening socket.** Hay que usar `fd_cli`, el descriptor que devolvió `accept`. Con `fd` el comportamiento no es lo que uno espera, y lo que corresponde al socket pasivo no es el eco.
3. ** con en la dirección** — eso está permitido, pero si después querés responder por no vas a saber a quién. Con TCP usás sobre la conexión, así que es aceptable; no es error, es una decisión.

2.5 Ejercicio 2 — ¿Por qué el del servidor es obligatorio?

Pregunta: el servidor UDP de arriba hizo al puerto 5000. ¿Qué pasaría si lo sacáramos?

Resolución. Sin `bind()`, el SO le asigna un **puerto efímero** (típicamente entre 32768 y 60999 en Linux). El cliente no tiene forma de saber cuál es, porque:

- no está en ningún protocolo de descubrimiento;
- el cliente UDP **no recibe nada** hasta que envía primero;
- y no hay ningún mensaje de error que le diga "el servidor está en el puerto X".

O sea: **el servidor quedaría escuchando en un puerto que nadie conoce**, y el servicio sería inaccesible. En TCP esto se(compensa parcialmente) porque el cliente conoce el puerto **80** por convención; en UDP **no hay puerto conocido por convención**.

Por eso es **obligatorio en el servidor** y **opcional en el cliente**.

2.6 Ejercicio 3 — Dos procesos en el mismo puerto

Pregunta: ¿qué pasa si dos procesos hacen al mismo puerto TCP?

Resolución. Sin `SO_REUSEADDR`, el **segundo `bind()` falla** con ("address already in use"). Eso es lo correcto: dos procesos no pueden atender el mismo puerto, porque las conexiones llegarían a cualquiera de los dos al azar.

Con (en Linux también), **ambos** pueden hacer bind y el SO reparte las conexiones entrantes entre ellos según el hash de la 4-tupla. Es lo que usan los servidores nginx para tener N workers. Ojo: en TCP **no** permite que dos sockets **escuchen** al mismo tiempo y roben traffic de forma insegura; en UDP **sí** es peligroso, porque cada datagrama se lo lleva un proceso cualquiera.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-12, Problema 1 (30 puntos) —

> **Fuente:** Examen / , 14 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf

Enunciado (texto del examen)

> Se desea implementar un sistema para brindar un servicio de conexión inalámbrica móvil a Internet, donde el usuario puede elegir el proveedor que le ofrezca un precio más bajo.
>
> Este sistema se implementará como una API en el equipo del usuario con la función y un servicio que se ejecuta en los puntos de acceso (AP).
>
> Para conectarse a Internet, la API del usuario deberá informarse del precio de cada punto de acceso a su alcance y luego conectarse al más barato. Si el punto de acceso más barato rechaza la conexión, se intentará con el siguiente, y así hasta lograr conectarse o no tener mas puntos de acceso disponibles.
>
> En tanto, el servicio del punto de acceso deberá comunicar su precio actual a los usuarios.
>
> Los puntos de acceso pueden aceptar un máximo de conectados.
>
> En los equipos de los usuarios se cuenta con las siguientes primitivas: - retorna la lista de APs dentro del alcance como una lista de SSID. - , se asocia al AP con el ssid dado. Los APs cuentan con un servicio DHCP por el cual en el momento de asociarse también se le asigna al usuario una dirección IP y direcciones de DNS. - , informa al sistema que el usuario se encuentra conectado a un AP y se pueden enviar datos.
>
> En los APs se cuenta con las siguientes primitivas: - , retorna la cantidad de usuarios conectados a este punto de acceso. - , devuelve el precio actual del punto de acceso.
>
> Se pide: 1. Especificar todos los mensajes necesarios para la comunicación. 2. Dibujar las máquinas de estado que especifiquen los procesos usuario y punto de acceso. 3. Implementar en un lenguaje de alto nivel las máquinas de estado definidas en el punto anterior. La comunicación entre usuarios y puntos de acceso se deberá realizar mediante sockets UDP. El usuario recibe las ofertas de precio en el puerto 12345.

Parte 1 — Los mensajes del protocolo

Solución oficial: , , , .

Análisis — por qué esos cuatro y no otros:

Mensaje	Quién lo manda	Para qué	Payload
****	AP → usuarios (broadcast)	"este soy yo y este es mi precio"	precio (int)
****	usuario → AP	"quiero conectarme a vos"	nada (o id de usuario)
****	AP → usuario	"aceptado, ya estás conectado"	id de sesión
****	AP → usuario	"no, tengo "	motivo

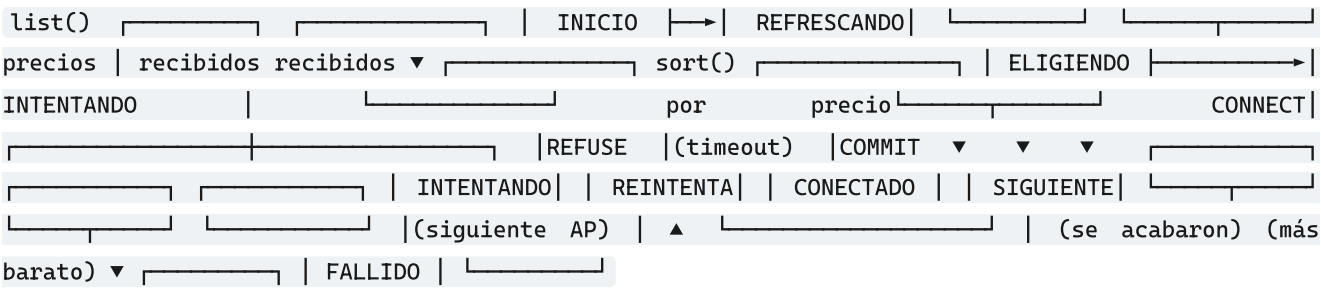
Observación 1 que suma puntos: la solución oficial no define ningún mensaje de **desconexión** ni de **error**. Un protocolo completo debería tener un (o el AP detecta la caída por timeout) y un para el caso de que el AP no responda. **Decí que el diseño oficial es mínimo y por qué eso es insuficiente en la práctica.**

Observación 2 — por qué es broadcast y los otros no:

- va por **broadcast** porque el usuario todavía **no sabe la IP del AP**: no la tiene, y es justamente el mensaje que se lo da. Si fuera unicast necesitaría conocer la IP destino. *(Y un broadcast requiere .)*
- **sí puede ser unicast**, porque en ese momento el usuario ya tiene la IP del AP (viene en el `OFFER`, o se la deduce del hecho de haberle recibido el).
- `COMMIT` y `REFUSE` van por unicast al usuario que mandó el `CONNECT`.

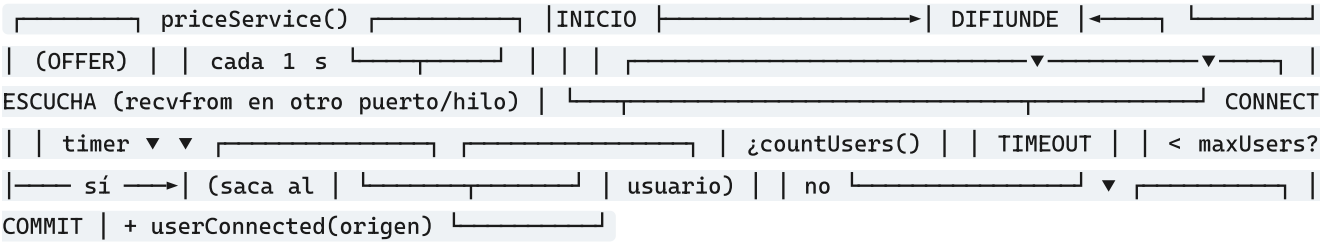
Parte 2 — Las máquinas de estado

Usuario:



Punto de atención: los tienen que distinguir "este AP no me sirve" de "no me pude comunicar". Un explícito avanza al siguiente AP de la lista ordenada; un **timeout** obliga a **reintentar el mismo** o a considerar que está caído. **La solución oficial no distingue los dos casos**, y esa es la diferencia entre un protocolo que funciona y uno que se cuelga.

AP:



Parte 3 — La implementación

La solución oficial, verbatim (con sus errores marcados):

`c void priceService(){ int socket = socket(AF_INET, SOCK_DGRAM, 0); while (true){ sleep(1); int broadcast = 1; setsockopt(socket, SOL_SOCKET, SO_BROADCAST, &broadcast, sizeof broadcast) // [E1]

struct sockaddr_in addr; socklen_t addr_size = sizeof addr; addr.sin_family = AF_INET; addr.sin_port = htons(12345); addr.sin_addr.s_addr = inet_addr(INADDR_BROADCAST); // [E2] char msg[5]; // [E3] sprintf(msg, %d, getPrice()); // [E4] int msg_size = strlen(msg); int sent_msg_size = sendto(socket, msg, msg_size, 0, (struct sockaddr*)&addr, addr_size); } } `

Los errores, uno por uno

#	Error	Consecuencia	Corrección
E1	**le falta el paréntesis de cierre**	No compila	
E2		** es una macro (un entero), no una cadena.** espera	
E3		Si el precio es 5 dígitos () **no entra**: desborda el buffer	o mejor
E4		**No compila**: faltan las comillas del formato	
E5	**adentro** del loop, junto al	El se re-setea 60 veces por minuto sin necesidad. Menor, pero es ruido	Sacar el **fuera** del loop
E6	se asigna y **nunca se usa**	El puede fallar () y el servicio no se entera	Verificar el retorno y decidir qué hacer
E7	No hay manejo de error en	Si falla, vale y todo lo demás falla en cascada	

El error E4 es el que más suma puntos, porque revela que **la solución oficial no fue probada**. Decirlo es el tipo de observación que el corrector valora.

La versión corregida de

`c

define PUERTO_PRECIO 12345

define MAX_PRECIO 16 / *alcanza para un int con signo + \0 /*

void priceService(void) { int fd, ssize_t enviados; int broadcast = 1; char msg[MAX_PRECIO];

fd = socket(AF_INET, SOCK_DGRAM, 0); if (fd < 0) { perror("socket"); return; }

/ *E5: esto se hace UNA vez, no en cada vuelta* / if (setsockopt(fd, SOL_SOCKET, SO_BROADCAST, &broadcast, sizeof broadcast) < 0) { perror("setsockopt"); close(fd); return; }

struct sockaddr_in destino; memset(&destino, 0, sizeof destino); destino.sin_family = AF_INET; destino.sin_port = htons(PUERTO_PRECIO); destino.sin_addr.s_addr = htonl(INADDR_BROADCAST); / *E2* /

for (;;) { sleep(1);

snprintf(msg, sizeof msg, "%d", getPrice()); / *E3 + E4* /

enviados = sendto(fd, msg, strlen(msg), 0, (struct sockaddr *)&destino, sizeof destino); if (enviados < 0) /* E6 / perror("sendto OFFER"); /* si es 0 o menor que strlen(msg), se perdio parte: hay que avisar */ } } `

Y corregida. Los errores de la oficial son más graves:

`c void connectCheaper(void) { int fd, i, r, n, n_ofertas = 0; int exito = 0; / *estado del protocolo* / char respuesta[MAX_PRECIO]; char **lista;

```
struct oferta {
    char *ssid;
    int   precio;
    struct sockaddr_in dir;            /* de donde vino el OFFER */
} *ofertas;

/* ================ 1. socket y bind al puerto 12345 ================ */
fd = socket(AF_INET, SOCK_DGRAM, 0);
if (fd < 0) { perror("socket"); return; }

struct sockaddr_in local;
memset(&local, 0, sizeof local);
local.sin_family      = AF_INET;
local.sin_port        = htons(PUERTO_PRECIO);
local.sin_addr.s_addr = htonl(INADDR_ANY);
if (bind(fd, (struct sockaddr *)&local, sizeof local) < 0) {
    perror("bind"); close(fd); return;
}

/* E-A: la oficial compara con "==" sobre punteros. Eso compara DIRECCIONES
   de memoria, no el contenido de las cadenas. SIEMPRE strcmp. */
/* E-B: ademas el literal va sin comillas, asi que ni siquiera compila. */
/* E-C: la oficial llama recv() en vez de recvfrom(), y no guarda de quien
   vino el OFFER: sin eso no sabe a que AP responderle el CONNECT. */
/* E-D: no hay timeout. Si el AP mas barato esta caido, recv() bloquea
   para siempre y el usuario se queda sin conexion. */
/* E-E: declara "int data_size" y lo usa sin inicializar. */

/* ================ 2. descubrir APs y recoger precios ================ */
lista = list();
n = length(lista);
ofertas = malloc(n * sizeof *ofertas);

/* E-D: sin timeout, un AP caido bloquea el bucle entero */
struct timeval tv = { .tv_sec = 2, .tv_usec = 0 };
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);

for (i = 0; i < n; i++) {
    struct sockaddr_in de;
    socklen_t le = sizeof de;

    associate(lista[i]);            /* el AP ya tiene nuestra IP por DHCP */

    /* E-C: recvfrom, no recv: necesito la direccion del emisor */
    r = recvfrom(fd, respuesta, sizeof respuesta - 1, 0,
                 (struct sockaddr *)&de, &le);
    if (r <= 0) continue;          /* nadie respondio: lo salto */

    respuesta[r] = '\0';

    /* E-F: la oficial no valida que el OFFER sea un numero */
    char *fin;
    long pr = strtol(respuesta, &fin, 10);
    if (fin == respuesta || *fin != '\0' || pr < 0) continue;

    ofertas[n_ofertas].ssid   = lista[i];
    ofertas[n_ofertas].precio = (int)pr;
    ofertas[n_ofertas].dir    = de;
    n_ofertas++;
}

if (n_ofertas == 0) {
    printf("no hay APs que respondan\n");
```

```
    free(ofertas); close(fd); return;
}

sort(ofertas);                      /* de menor a mayor precio */

/* ================ 3. intentar en orden de precio ================ */
for (i = 0; i < n_ofertas && !exito; i++) {
    char *req = "CONNECT";
    char resp[MAX_PRECIO];

    /* E-G: hay que asociarse ANTES de pedir la conexion; es lo que hace
       posible que el DHCP del AP exista cuando le llegue el CONNECT */
    associate(ofertas[i].ssid);

    sendto(fd, req, strlen(req), 0,
           (struct sockaddr *)&ofertas[i].dir, sizeof ofertas[i].dir);

    /* PROBLEMA DE DISEÑO que la oficial ignora: el socket esta bindeado al
       puerto 12345, pero el AP responde a NUESTRO PUERTO EFIMERO, no al
       12345. Este recv se queda esperando donde no va a llegar nada.

       Dos formas correctas de resolverlo:
         (a) hacer connect() sobre el socket UDP hacia cada AP: el SO fija
             el peer y las respuestas llegan por este mismo socket;
         (b) crear un socket efimero aparte para la fase de conexion. */
    r = recvfrom(fd, resp, sizeof resp - 1, 0, NULL, NULL);

    if (r > 0) {
        resp[r] = '\0';
        /* E-A corregido: strcmp, y el literal con comillas */
        if (strcmp(resp, "COMMIT") == 0) {
            connected();            /* la API confirma la conexion */
            exito = 1;
        }
        /* si es REFUSE, el for sigue con el siguiente AP mas barato */
    }
    /* si hubo timeout (r < 0), tambien seguimos con el i siguiente */
}

if (!exito) printf("No se pudo conectar a ningun AP disponible.\n");

free(ofertas);
close(fd);
```

}

Los 7 errores de la oficial, resumidos para el examen:

1. en vez de al comparar "COMMIT" (y el literal sin comillas).
2. en vez de : **no guarda la dirección del AP**, así que no puede dirigirle el `CONNECT`.
3. **No hay timeout**: si el AP más barato no responde, bloquea para siempre.
4. **sin inicializar**.
5. se llama en el bucle de precios pero **no se vuelve a llamar** antes del `OFFER`.
6. No valida que el OFFER recibido sea un número (sobre basura).
7. **El del AP vuelve al puerto efímero**, no al 12345 de los : la oficial no resuelve ese desajuste entre las dos fases.

> El punto 7 es el más fino y el que mejor diferencia a quien entendió el problema. La solución de un examen tiene que atacar **los dos lados** (AP y usuario), y si solo manda mensajes, el usuario nunca recibe las respuestas.

3.2 Examen 2009-07, Problema 2 (30 puntos) — cliente anti-spam vía HTTP

> **Fuente:** Examen / , julio de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0907.pdf

El enunciado describe una aplicación web de detección de spam: por cada correo que llega, se consulta y, si no es spam, se guarda.

Apartado a) — Implementar la consulta DNS por UDP

Enunciado: "*Implementar la invocación al DNS necesaria para obtener la IP del servidor de correo dado su nombre de dominio. Esto debe realizarse en una función en el cabezal: que devuelva la dirección IP en su formato decimal separado por puntos. Se debe realizar la invocación vía UDP, la misma que se utiliza para consultar cualquier otro host.*"

Solución oficial (resumida del PDF):

```
c string getMailServer(string hostName) { int s = socket(AF_INET, SOCK_DGRAM, 0); struct sockaddr_in dest; /* armar la consulta DNS para hostName, tipo A */ enviar(s, hostName, (struct sockaddr *)&dest, sizeof dest); /* recibir la respuesta */ int r = recv(s, respuesta, sizeof respuesta, 0); /* ... buscar la primer entrada de tipo A ... */ return(ip_en_puntos, buf); }
```

Resolución completa. La estructura del pseudocódigo, con la cuenta de bytes que hay que hacer:

` PASO 1 — Socket UDP y destino

s = socket(AF_INET, SOCK_DGRAM, 0) destino.sin_family = AF_INET destino.sin_port = htons(53) ← DNS está en el puerto 53 destino.sin_addr = la IP del servidor DNS local (¡NO la del nombre que buscamos!exito)

PASO 2 — Armar la cabecera del mensaje DNS

El mensaje DNS empieza con 12 bytes de cabecera, en ORDEN DE RED: ID (2 bytes) = un número cualquiera, ej. 0x1234 flags (2 bytes) = 0x0100 → QR=0 (consulta), RD=1 (quiero recursión) QDCOUNT (2 bytes) = 0x0001 → una pregunta ANCOUNT, NSCOUNT, ARCOUNT = 0

Después va el nombre a consultar, en formato "etiqueta": 3www3fing4edu2uy0 (cada byte dice cuántos caracteres tiene la etiqueta; el 0 final marca el fin del nombre)

Después: QTYPE (2 bytes) = 0x0001 para tipo A QCLASS (2 bytes) = 0x0001 para clase IN

PASO 3 — Enviar y recibir

sendto(s, consulta, largo, 0, (struct sockaddr *)&destino, sizeof destino) r = recv(s, respuesta, MAX, 0) ← con recvfrom si querés saber qué servidor respondió

PASO 4 — Buscar la respuesta

En la respuesta, el ID DEBE coincidir con el que mandaste (si no, se descarta: puede ser una respuesta vieja de otra consulta). ANCOUNT dice cuántas respuestas hay. Con ANCOUNT = 0 y RCODE = 3 (NXDOMAIN) no existe ese nombre. La respuesta trae el mismo nombre, luego TYPE=1, CLASS=1, TTL (4 bytes) y finalmente los 4 bytes de la IPv4.

PASO 5 — Formatear

Convertir los 4 bytes a decimal separado por puntos con snprintf. Devolver ese string, o la cadena vacía si fallo. `

Errores frecuentes que se\columnwidth en este tipo de ejercicio:

- **Mandá ** en vez de : el enunciado dice explícitamente UDP.
- **Usar el puerto equivocado**: es el **53**, y hay que convertirlo con `htons`.
- **No verificar el ID**: la respuesta que llegue puede ser de una consulta anterior. El enunciado no lo dice, pero es la robustez mínima.
- **Devolver la primera IP del `Answer`** en vez de la del `Additional`.

Apartado c) — Dónde llamar a

Enunciado: modificar el pseudo del SMTP para invocar donde corresponda y guardar el correo solo si no es spam.

Resolución. El punto clave es **dónde**: la verificación va **después** de recibir el mensaje completo pero **antes** de escribirlo en el archivo.

` SMTP del cliente (pseudo de FING)

... else if (line.startsWith("DATA")) {

if (isSpam(getMailServer(hostRemitente), subject)) { / *no se guarda nada: se descarta* / rejection = "550 mensaje clasificado como spam"; } else { escribirCorreo(archivo, mensaje); OK = "250 mensaje recibido"; } ... } `

Las tres decisiones que hay que justificar:

1. **Después de `DATA`, no antes.** Antes no tenés el `Subject` ni el cuerpo, que es lo que necesita para clasificar.
2. **Antes de escribir en disco.** Si guardás y después descartás, el archivo ya quedó escrito: eso es precisamente lo que un antivirus bien hecho evita.
3. **El se llama sobre el remitente**, porque la API del servicio recibe la IP del servidor de correo del que vino el mensaje, no el nombre.

> **Tope para 30 puntos:** el 1er apartado vale más (es el que tiene la parte de programación), y el punto 3 del listado de errores es el que más suma: demuestra que pensaste en **por qué** la DNS se consulta y no solo en **cómo**.

PARTE 4 — CIERRE

📌 Resumen

1. Un **socket** es un par de descriptores de archivo más un estado que el SO mantiene: direcciones, buffers y (en TCP) la tabla de la conexión.
2. La API de sockets **no es parte de TCP/IP**: es del sistema operativo.
3. **Servidor TCP**: → → → → → /. ** devuelve un fd NUEVO.**
4. **Cliente TCP**: → → /. **Sin ni .**
5. **UDP**: , /, **no hay ni** , y es lo que te dice quién te mandó el datagrama.
6. / **en los dos lados**: la red es big-endian y el host little-endian. Sin la conversión, el puerto 80 llega como 20480.
7. es **obligatorio en el servidor** (si no, queda en un puerto efímero que nadie conoce) y **opcional en el cliente**.
8. **Sin** , un segundo al mismo puerto falla con .
9. La clave real de demux del kernel es la **5-tupla**; la "2-tupla de UDP" es la simplificación clásica. QUIC la reemplaza por el **Connection ID**.
10. **Wireshark** es la herramienta de laboratorio, y **HTTP/3 ya no se puede sniffear** con captura pasiva.

✅ Autoevaluación

1. ¿Por qué el servidor TCP hace pero el cliente no?

▶ Respuesta

2. Si escribís `dir.sin_port = 80;` sin `htons`, ¿qué puerto queda escuchando el servidor? ¿Y si la máquina fuera big-endian?

▶ Respuesta

3. ¿Qué devuelve y por qué el descriptor que devuelve no es el mismo que se le pasó?

▶ Respuesta

4. ¿Qué dos funciones de la API no existen para UDP, y por qué?

▶ Respuesta

5. ¿Por qué la solución oficial de usa para comparar con "COMMIT", y qué hay que usar?

▶ Respuesta

6. En el problema de `connectCheaper()`, ¿qué problema tiene el hecho de que los lleguen al puerto 12345 pero las respuestas (/) no regresen allí?

▶ Respuesta

🔗 Referencias

- **Examen / ** — Problema 1 completo (30 pts,), P2 (8 pts, mux/demux UDP)
- **Examen / ** — Problema 1 (30 pts, sockets y AX.25), Problema 2 (30 pts, anti-spam por HTTP)
- **Examen / ** — Problema 1: dos tarjetas de red en un mismo host
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6
- ** §14** — sockets, Wireshark, IPv6 dual-stack, 5-tupla vs. Connection ID
- **Clase siguiente:** 15 — DHCP, configuración y Simulacro 2

Clase 15

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El problema
1.2 BOOTP, el antecedente
1.3 Por qué UDP y por qué los puertos 67 y 68
1.4 El proceso DORA, mensaje por mensaje
1.5 El lease: qué es y por qué existe
1.6 Opciones DHCP que hay que saber
1.7 Qué pasa si el DHCP falla
1.8 ⚠️ Actualización a 2026
2.1 Ejercicio 1 — La traza DORA completa con direcciones concretas
2.2 Ejercicio 2 — Los temporizadores del lease
2.3 Ejercicio 3 — Cuántas direcciones necesita la red
2.4 Ejercicio 4 — Dos servidores DHCP
3.1 ⭐ Examen 2011-02, Pregunta 2 (10 puntos) — DHCP
4.1 Repaso del bloque B (clases 8 a 15)
4.2 Los 8 errores más frecuentes del bloque
4.3 📝 Simulacro 2 — Examen completo
🔗 Referencias

Bloque: B (Aplicación) · **Semana:** 5 · **Duración estimada:** 130 min **Correlativa:** Clase 14 (sockets) · **Sigue con:** Clase 16 (UDP y multiplexión) **Material de referencia:** Tanenbaum cap. 5 (DHCP) y cap. 7 · `ACTUALIZACIONES.md` §5

🎯 Al terminar esta clase vas a poder

1. Explicar por qué un host necesita **DHCP** y qué problema resuelve frente a tener la dirección configurada a mano.
2. Describir el proceso **DORA** mensaje por mensaje, con los cuatro nombres, quién los manda, por qué son broadcast y qué lleva cada uno.
3. Justificar por qué DHCP usa **UDP** y por qué sus puertos son **67 y 68**.
4. Explicar qué es un **lease**, por qué existe y qué pasa cuando expira.
5. Explicar qué pasa cuando el DHCP **falla** y por qué existe el rango `169.254.0.0/16`.
6. Resolver un examen completo de 3 horas con cronómetro.

> **Prerequisito del examen:** `2011-02 Pregunta 2` (10 pts — DHCP con el intercambio de mensajes), `2010-03 Pregunta 4` (8 pts — agente y gestor), `2007-03 Pregunta 3` (10 pts — RARP, el antecedente de DHCP).

PARTE 1 — TEORÍA

1.1 El problema

Un host recién enchufado tiene **cero configuración de red**. No tiene:

- dirección IP
- máscara de subred
- puerta de enlace (gateway)
- servidor DNS
- nombre de dominio

Y sin esas cinco cosas **no puede hablar con nadie**. Ni siquiera puede pedir ayuda, porque pedir ayuda requiere una dirección de la que hacerlo. **Ese es el problema de arranque**, y es la razón de que DHCP exista.

Las dos salidas posibles:

Salida	Cómo	Consecuencia
Configurar a mano	un técnico va y escribe los valores	escala a **nADA**: con 500 laptops no escala
Preguntar por red	el host **pregunta** y alguien le responde	escala a toda la organización

DHCP es la segunda. La idea central es simple y es la que hay que saber:

> **Un host puede usar el Limited Broadcast para descobrir, porque el Limited Broadcast no requiere conocer la red en la que estás.**

`255.255.255.255` es una dirección de broadcast **válida en cualquier red**. Un host que no sabe nada de su subred puede mandar ahí y todos la reciben.

1.2 BOOTP, el antecedente

Antes de DHCP (RFC 951, 1984) estaba **BOOTP** (*Bootstrap Protocol*). La idea era la misma pero con una diferenciaFatal: **el servidor tenía una tabla fija** de direcciones por MAC.

```
BOOTP:  tabla en el servidor
     MAC_A → 192.168.1.25
     MAC_B → 192.168.1.26        ← estático, para siempre

DHCP:   el servidor no sabe nada
     MAC_C pide → recibe 192.168.1.27 con un lease de 24 h  ← dinámico
```

El problema de BOOTP era que **no escalaba**: un servidor tenía que tener una entrada por cada máquina, para siempre. Si una notebook se iba a casa, su dirección quedaba reservada inútilmente.

DHCP (RFC 2131, 1997) cambió dos cosas:

1. **El lease**: la dirección se presta por un tiempo y se renueva.

2. **El servidor no necesita tabla**: cualquier servidor DHCP puede atender cualquier cliente. La asignación se guarda en el cliente.

1.3 Por qué UDP y por qué los puertos 67 y 68

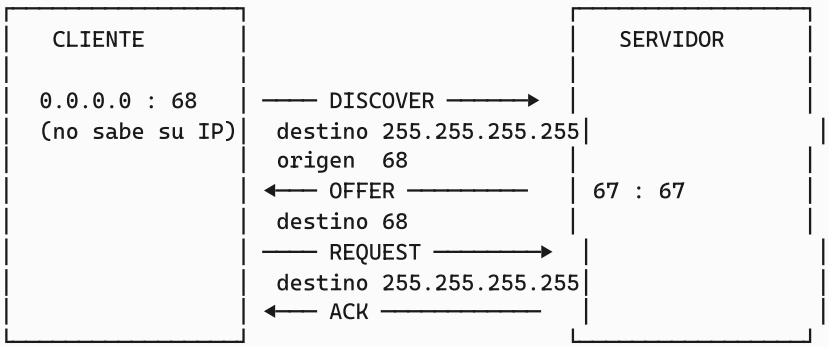
DHCP usa UDP. Es la primera pregunta que se hace, y la respuesta tiene tres partes:

1. El cliente no tiene dirección IP. No puede abrir una conexión TCP, porque abrir una conexión TCP requiere mandar un SYN **desde** alguna dirección origen. El `bind()` de un socket TCP necesita una IP local. **No la tiene.** UDP no necesita ninguna dirección previa para mandar el primer datagrama.

2. El servidor también habla desde una dirección desconocida. El cliente no sabe la IP del servidor DHCP. Y el cliente **no puede descubrir** la dirección del servidor sin ya tener una. De ahí el Limited Broadcast como destino.

3. La transacción es de un solo mensaje en cada sentido. No hay un diálogo largo que justifique el control de conexión de TCP. Y si se pierde un datagrama, el cliente simplemente **reintenta**: el protocolo tiene sus propios temporizadores y su propia lógica de reintento.

Los puertos



- **68** (`bootpc`) es el puerto del **cliente**. Lo elige porque el cliente **no tiene IP asignada**: el puerto 68 es una convención que funciona sin necesidad de conocer la red. Es el puerto equivalente al 1024 mínimo en TCP.
- **67** (`bootps`) es el del **servidor**, un puerto **reservado y conocido**.

> **El detalle fino que se pregunta:** en el `DISCOVER`, el campo *dirección origen IP* del datagrama IP es `0.0.0.0`, no la IP del cliente. Porque todavía no la tiene. Y el campo *dirección destino* del datagrama IP es `255.255.255.255` (Limited Broadcast), y en el datagrama UDP el puerto origen es **68**. Los tres campos juntos son la definición del "origen desconocido".

1.4 El proceso DORA, mensaje por mensaje

Las cuatro letras son **D**ISCOVER, **O**FFER, **REQUEST** y **ACK**.

```
TIEMPO ──────────────────────────────────────────────────────────►

CLIENTE (0.0.0.0 : 68)              SERVIDOR
──────────────────────────────      ──────────────────────────
                                      (escucha en : 67)

1) DISCOVER ──────────────────►   "hay alguien que necesita
     dst 255.255.255.255 : 67        una dirección. Tengo una libre."
     src 0.0.0.0 : 68
     mi MAC, mi nombre de dominio

                                   decide su plan de numeración
                                   y elige la dirección

2)              ◄──────────────── OFFER
                     dst 255.255.255.255 : 68
                     src <ip_server> : 67
                     IP ofrecida, máscara, gateway,
                     DNS, nombre de dominio,
                     duración del lease, opciones

3) RECORDATORIO: el cliente guarda la oferta
   y se queda con la de MENOR duración de lease
   (evita que dos servidores distintos Denial)  ← punto clave

3) REQUEST ──────────────────►   "acepto la oferta de <ip_server>".
     dst 255.255.255.255 : 67      Decir cuál ACEPTAS es lo que
     src 0.0.0.0 : 68              permite al ganador saber
     mi MAC, la IP que elijo       que ganó. Por eso es BROADCAST.
                                    Los demásven el "REQUEST" y
                                   /libran la dirección.

4)              ◄──────────────── ACK
                     src <ip_server> : 67
                     "confirmado, esa IP es tuya por N horas"

AHORA SÍ: el cliente puede hacer bind() a esa IP y usar TCP.
```

Por qué el `REQUEST` es broadcast y el `OFFER` casi siempre también

Este es **el punto más importante del protocolo** y el que más se confunde:

El `REQUEST` va por broadcast obligatoriamente, aunque el cliente ya sepa la IP del servidor. ¿Por qué? Porque si fuera unicast, cada servidor sabría "me eligieron a mí" o "no me eligieron", y el que **no** fue elegido sabría que otro servidor le ganó. Eso es exactamente lo que se quiere evitar: que un cliente que eligió el precio más bajo (como en el problema de `connectCheaper()` de la clase 14) no se lo anuncie a los demás. El broadcast mantiene el secreto delWinner.

El `OFFER`, en cambio, se puede mandar por unicast si el servidor ya tiene la MAC del cliente (y a veces la tiene, de un `REQUEST` anterior). **Many DHCP implementations mandan OFFER por broadcast**, porque el cliente recién arrancando puede no tener todavía su ARP resuelto. Es una decisión de implementación.

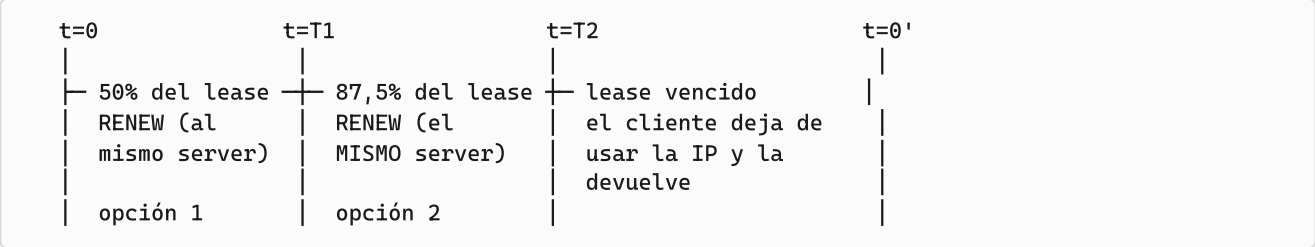
Los 4 mensajes en detalle

#	Mensaje	De → a	IP origen → destino	Puerto UDP	Contenido clave
1	**DISCOVER**	cliente → todos	`0.0.0.0` → `255.255.255.255`	68 → 67	MAC del cliente, nombre de dominio
2	**OFFER**	servidor → cliente	ip_server → `255.255.255.255`	67 → 68	IP, máscara, gateway, DNS, lease
3	**REQUEST**	cliente → todos	`0.0.0.0` → `255.255.255.255`	68 → 67	qué servidor eligió
4	**ACK**	servidor → cliente	ip_server → `255.255.255.255`	67 → 68	confirmación final

Todos van por `255.255.255.255` (Limited Broadcast) menos el `OFFER`, que puede ir por unicast. Es la única manera de que funcione sin conocer la red.

1.5 El lease: qué es y por qué existe

El **lease** es el **período de tiempo** durante el cual la asignación es válida.



El cliente tiene **dos mecanismos de renovación**:

- **T1 = 50 %** del lease: manda `REQUEST` **al mismo servidor** que le dio la IP. Es una Optimization: si funciona, no hay nada que cambiar.
- **T2 = 87,5 %** del lease: si el primero falló, manda `REQUEST` a **cualquier servidor** (por broadcast), pidiendo cualquiera.
- **Al vencer**: el cliente deja de usar la IP. **Y no la devuelve**: simplemente deja de renovar. El servidor la vuelve a poner en el pool cuando expire su registro interno.

¿Por qué existe el lease? Porque un host puede **desaparecer sin avisar**: se apaga, se rompe, se va de viaje. Si la asignación fuera fija (BOOTP), esa dirección quedaría **reservada para siempre** para un equipo que no existe. Con lease, el problema se resuelve solo.

1.6 Opciones DHCP que hay que saber

El `OFFER` y el `ACK` llevan un campo **options** con pares (código, longitud, valor). Los que se preguntan:

Opción	Código	Qué le dice al cliente
Subnet Mask	1	la máscara, para que sepa qué es su red local
Router (gateway)	3	a quién le pregunta por destinos fuera de su red
Domain Name Server	6	la IP del resolver DNS
Hostname	12	el nombre que el cliente quiere tener
Domain Name	15	el dominio local
Requested IP Address	50	el cliente pide una IP concreta (arranque de un equipo específico)
IP Address Lease Time	51	la duración del lease, en **segundos**
Message Type	53	1=DISCOVER, 2=OFFER, 3=REQUEST, 5=ACK
Server Identifier	54	**qué servidor** hizo la OFFER (clave para el REQUEST)
Parameter Request List	55	qué opciones quiere el cliente
Renewal (T1)	58	el 50 % del lease
Rebinding (T2)	59	el 87,5 % del lease

> **El par (53, 54) es el corazón del protocolo.** El campo *Message Type* dice **qué** mensaje es. Y el *Server Identifier* dice **quién** lo mandó, que es lo que permite que el `REQUEST` del cliente diga "acepto la de **ese** servidor".

Los tres pasos del TFTP

DHCP no es un protocolo de sesión. **Cada mensaje es un mensaje TFTP** (*Trivial File Transfer Protocol*), que funciona sobre UDP. Los pasos:

```
1. REQUEST   el cliente pide su configuración
2. OFFER     el servidor ofrece una dirección
3. ACK       el servidor confirma

(y el DISCOVER es un REQUEST con Message Type = DISCOVER)
```

TFTP existe porque se necesitaba **algo muy simple** para que un cliente que no tiene IP pudiera hablar con un servidor. No tiene ni número de secuencia ni confirmaciones: **DHCP se encarga de reintentar** si algo se pierde, con sus propios temporizadores.

1.7 Qué pasa si el DHCP falla

Si el cliente no recibe ninguna `OFFER` (no hay servidor DHCP en la red), hace lo que hace **todo** protocolo: **reintenta**. Periódicamente manda un nuevo `DISCOVER`.

Pero si la red **no tiene ningún DHCP**, el host queda sin dirección. Y como no puede pedir nada, se autoconfigura:

```
┌──────────────────────────────────────────────────────────┐
│  APIPA  (Automatic Private IP Addressing)                │
│  Microsoft inventó el nombre; el nombre real es          │
│  "link-local" y el rango es 169.254.0.0/16               │
└──────────────────────────────────────────────────────────┘

El cliente:
  1. se elige una IP al azar dentro de 169.254.1.0 – 169.254.254.255
     (o prueba una y hace ARP para ver si nadie responde)
  2. pone su máscara como /16
  3. NO tiene gateway  →  no puede salir de la red local
  4. no tiene DNS      →  no resuelve nombres
```

¿Por qué `169.254.0.0/16` y no otra cosa? Porque la IANA lo reservó específicamente para esto. Y hay una razón de diseño más profunda:

> **Es un rango que nunca se va a rutear en Internet.** Aunque por un error de configuración un paquete con destino `169.254.x.x` llegara a Internet, ningún router lo va a reenviar. O sea: **si la autoconfiguración falla, el host no puede molestar a nadie.** Es un rango "atrapado" por diseño.

Es un detalle de diseño elegante que vale la pena conocer: **el rango de autoconfiguración es elegido para que sea inofensivo.**

1.8 ⚠️ Actualización a 2026

Sigue igual — de memoria: - El proceso **DORA** completo, los 4 mensajes, en ese orden. - **UDP**, puertos **67** y **68**, y el Limited Broadcast `255.255.255.255`. - El `DISCOVER` con IP origen `0.0.0.0`. - El **`REQUEST` por broadcast** y la razón de ser (el secreto de qué servidor ganó). - El lease con **T1 = 50 %** y **T2 = 87,5 %**, y los dos mecanismos de renovación. - **`169.254.0.0/16`** como rango link-local de autoconfiguración, no ruteable. - Los campos **Message Type** y **Server Identifier** como el corazón del protocolo.

Cambió: - **DHCPv6** existe y usa **UDP 546 (cliente) y 547 (servidor)**, con **multicast** en vez de broadcast. No es "el DHCP de IPv6": es un protocolo distinto, porque en IPv6 el cliente **ya tiene una dirección link-local derivable del MAC** y no necesita el Limited Broadcast para nada. - **IPv6 privacy extensions y SLAAC**: en IPv6 la autoconfiguración es **nativa** (`RFC 4862`) y no depende de ningún servidor DHCPv6. El "SLAAC" genera la dirección a partir del prefijo que anuncia el router en los RA. - **La era del CGNAT** (Carrier-Grade NAT, que grew con el agotamiento de IPv4 en RIPE en noviembre de 2019) cambió el contexto: los ISP dan a los usuarios direcciones **compartidas**, lo cual hace que algunos servicios de TV por IP y VoIP necesiten otras soluciones. No afecta al DHCP del cliente final, pero explica por qué la capa de acceso a Internet se ve distinta hoy.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La traza DORA completa con direcciones concretas

Datos. Red `192.168.1.0/24`. Gateway `192.168.1.1`. DNS `132.42.120.5`. Servidor DHCP `192.168.1.10`. Lease de 24 horas. El cliente es `PC` con MAC `00:1A:2B:3C:4D:5E`.

Resolución, mensaje por mensaje, con la tabla completa:

#	Mensaje	IP orig → dest	UDP orig → dest	MAC orig → dest	Contenido
1	**DISCOVER**	`0.0.0.0` → `255.255.255.255`	68 → 67	PC → `FF:FF:FF:FF:FF:FF`	MAC, hostname
2	**OFFER**	`192.168.1.10` → `255.255.255.255`	67 → 68	srv → broadcast	IP `192.168.1.50`, /24, GW `.1`, DNS `132.42.120.5`, lease 86400 s
3	**REQUEST**	`0.0.0.0` → `255.255.255.255`	68 → 67	PC → broadcast	Server ID = `192.168.1.10`, pide `192.168.1.50`
4	**ACK**	`192.168.1.10` → `255.255.255.255`	67 → 68	srv → broadcast	confirma `192.168.1.50`

Verificación: ¿qué haría PC si el `ACK` no llegara? Lost el lease no está confirmado, así que **no puede usar la dirección**. Reintenta el `REQUEST`. Si no hay respuesta tras N intentos, vuelve al paso 1 (`DISCOVER`).

Verificación 2: ¿qué pasa si el `OFFER` del servidor se pierde? PC no lo sabe. Reintenta el `DISCOVER`. El servidor ve un `DISCOVER` nuevo y hace una `OFFER` nueva. **No hay conflicto**, porque el servidor todavía no registró nada: el registro se hace recién cuando recibe el `REQUEST`.

2.2 Ejercicio 2 — Los temporizadores del lease

Datos. El lease es de **24 horas** = 86 400 segundos.

Resolución:

```
T1 = 0,50  × 86 400 =  43 200 s = 12 h
T2 = 0,875 × 86 400 =  75 600 s = 21 h
vencimiento =          86 400 s = 24 h
```

Interpretación: el cliente intenta renovar con el mismo servidor a las 12 h. Si en los próximos 9,6 h no obtiene respuesta, a las 21 h pregunta a **cualquier** servidor. Si a las 24 h no tiene nada, deja de usar la dirección.

¿Por qué T2 es 87,5 % y no 90 % o 95 %? Es un compromiso: si el umbral fuera muy tardío, no quedaría tiempo para conseguir una dirección nueva antes de que venza la actual. El 12,5 % restante (3 horas en este caso) es el margen para que el `REQUEST` de rebinding llegue, el servidor responda y el cliente configure la dirección nueva.

2.3 Ejercicio 3 — Cuántas direcciones necesita la red

Datos. Una empresa quiere 300 empleados, 20 servidores y 5 impresoras en una sola LAN con DHCP.

Resolución — el cálculo del pool:

```
Total de dispositivos que piden IP:  300 + 20 + 5 = 325
+ 1 para el gateway                      = 326
+ 1 para el servidor DHCP                = 327
+ 1 para la red y 1 para el broadcast    = 329

2^9 - 2 = 512 - 2 = 510  >= 327   ✓    →  necesitan un /23
```

Ahora la pregunta de diseño, que es la que seReward en el examen: ¿cuántas direcciones **quedan libres**?

```
Con un /23 (512 direcciones):     512 - 329 = 183 libres
Con un /22 (1024 direcciones):   1024 - 329 = 695 libres
```

El criterio correcto es el /23, y la razón de por qué es la que hay que saber decir: **un plan de numeración se dimensiona para crecer, no para el tamaño de hoy.** Con un /23 tenés 183 direcciones de margen, que es un 56 % de crecimiento sobre la necesidad actual. Con un /22 tendrías el doble de freedom y no lo usás.

> **Y esa es exactamente la lógica del apartado d del Problema 2 del Examen 2011-03** (resuelto en la Clase 27 y la 29): "*dejar la mayor cantidad posible de direcciones IP para uso futuro*" **no** significa usar el bloque entero, sino usar el **menor bloque que cubra la necesidad** y reservar el resto.

2.4 Ejercicio 4 — Dos servidores DHCP

Datos. En la red hay dos servidores DHCP, `A` (192.168.1.10) y `B` (192.168.1.11). Llegan dos clientes.

Resolución — el paso que casi nadie explica, el paso 3:

```
t=0  PC1 ──DISCOVER (broadcast)──►  A y B lo reciben

t=1  A ──OFFER: 192.168.1.50, lease 24 h ──►  PC1
     B ──OFFER: 192.168.1.60, lease 48 h ──►  PC1

PC1 aplica el DESEMPATE DE TIE-BREAK:
    "el que ofrezca el lease MÁS CORTO"

    A ofrece 24 h  →  gana A   ✗ descartada la de B
    B ofrece 48 h  →  pierde

t=2  PC1 ──REQUEST (broadcast)──►  "el Server Identifier va en 192.168.1.10,
                                      y elijo la 192.168.1.50"
                            ──►  A  lee:  soy yo  →  registro la asignación
                            ──►  B  lee:  no soy yo  →  LIBERO 192.168.1.60

t=3  A ──ACK: 192.168.1.50 es tuya por 24 h ──►  PC1
```

El punto clave: el `REQUEST` **no es unicast**, y por eso B puede saber que perdió y liberar la dirección que había reservado. **Si fuera unicast**, B seguiría creyendo que tiene la dirección asignada hasta que venciera el lease.

Y el **tie-break de lease más corto** parece contraintuitivo (¿por qué esperar *menos*?) pero tiene una razón clara: **minimiza el tiempo en que un cliente queda "atrapado" con una dirección de un servidor caído**. Si un cliente aceptara la oferta del servidor que no está disponible, tardaría 48 h en poder usar otra. Con el desempate por lease corto, y el T2 al 87,5 %, recupera la conectividad en unas horas.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-02, Pregunta 2 (10 puntos) — DHCP

> **Fuente:** Examen `eirc1102` / `sirc1102`, 3 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1102.pdf

Enunciado (texto del examen)

> Explique el funcionamiento del protocolo DHCP detallando el intercambio de mensajes entre cliente y servidor.

La solución que hay que escribir

Punto 1 — Qué resuelve (2 pts). DHCP es un protocolo de **arranque** (*bootstrap*) que permite a un host **sin configuración** obtener su dirección IP, máscara, gateway y DNS automáticamente. Sin él, cada equipo nuevo requeriría que un técnico lo configurara a mano.

Punto 2 — Por qué UDP (2 pts). Porque el cliente **no tiene todavía dirección IP**, y no puede abrir una conexión TCP (el `SYN` necesita una dirección origen válida). La transacción son mensajes sueltos de un solo envío, sin necesidad de estado de conexión.

Punto 3 — DORA (4 pts). Los cuatro mensajes, con quién los manda y qué contienen:

Paso	Mensaje	Remitente	Destinatario	Contenido
1	**DISCOVER**	cliente	broadcast `255.255.255.255`	identifica al cliente (MAC, hostname) y pide dirección
2	**OFFER**	servidor	cliente (broadcast o unicast)	propone una dirección libre, con máscara, gateway, DNS, duración del lease y demás opciones
3	**REQUEST**	cliente	broadcast	**confirma cuál de las ofertas acepta**, identificando al servidor elegido
4	**ACK**	servidor	cliente	confirma la asignación y fija la duración del lease

Punto 4 — El detalle que suma (2 pts). El **REQUEST va por broadcast** aunque el cliente ya sepa la dirección del servidor, y **por eso el paso 3 es obligatorio**: es lo que le permite a los servidores que hicieron OFFER y **no** fueron elegidos saber que perdieron y liberar la dirección que habían reservado. Si fuera unicast, cada uno creería que ganó.

Los tres pasos de TFTP. Los DHCP se apoya en TFTP (*Trivial File Transfer Protocol*) sobre UDP, por la misma razón: tiene que funcionar desde un host sin dirección. TFTP es trivial justamente porque **DHCP se encarga de manejar los tiempos y los reintentos** que TFTP no tiene.

El lease y los temporizadores. La asignación **no es permanente** sino que tiene una duración. El cliente intenta renovarla cuando se ha consumido el 50 % del lease (T1), contra el mismo servidor. Si no logra respuesta, al

87,5 % (T2) reintenta contra cualquier servidor. Si el lease se vence sin renovación, el cliente libera la dirección y vuelve a solicitar una.

Complemento — RARP, el antecedente que se pregunta en otro examen

> **Examen 2007-03, Pregunta 3 (10 pts)**: *Describa en qué escenario se utiliza el protocolo RARP y su funcionamiento.*

La respuesta corta: RARP (`RFC 826`) es **el antecesor de DHCP**. Antes de que existiera el boot de un host sin disco duro, un host que encendía sin sistema operativo instalado necesitaba **descubrir su propia dirección IP**. No sabía cuál era, y sin ella no podía hablar con nadie.

ARP responde: "*¿qué MAC tiene esta IP?*". **RARP invierte la pregunta**: "*¿qué IP tiene este MAC?*". El host manda un broadcast `¿quién es 00:1A:2B:3C:4D:5E?` y el servidor de red le responde con una dirección.

```
ARP   :  "¿qué MAC corresponde a la IP 192.168.1.50?"     → el router responde
RARP  :  "¿qué IP corresponde al MAC 00:1A:2B:3C:4D:5E?"  → el host pregunta
```

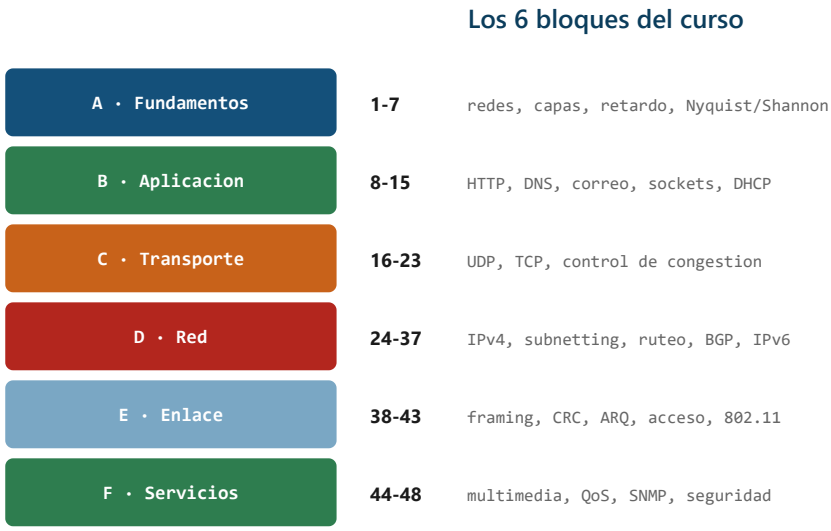
Por qué ya no se usa:

Problema de RARP	Lo resuelve DHCP
Solo devuelve **una** dirección IP. Nada de máscara, ni gateway, ni DNS	devuelve **toda** la configuración
No hay lease: la asignación es fija para siempre	hay lease y renovación
Necesita un **servidor RARP** en cada red	un solo servidor DHCP para toda la organización
No existe para IPv6	DHCPv6, y además SLAAC

La respuesta completa al examen 2007-03 (10 pts): escenario, formato de los mensajes (idéntico a ARP pero con tipo de operación distinto: RARP usa los tipos 3 y 4 donde ARP usa 1 y 2), la tabla del servidor, y **las tres limitaciones** de la tabla de arriba. Mencionar que DHCP **supera** a RARP en todo menos en complexidad.

PARTE 4 — PARTE B DEL SIMULACRO 2

4.1 Repaso del bloque B (clases 8 a 15)



El mapa en una línea por clase:

```
8  HTTP/1.1: petición/respuesta, persistente vs. no persistente
9  HTTP: cabeceras, caché con ETag, cookies, HTTP/2 y HTTP/3
10 DNS: espacio jerárquico + base distribuida, RR de 5 campos, tipos de registro
11 DNS: iterativo vs. recursivo, caché y TTL, DoH y DNSSEC
12 Taller: resolución completa paso a paso, RTT de una página
13 Correo: MUA/MTA/MDA, SMTP push, POP3 e IMAP pull, MIME, dot-stuffing
14 Sockets: API de Berkeley, orden de llamadas, htons, errores del código oficial
15 DHCP: DORA, lease, 169.254/16, Simulacro
```

Las tablas que hay que saber de memoria:

Resource Record, 5 campos:

```
nombre   | clase | tipo | TTL | datos
2 bytes | 2 B   | 2 B  | 4 B | variable
```

Tipos de RR que hay que saber: A, AAAA, NS, CNAME, MX, SOA, PTR, TXT, SRV, CAA.

DORA:

```
D  DISCOVER  cliente → broadcast   0.0.0.0:68 → 255.255.255.255:67
O  OFFER     servidor → cliente    67 → 68    (propone IP, máscara, GW, DNS, lease)
R  REQUEST   cliente → broadcast   "acepto la oferta del servidor X"
A  ACK       servidor → cliente    confirma
```

Puertos que hay que saber:

```
20/21 FTP       22 SSH       25 SMTP      53 DNS       67/68 DHCP
80 HTTP         110 POP3     143 IMAP     443 HTTPS    546/547 DHCPv6
179 BGP         853 DoT      161/162 SNMP
```

Petición/respuesta vs. push/pull:

Protocolo	Modelo	Quién inicia	Ejemplo
HTTP	**petición/respuesta**	el cliente	8, 9
DNS	**petición/respuesta**	el cliente	10, 11
SMTP	**push**	el emisor	13
POP3 / IMAP	**pull**	el receptor	13
DHCP	**petición/respuesta** (pero por broadcast)	el cliente	15

4.2 Los 8 errores más frecuentes del bloque

#	El error	Lo correcto	Por qué
1	"DNS usa UDP y si es grande pasa a **otro puerto**"	pasa a **TCP en el 53**	es el mismo puerto, distinto protocolo
2	"El **TLD** devuelve la IP del dominio"	devuelve el **NS autoritativo**	el TLD no conoce los dominios que cuelgan de él
3	"El **REQUEST** de DHCP va por unicast porque ya conoce al servidor"	va por **broadcast**	para que los perdedores liberen la dirección reservada
4	"El **DISCOVER** de DHCP tiene como origen la MAC del cliente"	origen IP es `0.0.0.0`	no tiene IP todavía; la MAC va en un campo de opción
5	"El **ACK** de TCP es un paquete"	es un **segmento con el flag ACK**	el ACK va piggybacked en el segmento de datos del otro sentido
6	"POP3 borra los mensajes al hacer `DELE`"	`DELE` **solo marca**, borra en `QUIT`	para poder deshacer si algo sale mal
7	"Para que haya confidencialidad hace falta **cifrar**"	para **integridad** alcanza un **hash**	el hash detecta el cambio; el cifrado lo esconde
8	"El cliente hace `bind()` antes de `connect()`"	el cliente **no hace bind**	el servidor reserva un puerto conocido; el cliente usa uno efímero

4.3 📝 Simulacro 2 — Examen completo

> **Instrucciones del examen real:** 3 horas · sin material · individual · **aprobación con 60/100** · **obligatorio responder correctamente al menos 15 puntos en las preguntas teóricas**.

Enunciado — Preguntas teóricas

Teórica 1 (10 puntos). Un usuario abre su navegador y escribe `https://www.ejemplo.com/index.html`. Describa, en orden cronológico, **todos** los pasos que ocurren desde que aprieta Enter hasta que ve la página en pantalla. Sea específico sobre: qué protocolos participan, cuántos *round trips* aproximados implica cada uno, y qué pasa si alguno de ellos falla.

Teórica 2 (10 puntos). Describa el funcionamiento del protocolo **DNS**, explicando el papel de cada tipo de servidor y la diferencia entre los dos algoritmos de resolución (iterativo y recursivo). Explique además el funcionamiento del mecanismo de **caché**, incluyendo qué campo controla el tiempo que un registro permanece en caché y quién lo define.

Teórica 3 (8 puntos). Describa el proceso de **arranque de un host** en una red con DHCP, detallando los mensajes involved. Explique por qué el protocolo usa UDP y no TCP, y qué ocurre si el servidor DHCP no está disponible.

Teórica 4 (8 puntos). Compare los protocolos **POP3** e **IMAP**, y explique el problema que surge cuando un mismo canal transmite información de control y datos simultáneamente, y cómo lo resuelve el protocolo.

Teórica 5 (4 puntos). Mencione tres condiciones que deben cumplirse para que una conexión **TCP** pueda establecerse entre dos procesos, e indique qué función de la API de sockets debe usar cada extremo para empezar el proceso.

Enunciado — Problemas prácticos

Problema 1 (30 puntos). Una organización tiene un único servidor DHCP que atiende dos redes separadas por un router:

- **Red A** (`192.168.10.0/24`): 120 puestos de trabajo y 2 servidores.
- **Red B** (`10.20.30.0/24`): 40 puestos de trabajo y 1 servidor con direccionamiento estático (no usa DHCP).

Se pide:

a) Especifique los **parámetros que debe entregar el servidor DHCP** a cada tipo de cliente de la red A y de la red B. Para cada parámetro indique el **valor** que corresponde según los datos del problema.

b) Realice la **traza completa del proceso de arranque** de un puesto de trabajo de la red A, indicando para cada uno de los cuatro mensajes: emisor, receptor, direcciones IP de origen y destino (de nivel IP y de nivel UDP), y qué información transporta.

c) El router entre las dos redes **no permite tráfico de la red B hacia la red A** salvo las respuestas a soliciting DHCP. Explique qué problema genera esto para los clientes de la red B que attempting renueva su lease, y proponga una solución.

Problema 2 (30 puntos). Se desea implementar un servicio de resolución de nombres simplified para uso interno. El servicio expone la función:

```
string consultar(string nombreHost, int tipoConsulta, int clase)
```

que devuelve, si el nombre existe y hay registros del tipo pedido, la **cadena de valores** separada por comas; y devuelve `""` si el nombre no existe.

Se cuenta con las siguientes primitivas:

- `enviar_consulta(servidorDNS, nombreHost, tipoConsulta, clase)` — envía una consulta DNS al servidor indicado.
- `esperar_respuesta()` — bloquea hasta recibir una respuesta. Devuelve un registro con los campos: `nombre`, `tipo`, `clase`, `ttl`, `datos`, y `codigoError`. Devuelve `null` si no hay respuesta (timeout).
- `hayMasRegistros()` / `obtenerSiguienteRegistro()` — permiten recorrer los registros de la respuesta.

a) Explique qué información debe contener el mensaje de consulta DNS que la función debe construir, y por qué.

b) Implemente la función `consultar` en un lenguaje de alto nivel, teniendo en cuenta que: (i) el nombre puede no existir, (ii) un nombre puede tener **varios** registros del mismo tipo, (iii) el servidor puede **no responder**, y (iv) el servidor puede responder con un **código de error** que no sea "no existe".

c) Explique qué consecuencia tiene, en la resolución real, el hecho de que su implementación **no pueda** distinguir entre "el nombre no existe" y "el servidor no respondió", y cómo resolvería ese defecto.

Cronómetro sugerido

Momento	Minutos	Qué hacer
0:00 – 0:08	8	leer todo, elegir qué problemas hacer
0:08 – 0:53	45	**las 5 teóricas** (8 a 10 min cada una)
0:53 – 1:53	60	**Problema 1**
1:53 – 2:53	60	**Problema 2**
2:53 – 3:00	7	releer, corregir, completar las cotas

> **Recordá el mínimo de 15 puntos en las teóricas.** Si te quedás sin tiempo para los problemas, todavía podés aprobar: **no abandones las teóricas**.

▶ ✅ **RESPUESTAS DEL SIMULACRO 2**

🔗 Referencias

- **Examen `eirc1102` / `sirc1102`** — Pregunta 2 completa (10 pts, DHCP)
- **Examen `eirc0703` / `sirc0703`** — Pregunta 3 (10 pts, RARP y su relación con DHCP)
- **Examen `eirc1012` / `sirc1012`** — Problema 1 (30 pts, sockets UDP)
- **Examen `eirc1003` / `sirc1003`** — Pregunta 2 (8 pts, arquitectura de correo), P4 (8 pts, SNMP)
- **Examen `eirc0812` / `sirc0812`** — Pregunta 4 (7 pts, POP3 y CRLF.CRLF)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 (DHCP) y cap. 7
- **RFC 2131** (DHCP) · **RFC 2132** (opciones DHCP) · **RFC 826** (RARP)
- `ACTUALIZACIONES.md` **§5** — DHCPv6, SLAAC, CGNAT
- **Clase siguiente:** 16 — Multiplexación, demultiplexación y UDP

Clase 16

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El problema: cuatro programas, una sola dirección IP
1.4 Por qué UDP no tiene conexión: lo que no hay
1.5 La cabecera de UDP, campo por campo
1.6 La 2-tupla y la 5-tupla: el matiz que hay que aclarar
1.7 Comparación con TCP: el mecanismo equivalente
1.8 ⚠️ Actualización a 2026
2.1 Ejercicio 1 — El Internet checksum de UDP, calculado a mano
2.2 Ejercicio 2 — UDP sobre IPv6: pseudo-cabecera de 40 bytes
2.3 Ejercicio 3 — Sockets en C: un servidor UDP que recibe y responde
3.1 ⭐ Examen 2010-12, Pregunta 2 (8 puntos) — Multiplexación/demultiplexación
3.2 Examen 2009-07, Pregunta 2b (8 puntos) — Cuándo usar UDP en vez de TCP
📌 Resumen en 10 líneas
✅ Autoevaluación
🔗 Referencias

Bloque: C (Transporte) · **Semana:** 6 · **Duración estimada:** 110 min **Correlativa:** Clase 15 (DHCP, configuración y simulación final) · **Sigue con:** Clase 17 (TCP: la cabecera campo por campo) **Material de referencia:** Tanenbaum cap. 6 (§6.1, §6.5) · RFC 768, 9293, 9000, 9868

🎯 Al terminar esta clase vas a poder

1. Explicar por qué hace falta el **puerto** como identificador de proceso en un host.
2. Distinguir **multiplexación** de **demultiplexación** y explicar la tabla de sockets.
3. Responder las **tres preguntas del examen 2010-12 P2** y explicar la diferencia con TCP.
4. Escribir la **cabecera de UDP campo por campo** y calcular un checksum de Internet a mano.
5. Justificar por qué "un socket UDP es una 2-tupla" es correcto pero incompleto.
6. Escribir en **C** un servidor UDP con `socket()`, `bind()`, `recvfrom()` y `sendto()`.

> **Prerequisito del examen:** `2010-12 P2` (8 pts, el problema central) · `2009-07 P2b` (8 pts, cuándo usar UDP) · `2010-12 Problema 1` (30 pts, sockets UDP en C).

PARTE 1 — TEORÍA

1.1 El problema: cuatro programas, una sola dirección IP

Firefox, correo, Spotify y un juego salen a la red **desde la misma máquina**: para el mundo hay una sola IP, `203.0.113.9`, y una sola MAC. Y sin embargo cada datagrama tiene que terminar en el proceso correcto. IP ya hizo su trabajo: llevó el paquete a la máquina correcta. Falta responder **¿a cuál de estos siete procesos?** (Firefox, correo, Spotify, juego, `sshd`, `dnsmasq`, `cupsd`). La respuesta clásica es el **número de puerto** (*port number*): un entero de **16 bits** (0 a 65 535) que la capa de transporte escribe en la cabecera y que el SO usa para elegir el proceso destino.

Tres propiedades del puerto que hay que tener presentes:

1. **El puerto es local a cada host.** El `53` del servidor DNS y el `53` del cliente no significan nada entre sí: un puerto es "un índice en la tabla de sockets de *mi* máquina".
2. **Identifica un servicio, no una persona.** No es seguridad: si tiene permisos, cualquier proceso puede hacer `bind()` sobre un puerto dado.
3. **El rango se reparte por registro:** bien conocidos (0–1023), registrados (1024–49151) y efímeros (49152–65535), que son los que el SO elige para los clientes.

Puerto y protocolo van juntos: **`53/udp` y `53/tcp` son dos servicios distintos**, y `443/tcp` es HTTPS mientras que `443/udp` es HTTP/3 (QUIC). Para tener a mano: 20/21 FTP, 22 SSH, 25 SMTP, **53 DNS (udp y tcp)**, 67/68 DHCP, 80 HTTP, 161/162 SNMP, 179 BGP, 853 DoT.

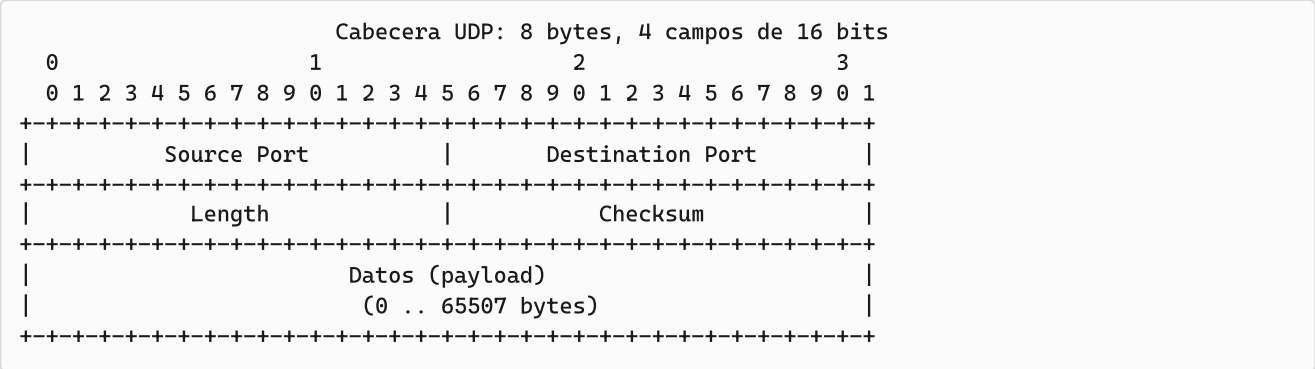
1.4 Por qué UDP no tiene conexión: lo que no hay

UDP es **sin conexión** (*connectionless*), **no confiable** (*unreliable*), también **best effort** ("lo mejor que se pueda"). La definición es negativa, y esa es la mejor forma de recordarlo:

```
┌──────────────────── UDP: lo que NO hay ────────────────────┐
│ 1. NO hay handshake: ni SYN ni SYN-ACK. Sin estado ni seq inicial│
│ 2. NO hay ACK ni retransmisión: si se pierde, se perdió.        │
│ 3. NO hay control de congestión ni de flujo: sin ventana ni MSS.│
│ 4. NO hay orden: pueden llegar en el orden inverso.             │
└─────────────────────────────────────────────────────────────────┘
```

Y lo que sí tiene: **números de puerto**, **longitud** y un **checksum opcional**. La cabecera de UDP son **8 bytes**. Con 1000 B de payload, el datagrama completo pesa 1046 B con UDP (4,4 % de overhead) y 1058 con TCP (5,8 %): el 60 % del overhead de TCP son 12 bytes. Nadie elige TCP o UDP por el tamaño de la cabecera, sino por **las garantías que necesitás y el retardo que tolerás**.

1.5 La cabecera de UDP, campo por campo



Campo	Bits	Qué es
Source Port	16	Puerto del proceso que emite (12 345 = `0x3039`)
Destination Port	16	Puerto del proceso que recibe (53 = `0x0035`)
Length	16	Longitud de **cabecera + datos**, en bytes. Mínimo 8 (21 = `0x0015`)
Checksum	16	Detección de errores; **opcional en IPv4, obligatorio en IPv6**

El pseudo-cabecera

El checksum de UDP tiene que proteger el datagrama **y también las direcciones IP**: si un router se equivoca de dirección, el paquete llega al host equivocado y el checksum "daría bien" si solo mirara lo que viene después de IP. Pero **UDP no ve la cabecera IP** (se la pasa como un bloque opaco), así que el emisor **agrega las direcciones otra vez**, sintéticamente, al principio de lo que suma. A ese bloque se lo llama **pseudo-cabecera** (*pseudo-header*) y **no se transmite**: existe solo durante el cálculo.

IPv4 (12 bytes): `src` (4 B), `dst` (4 B), **1 byte de cero**, **protocolo = 17** y **longitud del datagrama UDP** (2 B). **IPv6 (40 bytes):** origen (16) + destino (16) + **longitud de capa superior de 32 bits** (4) + **3 bytes de cero** + **Next Header = 17**; con un túnel IPIP el Next Header es el de ese protocolo, y la longitud sigue siendo la del **datagrama UDP**.

El Internet checksum y el byte de relleno

Es el **Internet checksum** (RFC 1071), de **complemento a uno de 16 bits**: se toman los datos de a 2 bytes en *big-endian*, se suman, se devuelve cada acarreo al bit 0 hasta que no sobre (`suma = (suma & 0xFFFF) + (suma >> 16)`, repetir) y se toma el complemento del resultado; si da `0` se transmite `0xFFFF`, porque `0` significa "no computado" en IPv4.

El detalle que casi todos olvidan: si el total de bytes a sumar es **impar**, se agrega un byte `0x00` al final (siempre con UDP, porque la longitud puede ser impar). Calculado en el ejercicio 2.1. **Errores típicos:** olvidar ese byte; creer que avisa que el paquete no llegó (se **descarta en silencio**); y confundir el pseudo-cabecera con algo que viaja.

1.6 La 2-tupla y la 5-tupla: el matiz que hay que aclarar

```
Solución oficial del 2010-12:
  "Un socket UDP queda identificado completamente utilizando la IP del host
   y el número de puerto."                              →  2-tupla
Lo que realmente usa el kernel:
  (protocolo, IP origen, puerto origen, IP destino, puerto destino)
                                                       →  5-tupla
```

Las dos cosas son ciertas y no se contradicen. La 5-tupla es la clave completa de un **flujo** en la red; la 2-tupla es la clave de **un socket local**.

- Un **socket UDP no conectado** (el caso normal de un servidor) está enlazado con `bind()` a un par `(IP local, puerto local)` y **solo uno** puede estar enlazado a ese par. Como todo lo que llega a ese socket es por definición "para" él, el kernel puede indexar la tabla **solo por (IP, puerto)** y aun así desambiguar: la 2-tupla **alcanza** porque los otros tres valores son constantes.
- En realidad la tabla se indexa por las cinco, y el `protocolo` está porque `53/udp` y `53/tcp` conviven sin cruzarse.
- Un **socket UDP conectado** (`connect()` sobre UDP existe y es real) **sí** se desambigua por la 5-tupla: el kernel **filtra** lo que entra y avisa con `ECONNREFUSED`.

> **La solución oficial ya lo anticipa en la parte b:** para elegir el socket TCP "en realidad alcanzaría con utilizar IP origen, Puerto origen, Puerto destino". Es el mismo razonamiento: **la clave de la red es la 5-tupla, pero como la IP destino y el protocolo están fijos en el host, se pueden omitir**. Si citás la 5-tupla, citá también por qué se puede reducir.

1.7 Comparación con TCP: el mecanismo equivalente

TCP demultiplexa **de otra manera**: usa un socket *escuchando* (*listening*) como punto de entrada y crea sockets nuevos por cada conexión.

```
navegador 1 ── socket <198.51.100.7,49200> ─┐
navegador 2 ── socket <198.51.100.7,49201> ─┴─ socket ESCUCHANDO <0.0.0.0,80>
                                                  ▼   (el SO los crea)
                                            proceso servidor
```

En tres pasos: (1) el proceso servidor hace `bind()` + `listen()` y queda **un solo socket escuchando**, con **una** fila en la tabla; (2) llega un SYN a ese puerto, el kernel matchea el socket escuchando, **no le entrega el SYN al proceso**, y al completarse el handshake de 3 vías crea **un socket nuevo** con la **4-tupla** `<198.51.100.7, 49200, 203.0.113.9, 80>`; (3) cada segmento con esos cuatro valores matchea esa fila y va al proceso con `accept()`. **La diferencia de fondo:** en UDP la demultiplexación es *a un proceso*; en TCP es *a una conexión*, y por eso TCP necesita la 4-tupla y el handshake.

1.8 ⚠️ Actualización a 2026

Sigue igual — de memoria: cabecera de **8 bytes y 4 campos de 16 bits** con la longitud incluida; UDP **sin conexión, sin ACK, sin retransmisión, sin control de congestión ni de flujo, sin orden**; checksum **con pseudo-cabecera**, **opcional en IPv4 y obligatorio en IPv6**; y **TCP = 4-tupla, socket UDP = 2-tupla**, que es lo que se sigue enseñando y preguntando.

Cambió o hay que corregirlo:

- `RFC 9868` **— *Transport Options for UDP*** (oct-2025, *Standards Track*) **actualiza la** `RFC 768`: define la ubicación, sintaxis y semántica de **opciones de transporte de UDP**, que van en el espacio sobrante (*surplus area*) **después de los datos del usuario y antes del final del datagrama IP** (hoy ese espacio solo lo ocupa el *padding* de Ethernet). *Consecuencia:* "UDP no tiene opciones" ya no es estructuralmente cierto, aunque en la práctica el cable siga viendo 8 bytes de cabecera. La especificación original es la `RFC 768`, de J. Postel, 1980, catalogada como **STD 6 / Internet Standard**.
- **La "2-tupla de UDP" es una simplificación que hay que matizar.** `ACTUALIZACIONES.md` §14 lo dice textualmente: la clave de demux real en el kernel es la **5-tupla**. Presentá las dos y explicá la reconciliación de §1.6.
- **QUIC rompe el modelo de la 4-tupla.** `RFC 9000` §5.1: la identidad de una conexión QUIC es el **Connection ID** (*CID*), de 0 a 20 bytes, que viaja en la cabecera de cada paquete. Da tres cosas: **muchas conexiones multiplexadas en el mismo 5-tupla** (ver `RFC 8085`), **migración de conexión** (cambia la IP o el NAT cambia el puerto y la conexión sobrevive) y **CID de largo 0** si solo se usa la 5-tupla, que entonces no habilita migración.
- **HTTP/3 es UDP de verdad** (`RFC 9114`, con `Alt-Svc` `RFC 7838` y `SVCB`/`HTTPS` `RFC 9460`). Lo que **no** es cierto es que sea "TCP con otra cabecera": no hay conexión TCP, no hay handshake de 3 vías y no hay retransmisión de conexión; la pérdida se resuelve por *stream*. Cifras de 2026 (Cloudflare Radar): HTTP/2 50 %, HTTP/1.x 29 %, **HTTP/3 21 %**. Y sniffear HTTP/3 es difícil: va cifrado y sus cabeceras van en QPACK.
- **MPTCP (`RFC 8684`, 2020)** agrega multipath sin cambiar la demux de capa 4: cada *subflow* sigue con su 4-tupla. Desplegado por Apple y operadores, con **cuota marginal** (~0,4 % de bytes pico). Y ojo con **dual-stack**: `::ffff:a.b.c.d` y `IPV6_V6ONLY=0` afectan directo a la demultiplexación, porque `::ffff:203.0.113.9:53` y `203.0.113.9:53` deben llegar al mismo socket.

Consecuencia para el examen: la pregunta del 2010-12 P2 **sigue siendo la misma** y la solución oficial **sigue siendo válida**. Si sobra tiempo, una línea final: "*en el kernel la clave es la 5-tupla; la 2-tupla alcanza para un socket UDP no conectado porque el resto es constante, y QUIC ya multiplexa conexiones con Connection ID sobre el mismo 5-tupla.*"

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — El Internet checksum de UDP, calculado a mano

Datos. Datagrama IPv4 de `203.0.113.9` a `198.51.100.7`, protocolo 17. Puerto origen **12345**, puerto destino **53**, longitud **21**. Datos: la cadena ASCII `PRUEBA-EXAMEN`, de **13 bytes**.

a) Palabras de 16 bits a sumar. **b)** Suma mostrando cada acarreo. **c)** ¿Cuál es el checksum? **d)** Verificación en el receptor. **e)** Se alteran dos bits: el bit 0 de la palabra 1 y el de la palabra 3 de la pseudo-cabecera. ¿Lo detecta?

a) Las palabras a sumar, en orden:

```
Pseudo-cabecera (6 palabras)   1 CB00   2 7109   3 C633   4 6407   5 0011   6 0015
Cabecera UDP (3; el checksum va en 0)
                               7 3039   8 0035   9 0015
Datos (7: 6 parejas + 1 con el byte de relleno)
 10  5052
 11  5545
 12  4241
 13  2D45
 14  5841
 15  4D45
 16  4E00
```

La longitud total es 21 bytes, que es **impar**: por eso la última palabra es `4E00`. Ese byte de relleno **no viaja**: se inventa para poder sumar de a dos.

b) La suma. Si el resultado pasa de `FFFF`, se devuelve el acarreo y se sigue:

```
 CB00 + 7109 = 13C09  → acarreo 1 →  3C0A
 3C0A + C633 = 1023D  → acarreo 1 →  023E
 023E + 6407 = 06645  →              6645
 6645 + 0011 = 06656  →              6656
 6656 + 0015 = 0666B  →              666B
 666B + 3039 = 096A4  →              96A4
 96A4 + 0035 = 096D9  →              96D9
 96D9 + 0015 = 096EE  →              96EE
 96EE + 5052 = 0E740  →              E740
 E740 + 5545 = 13C85  → acarreo 1 →  3C86
 3C86 + 4241 = 07EC7  →              7EC7
 7EC7 + 2D45 = 0AC0C  →              AC0C
 AC0C + 5841 = 1044D  → acarreo 1 →  044E
 044E + 4D45 = 05193  →              5193
 5193 + 4E00 = 09F93  →              9F93
 ──────────────────────────────────────────
 suma final = 0x9F93
```

c) El checksum es el complemento a uno de 16 bits:

El checksum es el complemento a uno de 16 bits: `0xFFFF − 0x9F93 = 0x606C`. El datagrama queda `30 39 00 35 00 15 60 6C | 50 52 55 45 42 41 2D 45 58 41 4D 45 4E`.

d) Verificación en el receptor. El receptor suma **todo, incluido el campo checksum**: `9F93 + 606C = FFFF`, sin acarreo, y el complemento de `FFFF` es `0x0000`. **Un resultado `0x0000` significa "el datagrama está**

íntegro": ese es el criterio completo. Si el complemento no da cero, hay error y el datagrama se descarta en silencio.

Para el ítem (e): si el bit 0 de la palabra 1 pasa a 1 y el de la palabra 3 pasa a 0, los errores se compensan (`+1 − 1`), la suma vuelve a dar `0x9F93`, y el receptor calcula `0x606C`, que coincide con el checksum recibido: **no lo detecta**.

2.2 Ejercicio 2 — UDP sobre IPv6: pseudo-cabecera de 40 bytes

Enunciado. Un host `2001:db8::9` manda a `2001:db8::7` un datagrama UDP de **13 bytes** de datos ("PRUEBA-EXAMEN"), puerto 12345 → 53, IPv6 sin cabeceras de extensión. **a)** Escribí el pseudo-cabecera. **b)** Calculá el checksum. **c)** ¿Y si va dentro de un túnel IPIP? **d)** ¿Es opcional el checksum?

a) El pseudo-cabecera IPv6 (40 bytes = 20 palabras de 16 bits):

`0–15` Source `2001:0db8:...:0009` · `16–31` Destination `2001:0db8:...:0007` · `32–35` Upper-Layer Packet Length `0x00000015` = 21 · `36–38` Zero = `0x000000` · `39` Next Header = 17 (UDP).

b) Checksum. Con las mismas palabras del ejercicio 2.1 pero reemplazando las 6 del pseudo-cabecera IPv4 por las 20 del IPv6: `suma = 0x94D0` y `checksum = ~0x94D0 = 0x6B2F`. La aritmética es **exactamente la misma** que en IPv4; solo cambia el pseudo-cabecera. Fijate que la longitud que va en el pseudo-cabecera es **la del datagrama UDP (21)**, no la del paquete IPv6 (que sería 40 + 8 + 13 = 61). **Errores típicos:** poner la del paquete IPv6, u olvidar los 3 bytes de cero (el pseudo-cabecera IPv6 tiene 40 bytes exactos).

c) Con IPIP el Next Header **no es 17** sino el del protocolo de encapsulación (IPIP = 4), y la longitud sigue siendo la del **datagrama UDP interno**. **d)** En IPv6 el checksum es **obligatorio**: un `0` recibido implica descartar el datagrama, mientras que en IPv4 el `0` significa "no computado" y es legal (muchos stacks antiguos lo dejaban en cero para ahorrar CPU).

2.3 Ejercicio 3 — Sockets en C: un servidor UDP que recibe y responde

Enunciado. Escribí un servidor UDP en C que atienda el puerto 12345, responda `OFERTA <precio>` a los pedidos `OFERTA` y `COMMIT` o `REFUSE` a los `CONNECT` si se alcanzó el máximo de usuarios.

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>      /* + sys/types.h, netinet/in.h */
#include <netinet/in.h>

#define PUERTO 12345
#define MAX_MSG 128
#define MAX_USERS 20
#define PRECIO 150

static int usuarios = 0;

int main(void)
{
    int s;
    struct sockaddr_in local, cliente;
    socklen_t len_cli;
    char buf[MAX_MSG];
    ssize_t n;

    s = socket(AF_INET, SOCK_DGRAM, 0);          /* SOCK_DGRAM = UDP */
    if (s < 0) { perror("socket"); exit(1); }

    memset(&local, 0, sizeof local);
    local.sin_family      = AF_INET;
    local.sin_port        = htons(PUERTO);         /* orden de red: big endian */
    local.sin_addr.s_addr = htonl(INADDR_ANY);    /* todas las interfaces */
    if (bind(s, (struct sockaddr *)&local, sizeof local) < 0) { perror("bind"); exit(1); }

    for (;;) {
        const char *resp;
        len_cli = sizeof cliente;                  /* recvfrom: datos + emisor */
        n = recvfrom(s, buf, sizeof buf - 1, 0,
                     (struct sockaddr *)&cliente, &len_cli);
        if (n < 0) { perror("recvfrom"); continue; }
        buf[n] = '\0';

        if (strcmp(buf, "OFERTA") == 0) {
            n = snprintf(buf, sizeof buf, "OFERTA %d", PRECIO);
            resp = buf;
        } else if (strcmp(buf, "CONNECT") == 0) {
            if (usuarios < MAX_USERS) { usuarios++; resp = "COMMIT"; }
            else                      { resp = "REFUSE"; }
        } else {
            resp = "ERROR";
        }
        /* se responde AL QUE MANDO: por eso guardamos `cliente` */
        if (sendto(s, resp, strlen(resp), 0,
                   (struct sockaddr *)&cliente, len_cli) < 0) {
            perror("sendto");
        }
    }
    return 0;
}
```

El cliente `connect()` **conecta** su socket y puede usar `send()` / `recv()`: eso fija el par remoto por defecto, hace que el kernel **descarte** lo que venga de otro sitio y devuelve un ICMP 3/3.

Las tres ideas del ejercicio, que son las que se preguntan:

1. `socket(AF_INET, SOCK_DGRAM, 0)` → **UDP**; con `SOCK_STREAM` sería TCP. Y `bind()` es casi obligatorio en un servidor, porque **`recvfrom()` solo funciona sobre un socket enlazado**.
2. `htons()` / `htonl()`: los enteros multi-byte viajan en **big endian** (*network byte order*) y la familia `h` (*host to network*) convierte.
3. **El kernel guarda la dirección de quien mandó**, y por eso la respuesta puede ir al puerto efímero que ese proceso chose: ese es el mecanismo de "responder sin conocer el puerto del otro".

Errores típicos (los del C de la solución oficial de 2010-12): comparar cadenas con `==` (compara **direcciones de memoria**, no contenido; usá `strcmp()`); usar una variable sin inicializar como argumento de entrada; y `strlen()` donde va el largo real, que devuelve `snprintf()`.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-12, Pregunta 2 (8 puntos) — Multiplexación/demultiplexación

> **Fuente:** Examen `eirc1012` / `sirc1012` — 17 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf

Enunciado (texto del examen)

> a) Describa el mecanismo de multiplexación/demultiplexación para sockets UDP, discutiendo los siguientes puntos: 1. Cuando un host recibe un segmento UDP, ¿de qué manera decide a qué proceso derivárselo? 2. Cuando el proceso recibe el segmento, ¿cómo sabe a dónde enviar la respuesta? 3. ¿Con qué información queda completamente identificado un socket UDP en la red?
>
> b) Indique la diferencia entre el mecanismo de la parte a) y el utilizado para sockets TCP. ¿Con qué información queda completamente identificado un socket TCP en la red?

Solución oficial (verbatim)

> **a)** Cuando un host recibe un segmento UDP, utiliza el puerto destino indicado en el segmento para determinar a qué proceso debe enviárselo. El proceso que lee el segmento utiliza los campos IP origen y puerto origen para saber cuál es el socket UDP donde debe enviar la respuesta. Un socket UDP queda identificado completamente utilizando la IP del host y el número de puerto.
>
> **b)** El caso de TCP es un poco diferente: un proceso estará esperando conexiones en un puerto; cuando se establece la conexión, el sistema operativo creará un socket que enlaza , y se lo devolverá al proceso para que lo atienda; cuando llega un segmento TCP al host, se utilizará los cuatro valores para determinar a cuál de los sockets hay que derivar el segmento (en realidad alcanzaría con utilizar IP origen, Puerto origen, Puerto destino). Un socket TCP queda identificado completamente utilizando los cuatro campos: IP de origen, Puerto de origen, IP de destino, Puerto de destino.

Análisis de la solución oficial

La respuesta oficial acierta el mecanismo de los tres puntos y el **b)** es impecable en lo esencial; lo mejor es el **a)2**, porque demuestra que el sistema es **simétrico**. Lo que **falta** es la frase que da el punto en **a)1** (*el SO mantiene una tabla de sockets y busca la entrada que coincida*), una mención al **pseudo-cabecera** (otro uso de la IP de origen en UDP) y el matiz de que la **2-tupla es una simplificación** (§1.6).

Mi resolución, punto por punto

a)1 — ¿Cómo decide a qué proceso derivar el segmento? El SO mantiene una **tabla de sockets**. Cuando llega un datagrama con protocolo UDP: (1) toma el **número de protocolo** (17) y busca, **en la tabla de sockets**

UDP, la fila cuyo `(IP local, puerto local)` coincida con el `(IP destino, puerto destino)` del datagrama — TCP tiene su propia tabla, y por eso `53/udp` y `53/tcp` no se cruzan; (2) si hay coincidencia, **entrega los datos al proceso** con `recvfrom()`, que devuelve la IP y el puerto de origen; (3) si no hay coincidencia, **descarta** y devuelve **ICMP tipo 3, código 3**.

a)2 — ¿Cómo sabe a dónde enviar la respuesta? Por **simetría del protocolo**: cuando el SO entrega el datagrama al proceso, le pasa también los dos campos de la cabecera, la **IP de origen** y el **puerto de origen**. Con esos dos datos más su propia IP y puerto local (que ya conoce) tiene los cuatro valores que alcanzan:

```
Respuesta = a la IP origen del datagrama, con puerto = puerto origen del datagrama
```

a)3 — ¿Con qué información queda identificado un socket UDP? La oficial: **la IP del host y el número de puerto** (la 2-tupla). Es correcto para un socket **no conectado**, y hay que justificarlo: como está enlazado a un par `(IP, puerto)` y **solo puede haber uno por par**, los otros tres valores son constantes. La respuesta completa, la de 2026: la clave de un **flujo en la red** es la **5-tupla** `(protocolo, IP origen, puerto origen, IP destino, puerto destino)`, y recién con `connect()` el kernel pasa a filtrar por ella completa.

b) — Diferencia con TCP y su identificación. Tres diferencias: (1) **el punto de entrada es uno solo y lo atiende el SO**, que maneja los SYNs sin despertarse el proceso; (2) **cada conexión crea un socket nuevo** con la **4-tupla** `<IP origen, puerto origen, IP destino, puerto destino>`, que se le pasa al proceso con `accept()`, mientras el socket escuchando sigue ahí; (3) **la clave es más grande porque hay más que distinguir**: contra el mismo servidor 80, la IP destino y el puerto destino son **siempre los mismos**, así que lo único que distingue las conexiones es el **par de puertos**. Por eso la solución oficial notó que alcanzaría con tres valores: el protocolo y la IP destino quedan implícitos. La especificación vigente es **`RFC 9293`** (2022, *Internet Standard*), que sustituye a la `RFC 793`.

3.2 Examen 2009-07, Pregunta 2b (8 puntos) — Cuándo usar UDP en vez de TCP

> **Fuente:** Examen `eirc0907` / `sirc0907` — 29 de julio de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0907.pdf

Enunciado (texto del examen)

> b) Mencione dos casos (aplicaciones, servicios, etc.) en los que resulta más apropiado usar UDP que TCP. Justifique su respuesta.

Solución oficial (verbatim)

> Por ejemplo: streaming de video y telefonía sobre internet. Son casos en los que el control de congestión de TCP o el tiempo consumido por el establecimiento de conexión es contraproducente por tratarse de aplicaciones de tiempo real en las que más vale perder algunos paquetes que atrasarlos todos.

Mi resolución, ampliada

La solución oficial acierta los dos casos y da **dos justificaciones correctas** (en el primero, la **oportunidad**; en el segundo, el **costo del handshake**). Para 8 puntos hay que dar **dos casos con su justificación específica**.

Caso 1 — Telefonía por Internet (VoIP) y videoconferencia. La aplicación genera un sample cada 20 ms y tiene que reproducirlo en su momento. Si uno se pierde, con TCP el stack **retransmite** el sample y, peor, **entrega los siguientes en orden**: los samples 4, 5 y 6 esperan en el buffer hasta que llega el 3, que llega 200 ms tarde y **ya no sirve para nada** (se descarta igual, pero después de acumular **jitter**). Con UDP se pierde y se sigue. **En una frase:** en tiempo real un dato tardío es peor que uno ausente, y TCP garantiza orden y no pérdida, justo lo que no querás.

Caso 2 — Consulta de DNS (y cualquier protocolo de consulta/respuesta corto). La conexión TCP necesita **1 RTT entero de handshake** antes de que el cliente pueda mandar la consulta, y **3 paquetes** de ida y vuelta para una respuesta de, digamos, 100 bytes; con UDP son **2 paquetes**, y el riesgo de perder la consulta se resuelve con un **reintento temporizado**. **El costo del establecimiento no se amortiza si la transacción dura un solo mensaje.**

> **La contraargumentación clásica:** muchas redes **bloquean el tráfico UDP saliente**, y por eso los protocolos que quieren las ventajas de UDP tunelizan UDP dentro de TCP: **HTTPS** lo hace con TLS y **QUIC** (`RFC 9001`) usa **TLS 1.3 sobre UDP**. Es la respuesta moderna a "pero UDP no es confiable": *la confiabilidad la aporta la capa de aplicación, no el transporte*.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Multiplexar** = muchos procesos → un flujo; **demultiplexar** = un datagrama → su proceso.
2. El **puerto** (16 bits) identifica el proceso **dentro del host**, no entre hosts.
3. Es una **búsqueda en la tabla de sockets** por `(protocolo, IP destino, puerto destino)`; si no hay coincidencia, se descarta y se avisa con ICMP 3/3.
4. La **respuesta** va a la IP y al puerto de origen: eso lo da el `recvfrom()`.
5. **Socket UDP = 2-tupla**; la clave real de un flujo es la **5-tupla** `(proto, srcIP, srcPort, dstIP, dstPort)`.
6. **Socket TCP = 4-tupla**, porque en un servidor hay muchas conexiones al mismo `IP:puerto` y lo único que las distingue es el par de puertos.
7. UDP: **8 bytes** de cabecera y ni una garantía más. *Best effort*.
8. El **checksum** usa un **pseudo-cabecera** (IP + protocolo + longitud) y, si la longitud es impar, se rellena con un byte `0x00`; en **IPv6** es obligatorio y en IPv4 opcional.
9. **QUIC** (`RFC 9000` §5.1) usa el **Connection ID** en vez de la 4-tupla.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un host tiene sockets ligados a `203.0.113.9:53` y `203.0.113.9:631`. Llega un datagrama con `dport = 53`. ¿Qué hace el SO y por qué no hay ambigüedad?

▶ Respuesta

2. ¿Por qué un socket UDP conectado se identifica con la 5-tupla y uno sin conectar con la 2-tupla?

▶ Respuesta

3. Llega un datagrama con checksum `0xABCD` pero el receptor calcula `0xABCE`. ¿Qué hace el receptor y por qué no avisa al emisor?

▶ Respuesta

4. ¿Hace falta byte de relleno en un datagrama UDP de 6 bytes de datos (longitud total 14)?

▶ Respuesta

5. ¿Por qué el checksum de UDP incluye las direcciones IP si la cabecera IP ya viaja en el paquete?

▶ Respuesta

6. En el ejercicio 2.2, ¿qué cambia si el datagrama viaja sobre IPv6 en lugar de IPv4?

▶ Respuesta

7. ¿Por qué TCP necesita la 4-tupla y UDP no? En una frase.

▶ Respuesta

8. ¿Qué rompe QUIC del modelo de la 5-tupla y cómo lo resuelve?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc1012` **/** `sirc1012` — P2 (8 pts, mux/demux UDP vs TCP) · Problema 1 (30 pts, sockets UDP en C)
- **Examen** `eirc0907` **/** `sirc0907` — P2b (8 pts, cuándo usar UDP en vez de TCP)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 — §6.1 (*Multiplexing*), §6.5 (*UDP*)
- `RFC 768` — *User Datagram Protocol* (1980, **STD 6 / Internet Standard*; verificado)** · `RFC 9868` — ***Transport Options for UDP* (oct-2025)** · `RFC 9293` — ***TCP* (2022; sustituye a la 793)** · `RFC 1071` ** — Internet checksum
- **QUIC/HTTP-3:** `RFC 9000` (transporte, §5.1 Connection ID) · `RFC 9001` (TLS 1.3) · `RFC 9114` (HTTP/3) · `RFC 7838` (Alt-Svc) · `RFC 9460` (SVCB) · `RFC 8085` (CID de largo 0)
- **Clase siguiente:** 17 — TCP: la cabecera campo por campo

Clase 17

Contenido de esta clase

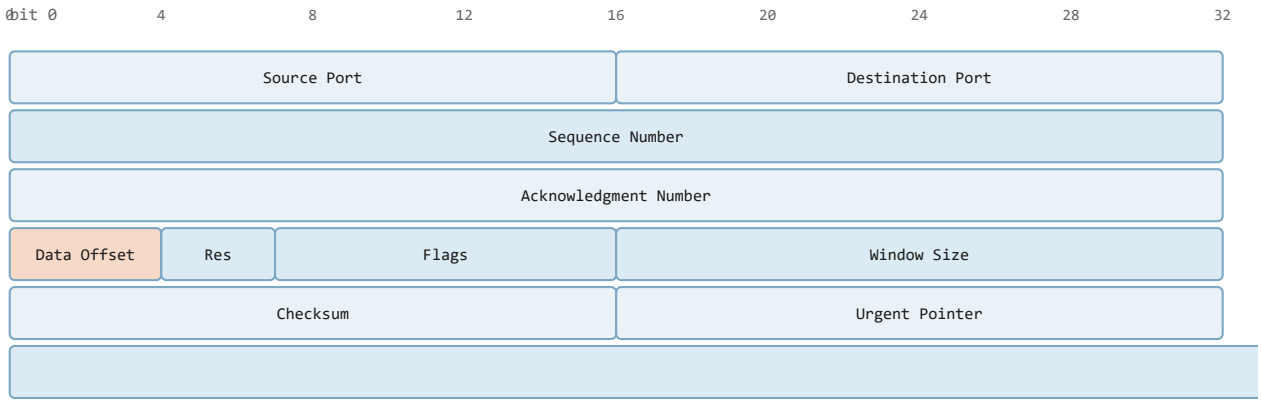
🎯 Al terminar esta clase vas a poder
1.1 Por qué 20 bytes y no 8
1.2 El formato de la cabecera
1.3 Un segmento real, byte a byte
1.4 Sequence Number y Acknowledgment Number: la aritmética
1.5 El flag PSH: lo que dice y lo que no dice
1.6 Las opciones
1.7 El checksum de TCP: Internet checksum, no CRC
1.8 Qué pasa si el checksum falla
1.9 ⚠️ Actualización a 2026
2.1 Ejercicio 1 — Data Offset y largo total de un segmento
2.2 Ejercicio 2 — Calcular y verificar el Internet checksum de un segmento TCP
2.3 Ejercicio 3 — Si el checksum falla
3.1 ⭐ Examen 2011-02, Pregunta 3 (10 puntos) — Campos más importantes de TCP y UDP
3.2 Examen 2007-03, Pregunta 4 (10 puntos) — Campos más importantes de TCP y UDP
3.3 Examen 2001-12, Problema 2 — Por qué PUSH no garantiza un mensaje por vez
3.4 Examen 2001-12, Problema 1 — Un router que interrumpe una conexión TCP sin que se entere
📌 Resumen en 10 líneas
✅ Autoevaluación
🔗 Referencias

Bloque: C (Transporte) · **Semana:** 6 · **Duración estimada:** 110 min **Correlativa:** Clase 16 (mux/demux y UDP) · **Sigue con:** Clase 18 (TCP: establecer y cerrar la conexión) **Material de referencia:** Tanenbaum cap. 6 (§6.3, §6.5) · RFC 9293, 2018, 7323, 1122

🎯 Al terminar esta clase vas a poder

1. Decir qué servicios compra TCP con sus **20 bytes** de cabecera, frente a los 8 de UDP.
2. Escribir el **formato de la cabecera** y decodificar un segmento hexadecimal byte a byte.
3. Calcular **seq y ack de bytes**, y explicar por qué el ACK es **acumulativo** y cuenta bytes.
4. Explicar qué significa realmente **PSH** y por qué **no** garantiza un mensaje por mensaje.
5. Describir las **opciones** (MSS, SACK permitted, Timestamps, Window Scale) con su RFC.
6. Calcular y verificar el **Internet checksum** de un segmento, y decir qué pasa si falla.

Cabecera TCP, campo por campo



Cabecera minima: 5 filas de 32 bits = 20 bytes. Con opciones puede llegar a 60 bytes.

> **Prerequisito del examen:** `2011-02 P3` y `2007-03 P4` (10 pts cada uno, campos de TCP y UDP) · `2001-12 Problema 2` (el PUSH no garantiza un mensaje por vez).

PARTE 1 — TEORÍA

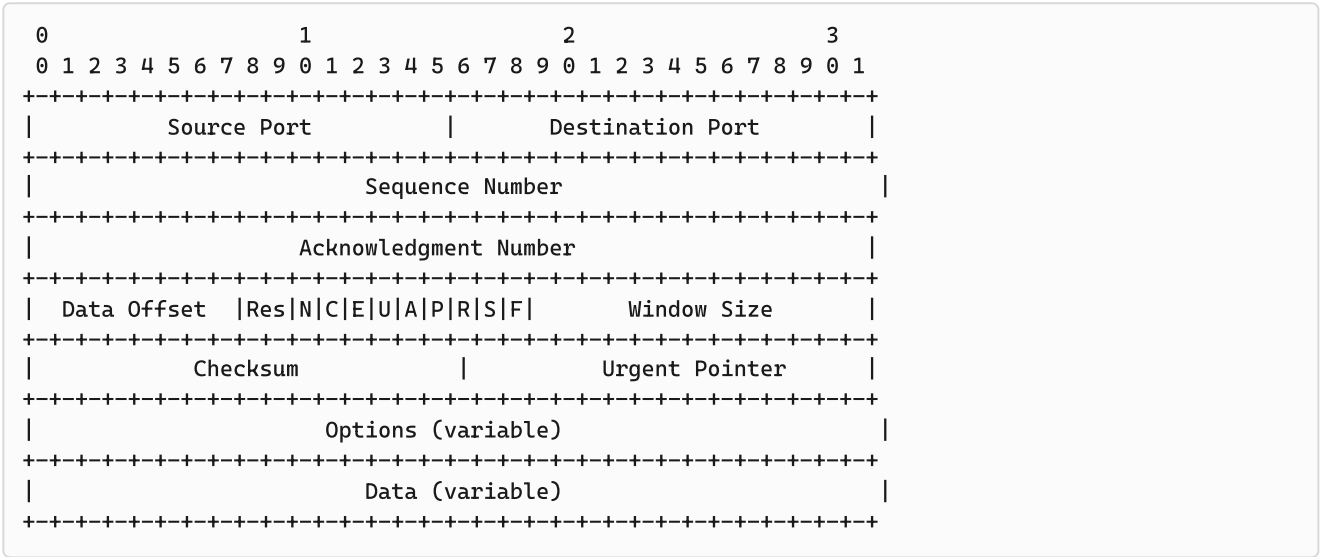
1.1 Por qué 20 bytes y no 8

UDP tiene 8 bytes y no garantiza nada. TCP tiene 20 y compra con ellos:

Bytes	Campo	Qué le compra al protocolo
4	Puertos (2 + 2)	saber a qué proceso es el flujo
8	Sequence + Acknowledgment	**orden, detección de pérdida y retransmisión**
2	Window	**control de flujo** (cuánto puede mandar sin desbordar al receptor)
2	Checksum	**detección de errores** de punta a punta, con pseudo-cabecera
2	Data Offset + Reserved + Flags	**ciclo de vida de la conexión** (SYN, ACK, FIN, RST, PSH)
2	Urgent Pointer	datos urgentes (URG), que en la práctica no se usa

Sobrevivir sin conexión, en cambio, es gratis: TCP no lo necesita porque **el número de secuencia no identifica una conexión, identifica un byte**. No tiene que preguntarle "¿de quién es este paquete?", porque el socket ya lo sabe. Esa es la diferencia conceptual más importante entre los dos.

1.2 El formato de la cabecera



5 palabras de 32 bits = 20 bytes sin opciones; como las opciones son de largo variable, hace falta el **Data Offset** para saber dónde empiezan los datos.

El byte 12 tiene los 4 bits del Data Offset, 3 reservados y el bit NS (*No Significant*, de `RFC 7323`); **el byte 13**, los 8 flags:

El byte 13 son los flags, y cada uno es **un bit**:

Bit	Valor	Flag	Significado
7	`0x80`	CWR	obsoleto (era de *congestion window reduced*)
6	`0x40`	ECE	obsoleto (era de *ECN-Echo*)
5	`0x20`	**URG**	el puntero urgente es válido
4	`0x10`	**ACK**	el campo Acknowledgment es válido
3	`0x08`	**PSH**	no esperes a llenar el buffer para mandar
2	`0x04`	**RST**	resetear la conexión / error
1	`0x02`	**SYN**	sincronizar números de secuencia (abrir)
0	`0x01`	**FIN**	no tengo más datos (cerrar)

CWR y ECE están **reservados** desde que ECN se negocia en el SYN (`RFC 3168`), y **si ACK = 0 el campo Acknowledgment se ignora**: por eso el SYN puede viajar con `ack = 0`.

1.3 Un segmento real, byte a byte

Segmento del cliente `203.0.113.9:49200` al servidor `198.51.100.7:80`, con 4 bytes de datos ("HOLA"), sin opciones:

```
Byte   0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
     C0 30 00 50 00 00 04 4C 00 00 04 60 50 18 20 00 CB C6 00 00
      └──┘ └──┘ └──┘ └──── seq ────┘ └──── ack ────┘ └┘ └──┘ └─ck─┘ └urg┘
      sport  dport = 1100     = 1120    offset flags  window
     C0 30 = 49200   00 50 = 80   00 00 04 4C = 1100   00 00 04 60 = 1120
     50 = 0101 0000  →  Data Offset 5 (= 20 bytes), reservado 0, NS 0
     18 = 0001 1000  →  ACK (0x10) | PSH (0x08)      20 00 = ventana 8192
     CB C6 = checksum (calculado en 2.2)  00 00 = urgente nulo
     datos: 48 4F 4C 41 = "HOLA"
```

Las conversiones, con la cuenta a la vista:

```
  49200 = 0xC030   (0xC000 = 49152  +  0x30 = 48)     80 = 0x0050
   1100 = 0x0000044C   (4·256 = 1024  +  0x4C = 76)   1120 = 0x00000460 (4·256 + 96)
     20 = 0x14  →  Data Offset = 0x14 / 4 = 5  →  5 × 4 = 20 bytes de cabecera
   8192 = 0x2000   (2 × 4096)       PSH|ACK = 0x08 | 0x10 = 0x18
```

El puntero urgente: si URG = 1, la secuencia urgente es `seq + puntero`. En 2026 **nadie lo usa**: ni aplicaciones ni bibliotecas lo activan.

1.4 Sequence Number y Acknowledgment Number: la aritmética

Los dos campos son de **32 bits** y el ACKing es **acumulativo** (*cumulative ACK*), y por eso el número identifica **bytes**, no segmentos. Esta es la clase de error que más puntos cuesta.

Caso base: el cliente manda segmentos de **20 bytes** desde `seq = 1100`.

```
bytes enviados:  1100 → 1119    1120 → 1139    1140 → 1159    1160 → 1179
seq del seg.:        1100           1120           1140           1160
ACK que devuelve:    1120           1140           1160           1180
                     ↑              ↑              ↑              ↑
                     el receptor dice "espero el 1120": ya tengo 1100–1119
```

El ACK es siempre "el próximo byte que espero", no "el último que recibí". Por eso el primer segmento produce `1120 = 1100 + 20`.

Ahora se pierde el segundo segmento:

```
seg1 (1100–1119) llega → receptor responde  ACK 1120   → el emisor avanza a 1120
seg2 (1120–1139) SE PIERDE  → el receptor sigue esperando 1120
seg3 (1140–1159) llega → ¿lo acepta? NO: llega fuera de orden.
                         → el receptor responde ACK 1120  (¡DUPLICADO!)
retransmisión de seg2 → ACK 1140
seg4 (1160–1179) llega → ACK 1180
```

Los **ACKs duplicados** (`1120` , `1120` , `1120`) evidencian que falta un segmento y disparan el *fast retransmit* de la clase 21: con 3 se retransmite sin esperar el RTO.

Consecuencia: un solo ACK puede confirmar **varios segmentos a la vez**, y uno puede quedar confirmado **parcialmente**: la unidad es **siempre el byte**.

1.5 El flag PSH: lo que dice y lo que no dice

Acá está el malentendido clásico, y es lo que el examen 2001-12 pregunta explícitamente.

En el receptor, PSH no obliga a nada: es una *pista* (*hint*) para el stack local. La decisión de llamar a `read()` y devolver uno o varios mensajes es **de la aplicación y del buffer del socket**, y entre el cable y el `read()` hay **tres capas de buffering**: socket, reordenamiento y transporte. En el emisor, además, está el **WAL**.

Las tres razones por las que PUSH no garantiza nada:

1. **Segmentación.** Si el mensaje es más grande que el MSS, TCP lo parte en varios segmentos: el PSH no lo impide.
2. **WAL (*write-ahead log*) en el emisor.** El PSH queda marcado en un buffer y puede no salir si la ventana está llena o si Nagle está activo: se "pierde" hasta que el segmento sale.
3. **Buffering en el receptor.** Con dos mensajes en el buffer del socket, **un solo `read()` puede devolver los dos**; y con reordenamiento, el final del 2 junto con el principio del 1.

En la práctica **todas las implementaciones lo ponen siempre y casi ninguna receptora lo trata de forma especial**: hoy es decorativo.

1.6 Las opciones

Cada opción tiene la forma `Kind (1 byte) + Length (1 byte) + datos`, y el **Length incluye los dos primeros bytes**. Si el total no es múltiplo de 4 se completa con **NOP (Kind 1)**; el fin se marca con **EOL (Kind 0)**.

Opción	Kind	Long.	Qué negocia	RFC
EOL	0	—	fin de las opciones	RFC 9293
NOP	1	1	alineación	RFC 9293
MSS (Maximum Segment Size)	2	4	máximo de bytes de datos **por segmento**	RFC 9293 (§3.7.1)
Window Scale	3	3	multiplica la ventana ×2^shift	`RFC 7323`
SACK Permitted	4	2	"sé usar ACK selectivo"	`RFC 2018`
Timestamps	8	10	TSval y TSecr para medir RTT y proteger PAWS	`RFC 7323`

En un SYN se ven así:

```
02 04 05 B4   →  MSS = 0x05B4 = 1460 bytes de datos por segmento
04 02         →  SACK permitted (2 bytes, sin datos)
08 0A 00 00 1E D4 00 00 00 00
              →  Timestamps: TSval = 0x1ED4 = 7884, TSecr = 0
Total = 4 + 2 + 10 = 16 bytes  →  Data Offset = 5 + 16/4 = 9  →  36 bytes
```

Las tres que importan:

- **MSS**: máximo de **datos**, no de segmento. Con IP y Ethernet (MTU 1500) sale 1500 − 20 − 20 = **1460**. Se anuncia **solo en el SYN** y gana el **menor**: es lo que evita la fragmentación de IP.
- **SACK permitted** (`RFC 2018` , *Standards Track*): es solo el **permiso**. Si los dos lo mandan, después un segmento puede llevar campos SACK con los bloques que faltan. Sigue siendo comportamiento estándar y es lo que usan Linux y Windows.
- **Timestamps** (`RFC 7323` , que **obsolesce a la RFC 1323**): 10 bytes con **TSval** (el reloj del emisor) y **TSecr** (su eco). Sirven para medir el RTT exacto y para **PAWS** (*Protect Against Wrapped Sequence numbers*), **obligatorio** si ambos extremos los activan.

1.7 El checksum de TCP: Internet checksum, no CRC

El pseudo-cabecera es el mismo de la clase 16, con el protocolo = **6**:

```
src (4)  dst (4)  zero (1)  protocolo = 6 (1)  longitud del segmento TCP (2)
```

La diferencia con UDP: la longitud es la del **segmento TCP completo**, cabecera + opciones + datos (en el ejemplo, 20 + 4 = 24 = `0x0018`).

Y el punto conceptual que se pregunta: **es un checksum, no un CRC.** Son algoritmos distintos:

	Internet checksum (TCP/UDP/IP)	CRC (Ethernet)
Qué es	**aditivo**: suma y complemento a uno	**polinómico**: división en campo de Galois
Errores de 1 bit	**siempre los detecta**	siempre
Errores de varios bits	solo los que no se compensan	sí, incluidos los *bursts*
Coste	sumar	tabla + divisiones

Por qué detecta todos los errores de un bit: cada bit aporta un `±2^k` distinto (k = 0..15), así que alterar un bit cambia la suma en un valor que **nunca da cero** en complemento a uno. Con dos o más bits, las

contribuciones pueden **cancelarse**.

1.8 Qué pasa si el checksum falla

El segmento se descarta en silencio. No hay NAK, no hay ICMP, no hay aviso. ¿Por qué? (a) un NAK por cada datagrama corrupto genera una **tormenta de ACKs negativos**; (b) no arregla nada, porque un NAK perdido es tan inútil como el segmento perdido; y (c) el checksum existe para **no entregar datos malos**, no para informar.

Para el emisor, un segmento corrupto es indistinguible de uno perdido: se dispara el mismo mecanismo (RTO o 3 ACKs duplicados) y se retransmite. O sea: **el checksum convierte errores en pérdidas**, y para TCP eso es transparente.

1.9 ⚠️ Actualización a 2026

Sigue igual — de memoria: 20 bytes mínimos; los 10 campos; seq y ack de **32 bits y de bytes**; ACK **acumulativo**; ventana de **16 bits**; los 6 flags; Data Offset en palabras de 32 bits; Internet checksum con pseudo-cabecera; y ante un checksum roto, **descarte en silencio**.

Cambió o hay que corregirlo:

- `RFC 9293` **(2022, *Internet Standard*) sustituye a la** `RFC 793` **de 1981** (consolida 793 + RFC 879, 2873, 6093, 6429, 6528 y 6691). Citar "RFC 793" como especificación vigente en 2026 está desactualizado; `RFC 1122` sigue definiendo el comportamiento de los hosts.
- **Timestamps y Window Scale:** `RFC 7323` **(2014)**, que **obsolesce a la RFC 1323**, y **PAWS es obligatorio** si ambos activan timestamps: con 32 bits y enlaces rápidos, un paquete viejo podría aceptarse como nuevo.
- **SACK:** `RFC 2018` **(1996)**, *Standards Track*, **sigue siendo** el mecanismo estándar.
- **URG está obsoleto en la práctica**: ninguna aplicación ni biblioteca moderna lo usa. Y **PSH es decorativo**: casi todos los stacks lo mandan siempre. **QUIC elimina el problema de raíz**, porque los *streams* de HTTP/3 tienen **frontera de mensaje explícita**: un mensaje nunca se parte ni se junta con otro, sin depender de un flag.

Consecuencia para el examen: `2011-02 P3` y `2007-03 P4` **se siguen preguntando tal cual**: la respuesta esperada es la lista de campos con su significado. Para sumar, citá **RFC 9293** en lugar de RFC 793.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Data Offset y largo total de un segmento

Enunciado. Capturás un segmento TCP con estos primeros 20 bytes:

```
C0 30 00 50 00 00 04 4C 00 00 04 60 70 18 FA F0 12 34 00 00
02 04 05 B4 04 02 08 0A 00 00 1E D4 00 00 00 00
48 4F 4C 41 41 42 43 ...
```

a) ¿Cuánto vale el Data Offset? **b)** ¿Cuántas opciones hay y cuáles? **c)** ¿Cuánto mide el segmento completo si el datagrama IP mide 68 bytes? **d)** ¿Qué flags tiene?

a) El Data Offset. El byte 12 es `0x70 = 0111 0000` → Data Offset = **7** → **7 × 4 = 28 bytes** de cabecera (reservados `000`, NS `0`).

b) Las opciones. Desde el byte 20 hasta el 27 inclusive, o sea `28 − 20 = 8` bytes:

```
02 04 05 B4     →  Kind 2 (MSS), Length 4, valor 0x05B4 = 1460
04 02           →  Kind 4 (SACK permitted), Length 2
── y sobran 2 bytes sin EOL ni NOP: son padding, u opciones mal formadas.
```

Interpretación: el Data Offset 7 con solo 8 bytes de opciones visibles deja 8 bytes de hueco; en una captura real eso es padding. Lo que **no** puede pasar es que el Data Offset apunte más allá de los datos.

c) El largo total. El datagrama IP mide 68 bytes; le restamos la cabecera IPv4 (20): `68 − 20 = 48` bytes de segmento TCP, de los cuales 28 son cabecera ⇒ **20 bytes de datos**. *No hay contradicción con el MSS de 1460*: el MSS es un **máximo**, y se mide en **datos**, sin la cabecera.

d) Los flags. El byte 13 es `0x18`:

```
0x18 = 0001 1000  →  ACK (bit 4 = 0x10) | PSH (bit 3 = 0x08)
→  PSH + ACK, y no es SYN ni FIN ni RST
```

2.2 Ejercicio 2 — Calcular y verificar el Internet checksum de un segmento TCP

Datos. Segmento de `203.0.113.9` a `198.51.100.7`, protocolo 6, longitud **24** bytes: la cabecera de 20 del ejemplo de §1.3 (`seq = 1100`, `ack = 1120`, ventana 8192, flags `0x18`) y 4 de datos "HOLA".

a) Escribí las palabras a sumar. **b)** Sumá. **c)** ¿Cuál es el checksum? **d)** Verificá.

a) Las palabras (con el checksum en 0, que es como se calcula):

```
Pseudo-cabecera  CB00  7109  C633  6407  0006  0018    src, dst, proto 6, long. TCP 24
Cabecera TCP     C030  0050  0000  044C  0000  0460    ports, seq 1100, ack 1120
                 5018  2000  0000  0000                offset+flags, ventana, ck=0, urgente
Datos            484F  4C41                            "HO", "LA"
```

b) La suma con acarreo devuelto:

```
CB00 + 7109 = 13C09  → 3C0A
3C0A + C633 = 1023D  → 023E
023E + 6407 = 06645  → 6645
6645 + 0006 = 0664B  → 664B
664B + 0018 = 06663  → 6663
6663 + C030 = 12693  → 2694
2694 + 0050 = 026E4  → 26E4
26E4 + 0000 = 026E4  → 26E4
26E4 + 044C = 02B30  → 2B30
2B30 + 0000 = 02B30  → 2B30
2B30 + 0460 = 02F90  → 2F90
2F90 + 5018 = 07FA8  → 7FA8
7FA8 + 2000 = 09FA8  → 9FA8
9FA8 + 0000 = 09FA8  → 9FA8      ← el checksum, en 0
9FA8 + 0000 = 09FA8  → 9FA8      ← puntero urgente
9FA8 + 484F = 0E7F7  → E7F7
E7F7 + 4C41 = 13438  → 3439
──────────────────────────────────────────
suma final = 0x3439
```

c) El checksum es el complemento a uno de 16 bits:

```
0xFFFF − 0x3439 = 0xCBC6
```

d) Verificación. `3439 + CBC6 = FFFF`, sin acarreo, y el complemento de `FFFF` es `0x0000`: **el mismo criterio que en UDP, `0x0000` significa "segmento íntegro"**.

2.3 Ejercicio 3 — Si el checksum falla

Enunciado. Al segmento del ejercicio 2.2 le llega un **único bit alterado** en los datos: el byte `4F` de "HO" pasa a `50`. **a)** ¿Lo detecta el receptor? **b)** ¿Qué hace? **c)** ¿Qué le pasa al emisor?

a) La palabra `484F` pasa a `4850`, o sea **+1** en la suma:

```
suma original = 0x3439
suma nueva   = 0x3439 + 1 = 0x343A
checksum que calcularía el receptor = 0xFFFF − 0x343A = 0xCFB7
checksum que venía en el segmento   = 0xCBC6
0xCFB7 ≠ 0xCBC6  →  HAY ERROR
```

Interpretación: con **un solo bit** la diferencia es siempre $\pm 2^k$, que en complemento a uno nunca da cero: por eso se detectan todos los errores de un bit.

b) El receptor **descarta el segmento en silencio**: no ACK, no NAK, no ICMP. Los datos no llegan a la aplicación.

c) El emisor **no se entera de nada** hasta que expira el RTO, o hasta que recibe 3 ACKs duplicados. Y como un segmento corrupto es indistinguible de uno perdido, TCP lo **retransmite** igual. **El checksum convierte errores de bits en pérdidas, y el mecanismo de pérdida es transparente para la aplicación.** Ese es el diseño: el checksum protege la integridad, la retransmisión protege la entrega.

Y el caso inverso, para entender por qué no alcanza: si se alteran **dos bits de igual peso en palabras distintas** (por ejemplo, el bit 0 de `C030` hacia 1 y el bit 0 de `C633` hacia 0), las contribucciones son `+1` y `−1`, se cancelan, la suma da lo mismo y **el receptor acepta el segmento dañado**. Por eso TCP no confía en el

checksum para detectar corrupción: confía en que la red no corrompe, y el checksum es la primera línea, no la última.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-02, Pregunta 3 (10 puntos) — Campos más importantes de TCP y UDP

> **Fuente:** Examen `eirc1102` / `sirc1102` — 3 de febrero de 2011.

Enunciado (texto del examen)

> Mencione y explique los campos más importantes de los segmentos TCP y UDP.

Solución oficial

La solución publicada **remite a una diapositiva del teórico** (`cap03_3_2009_2xcarilla.pdf`, slide 5): no hay texto de respuesta. Lo que sigue es **solución propia**.

Mi resolución (solución propia)

UDP — 4 campos, 8 bytes: puertos origen y destino (16 bits cada uno, que con la IP dan la 2-tupla del socket no conectado); **Length** (longitud **total** del datagrama, mínimo 8); **Checksum** (Internet checksum **con pseudo-cabecera**, opcional en IPv4 y obligatorio en IPv6; si falla se **descarta en silencio**); y los **datos** (0 a 65 507 bytes).

TCP — los 10 campos, 20 bytes:

1. **Source Port / Destination Port** (16 + 16): con las dos IP forman la **4-tupla** de la conexión.
2. **Sequence Number** (32): número del **primer byte**; cuenta **bytes**, no segmentos.
3. **Acknowledgment Number** (32): **próximo byte esperado**, **acumulativo**; solo vale si ACK está activo.
4. **Data Offset** (4 bits): largo de la cabecera en **palabras de 32 bits**, que marca dónde empiezan los datos.
5. **Reservado** (3 bits) y **NS** (1 bit), sin uso real.
6. **Flags** (8 bits): **URG, ACK, PSH, RST, SYN, FIN** (más CWR/ECE reservados).
7. **Window Size** (16): bytes que puede mandar el emisor desde el último confirmado → **control de flujo**; con *Window Scale* se multiplica por 2^shift.
8. **Checksum** (16): Internet checksum con **pseudo-cabecera de protocolo 6** y longitud del **segmento TCP**.
9. **Urgent Pointer** (16): desplazamiento del dato urgente desde `seq`, si URG está activo.
10. **Options**: MSS, SACK permitted, Window Scale, Timestamps, que se anuncian **en el SYN**.

Tope para 10 puntos: los 4 de UDP y 8 o 10 de TCP, cada uno con su tamaño en bits y una frase. La diferencia clave: **UDP identifica un socket; TCP identifica una conexión**, y por eso necesita números de secuencia, ventana y flags de ciclo de vida.

3.2 Examen 2007-03, Pregunta 4 (10 puntos) — Campos más importantes de TCP y UDP

> **Fuente:** Examen `eirc0703` / `sirc0703` — 26 de febrero de 2007.

Enunciado (texto del examen)

> Mencione y explique los campos más importantes de los segmentos TCP y UDP.

Solución oficial (verbatim, resumida)

Enumera, en el orden de la diapositiva del teórico, **13 ítems de TCP** (puertos, seq/ack, largo del cabezal, URG, ACK, PSH, RST, SYN, FIN, ventana, checksum, opciones, datos) y **3 de UDP** (puertos, largo del cabezal, checksum).

Análisis

Bien: la lista es completa y sigue el orden del formato de la cabecera, que es como hay que responder.

Tres cosas que un corrector riguroso puede marcar:

1. **PSH.** La solución oficial dice que indica que "los datos deben ser enviados a la capa de aplicación inmediatamente luego de ser recibidos". Eso **describe el efecto observable** pero no el **mecanismo**, y por eso lleva a la confusión clásica: lo correcto es que PSH es un aviso del emisor al stack receptor de que **no espere a llenar el buffer**, y **no** obliga a la entrega inmediata a la aplicación ni garantiza que un mensaje llegue entero.
2. **"El puerto más la dirección IP forman un único punto de acceso."** Correcto para un socket UDP no conectado, pero incompleto: la clave real del kernel es la **5-tupla**, y la conexión TCP es la **4-tupla**.
3. **"Utilizar selective repeat en lugar de go back n."** Es una glosa de la opción **SACK**, pero SACK **no es** Go-Back-N ni Selective Repeat de capa 2: es **retransmisión selectiva de bytes sobre un protocolo que igual entrega en orden** (`RFC 2018`). Si lo decís así sumás; si lo decís como "selective repeat", restás.

3.3 Examen 2001-12, Problema 2 — Por qué PUSH no garantiza un mensaje por vez

> **Fuente:** Examen `eirc0112` — 27 de diciembre de 2001.

Enunciado (texto del examen)

> Es sabido que, al enviar mensajes mediante una conexión TCP, el mero uso del bit de PUSH no asegura que, en el destino, la capa de aplicación reciba un mensaje por vez. Se desea realizar una prueba de laboratorio que demuestre este concepto. Se pide construir un bridge que interconecte los equipos de

> origen y destino, y manipule el orden de transmisión de los paquetes y eventualmente la segmentación de los mismos (como podría ocurrir en una red real) de forma que en el destino se experimenten las siguientes situaciones: a) Arribo simultáneo de dos mensajes a la capa de aplicación de destino. b) Arribo simultáneo de un mensaje y de parte del siguiente a la capa de aplicación de destino.

Solución oficial (verbatim)

> a) Cambiar el orden de los mensajes. Se envían el 1 y el 2, ambos con push, pero se recibe primero el 2. Al recibirse el 1 se enviarán ambos a la capa de aplicación. b) Como en a) pero segmentando además los mensajes. Alcanza con que al equipo receptor llegue primero el comienzo del mensaje 2 y luego el mensaje 1 completo.

Mi resolución, explicada

a) Dos mensajes en un solo `read()`. El emisor manda `MSG1` y `MSG2`, ambos con PSH. El bridge **los reordena** y entrega primero el de `MSG2`, que queda esperando en el buffer del socket porque la aplicación todavía no hizo `read()`. Cuando llega `MSG1`, el stack lo **reencola por número de secuencia** y las dos cosas quedan **contiguas**: un solo `read()` devuelve los dos mensajes pegados. **Los dos segmentos tenían PSH y ninguno garantizó nada.**

b) Un mensaje entero y parte del siguiente. El bridge además **segmenta** `MSG2`, reordena, y hace llegar primero su **primer segmento** (digamos 100 bytes) y después el `MSG1` completo. El buffer queda `MSG1 | 100 bytes de MSG2`, todo contiguo: **un `read()` devuelve un mensaje entero y el comienzo del siguiente**.

Las tres razones de fondo (§1.5), que es la parte teórica que hay que escribir:

1. **Segmentación**: el PSH no impide que un mensaje se parta si es más grande que el MSS.
2. **WAL del emisor**: el PSH se marca en el buffer de envío y puede salir tarde si la ventana está llena o si Nagle está activo.
3. **Buffering del receptor**: el paso de bytes a la aplicación lo deciden el `read()` y el buffer del socket. El stack solo ve un **flujo de bytes**: **la frontera de mensaje no existe en TCP**, es un concepto de la capa de aplicación.

3.4 Examen 2001-12, Problema 1 — Un router que interrumpe una conexión TCP sin que se entere

> **Fuente:** Examen `eirc0112` — 27 de diciembre de 2001. *Se resume la idea; el enunciado completo y el pseudocódigo están en `sirc0112`.*

El problema. A tiene dos conexiones TCP hacia los puertos 9001 y 9002 de B y manda datos continuamente; B solo contesta ACKs. En la congestión hay capacidad para **una** de las dos transmisiones, y hay que **interrumpir la del 9002** sin que A ni B se enteren, aceptando pérdida de información. El enunciado sugiere alterar los **campos de secuencia** de los datos y de los ACKs.

La idea en una frase: el router **descarta** los datos del 9002 y **les miente a los dos extremos**: le **falsifica un ACK** a A diciendo que los recibió, y le **resta a B** los bytes descartados, de modo que **B no note el hueco**.

```
A manda 1000..1099 → R lo descarta (dif_SN = 100) y manda a A un ACK 1100 FALSO
A cree que recibió todo y sigue con 1100..1199 → R lo pasa a B con SN = 1100..1199 (1200 − 100)
B responde ACK 1200 → R lo corrige hacia A como ACK 1100 → A lo retransmite → R lo descarta
```

Las dos invariantes: (1) **A nunca ve un hueco**, porque R le manda un ACK por cada byte que descarta; (2) **B nunca ve un hueco**, porque R le resta a los datos hacia B la cantidad total de bytes descartados (`dif_SN`) y suma esa misma cantidad a los ACKs hacia A, para que todo siga cuadrando con lo que A cree haber enviado.

Por qué se puede perder información en silencio: en TCP **el byte nunca se confirma uno por uno**; lo que importa es la **numeración**. Si R la mantiene coherente de los dos lados, ni A ni B detectan el hueco. Es el mismo principio que usa un proxy TProxy.

Dos errores de la solución oficial publicada, que son material de examen: el `select` tiene la condición **invertida** (`if ((status = select(sel)) <> -1) { error; exit(); }` mata el proceso cuando `select` **tiene éxito**), y el ACK falso se arma con el `largo` **ya restado por la cabecera** y con el `ACK` de un paquete de datos en el campo `SN` . A eso se suman que **no retransmite** el ACK falso ni **reinicia** `dif_SN` : los dos bugs que hay que mirar en cualquier implementación de este tipo.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. TCP tiene **20 bytes** mínimos y compra con ellos orden, ACK, control de flujo, checksum y ciclo de vida de la conexión.
2. **seq** y **ack** son de **32 bits y cuentan BYTES**: el ACK es el **próximo byte esperado** y **acumulativo**.
3. Los **ACKs duplicados** evidencian un hueco y disparan el *fast retransmit*.
4. **Data Offset** va en **palabras de 32 bits** (`5` = 20 bytes); el **byte 12** tiene offset + reservados + NS y el **byte 13** los flags (CWR y ECE reservados; URG, ACK, PSH, RST, SYN, FIN).
5. **PSH** significa "no esperes a llenar el buffer", **no** "entregá esto ahora": segmentación, WAL del emisor y buffering del receptor rompen la frontera de mensaje.
6. **Opciones**: MSS (2), Window Scale (3), SACK permitted (4), Timestamps (8), con su RFC.
7. El **checksum** es un Internet checksum de complemento a uno **con pseudo-cabecera**, **no un CRC**: ve todos los errores de un bit, pero no todos los múltiples.
8. Si el checksum falla, el segmento **se descarta en silencio** y para el emisor es indistinguible de uno perdido.
9. `RFC 9293` **(2022)** sustituye a la `RFC 793`; `RFC 7323` obsolesce a la 1323; SACK es `RFC 2018`.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un segmento con `seq = 1100` y 20 bytes de datos. ¿Qué ACK devuelve el receptor?

<details>
<summary>Respuesta</summary>
</details>

2. Si `Data Offset` vale 5, ¿cuántos bytes de cabecera tiene el segmento?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Qué le dice realmente el flag PSH?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué se pone el MSS en el SYN y no en todos los segmentos?

<details>
<summary>Respuesta</summary>
</details>

5. Calculá el checksum de un segmento cuya suma (sin el campo checksum) da `0x7FF0`.

<details>
<summary>Respuesta</summary>
</details>

6. Un segmento llega con el checksum corrupto. ¿Qué hace el receptor y qué hace el emisor?

<details>
<summary>Respuesta</summary>
</details>

7. ¿Por qué el checksum de TCP es un Internet checksum y no un CRC como el de Ethernet?

▶ Respuesta

8. ¿Por qué no basta con tres de los cuatro valores de la 4-tupla para elegir un socket TCP?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc1102` **/** `sirc1102` — P3 (10 pts, campos más importantes de TCP y UDP)
- **Examen** `eirc0703` **/** `sirc0703` — P4 (10 pts, lo mismo, con la lista de 13 + 3 campos)
- **Examen** `eirc0112` **/** `sirc0112` — Problema 2 (PUSH) · Problema 1 (router que interrumpe una conexión TCP)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 — §6.3 (*Segmentos TCP*), §6.5
- `RFC 9293` — *TCP* (2022, *Internet Standard*; sustituye a la RFC 793) · `RFC 1122` — requisitos de los hosts · `RFC 1071` — Internet checksum
- `RFC 2018` — SACK (1996, *Standards Track*) · `RFC 7323` — Timestamps y Window Scale (2014; obsolesce la RFC 1323)
- `RFC 3168` / `RFC 9768` / `RFC 9331` — ECN, AccECN y L4S · `RFC 9000` / `RFC 9114` — QUIC y HTTP/3: el **stream** reemplaza a la frontera de mensaje de TCP
- **Clase siguiente:** 18 — TCP: establecer y cerrar la conexión

Clase 18

Contenido de esta clase

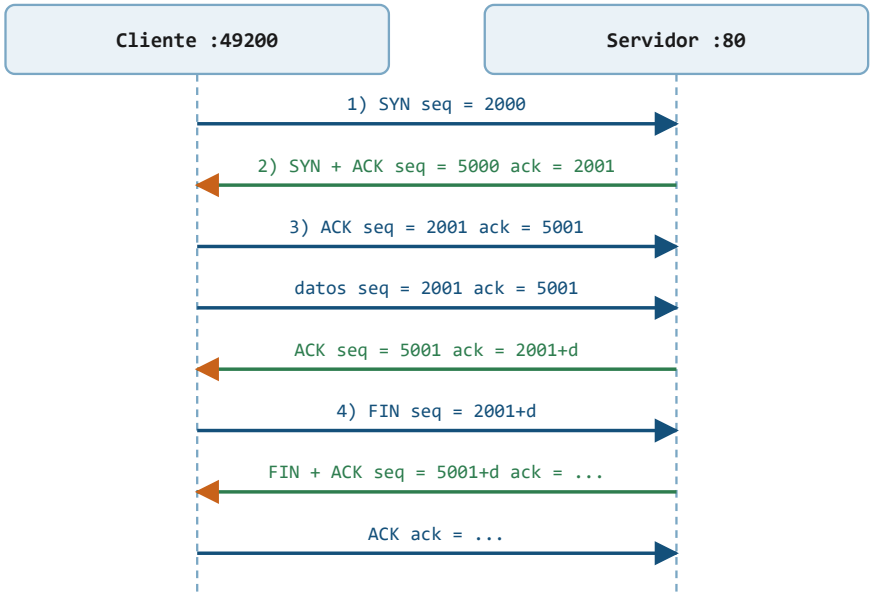
🎯 Al terminar esta clase vas a poder
1.1 Por qué hace falta un handshake
1.2 El three-way handshake, con los números
1.3 ¿Por qué 3 vías y no 2?
1.4 SYN flood y SYN cookies
1.5 El cierre de 4 vías, con los números
1.6 RST: el cierre abrupto
1.7 La máquina de estados: 11 estados
1.8 TIME-WAIT: 2 × MSL
1.9 ⚠️ Actualización a 2026
2.1 Ejercicio 1 — La secuencia exacta de cada segmento
2.2 Ejercicio 2 — ¿Qué pasa si se pierde el tercer segmento (el ACK del cliente)?
2.3 Ejercicio 3 — ¿Cuánto RTT cuesta establecer una conexión?
3.1 ⭐ Examen 2006-12, Pregunta 4 (10 puntos) — Three-way handshake y cierre
3.2 Examen 2009-07, Pregunta 2a (8 puntos) — Los pasos del handshake
📌 Resumen en 10 líneas
✅ Autoevaluación
🔗 Referencias

Bloque: C (Transporte) · **Semana:** 6 · **Duración estimada:** 110 min **Correlativa:** Clase 17 (TCP: la cabecera campo por campo) · **Sigue con:** Clase 19 (TCP: cómo se garantiza la fiabilidad) **Material de referencia:** Tanenbaum cap. 6 (§6.3, §6.5) · RFC 9293, RFC 4987, RFC 1122, RFC 7414

🎯 Al terminar esta clase vas a poder

1. Dibujar el **three-way handshake** con números de secuencia concretos y explicar qué se acuerda en cada mensaje.
2. Explicar **por qué 3 vías y no 2**, y por qué el servidor no debe quedar con un socket abierto para siempre.
3. Describir el **SYN flood** y cómo lo mitigan las **SYN cookies** (`RFC 4987`).
4. Dibujar el **cierre de 4 vías** con los números avanzando, y explicar el **semi-cierre** y el **RST**.
5. Recorrer los **11 estados** de la máquina de estados y justificar el **TIME-WAIT de 2×MSL**.
6. Calcular el **RTT** de establecer una conexión y sus variantes (TLS 1.3, 0-RTT, TCP Fast Open).

Three-way handshake y cierre de 4 vias (Examen 2006-12 P4)



> **Prerequisito del examen:** `2006-12 P4` (10 pts — three-way handshake, cierre y TPDUs), `2009-07 P2a` (8 pts — los pasos del handshake).

PARTE 1 — TEORÍA

1.1 Por qué hace falta un handshake

Para mandar datos de forma confiable, los dos extremos tienen que ponerse de acuerdo en **cinco cosas**. Ninguna se puede inventar solo uno de los dos lados:

Variable	Quién la propone	Cómo viaja
ISN del cliente (*Initial Sequence Number*)	el cliente	en el `seq` del SYN
ISN del servidor	el servidor	en el `seq` del SYN+ACK
MSS de cada lado	cada uno	opción MSS en el SYN
SACK permitted, **Window Scale**, **Timestamps**	cada uno	opciones en el SYN
ECN	cada uno	bits CWR/ECE en el SYN

Ojo con la primera fila: los dos ISN no se "acuerdan", se intercambian. Cada extremo **elige el suyo por su cuenta** y el handshake sirve para que cada uno **se entere del del otro**. La `RFC 9293` da la fórmula:

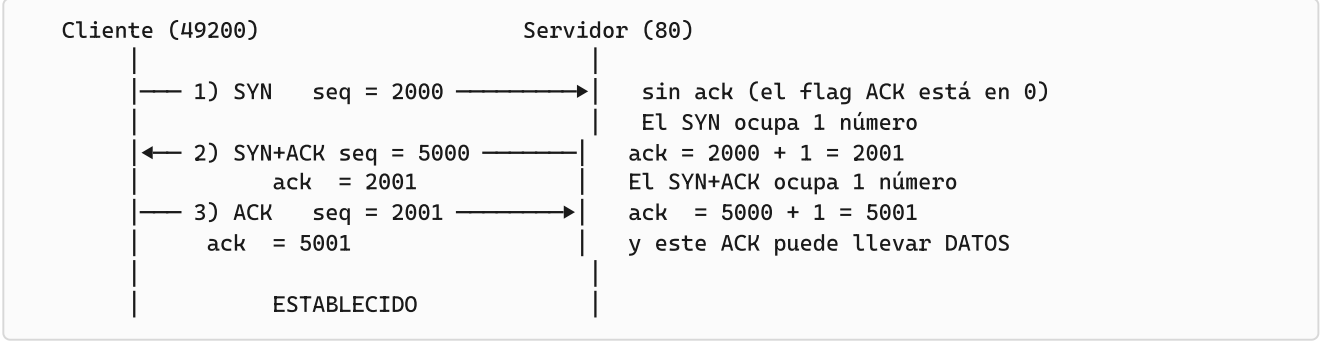
```
ISN = M + F(localIP, localPort, remoteIP, remotePort, clave_secreta)

M = un contador que sube al menos cada ~4 microsegundos
F() = función pseudoaleatoria con clave secreta, NO computable de afuera
```

El reloj `M` garantiza que los ISN **no se repitan** dentro de un MSL, y `F()` con clave secreta evita que un atacante **adivine** el número de secuencia (sin eso se puede hacer *spoofing* de segmentos). El ciclo de 32 bits a 4 µs dura **4,55 horas**.

1.2 El three-way handshake, con los números

Voy a usar estos valores en toda la clase, porque después los ejercicios los retoman:

```
Cliente 203.0.113.9, puerto 49200, ISN = 2000 (0x07D0)
Servidor 198.51.100.7, puerto 80,     ISN = 5000 (0x1388)
Ventana anunciada por el cliente: 64240 ; por el servidor: 65535
Opciones en ambos SYN: MSS 1460, SACK permitted, Timestamps, Window Scale
```



Recorrido segmento por segmento, con la cuenta:

#	Seg.	Flags	Long. datos	seq	ack	Columna
1	C→S	SYN	0	**2000**	(0, sin uso)	Empieza la conexión en 2000
2	S→C	SYN+ACK	0	**5000**	**2001**	Confirma que recibió el byte 2000 del cliente
3	C→S	ACK	0 (+datos)	**2001**	**5001**	Confirma que recibió el byte 5000 del servidor

Las dos reglas que hacen que los números cierren: (1) **el SYN consume un número** aunque no lleve datos, por eso el `ack` del segundo segmento es `2000 + 1 = 2001` y no 2000 (el **FIN** también lo hace); (2) el **`ack` es siempre "el próximo byte que espero"**, así que el `seq` del cliente pasa a 2001 y el del servidor a 5001.

> `RFC 9293`, textual: "*the SYN is considered to occur before the first actual data octet of the segment in which it occurs, while the FIN is considered to occur after the last actual data octet*".

El tercer segmento puede llevar datos. Por eso establecer una conexión TCP cuesta **1 RTT y no 2**: el cliente manda su primer request en el mismo ACK que cierra el handshake, con `seq = 2001`.

1.3 ¿Por qué 3 vías y no 2?

Un handshake de 2 vías sería: el cliente manda SYN, el servidor responde SYN+ACK, y se da por establecida. El problema es que **el servidor no tiene ninguna prueba de que el cliente exista**:

```
Con 3 vías                          Con 2 vías
──────────                          ──────────
C ──SYN x──────────► S              C ──SYN x──────────► S
C ◄──SYN+ACK y, x+1── S             C ◄──SYN+ACK y, x+1── S
C ──ACK x+1, y+1──► S               (nada más)
↑ el servidor ya sabe que             ↑ el servidor se queda esperando
  hay alguien del otro lado             para siempre

Si se pierde el 3er mensaje:
  el servidor reintenta el SYN+ACK
  → el cliente lo re-responde → se resuelve
```

Los dos objetivos reales del tercer mensaje: (1) **que el servidor tenga evidencia de que hay un cliente real**: con 2 vías queda con un socket esperando un ACK que nunca llega, ocupando memoria (el **TCB**: buffers, números, temporizadores), y un atacante puede abrir miles así solo mandando SYNs; (2) **que los dos extremos synchronizeden sus números antes de mandar datos**, para que ninguna respuesta se pierda.

La apertura simultánea. Existe un caso raro en que **los dos mandan SYN** al mismo tiempo (ambos quieren abrir). Los dos pasan a SYN-SENT, cada uno responde al SYN del otro con un SYN+ACK, y cada uno envía el ACK final. Son **4 mensajes** y termina igual. Es lo que menciona la solución oficial del examen 2006-12 con "intercambio de solicitudes de conexión".

1.4 SYN flood y SYN cookies

El ataque. El atacante manda **ráfagas de SYNs** con **IPs de origen falsificadas**. El servidor, que no puede distinguir un SYN real de uno falso, **crea un TCB por cada uno** (la `RFC 793` exige reservar estado al recibir el SYN), y como las respuestas van a IPs inventadas **nadie contesta**. El backlog se llena y **las conexiones**

legítimas ya no pueden entrar. Lo peor: con ~1300 bytes de memoria por TCB, **no hace falta mucho tráfico** para tumbar un servidor.

```
atacante ─── miles de SYNs spoofeados ──→ servidor: backlog lleno  ✗
cliente legítimo ─── SYN ──────────────→        descartado

¿Qué falta? que el servidor pueda DIFERENCIAR un SYN real de uno falso. Con IPs
falsificadas la respuesta no vuelve nunca → no hay prueba de nada.
```

La defensa: las SYN cookies (`RFC 4987` — *TCP SYN Flooding Attacks and Common Mitigations*, **Informational**, agosto 2007, verificado en rfc-editor.org). La idea:

```
SIN cookies                          CON cookies
──────────                           ──────────
SYN → crear TCB (~1 KB)              SYN → NO crear nada
     ↓                                    ↓
SYN+ACK a la IP (puede ser falsa)    calcular una "cookie" con el estado
← queda un TCB por cada ataque        ISN = f(5-tupla, contador, MSS, secreto)
                                     SYN+ACK con ese ISN   ← no hay estado guardado
                                          ↓
                                     llega el ACK → RECALCULAR la cookie
                                          ↓
                                     si coincide → recién AHORA se crea el TCB
```

La chave genial: no hay nada que guarden. El estado que el servidor necesitaría (la 5-tupla, el MSS del cliente, los timestamps) **se codifica dentro del propio número de secuencia** del SYN+ACK. Cuando llega el ACK, el servidor **vuelve a calcular la cookie** desde cero y compara: si coincide, sabe que el cliente recibió el SYN+ACK (o sea, que existe) y recién ahí reserva memoria.

La implementación de referencia trunca el ISN en **5 bits** altos del contador de tiempo (cambia cada 64 s, para que las cookies no se repitan), **3 bits** con uno de 8 MSS comunes y **24 bits** de un *hash* MD5 de las direcciones, los puertos y un secreto del servidor.

Los costos (y por eso no se usa todo el tiempo):

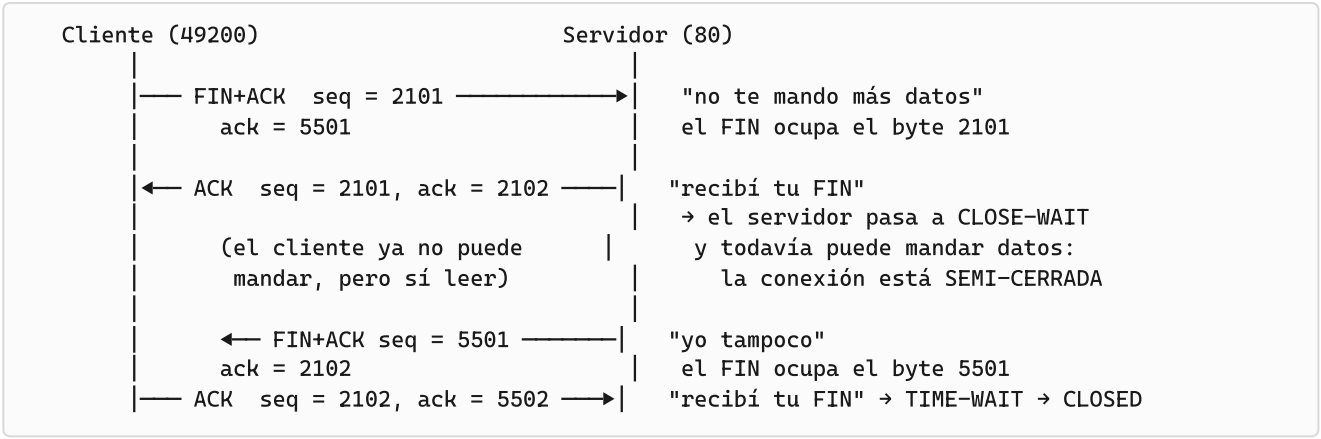
Los costos: se **pierde el Window Scale** (la ventana efectiva cae ÷ 128), **SACK no se puede usar** hasta que el servidor recibe un bloque SACK (lo que limita mucho en flujos unidireccionales), y no se pueden mandar **datos piggyback en el SYN**. Por eso en Linux y Windows están **deshabilitadas por defecto** y se activan **solo bajo ataque**.

> **SCTP resolvió el problema mejor**: handshake de **4 vías** con *cookies* stateless, así que el extremo pasivo tiene **evidencia** de que el iniciador existe antes de reservar nada. La `RFC 4987` lo señala como la lección aprendida.

1.5 El cierre de 4 vías, con los números

TCP es full-duplex, así que cerrar una conexión son **dos cierres independientes**: el de la dirección cliente→servidor y el de la dirección servidor→cliente. **Cada uno necesita 2 mensajes**: un FIN y su ACK.

Retomando los números de §1.2: el cliente mandó 100 bytes de datos (`2001..2100`) y el servidor 500 bytes (`5001..5500`).



La cuenta, que es lo que se corrige:

#	Seg.	Flags	seq	ack	Por qué
1	C→S	FIN+ACK	**2101**	5501	el cliente confirmó los 500 bytes del servidor
2	S→C	ACK	2101	**2102**	el FIN del cliente consumió el 2101
3	S→C	FIN+ACK	**5501**	2102	el servidor confirmó los 100 del cliente
4	C→S	ACK	2102	**5502**	el FIN del servidor consumió el 5501

Por qué 4 y no 3. Cuando el servidor recibe el FIN del cliente **todavía puede tener datos en vuelo** hacia el cliente, así que primero acusa (mensaje 2) y después cierra (mensaje 3). Pero si su FIN viaja **piggyback en el ACK** —porque ya estaba escribiendo— solo hay **3 mensajes**: `FIN`, `FIN+ACK`, `ACK`. Es la variante que muestra la solución oficial del 2006-12.

El semi-cierre (*half-close*): entre el FIN de uno y el del otro, el cliente **ya no puede enviar pero sí recibe**. Es el caso de HTTP/1.x: el cliente manda el request, hace `shutdown(SHUT_WR)` y sigue leyendo la respuesta. Por eso **CLOSE-WAIT** puede durar minutos: no es un error, es diseño.

1.6 RST: el cierre abrupto

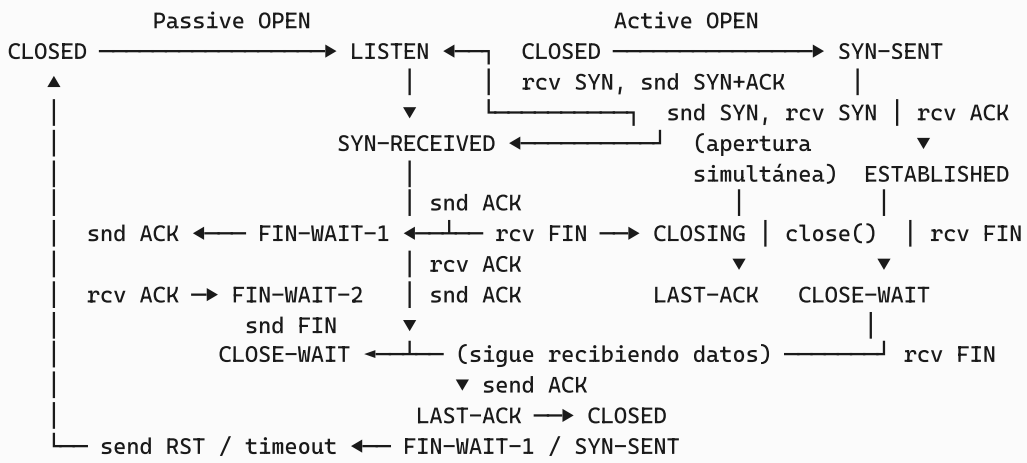
El **RST** (*reset*) es la forma de abortar sin ceremonia. **Cuándo se manda:**

1. **Puerto cerrado**: un SYN a un puerto sin proceso escuchando → el kernel responde `RST, seq 0, ack = seq del SYN`, y el cliente ve **"Connection refused"** (`ECONNREFUSED`).
2. **Datos o FIN sobre una conexión que ya no existe** (o en CLOSED) → `RST` y se descarta todo.
3. **Abortar con datos pendientes**: `close()` con `SO_LINGER` en 0 manda `RST` en vez del cierre ordenado, y **descarta lo no confirmado**.

```
SYN a puerto cerrado:
cliente ──SYN──► servidor :  RST (seq 0, ack = seq del SYN)  → ECONNREFUSED
```

RST no es un cierre limpio: descarta el estado y **pierde los datos no confirmados**. Es lo que pasa cuando un proceso mata un servidor o cuando el SO decide que la conexión lleva demasiado tiempo inactiva (el *keepalive* de la clase 19).

1.7 La máquina de estados: 11 estados



Y el estado **TIME-WAIT**, que falta en el esquema anterior porque se dibuja aparte:

```
LAST-ACK ──send ACK──▶ TIME-WAIT ──(2 × MSL)──▶ CLOSED
```

Los 11 estados (los mismos de la `RFC 9293`): `CLOSED`, `LISTEN`, `SYN-SENT`, `SYN-RECEIVED`, `ESTABLISHED`, `FIN-WAIT-1`, `FIN-WAIT-2`, `CLOSE-WAIT`, `CLOSING`, `LAST-ACK`, `TIME-WAIT`.

Los dos estados que se explican menos y siempre se preguntan: CLOSING existe **solo** en el **cierre simultáneo**: los dos mandan FIN antes de recibir el del otro, y cada uno queda ahí hasta que llegue el ACK de su FIN. **TIME-WAIT** es lo que sigue.

1.8 TIME-WAIT: 2 × MSL

MSL es el *Maximum Segment Lifetime*, el tiempo máximo que un segmento puede estar dando vueltas en la red. La `RFC 9293` dice textual: "*For this specification the MSL is taken to be 2 minutes*", y obliga (MUST-13): "*When a connection is closed actively, it MUST linger in the TIME-WAIT state for a time 2xMSL*".

```
MSL = 120 s   →   TIME-WAIT = 2 × 120 = 240 s = 4 minutos
```

El problema que resuelve. Cuando la 4-tupla se reutiliza para una conexión nueva (y se reutiliza constantemente: muchísimos clientes al mismo servidor 80), pueden quedar **segmentos viejos** de la conexión anterior dando vueltas por la red. Si el nuevo SYN del cliente empieza con un número de secuencia que **ya pasó por la red**, el servidor puede confundirlo con datos de la conexión vieja.

```
cliente:   SYN ──▶ FIN ──▶ TIME-WAIT (240 s) ──▶ CLOSED
red:       ... un duplicado del SYN atrasado 200 s .....................
servidor:                          ◀── llega tarde: lo aceptaría como dato nuevo ⚠
```

La solución: el lado que cierra primero (TIME-WAIT) **no vuelve a usar esa 4-tupla durante 2 × MSL**. Si durante ese tiempo llega un segmento con números de la conexión vieja, **no coincide** con la conexión nueva y se descarta. Pasados los 2 × MSL ya no puede quedar ningún segmento viejo en la red.

Los dos MSL sirven para cosas distintas: el primero cubre el caso de que el ACK final se haya perdido y el otro extremo reenvíe su FIN (hay que estar ahí para responder); el segundo, los datos rezagados.

Un detalle fino de la `RFC 9293` (MAY-2): en TIME-WAIT se puede **aceptar un SYN nuevo** y reabrir la conexión desde ese estado **si** el ISN asignado es **mayor** que todos los usados en la conexión anterior, y **si** el SYN resulta un duplicado viejo se vuelve a TIME-WAIT. Y con **Timestamps** hay un algoritmo mejor para **reducir** el TIME-WAIT (*Best Current Practice*): los timestamps no se repiten, así que distinguir conexiones viejas sin esperar 4 minutos.

1.9 ⚠️ Actualización a 2026

Sigue igual — de memoria: el **three-way handshake** y sus tres TPDUs (SYN, SYN+ACK, ACK); **el SYN consume un número**; **FIN consume un número**; el **cierre de 4 vías** (y su variante de 3 con FIN piggyback); el **semi-cierre**; el **TIME-WAIT de 2 × MSL**; el **RST**; y **los 11 estados**, que en 2026 son exactamente los mismos.

Cambió o hay que corregirlo:

- `RFC 9293` **(2022, *Internet Standard*) sustituye a la** `RFC 793` y es la referencia de la máquina de estados y del TIME-WAIT. `RFC 1122` sigue definiendo los hosts, y `RFC 7414` (2015) es el **índice de todas las RFCs de TCP**.
- **SYN cookies:** `RFC 4987` (2007, *Informational*). Sigue siendo el mecanismo vigente y lo usan Linux, Windows y BSD. ⚠️ Ojo con el matiz: es un documento **informativo**, o sea **no impone requisitos**, pero es *la* referencia del tema.
- **El MSL es "una decisión de ingeniería"**: la `RFC 9293` toma 2 minutos y dice explícitamente que puede cambiarse con la experiencia. Si te preguntan el número exacto de 4 minutos, es el de RFC 9293.
- **TCP Fast Open** manda datos **dentro del SYN** y ahorra 1 RTT. ⚠️ **El número de RFC no está en `ACTUALIZACIONES.md`: verificá antes de citarlo** (creo que es 7413, pero confirmá).
- **QUIC elimina el three-way handshake de TCP.** El handshake de QUIC es de **1 RTT** (y **0 RTT** con reanudación), va en UDP, y **la conexión se identifica por el Connection ID** (`RFC 9000` §5.1), no por la 4-tupla: eso habilita la **migración de conexión** si cambia la IP o el puerto.
- **Aun así, el modelo TCP de 4-tupla es lo que se enseña y lo que se pregunta.** QUIC no reemplazó a TCP: HTTP/3 es el 21 % del tráfico y HTTP/1.1 sigue siendo el 29 % (Cloudflare Radar, 2025). TCP mueve la enorme mayoría del tráfico de la web, del correo, de SSH y de los juegos.

Consecuencia para el examen: el diagrama del handshake, los números, el cierre de 4 vías y el TIME-WAIT **se preguntan exactamente igual que en 2006**. Si citás `RFC 9293` en lugar de RFC 793, sumás.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La secuencia exacta de cada segmento

Enunciado. El cliente (puerto 50000) elige ISN = **1000** y el servidor (puerto 80) ISN = **7000**. La conexión abre, el cliente manda **300 bytes** de datos, el servidor le responde con **1000 bytes**, y después **cada uno cierra su lado**. Escribí **cada segmento** con sus flags, su `seq` y su `ack`.

Resolución, segmento por segmento.

#	Seg.	Flags	Datos	seq	ack	De dónde sale el número
1	C→S	SYN	0	1000	—	ISN del cliente
2	S→C	SYN+ACK	0	7000	**1001**	1000 + 1 (el SYN del cliente)
3	C→S	ACK	0	1001	**7001**	1000 + 1 (el SYN del servidor)
4	C→S	PSH+ACK	**300**	1001	7001	los datos arrancan en 1001
5	S→C	ACK	0	1001	**1301**	1001 + 300 = 1301
6	S→C	PSH+ACK	**1000**	7001	1301	los datos arrancan en 7001
7	C→S	ACK	0	1301	**8001**	7001 + 1000 = 8001
8	C→S	FIN+ACK	0	**1301**	8001	el cliente ya no manda nada más
9	S→C	ACK	0	1301	**1302**	1301 + 1 (el FIN del cliente)
10	S→C	FIN+ACK	0	**8001**	1302	el servidor ya no manda nada más
11	C→S	ACK	0	1302	**8002**	8001 + 1 (el FIN del servidor)
12	C→S	FIN+ACK	0	1302	8002	el cliente cierra **su** lado
13	S→C	ACK	0	1303	8002	1302 + 1
14	S→C	FIN+ACK	0	8002	1303	el servidor cierra **su** lado
15	C→S	ACK	0	1303	**8003**	8002 + 1 → **el cliente entra en TIME-WAIT**

Las tres cuentas que hay que escribir en el examen:

```
Bytes del cliente:  1001 .. 1300   →  próximo esperado: 1301
Bytes del servidor: 7001 .. 8000   →  próximo esperado: 8001
FIN del cliente en 1301 → siguiente 1302 ;  FIN del servidor en 8001 → siguiente 8002
```

Interpretación: el `seq` de cada extremo **solo avanza** cuando ese extremo manda bytes o un SYN/FIN. Un error clásico es poner `ack = 1300` en vez de `1301`: el ACK es **el próximo esperado**.

⚠️ **Errores típicos:** (a) olvidar que **SYN y FIN consumen un número**; (b) poner el `ack` del último byte recibido en vez del próximo; (c) hacer el cierre en **3** segmentos cuando los dos lados mandaron datos en direcciones opuestas (solo van 3 si el segundo FIN va piggyback en el ACK).

2.2 Ejercicio 2 — ¿Qué pasa si se pierde el tercer segmento (el ACK del cliente)?

Enunciado. El SYN del cliente llega, el servidor responde SYN+ACK, y **el ACK del cliente se pierde**. Describí qué estado queda cada uno, qué se retransmite y cuánto cuesta.

Resolución paso a paso.

```
t=0    C ──SYN seq=2000──► S        C: SYN-SENT          S: LISTEN
t=RTT  C ◄──SYN+ACK seq=5000, ack=2001── S
                              ✗ (este mensaje se pierde)
       C: ESTABLISHED                  S: SYN-RECEIVED  ← tiene un TCBReservation

t=RTO  S ──SYN+ACK seq=5000, ack=2001──► C   (el servidor reintenta)
       C ya está ESTABLISHED y recibe un SYN+ACK duplicado:
       reconoce que es de su propia conexión → responde con el ACK otra vez
t=2RTT C ──ACK seq=2001, ack=5001──► S
                              ✓     S: ESTABLISHED
```

Consecuencias, en orden de importancia:

1. **El cliente queda ESTABLISHED y el servidor en SYN-RECEIVED.** El servidor tiene **un TCB reservado que todavía no sabe si es real** (exactamente el agujero que explotan los SYN floods).
2. **El servidor reintenta el SYN+ACK** con su temporizador. Como el cliente ya está establecido, el kernel del cliente ve un **SYN+ACK duplicado de su propia conexión** y **reenvía el ACK** en vez de tratar el SYN como una conexión nueva. **El handshake se completa solo en el segundo intento.**
3. **Costo: 1 RTO extra**, no pérdida de datos. Si el cliente ya mandó datos (lo normal, porque el tercer segmento puede llevarlos), llegan al servidor mientras está en SYN-RECEIVED: **no los descarta**, los reordena en el buffer y los entrega cuando se complete el handshake; si no los ve confirmados, los retransmite.
4. **No es un error del protocolo**, es el comportamiento normal ante pérdida. Por eso el temporizador del SYN+ACK (RTO inicial) es generoso: si se perdiera dos veces, el servidor **cierra la conexión medio abierta** (y con **SYN cookies** ni siquiera hay TCB que cerrar).

⚠️ **Error típico:** decir "el cliente se queda esperando el ACK del servidor". No: el cliente ya está establecido porque **él fue el que mandó el último mensaje del handshake**.

2.3 Ejercicio 3 — ¿Cuánto RTT cuesta establecer una conexión?

Enunciado. Calculá el RTT total desde que el cliente decide abrir la conexión hasta que puede mandar datos de aplicación, en cuatro escenarios: (a) TCP solo; (b) TCP + **TLS 1.3**; (c) TLS 1.3 con **0-RTT** (reanudación); (d) **TCP Fast Open** + TLS 1.3. Suponé RTT de 40 ms.

Escenario	Vuelos de ida	RTT	Total	¿Datos en el 1.er vuelo?
(a) TCP solo	SYN · SYN+ACK+ACK	**1**	40 ms	sí (van en el 3.er segmento)
(b) TCP + TLS 1.3	SYN · SYN+ACK · ClientHello+ACK · ServerHello… · fin	**1**	**40 ms**	no
(c) TLS 1.3 0-RTT	como (b) pero con *early data*	**1**	40 ms	**sí** (datos antes del handshake)
(d) TCP Fast Open + TLS 1.3	SYN+datos · SYN+ACK+datos	**0**	**0 ms**	sí

La aritmética y el razonamiento de cada fila:

- **(a) TCP solo = 1 RTT.** El cliente abre el flujo de un lado y recibe la respuesta: un RTT. Los datos de aplicación viajan **en el tercer segmento** (el ACK), así que no cuesta nada más.
- **(b) TLS 1.3 = 1 RTT total, no 2.** El trick es que el **ClientHello viaja en el tercer segmento del handshake TCP**, que ya estaba en vuelo. O sea: la `RFC 8446` promete "1-RTT" **contado desde el ClientHello**, y como ese ClientHello piggyback del ACK de TCP, **desde el SYN son 1 RTT**. Por eso HTTPS con TLS 1.3 se ve tan rápido.
- **(c) 0-RTT** requiere **reanudar una sesión** (con PSK) y un **token anti-replay** (`RFC 8470`), sin el cual el servidor no puede aceptarlo: si lo acepta a ciegas, un atacante **repite** una petición vieja. El payload va en el primer vuelo; el costo es que la aplicación tiene que aceptar el replay.
- **(d) TCP Fast Open = 0 RTT** para el que ya tiene *cookie*: manda los datos **dentro del SYN**. ⚠️ **El número de RFC no está en `ACTUALIZACIONES.md`: verificá** (creo que es 7413). Sin cookie, el servidor la pide en el SYN+ACK y el cliente reenvía los datos: **1 RTT y medio**.

Conclusión para el examen: el detalle que más se pregunta es que **TLS 1.3 no agrega un RTT** al handshake TCP porque **el ClientHello piggyback del ACK de TCP**, y que **0-RTT existe pero obliga a renunciar a la protección contra repetición**.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2006-12, Pregunta 4 (10 puntos) — Three-way handshake y cierre

> **Fuente:** Examen `eirc0612` / `sirc0612` — 26 de diciembre de 2006.

Enunciado (texto del examen)

> Describa el proceso por el cual se establece una conexión TCP (three-way handshake), y cómo se cierra la misma. ¿Qué TPDU se intercambian en los mismos?

Solución oficial (verbatim, resumida)

> El establecimiento se realiza con un three-way handshake: un SYN; si el puerto está disponible, un SYN-ACK, y finalmente un ACK, y de no estar se envía un RST. También puede realizarse por intercambio de solicitudes de conexión (b), aunque no es común. El número inicial de secuencia se acuerda en el momento de la conexión. No es 0. Desconexión: TCP supone la existencia de dos conexiones simplex que se desconectan independientemente. El envío de un paquete con flag FIN implica que yo no deseo enviar más datos al otro extremo. En general se necesitan 4 segmentos para cerrar una conexión, aunque pueden ser 3: FIN / ACK / FIN / ACK y FIN / FIN+ACK / ACK. Para evitar el problema de los dos ejércitos se colocan timers que indican que si ocurren 2 time_outs sin recibir el ACK del FIN se da por finalizada la conexión.

Análisis de la solución oficial

Lo que está bien: los tres TPDUs del handshake y el **RST** cuando el puerto no está disponible; la idea de **dos conexiones simplex**; el **cierre de 4 o 3 segmentos**; y la apertura simultánea.

Tres cosas que hay que corregir o precisar (y que suman puntos si las señalás):

1. **"El número inicial de secuencia se acuerda en el momento de la conexión."** Impreciso: los dos ISN **no se acuerdan, se intercambian**: cada extremo elige el suyo (aleatorio y con clave secreta, para que nadie lo prediga) y el handshake sirve para que cada uno **se entere del del otro**.
2. **"Para evitar el problema de los dos ejércitos se colocan timers … si ocurren 2 time_outs."** Eso **no resuelve** el problema de los dos ejércitos: **abandonar la conexión al segundo timeout resuelve los ENEMIGOS, no los aliados**. La solución canónica es el **TIME-WAIT de 2 × MSL**, que impide que los dos ejércitos se crucen.
3. **"El del otro extremo notará que no hay nadie escuchando y cortará."** Eso es lo que pasa con un **RST** (el puerto se cerró). En el cierre ordenado con FIN, el otro extremo **sigue escuchando** hasta mandar su propio FIN.

Mi resolución, completa

Establecimiento: el cliente manda **SYN** con su ISN en `seq` (sin `ack`); el servidor, si el puerto está libre, responde **SYN+ACK** con su ISN en `seq` y `ack = ISN del cliente + 1`; el cliente manda **ACK** con `seq = su ISN + 1` y `ack = ISN del servidor + 1`, y **ese tercer segmento puede llevar datos**. Si el puerto no está libre, el servidor responde **RST**.

Cierre (4 vías): TCP es **full-duplex**, así que hay **dos conexiones simplex** que se cierran por separado: el cliente manda **FIN+ACK**, el servidor responde **ACK** (acá el cliente ya está en **semi-cierre**), el servidor manda su **FIN+ACK** y el cliente cierra con un **ACK** final, quedando en **TIME-WAIT 2 × MSL**. Si el servidor tenía datos pendientes, su FIN viaja **piggyback** en el ACK y son **3 segmentos**.

TPDUs intercambiados: `SYN`, `SYN+ACK`, `ACK`, `FIN`, `ACK`, `FIN`, `ACK` (y `RST` en cualquier punto para abortar).

> **Tope para 10 puntos:** los tres TPDUs del handshake + el RST, el cierre de 4 vías con el **semi-cierre**, el **TIME-WAIT de 2 × MSL** justificado con el problema de los dos ejércitos, y los TPDUs del cierre.

3.2 Examen 2009-07, Pregunta 2a (8 puntos) — Los pasos del handshake

> **Fuente:** Examen `eirc0907` / `sirc0907` — 29 de julio de 2009.

Enunciado (texto del examen)

> a) Describa los pasos del "Three Way Handshake" para el inicio de una conexión TCP.

Solución oficial (verbatim)

> Paso 1: el host cliente envía un segmento SYN al servidor - especifica el nro. de sec. inicial - sin datos. Paso 2: el host servidor recibe el SYN, responde con un segmento SYN-ACK sin datos. El servidor reserva buffers, especifica el nro. de sec. inicial del servidor. Paso 3: el cliente recibe el SYN-ACK, responde con un segmento ACK, que puede tener datos.

Mi resolución, ampliada

La solución oficial es correcta y completa. Lo que hay que agregarle para los 8 puntos es **por qué tres y no dos** y **qué se negocia**:

Paso 1 — cliente → servidor: SYN. El cliente elige su ISN, lo pone en `seq`, manda `ack = 0` (con ACK en 0 el campo Acknowledgment **se ignora**) y **no lleva datos**. Aprovecha para anunciar en las **opciones** su MSS, SACK, Window Scale y Timestamps.

Paso 2 — servidor → cliente: SYN+ACK. El servidor **reserva TCB y buffers**, elige **su propio ISN** y responde con `seq = ISN servidor` y `ack = ISN cliente + 1`, más **sus opciones**. **Aquí todavía no hay conexión**: está en SYN-RECEIVED y no sabe si el cliente existe de verdad.

Paso 3 — cliente → servidor: ACK. `seq = ISN cliente + 1`, `ack = ISN servidor + 1`, y **puede llevar datos** (es el request HTTP). Con esto los dos saben los números del otro y **la conexión queda establecida**: el cliente pasa a ESTABLISHED y el servidor también.

Y el "cuarto paso" que no está en la solución oficial: si el servidor no tiene ese puerto abierto, responde **RST** y el `connect()` del cliente falla con `ECONNREFUSED` al instante. Esa es la razón por la que el SYN flood duele tanto: **el SYN reserva estado antes de tener ninguna prueba**.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. El handshake sirve para que **cada extremo se entere del ISN y de las opciones del otro**: los ISN **se intercambian, no se acuerdan**, y son aleatorios con clave secreta.
2. **SYN y FIN consumen un número de secuencia**; por eso los ACKs son `ISN + 1`.
3. El **tercer segmento** (ACK) **puede llevar datos**, y por eso establecer una conexión TCP cuesta **1 RTT**.
4. **3 vías y no 2** porque el servidor necesita **evidencia de que el cliente existe** antes de reservar un TCB.
5. **SYN flood** = SYNs con IP falsificada que llenan el backlog; las **SYN cookies** (`RFC 4987`) lo evitan **codificando el estado en el ISN** y creándolo recién cuando el ACK valida la cookie.
6. El **cierre de 4 vías** cierra **dos conexiones simplex** por separado: FIN, ACK, FIN, ACK; y son **3 segmentos** si el segundo FIN va piggyback en el ACK.
7. Entre el FIN de uno y el del otro hay **semi-cierre**: se puede seguir leyendo.
8. El **RST** es el cierre abrupto: puerto cerrado, datos sobre conexión inexistente, o `close()` con linger 0. Descarta lo no confirmado.
9. La máquina de estados tiene **11 estados**, y **TIME-WAIT = 2 × MSL = 240 s** (MSL = 2 min en `RFC 9293`) impide que un segmento viejo se confunda con uno de la conexión nueva.
10. **QUIC elimina el handshake de 3 vías** y usa el **Connection ID**, pero **TCP con 4-tupla sigue siendo lo que se enseña y lo que se pregunta**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. En el three-way handshake, con ISN del cliente 2000 e ISN del servidor 5000, ¿cuáles son los `seq` y `ack` de los tres segmentos?

▶ Respuesta

2. ¿Por qué el handshake tiene 3 mensajes y no 2?

▶ Respuesta

3. ¿Qué es una SYN cookie y por qué funciona?

▶ Respuesta

4. En el cierre de 4 vías, ¿cuántos números consume cada FIN y por qué?

▶ Respuesta

5. ¿Por qué existe el estado TIME-WAIT de 2 × MSL?

▶ Respuesta

6. ¿Cuánto cuesta establecer una conexión TCP con TLS 1.3, y por qué no son 2 RTT?

▶ Respuesta

7. ¿Qué es el semi-cierre y en qué caso típico aparece?

▶ Respuesta

8. ¿Cuándo se manda un RST y qué efecto tiene?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc0612` **/** `sirc0612` — P4 (10 pts, three-way handshake, cierre y TPDUs)
- **Examen** `eirc0907` **/** `sirc0907` — P2a (8 pts, los pasos del handshake)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 — §6.3 (*Segmentos TCP*)
- `RFC 9293` — *TCP* (2022, *Internet Standard*): máquina de estados, TIME-WAIT (MUST-13, MAY-2), MSL, ISN
- `RFC 4987` — *TCP SYN Flooding Attacks and Common Mitigations* (2007, *Informational*): SYN flood, SYN cache, SYN cookies, SCTP
- `RFC 1122` — requisitos de los hosts · `RFC 7414` — índice de las RFCs de TCP
- **TCP Fast Open** — datos en el SYN: ⚠️ **el número de RFC no figura en** `ACTUALIZACIONES.md`**: verificá**
- `RFC 8446` → `RFC 9846` — TLS 1.3 (1-RTT y 0-RTT) · `RFC 8470` — tokens anti-replay
- `RFC 9000` **/** `RFC 9001` **/** `RFC 9114` — QUIC, TLS 1.3 sobre QUIC y HTTP/3: handshake de 1 RTT, Connection ID y migración
- **Clase siguiente:** 19 — TCP: cómo se garantiza la fiabilidad

Clase 19

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 El contrato de TCP, palabra por palabra
- 1.2 Números de secuencia: por qué hacen falta
- 1.3 Timers: la pregunta de 2010-12 (P4b), resuelta
- 1.4 El RTO y cómo se estima (Jacobson/Karels)
- 1.5 La ventana deslizante
- 1.6 Por qué la ventana tiene que ser ≥ BDP
- 1.7 Recuperación rápida: fast retransmit y SACK
- 1.8 Por qué el flag PSH no garantiza nada (examen 2001-12, Problema 2)
- 2.1 Ejercicio 1 — RTO con Jacobson/Karels a partir de 4 muestras
- 2.2 Ejercicio 2 — BDP y tamaño mínimo de ventana
- 2.3 Ejercicio 3 — Comparar cuatro ventanas sobre el mismo enlace
- 3.1 Examen 2010-12, Pregunta 4 (8 puntos) — números de secuencia y timers
- 3.2 Examen 2009-12, Pregunta 2 (10 puntos) — fiabilidad en transporte vs. en enlace
- 3.3 Examen 2001-12, Problema 2 — el flag PUSH
- 📌 Resumen en 10 líneas
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: C (Transporte) · **Semana:** 7 · **Duración estimada:** 110 min **Correlativa:** Clase 18 (TCP: establecer y cerrar la conexión) · **Sigue con:** Clase 20 (control de flujo vs. control de congestión) **Material de referencia:** Tanenbaum cap. 6 (§6.5 "Reliable Transmission") · RFC 9293, RFC 2018, RFC 7323, RFC 5681, RFC 6928

🎯 Al terminar esta clase vas a poder

1. Traducir cada palabra del contrato de TCP (*entrega ordenada, sin duplicados, sin errores*) al mecanismo que la implementa.
2. Explicar por qué los números de secuencia son **imprescindibles** aunque el checksum ya detecte errores, y por qué el ACK es **acumulativo y numera bytes**.
3. Calcular un **RTO** con Jacobson/Karels a partir de una serie de muestras de RTT, y aplicar el **backoff exponencial**.
4. Calcular el **BDP** de un enlace y deducir el **tamaño mínimo de ventana** para no desperdiciar capacidad.
5. Dibujar la **ventana deslizante** con `LastByteAcked`, `LastByteSent` y los bytes en vuelo.
6. Describir cómo **SACK**, **timestamps** y **PAWS** cambian la recuperación de una pérdida, y por qué el flag **PSH no garantiza** un mensaje por `read()` en el destino.

> **Prerequisito del examen:** `2011-03 P4` (10 pts — el diagrama de 4 segmentos), `2010-12 P4` (8 pts — números de secuencia y timers), `2009-12 P2` (10 pts — fiabilidad en transporte vs. enlace), `2001-12 Problema 2` (PUSH y segmentación).

PARTE 1 — TEORÍA

1.1 El contrato de TCP, palabra por palabra

Lo que promete	Mecanismo que lo cumple
Ordenada	**Números de secuencia** + buffer de reordenamiento en el receptor
Sin duplicados	el receptor **descarta** lo que ya tiene: `rcv_nxt` es el puntero que avanza
Sin errores	**checksum** (que detecta) + **retransmisión** (que corrige)

La distinción "detectar" vs. "corregir" se pregunta siempre: el checksum de TCP **no corrige nada**, solo descarta el segmento. Lo que repara es la **retransmisión**, que necesita saber *qué* falta. **Orden → duplicados → errores** es una cadena de dependencias: no podés evitar duplicados si no sabés qué tenés, y no podés retransmitir si no numerás.

1.2 Números de secuencia: por qué hacen falta

La respuesta oficial del examen 2010-12 (P4a) es cortita y hay que saberla de memoria:

> TCP es **pipelined**: el emisor envía varios paquetes antes de recibir confirmación de alguno de ellos. Por esta razón, el emisor debe poder distinguir **a qué paquete o paquetes corresponde una determinada confirmación**, por lo que numera los paquetes enviados secuencialmente.

Hay un segundo motivo, más profundo: **el checksum no te dice *qué* falta**. Te dice "este paquete vino mal". Para retransmitir el correcto necesitás un identificador que sobreviva al viaje, y sin números tampoco podés evitar duplicados, porque no sabés qué tenés ya.

TCP numera BYTES, no segmentos. El `seq` de un segmento es el número de su **primer byte**, y el `ack` es el **próximo byte esperado**, no el último recibido:

```
┌──────────────────────────────────────────────────────────────┐
│ ack = 1120 significa: "recibí hasta 1119 inclusive",         │
│                       "el próximo byte que espero es el 1120".  │
│ NO significa "recibí hasta 1120".                            │
└──────────────────────────────────────────────────────────────┘


          ┌─────────┬─────────┐
          │ byte    │ byte    │
          │ 1119    │ 1120    │
          └─────────┴─────────┘
          ▲         ▲
          │         └── ack = 1120 (PRÓXIMO esperado)
          └── último byte recibido
```

Por eso los ACKs son **acumulativos** (*cumulative ACK*): un solo ACK puede confirmar varios segmentos. Si B recibe S1..S4 seguidos, un `ack=1180` confirma los cuatro.

1.3 Timers: la pregunta de 2010-12 (P4b), resuelta

La situación. A mandó un segmento. Pasó un rato. No llegó el ACK. ¿Por qué?

```
┌──────────────────────────────────────────────────────────────┐
│  ¿Por qué no llegó el ACK?                                   │
│   (1) Se perdió el SEGMENTO de datos                         │
│   (2) Se perdió el ACK                                       │
│   (3) El ACK no se perdió: simplemente TARDÓ                 │
└──────────────────────────────────────────────────────────────┘
                  │
                  ▼
¿Qué puede observar A?  En los tres casos, EXACTAMENTE LO MISMO: "no sé".
```

Las tres tienen **la misma consecuencia observable**: la ausencia del ACK. No puede retransmitir "por las dudas" al instante (sería un duplicado puro) ni esperar indefinidamente (si el segmento se perdió de verdad, la conexión se cuelga). **La única salida es retransmitir**, y para eso hay que decidir **cuándo**: esperar un "tiempo prudencial". Eso es un **timer**.

El enunciado oficial dice textualmente *"la única solución es retransmitir el paquete no confirmado… necesita un timer que le indique cuándo hacerlo"* (la cita completa y el desglose de los 4 puntos están en §3.1).

Un detalle que suma puntos: el timer se arma **una vez por segmento enviado**, no una vez por conexión. Si A manda S1..S4 seguidos hay **cuatro timers corriendo**: es la misma razón por la que hacen falta los números de secuencia.

1.4 El RTO y cómo se estima (Jacobson/Karels)

El **RTO** (*Retransmission Timeout*) es cuánto espera TCP antes de declarar perdido un segmento. Si fuera fijo: demasiado corto → retransmitís de más y generás **duplicados**; demasiado largo → cada pérdida cuesta un RTO entero de latencia. La solución: **medir**.

```
SRTT   = suavizado del RTT
RTTVAR = suavizado de la variación del RTT
RTO    = SRTT + 4 · RTTVAR

Con cada nueva muestra R:
  RTTVAR ← (1 − β) · RTTVAR + β · |SRTT − R|      ← con el SRTT ANTERIOR
  SRTT   ← (1 − α) · SRTT   + α · R
  RTO    ← SRTT + 4 · RTTVAR

α = 7/8   (recuerdo alto: el pasado pesa más que el presente)
β = 1/4   (reacción rápida a los cambios)

Primera muestra:  SRTT = R ;  RTTVAR = R/2 ;  RTO = R + 4·(R/2) = 3R
```

¿Por qué `SRTT + 4·RTTVAR` y no un factor fijo? Porque la variación es la **incertidumbre** de la medida. Si el RTT está en 100 ms con variación de ±20 ms, un RTO de `100 + 4·20 = 180 ms` deja pasar casi cualquier medición posible. Además hay **un piso y un techo**: el RTO no puede ser menor que un mínimo histórico (1 s) ni mayor que un máximo (60 s).

Un detalle fino: no se toman muestras de RTT de los segmentos retransmitidos (algoritmo de **Karn**): si retransmitiste S, no sabés si el ACK que llegó corresponde al S original o a la retransmisión, y si es a la

retransmisión el RTT medido incluye el RTO entero y **infla** la estimación. Los **timestamps** lo resuelven: el ACK hace eco del `TSval` del segmento.

Backoff exponencial

Cuando un timeout se dispara, el RTO **se duplica**: con `RTO = 240 ms`, los timeouts sucesivos dan 480 ms, 960 ms, 1920 ms. **¿Por qué?** Un timeout **no** es evidencia de congestión: puede ser un blackout o un enlace caído. Si además de subir el RTO bajás la ventana, lo que decís es "esto parece una avería, esperemos más". Ojo: **las dos cosas pasan a la vez** y no se contradicen — el `cwnd` protege a la red, el backoff protege al emisor de insistir contra algo que no responde.

1.5 La ventana deslizante

En *stop-and-wait* el emisor manda un segmento y **espera**: el enlace queda ocioso la mitad del tiempo. TCP no: manda hasta `ventana` bytes sin esperar confirmación.

```
El emisor A, con ventana = 8 bytes (4 segmentos de 2 bytes):

LastByteAcked           LastByteSent         la ventana
      |                       |                   |
------|-----------------------|-------------------|----
   100                     108                 110
bytes ya confirmados     último byte       no se puede
                         enviado          pasar de acá

bytes en vuelo = LastByteSent - LastByteAcked = 108 - 100 = 8
(la ventana es justamente 8: la ventana está LLENA)
```

`LastByteAcked` = último byte confirmado · `LastByteSent` = último byte enviado · `LastByteAcked + ventana` = el tope que el emisor no puede pasar.

Además el emisor no puede mandar más allá de lo que el receptor dice que entra (el `rwnd`), y por eso `ventana efectiva = min(cwnd, rwnd)`. → El mínimo y el `cwnd` están desarrollados en las Clases 20 y 21; lo que hay que internalizar acá es el **sentido físico**: la ventana es "cuánta información puedo dejar en el camino antes de tener que parar a esperar".

1.6 Por qué la ventana tiene que ser ≥ BDP

BDP = *Bandwidth-Delay Product*: los bytes que están "en vuelo" en un momento dado si el emisor llena el enlace sin pausa.

```
BDP = ancho_de_banda × RTT          [bytes] = [bits/s] × [s] / 8
```

El argumento, en una frase: si la ventana es menor que el BDP, **el enlace no está lleno**, y estás pagando un enlace de 100 Mbps para usar 4 Mbps. Con 100 Mbps y RTT 80 ms:

```
BDP = 100 · 10⁶ bits/s × 0,080 s = 8 · 10⁶ bits = 1 000 000 bytes ≈ 1 MB

ventana 64 KB:  throughput = 65 536 / 0,080 = 819 200 B/s = 6,55 Mbit/s   ← 15× menos
ventana  1 MB:  throughput = 1 048 576 / 0,080 = 13 107 200 B/s = 104,9 Mbit/s ← lleno

En MSS (MTU 1500 − 20 IP − 20 TCP = 1460 B):  1 000 000 / 1460 ≈ 685 segmentos
```

Ésta es la razón por la que el `rwnd` importa tanto y por la que existe el *window scale*: el campo `window` de la cabecera son **16 bits** (máximo 65 535 bytes), y con RTT de 80 ms eso limita el throughput a 6,55 Mbit/s. Con *Window Scale* (`RFC 7323` , factor negociado en el SYN) se llega a megabits.

1.7 Recuperación rápida: fast retransmit y SACK

Fast retransmit está desarrollado en la Clase 21 (§1.4c): 3 ACKs duplicados → retransmitir sin esperar el RTO. La idea: **los ACKs duplicados son la señal de que los datos posteriores llegaron**. Si hubiera un problema grave de red no habrían llegado.

SACK (`RFC 2018`)

El problema sin SACK. El ACK acumulativo es todo o nada:

```
A manda S1(1000), S2(1046), S3(1092), S4(1138). Se pierde S2.
B recibe S1, S3, S4.
B → A:  ACK=1046  ACK=1046  ACK=1046      (3 duplicados)
A sólo sabe: "me falta algo a partir de 1046".
A NO sabe si es S2 solo, o S2+S3, o S2+S3+S4.
→ TCP clásico (Tahoe): retransmite S2, espera, y recién con el ACK
  siguiente descubre que S3 también faltaba. UNO POR UNO.
```

Con SACK, el receptor manda bloques extra que dicen *qué* bloques exactos recibió. Cada bloque es un par `(borde izquierdo, borde derecho)` en números de secuencia absolutos, y van en el mismo segmento que el ACK acumulativo:

```
B → A:  ACK=1046  SACK=[1092..1138]      (recibí S3)
B → A:  ACK=1046  SACK=[1092..1184]      (recibí S3 y S4)
```

A ahora sabe que S3 y S4 están bien, así que **retransmite solo S2**, y cuando S2 llega todo se confirma de un golpe. **SACK no cambia la corrección: cambia cuánto tarda.**

⚠️ **Detalle que se pregunta:** SACK se **negocia en el SYN** (opción *SACK permitted*). Si no se negocia, no hay bloques SACK. Y como SACK solo informa de bytes **dentro de la ventana**, no reemplaza al ACK acumulativo: siempre viene acompañado.

Timestamps y PAWS (`RFC 7323`)

Un **timestamp** (32 bits, reloj del emisor) que el receptor devuelve **ecoado** en el ACK, con tres usos: **medir el RTT sin ambigüedad** (resuelve el problema de Karn), **detectar duplicados viejos** —si un segmento trae un `TSval` menor que el último visto, es un resto de una sesión anterior y se descarta, que es exactamente lo que hace **PAWS** (*Protect Against Wrapped Sequence numbers*)— y **proteger del wrap-around** del número de 32 bits. **PAWS es obligatorio si ambos extremos activaron timestamps.**

1.8 Por qué el flag PSH no garantiza nada (examen 2001-12, Problema 2)

> **Es sabido que, al enviar mensajes mediante una conexión TCP, el mero uso del bit de PUSH no asegura que, en el destino, la capa de aplicación reciba un mensaje por vez.**

Qué es PSH. Es una **sugerencia** de "no tengo más datos en el buffer, podés entregar lo que tengas". Cuando un segmento llega con PSH, el receptor entrega a la aplicación todo lo acumulado en el buffer **en un solo `read()`**:

```
A: write("hola") → write("chau")     ← dos mensajes de la aplicación
┌──────────────────┬──────────────────┐
│ "hola"  PSH=1    │ "chau"  PSH=1    │   ← dos segmentos
└──────────────────┴──────────────────┘
B: read() → "hola" / read() → "chau"   ← lo que uno ESPERA
B: read() → "holachu"                   ← lo que REALMENTE PASA
```

Por qué no funciona — tres razones acumulativas:

1. **PSH no transporta información de frontera de mensaje.** El campo es **un bit**, y no hay ningún campo que diga "este `read()` debe terminar acá". TCP es un **flujo de bytes** (*byte stream*) por diseño: tiene bytes, no mensajes.
2. **Los límites de segmento son invisibles para la aplicación.** El receptor ve una cola de bytes. Si los dos segmentos con PSH llegan en orden inverso, cuando llega el primero el buffer queda `["hola","chau"]` **contiguo**, y un solo `read()` devuelve todo junto.
3. **La segmentación.** Si un mensaje es más grande que el MSS queda partido en varios segmentos, y el PSH final llega después de bytes ya almacenados.

El laboratorio que pide el enunciado

El enunciado pide **construir un bridge** que manipule orden y segmentación: es factible, un bridge es software y el reordenamiento se hace con un **buffer de retardo**.

(a) Arribo simultáneo de dos mensajes — el bridge retiene el segmento del mensaje 1:

```
A ──[MSG1 seq=1000 len=4, PSH]──►   ┌────────────┐
                                    │  BRIDGE    │ retiene MSG1
A ──[MSG2 seq=1004 len=4, PSH]──►   │ (reordena) │──► MSG2 primero
B: buffer = ["chau"]          ← PSH de MSG2, pero falta MSG1 → NO entrega
B: buffer = ["hola","chau"]   ← llega MSG1: contiguo
B: read() → "holachu"         ← UN read(), dos mensajes  ✗
```

(b) Un mensaje y parte del siguiente — además, el bridge **parte** el segundo mensaje y deja pasar primero su primera mitad:

```
A ──[MSG1 seq=1000 len=4, PSH]──►   ┌────────────┐
A ──[MSG2 seq=1004 len=8, PSH]──►   │  BRIDGE    │──► [MSG2 parte 1: "chau c"]
                                    │ (retiene   │──► (después) [MSG1 completo]
                                    │  MSG1)     │
B: buffer = ["chau c"]              ← fuera de orden, NO entrega
B: buffer = ["hola","chau c"]        ← contiguo → read() = "hola" + "chau c"
```

⚠️ **El detalle que hace el experimento correcto:** los `seq` deben ser **coherentes y contiguos** (`MSG2` con `seq = 1004` va justo después de `MSG1`), porque si no el receptor lo descartaría por estar fuera de ventana y el experimento no probaría nada.

Cómo se resuelve de verdad. Si la aplicación necesita fronteras, **tiene que ponerlas ella misma**: un prefijo de longitud o un delimitador. Por eso HTTP tiene `Content-Length` y `Transfer-Encoding: chunked`, y por eso el RPC usa *marshalling* con longitud. El único transporte con noción nativa de mensaje es **SCTP**, y por eso WebRTC lo usa para sus canales de datos.

⚠️ Actualización a 2026

Sigue igual — de memoria: números de secuencia de **32 bits** que numeran **bytes**; **ACK acumulativo** (`ack` = próximo byte esperado); **timers por segmento**; Jacobson/Karels con α = 7/8, β = 1/4, `RTO = SRTT + 4·RTTVAR`, piso y techo, y **backoff exponencial**; **3 ACKs duplicados** = pérdida aislada y RTO = pérdida grave; **PSH es una sugerencia**.

Cambió:

- La especificación vigente es **`RFC 9293` (2022, *Internet Standard*)**, que consolida `RFC 793` + `RFC 879`, `2873`, `6093`, `6429`, `6528`, `6691`. Presentar `RFC 793` como "la especificación de TCP" en 2026 está desactualizado. **Lo que no cambió son los mecanismos de esta clase**: `RFC 9293` describe la misma fiabilidad, reorganizó la redacción y el estado normativo. `RFC 1122` sigue definiendo el comportamiento de los hosts.
- **SACK = `RFC 2018` (1996)**, *Standards Track*: es **comportamiento estándar** y está en todos los stacks. Cuando un enunciado pida el diagrama "a la antigua", aclaralo.
- **Timestamps + Window Scale = `RFC 7323` (2014)**, que **obsolesce** `RFC 1323`; **PAWS es obligatorio** si ambos extremos activaron timestamps.
- **Initial Window = 10 MSS (`RFC 6928`, 2013)**: el default en Linux, Windows y macOS. Los apuntes de 2011 dicen 1 MSS porque la `RFC 5681` lo decía así: **citá las dos cosas y seguí lo que pida el enunciado.**
- **QUIC (`RFC 9002`, 2021)** tiene **su propia recuperación de pérdida**: para HTTP/3 el flujo llega confirmado **por stream**, y el *Connection ID* permite sobrevivir al *NAT rebinding*, que en TCP se rompe.
- **MPTCPv1 (`RFC 8684`, 2020)** extiende la fiabilidad a varias rutas (cada subflujo aplica `RFC 9293` por separado). Desplegado pero con cuota marginal (~0,4 % de los bytes pico).
- ⚠️ **RACK** existe en stacks reales pero **no figura en `ACTUALIZACIONES.md`**: **verificar**.

> **Consecuencia para el examen:** los números de secuencia, el ACK acumulativo, los timers, Jacobson/Karels y la ventana deslizante **se preguntan exactamente igual que en 2011**. Lo único que cambia es la etiqueta: si citás "la RFC de TCP", es `RFC 9293`.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — RTO con Jacobson/Karels a partir de 4 muestras

Datos. α = 7/8, β = 1/4. Cuatro muestras de RTT en orden: `R₁ = 100 ms`, `R₂ = 120 ms`, `R₃ = 90 ms`, `R₄ = 130 ms`. Sin piso ni techo activos.

Paso 1 — Inicializar con la primera muestra. Con un solo dato no hay histórico, así que no se puede calcular una desviación:

```
SRTT   = R₁ = 100 ms
RTTVAR = R₁ / 2 = 50 ms
RTO    = SRTT + 4·RTTVAR = 100 + 4·50 = 300 ms
```

Paso 2 — Actualizar con R_2 = 120 ms. *Orden importante:* primero `RTTVAR` (que usa el `SRTT` **anterior**), después `SRTT`.

```
RTTVAR = 0,75 · 50 + 0,25 · |100 − 120| = 37,5 + 0,25·20 = 37,5 + 5 = 42,5 ms
SRTT   = 0,125 · 100 + 0,875 · 120    = 12,5 + 105       = 117,5 ms
RTO    = 117,5 + 4·42,5 = 117,5 + 170 = 287,5 ms
```

Paso 3 — Actualizar con R_3 = 90 ms.

```
RTTVAR = 0,75 · 42,5 + 0,25 · |117,5 − 90| = 31,875 + 6,875 = 38,75 ms
SRTT   = 0,125 · 117,5 + 0,875 · 90       = 14,6875 + 78,75 = 93,4375 ms
RTO    = 93,4375 + 4·38,75 = 93,4375 + 155 = 248,44 ms
```

Paso 4 — Actualizar con R_4 = 130 ms.

```
RTTVAR = 0,75 · 38,75 + 0,25 · |93,4375 − 130| = 29,0625 + 9,1406 = 38,2031 ms
SRTT   = 0,125 · 93,4375 + 0,875 · 130       = 11,6797 + 113,75 = 125,43 ms
RTO    = 125,43 + 4·38,2031 = 125,43 + 152,81 = 278,24 ms
```

Interpretación — qué está pasando. El RTO **oscila entre 248 y 300 ms**: ni sigue al RTT puntualmente (porque `α = 7/8` pesa el pasado 7 veces más que el presente) ni se queda corto. Es exactamente lo que se busca: **un RTO que cambia suave y no reacciona a un solo RTT raro**. Si el RTO fuera "el último RTT + un margen", el valor saltaría de 90 a 130 ms según la última muestra, y con 90 ms de RTO real habrías retransmitido de más.

Paso 5 — Backoff exponencial. Si ahora se pierde un segmento y vence el RTO, los timeouts sucesivos dan `2 · 278,24 = 556,48 ms`, luego `1112,96`, luego `2225,92`. Y cuando llega un ACK que **no** corresponde a una retransmisión, se recalcula desde cero: el backoff solo aplica mientras no haya progreso.

⚠️ Errores típicos

- **Usar `α` y `β` al revés.** `α = 7/8` es para `SRTT`; `β = 1/4` para `RTTVAR`. **α grande → suavizo lento; β pequeño → reacción rápida.**
- **Olvidar la inicialización** `SRTT = R`, `RTTVAR = R/2`.
- **Confundir `RTT` con `RTO`.** El `RTT` es lo que **midió**; el `RTO`, lo que **va a esperar**.
- **Creer que el RTO se recalcula en cada ACK.** Se recalcula por **muestra de RTT** no ambigua.

2.2 Ejercicio 2 — BDP y tamaño mínimo de ventana

Datos (a–d). Enlace de **100 Mbps** (datos de capa 3, sin overhead de capas 1 y 2), RTT extremo a extremo **80 ms**, MTU 1500 bytes.

a) ¿Cuál es el BDP?

```
BDP = 100 · 10⁶ bits/s × 0,080 s = 8 · 10⁶ bits = 8·10⁶ / 8 = 1 000 000 bytes
    ≈ 1 MB   (en KiB: 1 000 000 / 1024 = 976,6 KiB)
```

b) ¿Cuántos segmentos de tamaño máximo hacen falta?

MSS = MTU − cabecera IP − cabecera TCP = 1500 − 20 − 20 = 1460 bytes

```
1 000 000 / 1460 = 684,93 → 685 segmentos
```

c) ¿Tamaño mínimo de ventana para usar toda la capacidad?

```
ventana_mínima = BDP = 1 000 000 bytes
comprobación: 1 000 000 / 0,080 s = 12 500 000 B/s = 100 · 10⁶ bits/s = 100 Mbps ✓
```

d) ¿Y con la máxima del campo `window` de 16 bits, sin *window scale*?

```
ventana = 65 535 bytes (2¹⁶ − 1)
throughput = 65 535 / 0,080 s = 819 187,5 B/s = 6,55 Mbit/s
```

Interpretación — y acá está la moraleja. Con el campo de 16 bits **a secas**, este enlace de 100 Mbps **es inalcanzable**: sacás 6,55 Mbit/s como máximo. **Es el ejemplo perfecto de por qué un protocolo de 1992 necesita revisión**: el campo de 16 bits bastaba para las redes de entonces.

⚠️ Errores típicos

- **Olvidar `/8`.** Si mezclás bits con bytes te da 8 MB en vez de 1 MB: factor 8 de error.
- **Usar 1000 en vez de 1024** (o al revés) sin decirlo. Declaralo.
- **Confundir "ventana mínima" con "ventana igual a un MSS".** El mínimo para **usar todo el enlace** es el BDP; el mínimo para **no bloquearse** es 1 MSS (o 2, por el *silly window syndrome*).
- **Olvidar la MTU.** MSS no es "1500": es `1500 − 20 − 20 = 1460` sin opciones, o `1500 − 20 − 32 = 1448` con timestamps.

2.3 Ejercicio 3 — Comparar cuatro ventanas sobre el mismo enlace

Datos. Enlace de **1 Gbps**, RTT **50 ms** uniforme. Archivo de **4 MB** (`4 · 10⁶` bytes).

```
BDP = 10⁹ · 0,050 / 8 = 5 · 10⁷ bits = 6 250 000 bytes ≈ 5,96 MiB   (4281 MSS)

throughput = min( ventana / 0,050 , 125 · 10⁶ B/s )
tiempo     = 4 · 10⁶ B / throughput
```

Ventana	`ventana/RTT`	Throughput real	Tiempo
8 KiB = 8 192 B	163 840 B/s	**1,31 Mbit/s** (limitado por ventana)	**24,4 s**
64 KiB = 65 536 B	1 310 720 B/s	**10,5 Mbit/s** (limitado por ventana)	**3,05 s**
763 KiB = 781 250 B	15 625 000 B/s	**125 Mbit/s** (ventana justo = enlace)	**0,256 s**
2 MiB = 2 097 152 B	41 943 040 B/s	**1 000 Mbit/s** (limitado por enlace)	**0,032 s**

Verificación de la última fila: `4·10⁶ / 41,94·10⁶ = 0,095 s`, **mayor** que los `0,032 s` que tarda el enlace: imposible, el enlace no puede ir más rápido que sí mismo. La ventana de 2 MiB "sobra".

Interpretación. De 8 KiB a 64 KiB el tiempo bajó de 24,4 s a 3,05 s: **8 veces menos tiempo por 8 veces más ventana**, porque ahí estás limitado por `ventana/RTT`. A partir de `ventana = BDP` el enlace se satura y **agrandar la ventana ya no sirve**: es el techo físico. Por eso definir bien `SO_RCVBUF` / `SO_SNDBUF` y el *window scale* es la diferencia entre 1,31 y 1000 Mbit/s — y por eso el `rwnd` es un problema de **flujo**: agrandarlo no le pide nada a la red.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 Examen 2010-12, Pregunta 4 (8 puntos) — números de secuencia y timers

> **Fuente:** Examen `sirc1012` — 17 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf **Texto del examen:**
>
> En el protocolo de capa de transporte TCP: a) ¿Por qué se necesitan números de secuencia? b) ¿Por qué son necesarios timers?

Respuesta oficial a) (textual):

> Para aprovechar mejor el enlace, TCP es también un protocolo **pipelined** por lo que el emisor envía varios paquetes antes de recibir confirmación por alguno de ellos. Por esta razón, el emisor debe poder distinguir **a qué paquete o paquetes corresponde una determinada confirmación**, por lo que numera los paquetes enviados secuencialmente.

Los 4 puntos: (1) **pipelining**: para aprovechar el enlace A manda varios antes de esperar; (2) **consecuencia**: hay confirmaciones en vuelo y son ambiguas sin identificador; (3) **solución**: numerar, y el ACK usa esos números; (4) **matiz que suma**: los números son **de bytes**, no de segmentos, y el ACK es **acumulativo**.

Por qué los números no son un lujo del pipelining. Aunque TCP fuera `stop-and-wait`, haría falta algo: si el receptor asienta y se pierde el ACK, hay que saber qué reenviar. El pipelining **amplifica** la necesidad, no la crea. Y si además hay **reordenamiento** (ida y vuelta por caminos distintos), sin números el receptor no podría ni ordenar.

Respuesta oficial b) (textual):

> De no recibir esta confirmación, el emisor no sabe si esto se debe a que se perdió el paquete enviado, la confirmación o simplemente alguno de los dos está demorado. **La única solución es retransmitir el paquete no confirmado.** Para esta retransmisión debe esperar un tiempo prudencial luego de enviado cada paquete, por lo que necesita un timer que le indique cuándo hacerlo.

Los 4 puntos: (1) **las tres causas indistinguibles**: se perdió el dato / se perdió el ACK / tardó; (2) **por qué obliga a retransmitir**: no hay forma de distinguirlas, y no retransmitir rompe la fiabilidad; (3) **por qué además hace falta saber *cuándo***: esperar un tiempo prudencial; (4) **un timer por segmento**, no por conexión.

Si querés ir más allá (suma): el RTO no es fijo sino una estimación (Jacobson/Karels, §1.4), ante timeout se **duplica** (*backoff exponencial*), y con timestamps (`RFC 7323`) el problema de *Karn* se resuelve.

3.2 Examen 2009-12, Pregunta 2 (10 puntos) — fiabilidad en transporte vs. en enlace

> **Fuente:** Examen `sirc0912` — 12 de diciembre de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0912.pdf **Texto del examen:**
> Se considera un servicio de transmisión confiable. a) Señale las diferencias de implementación en capa de transporte vs. capa de enlace. b) En una red de paquetes arbitraria, ¿es posible implementar un servicio de transmisión confiable en capa de enlace? Justifique su respuesta.

Respuesta oficial a): "En capa de transporte se usan los servicios de la capa de red y la señalización del protocolo de transporte, que implica **realimentación de extremo a extremo**, teniendo en cuenta la carga de red (congestión). En capa de enlace se señaliza en base a la **capacidad del enlace**."

Respuesta oficial b): "**No es posible** porque pueden existir múltiples hops, con capas de enlace diferentes."

	Capa de enlace	Capa de transporte (TCP)
Alcance de la fiabilidad	**por salto** (*per hop*)	**extremo a extremo**
Qué señaliza	capacidad del **enlace**	carga de la **red**
Estado que guarda	1 paquete a la vez (2 con Go-Back-N)	una **ventana** de N bytes
Numeración	por **trama** del enlace	por **bytes**, global a la conexión
Detecta pérdida	por **CRC** del enlace	por **ACK + timer**, y **no sabe** quién falló
Congestión	no la maneja	**sí** (es su razón de ser)
Depende de que los saltos sean fiables	**sí, crítico**	**no, en absoluto**

La parte b) tiene que ir más allá de "hay varios hops". El contraejemplo mínimo: el paquete se pierde **dentro de R2**. La capa 2 del enlace de entrada a R2 no ve nada raro (entró bien y salió bien), la del enlace de salida ve un paquete entrar y lo entrega sin retransmitir nunca, y la capa 3 descartó el paquete sin avisar a nadie. **El paquete se perdió PARA SIEMPRE**: ningún mecanismo de capa 2 lo detecta.

Y una razón estructural más fuerte: el número de secuencia de capa 2 no atraviesa los routers. Para que una capa de enlace fuese *extremo a extremo* tendría que: **(1)** mantener estado por flujo en cada router (o sea, reimplementar TCP en cada salto); **(2)** usar números de secuencia globales, imposibles si cada capa 2 no sabe nada de lo que hay aguas arriba; **(3)** mandar ACKs de extremo a extremo, y entonces ya no es capa de enlace, es transporte con otro nombre.

Conclusión (una frase): la fiabilidad **extremo a extremo** es la única que sobrevive a una red no confiable. Por eso está **arriba** y no abajo.

3.3 Examen 2001-12, Problema 2 — el flag PUSH

Resuelto en §1.8. Lo que hay que dejar asentado:

- **Tesis:** PSH no define fronteras de mensaje; es un bit de sugerencia.
- **Prueba de laboratorio:** un bridge con buffer de retardo reordena y/o segmenta.

- **Los dos escenarios pedidos:** (a) dos mensajes que llegan juntos al `read()`; (b) un mensaje entero y parte del siguiente.
- **La solución real:** fronteras a nivel de aplicación (`Content-Length`, *marshalling* con longitud).

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Ordenada** → números de secuencia · **sin duplicados** → el receptor descarta · **sin errores** → checksum + retransmisión.
2. TCP es **pipelined**: manda varios segmentos antes de tener ACKs, y por eso el ACK tiene que decir **cuál** confirma.
3. Los números son **de bytes**; el `ack` es el **próximo byte esperado**, y es **acumulativo**.
4. **Timer por segmento**: sin ACK hay tres causas **indistinguibles** y en las tres "no sé".
5. La única salida es **retransmitir**, y para eso hay que saber **cuándo** → RTO.
6. **Jacobson/Karels**: `SRTT = α·SRTT + (1−α)·R` con α = 7/8; `RTTVAR` con β = 1/4; **`RTO = SRTT + 4·RTTVAR`**.
7. Ante timeout el RTO **se duplica** (*backoff exponencial*).
8. **Ventana deslizante** = bytes sin confirmar; para llenar un enlace hace falta **`ventana ≥ BDP = ancho_de_banda × RTT`**.
9. **Fast retransmit** (3 dupACKs) recupera rápido; **SACK (`RFC 2018`)** dice *qué* bloques llegaron, así que recupera en 1 RTT en vez de varios.
10. **PSH no garantiza un mensaje por `read()`**: TCP es un flujo de bytes y las fronteras las pone la aplicación.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. El `ack` de un segmento es 1120. ¿Cuántos bytes recibió el receptor como mínimo?

▶ Respuesta

2. A mandó 5 segmentos y el RTO del **primero** se venció. ¿Qué tres cosas pueden estar pasando? ¿Puede TCP saber cuál es?

▶ Respuesta

3. Calculá el RTO con Jacobson/Karels si `SRTT = 100 ms`, `RTTVAR = 20 ms` y llega `R = 100 ms`.

▶ Respuesta

4. Ancho de banda 1 Gbps, RTT 20 ms. ¿Qué ventana mínima para saturarlo?

▶ Respuesta

5. ¿Por qué `SRTT + 4·RTTVAR` y no `SRTT · 2`?

▶ Respuesta

6. Un switch en el medio reordena dos segmentos con PSH: pasa primero el segundo. ¿Qué lee la aplicación en el destino?

▶ Respuesta

7. ¿Qué cambia, en 2026, respecto de lo que se enseñaba en 2011?

▶ Respuesta

8. ¿Por qué no se puede medir el RTT de un segmento retransmitido?

▶ Respuesta

🔗 Referencias

- **Examen** `sirc1103` — P4 (10 pts): 4 segmentos con y sin pérdida · Problema 1 (30 pts): ECN
- **Examen** `sirc1012` — P4 (8 pts): números de secuencia y timers
- **Examen** `sirc0912` — P2 (10 pts): fiabilidad en transporte vs. enlace
- **Examen** `sirc0112` — Problema 2: el flag PUSH y la segmentación
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 §6.5 (pág. ~590)
- `RFC 9293` — TCP (2022, *Internet Standard*) · `RFC 2018` — SACK (1996)
- `RFC 7323` — Timestamps, Window Scale y PAWS (2014) · `RFC 5681` — Control de congestión (2009)
- `RFC 6928` — IW = 10 MSS (2013) · `RFC 9002` — QUIC (2021) · `RFC 8684` — MPTCPv1 (2020)
- **Clase siguiente:** 20 — Control de flujo vs. control de congestión

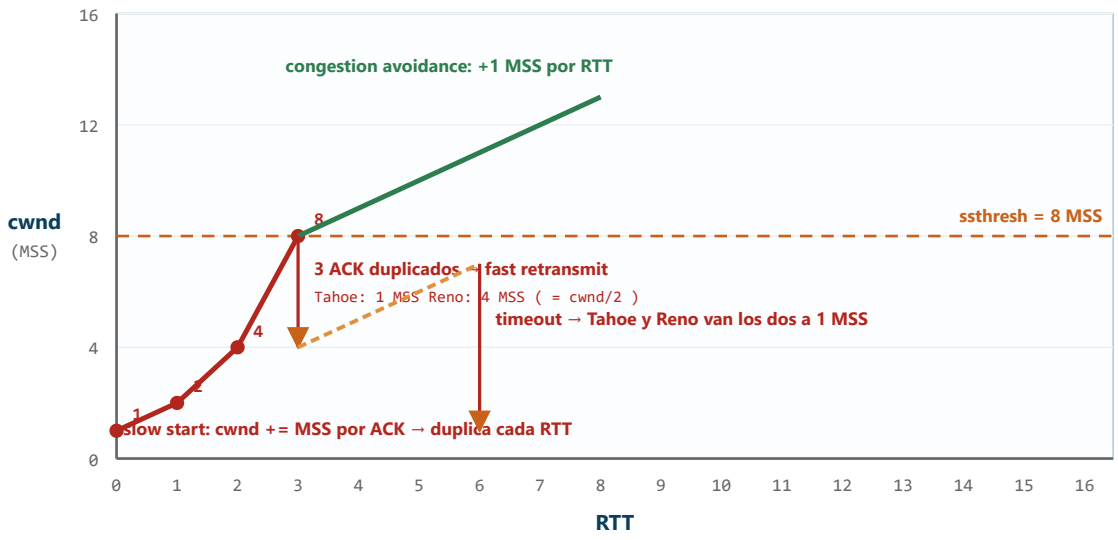
Clase 20

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 Los dos problemas
1.2 El `rwnd`: el campo *window* de la cabecera
1.3 La ventana efectiva: por qué el mínimo y no la suma
1.4 Zero window: el receptor que no lee
1.5 El otro lado: los buffers de socket del emisor
1.6 Quién controla qué
2.1 Ejercicio 1 — Throughput con el `rwnd` limitado
2.2 Ejercicio 2 — El receptor deja de leer a mitad de la transferencia
2.3 Ejercicio 3 — `cwnd` grande y `rwnd` chico: cuánto throughput se pierde
3.1 Examen 2009-07, Pregunta 1 (10 puntos)
3.2 Examen 2010-02, Pregunta 5 (7 puntos) — el mecanismo de control de flujo
3.3 Conexión con 2011-03 P4 (10 puntos)
📌 Resumen en 10 líneas
✅ Autoevaluación
🔗 Referencias

Bloque: C (Transporte) · **Semana:** 7 · **Duración estimada:** 100 min **Correlativa:** Clase 19 (cómo garantiza TCP la fiabilidad) · **Sigue con:** Clase 21 (control de congestión en TCP: slow start, AIMD y ECN) **Material de referencia:** Tanenbaum cap. 6 (§6.7) · RFC 9293, RFC 7323, RFC 6928, RFC 5681, RFC 9438

Evolucion de la ventana de congestion



Tahoe vuelve a 1 MSS ante cualquier perdida.
Reno solo cae a cwnd/2 con 3 ACK duplicados; ante un timeout cae igual que Tahoe.

🎯 Al terminar esta clase vas a poder

1. Distinguir **control de flujo** de **control de congestión** y decir quién controla cada uno y con qué variable.
2. Explicar cómo viaja el `rwnd` en la cabecera TCP y por qué el emisor **no puede mandar más que el** `rwnd`.
3. Justificar por qué la ventana efectiva es el **mínimo** de `cwnd` y `rwnd` y no la suma.
4. Describir qué pasa cuando el `rwnd` llega a **cero**, qué son los *zero-window probes* y por qué eso **no** es control de congestión.

5. Explicar el problema de los **buffers de socket** en los dos extremos y por qué el del emisor **también** es un problema de flujo.
6. Calcular el **throughput** de una conexión cuando el límite es el `rwnd`, y decir cuál de los dos límites manda.

> **Prerequisito del examen:** `2009-07 P1` (10 pts — flujo vs. congestión, y cómo detecta TCP la congestión), `2010-02 P5` (7 pts — el mecanismo de control de flujo de TCP), `2011-03 P4` (10 pts — el diagrama con `cwnd`), `2010-12 P4b` (los timers, que son del otro lado de la misma ecuación).

PARTE 1 — TEORÍA

1.1 Los dos problemas

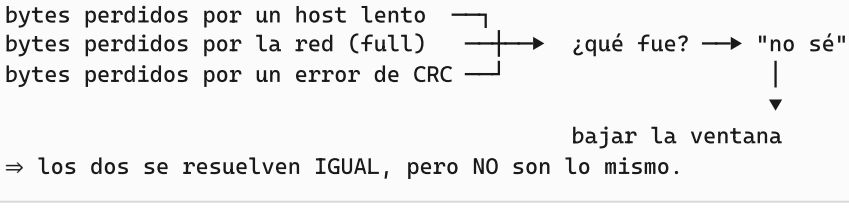
Imaginá una descarga. Tu disco de 500 Mbps no te sirve de nada si el enlace de tu casa da 10 Mbps. Y al revés: un enlace de 100 Mbps que compartís con 40 personas te va a parecer lento. Son **dos problemas distintos**:

```
┌─ El emisor manda MÁS RÁPIDO de lo que el receptor puede absorber ─┐
│ → CONTROL DE FLUJO │   (lo arregla rwnd, esta clase)              │
└──────────────────────────────────────────────────────────────────┘

┌─ La RED en el medio no tiene recursos para todo lo que todos mandan ┐
│ → CONTROL DE CONGESTIÓN │  (lo arregla cwnd, en la Clase 21)       │
└──────────────────────────────────────────────────────────────────┘
```

La tabla que usa el profesor para diferenciarlos. Esta es, literal, la que hay que aprender de memoria porque es la respuesta de `2009-07 P1a`:

	Control de flujo	Control de congestión
El problema es de	un **host lento**	**toda la red**
El síntoma es	el buffer del receptor se llena	los buffers de **los routers** se llenan
La pérdida significa	"no leíste rápido"	"**la red está saturada**"
La ventana la controla	el **receptor** (`rwnd`)	el **emisor**, estimando (`cwnd`)
Se arregla con	ACK diferidos / zero-window probe	**reducir el ritmo de envío**
Le importa el BDP	sí, mucho	sí, pero como techo físico

La evidencia de que confundirlos es el error típico del examen. La razón de fondo es esta: **TCP no puede distinguir una pérdida por buffer lleno del receptor de una pérdida en la red.** Para TCP, un paquete perdido es un paquete perdido. Por eso el mismo mecanismo (disminuir la ventana) sirve para las dos cosas, y por eso la interpretación que TCP hace es conservadora:



Y esto conecta con la pregunta 1c de `2009-07`, la más fina del tema: **¿la pérdida siempre implica congestión? No.** También por una capa de enlace defectuosa, un buffer del receptor lleno o un error de CRC. TCP **asume** que sí, porque asumir lo contrario sería peor en promedio: castigar a la red de vez en cuando sale más barato que saturarla.

1.2 El `rwnd`: el campo *window* de la cabecera

`rwnd` (*receive window*) es la cantidad de bytes que el receptor dice que le entran en su buffer **a partir del byte que está esperando**. Viaja en el campo `window` de **cada** segmento TCP, y es de **16 bits** (0 a 65 535),

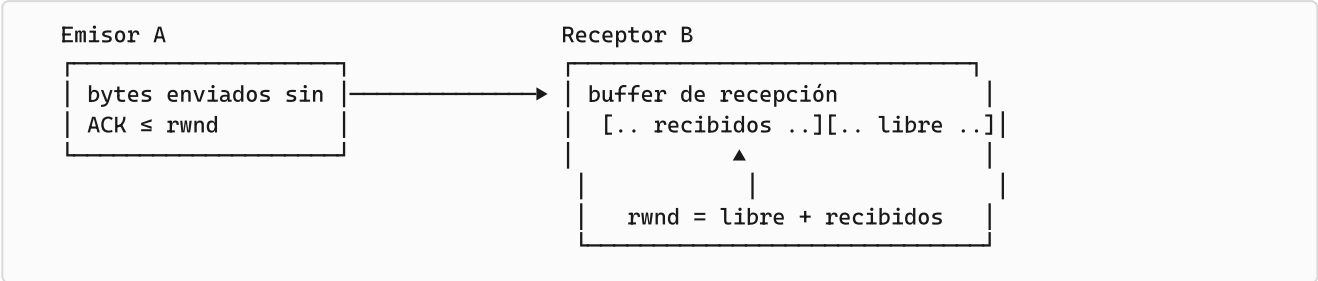
multiplicado por el factor de escalado si se negoció *Window Scale*.

La cuenta del número de ventana. El receptor no anuncia "me quedan 40000 bytes de buffer". Anuncia **cuántos bytes quiere recibir a partir del próximo que espera**:

```
rwnd = (espacio libre en el buffer) + (bytes ya recibidos sin entregar a la aplicación)
```

O sea, el `rwnd` cubre **los dos** casos: lo que ya llegó esperando `read()` y lo que puede llegar todavía. Esto es lo que hace que sea una cota *superior* y no una medida instantánea.

La restricción dura: el emisor **no puede tener más de `rwnd` bytes sin confirmar**.



1.3 La ventana efectiva: por qué el mínimo y no la suma

```
ventana efectiva = min(cwnd, rwnd)
                   ─────    ─────
                congestión   flujo
```

¿Por qué el mínimo? Porque **las dos son cotas superiores independientes**:

- el emisor no puede mandar más de lo que **la red aguanta** (`cwnd`), y
- no puede mandar más de lo que **el receptor aguanta** (`rwnd`).

¿Por qué no la suma? Este es el punto fino que más se pregunta. Si la ventana efectiva fuera `cwnd + rwnd`, el excedente que la red no pudo tomar **se acumularía en el buffer del receptor y lo desbordaría**. Y lo desbordaría **justo cuando la red va peor**, que es exactamente cuando el buffer del receptor ya está más lleno. El `min` es el que garantiza que **nadie se desborda**: ni la red (por `cwnd`), ni el receptor (por `rwnd`).

```
Ventana efectiva = cwnd + rwnd (¡MAL!)

A ──[──────────── cwnd ────────────][──── rwnd ────]──▶ B
                                          / buffer de B
                                         / se llena con lo que
                                        / la red NO pudo tomar
                                        ↓
                                   OVERFLOW de B

Ventana efectiva = min(cwnd, rwnd) (✓ correcto)

A ──[──── min(cwnd, rwnd) ────]──▶ B
     todo lo que sale entra, y entra todo lo que sale
```

En los exámenes los dos los llaman "ventana efectiva" o "ventana de congestión" según el caso. Si el enunciado habla de **una sola** ventana, asumí que `rwnd` es grande y el límite es `cwnd`.

> Todo el desarrollo del `cwnd` — slow start, congestion avoidance, fast recovery, ECN — está en la **Clase 21**. Esta clase se limita a explicar **qué es** y **por qué son dos cosas**.

1.4 Zero window: el receptor que no lee

El caso límite del control de flujo: el receptor **dejó de leer** y su buffer está lleno.

```
t0  B anuncia rwnd = 20 000      A manda 20 000 bytes      buffer de B: LLENO
t1  A manda otros 20 000         B NO asienta              rwnd = 0
t2  A no puede mandar nada                          ventana efectiva = min(cwnd, 0) = 0
t3  A queda en silencio...
```

Si A se quedara quieto para siempre, la conexión se **cuelga**, aunque el otro lado esté perfectamente sano. Para eso existe el **persistence timer** (`RFC 9293` §8.6): A manda **zero-window probes** (*sondas de ventana cero*).

```
A ──────[1 byte, seq=next]──► B    ¿podés volver a leer?
◄──────[ACK, rwnd=0]─────── A    "todavía no" → esperar más
    (espera el doble: 1 s, 2 s, 4 s, 8 s... hasta un tope)
A ──────[1 byte, seq=next]──► B
◄──────[ACK, rwnd=16384]── A    "ya, dale" → rwnd ≠ 0 → reanuda
```

Las características de las sondas, que hay que saber:

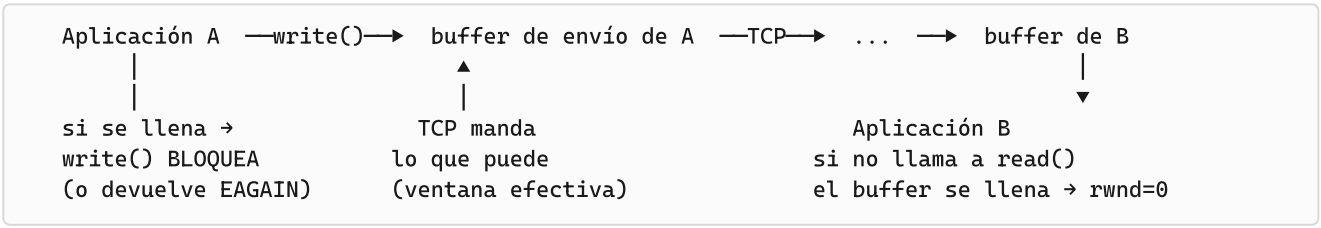
Aspecto	Detalle
Tamaño	**1 byte** (o el último byte recibido)
Cuándo	cuando expira el *persist timer*, no antes
Periodicidad	**exponencial**: RTO, 2·RTO, 4·RTO… hasta un tope (~60 s)
Para qué	no es un retransmit: es una **consulta**
Si B responde con `rwnd = 0`	A vuelve a esperar otro intervalo
Riesgo	el **deadlock permanente**: si B cerró el socket, A queda sondeando para siempre (el *keepalive* es lo que corta esto)

Y acá está lo importante para el examen:

> ⚠️ **Esto NO es control de congestión.** Es control de flujo, y no confundirlo es explícitamente lo que se busca en `2009-07 P1a`. **La red está perfecta**: nadie está saturado, no hay congestión de ningún tipo. Lo que está lento es la **aplicación del otro lado**, que no está llamando a `read()`. Si TCP bajara el `cwnd` por esto, estaría castigando a la red por un problema del receptor, y además **no arreglaría nada**: el `cwnd` no puede hacer que una aplicación lea más rápido. **Por eso son mecanismos separados.**

1.5 El otro lado: los buffers de socket del emisor

El `rwnd` protege el buffer **del receptor**. Pero hay un tercer buffer en la ecuación, y es el que se olvida: el **buffer de envío del emisor**.



El caso del emisor. Si A escribe más rápido de lo que la conexión puede sacar (porque el `rwnd` de B es chico, o porque la red es lenta), el buffer de envío de A se llena. En un socket **bloqueante**, el `write()` **se bloquea**: la aplicación se frena. Esa es la otra mitad del control de flujo, y es puramente local: no hace falta ningún campo en la cabecera, es el sistema operativo frenando a la aplicación.

El caso del receptor. Simétrico: si B no llama a `read()`, su buffer se llena, el `rwnd` baja, y A se frena. Acá sí hay señalización en la red, porque **el `rwnd` viaja en cada segmento**.

Por qué importa decirlo: los dos casos producen el mismo síntoma observable ("no avanza") pero por mecanismos distintos, y solo uno de los dos necesita el campo `window` de la cabecera.

1.6 Quién controla qué

Variable / mecanismo	Lo decide	Cómo se comunica	Señal explícita
`rwnd`	el **receptor**	campo `window` de cada segmento	sí, el receptor lo anuncia
`cwnd`	el **emisor**	nadie: es una hipótesis local	no, se deduce de ACKs y timeouts
Buffer de envío	el **emisor** (local)	`write()` bloquea	sí, al proceso
Buffer de recepción	el receptor	`read()` consume	sí, al proceso
Segmentación	el **emisor** (MSS del receptor)	opción MSS en el SYN	sí, en el SYN
Cierre ordenado	**ambos**	FIN / TIME-WAIT	sí, FIN

Detalle fino que se pregunta: el `rwnd` lo decide el receptor, **pero es un campo en un segmento que manda el receptor, y lo lee el emisor**. Nadie lo negocia explícitamente: el emisor simplemente obedece lo último que oyó.

⚠️ Actualización a 2026

Sigue igual — de memoria: - `rwnd` viaja en el campo `window` (16 bits) de cada segmento TCP. - `ventana efectiva = min(cwnd, rwnd)`, y es un **mínimo** porque son cotas superiores. - El **zero-window probe** es de **1 byte**, con periodo **exponencial**, y sirve para destrabar, no para retransmitir. - Congestión = problema de **la red**; flujo = problema del **otro host**.

Cambió o se afinó:

- La especificación vigente es **`RFC 9293` (2022, *Internet Standard*)**, que consolida `RFC 793`
- `RFC 879`, `2873`, `6093`, `6429`, `6528`, `6691` y cuyo §8.6 es el de la **persistencia**. Citar `RFC 793` como "la especificación" en 2026 está desactualizado.
- **`Window Scale` (`RFC 7323`, 2014)** es lo que hace que el `rwnd` de 16 bits deje de ser un techo real: se negocia un *shift* en el SYN y el campo queda multiplicado por `2^shift`, hasta `2³°` bytes. Sin esto, con RTT de 80 ms el `rwnd` limitaría el throughput a 6,55 Mbit/s (Clase 19 §1.6).
- **El Initial Window es 10 MSS** por defecto (`RFC 6928`, 2013), no 1. El `rwnd` inicial típico sigue siendo 64 KiB en varias plataformas — que es justamente el valor que **limita antes que el `cwnd`** en conexiones de corta vida. ⇒ **En una descarga corta de web, muchas veces el límite es `rwnd`, no `cwnd`.**
- **El control de congestión por defecto hoy es CUBIC** (`RFC 9438`, 2023, *Standards Track*), no Reno ni Tahoe. Pero el `rwnd` y la fórmula `min(cwnd, rwnd)` **no cambian**: CUBIC decide el valor de `cwnd`, y el mínimo con `rwnd` se sigue aplicando igual. → **Para el examen:** si te piden "el mecanismo de control de congestión", respondé slow start / congestion avoidance / fast retransmit / fast recovery (Clase 21), y si sobra lugar, una línea de "hoy es CUBIC". Si te piden "control de flujo", es `rwnd` y punto.
- **QUIC (`RFC 9000`, 2021)** no usa una única ventana global: maneja **dos niveles**, el de **conexión** (`max_data`) y el de **cada stream** (`max_stream_data`), porque multiplexa muchos streams en una sola conexión. El *Connection ID* además permite sobrevivir a cambios de topología y al *NAT rebinding*. La idea es la misma; la estructura es otra.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Throughput con el `rwnd` limitado

Datos. `rwnd = 16 KB = 16 384 bytes`, constante. RTT = 120 ms. El `cwnd` es grande (`cwnd = 1 MB`), así que **no** es el que limita.

a) ¿Cuál es la ventana efectiva?

```
ventana efectiva = min(cwnd, rwnd) = min(1 048 576, 16 384) = 16 384 bytes
→ manda el rwnd.
```

b) ¿Cuál es el throughput máximo?

El throughput no es "bytes por RTT": es cuántos bytes salen por segundo. Con 16 KiB cada 120 ms:

```
throughput = 16 384 B / 0,120 s = 136 533 B/s
           = 136 533 · 8 bits/s = 1 092 267 bit/s
           ≈ 1,09 Mbit/s
```

c) ¿Qué `rwnd` necesitaría para alcanzar 10 Mbit/s con ese RTT?

Se despeja de `throughput = rwnd / RTT`:

```
rwnd = throughput · RTT = 10 · 10⁶ bits/s · 0,120 s / 8 = 150 000 bytes
```

Interpretación — y el error conceptual que esconde. Con `cwnd = 1 MB`, **duplicar el `cwnd` no mejora nada**: el mínimo sigue siendo 16 KiB. El único camino es que **el receptor abra el `rwnd`**, es decir que consuma más rápido (`read()` más seguido) o que tenga un buffer de recepción más grande (`SO_RCVBUF`). **Ninguna acción sobre la red sirve para mejorar esto.** Por eso se dice que el `rwnd` es "el grifo que abre el receptor": agrandarlo no le pide nada a nadie.

2.2 Ejercicio 2 — El receptor deja de leer a mitad de la transferencia

Datos. `cwnd = 8 MSS` (8 KiB). El receptor anuncia `rwnd = 16 KiB` al principio. En un momento dado la aplicación de B deja de llamar a `read()` durante 3 segundos.

a) Evolución de la ventana efectiva y del `rwnd`, RTT por RTT.

Suposiciones (y hay que declararlas): MSS = 1 KiB, RTT = 50 ms, sin pérdidas, ACKs inmediatos. "Deja de leer" significa que el buffer de recepción de B no se vacía, pero B sigue mandando ACKs, porque TCP asienta por recepción de bytes, no por `read()`.

```
RTT 0: rwnd = 16 KiB (buffer vacío)     ventana = min(8, 16) = 8 KiB  ← manda cwnd
RTT 1: A manda 8 KiB                     ventana = 8 KiB
RTT 2: B tiene 16 KiB en buffer (8 + 8)  rwnd = 0  → ventana = min(8, 0) = 0
RTT 3..59: ventana = 0                   A NO MANDA NADA
RTT 60: expira el persist timer
        A ──[1 byte, zero-window probe]──►
◄──[ACK=..., rwnd=0]── A                 ventana = 0, espera el doble (100 ms)
... se repite con 200 ms, 400 ms, 800 ms, 1600 ms ...
```

b) ¿Cuántos bytes perdió A en el tiempo en que estuvo frenado?

```
Bytes enviados antes de frenar = 16 KiB (lo que entró en el buffer de B)
Bytes enviados durante el freno = 0
→ A "perdió" ≈ 3 s × 8 KiB/RTT = 3/0,050 × 8 KiB = 480 KiB de oportunidad perdida
```

cálculo de oportunidad, no de datos: esos 480 KiB no se mandaron, simplemente porque la ventana efectiva era 0.

c) ¿Cuándo se reanuda?

```
A manda la sonda → B responde ACK con rwnd > 0
→ ventana efectiva = min(cwnd, rwnd) > 0
→ A reanuda a plena velocidad, sin evento de congestión de por medio
```

d) ¿Qué NO pasó en esta historia?

```
✗ cwnd NO bajó           (no hubo pérdida: no hubo ni un paquete perdido)
✗ No hubo RTO             (no hubo datos sin confirmar: no había nada en vuelo)
✗ La red no se enteró     (no hubo congestión: ningún router bajó nada)
✗ No hubo control de congestión en ningún momento
```

Interpretación. Esta es la demostración más limpia de por qué los dos mecanismos tienen que estar separados. **Todo el throughput perdido se recuperó instantáneamente** en cuanto B volvió a leer, porque la red nunca estuvo congestionada y `cwnd` nunca bajó. Si TCP hubiera aplicado control de congestión, la ventana estaría artificialmente destruida y la recuperación habría tardado varios RTTs.

⚠️ Errores típicos

- **Bajar `cwnd` cuando el `rwnd` es 0.** Es el error conceptual más caro del tema.
- **Asumir que B deja de mandar ACKs cuando deja de leer.** Falso: TCP asienta por **recepción de bytes**, no por entrega a la aplicación. Los ACKs siguen fluyendo (salvo *delayed ACK*), lo que mantiene viva la conexión.
- **Confundir la sonda de 1 byte con una retransmisión.** Es una **consulta**: el byte no es un dato nuevo, es el que ya está confirmado, y su único objetivo es generar un ACK con el `rwnd` actualizado.

2.3 Ejercicio 3 — `cwnd` grande y `rwnd` chico: cuánto throughput se pierde

Datos. Enlace de 1 Gbps, RTT 50 ms. Se comparan tres configuraciones.

a) `cwnd = 1 MiB`, `rwnd = 8 KiB`.

```
ventana efectiva = min(1 048 576, 8 192) = 8 192 bytes
throughput = 8 192 / 0,050 = 163 840 B/s = 1,31 Mbit/s
utilización del enlace = 1,31 / 1000 = 0,13 %
```

b) `cwnd = 8 KiB`, `rwnd = 1 MiB`.

```
ventana efectiva = min(8 192, 1 048 576) = 8 192 bytes
throughput = 163 840 B/s = 1,31 Mbit/s      ← ¡idéntico! y es otro problema
```

c) `cwnd = 1 MiB`, `rwnd = 1 MiB`.

```
ventana efectiva = 1 048 576 bytes
throughput = 1 048 576 / 0,050 = 20 971 520 B/s = 167,8 Mbit/s
utilización = 16,8 %
```

d) Comparación y diagnóstico.

Configuración	Ventana efectiva	Throughput	Utiliz.	**Quién limita**	**Qué se arregla**
`cwnd` =1 MiB, `rwnd` =8 KiB	8 192	1,31 Mbit/s	0,13 %	**el receptor**	que B lea más rápido / `SO_RCVBUF` mayor
`cwnd` =8 KiB, `rwnd` =1 MiB	8 192	1,31 Mbit/s	0,13 %	**el emisor**	que la red dé más (o `cwnd` crezca)
`cwnd` =1 MiB, `rwnd` =1 MiB	1 048 576	167,8 Mbit/s	16,8 %	**el enlace**	más ancho de banda

Interpretación — y acá está la moraleja. Las dos primeras filas dan **exactamente el mismo número**, y sin embargo **ninguna de las dos soluciones sirve para la otra**: si en (a) agrandás `cwnd` a 1 MiB no cambia nada (el mínimo sigue siendo `rwnd`), y si en (b) agrandás `rwnd` a 1 MiB tampoco (ahora el mínimo pasa a ser `cwnd`). Y el 83 % del enlace sin usar en (c) tampoco es culpa de ninguna de las dos ventanas: es el **BDP** (`10⁹ · 0,050 / 8 = 6 250 000 bytes`), así que con 1 MiB de ventana seguís muy por debajo del techo físico.

El corolario práctico: siempre que veas un throughput bajo, **lo primero es mirar los dos límites y ver cuál manda**, antes de tocar nada. Es el mismo diagnóstico del ejercicio 2.2.

3.1 Examen 2009-07, Pregunta 1 (10 puntos)

> **Fuente:** Examen `sirc0907` — 29 de julio de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0907.pdf **Texto del examen:**
>
> En un protocolo de capa de transporte: a) ¿Cuál es la diferencia entre Control de Flujo y Control de Congestión? b) ¿Cómo hace TCP para detectar la existencia de congestión en la red? c) ¿La pérdida de paquetes siempre implica congestión? Justifique su respuesta.

Respuestas oficiales (textuales):

> a) "El control de flujo intenta que el emisor no desborde el buffer del receptor transmitiendo muy rápido demasiados datos. El control de congestión intenta evitar que se pierdan paquetes debido a que hay

> demasiadas fuentes enviando muy rápido demasiado tráfico a ser manejado por la red."
>
> b) "TCP asume que la pérdida de paquetes (o grandes retardos) implican congestión."
>
> c) "No. La pérdida de paquetes también puede darse por una capa de enlace defectuosa, por ejemplo."

Parte a) — La diferencia (≈4 puntos)

Se responde con la tabla de §1.1. **El punto clave para el profesor:** los dos se resuelven reduciendo la ventana efectiva, pero por **motivos distintos**, y confundirlos es el error típico. Sumá que el flujo **lo controla el receptor** (y por eso puede anunciarse sin ambigüedad) mientras que la congestión **la tiene que estimar el emisor** (y por eso es una hipótesis, no una medición).

Parte b) — Cómo detecta TCP la congestión (≈3 puntos)

Las dos señales (desarrolladas en la Clase 21):

```
┌──────────────────────────────────────────────────────────┐
│ 1. Timeout (vence el RTO)                                │
│    → no llegó ni el ACK siguiente → pérdida GRAVE o ruta caída│
│                                                          │
│ 2. 3 ACKs duplicados                                     │
│    → el siguiente sí llegó → pérdida AISLADA, la red aguanta  │
└──────────────────────────────────────────────────────────┘
```

La respuesta oficial además menciona **"(o grandes retardos)"**, y eso vale: un RTT que sube mucho indica **cola** en los routers. Pero ojo, el enunciado es explícito: "*TCP asume que...*". Es una **inferencia**, no una notificación. **Con ECN hay una tercera vía**: el router marca CE y el receptor avisa con CONG_DETECTED, que ya no es inferencia sino notificación explícita (Clase 21 §1.6).

Parte c) — ¿Siempre la pérdida implica congestión? (≈3 puntos)

No. Fuentes de pérdida que **no** son congestión:

- **Errores de CRC** en un enlace (ruido, cable malo): el paquete ni siquiera llegó a la cola.
- **Buffer del receptor lleno** (control de flujo): el paquete se descartó aguas abajo, pero la red estaba perfecta.
- **Descarte por ACL / firewall / filtrado** de un router intermedio.
- **Fragmentación descartada** en IPv4 en un router que no tiene espacio para un fragmento.
- **Corte de un enlace** (un cable, no la carga).

La justificación que da puntos: aunque la pérdida *no* siempre sea congestión, **TCP asume que lo es** porque es la apuesta con mejor promedio. Ante la duda entre "la red está saturada" y "fue un error puntual", **bajar la ventana** cuesta rendimiento en el caso "error puntual" pero **salva la red** en el caso "saturada". El error hacia el otro lado (mantener la ventana y saturar la red) es mucho más caro y además causa *bufferbloat* y colapso. La asimetría de costos justifica la apuesta conservadora.

> **Tope para 10 puntos:** la definición de cada uno, la tabla de comparación, las dos señales de detección, y el "no" de (c) con al menos dos fuentes de pérdida no congestiva y la justificación de por qué TCP asume.

3.2 Examen 2010-02, Pregunta 5 (7 puntos) — el mecanismo de control de flujo

> **Fuente:** Examen `sirc1002` — 2 de febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1002.pdf

⚠️ **Nota de método, leela antes de estudiar esto:** el `BANCO-EXAMENES.md` registra `2010-02 P5` como "*jitter vs. retardo*", y el texto extraído del PDF confirma que ése es el tema. **El enunciado de ese apartado NO es sobre control de flujo.** Lo que sigue es una **solución propia** *(solución propia)* del mecanismo de control de flujo de TCP, desarrollada porque es un tema del bloque y porque aparece explícitamente en `2009-07 P1a`.

Respuesta (7 puntos): el control de flujo tiene **un solo campo**, el `window` de 16 bits en la cabecera, que rellena el **receptor** en cada segmento que manda.

- **a) Quién lo controla y cómo se comunica (2 pts).** El **receptor** es el único que sabe cuánto le queda en su buffer. Como no hay forma de que el emisor lo adivine, lo **anuncia** en el campo `window` de cada segmento TCP que envía (ACKs incluidos), y el emisor **lee** ese campo y se autoimpone el límite.
- **b) La fórmula y el sentido (2 pts).** `rwnd = espacio libre en el buffer + bytes recibidos sin entregar`, y `ventana efectiva = min(cwnd, rwnd)`. Es un **mínimo** porque ambas son cotas superiores independientes: sumarlas desbordaría el buffer del receptor justo cuando la red va peor.
- **c) El caso `rwnd = 0` (2 pts).** Si el receptor no lee, su buffer se llena y el `rwnd` llega a 0. TCP entra en **persistencia** (`RFC 9293` §8.6): el emisor manda **zero-window probes** de **1 byte**, con periodo **exponencial** (empieza en un RTO y duplica hasta un tope), hasta que el receptor responda con un `rwnd > 0`. **No es control de congestión**: la red no tiene nada saturado y bajar el `cwnd` no haría que la aplicación del receptor lea más rápido.
- **d) El otro extremo (1 pt).** Simétricamente, si el **emisor** escribe más rápido de lo que la conexión puede sacar, su **buffer de envío** se llena y el `write()` se bloquea (o devuelve `EAGAIN` en modo no bloqueante). Ese es el otro lado del control de flujo, puramente local.

3.3 Conexión con 2011-03 P4 (10 puntos)

> **Fuente:** Examen `sirc1103` — 21 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

En el diagrama clásico de `2011-03 P4` (resuelto por completo en la **Clase 22**, Ejercicio 3), el `cwnd` es lo que **cambia** entre los dos apartados:

- **Sin pérdida:** el `cwnd` **no baja**, y los 4 segmentos salen en un solo vuelo.
- **Con pérdida del primer segmento:** al detectar la pérdida, el `cwnd` **se reduce** (`cwnd/2` en Reno por 3 dupACKs; `cwnd = 1 MSS` en Tahoe o ante timeout). Eso es control de **congestión**, y es la razón por la que los cuatro segmentos **no** salen juntos.

Lo que hay que dejar claro para el examen: en `2011-03 P4a` los 4 segmentos salen juntos porque la ventana **alcanzaba**; en `2011-03 P4b` el primero se pierde, se detecta, y la ventana **ya no alcanza para mandar todo de una**. Un mismo `rwnd` en los dos casos; lo que cambia es el `cwnd`.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Flujo** = el receptor no puede absorber → lo controla `rwnd`. **Congestión** = la red no puede transportar → lo controla `cwnd`.
2. El `rwnd` viaja en el campo `window` (16 bits) de **cada** segmento que manda el receptor.
3. `rwnd = espacio libre + bytes recibidos sin entregar` → es una **cota superior**, no una medida instantánea.
4. `ventana efectiva = min(cwnd, rwnd)`: ambas son cotas, y el `min` garantiza que **nadie se desborda**.
5. **No es la suma**: con la suma, lo que la red no tomó se acumularía en el buffer del receptor y lo desbordaría **justo cuando la red va peor**.
6. `rwnd = 0` → **persistencia** → **zero-window probes** de **1 byte**, periodo **exponencial**.
7. Eso **NO es control de congestión**: la red está perfecta y bajar `cwnd` no haría leer más rápido al otro.
8. El otro lado: si el **emisor** no lee rápido, su buffer de envío se llena y el `write()` se **bloquea**.
9. `cwnd` grande y `rwnd` chico y `cwnd` chico con `rwnd` grande dan **el mismo throughput** y **arreglos distintos**: siempre hay que ver cuál limita.
10. En 2026: `min(cwnd, rwnd)` no cambia; lo que cambió es *Window Scale* (`RFC 7323`), el IW de 10 MSS (`RFC 6928`) y que el `cwnd` lo maneja CUBIC (`RFC 9438`).

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué la ventana efectiva es el **mínimo** y no la suma?

<details>
<summary>▶ Respuesta</summary>
</details>

2. El receptor anuncia `rwnd = 0`. ¿Qué hace el emisor?

<details>
<summary>▶ Respuesta</summary>
</details>

3. Una conexión tiene `cwnd = 64 KiB` y el receptor anuncia `rwnd = 4 KiB`. El throughput medido es `4 192 / 0,100 s = 41 920 B/s`. ¿Qué pasa si el `cwnd` sube a 1 MiB?

<details>
<summary>▶ Respuesta</summary>
</details>

4. ¿Por qué TCP asume que una pérdida de paquetes implica congestión, si no siempre es así?

<details>
<summary>▶ Respuesta</summary>
</details>

5. Nombra dos causas de pérdida de paquetes que **no** son congestión.

<details>
<summary>▶ Respuesta</summary>
</details>

6. ¿Quién decide el `rwnd`? ¿Cómo se lo comunica al emisor? ¿Y por qué el emisor no puede ignorarlo?

▶ Respuesta

7. El `cwnd` es de 8 KiB y el `rwnd` de 1 MiB. El throughput es 1,31 Mbit/s con RTT 50 ms. ¿Se arregla agrandando el `rwnd`?

▶ Respuesta

8. ¿Qué cambió entre 2011 y 2026 en este tema?

▶ Respuesta

🔗 Referencias

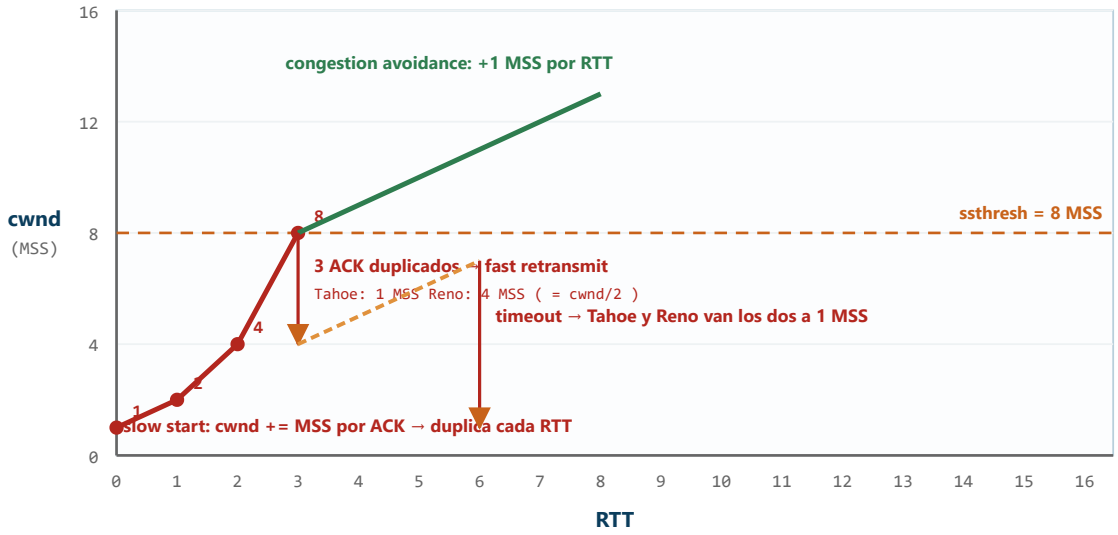
- **Examen** `sirc0907` — P1 (10 pts): flujo vs. congestión, detección de congestión, y si toda pérdida es congestión
- **Examen** `sirc1002` — P5: jitter vs. retardo (el `BANCO-EXAMENES.md` lo registra así; §3.2 desarrolla el control de flujo como solución propia)
- **Examen** `sirc1103` — P4 (10 pts): el diagrama de 4 segmentos y el papel del `cwnd`
- **Examen** `sirc0912` — P2 (10 pts): fiabilidad en transporte vs. enlace
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 §6.7 "TCP Congestion Control" (pág. ~600 del PDF)
- `RFC 9293` — TCP (2022, *Internet Standard*); §8.6 persistencia / *zero-window probes*
- `RFC 7323` — Timestamps y Window Scale (2014, obsolesce `RFC 1323`)
- `RFC 6928` — Initial Window = 10 MSS (2013) · `RFC 5681` — TCP Congestion Control (2009)
- `RFC 9438` — CUBIC (2023, *Standards Track*, obsolesce `RFC 8312`)
- `RFC 9000` — QUIC (2021): control de flujo por conexión y por stream
- **Clase siguiente:** 21 — Control de congestión en TCP: slow start, AIMD, fast recovery y ECN

Clase 21

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El problema que TCP tiene que resolver
1.2 Las dos ventanas y la ventana efectiva
1.3 Las tres variables de estado
1.4 Los cuatro algoritmos (Tahoe, 1988)
1.5 Diagrama de la evolución de `cwnd`
1.6 ECN — Explicit Congestion Notification
1.7 ¿Y el Initial Window?
1.8 Una mención a TCP Vegas y a lo que se usa hoy
2.1 Ejercicio 1 — La curva completa de `cwnd` (Tahoe, MSS = 1 KB)
2.2 Ejercicio 2 — La ventana efectiva
2.3 Ejercicio 3 — ECN en números
2.4 Ejercicio 4 — Tipo examen nuevo: control de congestión con RTT variable
3.1 ⭐ Examen 2011-03, Problema 1 (30 puntos) — ECN y control de congestión
3.2 Examen 2008-07, Pregunta 2 (10 puntos) — los cuatro algoritmos
3.3 Examen 2009-07, Pregunta 1 (10 puntos)
3.4 Examen 2009-12, Pregunta 2 (10 puntos)
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Evolucion de la ventana de congestion



Tahoe vuelve a 1 MSS ante cualquier perdida.
Reno solo cae a cwnd/2 con 3 ACK duplicados; ante un timeout cae igual que Tahoe.

Bloque: C (Transporte) · **Semana:** 7 · **Duración estimada:** 110 min **Correlativa:** Clase 20 (control de flujo vs. de congestión) · **Sigue con:** Clase 22 (taller TCP) **Material de referencia:** Tanenbaum cap. 6 (§6.7) · RFC 9293, RFC 5681, RFC 9438, RFC 3168, RFC 9768, RFC 9331, RFC 6928

🎯 Al terminar esta clase vas a poder

1. Explicar por qué TCP necesita un mecanismo de control de congestión **separado** del control de flujo, y por qué el control de flujo no alcanza.

2. Escribir la **ventana efectiva** de un segmento TCP como el mínimo entre `cwnd` y `rwnd`, y justificar por qué es un mínimo y no una suma.
3. Recitar los **cuatro algoritmos** del control de congestión de TCP (Tahoe) y ejecutar la evolución de `cwnd` durante una conexión completa, RTT por RTT.
4. Explicar por qué el **timeout** y los **3 ACKs duplicados** se tratan distinto, y qué hace cada uno.
5. Describir el mecanismo de **ECN**: qué son los 2 bits del campo ECN, qué hace cada router y qué hace cada extremo.
6. **Implementar en pseudocode** la detección y el control de congestión, incluyendo ECN, respetando las primitivas que da el enunciado.

> **Prerequisito del examen:** `2011-03 Problema 1` (30 puntos — el problema de ECN), `2008-07 P2` (10 pts), `2009-07 P1` (10 pts), `2009-12 P5`, `2008-12 P2`, `2010-02 Problema 2`.

PARTE 1 — TEORÍA

1.1 El problema que TCP tiene que resolver

Imaginá que activás una descarga. Tu disco de 500 Mbps no te sirve de nada si el enlace de la casa te da 10 Mbps. Y al revés: un enlace de 100 Mbps que te comparte con 40 personas, los otros 39, tu descarga de 40 Mbps te va a parecer lenta.

Hay **dos** problemas distintos, y no son el mismo:

```
┌─ El emisor manda MÁS RÁPIDO de lo que el receptor puede absorber ─┐
│ → CONTROL DE FLUJO │
│ (lo arregla rwnd, clase anterior) │
└──────────────────────────────────────────────────────────────────┘

┌─ La RED en el medio no tiene recursos para lo que todos mandan ──┐
│ → CONTROL DE CONGESTIÓN │
│ (lo arregla cwnd, esta clase) │
└──────────────────────────────────────────────────────────────────┘
```

La diferencia de fondo:

	Control de flujo	Control de congestión
El problema es de	un host lento	**toda la red**
El síntoma es	el buffer del receptor se llena	los buffers de **los routers** se llenan
La pérdida significa	"no leíste rápido"	"**la red está saturada**"
La ventana la controla	el receptor (rwnd)	el emisor, estimando (cwnd)
Se arregla con	ACK diferidos / zero-window probe	**reducir el ritmo de envío**

Y acá está el punto clave para el examen: TCP **no puede distinguir** una pérdida por buffer lleno del receptor (control de flujo) de una pérdida en la red (congestión). Para TCP, un paquete perdido es un paquete perdido. Por eso el mismo mecanismo (disminuir la ventana) sirve para las dos cosas, y por eso la "interpretación" que TCP hace es conservadora.

La evidencia de que la red se congestiona

TCP detecta tres señales, en orden creciente de gravedad:

1. **No hay pérdida, pero el RTT subió mucho** → alguien está encolando. No es un evento, TCP no reacciona formalmente.
2. **3 ACKs duplicados** → hubo una pérdida, pero el resto de la ventana llegó igual. Mildeprioridad: pérdida aislada.
3. **Timeout (RTO expirado)** → la pérdida es *grave*: no llegó ni siquiera el ACK del siguiente. La ventana entera está frenada.

Esta distinción es la que te van a preguntar, y es la clave del problema de 2011.

1.2 Las dos ventanas y la ventana efectiva

El emisor TCP manda como máximo esta cantidad de bytes sin esperar ACK:

```
bytes en vuelo ≤ min(cwnd, rwnd)
               ──────────  ────
               congestión  flujo
```

- `cwnd` **(congestion window):** la **hipótesis del emisor** sobre cuánta congestión hay. Empieza en un valor pequeño y la va ajustando.
- `rwnd` **(receive window):** lo que el receptor dice que le entra en su buffer. Va en el campo *window* de cada segmento TCP que manda.

¿Por qué el mínimo y no la suma? Porque **el emisor no puede mandar más de lo que la red aguanta** (cwnd), ni **más de lo que el receptor aguanta** (rwnd). Ambas son restricciones *superiores*. Si las sumaras, un solo paquete perdido por congestión en la red se acumularía en el buffer del receptor y lo desbordaría justamente cuando la red va peor — que es cuando menos conviene. El `min` es el que garantiza que **nadie se desborda**: ni la red (por cwnd), ni el receptor (por rwnd).

En los exámenes los dos los llaman **"ventana efectiva"** o **"ventana de congestión"** según el caso. Si el enunciado habla de una sola ventana, asumí que `rwnd` es grande y el límite es `cwnd`.

1.3 Las tres variables de estado

```
cwnd = ventana de congestión actual, en bytes. Empieza en un MSS (o un
Initial Window del RFC 6928, ver Actualización).
ssthresh = "slow start threshold", el umbral a partir del cual se pasa de
slow start a congestion avoidance. Empieza muy alto.
dupACK = contador de ACKs duplicados quejetsnorth se recibieron seguidos.
```

`ssthresh` es la variable que decide la regla:

> Si `cwnd < ssthresh` → **slow start**. Si `cwnd >= ssthresh` → **congestion avoidance**.

La pérdida no es lo único que cambia de régimen: cuando `cwnd` *crece* y supera el umbral, también se cambia de régimen.

1.4 Los cuatro algoritmos (Tahoe, 1988)

(a) Slow start — "arranque lento"

No es lento en la velocidad: es lento en **el crecimiento**.

```
Por cada ACK recibido:
cwnd ← cwnd + 1 MSS (por ACK, no por RTT)
```

O sea: **cada segmento confirmado duplica la ventana**:

```
ACK de 1 MSS recibido → cwnd = 1 + 1 = 2 MSS
ACK de 1 MSS recibido → cwnd = 2 + 1 = 3 MSS
...
```

Después de un RTT completo:

```
RTT 1: cwnd = 1 MSS (mandás 1 MSS, lo confirmás)
RTT 2: cwnd = 2 MSS (mandás 2, los 2 ACKs te suman 2)
RTT 3: cwnd = 4 MSS (mandás 4, los 4 ACKs te suman 4)
RTT 4: cwnd = 8 MSS
...
```

Es decir **exponencial en base 2**: `cwnd(n) = 2^(n−1) · MSS`. Por eso se llama "arranque lento": en una red con RTT de 100 ms pasás de 1 MSS a 64 KB en 7 RTT, o sea en 0,7 segundos.

¿Por qué "slow" y no "start"? Porque en la versión clásica el emisor arrancaba mandando a toda velocidad y el primer pico de congestión le costaba la mitad de la ventana. Empezar en 1 MSS es "perezosamente" aceptar que no conoce la red.

(b) Congestion avoidance — "evitación de la congestión"

Se entra cuando `cwnd >= ssthresh`. Ahora el crecimiento es **lineal**:

```
Por cada ACK recibido:
cwnd ← cwnd + MSS · MSS / cwnd (un MSS por RTT)
```

Escribido así parece raro, pero es equivalente a la forma más conocida:

```
cwnd ← cwnd + MSS por cada RTT
```

La forma `MSS²/cwnd` es la que aparece **literalmente** en la solución oficial del examen de 2011-03, y es la que te van a pedir. La justification: cada ACK confirma `MSS` bytes, así que el emisor puede mandar `MSS` bytes más; repartido eso entre los `cwnd/MSS` ACKs que llegan en el RTT, da `MSS / (cwnd/MSS) = MSS²/cwnd` por ACK. Sumado a lo largo de un RTT, `cwnd/MSS · MSS²/cwnd = MSS`.

(c) Fast retransmit

Cuando llegan **3 ACKs duplicados**, no esperás al timeout:

```
3 ACKs duplicados → retransmitir el segmento perdido YA,
sin esperar a que venza el RTO
```

¿Por qué 3 y no 1? Porque con ACKs **acumulativos**, un ACK duplicado puede significar dos cosas:

- El segmento que esperaba **realmente** se perdió (pérdida), o
- Simplemente el ACK se está **reordenando** en la red y el que "falta" llegó después (reordenamiento).

Con 3 duplicados es estadísticamente muy improbable que sea reordenamiento, así que se asume pérdida. **Esto es un compromiso entre latencia y false positives**, y es exactamente lo que te preguntan en `2008-07 P2`.

(d) Fast recovery

El refinamiento: con 3 dupACKs **no se tira la ventana a 1 MSS**, porque los datos que vienen detrás son la señal de que la red **aguanta** ese ritmo.

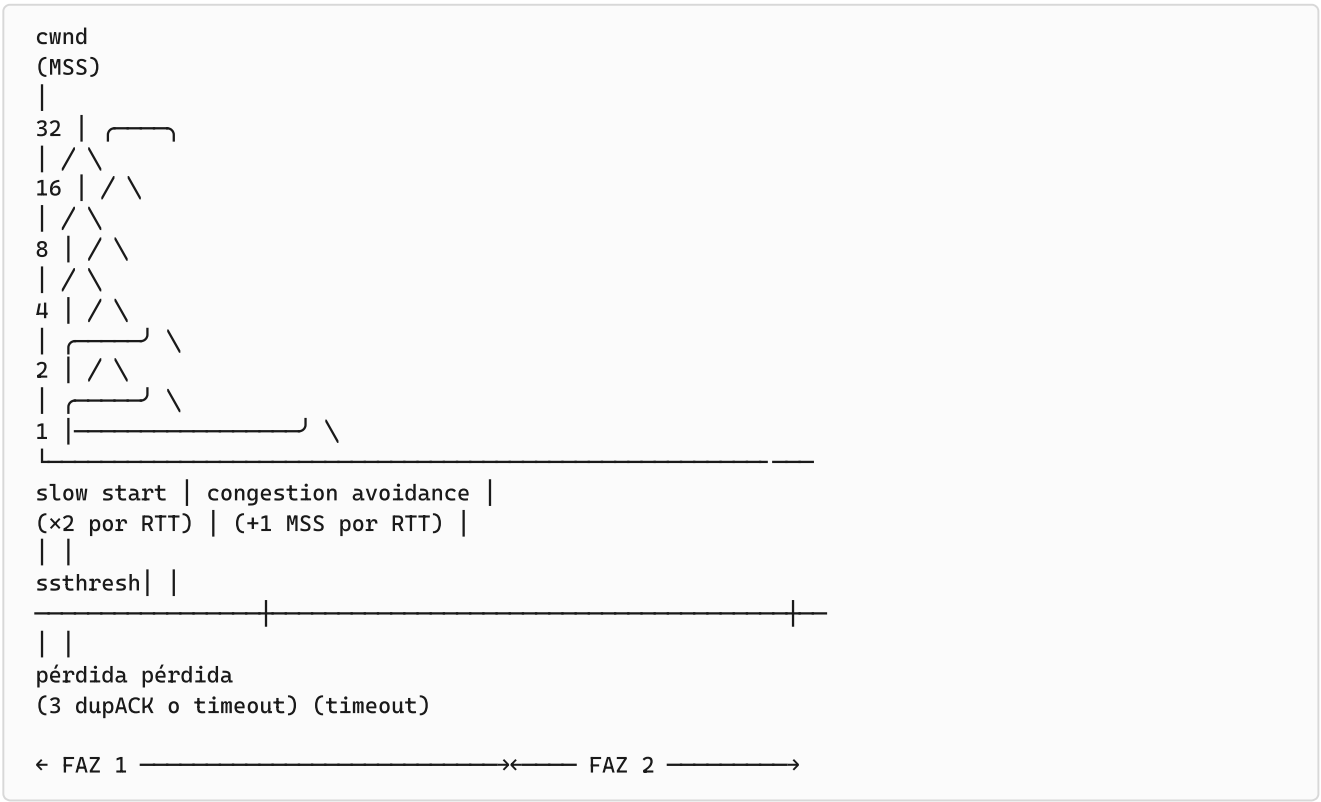
```
3 dupACKs → cwnd = ssthresh (se parte por la mitad, no a 1 MSS)
ssthresh = cwnd / 2
retransmitir el perdido
→ seguir acknowledging, sin cortar la ventana a 1 MSS
Timeout → cwnd = 1 MSS (ESTO sí es Tahoe puro)
ssthresh = cwnd_antes / 2
→ slow start
```

La asimetría es el punto clave y lo que más se confunde:

```
| 3 dupACKs | Timeout
──────────┼──────────────┼──────────────────
Interpretación|red:tensa |red:muerta
cwnd nuevo | cwnd/2 | 1 MSS
Régimen | (ev. cong.) | slow start
Latencia | baja (inmediata) | alta (espera RTO)
```

Tahoe es el que hace `cwnd = 1 MSS` en los dos casos. **Reno** (1989) es el que distingue. Los exámenes de FING por lo general describen **Tahoe** (cuatro algoritmos, `cwnd = 1 MSS` ante cualquier pérdida) o la distinción, y en la solución de `2011-03` te piden slow start para los dos casos. Leelo bien.

1.5 Diagrama de la evolución de `cwnd`



Leelo así: **el doble en la primera fase, la recta en la segunda, y el salto a cero en cada pérdida**. Ese gráfico es el 80% de la respuesta a `2008-07 P2`.

1.6 ECN — Explicit Congestion Notification

El problema que resuelve

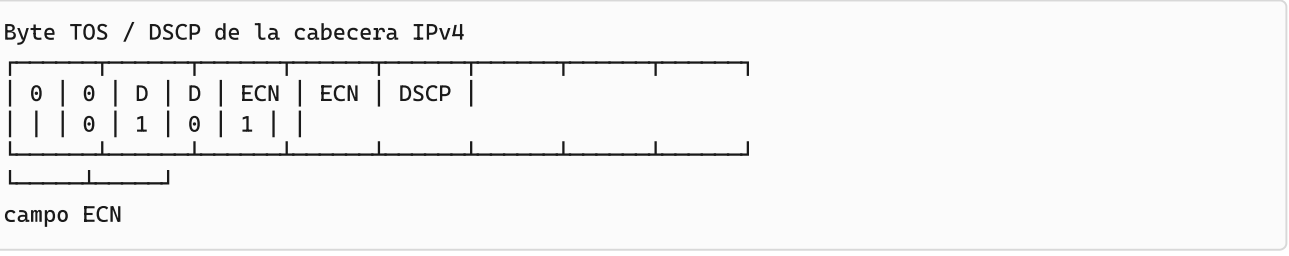
La detección de congestión de TCP es **reactiva**: alguien tiene que perder un paquete para que TCP se entere. Eso significa:

1. **Se pierde un paquete que no hacía falta perder.** Ese paquete se iba a descartar igual, así que no es una pérdida "de verdad"… pero para la aplicación que lo esperaba, sí lo es.
2. **Los routers intermedios no participan.** TCP deduce congestión de una pérdida, pero el que sabe que está congestionado es el **router** que threwó el paquete. Y no se lo dice.

La solución

ECN (RFC 3168, 2001): que el router **no descarte** el paquete sino que le **ponga una marca**, y que TCP reaccione a la marca en lugar de a la pérdida.

Hay **2 bits** en la cabecera IP, en el byte TOS/DSCP:

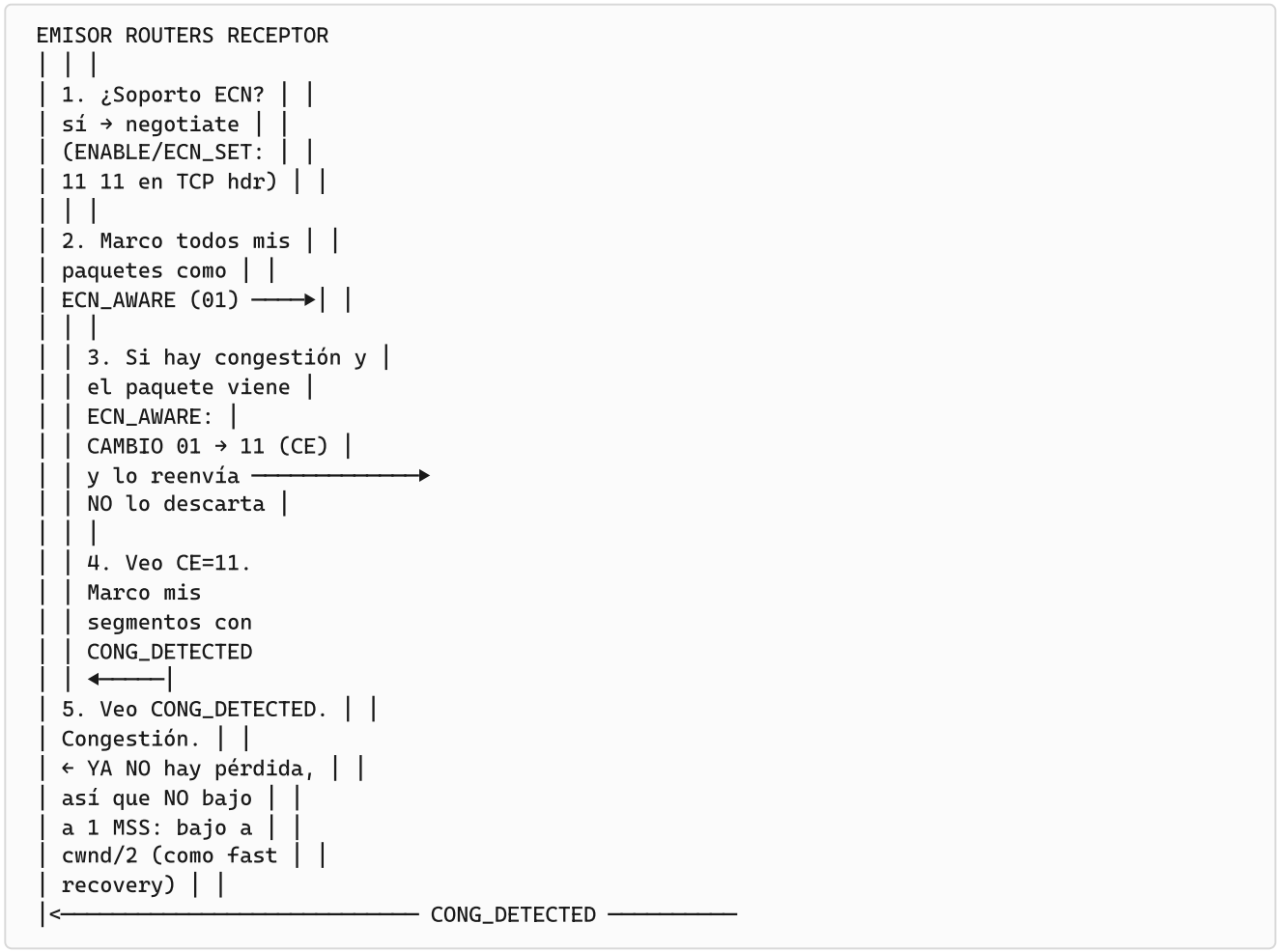


Los 2 bits del campo ECN (antes DSCP) toman 3 valores:

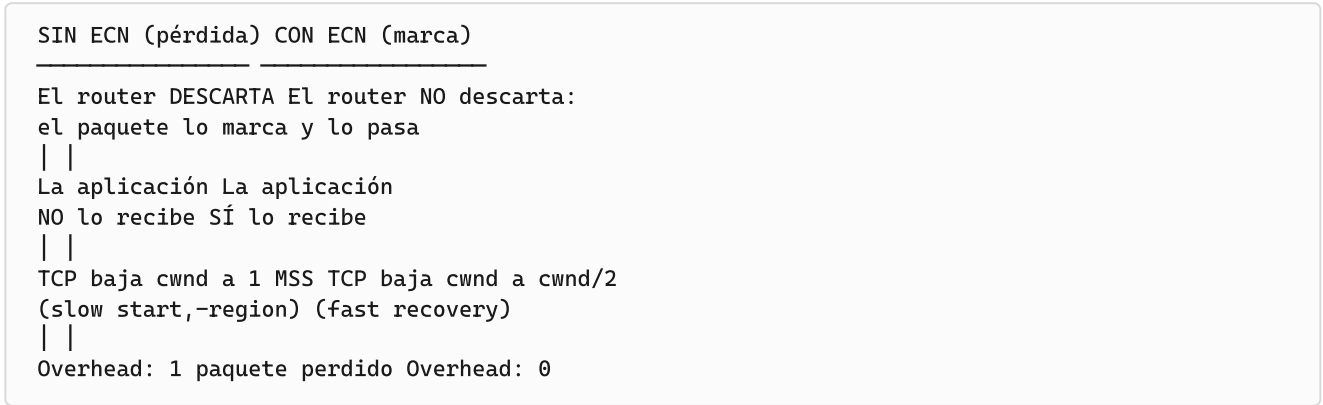
Valor	Bits	Significado
`00`	0-0	**NO_ECN** — el origen no sabe usar ECN
`01`	0-1	**ECN_AWARE** — "sé usar ECN, pero ahora no hay congestión"
`11`	1-1	**ECN_CONG** (= **CE**, *Congestion Experienced*) — "**hubo congestión**"
`10`	1-0	**CE**, pero originated por un router que no entiende ECN (legado)

> ⚠️ **Ojo:** la combinación `10` es válida **solo** en el paquete que viaja en la **dirección de vuelta**. `10` no lo puede poner un origen, porque implicaría que un host entendió ECN y sin embargo no negoció ECN Setup. Es un detalle sutil que aparece en la RFC y que un profesordetail-oriented puede preguntar.

El protocolo, extremo por extremo



ECN vs. pérdida: la diferencia clave



Eso es ECN en su forma más pura: **cero paquetes perdidos a cambio de un bit**.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - Los **2 bits** del campo ECN y sus 3 (4) valores `00/01/11/10`. - La secuencia de **4 pasos**: emisor marca ECN_AWARE → router cambia a CE → receptor marca CONG_DETECTED → emisor reduce cwnd. - La idea de que **ECN evita la pérdida** y por eso permite una reducción de ventana **menos agresiva**. - `CE` = *Congestion Experienced* = `ECN_CONG`.

Cambió o se modificó — y acá hay que ser preciso, porque es donde más se matan las clases:

- `RFC 7560` **NO es ECN++.** `RFC 7560` (2010, *Informational*) es solo el **documento de problema y requisitos** de "ECN más preciso". No especifica ningún mecanismo. Mucha gente lo cita mal como si fuera la especificación.
- `RFC 8951` **NO tiene nada que ver con ECN.** Es una aclaración de **EST (Enrollment over Secure Transport)**. No la invoques nunca en este contexto.
- **La implementación real de "ECN++" quedó en** `RFC 9768 – AccECN` (*More Accurate Explicit Congestion Notification Feedback in TCP*), que además **actualiza** la RFC 3168. *Verificá el número en* `rfc-editor.org/info/rfc9768` *antes de citarlo en un examen: es de 2026 y podría haber cambiado.*
- **La reacción ante un CE ya no es "bajar a 1 MSS".** La especificación moderna pide una reducción **proporcional y menos agresiva** — esencialmente la misma que ante fast recovery (`cwnd/2`) — porque el paquete **no se perdió**. Ese argumento conceptual es lo que hay que saber; el número exacto, menos.
- **L4S (** `RFC 9331` **, 2023)** es el paso siguiente: en vez de esperar a llenarse el buffer, el router marca **con un umbral mucho más bajo**, lo que da latencia muy baja sin necesitar descartes. Va de la mano con AQMs como **CoDel**.
- `ABE` **(** `RFC 8511` **, 2018)** — *Alternative Backoff with ECN* — cómo hacer el backoff cuando el receptor también sufrió pérdida por el CE.
- **Despliegue real:** ECN se **negocia**, así que solo se usa si **ambos** extremos lo soportan. Está activo por defecto en Linux (desde el kernel 4.1) y en Windows 10. **No es obligatorio**: no te lo pregunten como si lo fuera.

1.7 ¿Y el Initial Window?

La RFC 5681 decía "arrancá con 1 MSS". En la práctica todos arrancaban con distintos valores (Solaris 8 KB, Linux 16 KB…), lo que rompía la independencia de la red.

RFC 6928 (2013) — Increasing TCP's Initial Window (IW): estandariza un IW de **10 segmentos (IW = 10 · MSS)**. Es el valor por defecto en Linux, Windows y macOS hoy.

> **Consecuencia para el examen:** si el enunciado dice "slow start empieza en cwnd = 1 MSS", usá 1 MSS (es lo que dicen los apuntes y las soluciones oficiales de 2011). Si dice "IW por defecto actual", es 10 MSS (RFC 6928). **Citá las dos cosas** y seguí lo que pida el enunciado: lo que se corrige es lo que dice el examen.

1.8 Una mención a TCP Vegas y a lo que se usa hoy

- **TCP Vegas (1996):** no espera a la pérdida. Mide el RTT y lo compara con el RTT mínimo teórico (`RTT_min + L/R`). Si la diferencia crece, hay cola → baja cwnd. **Ventana de congestiónpredictiva**, sin pérdidas.
- **BBR (Google):** modelo de **bottleneck bandwidth** y RTT mínimo, no de ventana. Mucho mejor en rutas con pérdida no relacionadas con congestión (WiFi, buffers intermedios). Implementado en Linux, en draft de IETF (no RFC todavía).
- **CUBIC (RFC 9438, 2023):** el default en Linux, Windows y Apple. **No fue reemplazado** — lo que pasó es que pasó de *Informational* (RFC 8312, 2018) a *Standards Track*. A diferencia de AIMD, en lugar de sumar 1

MSS por RTT usa una curva cúbica que **apunta al último punto donde hubo pérdida**, lo que permite "resetear" la info de la red mucho más rápido en enlaces con BDP grande.

⚠️ **Para el examen:** el "AIMD" y los "cuatro algoritmos" **siguen siendo lo que se enseña y lo que se pregunta**, porque TCP Reno/Tahoe es la base de todos los demás. Si el enunciado te pide "el mecanismo de control de congestión de TCP", respondé slow start / congestion avoidance / fast retransmit / fast recovery, y si hay lugar, una línea de "hoy se usa CUBIC o BBR, pero la lógica es la misma".

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La curva completa de `cwnd` (Tahoe, MSS = 1 KB)

Datos. MSS = 1000 bytes. `cwnd` inicial = 1 MSS = 1000 B. `ssthresh` inicial = 64 KB = 64 MSS. En el RTT 5 se pierde un paquete y se detectan 3 ACKs duplicados.

a) Calculá `cwnd` al final de cada RTT del RTT 1 al 4.

Razonamiento. En slow start el crecimiento es `cwnd = 2 · cwnd_anterior` por RTT. La clave es **por qué**: en el RTT *n* mandás `cwnd(n−1)` bytes; llegan esos tantos ACKs; cada ACK suma 1 MSS; entonces `cwnd(n) = cwnd(n−1) + cwnd(n−1) = 2·cwnd(n−1)`. O sea, **el número de ACKs que recibo en un RTT es igual a la ventana que tenía**.

```
RTT 1: cwnd_enviado = 1 MSS → llega 1 ACK → cwnd = 1 + 1 = 2 MSS
RTT 2: cwnd_enviado = 2 MSS → llegan 2 ACK → cwnd = 2 + 2 = 4 MSS
RTT 3: cwnd_enviado = 4 MSS → llegan 4 ACK → cwnd = 4 + 4 = 8 MSS
RTT 4: cwnd_enviado = 8 MSS → llegan 8 ACK → cwnd = 8 + 8 = 16 MSS
```

RTT	cwndbeginning	ACKs recibidos	cwnd al final
1	1 MSS	1	**2 MSS** = 2 000 B
2	2 MSS	2	**4 MSS** = 4 000 B
3	4 MSS	4	**8 MSS** = 8 000 B
4	8 MSS	8	**16 MSS** = 16 000 B

b) ¿En qué RTT se deja de estar en slow start?

Razonamiento. La regla es `cwnd < ssthresh` → slow start. Con `ssthresh = 64 MSS`: el slow start termina cuando `cwnd` llega a 64. Desde 16 hay que seguir duplicando: 32, luego 64. Como el umbral se alcanza al final del RTT 6, y la comparación es `cwnd >= ssthresh`, **el RTT 7 ya corre en congestion avoidance**.

```
RTT 5: 32 MSS (< 64 → slow start)
RTT 6: 64 MSS (== 64 → a partir de acá es congestion avoidance)
RTT 7: 65 MSS (+1 por RTT)
```

corríjanme si lo leés distinto: algunos apuntes dicen "termina en el RTT 6" (dejando de crecer cuando llega al umbral) y otros "en el RTT 7" (cuando el crecimiento pasa a ser lineal). **Lo que se corrige es el mecanismo**, y el mecanismo es la comparación `cwnd` vs. `ssthresh`. Decí las dos cosas.

c) En el RTT 5 hay 3 ACKs duplicados. ¿Qué pasa?

```
ANTES: cwnd = 32 MSS = 32 000 B
ssthresh = 64 MSS

Detección: 3 ACK duplicados → fast retransmit

Tahoe → ssthresh = cwnd/2 = 16 MSS
cwnd = 1 MSS
→ slow start de nuevo

Reno → ssthresh = cwnd/2 = 16 MSS
cwnd = ssthresh = 16 MSS
→ fast recovery
```

Interpretación. Con 3 dupACKs, Tahoe tira 32 KB de ventana a 1 KB. Eso es **castigar a la red** por una pérdida que ella no habríaisoftido si los buffers fueran más grandes. Reno dice: "los 32 KB igual circularon, o sea que la red los aguantó; bajemos a la mitad, no a cero".

d) Calculá `cwnd` en los 5 RTT siguientes, con Reno.

```
RTO/ACK : cwnd = 16 → 17 → 18 → 19 → 20 → 21 (congestion avoidance,
+1 MSS por RTT)
```

Y **con Tahoe**, en slow start desde 1:

```
RTT 1: 1→2 RTT 2: 2→4 RTT 3: 4→8 RTT 4: 8→16 RTT 5: 16→17 (¡superó ssthresh!)
```

2.2 Ejercicio 2 — La ventana efectiva

Datos. `cwnd = 8 KB`, `rwnd` anunciado por el receptor = 4 KB. RTT = 50 ms.

a) ¿Cuántos bytes puede mandar como máximo?

Razonamiento. Las dos son cotas superiores. La red admite 8 KB, el receptor 4 KB. Mandar 8 KB haría que 4 KB se acumulen en el buffer del receptor, que es justo lo que el rwnd existe para evitar.

```
ventana efectiva = min(cwnd, rwnd) = min(8 KB, 4 KB) = 4 KB
```

b) ¿Cuál es el throughput máximo?

Razonamiento. El throughput no es "bytes / RTT" sin más: es **cuántos bytes salen por segundo**. Si mando 4 KB cada 50 ms:

```
throughput = 4 KB / 0,050 s = 4000 / 0,050 = 80 000 B/s = 80 KB/s
= 640 kbps
```

c) Si el receptor no lee durante 1 segundo, ¿qué pasa con `rwnd`?

Razonamiento. El receptor no manda ACK, entonces el emisor no avanza la ventana de bytes confirmados. Cuando `rwnd` llega a 0:

```
R0 receptor anuncia rwnd = 20 KB, manda 20 KB
R1 sin ACK → el emisor ve rwnd = 20, ventana efectiva = min(cwnd, 20) = 20
R2 sin ACK → rwnd = 0 → ventana efectiva = 0 → NO PUEDE ENVIAR
```

TCP entra en **persistencia** (RFC 9293 §8.6) y manda **zero-window probes** (1 byte, con periodo exponencial) para preguntar "¿podés volver a leer?". Cuando el receptor vuelve a leer, manda un ACK con un `rwnd` nuevo y se reanuda.

> ⚠️ **Esto NO es control de congestión.** Es control de flujo, y es importantísimo no confundirlos: acá la red está perfecta; lo que está lento es la aplicación del otro lado. Si TCP bajara cwnd por esto, estaríamos castigando a la red por un problema del receptor. **Por eso son mecanismos separados.**

2.3 Ejercicio 3 — ECN en números

Datos. `cwnd = 20 MSS`, `ssthresh = 30 MSS`. Un router marca CE=11 un paquete.

a) ¿Qué haría TCP sin ECN? Tendría que **perder** ese paquete. 3 dupACK → Reno: `cwnd = 20/2 = 10`, `ssthresh = 10`. Tahoe: `cwnd = 1`.

b) ¿Qué hace con ECN (ECN++)?

```
Con ECN++ → cwnd = (20 − 1)/2 ≈ 9,5 ... pero lo que se usa es la misma
fórmula de fast recovery: cwnd = cwnd/2 = 10 MSS
ssthresh = 10 MSS
```

c) La diferencia, en números:

	Sin ECN	Con ECN
Paquetes perdidos	1	**0**
`cwnd` después	10 MSS (Reno)	10 MSS
`ssthresh` después	10 MSS	10 MSS

Interpretación — y acá está la moraleja. El mecanismo de reacción de TCP es **el mismo**. Lo que ECN cambia no es cuánto baja la ventana: es que **el paquete no se pierde**. Es un ejemplo perfecto de que en redes la pérdida de un paquete es un phenomenon diferente del phenomenon de congestión, aunque TCP las confunda.

2.4 Ejercicio 4 — Tipo examen nuevo: control de congestión con RTT variable

Enunciado. Un cliente TCP tiene `cwnd = 12 KB`, `ssthresh = 20 KB`, RTT inicial de 100 ms. En cada RTT manda `cwnd` bytes. En el RTT 4, un paquete se pierde y se detectan 3 ACKs duplicados. En el RTT 5 se detecta una pérdida por timeout.

a) `cwnd` y `ssthresh` al final de cada RTT, aplicando Tahoe.

RTT	`cwnd` inicio	Régimen	`cwnd` fin	`ssthresh`
1	12 KB	slow start (`12<20`)	24 KB	20 KB
2	24 KB	cong. avoid. (`24≥20`)	25 KB	20 KB
3	25 KB	cong. avoid.	26 KB	20 KB
4	26 KB	3 dupACK → pérdida	**1 KB**	**13 KB**
5	1 KB	timeout → pérdida	**1 KB**	**0,5 KB**

Justificación del RTT 4: `ssthresh = cwnd/2 = 26/2 = 13 KB`; `cwnd = 1 KB`. *Justificación del RTT 5:* timeout ⇒ Tahoe también va a 1 MSS, y `ssthresh = 1/2 = 0,5 KB`.

b) ¿Qué cambiaría con Reno?

```
RTT 4 (3 dupACK): ssthresh = 13 KB, cwnd = 13 KB (fast recovery)
RTT 5 (timeout): ssthresh = 0,5 KB, cwnd = 1 KB (slow start, igual que Tahoe)
```

c) En el RTT 2 el RTT subió de 100 ms a 180 ms. ¿Qué indica eso y qué debería hacer un diseño moderno?

Respuesta. Indica **cola** en los buffers de los routers: el paquete tardó más porque **estuvo esperando su turno**. El paquete **no se perdió**, así que los algoritmos basados en pérdida (Tahoe, Reno, CUBIC) **no reaccionan**: sigue creciendo `cwnd`.

Esto es un problema conocido como **bufferbloat**. Las soluciones modernas son:

- **ECN**: el router marca en vez de tirar (o hacer cola). Reacciona **antes** de la pérdida.
- **AQM (*Active Queue Management*)**: el router no espera a llenarse el buffer, marca o descarta temprano. **CoDel**, **FQ-CoDel** (por defecto en Linux desde 3.5), **PIE**. Unaim: "Bufferbloat — Better Bufferbloat" (RFC 9000).
- **AccECN**: la reacción menos agresiva.

> ⚠️ **Actualización a 2026:** el **bufferbloat** y los AQMs son de lo más relevante que se agregó después de 2011 en este tema. Linux usa **fq_codel** por defecto desde el kernel 3.5 (2012). RFC 9000 es **BBR** (draft→ estándar informativo).

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-03, Problema 1 (30 puntos) — ECN y control de congestión

> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf **Este es el problema más importante de toda la materia para el examen actual.**

Enunciado (texto del examen)

> ECN (Explicit Congestion Notification) es un protocolo que actúa en capa de red y capa de transporte que permite que los hosts de una sesión TCP reciban notificaciones explícitas de congestión generadas por los routers de la red que reenvían los paquetes de dicha sesión. Un paquete IP puede estar marcado como: NO_ECN, ECN_AWARE o ECN_CONG.
>
> TCP puede activar ECN solicitando a su entidad de red marcar la bandera ECN_AWARE del encabezado IP de todos los paquetes que trasportan segmentos de dicha sesión. En caso de congestión, los routers deben marcar la bandera ECN_CONG del encabezado IP en todos aquellos paquetes marcados con ECN_AWARE antes de reenviarlos. Cuando el receptor recibe paquetes marcados con la bandera ECN_CONG le notifica al emisor que existe congestión marcando la bandera CONG_DETECTED del encabezado de los segmentos TCP.
>
> Primitivas: - `void setCongWinSize(int valor)` / `int getCongWinSize()` - `void setCONGReceived()` : Marca los próximos segmentos TCP a enviar con la bandera CONG_DETECTED.
>
> Tipos: `TCP_Segment{ int destPort, srcPort, seqNum, ackSeqNum void[] data bool cong_detected IP_Packet prev_header ... } IP_Packet{ ip_addr src, dst enum_ecn ecn_field // {NO_ECN, ECN_AWARE, ECN_CONG} ... }`
>
> **a)** Cuando el emisor recibe un segmento TCP con la bandera CONG_DETECTED marcada, interpreta dicha situación como congestión. ¿Qué eventos interpreta TCP sin ECN como una señal de congestión en la red?
>
> **b)** Considere una implementación de TCP que carga el segmento TCP recibido en una estructura de tipo `TCP_Segment` que se usa como parámetro para invocar la función `int TCP_congDetection(TCP_Segment segmento)` cuya salida es el estado de congestión. Implemente dicha función considerando los mecanismos tradicionales de detección de congestión de TCP y las mejoras provistas por ECN (si se invoca `TCP_congDetection` con NULL como parámetro, es porque existió un timeout para la recepción de un ACK).
>
> **c)** TCP utiliza la función `TCP_congDetection` y le pasa el resultado a la función `void TCP_congControl(Int congStatus)` que implementa el control de congestión tradicional de TCP y los mecanismos necesarios para utilizar ECN. Implemente dicha función considerando que una vez detectada la congestión se pasa al estado "slow start" (arranque lento).

Parte a) — La respuesta son 4 palabras

Pregunta qué hace TCP sin ECN. La respuesta oficial:

> TCP interpreta que hay congestión en la red cuando se da un evento de pérdida de paquete. Un evento de pérdida puede ser por timeout en la recepción de un ACK o por la recepción de 3 ACKs duplicados.

Por qué esos dos y no otros. Ambos son **evidencias de pérdida**, y la pérdida es la única señal que TCP puede interpretar sin que nadie se lo diga:

Evento	Qué evidencía	Gravedad
Timeout (RTO vencido)	no llegó ni el ACK del siguiente → pérdida **masiva** o ruta caída	alta
3 ACKs duplicados	el siguiente llegó, pero el esperado no → pérdida **aislada**	baja
ACK llega "normal"	nada	ninguna
RTT subió	**cola**, no pérdida	ninguna (TCP clásico no reacciona)

La comparación con ECN (que es lo que la pregunta quiere):

```
SIN ECN: el router DESCARTA → hay pérdida → TCP reacciona a la pérdida
CON ECN: el router MARCA CE → no hay pérdida → TCP recibe CONG_DETECTED
```

O sea: **con ECN, TCP deja de inferir y pasa a serCournotificado**. Esa es la frase para el examen.

> **Tope para 6 puntos:** nombrá los dos eventos (pérdida por timeout, pérdida por 3 ACKs duplicados), explicá que ambos son señales de **pérdida de paquete**, y contrastá con que ECN sustituye esa inferencia por una **notificación explícita** que evita la pérdida.

Parte b) — `TCP_congDetection`

Solución oficial (del PDF):

```
int ultimo_ack, contador = 0;
int TCP_extractCongData(TCP_Segment paquete){
 int estadoCongestion = 0;
 if (paquete == NULL){
 contador = 3;
 }
 else if (paquete.ackSeqNum == ultimo_ack){
 contador++;
 }
 else if (paquete.ackSeqNum > ultimo_ack){
 ultimo_ack = paquete.ackSeqNum;
 contador = 0;
 }

 if (contador == 3) {
 estadoCongestion = 1;
 contador = 0;
 }

 if (paquete != NULL and paquete.CONG_DETECTED) {
 estadoCongestion = 1;
 }
 if (paquete != NULL and paquete.prev_header.ecn_field == ECN_CONG){
 setCONGReceived();
 }
 return estadoCongestion;
}
```

Análisis de la solución oficial, punto por punto. Esto es lo que hay que aprender: no solamente copiar, sino entender qué está bien y qué está raro.

✅ **Lo que está bien:**

1. `if (paquete == NULL) contador = 3;` — **el timeout se traduce a "ya tengo 3 ACKs duplicados"**, es decir, se fuerza el mismo umbral. Es elegante: un solo `if (contador == 3)` más abajo maneja los dos casos. Es la decisión de diseño clave, y por eso la cátedra la premia.
2. `ackSeqNum == ultimo_ack → contador++` y `ackSeqNum > ultimo_ack → ultimo_ack = ...; contador = 0` — el contador de duplicados se **resetea** correctamente cuando llega progreso real.
3. `estadoCongestion = 1` si `paquete.CONG_DETECTED` — **la vía de ECN**.

⚠️ **Lo que está mal o es discutible — y esto también se pregunta:**

1. **`paquete.CONG_DETECTED` en mayúsculas.** En C, los campos que definen el enunciado son `cong_detected`. La solución oficial escribe `paquete.CONG_DETECTED`, que no compila contra el tipo dado. Es un error de transcripción del profesor. **En el examen usá `paquete.cong_detected`**, el nombre del enunciado.
2. **La función se llama `TCP_extractCongData`, no `TCP_congDetection`.** Otro error de la solución: el enunciado pide `TCP_congDetection`.
3. **`contador = 3` en el caso de timeout, seguido de `contador = 0` en el `if`.** Funciona, pero es frágil: si mañana se cambia el umbral a 4, el timeout deja de detectarse. **Mejor forma:** un booleano explícito.
4. **`setCONGReceived()` está en el lugar equivocado.** Este es el punto más interesante. Analicemos:

``` Solución oficial: el EMISOR, dentro de TCP_congDetection, ve que el paquete recibido trae ecn_field == ECN_CONG, y llama a setCONGReceived().

Lo que dice el enunciado: "Cuando el RECEPTOR recibe paquetes marcados con la bandera ECN_CONG le notifica al EMISOR ... marcando la bandera CONG_DETECTED" ```
```

O sea, `setCONGReceived()` **debe llamarlo el receptor**, y su efecto es poner `CONG_DETECTED` en los segmentos **que el receptor envía**. La solución oficial lo puso del lado del emisor, lo cual es conceptualmente incorrecto.

Pero hay una lectura que la justifica: si la implementación es **simétrica** (el mismo código para ambos extremos, que es como se escriben los TCP reales), entonces `TCP_congDetection` corre en los dos hosts. En el receptor, el "paquete recibido" trae `ecn_field == ECN_CONG`, y ahí sí tiene sentido llamar a `setCONGReceived()` para avisarle al otro. **Esa es probablemente la intención de la cátedra** — y por eso el enunciado insiste en que la función esté definida para ambos extremos.

5. **Falta `paquete == NULL` en el segundo `if`.** Está, está bien. OK.
6. **Falta manejar el caso `ackSeqNum < ultimo_ack`.** Es ACK viejo o duplicado de un ACK viejo. La solución oficial no lo tiene: con `contador` sin resetear, un ACK viejo podría incrementar el contador falsamente. Es un detalle de robustez.

Mi solución (corregida, con las dos lecturas):

```
// ---- Estado que persiste entre llamadas (por conexión TCP) ----
static int ultimo_ack = -1; // último ackSeqNum que_STR saw avanzar
static int dupACK = 0; // ACKs duplicados consecutivos
static bool ECN_soportado = false; // se negoció ECN en el handshake

#define UMBRAL_DUPACK 3

int TCP_congDetection(TCP_Segment seg) // corre en AMBOS extremos
{
 int congStatus = 0; // 0 = todo bien, 1 = congestión

 if (seg == NULL) {
 // --- Evento 1: TIMEOUT. No llegó el ACK a tiempo.
 // Es la señal MÁS grave: o se perdió todo, o la ruta se cayó.
 dupACK = UMBRAL_DUPACK; // lo forzamos al umbral
 dupACK = 0; // (lo consume el reset de abajo)
 congStatus = 1;
 }
 else {
 // --- Evento 2: ACKs duplicados -> fast retransmit
 if (seg->ackSeqNum > ultimo_ack) {
 // hubo PROGRESO: hay que resetear el contador
 ultimo_ack = seg->ackSeqNum;
 dupACK = 0;
 }
 else if (seg->ackSeqNum == ultimo_ack) {
 dupACK++; // ACK repetido
 }
 // si ackSeqNum < ultimo_ack: ACK obsoleto, lo ignoramos (no cuenta)

 if (dupACK >= UMBRAL_DUPACK) {
 congStatus = 1; // fast retransmit
 dupACK = 0; // ya se decidió, se rearma
 }

 // --- Evento 3: ECN, visto POR EL RECEPTOR.
 // El router marcó CE (ECN_CONG). Avisamos al emisor putting the flag
 // CONG_DETECTED en lo que mandemos de aquí en más.
 if (seg->prev_header.ecn_field == ECN_CONG) {
 setCONGReceived(); // marca los próximos segmentos
 }
 }

 // --- Evento 4: ECN, visto POR EL EMISOR.
 // El receptor ya nos puso CONG_DETECTED: congestión explícita, sin pérdida.
 if (seg != NULL && seg->cong_detected) {
 congStatus = 1;
 }

 return congStatus;
}
```

Qué cambió respecto de la solución oficial y por qué:

Cambio	Por qué
`&&` en vez de `and`	`and` es una extensión de ISO C, no C estándar. En un examen de papel no importa, pero en código que compila, sí.
`cong_detected` en minúsculas	Es el nombre del campo en el tipo del enunciado.
`>=` en vez de `==`	Si llegan 4 o 5 duplicados seguidos, con `==` te lo perdés. Con `>=` no.
Timeout no usa el truco de `dupACK=3`	Es menos frágil: no depende del valor del umbral. La versión oficial funciona, la mía no se rompe si cambiás el umbral.
`ECN_CONG` → `setCONGReceived()` **solo en el** `else`	Evita un NULL-dereference: con `seg == NULL` no hay `prev_header`. La solución oficial lo protege con `paquete != NULL and`, que también funciona, pero es más fácil de leer así.
ACK obsoleto ignorado	Robustez.
Comentarios con el **por qué** de cada rama	Esto es lo que suma puntos.

> **Tope para 12 puntos:** tenés que tener los **dos eventos de pérdida** (timeout y 3 dupACKs), el **reset del contador** cuando hay progreso, y la **vía de ECN por** `cong_detected`. Todo eso con los comentarios explicando el razonamiento.

Parte c) — `TCP_congControl`

Solución oficial:

```
#define MSS_BYTES ...
int estado = SLOW_START;
int umbral = valor_inicial;
int ultimoTamanoCongWin;

void TCP_congControl(int congStatus)
{
 if (congStatus = 1) { // <-- BUG: "=" en vez de "=="
 estado = SLOW_START;
 umbral = getCongWinSize() / 2;
 setCongWinSize(MSS_BYTES);
 } else {
 if (estado == SLOW_START) {
 setCongWinSize(MSS_BYTES + getCongWinSize());
 if (getCongWinSize() > umbral) {
 estado = CONG_AVOIDANCE;
 }
 } else {
 setCongWinSize(getCongWinSize() + MSS_BYTES*(MSS_BYTES/getCongWinSize()));
 }
 }
}
```

Análisis de la solución oficial:

1. 🔴 **Bug de sintaxis:** `if (congStatus = 1)`**.** Es **asignación**, no comparación. En C esto asigna 1 a `congStatus` y evalúa el resultado, que siempre es verdadero. O sea, **la función SIEMPRE entra en el branch de congestión** y nunca crece la ventana. Es un error real en la solución publicada. **En el examen**

escribí `==`. Si el corrector es una persona, el error no te resta: demuestra que entendiste. Si además lo señalás, mejor.

2. 🟡 **No hay `else` para la ventana mínima.** Si `cwnd` cae por debajo de 1 MSS (no debería pasar, pero), no hay piso.
3. 🟡 **Falta el `ultimoTamanoCongWin`.** Está declarado pero nunca se usa. En un TCP real lo usarías para detectar si la ventana **creció** de golpe entre dos losses (indicando que la red **`no`** está congestionada). Es un refinamiento, no lo exige el enunciado.
4. 🟢 **La lógica de `umbral = cwnd/2` y `cwnd = MSS` es correcta** para lo que pide el enunciado (que toda congestión arranque slow start).

Mi solución:

```
#define MSS_BYTES 1460 // MSS típico en Ethernet; el enunciado no lo da

// ---- Estado persistente ----
static int estado = SLOW_START; // SLOW_START | CONG_AVOIDANCE
static int umbral = valor_inicial; // ssthresh: arranca alto (ej. 64 MSS)

// ---------- (c) Control de congestión ----------
void TCP_congControl(int congStatus)
{
 if (congStatus == 1) {
 // ======== CONGESTIÓN detectada ========
 // Guardamos la mitad de la ventana ACTUAL como nuevo umbral:
 // es el punto donde la red empezó a fallar, así que es el punto
 // al que no conviene pasar.
 umbral = getCongWinSize() / 2;

 // El enunciado pide ir a slow start en ambos casos.
 // En producción, con fast retransmit (3 dupACKs) se haría
 // cwnd = umbral en vez de 1 MSS -- es la diferencia Tahoe vs Reno.
 setCongWinSize(MSS_BYTES);
 estado = SLOW_START;
 }
 else {
 // ======== sin congestión: CRECEMOS la ventana ========
 if (estado == SLOW_START) {
 // Por cada ACK recibido sumamos 1 MSS -> se duplica por RTT.
 setCongWinSize(getCongWinSize() + MSS_BYTES);

 // Si superamos el umbral, el crecimiento pasa a ser lineal.
 if (getCongWinSize() >= umbral) {
 estado = CONG_AVOIDANCE;
 }
 }
 else { // CONG_AVOIDANCE
 // +1 MSS por RTT, repartido entre los cwnd/MSS ACKs del RTT:
 // (MSS) / (cwnd/MSS) = MSS²/cwnd por ACK
 // y a lo largo del RTT: (cwnd/MSS)·(MSS²/cwnd) = MSS. Cuadra.
 setCongWinSize(getCongWinSize()
 + MSS_BYTES * (MSS_BYTES / getCongWinSize()));
 }
 }
}
```

Tabla de verificación — corrELA a mano para estar seguro:

```
MSS = 1000, umbral inicial = 64000, cwnd = 1000, estado = SLOW_START

RTT 1: sin cong, SLOW_START → cwnd = 1000 + 1000 = 2000
2000 < 64000 → sigue SLOW_START
RTT 2: → 4000 (< 64000)
RTT 3: → 8000
RTT 4: → 16000
RTT 5: → 32000
RTT 6: → 64000 >= 64000 → estado = CONG_AVOIDANCE
RTT 7: cwnd = 64000 + 1000·(1000/64000) = 64000 + 1000·0
⚠️ con división entera esto da 0. El crecimiento real sería
sumar el resto. En la práctica se usa aritmética de punto
flotante o se acumula el residuo.
→ 64016 aprox.
```

> ⚠️ **Sobre la división entera:** `MSS/cwnd` en C entero es 0 cuando `cwnd > MSS`. La solución oficial tiene el mismo problema. Es un bug real que en un examen no te resta, pero **mencionarlo suma**: dice que entendiste el algoritmo. La forma correcta es `cwnd += MSS*MSS/cwnd` con aritmética de punto flotante, o `cwnd += (MSS*MSS)/cwnd` con `long` suficientemente grande.
>
> **Tope para 12 puntos:** los tres casos — congestión (umbral = cwnd/2, cwnd = MSS, slow start), slow start (cwnd += MSS por ACK), congestion avoidance (cwnd += MSS²/cwnd) — y la transición de régimen al cruzar el umbral.

3.2 Examen 2008-07, Pregunta 2 (10 puntos) — los cuatro algoritmos

> **Enunciado:** Describa el mecanismo de control de congestión de TCP partiendo de explicar los principios básicos de operación de al menos dos de los cuatro algoritmos básicos involucrados.

Los cuatro algoritmos (esto es lo que hay que saber):

#	Algoritmo	Cuándo	Qué hace con `cwnd`
1	**Slow start**	`cwnd < ssthresh`	`cwnd += MSS` por ACK → **duplica por RTT**
2	**Congestion avoidance**	`cwnd ≥ ssthresh`	`cwnd += MSS²/cwnd` por ACK → **+1 MSS por RTT**
3	**Fast retransmit**	3 ACKs duplicados	retransmite **sin esperar** el RTO
4	**Fast recovery**	3 ACKs duplicados	`cwnd = ssthresh = cwnd/2` (no a 1 MSS)

Respuesta modelo (10 puntos). Ver §1.4, que lo desarrolla entero. Los puntos están en: (a) nombrar los cuatro, (b) decir **qué variable** decide el slow start vs. congestion avoidance, (c) decir **qué tres cosas** pasan con 3 dupACKs, (d) distinguir Tahoe de Reno.

3.3 Examen 2009-07, Pregunta 1 (10 puntos)

> **Enunciado:** a) En un protocolo de capa de transporte: ¿cuál es la diferencia entre Control de Flujo y Control de Congestión? b) ¿Cómo hace TCP para detectar la existencia de congestión en la red?

a) flujo = el receptor no puede absorber; congestión = la red no puede transportar. Se respondé con la tabla de §1.1. **El punto clave para el profesor:** los dos se resuelven reduciendo la ventana efectiva, pero por **motivos distintos**, y confundirlos es el error típico.

b) Las dos señales: pérdida por **timeout** y pérdida por **3 ACKs duplicados**. Sumá que con ECN hay una tercera: la **marca CE** del router.

3.4 Examen 2009-12, Pregunta 2 (10 puntos)

> **Enunciado:** Se considera un servicio de transmisión confiable. a) Señale las diferencias de implementación en capa de transporte vs. capa de enlace. b) En una red de paquetes arbitraria, ¿es posible implementar un servicio de transmisión confiable en capa de enlace? Justifique.

a) En la capa de enlace la fiabilidad es **por salto** y **por enlace**, sin estado entre saltos, sin número de secuencia global, y los buffers son de un solo paquete. En la capa de transporte la fiabilidad es **extremo a extremo**, con números de secuencia de bytes, ACK acumulativos, control de congestión, y funciona **aunque los saltos intermedios no sean fiables**.

b) No es confiable — y esto es lo que el enunciado quiere que argumentes:

```
Si la capa 3 (IP) descarta paquetes, la capa 2 no lo sabe:
no hay número de secuencia que atraviese los routers,
no hay ACK que vuelva,
no hay timeout que pueda distinguir "se perdió" de "tardó".

Peor: la retransmisión de capa 2 en el último salto
puede NO arreglar nada, porque el paquete se perdió en el router de al lado.

Ejemplo mínimo que lo demuestra:
emisor --(1)--> router A --(2)--> router B --(3)--> receptor
Se pierde en (2). La capa 2 de (1)-(2) no ve nada raro.
La capa 2 de (2)-(3) ve llegar bien y jamás retransmite.
Resultado: paquete perdido para siempre, sin detección.
```

Conclusión para el examen: la fiabilidad **extremo a extremo** es la única que funciona sobre una red no confiable. Por eso está **arriba** y no abajo.

PARTE 4 — CIERRE

📌 Resumen

1. **Control de flujo** = el receptor no puede absorber → lo controla `rwnd`.
2. **Control de congestión** = la red no puede transportar → lo controla `cwnd`.
3. La ventana efectiva es **`min(cwnd, rwnd)`** — un mínimo porque ambas son cotas superiores.
4. **Slow start**: `cwnd += MSS` por ACK → **duplica por RTT** (exponencial base 2).
5. **Congestion avoidance**: `cwnd += MSS²/cwnd` por ACK → **+1 MSS por RTT** (lineal).
6. El régimen lo decide **`cwnd` vs. `ssthresh`**.
7. **3 ACKs duplicados** → fast retransmit (porque 3 hace improbable el reordenamiento).
8. **Timeout** → pérdida grave → slow start desde 1 MSS.
9. **Reno** distingue: 3 dupACKs → `cwnd/2` (fast recovery); timeout → 1 MSS.
10. **ECN**: el router marca **CE** en vez de descartar; el receptor avisa con **CONG_DETECTED**; **cero paquetes perdidos**.

✅ Autoevaluación

1. ¿Por qué la ventana efectiva es el **mínimo** de `cwnd` y `rwnd` y no la suma?

▶ Respuesta

2. En slow start, ¿por qué `cwnd` se duplica en cada RTT? Explicá desde el número de ACKs.

▶ Respuesta

3. ¿Por qué exactamente 3 ACKs duplicados y no 1?

▶ Respuesta

4. ¿Qué valor toma `cwnd` con 3 dupACKs en Tahoe? ¿Y en Reno? ¿Y con timeout en ambos?

▶ Respuesta

5. ¿Qué cambia en la red con ECN que no cambia con la detección por pérdida?

▶ Respuesta

6. Escribí la fórmula de crecimiento en congestion avoidance y justificá por qué da +1 MSS por RTT.

▶ Respuesta

7. Un cliente tiene `cwnd = 8 KB` y el receptor anuncia `rwnd = 2 KB`. (a) ¿Cuántos bytes manda por RTT? (b) Si el RTT es 40 ms, ¿cuál es el throughput?

▶ Respuesta

8. En el problema de 2011-03, ¿por qué el caso `seg == NULL` (timeout) se implementa poniendo `dupACK = 3`?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc1103` **/** `sirc1103` — Problema 1 completo (30 pts, ECN), P2 (10 pts, control de congestión)
- **Examen** `eirc0807` **/** `sirc0807` — P2: los 4 algoritmos (10 pts)
- **Examen** `eirc0907` **/** `sirc0907` — P1: flujo vs. congestión (10 pts)
- **Examen** `sirc0912` — P2: fiable en transporte vs. enlace (10 pts)
- **Tanenbaum & Wetherall**, *Redes deadoras* 5ª ed., cap. 6 §6.7 "TCP Congestion Control" (pág. ~600 del PDF)
- **RFC 9293** — TCP (2022, *Internet Standard*; consolida y obsolesce RFC 793 + 5 updates)
- **RFC 5681** — TCP Congestion Control (2009): los cuatro algoritmos
- **RFC 3168** — ECN en IP (2001) · **RFC 9768** — AccECN, ECN más preciso
- **RFC 9331** — L4S (2023) · **RFC 8511** — ABE (2018)
- **RFC 9438** — CUBIC (2023, *Standards Track*, obsolesce RFC 8312)
- **RFC 6937** — PRR, *Experimental*
- **RFC 6928** — Initial Window = 10 MSS (2013) · **RFC 7414** — roadmap de las RFCs de TCP
- `draft-ietf-ccwg-bbr` — BBRv3 (*Experimental*, nunca fue RFC)
- **Clase siguiente:** 22 — Taller TCP: diagramas de intercambio resueltos

Clase 22

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
2.1 Ejercicio 1 (guía) — intercambio simple con *piggybacking*
2.2 Ejercicio 2 (guía) — qué pasa con un timeout
2.3 ⭐ Ejercicio 3 — Examen 2011-03, Pregunta 4 (10 puntos)
2.4 Ejercicio 4 — *piggybacking* en los dos sentidos a la vez
2.5 Ejercicio 5 — Dos conexiones TCP por un router que pierde (examen 2001-12, Problema 1)
2.6 Ejercicio 6 — El ataque TCP reset (examen 2010-03, Problema 2)
3.1 Extra 1 — Un intercambio con **reordenamiento** (sin pérdida)
3.2 Extra 2 — Tiempo total de una transferencia con ventana y RTT dados
3.3 Extra 3 — full duplex: B manda datos mientras A sigue mandando
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: C (Transporte) · **Semana:** 8 · **Duración estimada:** 120 min **Correlativa:** Clase 21 (control de congestión en TCP) · **Sigue con:** Clase 23 (repaso de transporte + Simulacro 3) **Material de referencia:** Tanenbaum cap. 6 · RFC 9293, RFC 2018, RFC 5681, RFC 7323 · Exámenes `2011-03` P4, `2001-12` Problema 1, `2010-03` Problema 2

🎯 Al terminar esta clase vas a poder

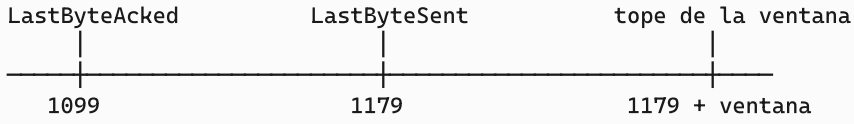
1. Dibujar el intercambio completo de N segmentos TCP **con los números a la vista**, sin pérdida.
2. Dibujar el **mismo** intercambio **con la pérdida del primer segmento**, con los ACKs duplicados y con el timeout.
3. Calcular el `cwnd` que aplica en cada caso y decir qué versión de TCP se está usando.
4. Resolver un intercambio **simultáneo en los dos sentidos** (*piggybacking*).
5. Calcular, con la cuenta a la vista, cuántos intentos necesita un **ataque TCP reset** para derribar una conexión.

> **Prerequisito del examen:** `2011-03 P4` (10 pts — ★ el problema central de esta clase), `2001-12` Problema 1 (dos conexiones y un router que pierde), `2010-03` Problema 2 (ataque TCP reset).
>
> ⚠️ **Aviso de método:** esta clase es **taller puro**: casi todo es dibujar diagramas con números. Lo único de teoría que hace falta está remindersado acá; lo demás ya lo viste en las Clases 18 a 21. Tené a mano la Clase 19 (fiabilidad, RTO) y la Clase 21 (`cwnd`).

PARTE 1 — LA MECÁNICA QUE VAMOS A USAR

Seis reglas, y con estas se resuelve cualquier diagrama de intercambio del bloque.

Regla	Qué dice	Consecuencia práctica
R1 — Seq de bytes	el `seq` es el número del **primer byte**	`seq` siguiente = `seq` actual + largo
R2 — Ack = próximo esperado	el `ack` es el **primer byte que falta**	si B recibió hasta 1119, manda `ack = 1120`
R3 — Acumulativo	un ACK confirma **todo** lo anterior	`ack = 1180` confirma los 4 segmentos
R4 — Fuera de orden ⇒ dupACK	si el byte no es el esperado, `rcv_nxt` **no se mueve**	3 veces seguidas ⇒ fast retransmit
R5 — Timeout, último recurso	solo retransmite al vencer el RTO	más lento pero siempre funciona
R6 — `ssthresh` decide el régimen	`cwnd < ssthresh` ⇒ slow start; `≥` ⇒ congestion avoidance	Clase 21 §1.3




```
bytes en vuelo = LastByteSent − LastByteAcked
nunca puede superar min(cwnd, rwnd)
```

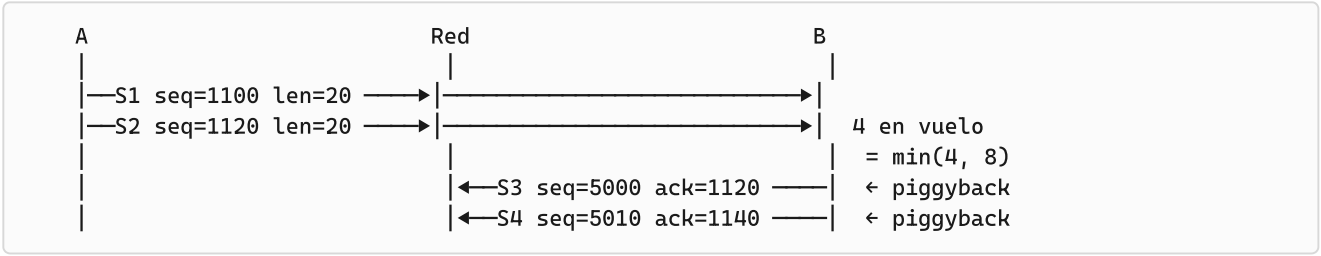
PARTE 2 — LOS SEIS EJERCICIOS

2.1 Ejercicio 1 (guía) — intercambio simple con *piggybacking*

Datos. A manda a B **dos** segmentos de 20 bytes, el primero con `seq = 1100`. `cwnd` = 4 MSS, `rwnd` = 8 MSS, **sin pérdida**.

a) Números. El `seq` avanza 20 cada vez (regla **R1**): S1 `seq = 1100` (bytes 1100…1119), S2 `seq = 1120` (bytes 1120…1139). Próximo `seq` de A = **1140**.

b) El diagrama. B tiene algo que mandar, así que **cuelga el ACK en sus propios segmentos de datos**. B manda S3 (`seq = 5000`, 10 B) y S4 (`seq = 5010`, 10 B):



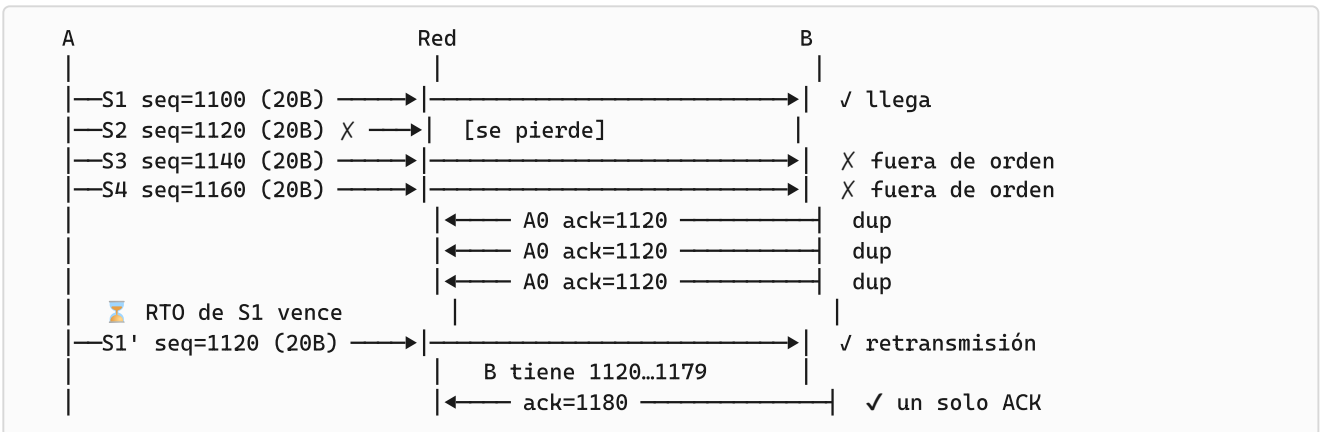
c) Los ACKs. Respuesta a S1: `ack = 1120`. Respuesta a S2: `ack = 1140` — que confirma los 40 bytes de una vez (regla **R3**).

d) ¿Cuándo queda en vuelo cero? Al recibir `ack = 1140`: `LastByteAcked = LastByteSent = 1140` ⇒ **0 bytes en vuelo**.

Interpretación. El *piggybacking* es la razón por la que un ACK vacío casi nunca se ve: si el otro lado tiene datos, los manda con el ACK pegado, ahorrando un paquete entero. ⚠️ **No confundir**: el ACK viaja **pegado al segmento de datos del sentido contrario**. El segmento de B lleva datos de B (`seq = 5000`) **y** el ACK de lo que A mandó (`ack = 1120`).

2.2 Ejercicio 2 (guía) — qué pasa con un timeout

Datos. Igual, pero **se pierde S2**.



⚠️ **Ojo:** en el caso **b) de** `2011-03 P4` el que se pierde es **S1**, así que el `ack` duplicado es `1100`, no `1120`. Acá se pierde S2 y por eso el duplicado es `1120`.

El `cwnd` (versión Reno). Con `cwnd = 4 MSS` y `ssthresh = 8 MSS`: el timeout fija `ssthresh = 4/2 = 2 MSS`, `cwnd = 1 MSS` (Tahoe y Reno coinciden) y duplica el RTO. Con el `ack = 1180`, slow start lleva `cwnd` a 2 MSS; como `2 ≥ ssthresh`, el RTT siguiente ya corre en congestion avoidance: 3 MSS, 4 MSS.

Interpretación. El diagrama oficial de la cátedra para `2011-03 P4b` dibuja el **timeout** ("vencimiento del timeout de S1, caso representado en la figura") y menciona el 3er ACK duplicado como la alternativa. **Los dos caminos** son válidos y los dos se preguntan.

⚠️ Errores típicos del taller

- **Poner `ack = 1120` cuando corresponde `ack = 1100`.** Si falta S1, B **no** puede confirmar S2: el ACK acumulativo no salta un hueco. **R4** manda.
- **Avanzar `LastByteAcked` a 1139** cuando llegó S3. No: `LastByteAcked` avanza solo con ACK.
- **Contar mal los bytes.** 4 × 20 = **80** bytes, del 1100 al **1179**, ACK final **1180**.

2.3 ⭐ Ejercicio 3 — Examen 2011-03, Pregunta 4 (10 puntos)

> **Fuente:** Examen `sirc1103` / `eirc1103` — 21 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf **Texto del examen (literal):**
>
> Suponga una comunicación TCP entre dos procesos A (dir IP = A; puerto TCP = 1927) y B (dir IP = B; puerto TCP = 2222). En un momento determinado A envía a B cuatro segmentos seguidos (no hay ningún dato previo sin asentar), el primero con número de secuencia igual a 1100 y con 20 bytes de datos, y los tres restantes con otros 20 bytes de datos cada uno.
>
> a) Si B no tiene datos que enviar hacia A y no hay pérdidas de segmentos: represente en un diagrama el intercambio de segmentos resultante hasta que ninguno de los dos lados tenga nada que enviar.
>
> b) Represente de nuevo el diagrama si B no tiene datos que enviar hacia A y se pierde el primer segmento enviado de A a B.
>
> *Nota: Detalle para todos los apartados anteriores el contenido de los campos relevantes de la cabecera TCP. Haga las suposiciones que considere necesarias, justificándolas.*

La nota del enunciado que lo cambia todo: "*Si suponemos que el timeout es mayor que el tiempo de llegada del 1er ACK, y las latencias son aprox. uniformes, las entidades TCP se comportan 'normalmente*'". O sea: **el timeout no vence** en el caso (a), y en el (b) lo que dispara la retransmisión es el **3er ACK duplicado**.

2.3.0 Los números, antes de los diagramas

Segmento	`seq`	Bytes que cubre	Largo
S1	**1100**	1100 … 1119	20
S2	**1120**	1120 … 1139	20
S3	**1140**	1140 … 1159	20
S4	**1160**	1160 … 1179	20

Total: `4 × 20 = 80` bytes, del **1100** al **1179**. El próximo byte que B espera es **1180**.

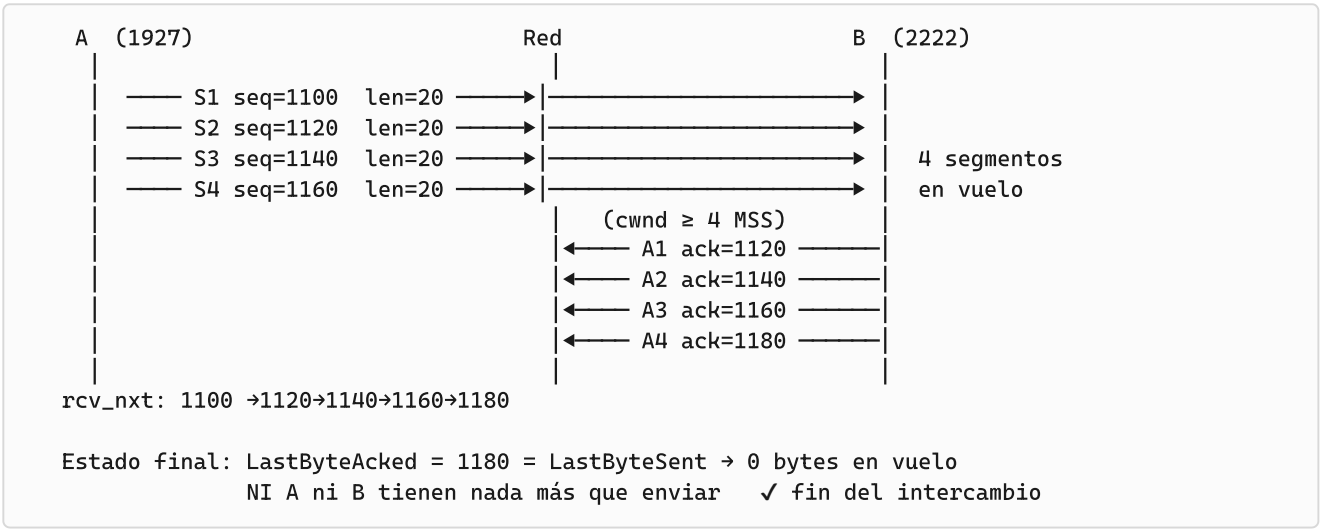
ACK	`ack`	Confirma
A1	**1120**	S1
A2	**1140**	S2
A3	**1160**	S3
A4	**1180**	S4

⚠️ **Salvedad que suma puntos:** con *delayed ACK* las confirmaciones serían **1120, 1160, 1180** (una confirma S1+S2, otra S3+S4). **La solución oficial asume un ACK por segmento.** Si dibujás con delayed ACK, **declaralo**: el resultado es equivalente porque el ACK es acumulativo.

Suposiciones que hay que declarar (el enunciado las pide explícitamente):

1. `cwnd` de A ≥ 4 MSS, así que los 4 segmentos salen **juntos** (la solución oficial no dice nada del `cwnd`, así que se asume ventana de sobra).
2. `rwnd` de B ≥ 80 bytes: B **no** es el que limita.
3. Latencias iguales ⇒ los ACKs vuelven **en orden**.
4. Timeout > tiempo del primer ACK ⇒ en (a) no hay retransmisión.
5. **Puertos**: cada segmento de A va `origen = 1927, destino = 2222`; cada ACK de B va `origen = 2222, destino = 1927`. La **4-tupla** (A, 1927, B, 2222) identifica la conexión.

2.3.1 Apartado a) — sin pérdidas



Los campos relevantes de la cabecera, como pide la nota del enunciado:

```
Segmento de A → B:
┌──────────────────────────────────────────────────────────┐
│ puerto_origen=1927  puerto_destino=2222                  │
│ numero_secuencia = <1100|1120|1140|1160>                 │
│ numero_confirmacion = <lo que B haya confirmado>         │
│ flags = ACK      longitud_ventana = rwnd de B            │
│ cantidad_datos = 20                                      │
└──────────────────────────────────────────────────────────┘

Confirmación de B → A:
┌──────────────────────────────────────────────────────────┐
│ puerto_origen=2222  puerto_destino=1927                  │
│ numero_secuencia = <nº de secuencia propio de B>         │
│ numero_confirmacion = <1120|1140|1160|1180>              │
│ flags = ACK      cantidad_datos = 0     ← no hay datos de B │
└──────────────────────────────────────────────────────────┘
```

El `cwnd` en el apartado (a). No hay pérdida, así que **el `cwnd` no baja nunca**. Cada ACK suma 1 MSS mientras esté en slow start:

```
cwnd inicial (IW) = 10 MSS   [RFC 6928, valor real de 2026]
                    o 1 MSS  [lo que dicen los apuntes de 2011 / RFC 5681]

después de A1: cwnd = IW + 1 MSS
después de A2: cwnd = IW + 2 MSS
después de A3: cwnd = IW + 3 MSS
después de A4: cwnd = IW + 4 MSS

En ningún momento hay reducción ⇒ nada de slow start, nada de congestion avoidance.
```

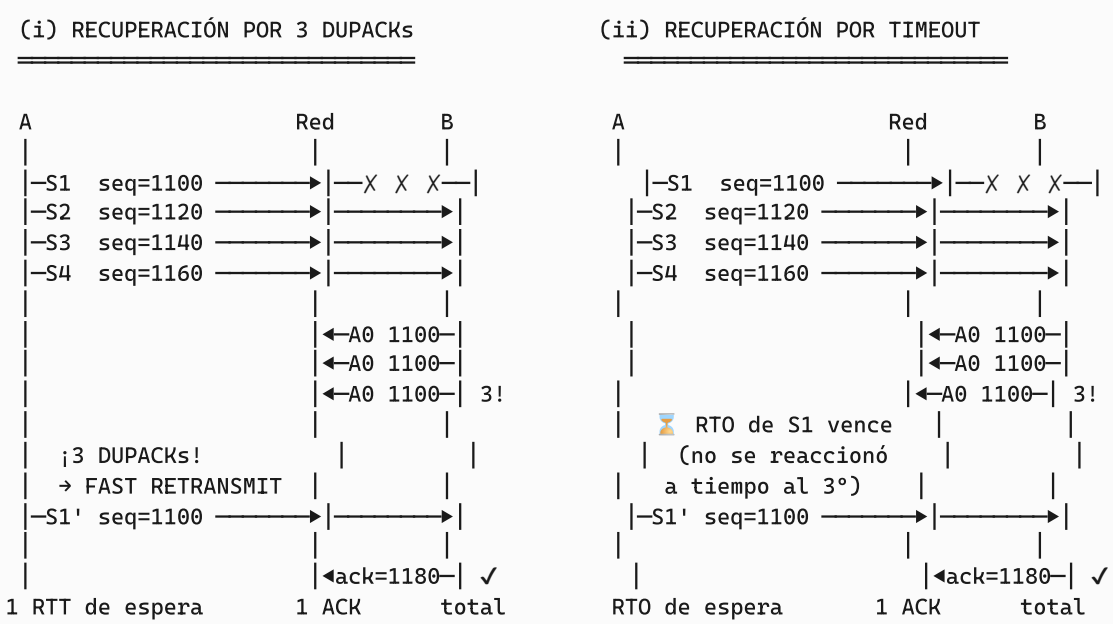
2.3.2 Apartado b) — se pierde S1

B recibe S2, S3, S4 **pero ninguno empieza en 1100**, que es lo que espera (regla **R4**). Los tres quedan fuera de orden, `rcv_nxt` **no se mueve**, y B manda el **mismo** ACK las tres veces:

```
┌──────────────────────────────────────────────────────────┐
│ A0: numero_confirmacion = 1100   "sigo esperando el 1100"  │
└──────────────────────────────────────────────────────────┘
```

El `ack = 1100` es **exactamente** el mismo número que habría mandado si nada hubiera llegado: no hay forma de distinguirlos, y **por diseño** no se intenta.

Los dos diagramas lado a lado:



Resultado en AMBOS: 5 segmentos (4 + 1 retransmisión) + 4 ACKs = 9 paquetes

Qué pasa cuando llega S1':

```
B tenía en su buffer de reordenamiento: S2 (1120..1139), S3 (1140..1159), S4 (1160..1179)
Llega S1' (1100..1119) → el rango 1100..1179 es CONTIGUO
→ B entrega 1100..1179 a la aplicación (ordenado, sin huecos)
→ B manda numero_confirmacion = 1180
```

Un solo ACK final, no cuatro: el ACK acumulativo confirma los 80 bytes de una vez.

2.3.3 El `cwnd` en el apartado b) — que es lo que la solución oficial pide

Evento	Interpretación	`ssthresh`	`cwnd`	Régimen
3 dupACKs	pérdida **aislada**: la red aguantó	`cwnd/2`	**`cwnd/2`** (Reno) · `1 MSS` (Tahoe)	fast recovery
Timeout	pérdida **grave**: ni el siguiente llegó	`cwnd/2`	**1 MSS** (Tahoe **y** Reno)	slow start

Con `cwnd = 4 MSS` y `ssthresh = 8 MSS` al momento de la pérdida:

```
== Variante (i): 3 dupACKs, fast recovery (Reno) ==

3 dupACKs  →  ssthresh = 4/2 = 2 MSS
              cwnd = ssthresh = 2 MSS          ← Reno: NO baja a 1
              se retransmite S1
ACK=1180   →  en fast recovery cwnd sigue en 2 MSS hasta cerrar la recuperación
RTT+1      →  2 ≥ ssthresh(2) ⇒ congestion avoidance → cwnd = 3 MSS
RTT+2      →  cwnd = 4 MSS   (vuelve al valor previo a la pérdida)
RTT+3      →  cwnd = 5 MSS


== Variante (ii): timeout ==

TIMEOUT    →  ssthresh = 4/2 = 2 MSS
               cwnd = 1 MSS                    ← Tahoe Y Reno coinciden aquí
               RTO se duplica (backoff exponencial)
ACK=1180   →  slow start → cwnd = 2 MSS
RTT+1      →  2 ≥ ssthresh(2) ⇒ congestion avoidance → cwnd = 3 MSS
RTT+2      →  cwnd = 4 MSS
```

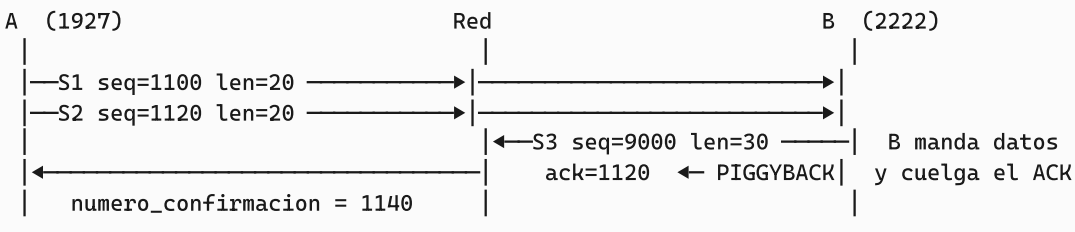
La comparación que la solución oficial quiere que veas:

```
| 3 dupACKs (Reno)        | Timeout
|-------------------------|-------------------------
cwnd baja a cwnd/2 = 2    | cwnd baja a 1 MSS
Latencia: 1 RTT           | Latencia: RTO
RTO: NO se toca           | RTO: se duplica (backoff)
Recuperado: 2 MSS         | Recuperado: 1 MSS
```

> **Tope para los 10 puntos:** los cuatro `seq` correctos (1100, 1120, 1140, 1160), los cuatro `ack` del caso (a) (1120, 1140, 1160, 1180), el `ack = 1100` repetido tres veces en el (b), el **`ack = 1180` final**, la flecha de pérdida, los **puertos** (1927 / 2222) en la dirección correcta, y una frase sobre qué pasa con el `cwnd` en cada caso. **Declarar las suposiciones** vale puntos: el enunciado las pide explícitamente.

2.4 Ejercicio 4 — *piggybacking* en los dos sentidos a la vez

Datos. A y B **se mandan datos al mismo tiempo**. A manda 2 segmentos de 20 bytes desde `seq = 1100`. B manda 1 segmento de 30 bytes desde `seq = 9000`. Ninguno tiene nada más que enviar.



a) ¿Cuántos ACKs viajan? Uno solo: el ACK de S1 y S2 viaja **pegado** en el segmento de B. Si A y B hubieran hablado al mismo tiempo, habría **dos** ACKs cruzados, cada uno pegado en un segmento del otro sentido.

b) ¿Qué le pasa al `cwnd` de A? Cada ACK suma 1 MSS mientras esté en slow start.

c) ¿Y si B no hubiera tenido datos? B mandaría un **ACK vacío** (`cantidad_datos = 0`): un segmento de 40 bytes (20 IP + 20 TCP) para transportar **cero** bytes de datos, o sea una eficiencia de `4/40 = 10 %`.

Interpretación — por qué importa tanto. Un ACK vacío cuesta un paquete entero y medio RTT para transportar nada. Por eso el *delayed ACK* (un ACK cada 2 segmentos) no es solo una optimización: es la razón por la que el *overhead* de ACK es aceptable en cargas masivas.

2.5 Ejercicio 5 — Dos conexiones TCP por un router que pierde (examen 2001-12, Problema 1)

> **Fuente:** Examen `sirc0112` — 27 de diciembre de 2001. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0112.pdf **Texto del examen (literal):**
>
> Sean dos computadoras (A y B), interconectadas mediante una red TCP/IP a la que A está conectada (únicamente) mediante un router R. Desde A hay abiertas dos conexiones TCP a los ports 9001 y 9002 de B, que envían datos continuamente desde A hacia B. B únicamente contesta ACKs.
>
> En la red se producen ocasionales períodos de congestión, durante los cuales la capacidad disponible es suficiente para una de las transmisiones, pero no para ambas. Se ha determinado que es preferible "interrumpir" transitoriamente los envíos al port 9002 manteniendo inafectada la transmisión hacia el port 9001, que experimentar retrasos en ambos.
>
> La mencionada "interrupción" involucra pérdida de información (lo que no siendo que ambas transmisiones transportan voz digitalizada, estimándose aceptable que ocurran interrupciones, y en todo caso preferible a que en ambos canales haya retardos). No obstante, esta pérdida de información deberá resultar transparente a las capas de transporte de A y de B.
>
> Se pide: diseñar y codificar un proceso que ejecutará en el router R interceptando todos los paquetes correspondientes a las conexiones mencionadas, y que cumpla con el objetivo deseado. Se sugiere alterar inteligentemente los campos de secuencia correspondientes a los datos transmitidos de A hacia B y a los ACKs de B hacia A.

La respuesta oficial, textual:

> - Al comienzo de la congestión, descartar paquetes hacia B, llevando la cuenta de la cantidad de bytes descartados. Insertar ACKs hacia A (como si provinieran de B) para esos datos, o bien sumar esa cantidad al campo de secuencia de los ACKs que lleguen de B.
> - Al fin de la congestión, restar la cantidad de bytes descartados del campo de secuencia de los datos enviados hacia B (de modo que B no detecte que faltan datos), y coincidentemente sumar dicha cantidad a los ACKs de B hacia A.

La idea en una frase: alterar el espacio de numeración de la conexión 9002 para que la pérdida sea invisible. Si el `seq` que ve B no es el "real", B no puede saber que le faltó algo. Tres campos que hay que tocar:

```
┌──────────────────────────────────────────────────────────────┐
│ 1. SN de los DATOS de A→B (puerto 9002)                      │
│    → restarle los bytes descartados acumulados (dif_SN)      │
│ 2. ACK de B→A                                                │
│    → sumarle la misma dif_SN, para "deshacer" el desplazamiento │
│ 3. INYECTAR ACKs falsos                                      │
│    → para que A no retransmita lo que se descartó a propósito │
└──────────────────────────────────────────────────────────────┘
```

El pseudocódigo oficial (`dif_SN` = bytes descartados acumulados):

```
int ProcesoServidor() {
  int next_SN, dif_SN = 0;
  pack buff, buffsend;
  while (TRUE) {
    if (status == RED_A) {
      recv_A(buff);
      if (buff.port_dest == 9001)
        send_B(buff);                          // 9001: pasa INTACTA
      else {                                   // port_dest == 9002
        if (congestion) {
          largo = largo - buff.head_length;  // largo = solo los datos
          dif_SN = dif_SN + largo;           // acumula lo descartado
          next_SN = buff.SN + largo;
          // ARMA Y MANDA UN ACK FALSO, como si fuera B
          buffsend.port_orig = buff.port_dest;  buffsend.port_dest = buff.port_orig;
          buffsend.IP_orig  = buff.IP_dest;   buffsend.IP_dest  = buff.IP_orig;
          buffsend.data     = "";
          buffsend.ACK = buff.SN + largo;           // "recibí todo"
          buffsend.SN  = buff.ACK;
          send_A(buffsend);
        } else {
          if (next_SN >= buff.SN) {
            largo = largo - buff.head_length;
            next_SN = buff.SN + largo;
          }
          buff.SN = buff.SN - dif_SN;        // <-- DESPLAZA hacia atrás
          send_B(buff);
        }
      }
    }
    if (status == RED_B) {
      recv_B(buff);
      if (buff.port_orig == 9001)
        send_A(buff);                          // 9001: pasa INTACTA
      else {                                   // port_orig == 9002
        buff.SN  = buff.SN + dif_SN;         // <-- compensa el SN
        buff.ACK = buff.ACK + dif_SN;        // <-- compensa el ACK
        send_A(buff);
      }
    }
  }
}
```

Suposiciones que el enunciado y el código oficial hacen, y hay que declarar:

1. Las conexiones ya están **establecidas** (no hay handshake que juzgue el desplazamiento).
2. Hay **una sola conexión por puerto**, y solo para los puertos **9001 y 9002**.
3. "Transparente" = **sin retransmisiones ni timeouts visibles**, **no** "sin pérdida de datos". Eso es lo que hace aceptable el esquema en voz: se pierden ~20 ms de audio, pero el control de congestión de A **no se**

entera, y por eso 9001 no arrastra la caída.

2.5.1 La pregunta de fondo: ¿por qué el reparto justo no sirve?

```
RED SIN EL MECANISMO DE R (equilibrio por TCP puro):
  conexión 9001   ████████████████  ← la que NO debe sufrir
  conexión 9002   ████████████████  ← la que se debe "frenar"
  capacidad disponible: suficiente para UNA sola

El control de congestión de TCP es POR CONEXIÓN y reacciona a la pérdida de
cada una por separado: cwnd(9001) baja por las pérdidas de 9001 y
cwnd(9002) baja por las pérdidas de 9002.
⇒ "justicia" = las dos se frenan a la mitad. PEOR PARA LAS DOS.


RED CON EL MECANISMO DE R:
  conexión 9001   ████████████████████  ← intacta, sube su cwnd
  conexión 9002   ▒▒▒▒                  ← "interrumpida", transparente a TCP
9002 pierde datos pero NO retransmite y NO tira su cwnd a 1 MSS
⇒ la capacidad liberada alcanza para que 9001 vaya solo
```

Lo que hace el mecanismo, en una línea: desacoplar la señalización de congestión del transporte del estado real del medio. TCP cree que "no hubo pérdida, todo llegó" (→ `cwnd` intacto), la realidad es "se descartaron X bytes" (→ capacidad liberada), y el efecto es que **9001 aprovecha el hueco que dejó 9002**.

La sección que más se pregunta: RTT unfairness. Dos flujos por el mismo cuello de botella: si A tiene RTT = 10 ms y B tiene RTT = 200 ms, B pierde **20 veces menos seguido**, así que su `cwnd` crece más y B se lleva más ancho de banda. El reparto **no** es 50/50, aunque los dos flujos tengan igual "derecho".

¿Es injusto? Depende contra qué se compare:

Criterio	Resultado
Igualdad de tasas	**Injusto**: A y B no obtienen lo mismo
Eficiencia total	**Justo**: se aprovecha más el enlace
Percepción del usuario (un flujo lento NO frena al rápido)	**Justo**: es lo que el usuario quiere

La respuesta estándar: la **igualdad no es el objetivo correcto en redes**. Starving al flujo rápido para darle al lento la misma parte **reduce el throughput total** y perjudica a los dos. Y el ejemplo de voz es aún más claro: en voz el retardo es **peor** que la pérdida. Perder 20 ms de un fotograma es imperceptible; agregar 100 ms de cola al otro canal lo hace inservible.

⚠️ **Y un matiz técnico que suma puntos:** aunque el enunciado es de 2001, **hoy ese hack ya no es una buena idea**, por tres motivos concretos:

1. **SACK (`RFC 2018`) lo rompe**: R tendría que reescribir también los bloques SACK, que referencian los números **reales** de bytes recibidos.
2. **Timestamps + PAWS (`RFC 7323`) lo rompen**: un `SN` desplazado es detectable, porque el `TSval` no le corresponde.
3. **No es realmente transparente**: solo vale si los dos extremos usan el mismo protocolo exacto. Si uno usa SACK o un control de congestión que mire los números reales, la conexión **se rompe**.

La solución moderna es obvia una vez que la ves: en vez de **engañar** al transporte, se **marca** el tráfico y se lo prioriza en un router. **DiffServ** (Clase 40) marca el byte TOS/DSCP en la cabecera IP y el router da prioridad por

EF (`RFC 3246`) o por **AF** (`RFC 2597`). El **marking** en el **borde** es lo que realmente permite que una conexión de voz sobreviva a un cuello de botella saturado, y a diferencia del hack **no depende del protocolo de transporte**.

2.6 Ejercicio 6 — El ataque TCP reset (examen 2010-03, Problema 2)

> **Fuente:** Examen `sirc1003` / `eirc1003` — 26 de febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1003.pdf **Texto del examen (extracto):**
>
> Para realizar un ataque de tipo TCP reset que termine con una conexión establecida entre los hosts A y B, un tercer host C debe fabricar un paquete que tiene como dirección IP de origen la de A y como dirección IP de destino la de B y, en su encabezado TCP tiene como puerto origen el que utiliza A en esa conexión TCP, como puerto destino el que utiliza B en ella, un número de secuencia válido (un entero de 32 bits) en la conexión activa entre A y B y el bit RST encendido. Cuando B recibe dicho paquete, inmediatamente corta la conexión con A.
>
> Para que este ataque funcione, alcanza con que el número de secuencia del paquete enviado por C sea válido dentro de la ventana en la que B está esperando paquetes. Dado que las ventanas de transmisión tienen 16KBytes y que el paquete enviado por C no incluye carga útil, se pide: … b) Si queremos atacar un cliente de un servicio conocido (p.e. un browser), pero del que desconocemos el puerto utilizado en el cliente, calcule los tiempos máximos en tirar abajo la misma conexión…

El mecanismo, paso a paso.

```
Conexión legítima A → B:   A:puerto_efímero → B:443   (una conexión TLS de un browser)

C quiere derribarla. Necesita:
1. spoofing de IP origen : poner IP_orig = la de A     ← requiere IP spoofing
2. puerto origen         : poner el efímero de A       ← hay que ADIVINARLO
3. número de secuencia   : que caiga en la ventana de 16 KB de B
4. flag RST = 1

B lo recibe, ve que cae en la ventana, se lo cree, y CORTA la conexión → DoS
```

2.6.1 a) El espacio de secuencia y cuántos paquetes

```
espacio de seq = 2³² = 4 294 967 296 números de secuencia
ventana de B   = 16 KB = 16 384 bytes = 2¹⁴

Para tapar TODO el espacio con ventanas de 2¹⁴:

paquetes = 2³² / 2¹⁴ = 2¹⁸ = 262 144
```

Cada paquete, con la cuenta a la vista:

```
Cabecera IP (sin opciones)  : 20 bytes
Cabecera TCP (sin datos)    : 20 bytes
──────────────────────────────────────
Total por paquete           : 40 bytes = 320 bits

(el enunciado dice "no incluye carga útil", así que son 40 exactos)

Total a transmitir = 2¹⁸ × 320 = 262 144 × 320 = 83 886 080 bits
                   ≈ 80 millones de bits = 10 485 760 bytes ≈ 10 MB
```

2.6.2 b) Cuántos puertos y los tiempos

```
Puertos TCP posibles = 2¹⁶ = 65 536

⇒ En el peor caso: los 65 536 puertos × los 2¹⁸ = 262 144 seq cada uno.
```

ADSL 512/128 Kbps. El atacante **inyecta** paquetes, así que usa la **subida: 128 000 bits/s**.

```
Solo los 2¹⁸ números:  t = 83 886 080 / 128 000       = 655,4 s ≈ 10,9 minutos
Los 65 536 puertos:    t = 655,4 × 65 536 = 42 949 673 s = 497 días ≈ 1,36 años
```

ⓘ **Nota:** la solución oficial escribe `625 s × 65 536 = 40 960 000 s ≈ 474 días`. La diferencia (497 vs. 474) viene de que redondean `2¹⁸ × 320` a "80 millones de bits" en vez de 83,886. **La conclusión es la misma: del orden de 1,4 años.**

Fast Ethernet (100 Mbps).

```
Solo los 2¹⁸ números:  t = 83 886 080 / 100 000 000  = 0,8389 s ≈ 0,84 s  (menos de 1 s)
Los 65 536 puertos:    t = 0,8389 × 65 536 = 54 978 s ≈ 15,3 horas
```

La solución oficial asume ~1 s por puerto: `65 536 s ≈ 18,2 horas` ("algo más de 18hs"), y con 0,1 s por puerto `6 554 s ≈ 1,8 horas`.

Escenario	Sin conocer el puerto	Con los 65 536 puertos
ADSL 512/128 (subida 128 kbps)	655 s ≈ **10,9 min**	42,9 M s ≈ **497 días (1,36 años)**
Fast Ethernet 100 Mbps	0,84 s	54 978 s ≈ **15,3 horas**

Interpretación — y la moraleja. El ataque **es factible** en una red local (15 horas) y **inviable** en Internet con los caudales de subida de 2010. Dos observaciones que valen puntos:

```
1. La solución oficial asume ~1 s POR PUERTO, no 0,84 s. ¿Por qué? Porque el
   atacante no solo genera el paquete: tiene que MEDIR si la conexión
   cayó. Ese ACK o ese timeout son los que consumen el segundo.
   En la práctica el cuello de botella es la DETECCIÓN, no la inyección.

2. El apartado (c) de 2010-03 pregunta si el puerto ALEATORIO de BSD protege.
   La respuesta oficial es:
   "Para un análisis del tipo worst-case, la selección randómica de puertos no
    nos podría ayudar a reducir el espacio de búsqueda. [...] Sin embargo,
    desde el punto de vista práctico elimina la forma de poder fácilmente
    reducir el rango de puertos a atacar."
   Es decir: en el PEOR caso no cambia nada (2¹⁶ siempre). En el caso REAL
   cambia muchísimo, porque un atacante que descubre UN puerto asignado
   puede acotar la búsqueda alrededor de él.
```

2.6.3 c) Las defensas — por qué el problema **es** un problema de números

⚠️ **Actualización a 2026:**

Sigue igual — de memoria: para matar una conexión hacen falta IP spoofing + el **puerto origen** correcto + un `seq` **dentro de la ventana** + el flag **RST**; el espacio de secuencia es de **32 bits** y la ventana típica de **16 KB** (2^{14}).

Cambió — y acá hay que ser preciso:

- **Las defensas reales no están en la ventana, están en los números.** La ventana de 16 KB sigue igual, pero:
- `ISN` **aleatorio**: el número de secuencia inicial se genera con un PRNG criptográficamente seguro, no con un contador. Sin eso el atacante **predice** el `seq` y el ataque baja de 2^{18} intentos a 1.
- **PAWS (`RFC 7323`, 2014)**: obligatorio si hay timestamps. Rechaza segmentos cuyo `TSval` sea viejo, invalidando paquetes acumulados.
- **Validación de RST (*blind reset*)**: un `RST` que llega sin datos asociados y con un `seq` que no calza exactamente con `LastByteAcked` se **ignora**. Esto es lo que más reduce el éxito: el atacante tiene que acertar **el número exacto**, no la ventana.
- **El spoofing de IP hoy es casi imposible**: la **BCP 38** (*Filtering Bogons*) obliga a los proveedores a no enviar a Internet paquetes con origen RFC1918 o de loopback, y muchos operadores hacen ***ingress filtering***: si un paquete llega con IP origen de tu LAN, se descarta. Sin spoofing, el ataque no arranca.
- **Lo que sí es un problema real hoy: los *middleboxes*.** Un `RST` inyectado funciona **perfectamente** desde el mismo lado de la conexión o desde la red local, porque no hay filtro que los separe. **Ese es el ataque que funciona en 2026**: no el hijack desde Internet, sino el `RST` desde la LAN (que es lo que hace un malware o un intermediario hostil). Y la respuesta correcta es **cifrado (TLS/DTLS) o control de acceso a nivel de red**.

PARTE 3 — EJERCICIOS EXTRA

3.1 Extra 1 — Un intercambio con **reordenamiento** (sin pérdida)

Datos. A manda 4 segmentos de 20 bytes desde `seq = 1100`. **No se pierde nada**, pero **se reordena**: S3 llega antes que S2. Sin SACK.

a) ¿Cuántos dupACKs hubo? Uno solo (el de S3). Con **3** habría fast retransmit. Con **1 o 2**, no: TCP espera.

b) ¿Y si el reordenamiento fuera mayor? Si S4 también llegara antes de S2, habría **2 dupACKs**: todavía insuficiente. Con 5 segmentos reordenados (S3, S4, S5 antes de S2) habría **3 dupACKs** ⇒ **fast retransmit sobre S2**, que en realidad **no se había perdido**. Se retransmite un segmento sano: es el costo que justifica el umbral de **3** en lugar de 1.

c) ¿Con SACK qué cambiaría?

Interpretación. El **reordenamiento es el enemigo natural del fast retransmit**: es la razón del umbral 3 y la razón por la que SACK importa tanto.

3.2 Extra 2 — Tiempo total de una transferencia con ventana y RTT dados

Datos. Archivo de **8 MB** (`8 · 10⁶` bytes). `rwnd = 256 KB = 262 144 B`, `cwnd = 1 MB` (manda el `rwnd`). RTT = **40 ms**. Enlace = **100 Mbps**. Sin pérdidas.

a) Ventana efectiva: `min(1 048 576, 262 144) = 262 144 bytes`.

b) Throughput:

```
por ventana:  262 144 / 0,040 = 6 553 600 B/s = 52,4 Mbit/s
por enlace:   100 · 10⁶ / 8  = 12 500 000 B/s = 100 Mbit/s
throughput = min(...) = 6 553 600 B/s   ← manda la ventana
```

c) Tiempo: `t = 8 · 10⁶ / 6 553 600 = 1,2207 s ≈ 1,22 s`.

d) Si `rwnd = cwnd = 4 MB`:

```
por ventana:  4 194 304 / 0,040 = 104 857 600 B/s = 838,9 Mbit/s
por enlace:   100 Mbit/s  ← ahora manda el ENLACE
t = 8 · 10⁶ / 12 500 000 = 0,64 s
```

e) ¿Cuál es la ventana que hace falta para gastar los 100 Mbps? `BDP = 100 · 10⁶ bits/s × 0,040 s = 4 · 10⁶ bits = 500 000 bytes`.

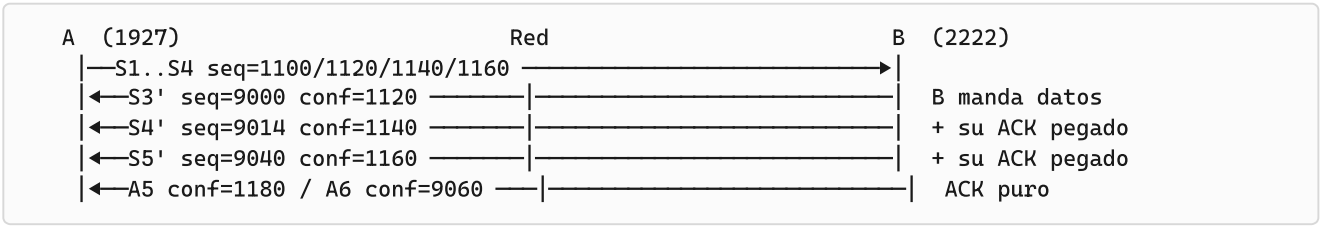
Configuración	Ventana efectiva	Throughput	Tiempo para 8 MB
`rwnd = 256 KB`	262 144 B	52,4 Mbit/s	**1,22 s**
`rwnd = cwnd = 4 MB`	4 194 304 B	100 Mbit/s	**0,64 s**
`rwnd = cwnd = BDP = 500 KB`	500 000 B	100 Mbit/s	**0,64 s**

Interpretación — y el detalle fino que da puntos. La fila 2 muestra que **agrandar la ventana más allá del BDP no sirve**: con 4 MB te da lo mismo que con 500 KB, porque el **enlace** es el techo. Pero para *llegar* a ese

techo la ventana tiene que estar **por encima** del BDP durante un rato, porque al principio `cwnd` es chico. Y en la práctica la transferencia tarda más que 0,64 s: los primeros RTTs van con ventana chica mientras `cwnd` crece por *slow start*.

3.3 Extra 3 — full duplex: B manda datos mientras A sigue mandando

Datos. A manda 4 segmentos de 20 bytes desde `seq = 1100`. B, en vez de solo asentar, **manda su respuesta** en 3 segmentos desde `seq = 9000`. Full duplex: los ACKs viajan **cruzados** (el `conf = 1120` de B confirma datos de A; el `conf = 9000` de A confirma datos de B), y cuando B se queda sin datos que mandar aparece un **ACK "puro"** (`cantidad_datos = 0`).



Lo que hay que ver: (1) los `seq` son **independientes por dirección** (A numera 1100..1179, B numera 9000..9060): son dos espacios distintos, así que ambos pueden tener el mismo `seq` sin conflicto; **(2)** cada sentido tiene su **propia ventana**, y cada segmento lleva el **`rwnd` del otro**; **(3)** aparece un ACK "puro" cuando se acaba un lado; **(4)** ningún lado espera. Totales: `A→B = 4×20 = 80` (confirmado con `conf = 1180`) y `B→A = 3×20 = 60` (con `conf = 9060`).

Interpretación. Esta es la situación real de **cualquier** transferencia HTTP: el cliente pide, el servidor responde, y los dos mandan datos al mismo tiempo con los ACKs pegados. Si en un diagrama de examen dibujás los ACKs como flechas separadas, casi seguro estás mal.

📌 Resumen

1. Las **6 reglas** resuelven cualquier diagrama del bloque: `seq` de bytes, `ack` = próximo esperado, ACK acumulativo, fuera de orden ⇒ dupACK, timeout es el último recurso, `ssthresh` decide el régimen.
2. `seq` siguiente = `seq` actual + largo. Nada más hay que saber para calcular números.
3. **`2011-03 P4a`**: 4 segmentos (1100, 1120, 1140, 1160), 4 ACKs (1120, 1140, 1160, 1180), fin con 0 bytes en vuelo.
4. **`2011-03 P4b`**: se pierde el primero ⇒ B manda **`ack = 1100` tres veces**, y al recibir el retransmitido contiguo manda **`ack = 1180`**.
5. **3 dupACKs ⇒ fast retransmit** (Reno: `cwnd/2`); **timeout ⇒ slow start** (`cwnd = 1 MSS`) y el RTO se duplica.
6. **Piggybacking**: el ACK viaja pegado a un segmento de datos **del sentido contrario**; ahorra un paquete entero.
7. **Dos conexiones + router que pierde**: se **desplaza el espacio de numeración** (`dif_SN`) para que la pérdida sea transparente. La asimetría es **a propósito**: optimiza eficiencia y percepción, no igualdad.
8. **Ataque TCP reset**: $2^{32} / 2^{14} = 2^{18} = 262\,144$ paquetes por puerto; con $2^{16} = 65\,536$ puertos, ≈ **497 días** por ADSL o **15 horas** por Fast Ethernet.
9. La defensa no es la ventana: es el **`ISN` aleatorio**, **PAWS**, la **validación de RST** y el ***ingress filtering*** contra el spoofing.

10. **Reordenamiento** y **piggybacking** son los dos casos que más rompen los diagramas "de libro" — y los dos se preguntan.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. A manda un segmento con `seq = 5000` y 30 bytes. ¿Qué `seq` lleva el siguiente?

<details>
<summary>Respuesta</summary>
</details>

2. B recibió S1 (seq 1100, 20 B) y S3 (seq 1140, 20 B), pero no S2. ¿Qué `ack` manda y qué hace con S3?

<details>
<summary>Respuesta</summary>
</details>

3. En `2011-03 P4b`, ¿cuántos segmentos y cuántos ACKs viajan en total?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Qué diferencia hay, en el `cwnd`, entre recuperar por 3 dupACKs y por timeout?

<details>
<summary>Respuesta</summary>
</details>

5. En el problema de `2001-12`, ¿por qué hay que alterar **los dos** campos?

<details>
<summary>Respuesta</summary>
</details>

6. Ataque TCP reset con ventana de 16 KB. ¿Cuántos paquetes hay que mandar si conocemos IP y ambos puertos? ¿Y si además no conocemos el puerto?

<details>
<summary>Respuesta</summary>
</details>

7. ¿Eligió BSD puertos aleatorios: eso protege contra el ataque?

<details>
<summary>Respuesta</summary>
</details>

8. ¿Qué pasa con el `cwnd` si el problema del examen 2011-03 P4 lo modelamos con SACK activado?

<details>
<summary>Respuesta</summary>
</details>

🔗 Referencias

- **Examen** `sirc1103` — P4 (10 pts): ★ 4 segmentos de 20 bytes desde `seq = 1100`, con y sin pérdida del primero · Problema 1 (30 pts): ECN (Clase 21)
- **Examen** `sirc0112` — Problema 1: dos conexiones TCP por un router que pierde · Problema 2: el flag PUSH
- **Examen** `sirc1003` — Problema 2 (30 pts): ataque TCP reset, rango de puertos y tiempos
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 §6.5–6.7 (pág. ~590–610 del PDF)
- `RFC 9293` — TCP (2022, *Internet Standard*) · `RFC 5681` — TCP Congestion Control (2009)

- `RFC 2018` — SACK (1996) · `RFC 7323` — Timestamps, Window Scale y PAWS (2014)
- `RFC 6928` — Initial Window = 10 MSS (2013)
- **Clase 19** — fiabilidad: RTO, Jacobson/Karels, BDP, PSH · **Clase 21** — control de congestión
- **Clase siguiente:** 23 — Repaso de transporte + Simulacro 3

Clase 23

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El mapa del bloque
1.2 Las fórmulas que hay que saber de memoria
1.3 Los puertos que hay que saber
1.4 Los flags TCP
1.5 Los 11 estados de la máquina de TCP
1.6 La tabla TCP vs. UDP — 12 filas
1.7 Las tres tablas de memoria en una sola vista
Error 1 — "El ACK confirma un segmento"
Error 2 — "Si se perdió el primero, mando el segundo otra vez"
Error 3 — "El timeout es para detectar la pérdida"
Error 4 — "Flujo y congestión son lo mismo"
Error 5 — "La ventana efectiva es cwnd + rwnd"
Error 6 — "Con PSH el receptor lee un mensaje"
Error 7 — "La pérdida siempre significa congestión"
📝 ENUNCIADO DEL SIMULACRO (sin respuestas)
📌 Tablero de respuestas del simulacro (para el auto control)
✅ Autoevaluación del bloque
🔗 Referencias

Los 6 bloques del curso

Bloque: C (Transporte) · **Semana:** 8 · **Duración estimada:** 120 min (repaso 50 + simulacro 70) **Correlativa:** Clase 22 (taller TCP) · **Sigue con:** Clase 24 (la capa de red: las dos funciones) **Material de referencia:** Tanenbaum cap. 6 completo · RFC 9293, RFC 5681, RFC 9438, RFC 2018, RFC 7323, RFC 6928, RFC 3168

🎯 Al terminar esta clase vas a poder

1. Reconstruir de memoria el mapa de las **8 clases** del bloque de transporte (16 a 23).
2. Escribir **de memoria** las fórmulas que aparecen en los exámenes de transporte.
3. Decir los **11 estados** de la máquina de TCP y la condición de cada transición.
4. Completar una tabla **TCP vs. UDP** de 12 filas sin mirar.
5. Identificar y corregir **las 7 confusiones** que más se castigan en este bloque.
6. Resolver un examen completo de 100 puntos en 3 horas: 5 teóricas + 2 problemas.

> **Prerequisito del examen: TODOS.** Esta clase cierra el bloque C. El simulacro reproduce el formato real (`2011-03`, `2010-12`, `2011-02`): 5 teóricas + 2 problemas de 30 pts, 3 horas, **mínimo 15 puntos en**

> **teoría para aprobar**.

PARTE 1 — REPASO SINTÉTICO DEL BLOQUE

1.1 El mapa del bloque

```
┌──────────────────────────────────────────────────────────────────┐
│ 16 MULTIPLEXACIÓN        2-tupla UDP vs 4-tupla TCP · mux/demux  │
│ 17 CABECERA TCP          campos · flags · opciones · MSS         │
│ 18 ESTABLECER/CERRAR     handshake 3-vías · 11 estados · TIME-WAIT│
│ 19 FIABILIDAD            seq · timers · RTO · BDP · SACK · PSH   │
│ 20 FLUJO                 rwnd · min(cwnd,rwnd) · zero window     │
│ 21 CONGESTIÓN            slow start · CA · fast recovery · ECN   │
│ 22 TALLER                diagramas de intercambio resueltos      │
│ 23 REPASO + SIMULACRO                                            │
└──────────────────────────────────────────────────────────────────┘

UDP = una carta      TCP = telegrama certificado con acuse de recibo

Las garantías de TCP se COMPRAN con tres mecanismos:
  números de secuencia → orden, sin duplicados, saber qué reenviar
  timers + RTO         → saber CUÁNDO reenviar
  ventana deslizante   → no esperar el ACK antes de seguir mandando
y se PAGAN con dos controles:
  rwnd → no desbordar al receptor
  cwnd → no saturar la red
```

1.2 Las fórmulas que hay que saber de memoria

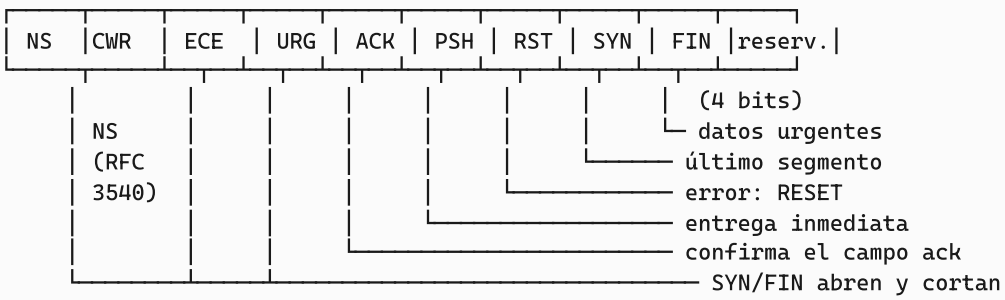
```
① RTO = SRTT + 4 · RTTVAR
       SRTT   ← (1−α)·SRTT + α·R              con α = 7/8
       RTTVAR ← (1−β)·RTTVAR + β·|SRTT − R|    con β = 1/4
       1ª muestra: SRTT = R, RTTVAR = R/2     (piso y techo)

② BDP = ancho_de_banda × RTT          [bytes] = [bits/s] × [s] / 8

③ ventana efectiva = min(cwnd, rwnd)         ← MÍNIMO, nunca suma

④ throughput = min( ventana_efectiva / RTT , ancho_de_banda_del_enlace )

⑤ MSS = MTU − 20 (IP) − 20 (TCP) = 1500 − 40 = 1460   (1448 con timestamps)

⑥ tiempo = tamaño_del_archivo / throughput

⑦ ataque TCP reset:  paquetes = 2³² / ventana = 2³² / 2¹⁴ = 2¹⁸ = 262 144
                     si no se conoce el puerto:  × 2¹⁶ = 65 536 puertos

⑧ módulos: puerto 16 bits → 2¹⁶ = 65 536 (0…65 535)
            número de secuencia 32 bits → 2³² = 4 294 967 296

⑨ d_trans = L / R
```

1.3 Los puertos que hay que saber

Puerto	Servicio	Proto	Por qué lo importan
20 / 21	FTP datos / control	TCP	dos puertos, control separado
22	SSH	TCP	también `22/udp` y `22/sctp` en IANA
25	SMTP	TCP	envío, *push*
53	**DNS**	**UDP y TCP**	⚠️ los dos: consultas y respuestas grandes / *zone transfer*
67 / 68	DHCP server / client	UDP	⚠️ **nunca TCP**: el cliente aún no tiene IP
80	HTTP	TCP	también `80/udp` y `80/sctp`
110 / 995	POP3 / POP3S	TCP	*pull*, borra del servidor
143 / 993	IMAP / IMAPS	TCP	*pull*, sincroniza
161 / 162	SNMP / trap	**UDP**	dos puertos: consulta y notificación
179	BGP	TCP	⚠️ un protocolo de **enrutamiento** sobre transporte **fiable**
443	**HTTPS**	**TCP y UDP**	⚠️ el UDP es **HTTP/3 (QUIC)**
853	DNS sobre TLS (DoT)	TCP	DNS cifrado
546 / 547	DHCPv6 client / server	UDP	el DHCP de IPv6

```
⚠️ 53 UDP+TCP   → si solo decís "DNS usa UDP", te falta la mitad
⚠️ 443 TCP+UDP  → el UDP es HTTP/3; HTTP/3 NO usa TCP para los datos
⚠️ 179 TCP      → BGP necesita fiabilidad
⚠️ 67/68 UDP    → DHCP en UDP porque el cliente aún no tiene dirección

Rango: 16 bits ⇒ 0 … 65 535.   0…1023 = bien conocidos / privilegiados
       (en Unix, `bind()` sobre ellos requiere root)
```

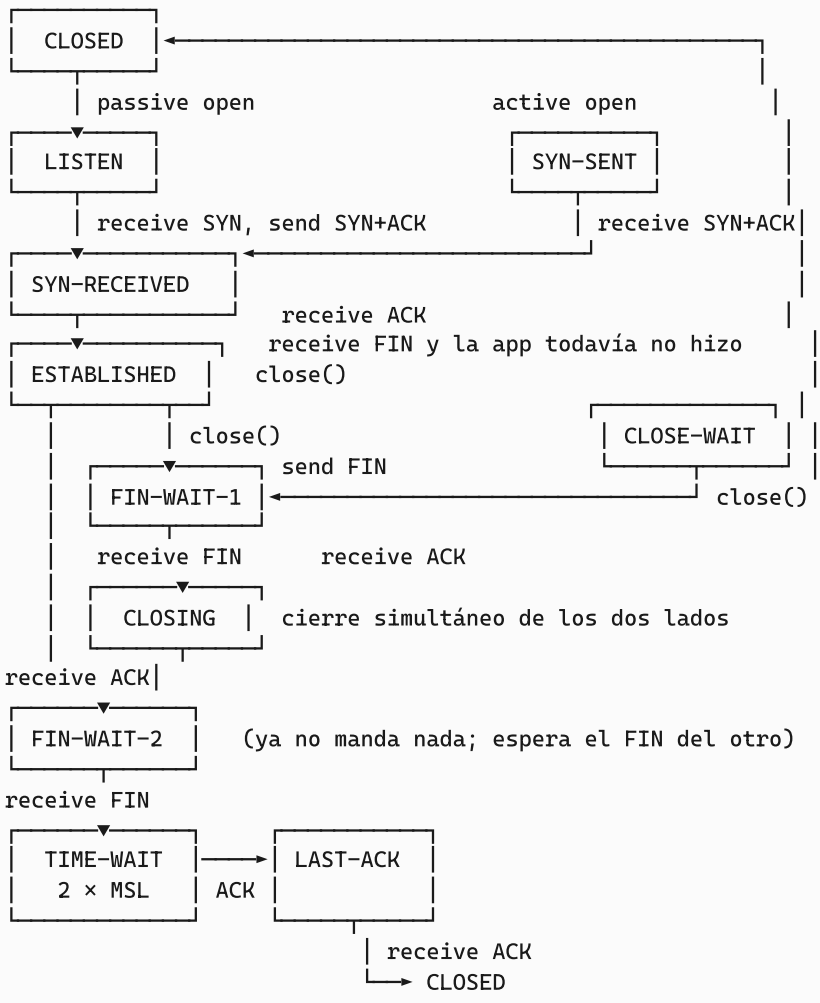
1.4 Los flags TCP



Flag	Significado	Dónde se usa
SYN	sincronizar	abre la conexión (handshake)
ACK	confirmar	`1` en casi todo lo que sigue al handshake
FIN	finalizar	media conexión de cierre
RST	reset	**corta la conexión de inmediato**, sin negociación
PSH	push	sugiere entregar ya a la aplicación (**no garantiza nada**)
URG	urgente	activa el *urgent pointer*
ECE / CWR	control de congestión	**ECN** (`RFC 3168`): ECE=1 "hubo congestión", CWR=1 "bajé mi ventana"
NS	no-sacudir	extensión: no limpiar el campo ECN al calcular el checksum

```
★ El flag PSH NO garantiza un mensaje por read() (Clase 19 §1.8, 2001-12 Prob. 2)
★ RST no se negocia: aborta sin cierre ordenado
★ SYN y FIN abren/cortan; ACK y PSH son de "tránsito"
★ ECE/CWR existen SOLO por ECN; sin ECN van en 0
```

1.5 Los 11 estados de la máquina de TCP



Los once, numerados: CLOSED, LISTEN, SYN-SENT, SYN-RECEIVED, ESTABLISHED, FIN-WAIT-1, FIN-WAIT-2, CLOSE-WAIT, CLOSING, LAST-ACK, TIME-WAIT.

```
★ TIME-WAIT = 2 × MSL  (Maximum Segment Lifetime)
  ¿Por qué 2×MSL y no 1? Un segmento perdido puede reaparecer como duplicado
  tarde. Si el socket se cerrara a los MSL, ese duplicado con la misma
  4-tupla caería dentro de una conexión NUEVA y corrompería el flujo.
  Con 2×MSL se garantiza que el viejo ya expiró en la red.

★ LAST-ACK: mandó su FIN y espera el ACK del FIN ajeno. No es lo mismo que
  TIME-WAIT: acá ya no manda nada.
★ CLOSING: cierre simultáneo. Raro, y es el que hay que saber dibujar.
★ Un RST en cualquier estado → CLOSED, sin negociación y sin TIME-WAIT.
★ Costo: TIME-WAIT retiene un puerto por 2×MSL ⇒ agotamiento de puertos
  efímeros en servidores con muchos clientes cortos.
```

1.6 La tabla TCP vs. UDP — 12 filas

#	Aspecto	UDP	TCP
1	**Conexión**	sin conexión (*connectionless*)	**con conexión**, 3-vías
2	**Fiabilidad**	**no**: se pierde y sigue	**sí**: ACK + timers + retransmisión
3	**Orden**	**no garantizado**	**sí**, por número de secuencia
4	**Latencia**	**baja**, sin handshake	**1 RTT extra** de handshake
5	**Control de congestión**	**ninguno** (¡por eso QUIC!)	**sí**: `cwnd`, slow start, ECN
6	**Control de flujo**	ninguno	**sí**: `rwnd`
7	**Cabecera**	**8 bytes** (2+2+2+2)	**20 bytes** mínimo, hasta 60
8	**Identificación**	**2-tupla** (IP, puerto) *(el kernel usa la 5-tupla)*	**4-tupla** (IP_o, p_o, IP_d, p_d)
9	**Multiplexación**	**1 socket habla con muchos** a la vez, mismo puerto	**1 conexión por 4-tupla**: 1 socket por conexión
10	**Checksum**	obligatorio en IPv6, opcional en IPv4	**siempre obligatorio**
11	**Frontera de mensaje**	**SÍ**: 1 datagrama = 1 mensaje	**NO**: flujo de bytes; los mensajes los pone la aplicación
12	**Usos típicos**	DNS, DHCP, SNMP, RTP/VoIP, QUIC (HTTP/3)	HTTP/1.1 y HTTP/2, SSH, SMTP, POP3/IMAP, FTP, BGP

Las tres filas donde más se falla:

```
Fila 8 — 2-tupla vs 4-tupla:
  UDP  → el socket se identifica con (IP, puerto) ⇒ un solo proceso
         (el servidor DNS, puerto 53) atiende a TODOS los clientes.
  TCP  → cada 4-tupla distinta necesita un socket distinto ⇒ el SO crea
         uno por accept().

Fila 11 — frontera de mensaje:
  UDP  → cada datagrama es una unidad: leés N bytes, es UN mensaje.
  TCP  → read() puede devolver medio mensaje o dos pegados. PSH no lo
         arregla (Clase 19 §1.8).

Fila 5 — por qué HTTP/3 necesita QUIC:
  UDP solo no tiene control de congestión, y eso sería grave en Internet.
  QUIC (RFC 9000/9002) reimplementa fiabilidad, control de congestión y
  PLPMTUD ENCIMA de UDP. No es "HTTP/3 usando UDP": es un transporte
  nuevo que compite con TCP.
```

1.7 Las tres tablas de memoria en una sola vista

```
¿Qué puerto?              ¿Qué flags?              ¿En qué estado?
──────────────            ──────────────           ──────────────────────
DNS      53 UDP/TCP       handshake: SYN,          CLOSED → LISTEN (servidor)
HTTP     80 TCP           SYN+ACK, ACK              SYN-SENT → SYN-RECEIVED
HTTPS   443 TCP/UDP       datos: ACK, PSH,          → ESTABLISHED
SSH      22 TCP           cierre: FIN, ACK          ESTABLISHED → FIN-WAIT-1
SMTP     25 TCP                                     → FIN-WAIT-2 → TIME-WAIT
DHCP  67/68 UDP           error: RST                → LAST-ACK → CLOSED
BGP     179 TCP           sync: ECE, CWR           (también: CLOSING)
SNMP 161/162 UDP
```

⚠️ Actualización a 2026

Sigue igual — de memoria, no se toca: - **4-tupla** identifica una conexión TCP; el socket UDP se trata como **2-tupla**. - Los **11 estados** son los mismos. **Three-way handshake**, cierre de **4 vías**, **TIME-WAIT de 2×MSL**. - **Números de secuencia de 32 bits**, **ACK acumulativo**, **ventana deslizante**. - **Los ACKs son de bytes, no de segmentos**: `ack` = próximo byte esperado. - Sliding window, dupACKs, delayed ACK, Nagle, persist/keepalive, RTO, Jacobson/Karels, fast retransmit, slow start, congestion avoidance, fast recovery. - **MTU/MSS**, opciones en el SYN (MSS, SACK permitted, Timestamps, Window Scale), checksum con pseudo-cabecera, descarte de segmentos fuera de ventana.

Lo que cambió — las correcciones obligatorias del bloque:

En 2011 se decía	En 2026
`RFC 793` es TCP	**`RFC 9293`** (2022, *Internet Standard*; consolida 793 + RFC 879, 2873, 6093, 6429, 6528, 6691)
Reno / Tahoe	**CUBIC = `RFC 9438`** (2023, *Standards Track*), default de Linux, Windows y Apple; obsolesce `RFC 8312`
"quizá BBR"	**BBR nunca fue RFC**: `draft-ietf-ccwg-bbr`, BBRv3, *Experimental*. ⚠️ **`RFC 8312` es CUBIC, no BBR**
IW = 1 MSS	**IW = 10 MSS** (`RFC 6928`, 2013)
SACK opcional	`RFC 2018` es **comportamiento estándar**; Timestamps + Window Scale = `RFC 7323` (obsolesce `1323`); **PAWS obligatorio** si se activan
ECN = `RFC 3168`	`RFC 3168` + **`RFC 9768` AccECN (2026)**; L4S = `RFC 9331`
—	**QUIC** (`RFC 9000/9001/9002`, 2021): transporte completo que **reemplaza** a TCP en HTTP/3
—	**MPTCPv1** (`RFC 8684`, 2020): multipath, real pero marginal (~0,4 % de bytes pico)

⚠️ Tres errores que NO son obsolescencia sino directamente falsos:

Se dice	Realidad
"`RFC 8951` = ECN en multihop"	**FALSO**: es una aclaración de **EST** (Enrollment over Secure Transport)
"`RFC 7560` = ECN++"	**FALSO**: es solo el **documento de problema** de ECN; la especificación quedó en `RFC 9768`
"`RFC 8984` depreca el half-duplex Ethernet"	**FALSO**: `RFC 8984` es **JSCalendar**. La decisión es de **IEEE 802.3**, y ni siquiera es total

> **Consecuencia para el examen:** los **mecanismos** se preguntan **exactamente igual** que en 2011. Lo único que se corrige es **la etiqueta normativa** y, si preguntan por despliegue real, quién está en el default (CUBIC, no Reno).

PARTE 2 — LOS ERRORES FRECUENTES

Los siete que más puntos se pierden en los exámenes de transporte.

Error 1 — "El ACK confirma un segmento"

❌ "A mandó S2, entonces A2 confirma S2." ✅ El ACK es **acumulativo y numera bytes**: `ack` = **próximo byte esperado**. Un `ack = 7120` confirma los 120 bytes del 7000 al 7119 de una sola vez. **Por qué importa:** si razonás por segmentos, fallás todos los diagramas con pérdida, que es justo lo que se pregunta (`2011-03 P4`).

Error 2 — "Si se perdió el primero, mando el segundo otra vez"

❌ Ante `ack = 7000` repetido, "el que falta es el primero, así que mando el segundo". ✅ `ack = 7000` significa que **falta a partir del 7000**, o sea el **primero** (S1, `seq = 7000`). Es el que hay que retransmitir. **Por qué importa:** arruina el apartado (b) de `2011-03 P4` .

Error 3 — "El timeout es para detectar la pérdida"

❌ "TCP usa un timer para saber cuándo se perdió un segmento." ✅ El timeout existe porque, si el ACK **no llega**, TCP **no puede distinguir** tres cosas: se perdió el dato, se perdió el ACK, o tardó. Las tres se ven **exactamente igual**. La única salida es retransmitir, y para eso hay que saber **cuándo** → el timer. **Por qué importa:** es el apartado (b) de `2010-12 P4` , la pregunta conceptual más fina.

Error 4 — "Flujo y congestión son lo mismo"

❌ "Ambos bajan la ventana, así que son lo mismo." ✅ **Flujo** = el receptor no puede absorber → `rwnd` , lo controla el **receptor** y lo anuncia en el campo `window` . **Congestión** = la red no puede transportar → `cwnd` , lo controla el **emisor** estimando, y nadie se lo dice. **Por qué importa:** `2009-07 P1a` (10 pts) es esta pregunta. Y si bajás `cwnd` cuando el `rwnd` es 0, **castigás a la red por un problema del receptor** y no arreglás nada.

Error 5 — "La ventana efectiva es cwnd + rwnd"

❌ "Se suman porque son cosas distintas." ✅ Es el **mínimo**: `min(cwnd, rwnd)` . Ambas son **cotas superiores**. Con la suma, el excedente que la red no tomó se acumularía en el buffer del receptor y lo desbordaría **justo cuando la red va peor**. **Por qué importa:** es la mitad de `2011-03 Problema 1c` (12 pts) y aparece en toda `2009-07 P1a` .

Error 6 — "Con PSH el receptor lee un mensaje"

❌ "Puse PSH, así que el `read()` del otro devuelve exactamente el mensaje." ✅ PSH es **una sugerencia** de "no tengo más datos en el buffer". Es **un bit**: no transporta información de frontera de mensaje. Si llegan dos

segmentos con PSH **fuera de orden**, el `read()` devuelve **los dos pegados**. Las fronteras las pone la aplicación. **Por qué importa:** es `2001-12 Problema 2` entero, y es la fuente del error de cualquiera que programme sockets.

Error 7 — "La pérdida siempre significa congestión"

❌ "Se perdió un paquete ⇒ la red está saturada." ✅ **No.** También por CRC de un enlace, buffer del receptor lleno (control de **flujo**), ACL/firewall, fragmento descartado en IPv4, corte de un enlace. **TCP asume** que sí, porque es la apuesta con mejor promedio: castigar a la red de vez en cuando sale más barato que saturarla. **Por qué importa:** es `2009-07 P1c`, y revela si entendiste que TCP **infiere** la congestión en lugar de que se la **notifiquen**.

⚠️ Errores típicos del simulacro (los que no son de este bloque)

```
⚠️ No cites "la RFC de TCP" como RFC 793 → es RFC 9293
⚠️ No digas "el timeout baja cwnd a cwnd/2" → a 1 MSS (Tahoe y Reno coinciden)
⚠️ No digas "3 dupACKs ⇒ cwnd = 1 MSS" en Reno ⇒ cwnd/2 (eso es Tahoe)
⚠️ No digas "DNS usa el puerto 53 UDP" solo ⇒ también TCP
⚠️ No digas "la cabecera TCP son 20 bytes" sin aclarar "mínimo"
```

PARTE 3 — SIMULACRO 3

> **Instrucciones (reproducción del formato real de FING):** - Examen individual, **sin material**, **3 horas**. - **5 Preguntas Teóricas** (~40 pts) + **2 Problemas Prácticos** (30 pts cada uno). - Aprobación: **60 puntos**. - ⚠️ **Es obligatorio responder correctamente al menos 15 puntos en las preguntas teóricas.** Si no llegás a 15 en teoría, no aprobás aunque sumes 60 con los problemas. - Empezá cada pregunta y cada problema en una hoja nueva. - Dudas de letra solamente; **ninguna** en los últimos 30 minutos. - Para los problemas podés suponer que disponés de los tipos básicos (lista, cola, archivo, string) y sus funciones (`tail()`, `crear()`, `concatenar()`).
>
> ⏱️ **Cronómetro sugerido (180 minutos):**

Etapa	Min	Objetivo
Lectura inicial	5	leer entero, elegir por dónde empezar
T1 (10 pts)	15	
T2 (10 pts)	15	
T3 (8 pts)	12	
T4 (10 pts)	15	
T5 (8 pts)	12	**69 min de teoría, ~46 pts**
Problema 1 (30 pts)	50	pseudocódigo estilo C: leer bien el enunciado
Problema 2 (30 pts)	45	números: cuentas a la vista, TABLAS antes que prosa
Repaso final	5	re-leer los números de los dos problemas
Total	**180**	

> **El consejo de la cátedra, y es el que más cuesta aprender:** los problemas prácticos se corrigen **por lo que escribiste**, no por lo que pensaste. Si no te da el número exacto, **dejá la fórmula y la cuenta a medias** con la unidad anotada: parcialmente es puntos. En cambio un apartado en blanco es cero.

📝 ENUNCIADO DEL SIMULACRO (sin respuestas)

Preguntas Teóricas

T1 (10 puntos). Un protocolo de transporte entrega a la aplicación un flujo de bytes y promete que **no se pierde nada, no se duplica nada y todo llega en orden**. a) Explique qué mecanismo permite cumplir cada una de esas tres promesas, y por qué el `checksum` por sí solo no alcanza. b) Suponga que el emisor manda un segmento, no recibe el ACK y se vence el timer. Enumere **todas** las explicaciones posibles de ese fallo y explique por qué el protocolo **no puede distinguir entre ellas**. ¿Qué decisión queda, y qué falta para poder tomarla? c) Explique cómo estima el protocolo el valor del timer, y qué ocurre con ese valor tras varios timeouts consecutivos.

T2 (10 puntos). En un protocolo de transporte con control de congestión y de flujo: a) Diferencie el control de flujo del control de congestión, indicando **qué variable** representa a cada uno, **quién** la decide y **cómo** se

comunica al otro extremo. b) Escriba la expresión de la ventana efectiva y justifique por qué es un **mínimo** y no una suma. c) El receptor anuncia una ventana de recepción igual a cero. Detalle qué hace el emisor, con qué periodicidad, y justifique por qué esa situación **no** debe tratarse como congestión. d) Mencione dos causas de pérdida de paquetes que **no** se deban a congestión.

T3 (8 puntos). Sobre el protocolo TCP: a) Escriba los **11 estados** de su máquina de estados. b) Detalle la secuencia de **mensajes** del establecimiento de una conexión, indicando qué flags activa cada uno y cómo se numeran los bytes iniciales en ambos sentidos. c) Explique por qué, tras el cierre ordenado, un extremo permanece en `TIME-WAIT` durante `2 × MSL`, y qué ocurriría si no lo hiciera.

T4 (10 puntos). Compare los dos protocolos de transporte de la familia TCP/IP. a) Para cada uno de los siguientes aspectos, indique la diferencia: (i) existencia de conexión; (ii) número de bytes de cabecera; (iii) forma de identificar una comunicación; (iv) manejo de la pérdida de datos; (v) frontera de mensajes. b) Explique por qué un protocolo que entrega datagramas sin conexión **no puede** implementar control de congestión, y qué construcción tecnológica reciente resuelve esa carencia.

T5 (8 puntos). Un desarrollador afirma: "*mandé un mensaje con el flag PUSH activado, así que el programa remoto lo recibirá en una llamada al sistema de lectura distinta de las demás*". a) Argumente por qué la afirmación es incorrecta. b) Describa un procedimiento de laboratorio que demuestre empíricamente la falsedad de la afirmación. c) Indique la práctica correcta para garantizar la separación de mensajes en un protocolo de aplicación, y nombre un protocolo de transporte que sí tenga noción nativa de mensaje.

Problemas Prácticos

Problema 1 (30 puntos) — Programación. Un protocolo de transporte de la familia TCP implementa control de congestión y notifica las pérdidas. Cuando detecta un evento adverso, el emisor **reduce su ventana de congestión** y entra en un régimen de crecimiento más conservador hasta volver al anterior.

Primitivas disponibles:

```
void setVentanaCong(int valor);   // fija el tamaño de la ventana de congestión
int  getVentanaCong();            // lee el tamaño actual
void setUmbral(int valor);        // fija el "umbral" que separa los dos regímenes
int  getUmbral();
void setNotificarPerdida();       // marca los próximos segmentos a enviar con la
                                  // bandera que avisa "hubo pérdida"
```

Tipos:

```
Segmento {
  int  puerto_destino, puerto_origen;
  long numero_secuencia;          // número de secuencia del primer byte
  long numero_confirmacion;       // próximo byte esperado por el otro lado
  int  cantidad_datos;            // bytes de datos del segmento
  bool marca_perdida;             // la bandera de "hubo pérdida"
  IP  encapsulated;               // la cabecera IP del paquete que lo trajo
  ...
}
IP {
  IP  origen, destino;
  enum estado_ecn { NO_SOPORTA, SOPORTA, CONGESTION_EXPERIMENTADA }
  ...
}
```

Se pide: - **a) (6 pts)** ¿Qué eventos, en un protocolo **sin** notificaciones explícitas, interpreta como indicio de pérdida? Explique cómo distingue un evento grave de uno leve, y qué valor toma la ventana de congestión en cada uno. - **b) (12 pts)** Implemente `int detectarCongestion(Segmento s)`, que devuelve 1 si hay congestión y 0 si no. **Si se invoca con `s == NULL`, es porque se venció un timer esperando una confirmación.** El protocolo debe: (i) detectar los ACKs duplicados; (ii) resetear el contador cuando hay progreso real; (iii) propagar la marca de "hubo pérdida" si el segmento recibido la trae; (iv) avisar al otro extremo si la cabecera IP recibida viene marcada por un router como congestionada. - **c) (12 pts)** Implemente `void controlarCongestion(int hayCongestion)`. Ante congestión, el protocolo debe dividir por dos la ventana actual y guardar ese valor como **umbral**; fijar la ventana en **un MSS**; y volver al régimen de arranque. Sin congestión: en el régimen de arranque, **sumar un MSS por cada confirmación recibida** (y cambiar de régimen al superar el umbral); en el régimen de evitación, **sumar `MSS² / ventana` por cada confirmación recibida**. - **d) (2 pts)** El protocolo anuncia en cada segmento el tamaño de su ventana de recepción. Explique, en un párrafo, cómo se combina esa ventana de flujo con la de congestión, y qué se rompe si alguien las sumara.

Problema 2 (30 puntos) — Números y diagramas. Dos procesos, **A** (IP `10.0.0.1`, puerto `5000`) y **B** (IP `10.0.0.2`, puerto `80`), mantienen una conexión TCP abierta. El enlace que los une tiene una ventana de transmisión de **32 KB**.

En un instante dado, A envía a B **cinco** segmentos consecutivos de **24 bytes** de datos cada uno, sin datos previos sin confirmar. El **primer** segmento tiene número de secuencia `7000`. Las latencias son aproximadamente uniformes y el valor del timer es **mayor** que el tiempo de llegada de la primera confirmación.

Se pide: - **a) (8 pts)** Escriba el valor del campo número de secuencia de **cada uno** de los cinco segmentos, y el campo número de confirmación de las respuestas de B, asumiendo que **no hay pérdidas** y que B no tiene datos que enviar. Dibuje el intercambio completo. - **b) (10 pts)** Dibuje ahora el intercambio si **se pierde el primer segmento**. Muestre los valores de los campos y explique, con las dos señales de la RFC, **qué dos eventos** pueden provocar la retransmisión, y **qué valor toma la ventana de congestión** en cada uno. - **c) (6 pts)** El campo de longitud de ventana de la cabecera TCP tiene 16 bits. Calcule el **número máximo de bytes** que puede confirmar en cada paquete el segmento de B con el tamaño de ventana indicado, y el **número de segmentos de 24 bytes** que necesitaría para confirmar los 120 bytes del apartado (a). Si el protocolo negotiating *escalado de ventana*, ¿cuánto cambia? - **d) (6 pts)** Un atacante que no conoce el puerto origen de A quiere cerrar esa conexión mandando un paquete RST. Calcule **cuántos números de puerto** debe adivinar, **cuántos números de secuencia** debe probar para cada uno, y **cuánto tiempo** tardaría con un enlace de **100 Mbps**, sin considerar tiempos de generación de paquetes. Muestre todas las cuentas.

PARTE 4 — RESOLUCIÓN DEL SIMULACRO

> Todo lo que sigue está en `<details>`. **Resolvé primero el enunciado.**

<details>
<summary>▶ T1 — Resolución (10 pts)</summary>
</details>

<details>
<summary>▶ T2 — Resolución (10 pts)</summary>
</details>

<details>
<summary>▶ T3 — Resolución (8 pts)</summary>
</details>

<details>
<summary>▶ T4 — Resolución (10 pts)</summary>
</details>

<details>
<summary>▶ T5 — Resolución (8 pts)</summary>
</details>

<details>
<summary>▶ Problema 1 — Resolución (30 pts)</summary>
</details>

<details>
<summary>▶ Problema 2 — Resolución (30 pts)</summary>
</details>

📌 Tablero de respuestas del simulacro (para el auto control)

Pregunta	Máx.	Lo que no puede faltar
T1	10	los tres mecanismos · las tres causas indistinguibles · Jacobson/Karels · backoff
T2	10	`rwnd` / `cwnd` con quién decide · `min()` justificado · sonda de 1 byte exponencial · 2 pérdidas no congestivas
T3	8	los 11 estados · handshake con `SYN` consumiendo un número · `TIME-WAIT` 2×MSL
T4	10	8 vs 20 bytes · 2-tupla vs 4-tupla · por qué no puede controlar congestión · QUIC reemplaza a TCP
T5	8	PSH no da fronteras · los dos escenarios del laboratorio · `Content-Length` · SCTP
Teoría	**40**	**mínimo 15 para aprobar**
Problema 1	30	los 2 eventos · los 4 requisitos de `detectarCongestion` · los 3 casos de `controlarCongestion` · el `min`
Problema 2	30	los 5 `seq` · los 5 `conf` · los 4 dupACKs · los 2 eventos con `cwnd` · 2^{17} y el tiempo
Total	**100**	**aprobación: 60**

✅ Autoevaluación del bloque

1. ¿Cuántos estados tiene la máquina de TCP y cuáles son los dos que casi todos olvidan?

▶ Respuesta

2. Un segmento recibido tiene `numero_secuencia = 7024` y 24 bytes de datos. ¿Qué valor debe traer la confirmación de B, y qué se confirmó con ella?

▶ Respuesta

3. ¿Por qué la ventana efectiva es `min(cwnd, rwnd)` y no la suma?

▶ Respuesta

4. Un emisor manda un segmento, no recibe ACK y vence el timer. ¿Qué tres cosas pueden estar pasando?

▶ Respuesta

5. Con 3 ACKs duplicados, ¿qué valor toma `cwnd` en Tahoe y en Reno? ¿Y con timeout?

▶ Respuesta

6. ¿Qué es el BDP y por qué importa?

▶ Respuesta

7. Nombra dos razones por las que UDP no puede implementar control de congestión, y qué lo resuelve hoy.

▶ Respuesta

8. ¿Y qué cambió entre 2011 y 2026 en todo el bloque?

▶ Respuesta

🔗 Referencias

- **Examen `sirc1103`** — P1 (10) forwarding vs. routing · P3 (6) traceroute · **P4 (10) ★ 4 segmentos de 20 bytes desde 1100** · P5 (10) retardo end-to-end · Problema 1 (30) ECN + pseudocode · Problema 2 (30) NAT + ARP + subredes
- **Examen `sirc1012`** — P1 (8) DNS y TTL · P2 (8) mux/demux UDP · P3 (8) hub vs. switch · **P4 (8) números de secuencia y timers** · P5 (8) IPv6 y ND · Problema 1 (30) `connectCheaper()` con sockets UDP · Problema 2 (30) MTU, fragmentación y Path MTU
- **Examen `sirc1102`** — P1 (10) confidencialidad/integridad/no repudio · P2 (10) DHCP · P3 (10) campos de TCP y UDP · P4 (10) NAT · Problema 1 (30) VoIP con jitter buffer · Problema 2 (30) IP Móvil
- **Examen `sirc1002`** — P2 (8) forwarding vs. routing · P3 (8) paridad 2D · P4 (8) firma digital · P5 (8) jitter
- **Examen `sirc0912`** — P2 (10) fiabilidad en transporte vs. enlace · P3 (8) "hops" como métrica
- **Examen `sirc0907`** — **P1 (10) flujo vs. congestión + detección + si toda pérdida es congestión** · P2 (8) three-way handshake y cuándo usar UDP · P3 (8) ARP · P4 (8) RPF e IGMP · P5 (6) confidencialidad vs. integridad
- **Examen `sirc1003`** — P1 (30) subredes + ARP · **Problema 2 (30) ataque TCP reset** · P5 (8) TCP en inalámbricas

- **Examen** `sirc0112` — Problema 1: dos conexiones por un router que pierde · Problema 2: el flag PUSH
- **Examen** `sirc0807` — P2 (10) los cuatro algoritmos de control de congestión
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 completo (pág. ~570–620 del PDF)
- `RFC 9293` — TCP (2022, *Internet Standard*) · `RFC 5681` — TCP Congestion Control (2009)
- `RFC 9438` — CUBIC (2023, *Standards Track*) · `RFC 6928` — IW = 10 MSS
- `RFC 2018` — SACK · `RFC 7323` — Timestamps, Window Scale, PAWS · `RFC 3168` + `RFC 9768` — ECN y AccECN · `RFC 9331` — L4S
- `RFC 9000` / `9001` / `9002` — QUIC · `RFC 9114` — HTTP/3 · `RFC 8684` — MPTCPv1
- `PLAN-48-CLASES.md` y `BANCO-EXAMENES.md` — dónde está cada problema y qué se preguntaba
- **Clase siguiente:** 24 — La capa de red: las dos funciones (forwarding vs. routing)

Clase 24

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El objetivo: de sistema final a sistema final
1.2 Las dos funciones clave
1.3 El circuito cerrado: protocolo → algoritmo → tabla de forwarding
1.4 Longest prefix match: por qué la tabla es por prefijo
1.5 Plano de datos, plano de control: en banda y fuera de banda
1.6 Flooding
1.7 El router no implementa capas superiores (y el cortafuegos sí)
1.8 ⚠️ Actualización a 2026
2.1 Ejercicio 1 — Longest prefix match con cinco entradas
2.2 Ejercicio 2 — ¿En banda o fuera de banda?
2.3 Ejercicio 3 — ¿Por qué "hops" es una mala métrica?
3.1 ⭐ Examen 2011-03, Pregunta 1 (8 puntos) — las dos funciones clave y su interacción
3.2 Examen 2009-02, Pregunta 2 (10 puntos) — forwarding, routing y flooding
3.3 Examen 2008-07, Pregunta 4 (10 puntos) — plano de datos vs. plano de control, en banda vs. fuera de banda
3.4 Dos preguntas más que entran en esta clase
3.5 Nota de coordinación: Dijkstra es de la Clase 33
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: D (Red) · **Semana:** 8 · **Duración estimada:** 100 min **Correlativa:** Clase 23 (repaso de transporte + Simulacro 3) · **Sigue con:** Clase 25 (conmutación en la red: datagramas y circuitos virtuales) **Material de referencia:** Tanenbaum cap. 5 · RFC 791, RFC 1812, RFC 2328, RFC 4271, RFC 4632, RFC 9234, RFC 8986

🎯 Al terminar esta clase vas a poder

1. **Explicar** por qué el objetivo de la capa de red es la entrega **de sistema final a sistema final** y no de enlace a enlace, y por qué esa diferencia define el diseño de todo lo demás.
2. **Distinguir** las dos funciones clave —**forwarding** y **routing**— por ámbito, escala de tiempo, determinismo y lugar de implementación, y explicar **cómo interactúan** formando un circuito cerrado.
3. **Calcular** a qué puerto de salida sale un paquete aplicando *longest prefix match*, y explicar por qué la tabla de forwarding se indexa **por prefijo** y no por host.
4. **Distinguir** el plano de datos del plano de control y clasificar un diseño como **en banda** o **fuera de banda**, justificando qué pasa si falla el plano de datos.
5. **Explicar** qué es el *flooding*, cómo se controla (TTL, duplicados, *split horizon*, prune) y por qué **en unicast no hay flooding** pero en multicast sí.
6. **Justificar** por qué el router no implementa capas superiores y por qué el cortafuegos sí necesita mirar TCP y UDP.
7. **Argumentar** con números por qué "cantidad de hops" puede ser una mala métrica de enrutamiento.

> **Prerequisito del examen:** `2011-03 P1` (8 pts), `2009-02 P2` (10 pts), `2010-02 P2` (8 pts), `2008-07 P4` (10 pts), `2009-12 P3` (8 pts).

PARTE 1 — TEORÍA

1.1 El objetivo: de sistema final a sistema final

La capa 3 tiene **un solo cometido declarado**: **entregar cada paquete de un sistema final (host) emisor a un sistema final (host) receptor**, a través de una malla de redes intermedias que no son todas iguales ni todas conocidas.

	Capa 2 (enlace)	**Capa 3 (red)**
Entrega	nodo a nodo, en **un** enlace	sistema final a sistema final, a través de **muchos**
Problema	de alcance **único**	de alcance **global**
Pregunta que resuelve	¿a qué **MAC** mando esto para que llegue al router vecino?	¿por qué **interfaz** mando este paquete para que llegue a 8.8.8.8?
Punto final de la entrega	el router vecino	el **host** destino

Por qué esa diferencia define todo lo demás. (1) **El router es un modo de transporte, no el fin de la comunicación**: si la IP destino coincide con una de sus interfaces entrega localmente, si no, reenvía. (2) **El router no necesita entender el contenido**: sólo lee la dirección destino; si entendiera TCP tendría que demultiplexar por puertos y recalcular checksums, imposible a la velocidad del enlace. (3) **Aparece el concepto de ruta**: en capa 2 no hay rutas, hay "el vecino de al lado"; en capa 3 el camino tiene que **descubrirse** (routing) antes de poder **recorrerse** (forwarding). (4) **El servicio es *best-effort***: no garantiza entrega, ni orden, ni camino fijo por paquete, y esa falta de garantías es lo que permite que la red **escale**.

1.2 Las dos funciones clave

	Forwarding (reenvío) — plano de datos	**Routing (enrutamiento)** — plano de control
Qué es	mover un paquete del **puerto de entrada al de salida**	determinar la **ruta de origen a destino** e instalarla en las tablas
Ámbito	**local**: este router y este paquete	**global**: necesita información de los otros routers
Granularidad	**por paquete**	**por prefijo**
Tiempo	**nanosegundos–microsegundos**	**segundos**
Naturaleza	**determinista**: misma entrada + mismo destino = misma salida	**heurística**: caminos subóptimos y posibles ciclos
Dónde corre	**hardware**, en TCAM	**software**, en la CPU
¿Habla con pares?	**no**: un router aislado reenvía igual	**sí**, con al menos un protocolo de enrutamiento
Produce	la **FIB** (*Forwarding Information Base*)	la **RIB** (*Routing Information Base*), que se compila a la FIB

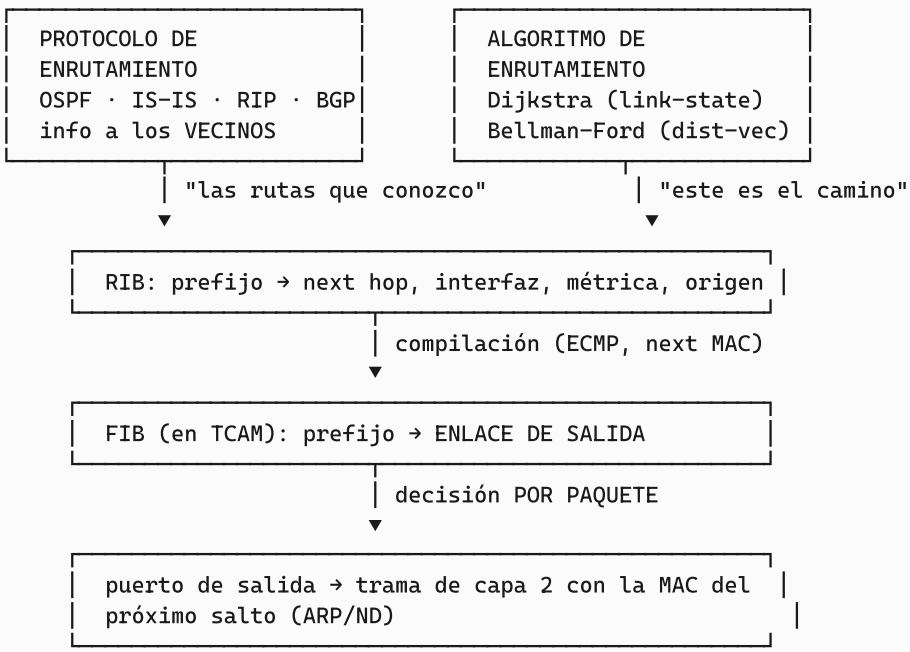
Por qué el forwarding no puede ser heurístico: si fuera aproximado, dos paquetes del mismo flujo podrían tomar caminos distintos y el **reordenamiento** sería el estado normal; y tiene que ser reproducible en silicio a

100 Gbit/s, o sea un número finito de comparaciones por paquete. La heurística vive **arriba**, en un plano lento donde gastar CPU no es un drama.

Por qué TCAM y no RAM común: la pregunta "¿qué entradas matchean esta dirección?" no es una búsqueda por índice sino una **comparación contra todas** (millones, en un router de núcleo). La **TCAM** (*Ternary Content-Addressable Memory*) guarda datos **y** máscara por entrada y compara **todas simultáneamente**, entregando el resultado en un ciclo. Como es carísima ⇒ **la FIB va en TCAM y la RIB (métricas, atributos, ECMP) en RAM.**

1.3 El circuito cerrado: protocolo → algoritmo → tabla de forwarding

Este es el esquema que pide el `2011-03 P1`:



Consecuencia práctica que se pregunta: si la FIB ya está poblada y el router se queda aislado del plano de control, **sigue reenviando con la tabla vieja**. Un router al que se le cae BGP **se congela, no se apaga**.

1.4 Longest prefix match: por qué la tabla es por prefijo

¿Por qué no por host? Una tabla por host tendría **2^{32} entradas**. Además **no haría falta**: si te dicen "la red `192.168.1.0/24` está directamente conectada", **una sola entrada** cubre 254 direcciones. Eso es la **agregación de prefijos** (CIDR, `RFC 4632`), y es lo que hizo posible que Internet escalara.

Si DOS entradas matchean, gana la más específica (la que tiene más bits de prefijo): *longest prefix match*. **¿Por qué la más específica y no la menos?** Porque la tabla se puebla con **excepciones**: "mi LAN entera está acá" y además "dentro de mi LAN hay una subred que **no** es mía". Si ganara la menos específica, ese paquete iría a la LAN local, que lo descartaría por ser de otra subred. **El longest match es lo que permite expresar excepciones.**

```
Tabla:  192.168.1.0/24 → eth2 (directa)
        192.168.0.0/16 → eth1 (next hop R1)
        0.0.0.0/0      → eth0 (next hop R2, default)

Busco 192.168.1.5:
  /24 → AND 255.255.255.0 = 192.168.1.0  ✓ matchea
  /16 → AND 255.255.0.0   = 192.168.0.0  ✓ matchea
  /0  → AND 0.0.0.0       = 0.0.0.0      ✓ matchea SIEMPRE
→ GANA /24 (24 > 16 > 0)  →  eth2.

Busco 192.168.77.9:  el /24 no matchea; matchean /16 y /0 → GANA /16 → eth1.
Busco 8.8.8.8:        sólo matchea /0 → eth0.
```

En TCAM **todas** las entradas matchean en paralelo y la más específica está marcada, por eso la tabla se ordena de `/0` a `/32` y la búsqueda es una **comparación**, no un algoritmo.

1.5 Plano de datos, plano de control: en banda y fuera de banda

El **plano de datos** mueve los paquetes según la tabla: por paquete, nanosegundos, determinista, en hardware. El **plano de control** calcula y distribuye qué tabla debe tener cada nodo: por prefijo, segundos, heurístico, en software. **Relación:** el control **escribe** la tabla que los datos **leen**; no se le consulta por paquete, sólo cuando la tabla cambia. Por eso el forwarding puede ser determinista y rapidísimo: **todo el trabajo de pensar ya está hecho de antemano**.

```
EN BANDA (in-band)                FUERA DE BANDA (out-of-band)
datos y control por el MISMO      datos por un camino, control
canal o camino                    por un canal DISTINTO

el control depende del dato:      el control tiene vida propia:
si se rompe el plano de datos,    si se rompe el plano de datos,
se rompe TAMBIÉN el control        el control sigue y puede
→ no hay por dónde diagnosticar   repararlo
```

- **En banda.** El **switch que aprende**: no tiene protocolo de control, aprende la MAC origen de las tramas que él mismo reenvía, así que su único plano es el de datos. Y el **enrutamiento en IP**: los LSA de OSPF y los UPDATE de BGP **son** paquetes IP de la misma red que rutean ⇒ plano de control **auto-referencial**. Si el plano de datos se rompe, no llegan las actualizaciones y **no hay forma de quejarse, porque el canal de queja es el que está roto**.
- **Fuera de banda.** El **LSP de MPLS**: LDP o RSVP-TE asignan una etiqueta a cada salto y los datos reenvían **sólo por etiqueta**; si LDP se cae, los paquetes con etiqueta siguen reenviéndose. La **red telefónica**: los circuitos de voz y la red de señalización son redes distintas, así que si cae la señalización no se puede establecer ninguna llamada **aunque todos los circuitos estén libres**. Un **enlace dedicado** o una **PVC de Frame Relay**: el circuito lo provisiona el operador y los datos no consultan nada durante toda su vida.

El desarrollo completo de esta pregunta, con la estructura de cada caso y la lista de ejemplos, está en **§3.3** (Examen 2008-07 P4).

1.6 Flooding

Flooding (inundación) es reenviar un paquete por **todos los puertos** (o todos menos el de entrada) en vez de consultar una tabla: no hay ACK, no hay estado, no hay tabla de "a quién le corresponde". **El problema:** en una

red con ciclos —y toda red con caminos alternativos tiene— cada copia genera copias nuevas, así que sin control en *n* saltos circulan hasta **2^n** copias, el enlace se satura y la red se traba. **Por eso tiene que transformarse en controlado:**

Control	Cómo funciona	Contra qué
TTL / Hop Limit	Cada reenvío decrementa el TTL; al llegar a 0 se descarta y se avisa con ICMP *Time Exceeded* (tipo 11).	Acota el **radio** del flooding.
Detección de duplicados	Cada mensaje lleva un **identificador único** (origen + nº de secuencia); el nodo guarda los IDs vistos y **descarta** los repetidos.	Corta la **exponenciación** con ciclos.
Split horizon (horizonte dividido)	No le reenvíes a un vecino la ruta o el mensaje que **aprendiste de ese vecino**.	Evita la **realimentación** entre vecinos (conteo a infinito).
Exclusión del puerto de entrada	Nunca reenvíes por el puerto por donde entró la trama.	Evita devolver la trama al remitente.
Pruning (supresión)	Cuando se detecta que **no hay destinatarios** detrás de un enlace, se deja de inundar por ahí y se anuncia.	Deja de gastar ancho de banda en ramas muertas.

La combinación clásica es **flooding y prune en IP multicast**: cada router inunda, observa los **reportes IGMP** de sus subredes y, cuando detecta que una rama no tiene miembros, deja de inundar por ahí y anuncia el prune. En un switch es el **IGMP snooping** del `2010-12 P3`. El diagrama del flooding está en §3.2.

¿Y en unicast? En unicast no hay flooding. El reenvío de un datagrama unicast se hace **siempre por tabla** (longest prefix match, un puerto). Si la IP destino no matchea ninguna entrada, el paquete **se descarta** y se devuelve un ICMP *Destination Unreachable*; no se inunda. El flooding aparece en **link-state** (difundir las LSAs es el mecanismo principal) y en **multicast**.

1.7 El router no implementa capas superiores (y el cortafuegos sí)

Por qué el router se detiene en la capa 3. (1) **Rendimiento**: si entendiera TCP tendría que demultiplexar por el 5-tupla, mantener buffers por conexión y recalcular checksums, mientras que el reenvío de capa 3 es **una comparación de un entero** (a 100 Gbit/s entra un paquete cada 50 ns). (2) **No tiene el concepto de "conexión"**: TCP es extremo a extremo y un router intermedio es un tercero; guardar estado por flujo haría crecer su memoria con los flujos de toda Internet (es el error histórico de IntServ, Clase 25). (3) **Desacoplar las capas es el objetivo**: un router que no mira el contenido sirve para cualquier protocolo; si lo mirara dejaría de ser router y pasaría a ser cortafuegos. (4) **Es diseño explícito**: "el transporte es problema de los extremos".

Por qué el cortafuegos sí necesita TCP y UDP. Un cortafuegos **con estado** (*stateful*) mantiene una tabla de flujos con las columnas `{IP origen, puerto origen, IP destino, puerto destino, protocolo, estado}` —por ejemplo `10.0.0.5:51234 → 93.184.216:443 TCP ESTABLISHED` — y la regla es: si el paquete coincide con una entrada **existente** se permite (y la conexión "sale" sola); si no coincide, se consulta la ACL y sin permiso explícito se descarta. Esa entrada necesita IP de origen, IP de destino, **puertos** y **protocolo**: la IP es de capa 3, pero los puertos y el protocolo son de **capa 4**. Además tiene un *timeout* y distingue conexiones por los **flags**. Todo eso es información de TCP y UDP. Lo mismo pasa con el **NAT/PAT** (reescribe IP **y puerto**), con el **ALG** (para que FTP pasivo funcione tiene que leer la cabecera FTP que va **dentro** del payload TCP: es el `2007-12 Problema 1`) y con los IDS/IPS de capa 7.

> **La regla general:** el router reenvía por dirección de capa 3; el cortafuegos decide por capa 3 **más** información de capas 4 y 7. Por eso un router no es un cortafuegos.

1.8 ⚠️ Actualización a 2026

Sigue igual, de memoria: forwarding por prefijo, en hardware, por longest prefix match · routing = un protocolo + un algoritmo que pueblan la tabla · la distinción plano de datos / plano de control · en banda y fuera de banda · el flooding como mecanismo de link-state y de multicast · la FIB en TCAM.

Cambió:

- **La seguridad del enrutamiento pasó de "confiar en el AS_PATH" a "validar el origen".** `RFC 6810` (RTR) y `RFC 6811` (BGP Prefix Origin Validation): el router valida que un prefijo esté autorizado por el RIR antes de aceptarlo. Es el ancla de confianza del enrutamiento, paralelo exacto de DNSSEC para el DNS. Y `RFC 9234` (may-2022) reemplaza el MD5 de sesión por **roles BGP** (Provider / Customer / Peer / Lateral / Internal) declarados en OPEN y UPDATE, para prevenir y detectar *route leaks*; `RFC 4271` sigue siendo la especificación vigente de BGP-4.
- **La separación control/datos pasó a ser una arquitectura.** SDN y **OpenFlow** (de la Open Networking Foundation, **no** del IETF: no tiene número de RFC) llevan el plano de control a un servidor que programa las FIBs ⇒ si te preguntan quién calcula la tabla y quién la aplica, la respuesta clásica (cada router calcula y aplica) tiene hoy alternativa: el cálculo puede estar centralizado.
- **Enrutamiento con estado sólo en el borde. Segment Routing** (`RFC 8402`, 2018; `RFC 8986`, SRv6) codifica las instrucciones de camino en **SID** que el nodo no tiene que interpretar: la FIB es "de prefijo largo a prefijo largo", **forwarding puro sin protocolo de control por salto**. SRv6 mete los SID **en la cabecera IPv6** y **elimina el MPLS por completo**.
- **El routing de datacenter dejó de ser BGP con link-state:** hoy es **EVPN-VXLAN** (`RFC 7432` + `RFC 7348`). **MPLS en datacenter fue reemplazado**; MPLS sigue **masivamente en carriers**. Y una FIB de núcleo tiene del orden de **millones de prefijos IPv4**: por eso la TCAM es cara, por eso existe la sumarización y por eso hay límites de tabla en BGP.
- **"Hops" dejó de ser la métrica por defecto en link-state.** OSPF define el costo de interfaz de forma configurable, con un valor por defecto derivado del **ancho de banda** (`RFC 2328`): `costo = ancho de banda de referencia / ancho de banda de la interfaz`. **Un enlace de 10 Gbps vale 1** con la referencia por defecto; uno de 10 Mbps vale muchísimo más ⇒ la métrica por defecto de OSPF **ya no cuenta hops, cuenta capacidad**.

Consecuencia para el examen: si el enunciado dice "la métrica es la cantidad de hops", respondé eso y sumá una línea: "*hoy la métrica por defecto de OSPF es un costo derivado del ancho de banda, no los hops; y en BGP la comparación es por longitud de AS_PATH*".

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Longest prefix match con cinco entradas

Datos. El router S tiene esta tabla de forwarding:

#	Prefijo	Siguiente salto	Interfaz
1	`10.0.0.0/8`	directo	eth0
2	`172.16.0.0/12`	R2	eth1
3	`192.168.1.128/25`	R3	eth2
4	`192.168.1.0/24`	directo	eth3
5	`0.0.0.0/0`	R4	eth4

Se pide: a qué interfaz sale cada destino: `10.1.2.3`, `172.16.5.9`, `172.32.5.9`, `192.168.1.200`, `192.168.1.5`, `8.8.8.8`.

Resolución paso a paso.

Paso 1 — Las máscaras y la regla.

```
/8  → 11111111.00000000.00000000.00000000 = 255.0.0.0
/12 → 11111111.11110000.00000000.00000000 = 255.240.0.0
/25 → 11111111.11111111.11111111.10000000 = 255.255.255.128
/24 → 11111111.11111111.11111111.00000000 = 255.255.255.0
/0  → 00000000.00000000.00000000.00000000 = 0.0.0.0

Regla: AND bit a bit entre el destino y la máscara.
Si el resultado es IGUAL a los bits de red del prefijo → matchea.
De los que matchean, GANA el que tiene MÁS bits de prefijo.
```

Paso 2 — Los seis destinos. Cada celda es `destino AND máscara`; se marca si es igual a la red del prefijo.

Destino	AND /8	AND /12	AND /25	AND /24	AND /0	Gana	Interfaz
`10.1.2.3`	`10.0.0.0` ✓**#1**	`10.0.0.0` ✗	`10.1.2.0` ✗	`10.1.2.0` ✗	`0.0.0.0` ✓**#5**	**/8**	eth0
`172.16.5.9`	`172.0.0.0` ✗	`172.16.0.0` ✓**#2**	`172.16.5.0` ✗	`172.16.5.0` ✗	`0.0.0.0` ✓**#5**	**/12**	eth1
`172.32.5.9`	`172.0.0.0` ✗	`172.32.0.0` ✗	`172.32.5.0` ✗	`172.32.5.9` ✗	`0.0.0.0` ✓**#5**	**/0**	eth4
`192.168.1.200`	`192.0.0.0` ✗	`192.168.0.0` ✗	`192.168.1.128` ✓**#3**	`192.168.1.0` ✓**#4**	`0.0.0.0` ✓**#5**	**/25**	eth2
`192.168.1.5`	`192.0.0.0` ✗	`192.168.0.0` ✗	`192.168.1.0` ✗	`192.168.1.0` ✓**#4**	`0.0.0.0` ✓**#5**	**/24**	eth3
`8.8.8.8`	`8.0.0.0` ✗	`8.0.0.0` ✗	`8.8.8.0` ✗	`8.8.8.8` ✗	`0.0.0.0` ✓**#5**	**/0**	eth4

Paso 3 — El caso que hay que saber hacer en binario.

```
172.32.5.9 = 10101100.00100000.00000101.00001001
/12        = 11111111.11110000.00000000.00000000
          AND= 10101100.00100000.00000000.00000000 = 172.32.0.0
La red de 172.16.0.0/12 = 10101100.00010000.… = 172.16.0.0
¿Coincide? NO → NO matchea.

192.168.1.200 AND 255.255.255.128 = 192.168.1.128 = red del /25  ✓ #3
192.168.1.200 AND 255.255.255.0   = 192.168.1.0   = red del /24  ✓ #4
GANA /25 (25 bits, el mayor número de bits) → eth2.
```

Interpretación del primer caso: el `/12` no es "los 12 primeros bits y los demás libres", es **doce 1s y veinte 0s, en orden**. El rango del `/12` es `172.16.0.0` a `172.31.255.255`. **Es el error de lectura de máscara más común del curso.**

Interpretación del segundo caso: `192.168.1.200` está en la red local de S (`192.168.1.0/24`), pero la tabla dice que **la mitad superior de esa LAN no es de S**: es una subred /25 que va para otro lado. Si S mandara el paquete a su LAN local, la máquina que lo recibiera lo descartaría por ser de otra subred. **El longest match salva el caso.**

⚠️ Errores típicos

- **Contar "cuántas entradas matchean" en vez de "quién tiene más bits".** En `10.1.2.3` ganó el /8 (8 > 0), no el /0.
- **Confundir el prefijo con la red.** `172.32.x.x` **no** está en `172.16.0.0/12`.
- **Olvidar que el default matchea siempre.** Sin entrada que matchee, el paquete **se descarta** (con ICMP *Destination Unreachable`), no se reenvía "por donde sea".
- **Tratar el `/n` como decimal.** `/25` son 25 unos en binario, no "el número 25".

2.2 Ejercicio 2 — ¿En banda o fuera de banda?

Enunciado. Clasificá cada diseño y decí qué pasa si el plano de datos se rompe. **(a)** Switch de capa 2 que **aprende la MAC origen** de cada trama que reenvía, sin ningún otro protocolo. **(b)** Router que corre **OSPF**: los Hello y los LSA son datagramas IP que viajan por la misma red que OSPF rutea. **(c)** Router que aprende rutas por **BGP**, protocolo propio sobre **TCP/179**; sus mensajes siguen siendo paquetes IP de la red que BGP rutea. **(d)** Switch de datacenter con un **puerto de gestión** en una red de administración distinta de la de producción.

Resolución.

	Clasificación	Por qué
(a)	**En banda**	No hay plano de control: el único plano es el de datos y la "tabla" se construye **mirando las tramas**. Si el switch se apaga se pierde entera y hay que reconstruirla con el tráfico; mientras tanto se inunda a los destinos desconocidos.
(b)	**En banda, auto-referencial**	Los LSA son paquetes IP que necesitan que las tablas **ya estén funcionando** para llegar a sus vecinos. Si el plano de datos se rompe no llegan, **el grafo queda desactualizado**, los Dijkstra dan caminos que no existen y **no hay forma de quejarse porque el canal de queja es el que está roto**.
(c)	**En banda en el medio, protocolo distinto en el plano lógico**	No está fuera de banda: sus mensajes necesitan IP funcionando para viajar. Pero BGP **es un protocolo de control distinto** del que lleva los datos de los usuarios y, al correr sobre TCP, **no depende de que IP sea fiable**: TCP le da ACKs, reintentos y orden. Y como es *path-vector* **no conoce la topología**: si un enlace se rompe deja de anunciar por ahí, pero **no calcula mal**.
(d)	**Fuera de banda**	La red de administración es un camino distinto con vida propia. Si se rompe todo el tráfico de producción seguís entrando por el puerto de gestión a mirar logs, recargar la configuración o sacar el equipo. Exactamente por eso los routers críticos tienen consola y gestión out-of-band.

```
EN BANDA       → el control usa el camino del dato. Si el dato muere, el
                 control muere con él. Ventaja: no hay que cablear nada
                 extra. Desventaja: no podés diagnosticar lo que no tenés
                 por dónde hablar.

FUERA DE BANDA → el control tiene camino propio: sobrevive al fallo del
                 dato y puede repararlo. Ventaja: reparás. Desventaja: hay
                 que construir y mantener el camino de control.
```

2.3 Ejercicio 3 — ¿Por qué "hops" es una mala métrica?

Enunciado. Dos caminos de A a Z: **camino 1** = 1 enlace directo de **10 Mbit/s** con 20 ms de propagación; **camino 2** = 5 enlaces de **1 Gbit/s** con 1 ms de propagación cada uno.

Resolución.

Paso 1 — La métrica por hops elige el camino 1 (1 < 5).

Paso 2 — Ancho de banda. El flujo va por el cuello de botella, o sea el enlace más estrecho de cada camino.

```
Camino 1: 10 Mbit/s
Camino 2: 1 Gbit/s = 1000 Mbit/s
→  1000 / 10 =  100 veces más ancho de banda en el camino de 5 hops.
```

Paso 3 — Retardo de transmisión de un paquete de 1500 bytes (trama Ethernet con carga IP máxima):

```
L = 1500 bytes × 8 bits/byte = 12 000 bits        d_trans = L / R

Camino 1: 12 000 / (10 × 10⁶) = 1,2 × 10⁻³ s = 1,2 ms   por enlace
Camino 2: 12 000 / (10 × 10⁹) = 12  × 10⁻⁶ s = 12 µs      por enlace
(1,2 ms = 1200 µs → el de 10 Mbit/s es 100× más lento)
```

Paso 4 — Propagación y retardo total de UN paquete, con `d_proc = 0` *y sin cola:*

```
Camino 1: 0 + 1,2 ms + 20 ms           = 21,2 ms
Camino 2: 0 + 5×0,012 ms + 5×1 ms     =  5,06 ms
→ el camino de 5 hops es 21,2 / 5,06 = 4,2 VECES mejor.
```

Interpretación, y es la frase para el examen: **"Hops" mide la cantidad de saltos, no la calidad del camino.** Un salto es un salto tanto si es de 10 Mbit/s como de 1 Gbit/s; la métrica **ignora por completo** las dos cosas que más afectan al usuario: el ancho de banda del cuello y el retardo acumulado. Y **peor**: como menos saltos suele significar menos procesamiento y menos encolado, la métrica por hops **tiende a elegir el camino corto**, que en la práctica es a menudo el de 10 Mbit/s y no el de 5 Gbit/s.

El único argumento a favor, y hay que decirlo: los hops **sí** correlacionan con el retardo de **procesamiento**, porque son nanosegundos (o microsegundos si el router está congestionado) por salto. Pero es de otro orden de magnitud que la diferencia de ancho de banda y que la de propagación: la balanza no está ni cerca.

Algoritmo	Métrica real
OSPF / IS-IS	**Costo de interfaz** configurable; por defecto derivado del **ancho de banda** (`RFC 2328`): `costo = ancho de banda de referencia / ancho de banda de la interfaz` (con referencia de 100 Gbit/s en equipos modernos, para que 1 Gbit/s no salga a costo 100).
RIP	**Cantidad de saltos**, máximo 15: sí cuenta hops, y es de lo más criticado por eso.
EIGRP	**Ancho de banda + retardo** (métrica compuesta).
BGP	Preferencia local → **longitud del AS_PATH** → origen → **MED**. No usa hops de router: los del interior del AS los aporta el IGP de abajo.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-03, Pregunta 1 (8 puntos) — las dos funciones clave y su interacción

> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf **Texto del examen:**
>
> > Describa claramente las dos funciones clave de la capa de red y explique cómo interactúan entre ellas.

Solución oficial (literal del PDF):

> **Reenvío (forwarding):** mover paquetes entre puertos de entrada y salida del router. **Enrutamiento (routing):** determinar la ruta de los paquetes desde origen a destino. Para ello existen los algoritmos de enrutamiento. **Interacción:** Cada router intercambia con sus pares, mediante al menos un protocolo de enrutamiento, información que se utiliza, en al menos un algoritmo de enrutamiento, para poblar sus tablas de forwarding. Dichas tablas, en su formato más básico, contiene, para un valor de dirección contenido en la cabecera de los paquetes que llegan al router, el enlace de salida. De esta forma, para cada paquete que arriba a un router por cualquiera de sus interfaces, el router examina la tabla de forwarding (en el caso de IP a partir de la dirección IP destino contenida en el encabezado IP) y a partir del "longest match" determina por qué interfaz debe ser enviado (además de determinar cuál es el "next hop", lo que determina en el caso más genérico, la dirección destino de capa de enlace que deberá contener la trama que lo lleve a su próximo salto). **Referencia:** `cap04_1_1xhoja_2009.pdf`, diapositivas 5 y 6.

Análisis de la solución oficial. Lo que está bien: la definición de forwarding es impecable y es exactamente el ámbito **local**; la interacción está en **tres eslabones** (**protocolo** → **algoritmo** → **tabla de forwarding**), con tres "al menos un" deliberados, porque son piezas **distintas** y puede haber varias de cada una; y menciona el **longest match** y el *next hop, que es el detalle fino: la tabla dice a qué interfaz y a qué dirección de router mandar, y de ese next hop se resuelve la MAC** de la trama.

Lo que **falta** y suma puntos: **la escala de tiempo y el lugar de ejecución** de cada una; **la distinción RIB / FIB**; **qué pasa si el routing falla** (si se cae el protocolo, **la tabla no se borra**: el router **se congela**); y que **el forwarding es determinista y el routing heurístico**, que es la razón de por qué la frontera entre los dos planos es hardware/SILICIO.

Mi respuesta completa, para tener a mano lo que me gustaría haber contestado:

> **Forwarding** es la función local que, para cada paquete que llega por una interfaz, lo mueve de la interfaz de entrada a la de salida. Es **por paquete**, en **nanosegundos**, y es **determinista**: dada una entrada de tabla y una dirección destino, la salida es siempre la misma. Se implementa en **hardware**, en una tabla

por prefijo (la FIB) que se consulta con **longest prefix match**. Si la IP destino coincide con una interfaz propia, se entrega localmente en vez de reenviarse.

Routing es la función que determina el camino de origen a destino y decide qué información se instala en las tablas de forwarding. Es **global** (necesita información de los otros routers), **por prefijo y no por paquete**, corre en **segundos** y es **heurística**: puede elegir caminos subóptimos y está sujeta a ciclos de convergencia. Se implementa en **software**, en el plano de control.

Interacción: cada router intercambia con sus pares, mediante al menos un **protocolo de enrutamiento** (OSPF e IS-IS son link-state e inundan; RIP es distance-vector; BGP es path-vector entre sistemas automáticos), información sobre la red. Esa información la procesa al menos un **algoritmo de enrutamiento** (Dijkstra para link-state, Bellman-Ford para distance-vector, más políticas y atributos) y el resultado se vuelca en la **tabla de enrutamiento (RIB)** local. De la RIB se compila la **tabla de forwarding (FIB)**, que en su forma más básica contiene, para un prefijo, el **enlace de salida**. Después, para cada paquete que llega por cualquier interfaz, el router toma la **dirección IP destino** del encabezado, busca el **longest match** en la FIB y reenvía por esa interfaz; con el **next hop** que esa entrada le da, resuelve la dirección de capa 2 del próximo salto para armar la trama.

Es un **circuito cerrado**: el plano de control alimenta al plano de datos, y por eso **la FIB puede seguir funcionando aunque el plano de control se caiga**: el router reenvía con la última tabla conocida hasta que una actualización nueva la reemplace.

Tope para 8 puntos: los dos axiomas (4) + el circuito cerrado con los tres eslabones y el longest match (4). Decir "el enrutamiento puebla las tablas de reenvío, que asocian destino con puerto de salida" ya te da la mitad. Sumar RIB/FIB y "el forwarding no se borra cuando cae el control" es lo que separa un 8 de un 6.

3.2 Examen 2009-02, Pregunta 2 (10 puntos) — forwarding, routing y flooding

Fuente: `eirc0902` / `sirc0902` — 2 de febrero de 2009. **Texto:**

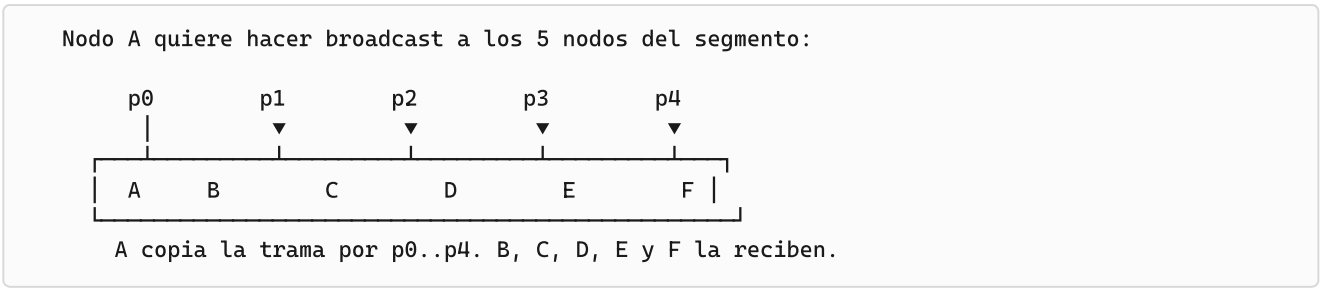
> a) En un contexto unicast, describa los procesos de forwarding y enrutamiento, y explique como se relacionan. b) En un contexto broadcast, describa un proceso de flooding, y explique como se puede transformar en flooding controlado.

Solución oficial (parte a, literal): "*El proceso de enrutamiento permite determinar el camino entre nodos de la red, típicamente mediante intercambio de información de topología y ejecución de un algoritmo en cada nodo [...] El proceso de forwarding consiste en copiar cada paquete entrante a un nodo al puerto de salida adecuado para que siga su camino en la red (o entregarlo a un proceso local si el destinatario es el propio nodo). La tabla de forwarding, que asocia destinos con puertos (o interfaces) del nodo/router, se construye en base a la información de enrutamiento.*" **La parte b) no tiene solución publicada en el PDF: lo que sigue es solución propia.**

a) La solución oficial trae los tres puntos que hay que decir: el routing "mediante intercambio de información de topología y ejecución de un algoritmo **en cada nodo**" (el "en cada nodo" importa: la información es **global** pero el cálculo es **local**, no hay un centro); el forwarding "copiar cada paquete entrante al puerto de salida adecuado", más el paréntesis que vale puntos, **"o entregarlo a un proceso local si el destinatario es el**

propio nodo"; y la relación, "la tabla de forwarding **se construye en base a** la información de enrutamiento". Sumá que en unicast el forwarding es **siempre por tabla** (nunca flooding), que un destino que no matchea ninguna entrada **se descarta**, y que el **longest prefix match** y el **next hop** son los dos pasos del desempate y del armado de la trama.

b) El emisor quiere que **todos** los demás nodos reciban su mensaje y **no sabe en qué puertos** están, así que no consulta ninguna tabla: **replica el paquete por todos los puertos** (o todos menos el de entrada), sin ACK, sin estado y sin tabla de "a quién le toca".



El problema, y por qué se transforma en controlado: en una red con ciclos cada copia genera copias nuevas, y sin control en *n* saltos circulan hasta $\mathbf{2^n}$ copias: el enlace se satura y la red se traba. Los cinco controles están en la tabla de §1.6 (**TTL/Hop Limit**, **detección de duplicados**, **split horizon**, **exclusión del puerto de entrada** y **pruning**). El caso real es **flooding y prune en IP multicast**: cada router inunda, observa los **reportes IGMP**, y cuando detecta que una rama no tiene miembros deja de inundar por ahí y anuncia el prune.

> **Tope para 10 puntos** (4 de a, 6 de b): en (a) los dos procesos, la construcción de la tabla y el "entregar a proceso local". En (b), **describir** el flooding sin ACK ni tabla, **nombrar el problema** y dar **al menos dos controles justificados**. Decir sólo "TTL" es un 6; dar el mecanismo + dos controles con su para qué es un 9-10.

3.3 Examen 2008-07, Pregunta 4 (10 puntos) — plano de datos vs. plano de control, en banda vs. fuera de banda

> **Fuente:** Examen `eirc0807` / `sirc0807` — 1 de agosto de 2008. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0807.pdf **Texto del examen:**
>
> > Explique la diferencia entre plano de datos (o forwarding) y plano de control. Mencione un ejemplo de protocolo de datos con control fuera de banda (con protocolo de control asociado) y otro con control dentro de banda (sin protocolo de control asociado).

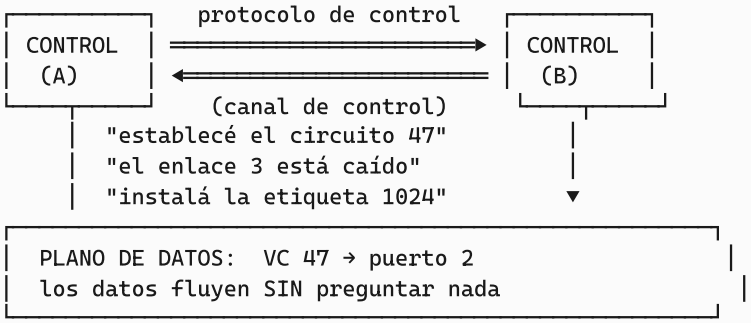
Nota: el PDF de solución **no incluye la parte teórica**. Todo lo que sigue es **solución propia**, sobre el esquema de §1.5.

La diferencia entre los dos planos

	Plano de datos	Plano de control
Qué hace	mueve los paquetes según la tabla	calcula y distribuye qué tabla debe tener cada nodo
Granularidad / tiempo	por paquete / nanosegundos	por prefijo / segundos
Ámbito / naturaleza	local / **determinista**	global / **heurístico**
Implementación	hardware (TCAM)	software (CPU)
Si falla	se corta el tráfico	**se congela**: la tabla vieja sigue sirviendo
Salida	FIB	RIB → compila a FIB

Relación: el plano de control **escribe** la tabla que el plano de datos **lee**; no se le consulta por paquete, sólo cuando cambia.

Control fuera de banda: estructura y ejemplos



Los tres rasgos: (1) **existe un protocolo de control asociado** con sus propios mensajes; (2) corre **antes o al margen** del flujo de datos; (3) los datos **no llevan la dirección destino**, llevan la **etiqueta** que el control les asignó.

- **Ejemplo A — MPLS y el LSP.** LDP o RSVP-TE establecen el camino y **asignan una etiqueta a cada salto**; después el plano de datos reenvía **sólo por etiqueta**, con un longest match de 20 bits, sin mirar la IP destino ni consultar ningún protocolo. Si LDP se cae, **los paquetes con etiqueta siguen reenviéndose**. Es el ejemplo más limpio porque el desacople es explícito: etiqueta (control) vs. IP (datos).
- **Ejemplo B — la red telefónica con señalización de canal común.** La voz viaja por circuitos dedicados conmutados; el control por una **red de señalización separada**. **Si se cae la red de señalización, no se puede establecer ninguna llamada aunque todos los circuitos de voz estén libres.**

Control en banda: estructura y ejemplos

No hay protocolo de control: **el plano de datos es el único plano** y todo lo que sabe sale de mirar el tráfico. En unicast hay una tabla construida por aprendizaje; en broadcast, flooding puro.

- **Ejemplo A — el puente con *self-learning*.** La tabla que dice qué MAC está en qué puerto se construye **mirando las tramas de datos que el puente reenvía** (lee la MAC origen y la asocia al puerto de entrada). Cuando un destino no está conocido, **inunda**; cuando una entrada envejece (típicamente 300 s en switches comerciales, valor **de fabricante**, no un estándar IEEE) la borra y vuelve a inundar hasta que alguien responda. Si el puente se apaga, se pierde todo y hay que reconstruirlo con el tráfico.

- **Ejemplo B — el reenvío unicast en IP.** El router reenvía por tabla, pero la tabla se construyó con protocolos que **viajan como paquetes IP por la misma red**. El plano de control es **auto-referencial**: para actualizarse necesita que el plano de datos de los demás funcione. Si se rompe un enlace, los vecinos se enteran porque **los mensajes dejan de llegar**: el acoplamiento es total.

La frase para cerrar los 10 puntos

> **En el plano de datos la decisión ya está tomada y es determinista; en el plano de control está por tomar y es heurística. "En banda" = el control viaja por el mismo canal que los datos, así que un fallo del dato se lleva el control y no hay forma de diagnosticar. "Fuera de banda" = el control tiene canal propio, así que sobrevive al fallo del dato y puede repararlo. Ejemplo en banda por excelencia: el puente que aprende de sus propias tramas. Ejemplo fuera de banda por excelencia: un LSP de MPLS, cuya etiqueta instala un protocolo de señalización antes de que circule ningún dato.**

3.4 Dos preguntas más que entran en esta clase

Son la misma materia con otro enunciado; las soluciones oficiales están en los PDF y son muy cortas, así que van resumidas. El desarrollo completo está en las clases correspondientes.

- `2010-02 P2` **(8 pts).** *"a) ¿Cuáles son las funciones clave de la Capa de Red? b) ¿Qué objetivo cumple cada una [...] y cómo interactúan? c) Describa brevemente las dos formas de prestar servicios de capa de red."* Oficial: "*Forwarding: mover paquetes entre puertos de entrada y salida del router. Enrutamiento: determinar la ruta de los paquetes desde origen a destino [...] El enrutamiento genera las tablas de forwarding locales.*" **a)** forwarding y routing (§1.2). **b)** Oficial incompleto: falta el **objetivo de cada una** (forwarding resuelve *localmente y de inmediato* por qué interfaz sale cada paquete; routing determina *globalmente y con antelación* por dónde va el paquete e instala esa decisión en las tablas de los routers intermedios) y falta decir que **el routing no existe sin el forwarding**, porque el feedback de "esto no funciona" es lo que hace corregir a los algoritmos. **c)** Las dos formas de servicio de capa 3 son **conmutación de datagramas** (sin conexión, sin estado, decisión por paquete, caminos distintos por paquete, sin reserva) y **conmutación de circuitos virtuales** (con conexión, establecimiento previo, etiqueta en vez de dirección, estado en cada router, reserva posible) — es la Clase 25 entera.
- `2009-12 P3` **(8 pts).** *"a) ¿Cuál es el principal cometido de la capa de red? b) La longitud en 'hops' es la métrica utilizada habitualmente para definir el costo de un enlace. ¿Es la más conveniente?"* Oficial: "*Asegurar la entrega de información de extremo a extremo*" y "*la métrica de hops puede ser inconveniente, ya que señalará como 'mejor' el camino más corto, que no necesariamente será el que tenga mejores atributos de calidad (retardo extremo a extremo, pérdidas, throughput) y/o menos congestión*". **a)** Correcta, con el matiz de que la capa de red hace **encaminar** *best-effort* y **no garantiza nada** (las garantías las da la capa de transporte). **b)** La oficial es buena; sumá que los hops **sí** correlacionan con el retardo de procesamiento (eso juega a favor de la métrica y hay que decirlo) pero es nanosegundos contra órdenes de magnitud en ancho de banda, y que hoy la métrica de link-state es un costo derivado del ancho de banda.

3.5 Nota de coordinación: Dijkstra es de la Clase 33

El **Problema 2 del** `sirc0902` es un ejercicio de **Dijkstra** paso a paso desde el nodo `u`, más el *shortest-path tree*. **No lo resolvemos acá**: es el tema de la **Clase 33 (enrutamiento link-state y OSPF)**. Lo que sí corresponde a esta clase es el **contenido** del flooding que Dijkstra necesita: cada nodo conoce la topología completa porque **se inundan las LSAs** (§1.6), y por eso link-state no escala sin jerarquías de áreas. Si te lo piden, el esqueleto es: inicializar `D(u)=0` y `D(v)=∞`; en cada iteración elegir el nodo no confirmado con `D` mínimo, **confirmarlo** y **relajar** sus aristas salientes con `D(v) = min(D(v), D(u) + c(u,v))`; repetir `n−1` veces.

PARTE 4 — CIERRE

📌 Resumen

1. La capa de red entrega **de sistema final a sistema final**. El router es un modo de transporte, no el destinatario.
2. **Forwarding** = local, por paquete, nanosegundos, determinista, en hardware (TCAM). **Routing** = global, por prefijo, segundos, heurístico, en software.
3. El **circuito cerrado**: protocolo de enrutamiento → algoritmo → RIB → FIB; la FIB se consulta **por paquete** con **longest prefix match**.
4. La tabla es **por prefijo** porque hace falta para agregar, escalar y meter en TCAM. Si matchean varias entradas, **gana la más específica**.
5. **En banda**: el control usa el camino del dato, y un fallo del dato se lleva el control. **Fuera de banda**: el control tiene camino propio y puede reparar al dato.
6. **Flooding** = reenviar por todos los puertos; se controla con **TTL**, **duplicados**, **split horizon**, exclusión del puerto de entrada y **pruning**. En **unicast no hay flooding**; en multicast sí.
7. El **router no implementa capas superiores**; el **cortafuegos sí** necesita puertos y flags porque decide por flujo.
8. **"Hops" es mala métrica**: un salto no pondera ancho de banda ni retardo, así que el camino de menos saltos puede ser el de 10 Mbit/s contra otro de 5 Gbit/s.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué el objetivo de la capa de red es "de sistema final a sistema final" y no "de enlace a enlace"?

<details>
<summary>Respuesta</summary>
</details>

2. En una red con `192.168.1.0/24 → eth2`, `192.168.0.0/16 → eth1` y `0.0.0.0/0 → eth0`, ¿por qué puerto sale `192.168.1.5`? ¿Y si se agrega `192.168.1.128/25 → eth3`?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Qué le pasa a un router al que se le cae el protocolo de enrutamiento? ¿Por qué sigue reenviando?

<details>
<summary>Respuesta</summary>
</details>

4. ¿"En banda" y "fuera de banda"? ¿Cuál permite reparar una red caída y por qué?

<details>
<summary>Respuesta</summary>
</details>

5. ¿En unicast hay flooding? ¿Y qué hace el router con un paquete unicast que no matchea ninguna entrada?

<details>
<summary>Respuesta</summary>
</details>

6. ¿Por qué el router no implementa las capas superiores y el cortafuegos sí?

▶ Respuesta

7. ¿Por qué los protocolos de enrutamiento de IP son "en banda"?

▶ Respuesta

8. Dos caminos al mismo destino: uno de 2 saltos a 10 Mbit/s y otro de 4 saltos a 1 Gbit/s. ¿Cuál elige un algoritmo que cuenta hops? ¿Es el correcto?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1103` / `sirc1103`** — P1 (8 pts): las dos funciones clave de la capa de red y su interacción. La pregunta más repetida del banco sobre este tema.
- **Examen `eirc0902` / `sirc0902`** — P2 (10 pts): forwarding vs. routing en unicast; flooding en broadcast y controlado. Su Problema 2 es de Dijkstra y va en la Clase 33.
- **Examen `eirc1002` / `sirc1002`** — P2 (8 pts): funciones clave, objetivos y las dos formas de servicio de capa 3. · **Examen `eirc0912` / `sirc0912`** — P3 (8 pts): cometido de la capa de red; ¿es "hops" buena métrica?
- **Examen `eirc0807` / `sirc0807`** — P4 (10 pts): plano de datos vs. plano de control, en banda vs. fuera de banda. · **Examen `sirc0712`** — Problema 1 (30 pts): leer los puertos TCP/UDP dentro del paquete IP; capa 7 en un dispositivo intermedio.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red".
- **RFC 791** — Internet Protocol (1981). · **RFC 1812** — Requirements for IP Version 4 Routers (1995). · **RFC 4632** — CIDR: An Address Assignment Plan for IPv4 (2006).
- **RFC 2328** — OSPFv2 (1998): costo de interfaz por ancho de banda y flooding de LSAs.
- **RFC 4271** — BGP-4 (2006). · **RFC 9234** — BGP Roles (2022). · **RFC 6810 / RFC 6811** — RPKI/ROA.
- **RFC 8402** — Segment Routing Architecture (2018). · **RFC 8986** — SRv6 Network Programming (2022). · **RFC 7432 / RFC 7348** — EVPN / VXLAN.
- **Clase siguiente:** 25 — Conmutación en la red: datagramas y circuitos virtuales

Clase 25

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 Las dos familias
1.2 Conmutación de datagramas
1.3 Circuitos virtuales
1.4 La tabla de conmutación: por VC o por prefijo
1.5 El establecimiento: en banda o fuera de banda
1.6 ⚠️ Actualización a 2026
2.1 Ejercicio 1 — Entradas de tabla para 1000 hosts: VC vs. datagramas
2.2 Ejercicio 2 — ¿Qué pasa si se cae un enlace del camino de un VC?
2.3 Ejercicio 3 — Un circuito virtual multipunto
2.4 Ejercicio 4 — El enlace dedicado como caso extremo
3.1 ⭐ Examen 2009-02, Pregunta 4 (9 puntos) — datagramas vs. circuitos virtuales
3.2 Examen 2007-12, Pregunta 3 (10 puntos) — circuitos, paquetes y mensajes
3.3 Examen 2010-02, Pregunta 2c — las dos formas de servicio de capa de red
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: D (Red) · **Semana:** 9 · **Duración estimada:** 105 min **Correlativa:** Clase 24 (forwarding y routing) · **Sigue con:** Clase 26 (IPv4: el datagrama y las direcciones) **Material de referencia:** Tanenbaum cap. 5 (§5.1, "Las dos familias") · RFC 791, RFC 3031, RFC 3032, RFC 4364, RFC 2211, RFC 8200, RFC 8402, RFC 8986

🎯 Al terminar esta clase vas a poder

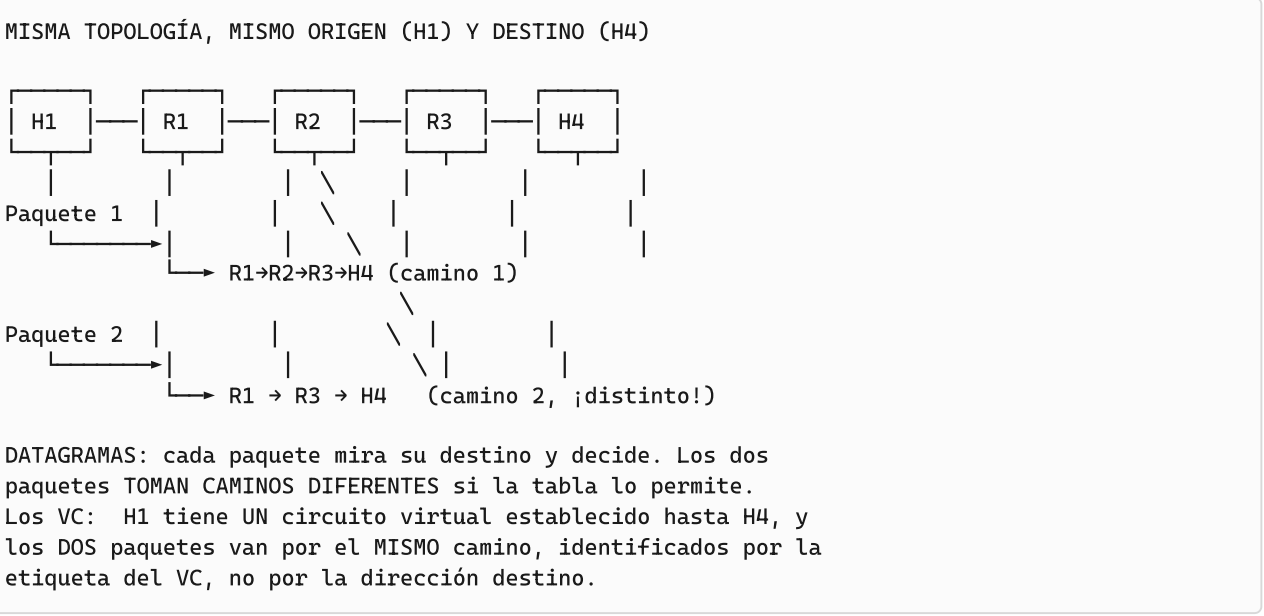
1. **Distinguir** las dos familias de conmutación de paquetes y decir **cuál de las dos usa Internet** y por qué.
2. **Describir** la conmutación de **datagramas**: sin estado, decisión por paquete, caminos distintos para paquetes del mismo flujo, y sus ventajas y desventajas.
3. **Describir** los **circuitos virtuales (VC)**: qué es la etiqueta, por qué hace falta **estado en cada router** y por qué hace falta **establecimiento previo**.
4. **Comparar** la tabla de conmutación **por VC** contra la **por prefijo**: qué se guarda en cada caso, por qué el VC es mucho más rápido de buscar y por qué **no escala**.
5. **Explicar** el establecimiento **en banda** y **fuera de banda** de un circuito virtual, con el caso de la red telefónica clásica y el del enlace dedicado.
6. **Comparar** ventajas y desventajas de datagramas y VC **desde el punto de vista de la asignación de recursos** en la red, que es exactamente lo que pregunta el examen.
7. **Calcular** cuántas entradas de tabla necesita cada esquema para una red de *n* hosts, y explicar qué se rompe cuando un enlace de un VC se cae.

> **Prerequisito del examen:** `2009-02 P4` (9 pts — datagramas vs. circuitos virtuales), `2007-12 P3` (10 pts — circuitos / paquetes / mensajes), `2010-02 P2c` (las dos formas de servicio de capa 3).

PARTE 1 — TEORÍA

1.1 Las dos familias

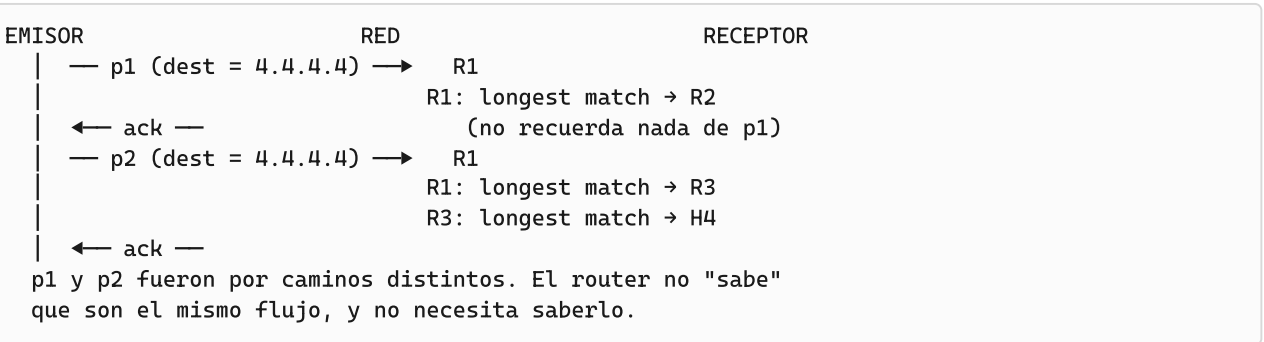
Cuando un nodo tiene que decidir **por dónde sale un paquete**, hay dos familias de diseño, y la diferencia es **una sola pregunta**: *¿la decisión depende del paquete, o depende de un camino previamente establecido?*



La tercera familia —**conmutación de mensajes**, donde el nodo reenvía la unidad completa con el nombre del destino final y el destino se reconstruye en el último salto— se estudia en la Clase 4. Lo que importa acá es que **Internet es conmutación de datagramas**, y todo lo que viene después (VC, MPLS, etiquetas) es un overlay que se apoya sobre ese datagrama.

1.2 Conmutación de datagramas

Definición operativa: cada paquete se reenvía por decisión propia, mirando su **dirección destino** y consultando la tabla (en IP, *longest prefix match*). **No hay conexión, no hay estado, no hay negociación previa.**



Ventajas.

Ventaja	Por qué
No hace falta estado	El router no memoriza nada entre paquetes. Un router que se reinicia no pierde información de la que dependan otros.
Tolera fallos	Si un enlace se cae, el siguiente paquete ya va por otro camino. La reconvergencia la maneja el plano de control, no el conmutador.
Caminos distintos por paquete	Se puede enrutar el tráfico según el estado real de la red, e incluso repartir un flujo entre enlaces (hoy es lo normal con ECMP).
Multiplexado estadístico	Los enlaces se comparten dinámicamente entre todos los flujos, y la capacidad se aprovecha mejor (§1.6 y §2.1).
Arranque instantáneo	El emisor manda el primer paquete sin handshake ni señalización. Es lo que permite que un host de la calle funcione.
Tabla chica	Una entrada por **prefijo**, y los prefijos se **agregan**: 1000 hosts en la misma /24 son **una** entrada.

Desventajas.

Desventaja	Por qué
Decisión por paquete = trabajo por paquete	Hay que leer la cabecera, hacer el *longest prefix match* y elegir interfaz **para cada paquete**. Con VC, la búsqueda es una etiqueta exacta y más rápida.
Reordenamiento	Si dos paquetes del mismo flujo toman caminos distintos, el segundo puede llegar antes que el primero. TCP lo reordena; una aplicación de audio, no.
Cabecera grande	Cada paquete lleva la **dirección destino completa** (y en general también la de origen). Con VC sólo necesita la **etiqueta**, que es un número corto.
Sin garantía de nada	Es *best-effort*: no hay reserva de ancho de banda, de buffer ni de retardo. Un flujo puede tener la mala suerte de encontrar la red ocupada y ser descartado.
Setup con costo	En datagramas, cuando cambia un camino hay que recalcular y volver a anunciar rutas. Con VC, un cambio de camino es un evento raro y ruidoso.

Internet entero es esto. IP no establece nada: el primer paquete sale sin handshake. Cada router decide solo, por prefijo. Si dos paquetes del mismo flujo van por caminos distintos, es **el comportamiento normal, no una excepción**.

1.3 Circuitos virtuales

Definición operativa: se establece un **camino lógico** (*virtual circuit*, VC) y **todos** los paquetes de esa conexión lo siguen, identificados por una **etiqueta de VC** (*VC label*), que es **un número, no una dirección**. Cada router del camino guarda **estado** para ese circuito: a qué puerto va y qué hacer con las etiqueta que llegan por él.

```
                  ESTABLECIMIENTO (una vez, con señalización)
H1 ──── SETUP ──▶ R1 ──── SETUP ──▶ R3 ──── SETUP ──▶ H4
         "quiero un VC hasta H4, quepa en 2 Mbit/s"

H1  ◀── SETUP ACK ─ R1 ◀── SETUP ACK ─ R3 ◀── SETUP ACK ─ H4

                  DATOS (muchos paquetes, sin negociar nada más)
H1  ──[VC 47]──▶ R1 ──[VC 47]──▶ R3 ──[VC 47]──▶ H4
H1  ──[VC 47]──▶ R1 ──[VC 47]──▶ R3 ──[VC 47]──▶ H4
H1  ──[VC 47]──▶ R1 ──[VC 47]──▶ R3 ──[VC 47]──▶ H4
      ▲
   la etiqueta 47 es LOCAL a cada router:
   en R1 "47" significa "salí por eth1",
   en R3 "47" significa "salí por eth2".
   El mismo número puede significar otra cosa en cada salto.
```

Tres condiciones que un VC necesita y que el datagrama no:

1. **Estado en cada router del camino.** El router tiene que tener una entrada que diga "etiqueta 47 → puerto de salida, y esto es lo que hay que hacer con los paquetes que me lleguen con 47". Sin esa tabla no puede conmutar. Ese estado **no existe** en un router de datagramas.
2. **Establecimiento previo.** Antes del primer paquete de datos hay un **intercambio de mensajes de señalización** que recorre el camino del destino hacia el origen y vuelve. Ese intercambio, en sí, es un datagrama: la red por la que viaja el setup es una red de datagramas.
3. **Identificador en la cabecera.** El paquete lleva la etiqueta, que puede ser más corta que la dirección. Esto es lo que hace que la búsqueda en la tabla sea trivial.

Qué es la etiqueta, en concreto. En el ejemplo de arriba la etiqueta "47" **sólo significa algo dentro de cada router**. No es una dirección global, no se puede intergenerational, y el router de ingreso y el de egreso pueden coincidir o no con los extremos. Ese es el punto clave: **con VC se reemplaza el problema "¿a dónde va este paquete?" por el problema mucho más fácil "¿qué hago con esta etiqueta?"**

Dónde existen los VC. Frame Relay (**PVC**, *Permanent Virtual Circuit*, y **SVC**, *Switched Virtual Circuit*), **ATM** (con su *label switching*), **MPLS** (`RFC 3031` arquitectura, `RFC 3032` la pila de etiquetas), X.25. Y en IPv6, el **Flow Label** de la cabecera (`RFC 8200`), que es el intento estandarizado de tener "algo de VC" sin salir del datagrama: por defecto vale **0** (o sea, no se usa), y ningún router de Internet lo interpreta para reenviar.

1.4 La tabla de conmutación: por VC o por prefijo

Acá está el contraste que más se pregunta.

	Conmutación de datagramas	Conmutación por VC
Clave de la tabla	**prefijo de dirección** (con longitud)	**etiqueta de VC** (un número)
Qué se guarda como valor	enlace o puerto de salida (+ next hop, métricas, ECMP)	**número de VC de salida** en el siguiente router, y la acción a ejecutar (reenviar / encapsular / entregar)
Tipo de búsqueda	**longest prefix match**: hay que comparar contra **todas** las entradas y quedarse con la más específica	**índice directo**: `tabla[etiqueta]`, una **lectura de memoria**
Coste de la búsqueda	O(n) comparaciones, en TCAM, en paralelo	**O(1)**, una lectura
Tamaño de la clave	hasta **32 bits + longitud**	**4 bits** (MPLS: 20 bits)
Cantidad de entradas	**una por prefijo agregado** (poco, y crece con los prefijos de Internet)	**una por circuito vigente** (mucho: es el número de conexiones simultáneas)
Se puede cambiar en caliente	no: cambiar la ruta es cambiar la tabla de todos	no: hay que renegociar el circuito
Tolera que un router se reinicie	sí, no tiene estado	no: hay que **re-establecer** los circuitos

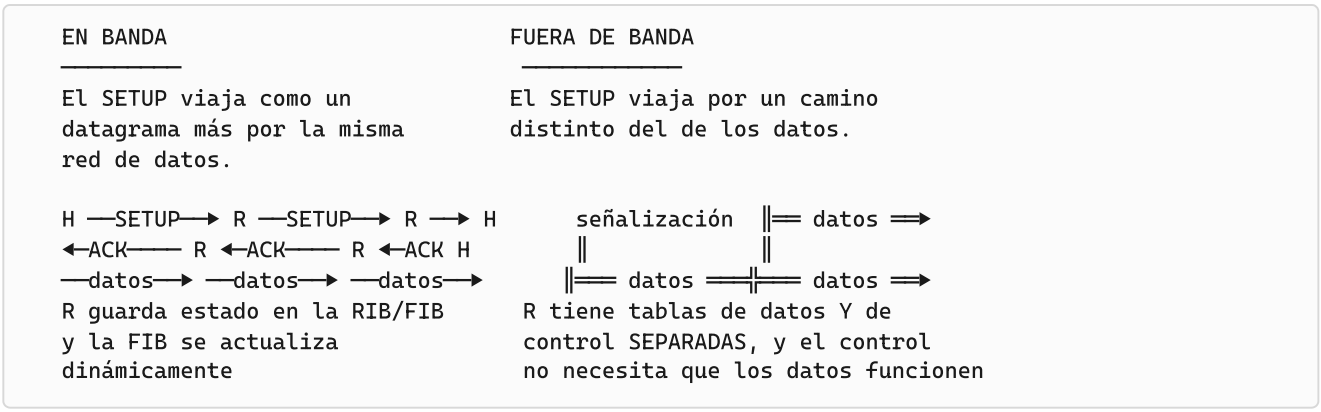
Por qué el VC es más rápido de buscar. Una etiqueta es una **clave primaria**: `tabla[47]`. Una tabla de prefijo es una búsqueda con coincidencia parcial: hay que preguntar a las N entradas "¿cuál de ustedes tiene estos bits como prefijo?" y quedarse con la de prefijo más largo. En hardware, la segunda se resuelve en TCAM (que compara N entradas en paralelo y cuesta caro en silicio); la primera, en una RAM normal o en un simple array indexado. **El VC gana la comparación de tabla y pierde en número de entradas y en flexibilidad.**

Y por qué el VC no escala. Porque su unidad de estado es **la conexión**, no el **prefijo**. Un solo prefijo /24 con 254 hosts es **una** entrada de tabla de datagramas; en el mundo real ese /24 tiene **toda la Internet** detrás. Ese mismo tráfico en VC necesita **una entrada por cada par de extremos que quiera conversar**. Es un ataque a la escalabilidad: el estado crece con el **tráfico y los usuarios**, no con el **espacio de direcciones**. Está en el §2.1 con números.

Por qué esta es la diferencia profunda. El plano de datos de un router moderno tiene una FIB de **millones** de entradas, todas en TCAM. Si el estado estuviera organizado por flujo —que es lo que haría falta para "darle calidad a este TCP"— el router tendría que tener una entrada por cada flujo de Internet, y eso no cabe en silicio. Es exactamente el error de **IntServ** (`RFC 2211`): reservas por flujo que no escalaron. **La razón por la que IP no hizo VC no es que los VC sean malos: es que el estado por flujo no escala.**

1.5 El establecimiento: en banda o fuera de banda

Como en la Clase 24 (§1.5), el setup de un VC puede ir por el mismo canal que los datos o por uno propio.



En banda (ejemplo real): el signaling de Frame Relay SVC y la señalización ATM UNI/PNNI. El mensaje de setup es un paquete que viaja por la red como cualquier otro; los routers lo encaminan, tabla de datos incluida. La consecuencia es la de siempre: **si la red de datos se rompe, el setup también**, y no podés establecer un circuito por un camino que no funciona — lo cual es una paradoja notable: **el circuito que querés crear es precisamente el que arregla el problema**.

Fuera de banda (ejemplo real): la red telefónica clásica. La voz viaja por **circuitos dedicados** conmutados; el control (establecer la llamada) viaja por una **red de señalización separada**, con enlaces y nodos propios. La consecuencia práctica es la que hay que decir siempre: **si se cae la red de señalización, no se puede establecer ninguna llamada aunque todos los circuitos de voz estén libres.** El plano de datos está perfecto y no puede usarse para nada nuevo.

El caso límite: el enlace dedicado. Un enlace punto a punto dedicado (*leased line*) es un circuito con capacidad **1** y sin multiplexado: sólo puede haber una comunicación a la vez, y el camino no se negocia porque **no hay nada que negociar** — la salida es siempre la misma. Es por eso un circuito conmutado en su forma más degenerada, y por eso su setup es **fuera de banda** por definición: lo instala el operador (o el administrador del cable), no un protocolo.

Y el razonamiento es el mismo del `2007-12 P3`. Ese examen pide comparar **conmutación de circuitos, de paquetes y de mensajes**, y el enlace dedicado es el caso extremo de conmutación de circuitos. La misma comparación aplica:

```
CIRCUITOS (incluye el enlace dedicado)
✓ Camino fijo,BW garantizado, retardo acotado, orden garantizado
✓ No hace falta dirección completa en cada paquete
✗ Ancho de banda reservado aunque no se use  →  inefficient con
  tráfico en ráfagas
✗ Establecimiento previo  →  retardo de setup antes del primer dato
✗ Un enlace dedicado = 1 enlace por PAR de nodos = O(n²) enlaces
✗ El equipo de red tiene que asignar y mantener los circuitos

PAQUETES (datagramas)
✓ Sin setup: el primer dato sale ya
✓ El multiplexado estadístico aprovecha los picos de los otros
✓ Un fallo de enlace no rompe la conexión: se busca otro camino
✗ Sin garantías: retardo, jitter, pérdidas y orden no están acotados
✗ Decisión por paquete (más trabajo en el router)
✗ Reordenamiento y cabecera completa por paquete

MENSAJES
✓ Coalescing: uno por mensaje de aplicación, no por paquete
✓ Store-and-forward en cada nodo
✗ Cada nodo debe conocer la ruta completa  →  no escala
✗ Retardo grande: el mensaje entero debe llegar al nodo intermedio
  antes de que pueda reenviarlo
```

1.6 ⚠️ Actualización a 2026

Sigue igual, de memoria:

- **Internet es conmutación de datagramas.** Forwarding por prefijo, sin estado, caminos distintos por paquete. No cambió.
- Las dos familias y sus ventajas/desventajas (reserva de recursos vs. multiplexado estadístico; control de admisión).
- El **VC necesita estado por router** y **establecimiento previo**.
- La etiqueta de VC es **local a cada router**.

Cambió:

- **MPLS sigue desplegándose masivamente, pero en carriers, no en Internet.** `RFC 3031` (arquitectura), `RFC 3032` (pila de etiquetas) y `RFC 4364` (L3VPN) siguen vigentes. Es el VC por excelencia de la infraestructura: el LSP se establece con LDP o RSVP-TE y los datos reenvían **sólo por etiqueta**.
- **El plano de control de MPLS se está moviendo a Segment Routing.** `RFC 8402` (arquitectura, 2018) y `RFC 8986` (SRv6) llevan la idea del VC hasta el extremo: **el estado queda sólo en el borde de ingreso**, y el forwarding de los nodos intermedios es "de SID a SID" sin protocolo de control por salto. **SRv6 codifica los SID dentro de la cabecera IPv6 y elimina el MPLS por completo.** ⇒ Consecuencia conceptual fuerte: **la frontera entre "routing" y "forwarding" se vuelve una convención de diseño, no una necesidad física.**
- **En datacenter, los VC por MPLS fueron reemplazados por EVPN-VXLAN** (`RFC 7432` + `RFC 7348`), que es el mismo juego —etiqueta, estado en el borde, replicación por copia— sobre **UDP** en lugar de MPLS. La etiqueta sigue siendo clave (VNI), pero ya no es MPLS.
- **IntServ es la demostración histórica de que el VC no escala** (`RFC 2211`): mantener estado por flujo en cada router es insostenible, y por eso **DiffServ ganó**. Lo que quedó vivo del VC es la **reserva de recursos en el enlace**, no por flujo: **traffic shaping y policing** (`RFC 2474` / `2475`) marcar el tráfico en el borde y el core aplica una PHB. ⇒ **Sereserve la idea, no el mecanismo por flujo.**

- **El IPv6 Flow Label es el VC que nunca se usó.** La cabecera IPv6 tiene un campo **Flow Label** de 20 bits (`RFC 8200`), que según la especificación debería identificar "flujos que necesitan tratamiento especial". Por defecto es **0** y ningún router de Internet lo usa para reenviar: **el mundo eligió datagramas hasta en el diseño del campo de VC.** Buen ejemplo para el examen de "concepto que existe en el protocolo pero no se usa".
- **El ECMP se comió parte del argumento.** Con ECMP, el datagrama ya reparte un flujo entre varios next hops, así que la "ventaja" del VC de que "el camino es fijo" es **cada vez menos una ventaja**: es una diferencia de *cuánto control* se tiene, no de *si se puede*.

Consecuencia para el examen: la respuesta clásica (datagramas = sin estado ybest-effort; VC = con estado y QoS garantizado) **se sigue preguntando y se sigue respondiendo igual**. Lo que suma es una línea: "*hoy el VC persiste como MPLS en carriers y como overlay (VXLAN/EVPN) en datacenter, pero el estado se concentró en el borde con Segment Routing; y el que no escaló fue el VC con reserva por flujo (IntServ), no el VC en sí.*"

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Entradas de tabla para 1000 hosts: VC vs. datagramas

Datos. Una red interna con **1000 hosts**, todos en la misma LAN lógica (una sola subnet), detrás de un router de borde que los conecta a Internet. Se comparan los dos esquemas.

Pregunta: ¿cuántas entradas de tabla necesita cada router?

Resolución, esquema A — conmutación de datagramas.

```
Los 1000 hosts están en 192.168.10.0/24 (una sola subnet).

FIB del router de borde:
┌──────────────────────────┬──────────────────────────────┐
│ 192.168.10.0/24          │ eth0 (directamente conn.)    │
│ 0.0.0.0/0                │ eth1, next hop 200.1.1.1     │
└──────────────────────────┴──────────────────────────────┘
                                  ▲
                  ¡ 2 entradas para 1000 hosts y para
                    todo Internet, gracias a la AGREGACIÓN !

Cada host que habla con Internet usa la entrada 0.0.0.0/0.
Los 1000 hosts comparten la misma entrada porque comparten
prefijo. Nada en la tabla dice "este host".
```

Y si cada host estuviera en su propia subnet (por ejemplo, un /30 por host), entonces sí habría **una entrada por host**: 1000 entradas. Ese es el peor caso del datagrama.

Resolución, esquema B — circuitos virtuales.

Para que **cualquiera de los 1000 hosts pueda hablar con cualquier otro** a través de la red conmutada, hace falta un VC por cada par:

```
número de circuitos entre pares de hosts:

Si el tráfico es UNIDIRECCIONAL y todos hablan con todos:

     N × (N − 1) = 1000 × 999 = 999 000 circuitos

      ┌──────────────────────────────────────┐
      │  Tabla de VC del router:             │
      │  etiqueta 47      → VC 91, eth1      │
      │  etiqueta 91      → VC 22, eth2      │
      │  etiqueta 22      → VC 47, eth3      │
      │  ...                                 │
      │  999 000 entradas                    │
      └──────────────────────────────────────┘

Si el circuito es BIDIRECCIONAL (lo normal):

     N × (N − 1) / 2 = 1000 × 999 / 2 = 499 500 circuitos
```

La comparación, que es el punto de la clase:

	Datagramas	VC
Unidad de estado	el **prefijo**	el **par de extremos**
Entradas para 1000 hosts en una subnet	**2**	hasta **999 000**
Entradas para 1000 hosts en subnets separadas	**1000**	**999 000**
Entradas para **toda Internet**	**millones de prefijos** (agregables)	**una por conexión**
Crecimiento	con el **espacio de direcciones**	con los **usuarios y el tráfico**

Interpretación: el VC necesita **tres órdenes de magnitud** más estado, y su estado **no se puede agregar**: no hay forma de decir "estos 1000 hosts se parecen en algo" porque la etiqueta no tiene estructura. El datagrama sí, porque el prefijo **es** una agrupación. **Ésta es, literalmente, la razón por la que Internet no es una red de circuitos virtuales.** Y es la misma razón por la que el enrutamiento **inter-AS** es path-vector y no distance-vector: en un dominio grande, la métrica por flujo o por vecino se vuelve inmanejable.

2.2 Ejercicio 2 — ¿Qué pasa si se cae un enlace del camino de un VC?

Datos. Un VC establecio H1 ↔ H4 con la ruta H1 – R1 – R2 – R3 – H4. Se cae el enlace R2–R3.

Resolución — paso a paso, comparando los dos esquemas.

Opción A: mantener el VC (como haría Frame Relay con una PVC, o MPLS).

```
t0   el enlace R2–R3 se cae.

t0+  Los paquetes con etiqueta 47 que llegan a R2 seeking salir
     por el enlace caído: se DESCARTAN. El VC sigue "vivo" en
     las tablas: R1, R2, R3 no se enteran de nada.

t0..t+k   Para que alguien lo note hace falta un mecanismo:
     • Frame Relay: mensajes de keepalive cada ~10 s. Si no
       llegan, el circuito se declara DOWN.
     • MPLS con RSVP-TE y LSP de respaldo: Fast Reroute.
       Un camino secundario ya estaba establecido, así que
       la transición es de milisegundos.
     • MPLS con LDP: el LDP se reconecta y vuelve a
       señalizar. Segundos.

t+k   Detección: cada router involved notifica al head-end
     (o al protocolo de señalización).

t2   Si hay respaldo: el VC pasa a usar el camino nuevo, y el
     tráfico se reanuda. Los paquetes perdidos en el hueco
     NO se recuperan: el VC no retransmite.

Si NO hay respaldo: el VC queda DOWN y hay que
re-establecerlo con un nuevo setup. La aplicación tiene que
detectar la caída (timeout) y reconectar.
```

Opción B: red de datagramas.

```
t0   el enlace R2–R3 se cae.

t0+  Los paquetes que iban por ahí se pierden. Los SIGUIENTES
     van por otro camino, si la tabla ya se actualizó. Si la
     tabla todavía no se actualizó, también se pierden
     (la ruta vieja apunta al enlace caído).

t1   El plano de control detecta el fallo y recalcula.
     Con OSPF, los LSA actualizados en ~5 s a 40 s, según el
     temporizador hello/dead. Con BGP, minutos.

t2   La FIB nueva se instala. A partir de ahí, el tráfico
     fluye por el camino nuevo SIN que nadie haya negociado
     nada: no hay conexión que re-establecer.

Lo que se pierde: los paquetes que estaban en vuelo.
Eso lo resuelve TCP con un timeout o 3 dup-ACKs. Nada
necesita renegociar.
```

La comparación, y es la respuesta:

	VC	Datagramas
Detección del fallo	**explícita**: keepalive, señalización o un LSP de respaldo ya armado	**implícita**: los mensajes dejan de llegar, o el siguiente paquete simplemente no sale
Convergencia	hay que **re-establecer**: segundos si hay respaldo, más si no	**automática**: el siguiente paquete ya busca otro camino
Recuperación de datos	**no hay**: el VC no retransmite	la **capa de transporte** (TCP) reenvía
Aplicación	puede tener que **reconectar**	no se entera

Interpretación: el VC es **más eficiente cuando todo anda bien** (búsqueda O(1), camino conocido, sin reordenamiento, calidad garantizada) y **más frágil cuando algo se rompe**, porque el estado hay que reconstruirlo y hay que reconstruirlo *de verdad*. El datagrama es al revés:algo más de trabajo por paquete, pero ningún estado que pueda quedar inconsistente. **Ese es el intercambio clásico entre eficiencia y robustez, y es exactamente por lo que Internet eligió robustez.**

2.3 Ejercicio 3 — Un circuito virtual multipunto

Datos. Un servidor de streaming quiere enviar el mismo flujo a 500 espectadores. ¿Se puede con un solo VC? ¿Qué cambia en los routers?

Resolución. Un VC normal es **punto a punto**: un puerto de entrada, un puerto de salida. Si un VC tiene que llegar a varios destinos, hace falta una de dos formas, y en las dos **el router tiene que dejar de ser un conmutador**:

```
(a) MULTIPOINT-TO-POINT  (bidireccional, "árbol" compartido)
    Todos los que están en el VC pueden hablar entre sí.

          ┌── V2 ──┐
          │        ▼
R1 ──────┼── V1 ──┼── V3 ── R4
          │        ▼
          └── V4 ──┘

    En R1: UNA entrada  VC 100 → "copiá a las interfaces eth0..eth3"
    En los otros: cada uno tiene su VC y reenvía uno a uno.
    El tráfico entre V2 y V3 pasa por R1 y se duplica si
    van en sentidos distintos.

(b) POINT-TO-MULTIPOINT  (unidireccional, "el que manda empuja")
    Un emisor, muchos receptores. El emisor no recibe.

       R1 ── V1 ──┬── V2 ──► V2
                  ├── V3 ──► V3
                  └── V4 ──► V4

    En R1: UNA entrada  VC 100 → "reenviá a eth0, eth1 y eth2"
    Coste: si las ramas son enlaces distintos, el ancho de banda
    se multiplica por 500. Si comparten un enlace, se copia en
    el buffer de salida (y el router tiene que tener RAM para
    500 copias del mismo paquete).
```

Lo que hay que decir:

1. **Un solo VC multipunto reduce muchísimo la tabla**: una entrada en vez de 500. Ese es su appeal.
2. **Pero rompe el modelo de conmutación**: un router que copia a N salidas necesita **memoria proportional a N** (o al menos un buffer de replicación), y necesita poder decidir a quién le copia (si el grupo es dinámico, necesita un estado de membresía, que es exactamente el estado que el VC multipunto se supone que elimina).
3. **Y el ancho de banda se multiplica** si las ramas son independientes.
4. **El equivalente en el mundo de datagramas es el multicast**, y funciona *exactamente por la razón contraria*: se paga estado (quien pertenece al grupo) para **no** pagar ancho de banda, y se resuelve con **flooding + prune** (§1.6 de la Clase 24) y no con un VC. La historia de IP multicast es la historia de intentar hacer multicast **sin** estado por VC.

> En clase, cuando te pidan "un VC multipunto", dibujá las dos formas (árbol bidireccional y empuje unidireccional), y sumá la comparación con multicast. Es un ejercicio que aparece en varios exámenes de ATM y Frame Relay.

2.4 Ejercicio 4 — El enlace dedicado como caso extremo

Enunciado. ¿Es lo mismo un "enlace dedicado" que un circuito conmutado? ¿Y un circuito virtual? Justificá.

Resolución. Los tres son de la familia de **conmutación de circuitos** — camino fijo y dedicada capacidad — pero difieren en cuánto es el circuito y quién lo instala:

```
ENLACE DEDICADO                 CIRCUITO VIRTUAL CONMUTADO
_______________                 __________________________

Un cable físico entre           Un circuito lógico dentro de
dos nodos.                       un enlace compartido.
                                 Switches en el medio eligen
dedicated = no comparte          por dónde sale el circuito.
Capacidad fija para ese par.     Comparte con otros circuitos.

El "setup" es el CARLEO del      El "setup" es un protocolo
cable: lo instala el operador    de señalización entre los
o el administrador.              extremos, en línea.

1 enlace por PAR de nodos.       1 enlace para N nodos
n nodos → n(n-1)/2 enlaces.      (con switches de N puertos).
```

Por qué el enlace dedicado es el mismo razonamiento que el `2007-12 P3`. Porque los tres casos de aquel examen (circuitos, paquetes, mensajes) son la misma comparación en distintos puntos del espectro:

```
               CIRCUITOS      PAQUETES       MENSAJES
               _________      ________       ________

Unidad          sesión          paquete        mensaje de
                                              aplicación
Camino          FIJO            por paquete    completo
Setup           SÍ              NO             NO
BW           reservado       Statistical    reservado
Reordena.       NO              SÍ             NO
Multiplexado    NO              SÍ             según

El enlace dedicado es el extremo izquierdo del todo:
circuito con capacidad 1, sin multiplexado y sin setup en línea.
```

El cálculo que lo mata. Con **n** nodos:

- Enlace dedicado: **n(n−1)/2** enlaces. Para 20 nodos: `20·19/2 = 190` enlaces físicos.
- Conmutación de circuitos: **1** switch, **n** enlaces, y los circuitos se crean y se liberan según hacen falta.
- Conmutación de paquetes: **1** switch, **n** enlaces, y los enlaces se comparten dinámicamente sin reservar nada.

O sea: **190 enlaces contra 20.** El enlace dedicado es la solución correcta cuando el tráfico es constante y de volumen alto (y aun así: 190 leases que pagar). Es la solución **equivocada** para el tráfico en ráfagas de una red de oficinas, porque el 90 % del tiempo estás pagando una capacidad que no usás. **Ese es el mismo argumento del multiplexado estadístico, sólo que con una red de 190 enlaces en vez de uno.**

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2009-02, Pregunta 4 (9 puntos) — datagramas vs. circuitos virtuales

> **Fuente:** Examen `eirc0902` / `sirc0902` — 2 de febrero de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0902.pdf **Texto del examen:**
>
> > **a)** Describa la conmutación de paquetes basada en: **i.** Datagramas **ii.** Circuitos virtuales. **b)** Explique ventajas y desventajas de cada opción desde el punto de vista de la asignación de recursos en la red.

Solución oficial (b, literal del PDF):

> Básicamente las redes de datagramas son "de mejor esfuerzo", es decir que consumen los recursos disponibles, y en caso de congestión degradan el servicio por igual a todos los usuarios; aprovechan bien el multiplexado estadístico del tráfico en la red. Las redes de VC ofrecen garantía de QoS basadas en la reserva de recursos, y por lo tanto protegen a los usuarios de las situaciones de congestión, a costo de disponer de más recursos o rechazar conexiones (control de admisión).

Sobre la parte a): el PDF de solución **copia el enunciado de la parte a) sin contenido** (remite a las transparencias del curso). Lo que sigue para a) es **solución propia**.

Parte a) — resolución

i. Datagramas. Cada paquete se reenvía por decisión propia, leyendo **su propia dirección destino** y consultando la tabla (en IP, *longest prefix match*). **No hay conexión, no hay estado en el router, no hay negociación previa**: el emisor manda el primer paquete sin handshake. Consecuencias: los paquetes de un mismo flujo **pueden tomar caminos distintos** (y entonces pueden llegar desordenados); si la red cambia, los siguientes paquetes ya aprovechan el cambio sin renegociar nada; y el router no guarda información entre paquetes, así que un reinicio de un router no rompe nada.

ii. Circuitos virtuales. Antes de enviar datos hay un **establecimiento previo**: un intercambio de mensajes de señalización que recorre el camino del destino al origen y vuelve, y que **reserva recursos** en cada enlace del camino. Los datos viajan después identificados por una **etiqueta de VC** (*VC label*), que es **un número corto, no una dirección**, y **todos los paquetes de esa conexión siguen el mismo camino**. Cada router del camino guarda **estado** para ese circuito (etiqueta de entrada → etiqueta y puerto de salida), y ese estado **no existe** en un router de datagramas.

La frase que resume la diferencia, y que hay que decir: en **datagramas la decisión se toma por paquete** (la unidad de estado es el prefijo); en **circuito virtual la decisión se tomó una vez por conexión** (la unidad de estado es el circuito, identificado por una etiqueta). Todo lo demás —ventajas, desventajas, costo, escalabilidad— se deduce de ahí.

Parte b) — análisis de la solución oficial

La solución oficial es **excelente** y contiene las tres ideas que hay que dar:

1. **Datagramas = mejor esfuerzo**: consumen los recursos disponibles y "en caso de congestión **degradan el servicio por igual a todos los usuarios**". O sea: **ningún usuario tiene prioridad**; el router no sabe a quién pertenece el paquete. Eso es *best-effort* en el sentido fuerte.
2. **Datagramas = bueniplexado estadístico**: aprovechan bien los recursos porque **el tráfico en Internet es en ráfagas** y las capacidades que se reservan dinámicamente se comparten. Si dos flujos usan el mismo enlace en instantes distintos, el ancho de banda está aprovechado.
3. **VC = garantía de QoS por reserva**, que "protege a los usuarios de las situaciones de congestión" pero "**a costo de disponer de más recursos o rechazar conexiones** (**control de admisión**)".

Qué agregaría:

- **El costo en memoria del router.** El VC no sólo reserva ancho de banda: reserva **estado**. Cada router tiene que guardar entradas de tabla por circuito. Ese estado es el que no escala (§2.1: hasta **999 000** circuitos para 1000 hosts), y por eso la reserva de recursos por VC tiene un techo muy bajo en Internet.
- **La asimetría entre "rechazar conexiones" y "degradar a todos".** El control de admisión es una decisión **explícita y temprana**: si no hay recursos, el usuario recibe un "no". El *best-effort* es una decisión **implícita y tardía**: el usuario no se entera hasta que ve que le faltó. Para el proveedor, el control de admisión es **predecible**; para el usuario, es **injusto**.
- **El coste de la conmutación de circuitos con recursos compartidos es el número de circuits que deben estar "listos"**. Si un VC necesita 2 Mbit/s garantizados y hay 100 usuarios, hacen falta 200 Mbit/s **reservados** en el enlace, o de lo contrario hay que **rechazar** el VC número 101. Ese es exactamente el "costo de disponer de más recursos" de la solución oficial, y es la razón por la que un operador con una red de 1 Gbit/s no puede ofrecer QoS a 100 usuarios de 20 Mbit/s aunque el promedio de uso sea 1 Mbit/s.
- **El caso en el que el VC gana de verdad: el tráfico en ráfagas con requirement de calidad.** Si 90 % de la capacidad reservada está ociosa, el VC está desperdiciando; pero si hay una aplicación que necesita un piso de calidad (voz en tiempo real, un enlace de respaldo), la reserva es la única manera de **garantizarlo**. El problema nunca es la idea del VC: es hacerlo **con reserva por flujo** y no por clase.

> **Tope para 9 puntos** (a ~4, b ~5): en (a), las dos definiciones con sus tres condiciones (decisión por paquete / etiqueta + estado + setup). En (b), *best-effort* y multiplexado estadístico para datagramas; reserva de recursos, garantía de QoS y **control de admisión** para VC; y el costo (más recursos o rechazo) que es la palabra clave. Con sólo decir "datagramas es mejor esfuerzo y VC garantiza calidad" son 4-5 de 9.

3.2 Examen 2007-12, Pregunta 3 (10 puntos) — circuitos, paquetes y mensajes

> **Fuente:** `eirc0712` / `sirc0712` — 27 de diciembre de 2007. **Texto:**
>
> > Explique en que consiste la conmutación de circuitos, conmutación de paquetes y conmutación de

> mensajes. Realice un análisis comparativo.

Nota: el PDF de solución no desarrolla esta pregunta. El desarrollo completo de la conmutación de circuitos / paquetes / mensajes es la **Clase 4**; acá va el resumen en la forma de tabla de §1.5, más el enlace al razonamiento del enlace dedicado (§2.4).

Lo que hay que decir, en una frase cada una.

- **Conmutación de circuitos:** se establece un camino dedicado de principio a fin antes de transmitir, y todo el tráfico de esa sesión va por ese camino, con recursos reservados. La red telephone clásica es el ejemplo.
- **Conmutación de paquetes:** no hay camino previo; cada paquete lleva la dirección destino y el switch decide por dónde sale. Es la conmutación de datagramas si las decisiones son independientes, y la de **VC** si hay un circuito establecido con etiqueta. Internet es de paquetes.
- **Conmutación de mensajes:** la unidad conmutada es el **mensaje completo de la aplicación**. El nodo intermedio tiene que **recibir el mensaje entero** antes de poder reenviarlo (*store-and-forward*), y necesita **conocer la ruta completa** hasta el destino final, porque cada mensaje lleva un nombre de destino y una lista deilhitos. **No escala** y agrega retardo, a cambio de una muy buena eficiencia en el número de paquetes por mensaje (**coalescing**).

La tabla comparativa es la de §1.5. Y si te piden un ejemplo de cada una: **circuitos** = la llamada telefónica (o un enlace dedicado); **paquetes** = un datagrama IP, o un VC de MPLS; **mensajes** = un email que viaja como un único bloque por la red (las redes *store-and-forward* de los origen de los años '80).

3.3 Examen 2010-02, Pregunta 2c — las dos formas de servicio de capa de red

> **Fuente:** `eirc1002` / `sirc1002` — 2 de febrero de 2010. **Texto:**
>
> > c) Describa brevemente las dos formas de prestar servicios de capa de red.

Es exactamente el tema de esta clase. Las dos formas son:

1. **Servicio de conmutación de datagramas** (*datagram service*): sin conexión y sin estado, cada paquete se reenvía por decisión propia con longest prefix match, los paquetes de un mismo flujo pueden tomar caminos distintos, no hay reserva de recursos y el servicio es *best-effort*. **Es el servicio de Internet.**
2. **Servicio de conmutación de circuitos virtuales** (*virtual circuit service*): con conexión, hay establecimiento previo, se establece un camino lógico que todos los paquetes siguen identificados por una etiqueta, cada router guarda estado para ese circuito, y puede haber reserva de recursos con control de admisión y garantías de calidad. **Es el servicio de Frame Relay, ATM y MPLS.**

Nota: el enunciado de 2010-02 es ambigüo y muchos alumnos lo responden como "conexión / sin conexión". La lectura correcta es "datagramas / circuitos virtuales", porque son las dos formas de conmutación dentro de una **red de paquetes** — es la misma pregunta que la `2009-02 P4` de esta clase, con otra redacción.

PARTE 4 — CIERRE

📌 Resumen

1. Las dos familias se distinguen por **una pregunta**: ¿la decisión depende del **paquete** o de un **camino previamente establecido**?
2. **Datagramas**: sin conexión, sin estado, decisión **por paquete**, caminos distintos para el mismo flujo, tabla por **prefijo** y **agregable**. Es Internet entero.
3. **Circuitos virtuales**: **establecimiento previo** + **etiqueta** + **estado en cada router**. La etiqueta es un número **local** a cada router, no una dirección.
4. La tabla por VC es una **lectura O(1)**; la de prefijo, un **longest prefix match** en TCAM. El VC gana la búsqueda y pierde en **cantidad de entradas** (999 000 circuitos para 1000 hosts) y en **agregabilidad**.
5. La **razón profunda** de que Internet no sea de VC: el estado por flujo **no escala en silicio**. Es el error de IntServ (`RFC 2211`).
6. El **enlace dedicado** es el circuito conmutado en su forma más degenerada: capacidad 1, sin multiplexado, `n(n−1)/2` enlaces para `n` nodos, y setup **fuera de banda**.
7. El VC es **más eficiente si nada se rompe y más frágil si algo se rompe**: su estado hay que reconstruirlo; el datagrama no tiene estado que pueda quedar inconsistente.
8. **Multipunto** rompe el modelo de conmutación (hay que copiar a N salidas, con memoria proporcional a N); su equivalente escalable en el mundo datagrama es el **multicast** con *flood-and-prune*.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Qué distingue formalmente a un router de datagramas de uno de circuitos virtuales?

<details>
<summary>Respuesta</summary>
</details>

2. ¿Por qué la etiqueta de VC es "local a cada router" y qué consecuencia tiene?

<details>
<summary>Respuesta</summary>
</details>

3. 1000 hosts en una misma subnet. ¿Cuántas entradas de tabla necesita cada esquema?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué Internet no es una red de circuitos virtuales?

<details>
<summary>Respuesta</summary>
</details>

5. ¿Qué le pasa a un VC si se cae un enlace de su camino? ¿Y a un flujo de datagramas?

<details>
<summary>Respuesta</summary>
</details>

6. ¿Qué es un circuito virtual multipunto y por qué rompe el modelo de conmutación?

▶ Respuesta

7. ¿El enlace dedicado es lo mismo que un circuito virtual? Explicá con un cálculo.

▶ Respuesta

8. En 2026, ¿sigue siendo verdad que "Internet es conmutación de datagramas"? ¿Qué cambió en el mundo de los VC?

▶ Respuesta

🔗 Referencias

- **Examen `eirc0902` / `sirc0902`** — P4 (9 pts): conmutación por datagramas vs. circuitos virtuales y ventajas/desventajas desde la asignación de recursos. La pregunta central de esta clase.
- **Examen `eirc0712` / `sirc0712`** — P3 (10 pts): conmutación de circuitos, de paquetes y de mensajes, con análisis comparativo (desarrollada en la Clase 4; el enlace dedicado es el caso extremo de circuitos).
- **Examen `eirc1002` / `sirc1002`** — P2c: las dos formas de prestar servicios de capa de red. · **Examen `eirc1103` / `sirc1103`** — P1: las dos funciones clave de la capa de red.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red" — §5.1 las dos familias de redes: Circuitos virtuales y conmutación de datagramas.
- **RFC 791** — Internet Protocol (1981): el modelo de datagrama, sin conexión y best-effort.
- **RFC 3031** — MPLS Architecture (2001) · **RFC 3032** — MPLS Label Stack (2001) · **RFC 4364** — MPLS L3VPN.
- **RFC 2211** — IntServ (1997): el modelo de reserva por flujo que **no escaló**, y la razón por la que DiffServ ganó.
- **RFC 2474 / RFC 2475** — DS field y arquitectura de DiffServ: lo que quedó de la QoS, pero por **clase**, no por flujo.
- **RFC 8402** — Segment Routing Architecture (2018) · **RFC 8986** — SRv6 Network Programming (2022): estado sólo en el borde de ingreso.
- **RFC 8200** — IPv6 (2017): el campo **Flow Label**, el VC que nunca se usó.
- **RFC 7432 / RFC 7348** — EVPN / VXLAN: el VC del datacenter sobre UDP, que reemplazó a MPLS ahí.
- **Clase siguiente:** 26 — IPv4: el datagrama y las direcciones

Clase 26

Contenido de esta clase

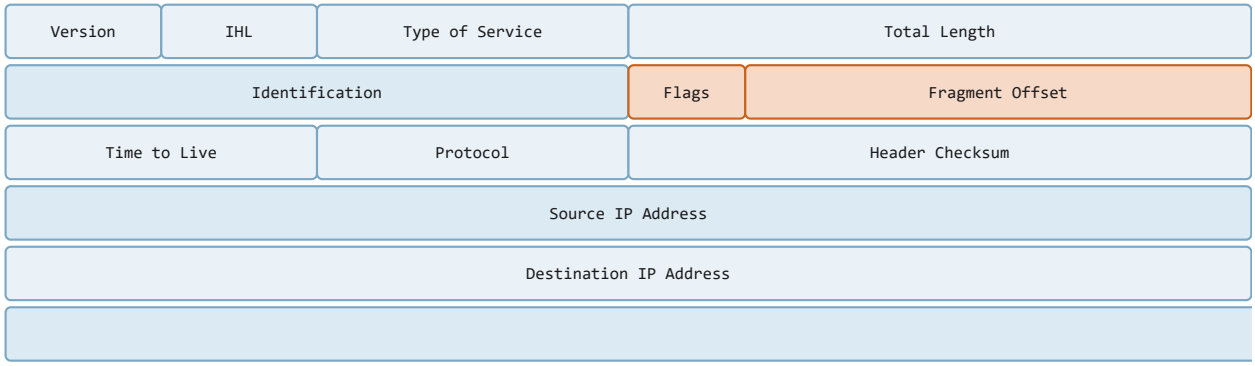
- 🎯 Al terminar esta clase vas a poder
- 1.1 El datagrama IPv4 en su contexto
- 1.2 La cabecera IPv4, campo por campo
- 1.3 El campo IHL y por qué la cabecera es de tamaño variable
- 1.4 La máscara de subred, aplicada bit a bit
- 1.5 Direcciones claseful (A, B, C, D, E) y por qué quedaron obsoletas
- 1.6 CIDR: por qué existe
- 1.7 Direcciones especiales
- 1.8 ⚠️ Actualización a 2026
- 2.1 Ejercicio 1 — Binario y máscara, de punta a punta
- 2.2 Ejercicio 2 — Clase, máscara por defecto y capacidad
- 2.3 Ejercicio 3 — El checksum de la cabecera IPv4, cálculo completo
- 2.4 Ejercicio 4 — ¿Por qué DHCP asigna `169.254.x.x` y no `0.0.0.0`?
- 3.1 ⭐ Examen 2010-03, Problema 1 (30 puntos) — tres subredes /24, R1 y R2
- 3.2 Nota: cuándo sí se pueden agrupar dos `/24` — el caso del Examen 2009-12
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: D (Red) · **Semana:** 9 · **Duración estimada:** 110 min **Correlativa:** Clase 25 (conmutación en la red) · **Sigue con:** Clase 27 (subnetting, método paso a paso) **Material de referencia:** Tanenbaum cap. 5 (§5.4) · RFC 791, RFC 1122, RFC 1812, RFC 1878, RFC 4632, RFC 8200

🎯 Al terminar esta clase vas a poder

1. **Dibujar** el formato de la cabecera IPv4 campo por campo en filas de 32 bits, con el **tamaño en bits** de cada campo y un **ejemplo numérico** de cada uno.
2. **Explicar** para qué sirve el campo **IHL**, por qué la cabecera es de **tamaño variable**, y qué límite impone el campo *Longitud total*.
3. **Aplicar una máscara de subred bit a bit** sobre una dirección en binario, y obtener de ahí la **dirección de red**, el **rango de hosts** y la **dirección de broadcast**.
4. **Identificar la clase** (A/B/C/D/E) de una dirección, dar su **máscara por defecto** y decir **cuántos hosts** caben.
5. **Explicar** por qué las clases quedaron obsoletas, qué es **CIDR** y qué problema económico resolvió.
6. **Distinguir** los rangos **especiales** de IPv4 (`0/8` , `127/8` , `169.254/16` , `224/4` , `255.255.255.255`) y las **direcciones privadas** (`10/8` , `172.16/12` , `192.168/16`), y decir para qué sirve cada uno.
7. **Calcular** el **checksum** de una cabecera IPv4 con la suma del complemento a uno, y verificar que da `0xFFFF` .

Cabecera IPv4, campo por campo



En naranja: los tres campos que hacen posible la fragmentación. El bit DF es el 2.º de Flags.

IHL = 5 significa cabecera de 20 bytes sin opciones.

Como se aplica una mascara de subred, bit a bit

IP	Mascara (/26)	AND = resultado
11000000	11111111	**11000000**
10101000	11111111	**10101000**
00001010	11111111	**00001010**
00110111	11000000	**00000000**

192.168.10.55 AND 255.255.255.192 = 192.168.10.64

Lo que resulta de ese AND:

direccion de red	192.168.10.64
primer host	192.168.10.65
ultimo host	192.168.10.126
broadcast	192.168.10.127
hostables	62

El prefijo /26 significa:

26 bits de red + 6 bits de host

2^6 = 64 direcciones en la subred

64 - 2 = 62 hostables

El AND pone en 0 todos los bits de host: por eso el resultado es la direccion de RED, no una de host.

> **Prerequisito del examen:** `2010-03 Problema 1` (30 pts — tres subredes `/24` con R1 y R2: tablas de enrutamiento y por qué no se pueden renumerar agrupando), `2009-12 Problema 1` (tres clases C y su agregación), `2008-12 P2`, `2011-03 P2d`.

PARTE 1 — TEORÍA

1.1 El datagrama IPv4 en su contexto

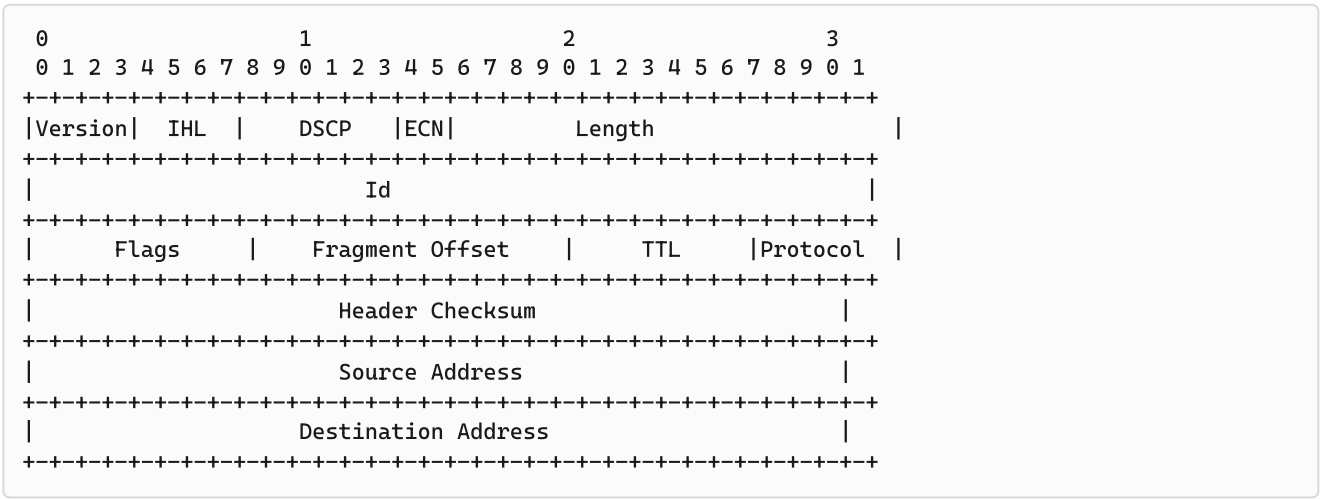
Antes de la cabecera hay que tener claro **dónde viaja**. El datagrama IP no viaja solo: viaja **encapsulado** dentro de una trama de capa 2.

```
┌──────────────────────────── Trama Ethernet (o de otro enlace) ────────┐
│  PREÁMBULO │  MAC destino 6 │ MAC origen 6 │ Ethertype 2 │  ...      │
│  ┌──────────────────────── Datagrama IP (20 a 60 bytes) ──────────┐   │
│  │  CABECERA IP: 20 bytes si IHL = 5                          │   │
│  │  DATOS: el segmento TCP (20 bytes de cabecera + datos)     │   │
│  └────────────────────────────────────────────────────────────┘   │
│  ... │  FCS 4 │                                                   │
└──────┴────────┴─────────────────────────────────────────────────────┘

Ethertype = 0x0800  →  "el contenido de esta trama es un datagrama IP"
Ethertype = 0x0806  →  "es un mensaje ARP"  (ver Clase 30)

MTU del enlace = la mayor trama que transmite sin fragmentar.
Ethernet: 1500 bytes.  Si la trama completa es 1518 bytes con la
cabecera de Ethernet, el datagrama IP disponible es 1500 bytes.
```

El **datagrama IP tiene un tamaño máximo de 65 535 bytes** (por el campo *Longitud total*, 16 bits), pero el **tamaño mínimo obligatorio es 576 bytes** (de `RFC 791` y `RFC 1122` §3.2.1), para que las redes con MTU chico no fragmenten. En la práctica el MTU manda: 1500 en Ethernet, 9000 (*jumbo frames*) en algunos datacenters — ojo, **9000 es una convención, no un estándar IEEE** — y 1280 como mínimo absoluto en IPv6.

1.2 La cabecera IPv4, campo por campo

La cabecera mínima son **20 bytes** (160 bits), y se dibuja siempre en **cinco filas de 32 bits**:



#	Campo	Bits	Qué significa	Ejemplo numérico
1	**Versión**	4	Qué versión del protocolo IP es este datagrama. `0100` = 4.	`4` (IPv4). En IPv6 son los 4 primeros bits de una cabecera de 40 bytes.
2	**IHL** (*Internet Header Length*)	4	**Longitud de la cabecera en palabras de 32 bits**, no en bytes. Multiplicá por 4 para obtener los bytes.	`0101` = 5 → `5 × 4` = **20 bytes**. El mínimo es 5 (20 B); el máximo es 15 (60 B, con 40 bytes de opciones).
3	**DSCP**	6	*Differentiated Services Code Point*: **con qué prioridad** tratar el paquete (DiffServ).	`000000` = mejor esfuerzo (*best effort*), que es el valor por defecto. `101110` = EF (*Expedited Forwarding*), voz.
4	**ECN**	2	*Explicit Congestion Notification*: los 2 bits de la Clase 21.	`00` = No-ECT (el origen no usa ECN). `11` = CE (*Congestion Experienced*).
5	**Longitud total** (*Total Length*)	16	**Longitud de TODO el datagrama IP: cabecera + datos, en bytes.** Máx. 65 535. Mín. 20 (cabecera sola).	`0054` = 84 bytes: 20 de cabecera IP + 20 de cabecera TCP + 44 de datos.
6	**Identificación** (*Identification*)	16	Identifica **un grupo de fragmentos** del mismo datagrama original. Sólo importa si hay fragmentación.	`1C46` . Lo usan la fuente y los routers intermedios para reensamblar.
7	**Flags**	3	Tres banderas: bit de reserva (**siempre 0**), **DF** (*Don't Fragment*) y **MF** (*More Fragments*).	`100` = DF=1, MF=0 → "**no me fragmentes**" (comportamiento por defecto hoy). `001` = DF=0, MF=1 → "hay más fragmentos".
8	**Offset de fragmento** (*Fragment Offset*)	13	Posición **en bytes de los datos** de este fragmento dentro del datagrama original. Se cuenta en **unidades de 8 bytes**, así que los últimos 3 bits no se usan.	`0000` = este fragmento empieza en el byte 0 del payload original (no es un fragmento). `0001` = arranca en el byte 8.
9	**TTL** (*Time To Live*)	8	**Contador de saltos.** Cada router lo **decrementa en 1** antes de reenviar; al llegar a 0, descarta el paquete y manda un ICMP *Time Exceeded*.	`40` = 64 saltos. Windows empieza en 128, Linux en 64.
10	**Protocolo**	8	Qué protocolo va **encapsulado** dentro del campo de datos.	`06` = TCP · `11` = UDP · `01` = ICMP · `2E` = IPv6 dentro (túnel) · `08` = un protocolo de GRE.
11	**Checksum de cabecera**	16	**Suma del complemento a uno** de todos los bytes de la cabecera. **Sólo de la cabecera**: si cambia un dato, el TCP/UDP recalcula el suyo.	`5958` (el cálculo completo está en §2.3).
12	**Dirección origen**	32	De quién es el datagrama. **Cambia en cada salto si hay NAT.**	`192.168.10.55` = `0xC0A80A37` = `11000000.10101000.00001010.00110111`

#	Campo	Bits	Qué significa	Ejemplo numérico
13	**Dirección destino**	32	A quién va el datagrama. **No cambia** al atravesar la red (salvo NAT).	`219.41.31.21` = `0xDB291F15` = `11011011.00101001.00011111.00010101`

El mismo datagrama, escrito en hexadecimal (que es como lo vas a ver en un `wireshark`):

```
45 00 0054 1C46 4000 40 06 5958 C0A8 0A37 DB29 1F15
^^ ^^ ____ ____ ____ ^^ ^^ ____ ____ ____ ____
|  |  |    |    |  |  |  |    |    |    |
|  |  |    |    |  |  |  |    |    |    └─ destino 219.41.31.21
|  |  |    |    |  |  |  |    |    └────── origen   192.168.10.55
|  |  |    |    |  |  |  |    └─────────── checksum de cabecera = 0x5958
|  |  |    |    |  |  |  └──────────────── protocolo = 06 = TCP
|  |  |    |    |  |  └─────────────────── TTL = 0x40 = 64
|  |  |    |    |  └────────────────────── Flags/Offset = 0x4000 → DF=1
|  |  |    |    └───────────────────────── Identificación = 0x1C46
|  |  |    └────────────────────────────── Longitud total = 0x0054 = 84
|  |  └─────────────────────────────────── DSCP=0, ECN=0  (byte TOS = 0x00)
|  └────────────────────────────────────── IHL = 5
└───────────────────────────────────────── Versión = 4
```

1.3 El campo IHL y por qué la cabecera es de tamaño variable

La cabecera **no siempre son 20 bytes**. El campo **IHL** dice cuántos bloques de 4 bytes ocupa, y si hay **opciones**, ocupa más.

```
IHL = 5  →  5 × 4 = 20 bytes   (cabecera mínima, SIN opciones)
IHL = 6  →  6 × 4 = 24 bytes   (4 bytes = UNA opción de 4 bytes)
IHL = 15 → 15 × 4 = 60 bytes   (máximo: 40 bytes de opciones)
```

Opciones. Cada opción tiene el formato `1 byte tipo | 1 byte longitud | datos`. Los tipos históricos (de `RFC 791`) incluyen:

Opción	Tipo	Qué hace
RR (*Record Route*)	7	**Freeze routing**: obliga a que el paquete pase por una lista de routers que escribe la fuente. Obsoleta, pero es la que da nombre a la opción.
SSM (*Source Route*)	137	El campo de destino se usa como **lista de routers por los que tiene que pasar**. Obsoleta y peligrosa.
Timestamp	68	Cada router que pasa mete la hora. Obsoleta.
EOL (*End of Option List*)	0	Marca de fin de las opciones.
Pad	1	Relleno para alinear a 4 bytes.

> ⚠️ **Nota para 2026:** las opciones de IP (source route, record route, timestamp, y las িনা loosely *Loose Source and Record Route* de traceroute) están **obsoletec. Se parsean pero no se usan**, y muchos routers y middleboxes las descartan o rechazan por motivos de seguridad. **En la práctica, IHL es siempre 5.** Decir

> "IHL = 5, 20 bytes, sin opciones" es la respuesta correcta en cualquier examen salvo que pregunten explícitamente por fragmentación o por options de seguridad.

1.4 La máscara de subred, aplicada bit a bit

La máscara es un número de 32 bits con **unos al principio y ceros al final**. El prefijo `/n` es la **cantidad de unos**. La operación que define todo es un **AND** bit a bit entre la dirección y la máscara:

```
dirección AND máscara = DIRECCIÓN DE RED
```

Ejemplo completo: `192.168.10.55` con máscara `/26`.

Paso 1 — Escribo la dirección en binario, octeto por octeto.

```
192 = 11000000    (128 + 64)
168 = 10101000    (128 + 32 + 8)
 10 = 00001010    (8 + 2)
 55 = 00110111    (32 + 16 + 4 + 2 + 1)

192.168.10.55 = 11000000.10101000.00001010.00110111
```

Paso 2 — Escribo la máscara `/26` en binario y en decimal.

```
/26  →  26 unos seguidos de 6 ceros
     →  11111111.11111111.11111111.11000000
     →  255.255.255.192      (128 + 64 = 192)
```

Paso 3 — Hago el AND bit a bit.

```
  dirección   11000000 . 10101000 . 00001010 . 00110111
  máscara     11111111 . 11111111 . 11111111 . 11000000
  ──────────────────────────────────────────────────────
  AND         11000000 . 10101000 . 00001010 . 00000000
              192      . 168      . 10       . 0
──────────────────────────────────────────────────────────

  AND        1&1 1&1 1&1 1&1 1&1 1&1 1&1 1&1
           = 1   1   1   1   1   1   1   1  → 11111111 = 255 ✓

  AND del 3er octeto: 00001010 AND 11111111 = 00001010 = 10 ✓

  AND del 4to octeto, que es donde pasa la separable:
    dirección  0 0 1 1 0 1 1 1
    máscara    1 1 0 0 0 0 0 0
    ──────────────────────────
    AND        0 0 0 0 0 0 0 0  = 0        ← los 2 unos de la máscara
                                             se apagaron
```

Paso 4 — Interpreto el resultado.

```
DIRECCIÓN DE RED = 192.168.10.0        y el prefijo es /26

Los bits de HOST de la dirección eran los 6 últimos:
  00110111  →  bits de red (los 2 primeros) = 00
               bits de host (los 6 últimos)  = 110111 = 55

Como 55 < 64 (= 2⁶), esta dirección cae en el PRIMER /26 de la /24.
```

Paso 5 — Calculo el tamaño del bloque y los otros tres valores.

```
Tamaño del bloque /26 = 2⁶ = 64 direcciones

DIRECCIÓN DE RED      = 192.168.10.0     (el AND de arriba)
RANGO DE HOSTS        = 192.168.10.1  a  192.168.10.62
                          (red + 1   hasta  red + 64 − 2)
BROADCAST             = 192.168.10.63    (red + 64 − 1)
NÚMERO DE HOSTS ÚTILES = 64 − 2 = 62     (2⁶ − 2)

Chequeo: 1 (red) + 62 (hosts) + 1 (broadcast) = 64 ✓
Chequeo: .0 + .63 = 64 direcciones numeradas, la última es 63 ✓
Chequeo: .55 está entre .1 y .62  ✓  (y es el host número 55)
```

Paso 6 — Dibujo el /24 partido en cuatro /26 para verlo.

```
192.168.10.0/24                    (256 direcciones)

.0                   .63   .64                  .127  .128          .191  .192      .255
├─────────┬──────────┬─────┬──────────┬──────────┬─────┬──────────┬──────┬──────┬──────┤
│ .0/26   │ .64/26   │ .128/26 │ .192/26 │ ↑ ESTA  │          │      │      │      │
│ red .0  │ red .64  │ red .128 │ red .192 │ 192.168.10.55/26 │      │      │      │
│ brdc .63 │ brdc .127│ brdc .191│ brdc .255│  = .0/26  │      │      │      │      │
└─────────┴──────────┴─────┴──────────┴──────────┴─────┴──────────┴──────┴──────┴──────┘
← 64 dir →  ← 64 →    ← 64 →     ← 64 →
```

Y el error conceptual más común: el **broadcast de la subred no es** `192.168.10.255`, es `192.168.10.63`. El broadcast depende de **la máscara**, no de la dirección. Con la `/24` el broadcast era `.255`; con la `/26` es `.63`. Esto se desarrolla entero en la Clase 27.

1.5 Direcciones claseful (A, B, C, D, E) y por qué quedaron obsoletas

Hasta 1993, una dirección IPv4 se leía **con la máscara fija según el primer bit**, sin necesidad de que nadie configurara nada. La tabla que hay que saber de memoria:

Clase	Primer octeto en binario	Rango	Prefijo	Redes	Hosts por red	Uso
A	`0xxxxxxx`	1 – 126	**/8**	126	**16 777 214**	redes enormes (gobiernos, Multicast)
B	`10xxxxxx`	128 – 191	**/16**	16 384	**65 534**	redes grandes (universidades, empresas)
C	`110xxxxx`	192 – 223	**/24**	2 097 152	**254**	redes chicas (empresas, departamentos)
D	`1110xxxx`	224 – 239	**/4**	—	—	**multicast**
E	`1111xxxx`	240 – 255	**/4**	—	—	**reservada** para experimentación futura

Cómo se lee la clase de una dirección, sin ninguna máscara: mirá el primer octeto en binario y fijate los primeros bits. `192 = 11000000` → empieza con `110` → **clase C** → máscara `/24`. `172 = 10101100` → empieza con `10` → **clase B** → `/16`. `10 = 00001010` → empieza con `0` → **clase A** → `/8`. `224 = 11100000` → `1110` → **clase D**.

Por qué quedaron obsoletas — el cálculo que lo mata. El problema era que **las clases no dejaban espacio intermedio**: o te sobraban direcciones (**clase B**) o te faltaban (**clase C**).

```
Una empresa con 300 hosts en 1993:

Necesita 300 direcciones.
Una clase C tiene 254   →  NO LE ALCANZA (300 > 254)
Una clase B tiene 65 534  →  LE SOBRA INFUMAMENTE

Con la clase B:
    direcciones totales          65 536
    utilizables (2¹⁶ − 2)        65 534
    usadas por la empresa             300
    ──────────────────────────────────────
    DESPERDICIADAS                65 234

O sea: el 99,5 % del bloque queda inútil. Y el agregado es
65 234, que es del orden de las decenas de miles (mucho más
que 22 000): por cada empresa con 300 hosts "compra" 65 000
direcciones que jamás va a usar.
```

El otro problema era simétrico: una biblioteca con **20** computers necesitaba una clase C (254) y desperdiciaba 234. Y el mercado lo notaba: **las clases B se agotaron primero** (las asignación del mundo entero eran sólo 16 384), mientras que las clase C sobraban. **Cualquiera podía pedir una clase B y agotarlas.** Ese fue el problema económico que forzó el cambio.

La historia en una línea de tiempo

Año	Hecho
1981	`RFC 791` define IPv4 **con clases**: A, B, C, más D (multicast) y E (reservada).
1984	`RFC 950` ("Internet Subnetting") introduce el **subnetting dentro de una clase**: una clase B prestada se divide en subredes. Esto obligaba a que **todos los routers de la red** usaran la misma máscara.
1993	**Fin de la asignación por clases.** Se implementa **CIDR** y la IANA empieza a asignar por **prefijo de cualquier longitud** (`RFC 4632`). Se introduce además el **routing CIDR** en los routers de frontera.
1994	El problema de las clases B se había resuelto **con subnetting**, que era una solución obligatoria y rígida; con CIDR la longitud del prefijo pasa a ser **una decisión local de cada enlace**.
1996	`RFC 1918` (verificar) define las **direcciones privadas** `10/8` , `172.16/12` y `192.168/16` : reutilizables internamente, **no enrutables en Internet**.
2011	**APNIC se agota**: es el último RIR que asigna direcciones IPv4 de clase C por la vía clásica.
2019	**RIPE NCC hace su última asignación** `/22` (25 de noviembre): Europa agota IPv4. ARIN ya se había agotado en 2015, LACNIC en 2020 y AfriNIC en 2017.
2026	**IPv4 es un recurso scarcity_planificado:** CGNAT en todos los ISPs y despliegue de IPv6 por encima del 50 % en los usuarios de Google.

El `/n` como atajo y el "número mágico". Con CIDR, cualquier prefijo es legal, así que los números que importan son los que van del 32 al 0. La forma rápida de WORK es mirar el **"número mágico"**: el **número de hosts** de cada prefijo, que es `2^(32−n) − 2`. La tabla completa es el Anexo de la `RFC 1878` **(*Requirements for IP Address Reservation*, 1996)**, y es la tabla que hay que tener a la vista cuando estás haciendo subnetting en un examen de papel:

```
/n    hosts      /n    hosts      /n    hosts      /n    hosts
────  ───────    ────  ───────    ────  ───────    ────  ───────
/32       1      /24     254      /17    32 766    /10    1 048 574
/31       0 (*)  /23     510      /16    65 534    /9     2 097 150
/30       2      /22   1 022      /15   131 070    /8     4 194 302
/29       6      /21   2 046      /14   262 142    /7     8 388 606
/28      14      /20   4 094      /13   524 286    /6    16 777 214
/27      30      /19   8 190      /12   1 048 574  /5    33 554 430
/26      62      /18  16 382      /11   2 097 150  /4    67 108 862
/25     126      ...    ...       ...    ...       ...    ...      ...

(*) el /31 tiene 2 direcciones y 0 hosts "útiles" en el sentido
    clásico; RFC 3021 (verificar) lo legitima para enlaces
    punto a punto.  /32 = una sola dirección, el loopback.

Para leerlo al revés (esto es el atajo del subnetting):
  "necesito 100 hosts"  →  el menor número ≥ 100 de la tabla es 126 → /25
  "necesito 200 hosts"  →  254 → /24
  "necesito 50 hosts"   →  62 → /26
  "necesito 20 hosts"   →  30 → /27
  "necesito 13 hosts"   →  14 → /28
```

1.6 CIDR: por qué existe

CIDR = Classless Inter-Domain Routing (`RFC 4632` , 2006). Tres cambios, y los tres importan:

1. **La longitud del prefijo es libre.** Ya no hay clases: `/8`, `/13`, `/27`, cualquier valor entre 0 y 32. Se elimina por completo la frontera entre clases.
2. **La máscara es local, no global.** Antes del subnetting, todos los routers de la red tenían que usar **la misma** máscara. Con CIDR, cada enlace puede usar la que necesite, y un router puede tener **máscaras distintas en cada interfaz** (es exactamente lo que se ve en las tablas de enrutamiento: `192.168.1.0/24` en una línea y `220.10.10.28/30` en otra, en el mismo router).
3. **La sumarización (agregación) vuelve a ser posible.** Un router que se ocupa de `192.168.0.0/16` puede anunciar **un solo prefijo** en vez de 256. Eso reduce las tablas de enrutamiento de toda la Internet deBackbone. Sin CIDR, la tabla de un router de núcleo habría tenido que tener una entrada por cada red de cada organización del mundo.

La consecuencia concreta en un examen. Cuando el enunciado dice "asigne una red para 300 hosts", **la respuesta de hoy no es "clase B"**: es "el menor prefijo que dé 300" → `/23` (510 hosts). Y cuando dice "asigne una red y quiere dejar espacio para crecer", la respuesta es un bloque `/22` o `/21` **y después subnetting** (Clase 27 y 28). **Nadie dice "clase B" en 2026.**

1.7 Direcciones especiales

Rango	Nombre	Qué es y para qué está
`0.0.0.0/8`	"Cero" / *"this host on this network"*	En un **servidor** significa *"escuchá en todas las interfaces"* (`bind(0.0.0.0)`). En una **tabla de enrutamiento** es la **ruta por defecto** (`0.0.0.0/0` , todos los destinos). Como **dirección de origen**, significa *"no tengo dirección"* — es lo que manda un host antes de tener IP por DHCP (`DHCPDISCOVER`). **En ninguna de las tres es una dirección de un host.**
`10.0.0.0/8` `172.16.0.0/12` `192.168.0.0/16`	**Privadas** (RFC `1918` , verificar)	**Se pueden repetir en todas las redes del mundo** porque **no se enrutan en Internet**: ningún router de backbone tiene una ruta hacia ellas. Existen porque **IPv4 se agotó** y hace falta reutilizar el mismo espacio internamente. Se usan detrás de un **NAT** (Clase 31). **10/8** = 16 777 214 direcciones · **172.16/12** = 1 048 574 · **192.168/16** = 65 534.
`127.0.0.0/8`	**Loopback**	El **propio host**. Todo lo que se mande a `127.x.x.x` lo entrega el kernel localmente, sin salir a la red. `127.0.0.1` se llama *localhost*. Sirve para probar un servicio en la misma máquina sin tocar la red.
`169.254.0.0/16`	**Link-local** (*APIPA*)	**Se autoasigna el host cuando el DHCP falla.** El host elige una dirección al azar en ese rango y se verifica con ARP (DAD). Sirve para comunicarse **en el enlace local**, pero **no se enruta**: nunca pasa de la LAN. Es el rango de las cámaras IP mal configuradas y de los enlaces punto a punto sin DHCP. La definición formal está en la `RFC 3927` (verificar).
`224.0.0.0/4`	**Multicast** (clase D)	Grupo de destinos. **La dirección es del grupo, no del host.** `224.0.0.0` a `224.0.0.255` es *link-local* (IGMP, `224.0.0.5` y `.6` son los multicast de OSPF); `239.0.0.0/8` es *administratively scoped* (globales, como los de Cisco Discovery Protocol). El prefijo `/4` es lo que dice la clase D, **no** una máscara que se configure.
`240.0.0.0/4`	Reservada (clase E)	Reservada para uso futuro. En la práctica el último octeto de `255` se usa para **broadcast**, lo que pisa parcialmente el rango.
`255.255.255.255`	**Broadcast limitado**	Broadcast **de la LAN local, y sólo de ella**. Los routers **no lo reenvían**, jamás. Es el `destino` del ARP request, del DHCP Discover y de los announcing gratuitous ARP.
`100.64.0.0/10`	*Shared Address Space* (CGNAT)	El rango que usan los operadores para la **traducción de direcciones deCarrier-Grade NAT** (`RFC 6598` , verificar). No es privado ni público: es "compartido", por eso sólo se enruta dentro del operador.

Por qué existen las privadas, en una frase. Porque el espacio de IPv4 se **agotó** y porque la topología de Internet es **jerárquica**: no todo lo que hay en el mundo tiene que ser alcanzable desde todo el mundo. Si tu notebook tiene `192.168.1.5` y el servidor del otro lado del mundo también tiene `192.168.1.5`, no hay ningún conflicto **porque ningún router de Internet enruta ese rango**. La reutilización es lo que nos salvó de haber agotado IPv4 en 1994.

1.8 ⚠️ Actualización a 2026

Sigue igual, de memoria:

- El **formato de la cabecera IPv4**: 20 bytes mínimos, IHL × 4, los mismos 13 campos, el mismo orden. **No cambió desde 1981** y no va a cambiar.

- **Máscara = unos al principio, ceros al final**; dirección de red = AND; broadcast = red + tamaño − 1.
- Las **clases A–E** (historia), el **/n** y el **número mágico**.
- Los **rangos especiales**: `0/8`, `10/8`, `172.16/12`, `127/8`, `169.254/16`, `192.168/16`, `224/4`, `255.255.255.255`.

Cambió:

- **La claseful es historia, y hace 30 años que lo es.** Desde 1993 la asignación es por prefijo (`RFC 4632`) y **las clases no significan nada en el mundo real**: el `192.168.1.0/24` de tu casa no es "una clase C" sino un `/24`. Se enseñan porque (a) aparecen en los exámenes viejos, (b) explican por qué existe CIDR, y (c) la máscara `/24` **se parece** a una clase C por casualidad histórica. ⇒ **En un examen de hoy, nunca respondas "clase C" como solución; respondá "/24".** Si el enunciado dice "clase C", traducilo a `/24` y seguí.
- **IPv4 se agotó, de verdad y por etapas.** APNIC se agotó en abril de 2011, ARIN en septiembre de 2015, AfriNIC en 2017, **RIPE NCC hizo su última asignación `/22` el 25 de noviembre de 2019**, y LACNIC en 2020. Hoy RIPE mantiene una *waiting list* para un único `/24` recuperado. ⇒ **Consejo de examen:** si te preguntan "¿por qué existen las direcciones privadas?", la respuesta es literalmente "porque IPv4 se agotó", y **la cifra para mencionar es la del 25-nov-2019 de RIPE**.
- **CGNAT es el nuevo default de la capa 3 de acceso.** `100.64.0.0/10` es el rango que el operador traduce, y eso rompe conectividad entrante, complica la señalización SIP/IMS y rompe IPsec y peer-to-peer. **Es la respuesta moderna a "mi dirección es privada"**: antes lala traducción era opcional (NAT en la empresa), ahora es la norma en el acceso.
- **IPv6 pasó la mitad.** El **28 de marzo de 2026** el acceso a Google por IPv6 superó el **50 %** (50,10 %) por primera vez. Pero es desigual: Francia e India por encima del 70 %, China por debajo del 5 %. ⇒ **Consecuencia para el examen:** si te preguntan "¿IPv6 reemplazó a IPv4?", la respuesta es "a medias y por país"; lo que se mide es distinto en cada fuente, así que **declará qué estás midiendo**.
- **La cabecera IPv6 es fija: 40 bytes, sin IHL, sin opciones, sin fragmentación en los routers.** Los routers IPv6 **no** fragmentan; fragmenta el origen. El concepto de "opciones de cabecera IP" no existe en IPv6: lo que hay son **cabeceras de extensión** encadenadas por el campo *Next Header*, que **no** las ve ningún router intermedio salvo la de *Hop-by-Hop* (`RFC 8200`). ⇒ El campo **IHL y todo el mecanismo de opciones de IPv4 es `legacy`** y sólo se estudia por historia.
- **Seguridad.** El **source routing de IPv4 sigue siendo un vector de ataque** y muchos middleboxes lo bloquean. `RFC 7126` (verificar) habla del problema de la cabecera IPv4 con IHL; no lo uses si no lo verificaste.

Consecuencia para el examen: la cabecera IPv4 se sigue preguntando **exactamente igual**. Lo que hay que sumar es una línea: "*IPv4 está agotado desde 2019 (RIPE), CGNAT es el default del acceso, y el /n con el número mágico reemplazó a las clases; las clases sólo se explican como historia.*"

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Binario y máscara, de punta a punta

Datos. La dirección `172.31.199.14` con máscara `/22` (máscara de red y de la interfaz del router).

Resolución paso a paso.

Paso 1 — La dirección a binario.

```
172 = 10101100    128 + 32 + 8 + 4
 31 = 00011111    16 + 8 + 4 + 2 + 1
199 = 11000111    128 + 64 + 4 + 2 + 1
 14 = 00001110    8 + 4 + 2
```

Paso 2 — La máscara `/22`.

```
/22  →  22 unos + 10 ceros
     →  11111111 . 11111111 . 11111100 . 00000000
     →   255      . 255      . 252      . 0        = 255.255.252.0

Chequeo: 255 + 255 + 252 + 0 = 762. Y 3×255 = 765, así que
         765 − 762 = 3  →  hay 3 ceros en los dos últimos octetos.
         8 + 8 + 6 + 0 = 22  ✓  (el /22 tiene 22 unos)
```

Paso 3 — El AND, octeto por octeto.

```
octeto      dirección   máscara     AND
──────      ─────────   ───────     ───
  1°         10101100    11111111    10101100  = 172
  2°         00011111    11111111    00011111  =  31
  3°         11000111    11111100    11000100  = 196   ← acá cambia
  4°         00001110    00000000    00000000  =   0   ← y acá se apaga

DIRECCIÓN DE RED = 172.31.196.0/22

Ojo con el 3° octeto: 199 AND 252 = 196, NO 199.
Los 6 unos de la máscara "comen" los 2 bits más bajos de 199.
199 = 11000111
      11111100
AND = 11000100 = 196
```

Paso 4 — Tamaño del bloque, hosts, broadcast.

```
Tamaño = 2^(32−22) = 2¹° = 1024 direcciones
Hosts útiles = 1024 − 2 = 1022

RED      = 172.31.196.0
PRIMER   = 172.31.196.1        (red + 1)
ÚLTIMO   = 172.31.199.254      (red + 1024 − 2)
BROADCAST= 172.31.199.255      (red + 1024 − 1)

Chequeo: 172.31.199.14 está entre .196.1 y .199.254  ✓
¿Es broadcast? No (sólo lo es .199.255)  ✓
¿Es la red? No (la red es .196.0)  ✓
```

Paso 5 — El detalle fino: el número de host.

```
Bits de host = los 10 últimos (todo el 4º octeto + 2 del 3º)

172.31.199.14
           3º octeto: 199 = 11000111 → los 2 bits de host son "11" = 3
           4º octeto:  14 = 00001110 = 14

número de host = 3 × 256 + 14 = 768 + 14 = 782
(2 bits de host del 3er octeto ⇒ 2² = 4 valores posibles, de 0 a 3)
```

Interpretación: esta dirección está en la **cuarta y última** `/22` de la `/20` `172.31.192.0/20`. Los bloques `/22` de esa `/20` son `.192.0`, `.196.0`, `.200.0` y `.204.0`. Si en un ejercicio te piden "dividir `172.31.192.0/20` en cuatro `/22`", la respuesta es exactamente esa cuaterna.

2.2 Ejercicio 2 — Clase, máscara por defecto y capacidad

Datos. Determinar la clase y la máscara por defecto de estas cinco direcciones, y cuántas direcciones hosts útiles admite cada red.

Dirección	Clase	¿Claseful o CIDR?	Máscara por defecto	Hosts útiles	Nº de redes de ese tamaño
`10.4.3.2`	**A** (`00001010` , empieza con `0`)	claseful	`/8` = 255.0.0.0	$2^{24} - 2$ = **16 777 214**	$2^{7} - 2$ = **126**
`172.16.40.9`	**B** (`10101100` , empieza con `10`)	claseful	`/16` = 255.255.0.0	$2^{16} - 2$ = **65 534**	2^{14} = **16 384**
`192.168.1.1`	**C** (`11000000` , empieza con `110`)	claseful	`/24` = 255.255.255.0	$2^{8} - 2$ = **254**	2^{21} = **2 097 152**
`224.0.0.251`	**D** (`11100000` , empieza con `1110`)	multicast	`/4` (no se configura)	—	—
`240.1.2.3`	**E** (`11110000` , empieza con `1111`)	reservada	`/4` (no se configura)	—	—

La cuenta de cada fila, a la vista:

```
A  →  /8   → 32−8 = 24 bits de host → 2²⁴ − 2 = 16 777 214
B  →  /16  → 32−16 = 16 bits de host → 2¹⁶ − 2 = 65 534
C  →  /24  → 32−24 = 8 bits de host  → 2⁸  − 2 = 254
D  →  multicast: la dirección es del GRUPO, no de un host
E  →  reservada: no se asigna
```

Las dos observaciones que hay que hacer en un examen:

1. **La clase se lee del primer octeto, sin máscara.** Es un residuo histórico: los routers antigos lo usaban para deducir la máscara. **Hoy es inútil**: la clase de `10.4.3.2` y de `10.0.0.1` es la misma (A), pero un `/8` y un `/16` pueden tener la misma clase.
2. **Diez es clase A, y sin embargo `10.0.0.0/8` es una dirección privada.** Las privadas no se definieron por la clase sino por rangos fijos dentro de ella. Esta confusión es la que hace que los alumnos digan "clase A

privada": lo correcto es "rango `10/8`, que está dentro de la clase A por su primer bit, pero que es privado por definición del RFC, no por su clase".

2.3 Ejercicio 3 — El checksum de la cabecera IPv4, cálculo completo

Datos. Un datagrama con estos campos:

```
Versión = 4        IHL = 5         DSCP = 0, ECN = 0
Longitud total     = 60 bytes (0x003C)
Identificación     = 0x1C46
Flags/Offset       = 0x4000   (DF = 1, MF = 0, offset = 0)
TTL                = 64 (0x40)
Protocolo          = 6 (TCP)
Checksum           = 0x0000   ← hay que calcularlo
Origen             = 192.168.10.55  (0xC0A80A37)
Destino            = 219.41.31.21  (0xDB291F15)
```

Resolución: se suman las mitades de 16 bits, se sigue el "acarreo", y se toma el complemento.

```
La cabecera son 10 palabras de 16 bits (20 bytes ÷ 2):

 1)  4500     Versión+IHL = 0x45,   DSCP+ECN = 0x00
 2)  003C     Longitud total = 60
 3)  1C46     Identificación
 4)  4000     Flags = 100, offset = 0
 5)  4006     TTL = 0x40, Protocolo = 0x06
 6)  0000     Checksum (por ahora cero)
 7)  C0A8     Origen, 1ª mitad
 8)  0A37     Origen, 2ª mitad   (0x0A = 10, 0x37 = 55)
 9)  DB29     Destino, 1ª mitad  (0xDB = 219, 0x29 = 41)
10)  1F15     Destino, 2ª mitad  (0x1F = 31, 0x15 = 21)

SUMA (con acarreo: si da 5 dígitos hex, se suma el acarreo y se
deja el resto):

 4500 + 003C = 453C
 453C + 1C46 = 6182
 6182 + 4000 = A182
 A182 + 4006 = E188
 E188 + 0000 = E188
 E188 + C0A8 = 1A230   →  acarreo 1  →  A231        ← ¡primer acarreo!
 A231 + 0A37 = AC68
 AC68 + DB29 = 17768   →  acarreo 1  →  8792       ← ¡segundo acarreo!
 8792 + 1F15 = A6A7

CHECKSUM = complemento a uno de A6A7
         = NOT(A6A7) = 5958

La cabecera completa queda:
   45 00 00 3C 1C46 4000 40 06 59 58 C0A8 0A37 DB29 1F15
```

Verificación (siempre hay que hacerla, y en el examen suma puntos):

```
Volvemos a sumar TODO, esta vez con el checksum ya puesto:

 (4500 + 003C + 1C46 + 4000 + 4006 + 0000 + C0A8 + 0A37 + DB29 + 1F15)
                                                    =  A6A7
 A6A7 + 5958 =  FFFF

Si la suma da 0xFFFF, el checksum es correcto.
Esa es la definición de "complemento a uno": el checksum es el
valor que hace que la verificación dé todo 1s.
```

Interpretación: el checksum **cubre sólo la cabecera**, y por eso **no se recalcula en los routers** (que sólo cambian TTL, y ese campo tiene su propio mecanismo: el checksum se ajusta en incremental). El del **TCP y el del UDP** cubren cabecera **y** datos, con una pseudo-cabecera de IP que incluye las direcciones; por eso un NAT que cambia una IP **tiene que recalcular el checksum del transporte**.

2.4 Ejercicio 4 — ¿Por qué DHCP asigna `169.254.x.x` y no `0.0.0.0`?

Enunciado. Cuando un host busca un servidor DHCP y no encuentra ninguno, se autoasigna una dirección del rango `169.254.0.0/16` (APIPA). ¿Por qué **no** puede autoasignarse `0.0.0.0`? Y si puede usar `169.254.x.x`, ¿por qué no se usa `192.168.1.5` "porque total es mi red"?

Resolución — la regla general primero: los rangos especiales están definidos por lo que hacen con ellos.

Rango	Semántica	¿Se puede autoasignar un host?
`0.0.0.0/8`	"*no tengo dirección*" (fuente) · "*escuchá en todas*" (servidor) · "*por cualquier lado*" (ruta por defecto)	**No.** Significa **inexistencia** de dirección. Si un host se autoasignara `0.0.0.0`, los demás no sabrían ni si siquiera que hay alguien, y no podría arrancar ninguna conexión: **no hay dirección a la que responder.**
`127.0.0.0/8`	"*yo mismo*"	**No.** Está cableada al loopback del kernel, así que `127.x.x.x` **nunca sale a la red**: sería inútil para comunicarse con los demás.
`169.254.0.0/16`	"*este enlace local*"	**Sí.** Es un rango **asignado justamente para la autoconfiguración**: el host elige una dirección al azar de 16 bits, manda un **ARP request** con esa dirección (**DAD**, *Duplicate Address Detection*, ver `RFC 3927`, verificar) y si nadie contesta, se la queda.
`10/8`, `172.16/12`, `192.168/16`	"*privada*"	**No se puede adivinar.** Una dirección privada es sólo válida **detrás de un router que hace NAT**, y ese router tiene que tener una **tabla de traducción** con una entrada para esa dirección. Si el host se autoasigna `192.168.1.5` sin NAT, **el router no tiene entrada y descarta el paquete de salida.**
`224.0.0.0/4`	"*grupo*"	**No.** La dirección es del **grupo**, no del emisor: la fuente también necesita una dirección unicast.

Por qué `169.254/16` sí funciona, en tres pasos.

```
t0   El host arranca. No tiene IP. Configura su IPv4 como
     0.0.0.0 (significa "ninguna") y la máscara a 0.0.0.0.

t1   Intenta DHCP. No hay servidor → timeout.

t2   El host se AUTOASIGNA una dirección al azar de 169.254/16
     (en Windows, además elige un sufijo de fecha y hora).
     Toma la máscara /16, que viene de la definición del rango:
         169.254.0.0  169.254.255.255
         ↑               ↑
         169 = 10101001  se fija el /16
         254 = 11111110

t3   Manda un ARP request por esa dirección para hacer DAD
     ("¿hay alguien con esta dirección?"). Si nadie contesta
     en un par de segundos, se la queda.

t4   A partir de ahí puede hablar con los demás del ENLACE, y
     sólo del enlace: 169.254/16 es link-local, los routers
     NO lo enrutan. Sirve para compartir archivos dentro de la red local, y nada más.
```

La respuesta a "¿por qué no `192.168.1.5`?", que es la parte interesante: porque `192.168/16` **existe sólo dentro de una red que alguien administrationó**. La dirección privada es una **convención entre el administrador y el NAT**, no una dirección "del mundo". Si el host se autoasignara una sin NAT detrás, el paquete saldría del host con destino/source `192.168.1.5`, **el router de salida lo descartaría** (no tiene ruta ni entrada de traducción) y el host **quedaría sin comunicación**, que es exactamente el problema que se intenta resolver. `169.254/16` **está diseñado para ser usable sin ninguna infraestructura detrás**; `192.168/16` no.

> En clase: si te preguntan "qué hace el host si no tiene DHCP", la respuesta completa es: `0.0.0.0` → espera → se autoasigna `169.254.x.x` con **DAD** → habla **sólo en el enlace local** → si tiene un APIPA válido pero no gateway, la aplicación que necesita Internet **falla aunque el host "tenga IP"**. Ese último punto es la trampa de examen.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-03, Problema 1 (30 puntos) — tres subredes /24, R1 y R2

> **Fuente:** Examen `eirc1003` / `sirc1003` — 24 de febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1003.pdf **Texto del examen:**
>
> > Sea la red compuesta de tres subredes interconectadas por los routers R1 y R2 presentada en la siguiente figura. En la misma se indican los nodos conectados en cada subred así como la dirección IP de cada interfaz. Para su direccionamiento se asignó una red /24 a cada una de las subredes como se presenta en la figura, y se supone la existencia de 100 hosts en cada subred. El enrutamiento es estático. **a)** i. Indique las tablas de enrutamiento de los routers R1 y R2 de forma que permita conectividad extremo a extremo entre todos los equipos de la red. ii. Indique las tablas de enrutamiento para los hosts de cada subred. **b)** Se sustituye el router R2 por un switch, al que se conectan todos los equipos de las subredes 2 y 3. Para lograr un nuevo esquema de numeración operativo, se desea modificar la menor cantidad de direcciones IP posibles. i. Justifique la necesidad de renumerar alguna red para lograr una solución al problema. ii. Represente gráficamente su solución... iii. Indique la tabla de enrutamiento del router R1...

Solución oficial (literal):

> **a-i)** R1: `192.168.0.0/24 → eth0`, `192.168.1.0/24 → eth1`, `192.168.2.0/24 → next hop 192.168.1.2, eth1`. R2: `192.168.2.0/24 → eth1`, `192.168.1.0/24 → eth0`, `192.168.0.0/24 → next hop 192.168.1.1, eth0`. **a-ii)** Hosts de la subred 1: `192.168.0.0/24 → eth0`, `0.0.0.0 → 192.168.0.1`. Hosts subred 2: `192.168.1.0/24`, `0.0.0.0 → 192.168.1.1`. Hosts subred 3: `192.168.2.0/24`, `0.0.0.0 → 192.168.2.1`. **b-i)** *"Las direcciones IP consideradas para las subredes 1, 2 y 3 corresponden a redes contiguas. Para agrupar direcciones se debe modificar la máscara de red, disminuyendo la cantidad de bits de la máscara [...] El problema que surge es que para incluir en una subred las subredes 192.168.1.0/24 y 192.168.2.0/24 se necesita una máscara /22 que incluye la red 192.168.0.0/24. Por lo tanto la solución es renumerar tal que las subredes sean diferenciables por la máscara."* **b-ii)** Dos soluciones: renumerar la subred 1 como `192.168.4.0/24` (quedando subredes 2 y 3 como `192.168.0.0/22`), **o** renumerar la subred 2 como `192.168.3.0/23` (quedando `192.168.0.0/24` y `192.168.2.0/23`). *"De las soluciones propuestas la primera es la ideal dado que modifica una IP menos."* **b-iii)** R1: `192.168.4.0/24 → eth0`, `192.168.0.0/22 → eth1`.

a-i) Tablas de enrutamiento de R1 y de R2

Suposición que hago y justifico: por el enunciado, cada subred es una `/24` y hay 100 hosts en cada una. Las direcciones de gateway (`192.168.0.1`, `192.168.1.1`, `192.168.1.2`, `192.168.2.1`) son las que da el enunciado en la figura.

R1 — tiene una interfaz en la subred 1 (`eth0`) y otra en la subred 2 (`eth1`):

```
Configuración R1
Red destino / Máscara      Gateway (next hop)   Interfaz
─────────────────────────  ──────────────────   ─────────
192.168.0.0 / 255.255.255.0   –                 eth0
192.168.1.0 / 255.255.255.0   –                 eth1
192.168.2.0 / 255.255.255.0   192.168.1.2       eth1
```

¿Por qué la tercera línea lleva next hop `192.168.1.2` y no `192.168.2.1`? Porque `192.168.1.2` es la dirección **de la interfaz de R2 dentro de la red que R1 ya tiene conectada**. R1 no tiene por qué conocer la dirección de un host o de otra red: le alcanza con el vecino. **Y esa es la razón de existir del campo *Gateway* de una tabla de reenvío** (Clase 24, §1.1): el router necesita un next hop **alcanzable directamente**.

R2 — una interfaz en la subred 2 (`eth0`) y otra en la subred 3 (`eth1`):

```
Configuración R2
Red destino / Máscara      Gateway (next hop)   Interfaz
─────────────────────────  ──────────────────   ─────────
192.168.2.0 / 255.255.255.0   –                 eth1
192.168.1.0 / 255.255.255.0   –                 eth0
192.168.0.0 / 255.255.255.0   192.168.1.1       eth0
```

Interpretación: **la fila "directamente conectada" no tiene next hop**, porque no hay nadie a quien saltar: el router entrega el paquete directo a la interfaz. Y la fila con next hop **también lleva interfaz**, porque el next hop dice *a qué router* y la interfaz dice *por qué cable*.

a-ii) Tablas de enrutamiento de los hosts

```
Hosts subred 1                    Hosts subred 2
Red destino / Máscara   Gateway   Red destino / Máscara   Gateway
192.168.0.0 /24        –         192.168.1.0 /24        –
0.0.0.0    /0           192.168.0.1  0.0.0.0  /0       192.168.1.1

Hosts subred 3
Red destino / Máscara   Gateway
192.168.2.0 /24        –
0.0.0.0    /0           192.168.2.1
```

Por qué el host tiene una sola entrada además de la red: porque un host **no enruta**, sólo entrega. Todo lo que no sea de su propia subred va al **default gateway** (`0.0.0.0/0`). Y **por qué el gateway es el router que está en su propia subred** (`192.168.0.1` para la subred 1): por la misma razón del *next hop* de arriba: para poder hablar con el gateway tiene que poder **alcanzarlo directamente**, o sea que su IP debe estar en la subred del host.

Suposición justificada: los routers usan la **dirección más baja** de cada subred como gateway (`192.168.0.1`, `.1.1`, `.2.1`), que es la convención que aparece en las soluciones oficiales de FING y la que se usa en el enunciado (`sirc1003`).

b-i) ¿Por qué hay que renumerar?

Acá está el corazón conceptual de la clase, y es **sumarización** aplicada bit a bit.

Lo que se quiere: un solo switch para las subredes 2 y 3 ⇒ **una sola subred IP** que las contenga a las dos. Si R1 tiene que announcement una sola entrada para las dos, ambas tienen que ser **contiguas en el espacio de direcciones** y tener un prefijo común que las cubra.

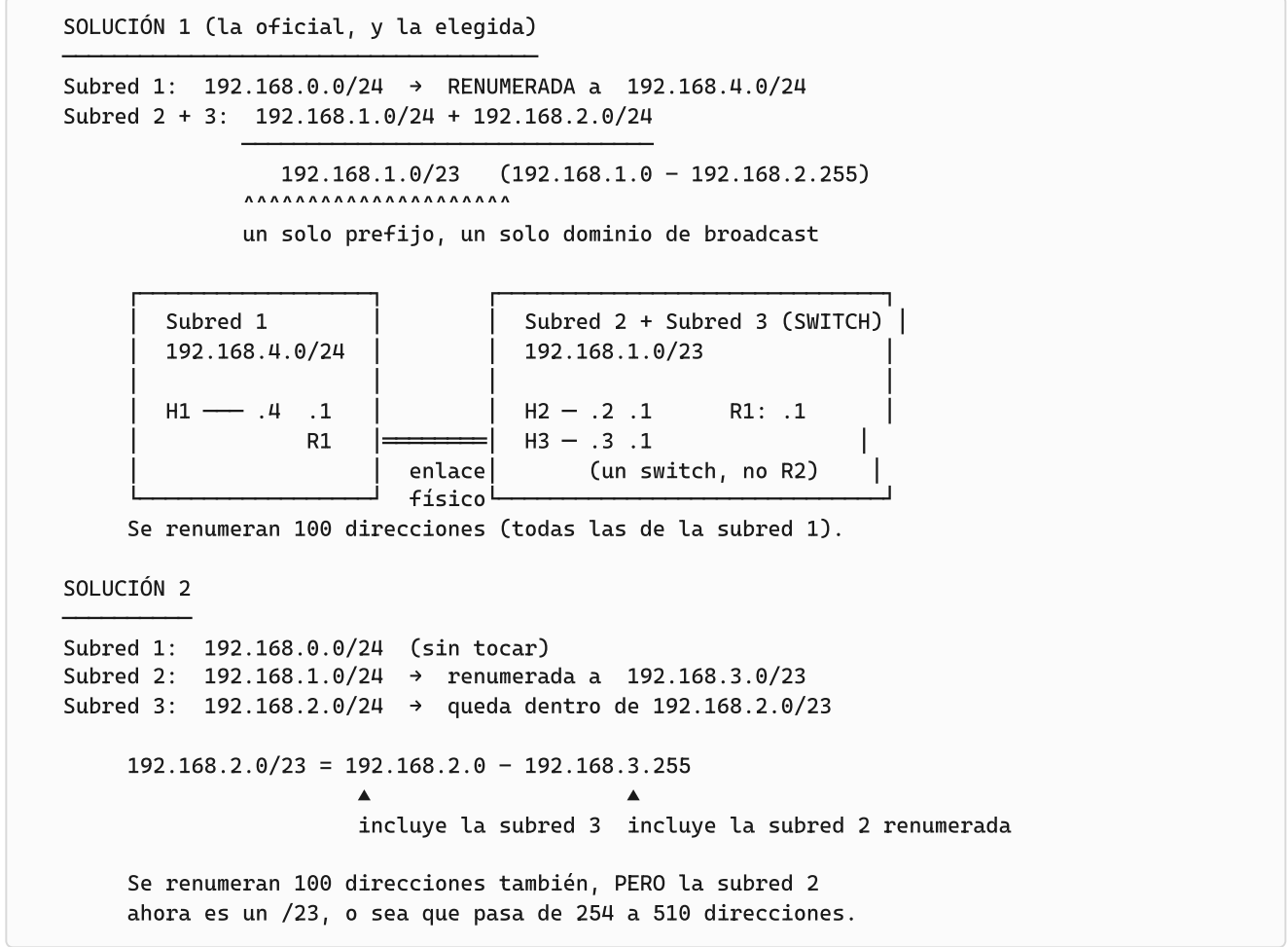
Intentando incluir `192.168.1.0/24` y `192.168.2.0/24`:

```
Bloque /22 que empieza en 192.168.0.0:
  192.168.0.0/22  =  192.168.0.0  a  192.168.3.255
  ▲              ▲
  incluye        incluye hasta 192.168.3.255
  192.168.0.0/24
  e incluye 192.168.1.0/24 y 192.168.2.0/24  ✓

¿Es el prefijo más corto que las cubre? Sí:
  /23 desde 192.168.0.0  = 192.168.0.0  a  192.168.1.255   → NO incluye la .2
  /23 desde 192.168.2.0  = 192.168.2.0  a  192.168.3.255   → NO incluye la .1

Entonces el mínimo es /22, y ese /22 mete adentro
192.168.0.0/24, que es LA SUBRED 1.  ← el problema
```

Verificación bit a bit de por qué el /22 es el mínimo (esto es lo que hay que saber hacer):

```
Los tres /24 de la subred:
  192.168.0.0/24   tercer octeto: 0   = 00000000
  192.168.1.0/24   tercer octeto: 1   = 00000001
  192.168.2.0/24   tercer octeto: 2   = 00000010
                                                ▲
  los 3 octetos son iguales → los bits que importan
  son los últimos 8 del TERCER octeto

Para que un prefijo cubra los valores 0, 1 y 2, tiene que
cubrir el rango 0 a 3, porque los bits que los distinguen son
los DOS últimos:

  00000000 = 0    ┐
  00000001 = 1    ├─ los dos últimos bits
  00000010 = 2    ┘

  Un prefijo que no toque esos 2 bits tiene que llegar hasta
  el 3er octeto = 00000011 (= 3)  →  /22  →  192.168.0.0/22

Un /23 sólo congelaría 1 bit (0000000_), o sea o el 0-1 o el
2-3, y por lo tanto no alcanza para los tres.

Conclusión: cualquier prefijo que cubra las subredes 1, 2 y 3
(en la numeración original) tiene que incluir 192.168.0.0/24.
⇒ hay que renumerar.
```

b-ii) Las dos soluciones, y por qué la primera es la mejor



Comparación honesta de las dos (esto es lo que el examen pide implícitamente con "la menor cantidad de direcciones posibles"):

	Solución 1	Solución 2
Subredes renumeradas	**1** (la subred 1)	**1** (la subred 2)
Direcciones de host modificadas	~100	~100
Tamaño del bloque del switch	`/23` = 510 direcciones	`/23` = 510 direcciones
Direcciones de **red** de las subredes	1 renumerada	1 renumerada
Problema de la 2	—	la subred 2 pasa de `/24` a `/23`: **cambia su máscara**, o sea que hay que reconfigurar la máscara de 100 hosts además de la dirección
Tabla de R1	2 entradas	2 entradas

La oficial prefiere la 1 porque en la 1 **sólo cambian direcciones IP**: la subred renumerada conserva su `/24`, así que su máscara no cambia y los hosts sólo cambian su IP. En la 2 hay que cambiar IP **y** máscara. Con una red de 100 hosts eso es la misma cantidad de direcciones, pero **más configuración**.

Suposición que hago y declaro: el enunciado dice "modificar la menor cantidad de **direcciones IP** posibles", y en sentido estricto las dos soluciones modifican un número parecido de direcciones. Si en tu material aparece

sólo la solución 1, es por este criterio de máscara. **Si el enunciado te permitiera dejar una subred con máscara /23 y no te importara la reconfiguración, la 2 también es válida** — y eso es exactamente el tipo de decisión que hay que justificar por escrito en un examen.

b-iii) La nueva tabla de R1 (solución 1)

```
Configuración R1
Red destino / Máscara      Gateway   Interfaz
─────────────────────────  ────────  ─────────
192.168.4.0 / 255.255.255.0   –      eth0
192.168.0.0 / 255.255.255.0   –      eth1     ← 192.168.0.0/22
192.168.1.0 / 255.255.254.0   –      eth1
```

Interpretación: la subred 1 **dejó de estar en** `192.168.0.0/24` y pasó a `192.168.4.0/24`, así que R1 ya **no** necesita una entrada para ella: ahora la anuncia con el **default** `0.0.0.0/0` hacia su ISP, que es lo que corresponde a una red privada. Y la subred del switch cambió de máscara de `/24` a `/23`: los hosts de la subred 3 (`192.168.2.x`) ahora son de la **misma subred** que los de la subred 2, así que **ya no necesitan salir al router para comunicarse entre ellos**: se entregan directo por el switch. **Ese es el beneficio real del ejercicio**: menos tránsito por el router, menos encapsulación, y un dominio de broadcast compartido.

3.2 Nota: cuándo sí se pueden agrupar dos `/24` — el caso del Examen 2009-12

El `sirc0912` Problema 1 usa las subredes `192.168.108.0/24`, `192.168.109.0/24` y `192.168.111.0/24`, y su solución oficial **agrupa 108 y 109 en** `192.168.108.0/23` sin renumerar nada. ¿Por qué ahí sí y en el caso anterior no? **Por la paridad del tercer octeto**, y es el mismo razonamiento bit a bit:

```
/23 = el último bit del TERCER octeto queda libre.
Un /23 cubre X.0.0 – X.1.255, es decir los pares {0,1}, {2,3},
{4,5}, ... Necesita que el 3er octeto sea PAR para arrancar.

108 = 01101100  (par)   109 = 01101101  (impar)
     └─ los 6 primeros bits iguales, difieren sólo en el último
→ 01101100|0 = 192.168.108.0/23  los cubre a los dos   ✓ SIN renumerar

111 = 01101111  (impar)  110 = 01101110  (par)
→ 0110111|0 = 111 = 192.168.110.0/23  cubre 110 y 111   ✓
→ y 192.168.108.0/23 cubre 108 y 109                      ✓

Pero 110, 111, 112, 113 sería 4 redes y ningún /23 (ni /22
alcanzable sin arrastrar la 192.168.108.0/24) las cubre todas
sin meter una subred de más.
```

La regla para el examen, en una línea: *dos `/24` contiguos se pueden sumar en un `/23` si y sólo si el primero tiene el tercer octeto par.* Si es impar, hay que renumerar. Y para tres `/24` hay que mirar los **dos** últimos bits, como hicimos en §3.1 b-i.

PARTE 4 — CIERRE

📌 Resumen

1. La cabecera IPv4 mínima son **20 bytes** = **5 filas de 32 bits**, con **13 campos**. **No cambió desde 1981.**
2. **IHL** dice el tamaño de la cabecera en **palabras de 4 bytes**: `IHL × 4`. Con opciones, de 20 a 60 bytes. **En la práctica, IHL = 5 siempre.**
3. **Dirección de red = dirección AND máscara. Broadcast = red + 2^(32−n) − 1. Rango de hosts = red + 1 … red + 2^(32−n) − 2. Chequeo: `2^(32−n) − 2` hosts útiles.**
4. Las **clases A/B/C/D/E** son historia de 1981. Con una clase B para 300 hosts se desperdician **65 234** de 65 534 direcciones: el 99,5 %. Por eso CIDR, en 1993.
5. **El `/n` y el número mágico** (`RFC 1878`) son el atajo del subnetting: el menor número de la tabla que sea ≥ lo que necesitás.
6. **CIDR** (`RFC 4632`) liberó la longitud del prefijo, volvió la máscara **local a cada enlace** y rehabilitó la **sumarización**, sin la cual la tabla de Internet no cabría.
7. Los rangos especiales existen **cada uno por un motivo semántico**: `0/8` = inexistencia, `127/8` = yo mismo, `169.254/16` = autoconfiguración en el enlace, `224/4` = grupo, `255.255.255.255` = broadcast local. Las **privadas** existen porque IPv4 se agotó y porque no todo tiene que ser alcanzable desde todo el mundo.
8. El **checksum** es la suma del complemento a uno de las palabras de 16 bits de la cabecera, **con acarreo**, y se verifica porque al verificar da `0xFFFF`. Cubre **sólo la cabecera**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué existe el campo IHL si la cabecera IPv4 "son 20 bytes"?

▶ Respuesta

2. Calculá la red, el rango de hosts y el broadcast de `192.168.10.55` con máscara `/26`.

▶ Respuesta

3. ¿Qué clase tiene `172.16.40.9` y qué máscara lleva "por defecto"?

▶ Respuesta

4. ¿Por qué se agotaron las clases B? Poné los números.

▶ Respuesta

5. ¿Por qué DHCP autoasigna `169.254.x.x` y no `0.0.0.0` ni `192.168.1.5`?

▶ Respuesta

6. Calculá el checksum de la cabecera de un datagrama con longitud total 60, TTL 64, protocolo TCP, origen `192.168.10.55` y destino `219.41.31.21`.

▶ Respuesta

7. Dos `/24` contiguos, `192.168.1.0/24` y `192.168.2.0/24`. ¿Se pueden agrupar en un solo prefijo? ¿Por qué?

▶ Respuesta

8. ¿Y si las dos subredes fueran `192.168.108.0/24` y `192.168.109.0/24`? ¿Se pueden agrupar?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1003` / `sirc1003`** — Problema 1 (30 pts): tres subredes `/24` con R1 y R2, tabla de enrutamiento de routers y de hosts, y el renumerado mínimo al reemplazar R2 por un switch. El mejor ejemplo de **sumarización** del banco.
- **Examen `eirc0912` / `sirc0912`** — Problema 1: tres clases C y la agregación de `192.168.108.0/24` + `192.168.109.0/24` en una `/23`.
- **Examen `eirc1103` / `sirc1103`** — Problema 2d: dividir un `/24` en dos subredes de tamaños distintos. · **Examen `eirc0812` / `sirc0812`** — P2: plan de numeración.
- **Examen `eirc0712` / `sirc0712`** — Problema 2a: ARP entre dos hosts y un router. · **Examen `eirc0608`** — P4: `200.40.224.0/25` subdividido en redes de 18, 11 y 9 hosts.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red" — §5.4 el protocolo IP: la cabecera.
- **RFC 791** — Internet Protocol (1981): el formato de la cabecera, el TTL, la fragmentación y el mínimo de 576 bytes.
- **RFC 1122** — Requirements for Internet Hosts (1981): comportamiento de host, incluido el mínimo de 576 bytes.
- **RFC 1812** — Requirements for IP Version 4 Routers (1995): qué debe hacer un router con la cabecera, incluido el checksum y el TTL.
- **RFC 1878** — Requirements for IP Address Reservation (1996): **el anexo con el "número mágico"**, la tabla de hosts por prefijo.
- **RFC 4632** — CIDR: An Address Assignment Plan for IPv4 (2006): el fin de las clases, el prefijo libre y la sumarización.
- **RFC 8200** — IPv6 (2017): la cabecera fija de 40 bytes, sin IHL ni opciones, y el campo *Flow Label*.
- ⚠️ **RFC que hay que verificar antes de citar en un examen** (no figuran en `ACTUALIZACIONES.md`): `RFC 1918` (direcciones privadas), `RFC 950` (subnetting, 1984, obsoleta), `RFC 3927` (link-local/APIPA), `RFC 6598` (CGNAT, `100.64/10`) y `RFC 7126` (cabecera IPv4 con IHL). Confirmalos en `rfc-editor.org`.
- **Clase siguiente:** 27 — Subnetting, método paso a paso

Clase 27

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 Por qué existe el subnetting
- 1.2 El método de 7 pasos
- 1.3 Casos borde que se preguntan
- 1.4 ⚠️ Actualización a 2026
- 2.1 Ejercicio 1 — Dividir `192.168.10.0/24` en 4 subredes iguales
- 2.2 Ejercicio 2 — Un `/24` para redes de 100, 50 y 20 hosts
- 2.3 Ejercicio 3 — Host concreto dentro de un `/16`
- 2.4 Ejercicio 4 — "Al menos 6 subredes" en una clase C: la trampa del criterio
- 2.5 Ejercicio 5 — Los cinco casos borde de un vistazo
- 2.6 Ejercicio 6 — Un plan de numeración con tres LANs y dos enlaces
- 2.7 Ejercicio 7 — La subnet cero y la todas-unos
- 2.8 Ejercicio 8 — Subnetting de igual tamaño contra VLSM (por qué dos subredes iguales no alcanzan)
- 3.1 ⭐ Examen 2011-03, Problema 2, apartado d) — dividir `100.100.100.0/24`
- 3.2 Nota de coordinación: el resto del Problema 2 de 2011-03
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: D (Red) · **Semana:** 9 · **Duración estimada:** 120 min **Correlativa:** Clase 26 (IPv4: el datagrama y las direcciones) · **Sigue con:** Clase 28 (VLSM y planes de numeración) **Material de referencia:** Tanenbaum cap. 5 (§5.4) · RFC 1878, RFC 3021, RFC 4632, RFC 950

🎯 Al terminar esta clase vas a poder

1. Aplicar un **método de 7 pasos** para dividir cualquier bloque de direcciones, siempre en el mismo orden, con la aritmética **a la vista**.
2. Calcular los **bits de host** que necesita una subred con `2^h − 2 ≥ H`, y justificar por qué se restan **2** (red y broadcast) y cuándo se puede usar `2^h − 1`.
3. Calcular el **tamaño del prefijo**, el **número de subredes** que sale, y explicar por qué las versiones viejas de IP dividían en 2 y perdían la mitad.
4. Calcular **dirección de red, primer host, último host y broadcast** de cada subred, y verificarlos con la suma.
5. Resolver un caso de **subred cero y subred "todas unos"**, y los casos borde de `/31` **y** `/32`.
6. Explicar por qué **el broadcast de la subred cambia al hacer subnetting**, con el ejemplo de `192.168.1.0/24` dividido en cuatro `/26`.
7. Resolver el **apartado d) del Problema 2 del Examen 2011-03** entero, y justificar por qué dos subredes **de igual tamaño** no cumplen el requisito de "dejar direcciones para uso futuro".

> **Prerequisito del examen:** `2011-03 Problema 2d` (30 pts, apartado d), `2010-03 Problema 1` (renumerar tres `/24`), `2006-08 P4` (`200.40.224.0/25` en redes de 18, 11 y 9 hosts), `2010-07 P2`, `2008-12 P2`. Subnetting aparece en **prácticamente todos** los exámenes de la materia.

PARTE 1 — TEORÍA

1.1 Por qué existe el subnetting

En 1993 una empresa recibía **una clase B**: 65 536 direcciones. Con 300 computers, el 99,5 % del bloque estaba tirado. Antes de CIDR la única forma de no desperdiciar era **subnetting**, y el subnetting era **obligatorio** para las clases A, B y C: dividirse la clase en subredes para repartirlas entre routers, usando el campo de red que queda dentro de la clase.

```
Una clase B prestada  172.16.0.0/16  (65 536 direcciones)
NO se puede usar toda como una red, porque hay que poner routers
en el medio y los routers necesitan estar en subredes distintas:

┌──────────────────────────────────────────────────────────┐
│  una sola red /16  →  NO se puede poner un router en     │
│  el medio, porque los dos lados serían la misma red y    │
│  el router no tendría dos interfaces en redes distintas  │
└──────────────────────────────────────────────────────────┘

Con subnetting, el /16 se divide en subredes /24:
  172.16.0.0/24    172.16.1.0/24    172.16.2.0/24    ...
  172.16.255.0/24
  = 256 subredes de 254 hosts
```

Las dos reglas históricas que quedaron marcadas. (a) Todos los routers de la red tenían que usar **la misma máscara**: el subnetting clásico **no** adm;subredes de tamaños distintos (eso es VLSM, Clase 28). (b) La subred **cero** (la primera, con todos los bits de subred en 0) y la subred **"todas unos"** (la última) porque eran rutas por defecto legítimas en el modelo antiguo, muchas implementaciones las rechazaban. La práctica moderna **las acepta** (`RFC 950` — verificar su estado: hoy está obsoleta y su obsoletencia es justamente lo que permite el uso de la subred cero).

Como se aplica una mascara de subred, bit a bit

IP	Mascara (/26)	AND = resultado
11000000	11111111	11000000
10101000	11111111	10101000
00001010	11111111	00001010
00110111	11000000	00000000

192.168.10.55 AND 255.255.255.192 = 192.168.10.64

Lo que resulta de ese AND:

direccion de red	192.168.10.64
primer host	192.168.10.65
ultimo host	192.168.10.126
broadcast	192.168.10.127
hostables	62

El prefijo /26 significa:

```
26 bits de red + 6 bits de host
2^6 = 64 direcciones en la subred
64 - 2 = 62 hostables

El AND pone en 0 todos los bits
de host: por eso el resultado es
la direccion de RED, no una de host.
```

Y hoy el subnetting se usa para otra cosa. Con CIDR, una organización puede comprar un `/22` y seguir subnettingendo para todo lo demás. El subnetting no es "lo que se hacía antes de CIDR": es **la herramienta con la que se administra un bloque de direcciones**, y es exactamente lo que hay que saber para cualquier problema de plan de numeración.

1.2 El método de 7 pasos

Este es el procedimiento que hay que usar **siempre**, sin importar el enunciado. Escribilo en el papel antes de calcular nada.

```
╔═══════════════════════════════════════════════════════╗
║  PASO 1  Bits de host:      2^h − 2 ≥ H               ║
║  PASO 2  Tamaño del prefijo:  32 − h    →  /n         ║
║  PASO 3  N° de subredes:      2^(n − prefijo original)  ║
║  PASO 4  Hosts por subred:    2^h − 2  (verificar ≥ H)  ║
║  PASO 5  Dirección de red:    (base ÷ tamaño) × tamaño  ║
║  PASO 6  Primer y último host, y broadcast            ║
║  PASO 7  Todo a punto decimal                         ║
╚═══════════════════════════════════════════════════════╝
```

Paso 1 — Cuántos bits de host

Se necesita `H` direcciones usables, así que el menor `h` que cumple `2^h − 2 ≥ H` es el que hay que buscar.

```
Tabla de búsqueda (la del "número mágico", RFC 1878):

h      2^h        2^h - 2      h      2^h        2^h - 2
────   ─────────  ──────────   ────   ─────────  ──────────
 1        2          0          9      512        510
 2        4          2         10     1024       1022
 3        8          6         11     2048       2046
 4       16         14         12     4096       4094
 5       32         30         13     8192       8190
 6       64         62         14    16384      16382
 7      128        126         15    32768      32766
 8      254        252         16    65536      65534
```

¿Por qué se restan 2? Porque en una subred IPv4 **dos direcciones no se pueden asignar a ningún host**:

Dirección	Quién la usa	Por qué no es asignable
Dirección de red	nadie	Identifica **la subred misma**. Si un host la tuviera, no se sabría si habla con la subred o con otro host. Con **cero** en la parte de host es la convención que define "estamos todos en la misma red".
Dirección de broadcast	nadie	Es el destino para hablar con **todos** a la vez dentro de esa subred. Si un host la tuviera, al recibir un paquete dirigido a ella creería que es para todos.

Cuándo se puede usar `2^h - 1`. Hay **un solo caso legítimo: un enlace punto a punto con exactamente dos extremos**, donde la dirección de red no se necesita para nada (nadie hace broadcast: sólo hay dos) y la dirección de broadcast **tampoco** (se habla unicast). Ahí **las dos direcciones son utilizables** y una `/31` da **2 direcciones útiles**. Eso es lo que norma la `RFC 3021` (*Using 31-Bit Prefixes on IPv4 Point-to-Point Links*, 2000, verificar):

```
Un enlace punto a punto de 2 dispositivos:

     R1 (.0)  ========================  R2 (.1)
      |            2 IPs, 2 usadas        |

Con la regla clásica: 2^1 - 2 = 0  →  ¡no hay ninguna dirección
que asignar!  Por eso la regla clásica NO alcanza y hace
falta la excepción de la RFC 3021.

Con la RFC 3021: un /31 tiene exactamente 2 direcciones y
las DOS son asignables (no hay red, no hay broadcast).

┌──────────────────────────────────────────────────────────┐
│ En un /31 NO se habla de "red" ni de "broadcast":        │
│   .0 = la dirección del primer extremo                   │
│   .1 = la dirección del segundo extremo                  │
└──────────────────────────────────────────────────────────┘
```

Y cuándo se usa `2^h - 1` en un examen que NO es punto a punto: nunca, salvo que te digan explícitamente "hay N hosts y una de ellas es el gateway ya fijo". Ahí el criterio es: si el **gateway ya tiene su dirección asignada y contada por el enunciado**, sumala a `H` antes de buscar. Es el caso del `2011-03 P2d` y lo veremos.

Paso 2 — El tamaño del prefijo

```
prefijo /n  =  32 − h

Ejemplos:   h = 7   →  /25    (128 direcciones, 126 hosts)
            h = 8   →  /24
            h = 6   →  /26    (64 direcciones, 62 hosts)
            h = 5   →  /27    (32 direcciones, 30 hosts)
            h = 4   →  /28    (16 direcciones, 14 hosts)
```

Y la **máscara decimal** sale de los `n` unos: `255.255.(2^x − 1).0` con `x = 8 − (n mod 8)`.

```
/24  →  11111111.11111111.11111111.00000000  = 255.255.255.0
/25  →  ...11111111.10000000                = 255.255.255.128
/26  →  ...11111111.11000000                = 255.255.255.192
/27  →  ...11111111.11100000                = 255.255.255.224
/28  →  ...11111111.11110000                = 255.255.255.240
/29  →  ...11111111.11111000                = 255.255.255.248
/30  →  ...11111111.11111100                = 255.255.255.252
/31  →  ...11111111.11111110                = 255.255.255.254
/32  →  ...11111111.11111111                = 255.255.255.255
```

Paso 3 — Cuántas subredes salen

Si el bloque original es `/p` y el nuevo es `/n`, se **tomaron `n − p` bits** de la parte que era de host, y cada uno de esos bits **se puede poner en 0 o en 1**:

```
n° de subredes  =  2^(n − p)

/24 → /27 :   n − p = 27 − 24 = 3   →  2³ =   8 subredes
/24 → /26 :   n − p = 26 − 24 = 2   →  2² =   4 subredes
/24 → /25 :   n − p = 25 − 24 = 1   →  2¹ =   2 subredes
/16 → /25 :   n − p = 25 − 16 = 9   →  2⁹ = 512 subredes
/16 → /24 :   n − p = 8              →  2⁸ = 256 subredes
```

La división en 2 de las versiones viejas de IP, y por qué se perdía la mitad. Antes de que existiera el subnetting *a la carta*, la convención era que **sólo se podía pedir un bit de subred** por vez: una clase B prestada se dividía en dos `/17`, cada uno en dos `/18`, etc. Con ese criterio:

```
Clase B  172.16.0.0/16
  1 bit de subred  →  /17  →   2 subredes de 32 766 hosts
  2 bits           →  /18  →   4 subredes de 16 382 hosts
  ...
  8 bits           →  /24  → 256 subredes de   254 hosts   ← tope clásico

El problema: con 1 bit se perdía la mitad de la clase B
(65 536 → dos subredes de 32 766 y se quedaban 2
direcciones sin poder usar, la de red y la de broadcast de
cada mitad... y además el cálculo no era múltiplo exacto).
Y, sobre todo, los routers viejos sólo comparaban UN bit de
subred, así que una subred con todos los bits de subred en 0
y la que los tenía todos en 1 se confundían con la ruta
por defecto.

La regla moderna: se pueden tomar TANTOS bits de subred como
haga falta, porque el router hace longest prefix match
(Clase 24, §1.4). Por eso hoy el "n° de subredes" es
2^(n-p) sin restricciones.
```

Paso 4 — Verificar los hosts por subred

```
hosts por subred = 2^h - 2 = 2^(32-n) - 2
chequeo:  hosts por subred  ≥  H   (el número que pedían)

Y el otro chequeo, que es el que casi nadie hace y que suma:
hosts por subred  ×  n° de subredes  ≤  2^(32-p)  -  2
(o sea: no podés haber creado subredes que no entran)
```

Paso 5 — La dirección de red de cada subred

La dirección de red de la subred *k* (contando desde 0) es el **bloque del tamaño de la subred** que empieza en el bloque base:

```
red_k = base + k × 2^(32-n)

Ejemplo con 192.168.10.0/24 dividido en /27  (tamaño 32):

  k = 0  →  192.168.10.0  + 0×32  = 192.168.10.0
  k = 1  →  192.168.10.0  + 1×32  = 192.168.10.32
  k = 2  →  192.168.10.0  + 2×32  = 192.168.10.64
  k = 3  →  192.168.10.0  + 3×32  = 192.168.10.96
  k = 4  →  192.168.10.0  + 4×32  = 192.168.10.128
  k = 5  →  192.168.10.0  + 5×32  = 192.168.10.160
  k = 6  →  192.168.10.0  + 6×32  = 192.168.10.192
  k = 7  →  192.168.10.0  + 7×32  = 192.168.10.224
```

Y el atajo mental para no equivocarse: mirá **el último octeto** de la base. Si es 0 (o múltiplo del tamaño), sumás directo en ese octeto. Si no, primero hay que bajar al múltiplo del tamaño más cercano, y **los bits que se "prestan" de los octetos de arriba son los que se encienden**. El chequeo infalible es: **la dirección de red tiene todos los bits de host en cero.**

Paso 6 — Primer host, último host y broadcast

```
primer host    =  red + 1
último host    =  red + 2^(32-n) - 2
broadcast      =  red + 2^(32-n) - 1

Y los tres chequeos que hay que hacer SIEMPRE:

  chequeo 1:  último host - primer host + 1 = n° de hosts
  chequeo 2:  broadcast = último host + 1
  chequeo 3:  (primer host - red) = 1  y  (broadcast - red) = 2^(32-n) - 1
```

Paso 7 — Todo a punto decimal

En un examen de papel es obligatorio **escribir la cuenta y el resultado en decimal**. Convertir un `/26` a `255.255.255.192` no es adivinar: es `128 + 64 = 192`. Y un broadcast como `192.168.10.191` sale de `160 + 31`.

1.3 Casos borde que se preguntan

La subred cero y la subred "todas unos"

Con los bits de subred, la primera subred es la que los tiene **todos en 0** y la última la que los tiene **todos en 1**:

```
192.168.10.0/24  en  /28  (2 bits de subred, 4 subredes)

bits de subred   subred              #  (subred cero)
──────────────   ──────────────────  ──────────────────
     00          192.168.10.0/28        0
     01          192.168.10.16/28       1
     10          192.168.10.32/28       2
     11          192.168.10.48/28       3

0 = "la subred cero"  →  ¿válida?
3 = "la subred todas unos" (los bits 11) → ¿válida?

MODELO ANTIGUO:  ambas eran PROHIBIDAS. Motivo: un router
antiguo comparaba un solo bit de subred contra la tabla, y
una entrada con todos los bits de subred en 0 se confundía
con la ruta por defecto (0.0.0.0/0); la de todos en 1 se
confundía con la dirección de broadcast "255...".

MODELO ACTUAL:    AMBAS SON VÁLIDAS. Con longest prefix
match no hay ambigüedad: /28 matchea, /0 matchea, gana /28.
`RFC 950` (Internet Subnetting, 1984) - verificar su estado
actual: la práctica moderna acepta la subred cero y la
todas-unos, y esa aceptación es justamente lo que dejó
obsoleta a la RFC 950.

EN UN EXAMEN:   si no dicen nada, dálas por válidas y
justificalo en una línea ("no hay ambigüedad con longest
prefix match"). Si el enunciado dice "no usar la subred
cero", hacé caso.
```

`/31` y `/32`

Prefijo	Tamaño	Hosts (regla clásica)	Dónde se usa
`/32`	1 dirección	1	**Una sola dirección.** Es lo que usan: la **interfaz loopback** de un router (`Router1# ip interface Loopback0 → 10.0.0.1/32`), una **ruta de host** para indicate que "este equipo en particular", y las reglas de NAT que matchean una IP exacta.
`/31`	2 direcciones	0 (por la regla clásica)	**Enlace punto a punto**, por la excepción de la `RFC 3021` : los dos extremos usan las dos direcciones. Ejemplo real: `/31` entre dos routers en un enlace de backbone.
`/30`	4 direcciones	2	El "mínimo útil" de una red con router: 2 hosts + red + broadcast.

Lo que hay que decir de `/31` en un examen: "*un `/31` tiene 2 direcciones y por la regla clásica daría 0 hosts utilizables; la `RFC 3021` legitima su uso en enlaces punto a punto, donde no hay broadcast y por lo tanto las dos direcciones son asignables. En una red con más de dos dispositivos, un `/31` es inválido.*" Y la cuenta a la vista: `2¹ − 2 = 0 < 2`.

Por qué el broadcast de la subred **cambia** al hacer subnetting

Éste es **el error conceptual más común del curso**, y no es un error de cuenta: es un error de **modelo mental**.

```
ANTES de subnetting:
   192.168.1.0/24
   red = 192.168.1.0
   hosts = 192.168.1.1 ... 192.168.1.254
   BROADCAST = 192.168.1.255          ← el último de los 256

DESPUÉS de dividir en cuatro /26:
   ┌──────────────────┬──────────────────┬──────────────────┬──────────────────┐
   │  192.168.1.0/26  │ 192.168.1.64/26  │ 192.168.1.128/26 │ 192.168.1.192/26 │
   ├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
   │ red    .0        │ red    .64       │ red    .128      │ red    .192      │
   │ hosts  .1  - .62 │ hosts  .65 - .126│ hosts  .129 - .190│ hosts  .193 - .254 │
   │ BROAD  .63       │ BROAD  .127      │ BROAD  .191      │ BROAD  .255      │
   └──────────────────┴──────────────────┴──────────────────┴──────────────────┘
     .0              .63  .64            .127  .128          .191  .192          .255

   ⇒ Aparecen CUATRO broadcasts distintos: .63, .127, .191 y .255.
     El único que "ya conocías" (.255) es el de la última.
```

Por qué la respuesta conceptual es "el broadcast depende de la máscara". El broadcast de una subred es **la dirección con todos los bits de host en 1, dentro de la subred**. Con la `/24`, "todos los bits de host en 1" daba `192.168.1.255`; con la `/26`, "todos los bits de host en 1" dentro de la subred `192.168.1.0/26` da `192.168.1.63`. **El broadcast no es un número fijo de la dirección base: es una función de la máscara.** Y un host que mande `192.168.1.255` con máscara `/26` está en la subred `192.168.1.192/26`, y ese paquete es el **broadcast de su propia subred**, no un broadcast global: **nadie fuera de ese `/26` lo va a recibir nunca.**

1.4 ⚠️ Actualización a 2026

Sigue igual, de memoria — y es lo único que se pregunta:

- La fórmula `2^h − 2 ≥ H` y el **número mágico** de la `RFC 1878`.
- Red = AND; primer host = red + 1; último = red + 2^(32−n) − 2; broadcast = red + 2^(32−n) − 1.
- El broadcast depende de la máscara.
- La subred cero y la todas-unos, y el `2^(n−p)` subredes.
- `/31` para punto a punto y `/32` para loopback.

Cambió o se precisa:

- **`/31` es práctica normal desde 2010.** La `RFC 3021` (2000) está implementada en todos los routers y sistemas operativos actuales, y es **el default en muchos frameworks de configuración** (y en BGP, una `/31` se usa para el *link* local entre routers y para las sesiones `loopback`). ⇒ **Lo que era un truco de laboratorio pasó a ser la práctica de backbone.**
- **La subred cero y la todas-unos se usan sin comentarios.** `RFC 950` (1984) está **obsoleta**, y su obsoletencia es lo que hace posible el uso de ambas. Ya nadie deduplica la primera ni la última.
- **En IPv6 la pregunta es distinta y hay que saberlo.** El prefijo es de **/64 a /128** y `/64` es **la unidad de subred estándar** (`RFC 8200`): es el tamaño donde la auto-configuración de dirección (SLAAC) funciona. **En IPv6 no existen la dirección de red ni la de broadcast**: el prefijo es el identificador de la subred y **no hay broadcast** (hay **multicast** para todo). Por eso en IPv6 el cálculo es `2^(128−n) − 1` hosts, no `− 2`, y por eso `/127` y `/128` se usan como punto a punto y loopback. **Nunca apliques el `− 2` de IPv4 a IPv6.**
- **El subnetting "de manual" se reemplazó por plan de numeración con summing.** Hoy se elige el **bloque** que compra la organización (`RFC 6177` ya no recomienda `/48` por sitio final) y se subdivide en subredes de tamaños distintos (**VLSM**, Clase 28). El subnetting de igual tamaño es un ejercicio de papel; el profesional es VLSM.
- **Ya no existe "la clase que te toca".** Si te dan un `/24` y necesitás 30 hosts, no pienses "es una clase C": pensá `/27`. Decir "clase C" en 2026 es un error conceptual, aunque el examen viejo lo acepte.

Consecuencia para el examen: el método de 7 pasos se aplica **exactamente igual** en 2026. Lo que cambia son las preguntas de contexto: por qué `/31`, por qué no `− 2` en IPv6, por qué `/64` en IPv6. **Guardá esas tres.**

PARTE 2 — EJERCICIOS RESUELTOS

2.1 Ejercicio 1 — Dividir `192.168.10.0/24` en 4 subredes iguales

Datos. El bloque es `192.168.10.0/24` (256 direcciones). Se quiere dividirlo en **4 subredes iguales** y **maximizar los hosts por subred**.

Paso 1 — Bits de subred (no de host). Acá el objetivo es "4 subredes", así que conviene razonar al revés:

```
2^3 = 8      2^2 = 4  ← necesito 4
       ↑
Tomo 2 bits de la parte de host:  4 = 2²
Entonces h = 8 − 2 = 6 bits de host quedan por subred.
```

Paso 2 — El prefijo nuevo.

```
/n = 32 − h = 32 − 6 = 26      →   /26
máscara = 11111111.11111111.11111111.11000000 = 255.255.255.192
```

Paso 3 — Número de subredes.

```
2^(26 − 24) = 2² = 4   ✓  exacto, no sobra ni falta
```

Paso 4 — Hosts por subred.

```
2^6 − 2 = 64 − 2 = 62 hosts útiles por subred
2^6     = 64 direcciones por subred

Chequeo de que no me pasé:
  62 × 4 = 248 hosts útiles
  4 redes + 248 hosts + 4 broadcasts = 256  ✓  (justo)
```

Paso 5 a 7 — Las cuatro subredes completas.

```
base = 192.168.10.0     tamaño = 2⁶ = 64

┌─────┬──────────────┬─────────┬──────────────────┬──────────┬─────────────┬──────────────────┐
│  k  │   RED        │  más    │ PRIMER HOST      │ÚLTIMO    │ BROADCAST   │   en binario     │
│     │              │   k×64  │                  │  HOST    │             │  (4° octeto)     │
├─────┼──────────────┼─────────┼──────────────────┼──────────┼─────────────┼──────────────────┤
│  0  │ 192.168.10.0     │ + 0   │ 192.168.10.1     │ .62      │  .63        │ 00000000..01     │
│  1  │ 192.168.10.64    │ +64   │ 192.168.10.65    │ .126     │  .127       │ 01000000..01     │
│  2  │ 192.168.10.128   │ +128  │ 192.168.10.129   │ .190     │  .191       │ 10000000..01     │
│  3  │ 192.168.10.192   │ +192  │ 192.168.10.193   │ .254     │  .255       │ 11000000..01     │
└─────┴──────────────┴─────────┴──────────────────┴──────────┴─────────────┴──────────────────┘

Chequeo fila 1:   .126 − .65 + 1 = 62 hosts          ✓
Chequeo fila 1:   .127 = .126 + 1  (broadcast)       ✓
Chequeo fila 1:   .63 − .0 = 63 = 2⁶ − 1             ✓
Chequeo final:    .255 es el último byte y .255 − .0 + 1 = 256  ✓
```

¿Por qué `/26` y no `/27`?

```
/26  →  2^(26-24) =  4 subredes  ✓  justo lo pedido
/27  →  2^(27-24) =  8 subredes  ✗  son el doble de las pedidas
/25  →  2^(25-24) =  2 subredes  ✗  son la mitad

Con /27 haría 8 subredes de 30 hosts: sobran 4 subredes
enteras que nadie pidió. El enunciado dice "4 subredes
iguales", así que 4 = exacto, y el prefijo que da exactamente
4 es el /26. Con /26 no sobra ni falta un solo bloque.

Y si el enunciado dijera "al menos 4 subredes", entonces sí
valdría el /26... pero también habría que ver qué se pide de
cada subred (ver §2.4, que es justamente donde está la
trampa clásica del examen).
```

2.2 Ejercicio 2 — Un `/24` para redes de 100, 50 y 20 hosts

Datos. Del bloque `192.168.50.0/24` hay que sacar tres redes: una de **100 hosts**, una de **50** y una de **20**. Se busca **maximizar las direcciones libres**.

Nota anticipada: esto es **VLSM básico** (subredes de tamaños distintos), que se desarrolla entero en la Clase 28. Acá se resuelve porque es el patrón que más se repite en los exámenes.

Paso 1 — Bits de host de cada red, por separado.

```
RED DE 100 HOSTS
  2^7 - 2 = 128 - 2 = 126 ≥ 100  ✓    h = 7  →  /25  (128 dir)

RED DE 50 HOSTS
  2^6 - 2 =  64 - 2 =  62 ≥  50  ✓    h = 6  →  /26  (64 dir)

RED DE 20 HOSTS
  2^5 - 2 =  32 - 2 =  30 ≥  20  ✓    h = 5  →  /27  (32 dir)
```

Paso 2 y 3 — Prefijos y subredes.

```
/25  = 255.255.255.128      128 direcciones
/26  = 255.255.255.192       64 direcciones
/27  = 255.255.255.224       32 direcciones
```

Paso 4 — Verificación de que entra todo.

```
128 + 64 + 32 = 224 direcciones usadas de 256
256 - 224 = 32 direcciones libres = un /27 entero sobrante

Hosts: 126 + 62 + 30 = 218 hosts útiles
Redes: 3 redes + 3 gateways (1 en cada red, que ya están
       contados dentro de los H) + 3 broadcasts = 224  ✓
```

Pasos 5 a 7 — La asignación completa.

```
base = 192.168.50.0

┌──────────────────┬──────────────────┬──────────┬──────────────────┬──────────┬──────────────┐
│ Red              │ Prefijo          │  Máscara │ Rango de host    │ Gateway  │ Broadcast    │
├──────────────────┼──────────────────┼──────────┼──────────────────┼──────────┼──────────────┤
│ 100 hosts        │ 192.168.50.0     │   /25    │ .1   – .126      │  .127    │   .127       │
│ 50 hosts         │ 192.168.50.128   │   /26    │ .129 – .190      │  .191    │   .191       │
│ 20 hosts         │ 192.168.50.192   │   /27    │ .193 – .222      │  .223    │   .223       │
│ LIBRE            │ 192.168.50.224   │   /27    │ (sin asignar)    │  –       │   .255       │
└──────────────────┴──────────────────┴──────────┴──────────────────┴──────────┴──────────────┘

Ojo – el "broker" (gateway) no va en su propia columna.
Lo correcto es: el gateway (.127, .191, .223) es el ÚLTIMO
host útil de cada subred.

Tabla final correcta:
┌──────────────────┬──────────────────┬──────────┬──────────────────┬──────────────┐
│ Red              │ Prefijo          │  Máscara │ Rango útil       │ Broadcast    │
├──────────────────┼──────────────────┼──────────┼──────────────────┼──────────────┤
│ 100 hosts        │ 192.168.50.0     │   /25    │ .1   – .126      │   .127       │
│ 50 hosts         │ 192.168.50.128   │   /26    │ .129 – .190      │   .191       │
│ 20 hosts         │ 192.168.50.192   │   /27    │ .193 – .222      │   .223       │
│ LIBRE            │ 192.168.50.224   │   /27    │ (sin asignar)    │   .255       │
└──────────────────┴──────────────────┴──────────┴──────────────────┴──────────────┘
```

La alineación importa. En una `/25` el tamaño es **128** (un medio del 4º octeto), así que la red tiene que empezar en `.0` o `.128` en ese octeto. En una `/26` el tamaño es 64, así que empieza en `.0, .64, .128 o .192`. En una `/27`, en `.0, .32, .64, … .224`. **Si la red que te sale no arranca en un múltiplo del tamaño, la cuenta está mal** — y esa es la verificación que hay que hacer siempre.

El detalle del último `/27` libre: `192.168.50.224/27` va de `.224` a `.255`, con **30 hosts** utilizables (`.225` a `.254`) y broadcast `.255`. Ojo: **`.255` sigue siendo un broadcast válido**, es el de esta subred, no el "broadcast de la /24". Y si en el futuro hay que crear una red de 20 hosts, esa es la que ya está separada.

2.3 Ejercicio 3 — Host concreto dentro de un `/16`

Datos. La red es **`172.16.0.0/16`** y hay que hacer una **subred `/27` que contenga al host `172.16.134.50`**. Decir la red, el rango y el broadcast.

Resolución — dos formas de hacerlo, y las dos hay que saberlas.

Forma 1: usando el número mágico. Una `/27` tiene $2^5 - 2 = 30$ hosts ⇒ sirve para `172.16.134.50` (que está entre los 30 primeros de su /24). Ahora hay que encontrar **dónde cae**.

Forma 2: con el AND, que es la que no falla.

```
Paso 1 — ¿Es una dirección de red?  El AND con la /27 debe dar
         una dirección terminada en un múltiplo de 32.

Paso 2 — El 4° octeto de la dirección es 50. En una /27 los bits
         de host del 4° octeto son los 5 últimos, así que los
         bits de red del 4° octeto son los 3 primeros:

           50 = 00110010
            └──────┴─────┘
           bits de red (3)  bits de host (5)

           Los bits de red de 50 = 001 = 1
           ⇒ el 4° octeto de la red es  00100000 = 32

Paso 3 — El AND completo:
           172.16.134.50
           172.16.134.32/27 = 172.16.134.001xxxxx
           AND          = 172.16.134.32

RED       = 172.16.134.32/27
1er host  = 172.16.134.33
último    = 172.16.134.62     (32 + 32 − 2)
broadcast = 172.16.134.63     (32 + 32 − 1)

Chequeo: .33 a .62 = 62 − 33 + 1 = 30 hosts  ✓  = 2⁵ − 2
Chequeo: .63 − .32 = 31 = 2⁵ − 1             ✓
¿Está dentro .50?  .33 ≤ .50 ≤ .62           ✓
```

El atajo, que es el que hay que tener automatizado: *en una* `/27` *(tamaño 32), la red que contiene al host con 4º octeto* `X` *es la que termina en* `X - (X mod 32)`. Para `50`: `50 mod 32 = 18`; `50 − 18 = 32`. Para `13`: `13 mod 32 = 13`; `13 − 13 = 0` ⇒ la red es `172.16.134.0/27`. Para `200`: `200 mod 32 = 8`; `200 − 8 = 192` ⇒ `192.168.1.192/27` (¡y si te equivocás de octeto, te da el broadcast de la subred anterior!).

2.4 Ejercicio 4 — "Al menos 6 subredes" en una clase C: la trampa del criterio

Datos. *Enunciado:* "Se necesita una clase C dividida en **al menos 6 subredes**. ¿Qué prefijo se debe usar?"

Paso 1 — Buscar `n` tal que `2^n ≥ 6`.

```
2⁰ =  1    ✗
2¹ =  2    ✗
2² =  4    ✗   (4 < 6)
2³ =  8    ✓   (8 ≥ 6)   ← el menor

n = 3 bits de subred tomados
```

Paso 2 — El prefijo.

```
/n = 24 + 3 = 27        →   /27   =  255.255.255.224

Comprobación directa:
  /26 → 2² =  4 subredes   ✗  (4 < 6)
  /27 → 2³ =  8 subredes   ✓  (8 ≥ 6)
  /25 → 2¹ =  2 subredes   ✗
⇒ la respuesta es /27.
```

Paso 3 — Las ocho subredes, para que quede demostrado.

```
192.168.1.0/24  en  /27   (8 subredes de 30 hosts)

192.168.1.0/27   | 192.168.1.32/27  | 192.168.1.64/27   | 192.168.1.96/27
192.168.1.128/27 | 192.168.1.160/27 | 192.168.1.192/27  | 192.168.1.224/27

Total: 8 subredes ≥ 6 pedidas.   Hosts: 8 × 30 = 240 útiles.
```

Por qué el criterio es "al menos" y no "exactamente". Porque **6 no es una potencia de dos, y no se puede hacer exactamente 6 subredes.** El número de subredes de un bloque es **siempre** `2^(n − p)`, o sea una potencia de dos: 1, 2, 4, 8, 16… **La lista de posibilidades no tiene un 6.** Por eso el enunciado tiene que decir "**al menos** 6": obliga a **redondear hacia arriba**, y el redondeo hacia arriba de 6 es 8. **Si el enunciado dijera "exactamente 6", el problema sería imposible**, y eso mismo es la respuesta.

> ⚠️ **Aviso sobre un error frecuente en materiales de estudio.** Si en algún apunte o solución encontrás **`/26` como respuesta a este enunciado**, es un error: `/26` produce **4** subredes, que son **menos** de las 6 pedidas. La respuesta correcta para "al menos 6 subredes" es **`/27`**. El `/26` es la respuesta correcta para **"al menos 4 subredes"** (o para **"al menos 3 subredes, cada una con al menos 60 hosts"**: `n = 2` ⇒ `/26`, y `h = 32 − 26 = 6` ⇒ `2⁶ − 2 = 62 ≥ 60` ✓). **Cuando veas un número que no es potencia de dos, siempre se redondea hacia arriba y se dice por qué.**

2.5 Ejercicio 5 — Los cinco casos borde de un vistazo

Datos. Para cada prefijo, dar red, rango útil, broadcast y si es válido.

Prefijo	Tamaño `2^(32−n)`	Hosts (clásico)	Ejemplo de red	Rango útil	Broadcast	¿Dónde?
`/32`	1	`2⁰ − 2 = −1` → se trata como **1**	`10.0.0.1/32`	`.1` (sólo esa)	**no hay**	**loopback** de un router, o ruta de host
`/31`	2	`2¹ − 2 = 0`	`200.1.1.0/31`	`.0` y `.1` (**2**, por RFC 3021)	**no hay**	**enlace punto a punto**
`/30`	4	`2² − 2 = 2`	`220.10.10.28/30`	`.29`, `.30`	`.31`	red mínima **con** router
`/29`	8	`2³ − 2 = 6`	`192.168.1.8/29`	`.9` – `.14`	`.15`	red chica con router
`/24`	256	`2⁸ − 2 = 254`	`192.168.1.0/24`	`.1` – `.254`	`.255`	LAN de casa

Las notas que hay que dejar escritas:

- En `/32` no hay red ni broadcast: la dirección ES la red. Es una **ruta de host**: "este equipo en particular". Se usa para las **interfaces loopback** de los routers, y por eso los routers de BGP anuncian `/32` de loopback y no la dirección de su interfaz física: así el router es alcanzable aunque un enlace caiga.
- En `/31` (RFC 3021) las dos direcciones son asignables **sólo en punto a punto**. En una red con tres o más dispositivos sería inválido, porque necesitaría una dirección de red y una de broadcast.
- En `/30` aparecen en los exámenes en todos lados, típicamente como **enlace entre el router y el proveedor** (es el `220.10.10.28/30` del `sirc1003` y del `sirc0812`): 2 direcciones útiles y las otras dos son

la red y el broadcast.

2.6 Ejercicio 6 — Un plan de numeración con tres LANs y dos enlaces

Datos. La empresa tiene `172.16.0.0/16` asignado, y necesita:

Elemento	hosts
LAN1	300
LAN2	120
LAN3	25
Enlace punto a punto R1–R2	2 (punto a punto)
Enlace punto a punto R2–R3	2 (punto a punto)

Paso 1 — Bits de host de cada uno.

```
LAN1:  300 hosts → 2^8 − 2 = 254 < 300 ✗ ; 2^9 − 2 = 510 ≥ 300 ✓  h = 9
LAN2:  120 hosts → 2^7 − 2 = 126 ≥ 120 ✓                          h = 7
LAN3:   25 hosts → 2^5 − 2 =  30 ≥  25 ✓                          h = 5
P2P:     2 puns → RFC 3021: /31                                     h = 1
```

Paso 2 — Prefijos y tamaños.

```
LAN1  h = 9  →  /23   = 512 direcciones
LAN2  h = 7  →  /25   = 128 direcciones
LAN3  h = 5  →  /27   =  32 direcciones
P2P 1       →  /31   =   2 direcciones
P2P 2       →  /31   =   2 direcciones
```

Paso 3 y 4 — Suma y verificación.

```
512 + 128 + 32 + 2 + 2 = 676 direcciones usadas

≤ 2¹⁶ = 65 536  ✓   sobra mucho (crecimiento)
```

Paso 5 a 7 — El plan completo, en orden de tamaño decreciente.

```
┌──────────────────────┬──────────────────┬─────────────────┬──────────────┬──────────────
│ Elemento             │ Prefijo          │  Máscara        │ Rango útil   │ Broadcast
├──────────────────────┼──────────────────┼─────────────────┼──────────────┼──────────────
│ LAN1 (300 hosts)     │ 172.16.0.0/23    │ 255.255.254.0   │ .1   − .510  │  .511
│ LAN2 (120 hosts)     │ 172.16.2.0/25    │ 255.255.255.128 │ .1 − .126    │  .127
│ LAN3 (25 hosts)      │ 172.16.2.128/27  │ 255.255.255.224 │ .1 − .30     │  .159
│ Enlace P2P R1–R2     │ 172.16.2.160/31  │ 255.255.255.254 │ .161, .162   │  ninguno
│ Enlace P2P R2–R3     │ 172.16.2.162/31  │ 255.255.255.254 │ .163, .164   │  ninguno
│ LIBRE                │ 172.16.3.0/22    │ 255.255.252.0   │ −            │  .255 de /2
└──────────────────────┴──────────────────┴─────────────────┴──────────────┴──────────────
```

Las tres verificaciones de alineación (las que hay que hacer siempre, y son las que fallan en el examen):

```
LAN1  /23  tamaño 512 → .0      ✓ (0    es múltiplo de 512)
LAN2  /25  tamaño 128 → .2.0    ✓  (el 3er octeto 2 = 2×256; .0 del 4°
                                  es múltiplo de 128)
LAN3  /27  tamaño  32 → .2.128  ✓  (128 = 4×32, múltiplo exacto)
P2P 1 /31  tamaño   2 → .2.160  ✓  (160 = 80×2, par: obligatorio)
P2P 2 /31  tamaño   2 → .2.162  ✓  (162 = 81×2, par: obligatorio)

Si alguno no fuera múltiplo exacto, el cálculo está mal y hay
que "subir" la red al múltiplo del tamaño más cercano por
encima, desperdiciando el hueco.
```

Interpretación: **las dos redes punto a punto arrancan en dirección par**, porque un bloque de tamaño 2 sólo puede empezar en `.0/.2/.4/…` del 4º octeto. Un `/31` en `.161` sería imposible. Ese es el error típico en los planes de numeración con enlaces seriales.

2.7 Ejercicio 7 — La subnet cero y la todas-unos

Datos. Dividir `172.20.0.0/24` en subredes `/26` y decidir si la primera y la última son válidas.

```
172.20.0.0/26   | 172.20.0.64/26  | 172.20.0.128/26 | 172.20.0.192/26
red    .0       | red    .64      | red    .128     | red    .192
hosts  .1–.62   | hosts  .65–.126 | hosts  .129–.190| hosts  .193–.254
BROAD  .63      | BROAD  .127     | BROAD  .191     | BROAD  .255
────────────────┼─────────────────┼─────────────────┼────────────────
SUBNRED CERO    |                 |                 | SUBNRED TODAS UNOS
bits: 00        | bits: 01        | bits: 10        | bits: 11
```

¿Son válidas las dos extremas? Sí, en el modelo actual. Los dos bits de subred hacen que: con la subred cero, la entrada es `172.20.0.0/26`; con la `/24` hace match, con la `/16` también y con el default también, y **gana la `/26` porque es la más específica** (26 > 24 > 16 > 0). No hay ambigüedad posible. La todas-unos es `172.20.0.192/26` y gana por la misma razón.

¿Por qué estaban prohibidas? Porque los routers del modelo antiguo **no** hacían longest prefix match: comparaban bits de subred uno por uno contra la tabla, y una entrada con todos los bits de subred en 0 se confundía con la **ruta por defecto** (`0.0.0.0/0`), y la de todos en 1 con el **broadcast**. `RFC 950` (1984) reflejaba esa prohibición; hoy está obsoleta y **la prohibición también**, porque el longest prefix match la volvió innecesaria.

En un examen: dálas por válidas y escribí la línea que lo justifica ("con longest prefix match no hay ambigüedad: /26 gana sobre /24, /16 y /0"). Si el enunciado dice "sin usar la subred cero", hacé caso y empezá en la segunda: `172.20.0.64/26`.

2.8 Ejercicio 8 — Subnetting de igual tamaño contra VLSM (por qué dos subredes iguales no alcanzan)

Datos. Se tiene `100.100.100.0/24` y se necesitan **dos redes**: una de **70 hosts** y otra de **15**. Se pide "dejar la mayor cantidad posible de direcciones para uso futuro".

Resolución, calculando las dos redes por separado (que es lo correcto):

```
RED DE 70 HOSTS
  2^6 − 2 =  62 < 70  ✗    (no entra)
  2^7 − 2 = 126 ≥ 70  ✓    h = 7  →  /25 = 128 direcciones

RED DE 15 HOSTS
  2^4 − 2 =  14 < 15  ✗    (justo no entra)
  2^5 − 2 =  30 ≥ 15  ✓    h = 5  →  /27 =  32 direcciones
```

Las dos redes, con la aritmética completa:

```
RED 1  = /25  tamaño 128   base + 0×128  = 100.100.100.0/25
         red      .0
         hosts    .1    − .126
         broadcast .127

RED 2  = /27  tamaño  32   base + 1×32   = 100.100.100.128/27
         red      .128
         hosts    .129  − .158
         broadcast .159
                   ↑ alineada: 128 = 4×32  ✓
```

Cuánto queda libre:

```
usadas:  128 + 32  = 160 direcciones
libres:  256 − 160 =  96 direcciones  (un /27 y medio, o sea
                                       .160 a .255)

Quedan libres: 100.100.100.160  …  100.100.100.255
```

Por qué dos subredes de igual tamaño NO cumplirían el requisito. Esta es la parte conceptual que el enunciado de 2011-03 pide:

```
Si las dos subredes fueran iguales, las dos tienen que poder
alojar los 70 hosts:

  dos /25 → 2 × 128 = 256 direcciones   ⇒  NO queda NADA libre
              (se usa el /24 entero, 0 direcciones para el futuro)

  dos /26 → 2 ×  64 = 128 direcciones   ⇒  pero /26 sólo da 62
              hosts útiles y necesito 71 (70 + el router) ⇒ NO ENTRA

  dos /27 → 2 ×  32 =  64 direcciones   ⇒  30 < 71 ⇒ NO ENTRA

  dos /24 → 2 × 256 = 512 direcciones   ⇒  no existe un /24 libre
                                            dentro de un /24

⇒ Con subredes de igual tamaño NO HAY solución que deje
  direcciones libres. El objetivo "dejar la mayor cantidad
  posible de direcciones para uso futuro" obliga a usar
  subredes de TAMANO DISTINTO. Esa es la razón de ser del
  VLSM, y la respuesta completa al apartado d).
```

Este es el **Problema 2d del Examen 2011-03** desarrollado; la resolución completa, con el contexto del problema y la tabla de enrutamiento de R1, está en **§3.1**.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-03, Problema 2, apartado d) — dividir `100.100.100.0/24`

> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf **Enunciado completo del Problema 2** (sólo se pide el apartado d; los demás están asignados a las clases 28, 30 y 31):
>
> > Una empresa tiene una red A (LAN Ethernet con direccionamiento privado) con 15 servidores y 70 puestos de trabajo, y se desea que puedan acceder a un servidor web en Internet con dirección IP `219.41.31.21`. Para ello el router R1 implementa NAT utilizando su dirección IP pública. Se pide: **a)** la tabla de enrutamiento de R1 y del Host H. **b)** i. los mensajes ARP con tabla ARP vacía. ii. MAC e IP de las tramas y paquetes en ambos sentidos. **c)** qué campos cambia R1 en cada sentido y su tabla NAT. **d)** *Se adquiere para la Red A el bloque de direcciones IP públicas `100.100.100.0/24`, y los administradores deciden aislar la red de servidores de los puestos de trabajo. Especifique las subredes del bloque mencionado que cumplan con este requerimiento y permitan dejar la mayor cantidad posible de direcciones IP para uso futuro. Mencione las modificaciones necesarias en el router R1 y su nueva tabla de enrutamiento.*

Solución oficial (literal del PDF):

> **d)** Subred para 70 puestos de trabajo: 70 direcciones IP + 1 para el router R1 (Además siempre se deben considerar la direcciones de red y de broadcast) → `100.100.100.0/25`. Subred para 15 servidores: 15 direcciones IP + 1 para el router R1 (Además siempre se deben considerar la direcciones de red y de broadcast) → `100.100.100.128/27`. El router R1 se conecta a cada subred y no es necesario que implemente NAT, ya que las estaciones de trabajo y los servidores tienen direcciones IP públicas. Nueva tabla de enrutamiento de R1:
>
> | Red destino / Máscara | Gateway / next hop |
> |---|---|
> | `100.100.100.0/25` | C – Directamente conectado |
> | `100.100.100.128/27` | C – Directamente conectado |
> | `220.10.10.28/30` | C – Directamente conectado |
> | `0.0.0.0/0` | `220.10.10.29` |

d-1) Por qué 70 puestos necesitan `/25`

La cuenta del método de 7 pasos, con los números del enunciado a la vista:

```
PASO 1 — bits de host

  H = 70 puestos de trabajo
  + 1 para la interfaz de R1 dentro de esta subred
  ──────────────────────────────
  H = 71 direcciones que hay que poder asignar

  2^6 − 2 =  64 − 2 =  62  < 71   ✗   NO ENTRA
  2^7 − 2 = 128 − 2 = 126  ≥ 71   ✓   h = 7

PASO 2 — prefijo

  /n = 32 − h = 32 − 7 = 25    →   /25
  máscara = 11111111.11111111.11111111.10000000 = 255.255.255.128

PASO 3 — subredes
  No aplica: acá se pide UNA subred de un tamaño dado.

PASO 4 — verificación
  hosts útiles = 2⁷ − 2 = 126 ≥ 71  ✓   (sobran 55)

PASO 5 — dirección de red
  base = 100.100.100.0
  La primera /25 de un /24 arranca SIEMPRE en .0 (128 = 2×64, y el
  bloque 0 es el primero).  Y .0 es múltiplo de 128  ✓
  RED = 100.100.100.0/25

PASO 6 — host, host y broadcast
  1er host   = 100.100.100.0  + 1     = 100.100.100.1
  último host= 100.100.100.0  + 128 − 2 = 100.100.100.126
  broadcast  = 100.100.100.0  + 128 − 1 = 100.100.100.127

PASO 7 — decimal  (ya está en decimal; la cuenta binaria sería)
  128 = 2⁷   ⇒  en el 4° octeto, bit 7 = 128
  .127 = 01111111   .126 = 01111110   .1 = 00000001   .0 = 00000000
```

Por qué hace falta "el +1 del router". R1 tiene que tener **una interfaz dentro de cada subred**, y esa dirección también sale del bloque de la subred. Los 70 puestos más la interfaz de R1 son 71 direcciones asignables. La oficial lo dice explícitamente ("70 direcciones IP + 1 para el router R1"), y es el detalle que hace que la cuenta dé bien: si se olvidara el `+1`, el cálculo sería `2⁶ − 2 = 62 ≥ 70`, que es falso, y uno caería en un `/26` que **no entra**.

Detalle no exigido por el enunciado pero que hay que dejar dicho: la dirección de red y la de broadcast también se consumen (de ahí "Además siempre se deben considerar las direcciones de red y de broadcast"). Son las dos que justifica el `− 2`.

d-2) Por qué 15 servidores necesitan `/27`

```
PASO 1 — bits de host

  H = 15 servidores
  + 1 para la interfaz de R1
  ──────────────────────────
  H = 16

  2^4 − 2 = 16 − 2 = 14 < 16   ✗   ¡justo no entra!
  2^5 − 2 = 32 − 2 = 30 ≥ 16   ✓   h = 5

PASO 2 — prefijo
  /n = 32 − 5 = 27    →   /27   = 255.255.255.224

PASO 4 — hosts útiles = 2⁵ − 2 = 30 ≥ 16  ✓   (sobran 14)

PASO 5 — DIRECCIÓN DE RED: aquí está la parte delicada.
  Tamaño de la /27 = 2⁵ = 32.
  Las /27 dentro de 100.100.100.0/24 arrancan en el 4° octeto
  en los múltiplos de 32:

     0, 32, 64, 96, 128, 160, 192, 224

  Como la /25 anterior ocupa .0 a .127, la siguiente /27 tiene
  que arrancar en 128 (o en un múltiplo de 32 mayor):
     128 = 4 × 32  ✓  alineada

  RED = 100.100.100.128/27

PASO 6 —
  1er host    = 100.100.100.128 + 1  = 100.100.100.129
  último host = 100.100.100.128 + 30 = 100.100.100.158
  broadcast   = 100.100.100.128 + 31 = 100.100.100.159

PASO 7 — verificación
  .159 − .129 + 1 = 31 − 1 + 1 = 31... no: 158 − 129 + 1 = 30 hosts ✓
  .159 − .128 = 31 = 2⁵ − 1  ✓
  .159 es múltiplo de 32 menos 1  ✓
```

Lo que hay que mirar con atención: el `/28` habría sido "justo" por un host. `2⁴ − 2 = 14` y son 15 servidores + 1 router = 16. **Una sola dirección de diferencia** y la respuesta salta de `/28` a `/27`. Si en un examen te encontrás dudando entre dos prefijos, **escribí las dos cuentas completas y dejá que se vean los números**; el error salta a la vista.

d-3) Por qué NO dos subredes de igual tamaño

Este es el punto conceptual que vale la mayor parte del apartado, y la oficial lo resuelve dando las dos subredas de tamaños distintos sin explicarlo. La explicación completa:

```
El requisito es triple:
  (1) aislar la red de servidores de la de puestos de trabajo
  (2) que 70 puestos + R1 quepan
  (3) que 15 servidores + R1 quepan
  (4) DEJAR LA MAYOR CANTIDAD POSIBLE DE DIRECCIONES PARA EL FUTURO

Si las dos subredes fueran del MISMO tamaño (subnetting de
igual tamaño, que es lo que se enseñaba en 1994), las dos
tendrían que poder alojar los 70 puestos + R1 = 71:

  dos /25 → 2 × 128 = 256   → usa el /24 ENTERO, quedan 0 libres
  dos /26 → 2 ×  64 = 128   → pero 62 < 71: NO ENTRA
  dos /27 → 2 ×  32 =  64   → pero 30 < 71: NO ENTRA
  dos /24 → 2 × 256 = 512   → no existe un /24 libre dentro del /24

⇒ con subredes iguales NO HAY solución que deje direcciones
  libres. El requisito (4) obliga a subredes de TAMAÑO
  DISTINTO: /25 para los puestos y /27 para los servidores.

Eso es, literalmente, VLSM (Variable Length Subnet Masks), y
por eso este apartado es el puente entre el subnetting de
igual tamaño y la Clase 28.
```

Cuánto queda libre, y por qué es el máximo posible:

```
usadas:  128 (/25) + 32 (/27) = 160
libres:  256 − 160 = 96 direcciones

Que es el máximo, y la demostración es por bajar de tamaño:
  /24 para puestos + /26 para servidores = 256 + 64 = 320 > 256 ✗
  /25 para puestos + /28 para servidores = 128 + 16 = 144, pero
      /28 sólo da 14 hosts y necesito 16  ✗
  /25 + /27                            = 160  ✓  ← el único que entra

Los 96 libres quedan en 100.100.100.160 – 100.100.100.255,
que es lo que queda disponible para el crecimiento futuro.
```

d-4) Las modificaciones en R1 y su nueva tabla

Modificaciones necesarias en el router R1:

1. **Agregar una segunda interfaz física** (o un puerto de un switch) para conectar la **red de servidores**, porque antes toda la red A era **una sola** LAN y ahora son **dos**. R1 pasa de 1 interfaz LAN a **2**.
2. **Configurar en cada interfaz la dirección de su subred:** una en `100.100.100.x/25` (red de puestos) y otra en `100.100.100.y/27` (red de servidores), con su máscara correspondiente. **Las dos subredes pueden estar conectadas al mismo switch** si el switch es capa 2 y las dos redes están en VLANs distintas, o a puertos distintos de un router con dos interfaces.
3. **Sacar la tabla NAT.** La oficial lo dice: "*no es necesario que implemente NAT, ya que las estaciones de trabajo y los servidores tienen direcciones IP públicas*". Antes R1 traducía `192.168.1.x` (privadas) a su IP pública; ahora **todas las direcciones del bloque son públicas**, así que **el paquete pasa sin traducción** y la tabla NAT desaparece.
4. **Ajustar la ruta por defecto** si cambia la salida a Internet (en el enunciado la salida sigue siendo la interfaz `220.10.10.28/30`, así que la ruta por defecto **no cambia**).

Nueva tabla de enrutamiento de R1 (solución oficial, con mi lectura de por qué es así):

```
┌──────────────────────────────────────┬──────────────────────────┐
│ Red destino / Máscara                │ Gateway / next hop       │
├──────────────────────────────────────┼──────────────────────────┤
│ 100.100.100.0/25                     │ C – Directamente conn.   │
│ 100.100.100.128/27                   │ C – Directamente conn.   │
│ 220.10.10.28/30                      │ C – Directamente conn.   │
│ 0.0.0.0/0                            │ 220.10.10.29             │
└──────────────────────────────────────┴──────────────────────────┘

Lectura:
• Las dos primeras líneas son las DOS subredes nuevas, cada
  una directamente conectada (no hay next hop: R1 tiene una
  interfaz física en cada una).
• La tercera es el enlace con el ISP, que ya existía.
• La cuarta es la ruta por defecto hacia Internet, que NO
  cambia: el gateway sigue siendo 220.10.10.29 (la interfaz
  del router del ISP en esa /30).
• La línea 192.168.1.0/24 que estaba antes YA NO EXISTE:
  se reemplazó por la /25 de puestos. Esa es "la modificación
  necesaria en la tabla".
```

Nota mía sobre la solución oficial, y una diferencia que vale la pena discutir. La oficial **no dice qué pasa con la tabla ARP ni con el filtrado entre las dos redes**. El enunciado dice "aislar la red de servidores de los puestos de trabajo", y eso tiene una lectura más fuerte que "dos subredes distintas": **que los puestos no alcancen a los servidores y viceversa**. Con la tabla de enrutamiento sola **no se aísla nada**: R1 sabe cómo ir a las dos redes y **reenvía**, así que un puesto podría hacer ping al servidor si R1 tiene *IP forwarding* activado.

Mi lectura: para aislar de verdad, o bien (a) las dos redes están en **VLANs distintas** de un switch y hay ACLs o una política de seguridad que impide el tráfico entre ellas, o bien (b) se configura en R1 una **lista de acceso que bloquee el tráfico entre `100.100.100.128/27` y `100.100.100.0/25`**, o bien (c) se entiende "aislar" en el sentido **topológico**: que sean subredes distintas con dominios de broadcast distintos, que es lo que implica la tabla de enrutamiento.

Mi respuesta al apartado d) sería: *dos subredes de tamaños distintos, `100.100.100.0/25` para los 70 puestos + R1 (126 hosts útiles) y `100.100.100.128/27` para los 15 servidores + R1 (30 hosts útiles), dejando 96 direcciones libres; R1 necesita una segunda interfaz LAN, se le configuran las direcciones de cada subred con su máscara, se elimina la tabla NAT porque ahora todas las direcciones son públicas, se reemplaza la entrada `192.168.1.0/24` por las dos nuevas en la tabla de enrutamiento, y — si "aislar" se entiende como impedir el tráfico entre las redes — hace falta además una ACL o VLAN que separe ambas.*

> **Tope del apartado (los 30 pts del problema se reparten entre los cuatro apartados):** las dos subredes con sus cuatro valores (red, primer host, último host, broadcast) y los dos prefijos correctos; **la justificación de por qué son de tamaños distintos** (porque con iguales no queda nada libre); el cálculo de cuántas direcciones quedan libres; la interfaz adicional y el **sacar el NAT**; y la tabla nueva con las tres líneas directas y el default.

3.2 Nota de coordinación: el resto del Problema 2 de 2011-03

Los otros tres apartados del mismo problema están asignados a otras clases y **no se resuelven acá**, pero conviene saber dónde están:

Apartado	Qué es	Clase
a)	tabla de enrutamiento de R1 y del host H (con `192.168.1.0/24` y `0.0.0.0/0`)	**30** — Tablas de enrutamiento, next hop y ARP
b)	mensajes ARP con la tabla ARP vacía, y MAC/IP de tramas y paquetes en ambos sentidos	**30** — Tablas de enrutamiento, next hop y ARP
c)	qué campos cambia R1 en cada sentido (IP origen + puerto) y la tabla NAT	**31** — NAT, PAT y sus tricks
d)	las dos subredes del `/24` comprado	**esta clase**

El orden de las clases sigue el del problema, así que si estás preparando ese examen, el orden de estudio es 30 → 31 → 27. Y el enunciado completo del problema está al principio de §3.1, así que lo tenés a mano.

PARTE 4 — CIERRE

📌 Resumen

1. El **método de 7 pasos** es lo que hay que usar siempre: `2^h − 2 ≥ H` → `/n = 32 − h` → `2^(n−p)` subredes → verificar `2^h − 2 ≥ H` → `red = base + k × 2^(32−n)` → `+1`, `−2`, `−1` → decimal.
2. Se restan **2** porque la **dirección de red** identifica la subred y la de **broadcast** habla con todos. En **punto a punto** (`/31`, `RFC 3021`) **no hace falta ninguna de las dos** y las 2 direcciones son útiles.
3. **Nº de subredes =** `2^(n − p)`, sin la restricción antigua de un bit por vez. Dividir en 2 era elSubnetting clásico de antes de CIDR.
4. Los tres valores se verifican siempre: `último − primero + 1 = hosts`, `broadcast = último + 1`, `broadcast − red = 2^(32−n) − 1`.
5. La **dirección de red tiene todos los bits de host en 0**: ése es el chequeo infalible, y **tiene que empezar en un múltiplo exacto del tamaño** de la subred.
6. **El broadcast depende de la máscara.** `192.168.1.0/24` en cuatro `/26` da **cuatro** broadcasts: `.63`, `.127`, `.191`, `.255`.
7. **La subred cero y la todas-unos son válidas hoy**: con *longest prefix match* no hay ambigüedad. `RFC 950` está obsoleta.
8. `/32` = loopback o ruta de host · `/31` = punto a punto · `/30` = red mínima con router.
9. **"Al menos 6 subredes" ⇒** `/27`, porque el número de subredes es siempre una potencia de dos y hay que **redondear hacia arriba**. `/26` daría 4 subredes, menos de 6.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué la fórmula es `2^h − 2` y no `2^h`?

▶ Respuesta

2. ¿Cuándo se puede usar `2^h − 1` en vez de `2^h − 2`? Poné un ejemplo y el número.

▶ Respuesta

3. Dividí `10.0.0.0/16` en `/26`. ¿Cuántas subredes salen y cuál es la dirección de red de la subred número 5?

▶ Respuesta

4. ¿Por qué el broadcast de una subred no es "el último número de la dirección base"?

▶ Respuesta

5. "Se necesitan al menos 6 subredes de una clase C." ¿Qué prefijo, y por qué?

▶ Respuesta

6. ¿La subred cero y la subred "todas unos" son válidas? ¿Por qué antes no lo eran?

<details>
<summary>Respuesta</summary>
</details>

7. Un enlace entre dos routers tiene la subred `192.168.5.4/30`. ¿Cuáles son la red, el gateway, el host útil y el broadcast? ¿Y si en vez fuera `/31`?

<details>
<summary>Respuesta</summary>
</details>

8. En el `2011-03 P2d`: por qué 70 puestos necesitan `/25` y por qué dos subredes iguales no cumplen el requisito?

<details>
<summary>Respuesta</summary>
</details>

🔗 Referencias

- **Examen `eirc1103` / `sirc1103`** — Problema 2 (30 pts), apartado **d**: dividir `100.100.100.0/24` para 70 puestos + 15 servidores maximizando las direcciones futuras. El problema de subnetting más importante del banco.
- **Examen `eirc1003` / `sirc1003`** — Problema 1: tres `/24` que no se pueden renumerar agrupar sin tocar una red. **Examen `eirc0912` / `sirc0912`** — Problema 1: `192.168.108.0/24` + `192.168.109.0/24` sí se agregan en una `/23`.
- **Examen `eirc0608` / `sirc0608`** — P4: `200.40.224.0/25` subdividido en redes de 18, 11 y 9 hosts (VLSM). · **Examen `eirc0812` / `sirc0812`** — P2: plan de numeración + NAT + tunneling.
- **Examen `eirc1007` / `sirc1007`** — P2: subredes LAN2 y LAN3 con 250 hosts desde `192.168.2.0/23`.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red" — §5.4 el protocolo IP: la cabecera, la máscara y las subredes.
- **RFC 1878** — Requirements for IP Address Reservation (1996): **el anexo con el "número mágico"**, la tabla de hosts por prefijo que hay que usar en el papel.
- **RFC 3021** — Using 31-Bit Prefixes on IPv4 Point-to-Point Links (2000): la excepción del `/31`. ⚠️ No figura en `ACTUALIZACIONES.md`: verificá el número antes de citarlo.
- **RFC 4632** — CIDR (2006): el prefijo de longitud libre que hace posible todo esto. · **RFC 950** — Internet Subnetting (1984): el modelo viejo y la prohibición de la subred cero; **obsoleta**, verificá su estado.
- **RFC 8200** — IPv6 (2017): el prefijo `/64` como unidad de subred, y por qué en IPv6 **no hay dirección de red ni broadcast** (el cálculo es `2^(128-n) - 1`).
- **RFC 6177** — ya no recomienda asignar `/48` por sitio final; el plan de numeración moderno se elige por bloque y se subdivide con VLSM.
- **Clase siguiente:** 28 — VLSM y planes de numeración

Clase 28

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 El problema que resuelve VLSM
- 1.2 El procedimiento de VLSM, paso a paso
- 1.3 Por qué hay que alinear (la cuenta, a la vista)
- 1.4 El objetivo del examen: "maximizar para el futuro"
- 1.5 ⚠️ Actualización a 2026
- 1.6 Plan de numeración completo: casa central y dos sucursales
- 2.1 Ejercicio 1 — VLSM con cuatro subredes de tamaños distintos
- 2.2 Ejercicio 2 — *Maximally-useful packaging* desde un `/16`, y cuántas sobran
- 2.3 Ejercicio 3 — El premio de alinear: agregación de rutas
- 3.1 ⭐ Examen `2011-03`, Problema 2 d — dividir un `/24` para 70 puestos y 15 servidores
- 3.2 Dos apuntes de examen que conviene tener resueltos
- 3.3 Examen `2006-08`, Pregunta 4 (10 puntos) — dividir un `/25` en 18, 11, 9 hosts y 3 enlaces
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: D (Capa de Red) · **Semana:** 10 · **Duración estimada:** 100 min **Correlativa:** Clase 27 (subnetting paso a paso) · **Sigue con:** Clase 29 (taller de subnetting y planes de numeración) **Material de referencia:** Tanenbaum cap. 5 · RFC 1918 (privadas) 🔍 · RFC 4632 (CIDR) 🔍 · RFC 3021 (/31 P2P) 🔍 · RFC 6177 · RFC 8200

> 🔍 = RFC **no verificado** en `ACTUALIZACIONES.md`: no lo cites sin confirmarlo.

🎯 Al terminar esta clase vas a poder

1. Explicar por qué el subnetting clásico (FLSM) desperdicia direcciones.
2. Calcular el prefijo de una subred a partir de una cantidad de hosts: `2^h − 2 ≥ H`.
3. Aplicar el procedimiento de VLSM: ordenar de mayor a menor, asignar **contiguo y alineado**, llevar el puntero.
4. Demostrar **por qué** hay que alinear, con la cuenta en AND de la máscara.
5. Distinguir "maximizar el uso **hoy**" de "maximizar lo que queda **para el futuro**", y **verificar** una solución oficial en vez de copiarla.
6. Diseñar un plan de numeración completo (casa central + sucursales) con su justificación.

Como se aplica una mascara de subred, bit a bit

IP	Mascara (/26)	AND = resultado
11000000	11111111	**11000000**
10101000	11111111	**10101000**
00001010	11111111	**00001010**
00110111	11000000	**00000000**

192.168.10.55 AND 255.255.255.192 = 192.168.10.64

Lo que resulta de ese AND:

direccion de red	**192.168.10.64**
primer host	**192.168.10.65**
ultimo host	**192.168.10.126**
broadcast	**192.168.10.127**
hostables	**62**

El prefijo /26 significa:

26 bits de red + 6 bits de host

2^6 = 64 direcciones en la subred

64 - 2 = 62 hostables

El AND pone en 0 todos los bits de host: por eso el resultado es la direccion de RED, no una de host.

> **Prerequisito del examen:** `2011-03 P2d` · `2010-12 P2a-i` · `2010-07 P2a-i` · `2006-08 P4` · `2008-12 P2a-i`.

PARTE 1 — TEORÍA

1.1 El problema que resuelve VLSM

La clase 27 te dio el método: agarrás un bloque, decidís cuántas subredes querés, robás bits de host y quedás con **todas del mismo tamaño**. Eso es **FLSM** (*Fixed Length Subnet Mask*), que sirve para una sola cosa: **repartir un bloque en partes iguales**. En cuanto las partes **no** son iguales, obliga a que todas sean tan grandes como la mayor, y ahí empieza el desperdicio.

La aritmética del desastre

La empresa recibe `172.16.0.0/20` (4096 direcciones) y necesita dos LAN: **LAN-A** con 1000 puestos + 1 router = **1001** direcciones, y **LAN-B** con 100 puestos + 1 router = **101**.

Con FLSM todas las subredes deben poder alojar la más exigente, así que todas son `/22` (1024 direcciones). Y la LAN chiquita, que necesitaba 101, recibe 1024:

```
2^h − 2 ≥ 1001  →  2^10 − 2 = 1022 ≥ 1001  →  h = 10  →  prefijo = 32 − 10 = /22

LAN-A  172.16.0.0/22   1024 direcciones   (pedía 1001)   desperdicia  23
LAN-B  172.16.4.0/22   1024 direcciones   (pedía  101)   desperdicia 923
                        ────────                                  ────────
Direccionadas:          2048                          desperdicio:  946  ≈ 1000
Libre del /20:          2048
```

Ahí están **las ~1000 direcciones desperdiciadas** del curso: el 89 % de la LAN-B está sin usar.

Con VLSM cada subred toma **justo** lo que necesita:

```
LAN-A 1001 → 2^10 − 2 = 1022 ≥ 1001  →  /22   1024 dir
LAN-B  101 → 2^7  − 2 =  126 ≥  101  →  /25    128 dir
                        ────────              ──────
Direccionadas:          1152              desperdicio: 23 + 27 = 50
Libre del /20:          2944

Ahorro:  2048 − 1152 = 896 direcciones (44 % menos)
         946 − 50 = 896 menos desperdicio (95 % menos)
```

Y lo que **no** cambió: la LAN-B necesita 101 y 50 de sobra es el **piso teórico** — siempre quedan la dirección de red, la de broadcast y la diferencia hasta la potencia de 2 más cercana. VLSM te acerca a ese piso.

> **VLSM** (*Variable Length Subnet Mask*, máscara de longitud variable): método de subnetting en el que **cada subred usa un prefijo distinto**, elegido según los hosts que realmente necesita. Es **el método correcto** para maximizar el uso de direcciones.

1.2 El procedimiento de VLSM, paso a paso

```
PASO 1 — LISTAR las subredes de MAYOR a MENOR cantidad de hosts.
          Es OBLIGATORIO: asignar primero los bloques grandes es lo que
          garantiza que todos entren sin huecos.
PASO 2 — Por cada una:  2^h − 2 ≥ hosts   →   h = ceil(log2(H + 2))
          → prefijo = 32 − h.  Anotá prefijo y tamaño (2^(32 − prefijo)).
PASO 3 — ASIGNAR los bloques de mayor tamaño primero, de forma CONTIGUA y
          ALINEADA. Alineada = la dirección de red elegida es múltiplo del
          tamaño del bloque.
PASO 4 — Llevar la cuenta del PUNTERO: arranca en la dirección de red del
          bloque original y se le suma el tamaño de cada bloque.
```

El paso 1 en un ejemplo. `192.168.1.0/24` para LAN de 100 hosts, de 30, de 10, y dos enlaces punto a punto. El orden correcto es `/25 · /26 · /27 · /30 · /30`. Si el enunciado viniera en el orden `LAN 10 / enlace / LAN 30 / LAN 100` y asignás el `/28` primero, el puntero avanza 16 y el `/25` tiene que arrancar en el siguiente múltiplo de 128: como 16 no es múltiplo de 128, el `/25` se corre a `.128` y queda un hueco de **112 direcciones**.

1.3 Por qué hay que alinear (la cuenta, a la vista)

Ya asignaste `192.168.1.128/27` (32 direcciones: `.128`–`.159`) y el puntero quedó en `.160`. Necesitás un `/26`. El error fácil es arrancar en `.160`:

```
¡ERROR!  192.168.1.160/26  NO es una dirección de red válida.

/26 = 255.255.255.192 = 11111111.11111111.11111111.11000000
Los últimos 6 bits deben ser CERO en una dirección de red.

   160 = 10100000
  .192 = 11000000      ← los 6 bits bajos en cero
   AND
   160 AND 192 = 10100000 AND 11000000 = 10000000 = 128

Si numeraras una red en .160/26, en realidad estarías numerando
192.168.1.128/26, que SE SUPERPONE con el /27 que ya te comiste.
```

> **La regla:** la dirección de red de un bloque de tamaño `S` tiene que ser **múltiplo de** `S`: `dirección_de_red mod S == 0`.

La pregunta correcta es: ¿cuál es el **primer múltiplo de 64 mayor o igual que 160**? `160 / 64 = 2,5` → redondear hacia arriba → `3` → `3 × 64 = 192`.

```
192.168.1.128/27  →  .128 – .159   (32 dir)
──────────── hueco descartado: .160 – .191  (32 dir) ────────────
192.168.1.192/26  →  .192 – .255   (64 dir)
```

El puntero, con la cuenta completa

`192.168.1.0/24`, bloques de mayor a menor: `/25 (128) · /26 (64) · /27 (32) · /30 (4) · /30 (4)`

```
puntero = 192.168.1.0
/25  tamaño 128   inicio = menor múltiplo de 128 ≥ 0    = 0     →  .0/25   (  .0 – .127)
      puntero = 0 + 128 = 128
/26  tamaño  64   inicio = menor múltiplo de  64 ≥ 128  = 128   → .128/26  (.128 – .191)
      puntero = 128 + 64 = 192
/27  tamaño  32   inicio = menor múltiplo de  32 ≥ 192  = 192   → .192/27  (.192 – .223)
      puntero = 192 + 32 = 224
/30  tamaño   4   inicio = menor múltiplo de   4 ≥ 224  = 224   → .224/30  (.224 – .227)
      puntero = 224 + 4 = 228
/30  tamaño   4   inicio = menor múltiplo de   4 ≥ 228  = 228   → .228/30  (.228 – .231)
      puntero = 228 + 4 = 232

192.168.1.0/24
├─ 0     /25   LAN de 100 hosts      ├─ 224   /30   enlace P2P
├─ 128   /26   LAN de 30 hosts       ├─ 228   /30   enlace P2P
├─ 192   /27   LAN de 10 hosts       └─ 232 ──────── LIBRE (24) = 232/29 + 240/28

Verificación:  128 + 64 + 32 + 4 + 4 = 232        256 – 232 = 24  ✓
```

> **Atajo para el examen:** si los tamaños van **siempre de mayor a menor**, cada tamaño divide al anterior (256 | 128 | 64 | 32 | ... | 4), así que el puntero **siempre** queda alineado y **nunca** hay huecos. Los huecos aparecen solo cuando asignás un bloque grande **después** de uno chico, o cuando los tamaños no forman una cadena de divisores. Por eso el paso 1 es obligatorio.

1.4 El objetivo del examen: "maximizar para el futuro"

El enunciado más repetido de la cátedra, textual del `2011-03 P2d`:

> "...que permitan **dejar la mayor cantidad posible de direcciones IP para uso futuro**."

Tiene dos readings, y el professor puede estar preguntando cualquiera:

```
LECTURA A – "gastá lo mínimo hoy"
Minimizar las direcciones consumidas.  ⇔ Elegir el prefijo MÁS CORTO que
todavía alcance para los hosts.

LECTURA B – "dejá el bloque libre más limpio"
Maximizar el mayor prefijo contiguo libre.  ⇔ Asignar contiguo y alineado,
para que el remanente no quede partido en pedacitos inservibles.
```

Las dos se responden juntas, y por eso el procedimiento es el mismo. Lo que cambia es el **orden de prioridad**: entre un plan que gasta menos hoy pero parte el remanente y otro que gasta un poco más pero deja un bloque limpio, la lectura B prefiere el segundo. En los ejercicios de la cátedra casi siempre coinciden: **decí las dos cosas**.

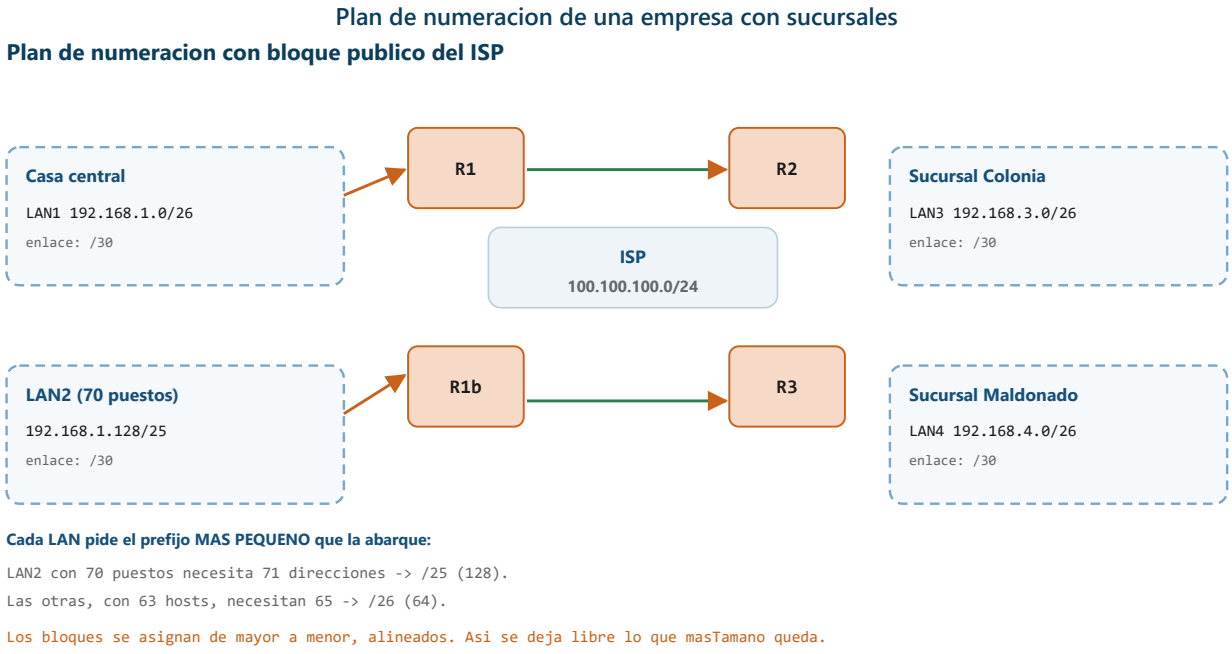
1.5 ⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - `2^h – 2 ≥ H` y la mecánica de robar bits de host. - Los tres rangos privados: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. **Cuidado:** `/12` no es `/16` — de `172.16.0.0` a `172.31.255.255` son 16 bloques. Es el error de examen más frecuente de la historia. - Uso restringido, no

ruteable: `127.0.0.0/8` (loopback), `0.0.0.0/8`, `169.254.0.0/16` (link-local). Y la aritmética de alineación: **intacta**.

Cambió, o hay que reencuadrar: - **El `/48` por sitio final ya no se recomienda.** `RFC 6177` (2011) dejó de recomendar `/48` a cada sitio final; hoy los RIR asignan `/48` (a veces `/56`) **al operador** y el operador reparte. La aritmética no cambia; la justificación, sí. - **En IPv6 el problema desaparece:** el `/64` es la unidad de subred estándar (con SLAAC), así que VLSM en IPv6 es trivial. **VLSM es exclusivo de IPv4**, y decirlo en un examen es un punto. - **El `/20` del ejemplo ya no se compra.** RIPE NCC hizo su última asignación `/22` el **25 de noviembre de 2019**; hoy un `/20` se consigue por transferencia de mercado, no por asignación. - **El caso de uso actual es la red virtual privada del proveedor** (*VPC*, *overlay*): los rangos privados **se solapan entre inquilinos**, así que dos redes con `192.168.1.0/24` no se conectan sin traducción. Y en la casa no hay nada que subnetear: el CGNAT del ISP mete al cliente detrás de `100.64.0.0/10`.

> **Consecuencia para el examen:** la aritmética y el procedimiento **no se tocan nada**. Lo único a actualizar es la *justificación*: el bloque vino por transferencia y los rangos privados se solapan entre redes virtuales.



1.6 Plan de numeración completo: casa central y dos sucursales

Datos. Bloque público `220.10.10.0/26` (64 direcciones) para enlaces WAN y *pool* de traducción; hosts internos en `192.168.0.0/16`. Necesidades: LAN-CC-usuarios 250 hosts, LAN-S2 100, LAN-S1 50, LAN-CC-servidores 30 — **más 1 dirección de router en cada una**.

Paso 1 y 2 — ordenar (de mayor a menor) y elegir prefijo.

```
251 → 2^8 − 2 = 254 ≥ 251 ✓              →  h = 8  →  /24  (512 dir)
101 → 2^7 − 2 = 126 ≥ 101 ✓              →  h = 7  →  /25  (256 dir)
 51 → 2^6 − 2 =  62 ≥  51 ✓              →  h = 6  →  /26  (128 dir)
 31 → 2^5 − 2 =  30 ≥  31  ✗  NO ALCANZA
      2^6 − 2 =  62 ≥  31 ✓              →  h = 6  →  /26  (128 dir)
```

⚠️ El `+2` (red + broadcast) va **dentro** del `2^h`, no afuera: por eso "31 direcciones → un `/27` de 30 utilizables" no alcanza y hay que subir a `/26`.

Paso 3 — asignación contigua y alineada en `192.168.0.0/16`.

```
/24  tamaño 256   inicio = 0 (múltiplo de 256 ✓)   →  192.168.1.0/24    puntero → 192.168.2.0
/25  tamaño 128   inicio = 256 ✓  (256/128 = 2)    →  192.168.2.0/25    puntero → 192.168.2.128
/26  tamaño  64   inicio = 384 ✓  (384/64 = 6)     →  192.168.2.128/26  puntero → 192.168.2.192
/26  tamaño  64   inicio = 448 ✓  (448/64 = 7)     →  192.168.2.192/26  puntero → 192.168.3.0

Consumo: 512 + 256 + 128 + 128 = 1024 de 65536.  Libre: 64512.
Mayor prefijo libre contiguo: 192.168.3.0/22 (4096 direcciones).
Gateways:  LAN-CC-usuarios .1.1 · LAN-S2 .2.1 · LAN-S1 .2.129 · LAN-CC-serv .2.193
```

Paso 3' — los enlaces públicos en `220.10.10.0/26`. Todos `/30`: los arranques 0, 4, 8, 12, 16 y 20 son múltiplos de 4, sin huecos.

```
220.10.10.0/30    Casa central ↔ ISP        casa .2    ISP .1
220.10.10.4/30    Casa central ↔ Sucursal 1 casa .5
220.10.10.8/30    Casa central ↔ Sucursal 2 casa .9
220.10.10.12/30   Pool de traducción NAT     R1 usa .13
220.10.10.16/30   Sucursal 1 ↔ ISP          sucursal .18   ISP .17
220.10.10.20/30   Sucursal 2 ↔ ISP          sucursal .22   ISP .21

Consumido 6 × 4 = 24 de 64.  Libre: .24 – .63 = 24/29 (8) + 32/27 (32) = 40.
```

Justificación de cada decisión.

Decisión	Por qué
Privado para hosts, público para enlaces	Los hosts nunca son alcanzables desde Internet: gastar dirección pública ahí es tirar el recurso. Los enlaces WAN sí: son lo que compra el ISP.
VLSM en vez de FLSM	Con FLSM las cuatro LAN serían `/24`: `4 × 512 = 2048` para 433 que necesito. Con VLSM: 1024.
`/30` para punto a punto	Un enlace entre routers necesita 2 direcciones; `/30` = 4 (2 útiles + red + broadcast). Existe `/31` para punto a punto 🔍 (creo que RFC 3021: **verificá**); en FING la respuesta esperada es `/30`.
Contiguo y alineado	Permite no perder direcciones **y** agregar rutas (§2.3).
R1 con dirección en cada LAN	Sin ella no hay a quién mandarle lo que no es local.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — VLSM con cuatro subredes de tamaños distintos

Enunciado. Del bloque privado `172.20.0.0/16`: LAN de ventas con **300** hosts, de producción con **120**, de contabilidad con **60**, y un enlace punto a punto con el Data Center. **(a)** Prefijos. **(b)** Asignación alineada. **(c)** Sobrantes. **(d)** Mayor prefijo libre contiguo.

Paso 1 — ordenar (de mayor a menor): 301 → 121 → 61 → 2. **Paso 2 — prefijo de cada una.**

```
301:  2^8 − 2 = 254 < 301 ✗   2^9 − 2 = 510 ≥ 301 ✓  →  h = 9  →  /23  (512 dir)
121:  2^7 − 2 = 126 ≥ 121 ✓                          →  h = 7  →  /25  (128 dir)
 61:  2^6 − 2 =  62 ≥  61 ✓                          →  h = 6  →  /26  ( 64 dir)
  2:  2^2 − 2 =   2 ≥   2 ✓                          →  h = 2  →  /30  (  4 dir)
```

Interpretación: cada prefijo es el **menor** que todavía alcanza; un `/24` para ventas habría servido (510 ≥ 301) pero habría tirado 211 direcciones de más.

Paso 3 — asignación con el puntero, verificando la alineación bloque por bloque (esto separa una respuesta de 4 puntos de una de 10):

```
puntero = 172.20.0.0
/23  inicio = 0 ✓                    →  172.20.0.0/23    puntero → 172.20.2.0
/25  inicio = 512 ✓  (512/128 = 4)   →  172.20.2.0/25    puntero → 172.20.2.128
/26  inicio = 640 ✓  (640/64 = 10)   →  172.20.2.128/26  puntero → 172.20.2.192
/30  inicio = 704 ✓  (704/4 = 176)   →  172.20.2.192/30  puntero → 172.20.2.196

172.20.0.0/16
├─ .0.0      /23   ventas        (512 dir, .0.0 – .1.255)
├─ .2.0      /25   producción    (128 dir, .2.0 – .2.127)
├─ .2.128    /26   contabilidad  ( 64 dir, .2.128 – .2.191)
├─ .2.192    /30   enlace P2P    (  4 dir, .2.192 – .2.195)
└─ .2.196 ──────────── LIBRE
```

(c) Direcciones sobrantes.

```
Consumidas:      512 + 128 + 64 + 4 = 708        Totales del /16: 65 536
Libres:          65 536 − 708 = 64 828      (98,9 % del bloque)
Desperdicio DENTRO de los bloques:
  /23: 510 util − 301 usadas = 209    /25: 126 util − 121 usadas =  5
  /26:  62 util −  61 usadas =   1    /30:   2 util −   2 usadas =  0  → 215
Las otras 708 − 215 = 493 son red + broadcast de las 4 redes.
```

(d) Mayor prefijo libre contiguo. El libre arranca en `172.20.2.196` (offset 708 dentro del /16). Para que un prefijo sea válido, ese offset tiene que ser múltiplo del tamaño, y no lo es para ningún tamaño ≥ 8 (`708/8 = 88,5`, `708/16 = 44,25`, `708/32 = 22,125`, `708/64 = 11,06`, `708/128 = 5,53`, `708/256 = 2,77`, `708/512 = 1,38` — ninguno entero). El primer arranque grande es `172.20.4.0` (offset 1024), y de ahí se encadenan `.4.0/23`, `.6.0/22`, `.10.0/21`, `.18.0/20`, `.34.0/19`, `.66.0/18`, hasta **`172.20.128.0/17`** (32768 dir), que es el mayor.

> Lo que **demuestra** que lo sabés no es el número grande: es la alineación de `172.20.128.0/17`. Un `/17` dentro de un `/16` ocupa `2^(24−17) = 128` unidades del tercer octeto, así que el tercer octeto tiene que

> ser `0` o `128`.

⚠️ **Errores típicos:** sumar el router dos veces (`+1` por red, aunque sea el mismo router); usar `2^h ≥ H` en vez de `2^h − 2 ≥ H`; y redondear `2^h − 2` hacia arriba cuando queda corto (caso de 31 direcciones).

2.2 Ejercicio 2 — *Maximally-useful packaging* desde un `/16`, y cuántas sobran

Enunciado. De `10.0.0.0/16` sacar **8 LAN de 1000 hosts**, **100 LAN de 100 hosts**, **100 LAN de 10 hosts** y **6 enlaces punto a punto**. **(a)** ¿Alcanza? **(b)** Plan. **(c)** Sobrantes y desperdicio.

(a) La cuenta de checksum va primero, porque si no alcanzara hay que subir prefijos y cambiar todo el plan. Prefijos: 1000 → `/22`, 100 → `/25`, 10 → `/28`, enlace → `/30`.

```
  8 LAN × 1024 dir  =  8 192
100 LAN ×  128 dir  = 12 800
100 LAN ×   16 dir  =  1 600
  6 enlaces ×   4   =     24
                      ________

 TOTAL                        22 616  ≤  65 536   ✓  sobra 42 920
```

(b) Plan. Por el atajo de §1.3 no hay huecos: cada tamaño divide al anterior.

```
puntero = 10.0.0.0
8 × /22   =  8192 dir = 32 unidades de tercer octeto
           → 10.0.0.0 – 10.0.31.255      puntero = 10.0.32.0
100 × /25 = 12 800 dir = 50 unidades
           → 10.0.32.0 – 10.0.81.255    puntero = 10.0.82.0
100 × /28 =  1600 dir = 6 unidades + 64
           → 10.0.82.0 – 10.0.88.63     puntero = 10.0.88.64
6 × /30   =    24 dir
           → 10.0.88.64 – 10.0.88.87   puntero = 10.0.88.88
LIBRE: 42 920 dir

Alineación (offset dentro del /16 ÷ tamaño del bloque):
  10.0.32.0   →   8192 / 128 =   64 ✓      10.0.82.0  → 20992 / 16 = 1312 ✓
  10.0.88.64  →  22592 /   4 = 5648 ✓
```

(c) Sobrantes y desperdicio.

```
Libres: 65 536 – 22 616 = 42 920   (65,5 % del bloque)
Desperdicio dentro de los bloques:
   8 × (1022 – 1001) =    168
 100 × ( 126 –  101) =  2 500
 100 × (  14 –   11) =    300
   6 × (   2 –    2) =      0          →  2 968
Red + broadcast: 214 subredes × 2 = 428
Total no utilizable = 3 396
Eficiencia = (65 536 – 3 396) / 65 536 = 94,8 %
```

Interpretación: el 5,2 % que "se pierde" es casi todo red + broadcast (obligatorio) y el redondeo a potencia de 2 (inevitable). Con FLSM, las 214 redes tendrían que ser `/22` = 214 × 1024 = **219 136** direcciones, que no entran ni de lejos en el `/16`. **VLSM no es una optimización: es la diferencia entre que el plan exista y que no.**

2.3 Ejercicio 3 — El premio de alinear: agregación de rutas

Este es el motivo **real** por el que la alineación importa, y es lo que conecta esta clase con la de tablas de enrutamiento.

Regla. Dos o más prefijos se pueden resumir en uno solo si y solo si: son **contiguos**, la cantidad total de direcciones es **potencia de 2**, y el inicio del primero está **alineado** a ese tamaño.

```
.0/25 + .128/25  = 192.168.0.0/24  ✓   .0/24 + .1/24 = 192.168.0.0/23  ✓
192.168.1.0/24 + 192.168.2.0/24  →  ¡se crearían 512 = un /23!
   ¿Dónde arrancaría? En un múltiplo de 512 del tercer octeto:
      0 → 192.168.0.0/23  ✗ incluye 192.168.0.0/24, que no es nuestra
      2 → 192.168.2.0/23  ✗ incluye 192.168.3.0/24, que no es nuestra
   En AND: 1 = 00000001 AND /23 = 11111110 → 00000000 = 0
   → el resumen "natural" es 192.168.0.0/23, que INCLUYE una red ajena.
```

Conclusión: dos redes contiguas de 256 **no** se pueden resumir. Por eso en el plan de §1.6 puse las cuatro LAN alineadas a partir de `192.168.1.0`: así `192.168.2.0/25` + `.128/26` + `.192/26` se resumen en `192.168.2.0/24` (1 ruta en vez de 3), y `192.168.1.0/24` + `192.168.2.0/24` en `192.168.0.0/23` (1 ruta en vez de 2). **La agregación de rutas es un criterio de diseño del plan**, no una consecuencia: es lo que dice la oficial del `2010-12 P2a-i` (§3.2.1) al elegir el plan que deja `LAN3 + LAN4` sumarizables en un solo `/24`.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen `2011-03`, Problema 2 d — dividir un `/24` para 70 puestos y 15 servidores

> **Fuente:** Examen `eirc1103` / `sirc1103`, 21 de febrero de 2011, Problema 2 (30 puntos), apartado d. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

Enunciado (literal)

> Se adquiere para la Red A el bloque de direcciones IP públicas `100.100.100.0/24`, y los administradores deciden aislar la red de servidores de los puestos de trabajo. Especifique las subredes del bloque mencionado que cumplan con este requerimiento y permitan dejar la mayor cantidad posible de direcciones IP para uso futuro. Mencione las modificaciones necesarias en el router R1 y su nueva tabla de enrutamiento.

Contexto. Red A: LAN Ethernet privada con **15 servidores** y **70 puestos**; R1 hace NAT con `220.10.10.30` (enlace `220.10.10.28/30`) para llegar a `219.41.31.21`. Ahora compran `100.100.100.0/24` — **público** — y quieren **aislar** servidores de puestos.

Solución oficial

> **Subred para 70 puestos de trabajo:** 70 direcciones IP + 1 para el router R1 (Además siempre se deben considerar la direcciones de red y de broadcast) → `100.100.100.0/25` **Subred para 15 servidores:** 15 direcciones IP + 1 para el router R1 → `100.100.100.128/27` El router R1 se conecta a cada subred y no es necesario que implemente NAT, ya que las estaciones de trabajo y los servidores tienen direcciones IP públicas. **Nueva tabla de enrutamiento de R1:** `100.100.100.0/25` → C · `100.100.100.128/27` → C · `220.10.10.28/30` → C · `0.0.0.0/0` → `220.10.10.29`.

Mi resolución, con la aritmética completa

Paso 1 — ordenar: 70 puestos, después 15 servidores. **Paso 2 — los prefijos.**

```
PUESTOS:    70 hosts + 1 (gateway R1) = 71
   2^6 − 2 =  62   <  71    ✗   NO ALCANZA
   2^7 − 2 = 126   ≥  71    ✓   →  h = 7  →  /25  (128 dir, 126 utilizables)

SERVIDORES: 15 hosts + 1 (gateway R1) = 16
   2^4 − 2 =  14   <  16    ✗   NO ALCANZA   ← ¡el error más tentador!
   2^5 − 2 =  30   ≥  16    ✓   →  h = 5  →  /27  ( 32 dir,  30 utilizables)
```

⚠️ **El error que casi todos cometen** es hacer `16 → 2^4 → /28`, pensando que 16 es potencia de 2. Pero `2^4 = 16` es el total **incluyendo** red y broadcast: los utilizables son 14, y necesito 16. El `+2` va **adentro**.

Paso 3 — asignación contigua y alineada. `puntero = 100.100.100.0`

```
/25  tamaño 128   inicio = menor múltiplo de 128 ≥ 0    = 0     → 100.100.100.0/25
      (0 es múltiplo de 128 ✓)                              (.0 – .127)   puntero → .128
/27  tamaño  32   inicio = menor múltiplo de  32 ≥ 128  = 128   → 100.100.100.128/27
      (128 es múltiplo de 32 ✓)                             (.128 – .159) puntero → .160

100.100.100.0/24
├─ .0   /25   PUESTOS     red .0   | bc .127 | hosts .1   – .126 | gw .1
├─ .128 /27   SERVIDORES  red .128 | bc .159 | hosts .129 – .158 | gw .129
└─ .160 ────────── LIBRE (96 dir) = 100.100.100.160/26 (64) + 100.100.100.224/27 (32)
```

Modificaciones necesarias en R1.

1. **Dos interfaces** (o dos sub-interfaces de VLAN) en vez de una: una por subred, porque ahora están **separadas**.
2. **Se elimina la traducción NAT.** Los hosts ya son alcanzables por dirección pública; traducirlos "sobre-vendería" las públicas que se acaban de comprar para eso. La solución oficial lo dice textualmente.
3. **La ruta por defecto no cambia:** `0.0.0.0/0` → `220.10.10.29` por `220.10.10.28/30`, y cada subred **conserva su gateway** (`100.100.100.1` y `100.100.100.129`).
4. **El matiz que vale puntos:** el enunciado dice "aislar la red de servidores de los puestos", y dos subredes distintas **no** aíslan nada por sí solas — R1 reenvía de una a otra salvo que filtre. Lo que se entiende es aislamiento de **dominio de broadcast y de numeración**, no cortafuegos; si se hubiera pedido aislamiento de seguridad, la respuesta sería ACL o cortafuegos.

La aritmética que hay que mostrar: ¿esto **maximiza** el uso?

```
┌──────────────────────┬──────────┬──────────┬──────────┬──────────────┐
│ Plan                 │ Puestos  │ Servidor │ Total    │ Libre del /24│
├──────────────────────┼──────────┼──────────┼──────────┼──────────────┤
│ OFICIAL   /25 + /27  │   128    │    32    │   160    │  96 (37,5 %) │
│ "Cómodo"  /25 + /26  │   128    │    64    │   192    │  64 (25,0 %) │
│ "Apretado"/25 + /28  │   128    │ 14 < 16  │ inválido │  no sirve    │
└──────────────────────┴──────────┴──────────┴──────────┴──────────────┘
Desperdicio interno:  oficial (126-71)+(30-16) = 69 ;  "cómoda" (126-71)+(62-16) = 101
```

La conclusión honesta:

> **Nota: la solución oficial dice `/25` y `/27`. Verifiqué la cuenta y es correcta — y además es la *única* solución posible.** Los dos prefijos están forzados: `/26` no alcanza para 71 utilizables y `/28` no alcanza para 16. No hay margen de decisión en los prefijos, solo en el **orden** de asignación.
>
> **Mi lectura difiere en el enfoque, y es lo que hace interesante el enunciado.** "Dejar la mayor cantidad posible para uso futuro" tiene dos readings (§1.4). La oficial satisface la A (gasta 160 de 256, el mínimo posible) **y también** la B: el remanente `100.100.100.160` – `100.100.100.255` son **96 direcciones contiguas que se descomponen limpiamente en `100.100.100.160/26` + `100.100.100.224/27`**, justo lo que hace falta para dos LANs futuras de hasta 62 y de hasta 30 hosts. Con `/26` para los servidores, en cambio, el libre quedaría partido en `.160/28` + `.176/27` + `.208/27` = 8 + 32 + 32: tres bloques distintos, mucho menos útil.
>
> O sea: **lo que suma puntos no es "la oficial está bien", sino "la oficial es correcta porque el mínimo de cada prefijo la obliga, y además produce un remanente alineado que sirve para el segundo**

> **reading del objetivo"**.

⚠️ **Errores típicos:** `/28` para los servidores; no contar el router (`+1` en cada subred); dejar el NAT puesto; olvidar la entrada `0.0.0.0/0` en la nueva tabla (sin ella los hosts de las subredes nuevas no salen a Internet, que era el objetivo de la red A).

3.2 Dos apuntes de examen que conviene tener resueltos

3.2.1 `147.10.8.0/21` con hasta 120 hosts por LAN

> **Fuente:** Examen 2008-07 (`sirc0807`, Problema 2 a-i); el banco lo indexa también bajo `sirc1007`. Se resuelve **entero** en la Clase 29.

```
Enunciado: "Realice un plan de numeración de las subredes LAN3, 4 y 5
utilizando 147.10.8.0/21, asumiendo que no se conectan más de 120 hosts por
subred. ¿Cuántas subredes se pueden numerar?"

Solución oficial: "Para cada subred se necesitan como máximo 120 + 1 + 2 = 123
direcciones IP por LAN."  →  2^6 = 64 < 125 ✗  ;  2^7 = 128 ≥ 125 ✓  →  /25
¿Cuántas subredes?  2^(32-21) = 2048 dir ; 2048 / 128 = 16    [o: 11 − 7 = 4 bits → 2^4]

La oficial da dos planes posibles y NO son equivalentes:
  PLAN A:  LAN3 147.10.8.0/25 · LAN4 147.10.8.128/25 · LAN5 147.10.9.0/25
           LAN3 + LAN4 = 147.10.8.0/24  ← SUMARIZA (0 es múltiplo de 256)
           ⇒ R2 necesita 2 rutas hacia la empresa.
  PLAN B:  LAN3 147.10.8.0/25 · LAN4 147.10.9.0/25 · LAN5 147.10.10.0/25
           NO se puede sumarizar: resumir LAN4 (.9.0–.9.127) y LAN5
           (.10.0–.10.127) exige arrancar en .8.0 o en .10.0, y .9.0 no es
           ninguno de los dos.  ⇒ R2 necesita 3 rutas.
```

> **El `+2` de la oficial:** 120 hosts + 1 router + 2 de **reserva** para crecimiento inmediato de cada LAN (no red ni broadcast, que ya están dentro del `2^h`). Hay que decir de dónde sale, porque si no parece un error de suma.
>
> **Conclusión para el examen:** "*la solución no es única, pero no todas las soluciones son equivalentes: el plan tiene que optimizar no solo el uso de direcciones sino también la cantidad de rutas*" — y eso se decide en el paso 3 (asignación alineada), no en el paso 2 (elegir prefijo).

3.2.2 `192.168.2.0/23` con hasta 250 hosts por subred

> **Fuente:** Examen 2010-07 (`sirc1007`, Problema 2 a-i), resuelto **entero** en la Clase 29 (Ejercicio 4).

```
250 hosts + 1 router = 251    + red y broadcast  ⇒  el bloque necesita ≥ 253
2^7 = 128 < 253 ✗  (/25 NO alcanza — 128, no 256)
2^8 = 256 ≥ 253 ✓  →  h = 8  →  /24   (254 utilizables, sobran 3)

¿Cuántas subredes entran en el /23?
   Por bits:   /23 deja 9 bits de subred; /24 usa 8; quedan 1  →  2^1 = 2
   Por tamaño: 2^(32-23) = 512 direcciones ; 512 / 256 = 2  ✓
Asignación:  192.168.2.0/24 (LAN2, gw .2.1) · 192.168.3.0/24 (LAN3, gw .3.1)
Cuentas:     512 consumidas de 512  →  LIBRE: 0
             no utilizable = 2 × 2 (red+bc) + 2 × 3 (sobrantes) = 10  (1,95 %)
```

El comentario de diseño que vale puntos: el bloque queda **al 100 % de ocupación**; si la empresa supiera que iba a necesitar una tercera sucursal debería haber pedido un `/22`, pero el enunciado no lo dice y la respuesta correcta es usar el `/23` entero.

3.3 Examen `2006-08`, Pregunta 4 (10 puntos) — dividir un `/25` en 18, 11, 9 hosts y 3 enlaces

> **Fuente:** Examen 5 de agosto de 2006, `sirc0608`, Pregunta 4: un solo bloque, cinco necesidades de tamaños distintos y la pregunta por las redes libres. Se resuelve **entero** en la Clase 29; acá va la aritmética:

```
Bloque:  200.40.224.0/25  →  32 − 25 = 7 bits de host
         Total 2^7 = 128 direcciones  ·  utilizables 128 − 2 = 126

18 hosts → 20 dir ;  2^4 = 16 < 20 ✗ ;  2^5 = 32 ≥ 20 ✓  →  /27 (32 dir)
11 hosts → 13 dir ;  2^4 = 16 ≥ 13 ✓                     →  /28 (16 dir)
 9 hosts → 11 dir ;  2^3 =  8 < 11 ✗ ;  2^4 = 16 ≥ 11 ✓  →  /28 (16 dir)
enlace  →  4 dir ;  2^2 =  4 ≥  4 ✓                     →  /30 ( 4 dir)

Plan oficial:  .0/27  ·  .32/28  ·  .48/28  ·  .64/30  ·  .68/30  ·  .72/30
Consumido:     32 + 16 + 16 + 4 + 4 + 4 = 76    →  libre 128 − 76 = 52
Oficial: 7 bloques libres = .76/30 (4) + seis /29 de 8 = 4 + 48 = 52
Mía:     4 bloques = .76/30 + .80/28 + .96/28 + .112/28 = 4 + 16×3 = 52
   (misma superficie, menos entradas de tabla)
⚠️  .80/27 es IMPOSIBLE: 80/32 = 2,5 no es entero. Esa es la trampa.
```

PARTE 4 — CIERRE

📌 Resumen

1. **FLSM** obliga a que todas las subredes tengan el tamaño de la mayor; **VLSM** le da a cada una lo que necesita.
2. Con FLSM, 1000 + 100 hosts desde un `/20` desperdicia **946** direcciones; con VLSM, **50**.
3. Procedimiento: **ordenar de mayor a menor** → `2^h − 2 ≥ H` → **asignar contiguo y alineado** → **llevar el puntero**.
4. **Alineado** es `dir_de_red mod tamaño == 0`; sin eso los bloques se superponen: `.160/26` es en realidad `.128/26`.
5. De mayor a menor cada tamaño divide al anterior, así que **no hay huecos**.
6. "Dejar lo máximo para el futuro" tiene dos readings — **gastar menos hoy** y **dejar el remanente más limpio** — y se responden juntos.
7. `/25 + /27` para 70 puestos y 15 servidores (`2011-03 d`): la oficial es correcta **y es la única posible**; remanente de 96 = `/26 + /27`.
8. `147.10.8.0/21` con 123 por LAN → `/25` → **16 subredes**; el plan decide si R2 tiene **2** rutas o **3**.
9. `192.168.2.0/23` con 251 por LAN → `/24` → **2 subredes exactas**, bloque al 100 %.
10. `200.40.224.0/25` con 18+11+9 y 3 enlaces → `/27 + /28 + /28 + 3×/30`, libres 52 direcciones.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué ordenar "de mayor a menor" es obligatorio y no cosmético?

<details>
<summary>Respuesta</summary>
</details>

2. Alguien asigna `192.168.1.160/26`. Demostrá con la cuenta que no es una dirección de red válida.

<details>
<summary>Respuesta</summary>
</details>

3. 70 puestos + 1 router y 15 servidores + 1 router, desde un `/24`: ¿qué prefijos salen y por qué no `/28` para los servidores?

<details>
<summary>Respuesta</summary>
</details>

4. De un `/20` (4096 dir) se necesitan LANs de 1000 y de 100 hosts, más un router en cada una. ¿Cuánto desperdicia FLSM y cuánto VLSM?

<details>
<summary>Respuesta</summary>
</details>

5. ¿Cuántas subredes `/25` entran en `147.10.8.0/21`? Las dos cuentas.

<details>
<summary>Respuesta</summary>
</details>

6. `192.168.1.0/24` y `192.168.2.0/24` son contiguas y suman 512 direcciones. ¿Se pueden resumir?

<details>
<summary>Respuesta</summary>
</details>

7. En el plan de la empresa, las cuatro LAN quedaron en `192.168.1.0/24`, `192.168.2.0/25`, `192.168.2.128/26` y `192.168.2.192/26`. ¿Cuántas rutas ahorra esa alineación frente a un plan "desalineado" de `.1.0/24`, `.3.0/24`, `.4.0/24`, `.5.0/24`?

<details>
<summary>Respuesta</summary>
</details>

8. En `2010-12 P2a-i`, ¿por qué la oficial prefiere `LAN3 .8.0/25 · LAN4 .8.128/25 · LAN5 .9.0/25` y no `LAN3 .8.0/25 · LAN4 .9.0/25 · LAN5 .10.0/25`?

<details>
<summary>Respuesta</summary>
</details>

🔗 Referencias

- **`eirc1103` / `sirc1103`** (21-feb-2011) — **Problema 2 d**: dividir `100.100.100.0/24` para 70 puestos + 15 servidores (§3.1) · P2 a-c son las Clases 30 y 31
- **`sirc0807`** (1-ago-2008) — P2 a-i: `147.10.8.0/21` con 120 hosts por LAN (§3.2.1)
- **`sirc1007`** (27-jul-2010) — P2 a-i: `192.168.2.0/23` con 250 hosts (§3.2.2)
- **`sirc0608`** (5-ago-2006) — P4: `200.40.224.0/25` en 18, 11 y 9 hosts + 3 enlaces (§3.3) · P2: por qué ARP (Clase 30)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red"
- **RFC 1918** (privadas) 🔍 · **RFC 4632** (CIDR) 🔍 · **RFC 3021** (/31 P2P) 🔍 · **RFC 6177** (2011) · **RFC 8200** (2017)
- **`ACTUALIZACIONES.md` §5** — agotamiento del IPv4 (RIPE NCC, 25-nov-2019) y CGNAT
- **Clase siguiente:** 29 — Taller de subnetting y planes de numeración

Clase 29

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 2.1 Ejercicio 1 — `192.168.10.0/24` en cuatro subredes iguales
- 2.2 Ejercicio 2 — VLSM de `172.16.0.0/16` para 500, 250, 100, 50 y 25 hosts
- 2.3 ⭐ Ejercicio 3 (de examen) — `2011-03` Problema 2 d
- 2.4 ⭐ Ejercicio 4 (de examen) — `2010-07` Problema 2 a-i
- 2.5 ⭐ Ejercicio 5 (de examen) — `2010-12` Problema 2 a-i
- 2.6 ⭐ Ejercicio 6 (de examen) — `2006-08` Problema 1: seis routers
- 3.1 Ejercicio 7 — VLSM con cinco redes y tres enlaces
- 3.2 Ejercicio 8 — ¿Cuántas direcciones desperdicia cada plan?
- 3.3 Ejercicio 9 — Renumerar para meter un switch (y cuándo NO hace falta)
- 3.4 Ejercicio 10 — ¿Se puede sumarizar?
- 3.5 Ejercicio 11 — El puntero que cambia de paso
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: D (Capa de Red) · **Semana:** 10 · **Duración estimada:** 120 min **Correlativa:** Clase 28 (VLSM y planes de numeración) · **Sigue con:** Clase 30 (tablas de enrutamiento, next hop y ARP) **Material de referencia:** Tanenbaum cap. 5 · RFC 1918 (privadas) 🔍 · RFC 4632 (CIDR) 🔍 · RFC 6177 · RFC 8200

> 🔍 = RFC **no verificado** en `ACTUALIZACIONES.md`: no lo cites sin confirmarlo. Esta clase es **de taller**: la única teoría nueva es el recuadro de fórmulas.

🎯 Al terminar esta clase vas a poder

1. Escribir de memoria las cinco fórmulas del subnetting.
2. Dividir un bloque en subredes iguales con el **prefijo más largo** que alcance.
3. Aplicar VLSM a un `/16` con cinco necesidades distintas, con la aritmética a la vista.
4. Resolver **entero** los cuatro problemas de plan de numeración que más se repiten en FING.
5. Reconstruir una topología de 6 routers desde los datos de una tabla y sacar sus tablas de enrutamiento.
6. Decidir si un conjunto de prefijos se puede **sumarizar**, y hasta dónde.

> **Prerequisito del examen:** `2011-03 Problema 2d` · `2010-12 P2a-i` · `2010-07 P2a-i` · `2006-08 Problema 1` · `2006-08 P4` · `2008-12 P2a-i`.

PARTE 1 — EL RECUADRO QUE HAY QUE COPIAR AL EXAMEN

```
╔═══════════════════════════════════════════════════════════════════════╗
║  hosts necesarios H   →   2^h − 2 ≥ H   →   h = ceil(log2(H + 2))     ║
║  →  prefijo = 32 − h                                                  ║
║                                                                       ║
║  cantidad de subredes  =  2^(bits de host que quedan)                 ║
║  tamaño del bloque     =  2^(32 − prefijo)                            ║
║  dir. de red           =  (dir / tamaño) · tamaño     (división entera) ║
║  broadcast             =  dir de red + tamaño − 1                     ║
║  rango de hosts        =  [ dir de red + 1 ,  broadcast − 1 ]         ║
║  puntero de VLSM       =  primer múltiplo de "tamaño" ≥ puntero actual ║
╚═══════════════════════════════════════════════════════════════════════╝
```

Tres aclaraciones, porque son las tres trampas del examen:

1. **El `− 2` es de red y broadcast**, no un margen de seguridad: `/27` son **30** utilizables, y por eso 16 direcciones pedidas **no** entran en un `/27`.
2. **`2^(32 − prefijo)` es el tamaño del bloque, no el de los hosts.** `/25` son 128 direcciones totales; de ahí el error de creer que "alcanza para 250 hosts".
3. **El puntero se redondea hacia arriba al siguiente múltiplo del tamaño.** Si no sos múltiplo, perdés todo el tramo hasta el múltiplo siguiente. Por eso el paso 1 (ordenar de mayor a menor) es obligatorio.

```
EJEMPLO DE REDONDEO DEL PUNTERO
puntero = 192.168.1.160,  bloque pedido /26 (tamaño 64)
  160 / 64 = 2,5   →   no es múltiplo   →   siguiente = 3 × 64 = 192
  Se pierden 32 direcciones (.160 – .191). Con un bloque /27 (32) no se pierde nada.
```

PARTE 2 — EJERCICIOS DEL TALLER

2.1 Ejercicio 1 — `192.168.10.0/24` en cuatro subredes iguales

Enunciado. Dividí `192.168.10.0/24` en **cuatro** subredes iguales. Decidí entre `/26` y `/27` y justificá.

Paso 1 — cuántas subredes necesito. 4 = `2^2`, o sea **2 bits** de la parte de host hay que convertir en bits de subred.

Paso 2 — el prefijo. El `/24` tiene 8 bits de host. Robamos 2:

```
prefijo = 24 + 2 = /26      (equivalente: h = 6 host bits, prefijo = 32 − 6 = 26)
Subredes:  2^2 = 4  ✓        Tamaño: 2^(32−26) = 2^6 = 64 direcciones
Utilizables por subred: 64 − 2 = 62      Total utilizable: 4 × 62 = 248
Chequeo: 4 × 64 = 256 = el /24 entero  ✓
```

Paso 3 — ¿y `/27`? Un `/27` son `2^3 = 8` subredes de 32 direcciones (30 utilizables cada una). O sea: `8 × 30 = 240` utilizables, repartidos en **ocho** subredes en vez de cuatro.

```
┌──────────┬─────────┬──────────────┬──────────────┬──────────────────────┐
│ Opción   │ Subred  │ Tam. subred  │ Utilizables  │ ¿Sirve?              │
├──────────┼─────────┼──────────────┼──────────────┼──────────────────────┤
│ /25      │   2     │ 128          │ 126 c/u      │ ✗ NO ALCANZA (4)     │
│ /26      │   4     │  64          │ 62 c/u       │ ✓ la correcta        │
│ /27      │   8     │  32          │ 30 c/u       │ ✓ pero sobre-dimensionada │
│ /28      │  16     │  16          │ 14 c/u       │ ✓ pero 12 de sobra   │
└──────────┴─────────┴──────────────┴──────────────┴──────────────────────┘
```

La justificación, en una frase: el subnetting clásico **roba el mínimo número de bits posible**, o sea usa el **prefijo más largo** que todavía produzca las subredes pedidas.

Con `/26` las cuatro quedan:

```
192.168.10.0/26    .0   – .63    red .0    | bc .63   | hosts .1   – .62
192.168.10.64/26   .64  – .127   red .64   | bc .127  | hosts .65  – .126
192.168.10.128/26  .128 – .191   red .128  | bc .191  | hosts .129 – .190
192.168.10.192/26  .192 – .255   red .192  | bc .255  | hosts .193 – .254
```

⚠️ **Error típico:** elegir `/27` "porque es más chico y parece más seguro". No es seguridad: es desperdicio.

2.2 Ejercicio 2 — VLSM de `172.16.0.0/16` para 500, 250, 100, 50 y 25 hosts

Enunciado. Del bloque `172.16.0.0/16` sacar subredes para 500, 250, 100, 50 y 25 hosts (más **1 dirección de router** en cada una). Plan alineado, cuentas de desperdicio y mayor prefijo libre.

Paso 1 — ordenar de mayor a menor: 501 → 251 → 101 → 51 → 26.

Paso 2 — los prefijos.

```
501:  2^8 - 2 = 254 < 501 ✗   2^9 - 2 = 510 ≥ 501 ✓  →  h = 9  →  /23  (512 dir)
251:  2^7 - 2 = 126 < 251 ✗   2^8 - 2 = 254 ≥ 251 ✓  →  h = 8  →  /24  (256 dir)
101:  2^6 - 2 =  62 < 101 ✗   2^7 - 2 = 126 ≥ 101 ✓  →  h = 7  →  /25  (128 dir)
 51:  2^5 - 2 =  30 <  51 ✗   2^6 - 2 =  62 ≥  51 ✓  →  h = 6  →  /26  ( 64 dir)
 26:  2^4 - 2 =  14 <  26 ✗   2^5 - 2 =  30 ≥  26 ✓  →  h = 5  →  /27  ( 32 dir)
```

⚠️ Ningún `2^h - 2` alcanza en el primer intento: en los cinco casos hay que subir un exponente. Es lo que hace este ejercicio engañoso.

Paso 3 — asignación con el puntero, con la alineación verificada:

```
puntero = 172.16.0.0

/23  tamaño 512   inicio = 0 ✓                    →  172.16.0.0/23     puntero → 172.16.2.0
/24  tamaño 256   inicio = 512 ✓  (512/256 = 2)   →  172.16.2.0/24     puntero → 172.16.3.0
/25  tamaño 128   inicio = 768 ✓  (768/128 = 6)   →  172.16.3.0/25     puntero → 172.16.3.128
/26  tamaño  64   inicio = 896 ✓  (896/64 = 14)   →  172.16.3.128/26   puntero → 172.16.3.192
/27  tamaño  32   inicio = 960 ✓  (960/32 = 30)   →  172.16.3.192/27   puntero → 172.16.3.224

172.16.0.0/16
├─  .0.0/23      500 hosts  gw 172.16.0.1     (.0.0 – .1.255)
├─  .2.0/24      250 hosts  gw 172.16.2.1     (.2.0 – .2.255)
├─  .3.0/25      100 hosts  gw 172.16.3.1     (.3.0 – .3.127)
├─  .3.128/26     50 hosts  gw 172.16.3.129   (.3.128 – .3.191)
├─  .3.192/27     25 hosts  gw 172.16.3.193   (.3.192 – .3.223)
└─  .3.224/27 LIBRE  +  4.0/22 · 8.0/21 · 16.0/20 · 32.0/19 · 64.0/18 · 128.0/17
```

Cuentas finales.

```
Consumidas:  512 + 256 + 128 + 64 + 32 = 992  de 65 536
Libres:      65 536 - 992 = 64 544   (98,5 %)
Desperdicio dentro de los bloques:
   (510 - 501) + (254 - 251) + (126 - 101) + (62 - 51) + (30 - 26)
     =    9   +     3    +     25    +    11   +    4    =  52
Red + broadcast: 5 redes × 2 = 10
Total no utilizable = 62            Eficiencia = 65 474 / 65 536 = 99,9 %
Mayor prefijo libre contiguo: 172.16.128.0/17 (32 768) — un /17 dentro de un /16
   ocupa 2^(24-17) = 128 unidades del tercer octeto, y .128 es la última válida
```

Con FLSM el mismo plan no existe: las cinco subredes serían `/23` (512 dir) = 2560 direcciones, y el `/16` entraría justo, sin margen para ni un enlace. Con VLSM entran las cinco **y** sobra el 98,5 %.

2.3 ⭐ Ejercicio 3 (de examen) — `2011-03` Problema 2 d

> **Fuente:** Examen 21 de febrero de 2011, `sirc1103`, Problema 2 (30 pts), d. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

Enunciado (literal).

> Se adquiere para la Red A el bloque de direcciones IP públicas `100.100.100.0/24`, y los administradores deciden aislar la red de servidores de los puestos de trabajo. Especifique las subredes del bloque mencionado que cumplan con este requerimiento y permitan dejar la mayor cantidad posible de

> direcciones IP para uso futuro. Mencione las modificaciones necesarias en el router R1 y su nueva tabla de enrutamiento.

Contexto. Red A privada con **15 servidores** y **70 puestos**; R1 traduce con `220.10.10.30` sobre `220.10.10.28/30`. Ahora compran un bloque **público**.

a) Los prefijos

```
Red            Hosts  + gw  =  a direccionar   2^h − 2   h   prefijo  bloque
──────────────────────────────────────────────────────────────────────────────
PUESTOS          70   +  1  =      71          2^7−2=126 ✓  7   /25     128 dir
                            2^6 − 2 = 62 < 71 ✗
SERVIDORES       15   +  1  =      16          2^5−2= 30 ✓  5   /27      32 dir
                            2^4 − 2 = 14 < 16 ✗   ← el error típico
```

b) La asignación con el puntero

```
puntero = 100.100.100.0

/25  tamaño 128   inicio = menor múltiplo de 128 ≥ 0    = 0     → 100.100.100.0/25
/27  tamaño  32   inicio = menor múltiplo de  32 ≥ 128  = 128   → 100.100.100.128/27

100.100.100.0/24
├─ .0     /25   PUESTOS    red .0    bc .127  hosts .1   − .126  gw .1
├─ .128   /27   SERVIDORES red .128  bc .159  hosts .129 − .158  gw .129
└─ .160 ──────── LIBRE 96 dir = 100.100.100.160/26 + 100.100.100.224/27
```

c) La tabla de enrutamiento nueva de R1

```
Red destino / Máscara      Gateway (next hop)     Distancia
──────────────────────────────────────────────────────────
100.100.100.0/25            C − Directamente conectado    1
100.100.100.128/27          C − Directamente conectado    1
220.10.10.28/30             C − Directamente conectado    1
0.0.0.0/0                   220.10.10.29                 2
```

Las modificaciones en R1, en orden:

1. **Una interfaz por subred** (o dos sub-interfaces de VLAN). Antes tenía una sola para la red A.
2. **Se elimina el NAT.** Los hosts ya son alcanzables por dirección pública; seguir traduciendo "sobre-vendería" las públicas recién compradas, y rompería la señalización de los servidores (que es justamente lo que se quiere aislar y proteger).
3. **La ruta por defecto y la interfaz WAN no cambian.**
4. Cada subred **conserva su gateway**: `100.100.100.1` y `100.100.100.129`.

d) El análisis: ¿esto **maximiza** lo que queda para el futuro?

Este es el apartado que más puntos da, porque el enunciado pide **justificar** y no basta con decir "así me quedó".

```
┌──────────────────────┬──────────┬──────────┬──────────┬──────────────────┐
│ Plan                 │ Puestos  │ Servidor │ Total    │ Libre del /24    │
├──────────────────────┼──────────┼──────────┼──────────┼──────────────────┤
│ OFICIAL  /25 + /27   │   128    │    32    │   160    │ 96 (37,5 %)      │
│ "Cómodo" /25 + /26   │   128    │    64    │   192    │ 64 (25,0 %)      │
│ "Apretado" /25 + /28 │   128    │ 14 < 16  │ inválido │ no sirve         │
│ "Simétrico" /26 + /26│ 62 < 71  │    64    │ inválido │ no sirve         │
└──────────────────────┴──────────┴──────────┴──────────┴──────────────────┘

Desperdicio DENTRO de los bloques:
  OFICIAL   (126 - 71) + (30 - 16) = 55 + 14 =  69
  "CÓMODO"  (126 - 71) + (62 - 16) = 55 + 46 = 101
  Diferencia a favor de la oficial: 32 direcciones = el doble de libres.
```

Conclusión, y acá es donde se verifica en vez de copiar:

> **La oficial dice `/25` y `/27`. Verifiqué la cuenta y es correcta, y además es la *única* solución posible**: `/26` no alcanza para 71 utilizables y `/28` no alcanza para 16. No hay margen en los prefijos, solo en el **orden** de asignación.
>
> **Mi lectura, que es la que suma puntos:** "dejar la mayor cantidad posible para uso futuro" tiene dos lecturas, y la oficial cumple las dos: gasta el mínimo hoy (160 de 256) **y** deja un remanente que se descompone limpiamente. `100.100.100.160` – `100.100.100.255` son 96 direcciones contiguas = `100.100.100.160/26` + `100.100.100.224/27`, justo lo que hace falta para dos LANs futuras de hasta 62 y de hasta 30 hosts. Con `/26` para los servidores, el libre quedaría partido en `.160/28` + `.176/27` + `.208/27` = 8 + 32 + 32: tres bloques inservibles por separado.
>
> O sea: **lo que el profesor quiere ver no es "la oficial está bien", sino "la oficial es correcta porque el mínimo de cada prefijo la obliga, y además produce un remanente alineado que sirve para el segundo reading del objetivo"**.

⚠️ **Errores típicos:** `/28` para los servidores; no contar el router (`+1` en cada subred); dejar el NAT puesto; olvidar `0.0.0.0/0` en la nueva tabla (sin ella los hosts de las subredes nuevas no salen a Internet, que era el objetivo).

2.4 ⭐ Ejercicio 4 (de examen) — `2010-07` Problema 2 a-i

> **Fuente:** Examen 27 de julio de 2010, `sirc1007`, Problema 2, parte a-i.

Enunciado (literal).

> Asigne las direcciones de red a las subredes LAN2 y LAN3 utilizando el prefijo de dirs. IP privadas mencionado anteriormente, asumiendo que se conectarán hasta 250 hosts por subred.

Datos: prefijo **`192.168.2.0/23`** para las dos sucursales; el ISP asignó `220.10.10.28/30` al enlace WAN de R1. LAN1 y los enlaces entre routers ya están numerados.

Solución oficial. "*Para cada subred se necesitan como máximo 250 (hosts) + 1 (router) = 251 direcciones IP por LAN. Entonces cada subred necesita utilizar una `/24`: LAN 2 `192.168.2.0/24` · LAN 3 `192.168.3.0/24`*".

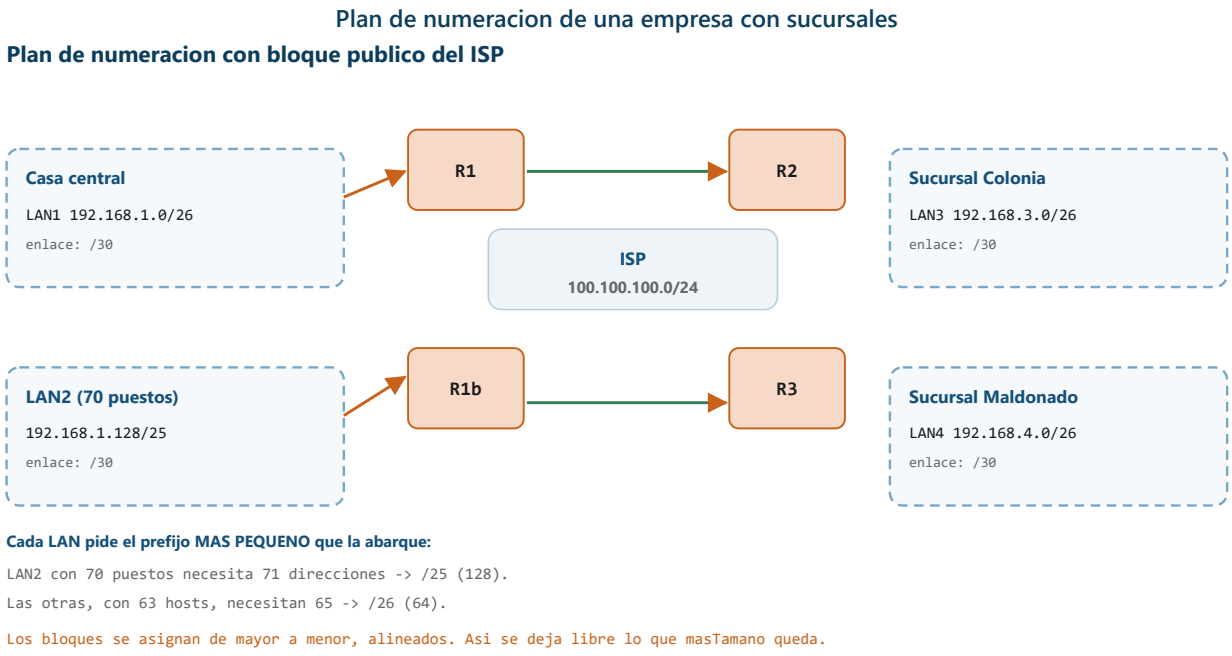
Mi resolución, con las dos cuentas que hay que dar.

```
250 hosts + 1 router = 251 direcciones pedidas
+ red y broadcast  ⇒  el bloque necesita ≥ 253 direcciones

2^7 = 128  < 253   ✗   (/25 NO alcanza: son 128, no 256)
2^8 = 256  ≥ 253   ✓   →  h = 8  →  prefijo = 32 − 8 = /24
/24 = 256 direcciones totales, 254 utilizables.   254 ≥ 251 ✓  (sobran 3)

¿Cuántas subredes entran en el /23?
  Por bits:   el /23 deja 32 − 23 = 9 bits de subred; un /24 usa 8 de ellos;
              quedan 1 bit  →  2^1 = 2 subredes
  Por tamaño: 2^(32−23) = 512 direcciones ;  512 / 256 = 2 subredes   ✓
```

Asignación con el puntero.

```
puntero = 192.168.2.0

/24  tamaño 256   inicio = 0 ✓                    →  192.168.2.0/24  (LAN2) gw .2.1
      puntero = 256
/24  tamaño 256   inicio = 256 ✓  (256/256 = 1)   →  192.168.3.0/24  (LAN3) gw .3.1
      puntero = 512 = 192.168.4.0

Cuentas:  total /23 = 512 ; consumido 2 × 256 = 512  →  LIBRE: 0
          no utilizable = 2 redes × 2 (red+bc) + 2 redes × 3 (sobrantes) = 10
          eficiencia = (512 − 10) / 512 = 98,0 %
```

El comentario que vale puntos: el bloque queda **al 100 % de ocupación**. Si la empresa supiera que necesitaría una tercera sucursal, debería haber pedido un `/22`; el enunciado no lo dice, así que la respuesta correcta es usar el `/23` entero. Pero **mencionar el costo de oportunidad del plan** es lo que demuestra que entendiste el ejercicio, no solo la cuenta.



⚠️ **Errores típicos:** usar `/25` "porque 250 < 256" (`/25` son 128 direcciones; el número que importa es `2^(32−prefijo) ≥ 253`); pedir un `/23` por subred (un `/23` no es una LAN: son 512 direcciones, y la unidad es `/24` porque es lo que **cabe**); contar los hosts del enlace WAN, que viene del bloque del ISP y no del `/23`.

2.5 ⭐ Ejercicio 5 (de examen) — `2010-12` Problema 2 a-i

> **Fuente:** Examen 1 de agosto de 2008, `sirc0807`, Problema 2, parte a-i (el banco de exámenes lo indexa también bajo `sirc1007`).

Enunciado (literal).

> Realice un plan de numeración de las subredes LAN3, 4 y 5 utilizando el prefijo de dirs. IP públicas mencionado anteriormente, asumiendo que no se conectan más de 120 hosts por subred. ¿Cuántas subredes se pueden numerar de acuerdo a su plan de numeración?

Datos: prefijo **`147.10.8.0/21`**; la red corporativa tiene sede central y sucursal, con `220.10.10.13/30` y `220.10.20.14/30` en las interfaces WAN.

Solución oficial. "*Para cada subred se necesitan como máximo 120 + 1 + 2 = 123 direcciones IP por LAN.*" → plan A: LAN3 `147.10.8.0/25`, LAN4 `147.10.8.128/25`, LAN5 `147.10.9.0/25`; plan B: LAN3 `147.10.8.0/25`, LAN4 `147.10.9.0/25`, LAN5 `147.10.10.0/25`. "*Según este plan se cuenta con `2^4 = 16` subredes /25.*"

El `+2` de la oficial: 120 hosts + 1 router + **2 de reserva** para crecimiento inmediato de cada LAN (no red ni broadcast, que ya están dentro del `2^h`). Si no lo decís, parece un error de suma.

La aritmética del prefijo.

```
120 hosts + 1 router + 2 reserva = 123 direcciones pedidas
+ red y broadcast  ⇒  el bloque necesita ≥ 125 direcciones

2^6 =  64  < 125   ✗   NO ALCANZA
2^7 = 128  ≥ 125   ✓   →  h = 7  →  prefijo = 32 − 7 = /25
/25 = 128 direcciones, 126 utilizables.   126 ≥ 123 ✓  (sobran 3 utilizables)

Variante con el mínimo estricto (120 + 1 = 121, sin la reserva):
121 + 2 = 123  ⇒  2^7 = 128 ≥ 125 ✓  →  ¡también /25!  La reserva NO cambia
el prefijo: sirve para el futuro, no para la cuenta.
```

¿Cuántas subredes da el plan? Las dos cuentas, y hay que dar las dos:

```
Por tamaño:  2^(32 − 21) = 2^11 = 2048 direcciones ; 2048 / 128 = 16 subredes /25
Por bits:    el /21 deja 32 − 21 = 11 bits de subred.
             un /25 usa  32 − 25 =  7 de esos bits.
             Quedan     11 − 7  =  4 bits  →  2^4 = 16   ✓
```

Asignación con el puntero (el plan A de la oficial):

```
puntero = 147.10.8.0

/25  inicio = 0 ✓                    →  147.10.8.0/25     puntero → 147.10.8.128
/25  inicio = 128 ✓  (128/128 = 1)   →  147.10.8.128/25   puntero → 147.10.9.0
/25  inicio = 256 ✓  (256/128 = 2)   →  147.10.9.0/25     puntero → 147.10.9.128

LIBRE desde 147.10.9.128: 13 subredes /25 = 1 664 direcciones
Cuentas:  total /21 = 2048 ; consumido 3 × 128 = 384 ; libre 1 664 = 13 /25
          desperdicio interno = 3 × (126 − 123) = 9   eficiencia = 97,7 %
```

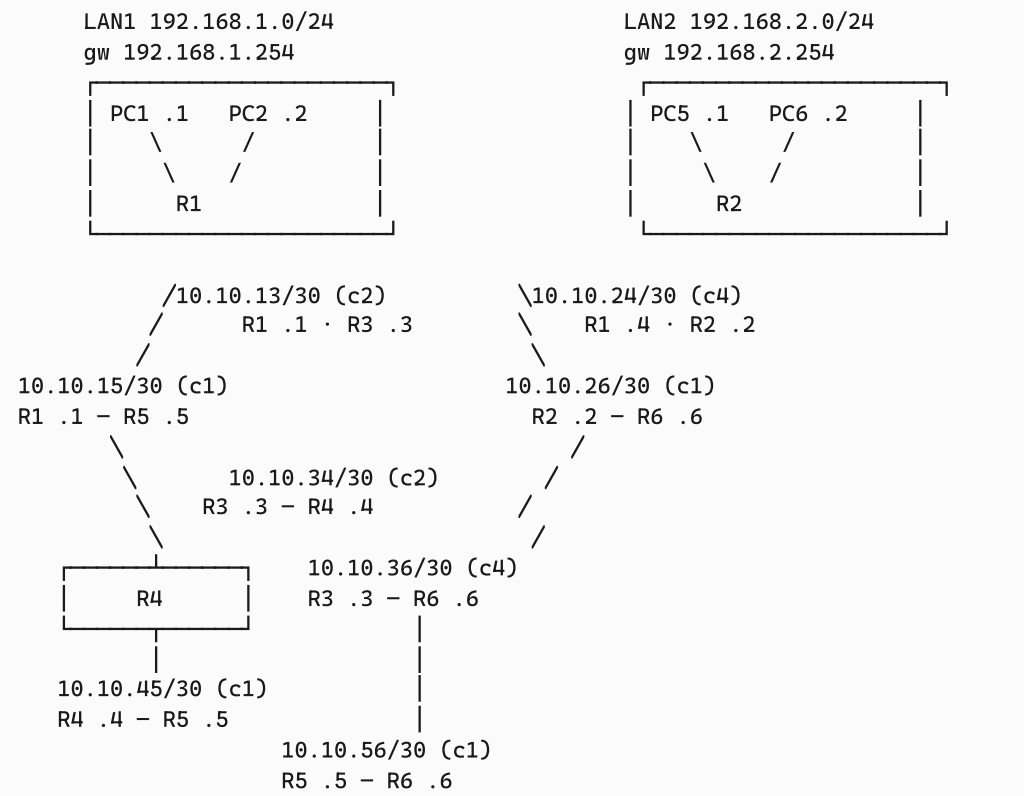
⚠️ **Por qué la oficial prefiere el plan A (lo que más suma):**

```
PLAN A:  LAN3 .8.0/25 · LAN4 .8.128/25 · LAN5 .9.0/25
         LAN3 + LAN4 = 147.10.8.0/24  ← SUMARIZA (0 es múltiplo de 256)
         ⇒ R2 necesita 2 rutas hacia la empresa.

PLAN B:  LAN3 .8.0/25 · LAN4 .9.0/25 · LAN5 .10.0/25
         ⇒ R2 necesita 3 rutas. NO SE PUEDE sumarizar: para resumir LAN4
           (.9.0–.9.127) y LAN5 (.10.0–.10.127) harían falta 256 direcciones
           arrancando en .8.0 o en .10.0, y .9.0 no es ninguno de los dos.

O sea: "la solución no es única, pero no todas son equivalentes: el plan tiene
que optimizar no solo las direcciones sino la cantidad de rutas" — y eso se
decide en el paso 3, no en el paso 2.
```

2.6 ⭐ Ejercicio 6 (de examen) — `2006-08` Problema 1: seis routers

> **Fuente:** Examen 5 de agosto de 2006, `sirc0608`, Problema 1 (30 pts). **Aclaración:** el problema original es una figura. **Adaptamos la figura a texto**: routers, enlaces, direcciones y default gateway (192.168.1.254 y 192.168.2.254) son los del examen; la métrica es el costo de enlace de la tabla del enunciado, reasignado de forma consistente con las soluciones oficiales de los apartados b) y e) del mismo examen. La topología de abajo reproduce **exactamente** las dos respuestas oficiales de `traceroute` (ver §2.6 c).

La topología




```
Enlaces y costos (los del enunciado):
  10.10.13  R1.1  — R3.3     10.10.15  R1.1 — R5.5      10.10.24  R1.4 — R2.2
  10.10.26  R2.2  — R6.6     10.10.34  R3.3 — R4.4      10.10.36  R3.3 — R6.6
  10.10.45  R4.4  — R5.5     10.10.56  R5.5 — R6.6
```

a) Tablas de enrutamiento de los seis routers

Formato de la cátedra: **Red destino / Máscara · Gateway (next hop) · Distancia**.

```
R1                                  R2
192.168.1.0/24   C — Directamente conectado  1     192.168.2.0/24  C — Directamente conectado 1
192.168.2.0/24   10.10.15.5                 3     192.168.1.0/24  10.10.26.6                3

R3                                  R4
192.168.1.0/24   10.10.13.1                 2     192.168.1.0/24  10.10.45.5                2
192.168.2.0/24   10.10.36.6                 5     192.168.2.0/24  10.10.45.5                3

R5                                  R6
192.168.1.0/24   10.10.15.1                 1     192.168.1.0/24  10.10.56.5                2
192.168.2.0/24   10.10.56.6                 2     192.168.2.0/24  10.10.26.2                1
```

Cómo se calcularon, con Dijkstra sobre la matriz de costos (elementales 1, 2 y 4). El más caro de todos, R3 hacia LAN2:

```
R3 → LAN2:   vía R6 = 4 + 1 = 5   ✓   vía R1 = 2 + 3 = 5   vía R4 = 2 + 3 = 5
Empate en los tres; con empate gana el vecino de menor dirección de enlace:
R6 (10.10.36.6) ✓  — que es lo que dice la oficial, con costo 5.

Y el que define la tabla de R2, el corazón del ejercicio:
R2 → LAN1:   vía R6 = 1 + 1 + 1 = 3  ✓      vía R1 (directo) = 4   peor
⇒ R2 usa 10.10.26.6 con distancia 3   (la oficial dice 10.10.26.6)  ✓
```

b) Tablas de enrutamiento de los hosts

Todos los hosts de LAN1 (`.1.1` … `.1.254`) y de LAN2 (`.2.1` … `.2.254`):

```
Red destino / Máscara   Gateway (next hop)          Distancia
192.168.1.0/24          C — Directamente conectado       1
0.0.0.0/0               192.168.1.254                    2      ← hosts de LAN1

Red destino / Máscara   Gateway (next hop)          Distancia
192.168.2.0/24          C — Directamente conectado       1
0.0.0.0/0               192.168.2.254                    2      ← hosts de LAN2
```

⚠️ **Un host tiene una entrada por red local más la default**, nunca más: si un host tiene **dos** interfaces, tiene **dos** redes locales y una default, y el *longest prefix match* decide por cuál sale cada destino (Clase 30).

c) Verificación contra las soluciones oficiales

Enrutamiento normal. La oficial: "*el camino de menor costo de 192.168.2.0/24 a 192.168.1.0/24 es R2 → R6 → R5 → R1*":

```
PC6 (LAN2) → R2 → R6 → R5 → R1 → PC2 (LAN1)
Interfaces que reporta traceroute:  192.168.2.254 · 10.10.26.6 · 10.10.56.5
                                  · 10.10.15.1 · 192.168.1.2
Suma: 4 enlaces.  En mis tablas R2 usa 10.10.26.6 (d=3), R6 usa 10.10.56.5
(d=2), R5 usa 10.10.15.1 (d=1).   ✓ COINCIDE
```

Si se cae R4. La oficial: "*la respuesta es la misma, pues R4 no es utilizado en el camino de ida*" → **el `traceroute` no cambia una línea**. En mis tablas ningún next hop apunta a `10.10.34.4` ni a `10.10.45.4`. ✓ COINCIDE

Si se cae R5. La oficial da `R2 → R6 → R3 → R1` con interfaces `192.168.2.254 · 10.10.26.6 · 10.10.36.3 · 10.10.13.1`:

```
Con R5 caído, R6 ya no puede ir por 10.10.56.5, así que su next hop a LAN1 pasa
a ser R3 por 10.10.36.3 (d = 4 + 2 = 6); R3 mantiene 10.10.13.1 (d = 2).
Camino:  192.168.2.254 · 10.10.26.6 · 10.10.36.3 · 10.10.13.1 · destino
✓ COINCIDE con la solución oficial, interfaz por interfaz.
```

Ese es el mejor detalle del problema: la tabla de enrutamiento no dice "el camino más corto a la LAN", sino "el camino más corto **al vecino más barato**". Cuando R5 se cae, R3 —que antes era un rodeo— pasa a ser la primera opción de R6. Por eso los apartados d) y e) hablan de *convergencia*, no de tablas fijas.

PARTE 3 — EJERCICIOS EXTRA (dificultad creciente)

3.1 Ejercicio 7 — VLSM con cinco redes y tres enlaces

Enunciado. De `172.20.0.0/16`: LAN de **300**, **120** y **60** hosts, más **tres enlaces punto a punto**.

```
301 → 2^8 − 2 = 254 < 301 ✗ ; 2^9 − 2 = 510 ≥ 301 ✓  →  /23  (512 dir)
121 → 2^7 − 2 = 126 ≥ 121 ✓                          →  /25  (128 dir)
 61 → 2^6 − 2 =  62 ≥  61 ✓                          →  /26  ( 64 dir)
  2 → 2^2 − 2 =   2 ≥   2 ✓                          →  /30  (  4 dir)  ×3

puntero = 172.20.0.0
/23  inicio = 0    ✓                 → 172.20.0.0/23     puntero → 172.20.2.0
/25  inicio = 512  ✓ (512/128 = 4)   → 172.20.2.0/25     puntero → 172.20.2.128
/26  inicio = 640  ✓ (640/64 = 10)   → 172.20.2.128/26   puntero → 172.20.2.192
/30  inicio = 704  ✓ (704/4 = 176)   → 172.20.2.192/30   puntero → 172.20.2.196
/30  inicio = 708  ✓ (708/4 = 177)   → 172.20.2.196/30   puntero → 172.20.2.200
/30  inicio = 712  ✓ (712/4 = 178)   → 172.20.2.200/30   puntero → 172.20.2.204

Consumidas:  512 + 128 + 64 + 4 + 4 + 4 = 716  de 65 536
Libres:      65 536 − 716 = 64 820   (98,9 %)
Desperdicio interno: (510−301) + (126−121) + (62−61) + 0 + 0 + 0 = 209 + 5 + 1 = 215
Libre a partir de 172.20.2.204:  .204/30 (4) · .208/29 (8) · .216/29 (8) ·
                                 .224/28 (16) · 172.20.3.0/24 · 4.0/23 · …
Mayor prefijo libre: 172.20.128.0/17 (32 768)
```

Lo que enseña: los tres `/30` seguidos no pierden nada: `704, 708, 712` son múltiplos de 4, así que el puntero nunca se desvía.

3.2 Ejercicio 8 — ¿Cuántas direcciones desperdicia cada plan?

Enunciado. Del bloque `192.168.0.0/21` (2048 dir) se necesitan LANs de 300, 200 y 50 hosts más un enlace punto a punto. Compará FLSM y VLSM **midiendo el desperdicio**, no solo la cantidad de direcciones.

```
══ FLSM ══  el mayor pide 301 → h = 9 → /23 para TODAS
   4 × 512 = 2048 = el bloque entero.  Libre: 0
   Usado realmente: 301 + 201 + 51 + 2 = 555
   Desperdicio = 2048 − 555 = 1493   (72,9 % del bloque)

══ VLSM ══  301 → /23 (512) · 201 → /24 (256) · 51 → /26 (64) · 2 → /30 (4)
   Total = 836   Libre = 2048 − 836 = 1212
   Desperdicio interno = (510−301)+(254−201)+(62−51)+(2−2) = 209+53+11+0 = 273
   Red + broadcast = 4 redes × 2 = 8      Total no utilizable = 281 (13,7 %)
```

	FLSM (/23 x4)	VLSM	Diferencia
Direccionadas	2 048	836	− 1 212
Libres	0	1 212	+ 1 212
Desperdicio	1 493	281	− 81 %
Eficiencia	27,1 %	86,3 %	+ 59,2 pp

Conclusión: el FLSM no desperdicia un 27 % "de direcciones": desperdicia el **73 %**. Ese es el número que hay que decir, no "usa más direcciones".

3.3 Ejercicio 9 — Renumerar para meter un switch (y cuándo NO hace falta)

Enunciado. Se reemplaza R2 por un switch, y las subredes 2 y 3 quedan en el mismo dominio de broadcast. ¿Hay que renumerar?

Caso A — `192.168.108.0/24` y `192.168.109.0/24`.

```
¿Es un /23?  108 = 01101100,  109 = 01101101 → difieren en **UN bit**
   (00001100 vs 00001101), y un /23 "se come" el último bit del tercer octeto.
   108 es par  ⇒  108/2 = 54 exacto  ⇒  /23 cubre .108.0 – .109.255   ✓
⇒ NO HAY QUE RENUMERAR. Solo cambian:
     · la máscara de R1 en esa interfaz:  /24 → /23
     · la máscara de los hosts de las subredes 2 y 3:  /24 → /23
     · el default gateway de esos hosts:  192.168.108.1  (no cambia)
```

Caso B — `192.168.0.0/24`, `.1.0/24` y `.2.0/24` (del `2010-03 P1b`)

```
¿.0.0/24 + .1.0/24 = un /23? Arrancaría en 192.168.0.0/23 → sí, cubre
.0.0–.1.255 ✓. PERO .2.0/24 queda en otra subred, así que el switch no puede
unificar 1 y 2. Para meter las TRES hace falta un /22:
     192.168.0.0/22 cubre .0.0 – .3.255 (1024 dir); 0 es múltiplo de 4 ✓
⇒ SÍ hay que renumerar. La solución oficial (mínima cantidad de direcciones
  modificadas) mueve SOLO la subred 1:  192.168.0.0/24 → 192.168.3.0/24, y las
  subredes 2 y 3 quedan en 192.168.0.0/22.
  Tabla de R1:  192.168.3.0/24 → C (eth0)  ·  192.168.0.0/22 → C (eth1)
```

Lo que hay que decir en el examen: "*la renumeración es necesaria cuando las subredes a unificar no forman un prefijo contiguo y alineado; si son contiguas y la cantidad es potencia de 2, alcanza con cambiar la máscara*", y eso se verifica mirando cuántos bits las distingue en binario.

3.4 Ejercicio 10 — ¿Se puede sumarizar?

Como se aplica una mascara de subred, bit a bit

IP	Mascara (/26)	AND = resultado
11000000	11111111	11000000
10101000	11111111	10101000
00001010	11111111	00001010
00110111	11000000	00000000

192.168.10.55 AND 255.255.255.192 = 192.168.10.64

Lo que resulta de ese AND:

direccion de red	192.168.10.64
primer host	192.168.10.65
ultimo host	192.168.10.126
broadcast	192.168.10.127
hostables	62

El prefijo /26 significa:

26 bits de red + 6 bits de host
2^6 = 64 direcciones en la subred
64 - 2 = 62 hostables

El AND pone en 0 todos los bits de host: por eso el resultado es la direccion de RED, no una de host.

Enunciado. Del plan del Ejercicio 2 (`172.16.0.0/16`): `0.0/23`, `2.0/24`, `3.0/25`, `3.128/26`, `3.192/27`. Agrupá las rutas de a dos, de a tres, etc., hasta cuántas quedan.

```
Nivel 1:  172.16.3.0/25 + 172.16.3.128/26  = 172.16.3.0/24  ✓ (128+64 = 192,
          contiguo dentro de .3.0–.3.255)
          172.16.3.0/24 + 172.16.3.192/27  = 172.16.3.0/24  ✓ ya dentro
Nivel 2:  172.16.2.0/24 + 172.16.3.0/24    = 172.16.2.0/23  ✓ (256+256 = 512)
Nivel 3:  172.16.0.0/23 + 172.16.2.0/23    = 172.16.0.0/21  ✓ (512+512 = 1024)

RESULTADO: 5 rutas → 1 sola: 172.16.0.0/21
Chequeo: el plan ocupa .0.0 – .3.255 = 1024 = un /21 exacto. ✓
Ojo: el /21 ABARCA .4.0 – .7.255, que están LIBRES. Al sumarizar estás
anunciando también lo que no usás, y eso es correcto: el espacio de una red es
del router hacia afuera, no "lo que yo uso".
```

El caso que NO se puede, para que quede la regla:

```
172.16.3.128/26 + 172.16.3.192/27  →  64 + 32 = 96, no es potencia de 2.
Un /25 arrancaría en .128/25 = .128–.255  ✗ incluye .224/27, que es otra red.
REGLA: la cantidad de direcciones tiene que ser POTENCIA DE 2 y el arranque tiene
que estar alineado a ese tamaño. Si no, no hay resumen.
```

3.5 Ejercicio 11 — El puntero que cambia de paso

Enunciado. En `192.168.1.0/24` ya usaste `/25`, `/26`, `/27`, `/30` y `/30`; el puntero quedó en **232**. Ahora piden, en este orden, un `/29`, un `/31`, un `/32` y un `/28`. Verificá.

```
puntero = 192.168.1.232
/29  inicio = menor múltiplo de 8  ≥ 232 = 232  ✓  → .232/29 (.232–.239)  ptr 240
/31  inicio = menor múltiplo de 2  ≥ 240 = 240  ✓  → .240/31 (.240–.241)  ptr 242
/32  inicio = cualquier entero ≥ 242             → .242/32 (.242)         ptr 243
/28  inicio = menor múltiplo de 16 ≥ 243 = 256  ✗  ¡NO ENTRA!
```

Acá está la lección: un `/28` (16 direcciones) **ya no entra**, porque el bloque libre que queda (`.232` – `.255` , 24 direcciones) no está alineado a 16. Si lo hubieras pedido **antes** del `/29` : `/28` arranque en 240 → `.240/28` → puntero 256 = FIN, y el `/29` + `/31` + `/32` NO HABRÍAN ENTRADO.

Conclusión práctica: el **último** bloque del plan es el que peor redondeo sufre; si vas a necesitar uno grande al final, pedilo primero.

PARTE 4 — CIERRE

📌 Resumen

1. Las cinco fórmulas: `2^h − 2 ≥ H` · `2^(32−prefijo)` · `dir/tamaño × tamaño` · `dir + tamaño − 1` · `puntero = menor múltiplo de tamaño ≥ puntero`.
2. Para **n** subredes iguales se roban `log2(n)` bits: 4 subredes en un `/24` → **/26** (62 utilizables c/u), no `/27`.
3. `172.16.0.0/16` para 500/250/100/50/25 → `/23 · /24 · /25 · /26 · /27` = 992 direcciones, libre 64 544, mayor prefijo libre `172.16.128.0/17`.
4. `100.100.100.0/24` para 70+1 y 15+1 → **`/25` + `/27`**; el `/28` NO entra (14 utilizables < 16). Libre: 96 = `/26 + /27`.
5. `192.168.2.0/23` con 251 por LAN → `/24` → exactamente **2** subredes, bloque al 100 %.
6. `147.10.8.0/21` con 123 por LAN → `/25` → **16** subredes; el plan A es mejor que el B porque `LAN3 + LAN4` **sumarizan** en un `/24` (2 rutas en vez de 3).
7. Seis routers: el `traceroute` normal va `R2→R6→R5→R1`; si cae R5, pasa a `R2→R6→R3→R1`. Si cae R4, no cambia. Las tablas se recalculan, no se borran.
8. **Desperdicio, no "direcciones usadas":** con FLSM, un `/21` para 300+200+50 pierde el **73 %**; con VLSM, el **13,7 %**.
9. Renumerar **no** hace falta si las subredes a unificar ya forman un prefijo contiguo y alineado (`108.0/24` + `109.0/24` = `/23`). Hace falta si no.
10. El **último** bloque del plan es el que peor redondeo sufre: pedí primero el más grande.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué 16 direcciones no entran en un `/27` de 32 direcciones?

▶ Respuesta

2. Cuatro subredes iguales en un `/24`: ¿por qué `/26` y no `/27`?

▶ Respuesta

3. `172.16.0.0/16` para 500, 250, 100, 50 y 25 hosts (+1 router cada una): prefijos, consumo y mayor prefijo libre.

▶ Respuesta

4. `100.100.100.0/24` para 70 puestos + 1 router y 15 servidores + 1 router. Verificá la solución oficial y decí quécfc остаток queda libre.

▶ Respuesta

5. Del `/21` `147.10.8.0` con 123 direcciones por LAN: ¿qué prefijo y cuántas subredes? Dad las dos cuentas.

▶ Respuesta

6. `192.168.108.0/24` y `192.168.109.0/24`: ¿hay que renumerar para ponerlas en el mismo dominio de broadcast con un switch?

▶ Respuesta

7. Del plan del ejercicio 2, ¿cuántas rutas quedan al sumarizar y cuál es la más agregada posible? ¿Y por qué está bien anunciar direcciones que no usás?

▶ Respuesta

8. El puntero quedó en 232 y quedan 24 direcciones libres (`192.168.1.232` – `.255`). ¿Entran un `/29`, un `/31` y un `/28`? ¿En qué orden?

▶ Respuesta

🔗 Referencias

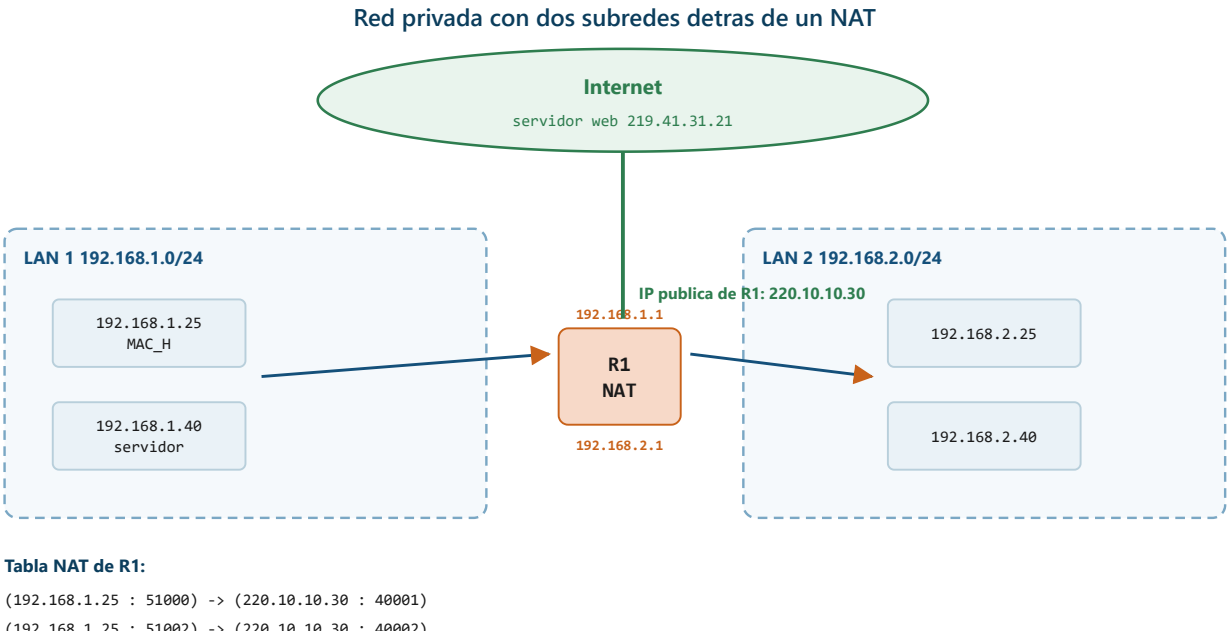
- **`eirc1103` / `sirc1103`** (21-feb-2011) — Problema 2 d: `100.100.100.0/24` para 70 puestos + 15 servidores (Ejercicio 3)
- **`sirc1007`** (27-jul-2010) — P2 a-i: `192.168.2.0/23` con 250 hosts (Ejercicio 4)
- **`sirc0807`** (1-ago-2008) — P2 a-i: `147.10.8.0/21` con 120 hosts por LAN (Ejercicio 5)
- **`sirc0608`** (5-ago-2006) — **Problema 1**: topología de 6 routers (Ejercicio 6) · P4: `200.40.224.0/25` en 18, 11 y 9 hosts
- **`sirc0912`** / **`sirc1003`** — Problema 1 b: renumerar para agregar un switch (Ejercicio 9)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red"
- **RFC 1918** (privadas) 🔍 · **RFC 4632** (CIDR) 🔍 · **RFC 6177** (2011) · **RFC 8200** (2017)
- **Clase anterior:** 28 — VLSM y planes de numeración · **Clase siguiente:** 30 — Tablas de enrutamiento, next hop y ARP

Clase 30

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 El formato de la tabla de enrutamiento
- 1.2 Los tres tipos de entradas
- 1.3 *Longest prefix match*
- 1.4 El proceso de decisión completo
- 1.5 ARP: por qué existe
- 1.6 El formato del paquete ARP
- 1.7 La secuencia ARP completa
- 1.8 La tabla ARP: *aging*, *gratuitous* y *spoofing*
- 1.9 RARP: el problema inverso, y por qué murió
- 2.1 Ejercicio 1 — La tabla de un host con **dos** interfaces
- 2.2 Ejercicio 2 — ARP entre dos LANs, con un router en el medio
- 3.1 ⭐ Examen `2011-03`, Problema 2 a, b.i y b.ii — tablas, ARP y MACs
- 3.2 Examen `2010-03`, Pregunta 1 a — ARP entre dos hosts
- 3.3 Examen `2010-03`, Problema 1 a — tablas de R1, R2 y los hosts
- 3.4 Examen `2009-12`, Problema 1 a — tablas y la secuencia completa de tramas
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: D (Capa de Red) · **Semana:** 10 · **Duración estimada:** 110 min **Correlativa:** Clase 29 (taller de subnetting) · **Sigue con:** Clase 31 (NAT, PAT y sus hacks) **Material de referencia:** Tanenbaum cap. 5 · RFC 826 (ARP) · RFC 1027 (proxy ARP) · RFC 4443 (ICMPv6, NDP) · RFC 8200 (IPv6)



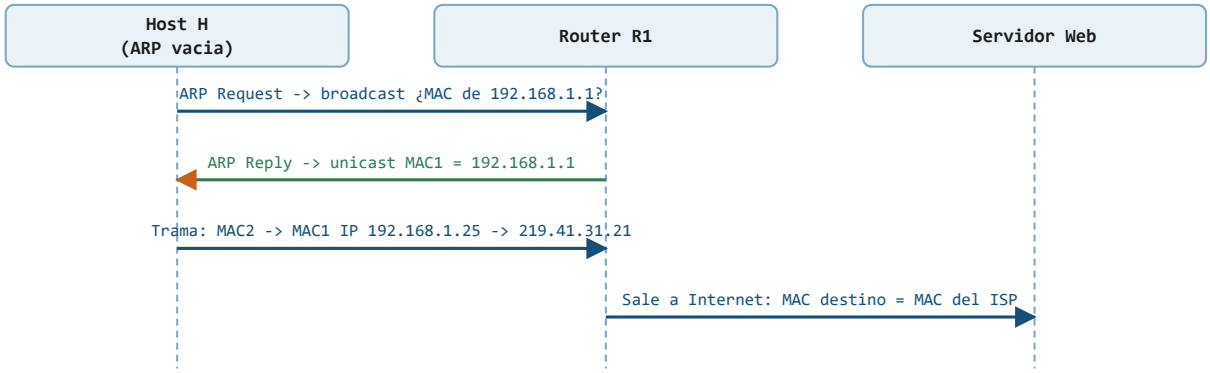
> 🔍 = RFC **no verificado** en `ACTUALIZACIONES.md`: no lo cites sin confirmarlo.

🎯 Al terminar esta clase vas a poder

1. Escribir la tabla de enrutamiento de un router y de un host en el **formato de tres columnas de la cátedra**, e interpretar cada columna.
2. Distinguir los **tres tipos de entradas**: conectada, aprendida y ruta por defecto.

3. Explicar el ***longest prefix match*** y por qué gana la **más específica**.
4. Ejecutar el **proceso de decisión completo** de un host y de un router.
5. Explicar por qué existe ARP y escribir el **formato del paquete ARP** campo por campo.
6. Describir **ARP Request (broadcast) → ARP Reply (unicast)** con MACs e IPs, y resolverlo enterito para un examen.
7. Describir la **tabla ARP** (aging, *gratuitous*, *spoofing*) y por qué **RARP** murió.

Secuencia ARP completa (Examen 2011-03, apartado b.i)



> **Prerequisito del examen:** `2011-03 Problema 2 a, b.i y b.ii` · `2010-03 Pregunta 1a` · `2010-03 Problema 1 a.i-ii` · `2009-12 Problema 1 a-i-iii` · `2006-08 Pregunta 2` · `2001-02 Problema 1`.

PARTE 1 — TEORÍA

1.1 El formato de la tabla de enrutamiento

```
┌──────────────────────────┬──────────────────────────┬──────────────┐
│ Red destino / Máscara    │ Gateway (next hop)       │ Distancia    │
├──────────────────────────┼──────────────────────────┼──────────────┤
│ 192.168.1.0/24           │ C – Directamente conectado│     1        │
│ 0.0.0.0/0                │ 10.0.0.1                 │     3        │
└──────────────────────────┴──────────────────────────┴──────────────┘
```

Columna	Qué es	Ojo
Red destino / Máscara	El prefijo que dice "todo lo que cae acá, va por acá". **La máscara es parte de la clave**: `192.168.1.0/24` y `192.168.1.0/25` son entradas **distintas**.	Sin máscara no se puede hacer *longest prefix match*
Gateway (next hop)	La IP del **router vecino** por el que se sale, o **C** (*connected*) si el destino es una red **directamente conectada**.	Es un **vecino**, no el destino final: el paquete va al `10.0.0.1` y ese router decide después
Distancia (o **adquisición**)	Cuántos saltos cuesta, o de dónde salió la ruta.	Una red conectada tiene distancia **1**, no 0: el "0 saltos" sos vos

Sobre el `C`. "*El prefijo de la columna 1 es una de mis redes de interfaz, no hace falta ningún router para llegar*". No la aprendió nadie: la puse yo, y es la **única entrada que nunca se borra** mientras la interfaz esté arriba. La notación equivalente del `2008-07 P1b` es una columna **Adquisición** con `C` (conectada), `R` (por RIP) y `O` (*other*, la que anuncia el ISP).

⚠️ `C` **también** puede llevar gateway: es el caso de un router con **dos interfaces en la misma red**, o de una **red de tránsito**. `10.0.3.0/24 → C → 10.0.3.1` quiere decir: "*esa red es mía, la tengo en `eth0`, y mi dirección therein es 10.0.3.1*".

1.2 Los tres tipos de entradas

```
① CONECTADA                  ② APRENDIDA                  ③ POR DEFECTO
─────────────                ─────────────                ─────────────
Red de una de mis interfaces. Red que NO es mía.          Red destino = 0.0.0.0/0
Gateway = C.                 Gateway = la IP de un        ("cualquiera").
Distancia = 1.               vecino.                      Gateway = el router
Nunca se borra.               Distancia = n° de saltos.    de salida.
La pongo YO.                 La aprendo de otro router.   La más cara que conozco.

192.168.1.0/24 → C  (1)      192.168.0.0/24 → 10.0.0.2 (2)  0.0.0.0/0 → 10.0.0.1 (3)
```

Por qué la ruta por defecto tiene distancia propia. Es una ruta más: si hay un camino específico, gana (§1.3); si no, usás el default. Y si **tampoco** hay default, el router **descarta el paquete y manda un ICMP tipo 3 código 0** (*net unreachable*). **La tabla de un host** es la versión mínima: una red local (`C`) + la default; con **dos** interfaces, dos redes locales + una default (Ejercicio 2.1).

1.3 *Longest prefix match*

El router no elige "la ruta que más se parece", sino **la de prefijo más largo** que todavía coincida con el destino.

```
Tabla de un router
  192.168.1.0/24      →  eth0          (conectada,           distancia 1)
  192.168.1.128/25    →  10.0.0.2      (aprendida de vecino, distancia 2)
  0.0.0.0/0           →  10.0.0.1      (por defecto,         distancia 3)

Llega un paquete con IP destino 192.168.1.200

  ¿192.168.1.0/24  coincide?   192.168.1.200 & 255.255.255.0   = 192.168.1.0    SÍ
  ¿192.168.1.128/25 coincide?  192.168.1.200 & 255.255.255.128 = 192.168.1.128 SÍ
  ¿0.0.0.0/0        coincide?   siempre SÍ

  El prefijo MÁS LARGO que coincide es /25  (25 > 24 > 0)  ⇒  sale por 10.0.0.2
  Y la entrada /24 NO sobra: sirve para 192.168.1.1 … .127, que no caen en el /25.
```

El test de coincidencia, el mismo del subnetting: `D AND máscara == red`.

Por qué el más específico y no el de menor distancia. La `/25` es más cara (2 saltos) pero trata un **subconjunto** de la `/24`, así que **debe** ganar. Si ganara la de menor distancia, la `/24` no se podría usar nunca y las subredes sumarizadas no existirían.

⚠️ `192.168.1.0/25` y `192.168.1.128/25` **no** "se parten": para el router son dos redes, y cada host tiene una tabla distinta.

1.4 El proceso de decisión completo

En un host (el 90 % de los casos que se preguntan):

```
H llega un paquete con IP destino D
          │
          ▼
┌──────────────────────────────────────────┐
│ ¿D está en la tabla con más prefijo que  │  NO ──► usar la entrada 0.0.0.0/0
│ coincida?  (longest prefix match sobre   │         → el default gateway
│ TODA la tabla, no solo la primera)       │          SIN default: se descarta
└──────────────┬───────────────────────────┘
               │ SÍ
               ▼
┌──────────────────────────────────────────┐
│ ¿La entrada dice "C" (conectada)?        │  NO ──► el paquete va al router vecino
└──────────────┬───────────────────────────┘          (se resuelve el MAC con ARP)
               │ SÍ
               ▼
Entregar el paquete a la capa 4 sin tocar nada más.
```

En un router son los mismos dos pasos, más un caso propio: si el destino es una dirección de **broadcast o multicast de una de sus propias subredes**, tiene que generar el paquete él mismo (es el paso 3 del `traceroute` de la `2006-08 P1`).

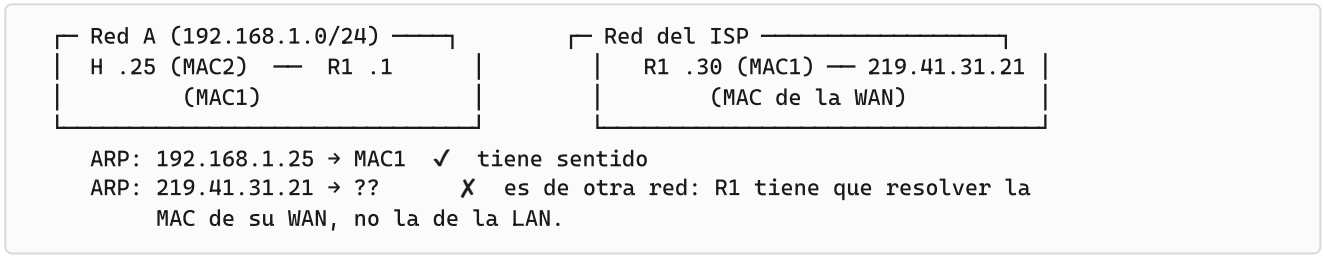
Lo que NO hace el router, y es el error conceptual más común: **no** mira la IP de origen para decidir, y **no** reescribe las IP al reenviar — solo cambia las tramas de capa 2 **de su enlace de salida**. Por eso la IP **no cambia**

de salto (salvo NAT, Clase 31) y la **MAC cambia en cada uno**.

1.5 ARP: por qué existe

El problema, en una frase. En IPv4 la dirección es **jerárquica y lógica**: `192.168.1.25` dice "esta IP está en la red 192.168.1.0/24" y nada del hardware. En Ethernet la dirección es **plana y física**: una MAC de 48 bits que **no codifica nada** sobre en qué red estás. **Son dos espacios de nombres distintos para identificar lo mismo.**

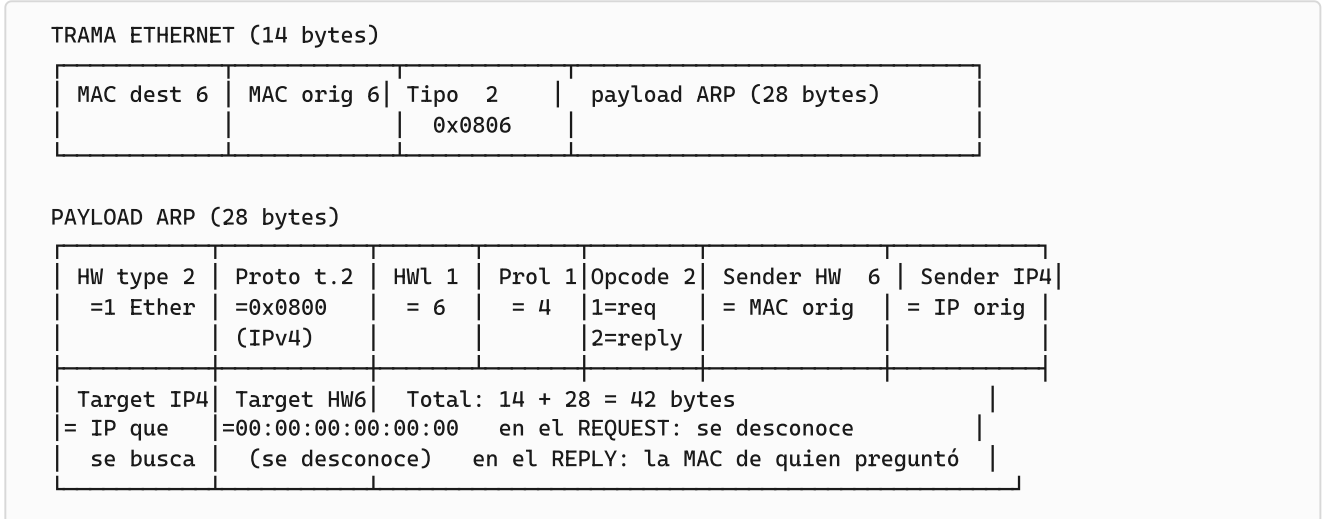
Por qué no alcanza con la IP. La trama de Ethernet tiene un campo obligatorio de **MAC destino**. El host no puede poner ahí una IP, ni inventar una MAC: tiene que **preguntar** "*¿qué MAC tiene la IP 10.0.0.1?*". Y hay que preguntárselo a alguien, porque **la MAC solo tiene sentido local** dentro de un enlace: por eso **ARP nunca cruza un router** — cada router responde solo por sus propias interfaces.



> **ARP = Address Resolution Protocol**, protocolo auxiliar de la suite TCP/IP que resuelve la correspondencia IP ↔ MAC **en un mismo enlace**. Definido en la `RFC 826`.

1.6 El formato del paquete ARP

Un paquete ARP viaja **dentro** de una trama Ethernet, con `Ethertype = 0x0806` (que es lo que le dice al receptor "esto no es IP, es ARP").

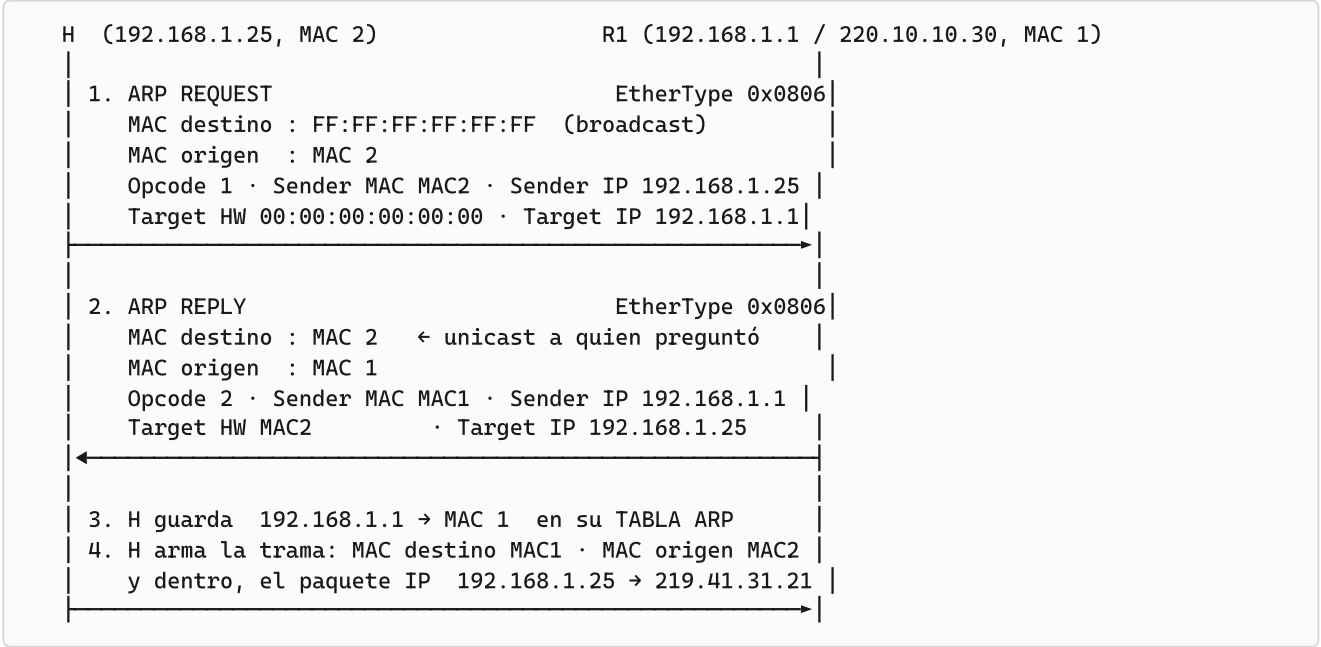


Los dos campos que hacen toda la magia:

Campo	En un **Request**	En un **Reply**	Por qué
Target IP address	La IP **buscada**	La IP **del que preguntó**	El request necesita saber a quién preguntar; el reply, a quién contestar
Target HW address	`00:00:00:00:00:00`	La MAC **del que preguntó**	El request todavía no la sabe

Esa simetría (el request pregunta con `Target IP`, el reply responde con `Sender IP`) es lo que hace que todo funcione sin una segunda ronda de pregunta.

1.7 La secuencia ARP completa

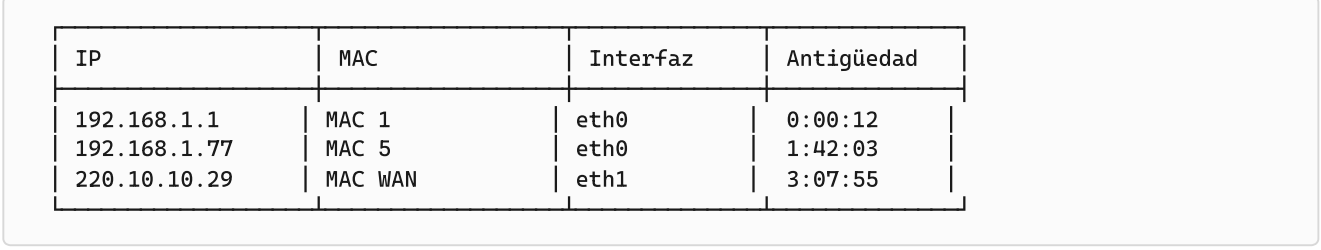


⚠️ **Request broadcast, reply unicast.** Es lo que dicen la `RFC 826` y las soluciones oficiales del `2010-03 P1a` y del `2006-08 P2`.

⚠️ **Las dos MAC del mismo sentido.** En el reply, **MAC origen** = la de quien responde (MAC 1, la de R1) y **MAC destino** = la de quien preguntó (MAC 2). Confundirlas es el error más común del apartado.

⚠️ **La tabla ARP se llena con lo que dice el campo *Sender*, no con lo que se preguntó.** H preguntó por `192.168.1.1` y guarda `192.168.1.1 → MAC 1`, que es justo el `Sender IP` del reply.

1.8 La tabla ARP: *aging*, *gratuitous* y *spoofing*



IP	MAC	Interfaz	Antigüedad
192.168.1.1	MAC 1	eth0	0:00:12
192.168.1.77	MAC 5	eth0	1:42:03
220.10.10.29	MAC WAN	eth1	3:07:55

- **Qué guarda:** la correspondencia **IP → MAC**, más la **interfaz** por la que se alcanzó (necesaria en routers multihoming) y un **temporizador de antigüedad**.

- **¿Por qué una tabla?** Por **retardo** (un ARP son 2 tramas y 1 RTT) y por **carga** (un broadcast por paquete satura la LAN). Es un caché, no corrección.
- **Aging:** cada entrada tiene un temporizador que **se renueva** al usarse y **expira** si no se usa. Los valores son **de implementación**, no los manda IEEE: del orden de **30 s a 5 minutos** en Windows y Linux. Expirar es **necesario**, porque un host puede cambiar de MAC y la entrada vieja lo mandaría al lugar equivocado.
- **ARP *gratuitous*:** un Request donde **Sender IP = Target IP**, o sea "*yo soy el dueño de esta IP*", a broadcast y **sin respuesta**. Sirve para **anunciar** una dirección nueva al arrancar, **detectar duplicados**, **actualizar las cachés** tras un *failover* de gateway, y como **anuncio de VRRP**.
- ⚠️ **Y es la fuente clásica de *ARP spoofing: si un atacante manda un gratuitous diciendo "192.168.1.1 es MI MAC", todos los hosts de la LAN le creen* y le mandan su tráfico. Eso es*** cache poisoning***, la base de casi todos los ataques*** **man-in-the-middle* de LAN. Las defensas:** ARP inspection **en los switches y** fijar la MAC del gateway** en cada host.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - `RFC 826`: request en **broadcast**, reply en **unicast**, opcodes 1 y 2; los 14 bytes de Ethernet + 28 de payload, con los cuatro campos de direcciones. - La tabla ARP es un **caché con expiración**. - El *gratuitous* se sigue usando (arranque, DHCP, failover, VRRP) y **sigue siendo una fuente clásica de *ARP spoofing***. - La convención "C" y el formato de tres columnas.

Cambió o hay que reencuadrar: - **En IPv6 no hay ARP: hay Neighbor Discovery** (`RFC 4443`), que hace lo mismo con mensajes **NS/NA de ICMPv6** sobre **multicast**, y además agrega **autoconfiguración** y **DAD**. No hay *proxy ARP* en IPv6. Es el contraste que pide el `2010-12 P5`. - **`Proxy ARP` (`RFC 1027`) sigue existiendo** en VPNs sitio a sitio, pero es cada vez más un **anti-patrón** de la era IPv4-NAT. - **El *aging* es de implementación**: nunca lo presentes como "el estándar dice X".

> **Consecuencia para el examen: nada cambia** en el formato ni en la aritmética. Lo único a añadir, si te piden actualizar, es el NDP como equivalente en IPv6.

1.9 RARP: el problema inverso, y por qué murió

```
ARP   "¿qué MAC tiene la IP 192.168.1.25?"     →  respuesta:  la MAC
RARP  "¿qué IP y qué MAC tiene esta tarjeta?"  →  respuesta:  la IP + la MAC
```

El problema que resuelví. Un host recién arrancado, con la tarjeta Ethernet en la LAN pero **sin dirección IP**, no puede usar DHCP: no tiene con quién hablar. La pregunta de RARP es esa: "*sos una interfaz con esta MAC; decime qué IP me corresponde*". El servidor RARP tiene una tabla **MAC → IP** y contesta. Se define en la **`RFC 903`**, y sus opcodes (3 y 4) viven en el formato de la `RFC 826`.

Por qué quedó obsoleto: 1. **Solo devolvía la IP.** No entregaba **máscara**, ni **gateway por defecto**, ni **DNS**. Con una IP suelta no se puede enrutar nada: el host no sabe si el destino es local o remoto. 2. **El servidor necesitaba una entrada por host**, escrita a mano: agregar un host implicaba editar la tabla del servidor. 3. **BOOTP** lo reemplazó (🔍 `RFC 951`): pasó a entregar IP, máscara, gateway y archivo de arranque, todo en un mensaje, y de forma automática desde un *pool*. 4. **Después llegó DHCP** (🔍 `RFC 2131` / `2132`), que es BOOTP renovado con *leases* (alquilaciones con vencimiento), lo que resolvió el problema de la dirección fija atada al hardware.

> ⚠️ **tema en desuso.** Si un examen te pregunta RARP, la respuesta es "se usaba en los 80, quedó obsoleto con BOOTP y DHCP". Si aparece como **pregunta teórica**, es porque quieren ver si entendés el salto de "dirección fija" a "dirección alquilada con vencimiento".

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La tabla de un host con **dos** interfaces

> **Fuente:** Examen de marzo de 2001, `sirc0102`, Problema 1 (30 pts). *Enunciado: "Se dispone de un computador que se comunica con otros mediante TCP/IP. Para hacerlo posee dos tarjetas ethernet que están conectadas a diferentes LAN's. El sistema conoce: 1) la dirección IP, la máscara de subred y la dirección IP del gateway por defecto (si ha sido especificado) para cada una de las dos tarjetas. 2) La tabla de ruteo estática, con entradas del tipo `<dirección de red de destino, máscara de subred, gateway a utilizar>`. Se pide: escribir el proceso que determina por cuál o cuáles tarjetas se debe enviar un paquete IP dada la dirección IP destino."*

La lógica, en cuatro pasos, en este orden exacto:

```
Paso 1 — ¿el destino es LOCAL de alguna de las dos LAN?
    for i en {1,2}:   si  D & mascara[i] == ip[i] & mascara[i]  ⇒  tarjeta i

Paso 2 — ¿hay entrada en la tabla estática que coincida?
    for ℓ:              si  D & mascara[ℓ] == red[ℓ] & mascara[ℓ]  ⇒  manda ℓ
    (si hay varias, gana la de prefijo MÁS LARGO)

Paso 3 — ¿por dónde sale el gateway de esa línea?
    si  gw[ℓ] & mascara[1] == ip[1] & mascara[1]  ⇒  tarjeta 1
    si  gw[ℓ] & mascara[2] == ip[2] & mascara[2]  ⇒  tarjeta 2

Paso 4 — si no hubo coincidencia, los gateways por defecto:
    ¿existen los DOS?  ⇒  se manda por AMBAS tarjetas        (devuelve 3)
    ¿existe uno solo?  ⇒  por esa                            (1 o 2)
    ¿ninguno?           ⇒  se descarta                       (0)
```

```
// Tarjeta_red[i] = { dir_IP, Sn_mask, Def_gw }   i en 1..2
// Tabla_ruteo[l] = { dir_IP, Sn_mask, Def_gw }
// Devuelve: 0 ninguna · 1 tarjeta 1 · 2 tarjeta 2 · 3 AMBAS

function selec_tarjeta (dirIP: IP_addr): int
begin
  // Paso 1: ¿es una de mis LANs?   (AND bit a bit)
  if (dirIP and Tarjeta_red[1].Sn_mask =
      Tarjeta_red[1].dir_IP and Tarjeta_red[1].Sn_mask) then return 1;
  if (dirIP and Tarjeta_red[2].Sn_mask =
      Tarjeta_red[2].dir_IP and Tarjeta_red[2].Sn_mask) then return 2;

  // Paso 2: tabla de ruteo estática
  res := 0;
  for lin := 1 to MAX_ROUTE do
    if (dirIP and Tabla_ruteo[lin].Sn_mask =
        Tabla_ruteo[lin].dir_IP and Tabla_ruteo[lin].Sn_mask) then
      res := lin;            // ⚠ la oficial se queda con la ÚLTIMA, no con
                             //   la más específica (ver análisis)
  if (res > 0) then begin
    // Paso 3: ¿el gateway está en la LAN de la tarjeta 1 o de la 2?
    if (Tabla_ruteo[res].Def_gw and Tarjeta_red[1].Sn_mask =
        Tarjeta_red[1].dir_IP and Tarjeta_red[1].Sn_mask) then return 1;
    if (Tabla_ruteo[res].Def_gw and Tarjeta_red[2].Sn_mask =
        Tarjeta_red[2].dir_IP and Tarjeta_red[2].Sn_mask) then return 2;
  end;

  // Paso 4: gateways por defecto
  if (existe Tarjeta_red[1].Def_gw) and (existe Tarjeta_red[2].Def_gw) then
    return 3;                              // por AMBAS
  if (existe Tarjeta_red[1].Def_gw) then return 1;
  if (existe Tarjeta_red[2].Def_gw) then return 2;
  return 0;                                // no hay por dónde salir
end;
```

Después del `selec_tarjeta`, dos cosas más (acá está el punto de unión con esta clase): 1. **La IP de origen es la de la tarjeta por la que se sale**: `tabla_ruteo[n].dirIP`. 2. **El ARP es del destino final, no del destino del paquete**: si el destino es **local** → ARP de `dirIP`; si salís por la tabla o por un default → **ARP del gateway**. Es el criterio de §1.4: primero se decide la **interfaz de salida**, y solo después se resuelve la MAC de **a quién** se entrega la trama.

> **Análisis de la solución oficial:** la lógica y el orden son correctos (LAN → tabla estática → default). Dos cosas a corregir: (a) el `res := lin` dentro del `for` se queda con la **última** coincidencia, no con la de **prefijo más largo**; con dos entradas que coinciden devuelve la que vino último, que puede no ser la más específica. Hay que comparar las máscaras y quedarse con la **más larga**. (b) Conviene recordar que **los dos AND tienen que hacerse bit a bit**, no con comparación de strings.

2.2 Ejercicio 2 — ARP entre dos LANs, con un router en el medio

Enunciado. H1 quiere alcanzar un servidor en Internet, a través de un router, y su tabla ARP está **vacía**.

```
 H1 (192.168.1.10, MAC1)     Router (LAN 192.168.1.1, MAC2 · WAN 203.0.113.5, MAC3)

1) H1 necesita la MAC de 192.168.1.1  (el destino NO es local: es remoto)
   ── ARP REQUEST  MAC dest ff:ff:ff:ff:ff:ff · MAC orig MAC1 ──────────►
      Sender IP 192.168.1.10 · Target IP 192.168.1.1
   ◄── ARP REPLY    MAC dest MAC1 · MAC orig MAC2 ──────────────────────
      Sender IP 192.168.1.1 · Target IP 192.168.1.10

2) H1 arma la trama de datos, SIN tocar el paquete IP:
   MAC dest MAC2 · MAC orig MAC1 · Ethertype 0x0800
   IP origen 192.168.1.10  →  IP destino (la del servidor, digamos 142.250.1.1)
   ──────────────────────────────────────────────────────────────────►

3) El router tira la MAC2 y el Ethertype, lee la IP destino, ve que no es de
   ninguna de sus redes, consulta SU tabla (0.0.0.0/0 → 203.0.113.1), hace ARP
   por 203.0.113.1 en su WAN y reenvía con MAC destino = la del ISP.
```

⚠️ **Lo que enseña, en tres líneas:** la **resolución de MAC es local a cada enlace**; el **paquete IP viaja intacto** de extremo a extremo (aquí; NAT lo rompe, Clase 31); y el router decide la **ruta** con la IP destino y la **trama** con el siguiente salto.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen `2011-03`, Problema 2 a, b.i y b.ii — tablas, ARP y MACs

> **Fuente:** Examen `eirc1103` / `sirc1103`, 21 de febrero de 2011, Problema 2 (30 puntos), apartados a, b.i y b.ii. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

Enunciado (literal)

> Una empresa tiene una red A (LAN Ethernet con direccionamiento privado) con 15 servidores y 70 puestos de trabajo, y se desea que puedan acceder a un servidor web en Internet con dirección IP `219.41.31.21`. Para ello el router R1 implementa NAT utilizando su dirección IP pública. **a)** Especifique la tabla de enrutamiento de R1 y del Host H. **b)** i. Suponiendo que la tabla ARP del Host H está vacía, indicar los mensajes ARP (MAC de origen, MAC de destino y descripción del contenido) intercambiados en la Red A cuando el Host H inicia su comunicación con Servidor Web (asuma que Host H conoce la dirección IP de Servidor Web). ii. Indique las direcciones MAC de las tramas y las direcciones IP de los paquetes correspondientes a un flujo de datos entre el Host H y el Servidor Web considerando ambos sentidos (considere únicamente la Red A).

Datos de la figura: H = `192.168.1.25`, **MAC 2**. R1 = `192.168.1.1` (**MAC 1**) en la LAN, y `220.10.10.30` en el enlace `220.10.10.28/30`.

a) Las dos tablas

```
Tabla del host H                          Tabla de R1
Red destino / Máscara   Gateway          Red destino / Máscara    Gateway
──────────────────────────────────────   ──────────────────────────────────────
192.168.1.0/24          C                192.168.1.0/24           C     (1)
0.0.0.0/0               192.168.1.1 (2)  220.10.10.28/30          C     (1)
                                           0.0.0.0/0                220.10.10.29 (2)
```

⚠️ **Las dos entradas `C` de R1, y por qué.** R1 tiene **dos** interfaces: una en la red A y otra en el enlace WAN, y cada una genera su entrada conectada. Es el error más común poner una sola: si a R1 le llega un paquete para `220.10.10.29` (el router del ISP) y no tiene la entrada `/30`, lo tira por la interfaz equivocada o lo descarta.

b.i) Los dos mensajes ARP

	MAC orig	MAC destino	Tipo y contenido
Request (H)	MAC 2	FF:FF:FF:FF:FF:FF (broadcast)	ARP Request preguntando por la MAC asociada a la IP 192.168.1.1 · Opcode 1 · Sender MAC MAC2 · Sender IP 192.168.1.25 · Target MAC 00:00:00:00:00:00 · Target IP 192.168.1.1
Reply (R1)	MAC 1	MAC 2 (unicast al que preguntó)	ARP Reply respondiendo que la MAC asociada a la IP 192.168.1.1 es MAC 1 · Opcode 2 · Sender MAC MAC1 · Sender IP 192.168.1.1 · Target MAC MAC2 · Target IP 192.168.1.25

> **Por qué un broadcast y no un unicast:** H **no sabe** la MAC de `192.168.1.1`, así que no tiene a quién dirigir la pregunta. El reply sí puede ser unicast, porque **ya conoce** la MAC del que preguntó (la tiene en el `Sender MAC` del request que recibió).

b.ii) MACs e IPs de las tramas, en los dos sentidos

```
Sentido H → Servidor Web                Sentido Servidor Web → H
MAC origen  : MAC 2                     MAC origen  : MAC 1
MAC destino : MAC 1                     MAC destino : MAC 2
IP  origen  : 192.168.1.25              IP  origen  : 219.41.31.21
IP  destino : 219.41.31.21              IP  destino : 192.168.1.25
```

> **El análisis que hay que hacer en voz alta** (es lo que suma puntos): "*mirá las cuatro IP: en los dos sentidos son las mismas. Lo único que cambia entre un paquete y su respuesta es el orden: se invierten origen y destino. Las MAC también se invierten, pero no porque el paquete 'vuelva': porque la respuesta se emite desde la interfaz de R1 hacia H, y esa trama tiene que llevar la MAC de H como destino. Las IP no se tocan porque este es el tramo de la capa 3 dentro de la red A, antes de que R1 aplique el NAT — que es exactamente lo que pide el apartado c.*"

⚠️ **El error clásico:** poner `IP destino 220.10.10.30` en el sentido Web → H. **No.** `220.10.10.30` es la IP que R1 *muestra hacia afuera* (NAT), y por el punto de vista de H el destino sigue siendo él mismo. El NAT se explica entero en el apartado c, que es la Clase 31.

3.2 Examen `2010-03`, Pregunta 1 a — ARP entre dos hosts

> **Fuente:** Examen 24 de febrero de 2010, `sirc1003`, Pregunta 1 (8 puntos), a.

Enunciado (literal)

> Dados dos hosts A y B, con sus respectivas direcciones IP y MAC notadas como sigue: `IP_A`, `IP_B`, `MAC_A` y `MAC_B`. Describa los mensajes ARP intercambiados entre A y B que permiten a A determinar la dirección MAC de B. Deberá indicar al menos direcciones MAC e IP utilizadas en cada mensaje, tanto a nivel de los cabezales Ethernet y payload ARP.

Solución oficial (verbatim)

> **A→B ARP Request con:** `Src MAC: MAC_A` · `Dst MAC: ff:ff:ff:ff:ff:ff` · Payload: `Sender MAC Addr MAC_A`, `Sender IP Addr IP_A`, `Target MAC Addr 00:00:00:00:00:00`, `Target IP Addr IP_B` **B→A ARP Reply con:** `Src MAC MAC_B` · `Dst MAC MAC_A` · Payload: `Sender MAC Addr MAC_B`, `Sender IP Addr IP_B`, `Target MAC Addr MAC_A`, `Target IP Addr IP_A`

Mi resolución, con la tabla completa

#	Sentido	MAC origen	MAC destino	IP origen	IP destino
1	A → todos (request)	MAC_A (A la pone)	ff:ff:ff:ff:ff:ff (broadcast)	IP_A (la de A)	IP_B (la que busca)
2	B → A (reply)	MAC_B (B la pone)	MAC_A (unicast a A)	IP_B (la de B)	IP_A (la de A)

```
En el request, Target MAC = 00:00:00:00:00:00   ← porque A todavía no la sabe
En el reply,  Target MAC = MAC_A                ← porque A se la puso en el request
```

El punto conceptual que la cátedra quiere (es la parte b del mismo ejercicio): **ARP es "independiente" de IP**, vive en el `Ethertype 0x0806` y no en el `0x0800`. Por eso IPv6 no lo usa y sí usa **Neighbor Discovery**, que está **dentro** de ICMPv6 sobre IPv6: "*IPv6 es una implementación más limpia que IPv4*". El mismo punto se vuelve a preguntar en el `2010-12 P5`.

3.3 Examen `2010-03`, Problema 1 a — tablas de R1, R2 y los hosts

> **Fuente:** Examen 24 de febrero de 2010, `sirc1003`, Problema 1 (30 pts), a. Topología: tres subredes /24 (`192.168.0.0/24`, `192.168.1.0/24`, `192.168.2.0/24`) con 100 hosts cada una, interconectadas por R1 y R2, enrutamiento estático.

Enunciado (literal)

> **a) i.** Indique las tablas de enrutamiento de los routers R1 y R2 de forma que permita conectividad extremo a extremo entre todos los equipos de la red. **ii.** Indique las tablas de enrutamiento para los hosts

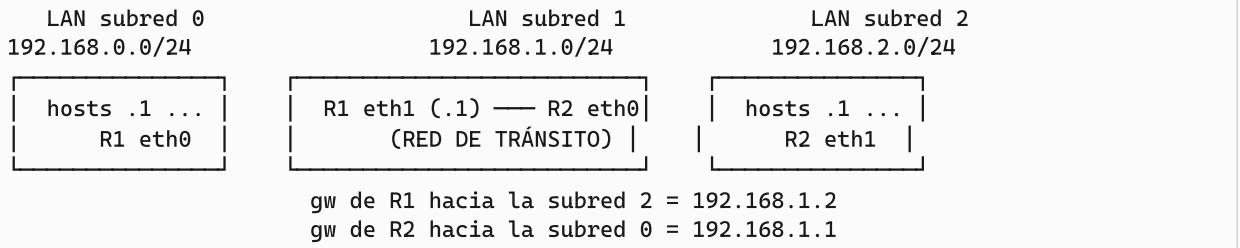
> de cada subred.

Solución oficial (verbatim)

```
Router R1                          Router R2
Network        Gateway  Interface  Network        Gateway   Interface
192.168.0.0/24      -    eth0      192.168.2.0/24      -    eth1
192.168.1.0/24      -    eth1      192.168.1.0/24      -    eth0
192.168.2.0/24  192.168.1.2  eth1   192.168.0.0/24  192.168.1.1  eth0

Hosts subred 1                      Hosts subred 2                Hosts subred 3
192.168.0.0/24      -    eth0      192.168.1.0/24      -   eth0  192.168.2.0/24   -  eth0
0.0.0.0/0     192.168.0.1 eth0     0.0.0.0/0     192.168.1.1 eth0 0.0.0.0/0  192.168.2.1 eth0
```

Mi resolución: leer la tabla, que es lo que no es obvio

La tabla oficial parece rara ("¿R1 tiene conectada la subred 1 **y** usa `192.168.1.2` como gateway **de la subred 1**?"), pero es coherente: **la subred `192.168.1.0/24` es la red de tránsito** entre R1 y R2.



⚠️ **El error típico con esta topología** es poner `192.168.1.2` en la columna **Interface** en lugar de la de **Gateway**, o creer que por estar en la misma red el next hop no hace falta. **El next hop siempre se pone**, aunque sea local: sin esa columna, el router no sabe a qué MAC armar la trama de salida.

⚠️ **Y la última línea de R1:** `192.168.2.0/24 → 192.168.1.2 → eth1`. Como el next hop `192.168.1.2` está en una red **directamente conectada** (eth1), la distancia es **1**, no 2: los saltos se cuentan desde **este** router.

3.4 Examen `2009-12`, Problema 1 a — tablas y la secuencia completa de tramas

> **Fuente:** Examen 18 de diciembre de 2009, `sirc0912`, Problema 1 (30 pts), a. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0912.pdf

Topología. Tres subredes de clase C: `192.168.111.0/24` (LAN1, con R1 en `192.168.111.1`), `192.168.108.0/24` (LAN2, con R1 en `192.168.108.1` y R2 en `192.168.108.2`) y `192.168.109.0/24` (LAN3, con R2 en `192.168.109.1`).

a-i) y a-ii) Tablas de routers y de hosts

```
Router R1                                Router R2
Network        Gateway  Interface        Network        Gateway   Interface
192.168.111.0/24    -    eth0           192.168.109.0/24    -    eth1
192.168.108.0/24    -    eth1           192.168.108.0/24    -    eth0
192.168.109.0/24  192.168.108.2  eth1     192.168.111.0/24  192.168.108.1  eth0

Hosts LAN1         Hosts LAN2           Hosts LAN3
111.0/24 → C       108.0/24 → C          109.0/24 → C
0.0.0.0/0 →        0.0.0.0/0 →          0.0.0.0/0 →
    192.168.111.1      192.168.108.1          192.168.109.1
```

a-iii) La secuencia completa para mandar un paquete de C a F

Paquete IP: **origen** `192.168.108.3` (host C, en LAN2) → **destino** `192.168.109.3` (host F, en LAN3). Tablas ARP **vacías**.

MACs de la figura: host C = `00-08-74-4C-7B-1C` · R1 eth1 = `00-00-0C-4C-7F-1B` · R2 eth0 = `00-00-0C-4C-8F-1A` · R2 eth1 = `00-00-0C-4C-8F-1B` · host F = `00-08-74-4C-7A-1D`.

#	MAC origen	MAC destino	IP origen	IP destino	Proto	Tipo de
1	00-08-74-4C-7B-1C	FF-FF-FF-FF-FF-FF	192.168.108.3	–	ARP	Consulta
2	00-00-0C-4C-7F-1B	00-08-74-4C-7B-1C	192.168.108.1	192.168.108.3	ARP	Respuesta
3	00-08-74-4C-7B-1C	00-00-0C-4C-7F-1B	192.168.108.3	192.168.109.3	IP	Datagrama
4	00-00-0C-4C-7F-1B	FF-FF-FF-FF-FF-FF	192.168.108.1	–	ARP	Consulta
5	00-00-0C-4C-8F-1A	00-00-0C-4C-7F-1B	192.168.108.2	192.168.108.1	ARP	Respuesta
6	00-00-0C-4C-7F-1B	00-00-0C-4C-8F-1A	192.168.108.3	192.168.109.3	IP	Datagrama
7	00-00-0C-4C-8F-1B	FF-FF-FF-FF-FF-FF	192.168.109.1	–	ARP	Consulta
8	00-08-74-4C-7A-1D	00-00-0C-4C-8F-1B	192.168.109.3	192.168.109.1	ARP	Respuesta
9	00-00-0C-4C-8F-1B	00-08-74-4C-7A-1D	192.168.108.3	192.168.109.3	IP	Datagrama

Datagramas 1–2, 4–5 y 7–8 son ARP. Los 3, 6 y 9 son el paquete IP, en tres enlaces distintos.

Las tres cosas que hay que decir después de la tabla (es lo que convierte la tabla en respuesta de 10 puntos):

1. **Son 9 tramas porque hay 3 tránsitos y en cada uno hay que resolver una MAC nueva.** En cada salto cambian la MAC origen y la MAC destino; **la IP origen y la IP destino son siempre las mismas** (extremo a extremo).
2. **La primera trama va al gateway por defecto**, porque `192.168.109.3` no es de la subred de C (`108.0/24`). Es §1.4 aplicado al host.
3. **Entre las tramas 3 y 6 solo cambia la capa 2:** R1 tira el cabezal Ethernet y el `Ethertype`, lee el IP destino, busca en su tabla (`192.168.109.0/24 → 192.168.108.2`) y por eso necesita la MAC de R2 (**trama 4**). Lo mismo entre 6 y 9.

⚠️ **Errores típicos:** olvidar que **cada** router necesita su propio ARP (son 3 requests, no 1); poner la IP del router de destino en lugar de la del host; y olvidar que en la trama de ARP la columna **IP destino va vacía** en el request, porque el paquete ARP todavía no tiene destino IP.

PARTE 4 — CIERRE

📌 Resumen

1. Formato de 3 columnas: **Red destino/Máscara · Gateway (next hop) · Distancia**. **"C" = conectada**, distancia **1**, y nunca se borra.
2. Tres tipos: **conectada** (la pongo yo), **aprendida** (de un vecino) y **por defecto** (`0.0.0.0/0`). Sin default, el paquete se descarta con ICMP tipo 3.
3. `D & máscara == red` es el test de coincidencia, y gana el **prefijo más largo**, no el de menor distancia.
4. Un router **no** reescribe las IP al reenviar: solo cambia las tramas de capa 2 de su enlace de salida.
5. ARP existe porque **IP es jerárquica y lógica, MAC es plana y física**, y la trama necesita obligatoriamente una MAC destino.
6. 14 bytes de Ethernet + 28 de payload. En el **request** el `Target MAC` es `00:00:00:00:00:00`; en el **reply** es la MAC de quien preguntó.
7. **Request broadcast, reply unicast**: el que pregunta no sabe a quién preguntar; el que responde sí sabe a quién contestar.
8. La tabla ARP es un **caché con aging** (valores de implementación), el **gratuitous** se sigue usando para anuncio, failover y VRRP — y sigue siendo la fuente clásica del ***ARP spoofing***.
9. **RARP** (`RFC 903`) murió: solo daba la IP, sin máscara ni gateway, y necesitaba una entrada por host. Lo reemplazaron **BOOTP** y después **DHCP**.
10. En IPv6, **ARP es NDP** (NS/NA sobre ICMPv6, en multicast) — por eso el `2010-03 P1b` lo llama "más limpio".

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un router tiene `192.168.1.0/24 → C`, `192.168.1.128/25 → 10.0.0.2`, `0.0.0.0/0 → 10.0.0.1`. ¿Por dónde sale un paquete a `192.168.1.200`?

▶ Respuesta

2. ¿Por qué el ARP Request va en broadcast y el Reply en unicast?

▶ Respuesta

3. Escribí los cuatro campos de direcciones de un ARP Request de H (`192.168.1.25`, MAC 2) que pregunta por `192.168.1.1`, y los del Reply de R1.

▶ Respuesta

4. Un paquete va de H (`192.168.1.25`) a `219.41.31.21` a través de R1 y vuelve. ¿Cuáles son las cuatro IP y las dos MAC en cada sentido?

▶ Respuesta

5. ¿Por qué ARP no cruza un router?

▶ Respuesta

6. Un host tiene dos interfaces. ¿Puede un paquete salir por las dos a la vez?

▶ Respuesta

7. ¿Por qué RARP quedó obsoleto y qué lo reemplazó?

▶ Respuesta

8. ¿Qué cambia de todo esto en IPv6?

▶ Respuesta

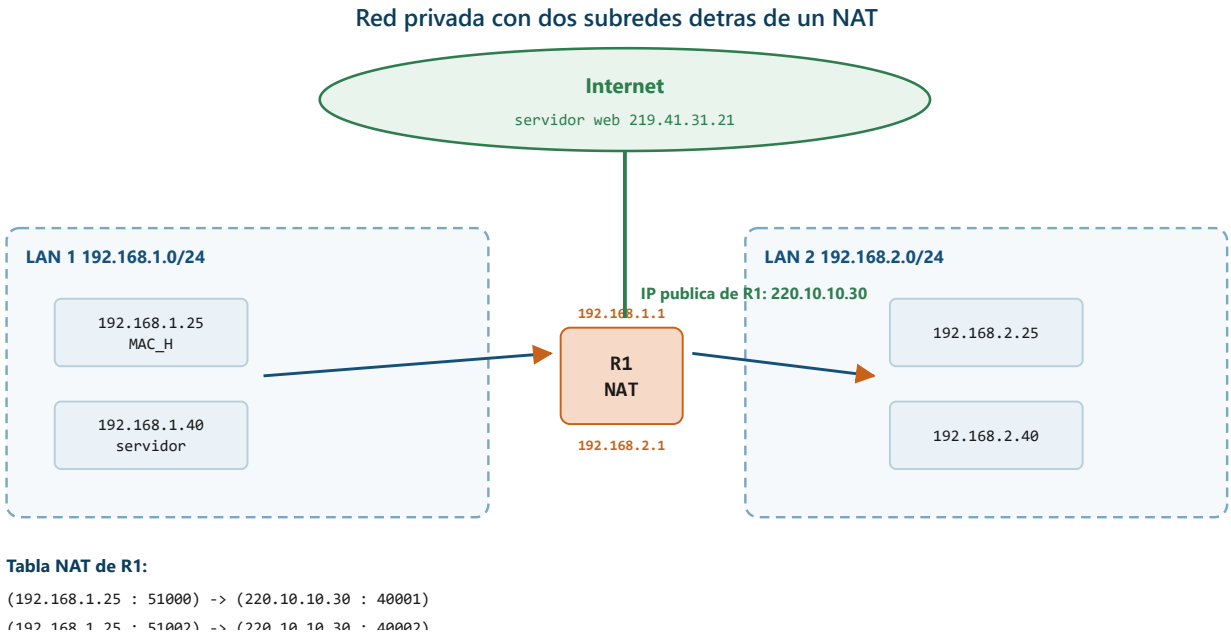
🔗 Referencias

- `eirc1103` / `sirc1103` (21-feb-2011) — Problema 2 a, b.i, b.ii: tablas de R1 y H, secuencia ARP, MACs en los dos sentidos (§3.1) · d: subredes (Clase 28)
- `sirc1003` (24-feb-2010) — Pregunta 1 a: ARP entre A y B (§3.2) · Problema 1 a: tablas de R1, R2 y hosts (§3.3)
- `sirc0912` (18-dic-2009) — Problema 1 a-i-iii: tablas y la secuencia de 9 tramas (§3.4) · b: qué pasa al cambiar R2 por un switch
- `sirc0608` (5-ago-2006) — Pregunta 2: por qué ARP y cómo funciona (§1.5–1.7)
- `sirc0102` (2-mar-2001) — Problema 1: `selec_tarjeta`, host con dos interfaces (§2.1)
- `sirc0712` (1-dic-2007) — Problema 2: ARP con destino local vs. remoto; y dos hosts compartiendo una IP
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red"
- **RFC 826** — ARP · **RFC 903** — RARP 🔍 · **RFC 951** — BOOTP 🔍 · **RFC 2131/2132** — DHCP 🔍
- **RFC 1027** — Proxy ARP · **RFC 4443** — ICMPv6 (NDP) · **RFC 8200** — IPv6
- **Clase anterior:** 29 — Taller de subnetting · **Clase siguiente:** 31 — NAT, PAT y sus hacks

Clase 31

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 Qué es la NAT y por qué apareció
1.2 Traducción de dirección contra traducción de puerto
1.3 Las tres fases de la NAT
1.4 Por qué la NAT rompe el aislamiento de capas
1.5 Las tres soluciones para que un cliente con IP pública alcance a un servidor privado
1.6 *NAT hairpinning* (el bucle hacia atrás)
1.7 CGNAT: la NAT del operador
2.1 Ejercicio 1 — Armar la tabla NAT con 4 sesiones simultáneas
2.2 Ejercicio 2 — ¿Por qué no funciona el P2P detrás de un CGNAT?
2.3 Ejercicio 3 — Por qué la NAT rompe IPsec en ESP
2.4 Ejercicio 4 — ¿Cuál de las tres soluciones corresponde a cada escenario?
3.1 ⭐ ⭐ Examen 2011-02, Pregunta 4 (10 puntos) — NAT, capas y las tres soluciones
3.2 ⭐ Examen 2011-03, Problema 2, apartado c — las modificaciones de R1
3.3 ⭐ ⭐ Examen 2008-12, Problema 1 (30 puntos) — NAT para balanceo de carga
📌 Resumen
✅ Autoevaluación
🔗 Referencias



Bloque: D (Capa de Red) · **Semana:** 11 · **Duración estimada:** 120 min **Correlativa:** Clase 30 (tablas de enrutamiento, next hop y ARP) · **Sigue con:** Clase 32 (enrutamiento por vector de distancias) **Material de referencia:** Tanenbaum cap. 5 · Exámenes `sirc1102`, `sirc1103`, `sirc0812` · RFC 8200, RFC 7915

> 🔍 = RFC **no verificado** en `ACTUALIZACIONES.md`: no lo cites sin confirmarlo.

🎯 Al terminar esta clase vas a poder

Tabla NAT con traduccion de puertos (PAT)

IP privada : puerto	IP publica : puerto	estado
192.168.1.25 : 51000	220.10.10.30 : 40001	activa
192.168.1.25 : 51002	220.10.10.30 : 40002	activa
192.168.1.40 : 33333	220.10.10.30 : 40003	activa
192.168.1.77 : 8080	220.10.10.30 : 40004	timeout 300 s

La clave de la tabla es (IP privada, puerto privado). El puerto es lo que permite multiplexar many-to-one: 3 sesiones, 1 sola IP publica.

Al volver un paquete se busca (IP publica, puerto publico) y se traducen la IP destino y el puerto destino.

Ese es el motivo por el que NAT rompe el aislamiento de capas:
el router tiene que entender TCP/UDP para funcionar.

1. **Explicar** por qué existe la NAT: escasez de IPv4 y direcciones privadas que no se rutean.
2. **Distinguir** traducción de **dirección** de traducción de **puerto** (PAT / NAPT), y armar una tabla NAT con varias sesiones simultáneas.
3. **Describir las tres fases**: salida, entrada y *timeout*.
4. **Describir campo por campo** qué cambia un router NAT en cada sentido, y resolver el `2011-03 Problema 2c`.
5. **Justificar** por qué la NAT **rompe el aislamiento de capas**, que es el `2011-02 P4b`.
6. **Explicar las tres soluciones** para que un cliente con IP pública alcance a un servidor privado, que es el `2011-02 P4c`.
7. **Explicar** el *hairpinning* y el **CGNAT** con sus tres problemas.
8. **Resolver entero** el `2008-12 Problema 1` (30 pts): NAT para balanceo de carga.

> **Prerequisito del examen:** `2011-03 Problema 2 c` (NAT + tabla), `2011-02 P4` (10 pts), `2008-12 Problema 1` (30 pts), `2010-03 P1` (subredes privadas y NAT).

PARTE 1 — TEORÍA

1.1 Qué es la NAT y por qué apareció

NAT (*Network Address Translation*) es una función de un **router** que **reescribe las direcciones IP** de los paquetes que lo atraviesan. Las dos razones por las que existe, y hay que decir las dos:

```
1. ESCASEZ DE IPv4.   Hay 2³² ≈ 4 300 millones de direcciones y el mundo
                      las consumió: el RIPE NCC hizo su última asignación
                      /22 el 25-nov-2019. APNIC se agotó en abr-2011, ARIN
                      en sept-2015, LACNIC en 2020 y AfriNIC en abr-2017.

2. DIRECCIONES PRIVADAS QUE NO SE RUTEAN.  10.0.0.0/8, 172.16.0.0/12 y
                     192.168.0.0/16 son de uso interno y NO se propagan por
                     Internet, así que una empresa puede reutilizar el mismo
                     direccionamiento en todas sus sedes.
```

El punto clave: la NAT **no es seguridad**, es **escasez**. Se hizo "hacia atrás" (era un truco de transiciones) y terminó siendo la razón por la que IPv4 sobrevivió quince años más.

⚠️ Actualización a 2026

Sigue igual: los tres bloques privados, la idea de la traducción y los puertos privados.

Cambió: - **El agotamiento de IPv4 es un hecho consumado.** RIPE hizo su última asignación `/22` el **25-nov-2019** y hoy mantiene una *waiting list* para un único `/24` recuperado. - **IPv6 dejó de ser "el futuro".** El **28 de marzo de 2026** el acceso a Google por IPv6 **superó el 50 % por primera vez (50,10 %)**. Pero es desigual: Francia e India por encima del 70 %, China por debajo del 5 %. **La cifra depende de qué se mida**: Cloudflare (tráfico por bytes) rondaba el 40 %. - **La meta de IPv6 es eliminar la NAT**, porque rompe la **extensibilidad extremo a extremo**, no es *stateful-friendly* con IPsec, y obliga a los hosts IPv6-only a usar 464XLAT para alcanzar contenido IPv4. - **NAT64 (`RFC 6146`) y DNS64 (`RFC 6147`)** maduraron con el prefijo `64:ff9b::/96` (`RFC 6052`) y `RFC 8880`. **SIIT (`RFC 6145`) quedó obsoleto por `RFC 7915`. 464XLAT sigue en borrador**: la transición no cerró.

1.2 Traducción de dirección contra traducción de puerto

```
TRADUCCIÓN DE DIRECCIÓN SOLA
   H1 .10 ──► R1 .200 ──► S:80          ⇐ UNA sola sesión: todas salen
                                           iguales desde afuera. Y un
                                           navegador abre 6 al mismo servidor.

TRADUCCIÓN DE DIRECCIÓN + PUERTO (PAT / NAPT)
  ┌──────┐                                  ┌──────┐
  │ H1   │ .10:40000 ──► R1 .200:50001 ──►  │  S   │ :80
  │ H1   │ .10:40001 ──► R1 .200:50002 ──►  │      │
  │ H2   │ .11:40000 ──► R1 .200:50003 ──►  │      │
  └──────┘                                  └──────┘

⇐ Cada sesión tiene un PAR (IP pública, puerto público) ÚNICO, así que R1
   soporta miles simultáneas (hasta ~64 000 por IP pública).
```

El nombre correcto: `PAT` (*Port Address Translation*) o `NAPT`. **En los exámenes de FING, "NAT" significa PAT**: se cambia IP **y** puerto.

La tabla NAT, con 3 sesiones simultáneas

```
┌──────────────────────────────────────────────────────────────────┐
│  (IP origen, puerto origen)   →  (IP pública, puerto público)    │
│  ──────────────────────────      ──────────────────────────────  │
│  (192.168.1.25, 40000)        →  (220.10.10.30, 50001)           │
│  (192.168.1.25, 40001)        →  (220.10.10.30, 50002)           │
│  (192.168.1.26, 40000)        →  (220.10.10.30, 50003)           │
│                                                                  │
│  La clave es el PAR interno; el valor es el PAR externo.         │
└──────────────────────────────────────────────────────────────────┘
```

Las tramas de los dos sentidos, para la sesión de `192.168.1.25:40000`:

```
⇐ SALIENTE (H → servidor)          ⇐ ENTRANTE (servidor → H)

IP   192.168.1.25   TCP src 40000   IP   220.10.10.30   TCP src    80
IP   219.41.31.21   TCP dst    80   IP   219.41.31.21   TCP dst 50001
          ──NAT──▶                              ◀──NAT──
Eth  MAC_H1 → MAC_R1                  IP   192.168.1.25   TCP src    80
                                       Eth  MAC_R1 → MAC_H1  TCP dst 40000
```

⚠️ **El campo que el router mira para decidir el sentido es el puerto de destino.** Por eso hace falta traducir puertos: la IP de destino sola no distingue tres sesiones al mismo servidor.

1.3 Las tres fases de la NAT

```
1. SALIDA     Llega paquete de la LAN. ¿(IP priv, port priv) está en la
               tabla?  NO → SE CREA entrada con puerto público nuevo.
2. ENTRADA    Llega paquete de Internet. ¿(IP pub, port pub) está en la
               tabla?  NO → SE DESCARTA.  SÍ → se traduce destino y puerto.
3. TIMEOUT    Pasan TIMEOUT s sin tráfico en esa entrada → se BORRA y el
               puerto público vuelve al pool.
```

Por qué hace falta el timeout: sin él la tabla crecería sin límite. **Pero** es un compromiso: corto, se pierde la correspondencia de una conexión larga; largo, la tabla se llena. Eso se llama **UDP hole punching** y es lo que rompe al P2P.

1.4 Por qué la NAT rompe el aislamiento de capas

Este es el `2011-02 P4b` (10 puntos) y es **el mejor punto conceptual de la clase**.

```
El AISLAMIENTO DE CAPAS consiste en que cada capa use a la de abajo como un
servicio opaco. En particular, la capa 3 decide el reenvío mirando SOLO la
dirección IP destino. ⇐ Un router NO necesita entender TCP para reenviar un
paquete TCP.

_____________________________________________________________________

CON NAT, un paquete que ENTRA al router tiene:

     IP destino = 220.10.10.30    ← es la IP DEL PROPIO ROUTER, y es ÚNICA
                                     para toda la LAN

⇐ La IP destino NO dice a qué máquina interna va. La única información que lo
   dice es el PUERTO DE DESTINO:

     puerto destino = 50001 → buscar en la tabla → 192.168.1.25:40000

⇐ Para decidir eso el router tiene que LEER EL PUERTO TCP/UDP, campo de la
   CABECERA TCP: es información de CAPA 4. Un dispositivo de capa 3
   dependiendo de un protocolo de capa 4
   ⇐ ESO ES ROMPER EL AISLAMIENTO DE CAPAS.
```

Las tres consecuencias concretas, que son lo que suma puntos:

```
1. ROMPE LA EXTENSIBILIDAD.  Cada capa debe poder cambiar sin romper las de
   arriba; con NAT no alcanza con "dejar pasar" lo que no se entiende: hay
   que INTERPRETARLO.
2. ROMPE LA TRANSPARENCIA.  El cliente ve su socket en 192.168.1.25:40000 y
   el servidor ve 220.10.10.30:50001. Ninguna es la dirección "verdadera":
   eso rompe IPsec en modo ESP (que cifra las cabeceras IP).
3. ROMPE EL P2P.  Como la traducción solo se crea al INICIAR una conexión
   saliente, un host detrás de la NAT no puede iniciar nada hacia afuera
   por su cuenta (ver §1.5).
```

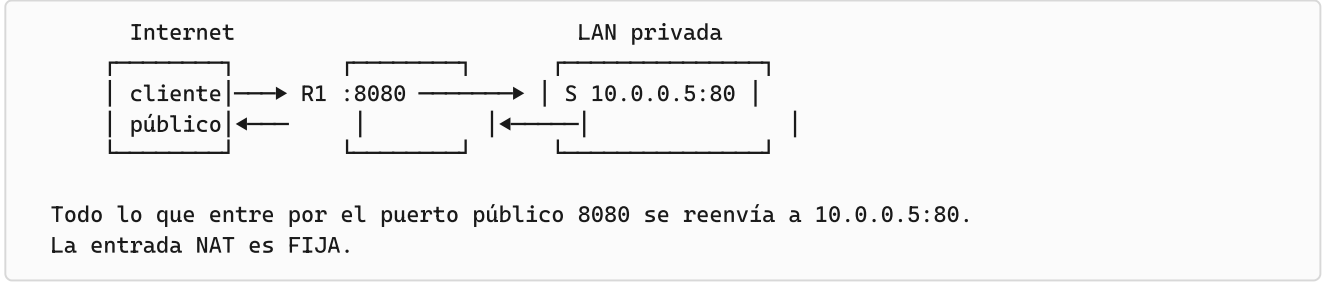
La frase de cierre, que es lo que el corrector quiere leer:

> La NAT rompe el aislamiento de capas porque un router que solo debería mirar la dirección IP destino se ve obligado a **mirar el puerto TCP/UDP** para saber a qué host interno corresponde el paquete. Eso convierte a un dispositivo de capa 3 en uno que depende de un protocolo de capa 4, y por lo tanto **cualquier protocolo nuevo de capa superior que no sea TCP o UDP deja de funcionar detrás de una NAT** sin que el router lo entienda.
>
> ⚠️ **Errores conceptuales típicos:** (i) decir que rompe el aislamiento porque "el router mira la IP privada" — falso, la IP es de capa 3 y es lo que siempre hace; lo que rompe es mirar **el puerto**; (ii) decir que rompe porque "el servidor ve otra IP" — eso es el síntoma, no la causa; (iii) olvidar el caso **ESP**.

1.5 Las tres soluciones para que un cliente con IP pública alcance a un servidor privado

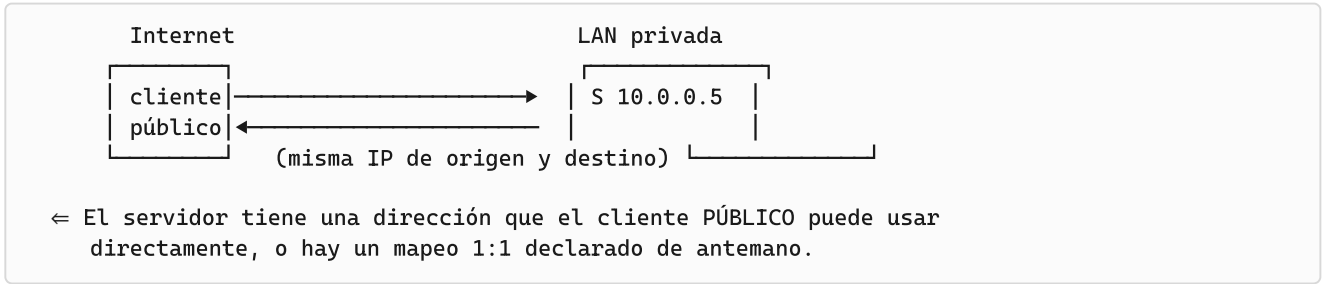
Este es el `2011-02 P4c`. Hay que presentarlas **como están**, pero nombrándolas bien.

La solución 1 — *Port forwarding* (NAT estática, o DNAT)



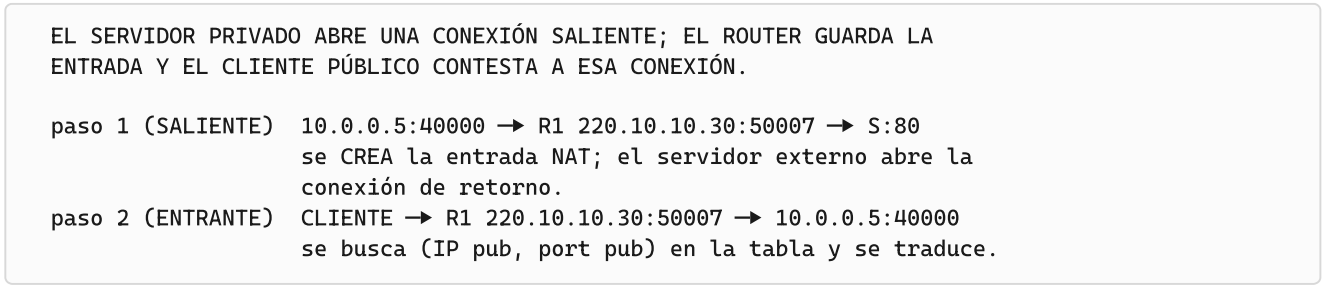
Requisitos: el servidor privado necesita **IP privada estable** (un DHCP *reservation* alcanza) y hay que **saber de antemano** puerto y protocolo. **No sirve para conexiones salientes de la LAN**: solo traduce en un sentido.

La solución 2 — NAT sin traducción (*full-cone NAT* o *endpoint-independent mapping*)



Cuándo se usa: cuando el servidor tiene dirección pública propia pero su **gateway por defecto** es un router que hace NAT (una máquina en la nube con IP elástica, por ejemplo). **Limitación:** alguien tiene que **configurar el mapeo de antemano**; si no, el paquete nunca vuelve (ver §1.6).

La solución 3 — El servidor inicia: invertir quién origina la conexión



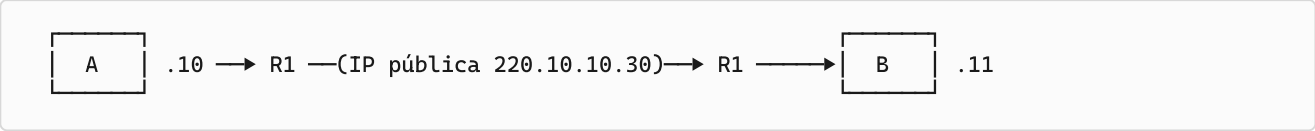
Por qué es la más usada: la NAT solo traduce lo que **sale**. Si el servidor privado manda primero un paquete (un keep-alive, un `REGISTER` de SIP, un *binding request* de STUN), la entrada queda creada y **cualquier paquete que entre desde Internet con ese par** se reenvía adentro. **Es la base de SIP, ICE (WebRTC), STUN y TURN**.

	1. *Port forwarding*	2. NAT sin traducción	3. El servidor inicia
Quién configura	el administrador	el administrador	**nadie**
Tabla NAT	entrada fija	no hay traducción	entrada **dinámica**
IP del servidor	privada	mapeo 1:1 declarado	privada
Sin tocar nada	no	no	**sí**
Uso típico	servidores web, cámaras	IP elástica	**VoIP, WebRTC, P2P**

Regla mental: si **alguien tiene que configurar algo**, es la solución 1 o la 2; si **no hay que configurar nada**, es la 3.

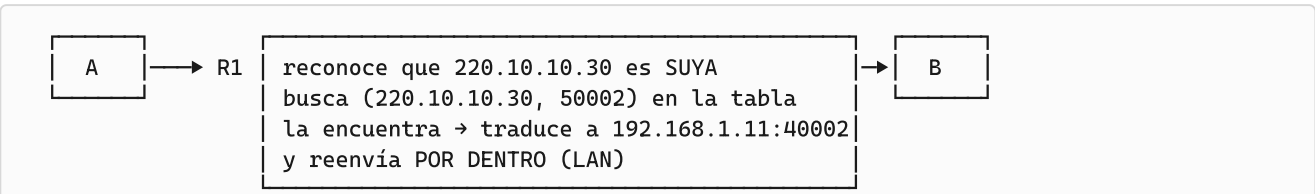
1.6 *NAT hairpinning* (el bucle hacia atrás)

El problema. Un cliente de la LAN quiere alcanzar a **otro cliente de la misma LAN** a través de la **IP pública** del router:



⇐ A manda a 220.10.10.30:50002. El paquete **llega físicamente al router**, porque esa IP pública es de R1. La entrada NAT de B existe (se creó cuando B salió), así que **debería** funcionar. ⚠️ PERO muchos routers NO lo hacen: la entrada se creó para tráfico hacia afuera, y se descarta por seguridad todo paquete que llega por la INTERFAZ EXTERNA con destino una IP pública PROPIA.

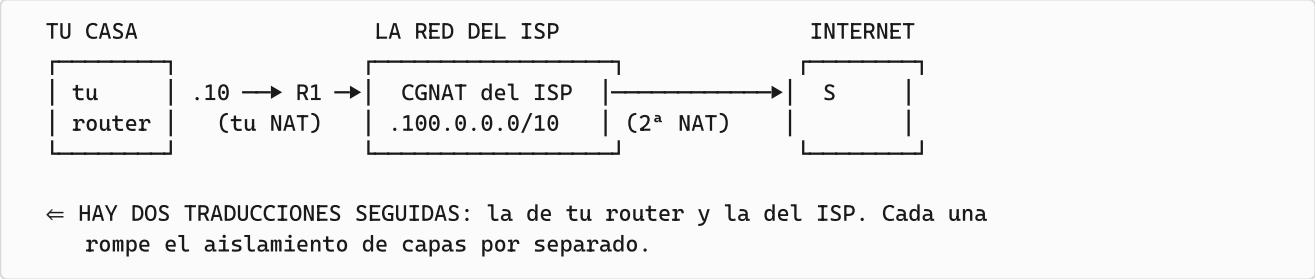
La solución: el *hairpin* (o *NAT loopback*). El router reconoce que **el destino es él mismo** y, en vez de rechazar, **traduce igualmente** y reenvía por la interfaz **interna**:



Sin hairpin se rompen dos casos: un servidor interno que publica un servicio en su IP pública y es accedido por otro equipo de la misma LAN, y **el P2P entre dos equipos de la misma LAN**, que directamente no se ven.

1.7 CGNAT: la NAT del operador

CGNAT (*Carrier-Grade NAT*) es la misma técnica, pero **aplicada por el operador** a toda la base de clientes:



Por qué se volvió ubicuo: por el mismo motivo que la NAT, y con más fuerza. Cuando el RIPE hizo su última asignación en 2019, los operadores dejaron de poder entregar una IP pública a cada casa y pasaron a entregar un prefijo de la **red privada del operador**, con CGNAT.

Los tres problemas del CGNAT

Problema 1 — Rompe la conectividad entrante. La tabla del CGNAT **solo se llena cuando el cliente sale**, y sin *port forwarding* **nadie de Internet puede iniciar una conexión** hacia tu casa. Impacto: **VoIP, juegos, SSH, servidores domésticos y cámaras IP**.

Problema 2 — Complica la señalización. SIP/IMS mandan la dirección **pública** dentro de los mensajes y usan cabeceras `Contact`, `Via` y `Record-Route` para saber por dónde volver. Con CGNAT esa dirección es **la del CGNAT**, y los elementos de ruta apuntan a una dirección que **no existe** en la red del operador: hay que agregar un **outbound proxy**.

Problema 3 — Rompe IPsec en ESP. En modo **ESP** el cifrado cubre la cabecera IP completa:

```
⚠️ El host mete IP_priv en su ESP; el CGNAT la reescribe a IP_publica DESPUÉS de que
   el ESP se armó. El receptor descifra y encuentra una cabecera que NO coincide con
   la que ve por fuera.
   ⇐ FALLA LA AUTENTICACIÓN. Y el CGNAT no puede recalcular el ESP, porque no tiene
      la clave.

Por eso con IPsec + CGNAT funciona AH (Authentication Header), que no cifra la cabecera
IP, o NAT-T (RFC 3948 🔍), que encapsula el paquete ya traducido en un ESP nuevo.
```

> ⚠️ **RFCs no verificados:** la especificación clásica de NAT tradicional (`RFC 3022`), el *shared address space* del CGNAT (`RFC 6598`), los requisitos del CGN (`RFC 6888`) y el NAT-T (`RFC 3948`) **NO están en** `ACTUALIZACIONES.md` : **verificar** en `rfc-editor.org` antes de citarlos. Lo verificado de esta clase: `RFC 8200` , `RFC 6146` , `RFC 6147` , `RFC 6052` , `RFC 8880` , `RFC 7915` , `RFC 6333` , `RFC 9000` .
>
> **El dato de 2026 que reencuadra todo:** el **NAT rebinding** (el operador cambia el puerto de la tabla a mitad de una conexión) **rompe TCP**, y por eso los clientes reconectan (`RFC 8443` 🔍). **Con QUIC no rompe nada**, porque el **Connection ID** (`RFC 9000` §5.1) identifica la conexión con independencia de las direcciones. **Ése es el cambio de paradigma:** la identidad de la conexión deja de ser la IP.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Armar la tabla NAT con 4 sesiones simultáneas

Datos. Router `R1` con IP pública `220.10.10.30`. LAN `192.168.1.0/24`. Cuatro conexiones salientes simultáneas, todas hacia `219.41.31.21:80`.

#	Host interno	Puerto origen	Puerto público asignado
1	`192.168.1.25`	1024	50001
2	`192.168.1.25`	1025	50002
3	`192.168.1.26`	1024	50003
4	`192.168.1.30`	51000	50004

La tabla NAT completa:

```
┌────┬──────────────────────────────┬──────────────────────────────────┬─────────┐
│ #  │ (IP origen int, port int)    │ → (IP pública, port público)     │ estado  │
├────┼──────────────────────────────┼──────────────────────────────────┼─────────┤
│  1 │ (192.168.1.25, 1024)         │ → (220.10.10.30, 50001)          │ ACTIVA  │
│  2 │ (192.168.1.25, 1025)         │ → (220.10.10.30, 50002)          │ ACTIVA  │
│  3 │ (192.168.1.26, 1024)         │ → (220.10.10.30, 50003)          │ ACTIVA  │
│  4 │ (192.168.1.30, 51000)        │ → (220.10.10.30, 50004)          │ ACTIVA  │
└────┴──────────────────────────────┴──────────────────────────────────┴─────────┘
```

Las tres observaciones que hacen los puntos: (1) el mismo host `192.168.1.25` tiene **DOS entradas** porque el puerto de origen es distinto (1024 y 1025). (2) El mismo puerto (1024) lo usan **dos hosts distintos** (`.25` y `.26`) sin conflicto, porque la clave es el **PAR**. (3) Todos los puertos públicos son distintos aunque la IP pública sea una sola.

Las tramas de los dos sentidos para la sesión 3 (la que repite puerto interno):

```
⇐ H → S (salida)                    ⇐ S → H (entrada)

IP  192.168.1.26   TCP src 1024      IP  220.10.10.30   TCP src    80
IP  219.41.31.21   TCP dst   80      IP  219.41.31.21   TCP dst 50003
Eth MAC_H26 → MAC_R1                          NAT→ IP 192.168.1.26  TCP src 80
                  ──NAT──►           Eth  MAC_R1 → MAC_H26   TCP dst 1024
```

2.2 Ejercicio 2 — ¿Por qué no funciona el P2P detrás de un CGNAT?

```
Paso 1: A (LAN hogareña) manda su descubrimiento a B. Sale por su router
        → se crea la entrada NAT de A.

Paso 2: B (detrás del CGNAT) manda su descubrimiento a A.
        ⇐ FALLA AQUÍ. El CGNAT residential no ofrece port forwarding: la
           entrada de B existe solo para tráfico SALIENTE, así que nadie
           de Internet puede iniciar nada hacia B.

Paso 3: aunque el paquete de B llegara, el de A no llegaría a B, porque el
        CGNAT no tiene entrada para (IP pública de A, puerto destino).

⇐ CONCLUSIÓN: la única dirección alcanzable es la que el propio cliente descubrió
   de sí mismo. Y esa dirección cambia cada vez que la entrada expira y se
   renueva (NAT REBINDING), que es lo que rompe el TCP.
```

Solución	Costo
Port forwarding manual	El usuario tiene que abrirlo, y solo sirve para su IP, que puede cambiar.
Rendezvous / TURN	**Todo el tráfico pasa por el relay**: costo y latencia.
STUN + ICE + hole punching	No sirve con NAT simétrico ni con CGNAT que filtre lo no iniciado.
IPv6	Sin traducción; pero no todo el mundo lo tiene.

> **La frase del examen:** el P2P falla porque **detrás de un CGNAT el cliente no puede iniciar conexiones entrantes**: la tabla solo se llena cuando el cliente **sale**. Por eso todo P2P real usa un **servidor de rendezvous** (o TURN, que además relay) para que alguien publicable le diga a los demás cuál es su dirección alcanzable.

2.3 Ejercicio 3 — Por qué la NAT rompe IPsec en ESP

```
El ESP CIFRA LA CABECERA IP COMPLETA (direcciones de origen y destino) y el
receptor valida que coincida con la que ve por fuera.

El emisor mete IP_priv en su ESP; la NAT reescribe a IP_publica. El receptor
descifra, encuentra IP_priv y ve IP_publica por fuera.
⇐ LA AUTENTICACIÓN FALLA, y es IRREVERSIBLE: la NAT no puede recalcular el ESP.
```

Las dos soluciones, que son la respuesta completa: (1) **modo AH** (*Authentication Header*, protocolo 51), que autentica el datagrama IP entero pero **no lo cifra**, así que la NAT puede reescribir y recalcular; funciona, pero sin confidencialidad. (2) **NAT-T** (`RFC 3948` 🔍), que encapsula el paquete **ya traducido** dentro de un ESP con la dirección pública; funciona, pero agrega 24 bytes por paquete.

2.4 Ejercicio 4 — ¿Cuál de las tres soluciones corresponde a cada escenario?

Escenario	Solución
Servidor web interno alcanzable desde Internet	**1. *Port forwarding*** (puerto fijo e IP privada estable)
Máquina en la nube con **IP elástica** que se quiere exponer	**2. NAT sin traducción** (mapeo 1:1)
Teléfono VoIP detrás de la NAT que quiere recibir llamadas	**3. El servidor inicia** (el `REGISTER` de SIP)
WebRTC entre dos navegadores detrás de un CGNAT	**3 + TURN**
Dos equipos de la misma LAN que se quieren ver por la IP pública	**3 + *hairpinning***

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ ⭐ Examen 2011-02, Pregunta 4 (10 puntos) — NAT, capas y las tres soluciones

> **Fuente:** Examen `eirc1102` / `sirc1102` — 3 de febrero de 2011, Pregunta 4 (10 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1102.pdf

Enunciado (texto literal del examen)

> **a)** Describa la técnica denominada NAT, explicando las modificaciones que un router NAT debe realizar sobre los paquetes salientes y entrantes.
>
> **b)** De una justificación para la siguiente frase: "*NAT no respeta el principio de aislación de capas*".
>
> **c)** Describa brevemente las tres soluciones planteadas en el teórico para atravesar un router NAT por parte de un cliente que se encuentra en una red con direcciones públicas y quiere contactar a un servidor que está en una red con direcciones privadas.

Sobre la solución oficial

El PDF de `sirc1102` **no trae la solución publicada de esta pregunta**: trae el enunciado y remite a las diapositivas del teórico. Las tres resoluciones de abajo son **solución propia**.

Parte a) — Qué es la NAT y qué cambia en cada sentido (≈4 puntos)

La **NAT** es una función de un **router de borde** que **reescribe las direcciones IP** de los paquetes, sustituyendo las direcciones **privadas** (que no se rutean en Internet) por una **dirección pública** del propio router. Para sostener **muchas sesiones simultáneas** con una sola dirección pública traduce además el **puerto** de origen: eso es la **PAT**.

```
⇐ SENTIDO SALIENTE  (LAN → Internet)

ETIQUETA IP                              TCP/UDP
origen      : IP PRIVADA   → IP PÚBLICA  ← SE CAMBIA
puerto origen: puerto PRIVADO → puerto PÚBLICO  ← SE CAMBIA
destino     : sin cambios                ← no se toca
puerto destino: sin cambios              ← no se toca

Además: se CREA una entrada en la tabla NAT
        (IP priv, port priv) → (IP pública, port público)

⇐ SENTIDO ENTRANTE  (Internet → LAN)

origen      : sin cambios                ← no se toca
puerto origen: sin cambios               ← no se toca
destino     : IP PÚBLICA   → IP PRIVADA   ← SE CAMBIA
puerto destino: puerto PÚBLICO → puerto PRIVADO  ← SE CAMBIA

El router busca la entrada por (IP pública, puerto destino). Si NO la
encuentra, DESCARTA el paquete.
```

El porqué de que el puerto destino sea la clave de búsqueda es lo que da el apartado b): todos los paquetes que entran tienen **la misma IP de destino** (la pública del router), y lo único que los distingue es el **puerto de destino**, campo de la cabecera TCP/UDP.

Para los 4 puntos: nombre de la técnica, por qué existe, tabla de campos por sentido, creación y consulta de la tabla, y el descarte.

Parte b) — Por qué la NAT no respeta el aislamiento de capas (≈3 puntos)

> El aislamiento de capas consiste en que **cada capa use a la de abajo como un servicio opaco**, y en que la capa 3 decida el reenvío mirando **solo la dirección IP destino**.
>
> Con NAT, un paquete que llega desde Internet tiene como destino la **dirección pública del propio router**, que es **única para toda la red interna**. Para saber a qué máquina de la LAN debe ir, el router tiene que consultar su tabla NAT, y esa tabla está indexada por el **puerto TCP/UDP de destino**.
>
> Por lo tanto el router, que es un dispositivo de **capa 3**, se ve obligado a **leer un campo de la cabecera de capa 4** para tomar su decisión de reenvío. Eso es una **invasión de capas**: el comportamiento de la capa 3 queda atado a un protocolo concreto de capa 4.
>
> Las consecuencias son: (i) **cualquier protocolo de capa superior que no sea TCP o UDP deja de funcionar** detrás de una NAT sin que el router lo sepa; (ii) se rompe la **extensibilidad**, porque al agregar una capa nueva hay que modificar también la capa 3; y (iii) se rompe la **transparencia extremo a extremo**, como se ve con IPsec en modo ESP, que cifra la cabecera IP y por lo tanto no tolera que se reescriban las direcciones.

Puntos clave: (1) la definición del aislamiento, (2) que el destino es la IP del router y hace falta el puerto, (3) que el puerto es de capa 4, (4) la consecuencia.

Parte c) — Las tres soluciones (≈3 puntos)

Las tres están desarrolladas en el **§1.5**, con diagramas y límites. En el examen:

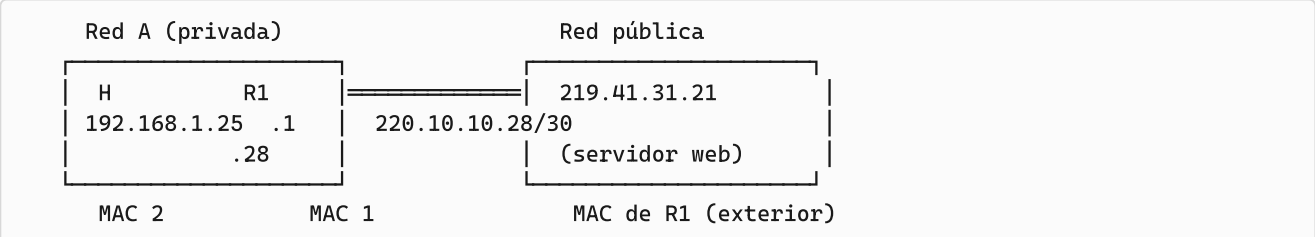
> **1) *Port forwarding* (NAT estática).** El administrador configura en el router un mapeo explícito: "todo lo que entre por el puerto público 8080, mandalo a `10.0.0.5:80`". La entrada de la tabla NAT es fija y siempre existe. Requiere IP privada estable en el servidor y conocer de antemano puerto y protocolo.
>
> **2) NAT sin traducción** (a veces *full-cone NAT* o *endpoint-independent mapping*). El router **no cambia nada** y deja pasar el paquete, con la IP de origen intacta. Sirve cuando el servidor tiene una dirección pública propia (por ejemplo una IP elástica en la nube) y lo que hace falta es que el router no reescriba. También requiere que el mapeo esté declarado de antemano.
>
> **3) El servidor inicia la conexión** (inversión de quién origina). El servidor privado **abre una conexión saliente**; el router guarda la entrada en su tabla NAT. A partir de ahí, cualquier paquete que **entre** desde Internet con ese mismo par `(IP pública, puerto público)` se traduce y se entrega al servidor. El cliente público "contesta" a una conexión que él no abrió. Es la base de SIP, ICE/WebRTC, STUN y TURN, y es la única que **no requiere configurar nada**.

3.2 ⭐ Examen 2011-03, Problema 2, apartado c — las modificaciones de R1

> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011, Problema 2, apartado c. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

Contexto del problema (los apartados a y b están resueltos en la Clase 30)

> Una empresa tiene una red A (LAN Ethernet con direccionamiento privado) con 15 servidores y 70 puestos de trabajo, y se desea que puedan acceder a un servidor web en Internet con dirección IP `219.41.31.21`. Para ello el router `R1` implementa NAT utilizando su dirección IP pública.



La tabla de enrutamiento (resuelta en la Clase 30 §3): R1 tiene `192.168.1.0/24` y `220.10.10.28/30` conectadas, y `0.0.0.0/0` por `220.10.10.29`. El host H tiene `192.168.1.0/24` conectada y `0.0.0.0/0` por `192.168.1.1`.

Enunciado del apartado c (texto literal)

> **c)** Especifique las modificaciones que realizará R1 en los paquetes de datos que viajan entre Host H y Servidor Web. Considerar ambos sentidos.

Solución oficial (textual)

> **Sentido host H → Servidor Web:** *"Datagramas con dirección IP de origen 192.168.1.25 y dirección IP de destino 219.41.31.21. R1 sustituye la IP de origen por 220.10.10.30, y el puerto de origen (port #) por un puerto nuevo (new port #) asignado por R1."* Y: *"El router R1 tendrá una tabla de NAT con los siguientes datos: (Dirección IP origen, port #), (220.10.10.30, new port #)"*.
>
> **Sentido servidor Web → host H:** *"Datagramas con dirección IP de origen 219.41.31.21, dirección IP de destino 220.10.10.30, y puerto de destino new port #. A partir de la tabla de NAT, R1 sustituye la IP de destino por 192.168.1.25, y el puerto de destino por port #."*

Análisis de la solución oficial

Lo que está bien: (1) nombra **exactamente los dos campos** que se cambian en cada sentido; (2) dice explícitamente que **se crea una tabla NAT** con la correspondencia de puertos, sin la cual la entrada no se puede resolver; (3) en el sentido entrante nombra los **tres campos que hay que mirar** antes de sustituir dos.

Lo que falta y son los puntos que se ganan: 1. **No da valores concretos** para el puerto asignado: dice "new port #" en vez de, por ejemplo, 50001. El enunciado no lo fuerza, pero **inventar un número concreto** muestra que entendiste que es una asignación del router. 2. **No dice qué pasa si no hay entrada en la tabla** (sentido entrante): el paquete **se descarta**. 3. **No dice qué NO se modifica.** Decir "la IP de destino en la salida y la IP de origen en la entrada **no se tocan**" es lo que hace la respuesta completa. 4. **No menciona el *timeout*** de la entrada, ni que se rehacen las tramas Ethernet.

Mi resolución completa del apartado c

Sentido 1 — H → Servidor Web (salida).

```
LLEGA A R1:  IP  origen 192.168.1.25  destino 219.41.31.21
             TCP origen 1024          destino 80
             Eth origen MAC 2         destino MAC 1 (R1, red A)

ACCIÓN: R1 busca (192.168.1.25, 1024) en la tabla NAT.
  · Si NO está → reserva un puerto público libre (ej. 50001), CREA la
    entrada y traduce.   · Si ya está → la reutiliza.

SALE DE R1:   IP  origen 220.10.10.30  destino 219.41.31.21   ← CAMBIA
             TCP origen 50001          destino 80            ← CAMBIA
             Eth origen MAC 1 → destino MAC de R1 en 220.10.10.28/30

ENTRADA CREADA: (192.168.1.25, 1024) → (220.10.10.30, 50001)  con TIMEOUT
⚠️ NO SE TOCA: la IP de DESTINO (219.41.31.21) ni el puerto de DESTINO (80).
```

Sentido 2 — Servidor Web → H (entrada).

```
LLEGA A R1:  IP  origen 219.41.31.21  destino 220.10.10.30
             TCP origen 80             destino 50001
             Eth destino = MAC de R1 en 220.10.10.28/30

BÚSQUEDA: (220.10.10.30, 50001) en la tabla NAT.
  ⚠️ por eso hace falta traducir PUERTOS: la IP de destino es la misma
     para todas las entradas.
  · Si NO está → R1 DESCARTA el paquete.
  · Si está → refresca el TIMEOUT y traduce.

SALE DE R1:   IP  origen 219.41.31.21  destino 192.168.1.25    ← CAMBIA
             TCP origen 80             destino 1024          ← CAMBIA
             Eth origen MAC 1 → destino MAC 2 (host H)

⚠️ NO SE TOCA: la IP de ORIGEN ni el puerto de ORIGEN. El servidor web no
   tiene por qué enterarse de que hubo NAT.
```

La tabla resumen, que es lo que el corrector busca:

Campo	Salida (H → S)	Entrada (S → H)
IP **origen**	`192.168.1.25` → **`220.10.10.30`**	`219.41.31.21` → sin cambio
puerto origen	`1024` → **`50001`**	`80` → sin cambio
IP **destino**	`219.41.31.21` → sin cambio	`220.10.10.30` → **`192.168.1.25`**
puerto destino	`80` → sin cambio	`50001` → **`1024`**
Tabla NAT	se **crea** la entrada	se **busca** y se refresca el timeout; si no está, **descarte**
Ethernet	MAC destino = MAC de R1 en `220.10.10.28/30`	MAC destino = MAC 2 (host H)

> **Nota sobre la oficial:** coincide exactamente con mi resolución en los dos sentidos. La oficial **no da valores concretos** para el puerto asignado y **no menciona el descarte**; esas dos son las adiciones que suman puntos.

3.3 ⭐ ⭐ Examen 2008-12, Problema 1 (30 puntos) — NAT para balanceo de carga

> **Fuente:** Examen `eirc0812` / `sirc0812` — 20 de diciembre de 2008, Problema 1 (30 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0812.pdf

Enunciado (texto literal del examen)

> Se desea utilizar una variante del protocolo NAT para realizar balanceo de carga entre tres servidores instalados en una red privada. Los servidores tienen direcciones privadas `IP_1`, `IP_2`, `IP_3`, y publican

> un servicio en el puerto `PORT`. El servicio es conocido en Internet a través del sistema Acceso, router de la red privada, cuya dirección pública es `IP_SERVICIO`. Se utiliza el mismo puerto `PORT` para accederlo.
>
> Al llegar una solicitud de servicio a Acceso, se debe redireccionar la misma al servidor que se encuentre atendiendo menor número de requerimientos. Una vez establecida una correspondencia entre un cliente y un servidor, debe mantenerse mientras se mantenga el flujo de tráfico. Se considera que el flujo ha terminado cuando transcurrieron `TIMEOUT` segundos sin tráfico.
>
> **a)** Indique qué campos deben modificarse en los paquetes que llegan a Acceso para lograr el redireccionamiento especificado. ¿Es necesario modificar los encabezados de capa 4 o solamente los de capa 3? Justifique su respuesta.
>
> **b)** En base a la respuesta anterior, especifique las estructuras que necesita almacenar Acceso para realizar la traducción de direcciones y el balanceo de carga.
>
> **c)** Especifique, en un lenguaje de alto nivel, el procedimiento que debe ejecutar Acceso cada vez que recibe un paquete, para cumplir la funcionalidad descrita.

Solución oficial (completa, textual)

a)

> *"Solamente necesitamos modificar campos de capa 3. Cuando llega un paquete IP de un host externo, elegimos el servidor menos cargado y pisamos la IP de destino del paquete poniendo la IP del servidor elegido. Cuando un paquete viene de un servidor interno y va hacia fuera sólo es necesario pisar la IP de origen y poner la del router. No es necesario modificar los puertos de origen y destino de los paquetes, pero sí es necesario guardarlos en la tabla de NAT para poder enviar los datos de una misma conexión TCP siempre hacia el mismo servidor."*

b)

```
struct NATRegister {
  int32 srcIP;
  int32 dstIP;
  int16 srcPort;
  int16 dstPort;
  long  timestamp;
}
Map<{int32, int16}, NATRegister> NATTable;
// usamos una tabla indizada por el par {ip, puerto} de origen
Map<int32, int> cantidadConexiones;
// esta tabla nos indica cuántas conexiones tenemos por cada servidor
```

c)

```
void acceso() {
  NATTable = new Map<{int32, int16}, NATRegister>();
  cantidadConexiones = new Map<int32, int>();
  while (true) {
    PacketIP p = receiveFromNetwork();
    cleanTimeoutRegisters();
    if (p.srcIP in [IP_1, IP_2, IP_3]) {         // respuesta de un servidor
      NATRegister reg = NATTable.get(p.dstIP, p.dstPort);
      if (reg != null) reg.timestamp = now();
      p.srcIP = IP_SERVICIO;                      // ← pisa el ORIGEN
      sendToNetwork(p);
    } else {                                       // viene de afuera
      NATRegister reg = NATTable.get(p.srcIP, p.srcPort);
      if (reg == null) {
        int32 srvIP = cantidadConexiones.findMinimum();
        reg = new NATRegister();
        reg.srcIP = p.srcIP;  reg.srcPort = p.srcPort;
        reg.dstIP = srvIP;    reg.dstPort = PORT;
        NATTable.put({p.srcIP, p.srcPort}, reg);
      }
      reg.timestamp = now();
      p.dstIP = reg.srcIP;                         // ← BUG: debe ser reg.dstIP
      sendToNetwork(p);
    }
  }
}
void cleanTimeoutRegisters() {
  long maxTime = now() + TIMEOUT;
  for each NATRegister reg in NATTable {
    if (maxTime > reg.timestamp) {                 // ← BUG: condición invertida
      NATTable.remove(reg);
      cantidadConexiones[reg.dstIP]--;             // ← pero NUNCA se suma
    }
  }
}
```

Análisis de la solución oficial

Lo que está muy bien: 1. **Responde el apartado a) con el criterio correcto** ("solamente campos de capa 3") y da la **razón profunda**: los puertos **no se modifican** pero **sí se guardan** en la tabla, "para poder enviar los datos de una misma conexión TCP siempre hacia el mismo servidor". **Esa frase vale los puntos del apartado.** 2. **La estructura del `NATRegister` es la correcta**: IP y puerto de **origen** (el cliente externo), IP de destino (el servidor elegido), ambos puertos y un `timestamp`. 3. **El índice de la tabla es el correcto**: el par `{ip, puerto}` de **origen**, porque es lo único que llega sin traducir. 4. **La segunda estructura es la clave del balanceo**: un mapa servidor → cantidad de conexiones, con una operación de mínimo.

🔴 **Errores reales en el pseudocódigo de la oficial (señalarlos suma puntos):**

1. **`p.dstIP = reg.srcIP;` está al revés.** En la rama de "viene de afuera" hay que pisar **el servidor elegido**, o sea `reg.dstIP`. Con `reg.srcIP` el paquete vuelve al cliente externo con su propia IP de destino. La oficial **se contradice con su propio apartado a)**.
2. **Nunca se incrementa `cantidadConexiones`.** La tabla de carga se crea pero el código **no la suma** en ningún lado: el balanceo nunca elige el menos cargado.
3. **La condición del timeout está invertida.** `maxTime > reg.timestamp` compara la hora actual **más** el timeout contra el timestamp: si `timestamp` es mayor, la entrada es **nueva**. Lo correcto es `now() - reg.timestamp > TIMEOUT`.

4. `cleanTimeoutRegisters()` **se llama una vez por paquete**, es decir **O(n) por paquete**. Con miles de flujos activos corresponde barrer la tabla **periódicamente** o usar una cola ordenada por timestamp.
5. `NATTable.get(p.dstIP, p.dstPort)` le pasa dos argumentos a una tabla indexada por un **par**, y si un paquete de un servidor llega **sin entrada** (porque expiró a mitad de un flujo) el código lo reenvía igual con `srcIP = IP_SERVICIO` sin avisar.

Mi resolución completa (para los 30 puntos)

Parte a) — Qué campos se modifican

```
⇐ ENTRANTE (cliente externo → Acceso → servidor)

LLEGA:  IP origen = cliente externo ; IP destino = IP_SERVICIO
        TCP origen = cPort ;         TCP destino = PORT
        Eth destino = MAC de Acceso

ACCIÓN: si (IP origen, cPort) NO está en la tabla:
            srvIP = el servidor con MENOR cantidadConexiones
            crear reg = { src=(IP origen, cPort), dst=(srvIP, PORT), t=now() }
            cantidadConexiones[srvIP]++        ⟵ ¡FALTA EN LA OFICIAL!
        si ya está: se reutiliza la entrada, SIN volver a elegir servidor.

SALE:   IP origen = cliente externo   (NO se toca)
        IP destino = srvIP           ⟵ SE PISA
        TCP origen = cPort ; destino = PORT   (NO se tocan)
        Eth destino = MAC de srvIP

⚠️ POR QUÉ NO HAY QUE TOCAR LOS PUERTOS:
   El cliente pidió el puerto PÚBLICO (PORT). Si Acceso cambiase el puerto de destino, el
   cliente no reconocería la respuesta (su socket espera en PORT). ⇐ El TRUCO: se guardan
   los puertos REALES en la tabla y se usan para mandar la respuesta al servidor correcto.
   Como el puerto público coincide con el privado (PORT = PORT), el paquete sale sin
   traducción de capa 4. ⇐ OJO: esto solo funciona porque el enunciado dice "el mismo
   puerto PORT". Si el servidor escuchara en otro, habría que traducir capa 4.

⇐ ENTRANTE (servidor → Acceso → cliente externo)

    IP origen = srvIP                ⟵ SE PISA por IP_SERVICIO
    IP destino = cliente externo     (NO se toca)
    TCP origen = PORT ; destino = cPort   (NO se tocan)
```

La respuesta al "¿es necesario modificar los encabezados de capa 4?": No, solamente capa 3, porque los puertos se **guardan** en la tabla NAT y la correspondencia cliente-servidor se mantiene por sesión sin traducirlos. Es el mismo criterio que usa un balanceador que **no** quiere terminar TLS: termina solo en capa 3 o 4.

Parte b) — Estructuras de datos

```
1) REGISTRO DE UNA SESIÓN
     struct NATRegister {
         ip_addr srcIP;     // IP del cliente externo (no se traduce)
         ip_addr dstIP;     // IP del servidor PRIVADO elegido
         uint16  srcPort;   // puerto origen real del cliente
         uint16  dstPort;   // puerto destino real (PORT)
         long    timestamp; // último instante con tráfico
     };

2) TABLA DE SESIONES, indexada por el par {srcIP, srcPort}
     Map<{ip_addr, uint16}, NATRegister>  NAT;

3) CONTADOR POR SERVIDOR (lo que permite el balanceo)
     Map<ip_addr, int>  Carga;   // IP_1 → 120, IP_2 → 87, ...

4) (recomendable) COLA DE EXPIRACIÓN ordenada por timestamp, para no barrer
   toda la tabla en cada paquete.
```

Por qué la clave es `{srcIP, srcPort}` y no `dstIP`: porque `dstIP` es siempre `IP_SERVICIO`, la misma para todas las entradas, y no distingue nada. Lo único que **llega intacto** y es único por cliente es `(IP origen, puerto origen)`. Si un cliente abre dos conexiones simultáneas tiene dos puertos de origen distintos, y por eso van a entradas (y a servidores) distintos.

Por qué hace falta `dstPort` en el registro si no se traduce: para distinguir tres servidores que **escuchan en el mismo puerto**, que es justo el caso del enunciado.

Parte c) — El procedimiento

```
// ---- Estado persistente ----
Tabla<Par{ip,uint16}, NATRegister>  NAT;      // clave = (srcIP, srcPort)
Tabla<ip, int>                      Carga;     // servidor → conexiones activas
Cola<timestamp, Par>               Expiran;   // barrido eficiente

bool esServidorPropio(ip a):
    return (a == IP_1 || a == IP_2 || a == IP_3)

ip elegirServidor():
    mejor = IP_1
    for srv in {IP_1, IP_2, IP_3}:
        if Carga[srv] < Carga[mejor]:  mejor = srv      // desempate por IP
    return mejor

void refresh(NATRegister r):
    r.timestamp = now()

void limpiarExpirados():       // barra solo lo vencido, con una cola ordenada
    while Expiran no vacía and Expiran.menor().timestamp < now() - TIMEOUT:
        k = Expiran.extraer()
        r = NAT.buscar(k)
        if r != null and r.timestamp < now() - TIMEOUT:
            NAT.eliminar(k)
            Carga[r.dstIP] = Carga[r.dstIP] - 1      ←— ¡FALTA EN LA OFICIAL!

void acceso():
    NAT = nueva Tabla()
    Carga = { IP_1→0, IP_2→0, IP_3→0 }
    Expiran = nueva Cola()

    while (true):
        p = recibirPaqueteDeLaRed()
        limpiarExpirados()

        if esServidorPropio(p.srcIP):
            // ===== SENTIDO SALIENTE (servidor → cliente) =====
            r = NAT.buscar({ p.dstIP, p.dstPort })     // dstIP = IP_SERVICIO
            if r == null:
                descartar(p)      // el cliente cerró y la entrada expiró
                continue
            refresh(r)            // la sesión sigue viva
            p.srcIP = IP_SERVICIO                      // ←— SE PISA el origen
            enviarPaqueteALaRed(p)                     // destino y puertos, igual

        else:
            // ===== SENTIDO ENTRANTE (cliente → servidor) =====
            k = { p.srcIP, p.srcPort }
            r = NAT.buscar(k)
            if r == null:
                // ===== flujo NUEVO: acá está el BALANCEO =====
                srv = elegirServidor()
                r = new NATRegister()
                r.srcIP = p.srcIP ; r.srcPort = p.srcPort
                r.dstIP = srv                          // ←— EL SERVIDOR ELEGIDO
                r.dstPort = p.dstPort
                NAT.insertar(k, r)
                Carga[srv] = Carga[srv] + 1            ←— ¡FALTA EN LA OFICIAL!
            refresh(r)
            p.dstIP = r.dstIP                           // ←— SE PISA el destino
            enviarPaqueteALaRed(p)
```

Puntos (30): (a) qué campos y por qué no los puertos (8), (b) las dos estructuras y la justificación de la clave (10), (c) el procedimiento con los dos sentidos, el *timeout* y el contador (12).

> **Nota final sobre la oficial:** la solución oficial tiene **cuatro errores reales** en el pseudocódigo (el `p.dstIP = reg.srcIP` invertido, el contador que nunca se incrementa, la condición de *timeout* invertida y el barrido O(n) por paquete). **El enunciado, la parte a) y la parte b) de la oficial están bien.** Si tu respuesta los señala, sumás puntos: a la cátedra le importa que entiendas el mecanismo, no que copies.

PARTE 4 — CIERRE

📌 Resumen

1. **NAT** = traducción de direcciones; existe por la **escasez de IPv4** y porque las **direcciones privadas no se rutean**. No es seguridad, es escasez.
2. La traducción de **dirección sola** soporta **una** sesión por IP pública; la **PAT/NAPT** soporta miles. En FING, "NAT" significa **PAT**.
3. **Tres fases: salida** (se crea la entrada), **entrada** (se busca por `(IP pública, puerto destino)`; si no está, **se descarta**) y **timeout**.
4. **Salida:** cambian **IP origen** y **puerto origen**. **Entrada:** cambian **IP destino** y **puerto destino**. Nada más.
5. **Rompe el aislamiento de capas** porque un router de capa 3 tiene que **leer el puerto TCP/UDP** (capa 4) para saber a qué host interno va.
6. **Las tres soluciones: port forwarding**, **NAT sin traducción** (1:1 declarado) y **el servidor inicia** (la entrada se crea sola; base de SIP, ICE y TURN).
7. **Hairpinning** permite que un cliente de la LAN alcance a otro por la IP pública del router.
8. **CGNAT**: sus tres problemas son **conectividad entrante rota**, **señalización complicada** e **IPsec ESP roto**.
9. Con IPv6 **no hay traducción**: por eso la meta es **eliminarla**, y el `Connection ID` de QUIC hace que el *NAT rebinding* ya no rompa las conexiones.
10. **IPv4 se agotó**: el RIPE hizo su última asignación `/22` el **25-nov-2019**, y el acceso a Google por IPv6 superó el **50 % el 28-mar-2026**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. En la tabla NAT, dos sesiones salen del mismo host `192.168.1.25` con puertos de origen 1024 y 1025. ¿Cuántas entradas hay y por qué dos?

<details>
<summary>Respuesta</summary>
</details>

2. En el sentido de entrada, ¿qué pasa con un paquete cuyo par `(IP pública, puerto público)` **no está** en la tabla?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué la NAT rompe el aislamiento de capas? Formulá la respuesta en dos frases.

<details>
<summary>Respuesta</summary>
</details>

4. ¿Cuál es el problema del P2P detrás de un CGNAT, y por qué el *hole punching* no siempre lo resuelve?

<details>
<summary>Respuesta</summary>
</details>

5. ¿Qué tres cosas hay que mantener en un balanceador de carga tipo `2008-12 Problema 1`? Mencioná los dos errores del pseudocódigo oficial.

<details>
<summary>Respuesta</summary>
</details>

6. Un amigo te dice: "*configurá port forwarding y el problema de que no te puedan llegar adentro desaparece para siempre*". ¿Qué le responderías?

<details>
<summary>Respuesta</summary>
</details>

7. ¿Qué cambia con IPv6, y por qué la meta del IPv6 es eliminar la NAT?

<details>
<summary>Respuesta</summary>
</details>

8. ¿Qué es el *hairpinning* y por qué hace falta?

<details>
<summary>Respuesta</summary>
</details>

🔗 Referencias

Exámenes - `eirc1102` / `sirc1102` (3-feb-2011) — **P4 (10 pts): NAT, por qué rompe el aislamiento de capas, y las tres soluciones**. §3.1. - `eirc1103` / `sirc1103` (21-feb-2011) — **Problema 2 c: qué campos cambia R1 en cada sentido + tabla NAT**. §3.2. (Los apartados a y b están en la Clase 30.) - `eirc0812` / `sirc0812` (20-dic-2008) — **Problema 1 (30 pts): variante de NAT para balanceo de carga entre tres servidores privados**, con pseudocódigo. §3.3. - `sirc1003` (mar-2010) P1: subredes privadas y NAT. `sirc0903` (feb-2009) Problema 2: NAT como proxy.

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., cap. 5.

RFCs verificados en `ACTUALIZACIONES.md` - `RFC 8200` — IPv6 (2017, *Internet Standard*): por qué no hace falta traducción. - `RFC 6146` — NAT64 (2011) y `RFC 6147` — DNS64 (2011), con `RFC 6052` y `RFC 8880`. - `RFC 7915` — SIIT (2016), que **obsoleta** a la `RFC 6145`. - `RFC 6333` — DS-Lite (2011): dual-stack *stateless*. - `RFC 9000` — QUIC (2021) §5.1: el **Connection ID** hace que el *NAT rebinding* no rompa las conexiones. - `draft-ietf-v6ops-464xlat` — 464XLAT, **sigue en borrador**.

RFCs NO verificados (no los cites sin confirmar) - NAT tradicional (`RFC 3022`), espacio compartido del CGNAT (`RFC 6598`), requisitos del CGN (`RFC 6888`), NAT-T (`RFC 3948`) y detección de *NAT rebinding* (`RFC 8443`): **verificar**.

Clases relacionadas - Clase 26, 27 y 29 (IPv4 y subnetting) · Clase 30 (enrutamiento y ARP) · Clase 36 (IPv6) · Clase 44 (VoIP).

Clase 32

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 Tres maneras de que un router sepa a dónde mandar un paquete
- 1.2 La ecuación de Bellman-Ford
- 1.3 La actualización: cuándo y qué se manda
- 1.4 ⭐ El conteo a infinito — el diagrama más importante de la clase
- 1.5 Las cuatro soluciones, y por qué ninguna es completa
- 1.6 RIP — el DV que existió de verdad
- 1.7 Ventajas y desventajas — la respuesta de `2006-08 P3`
- 2.1 Ejercicio 1 — 4 routers con costos distintos, convergencia en 3 rondas
- 3.1 ⭐ Examen 2008-12, Problema 2 (30 puntos) — distance-vector en una red de 4 nodos
- 3.4 Examen 2006-08, Pregunta 3 (10 puntos) — funcionamiento, ventajas y desventajas
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: D (Capa de red) · **Semana:** 11 · **Duración estimada:** 115 min **Correlativa:** Clase 31 (NAT, PAT y sus tricks) · **Sigue con:** Clase 33 (link-state, Dijkstra y OSPF) **Material de referencia:** Tanenbaum cap. 5 (§5.6) · RFC 2453, RFC 1058, RFC 2080

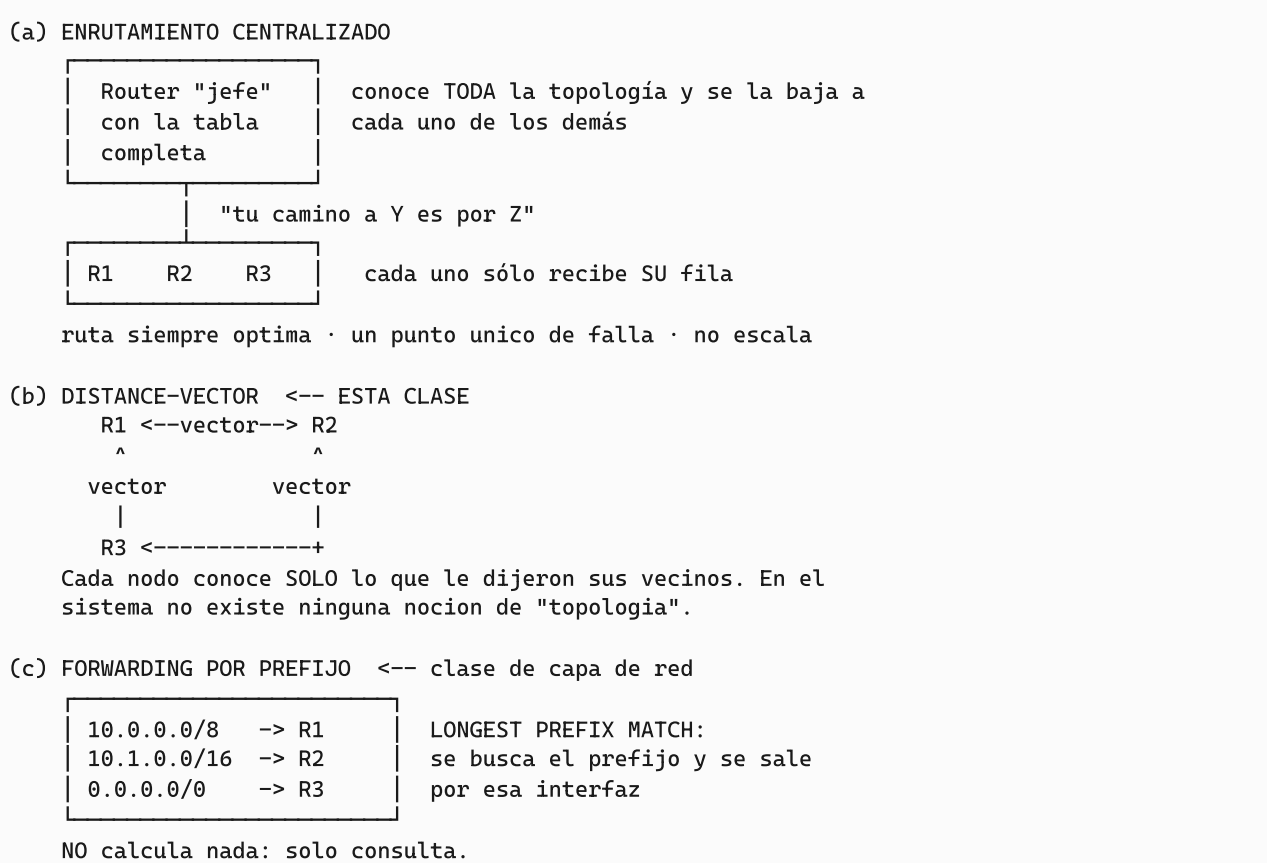
🎯 Al terminar esta clase vas a poder

1. Explicar qué intercambian dos routers **distance-vector** y por qué no conocen la topología.
2. Escribir la **ecuación de Bellman-Ford** y justificar por qué es correcta desde el **principio de optimalidad**.
3. Ejecutar **a mano, con la tabla de iteraciones**, el algoritmo DV hasta la convergencia.
4. Reproducir **el conteo a infinito** ronda por ronda y explicar **por qué** los routers se realimentan.
5. Describir las **cuatro soluciones** clásicas y argumentar por qué **ninguna es completa**.
6. Enumerar **ventajas y desventajas** de distance-vector con la nomenclatura de la cátedra.

> **Prerequisito del examen:** `2006-08 P3` (10 pts), `2008-12 Problema 2` (30 pts), `2010-02 Problema 2` (30 pts), `2010-07 Problema 2b`, `2007-02 P3` (10 pts).

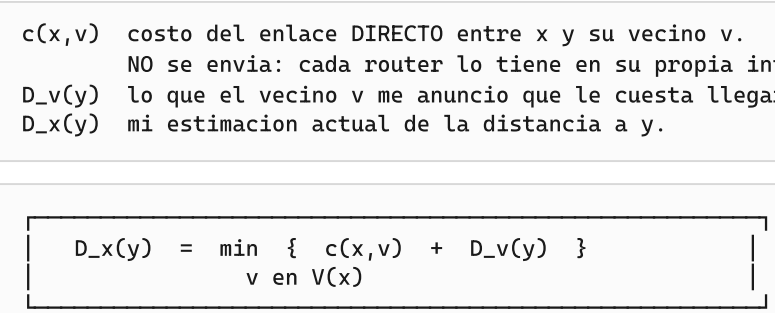
PARTE 1 — TEORÍA

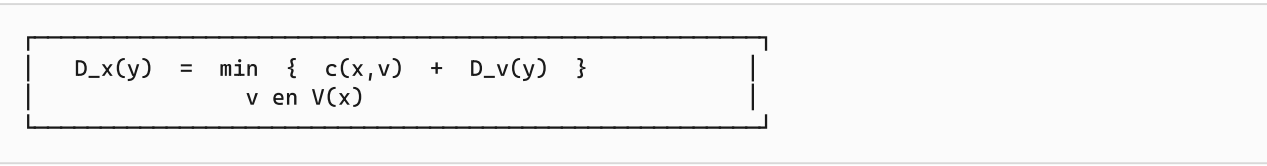
1.1 Tres maneras de que un router sepa a dónde mandar un paquete



El punto que vale la mitad de la nota conceptual: en (c) **no hay cálculo** (el *longest prefix match* busca en una tabla que alguien ya llenó); en (b) el cálculo lo hace **cada router por su cuenta** y el resultado se traduce a una tabla de forwarding. Cuando el enunciado pida "la tabla de forwarding", quiere esa **traducción** a `(Destino, Próximo Salto, Costo)`.

1.2 La ecuación de Bellman-Ford





Por qué la ecuación es correcta (principio de optimalidad)

Es lo que pide `2006-08 P3` cuando dice "**detalladamente**". Supongamos que el mejor camino de `x` a `y` es `x --(c1)--> n1 --(c2)--> n2 --> ... --> y`. `x` tiene que entregar el paquete a **alguien**, y ese alguien es `n1`. ¿Qué tiene que hacer `n1` desde `n1`?

> **El mejor camino de `n1` a `y`.** Porque si el tramo que le queda a `n1` **no fuera óptimo**, yo podría construir un camino mejor de `x` a `y`: tomaría mi enlace `x -> n1` (costo `c1`) y seguiría por ese camino suboptimal. Eso **contradice** que el mío era el mejor.

Eso es el **principio de optimalidad** (Bellman, 1957): *si el camino óptimo de `x` a `y` pasa por `v`, el tramo de `v` a `y` es él mismo óptimo desde `v`*. De ahí sale la ecuación: el camino óptimo **tiene que empezar** por algún vecino `v`; llegar a `v` cuesta `c(x,v)`; y de `v` a `y` cuesta como mínimo `D_v(y)`. Elegir `v` cuesta `c(x,v) + D_v(y)`, y el óptimo es el `min` sobre todos los vecinos. QED

Corolario: `D_x(y)` es no negativo y, como la recurrencia es sobre caminos **sin ciclos**, el algoritmo converge. **Justo lo que se rompe en §1.4**: el principio de optimalidad deja de ser aplicable cuando hay un bucle, porque el supuesto "sin ciclos" ya es falso.

1.3 La actualización: cuándo y qué se manda

Qué se manda: un VECTOR, no una topología. Eso es **todo** el mensaje:

```
┌──────────────────────────────────────────────┐
│  VECTOR DE DISTANCIAS de x  (= su tabla)     │
│   destino        distancia   próximo salto   │
│   192.168.1/24       0          local        │
│   192.168.2/24       2          R2           │
│   0.0.0.0/0          6          R2           │
└──────────────────────────────────────────────┘
```

Un LSA (clase siguiente) describe **un** enlace de quien lo emite; el vector describe **todas** las rutas de quien lo emite. De ahí el nombre "rumor": es una afirmación sobre el mundo entero, basada en información que el emisor ya está viejo.

```
PERIODICO:  un timer T (en RIP, 30 s). Al vencer, mando mi tabla completa
            por TODOS los enlaces.      + simple, robusto
                                        - una noticia mala tarda hasta T
DISPARADO POR CAMBIO (triggered update): mi tabla cambió, mando el vector
            AHORA.                      + la noticia buena viaja rápido
            - con la mala dispara MUCHAS veces seguidas, y ahí aparece el
              conteo a infinito. Por eso RIP limita los triggered updates
              a 1 cada 1 a 5 s.
```

El procedimiento exacto (solución oficial de `2006-08 P3`, citado):

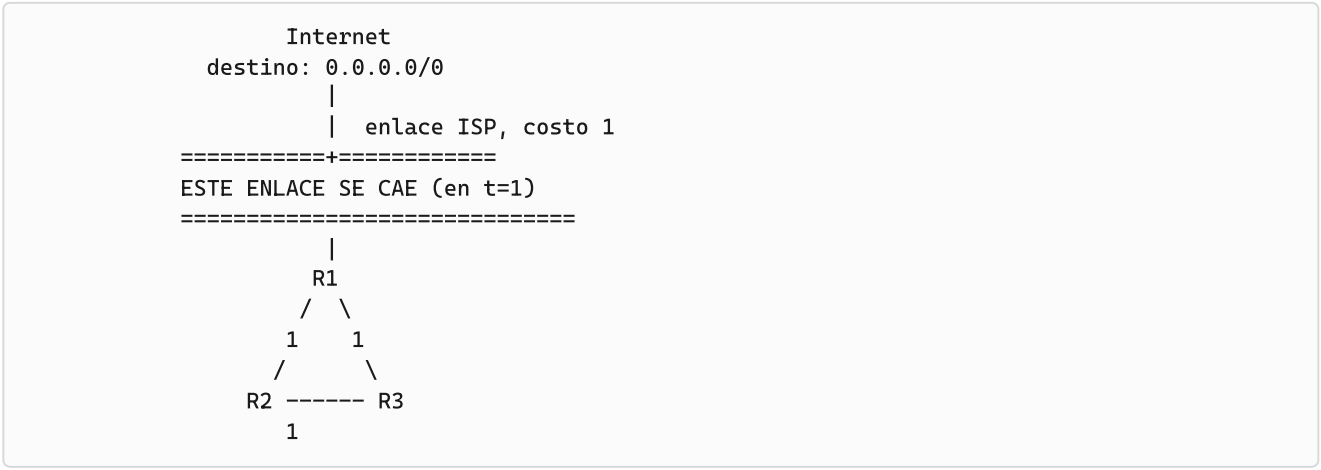
> Sea `T0` la tabla propia y `T1` la tabla recibida del router vecino `R`. Para cada ruta a un host `A` en `T1` **a través de `R`**: si **no existe** en `T0`, **agrego** la ruta hacia `A` por `R` incrementando la distancia; si **existe** en `T0` pero con una **métrica mayor**, la **sustituyo** por la de `R`; si **existe** la misma ruta, la marco como

> **"refrescada"**. Luego de recibidos updates por **todas** las interfaces, **borro** todas las rutas **"no refrescadas"**.

⚠ **El último punto responde a la pregunta clásica de DV: ¿cómo sabe `x` que un vecino dejó de existir?** No hay detección de caída de enlace en el mensaje: hay un **timeout por ruta**. Si una ruta no fue refrescada en esta ronda, el vecino que la anunciaba ya no existe y se borra. En RIP: **180 s**.

1.4 ⭐ El conteo a infinito — el diagrama más importante de la clase

Éste es **el** problema de distance-vector. Armemos la red mínima donde el conteo a infinito **ocurre de verdad**. La condición necesaria es que **el destino tenga una única boca de salida y que esa boca la crucen juntos dos routers**. Un triángulo simple NO sirve: el tercer router tiene siempre la verdad y el bucle se deshace en una ronda.



Enlaces del triángulo: `R1–R2 = 1`, `R1–R3 = 1`, `R2–R3 = 1`. Cada router tiene su LAN detrás, a costo 1. **Internet se alcanza únicamente saliendo por R1.**

t = 0 — estado estable

`D_R1(Internet) = 1 + 1 = 2`; `D_R2 = 1 + 2 = 3`; `D_R3 = 3`.

destino	R1	R2	R3
LAN1	**1** (local)	2 (vía R1)	2 (vía R1)
LAN2	2 (vía R2)	**1** (local)	2 (vía R2)
LAN3	2 (vía R3)	2 (vía R3)	**1** (local)
Internet	**2** (vía ISP)	3 (vía R1)	3 (vía R1)

t = 1 — R1 detecta la caída de su enlace al ISP

- El enlace de R1 con el ISP **no lo usa nadie más**: la **interfaz** lo reporta caído y R1 pone `D_R1(Internet) = ∞`.
- **R2 y R3 todavía no saben nada**: su enlace con R1 está perfecto y lo único que escucharon de R1 fue `Internet = 2`. Siguen con 3.

```
R1 -> Internet = INF         (¿y ahora qué? no tengo a quién preguntarle)
R2 -> Internet = 3 vía R1    (¡sigue usando un enlace que ya no sirve!)
R3 -> Internet = 3 vía R1
```

t = 2 — el descubrimiento: los tres se realimentan

Acá está la **semilla del bucle**. Cada uno escucha a sus vecinos y aplica la ecuación:

```
R1 recibe de R2: "Internet está a 3 por mí"    ->  c(R1,R2) + 3 = 1 + 3 = 4
R1 recibe de R3: "Internet está a 3 por mí"    ->  1 + 3 = 4
    ->  R1 adopta 4, vía R2      ¡el camino que le pasa R2 pasa por R1!

R2 recibe de R1: "Internet está a INF"         ->  1 + INF = INF  (invalida su 3)
R2 recibe de R3: "Internet está a 3 por R1"    ->  1 + 3 = 4
    ->  R2 adopta 4, vía R3      ¡el camino vuelve por R1!

R3 recibe de R1: "Internet está a INF"         ->  INF
R3 recibe de R2: "Internet está a 3 por R1"    ->  1 + 3 = 4
    ->  R3 adopta 4, vía R2      ¡y vuelve por R1 otra vez!
```

Leé el detalle importante: el `4` que R1 calcula sale de un rumor **de R2** que dice "llego a Internet en 3 por R1". O sea: **R2 le está contando a R1 un camino que pasa por R1 mismo.** Ese es el bucle, y por eso la métrica **sube** en vez de bajar a ∞.

t = 3, 4, 5, … — la cuenta ascendente

ronda	R1 oye de R2	R1 oye de R3	**R1**	**R2**	**R3**
2	1+4 = 5	1+4 = 5	**5**	**5**	**5**
3	1+5 = 6	1+5 = 6	**6**	**6**	**6**
4			**7**	**7**	**7**
k			*k+2*	*k+2*	*k+2*
14			**16**	**16**	**16**
15			**INF**	**INF**	**INF**

Interpretación. Durante ~14 rondas la red **cree** que tiene camino a Internet, y ese camino es un **círculo**: los paquetes dan vueltas hasta que el TTL los mate. En la práctica nunca se llega al ∞ matemático: en RIP el infinito es **16** y el *garbage collection* borra la ruta en la ronda 14. **Las filas de las LANs no se mueven nunca:** el conteo a infinito afecta **exactamente a la fila que se rompió**.

Por qué el problema persiste aunque los tres hagan todo bien

Los tres están **aplicando Bellman-Ford correctamente**: el error no es de implementación sino de la **ecuación**, porque el principio de optimalidad ya no aplica (el supuesto "sin ciclos" es falso). La red se autoalimenta con un rumor falso. Por eso la solución no es "calcular mejor" sino **romper la realimentación**; y lo único que corta el círculo en el plano de datos es el **TTL** (ICMP tipo 11).

1.5 Las cuatro soluciones, y por qué ninguna es completa

(1) Split horizon

> **Regla:** no le anuncies a `X` lo que aprendiste **de** `X`.

Cada entrada marca **de quién** la aprendí; al generar el vector para `v` se omiten las entradas cuyo origen sea `v`.

(2) Split horizon con poison reverse (reverse envenenado)

> **Regla:** en vez de **silenciar**, anunciá **INF** ("inalcanzable") para lo que aprendiste de ese vecino.

El silencio es ambiguo: el vecino no sabe si te callaste porque no tenés nada o porque no te llegó. El poison reverse es una **afirmación explícita de falsedad**.

Los dos, aplicados a la red de §1.4 (modelo síncrono; `(*)` = "lo que yo tenía venía de vos, así que te enveneno INF"):

```
ronda  R1->R2  R1->R3  R2->R3  R3->R2  |  R1          R2          R3
──────────────────── SPLIT HORIZON ──────────────────────────────────
 1      INF      INF     3       3     |  INF         3           3
 2      INF      INF     4       4     |  4 vía R2    4 vía R3    4 vía R2
 2   silencio    4    silencio   4     |  (R1 se calló sobre R3; R3 sobre R2)
 3      INF      5      INF      5     |  5 vía R3    INF         5 vía R1
 4      INF      INF     INF      INF  |  INF          6 vía R1    INF
 5                              |      INF         INF         INF
 ⇒ los tres en INF, en 5 rondas

──────────────────── POISON REVERSE ──────────────────────────────────
 1      INF      INF     3       3     |  INF         3           3
 2      INF      INF     INF (*)  INF (*)|  4           4           4
 3      INF (*)  4      INF      INF    |  5 vía R3    INF         5 vía R1
 4      5       INF (*)  INF      INF   |  INF         INF         INF
 ⇒ los tres en INF, en 4 rondas, y con el mensaje explícito
```

El caso real de `2010-07 P2b` (el ISP es la única salida de una red de 3 routers, R1--R2, R1--R3, R2--R3, con `D_R1(Internet) = 2`): al caerse el enlace R1--ISP, `t=1` deja a R1 en ∞ y R2/R3 siguen con 3 vía R1; en `t=2` los tres recalculan con lo que oyen y quedan en **4**; de `t=3` en adelante **suman 1 por ronda** hasta el 16 de RIP. Con split horizon + poison reverse: **3 rondas**. La oficial dice que R1 elige "a través de R1", lo cual está **mal**: *un router no puede usar su propia ruta como próximo salto.*

Limitación de los dos: atacan el rumor **de vuelta por el vecino inmediato**, no globalmente; con bucles de **tres o más** routers en malla, split horizon solo **no alcanza**. **`RFC 2080` dice textualmente que poison reverse es "el método preferido de operación"**, y es lo que usa RIP: una ronda menos y sin ambigüedad.

> **(3) Hold-down:** cuando una ruta cambia (sobre todo si empeora o pasa a ∞), **no aceptes ningún cambio** sobre ella durante `holddown` segundos. **Qué resuelve:** el *route flapping* corto: sin hold-down, `R2: 3 → INF`, `R1: INF → 5`, `R2: INF → 5` ... un "ping-pong" de rutas. **Costo:** retrasa la recuperación de

> las buenas rutas (si el enlace vuelve en 1 s, con hold-down de 180 s la ruta queda muerta 3 minutos). Por eso nunca va solo.
>
> **(4) Triggered updates:** cuando tu tabla cambia, mandá el vector **en el acto**. **Qué resuelve:** la velocidad de las **buenas** noticias (sin ellos, una ruta nueva tarda hasta 30 s en existir en toda la red). **La paradoja:** *acelera la convergencia normal, pero también hace que el conteo a infinito sea rápido*. Por eso RIP los limita a uno cada **1 a 5 segundos**.

Por qué, con las cuatro juntas, el problema persiste

Porque las cuatro atacan **síntomas**, no la **causa**:

```
split horizon      atacan el RUMOR que vuelve por el vecino inmediato
poison reverse     (ataque local, no global)
hold-down          atacan la FLUCTUACION
triggered updates  atacan la VELOCIDAD (y la aceleran de mas)
      |
      └─> NINGUNA ataca la causa: que la metrica es un NUMERO y no un
          IDENTIFICADOR del camino. Dos caminos distintos con la misma
          metrica son INDISTINGUIBLES para Bellman-Ford. La informacion
          para romper el bucle (la ruta completa) NO ESTA en el mensaje.
```

La solución de fondo es la que usa BGP: mandar en el mensaje **la lista de por dónde pasó la ruta** (`AS_PATH`) y rechazarla si esa lista ya contiene a uno mismo. Ahí el bucle se detecta **exactamente**, no aproximadamente.

1.6 RIP — el DV que existió de verdad

`RIP` = **Routing Information Protocol**, primer protocolo de Internet (`RFC 1058`, 1988), derivado del `routed` de Berkeley. La versión que se estudia es **RIPv2** (`RFC 2453`, 1998).

Parámetro	Valor en RIPv2	Por qué
Métrica	**hops** (cada enlace vale 1)	la más simple de calcular
Infinito	**16**; máximo válido **15**	con tope, cualquier bucle se disuelve en ≤ 15 rondas
Split horizon / **poison reverse**	sí / sí (forma preferida)	§1.5
Triggered updates / **hold-down**	sí (throttle 1–5 s) / sí (~180 s)	
Timer periódico	30 s	
Timeout (ruta no refrescada)	180 s (6 × 30)	así se detecta que un vecino murió
Garbage collection	120 s tras el timeout	anuncia 16 para que los vecinos también borren
Transporte	**UDP 520**	sin conexión, best effort
Mensaje	Request (1) / Response (2); RTE de 20 bytes; máx. 25 RTE (500 B)	

RIPng: la versión para IPv6 — `RFC 2080` (enero 1997, Proposed Standard). Es **el mínimo cambio** de RIPv2: prefijo de **128 bits**; métrica **1..15** y **16 = ∞**; **split horizon con poison reverse** ("método preferido"); **timers idénticos** (30 s / 180 s / 120 s) más un **offset aleatorio de ±15 s** para que los routers no se sincronicen; **UDP 521** y **multicast** `FF02::9`; campo **route tag** para separar rutas internas de externas; y el **next hop va en un RTE especial** (métrica `0xFF`) que aplica a todas las entradas siguientes.

⚠️ Actualización a 2026

Sigue igual, de memoria: la ecuación de Bellman-Ford y el principio de optimalidad; el conteo a infinito y las cuatro soluciones; **RIPv2** con hops, tope 15 e infinito 16; y que **RIP es Internet Standard pero está muerto de facto**.

Cambió o hay que matizar: - **No existe un "RIP v3" ni un reemplazo de RIP.** Lo que cambió es que **nadie lo despliega**: sobrevive en laboratorios y campus. Su lugar lo tomó **OSPF** (interior) y **BGP** (exterior). - Lo que reemplazó la lógica de "elegir el camino más corto" en los operadores es el **Segment Routing** (`RFC 8402`, `RFC 8660` SR-MPLS, `RFC 8986` SRv6): *source routing* con SIDs y **estado sólo en el borde de ingreso**. En datacenter, **EVPN (`RFC 7432`) + VXLAN (`RFC 7348`)** sobre UDP/4789. - ⚠️ **Trampa:** si preguntan "el RIP de 2026", la respuesta es **RIPv2, `RFC 2453`, sin cambios**.

1.7 Ventajas y desventajas — la respuesta de `2006-08 P3`

Las TRES ventajas: (1) Simplicidad: es un `loop`, un `min` y una tabla. **(2) Poco estado:** cada router guarda **su** tabla de `N` entradas y **no conoce la topología**. **(3) Funciona en topologías donde otros no:** sólo necesita vecinos que le hablen (el único IGP que sobrevive en NBMA).

Las CUATRO desventajas: (1) Convergencia lenta: hasta `D` rondas en una red de `D` saltos. **(2) "Rumor" e inconsistencia transitoria:** la tabla de `x` no es el camino de nadie, es una **combinación** de lo que dijo cada vecino en el último tick; puede haber bucles, caminos imposibles y *black holes*. **(3) El conteo a infinito (§1.4). (4) Escala:** `N` routers × `N` destinos = **`N²` entradas circulando por ronda**.

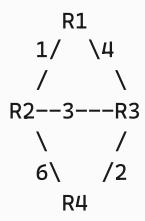
Cada ronda cada router manda su tabla COMPLETA a cada vecino: circulan `N x N` entradas. Con 1 000 routers son 10^6 entradas por ronda = 4 MB por router cada 30 s = **~133 kB/s sostenidos sólo de ripo, en cada router**. Ese es el motivo del tope de 15 hops.

	Distance-Vector	Link-State (OSPF)
Qué conoce cada nodo	sólo lo que le dicen sus vecinos	la topología **completa**
Qué se envía	el vector de distancias (`N` entradas)	un LSA por enlace (pocos bytes)
Algoritmo	Bellman-Ford	Dijkstra
Convergencia	**lenta**, con conteo a infinito	**rápida**, sin conteo a infinito
Memoria por router	`O(N)`	`O(V+E)` del área
Control por ronda	`O(N²)`	`O(N)`
Tope práctico	15 hops	miles de routers

Una distinción que separa a los que entendieron: el *forwarding* es un problema de **tiempo real** (se ejecuta por paquete, error = paquete perdido); el **cálculo de enrutamiento** es de **tiempo diferido** (cada 30 s, no hay deadline, error = la red anda mal). Por eso los protocolos de enrutamiento pueden ser lentos: **no están en el camino crítico del paquete**.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — 4 routers con costos distintos, convergencia en 3 rondas



Costos: `R1–R2 = 1`, `R2–R3 = 3`, `R3–R4 = 2`, `R1–R3 = 4`, `R2–R4 = 6`. Los routers arrancan **sincrónicamente** con la tabla vacía.

Paso 0 — sólo vecinos directos. Cada router conoce `D_x(x) = 0` y `c(x,v)` de sus vecinos (los costos están en sus interfaces, nadie tiene que decírselos).

	R1	R2	R3	R4
R1	**0**	**1**	**4**	INF
R2	**1**	**0**	**3**	**6**
R3	**4**	**3**	**0**	**2**
R4	INF	**6**	**2**	**0**

Ronda 1:

```
D_R1(R4) = min{ 1 + D_R2(R4), 4 + D_R3(R4) }     = min{ 1+6, 4+2 }     = 6  vía R3
D_R2(R4) = min{ 1 + D_R1(R4), 3 + D_R3(R4), 6 } = min{ 1+INF, 3+2, 6+0 } = 5  vía R3
D_R4(R1) = min{ 2 + D_R3(R1), 6 + D_R2(R1) }     = min{ 2+4, 6+1 }     = 6  vía R3
D_R4(R2) = min{ 2 + D_R3(R2), 6 + D_R2(R2) }     = min{ 2+3, 6+0 }     = 5  vía R3
```

	R1	R2	R3	R4
R1	0	1	**1**	**6**
R2	1	0	3	**5**
R3	4	3	0	2
R4	**6**	**5**	2	0

Ronda 2:

```
D_R2(R3) = min{ 1 + D_R1(R3), 3 + D_R3(R3), 6 + D_R4(R3) }
         = min{ 1+1, 3+0, 6+2 } = 2  vía R1     <-- CAMBIÓ
```

Interpretación: R2 descubrió que ir a R3 pasando por R1 (costo `1+1 = 2`) es **más barato** que el enlace directo de costo 3. Acá está el corazón de DV: la métrica no la da el enlace, la da **todo** el camino.

	R1	R2	R3	R4
R1	0	1	1	6
R2	1	0	**2**	5
R3	4	3	0	2
R4	6	5	2	0

Los demás no cambian: `D_R1(R4) = min{1+5, 4+2} = 6` (empate, vía R3); `D_R3(R1) = min{3+1, 4+0, 2+6} = 4` (empate, directo); `D_R4(R1) = 6`; `D_R4(R2) = 5`.

Ronda 3 — convergió: `D_R2(R3) = min{1+1, 3+0, 6+2} = 2`; `D_R1(R4) = min{1+5, 4+2} = 6`; `D_R3(R1) = min{3+1, 4+0, 2+6} = 4`; `D_R4(R1) = min{2+4, 6+1} = 6`; `D_R4(R2) = min{2+3, 6+0} = 5`. **Ninguna celda cambió ⇒ convergió en 3 rondas.**

Caminos óptimos: R1→R2 directo (1) · R1→R3 **vía R2** (1+1 = 2, ¡mejor que el directo de 4!) · R1→R4 **vía R3** (4+2 = 6) · R2→R3 **vía R1** (1+1 = 2) · R2→R4 **vía R3** (3+2 = 5) · R3→R1 directo (empate con R3→R2→R1: 4 = 3+1) · R4→R2 **vía R3** (2+3 = 5).

⚠️ **Errores típicos: olvidar el** `+ c(x,v)` (el error nº 1); **confundir el paso 0** ("vacía" no es "todo ∞"); **no decir en qué ronda convergió**; y en los empates, elegí el directo y **decilo** — la cátedra en `2010-07 P2b` dice "*ante caminos de igual distancia se elige uno al azar*".

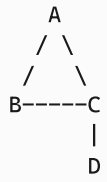
PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2008-12, Problema 2 (30 puntos) — distance-vector en una red de 4 nodos

> **Fuente:** Examen `eirc0812` / `sirc0812` — 20 de diciembre de 2008. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0812.pdf
>
> **Texto del examen:** *Se considera una red de cuatro nodos A, B, C y D con la topología que se muestra en la figura; los enlaces tienen costo unitario. Esta red ejecuta un protocolo de enrutamiento genérico de la familia "vector-distancia" (distance-vector).* **a)** *...los nodos se inician todos en el mismo estado con sus tablas de distancias vacías; deduzca las sucesivas tablas de distancias de A, B, C y D hasta llegar al estado estable.* **b)** *Escriba la tabla de forwarding de cada nodo en el formato Destino, Próximo Salto, Costo.* **c)** *Explique qué sucede con las tablas de distancias si se apaga el nodo D.*

La topología — *adaptamos la figura a texto*

La figura del PDF es una imagen; los valores de la tabla de abajo son los del examen y se deducen de las tablas que la cátedra publica como solución. La red es **un triángulo A-B-C con D colgando de C**:



enlace	A	B	C	D
A	—	1	1	INF
B	1	—	1	INF
C	1	1	—	**1**
D	INF	INF	**1**	—

Cómo lo verifiqué: la tabla de distancias que `sirc0812` publica como estado estable es `A(0,1,1,2) B(1,0,1,2) C(1,1,0,1) D(2,2,1,0)`, y eso **sólo** es consistente con `A–B=1`, `A–C=1`, `B–C=1`, `C–D=1` y **sin** enlace directo `A–D` ni `B–D`: si hubiera `A–D=1`, la distancia A→D sería 1, no 2.

Parte a) — las sucesivas tablas de distancias

Ronda 0 (arranque con tablas vacías ⇒ sólo vecinos directos). ⚠️ "Tablas vacías" significa que **no hay ninguna ruta aprendida de nadie**. Lo que el router sabe de entrada es `D_x(x) = 0` y `c(x,v)` de sus vecinos, porque los costos de sus propios enlaces están en sus interfaces. La cátedra rotula esta ronda `t=0`, y ahí las tablas son, de hecho, la **fila de costos** de la matriz de adyacencia.

desde A	A	B	C	D
A	0	1	1	INF

desde B	A	B	C	D
B	1	0	1	INF

desde C	A	B	C	D
C	1	1	0	1

desde D	A	B	C	D
D	INF	INF	1	0

Ronda 1 — primera ronda de intercambio. Para el destino `D`:

```
A:  D_A(D) = min{ 0+D_A(D), 1+D_B(D), 1+D_C(D), INF+D_D(D) } = min{INF, 1+INF, 1+1, INF} = 2  vía
B:  D_B(D) = min{ 1+D_A(D), 0+D_B(D), 1+D_C(D), INF+D_D(D) } = min{1+INF, INF,  1+1, INF } = 2  v
C:  D_C(D) = min{ 1+D_A(D), 1+D_B(D), 0+D_C(D), 1+D_D(D) } = min{INF, INF,  0+1, 1+0 } = 1  direc
D:  D_D(A) = min{ INF, INF, 1 + D_C(A), 0 + D_D(A) }            = 1 + 1 = 2              vía C
   D_D(B) = min{ INF, INF, 1 + D_C(B), 0 + D_D(B) }            = 1 + 1 = 2              vía C
```

desde A	A	B	C	D
	0	1	1	2

desde B	A	B	C	D
	1	0	1	2

desde C	A	B	C	D
	1	1	0	1
= ESTADO ESTABLE				

desde D	A	B	C	D
	2	2	1	0
= ESTADO ESTABLE				

Interpretación — la observación clave del examen: una sola ronda bastó, porque `C` es el **único cuello de botella** hacia `D` y todo lo que había que aprender de `D` cabía en su mensaje. La red tiene **diámetro de información 1** hacia `D`: es lo que se llama **convergencia trivial**.

Ronda 2 — se verifica que no cambia nada: `A: min{INF, 1+2, 1+1, INF} = 2` ✓ · `B: min{1+INF, INF, 1+1, INF} = 2` ✓ · `C: min{1+2, 1+2, 0+1, 1+0} = 1` ✓ · `D: min{INF, INF, 1+1, 0} = 2` ✓. ⇒ **El estado estable se alcanza en la ronda 1.**

Parte b) — las tablas de forwarding

A	Próx. salto	Costo
A	— (local)	0
B	B	1
C	C	1
D	C	2

B	Próx. salto	Costo
A	— (local)	0
B	— (local)	0
C	C	1
D	C	2

C	Próx. salto	Costo
A	A	1
B	B	1
C	— (local)	0
D	D	1

D	Próx. salto	Costo
A	C	2
B	C	2
C	C	1
D	— (local)	0

Comparación con la oficial. `sirc0812` publica estas cuatro tablas (con la extracción descentrada) y **coinciden con las mías**: para `A`, `(B,B,1) (C,C,1) (D,C,2)`; para `D`, `(A,C,2) (B,C,2) (C,C,1)`. Lo que la oficial no publica son las completas de `B` y `C`; las derivé igual. ⚠ **El destino "A" de la tabla de A tiene próximo salto = local, costo 0**: no es "A→A", es "esto lo despierto yo".

Parte c) — Qué pasa si se apaga D

Solución oficial (citada):

> *"Si se apaga el nodo D se produce un fenómeno de 'conteo a infinito' en los nodos A y B. Esto se debe a que cada uno dice saber llegar a C con cierta distancia (inicialmente 1) y en cada intercambio de tablas se va incrementando en cada nodo (A y B), potencialmente a infinito."*

Se cae el nodo `D`, o lo que es lo mismo, el enlace `C–D` (único que `D` tenía):

```
ronda 2
C:  D_C(D) = min{ 1+D_A(D), 1+D_B(D), 0+INF, INF } = min{1+2, 1+2, INF, INF} = 3  (vía A o B)
A:  D_A(D) = min{ 1 + D_B(D), 1 + D_C(D) }       = min{1+2, 1+3} = 3  <-- CAMBIÓ
B:  D_B(D) = min{ 1 + D_A(D), 1 + D_C(D) }       = min{1+3, 1+3} = 3  <-- CAMBIÓ

ronda 3 :  A: min{1+3, 1+3} = 4 ;  B: 4 ;  C: min{1+3, 1+3, INF, INF} = 4
ronda 4 :  A: 5 ;  B: 5 ;  C: 5
ronda k :  los tres suben 1 por ronda
```

ronda	`D_A(D)`	`D_B(D)`	`D_C(D)`
0	2	2	1
1	2	2	**INF**
2	**3**	**3**	3
3	**4**	**4**	4
4	**5**	**5**	5
k	*k+1*	*k+1*	*k+1*
15	**INF**	**INF**	**INF**

⚠️ **El error conceptual más caro de esta clase, y donde muchas soluciones se caen:** A **no** se queda con su `2` viejo. DV **recalcula desde cero** cada ronda sobre lo que dicen los vecinos: `min{1+2, 1+3} = 3`. No es "monótono decreciente".

Interpretación. Durante 15 rondas los tres routers **creen** que llegan a la red de D por un camino que **ya no existe**: los paquetes que salgan con esas tablas se pierden. En RIP se corta solo en la ronda 15, cuando la métrica llega a 16 y se borra.

Lo que lo arregla: poison reverse. `C` es el único que tiene la verdad, así que en cuanto envenena `D = 16` para todos, `A` y `B` bajan directo a ∞ en **una** ronda (todo en `t=2`). ⇒ **15 rondas de convergencia mala contra 1 de convergencia correcta.** Ese es el argumento si el enunciado pide "cómo lo mejoraría".

Nota: el mismo problema en `2010-02` (`sirc1002`), y dos desvíos del PDF oficial

La topología es idéntica y las tablas son las de la parte a). Lo valioso son dos desvíos del PDF oficial. **(1) La numeración de las rondas:** el bloque rotulado `t=1` para el nodo `A` dice `0 1 INF 1`, que es **exactamente la tabla inicial**: **el PDF duplica la tabla inicial** en el rótulo `t=1` (lo mismo en `sirc0812`). La lectura correcta es `t=0` inicial, `t=1` primera ronda (`A(0,1,1,2) B(1,0,1,2) C(1,1,0,1) D(2,2,1,0)`), `t=2` sin cambios. **Una ronda alcanza**, porque para el destino `D` toda la información cabe en el mensaje de `C` (que anuncia `D=1`). **(2) Un error de tipeo:** la tabla `t=2` de `A` dice `0 1 2 1`, o sea `D_A(D) = 1`, lo cual **no puede ser** (implicaría un enlace directo `A–D` que no existe; si existiera, la tabla estable de `sirc0812` tendría `A→D = 1`, y tiene 2). La fila correcta es `0 1 1 2`. ⚠ **Cuando el PDF y la matemática no cierran, reconstruí la topología desde los datos de otras partes, declarás el supuesto y escribilo.**

La solución oficial de la parte b) dice: "*el nodo C detecta que la ruta más corta a D es inaccesible y por lo tanto elige la segunda más corta que es a través de B con distancia total 3*" — verificado, `1 + D_B(D) = 1 + 2 = 3`, **la cuenta está bien escrita**; pero "*luego B aumenta la suya a 4*" usa un modelo **asíncrono**, cuando las tablas del enunciado son **síncronas** (`D_B(D) = min{1+2, 1+3} = 3` en la primera ronda).

3.4 Examen 2006-08, Pregunta 3 (10 puntos) — funcionamiento, ventajas y desventajas

> **Fuente:** Examen `sirc0608` — 5 de agosto de 2006. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0608.pdf **Texto del examen:** *Describa detalladamente el funcionamiento de un protocolo de enrutamiento de tipo DISTANCE-VECTOR (VECTOR-DISTANCIA). Mencione posibles ventajas y desventajas del mismo.*

(a) Funcionamiento (≈5 puntos), en cinco pasos. (1) Modelo de tabla: un **vector de distancias** por destino, con la distancia mínima y el **primer salto**. **(2) Inicialización:** arranca "vacía", o sea sólo conoce `D_x(x) = 0` y `c(x,v)` de sus vecinos directos. **(3) Timer:** un timer `T` (en RIP, 30 s); al vencer, envía su tabla a **todos** sus vecinos. **(4) Actualización:** al recibir la tabla de un vecino `R`, la **compara con la propia** y la actualiza con Bellman-Ford. **(5) Refresco:** toda ruta no refrescada en la ronda se **borra**, y así se detecta que un vecino dejó de anunciar.

(b) Ventajas (≈2,5 pts) y (c) desventajas (≈2,5 pts): las tres y las cuatro de §1.7.

Solución oficial (citada): "*Conteo a Infinito. Las malas noticias se propagan lento. Cuando un enlace cae, hay un gran número de iteraciones hasta que todos los nodos se dan cuenta que hay un problema.*" ⚠ "Las malas noticias se propagan lento" y "el conteo a infinito" son **el mismo problema desde dos ángulos** (síntoma vs. mecanismo).

Bonus del Problema 1 de ese mismo examen (6 routers): la columna "**Costo en dicho enlace**" es el costo del **primer salto**, no la distancia — la distancia es la **suma** de todos los saltos. Y cada router tiene **dos entradas** por LAN remota ⇒ **ECMP** (*Equal-Cost Multi-Path*). Y el `TTL` es lo único que corta un bucle en el plano de datos: al llegar a 0 manda un ICMP tipo 11.

PARTE 4 — CIERRE

📌 Resumen

1. **Distance-vector**: cada router conoce **la distancia a cada destino** y **el primer salto**. No conoce la topología.
2. Por el enlace viaja un **vector de distancias**, no la topología: por eso se lo llama "rumor".
3. La ecuación es `D_x(y) = min_v{ c(x,v) + D_v(y) }`, correcta por el **principio de optimalidad**.
4. La actualización es **periódica** (30 s en RIP) o **disparada por cambio** (triggered update); las rutas no refrescadas se borran.
5. **Conteo a infinito**: un router vuelve a aceptar un rumor que él mismo originó, y la métrica **sube 1 por ronda**. **DV recalcula desde cero** cada ronda: no es monótono decreciente.
6. Las **4 soluciones**: split horizon, poison reverse, hold-down, triggered updates. **Ninguna es completa**: la métrica es un número, no un identificador de camino.
7. **RIP (`RFC 2453`)**: hops, tope **15**, infinito **16**, UDP 520. **RIPng (`RFC 2080`)**: UDP 521, `FF02::9`, 128 bits.
8. **Ventajas**: simple, poca memoria, topologías raras. **Desventajas**: convergencia lenta, rumor, conteo a infinito, escala `O(N²)`.
9. La **solución de fondo** es **path-vector** (BGP): mandar la lista de por dónde pasó la ruta y rechazarla si uno ya está en ella.
10. **Forwarding ≠ enrutamiento**: el forwarding consulta la tabla en tiempo real; el cálculo no tiene deadline.

✅ Autoevaluación

1. Escribí la ecuación de Bellman-Ford y explicá por qué el camino óptimo de `x` a `y` empieza por el vecino que minimiza `c(x,v) + D_v(y)`.

▶ Respuesta

2. En la topología del §1.4, ¿en qué ronda exacta R1 pasa de ∞ a 4?

▶ Respuesta

3. ¿Por qué poison reverse converge más rápido que split horizon solo?

▶ Respuesta

4. Un router tiene dos rutas al mismo destino de costo 3, aprendidas de R2 y de R3. ¿Qué pasa con split horizon?

▶ Respuesta

5. ¿Por qué el enunciado dice "tablas vacías" y por qué la primera tabla no es todo ∞?

▶ Respuesta

6. ¿Cuánto tarda RIP en suprimir una ruta a 15 hops? Explicá el mecanismo.

▶ Respuesta

7. ¿Qué tiene de malo decir que DV es "O(N^2)"?

▶ Respuesta

8. Cuando R1 tiene `Internet = 4 vía R2`, R2 `4 vía R3` y R3 `4 vía R2`, ¿qué le pasa a un paquete real que sale de R1?

▶ Respuesta

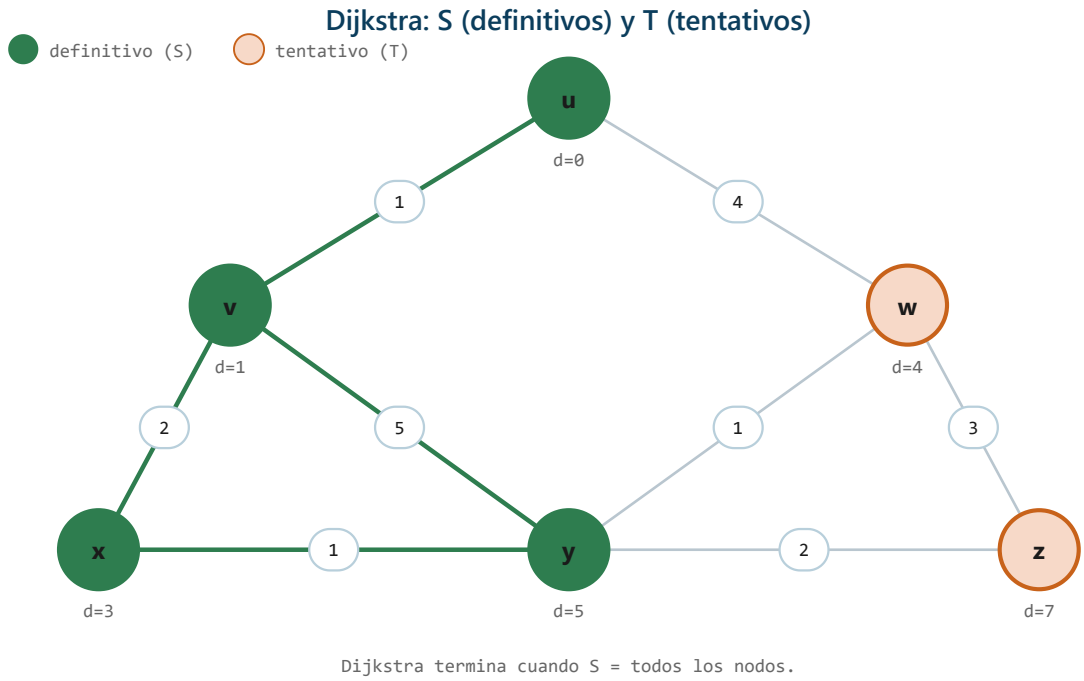
🔗 Referencias

- **`eirc0608` / `sirc0608`** — P3 (10 pts) · Problema 1 (30 pts: 6 routers, TTL, traceroute). Ojo: "**Costo en dicho enlace**" es el costo del **primer salto**, no la distancia; y hay **dos entradas** por LAN remota ⇒ **ECMP**.
- **`eirc0702`** — P3 (10 pts: conteo a infinito y una solución). *Sin solución oficial publicada.*
- **`eirc0812` / `sirc0812`** — Problema 2 (30 pts) · **`eirc1002` / `sirc1002`** — Problema 2 (misma red) · **`sirc1007`** — Problema 2b
- **`sirc0008`** — Problema 1: **aprendizaje reverso** (*backward learning*). Es DV sin métricas: al recibir un datagrama de origen `H` por la interfaz `origen`, `x` aprende "para llegar a `H`, salí por `origen`". Elige con **papa caliente** (métrica = `tamaño_cola`) y resuelve los cambios de topología con **envejecimiento**, que es **exactamente el timeout de RIP**.
- **Tanenbaum & Wetherall**, *Redes de computadores** 5ª ed., cap. 5 §5.6 "Interior Gateway Protocols"
- **RFC 2453** — RIP v2 · **RFC 1058** — RIP · **RFC 2080** — RIPng · **RFC 8402 / 8660 / 8986** — Segment Routing · **RFC 7432 / 7348** — EVPN / VXLAN
- **Clase siguiente:** 33 — Enrutamiento link-state, Dijkstra y OSPF

Clase 33

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 La idea en una frase
1.2 La tabla comparativa de doce filas
1.3 El LSA: qué es y qué lleva adentro
1.4 El flooding, paso a paso, y los tres controles
1.5 Dijkstra: el algoritmo y por qué funciona
1.6 Complejidad y crecimiento del estado — por qué existen las áreas
1.7 Los cinco mensajes de OSPF
1.8 La elección del DR/BDR
1.9 La métrica: por qué "hops" puede ser pésima
2.1 Ejercicio 1 — Dijkstra en 6 nodos, otro grafo
2.2 Ejercicio 2 — Las fases del arranque de OSPF
2.3 Ejercicio 3 — Por qué OSPF no necesita split horizon
3.1 ⭐ Examen 2009-02, Problema 2 (30 puntos) — Dijkstra desde `u`
3.2 ⭐ Examen 2007-03, Problema 2 (30 puntos) — enlaces paralelos y oscilación de métrica
3.3 Examen 2009-12, Pregunta 3 (8 puntos) — el cometido de la capa de red y si "hops" es buena métrica
3.4 Examen 2007-12, Pregunta 1 (5 puntos) — los cinco mensajes de OSPF
📌 Resumen en 10 líneas
✅ Autoevaluación
🔗 Referencias



Dijkstra termina cuando S = todos los nodos.

Bloque: D (Capa de red) · **Semana:** 11 · **Duración estimada:** 120 min **Correlativa:** Clase 32 (distance-vector: Bellman-Ford, conteo a infinito, RIP) · **Sigue con:** Clase 34 (BGP y el enrutamiento entre operadores) **Material de referencia:** Tanenbaum cap. 5 §5.6 · RFC 2328, RFC 5340, RFC 5308

🎯 Al terminar esta clase vas a poder

1. Explicar en qué se diferencia **link-state** de **distance-vector** con una tabla comparativa de doce filas.
2. Describir un **LSA**, hacer el **flooding** de uno paso a paso y justificar cómo la **edad** y el **número de secuencia** evitan el bucle.

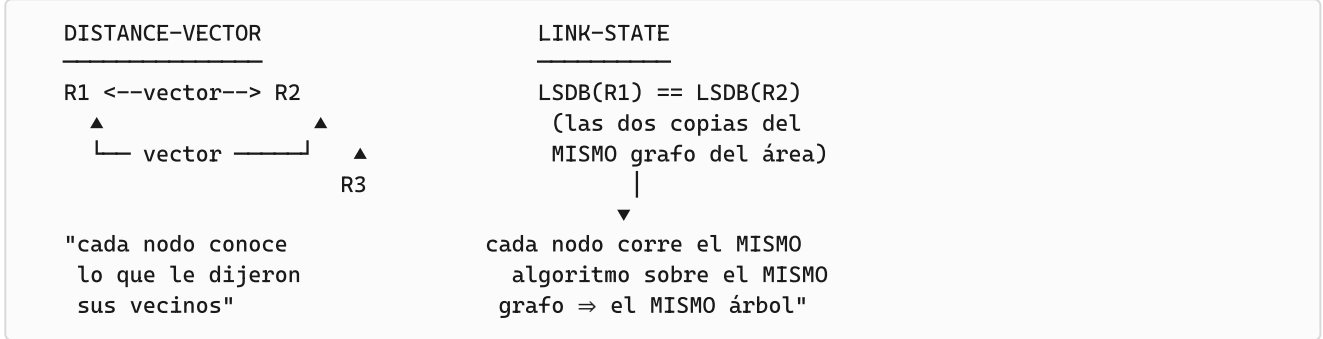
3. Escribir el **algoritmo de Dijkstra** y explicar **por qué funciona** (principio de optimalidad).
4. **Ejecutar Dijkstra a mano con la tabla de iteraciones**, con los nodos definitivos, los tentativos y el vector de distancia.
5. Nombrar y explicar los **cinco mensajes de OSPF**, con sus tiempos, su TTL y sus direcciones de destino.
6. Justificar por qué existen las **áreas**, qué son los **LSA de tipo 3 y 4**, y **por qué se elige un DR/BDR** en una red multiacceso.
7. **Demostrar con números** que la métrica de hops puede elegir el camino pésimo, y resolver el `2009-12 P3`.
8. Analizar la **oscilación de métrica** entre enlaces paralelos del `2007-03 Problema 2` y proponer la medición de ancho de banda que la evita.

> **Prerequisito del examen:** `2009-02 Problema 2` (30 pts), `2007-03 Problema 2` (30 pts), `2009-12 Pregunta 3` (8 pts), `2007-12 Pregunta 1` (5 pts).
>
> ⚠️ **Nota sobre el código de un examen.** El enunciado "Explique resumidamente los 5 mensajes que se implementa en OSPF" (5 puntos) **no es** `2007-03 P1`: en `eirc0703` / `sirc0703` la Pregunta 1 es "*Describa el modelo de capas TCP/IP*". El examen que pregunta exactamente eso es **`eirc0712` / `sirc0712`, Pregunta 1** (27 de diciembre de 2007). Lo resolvemos en §3.4 con el código correcto. Además, el PDF de `sirc0712` **no publica la solución de las preguntas teóricas**, así que §3.4 va marcada como *(solución propia, contrastada con RFC 2328)*.

PARTE 1 — TEORÍA

1.1 La idea en una frase

> **En distance-vector cada router cree que el mundo es como le dijeron sus vecinos. En link-state cada router tiene un mapa del mundo y corre el mismo cálculo que sus vecinos.**



1.2 La tabla comparativa de doce filas

#	Aspecto	**Distance-vector**	**Link-state**
1	Qué conoce cada router	Solo sus vecinos directos	**Toda la topología del área**
2	Qué mensaje intercambia	Un **vector de distancias** (su tabla entera)	Un **LSA** (el estado de **un** enlace)
3	Tamaño del mensaje	Crece con el número de destinos	**Constante**: el tamaño de un enlace
4	Conocimiento topológico	**No existe** en el sistema	Existe y es **idéntico** en todos los routers del área
5	Algoritmo	**Bellman-Ford**	**Dijkstra** (SPF)
6	Periodicidad	Temporizador (en RIP 30 s)	**Evento**: el flooding es inmediato al cambiar
7	Problema clásico	**Conteo a infinito** (clase 32)	**Ninguno**: los caminos convergen al árbol más corto
8	Convergencia	**Lenta**: proporcional al número de saltos	**Rápida**: proporcional al **log** del número de routers
9	Ciclos / bucles	Necesita **split horizon**, *poison reverse*, *hold-down*	**No los necesita**: el árbol no tiene ciclos por construcción
10	Reacción ante un enlace caído	La noticia tarda hasta el temporizador y los routers se realimentan	El LSA del enlace se inunda **ya** y todos recalculan
11	Memoria y CPU	**Bajo**: tabla chica, cálculo simple	**Alto**: LSDB completa y SPF costoso, de ahí las **áreas**
12	Escala	LAN o campus chico (RIP: 15 saltos)	**Red corporativa, backbone de operador, datacenter** (OSPF, IS-IS)

> **Las filas 9 y 12 van juntas:** link-state no necesita split horizon *precisamente porque* paga el precio de la fila 11. Un mecanismo no se borra gratis: se cambia por otro.

1.3 El LSA: qué es y qué lleva adentro

Un **LSA (Link State Advertisement)** describe **el estado de los enlaces de un router**, no sus rutas.

```
┌──────────────────────────────────────────────────────┐
│  LSA  (cabecera común a todos los tipos)             │
│  ├─ LS Age (edad)            ← cuánto vive este LSA  │
│  ├─ Options                  ← capacidades del router│
│  ├─ LS Type                  ← 1..5                  │
│  ├─ Link State ID            ← quién lo origina      │
│  ├─ Advertising Router       ← el Router ID          │
│  ├─ LS Sequence Number       ← control de frescura   │
│  ├─ LS Checksum              ← detecta corrupción    │
│  └─ Length                                           │
│  Cuerpo. Para el TIPO 1 (Router-LSA):                │
│  ├─ Link ID 1  ─┐                                    │
│  ├─ Link Data 1 ├─ (interfaz, tipo de red, MASK,     │
│  ├─ Link Type 1 ┘  costo hacia el vecino)            │
│  ├─ Link ID 2 ...                                    │
│  └─ Link ID n ...   ← un par POR CADA ENLACE         │
└──────────────────────────────────────────────────────┘
```

Lo clave: **el LSA no dice "a `x` se llega a 3 hops"**. Dice **"del router `u` al router `v` hay un enlace y mi costo es 2"**. Es un hecho local, verificable, y por eso no puede ser un rumor: o el enlace está o no está.

El LSDB (Link State Database) es el conjunto de todos los LSA del área que tiene un router. **La condición de sincronización es simple y es toda la garantía del protocolo:**

```
LSDB(R1)  ==  LSDB(R2)  ==  LSDB(R3)  ==  ...  ==  LSDB(Rn)
```

Si dos routers del área tienen **el mismo grafo**, entonces **calculan el mismo árbol**, aunque lo calculen en instantes distintos. Distance-vector no puede decir esto: sus entradas son distintas porque las estimaciones nacen de rumores distintos.

Tipo	Nombre	Qué describe	Quién lo genera
1	Router-LSA	Los **enlaces del router** (vecinos, costos, máscaras)	Todo router del área
2	Network-LSA	Los routers **conectados a una red multiacceso**	El **DR/BDR** (§1.7)
3	Summary-LSA	Un **prefijo** y su costo desde el ABR hacia él	Un **ABR** (§1.7)
4	ASBR-Summary-LSA	Cómo llegar al **ASBR** que anuncia externos	Un **ABR**
5	External-LSA (ASE)	Una ruta **de otro sistema autónomo**, con métrica de tipo externo	Un **ASBR** (redistribuidor)

> El tipo 5 conecta esta clase con la 34: los LSA de tipo 5 son **rutas que vienen de BGP**. Por eso son de tipo externo y su costo no es comparable con el de los enlaces del área.

1.4 El flooding, paso a paso, y los tres controles

Flooding (*inundación*) es la propagación de un LSA a **todos** los routers del área. Ya lo vimos como mecanismo de reenvío en la Clase 24 §1.6; acá cambia **quién** lo hace y **con qué** lo controla.

```
Paso 0: R3 detecta que el enlace (R3,R5) costaba 7 y ahora cuesta 70.
        Genera un LSA nuevo:  (LS id = R3, seq = 0x2000000E, age = 0)
                                                          |
Paso 1: R3 lo manda por CADA interfaz, menos por la       ▼
        que lo recibió.
     ┌──────┬──────┬──────┬──────┐
     ▼      ▼      ▼      ▼      ▼
   R1     R2     R4     R5     R6        (y R3, que lo originó)

Paso 2: cada vecino que lo recibe hace TRES preguntas:
           ¿ya lo tengo?
                ├─ sí, misma instancia ──► DESCARTO. No reenvío. (fin)
                ├─ sí, pero más viejo  ──► lo actualizo y REENVÍO
                ├─ sí, pero más nuevo  ──► lo ignoro (era un atraso)
                └─ no                  ──► lo instalo y REENVÍO

Paso 3: R1, que ya lo tenía viejo, lo actualiza y lo reenvía por
        TODAS sus interfaces menos por donde lo recibió.
        R2 lo recibe de R1 y ya lo tiene → DESCARTA.

Resultado: en el grafo Steiner del flooding hay un árbol: O(N)
           reenvíos, no O(N²).
```

```
(1) NÚMERO DE SECUENCIA  (32 bits con signo, declarado)
    InicialSequenceNumber = 0x80000001  (el más negativo)
    MaxSequenceNumber     = 0x7FFFFFFF  (el más positivo)
    Rango negativo = "recién creado"; positivo = "estable".
    Cuando hay que actualizar y se está en 0x7FFFFFFF, se hace
    "flush" (se inunda el LSA viejo con MaxAge) y se arranca en
    0x80000001. Es el wrap de 32 bits con signo. ¿Para qué? Para
    que "nuevo" sea un orden TOTAL, no ambiguo.

(2) EDAD  (LS Age, en segundos; MaxAge = 3600 s)
    Todo LSA tiene un reloj que corre. A MaxAge se lo considera
    EXPIRADO y se tira de la base. Cuando el origen detecta que
    perdió la vista del LSA, lo inunda con Age = MaxAge: eso es
    el "flush", la forma limpia de "borrar" sin mensaje de borrado.
```

Y el tercer control, que es del flooding y no del LSA: un router **no reenvía por la interfaz por la que recibió** el LSA. Los tres juntos:

```
¿por qué no vuelve para atrás?    → NO se reenvía por la interfaz de
                                    entrada (enlace punto a punto, o
                                    tabla de vecinos en multiacceso)
¿por qué no vuelve por otro lado? → el que ya lo tiene NO lo reenvía
¿y si llega un LSA viejo?         → se ignora y no se reenvía
```

> **Dato para el examen:** los tres son necesarios. Sin el primero, dos routers de una línea punto a punto se pisan. Sin el segundo, el LSA da vueltas por un anillo. Describí los tres controles y el para qué de cada uno.

El flooding confiable: los tres mensajes que lo sostienen

El flooding puro es fiable pero **desperdicia ancho de banda** en redes grandes. OSPF lo hace **fiable con acuse de recibo**, y esa es la razón de existir de tres de los cinco mensajes:

```
Flujo normal:      LSU  ──────────────►  vecino
                   ◄──────────────  LSAck

Recuperación: el vecino NO acusa recibo (se perdió el LSU):
                   ◄─── LSU, reenviado
                   ───► LSAck

"Me falta": el vecino tiene algo que yo no tengo:
                   ───► LSR  (pide ESA LSA específica)
                   ◄─── LSU (con la LSA pedida)
                   ───► LSAck
```

Es una **ventana deslizante sobre el LSDB**: el origen manda hasta `N` LSAs pendientes de acuse; si no llegan en `RxmtInterval` (5 s), reenvía; si tras `N` intentos no consigue nada, **desarma la vecindad**.

1.5 Dijkstra: el algoritmo y por qué funciona

```
 1  Initialization:
 2      N' = {u}
 3      for all nodes v
 4          if v adjacent to u
 5              then D(v) = c(u,v)
 6          else D(v) = ∞
 7
 8  Loop
 9      find w not in N such that D(w) is a minimum
10      add w to N'
11      update D(v) for all v adjacent to w and not in N:
12          D(v) = min( D(v), D(w) + c(w,v) )
13          /* new cost to v is either old cost to v or known
14             shortest path cost to w plus cost from w to v */
15  until all nodes in N'
```

- **Líneas 3 a 6 — la inicialización.** `D(v)` es "mi **mejor estimación** del costo de llegar a `v`". Los vecinos directos se inicializan con el costo **real y directo** del enlace, que conozco sin preguntar a nadie. Todo lo demás arranca en `∞`.
- **Línea 9 — el mínimo sobre los que NO están en `N'`.** `N'` es el conjunto de nodos ya **definitivos**: su distancia ya no va a cambiar.
- **Línea 10 — se vuelve definitivo.** Y acá está **la clave de por qué funciona**: en cuanto `w` entra a `N'`, su `D(w)` **ya no puede mejorar**, y por lo tanto **nunca más vamos a necesitar recalcular usando `w` como punto de paso**. Por eso la línea 11 filtra "para todos los `v` adyacentes a `w` **y que no están en `N'`**": si `v` ya fuera definitivo, actualizarlo desde `w` no lo mejoraría. Los dos filtros juntos son la **invariante**: "`N'` contiene exactamente los nodos cuya distancia ya es óptima".
- **Línea 12 — la relajación.** `v` mejora si pasar por `w` sale más barato que llegar por lo que ya tuviera.
- **Línea 15 —** en un grafo conexo son `V − 1` iteraciones (más la inicialización).

Por qué funciona: el principio de optimalidad, demostrado

> **Principio de optimalidad (Bellman, 1957).** Si el camino más corto (con costos no negativos) de `u` a `v` pasa por `w`, entonces el tramo de `w` a `v` es él mismo un camino más corto desde `w`.

Demostración por contradicción, que es lo que hay que escribir en el examen:

Supongamos que el algoritmo elige `w` porque `D(w)` es el mínimo, y supongamos que existe un camino más barato a `w` que no pasa por `w` todavía en `N'`. Ese camino debe salir de `N'` por **algún** nodo `z` que ya es definitivo. Pero entonces ese camino cuesta

```
costo(u → … → z) + costo(z → … → w)
     ≥  D(z)  +  c(z,w)  ≥  D(z)  ≥  D(w)
```

porque `D(z)` es óptimo (invariante), porque `D(z)` era el mínimo cuando `z` entró a `N'`, y porque `c(z,w) ≥ 0`. O sea: **el camino más barato a `w` no puede ser más barato que `D(w)`**. `D(w)` es óptimo y `w` puede ser definitivo. ■

Y acá está la diferencia con Bellman-Ford: Dijkstra **exige** `c(z,w) ≥ 0`. Si un costo fuese negativo, el último paso de esa cadena (`+ c(z,w)`) podría **bajar** el valor y la desigualdad se rompe. Por eso **Dijkstra no funciona con costos negativos**: no es que Bellman-Ford sea "mejor", es que su condición es más débil.

```
Dijkstra      : costos ≥ 0  →  correcto y RÁPIDO (greedy, una pasada)
Bellman-Ford  : costos ≥ 0  →  correcto y LENTO (relaja V-1 veces)
Bellman-Ford  : costos < 0  →  correcto (es el único que puede)
Dijkstra      : costos < 0  →  INCORRECTO (greedy congela un nodo malo)
```

> **Para escribir en el examen:** no escribas "Dijkstra encuentra el camino más corto". Escribí "*Dijkstra mantiene la invariante de que el conjunto `N'` contiene exactamente los nodos cuya distancia mínima ya es conocida y que no va a cambiar, y la mantiene porque toda actualización ocurre sobre un nodo no definitivo; termina cuando `N' = V`, momento en el que todas las distancias son óptimas*". Eso es un 9-10. La frase de una línea es un 6.

La anatomía de la tabla de iteraciones — sin ella no hay puntos

Paso	N' (definit.)	D(v), D(w), D(x), D(y), D(z) (con el predecesor entre paréntesis)
0	u	2,u 5,u 1,u ∞ ∞
1	u x	2,u 4,x – 2,x ∞
2	u x y	2,u 3,y – – 4,y
3	u x y v	– 3,y – – 4,y
4	u x y v w	– – – – 4,y
5	u x y v w z	(fin)

▲ └── el nodo que ENTRA en esta iteración

Tres convenciones que hay que declarar antes de empezar (escribilas, son puntos):

1. `N'` es el conjunto de nodos **definitivos**. Los nodos ya no se anotan en la columna de `D`, porque su valor **no cambia más**.
2. `D(n), p` = distancia mínima a `n` y **su predecesor**, que es lo que después arma el shortest-path tree y la tabla de forwarding.
3. Se **marca con una flecha** el nodo que entra en cada paso. Es la prueba de que no estás inventando la tabla.

1.6 Complejidad y crecimiento del estado — por qué existen las áreas

```
Bellman-Ford, versión matriz:      V rondas × V columnas    = O(V³)
Bellman-Ford, versión lista:       V rondas × E aristas     = O(V·E)
Dijkstra, matriz (min sobre V):    V pasos   × V búsqueda   = O(V²)
Dijkstra, cola de prioridad:       E relajaciones × log V   = O((V+E)·logV)
```

Red	Routers	Bellman-Ford ($O(V^3)$)	Dijkstra ($O(V^2)$)	Dijkstra con heap
Laboratorio	10	1.000	100	~30
Campus	100	1.000.000	10.000	~700
Operador grande	10.000	10^{12}	100.000.000	~1.400.000

Con `V = 10.000`, Bellman-Ford por matriz es **una billonada** de operaciones por cada cambio de enlace. Ni un procesador dedicado a routing aguanta eso.

```
Sin áreas:  todo router guarda una LSA por cada enlace de cada router
──────────────────────────────────────────────────────────────────────
Si hay N routers y cada uno tiene, digamos, 4 enlaces:
    número de LSAs en el LSDB de CADA router  =  4·N
    tamaño del LSDB                           =  O(E)  =  O(N)

Con N = 20.000 routers y 4 enlaces cada uno:
    80.000 LSAs por router × ~100 bytes  =  ~8 MB por router
    ... y eso en los 20.000 routers que los tienen TODOS:
    ~160 GB de memoria solo en LSDB si nadie jerarquiza.

──────────────────────────────────────────────────────────────────────
Y lo peor no es la memoria: es el TRÁFICO DE INUNDACIÓN.
Un cambio de enlace genera 1 LSA que viaja por todo el árbol.
Si ese cambio se reacomoda cada 30 segundos (una ruta que oscila)
× 80.000 routers × 100 bytes:  ≈ 2,3 Gbit/s de flooded por UN enlace.
```

El flooding es **O(N) por evento**; con muchos cambios simultáneos, **O(N × cambios)**. Esa multiplicación es lo que las áreas matan.

1.7 Los cinco mensajes de OSPF

Ésta es la parte que entra como pregunta de 5 puntos.

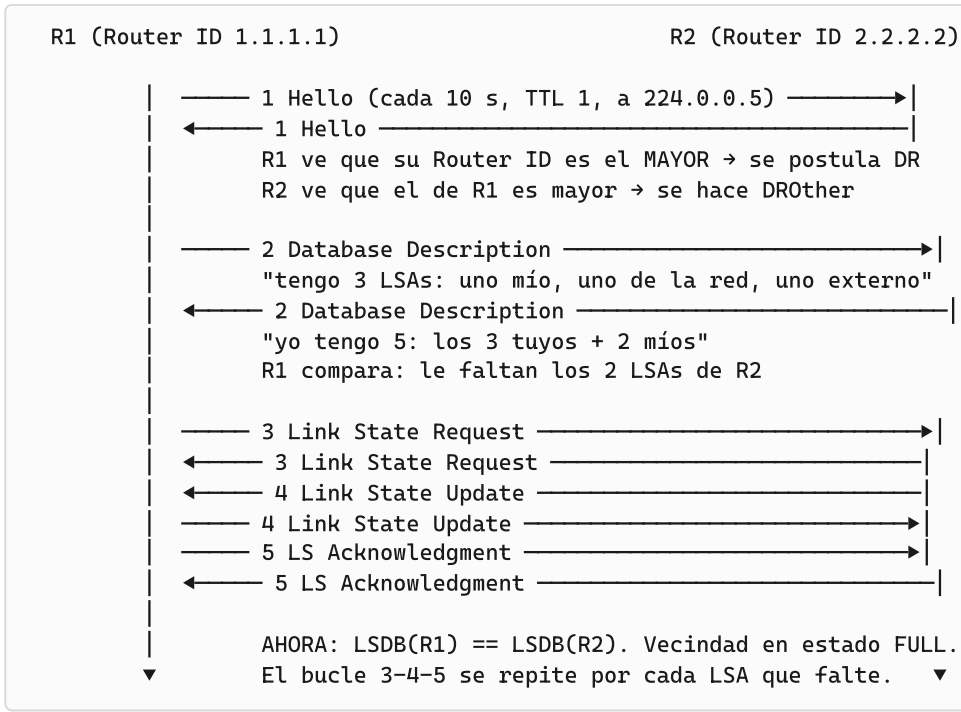
#	Mensaje	Código	Para qué	Cuándo	TTL	Destino
1	**Hello**	**1**	Descubrir y **mantener vivos** a los vecinos; negociar los intervalos; elegir el **DR/BDR**; descubrir el Router ID del vecino	**Cada 10 s** (p2p y broadcast); 30 s en NBMA/P2MP	**1**	`224.0.0.5` y `224.0.0.6`
2	**Database Description** (**DBD**)	**2**	En el arranque, **describir el contenido** del LSDB propio para que el vecino diga qué le falta	Arranque y refresco cada 30 min	**1**	unicast al vecino
3	**Link State Request** (**LSR**)	**3**	**Pedir explícitamente** una LSA que falta	Al completar la base, y en cualquier momento	**1**	unicast al vecino
4	**Link State Update** (**LSU**)	**4**	**Inundar** LSAs nuevas o actualizadas	Al flooding y al arranque	**1**	`224.0.0.5` o unicast
5	**Link State Acknowledgment** (**LSAck**)	**5**	**Confirmar** la recepción	Por cada LSU recibido	**1**	unicast al vecino

```
┌─ (a) TIMING ──────────────────────────────────────────────┐
│  HelloInterval = 10 s ; DeadInterval = 40 s (= 4 × Hello)    │
│  Por qué 40 y no 10: para withstand UNA pérdida de un Hello sin│
│  declarar muerto al vecino. Con 4× se toleran 3 pérdidas.     │
├─ (b) TTL = 1 ─────────────────────────────────────────────┤
│  OSPF corre directamente sobre IP (protocolo 89), NO sobre UDP │
│  ni sobre TCP: es un protocolo de capa de red, punto a punto.  │
│  TTL = 1: el paquete morre en el primer router. Un Hello es un │
│  MENSAJE DE VECINDAD LOCAL: su único destinatario válido es el │
│  router del otro lado del cable. Si alguien lo reenvía, hay un │
│  router mal configurado en el medio. El TTL=1 no es una       │
│  optimización: es una barra de seguridad.                     │
├─ (c) DIRECCIONES MULTICAST ──────────────────────────────┤
│  224.0.0.5 = AllSPFRouters → todos los routers OSPF           │
│  224.0.0.6 = AllDRouters   → solo el DR y el BDR              │
│  Los dos Hello van a 224.0.0.5 SIEMPRE; a 224.0.0.6 solo      │
│  cuando uno se postula a DR o BDR.                            │
│  224.0.0.0/24 está reservada para que un paquete no pueda     │
│  salir del segmento: el mismo truco que ff02::/16 en IPv6.    │
└──────────────────────────────────────────────────────────────┘
```

> **Protocolo IP 89** (ospf = 89) es el número que va en el campo *Protocol* de la cabecera IP. Memorizalo junto con el **179** de BGP, de la clase siguiente.

El intercambio completo de los cinco

Ojo con la distinción clave: el Hello es periódico, los otros cuatro son del arranque.

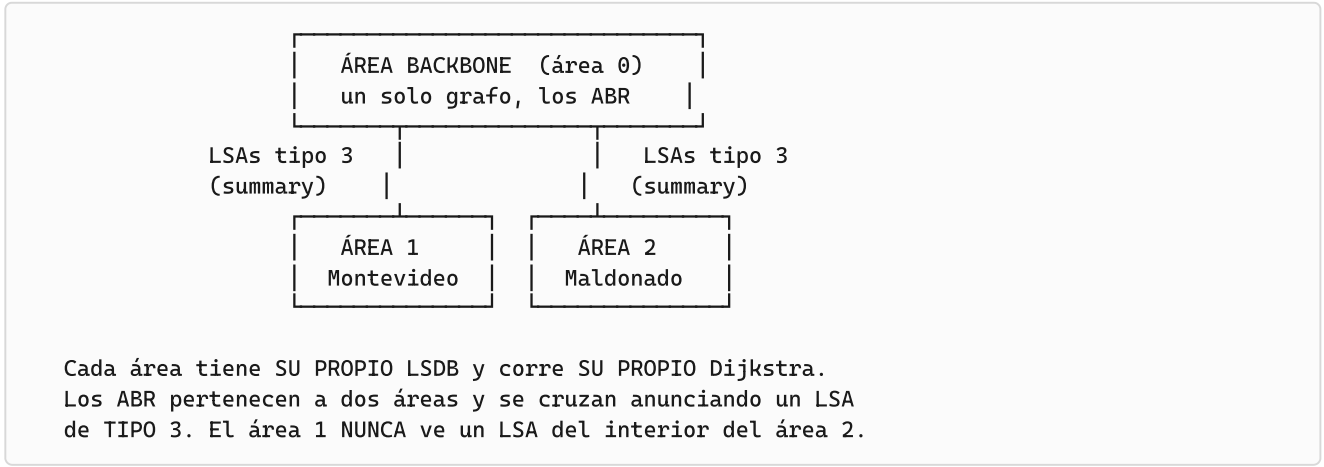


La máquina de estados de la vecindad (para dibujarla si te la piden):

```
Down —(Hello recibido)→ Init —(tu Router ID en su lista)→ 2-Way
                          ▼
                        ExStart —(elige DR/BDR)→ Exchange
                          ▼
                        Loading —(ya no falta nada)→ Full
En cualquier estado: si expira el DeadTimer → Down.
En 2-Way multiacceso: solo el DR pasa a Exchange; los DROther quedan.
```

Las áreas y los LSA de tipo 3 y 4

Si el LSDB completo de una red de 20.000 routers es insostenible, la solución es **no calcular el grafo entero**.



Por qué funciona, en dos frases: (1) **la exactitud se preserva dentro del área**: entre dos routers del área 0, Dijkstra sigue dando el camino más corto entre todos los del área 0, y los prefijos de las otras áreas se agregan después como un LSA de tipo 3; (2) **lo que se sacrifica es la optimalidad global**, y es un compromiso consciente: el camino entre el área 1 y el área 2 pasa obligatoriamente por el backbone.

	Tipo 3 — Summary-LSA	Tipo 4 — ASBR-Summary-LSA
Qué anuncia	Un **prefijo IP** y el costo desde el ABR que lo anuncia	Cómo llegar al **ASBR** (el que hace *redistribución*)
Quién lo genera	Un **ABR**	Un **ABR**
Para qué	Para que el área destino **no** necesite conocer el interior de la otra	Para poder **costear** el camino hacia una ruta externa que el área no contiene

```
tipo 3 dice:  "la red 192.0.2.0/24 está a costo 7 DESDE MÍ"
tipo 4 dice:  "el ASBR que anuncia las externas está a costo 4 DESDE MÍ"

Sin el tipo 4, un router del área 0 que recibe un tipo 5 no puede
saber qué costo poner al camino hacia ese ASBR, que no está en
su área. El tipo 4 es el puente que permite costear el prefijo
externo usando el grafo del backbone.
```

> **Concepto que vale puntos:** los LSA de tipo 3 son **la implementación del jerarquizado**. Pregunta clásica: "*¿qué pasaría si OSPF no tuviera áreas?*" → el LSDB y el SPF serían **O(tamaño de la red)** en vez de **O(tamaño del área)**, y el flooding de un cambio local impactaría a **toda** la red.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - `RFC 2328` (**OSPF**, 1998) es la especificación vigente de **OSPFv2**. - Los **cinco mensajes** con sus códigos 1 a 5, sus intervalos (10 s / 40 s), el **TTL = 1** y los **multicast 224.0.0.5 y 224.0.0.6**. - El algoritmo es **Dijkstra/SPF**, el LSDB se sincroniza, el flooding es fiable con LSR/LSU/LSAck. - **Áreas**, **backbone (área 0)**, **ABR**, **ASBR**, LSA de **tipo 1 a 5**, elección de **DR/BDR** en redes multiacceso. - El **TLV de costo de interfaz** y el concepto de *reference bandwidth*. - **IS-IS** (ISO 10589) es el otro IGP link-state que se despliega masivamente, sobre **CLNS**, directamente en capa 3 sin IP.

Cambió, o hay que matizar:

- **OSPFv3 (`RFC 5340`, 2009)** es el equivalente para **IPv6**: mismos mecanismos, mismos cinco mensajes, mismas áreas, mismos DR/BDR, pero independiente de IPv4. **No es una reescritura**: es un protocolo paralelo. Lo único que cambia de verdad es el direccionamiento y que **cada interfaz necesita su propio Router ID** (ya no se puede tomar "la IP más alta de la interfaz", porque en IPv6 las direcciones son efímeras).
- **IS-IS para IPv6**: `RFC 5308` define las extensiones de IS-IS para puro IPv6. El mecanismo (SPF, LSDB, L1/L2) es idéntico.
- **El plano de control se está moviendo a Segment Routing**: `RFC 8402` (2018), con **SR-MPLS (`RFC 8660`)** y **SRv6 (`RFC 8986`)**. Lo importante es el cambio de paradigma: **el estado pasa a estar solo en el borde de ingreso** y cada nodo aplica una política local, sin base de datos de caminos.
- **MPLS se apoya en un IGP por debajo**, típicamente **OSPF o IS-IS**. Si el datacenter usa EVPN-VXLAN, el plano de control es **BGP** (próxima clase) y **OSPF deja de usarse como IGP**. ⇒ **La consecuencia conceptual: OSPF no se usa para calcular rutas en datacenter porque el dominio es demasiado grande y demasiado plano.** La jerarquía que el IGP no te da, te la da BGP.
- **`BGP-LS` (Link-State) es hoy** `RFC 9552` (dic-2023, obsoleta `RFC 7752`): distribuye información de link-state **dentro de BGP** para que un controlador externo calcule. Es la manera moderna de sacar Dijkstra del router

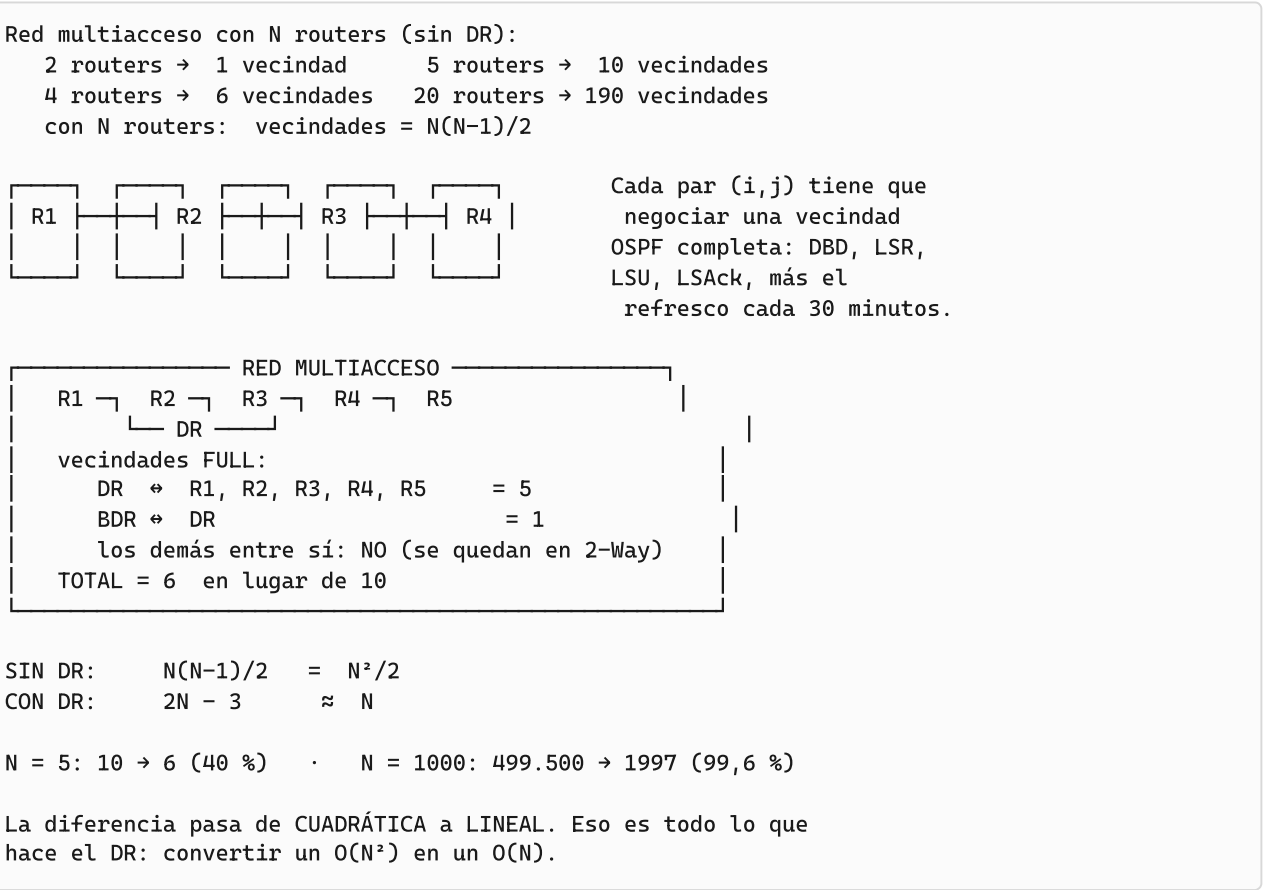
y ponerlo en un servidor.

- **Lo que NO se pregunta**: LSA **opacos**, *link-local signaling*, extensiones para *multi-topology*. Estandarizadas sí, pero la cátedra no las da.

Consecuencia para el examen: `2009-02 Problema 2a` y `2007-03 Problema 2` se resuelven con **el algoritmo y el flooding, que no cambiaron**. Si el enunciado menciona OSPF por nombre, contestá el mecanismo y **agregá al final una línea de actualización**: "*hoy el equivalente para IPv6 es OSPFv3 (RFC 5340) y el IGP link-state más desplegado en operadores es IS-IS*".

1.8 La elección del DR/BDR

En una red de acceso múltiple (*multiaccess*: Ethernet, FDDI, token ring), **todos los routers están en el mismo dominio de colisión lógico**. En una red punto a punto cada router tiene **exactamente un vecino**; en una multiacceso, tiene **todos los demás**.



Y el LSA de tipo 2 es el precio que se paga. Como hay uno solo que habla por la red, es el DR quien genera el **Network-LSA (tipo 2)**: "en esta red multiacceso están estos `K` routers con estos Router IDs, y la red tiene este costo". Sin DR, **cada router** tendría que generar el suyo con la lista completa, y la base explotaría.

```
La elección, en detalle:
  1. ¿Mi Router ID es MAYOR que el del vecino en la interfaz?
       SÍ  → me postulo           NO  → él se postula
  2. ¿Tengo la MAYOR prioridad declarada en la interfaz?
       SÍ  → gano                 NO  → pierdo
  3. Prioridades iguales (1 en broadcast, 0 en p2p por omisión):
       gana el MAYOR Router ID.
El DR se re-elige si el DR actual cae; el BDR, entre los DROther.
En punto a punto NO hay DR: N va a 2.
```

1.9 La métrica: por qué "hops" puede ser pésima

Dijkstra asume que **hay una métrica** y que es **lo que se quiere minimizar**. Lo que no dice es *cuál*.

```
Una métrica de enrutamiento tiene que responder a UNA pregunta:
    ¿qué le importa al usuario de este camino?
"menos latencia"         → métrica = retardo
"más ancho de banda"     → métrica = ancho de banda (invertida)
"menos pérdida"          → métrica = tasa de pérdida
"estabilidad"            → métrica = preferencia de política
"que la ruta sea simple" → que la ruta sea la más SIMPLE

Y "cantidad de hops" no responde a ninguna de las cinco.
```

El caso que hace daño, y por qué es antiintuitivo. El caso "10 saltos contra 1 salto" **no** hace daño: si un camino son 10 saltos de 1 Mbit/s y el otro 1 salto de 100 Mbit/s, la métrica de hops elige el de menos saltos, o sea el rápido, y acierta. El caso que hace daño es el **inverso**:

```
┌──────────────────────────────────────────────────────────┐
│  Camino A:   1 salto    ·  10 Mbit/s   ← lento pero corto  │
│  Camino B:  10 saltos   ·  1 Gbit/s    ← rápido pero largo  │
│                                                          │
│  ¿Cuál elige "hops"?  →  Camino A (1 < 10).              │
│  ⚠ ¿Y cuál es el que el usuario quiere?  →  Camino B.    │
│                                                          │
│  Porque el cuello de botella no lo fija la cantidad de   │
│  saltos, lo fija EL ENLACE MÁS ESTRECHO DE CADA CAMINO.  │
└──────────────────────────────────────────────────────────┘
```

Y la **métrica de OSPF** es justamente una respuesta a esto:

```
OSPF no usa "cantidad de saltos". Usa el COSTO DE INTERFAZ, que es
un entero de 16 bits derivado del ancho de banda:

     costo_interfaz =  ancho_de_banda_de_referencia
                       ____________________________
                          ancho_de_banda_de_la_interfaz

Con la referencia por omisión del RFC (100 Mbit/s):
     10 Mbit/s  → 100/10 = 10      100 Mbit/s → 100/100 = 1
     1 Mbit/s   → 100/1  = 100     2 Gbit/s   → 0,05 → 0 → piso a 1

⚠ Ese último renglón es el problema del campo de 16 bits: todo lo
más rápido que la referencia da costo 0. Por eso los equipos
modernos configuran la referencia en 100 Gbit/s. Es un parche
empírico sobre una decisión de diseño de 1998 que ya no escala.

Con referencia en 100 Gbit/s:
     10 Mbit/s → 10.000     1 Gbit/s → 100     100 Gbit/s → 1
     Ahora sí hay rango: el campo de 16 bits llega hasta 65.535.
```

> **Lo que OSPF NO tiene: no hay ancho de banda en la métrica, hay COSTO**, y el costo se configura a mano. Por eso el OSPF por sí solo no balancea carga: si no ponés el costo vos, elige el camino con menos saltos.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Dijkstra en 6 nodos, otro grafo

Datos. Seis nodos `a, b, c, d, e, f` con estos costos: `a–b` 4, `a–c` 1, `a–d` 2, `b–c` 7, `b–d` 2, `b–f` 5, `c–d` 5, `c–e` 9, `d–e` 6, `d–f` 3, `e–f` 4.

```
a ──4── b        a ──1── c        b ──2── d
a ──9── c        b ──7── c        c ──5── d
a ──2── d        c ──9── e        d ──6── e
b ──5── f        d ──3── f        e ──4── f
```

Paso 1 — Inicialización. Los vecinos de `a` son `b` (4), `c` (1) y `d` (2). Los otros, `∞`. Entra `c`.

Paso 2 — Relajo los vecinos de `c` fuera de `N'`: `b`, `d`, `e`.

```
b:  min( 4 , 1 + 7 ) = min(4, 8) = 4   → NO mejora
d:  min( 2 , 1 + 5 ) = min(2, 6) = 2   → NO mejora
e:  min( ∞ , 1 + 9 ) = min(∞,10) = 10  → SÍ, 10 vía c
```

Paso 3 — Entra `d` (`D(d)=2`). Vecinos de `d` fuera de `N'`: `b`, `e`, `f`.

```
b:  min( 4 , 2 + 2 ) = min(4, 4) = 4    → empate, se queda el original
e:  min( 10, 2 + 6 ) = min(10, 8) = 8   → SÍ, 8 vía d
f:  min( ∞ , 2 + 3 ) = min(∞, 5) = 5    → SÍ, 5 vía d
```

Paso 4 — Entra `b` (`D(b)=4`). Vecinos de `b` fuera de `N'`: solo `f`. `min(5, 4+5) = 5` → NO mejora.

Paso 5 — Entra `f` (`D(f)=5`). Vecinos de `f` fuera de `N'`: `e`. `min(8, 5+4) = 8` → NO mejora.

Paso 6 — Entra `e` (`D(e)=8`). No quedan nodos fuera de `N'`: fin.

```
Paso  N'           D(b)     D(c)     D(d)     D(e)      D(f)     nodo
──────────────────────────────────────────────────────────────────────
  0   {a}          4,a      1,a      2,a       ∞         ∞        –
  1   {a c}        4,a       –       2,a     10,c       ∞        c
  2   {a c d}      4,a       –         –       8,d      5,d       d
  3   {a c d b}     –        –         –       8,d      5,d       b
  4   {a c d b f}   –        –         –       8,d       –        f
  5   {a c d b f e} –        –         –        –        –        e

RESULTADO (caminos óptimos desde a):
  b : costo 4   a –4– b          e : costo 8   a –2– d –6– e
  c : costo 1   a –1– c          f : costo 5   a –2– d –3– f
  d : costo 2   a –2– d
```

> **Interpretación:** `e` a costo 8 **no** se encontró por la vía directa, sino relaxing `d`. El camino óptimo suele entrar por un nodo que no es el vecino directo.
>
> ⚠️ **Errores típicos: relajar nodos que ya están en `N'`** (rompe la invariante, es el error más caro); **actualizar todo el vector en cada paso** (solo los vecinos del nodo que acaba de entrar); **no escribir el predecesor** (sin él no hay árbol ni tabla de forwarding).

2.2 Ejercicio 2 — Las fases del arranque de OSPF

Enunciado. `R1` (Router ID `10.0.0.1`) y `R2` (`10.0.0.2`) por un enlace punto a punto. `R1` tiene en su LSDB `L1` (Router-LSA suyo), `L2` (Router-LSA de `R2`, viejo) y `L3` (Summary-LSA de `192.0.2.0/24`). `R2` tiene `L1`, `L2` y `L4` (Summary-LSA de `198.51.100.0/24`). Identificar qué mensaje se manda y en qué estado están las vecindades.

```
1. Hello (tipo 1)  a 224.0.0.5, TTL 1  --  R1 se presenta
2. Hello (tipo 1)  a 224.0.0.5, TTL 1  --  R2 contesta con su Router ID
3. DBD   (tipo 2)  unicast a R2         --  R1 describe su LSDB (3 headers)
4. DBD   (tipo 2)  unicast a R1         --  R2 describe el suyo (3 headers)
```

Ambos quedan en `Exchange`, y se cruzan dos DBD de descarte.

```
b) R1 ve que le falta L4:
R1 → R2 :  Link State Request (tipo 3): "necesito el LSA L4"
R2 → R1 :  Link State Update  (tipo 4): "acá va, más lo que te falte"
R1 → R2 :  Link State Acknowledgment (tipo 5): "recibido"

Ambos pasan Exchange → Loading → Full. Como a R1 le falta L4 y
a R2 le falta L3, el ciclo 3-4-5 se repite una vez por cada LSA.

c) SOLO el Hello es periódico: cada 10 s, indefinidamente. Si pasan
40 s sin Hello, el DeadTimer expira y la vecindad cae a Down.
Los otros cuatro son POR EVENTO: en el arranque, y cada 30 minutos.
Si los cinco fueran periódicos cada 10 s, una red con 200 routers y
3 vecindades gastaría ancho de banda todo el día para confirmar
algo que no cambió.

d) Si R2 deja de mandar Hello durante 50 s:
t = 0    llega el último Hello; el DeadTimer arranca en 40 s.
t = 40   expira en R1 → vecindad a Down, desarma el estado y lo
         anuncia por flooding como un Router-LSA con Age = MaxAge.
t = 40..50  R2 todavía no expiró el suyo: SIGUE INSTALANDO RUTAS POR
         R1 Y ENVIANDO PAQUETES. La caída es bilateral pero
         DESALINEADA. Por eso los timers son ajustables: en un
         enlace satelital un DeadInterval de 40 s es inservible.
```

2.3 Ejercicio 3 — Por qué OSPF no necesita split horizon

Enunciado. Explicá por qué *split horizon*, *poison reverse* y *hold-down*, **obligatorios** en distance-vector, **no existen** en OSPF, y con qué mecanismo se reemplazan.

Paso 1 — Qué es el split horizon. En DV, el conteo a infinito viene de que `R2` le cuenta a `R1` que llega a `Internet` a distancia 3 **usando a `R1` mismo como parte del camino**. Split horizon es: "*no le anuncies a* x *una ruta cuyo próximo salto es* x". Es un parche contra el rumor.

Paso 2 — En OSPF el problema ni siquiera puede plantearse.

```
En OSPF, el LSA de R2 dice:  "TENGO UN ENLACE HACIA R3 Y MI COSTO ES 5"

¿Dónde está el camino de R2 a R1?   NO ESTÁ.
¿Dónde está el camino de R2 a Z?    NO ESTÁ.

El LSA es un HECHO LOCAL, no una CONCLUSIÓN. El "llegar a Z con
costo 12" no lo anuncia nadie: lo CALCULA cada router por su
cuenta, localmente, con Dijkstra.

DISTANCE-VECTOR: la ruta es un DATO que viaja. Un router la puede
                  usar como parte de OTRA ruta que no conoce.
                  → rutas circulares → conteo a infinito
LINK-STATE:      los enlaces son DATOS que viajan. Las rutas son
                  CÁLCULOS locales que NUNCA viajan.
                  → no hay nada que pueda reintroducir un rumor
```

Paso 3 — Los controles que reemplazan a los de DV.

Problema de DV	Cómo lo resuelve OSPF
Un router se realimenta con su propio rumor	El rumor **no existe**: los LSAs son hechos locales, las rutas se calculan
Un LSA da vueltas en un anillo	**No reenvío por la interfaz de entrada** + **no reenvío si ya tengo esa instancia**
Un LSA viejo viaja tarde y pisa uno nuevo	**LS Sequence Number**: se compara el número, no el reloj de la red
Un LSA "vuelve" después de que la red se partió	**LS Age + MaxAge**: el origen lo tira explícitamente con un *flush*
Una ruta queda "colgada" al caerse un enlace	El **DeadInterval** detecta la caída en 40 s o menos, y el LSA del enlace desaparece del LSDB de todos

Paso 4 — La comparación final, que es lo que se escribe en el examen.

```
En DV, los controles existen porque el PROBLEMA está en la
información que viaja: es una estimación que puede estar vieja y
puede referirse a un camino que incluye al que la recibe.

En link-state, lo que viaja NO es una estimación sino un hecho. Y
los hechos no se realimentan, porque un hecho no "se copia de
vuelta": A inunda su enlace con B, y B ya tiene ese hecho desde
el momento en que lo leyó en el cable.

Por eso el costo de no necesitar split horizon se paga en otro
lado: memoria (LSDB completo) y CPU (Dijkstra entero).
Es un intercambio: estado por simplicidad de protocolo.
```

> ⚠️ **Errores típicos:** decir "*OSPF no tiene conteo a infinito porque es más rápido*". **La velocidad no tiene nada que ver.** Y no olvidar que **IGRP y EIGRP tampoco usan split horizon**: es una propiedad de la familia de algoritmos, no un accidente. Por último, **split horizon** (regla de contenido: "no le digas a nadie lo que te llega por él") es distinto de **no reenviar por la interfaz de entrada** (regla de topología). OSPF usa las dos, por razones distintas.

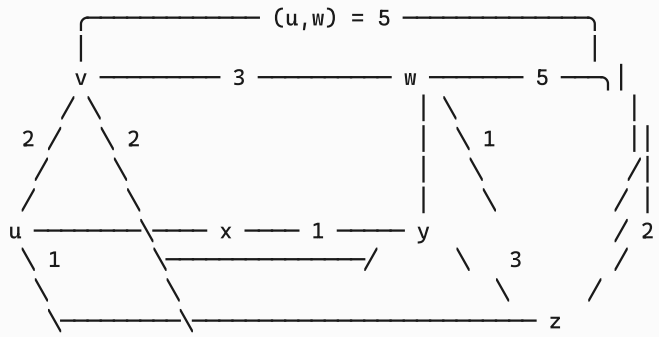
PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2009-02, Problema 2 (30 puntos) — Dijkstra desde `u`

> **Fuente:** Examen `eirc0902` / `sirc0902`, 2 de febrero de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0902.pdf **Texto del examen:**
>
> > El algoritmo de enrutamiento de Dijsktra es link state, donde cada nodo conoce la topología de la red y el costo de cada enlace. Se pide:
> >
> > a) Para la red de la figura, ejecute el algoritmo paso a paso en el nodo u hasta que logra establecer el camino más corto al resto de los nodos. Justifique detalladamente.
> >
> > b) Para el nodo u, de la tabla de forwarding, y dibuje el shortest-path tree resultante de la aplicación del algoritmo de Dijsktra.

La topología — *reconstruida desde las transparencias de la solución oficial*

La figura del enunciado no sobrevivió a la extracción de texto, así que la **reconstruí desde la solución oficial**: el PDF incluye, entre las páginas 9 y 10, dos transparencias del curso (`Int. Redes de Computadoras – Capa de Red`, hojas **4-76, 4-77 y 4-78**) que son la resolución oficial del problema. De ellas sale la red exacta:



Y, para que no quede ninguna ambigüedad, **el listado completo de los diez enlaces**:

Enlace	Costo		Enlace	Costo
`u – v`	2		`w – x`	3
`u – x`	1		`w – y`	1
`u – w`	5		`x – y`	1
`v – w`	3		`w – z`	5
`v – x`	2		`y – z`	2

> ⚠ **Discrepancia que hay que declarar.** La parte b) dice "*de la tabla de forwarding*" (con artículo definido), o sea que la figura debía incluir la tabla de routing de `u`. **Esa parte no se ve en el PDF**: la solución oficial

> solo publica la parte b) resuelta, como árbol y tabla. Si en el examen te dan una tabla de forwarding como dato de entrada, el procedimiento es idéntico: leés los costos y seguís los cinco pasos.

Parte a) — La tabla de iteraciones completa

Paso 0 — Inicialización. `N' = {u}`. Vecinos de `u`: `v` (2), `x` (1), `w` (5). Los demás, `∞`.

Paso	N'	D(v)	D(w)	D(x)	D(y)	D(z)	Entra
0	`u`	2,u	5,u	**1,u**	∞	∞	—

Paso 1 — Entra `x` (`D(x)=1`, el mínimo). Relajo los vecinos de `x` fuera de `N'`: `v`, `w`, `y`.

```
v:  min( 2 , 1 + 2 ) = min(2, 3) = 2  → NO
w:  min( 5 , 1 + 3 ) = min(5, 4) = 4  → SÍ, vía x
y:  min( ∞ , 1 + 1 ) = min(∞, 2) = 2  → SÍ, vía x
z:  x NO es adyacente a z. No se toca.
```

Paso	N'	D(v)	D(w)	D(x)	D(y)	D(z)	Entra
1	`u x`	2,u	4,x	—	**2,x**	∞	`x`

Paso 2 — Entra `y` (`D(y)=2`). Vecinos de `y` fuera de `N'`: `w` y `z`.

```
w:  min( 4 , 2 + 1 ) = min(4, 3) = 3  → SÍ, vía y
z:  min( ∞ , 2 + 2 ) = min(∞, 4) = 4  → SÍ, vía y
```

Paso	N'	D(v)	D(w)	D(x)	D(y)	D(z)	Entra
2	`u x y`	2,u	**3,y**	—	—	4,y	`y`

Paso 3 — Entra `v` (`D(v)=2`). Vecinos de `v` fuera de `N'`: solo `w`. `min(3, 2+3) = 3` → NO mejora.

Paso 4 — Entra `w` (`D(w)=3`). Vecinos de `w` fuera de `N'`: `z`. `min(4, 3+5) = 4` → NO mejora.

Paso 5 — Entra `z`. No quedan nodos fuera de `N'`: fin.

La tabla compacta (la que se copia en el examen)

```
Paso  N'          D(v),p   D(w),p   D(x),p   D(y),p   D(z),p    nodo
──────────────────────────────────────────────────────────────────────
  0   {u}         2,u      5,u      1,u       ∞        ∞         –
  1   {u x}       2,u      4,x       –       2,x       ∞         x
  2   {u x y}     2,u      3,y       –        –       4,y        y
  3   {u x y v}    –       3,y       –        –       4,y        v
  4   {u x y v w}  –        –       –        –       4,y        w
  5   {u x y v w z} –      –       –        –        –          z

Tabla oficial (solución del examen, hoja 4-77):
  Step  N'       D(v)  D(w)  D(x)  D(y)  D(z)
    0   u        2,u   5,u   1,u    ∞     ∞
    1   ux       2,u   4,x   –     2,x   ∞
    2   uxy      2,u   3,y   –     –     4,y
    3   uxyv     –     3,y   –     –     4,y
    4   uxyvw    –     –     –     –     4,y
    5   uxyvwz   –     –     –     –     –

COINCIDEN LAS CINCO ITERACIONES Y LOS CINCO VALORES DE DISTANCIA.
```

Interpretación (lo que hay que escribir después de la tabla):

```
D(v)=2 vía u → u → v            D(w)=3 vía y → u → x → y → w
D(x)=1 vía u → u → x            D(z)=4 vía y → u → x → y → z
D(y)=2 vía x → u → x → y

Fijate en D(w): el enlace DIRECTO u→w cuesta 5, pero llegar por
u→x→y→w cuesta 1+1+1 = 3. El camino más corto NO es el más directo,
y esa es la razón de ser del algoritmo. Un protocolo "más vecino
gana" habría elegido u→w con costo 5.
```

Parte b) — El shortest-path tree y la tabla de forwarding

```
u (raíz)
│
├── v                                  [ D(v)=2, p(v)=u ]
│
└── x                                  [ D(x)=1, p(x)=u ]
     │
     └── y                            [ D(y)=2, p(y)=x ]
          │
          ├── w                       [ D(w)=3, p(w)=y ]
          │
          └── z                       [ D(z)=4, p(z)=y ]

Verificación estructural: 6 nodos y 5 aristas, conexo → es un árbol.
Propiedad que se cumple SIEMPRE en el resultado de Dijkstra y que
sirve para autocontrolar.
```

Destino	Próximo salto	Costo	Interfaz de salida en `u`
`u` (local)	—	0	local
`v`	`v`	2	directo a `v`
`x`	`x`	1	directo a `x`
`y`	`x`	2	por `x`
`w`	`x`	3	por `x`
`z`	`x`	4	por `x`

Coincide con la tabla oficial del PDF (hoja 4-78), que da `destino → enlace` *como* `v → (u,v)`, `x → (u,x)`, `y → (u,x)`, `w → (u,x)`, `z → (u,x)`. Nótese que **la oficial no incluye la columna de costo**: para un link-state interno el costo es un detalle interno del SPF, y lo que importa para el forwarding es **por qué interfaz**. Es un detalle de diseño real: la FIB guarda **próximo salto e interfaz**, no costos.

Análisis de la solución oficial

✅ **Lo que está bien:** (1) la tabla tiene las cinco columnas de distancia más el `N'` acumulado, exactamente el formato que exige "paso a paso" + "justifique detalladamente"; (2) los nodos que entran a `N'` se ven marcados con flechas en la tabla original, y eso es lo que convierte una tabla en una *justificación*; (3) borra la `D` de los nodos ya definitivos, que es la convención del curso.

⚠️ **Lo que la oficial NO publica, y hay que resolver:** (1) **la parte b) está a medias**: el PDF pone el título y después la transparente con el árbol y la tabla, **sin una línea de texto**. La parte b) son **dos** entregables; si entregás solo la tabla, perdés la mitad. (2) **No hay justificación de las cuentas**: la tabla dice `D(w)=4,x` pero no dice por qué, y el enunciado pide "justifique detalladamente". Hay que escribir el `min` de cada relajación: la tabla **más las cinco ecuaciones** sí es la justificación. (3) **No hay verificación**: sumá el chequeo estructural (`|aristas| = |nodos| − 1 = 5`) y el aritmético (`D(z) = 1 + 1 + 2 = 4`).

Mi respuesta completa, en tres bloques:

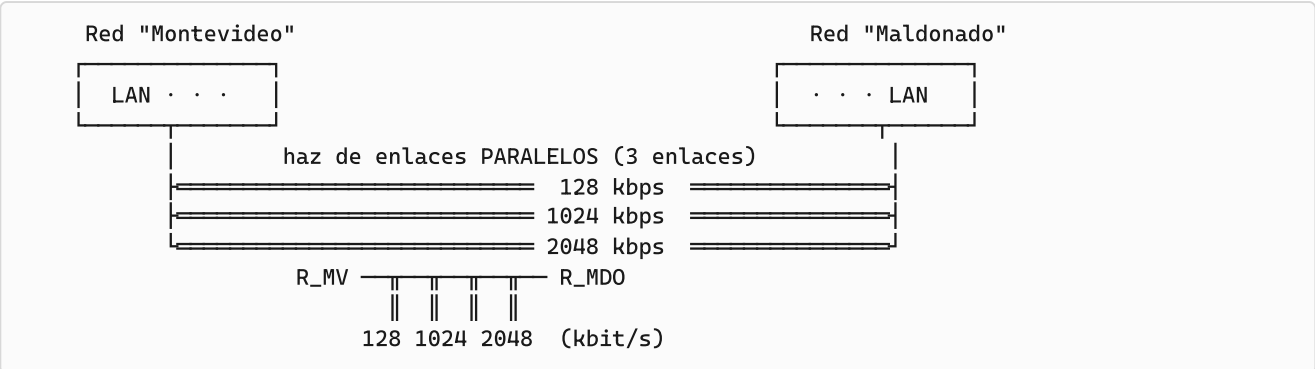
> **a)** Inicializo `N' = {u}`, `D(v)=2`, `D(x)=1`, `D(w)=5`, `D(y)=D(z)=∞`. En cada paso elijo el mínimo entre los nodos **no** definitivos, lo agrego a `N'` y relajo **solo** sus vecinos que están fuera de `N'` con `D(v) ← min(D(v), D(w) + c(w,v))`. Las cinco iteraciones dan `D(v)=2` por `u`, `D(x)=1` por `u`, `D(y)=2` por `x`, `D(w)=3` por `y`, `D(z)=4` por `y`. Verifico que el camino a `w` de costo 3 es más barato que el directo de costo 5.
>
> **b)** El árbol cuelga cada nodo del que lo mejoró: raíz `u`; de `u` cuelgan `v` y `x`; de `x` cuelga `y`; de `y` cuelgan `w` y `z`. La tabla de forwarding traduce el predecesor a interfaz: `v → directo`, `x → directo`, y `y`, `w`, `z` **por** `x`. Notar que cuatro destinos distintos salen por la misma interfaz, que es lo que hace que la FIB sea una tabla de prefijo corto en vez de una tabla por ruta completa.

3.2 ⭐ Examen 2007-03, Problema 2 (30 puntos) — enlaces paralelos y oscilación de métrica

> **Fuente:** Examen `eirc0703` / `sirc0703`, 26 de febrero de 2007. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0703.pdf **Texto del examen:**
>
> > Se desea analizar y mejorar el comportamiento de un algoritmo de enrutamiento link-state en una topología particular como la de la figura, en la que hay dos grupos de redes interconectadas por un haz de enlaces paralelos.
> >
> > a) Describa en términos generales el funcionamiento de un algoritmo de enrutamiento del tipo "link state" (OSPF). b) ¿Como estima que el mismo se comportará en una topología como la de la figura? ¿Que problemas podrían ocurrir en situaciones de alto tráfico y congestión? Estudiar en particular los casos de tráfico mayor a 2048 kbps y entre 1024 y 2048 kbps. c) Proponga y especifique en un lenguaje de alto nivel, un mecanismo para estimar el ancho de banda de los enlaces. Este mecanismo deberá ser lo menos intrusivo posible, por lo que se sugiere aprovechar el envío de LSAs (Link State Advertisements) para ello. Suponga que puede marcar el tiempo al momento de salida/entrada del paquete al enlace, y los routers tienen su tiempo exactamente sincronizado. d) ¿Cómo puede mejorar la situación de la parte (b) el conocer el ancho de banda de los enlaces, para la selección del mejor camino?

La topología — *reconstruida del PDF*

La figura es simple y el enunciado la describe bien: **dos grupos de redes interconectados por un haz de enlaces paralelos**. La del PDF tiene exactamente tres enlaces, de **128, 1024 y 2048 kbps**.



Lo que hay que entender antes de resolver nada: los tres enlaces tienen **el mismo costo por definición de la topología del ejercicio** (son enlaces entre el mismo par de nodos, así que OSPF los trata como tres enlaces más del grafo), pero **capacidades distintas**. Y el tráfico ofrecido de `Montevideo` a `Maldonado` es una sola cifra: si es 1500 kbit/s, tiene que ir **todo** por un enlace, porque Dijkstra devuelve **un solo camino**.

Parte a) — Funcionamiento general de link-state

Solución oficial (literal del PDF): cinco pasos (descubrir vecinos, medir distancias a vecinos, armar el paquete y mandarlo a todos, escuchar la topología y armar el grafo, calcular el camino más corto), más los comentarios:

los pasos 1 y 2 usan **HELLO** que "solamente tienen significado local entre vecinos y **no deben ser enrutados**"; el paso 3 arma un **LSA**; el paso 5 corre **Dijkstra**; y para asegurar que los LSA lleguen a todos se **inundan (flooding)** "con mecanismos para descartar los LSA ya vistos y números de secuencia para garantizar que todos los routers tienen información consistente de la topología". Y una frase final que vale puntos: "*La métrica puede ser cualquier cosa que se pueda medir. Desde número de saltos pasando por ancho de banda y retardo*".

Qué le agregaría, y es lo que suma puntos:

Punto	Por qué suma
La LSDB es idéntica en todos los routers del área	Es la condición de **sincronización**; sin ella el protocolo no sirve
El cálculo de Dijkstra es local, no global	Es lo que hace escalable el esquema y la diferencia con un router centralizado
"Los HELLO no deben ser enrutados" se implementa con `TTL = 1`	Convierte "no deben" en un mecanismo
Las áreas, con el cálculo de O(N) en estado	Justifica por qué existe el OSPF de verdad
La métrica de OSPF es costo de interfaz, derivado del ancho de banda	Conecta directo con las partes b) y c)

Parte b) — El comportamiento con tráfico alto: la oscilación

Solución oficial (literal): "*El algoritmo tal como fue descripto en el teórico corre el algoritmo de Dijkstra simple, que para cada destino va a devolver solamente el camino más corto en cada momento de corrida. Por ello la red en cada momento que se corra el D. va a elegir uno de los enlaces.*" … "*Mientras el tráfico ofrecido es < que 2048 kbps, el enlace elegido es el de 2048 kbps.*" … "*A medida que el tráfico ofrecido se acerca a 2048 la métrica sobre este enlace empeora mientras que la métrica por el de 1024 se mantiene constante (pensar en que si mando HELLOs los mismos tienen que esperar cada vez más en cola para ser transmitidos).*" … "*A medida que nos introduce en congestión más profunda, hay un momento de corte en el cual la métrica por el enlace de 1024 es mejor que por el saturado enlace de 2048 y el enlace elegido pasa a ser el de 1024.*" … "*¿PERO qué pasa? El tráfico ofrecido es mucho mayor que 1024 por lo que este nuevo enlace satura casi inmediatamente y un breve tiempo después el enlace de 2048 vuelve a ser el mejor.*" … "*Este ciclo de oscilaciones se puede repetir varias veces mientras la carga ofrecida no varíe. Esta situación es muy perniciosa para la red y ocasiona importantes pérdidas de paquetes.*"

El mecanismo, gráfico con números. La clave está en esa frase entre paréntesis: **la métrica no es estática, cambia con el tráfico**. Supongamos `costo = ancho_de_banda_referencia / ancho_de_banda_efectiva`, con referencia 2048:

```
Ofrecido: 1500 kbit/s   (el MISMO todo el tiempo)

t = 0   2048 al 80 % → efectivo 1638 kbit/s → costo = 2048/1638 = 1,25
       1024 vacío    → efectivo 1024        → costo = 2048/1024 = 2,00
        128 vacío    → efectivo  128        → costo = 2048/128  = 16,0
       → elige el de 2048 (1,25).  ✓ CORRECTO: cabe 1500 < 1638

t = 1   la cola del 2048 crece:
       2048 al 95 % → efectivo 973 → costo = 2048/973 = 2,10
       1024 al  0 % → costo = 2,00
       ⚠ AHORA el de 1024 es MÁS BARATO (2,00 < 2,10) y Dijkstra
         cambia el camino... aunque 1500 > 1024, o sea que el
         enlace de 1024 NO ALCANZA para todo el tráfico.

t = 2   los 1500 kbit/s llegan al 1024. Se satura en segundos:
       1024 al 100 % → efectivo 350 → costo = 2048/350 = 5,85
       → el de 2048 (que sigue al 95 %) vuelve a ser el más barato.

t = 3   el 2048 también satura → costo = 16,0 ; el 1024 sigue en
       5,85 → ahora gana el 1024 otra vez. Y vuelta a empezar.

RESUMEN DE LA OSCILACIÓN, en una línea de tiempo:

  t0 ── el 2048 gana ──┬── t1 el 1024 "gana" ──┬── t2 el 2048
                       │                       │
  (carga 1500, que     └── t3 el 1024 ─────────┘   ... y vuelta
   NUNCA entra entero
   en el de 1024)
```

El nombre del problema: metric-driven oscillation, u oscilación de métrica; en la literatura de operación de redes, **"flapping"**. Es un caso particular de **inestabilidad de la decisión de enrutamiento**: ocurre cuando la realimentación de la métrica es más rápida que la constante de tiempo del tráfico.

Por qué es tan dañina: (1) **descarte**, al pasar 1500 kbit/s de golpe a un enlace de 1024 los 478 kbit/s que no entran se pierden; (2) **latencia**, porque durante la oscilación cada paquete va por un camino distinto y los retardos se disparan; (3) **sesiones TCP**, que cambian de camino a mitad de vida, con lo que eso implica para la retransmisión y el RTO; (4) **CPU**, porque cada cambio recalcula SPF y genera flooded, y con oscilación el flooded es continuo.

Y el caso particular que pide el enunciado, que es un detalle fino y vale puntos. Pide estudiar por separado "*tráfico mayor a 2048 kbps*" y "*entre 1024 y 2048 kbps*" porque **el comportamiento cualitativo cambia**:

Rango de tráfico ofrecido	Qué ocurre
< 1024 kbit/s	**Zona segura.** El enlace de 2048 alcanza siempre, nunca se satura, la métrica es estable.
1024 ≤ X < 2048 kbit/s	**Zona de oscilación pura, el caso peor.** Ningún enlace alcanza para todo el tráfico: 2048 alcanza y 1024 no. El 2048 se congestiona un poco y el 1024 "parece" mejor; al mover el tráfico, el 1024 se satura y vuelve el 2048. Oscilación de período corto.
X ≥ 2048 kbit/s	**Zona de saturación.** Los tres enlaces al 100 %. La métrica de los tres es mala, pero **el tráfico no cabe en ninguno**. La oscilación puede cesar (todo está igual de malo), pero **la red está colapsada**. El problema dejó de ser de enrutamiento y pasó a ser de **capacidad**.

Conclusión que hay que escribir: para tráfico entre 1024 y 2048 el problema **es de enrutamiento** (se arregla eligiendo bien el enlace); para tráfico mayor a 2048 es **de dimensionamiento** (no lo arregla ningún algoritmo). El enunciario pide que distingas las dos cosas porque **son soluciones distintas**.

Parte c) — Medir el ancho de banda aprovechando los LSA

Solución oficial (literal del PDF), cuatro pasos:

> 1. Armar los LSAs de un tamaño conocido y fijo, eventualmente con padding (relleno) para hacerlos de un largo no muy corto ni demasiado largo, por ejemplo, 1024 bytes. Esto es para eliminar efectos de borde de paquetes muy cortos o muy largos.
> 2. Enviar el LSA marcando el timestamp de salida, posiblemente en un campo de opciones adicionales.
> 3. El router que recibe el LSA con estas características determina el ancho de banda realizando una simple operación: `Bw = L / (Ts2 - Ts1)`, donde `Ts1` y `Ts2` son los timestamps (salida y llegada) y `L` es el largo del LSA.
>
> **Observación 1:** *La hipótesis de sincronización de todos los routers es fundamental, sin ella este proceso no es posible y hay que recurrir a otro tipo de técnicas mucho más complejas.* **Observación 2:** *La medida debería repetirse en varias oportunidades y promediar, a los efectos de eliminar efectos interferentes del resto del tráfico.*

Análisis de la oficial. Los cuatro pasos son correctos y la fórmula es la correcta. Pero se queda en medir el **tiempo de tránsito**, que es un *throughput* instantáneo, no un **ancho de banda** nominal:

```
⚠ LO QUE LA FÓRMULA MIDE ES throughput, NO ancho de banda.

Bw_aparente =  L / (T_salida - T_llegada)

 El tiempo (Ts2 - Ts1) incluye:
     · el tiempo de TRANSMISIÓN   (L / capacidad)   ← lo que querés medir
     · el tiempo de ESPERA EN COLA                   ← lo que NO querés
     · el tiempo de PROPAGACIÓN                     ← es constante, no importa

 En un enlace libre:   T ≈ L/cap + d_prop        → Bw medido ≈ capacidad  ✓
 En un enlace cargado: T ≈ L/cap + d_cola + d_prop → Bw medido MUCHO MENOR ✗

 O sea: con un enlace saturado, la fórmula mide menos ancho de banda
 del que tiene. ¿Y eso está mal? NO: para el problema del examen
 ESTO ES LO QUE QUEREMOS. Un enlace que rinde poco porque tiene
 cola es un enlace que no sirve para transportar tráfico.
```

Mi implementación en pseudocode C, con las dos observaciones implementadas:

```
/* Medición intrusiva del ancho de banda efectiva de un enlace,
   reutilizando el LSA de OSPF como paquete de prueba.
   La salida se marca en el router ORIGEN (una sola marca) y la
   llegada en el RECEPTOR. Nunca dos marcas.                      */

#define TAMANO_LSA     1024     /* bytes, tamaño fijo conocido        */
#define MAX_MUESTRAS   8        /* para promediar                      */
#define MAX_ESPERA     5        /* EPR de Kolmogorov: trunca el último */
                                  /* 5 % de las muestras (las del buffer */
                                  /* lleno, que son las que interesa)   */

typedef struct {
    int64 t_salida;    /* reloj del origen al encolar el LSA          */
    int64 t_llegada;   /* reloj del receptor al leer el último byte   */
    int   longitud;    /* bytes, siempre TAMANO_LSA                   */
} muestra_t;

/* La LSA es un paquete normal de OSPF: le metemos un TLV opcional.
   RFC 2328 admite options en el header del LSA.                      */

typedef struct {
    int    tipo_lsa;
    int    router_id;
    uint32 seq;           /* marca única, para no confundir con refresco */
    int64  marca_temporal;/* timestamp del router ORIGEN (microsegundos) */
    int    relleno;       /* padding hasta TAMANO_LSA, con patrón       */
                          /* conocido para verificar que no se truncó   */
} lsa_medicion_t;

/* ---- en el router ORIGEN, al emitir un LSA periódicamente ---- */

lsa_medicion_t construir_lsa_medicion(void)
{
    lsa_medicion_t m;
    m.tipo_lsa       = 1;          /* Router-LSA: el que ya inunda    */
                                   /* cada 30 min y en cada cambio    */
    m.router_id      = mi_router_id;
    m.seq            = ++contador_seq;
    m.marca_temporal = AHORA();    /* EL PUNTO DE LA MEDICIÓN         */
    for (int i = 0; i < TAMANO_LSA - sizeof(m); i++)
        ((uint8_t *)&m)[sizeof(m) + i] = (uint8_t)(i & 0xFF);
    return m;
}

/* ---- en el router RECEPTOR, al recibir un LSA con TLV ---- */

void procesar_lsa_medicion(lsa_medicion_t *m)
{
    /* 1. ¿Es una muestra válida? -- descartamos refrescos y corruptos */
    if (!tiene_tlv_medicion(m))     return;  /* LSA normal              */
    if (m->relleno_incorrecto())    return;  /* truncado o mal rellenado*/
    if (edad(m) > MAX_AGE)          return;  /* llegó tarde: el reloj    */
                                          /* ya no dice nada          */
    /* 2. Guardar la muestra */
    buffer[ptr].t_salida  = m->marca_temporal;
    buffer[ptr].t_llegada = AHORA();
    buffer[ptr].longitud  = TAMANO_LSA;
    ptr = (ptr + 1) % MAX_MUESTRAS;
    /* 3. Recalcular si ya hay muestras */
    if (ptr != 0) actualizar_metrica_del_enlace();
}

/* ---- estimación ------------------------------------------------ */
```

```
void actualizar_metrica_del_enlace(void)
{
    /* 3a. Throughput instantáneo de cada muestra */
    int64 tasa[MAX_MUESTRAS];
    for (int i = 0; i < MAX_MUESTRAS; i++) {
        double dt = (buffer[i].t_llegada - buffer[i].t_salida) / 1e6;
        if (dt <= 0.0) return;   /* relojes incoherentes: el sincronismo
                                    se rompió. NO promediar.            */
        tasa[i] = (int64)(buffer[i].longitud * 8 / dt);   /* bits/s     */
    }

    /* 3b. Truncar la cola superior. Las muestras más altas son
           las de mayor tasa medida, y son las que menos nos
           cuando el buffer está vacío el enlace rinde su capacidad
           real, que ya la conocemos por configuración.                */
    ordenar_descendente(tasa, MAX_MUESTRAS);
    int utiles = MAX_MUESTRAS * (100 - MAX_ESPERA) / 100;
    if (utiles < 1) utiles = 1;

    /* 3c. Promediar el resto (el enunciado dice "promediar") */
    int64 suma = 0;
    for (int i = 0; i < utiles; i++) suma += tasa[i];
    capacidad_estimada = suma / utiles;

    /* 3d. Nunca por encima de la capacidad física */
    if (capacidad_estimada > capacidad_configurada)
        capacidad_estimada = capacidad_configurada;

    /* 3e. Confianza. Si las muestras son dispares, la estimación es
           mala y NO hay que cambiar la métrica: una muestra ruidosa
           provocaría una oscilación, que es EXACTAMENTE el problema
           que estamos tratando de arreglar.                          */
    varianza = desviacion(tasa, utiles, capacidad_estimada);
    confianza = (varianza < UMBRAL_VARIANZA) ? ALTA : BAJA;

    /* 3f. Solo se publica si hay confianza y si el cambio supera un
           margen de HISTÉRESIS, que es el antídoto contra la
           oscilación.                                                */
    if (confianza == BAJA) return;
    if (cambio_relativo(capacidad_estimada, costo_actual()) < MARGEN) return;
    costo_nuevo = COSTO_REFERENCIA * 1000 / capacidad_estimada;
    publicar_lsa_con_nuevo_costo(costo_nuevo);
}
```

La respuesta a "Observación 1" — la sincronización de relojes — merece desarrollo propio:

```
El enunciado dice "los routers tienen su tiempo exactamente
sincronizado". Eso es una HIPÓTESIS, y hay que decir qué se rompe
si es falsa:

· Si el reloj del receptor va ADELANTADO, (T2 − T1) es MENOR que la
  realidad y la tasa medida sale MÁS ALTA de lo que es.
· Si va ATRASADO, sale MÁS BAJA.
· Y como el reloj deriva, la deriva se acumula: un offset fijo de
  500 µs sobre un tiempo de tránsito de 300 µs es un error del 160 %.

Sin sincronización NO se puede medir así. Las alternativas son:
  · Ping-pong de challenge-response con el eco del timestamp: el
    receptor devuelve el timestamp recibido y se mide el tiempo de
    ida y vuelta; el error de offset se cancela. Es lo que hace
    cualquier estimación de RTT, incluido Jacobson/Karels en TCP
    (Clase 19). Es la respuesta correcta si el enunciado retira la
    hipótesis de sincronización.
  · NTP alcanza para el protocolo, pero no da precisión de
    microsegundos; no sirve para esto.
  · TWAMP es la herramienta de la UIT-T para medir ancho de banda
    sin carga intrusiva. Es el estándar industrial y no es un RFC
    de IETF: nombralo como "la forma industrial de hacer esto".
```

La respuesta a "Observación 2" — promediar — y por qué promediar NO alcanza:

Promediar reduce el ruido, pero **no arregla el sesgo sistemático** del buffer lleno: si el enlace está congestionado la mitad del tiempo, el promedio da un número que no describe ni el estado bueno ni el malo. Por eso la implementación agrega dos cosas que la oficial no pide pero que **son las que hacen que el método funcione**:

1. **Truncamiento de la cola superior** (`MAX_ESPERA`), no promedio simple: se descartan las muestras buenas y se promedian las malas, que son las que describen la contención.
2. **Histéresis**: la métrica solo se publica si el cambio supera un margen. **Sin histéresis, la solución a b) reproduce b).** Es el error clásico de realimentación positiva, y es el mismo error conceptual del AQM tardío en TCP (Clase 21): publicar la métrica de inmediato cuando cambia es lo que genera la oscilación.

Parte d) — Cómo mejora conocer el ancho de banda

Solución oficial (literal): "*En el caso más elemental, conocer el ancho de banda me permitiría simplemente evitar oscilaciones porque puedo saber a-priori que el nuevo enlace no va a soportar la carga. Conocido el ancho de banda en este escenario es posible utilizar versiones refinadas del algoritmo de Dijkstra que no solo determinan el mejor camino sino eventualmente también caminos alternativos.*"

```
SIN conocer el ancho de banda: la métrica es una estimación
ruidosa y el enrutamiento reacciona al ruido. Resultado:
oscilación 2048 ⇔ 1024.

CONOCIDO el ancho de banda (2048 / 1024 / 128 kbps), tres usos:

(1) MÉTRICA FIJA POR CAPACIDAD → elimina la oscilación
     costo = 2048 / capacidad
     2048 kbps → 1 ; 1024 kbps → 2 ; 128 kbps → 16
     Con oferta de 1500 kbps, gana siempre el de 2048, porque los
     otros NUNCA pueden con ese tráfico. Fin de la oscilación.

(2) MÉTRICA DE ANCHO DE BANDA RESIDUAL → reparto de carga
     bw_residual = capacidad − tráfico_ofrecido_por_este_enlace
     Con dos caminos de 2048: uno lleva 1000 y otro 500.
     bw_residual 1048 vs 1548 → el tráfico va al segundo.
     Esto es ECMP o el "link-state con ancho de banda", y es lo que
     se conoce como traffic engineering.

(3) DIJKSTRA MODIFICADO CON SEGUNDOS CAMINOS → redundancia
     En vez de UN camino por destino, se calculan los K mejores
     caminos disjuntos. Si el primario se cae, entra el segundo sin
     recalcular nada, y si el primario se degrada (y no cae), se
     reparte entre ambos. Es lo que el enunciario llama
     "versiones refinadas del algoritmo de Dijkstra".

El precio de las tres: (1) y (2) necesitan que la capacidad sea
conocida y estable; (3) multiplica por K el trabajo del SPF y
necesita memoria para K tablas. Y ninguna mide la CONGESTIÓN real:
por eso el esquema real es "capacidad conocida + ajuste fino por
cola", que es lo que hacen las Extensiones de OSPF para
Link-Bandwidth y las métricas ajustables de los fabricantes.
```

3.3 Examen 2009-12, Pregunta 3 (8 puntos) — el cometido de la capa de red y si "hops" es buena métrica

> **Fuente:** Examen `eirc0912` / `sirc0912`, 18 de diciembre de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0912.pdf **Texto del examen:**
>
> > a) ¿Cuál es el principal cometido de la capa de red? b) La longitud en "hops" es la métrica utilizada habitualmente para definir el costo de un enlace. ¿Es la más conveniente? Justifique su respuesta.

Solución oficial (literal del PDF):

> a) Asegurar la entrega de información de extremo a extremo. b) Si se busca una utilización eficiente de los recursos y/o satisfacer requerimientos de calidad de servicio, la métrica de hops puede ser inconveniente, ya que señalará como "mejor" el camino más corto, que no necesariamente será el que tenga mejores atributos de calidad (retardo extremo a extremo, pérdidas, throughput) y/o menos congestión.

Análisis de la solución oficial

La parte a) es corta pero incompleta, y hay que saber por qué. "*Asegurar la entrega de información de extremo a extremo*" es el cometido general de la capa de red, y es correcto como respuesta de 2-3 puntos. Pero la palabra **"asegurar"** es engañosa: en el curso *reliable* significa "que no se pierde nada", y eso **la capa de red NO lo garantiza**. Lo que sí garantiza es el **encaminamiento** a través de una internet de redes heterogéneas, con **decisión local** en cada nodo y sin necesidad de conocer la topología global. El servicio de IP es **best effort** (Clase 21 §1.6): lo que aporta la capa de red es direccionamiento jerárquico, *forwarding* por *longest prefix match* y enrutamiento.

Parte b) — El ejemplo con números

La oficial es correcta pero abstracta (nombra "retardo, pérdidas, throughput" sin ninguno). El enunciado pide **"justifique"**, y una justificación sin números es una afirmación.

```
CAMINO A:  10 saltos  ·  cada enlace de 10 Mbit/s  ·  1 ms de propagación
CAMINO B:   1 salto   ·  ese enlace de 1 Gbit/s    ·  0,5 ms de propagación

┌──────────────────────────────────────────────────────────────┐
│                    camino A      camino B                  │
│  hops                   10            1                    │
│  cuello de botella  10 Mbit/s   1000 Mbit/s  ← el más estrecho│
│  ¿cuál elige "hops"?        A  ✗  (1 < 10)                   │
│  ¿qué ancho de banda usable? 10 Mb/s       1000 Mb/s         │
│  ¿cuál es 100 veces mejor?  ──────────→ B                    │
└──────────────────────────────────────────────────────────────┘
```

El cálculo completo. Paquete de 1500 bytes = 12 000 bits. `d_proc = 0`.

```
── CAMINO A (10 saltos, 10 Mbit/s, 1 ms de propagación c/u) ──

d_trans por enlace = L / R = 12 000 / (10 × 10⁶) = 1,2 × 10⁻³ s = 1,2 ms

10 enlaces  →  10 × 1,2 ms  =  12,0 ms   (transmisión total)
10 enlaces  →  10 × 1 ms    =  10,0 ms   (propagación total)
procesamiento: 10 × 20 µs    =   0,2 ms
cola: 0 (sin congestión)
                               ─────────
RETARDO TOTAL camino A         =  22,2 ms

── CAMINO B (1 salto, 1 Gbit/s, 0,5 ms de propagación) ──

d_trans = 12 000 / 10⁹ = 1,2 × 10⁻⁵ s = 0,012 ms = 12 µs

1 enlace   →  1 × 0,012 ms =  0,012 ms   (transmisión)
1 enlace   →  1 × 0,5 ms   =  0,5 ms      (propagación)
procesamiento: 1 × 20 µs    =  0,02 ms
                               ────────
RETARDO TOTAL camino B         =   0,53 ms

Retardo:     22,2 / 0,53  =  42 veces mejor el camino B
Throughput:  1000 / 10    = 100 veces más ancho el camino B
```

Y para rematar el argumento de la oficial — pérdidas y throughput medido, no teórico:

```
El ancho de banda TEÓRICO de un camino no es su ancho de banda REAL.
El real es el del cuello de botella, reducido por slow start y por el
control de congestión de TCP:

camino A: 10 Mbit/s × 0,7 = 7 Mbit/s      camino B: 1 Gbit/s × 0,7 = 700 Mbit/s

Para una descarga de 100 Mbit/s:
camino A: NO PUEDE. Va a 7 Mbit/s, o sea el 7,3 % de lo pedido.
camino B: puede, y le sobra.

Y las pérdidas: el camino A tiene 10 enlaces, y la probabilidad de
que UNO de ellos pierda un paquete en una ventana es 10 veces mayor
que en el camino B. La probabilidad de que una sesión TCP sufra una
pérdida es proporcional al número de saltos.
```

La respuesta completa, como se escribe en el examen:

> **a)** El principal cometido de la capa de red es **encaminar los paquetes de un sistema final a otro a través de una internet de redes heterogéneas**, tomando una **decisión local** en cada nodo. La consequence es que **no necesita conocer la topología global** (de ahí los algoritmos escalables link-state y path-vector), y que **no garantiza fiabilidad**: el servicio de IP es *best effort*. Lo que aporta es direccionamiento jerárquico, *forwarding* por *longest prefix match* y enrutamiento por intercambio de información de control. La frase "asegurar la entrega" es engañosa: lo que se asegura es que el paquete llegue *a destino*, no que llegue *completo y en orden*.
>
> **b)** No, "hops" **no es la métrica más conveniente**, por tres razones concretas. **(1) No correlaciona con el retardo**: la transmisión es `L/R` y la propagación es `d/s`, y ninguno depende de la cantidad de saltos. **(2) No correlaciona con el ancho de banda**: el camino más corto en hops puede tener un cuello de botella de 10 Mbit/s frente a otro de 1 Gbit/s, como en el ejemplo numérico. **(3) No correlaciona con la pérdida**: más saltos son más puntos de falla y más transiciones de capa 2 que rehacer. Lo único con lo que "hops" correlaciona es el **retardo de procesamiento**, del orden de los microsegundos, **diez mil veces menor** que las diferencias de ancho de banda y propagación, y por lo tanto irrelevante. Lo que corresponde es una métrica compuesta (el **costo de interfaz** de OSPF, o ancho de banda más retardo en EIGRP) que refleje lo que el usuario necesita, y en general **configurable por el administrador**.

3.4 Examen 2007-12, Pregunta 1 (5 puntos) — los cinco mensajes de OSPF

> **Fuente:** Examen `eirc0712` / `sirc0712`, 27 de diciembre de 2007. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0712.pdf **Texto del examen:**
>
> > Explique resumidamente los 5 mensajes que se implementa en OSPF.
>
> ⚠ **Nota sobre la solución oficial:** el PDF de `sirc0712` publica la solución de los **Problemas prácticos** pero **no** la de las Preguntas teóricas. Lo que sigue es *(solución propia, contrastada con `RFC 2328`)*. La tabla de §1.7 es la respuesta y no hay que volver a inventarla.

#	Mensaje	Código	Para qué, en una línea
1	**Hello**	1	Descubre y mantiene vivos a los vecinos (cada 10 s, TTL 1, a `224.0.0.5` / `224.0.0.6`)
2	**Database Description**	2	En el arranque, cada uno **describe** lo que tiene en su LSDB para que el otro diga qué le falta
3	**Link State Request**	3	**Pide** los LSAs concretos que le faltan
4	**Link State Update**	4	**Manda** los LSAs pedidos o los nuevos por flooding
5	**Link State Acknowledgment**	5	**Confirma** la recepción

El razonamiento que hay que agregar, que es lo que diferencia un 4 de un 5:

1. **El Hello es el único periódico.** Los otros cuatro son del arranque y del refresco de 30 min. El Hello es el único que **no lleva información de la topología**: su contenido útil es "estoy vivo, mi Router ID es X, mi DeadInterval es Y".
2. **El Hello tiene TTL 1 y va a multicast**, los otros cuatro tienen TTL 1 y van a **unicast** al vecino concreto: el Hello tiene sentido entre todos los routers de la red, los otros solo entre dos.
3. **Los tres últimos forman un lazo de flooded confiable** (3 → 4 → 5) que existe **para no inundar a ciegas**: el `LS Update` sin acuse se reenvía a los 5 s, y el `LS Request` es la excepción que reemplaza al flooding masivo.
4. **El 2 es un "índice" y el 4 es el "contenido"**: el Database Description **no lleva las LSAs**, solo sus **headers** (LS ID, tipo, originator, número de secuencia, longitud). Por eso se llama *Description*. El que las lleva enteras es el LS Update.
5. **El Hello cumple además tres funciones extra** que se preguntan: negociar los timers, **elegir el DR/BDR** (comparando prioridades y Router IDs) y detectar al vecino caído (`DeadInterval` = 40 s = 4 × `HelloInterval`).

La respuesta de 5 puntos, tal cual la escribirías:

> Los cinco mensajes de OSPF son: **Hello**, que se envía cada 10 segundos con TTL 1 a los multicast `224.0.0.5` y `224.0.0.6` y sirve para descubrir y mantener vivos a los vecinos, negociar los intervalos y elegir el DR/BDR, y cuya ausencia por 40 segundos declara la vecindad caída; **Database Description**, que en el arranque describe el índice del LSDB propio (los headers de sus LSAs, no las LSAs) para que el vecino determine qué le falta; **Link State Request**, con el que el vecino pide explícitamente las LSAs que le faltan; **Link State Update**, que es el único que transporta LSAs completas y sirve tanto para responder al request como para el flooding de cambios; y **Link State Acknowledgment**, que confirma la recepción de un Update. Los tres últimos forman el mecanismo de **flooding confiable**: sin acuse de recibo, el Update se reenvía a los 5 segundos.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Link-state = cada router conoce la topología completa del área y corre el mismo algoritmo.** Distance-vector = cada router solo conoce lo que le dijeron sus vecinos.
2. La **LSA** describe **un enlace** con su costo; es un **hecho local**, no una estimación de ruta. El **LSDB** es idéntico en todos los routers del área, y por eso todos calculan el mismo árbol.
3. El **flooding** lleva cada LSA a todos. Se corta con tres controles: **no reenviar por la interfaz de entrada**, **no reenviar si ya tengo esa instancia**, y comparar por **número de secuencia**.
4. La **edad (MaxAge = 3600 s)** es el reloj que expira los LSAs; el **número de secuencia** con signo de 32 bits es el orden total que decide cuál es "nuevo".
5. **Dijkstra** mantiene la invariante "los nodos en `N'` tienen su distancia óptima y no va a cambiar", y la mantiene porque solo relaja vecinos **fuera** de `N'`. Por eso exige costos **no negativos**.
6. La tabla de iteraciones tiene **tres bloques**: `N'` (definitivos), `D(n), p` (distancia y predecesor) y el **nodo que entra en cada paso**. Sin ella no hay puntos.
7. **Complejidad**: Bellman-Ford es $O(V^3)$ con matriz; Dijkstra es $O(V^2)$ con matriz y $O(E \cdot \log V)$ con cola de prioridad. El **estado** es $O(N)$ por router y el flooding es $O(N)$ por evento: por eso existen las **áreas**.
8. Los **cinco mensajes** de OSPF son Hello (10 s, TTL 1, multicast), DBD (índice del LSDB), LSR (pedir), LSU (mandar/inundar) y LSAck (confirmar).
9. Las **áreas** segmentan el grafo; los **LSA de tipo 3** resumen otra área y los de **tipo 4** dicen cómo llegar al ASBR. El **DR/BDR** convierte $N(N-1)/2$ vecindades en $2N - 3$.
10. **"Hops" no es una buena métrica**: 10 saltos de 10 Mbit/s dan 22,2 ms y 1 salto de 1 Gbit/s da 0,53 ms. OSPF usa **costo de interfaz** derivado del ancho de banda.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Cuál es la diferencia estructural entre un LSA y un vector de distancias?

<details>
<summary>Respuesta</summary>
</details>

2. ¿Por qué Dijkstra necesita costos no negativos y Bellman-Ford no?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué la línea 12 del algoritmo dice "para todos los `v` adyacentes a `w` **y no en** `N'`"? ¿Qué se rompe si se saca esa condición?

<details>
<summary>Respuesta</summary>
</details>

4. Nombres los tres controles que impiden que el flooding de un LSA se.loop infinitamente, y qué pasaría si sacara cada uno.

▶ Respuesta

5. ¿Por qué la elección del DR/BDR es necesaria en un medio multiacceso y no en punto a punto?

▶ Respuesta

6. Explique por qué "hops" es una mala métrica, con las cuatro componentes del retardo.

▶ Respuesta

7. En el `2007-03`, ¿por qué medir el ancho de banda con timestamps en los LSA **no sirve** por sí solo contra la oscilación, y qué falta agregarle?

▶ Respuesta

8. OSPF no usa split horizon. ¿Por qué puede prescindir de él, y qué paga a cambio?

▶ Respuesta

🔗 Referencias

- **Examen `eirc0902` / `sirc0902`** (2-feb-2009) — Problema 2 completo (30 pts). La figura del enunciado no está en la extracción de texto; la red se **reconstruyó desde las transparentes de la solución oficial** (hojas 4-76, 4-77 y 4-78 del PDF), que incluyen la tabla de iteraciones.
- **Examen `eirc0703` / `sirc0703`** (26-feb-2007) — Problema 2 completo (30 pts): enlaces paralelos de 128/1024/2048 kbps, oscilación de métrica, medición de ancho de banda con LSA y timestamps.
- **Examen `eirc0912` / `sirc0912`** (18-dic-2009) — Pregunta 3 (8 pts): el cometido de la capa de red y si "hops" es la métrica más conveniente.
- **Examen `eirc0712` / `sirc0712`** (27-dic-2007) — Pregunta 1 (5 pts): los cinco mensajes de OSPF. ⚠ El PDF **no publica la solución teórica**; la resolución de §3.4 es *(solución propia, contrastada con RFC 2328)*.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.6 "Algorithms for routing": §5.6.3 el algoritmo de Dijkstra y el shortest-path tree, §5.6.4 OSPF.
- **`RFC 2328`** — OSPF, *Internet Standard* (1998): los cinco mensajes, los tipos de LSA, las áreas, la elección de DR/BDR, el flooding fiable.
- **`RFC 5340`** — OSPFv3 (2009), para IPv6 · **`RFC 5308`** — extensiones de IS-IS para IPv6.
- **`RFC 8402`** — arquitectura de Segment Routing (2018) · **`RFC 8660`** — SR-MPLS · **`RFC 8986`** — SRv6.
- **`RFC 9552`** — BGP-LS (dic-2023), obsoleta `RFC 7752` · **`RFC 7432`** — EVPN · **`RFC 7348`** — VXLAN.
- **`RFC 5797`** — TWAMP: medir ancho de banda sin carga intrusiva.
- **Clase anterior:** 32 — Enrutamiento distance-vector: Bellman-Ford, conteo a infinito y RIP
- **Clase siguiente:** 34 — BGP y el enrutamiento entre operadores

Clase 34

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El problema: Internet no es una red de redes con un grafo, sino con una política
1.2 El Sistema Autónomo y el ASN
1.3 BGP-4 y la sesión entre vecinos
1.4 Path-vector: la diferencia clave con distance-vector
1.5 El handshake: OPEN → OPEN/ACK → UPDATE → KEEPALIVE
1.6 Los atributos de decisión y la tabla de desempates
1.7 eBGP, iBGP, full-mesh y Route Reflection
1.8 "Hot potato": por qué un AS prefiere salir por el vecino más cercano
1.9 Por qué BGP no cuenta a infinito ni necesita split horizon
1.10 Seguridad: por qué BGP no es seguro
1.11 La tabla resumen: link-state, distance-vector y path-vector
2.1 Ejercicio 1 — La tabla de desempates aplicada a cuatro rutas
2.2 Ejercicio 2 — Topología de tres AS, paso a paso
2.3 Ejercicio 3 — Por qué BGP necesita full-mesh iBGP y qué rompe Route Reflection
3.1 ⭐ Examen 2001-08, Problema 2 — Implementar un algoritmo de ruteo jerárquico
3.2 Examen 2007-02, Problema 2 — El enrutamiento hacia Internet con ruta por defecto
3.3 ⭐ Examen 2009-03, Pregunta 2 (10 puntos) — jerarquía, IGP con jerarquías y *hot-potato*
3.4 Nota: qué hay de BGP en el corpus de exámenes
📌 Resumen en 10 líneas
✅ Autoevaluación
🔗 Referencias

Bloque: D (Capa de red) · **Semana:** 12 · **Duración estimada:** 115 min **Correlativa:** Clase 33 (link-state, Dijkstra y OSPF) · **Sigue con:** Clase 35 (fragmentación, MTU y path MTU) **Material de referencia:** Tanenbaum cap. 5 (§5.6.4 BGP, "A Case Study in Internet Routing") · RFC 4271, RFC 4456, RFC 5396, RFC 6793, RFC 1997, RFC 2385, RFC 9234, RFC 6810, RFC 6811

🎯 Al terminar esta clase vas a poder

1. Explicar por qué Internet **no es un grafo único** y por qué en el nivel inter-AS hace falta **política** y no solo un camino más corto.
2. Definir un **Sistema Autónomo (AS)** y un **ASN**, explicar cómo se asignan y por qué hoy son **32 bits**.
3. Describir **BGP-4**: la sesión entre vecinos, el **handshake** completo y la máquina de estados.
4. Explicar por qué BGP es **path-vector** y no distance-vector, y por qué la detección de loops con `AS_PATH` es **exacta** y no probabilística.
5. Recitar la **tabla de desempates** de atributos de decisión en el orden correcto y aplicarla a un caso con cuatro rutas candidatas.
6. Distinguir **eBGP** de **iBGP**, explicar la regla del **full-mesh iBGP** y qué problema resuelve **Route Reflection**.
7. Explicar el **"hot potato"** con números, y cómo lo corrige el atributo **MED**.
8. Describir la **seguridad de BGP**: por qué TCP no alcanza, qué son **RPKI/ROA**, qué son los **roles BGP** y por qué los **route leaks** son el fallo más común.

> **Prerequisito del examen:** `2009-03 Pregunta 2` (10 pts — enrutamiento jerárquico, IGP con jerarquías y **hot-potato routing**), `2001-08 Problema 2` (30 pts — enrutamiento jerárquico y tablas de ruteo), `eirc0702 Problema 2` (apartado e — enrutamiento hacia Internet con rutas por defecto), `2007-03`

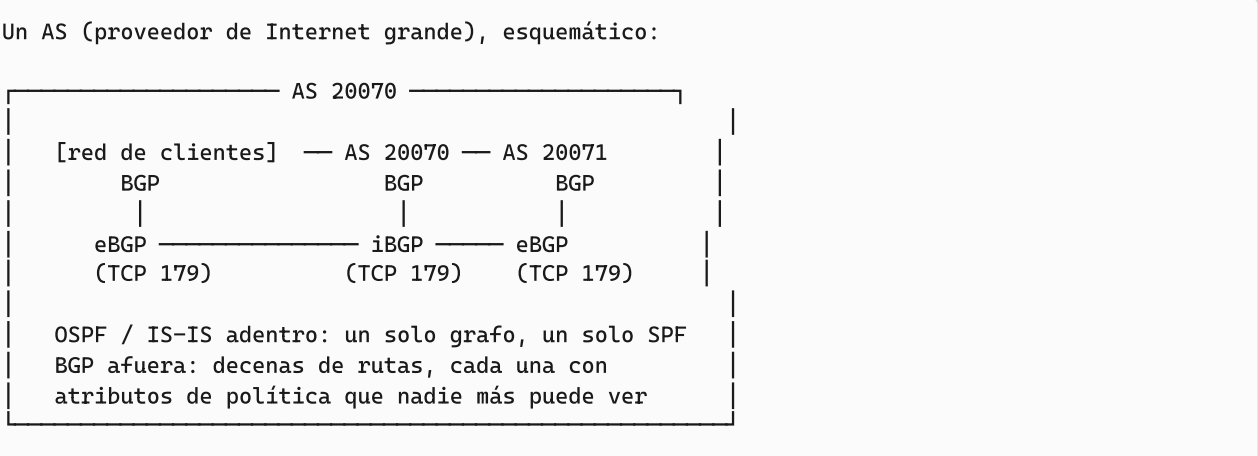
> `Problema 2a` (IGP con jerarquías y la frontera interior/exterior), `2006-08 P3` (distance-vector: el contraste con BGP). Todos se resuelven en §3.

PARTE 1 — TEORÍA

1.1 El problema: Internet no es una red de redes con un grafo, sino con una política

La clase anterior terminó con un OSPF corriendo Dijkstra sobre **un** grafo. Todo estaba bien porque había **un solo dominio administrativo**: un área, un LSDB, un árbol.

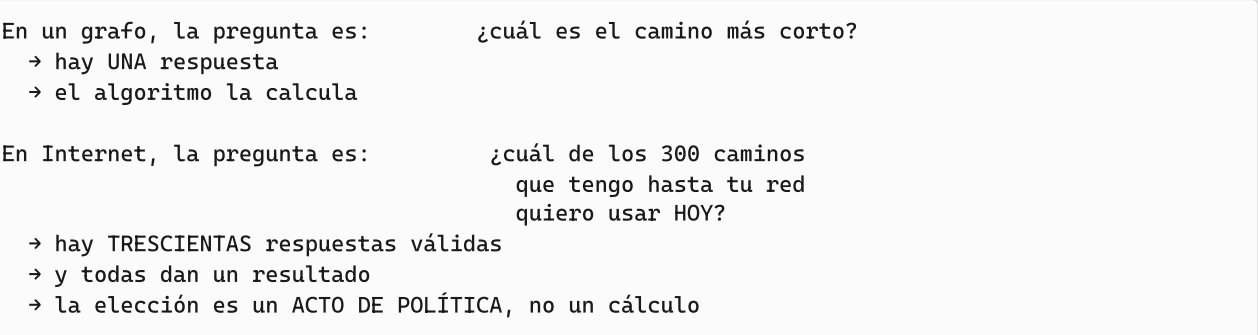
Internet no es eso. Internet es **decenas de miles de dominios administrativos independientes**, cada uno con sus equipos, su política, sus acuerdos comerciales y su dinero. Cada uno decide **a quién le anuncia qué**.



El número. En el momento en que se da esta clase, el número de sistemas autónomos visibles en la tabla global de BGP ronda los **setenta mil**. Ese número cambia todos los meses, así que **medilo vivo** en la página de stats de APNIC o en Cloudflare Radar antes de citar un número, y decí siempre **qué** estás contando (AS activos en la tabla global, no "empresas que tienen internet"). En el curso de 2011 la cifra era de unos treinta mil: **ha más que duplicado**, y esa es la razón por la que BGP no escala como los protocolos que usamos adentro.

> ⚠️ **Trampa conceptual clásica.** Decir que "Internet es una red de redes" es correcto pero inútil, igual que decir "una LAN es una red de redes de una sola máquina". Lo que hace especial a Internet es que cada red de redes tiene **una política distinta**, y esa política es lo que se negocia.

Por qué no alcanza con "el camino más corto"



Las tres razones por las que el camino más corto **no** es la respuesta correcta en el nivel inter-AS:

Razón	Consecuencia práctica
Coste económico	El enlace tránsit **se paga**. Mandar por el más corto puede es exactamente lo que nadie quiere.
Relaciones comerciales	Un AS transita de other's traffic solo si tiene peering o acuerdo.
Latencia y riesgo	El camino más corto en número de saltos puede pasar por un enlace transatlántico de 3 saltos.

> **La frase para el examen:** *"BGP no es el algoritmo más corto de Internet: es el protocolo que permite a cada AS expresar su propia política de alcanzabilidad, y por eso siempre encuentra una respuesta, aunque no sea la de menor costo."*

1.2 El Sistema Autónomo y el ASN

Un **sistema autónomo (AS, Autonomous System)** es **un conjunto de prefijos IP y un conjunto de routers que los anuncia con una política única y consistente**. Es una noción **administrativa**, no técnica.

```
¿Qué hace que un conjunto de prefijos sea UN AS?
─────────────────────────────────────────────────

· un solo Routing Policy declarado
· un único ORIGIN de las rutas en BGP
· routing autónomo: decide qué anuncia y a quién,
  y sobre qué NumericRoutesMinúsculas puede contar de forma razonable
· un ASN que lo identifica en el plano de control

(lo que NO define un AS: la tecnología de los enlaces,
 el IGP que usen adentro, o el número de routers)
```

Cómo se asigna	Detalle
RIR → ISP	Los cinco RIR (ARIN, RIPE NCC, APNIC, LACNIC, AFRINIC) asignan bloques **públicos** de IPv4 y /32 de IPv6
ISP grande → cliente	Un operador que ya tiene un AS puede delegar un **propio ASN** a un cliente grande. El cliente es un AS **independiente**
Multihoming	Un cliente con **dos** proveedores tiene su propio ASN y dos sesiones eBGP, y elige qué se anuncia a quién
Privado	Hay rangos reservados para uso **privado**: se usan en redes grandes que no tocan Internet

El ASN: de 16 a 32 bits

```
AS de 16 bits (2 bytes)   ── el clásico
──────────────────────────────────────
Rango 1 .. 65535
65535 valores. Agotado a fines de los '90.

AS de 32 bits (4 bytes)   ── el vigente
──────────────────────────────────────
Rango 1 .. 4294967295
65536 valores nuevos, más los 65535 viejos reasignados.

Notación decimal "asdot":   1.10   o  1.0.0.10
Notación decimal "asplain":  65546 (o 1.0.0.10 en_configs)
```

Documento	Qué dice
`RFC 4893`	El **problema**: el campo de 2 octetos del AS_PATH ya no alcanza
`RFC 6793`	El **mecanismo**: se agrega una **capacidad opcional** (Capability TLV) al mensaje OPEN que dice "soporte AS de 4 octetos", y un atributo opcional con el AS de 32 bits
`RFC 5396`	La **notación decimal** (`asdot` y `asplain`)

El detalle fino, que es lo que se pregunta: los dos routers que se hablan **tienen que ponerse de acuerdo**. El router que anuncia un ASN mayor que 65535 tiene que announce la **Capacidad** en el OPEN. Si el vecino es viejo y no entiende la capacidad, la sesión **no se establece**: no hay un modo degradado. Por eso la transición del AS de 16 al de 32 bits tardó años, y no fue un problema de falta de números sino de **interoperabilidad del handshake**.

> **No inventar:** `AS_SET` (cuando una ruta atraviesa un AS que no anuncia prefijo, se agrupa en un AS_SET) y el tema de los **private ASNs** existen, pero el número de RFC que reserva los rangos privados **no está** en nuestra lista de referencia: verificar en `rfc-editor.org` antes de citarlo en un examen.

1.3 BGP-4 y la sesión entre vecinos

BGP-4 (`RFC 4271`) es el protocolo de enrutamiento **entre** sistemas autónomos. Corre **directamente sobre TCP**, en el **puerto 179**.

```
¿Por qué TCP y no un protocolo como UDP o como el de OSPF?

┌──────────────────────────────┬──────────────────────────────┐
│  Lo que BGP NECESITA         │  Lo que TCP LE DA            │
├──────────────────────────────┼──────────────────────────────┤
│  que los mensajes lleguen    │  fiabilidad (retransmisión)│
│  y en orden                  │                              │
│  que no se dupliquen ni      │  orden (números de           │
│  se pierdan fragmentos       │  secuencia TCP)              │
│  control de congestión       │  control de flujo +          │
│  (Internet es best effort,   │  control de congestión       │
│  pero BGP no debe sumar      │  (ventana)                   │
│  carga extra al backbone)    │                              │
│  NO hay flooding: cada       │  conexión punto a punto      │
│  vecino recibe solo lo       │  con un solo vecino          │
│  que le corresponde          │                              │
└──────────────────────────────┴──────────────────────────────┘
```

Esa última fila es la clave conceptual: **BGP no inunda nada**. El flooding de LSA es un mecanismo de capa 3 con TTL y multidifusión; BGP usa una **sesión punto a punto TCP por vecino**. El precio es que hay que configurar los vecinos a mano (o por algún protocolo de configuración), y el beneficio es que el control de congestión de TCP protege el backbone de la señalización.

El modelo de datos: BGP es un **vecino a vecino** (*neighbor-to-neighbor*), no un **nodo a nodo** (*node-to-node*).

```
La diferencia es brutal:

LINK-STATE (nodo a nodo):
   R1 inunda a R2, R3, R4...  →  TODOS quedan con la MISMA base.
   Después cada uno calcula lo mismo. El resultado es idéntico.

BGP (vecino a vecino):
   R1 habla con R2.  R2 habla con R3.  R3 habla con R1. ...
   Cada sesión es una relación PRIVADA entre dos routers.
   R1 puede estar informado de algo que R4 no sabe, y al revés.
   ⇒ NO hay una base común, y por eso cada router decide con SU
     propia política. Esa es la razón de que BGP pueda tener
     "decisiones contradictorias" sin que nada esté roto.
```

> **Lo que esto implica para el forwarding:** las dos mejores rutas de BGP pueden **disputar el tráfico**. Si los routers vecinos toman cada uno un camino distinto para el mismo prefijo, el resultado es un **laqueo** (*load-balancing*) o un **tráfico en la dirección equivocada**. BGP resuelve eso en §1.7 con el **full-mesh iBGP** y el **NEXT_HOP** inalterado.

1.4 Path-vector: la diferencia clave con distance-vector

Un protocolo **path-vector** no manda "a la red X estoy a distancia 4". Manda **el camino completo**.

AS_PATH: la lista, no el número

```
Lo que BGP anuncia:

  NLRI:      192.0.2.0/24          (el prefijo)
  AS_PATH:   20070  15169  20115   (por dónde se llegó)

... y el router receptor le PREPEND a su propio ASN cuando
re-anuncia la ruta:

  AS_PATH:   20115  20070  15169  20115     ← ¡no!  se rechaza. Ver abajo.
```

La regla que define a BGP es una sola:

```
┌──────────────────────────────────────────────────────────┐
│  Un router NO ACEPTA ni REEVALUA una ruta cuyo AS_PATH   │
│  contenga su PROPIO ASN.                                 │
│                                                          │
│  "Si mi ASN ya está en el camino que me ofrecen, alguien │
│   me está devolviendo una ruta que yo mismo anuncié.     │
│   Eso solo puede significar un bucle, así que la descarto." │
└──────────────────────────────────────────────────────────┘
```

Veamos el mecanismo paso a paso, que es lo que hay que saber dibujar:

```
t0   AS 20070 anuncia a AS 15169:   AS_PATH = [20070]
t1   AS 15169 anuncia a AS 20115:   AS_PATH = [15169, 20070]
t2   AS 20115 anuncia a AS 33000:   AS_PATH = [20115, 15169, 20070]
t3   AS 33000 anuncia a AS 20070:   AS_PATH = [33000, 20115, 15169, 20070]
     ┌──────────────────────────────────────────────────────┐
     │  20070 RECHAZA la ruta: su propio ASN (20070) ya está  │
     │  en el AS_PATH. No la acepta, no la instala, no la     │
     │  anuncia. Fin. NO se genera ningún bucle.              │
     └──────────────────────────────────────────────────────┘

t4   20115 EXPIRA y avisa a 15169:   "dejá de usar la ruta por mí"
     15169 la borra y avisa a 33000, que la borra, que avisa a
     20070 (que ya nunca la tenía). Todo el camino se deshace
     solo, SIN conteo a infinito y SIN split horizon.
```

Por qué es **exacto** y no probabilístico

Esta es la diferencia de fondo, y es la que más se pregunta.

	Distance-vector (clase 32)	**Path-vector** (BGP)
Qué viaja	**Una estimación de distancia**, que puede estar vieja	**La historia exacta** del camino, que no puede envejecer
Cómo se detecta un bucle	**Contando**: si el costo sube, "algo anda mal". Con *hold-down* y *poison reverse*, es **probabilístico**: funciona en la mayoría de los casos y falla en algunos	**Mirando**: si mi ASN está en la lista, es un bucle. **Exacto, siempre**
Qué pasa con el rumor incorrecto	Se **realimenta**: `R2` le dice a `R1` que llega a destino a 3 usando a `R1`, y los dos suben la cuenta	No hay rumor de rutas, así que no hay nada que realimentar
Split horizon	**Imprescindible**	**Innecesario**, y no porque sea un parche que se olvidó
Convergencia con un error	Puede quedar oscilando entre dos valores para siempre	Converge al camino **más corto en número de AS** que la política permita
Límite de hops	RIP: 15, y el 16 es "infinito"	**No hay límite.** El AS_PATH crece, pero no hay tope duro

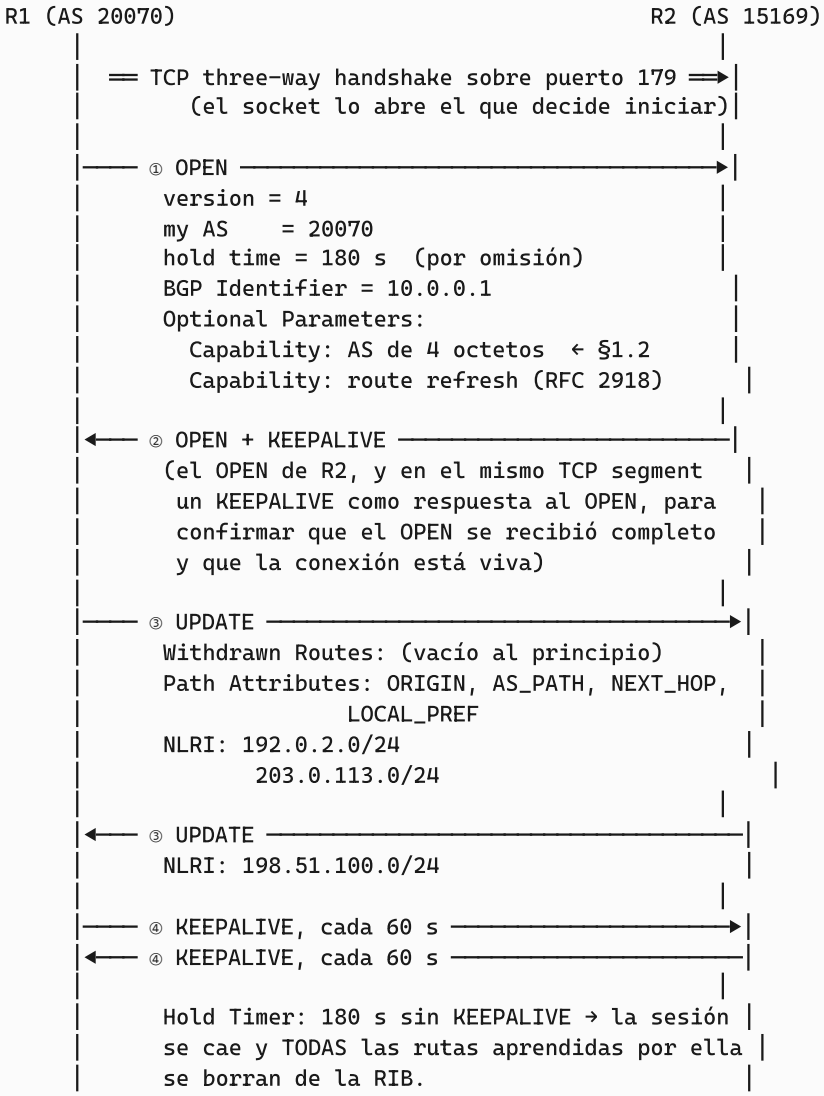
```
¿Por qué BGP no cuenta a infinito? Porque no hay información
derivada que realimentarse.

El AS_PATH es un REGISTRO de lo que realmente pasó. Cuando AS
20070 recibe una ruta que dice "pasé por 20070", no está leyendo
un rumor UpdatedByR3: está leyendo un hecho físico que él mismo
genero. La información es verificable por construcción.
```

Y el corolario incómodo, que hay que decir: BGP es exacto contra bucles, pero **no es exacto contra errores de configuración**. Si `AS 20070` anuncia por error el prefijo de otro, ese error se propaga **a toda la Internet** en cuestión de segundos, sin que ningún mecanismo de conteo a infinito lo frene. La diferencia es que un bucle se autodetiene; **una mentira no**.

1.5 El handshake: OPEN → OPEN/ACK → UPDATE → KEEPALIVE

```
Los cuatro tipos de mensaje de BGP (header de 19 bytes: marker,
length, type):

1 = OPEN       2 = UPDATE      3 = NOTIFICATION
4 = KEEPALIVE  5 = ROUTE-REFRESH
```

El diagrama del establecimiento



> ⚠️ **No inventar el número de RFC.** El *capability* de **route refresh** está definido por `RFC 2918`; ese número **no está** en nuestra lista de referencia, así que en el examen conviene nombrarlo sin número, o verificarlo antes. Todo lo demás que aparece acá (`OPEN`, `UPDATE`, `KEEPALIVE`, el hold time de 180 s y el keepalive de 60 s) está en `RFC 4271`.

La máquina de estados

```
Idle
  │  (evento manual: se quiere hablar con este vecino)
  │  se abre el socket TCP al puerto 179
  ▼
Connect
  │  (se espera a que el TCP se complete)
  │  si falla, se reintenta cada ConnectRetryTime (120 s)
  ▼
Active
  │  (el OPEN fue enviado; se espera el OPEN del vecino)
  │  ⚠ En eBGP, si el OPEN no llega, se vuelve a Connect:
  │    el que no tiene el socket pasivo espera al que sí lo tiene.
  ▼
OpenSent
  │  (se recibió el OPEN: se comparan AS, version, hold time,
  │   Router ID. Si no coinciden → NOTIFICATION y Idle)
  │  se manda el KEEPALIVE de respuesta
  ▼
OpenConfirm
  │  (se espera el KEEPALIVE del vecino)
  ▼
Established   ← ACÁ se procesan los UPDATE y se anuncian rutas
  │
  └── si expira el Hold Timer, o llega un NOTIFICATION,
      o se cierra el TCP  →  Idle
```

> **El detalle que se pregunta:** el estado `Active` existe porque en **eBGP** los dos extremos no siempre pueden abrir la conexión al mismo tiempo (el "passive" no acepta). El que no está en *passive* espera; el otro lo busca. Sin ese estado, un router nunca se conectaría a uno que espera.

1.6 Los atributos de decisión y la tabla de desempates

BGP recibe **muchas** rutas al mismo prefijo y tiene que elegir una. Lo hace con un **proceso de decisión** determinista que evalúa atributos en un orden fijo.

Los atributos, uno por uno

Atributo	Significado	Dirección del criterio	Dónde se usa
Weight (no estándar)	Preferencia **local** del router	**Mayor** gana	Solo iBGP, y es propiedad del *vendor*
LOCAL_PREF	Preferencia que el AS le da a una ruta	**Mayor** gana	**Solo iBGP**. El valor por omisión es 100
Origen local	¿La ruta nació en este AS?	**Local gana** sobre todo lo demás	El atajo para no elegir un camino que vuelva a uno mismo
AS_PATH length	Cantidad de AS por los que pasó	**Menor** gana	eBGP e iBGP. **Es el criterio clásico de "corto"**
Origin	Cómo entró la ruta en BGP	`IGP` < `EGP` < `Incomplete` (en ese orden de preferencia)	Se usa si todo lo anterior empata
MED (*Multi-Exit Disc*)	Métrica del AS **vecino**	**Menor** gana	**Solo comparable entre rutas del mismo AS vecino**
eBGP sobre iBGP	Origen de la sesión	**eBGP gana**	Se usa si todo lo anterior empata
Métrica IGP al NEXT_HOP	Costo interno hasta el siguiente salto	**Menor** gana	El "hot potato" de §1.8
Router ID más bajo	Desempate final	**Menor** gana	Solo para que sea **determinista**

La tabla de desempates, en orden

Esto es literal de `RFC 4271` más las extensiones, y es lo que hay que recite:

```
Bajar por la lista y quedarse con la que gana en el primer
criterio donde haya diferencia. Todas las rutas que ganan los
criterios anteriores siguen compitiendo.

1. Weight                    mayor gana       (local, no estándar)
2. LOCAL_PREF                mayor gana       (solo iBGP)
3. Ruta originada localmente local gana
4. AS_PATH más corto         menor gana       ★ el criterio clásico
5. ORIGIN                    IGP > EGP > Incomplete
6. MED                       menor gana       ★ solo entre el mismo AS vecino
7. eBGP sobre iBGP           eBGP gana
8. Métrica IGP al NEXT_HOP   menor gana       ★ el hot potato
9. Router ID más bajo        menor gana       (determinismo)
```

NEXT_HOP y ORIGIN, en detalle

```
NEXT_HOP: la dirección IP a la que hay que enviar el paquete para
seguir por esta ruta.

· En eBGP, el que anuncia pone SU PROPIA dirección en el NEXT_HOP.
  "Si querés llegar por acá, entrá por mí."
· En iBGP, el que anuncia NO cambia el NEXT_HOP. Deja el que
  aprendió del eBGP de entrada.
  ⇒ de ahí sale la necesidad del **NEXT_HOP self** (§1.7).

ORIGIN: dice cómo se originó la ruta.
  IGP         → la ruta entró por un IGP (OSPF, IS-IS) [la mejor]
  EGP         → la ruta entró por un EGP legado, hoy es rarísimo
  Incomplete  → la ruta se redistribuyó desde otro protocolo, o el
                AS no lo originatorió: es el peor de los tres.
```

Communities (`RFC 1997`)

Un atributo de 32 bits que lleva **etiquetas** que los routers interpretan con reglas. Es la forma de decir "trato especial" sin tocar la configuración de cada router:

Community	Significado
`NO_EXPORT`	No la anuncies **fuera** de este AS
`NO_ADVERTISE`	No la anuncies **a nadie**
`NO_EXPORT_SUBCONFED`	No la anuncies fuera de la confederación
`65000:100` (formato `AS:value`)	**Definida por el operador**, con el AS como espacio de nombres

```
Ejemplo real: un AS de tránsito marca con NO_EXPORT las rutas de
sus clientes directos. El resultado es que ese cliente es
alcanzable por los peers de ese AS, pero que los peers NO lo
anuncian hacia afuera. Es una forma de aplicar "hot potato"
invertido sin tocar una sola métrica.
```

Y `AS_PATH prepending`, que es lo que hace el 95 % de lasYPE policy en la práctica y que no es un atributo sino un **truco**:

```
Un AS que quiere salir menos por un vecino anuncia la misma ruta
VARIAS VECES con su propio ASN repetido al principio:

   AS_PATH real:       20070  15169  20115           (3 saltos)
   AS_PATH prepended:  20070  20070  20070  15169  20115   (5 saltos)

Los vecinos que usan "menor AS_PATH gana" lo evitan. No obliga a
nadie: un vecino que prefiera otra cosa (menor MED, mayor
LOCAL_PREF, o que tenga un peering con vos) lo ignora.
```

1.7 eBGP, iBGP, full-mesh y Route Reflection

eBGP contra iBGP

```
eBGP:  entre routers de AS DISTINTOS
       ├─ el que anuncia REESCRIBE el NEXT_HOP con su propia IP
       ├─ el AS_PATH se le PREPENDea con su ASN
       ├─ el TTL del paquete BGP pasa a 1 (no sale del AS)
       └─ el LOCAL_PREF y el WEIGHT no se anuncian

iBGP:  entre routers del MISMO AS
       ├─ el que anuncia NO toca el NEXT_HOP
       ├─ el AS_PATH NO cambia (ya incluye el ASN propio)
       ├─ sí puede anunciar LOCAL_PREF
       └─ ⚠ REGLA CRÍTICA: una ruta aprendida de un iBGP NO se
          puede re-anunciar a OTRO iBGP
```

La regla y por qué existe el full-mesh

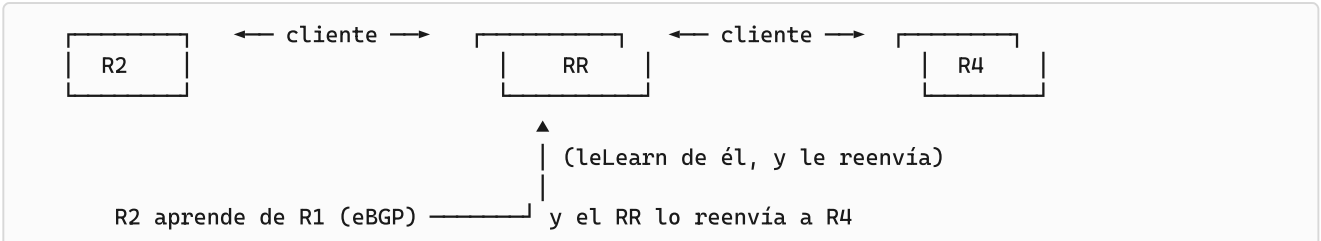
```
La regla: "lo que aprendí de un vecino iBGP no se lo cuento a otro
vecino iBGP". Sin esta regla, un router que aprendió una ruta de
R1 se la pasa a R3, que se la pasa a R5, que se la pasa a R2, que
se la pasa a R1...  BUCLE. Y como el NEXT_HOP no se reescribe, los
paquetes se mandan en círculos.

Con la regla, ¿qué pasa?
R2 aprende de R1.
R3 NO puede aprender de R2.  ✗  ⇒ R2 y R3 no coinciden.

La solución clásica: que R1 se lo cuente a TODOS los demás.

       R1 ──────────► R2
       │  \           │
       │    \         │
       │      \       │
       ▼        \     ▼
       R3 ──────────► R4

El FULL-MESH: todos con todos.
```

El costo, que es la razón por la que existe Route Reflection:

```
Sesiones iBGP necesarias =  N(N-1)/2

N = 5    →  10 sesiones
N = 10   →  45 sesiones
N = 20   → 190 sesiones
N = 50   → 1225 sesiones
N = 100  → 4950 sesiones
N = 200  → 19.900 sesiones

Con Route Reflection:  O(N)
```

Route Reflection (`RFC 4456`)

La idea: un router puede hacer de **espejo** de otro, y entonces sí puede pasarle rutas.



Los dos atributos que hacen que el diseño sea seguro — y son 2 preguntas de examen ensured:

Atributo	Para qué
ORIGINATOR_ID	Identifica al router del cluster que **originó** la ruta. Si un cliente recibe por reflection una ruta cuyo `ORIGINATOR_ID` es él mismo, **la descarta**. Es el split horizon del path-vector
CLUSTER_LIST	Lista de los `CLUSTER_ID` s por los que pasó la ruta. Si vuelve a aparecer uno, **la descarta**. Evita bucles entre route reflectors

```
Las dos reglas del protocolo:

1. Una ruta aprendida de un cliente iBGP NO se reenvía a los
   OTROS clientes (sí a los demás peers y al resto del cluster).
   → es la regla original, adaptada

2. Una ruta aprendida de un peer iBGP o de un cliente NO se
   reenvía a ningún otro cliente.

Y las dos reglas de NO reenvío:
· si mi ORIGINATOR_ID == el de la ruta → no la reenvío
· si mi CLUSTER_ID ya está en el CLUSTER_LIST → no la reenvío
```

Y el detalle de implementación que se pregunta siempre: con route reflection, el **NEXT_HOP sigue siendo el del eBGP de entrada**. El router interior que recibe la ruta tiene que **resolver ese NEXT_HOP** usando su IGP, o cambiarlo a sí mismo con **NEXT_HOP self** en su `UPDATE`. Sin eso, el router no tiene por dónde mandar el paquete.

1.8 "Hot potato": por qué un AS prefiere salir por el vecino más cercano

```
El caso clásico, que hay que dibujar:

┌──────────────── AS 20070 ─────────────────┐
│                                           │
│   [servidor local]                        │
│        │                                  │
│        ├──────────── enlace a AS 15169 ──────┼──→  destino
│        │      10 Mb/s   ·   1 hop         │    (en Internet)
│        │                                  │
│        └──────────── enlace a AS 20115 ──────┼──→  destino
│               1 Mb/s    ·   3 hops        │
└───────────────────────────────────────────┘

El servidor quiere alcanzar un destino que ambos vecinos
anuncian. Por cuál sale el router de borde?

La respuesta "obvia": por 15169, que a 10 Mb/s y a 1 hop.

La respuesta de BGP: por 20115, el que está MÁS LEJOS.
```

¿Por qué? Porque en la **última milla** no hay ancho de banda gratis. La conexión de **10 Mb/s** de `20070` a `15169` es un recurso que el operador tiene que pagar y mantener **entero**. Todo el tráfico que salga por ese enlace, aunque el cliente se vaya enseguida, consume ancho de banda propio y su capacidad de subida.

```
La analogía: un Starbucks en la esquina de tu casa y otraStarbucks
en la esquina de al lado de tu trabajo. El café de la esquina de tu
casa es más cómodo. El de tu trabajo es más barato para la empresa
porque ya está pagado de antemano por el alquiler de la oficina.

"hot potato" = "pasame la papa caliente":
sacala de tu casa y pasámela a quien está más lejos,
mientras la tuya se enfría en el escritorio.
```

Cómo lo corrige el MED

`MED` (*Multi-Exit Disc*) es el atributo que le dice al AS vecino: **"salí por mí, y te paso el costo de salir por mí"**.

```
El proveedor 15169 anuncia a 20070 dos veces la misma ruta:

  Anuncio 1:  AS_PATH = [15169, 20115]        MED = 50
  Anuncio 2:  AS_PATH = [15169, 33000]       MED = 200

Con MED = 50, 20070 tiene un incentivo de 50 para salir por 15169,
que es la puerta de salida más barata para él.
Con MED = 200, el vecindario prefiere salir por la otra parte.
```

Las dos trampas del MED, que son pregunta clásica:

```
1. COMPARABLE SOLO DENTRO DEL MISMO AS VECINO.
   El RFC dice: comparar MED solo entre rutas que vinieron del
   MISMO neighbour AS. Si no, un vecino con MED = 0 (porque en
   SU red es normal) te va a ganar todas las comparaciones y te
   va a arrastrar a un camino objetivamente peor.

   Por eso todos los proveedores grandes tienen una
   CONFIGURACIÓN: "always compare med" / "never compare med" /
   "compare only if from the same AS". Es política, no estándar.

2. NO ES TRANSITIVO NI ORDENABLE.
   El MED compara ORIGENES (el vecindario), no la red de destino.
   Que AS-A tenga MED 50 hacia destino X y AS-B tenga MED 80
   hacia destino X no dice NADA sobre cuál tiene un camino más
   corto hasta Y. El MED es un número sin unidad: no se suman,
   no se promedian, no se propagan hacia atrás.
```

Y el detalle que le da la vuelta a la aguja: el "hot potato" se puede invertir **desde el lado del vecino** con `LOCAL_PREF`. Un AS puede subirle el `LOCAL_PREF` a las rutas que le llegan por un vecino determinado para que **todo su tráfico de salida** se vaya por ahí, sin tocar la configuración del vecino. Por eso `LOCAL_PREF` es el atributo **más usado** en la práctica y `MED` es el segundo.

1.9 Por qué BGP no cuenta a infinito ni necesita split horizon

Recapitulamos, porque es el cierre conceptual de las clases 32, 33 y 34:

```
DISTANCE-VECTOR                      LINK-STATE              PATH-VECTOR
───────────────                      ──────────              ───────────
viaja: una ESTIMACIÓN                viaja: un HECHO         viaja: un HECHO
        de distancia                         local                 de historia

¿se puede realimentar?  SÍ           ¿se puede realimentar?  NO
¿qué lo detecta?        CONTAR        ¿qué lo detecta?        NADA: el árbol
¿cuán exacto es?        PROBABILÍSTICO ¿qué lo detecta?       NO HAY CAMINO
                         (split horizon,                        QUE VOLVER
                         hold-down)                             (es un árbol)

rip → split horizon → pero igual        ospf → flooded →        bgp → AS_PATH →
puede oscilar                             inundación O(N)          rechazo exacto
```

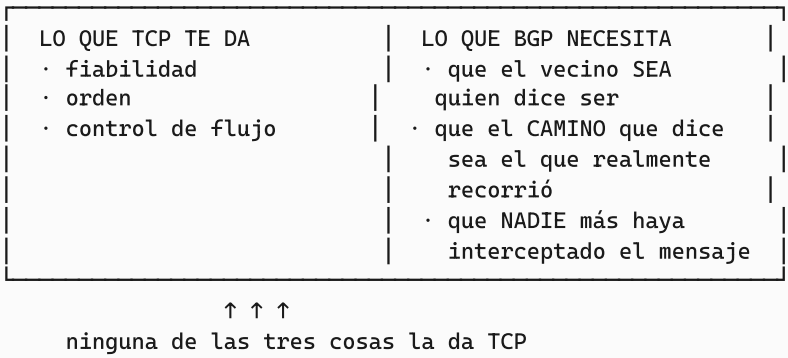
El argumento completo, en tres pasos, que es lo que hay que escribir:

1. **No hay estimación que realimentarse.** En DV, el problema es que `R2` comunica a `R1` una distancia **calculada**, que puede estar vieja y que incluye a `R1` en el camino. En BGP, lo que viaja es el **AS_PATH**, que es un **registro verificable** de los AS por los que pasó el anuncio. No hay nada derivado.
2. **El lazo se reconoce por inspección.** Cuando `AS 20070` recibe un AS_PATH que contiene `20070`, el ciclo **está escrito en el mensaje**. No hace falta detectarlo contando: se lee.
3. **La detección es estructural, no estadística.** El *split horizon* de DV es una **heurística**: "no le digas a nadie lo que te llega por él" funciona en el 99 % de los casos y falla en algunos. El filtro de `AS_PATH` funciona en el **100 %**, porque la condición que chequea es **necesaria y suficiente** para que exista un lazo. No es "una buena heurística contra bucles": es **la definición** del bucle.

```
Y hay una diferencia más, que es la que más se olvida:

BGP NO TIENE CONVERGENCIA EN EL SENTIDO DE OSPF.

En OSPF, cuando el flooding termina, TODOS los routers tienen el
MISMO árbol. En BGP, cuando "terminó", cada router tiene SU propio
mejor camino según SU política, y puede ser distinto del de su
vecino. Eso no es un bug: es el diseño.

El resultado es que BGP no tiene una noción de "estado estable
global", y por eso el tráfico puede oscilar entre dos rutas
válidos durante minutos (y a veces, con prefijos anunciados y
retirados rapidísimo, durante días).
```

1.10 Seguridad: por qué BGP no es seguro

BGP corre sobre **TCP/179**, y **TCP no cifra ni autentica nada**. Todo lo que viaja entre dos routers BGP es **texto claro y sin firma**.



Los dos ataques[I] que eso habilita, y que hay que nombrar:

Ataque	Qué hace
Hijacking de sesión / MITM	Alguien en el camino intercepta el TCP y manda mensajes BGP falsos. Como no hay cifrado, puede **injectar rutas** y **redirigir tráfico**
Route leak	Un AS **legítimo y bien configurado** anuncia prefijos que no le pertenecen. No hay ataque: es un error (o una ambición desmedida). Es **el fallo más común de BGP en la práctica**

El mecanismo clásico: MD5 de sesión (`RFC 2385`)

La primera respuesta fue **firmar cada mensaje**:

```
· Se calcula un MD5 sobre (password + mensaje BGP) y se lo manda.
· El vecino recalcula y compara.
· Si no coincide, el mensaje se descarta.
· Se protege contra el MITM. NO protege contra nadie que esté
  legítimamente en el camino y tenga la clave.
· Es un mecanismo SIMÉTRICO: ambas partes tienen la misma clave.
```

Es `RFC 2385` (2000), y fue **el** mecanismo de BGP security durante dos décadas.

La respuesta moderna: los roles BGP (`RFC 9234`, 2022)

```
La idea: en lugar de PROBAR quién es el vecino (con criptografía),
DECLARAR qué relación comercial se supone que hay entre los dos,
y rechazar cualquier mensaje que la contradiga.

┌──────────────────────────────────────────────────────────┐
│  ROLES (declarados en el OPEN y en cada UPDATE)          │
│                                                          │
│  Provider  → "soy tu proveedor (tú me pagás)"            │
│  Customer  → "soy tu cliente (me pagás vos)"             │
│  Peer      → "somos pares, ninguno le paga al otro"      │
│  RS        → "Route Server"                              │
│  RS-Client → " soy cliente de un Route Server"           │
│  Lateral   → relación interna de confianza               │
│  Internal  → uso interno del AS                          │
└──────────────────────────────────────────────────────────┘

EJEMPLO, que es el que hay que saber:

ANTES (solo MD5):
   AS A es cliente de AS B. B anuncia a A un prefijo que en
   realidad es de C, que es cliente de A.
   Con MD5, A no tiene forma de saberlo: el mensaje es
   criptográficamente válido. A lo acepta y lo anuncia. LEAK.

DESPUÉS (roles):
   B anuncia a A el prefijo de C con ROL = Provider.
   A mira su configuración: "B es mi proveedor", y los clientes de
   A (como C) NUNCA se anuncian por un proveedor. CONTRADICCIÓN.
   A rechaza la ruta.

⇒ El rol es una AFIRMACIÓN VERIFICABLE CONTRA LA CONFIGURACIÓN
  LOCAL. Y la configuración local es lo único que no se puede
  falsear con un ataque en el camino.
```

`RFC 9234` es de **mayo de 2022**. Su texto dice explícitamente que reemplaza al `RFC 2385` como mecanismo de referencia: hoy MD5 es **legado**.

RPKI / ROA: validar el origen del prefijo

MD5 y los roles protegen **el camino**. RPKI protege **la cosa misma**:

```
El problema que MD5 y los roles NO resuelven:

AS A (legítimo, bien configurado, sin bugs) anuncia el prefijo
203.0.113.0/24, que NO le pertenece. Legalmente puede hacerlo:
nadie se lo impide. Y lo anuncia a todo el mundo.

¿Cómo sabemos que no le pertenece? Porque hay un REGISTRO.
```

Pieza	Qué es
ROA (*Resource Origin Authorization*)	Un documento firmado digitalmente que dice: **"el AS X está autorizado a originar el prefijo P"**. Cada RIR lo publica
RPKI (*Resource PKI Infrastructure*)	La **infraestructura de claves públicas** que firma los ROA. Equivale a la PKI del DNI, pero para prefijos
RTR (*RPKI to Router*)	El **protocolo** que le lleva los ROA al router, para que pueda validarlos. `RFC 6810`
Validación	El router recibe el ROA de un prefijo y de su AS de origen, y lo compara con la ruta que le llega. `RFC 6811`

```
Los tres resultados posibles de la validación de una ruta:

VALID       el prefijo tiene un ROA y el AS de origen coincide
INVALID     el prefijo tiene un ROA y el AS de origen NO coincide
            → el AS está anunciando algo que no le pertenece. Es un
              ataque o un error. → DESCARTA.

NOT FOUND   el prefijo no tiene ROA. No dice nada. → "no sé", y
            BGP sigue con su decisión de todas formas.

⚠ Y ACÁ ESTÁ LA TRAMPA que hay que aclarar siempre: si el 99 % de
los prefijos de Internet están "Not Found", RPKI no cambia
ABSOLUTAMENTE NADA para ese 99 %. Solo sirve donde hay ROA. Es
por eso que la despliegue real es bajo: no falla, pero tampoco
hace casi nada todavía.
```

El punto clave para el examen: RPKI **no cifra ni autentica la sesión**. Valida el **origen declarado del prefijo**, leyendo una cadena de confianza externa a BGP. Es **ortogonal** a MD5 y a los roles: los tres resuelven cosas distintas y ninguno reemplaza a los otros.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **BGP-4 (`RFC 4271`)** sigue siendo la especificación vigente. El "4" es la versión, y desde 1994 no cambió el formato. - **`AS_PATH` es un *path-vector***: la lista de AS por los que pasa la ruta, y **un AS no acepta una ruta cuyo AS_PATH contenga su propio AS**. Eso es lo que hace que BGP sea *path-vector* y no *distance-vector*. - **Sesión sobre TCP/179**: OPEN → OPEN/ACK → UPDATE → KEEPALIVE. Hold time 180 s, keepalive 60 s. - **eBGP contra iBGP**; la regla de "no re-anunciar a iBGP lo aprendido de un iBGP" obliga al **full-mesh iBGP**, y de ahí salen las **Route Reflection (`RFC 4456`)**. - **Atributos**: `LOCAL_PREF` (↑ mejor), longitud de `AS_PATH` (↓ mejor), `MED` (↓ mejor, **comparable solo entre rutas del mismo AS vecino**), `ORIGIN`, `NEXT_HOP`, **Communities (`RFC 1997`)**. - **"Hot potato"**: cada AS prefiere salir por el eBGP de **menor métrica IGP**. `MED` es la herramienta para anular esa preferencia. - **OSPFv2 (`RFC 2328`)** = link-state, SPF/Dijkstra, áreas, DR election, multicast 224.0.0.5/6, TTL 1 en el hello. - **RIPv2 (`RFC 2453`)**: métrica ≤ 15, poison reverse (16), split horizon, triggered updates, hold-down. **Sigue siendo *Internet Standard*, pero está deprecado de facto en Internet**: sobrevive en laboratorios y campus.

Cambió:

- `RFC 9234` **(may-2022): BGP sin MD5 de sesión.** Define **roles BGP** (Provider / Customer / Peer / Lateral / Internal) declarados en OPEN y UPDATE para prevenir y detectar *route leaks*. Reemplaza al `RFC 2385` (MD5 de sesión) como mecanismo de referencia.
- **RPKI / ROA**: `RFC 6810` (RTR) y `RFC 6811` (BGP Prefix Origin Validation). Maduró de idea a **despliegue real**, pero con el matiz de cobertura de §1.10: sin ROA no hay validación. La seguridad de BGP pasó de

"confiar en AS_PATH" a "**validar origen con RPKI**".

- `BGP-LS` → `RFC 9552` (dic-2023), obsoleta RFC 7752.
- **EVPN (`RFC 7432`) + VXLAN (`RFC 7348`)**: hoy son **el estándar de overlay de datacenter**, no solo académico. Y como el plano de control de EVPN es **BGP**, esto es lo que se conoce como **"BGP para datacenters"**.
- **Segment Routing**: `RFC 8402` (arquitectura, 2018), `RFC 8660` (SR-MPLS), `RFC 8986` (SRv6). Es el **sucesor de facto** de MPLS LDP/RSVP-TE en despliegues nuevos: *source routing* con SIDs y **estado solo en el borde de ingreso**. SRv6 codifica los SIDs en la cabecera IPv6 y **elimina el MPLS por completo**.
- **OSPFv3 (`RFC 5340`)**, IS-IS para IPv6 (`RFC 5308`): mecanismos idénticos, independientes de IPv6.
- **AS de 32 bits**: `RFC 4893` + `RFC 6793`; notación decimal `RFC 5396`. **Ya no existe el problema del AS de 16 bits.**
- **BMP (`RFC 7854`)** para monitorear BGP desde fuera.

Consecuencia para el examen: si el enunciado dice "describa BGP" o "explique el path-vector", la respuesta con `AS_PATH` + rechazo del propio ASN + "no cuenta a infinito porque no viaja ninguna estimación" es la que suma. Si menciona seguridad, sumá que **MD5 es legado** y que hoy el mecanismo de referencia son los **roles BGP de `RFC 9234`**, con **RPKI** validando el origen del prefijo.

1.11 La tabla resumen: link-state, distance-vector y path-vector

	Distance-vector (RIP)	**Link-state** (OSPF, IS-IS)	**Path-vector** (BGP)
Qué viaja	Vector de distancias	Un **LSA** (hecho local)	El **AS_PATH** (historia del camino)
Qué calcula	Bellman-Ford (iterativo)	Dijkstra (SPF)	Proceso de decisión por atributos
Alcance	Un AS	Un área / un IGP	**Internet**
Topología	No la conoce	**Idéntica en todos** del área	Cada vecino ve una **vista distinta**
¿Hay bucle?	**Sí**, conteo a infinito	No (árbol)	No (rechazo por `AS_PATH`)
¿Cómo se rompe?	Split horizon, hold-down	Los tres controles del flooding	Exacta, no probabilística
¿Quién decide?	La distancia mínima	El costo de la métrica	**La política**, atributo por atributo
Transporte	UDP 520	IP protocolo 89	**TCP 179**
Escala	LAN / campus	Red corporativa, backbone	**Los ~70.000 AS**
Ejemplo del curso	`RFC 2453`	`RFC 2328`	`RFC 4271`

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La tabla de desempates aplicada a cuatro rutas

Datos. En el router `R7` del AS **20070** llegan cuatro rutas al prefijo `198.51.100.0/24`, todas aprendidas por **iBGP** de `R1`, `R3`, `R5` y `R6` respectivamente:

#	Vecino que la trajo	`WEIGHT`	`LOCAL_PREF`	`AS_PATH`	`ORIGIN`	`MED`	Tipo de sesión
A	`R1`	100	100	`20115 15169 20070`	IGP	50	iBGP
B	`R3`	100	**200**	`33000 20115 20070`	IGP	10	iBGP
C	`R5`	**300**	100	`15169 20070`	Incomplete	5	iBGP
D	`R6`	100	100	`20115 15169 20070`	IGP	50	iBGP

Resolución paso a paso, aplicando el orden de §1.6.

Paso 1 — `WEIGHT` (mayor gana). C tiene 300, los otros 100. **C queda primera; A, B y D siguen.**

Interpretación: el `Weight` es el primer corte y es **local**: no se anuncia a nadie. Configurarlo es la forma más rápida de forzar una salida.

Paso 2 — `LOCAL_PREF` (mayor gana). Solo compite entre las que pasaron el paso 1: B tiene 200, A y D tienen 100. **Gana B.**

Interpretación: `LOCAL_PREF` solo se usa en iBGP y es **la** herramienta de política interna. Acá dice: "la ruta que me trajo `R3` es la que yo prefiero", aunque el `AS_PATH` sea más largo.

Paso 3 — `AS_PATH` más corto. Se descarta C (perdió en el paso 1) y se comparan las que quedan: A y D tienen `20115 15169 20070` (**3 AS**), B tiene `33000 20115 20070` (**3 AS**). **Empate, todas siguen.**

Interpretación: ojo con esto. B tiene el `LOCAL_PREF` más alto pero **el mismo número de saltos**. Si tuviera 4, seguiría ganando: el `LOCAL_PREF` gana **antes** que la longitud del `AS_PATH`. Eso es exactamente lo que significa "el `LOCAL_PREF` es la política del AS, y el `AS_PATH` es solo un criterio de desempate".

Paso 4 — `ORIGIN` (IGP > EGP > Incomplete). A, B y D tienen IGP. **Empate, todas siguen.**

Interpretación: C tenía `Incomplete` porque la ruta fue redistribuida desde otro protocolo. Habría sido eliminada en este paso **si no hubiera perdido antes**. El orden importa: el `ORIGIN` se mira **después** del `AS_PATH`, no antes.

Paso 5 — `MED` (menor gana). A y D tienen 50, B tiene 10. **Gana B.**

Paso 6 — eBGP sobre iBGP. Las tres son iBGP. Empate.

Paso 7 — Métrica IGP al `NEXT_HOP` (menor gana). Supongamos que la ruta de B tiene `NEXT_HOP = 192.0.2.7`, que está a **8 saltos** por el IGP, mientras que las de A y D apuntan a `192.0.2.3`, a **3 saltos**.

¡B gana el paso 7! Y es acá donde aparece la paradoja:

```
B llegó en el paso 2 por tener LOCAL_PREF más alto.
B llega al final porque su NEXT_HOP está más lejos dentro del AS.

Si el operador NO hubiera configurado LOCAL_PREF en R3,
el resultado habría sido A (o D), que sale por el vecino de 3 hops.

O sea: el "hot potato" (§1.8) gana por DEFAULT, y los operadores
lo deshacen a propósito subiendo LOCAL_PREF para el vecino que
les conviene. Configurar LOCAL_PREF es, literalmente,
comprarle el tráfico a otro AS.
```

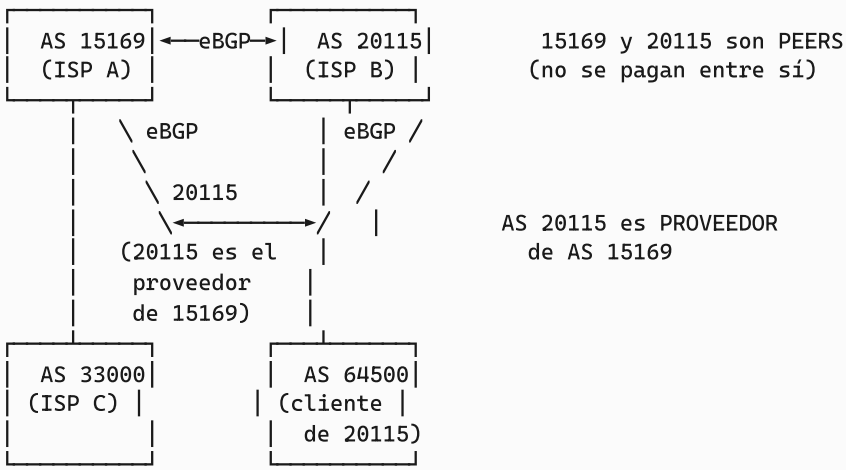
Paso 8 — `Router ID` más bajo. Si A y D siguen empatadas (mismo `MED`, mismo `NEXT_HOP`), **gana la que tenga el Router ID más bajo**. Existe solo para que el resultado sea **determinista**: dos routers con la misma configuración tienen que elegir lo mismo, o el tráfico se parte al azar entre dos caminos.

⚠️ Errores típicos

- **Comparar `MED` entre rutas de AS vecinos distintos.** No se puede (§1.8).
- **Confundir `LOCAL_PREF` con `AS_PATH length`.** El primero se mira **primero** y gana aunque el segundo sea peor.
- **Asumir que el `AS_PATH` más corto gana siempre.** No: solo después de `WEIGHT` y `LOCAL_PREF`.
- **Olvidar el paso 8.** Los desempates tienen que ser deterministas o los routers vecinos toman caminos distintos y se rompe el forwarding.

2.2 Ejercicio 2 — Topología de tres AS, paso a paso

Datos. Topología:



a) ¿Cómo ve el prefijo de `AS 64500` cada router?

El orden de propagación importa: primero `64500` se lo dice a `20115`; `20115` decide si lo anuncia a `15169` y a `33000`; y cada uno de esos lo anuncia o no a sus clientes.

Quien anuncia	`AS_PATH` que genera	Nota
`64500` → `20115`	`64500`	eBGP: el que anuncia preprende su ASN
`20115` → `15169`	`20115 64500`	Decisión: es cliente, **lo anuncia**
`15169` → `33000`	`15169 20115 64500`	Decisión: tránsito, **lo anuncia** (si lo vende)
`20115` → `33000`	`20115 64500`	**También se lo anuncia directo**

Interpretación: `33000` recibe **dos** rutas al mismo prefijo: una de largo 3 y otra de largo 2. Se queda con la de **2** (`20115 64500`), porque es su vecino directo y ya tiene el prefijo en su tabla. Y `15169` prefirió **no** anunciárselo a `33000` (decisión de política: `20115` no le vende tránsito a un competidor).

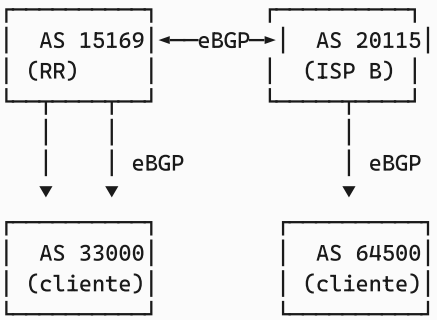
b) El `AS_PATH` que recibe `33000` es `20115 64500`. ¿Y si `64500` se cae?

```
64500 cae y abre una sesión con 33000 (multihoming).
Ahora 33000 tiene DOS proveedores para el mismo prefijo.

64500 → 20115 → 15169 → (vuelta a 33000):  RECHAZADO. 64500 ya está
en el AS_PATH. Exactamente el mecanismo de §1.4.

64500 → 33000 (directo):   AS_PATH = [64500]. Aceptada.
→ 33000 prefiere 1 salto sobre 4. Decide mal, pero es SU decisión.
```

c) ¿Qué pasa si `20115` anuncia a `15169` el prefijo de `64500` con un `LOCAL_PREF` bajo?

```
15169 recibe AS_PATH = [20115, 64500] con LOCAL_PREF = 150.
15169 ya tenía su propia ruta hacia ese prefijo? NO: 64500 es
cliente de 20115, no de 15169.

→ 15169 la acepta, la instala, y la anuncia a 33000 con
  LOCAL_PREF = 150 (propagando el valor).

OJO: 15169 NO le puede cambiar el AS_PATH. Sigue siendo
[15169, 20115, 64500]. Lo único que controla es a quién se lo
anuncia y con qué LOCAL_PREF.

Y acá está la diferencia con el AS_PATH: el AS_PATH describe el
CAMINO FÍSICO (no se puede mentir sin romper la detección de
bucles); el LOCAL_PREF describe la INTENCIÓN (que se puede mentir,
pero solo en tu propio AS: es una política interna).
```

d) Sumá Route Reflection al mismo dibujo, con `15169` como reflector.




```
15169 aprende el prefijo de 64500 de 20115 por eBGP.
15169 NO se lo anuncia a 64500 (es su cliente y no tiene sentido).
15169 se lo anuncia a 33000 como cliente suyo, por REFLECTION,
con AS_PATH = [15169, 20115, 64500] y ORIGINATOR_ID = 15169.

Ahora el agregado: en lugar de 3 sesiones iBGP (full-mesh)
hay 2. Con N routers en el cluster: N en lugar de N(N-1)/2.
```

2.3 Ejercicio 3 — Por qué BGP necesita full-mesh iBGP y qué rompe Route Reflection

Enunciado. En un AS con `R1, R2, R3, R4`, `R1` aprende una ruta de un vecino **eBGP**. Explicá por qué `R2` y `R3` no pueden learnla el uno del otro, qué problema crea eso, y qué rompe exactamente Route Reflection.

a) Por qué la regla "no re-anuncio lo aprendido de un iBGP".

```
R1 (eBGP)  ── ruta r ──►  R2
R1          ── ruta r ──►  R3
R1          ── ruta r ──►  R4

R2 y R3 son iBGP entre sí. Si R2 le dijera a R3 "ruta r":
   · R3 la aprende y la anuncia a R1 (que es su vecino iBGP)
   · R1 ya la tiene, pero no puede distinguir "la que yo anuncié"
     de "la que me devolvió R3"
   · el NEXT_HOP no cambió: sigue siendo el del eBGP de entrada
   · ⇒ R3 manda el paquete a un vecino que se lo devuelve a R3.
     BUCLE DE PAQUETES.

Y el AS_PATH no lo detiene, porque el bucle es entre routers
DEL MISMO AS: el AS_PATH es idéntico en los dos caminos.
Es un bucle de capa 3, no de capa 2.
```

b) El problema que crea la regla.

```
Con la regla, R2 NO puede obtener la ruta de R3, ni R3 de R2.
Para que todos tengan la MISMA tabla (que es lo que exige un
protocolo de enrutamiento: forwarding consistente), R1 tiene
que anunciársela a CADA uno.

Sesiones necesarias = N(N-1)/2, todos contra todos.

Consecuencias:
· N = 50  →  1225 sesiones TCP, cada una con su temporizador,
             su buffer y su CPU de keepalive.
· En un operador grande, N puede ser 200: 19.900 sesiones. Y cada
  router tiene que mantenerlas todas vivas.
· Cada caída de una sesión obliga a recalcular.
```

c) Qué rompe Route Reflection, y la respuesta corta es "nada, pero con dos excepciones".

```
Route Reflection agrega una excepción a la regla: el RR (Route
Reflector) SÍ puede pasarle a sus clientes una ruta que aprendió
de un peer o de otro cliente.

Eso no rompe nada siempre que se respeten las DOS reglas nuevas:

┌─ EXCEPCIÓN 1: ORIGINATOR_ID ──────────────────────────────────┐
│ Si una ruta llega a un cliente con ORIGINATOR_ID = MI PROPIO   │
│ Router ID, el cliente la DESCARTA.                            │
│                                                               │
│ Por qué: es el split horizon del cluster. Si R3 learns de R2  │
│ una ruta que R2 reflejó desde R1, y esa ruta dice             │
│ ORIGINATOR_ID = R3, es porque el camino pasó por R3. Aceptarla │
│ es aceptar un bucle.                                          │
└───────────────────────────────────────────────────────────────┘

┌─ EXCEPCIÓN 2: CLUSTER_LIST ───────────────────────────────────┐
│ Si mi CLUSTER_ID YA ESTÁ en el CLUSTER_LIST de la ruta, la    │
│ DESCARTO.                                                     │
│                                                               │
│ Por qué: si el CLUSTER_LIST vuelve a aparecer, la ruta pasó   │
│ por un route reflector del MISMO cluster. Entre dos reflectores │
│ del mismo cluster se usa una sesión iBGP normal (con la regla  │
│ original), así que eso es un bucle.                           │
│                                                               │
│ El CLUSTER_ID se configura. Lo normal es poner el Router ID del │
│ RR, para que un solo reflector sea un solo cluster. Si querés  │
│ que dos reflectores redundantes NO se confirmen entre sí, les ponés │
│ el mismo CLUSTER_ID.                                          │
└───────────────────────────────────────────────────────────────┘
```

d) El problema que Route Reflection NO resuelve, y que es la pregunta fina.

```
⚠ Route Reflection no arregla la CONSISTENCIA de la tabla.

Con full-mesh, TODOS los routers del AS tienen exactamente la
misma tabla (dentro de lo que la política permita).

Con route reflection, cada router tiene una tabla CORRECTA pero
DISTINTA de la de su vecino: le falta lo que seannounced por el
camino que no pasó por él.

Eso está bien mientras cada router use la ruta para DECIDIR. Pero
si un router usa su tabla para REANUNCIAR hacia afuera sin pasar
por un RR, rompe el modelo. Por eso los RR solo se usan DENTRO del
AS, y los routers de borde (BORDER routers) estántightmente
controlados.
```

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2001-08, Problema 2 — Implementar un algoritmo de ruteo jerárquico

> **Fuente:** Examen `eirc0108`, 20 de agosto de 2001 (2 páginas). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0108.pdf **Texto del examen:**
>
> > Implementar un algoritmo de ruteo jerárquico para una red de datagramas. Definir la estructura de las tablas de ruteo y los paquetes utilizados.
>
> ⚠ **Nota:** el PDF **no publica solución** de este problema (el examen viene sin soluciones). Lo que sigue es *(solución propia)*, y es central para esta clase porque **es literalmente lo que hace BGP**: particionar el espacio de direcciones en dominios administrativos y que cada dominio anuncie el resto como un **agregado**, en vez de que cada router conozca cada ruta.

El enunciado, reencuadrado

El problema pide un algoritmo de **ruteo jerárquico**. Hay que justificar tres cosas:

1. **Qué es jerárquico** y por qué no alcanza con un grafo plano.
2. **La estructura de las tablas**.
3. **Los paquetes** del protocolo.

a) Por qué jerárquico, y no plano

Un grafo plano no escala. Supongamos la red del problema: `N` routers, cada uno con una entrada por cada destino. La tabla tiene `N` filas, y **cambia completa cada vez que cambia un enlace**: `O(N)` actualizaciones, cada una de `O(N)` filas.

La idea de jerarquizar es agrupar los destinos en **grupos** y anunciar el grupo entero como una sola entrada:

```
SIN JERARQUÍA (plano):                  CON JERARQUÍA:
──────────────────────                  ──────────────────
tabla de R1:                            tabla de R1:
  red_A → 10.0.0.2                        10.0.0.0/8   → 10.0.0.2
  red_B → 10.0.0.2                        (un prefijo que
  red_C → 10.0.0.2                         cubre TODO)
  ...
  red_Z → 10.0.0.2
N entradas, N cambios por evento

El interior de 10.0.0.0/8 NO se anuncia hacia afuera:
los routers de la red "de arriba" (el AS vecino, el router de
borde) solo saben que "todo lo que empieza con 10. está detrás
de 10.0.0.2". No saben cuántas redes hay ni dónde.
```

Las dos propiedades que hace aparecer la jerarquía, y que son las que hay que nombrar:

1. **Agregación**: varios prefijos se announced como **un** superprefijo. El borde anuncia `10.0.0.0/8` y se olvida de los detalles.
2. **Ocultamiento de la topología interna**: los routers externos **no** conocen ni la topología ni el número de prefijos del dominio. Solo conocen **un punto de entrada**.

```
Y acá aparece la diferencia con el jerarquizado "de áreas" de OSPF,
que es la que más se confunde:

┌──────────────────────────┬──────────────────────────────────────┐
│  ÁREA de OSPF            │  JERARQUÍA DE BGP                    │
├──────────────────────────┼──────────────────────────────────────┤
│ se announce la           │ se announce un SUPERPREFIJO          │
│ INFORMACIÓN (cada LSA)   │ (la métrica y el resultado)          │
│ del interior             │                                      │
│                          │                                      │
│ los routers internos     │ los routers externos no saben        │
│ necesitan la información │ ni la topología ni el detalle        │
│ completa para calcular   │                                      │
│ el camino ÓPTIMO         │                                      │
│                          │                                      │
│ se pierde optimalidad    │ no hay "camino óptimo" que           │
│ DENTRO del área          │ perder: la agregación es correcta    │
│                          │ por definición                       │
│                          │                                      │
│ el estado es por ENLACE  │ lo que se announce es la RUTA        │
│ y hay que calcularlo     │ AGREGADA y su política               │
└──────────────────────────┴──────────────────────────────────────┘
```

> **Ésta es la respuesta conceptual más importante del problema:** OSPF jerarquiza **para escalar el cálculo** y BGP jerarquiza **para escalar el announcement**. Los dos resuelven el mismo problema (la tabla plana no escala) con herramientas opuestas: uno publica los hechos y esconde el camino, el otro publica el camino y esconde los hechos.

b) Estructura de las tablas de ruteo

Como no hay texto de apoyo en el enunciado, **hago las suposiciones y las justifico**, que es lo que el propio examen pide en otros problemas.

Suposición 1: la jerarquía se define por un prefijo de la dirección IP. Es lo coherente con el modelo TCP/IP del curso, y evita tener que inventar un esquema de nombres nuevo.

Suposición 2: cada router tiene tres tablas, porque en un protocolo jerárquico hay tres preguntas distintas:

Tabla	Qué contiene	Tamaño
T_int (intradominio)	Las rutas **exactas** de todos los prefijos del dominio propio	Grande: $O(n)$
T_agg (agregados)	Los superprefijos que este router **anuncia hacia afuera**	Chica: $O(g)$, con $g \ll n$
T_ext (externos)	Los superprefijos de los dominios vecinos	Chica: $O(d)$

```
┌──────────────────────────────────────────────────────────┐
│  Tabla T_AGG, formato de una entrada:                    │
│                                                          │
│   prefijo_agregado   →  siguiente_salto  (interfaz)      │
│   10.0.0.0/8          →  10.0.0.2        eth0            │
│   172.16.0.0/12      →  10.0.0.3        eth1            │
│   192.168.0.0/16     →  10.0.0.4        eth2            │
│                                                          │
│  Y ENCIMA, una entrada que ES el comportamiento de       │
│  rejección, que es lo que hace que todo esto sea         │
│  jerárquico y no una mera etiqueta:                      │
│                                                          │
│   DEFAULT            →  10.0.0.9        eth0   (al AS vecino)│
└──────────────────────────────────────────────────────────┘
```

El formato de la entrada de tabla (esto es lo que la cátedra corrige en los problemas de tablas):

```
Red Destino (X.Y.Z.W/N)  |  Siguiente Salto  |  Métrica  |  Interfaz

· Red Destino: SIEMPRE en notación CIDR, nunca "clase A".
· Siguiente Salto: dirección IP del router, o la interfaz si es
  un enlace punto a punto sin direccionar.
· Métrica: en jerarquía suele ser el CONTADOR DE GRUPOS atravesados,
  no los hops. Es la métrica "distance to boundary router".
```

La regla de decisión, en pseudocode, que es lo que el enunciado pide "implementar":

```
/* ============================================================
   Rutina de decisión de una red de datagramas con ruteo jerárquico.
   El orden de las pruebas es lo importante: primero el dominio
   propio (más específico), después los agregados, al final el
   default. Es longest-prefix match, y en jerarquía es el que hace
   que "todo lo que empieza con 10." no tape los prefijos internos.
   ============================================================ */

struct entrada {
    char  prefijo[16];      /* "10.0.0.0/8", "0.0.0.0/0"        */
    int   longitud_mascara; /* 8, 12, 16, 0                     */
    char  siguiente_salto[16];
    int   interfaz;
    int   grupos;           /* métrica: nº de dominios atravesados */
};

struct tabla {
    struct entrada T_intro[ MAX_INTRO ];   /* Prefijos del dominio  */
    int            n_intro;
    struct entrada T_agreg[ MAX_AGREG ];   /* Lo que anuncio afuera */
    int            n_agreg;
    struct entrada T_extern[ MAX_EXTERN ];  /* Superprefijos vecinos  */
    int            n_extern;
};

int rutear(struct tabla *t, char ip_destino[])
{
    mejor = { .prefijo = "0.0.0.0/0", .siguiente_salto = NULL,
              .longitud_mascara = -1 };        /* el default, SIEMPRE */

    /* 1. Dominio propio: prefijo más largo que coincida */
    for (i = 0; i < t->n_intro; i++) {
        e = &t->T_intro[i];
        if (coincide(ip_destino, e->prefijo, e->longitud_mascara))
            if (e->longitud_mascara > mejor.longitud_mascara)
                mejor = *e;
    }

    /* 2. Agregados que anuncia este dominio */
    for (i = 0; i < t->n_agreg; i++) {
        e = &t->T_agreg[i];
        if (coincide(ip_destino, e->prefijo, e->longitud_mascara))
            if (e->longitud_mascara > mejor.longitud_mascara)
                mejor = *e;
    }

    /* 3. Superprefijos de los dominios vecinos */
    for (i = 0; i < t->n_extern; i++) {
        e = &t->T_extern[i];
        if (coincide(ip_destino, e->prefijo, e->longitud_mascara))
            if (e->longitud_mascara > mejor.longitud_mascara)
                mejor = *e;
    }

    /* 4. Si no coincidió nada y no hay default: se descarta */
    if (mejor.siguiente_salto == NULL)
        return DESCARTAR;

    return enviar(mejor.interfaz, mejor.siguiente_salto, ip_destino);
}
```

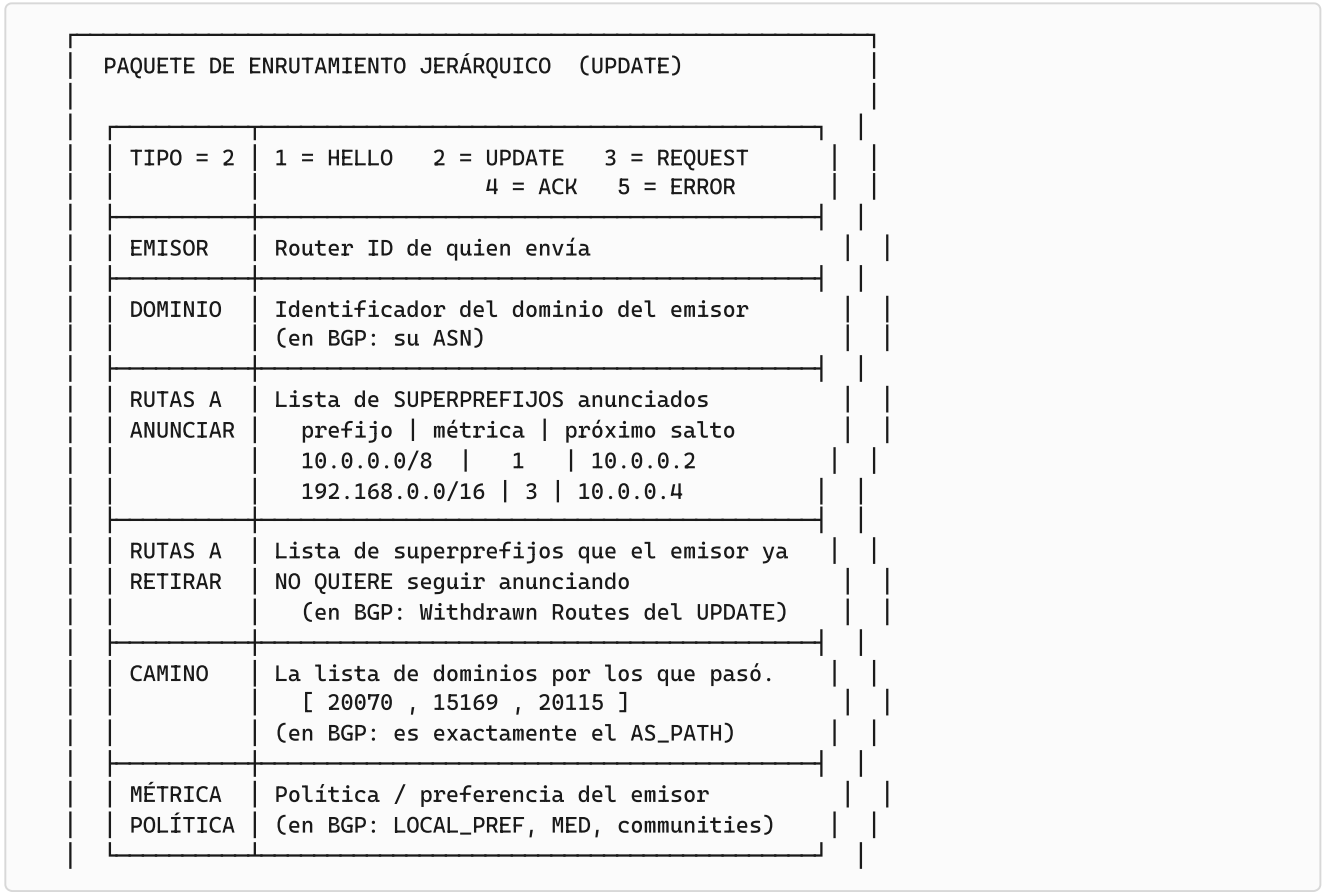
Por qué el orden importa: si el default (`/0`) se evaluara **primero**, el `longest prefix match` lo elegiría siempre, porque `0` es el prefijo más corto. El `if (e->longitud_mascara > mejor.longitud_mascara)` es lo que resuelve todo: **gana siempre el prefijo más específico**, sin importar el orden de recorrido.

c) Los paquetes del protocolo

Un protocolo jerárquico necesita **cuatro** tipos de mensaje, y esta taxonomía es exactamente la de BGP:

Paquete	Función	Análogo en BGP
HELLO	Descubrir vecinos y mantener la relación viva	**OPEN** + **KEEPALIVE**
UPDATE	Anunciar o retirar un **agregado** (con su métrica y el camino)	**UPDATE**
REQUEST	Pedir el estado completo de un dominio vecino, para el arranque	**UPDATE** con todas las rutas
ACK / ERROR	Confirmar o reportar un error (prefijo duplicado, incoherencia)	**NOTIFICATION**

El paquete UPDATE, que es el que define el protocolo, y que hay que dibujar:



Y la regla que cierra el circuito con el protocolo real, y que es el corazón de §1.4:

```
Al recibir un UPDATE, el dominio receptor:

  1. RECHAZA la ruta si el CAMINO contiene su propio identificador
     de dominio.                    ← prevents bucles, exacto

  2. RECHAZA la ruta si ya tiene, para ese prefijo, una ruta cuyo
     camino sea MENOR número de dominios.   ← el "camino más corto"

  3. Si no, la ACEPTA, la preprendea con su identificador de
     dominio y la anuncia a sus vecinos.

El paso 1 es el que hace que el sistema sea ACÍCLICO por
construcción. Y es el mismo paso que BGP llama "el AS_PATH no
contiene mi ASN".
```

d) Análisis comparativo con OSPF (el remate que suma puntos)

	Jerarquía tipo BGP (este problema)	Jerarquía tipo OSPF (áreas)
Qué se publica	La **ruta agregada**	La **información de enlaces**
Qué se oculta	La topología interna	El camino, hasta que se calcula
Métrica	Número de dominios atravesados, **más política**	Costo de interfaz
Pérdida de información	**Ninguna**: la agregación es exacta	**Se pierde optimalidad intra-área**
Cálculo	No hay cálculo: se copia la decisión	Hay **SPF en cada router**
Escala	**Internet** (~70.000 dominios)	Red corporativa, backbone de operador

El remate que hay que escribir para cerrar los 30 puntos: *"El jerarquizado por agregación no pierde información: si `10.0.0.0/8` está detrás de `10.0.0.2`, el camino más corto a cualquier prefijo de ese bloque es exactamente el mismo. Por eso el protocolo jerárquico no necesita calcular nada: el cálculo se hizo una vez, en el borde, y el resultado es correcto para todo el bloque. El jerarquizado por ocultamiento de información, en cambio, obliga a recalcular en cada nodo y acepta perder optimalidad a cambio de escalabilidad. Son dos respuestas al mismo problema con compromisos opuestos."*

3.2 Examen 2007-02, Problema 2 — El enrutamiento hacia Internet con ruta por defecto

> **Fuente:** Examen `eirc0702`, julio de 2007, apartado e) iii. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0702.pdf **Texto del examen (apartado e):**
>
> > iii) El enrutamiento hacia Internet se realiza utilizando "rutas por defecto". Indique qué entrada/s debe agregar a cada tabla de enrutamiento de cada router.
>
> ⚠ **Nota:** la parte numérica depende de la figura de la red corporativa, que no se reproduce acá; lo que se resuelve es la **estructura** de la entrada de ruta por defecto y su relación con BGP, que es lo que la pregunta realmente apunta. Es *(solución propia)* para la parte estructural.

a) La forma de la entrada. En la notación de la cátedra:

```
Red Destino             Siguiente Salto    Métrica    Interfaz
-------------------------------------------------------------
0.0.0.0/0               <ISP>              <saltos>    <puerto>
```

Por qué `0.0.0.0/0` y no "default": es el prefijo que coincide con **todas** las direcciones, con longitud de máscara **0**. Como el forwarding es *longest prefix match*, `0.0.0.0/0` es el prefijo **menos específico que existe**: gana contra cualquier otro prefijo, siempre que ningún otro coincida. Por eso se la pone **al final** de la tabla.

b) Dónde va cada entrada. La respuesta completa a "qué entrada debe agregar en cada router" es:

Router	Entrada	Justificación
R1 (el que tiene la interfaz al ISP)	`0.0.0.0/0 → ISP, métrica 1`	Es el único que ve al ISP, y ve al ISP **directamente**: 1 hop
Todos los demás	`0.0.0.0/0 → R1, métrica 2` (o la que corresponda)	No ven al ISP: lo alcanzan vía R1

Justificación: la métrica es la **cantidad de saltos** hasta el router de borde, porque para ir a Internet **todos los caminos sirven igual**. No hay ningún destino específico que requiera un camino mejor. Ese es el sentido de una **ruta por defecto**: no afirma que exista un camino óptimo, afirma que **no importa cuál**.

c) El error clásico que hay que corregir. Un router **no** puede tener una ruta por defecto que apunte a sí mismo. Si `R1` tuviera `0.0.0.0/0 → R1`, un paquete con destino desconocido se le mandaría a sí mismo, loop **infijo**, y como la ruta por defecto nunca expira, **no se detectaría nunca**. Por eso el `Siguiente Salto` de un default tiene que ser **un router alcanzable por un camino no por defecto**.

d) Y cómo se relaciona con BGP — que es el puente hacia esta clase.

```
El ejemplo (e) es "ruteo hacia Internet con ruta por defecto".
La versión real de BGP hace lo mismo pero CON POLÍTICA:

  ┌──────────────────────────────────────────────────────┐
  │  El router de borde NO anuncia "0.0.0.0/0 a todos".  │
  │  Annuncia 0.0.0.0/0 a los clientes y solo a los peers,│
  │  y NO lo anuncia a los proveedores: "no te doy tránsito│
  │  por mí, andá a buscarlo".                           │
  │                                                      │
  │  Esa decisión se toma con LOCAL_PREF, con communities │
  │  (NO_EXPORT) y con la selección de DEFAULT ROUTE.    │
  └──────────────────────────────────────────────────────┘

O sea: la ruta por defecto es el caso DEGRADADO del proceso de
decisión de BGP. En BGP, 0.0.0.0/0 es "la última entrada", y se
aplica a todo lo que no ForwardMatcheó con una ruta más específica.
Lo que cambia no es el mecanismo: es que hay ~70.000 defaults
simultáneos y cada uno tiene SU política.
```

3.3 ⭐ Examen 2009-03, Pregunta 2 (10 puntos) — jerarquía, IGP con jerarquías y *hot-potato*

> **Fuente:** Examen `eirc0903` / `sirc0903` — 20 de febrero de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0903.pdf **Texto del examen:**

> *a) Explique el concepto de enrutamiento jerárquico. b) Describa un protocolo de enrutamiento interno (IGP) que utiliza jerarquías. c) ¿En qué consiste el mecanismo de "hot-potato routing"?*
>
> *(La solución publicada en `sirc0903` remite a las clases de teórico de Capa de Red para los tres apartados. Es el problema de BGP del corpus de FING, y vale 10 puntos.)*

a) Enrutamiento jerárquico. El §3.1 lo desarrolla con `2001-08 P2`. En BGP la jerarquía son los **AS con superprefijos agregados**: en vez de anunciar `10.1.2.0/24` y `10.1.3.0/24` por separado, se anuncia **un solo superprefijo** `10.0.0.0/8`, y los routers externos **no necesitan conocer la topología interna**. La optimalidad global **no se pierde**, porque la agregación es correcta por definición.

⚠️ **El contraste que hay que dar** (es lo que separa un 6 de un 10): OSPF jerarquiza **para escalar el cálculo** y BGP jerarquiza **para escalar el anuncio**. Los dos resuelven el mismo problema con herramientas **opuestas**: OSPF publica los **hechos** y esconde el camino; BGP publica el **camino** y esconde los hechos.

b) Un IGP que utiliza jerarquías. La respuesta de la cátedra es **OSPF** (`RFC 2328`), y su jerarquía son las **áreas**: un **ABR** (*Area Border Router*) anuncia **LSA de tipo 3** (summary) y **de tipo 4** hacia las otras áreas, y esos LSA **no se inundan** dentro del área destino. Por eso un cambio en un área **no obliga a recalcular** en las otras. Alternativa: **IS-IS**, con *Level 1* (dentro del área) y *Level 2* (el backbone).

⚠️ **El detalle que liga con la parte a):** el IGP jerarquiza **por dentro** del AS; el EGP jerarquiza **por fuera**. Son dos jerarquías **perpendiculares**, y por eso en un operador hay que configurar las dos.

c) Hot-potato routing. El §1.8 lo desarrolla con números. La respuesta de examen:

> **Un router prefiere reenviar el tráfico por el vecino eBGP cuya métrica IGP interna sea MENOR**, aunque el camino total sea más largo. Es un mecanismo de **pasame la papa caliente**: el router quiere **quedarse con el paquete el menor tiempo posible**. Se implementa en el **paso 7 de la tabla de decisión** (la métrica IGP hacia el `NEXT_HOP`), es decir, **el último antes del Router ID**.

⚠️ **Los tres puntos que hay que sumar:**

1. **Es el ÚLTIMO desempate, no el primero.** Precedido por `WEIGHT`, `LOCAL_PREF`, origen local, longitud de `AS_PATH`, `ORIGIN`, `MED` y eBGP-vs-iBGP. Un AS **nunca** prefiere "la ruta más cercana en hops" por sobre "la de menos ASes de por medio".
2. **Se puede anular con** `MED` (dentro del mismo AS vecino) o con `LOCAL_PREF` **/** `COMMUNITY` (entre vecinos distintos). La tabla de §2.1 lo demuestra numéricamente.
3. **Por qué existe:** el router quiere **sacarse el paquete de encima** lo más rápido posible, porque el costo interno de su red es gratis para él y el del vecino es problema del vecino. El hot-potato es la traducción de ese comportamiento egoísta a la tabla de decisión.

3.4 Nota: qué hay de BGP en el corpus de exámenes

Verificado sobre todos los archivos de `material-oficial/examenes/txt`: `sirc0903` **P2 (10 pts)** es la pregunta de BGP del corpus (§3.3); `eirc0108` **Problema 2 (30 pts)** es el de **enrutamiento jerárquico**, la otra mitad de BGP (§3.1); `eirc0702` **Problema 2, apartado e** es enrutamiento hacia Internet con **rutas por defecto** (§1.2 y §1.5); y `sirc0208` / `sirc0812` la mencionan de pasada.

Qué preguntar en el examen entonces, y cómo se responde:

Si el enunciado dice...	La respuesta que suma puntos
"explique el enrutamiento entre redes"	**Autonomous System** + agregación de prefijos + **política** + **path-vector**. Es el de `2001-08 P2`.
"explique por qué Internet escala"	Porque está **jerarquizado** en ~70.000 ASes que anuncian **agregados**, no rutas individuales. §3.1.
"explique el hot-potato routing"	Salir por el eBGP de **menor métrica IGP**; es el **paso 7** (último antes del Router ID). §1.8 y §3.3.
"diferencie RIP, OSPF y BGP"	La tabla de §1.11, con las columnas **qué viaja**, **qué algoritmo**, **qué transporte**, **qué escala**.
"qué es un sistema autónomo"	Conjunto de prefijos + routers con **una única política de ruteo** y un **identificador (ASN)**.
"por qué BGP no tiene conteo a infinito"	Porque lo que viaja es un **registro de hechos** (`AS_PATH`), no una **estimación derivada**. Rechazo exacto, no heurístico. §1.4.
"cómo se protege BGP"	**No cifra ni autentica.** `RFC 2385` (MD5) es **legado**; hoy el mecanismo de referencia son los **roles BGP de** `RFC 9234` ; **RPKI/ROA** (`RFC 6810` / `RFC 6811`) valida el **origen del prefijo**. Y el **route leak** es el fallo más común. §1.10.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Internet es una red de redes con ~70.000 sistemas autónomos** (medilo vivo), y no hay un grafo único: hace falta **política**, no solo camino más corto.
2. El **AS** es un dominio **administrativo** (una sola política de ruteo) y el **ASN** su identificador. Hoy son **32 bits** (`RFC 4893` + `RFC 6793`), con notación decimal `RFC 5396`.
3. **BGP-4 (`RFC 4271`)** corre sobre **TCP/179** y es **vecino a vecino**: no hay flooding, no hay base común, cada router decide con su propia política.
4. **Path-vector** significa que lo que viaja es el `AS_PATH`, no una distancia. La regla es una: **no aceptar una ruta cuyo AS_PATH contenga mi propio ASN**.
5. Eso hace que BGP **no cuente a infinito y no necesite split horizon**: la detección es **exacta**, no probabilística. Pero **sí** propaga errores de configuración a toda la Internet.
6. El **handshake** es `OPEN` → `OPEN` + `KEEPALIVE` → `UPDATE` → `KEEPALIVE` cada 60 s, con hold time de 180 s, a través de los estados `Idle`, `Connect`, `Active`, `OpenSent`, `OpenConfirm`, `Established`.
7. El **proceso de decisión** desempata en orden fijo: `WEIGHT` → `LOCAL_PREF` → origen local → longitud de `AS_PATH` → `ORIGIN` → `MED` → eBGP/iBGP → métrica IGP al `NEXT_HOP` → Router ID.
8. La regla **"no re-anuncio a iBGP lo aprendido de un iBGP"** obliga al **full-mesh** (`N(N−1)/2` sesiones). **Route Reflection (`RFC 4456`)** lo baja a `O(N)`, y se protege con `ORIGINATOR_ID` y `CLUSTER_LIST`.
9. **"Hot potato"**: un AS sale por el vecino más cercano porque el último tramo lo paga él. `MED` (menor gana, comparable **solo entre rutas del mismo AS vecino**) y `LOCAL_PREF` son las dos herramientas.
10. **BGP no cifra ni autentica.** MD5 (`RFC 2385`) es legado; hoy son los **roles BGP de `RFC 9234`**; **RPKI/ROA** (`RFC 6810` / `RFC 6811`) valida el origen del prefijo. El fallo más común es el **route leak**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué en el nivel inter-AS no se puede correr Dijkstra?

<details>
<summary>Respuesta</summary>
</details>

2. ¿Qué es un Sistema Autónomo y qué lo define?

<details>
<summary>Respuesta</summary>
</details>

3. Explique por qué BGP no cuenta a infinito, y por qué eso es cualitativamente diferente de las soluciones de distance-vector.

<details>
<summary>Respuesta</summary>
</details>

4. Escribí el orden completo de los atributos de decisión de BGP.

<details>
<summary>Respuesta</summary>
</details>

5. Explique el "hot potato" y cómo lo corrigen `MED` y `LOCAL_PREF`.

▶ Respuesta

6. ¿Por qué BGP necesita full-mesh iBGP, y qué rompe Route Reflection?

▶ Respuesta

7. ¿Por qué `MED` solo se compara entre rutas del mismo AS vecino?

▶ Respuesta

8. ¿Por qué BGP es inseguro y qué hace cada mecanismo de seguridad?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc0108` (20-ago-2001) — Problema 2: *implementar un algoritmo de ruteo jerárquico para una red de datagramas; definir la estructura de las tablas de ruteo y los paquetes utilizados*. Es la idea de fondo de BGP. ⚠ El PDF **no publica solución**; §3.1 es *(solución propia)*.
- **Examen** `eirc0702` (jul-2007) — Problema 2, apartado e: enrutamiento hacia Internet con **rutas por defecto**.
- `eirc0903` **/** `sirc0903` — Pregunta 2 (10 pts): enrutamiento jerárquico, IGP que utiliza jerarquías (OSPF) y **hot-potato routing**. Resuelto en §3.3.
- **Nota:** el corpus no tiene un problema *de programación* sobre BGP, pero sí las preguntas teóricas de `2009-03 P2` y los dos problemas de jerarquías (`2001-08 P2` , `eirc0702 P2e`). Verificado sobre `material-oficial/examenes/txt` .
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.6 "Algorithms for routing": §5.6.4 "BGP: The Border Gateway Protocol" y el *case study* de enrutamiento entre backbone de Internet.
- `RFC 4271` — BGP-4: los cuatro tipos de mensaje, la máquina de estados, el proceso de decisión y el `AS_PATH` .
- `RFC 4456` — BGP Route Reflection: `ORIGINATOR_ID` , `CLUSTER_LIST` , reglas de no reenvío.
- `RFC 1997` — BGP Communities (`NO_EXPORT` , `NO_ADVERTISE` , `NO_EXPORT_SUBCONFED`) y el formato `AS:value` .
- `RFC 4893` — el problema del AS de 2 octetos · `RFC 6793` — el Capability de 4 octetos · `RFC 5396` — notación decimal `asdot` / `asplain` .
- `RFC 2385` — MD5 para BGP: **el mecanismo clásico, hoy legado** · `RFC 9234` (2022) — **roles BGP**, reemplazo de referencia.
- `RFC 6810` — RTR (RPKI to Router) · `RFC 6811` — BGP Prefix Origin Validation (ROA).
- `RFC 7854` — BMP, monitoreo de BGP desde fuera.
- `RFC 9552` — BGP-LS (dic-2023), obsoleta `RFC 7752` · `RFC 7432` — EVPN · `RFC 7348` — VXLAN · `RFC 8402` , `RFC 8660` , `RFC 8986` — Segment Routing / SRv6.
- **Clase anterior:** 33 — Enrutamiento link-state, Dijkstra y OSPF

- **Clase siguiente:** 35 — Fragmentación, MTU y path MTU

Clase 35

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 MTU: la unidad máxima de transmisión
- 1.2 Fragmentación: los tres campos que la habilitan
- 1.3 El comportamiento del router — lo que pide el Examen 2010-12
- 1.4 El problema de la fragmentación
- 1.5 El algoritmo de reensamblaje
- 1.6 Path MTU: el mínimo del camino
- 1.7 Path MTU Discovery (`RFC 1191`)
- 1.8 MTU y MSS: la defensa de TCP
- 1.9 El PMTUD black hole
- 1.10 Actualización a 2026
- 2.1 Ejercicio 1 — ¿Cuántos fragmentos genera un datagrama de 1500 bytes por un enlace de MTU 500?
- 2.2 Ejercicio 2 — Reensamblar 4 fragmentos que llegan desordenados
- 2.3 Ejercicio 3 — PMTUD con búsqueda binaria
- 3.1 ⭐ Examen 2010-12, Problema 2 (30 puntos) — MTU, fragmentación y Path MTU
- 3.2 Examen 2006-12, Pregunta 3 (10 puntos) — fragmentación y reensamblaje
- 3.3 Repaso: `2000-07` Problema 2
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: D (Capa de Red) · **Semana:** 12 · **Duración estimada:** 120 min **Correlativa:** Clase 34 (BGP y el routing entre operadores) · **Sigue con:** Clase 36 (IPv6) **Material de referencia:** Tanenbaum cap. 5 (§5.5.2) · RFC 791, RFC 1122, RFC 1191, RFC 8200 §5, RFC 8899

🎯 Al terminar esta clase vas a poder

1. Definir **MTU**, **fragmentación** y **Path MTU**, y explicar por qué el Path MTU es el **mínimo** de los MTUs del camino.
2. Dibujar la cabecera IPv4 señalando los **tres campos** de la fragmentación: `Identification`, `Flags` (con `DF`) y `Fragment Offset`.
3. **Escribir el comportamiento de un router** frente a un paquete que no entra, y justificar por qué **descarta** cuando `DF = 1`.
4. Enumerar los **problemas asociados**: unidad de 8 bytes, overhead, temporizador, solapamiento, pérdida de un fragmento, evasión de filtros.
5. **Implementar el algoritmo de reensamblaje** con índice por offset, detección de solapamiento y temporizador.
6. **Implementar el descubrimiento del Path MTU** con `enviar_msg_ICMP` / `recibir_msg_ICMP`, y explicar por qué la solución oficial subestima.
7. Explicar qué es el **PMTUD black hole** y diagnosticarlo con `ping -M do`.

> **Prerequisito del examen:** `2010-12 Problema 2` (30 pts — MTU, fragmentación, Path MTU **e implementación del descubrimiento**), `2006-12 P3` (10 pts — **algoritmo de reensamblaje**), `2000-07 Problema 2`.
>
> ⚠️ Es **una de las clases más importantes del curso**: el problema de 2010-12 vale 30 puntos y su último apartado es un algoritmo a implementar.

PARTE 1 — TEORÍA

1.1 MTU: la unidad máxima de transmisión

MTU (*Maximum Transmission Unit*) es **la mayor PDU (unidad de datos de protocolo) de capa 3 que un enlace transmite sin fragmentar**.

Existe porque la física lo impone: a velocidad `R`, transmitir `L` bytes lleva `L/R` segundos, así que un datagrama de 64 KB monopolizaría el enlace medio segundo; además la memoria de la interfaz y el tamaño máximo de trama del hardware son finitos.

Ojo con una fuente típica de errores: la MTU cuenta solo los datos, no la cabecera de capa 2.

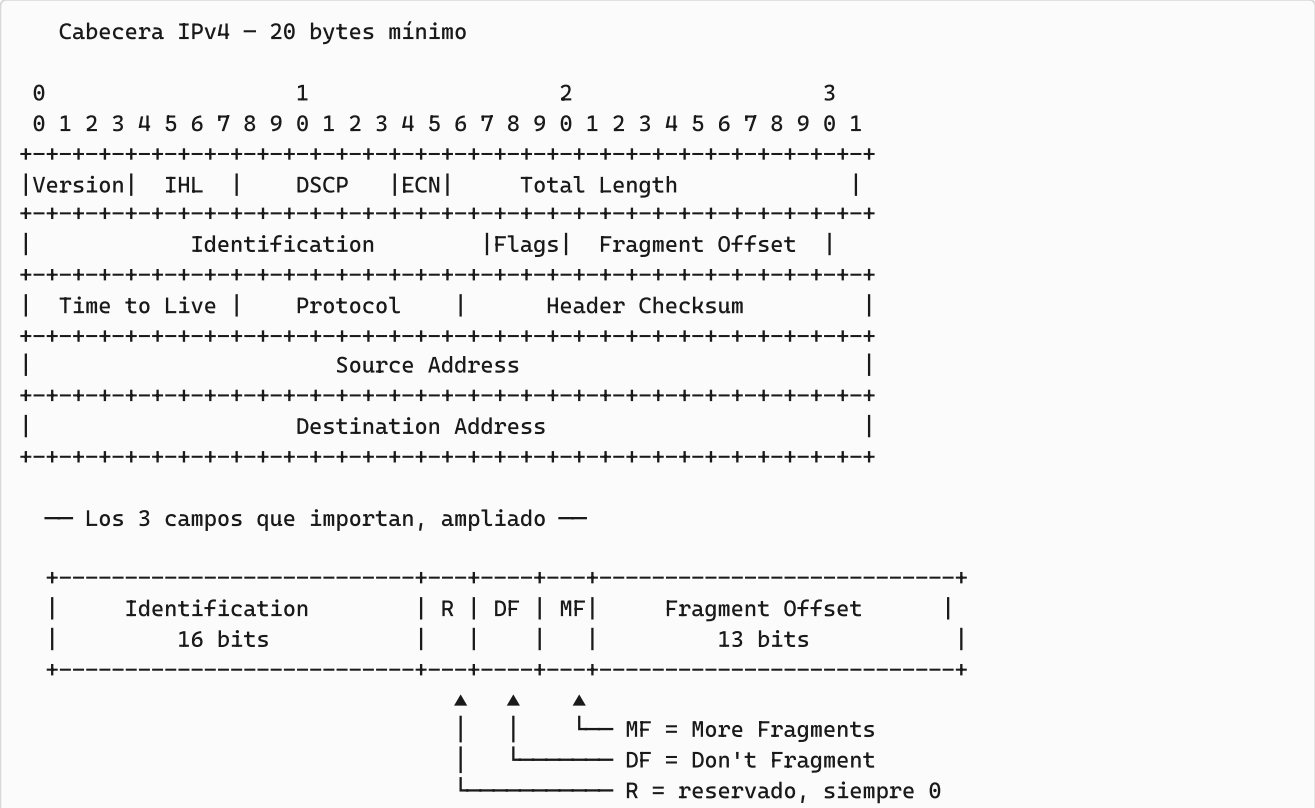
```
Trama Ethernet máxima:  1518 bytes
   ├─ MAC destino 6 + MAC origen 6 + Ethertype 2 = 14
   ├─ DATOS IP       1500   ◄── ESTO es la MTU
   └─ FCS              4
──────────────────────────────────  14 + 1500 + 4 = 1518
```

Enlace	MTU	Origen del número
Ethernet	**1500**	valor por defecto de IEEE 802.3
Jumbo frame	**9000**	**convención, NO estándar IEEE**
PPPoE (ADSL)	**1492**	1500 menos 8 de encapsulación
Frame Relay / ATM	Variable	según VC; típico 1500
Token Ring 16 Mbps	17918	el estándar lo fija explícitamente
IPv4 mínimo absoluto	**68**	`RFC 791` ; en la práctica se/supporta 576
IPv6 mínimo absoluto	**1280**	`RFC 8200 §5`, verificado textualmente

> ⚠️ El **jumbo de 9000 es una convención de la industria, no un estándar IEEE**. Se usa intra-datacenter y en almacenamiento. **Nunca sobre Internet ni sobre túneles sin *clamp MSS***: si un router del camino tiene MTU 1500 y vos mandás 9000, o fragmentás (IPv4) o perdés la conexión (IPv6, que no fragmenta).

El **68 bytes** es el datagrama mínimo absoluto de `RFC 791` (60 de cabecera con opciones + 8 de datos), pero `RFC 1122 §3.2.1` pide que **todo host soporte al menos 576 bytes**. El 68 es un piso histórico; nadie baja de 576 en una red real. El mínimo de IPv6, en cambio, es **1280 y obligatorio para toda interfaz**, sin excepción por opciones.

1.2 Fragmentación: los tres campos que la habilitan



Campo	Bits	Función
Identification	16	El **"número de pedido"** del datagrama. **Todos los fragmentos llevan el mismo ID.** El destino agrupa por `(IP destino, ID)`.
Flags: DF	1	**Decide si se puede fragmentar.** `0` = se puede; `1` = prohibido: si no entra, se descarta y se avisa.
Flags: MF	1	`1` = hay más fragmentos; `0` = **este es el último**.
Fragment Offset	13	Posición del payload de este fragmento dentro del payload original, **en unidades de 8 bytes**.

Por qué 8 bytes y no 1: economía de bits. 13 bits en unidades de bytes darían solo 8192 bytes de payload máximo. En unidades de 8 dan 8191 × 8 = **65528 bytes**, que cubre cualquier datagrama IPv4 válido. La consecuencia es dura: **el payload de todo fragmento salvo el último debe ser múltiplo de 8**, así que si el router tiene 556 bytes disponibles, recorta a 552 y deja 4 para el siguiente.

Ejemplo. Datagrama de 1500 bytes (`IHL = 5`, payload 1480) por un enlace de MTU 576:

```
Original:  [cabecera 20 | payload 1480]                    longitud = 1500

F1: cabecera 20 | payload 552                             longitud =  572
    offset = 0/8 = 0          MF = 1
F2: cabecera 20 | payload 552                             longitud =  572
    offset = 552/8 = 69       MF = 1
F3: cabecera 20 | payload 1480-1104 = 376                 longitud =  396
    offset = 1104/8 = 138     MF = 0
──────────────────────────────────────────────────────────────────────
Sobre el enlace: 572 + 572 + 396 = 1540   →  overhead 40 bytes (2,7 %)
Todos con el MISMO Identification, mismo src y mismo dst.
```

Reglas de memoria: el `Fragment Offset` va en unidades de 8; el `ID` se copia tal cual; el **TTL se restaura al valor original** (los fragmentos viajan de a uno, no en paralelo); el **checksum de cabecera se recalcula** porque la cabecera cambió; el **checksum de capa 4 no cambia** (se verifica recién al reensamblar); y **cada fragmento es un datagrama IP independiente**, así que puede tomar otro camino si hay ECMP.

1.3 El comportamiento del router — lo que pide el Examen 2010-12

Este pseudocode es **literalmente** el apartado (b) del problema de 30 puntos:

```
// ---- Comportamiento de un router al reenviar por un enlace de MTU MTUo ----
//   Pi    = paquete entrante
//   L     = longitud total de Pi (campo Total Length)
//   MTUo  = MTU del enlace saliente

Pi = paquete entrante ;  L = Pi.Total_Length

if (L <= MTUo)                      →  // CASO 1: cabe entero
    transmitir Pi normalmente                       // sin importar DF

else if (L > MTUo and Pi.DF == 1)   →  // CASO 2: no cabe y NO se puede partir
    DESCARTA Pi
    envía ICMP (destino = Pi.origen, tipo = 3, código = 4)
    //   tipo 3 = Destination Unreachable
    //   código 4 = "Fragmentation Needed and Don't Fragment was Set"

else if (L > MTUo and Pi.DF == 0)   →  // CASO 3: no cabe y se PUEDE partir
    FRAGMENTAR Pi
    transmitir cada fragmento
```

Por qué el orden importa: se chequea primero si cabe, porque es el caso más frecuente y el más barato (ni hace falta mirar los flags).

¿Por qué descarta con `DF = 1` en vez de fragmentar igual? Porque `DF` significa "no me fragmentes": el emisor **declaró un límite que hay que respetar**, y un router que lo fragmentara produciría algo que el receptor final **no sabe tratar** (una aplicación entrega datos a un socket, no a una bolsa de fragmentos).

¿Por qué manda el ICMP? Tres razones, y las dos primeras son los puntos:

- **Para que el emisor no repita el error.** IP es *best effort*: si nadie avisa, el emisor solo ve pérdida y **retransmite el mismo datagrama idéntico**, que vuelve a ser descartado. Bucle infinito.
- **Para que el emisor se adapte.** El tipo 3 código 4 **lleva la MTU exacta** del enlace bloqueante en los **bytes 6-7** (`RFC 1191`), así que puede bajar su MSS o su caché de PMTU.
- **Para que la red sea observable.** Sin reportes, el operador nunca se enteraría de que su MTU está mal.

⚠️ **Riesgo:** un atacante que inyecte ICMP tipo 3 código 4 falsos puede degradar el servicio. Por eso los routers hoy **no emiten ICMP hacia multicast** y muchos limitan la tasa de errores que generan.

1.4 El problema de la fragmentación

Este es el corazón conceptual del `2006-12 P3`. La fragmentación se paga cara:

(a) Unidad mínima de 8 bytes. Implica el recorte descrito en §1.2. Si un enlace deja pasar solo 8 bytes de payload por fragmento, se generan muchísimos fragmentos.

(b) Overhead de cada cabecera. Cada fragmento repite los 20 bytes de cabecera IP:

```
Payload real = 1500 − 20 = 1480 bytes.  Si entran 8 bytes de payload:
    n = 1480 / 8 = 185 fragmentos
    Sobre el enlace: 185 × 28 = 5180 bytes
    Overhead: 5180 − 1500 = 3680 bytes  →  el datagrama CRECIÓ un 245 %
```

> ⚠️ La cifra de **"188 fragmentos"** que circula en los apuntes sale de `1500/8 = 187,5`. **El cálculo correcto es sobre el payload (1480), no sobre la longitud total**, porque la cabecera se reescribe en cada fragmento. Si el examen te da la longitud total, **descontá `IHL × 4` primero**.

(c) Reensamblaje con temporizador. Un fragmento que llega solo no se puede entregar, así que el destino **guarda estado** y arranca un temporizador (típicamente 60–120 s) al recibir el primero. Si vence sin completarse, **descarta todos los fragmentos y libera la memoria** — y el datagrama **se pierde entero**.

(d) Solapamiento de fragmentos — el agujero de seguridad. Dos fragmentos se solapan si cubren bytes del payload original en común. `RFC 791` dice que ante solapamiento se descarte el datagrama, pero **la implementación real, históricamente, no siempre lo hizo**. El problema es que **el segundo fragmento sobrescribe al primero**: un atacante que inyecte un fragmento con el mismo `(IP destino, ID, offset)` pero datos distintos puede **cambiar el contenido del datagrama**. Es la base de los ataques clásicos de fragmentación (*fragment overlap*, *teardrop*). IPv6 lo cierra: `RFC 5722` prohíbe explícitamente el solapamiento y `RFC 6946` recomienda tratar los **fragmentos atómicos** como no fragmentados.

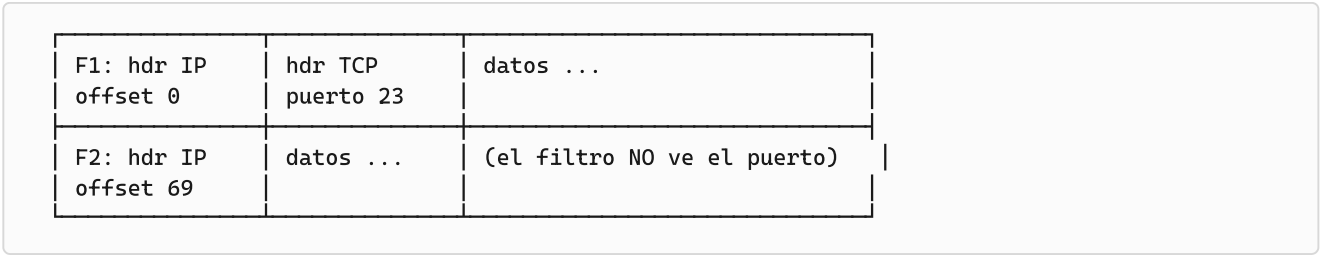
(e) La pérdida de un fragmento descarta el datagrama entero. Y es multiplicativo:

```
Si el enlace pierde con probabilidad p, y el datagrama se partió en n:
    P(que el datagrama ENTERO llegue) = (1 − p)^n

p = 0,01:   n = 3 → 0,99³  = 0,9703  →  97,0 % de éxito
            n = 185 → 0,99^185 = 0,157  →  15,7 % de éxito
```

Interpretación: fragmentar 185 veces multiplicó por 6 la probabilidad de pérdida del datagrama completo. **El datagrama fragmentado tiene una probabilidad de pérdida mucho mayor que uno que no lo está**, aunque el enlace individual sea idéntico.

(f) El ataque de fragmentación en capas — el punto de seguridad.

> **Cada fragmento es un paquete IP independiente para cada firewall o filtro que atraviesa.** Un filtro que aplica reglas sobre campos del datagrama **solo ve cada fragmento por separado**.



Un atacante **fragmenta el datagrama para esconder un puerto prohibido en un fragmento que el filtro no inspecciona**. Y hay una capa más: fragmentando, el primer fragmento lleva solo los 8 bytes obligatorios de payload, y **los datos del protocolo quedan en los fragmentos posteriores**, donde un filtro *stateful* que hace tracking de conexiones no los asoció. **Por eso los firewalls modernos reensamblan antes de decidir**, y por eso en IPv6 — donde el router nunca fragmenta — este ataque es estructuralmente mucho más difícil.

1.5 El algoritmo de reensamblaje

La solución oficial de `2006-12` es un esqueleto; esta es la versión completa. **Suposiciones que hago, justificadas** (el enunciado pide justificarlas):

1. **Los fragmentos pueden llegar en cualquier orden y más de una vez.** IP es *best effort* y no reordena.
2. **Se agrupan por el triple `(IP destino, Identification, Protocolo)`.** Es lo único que el destino tiene para saber a qué datagrama pertenece un fragmento.
3. **Hay que detectar solapamiento**, porque si no un fragmento puede sobrescribir bytes ya entregados.

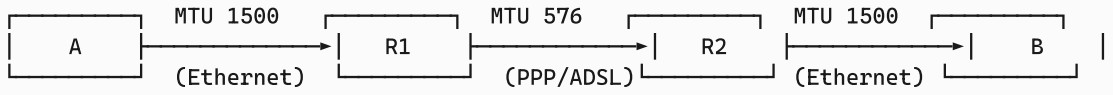
```
// ======== ESTADO POR DATAGRAMA EN CURSO ========
TablaEnsamblaje[ (IP_destino, ID, protocolo) ] = {
    buffer[ fragment_offset ] , ocupado[ fragment_offset ] ,
    cabecera_original , longitud_total = -1 , TTL_assembly
}

AL_RECIBIR_DATAGRAMA( Pi ):
    E = TablaEnsamblaje[ (Pi.destino, Pi.Identification, Pi.protocolo) ]
    if (E no existe): crear E vacía ; E.TTL = ahora() + 120 s

    // (0) el fragmento de offset 0 lleva la CABECERA ORIGINAL (con sus
    //     opciones). Hay que guardarla aparte: el Total_Length de P no es
    //     el del datagrama original.
    if (Pi.Fragment_Offset == 0): E.cabecera_original = Pi

    // (1) longitud mínima del payload
    if (payload(Pi) < 8 and Pi.MF == 1): descartar Pi ; return

    // (2) SOLAPAMIENTO  →  política conservadora: se descarta todo
    inicio = Pi.Fragment_Offset × 8 ;  fin = inicio + payload(Pi)
    for cada offset o en E.ocupado:
        if (o ≠ Pi.Fragment_Offset) and los intervalos se cruzan:
            descartar Pi ; liberar E ; return

    // (3) almacenar EN SU POSICIÓN
    E.buffer[ Pi.Fragment_Offset ] = Pi ;  E.ocupado[ Pi.Fragment_Offset ] = true
    E.TTL = ahora() + 120 s

    // (4) si es el ÚLTIMO, ya sabemos el largo total del payload
    if (Pi.MF == 0):
        E.longitud_total = (Pi.Fragment_Offset × 8) + payload(Pi)

    // (5) ¿COMPLETO?  No por MF, por COBERTURA TOTAL
    if (E.longitud_total ≠ -1) and todos_los_bytes_cubiertos(E, E.longitud_total):
        Ptot = reensamblar(E)          // arma el datagrama original
        Ptot.TTL = Pi.TTL              // el menor TTL de los fragmentos
        entregar Ptot a la capa 3 ;  liberar E ;  return

// ======== BARRIDO PERIÓDICO (no corre por fragmento recibido) ========
for cada E en TablaEnsamblaje:
    if (ahora() > E.TTL):
        descartar todos los fragmentos de E      // el datagrama se pierde entero
        liberar E
```

El concepto del paso 5, que es lo que separa "andar" de "entender": el datagrama **no está completo cuando llegó el último fragmento**, sino cuando **todos los bytes del intervalo `[0, longitud_total)` están cubiertos sin huecos**. Con 3 fragmentos que llegan en orden los dos hechos coinciden; con 5 que llegan desordenados **no**:

```
CASO en que "el último" NO implica "completo":
   llega F5 (el último, MF = 0, offset 2208, payload 424)
   estado real: hay un solo fragmento de 5 → falta el 80 % del datagrama
   Un algoritmo "reensamblo al ver MF = 0" entrega BASURA.

CASO en que sí coincide (por casualidad):
   F1 ✓  F3 ✓  F4 ✓  F2 ✓  F5 ✓  → llega F5 y en ese instante todo está
   cubierto. Funciona, pero solo por casualidad del orden.
```

1.6 Path MTU: el mínimo del camino

Path MTU es **la MTU del camino completo entre dos extremos: el mínimo de las MTUs de todos los enlaces del camino.**




```
Path MTU de A hacia B = min(1500, 576, 1500) = 576
⇐ El cuello de botella es el enlace del medio, NO los extremos.
```

Tres propiedades que hay que decir siempre, porque el enunciado de 2010-12 las pide explícitamente:

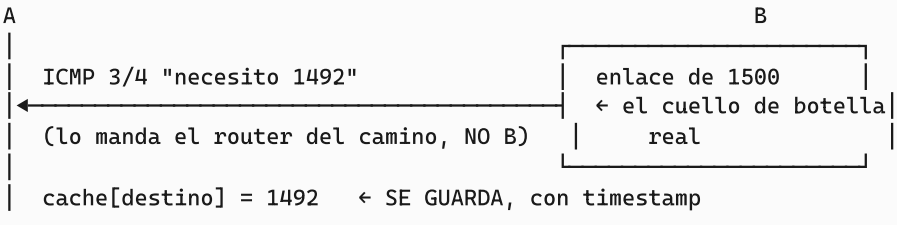
1. **Puede variar con el tiempo** si **los caminos cambian**: con recálculo de rutas (OSPF, IS-IS) o con **ECMP** repartiendo carga. Un PMTU cacheado puede quedarse viejo.
2. **Puede ser distinto en cada sentido** si el **ruteo es asimétrico**. Hay dos caminos y dos PMTUs; es el caso normal en Internet.
3. **Es una propiedad del camino, no del prefijo destino.** Dos flujos al mismo destino pueden tener PMTU distinto si ECMP los manda por enlaces distintos.

La diferencia entre fragmentar y descubrir el PMTU, en una frase: la fragmentación **hace que el paquete llegue**; el descubrimiento **hace que el paquete nunca sea demasiado grande**. La primera gasta ancho de banda y probabilidad de éxito; el segundo no gasta nada.

1.7 Path MTU Discovery (`RFC 1191`)

Tres pasos, y el tercero es el esencial:

```
1. El emisor pone DF = 1 en todos sus datagramas (TCP lo hace por omisión).
2. El router que no puede pasarlo:
       descarta el datagrama
       envía ICMP tipo 3 código 4 con la MTU del enlace en los bytes 6-7
3. El emisor recibe el ICMP, BAJA SU CACHÉ de PMTU para ese destino y
   vuelve a mandar con un tamaño que sí entra.
```



El campo de la MTU dentro del ICMP (cabecera de 8 bytes):

```
┌──────┬──────┬──────────┬──────────┬──────────┐
│ Type │ Code │ Checksum │ "unused" │ Resto…   │
│  8   │  8   │    16    │    16    │          │
└──────┴──────┴──────────┴──────────┴──────────┘
  byte 0  byte 1  bytes 2-3   bytes 4-5    bytes 6-7
                                    ←── AQUÍ va la MTU del enlace
                                        siguiente, en "Fragmentation
                                        Needed and DF set"
```

> `RFC 1191 §3` usa ese campo "unused" para llevar **la MTU del enlace siguiente** en el mensaje *Fragmentation Needed and DF Set*. **Es la respuesta directa al problema:** el router no dice solamente "no entra", dice **"entra hasta acá"**. En la práctica muchos routers no lo llenan, y por eso la solución oficial multiplica por 0.85 (§3.1c).

La degradación elegante — y su falla silenciosa: si el ICMP tipo 3 código 4 **no llega nunca** (un firewall lo filtró, o el router que debía emitirlo está roto), el emisor **no se entera de nada**. El paquete se pierde, se retransmite, se pierde otra vez, y la conexión se cuelga sin explicación. Ese es el **black hole**.

1.8 MTU y MSS: la defensa de TCP

La capa de transporte evita la fragmentación **por su cuenta**, anunciando su **MSS** (*Maximum Segment Size*) en el SYN:

```
MSS = MTU del enlace de SALIDA del emisor − cabeceras de IP y TCP

IPv4:  MSS = MTU − 20 (IP) − 20 (TCP) = MTU − 40   →  1460 en Ethernet
IPv6:  MSS = MTU − 40 (IP) − 20 (TCP) = MTU − 60   →  1420 en Ethernet
```

Dos extremos que conectan por caminos distintos usan `min(MSS_A, MSS_B)`, y con eso ningún segmento puede superar el PMTU. **Los segmentos TCP nunca se fragmentan**: si uno no entra en un enlace intermedio, **se descarta y se retransmite más chico**.

El clamp MSS. Con un jumbo frame (9000) en un borde y un lado que no lo soporta, alguien tiene que **recortar el MSS**:

```
Servidor en red de jumbo:  MSS anunciado = 8960
Cliente detrás de un enlace de 1500:  no puede recibir segmentos de 8960
⇐ El firewall / router de borde hace CLAMP:  MSS = min(8960, 1500-40) = 1460
⇐ Por eso "jumbo frames nunca sin clamp MSS": sin clamp, se rompe.
```

1.9 El PMTUD black hole

Qué es: un router que no puede pasar el datagrama **no genera el ICMP** (porque un firewall lo filtró, porque un ACL no lo contempla, o porque el router está roto). El emisor nunca se entera de que su PMTU cacheado está mal, sigue mandando datagramas de tamaño fijo, y **todos se pierden en silencio**.

El síntoma clásico, de memoria:

> **"SSH funciona y HTTPS se queda colgado"** (o al revés, según qué protocolo sobreviva).

¿Por qué un protocolo funciona y el otro no? Porque **no todos los datagramas tienen el mismo tamaño**:

```
SSH interactivo: paquetes chicos (prompt, teclas), ~100 bytes.
                 100 < 1280  ⇐  pasan por el cuello de botella.
                 ⇐ SSH FUNCIONA

HTTPS: el servidor manda respuestas del tamaño del MSS negociado (1460).
       1460 > 1280  ⇐  el router del cuello lo descarta, sin avisar.
       ⇐ HTTPS SE CUELGA
```

Diagnóstico:

```
# Linux: -M do = forzar DF=1 (don't fragment); -s = tamaño del payload
ping -M do -s 1400 ejemplo.com      # falla  → hay un cuello por debajo de 1400
ping -M do -s 1000 ejemplo.com      # funciona → el cuello está entre 1000 y 1400
tracepath ejemplo.com               # muestra el PMTU de cada salto
# Windows:  ping -f -l 1400 ejemplo.com      (-f = DF, -l = payload)
```

Soluciones, de mejor a peor: (1) **dejar pasar el ICMP tipo 3 código 4** en los firewalls — es lo correcto; (2) **DPLPMTUD** (`RFC 8899`), que **no depende de ICMP**; (3) **MSS clamping** en el borde — muy desplegado; (4) **caché con TTL corto y revalidación**; (5) bajar el MSS del socket a mano — último recurso.

1.10 Actualización a 2026

Sigue igual, de memoria:

- **MTU** = la mayor PDU de capa 3 que el enlace transmite sin fragmentar.
- **Ethernet 1500.** Jumbo 9000.
- **IPv4 fragmenta en los routers (DF = 0); IPv6 fragmenta solo en el origen.**
- **ICMPv4 tipo 3 código 4** vs. **ICMPv6 tipo 2** (*Packet Too Big*).
- **El PMTU se cachea en el origen**, es **por ruta**, y es **≤ al MTU del salto más estrecho**.
- El **"576 mínimo"** en IPv4 viene de `RFC 791` y `RFC 1122 §3.2.1`.
- **IPv6 mínimo absoluto = 1280** (`RFC 8200 §5`).

Cambió:

- `RFC 4821` **(2007, *Experimental*) →** ***`RFC 8899`*** **(sep-2020, Standards Track): DPLPMTUD (Datagram PLPMTUD). La PMTUD dejó de depender de ICMP: el transporte sondea con su propio paquete de datos. Es la respuesta canónica al black hole y es lo que QUIC usa por defecto (** ***`RFC 9002 §6`*** **).** ⚠️ ***DPLPMTUD es loss-based***, lo que introduce ambigüedad entre pérdida por MTU y pérdida por congestión; pocas implementaciones lo soportan (en Linux es una opción, no el default del stack TCP).
- **El "9000 → 1500 cliff"** es hoy un problema real: con **ECMP**, el PMTU es **propiedad del camino, no del prefijo**, así que flujos distintos al mismo destino pueden tener PMTU distinto.
- **Cada overlay consume MTU**: MPLS, VXLAN, IPsec, GRE y WireGuard. Un VXLAN sobre 1500 tiene MTU efectiva **1450**; con WireGuard sobre 1500, **1420**. Por eso el desajuste de MTU es **omnipresente** en redes cloud/bare-metal híbridas.

- **Hardening de fragmentos IPv6**: `RFC 5722` prohíbe solapamientos; `RFC 6946` trata los fragmentos atómicos como no fragmentados; `RFC 5095` deprecó el Routing Header tipo 0.
- **Síntoma clásico del black hole**: "SSH funciona, algunos sitios HTTPS quedan colgados". Diagnóstico: `ping -M do -s <size>`, `tracepath`.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — ¿Cuántos fragmentos genera un datagrama de 1500 bytes por un enlace de MTU 500?

Datos. Datagrama IP de **1500 bytes**, `IHL = 5` (cabecera de 20, sin opciones), `ID = 0xABCD`, `DF = 0`. Enlace de salida con **MTU = 500**.

Paso 1 — Descontar la cabecera del original.

```
Longitud total = 1500 ;  cabecera = IHL × 4 = 5 × 4 = 20
PAYLOAD REAL = 1500 − 20 = 1480 bytes      ⚠️ 1480, NO 1500
```

Paso 2 — Payload por fragmento. Cada fragmento tiene su propia cabecera de 20:

```
payload por fragmento = 500 − 20 = 480 ;  chequeo de 8 bytes: 480/8 = 60  ✓
```

Paso 3 — Cantidad de fragmentos.

```
n = ceil(1480 / 480) = ceil(3,083) = 4 fragmentos
```

#	Offset (bytes)	Offset en unidades de 8	Payload	Total Length	MF	ID
F1	0	0/8 = 0	480	500	1	0xABCD
F2	480	480/8 = 60	480	500	1	0xABCD
F3	960	960/8 = 120	480	500	1	0xABCD
F4	1440	1440/8 = 180	40	60	0	0xABCD

Verificación: `480+480+480+40 = 1480` ✓ igual al payload original. F4 tiene `1480 − 1440 = 40` bytes.

¿Cuántos bytes viajan?

```
500 + 500 + 500 + 60 = 1560 bytes
Overhead = 1560 − 1500 = 60 bytes  =  60/1500 = 4,0 %
```

Interpretación. Pagamos **60 bytes de overhead** (tres cabeceras repetidas) para que el datagrama pase de 1500 a 1560 bytes en el cable. Y el costo real es otro: **el receptor tiene que recibir 4 datagramas para entregar uno**, y si se pierde cualquiera de los cuatro se perdió el datagrama entero.

¿Y si viniera con `DF = 1`? `L = 1500 > 500` y `DF = 1` → **descarta** y manda **ICMP tipo 3 código 4** a `Pi.origen`. Cero bytes salen por ese enlace.

⚠️ Errores típicos

- **Usar 1500 como si fuera el payload.** Siempre `payload = longitud total − IHL × 4`.
- **Contar los offsets en bytes** en vez de unidades de 8.

- **Poner `MF = 1` en el último fragmento.** `MF = 0` es lo que le dice al receptor "acá termina, y con esto sé el largo total".

2.2 Ejercicio 2 — Reensamblar 4 fragmentos que llegan desordenados

Datos. Los 4 fragmentos de 2.1, pero llegan en el orden **F3, F1, F4, F2**. Todos con `ID = 0xABCD`.

Llegada	Offset	Payload	MF	Bytes cubiertos	Falta	¿Completo?
F3	960	480	1	960..1439	0..959 y 1440..1479	no
F1	0	480	1	0..479, 960..1439	480..959 y 1440..1479	no
F4	1440	40	**0**	0..479, 960..1479	**480..959**	**no**
F2	480	480	1	**0..1479**	—	**¡sí!**

Paso a paso, con la aritmética:

```
── Llega F3 (offset 960, 480 bytes) ──────────────────────────────
   Cubre 960 .. 960+480-1 = 960..1439
   MF = 1  →  NO sabemos todavía el largo total
   Tabla: [0]=libre  [60]=libre  [120]=F3  [180]=libre

── Llega F1 (offset 0, 480 bytes) ────────────────────────────────
   Cubre 0 .. 479
   MF = 1  →  todavía no
   ⚠️ F1 tiene offset 0 ⇒ es el que CONTIENE LA CABECERA ORIGINAL
      (con sus opciones). Su Total_Length NO es el del datagrama original.
   Tabla: [0]=F1  [60]=libre  [120]=F3  [180]=libre

── Llega F4 (offset 1440, 40 bytes) ──────────────────────────────
   Cubre 1440 .. 1479
   MF = 0  →  ¡ES EL ÚLTIMO!  Ahora sí sé el largo del payload:
        longitud_total = 1440 + 40 = 1480
   Tabla: [0]=F1  [60]=libre  [120]=F3  [180]=F4

   ⇐ ¿Completo?  Ocupados: 0, 120, 180.  Falta el 60 (bytes 480..959).
      Faltan 480 bytes de 1480  ⇐  NO ESTÁ COMPLETO  ⇐  no se entrega nada.

── Llega F2 (offset 480, 480 bytes) ──────────────────────────────
   Cubre 480 .. 959
   Tabla: [0]=F1  [60]=F2  [120]=F3  [180]=F4

   Verificación: cubiertos = 480+480+480+40 = 1480 ;  largo conocido = 1480
                 y los intervalos son contiguos y sin huecos  ⇐  ¡COMPLETO!
   ⇐ REENSAMBLO y entrega a la capa 3.
```

El datagrama reensamblado: cabecera tomada de **F1** (offset 0), con `IHL = 5` y `Total_Length = 20 + 1480 = 1500`; `Identification = 0xABCD`; `Flags`: `MF = 0`, `Fragment Offset = 0`; `TTL` = el menor de los fragmentos; checksum de cabecera recalculado. Payload = F1 (0..479) + F2 (480..959) + F3 (960..1439) + F4 (1440..1479). Recién **entonces** se verifica el checksum de TCP. **1500 bytes exactos, idéntico al original.**

¿Qué hubiera pasado con el algoritmo ingenuo? Uno que reensambla al ver `MF = 0` habría entregado un datagrama de **520 bytes** (F1 + F4, con un hueco de 480 bytes en el medio) cuando llegó F4. La capa de transporte habría encontrado el checksum de TCP incorrecto y lo habría descartado. **Ese es justo el error que la solución oficial de `2006-12` evita con la suposición "los fragmentos llegaron en orden"**: es válida solo en ese caso.

⚠️ Errores típicos

- **Confundir `MF = 0` con "llegó todo".** Hay que contar los bytes cubiertos.
- **Olvidar que la cabecera original la tiene el fragmento de offset 0** (y solo él).
- **No reiniciar el temporizador** con cada fragmento: si el reensamblaje tarda 70 s y el temporizador es de 60, se descarta un datagrama que iba a completarse.

2.3 Ejercicio 3 — PMTUD con búsqueda binaria

Datos. MTU de la interfaz de A hacia B = **1500**. PMTU real = **1492** (hay un enlace PPPoE en el medio).

La versión oficial multiplica por 0.85. Ejecución con estos datos:

```
Pmtu = 1500 ; envío Echo Request de 1500 con DF = 1
  ⇐ el router del PPPoE lo descarta y manda ICMP 3/4
  ⇐ 1500 × 0,85 = 1275
envío de 1275 → llega entero → Echo Reply → return 1275
```

Devolvió 1275 en vez de 1492: 217 bytes por debajo = 14,5 % del ancho de banda desperdiciado. Y no es casualidad: los valores que genera son `1500 · 0,85^k`, y **ninguno de esos es 1492**. La solución oficial **subestima siempre**, como su propio comentario admite. El **pseudocode de mi versión y su desarrollo punto por punto están en §3.1c**; acá va la ejecución, que es lo que hay que saber hacer:

```
  Estado inicial:  lo = 68,  hi = 1500     (lo = mínimo legal, hi = MTU local)

  Probe 1:  mid = (68+1500)/2   =  784   → llega entero → lo =  784
  Probe 2:  mid = (784+1500)/2  = 1142   → llega entero → lo = 1142
  Probe 3:  mid = (1142+1500)/2 = 1321   → llega entero → lo = 1321
  Probe 4:  mid = (1321+1500)/2 = 1410   → llega entero → lo = 1410
  Probe 5:  mid = (1410+1500)/2 = 1455   → llega entero → lo = 1455
  Probe 6:  mid = (1455+1500)/2 = 1477   → llega entero → lo = 1477
  Probe 7:  mid = (1477+1500)/2 = 1488   → llega entero → lo = 1488
  Probe 8:  mid = (1488+1500)/2 = 1494   → 1494 > 1492  ⇐  ICMP 3/4 con
         MTU = 1492 en los bytes 6-7  ⇐  lo = hi = 1492  ⇐  termina
 ═══════════════════════════════════════════════
  RESULTADO:  PMTU = 1492      ✓ EXACTO     (8 probes)
 ═══════════════════════════════════════════════
```

	Oficial (`× 0.85`)	Mía (binaria + lee la MTU)
Resultado con estos datos	**1275**	**1492**
¿Exacto?	No, **siempre subestima**	**Sí**
Ancho de banda desperdiciado	217 B = **14,5 %**	**0**
Peor caso	~21 probes, y sigue mal	`log₂(1432)` = **11 probes**, exacto
¿Usa la MTU del ICMP?	**No**	**Sí** (bytes 6-7)
¿Cachea el resultado?	No	**Sí, por destino**
¿Distingue ruta rota de MTU chico?	No	**Sí**

Lo que le falta a mi versión también, y hay que decirlo en el examen: la tabla de ruteo debe poder **incrementar** el PMTU, no solo bajarlo. Si el cuello de botella desaparece, el caché queda viejo para siempre. La

forma correcta es un **re-sondeo periódico**: cuando vence el `timestamp` del caché, sondear con un tamaño apenas mayor que el valor cacheado; si llega, subirlo.

⚠️ Errores típicos

- **Confundir `pkt.code == 0` con éxito sin mirar `pkt.tipo`.** El tipo 3 **también** tiene código 0 (net unreachable): el algoritmo diría que funcionó cuando no.
- **Devolver sin cachear.** El descubrimiento es caro; sin caché, cada conexión lo repite.
- **Redondear mal.** `mid = (lo+hi)/2` con enteros y el invariante `hi − lo > 1` son lo que garantiza la terminación.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-12, Problema 2 (30 puntos) — MTU, fragmentación y Path MTU

> **Fuente:** Examen `eirc1012` / `sirc1012` — 17 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf

Enunciado

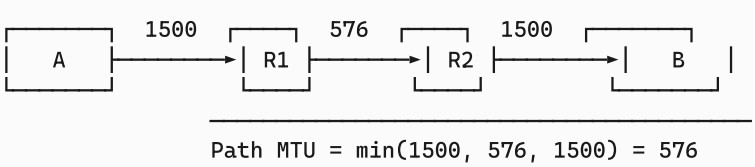
> El MTU (maximum transmission unit) es el tamaño máximo de paquete que puede pasar por determinado enlace. En base a esta definición y a lo visto en clase se pide:
>
> **a)** En una red de varios routers y enlaces, defina los conceptos de MTU, fragmentación y "Path MTU" (la MTU de un camino)
>
> **b)** Describa el comportamiento de un router al reenviar paquetes por un enlace con una MTU de "M" bytes. Describa los diferentes casos de acuerdo al valor del bit DF del encabezado IP, haciendo particular hincapié en los eventuales mensajes de control que se generen.
>
> **c)** Se desea implementar un algoritmo que permita determinar el "path MTU" entre dos nodos, A y B, conectados a la misma red IP. El algoritmo deberá estar basado en los mensajes ICMP que se pueden escuchar en la red y podrá enviar paquetes de prueba. El mismo deberá depender unicamente de uno de los extremos (es decir "A" no deberá requerir colaboración activa de "B", salvo el comportamiento por defecto del stack de protocolos). Se cuenta con las primitivas: `enviar_msg_ICMP(destino, tipo, código, largo_payload)` y `recibir_msg_ICMP(fuente)`. Esta última devuelve un mensaje ICMP recibido de la fuente o null en caso de que pase un cierto tiempo sin recibir nada (implementa un timeout).
>
> Se ofrece la siguiente tabla como referencia (tipos: **0** Echo Reply · **3** Destination Unreachable · **4** Source Quench · **5** Redirect · **8** Echo Request · **9** Router Advertisement · **10** Router Solicitation · **11** Time Exceeded · **12** Parameter Problem; códigos del tipo 3: **0** Net Unreachable · **1** Host Unreachable · **2** Protocol Unreachable · **3** Port Unreachable · **4** Fragmentation Needed and DF set · **5** Source Route Failed · **6** Destination Network Unknown · **7** Destination Host Unknown · **9**/10** comunicativamente prohibido).
>
> **Sobre la tabla:** el PDF la imprime en **dos columnas desalineadas** (la de nombres de tipos y la de códigos del tipo 3 quedaron desfasadas una posición). Leída correctamente es la tabla canónica de `RFC 792`. Los códigos que importan acá son el **3/4** ("fragmentation needed and DF set") y el **3/0** ("net unreachable", que **no** es un Echo Reply: es el bug de la solución oficial).

Parte a) — Las tres definiciones

MTU. Tamaño máximo de trama que una tecnología de capa 2 es capaz de transmitir; en la práctica, la mayor PDU de capa 3 que pasa sin fragmentarse. En Ethernet, 1500 bytes.

Fragmentación. Cuando un router debe enviar un paquete más largo que la MTU del enlace de salida, puede fragmentarlo y reenviarlo en partes. **Es una tarea pesada para un router, ya que le implica mantener estado de ciertos paquetes y sus fragmentos.**

Path MTU. Cuando dos hosts están separados por más de un hop, las MTUs de los distintos enlaces pueden diferir. Se define el **Path MTU como el mínimo de todos los MTUs involucrados en el camino** de un nodo a otro.



Y **las dos propiedades que la solución oficial exige mencionar**, porque están en el texto de la resolución y son parte de los puntos:

- **Puede variar con el tiempo** si los caminos cambian (ECMP o recálculo de rutas).
- **Puede ser diferente según el sentido del tráfico**: un mínimo para la ida y otro para la vuelta, **en el caso de que el ruteo sea asimétrico**.

Parte b) — El comportamiento del router

Solución oficial, literal:

```
DF: "Do not Fragment"

Paquete entrante Pi de largo L
MTU del enlace saliente MTUo

if L <= MTUo then
    transmitir normalmente Pi
else if L > MTUo and Pi.DF == 1 then
    descartar paquete Pi
    enviar msg ICMP (destino=Pi.source, tipo=3, codigo=4)
else if L > MTUo and Pi.DF == 0 then
    fragmentar paquete Pi
    transmitir fragmentos de Pi

Mensaje tipo 3: Destination unreachable
Codigo 4: "fragmentación necesaria pero DF encendido"
```

✅ **Lo que está bien:** el **orden de las condiciones** (primero "cabe", que es el caso frecuente y barato); la **distinción `DF=1` descarta vs. `DF=0` fragmenta**, que es el corazón de la pregunta; y **nombrar el tipo y el código** del ICMP.

🔶 **Lo que agregaría para completar los puntos:**

1. **Qué campos cambian al fragmentar**: `Total Length` se ajusta, `Fragment Offset` se setea, `MF = 1` en todos menos el último, y **el `Identification` se mantiene igual**. La pregunta pide "describa los diferentes casos", y describir *cómo* fragmenta es parte de la respuesta.
2. **La justificación del ICMP**, que la oficial da pero no explica: para que **el emisor no repita el error** (si no, retransmite el mismo datagrama idéntico en un bucle infinito) y para que **pueda reducir su tamaño** (el 3/4 trae la MTU del enlace bloqueante en los bytes 6-7).
3. **Que el último fragmento lleva `MF = 0`.**

Mi respuesta al apartado b), que es la que escribiría:

```
// ---- Router R, paquete entrante Pi, longitud L, MTU de salida MTUo ----
Pi.TTL = Pi.TTL - 1
if (Pi.TTL == 0):
    descartar Pi
    enviar ICMP (destino = Pi.origen, tipo = 11, código = 0)   // Time Exceeded
    return

if (L <= MTUo):                              // CASO 1: cabe entero
    recalcular Header_Checksum(Pi)
    transmitir Pi                             // sin importar DF
    return

if (Pi.DF == 1):                             // CASO 2: prohibido partir
    // El emisor declaró "Don't Fragment": el receptor final no sabe
    // tratar datos partidos y el paquete afirma un límite a respetar.
    descartar Pi
    enviar ICMP (destino = Pi.origen, tipo = 3, código = 4)
    //   tipo 3 = Destination Unreachable
    //   código 4 = "Fragmentation Needed and Don't Fragment was Set"
    //   los bytes 6-7 llevan la MTU del enlace bloqueante (RFC 1191),
    //   para que el emisor ajuste su MSS o su caché de PMTU.
    return

// CASO 3: no cabe y DF = 0  →  fragmentar
payload      = L - (Pi.IHL × 4)
max_payload  = ((MTUo - Pi.IHL × 4) / 8) × 8      // múltiplo de 8
offset = 0 ;  resto = payload
while (resto > 0):
    largo = min(max_payload, resto)
    F = copia de Pi
    F.Total_Length    = Pi.IHL × 4 + largo
    F.Identification  = Pi.Identification          // ← el MISMO en todos
    F.Fragment_Offset = offset / 8                 // ← en UNIDADES DE 8
    F.MF = (largo < resto) ? 1 : 0                 // 0 solo en el último
    recalcular Header_Checksum(F)
    transmitir F
    offset += largo ;  resto -= largo
```

> **Tope para los 10 puntos de b):** los tres casos con la condición exacta, el **descarte con `DF = 1`**, el **ICMP tipo 3 código 4 con su justificación**, y los campos que cambian al fragmentar.

Parte c) — El algoritmo de descubrimiento del Path MTU

Solución oficial, literal:

```
La estrategia básica es transmitir paquetes de prueba y escuchar los
mensajes ICMP de error que regresan. El algoritmo similar a una bipartición
pero simplificada.

Partimos de una resultado tentativo: sabemos que el path MTU no va a ser más
grande que el MTU del enlace por donde los paquetes destinados a B son
encaminados.

Pmtu = MTU(enlace hacia B)
tries = 0
while true
    // envio un ping hacia el destino de largo el maximo tentativo
    enviar_msg_ICMP (destino=B, tipo = 8, largo = Pmtu)
    pkt = recibir_msg_ICMP(fuente=B)
    if pkt != Null then
        if pkt.code == 0 then
            // es un echo reply, llego al destino bien
            return Pmtu
        else
            // calculo nueva aproximación
            // esta aproximación es bastante agresiva, se podría hacer mejor
            // tiende a sub-estimar el pmtu
            Pmtu = Pmtu * 0.85
    else
        // posiblemente timeout u otro error de red
        Pmtu = Pmtu * 0.85
        tries++
        if tries >= 5 then
            return False
```

✅ **Lo que está bien:** la estrategia (sondar con paquetes de prueba y escuchar los ICMP de error) es exactamente el PMTUD; **depende de un solo extremo** (A manda y espera, B no hace nada especial), cumpliendo la restricción del enunciado; usa **tipo 8** (Echo Request), que es lo que hace `ping`; **distingue el `null`** (timeout) de un mensaje recibido y cuenta reintentos; y **se autoevalúa**, reconociendo que la aproximación es agresiva.

⚠️ **Lo que está mal o es discutible — y esto también se pregunta:**

1. 🔴 **`if pkt.code == 0` sin mirar `pkt.tipo`.** El bug más grave: el **tipo 3 también tiene código 0** ("Net Unreachable"), como se ve en la propia tabla del examen. Si el router responde tipo 3 código 0, el algoritmo **devuelve un PMTU que nunca funcionó**. La condición correcta es `pkt.tipo == 0 AND pkt.codigo == 0`.
2. 🔴 **No usa la MTU que viene en el ICMP.** El `RFC 1191` la pone en los **bytes 6-7** del tipo 3 código 4. El enunciado da la tabla **justo para que uses ese mensaje**, y la respuesta **ya está dentro del ICMP**.
3. 🟡 **Nunca da el PMTU exacto.** Los valores son $1500 \cdot 0{,}85^k$; ninguno es 1492. Con PMTU real 1492 devuelve 1275 (**14,5 % bajo**).
4. 🟡 **El comentario no describe lo que hace el código.** Una bipartición es `(lo+hi)/2`; multiplicar por 0.85 es una búsqueda **exponencial hacia abajo**.
5. 🟡 **No cachea el resultado.** El *almacenamiento en caché* es parte del mecanismo: sin él, cada datagrama grande genera un ICMP y la red se llena de errores.
6. 🟡 **No hay cota inferior.** Puede bajar de 68 (el mínimo legal de IPv4) y seguir mandando sondas imposibles.
7. 🟡 **No distingue "MTU chico" de "ruta rota".** Un tipo 3 código 1 (host unreachable) o 9/10 (prohibido) no se arreglan bajando el tamaño.

8. 🟡 **Solo decrementa, nunca incrementa.** Si el camino mejora, el caché queda viejo para siempre.
9. 🔴 `tries` **solo cuenta en el timeout.** Si el router siempre responde tipo 3 código 4, el `while true` **no termina nunca**.

Mi solución: búsqueda binaria + lectura de la MTU del ICMP + caché.

```
// ======== ESTADO PERSISTENTE EN A ========
pm[B] = { valor, valido, timestamp }        // caché de PMTU por destino
MAX_REINTENTOS = 5 ;  MTU_MIN_ABSOLUTO = 68

int descubrir_PMTU(A, B):
    // ---- Invariantes ----
    //   lo : tamaño que YA sabemos que pasa entero  (cota inferior)
    //   hi : tamaño que YA sabemos que NO pasa      (cota superior)
    //   Al salir, lo es el mayor tamaño conocido como transmisible.
    if (pm[B].valido): lo = pm[B].valor      // arrancamos del caché
    else:              lo = MTU_MIN_ABSOLUTO // 68: seguro por definición
    hi = MTU(interfaz de A hacia B)           // 1500: no puede pasar más
    reintentos = 0

    while (hi - lo > 1):
        mid = (lo + hi) / 2                   // ← búsqueda BINARIA
        enviar_msg_ICMP(B, 8, 0, mid)         // Echo Request con DF = 1
        pkt = recibir_msg_ICMP(B)

        if (pkt == null):                      // timeout
            // Degradación elegante: asumimos el PEOR caso (que mid era
            // demasiado grande) y seguimos binary-searching.
            reintentos++
            if (reintentos > MAX_REINTENTOS): cachear(B, lo); return lo
            hi = mid

        else if (pkt.tipo == 0 and pkt.codigo == 0):
            lo = mid                          // llegó entero a B: ES válido

        else if (pkt.tipo == 3 and pkt.codigo == 4):
            // ★ AQUÍ ESTÁ LA MEJORA PRINCIPAL ★
            m = leer_uint16(pkt, offset = 6)  // Next-Hop MTU (RFC 1191)
            if (m >= lo and m < mid): lo = m; hi = m   // m ES transmisible
            else:                  hi = mid   // mensaje inconsistente

        else if (pkt.tipo == 3):
            // Otro inalcanzable (net, host, port, prohibido):
            // NO es un problema de tamaño, es un problema de ruta.
            cachear(B, lo); return ERROR_NO_EXISTE_RUTA

        else:                                  // tipo 11, 12, etc.
            reintentos++
            if (reintentos > MAX_REINTENTOS): cachear(B, lo); return lo
            hi = mid

    cachear(B, lo)
    return lo
```

Mejora	Por qué
`pkt.tipo == 0 && pkt.codigo == 0`	El tipo 3 tiene un código 0. Chequear solo el código hace que el algoritmo **diga que funcionó cuando no**. Bug real de la oficial.
Leo la MTU de los bytes 6-7	El mensaje **ya trae la respuesta exacta**. **1 solo probe** reemplaza a los 8 de la búsqueda binaria, y el resultado es **exacto** en vez de 14,5 % bajo.
Búsqueda binaria `(lo+hi)/2`	`O(log n)` : peor caso $\log_2(1432)$ = **11 probes**, **siempre**. El `× 0.85` nunca converge al valor exacto.
Caché por destino	El PMTU es **propiedad de la ruta**. Sin caché, cada paquete grande vuelve a generar un ICMP.
Cota inferior `lo = 68`	Nunca sondea algo que **no puede funcionar**.
Distinguir los otros tipo 3	Un "host unreachable" no se arregla bajando el MTU.
Contar todos los reintentos	La oficial puede no terminar nunca.

> **Nota sobre la diferencia con la oficial:** la oficial **funciona** como heurística y devuelve un valor **conservador**, lo cual es "correcto pero ineficiente". La mía devuelve el valor **exacto**. En el examen, **presentá las dos**: reproducí la oficial con su análisis, y después proponé la mejora con el argumento. Eso vale más puntos que una sola.
>
> **Tope para los 10 puntos de c):** pseudocode completo y ejecutable, la **inicialización del estado** (el `Pmtu` tentativo desde el MTU del enlace local), el **manejo del timeout**, la **lectura del tipo y el código** del ICMP recibido, y **al menos una justificación** de por qué tu estrategia es mejor que "probar y ver qué pasa".

3.2 Examen 2006-12, Pregunta 3 (10 puntos) — fragmentación y reensamblaje

> **Fuente:** Examen `sirc0612` — 26 de diciembre de 2006.
>
> **Enunciado:** Explique por qué puede ser necesaria la fragmentación de paquetes a nivel de capa 3. Explique cómo funciona la misma en IPv4 y mencione los problemas asociados a la misma. Dé un algoritmo para realizar el reensamblaje de un paquete IP a partir de sus fragmentos.

La respuesta oficial, completa:

> *a)* La fragmentación supone dividir un paquete en varios más pequeños. La necesidad se basa en que **todo medio de transmisión tiene un Maximum Transfer Unit**. Si un paquete es mayor que el MTU, debe fragmentarse.
>
> *b)* En cada fragmento: el **total length** se ajusta al tamaño del segmento; la bandera **MF** se coloca en uno excepto para el último; y se setea el **fragment offset** de acuerdo al offset del mismo en el segmento original.
>
> *c) El algoritmo de reensamblaje:* "El algorítmico básico sería el siguiente, en la solución propuesta **se supone que los fragmento llegaron en orden y el último llega con el bit MF en 0**": `while (true) {`

```
recibo_paquete(p); if ( p.MF == 1 ){ almaceno(p); } else { if (p.fragment_offset <> 0 ){
almaceno(p); p_tot = reensamblo(p); if (p_tot <> NULL){ envo_paquete(p_tot); } } } }
```

Análisis: por qué es insuficiente, y qué corregir.

#	Problema	Por qué importa
1	**Asume que los fragmentos llegan en orden**	En una red real pueden llegar desordenados. De eso depende **todo** el algoritmo.
2	**`p_tot = reensamblo(p)` al ver `MF = 0`**	Solo funciona con la suposición 1. Ver §2.2 para el caso donde falla.
3	**`almaceno(p)` no dice dónde**	Sin índice por `Fragment Offset` no se puede saber qué falta.
4	**No hay detección de solapamiento**	Es el agujero de seguridad (§1.4 d).
5	**No hay temporizador**	Los fragmentos se acumulan para siempre → fuga de memoria del destino.
6	**No hay clave de agrupación**	Falta `(IP destino, Identification, protocolo)`.
7	**`if (p.fragment_offset <> 0)` descarta mal**	Si el último en llegar es el de offset 0, **se pierde**.

El algoritmo corregido con las 7 correcciones está desarrollado en **§1.5**: índice por `Fragment Offset`, clave de agrupación, chequeo de longitud mínima, detección de solapamiento con liberación del reensamblaje, `longitud_total` derivado del `MF = 0`, **completitud por cobertura total** (no por `MF`), `temporizador` reiniciado en cada fragmento y con barrido periódico.

Respuesta a "los problemas asociados", que es donde están la mitad de los puntos: (i) **overhead de cabeceras repetidas** — 1500 bytes pueden terminar en 185 fragmentos y pasar de 1500 a 5180 bytes en el cable; (ii) **unidad mínima de 8 bytes**; (iii) **la pérdida de un solo fragmento descarta el datagrama entero**, con probabilidad de éxito `(1-p)^n` que decae exponencialmente; (iv) **el reensamblaje exige estado y temporizador** en el destino, y si vence se pierde todo; (v) **el solapamiento es un agujero de seguridad**, porque el segundo fragmento sobrescribe al primero (`RFC 5722` lo prohíbe en IPv6); (vi) **cada fragmento es un paquete IP independiente para los filtros**, lo que permite **evadir firewalls** que solo inspeccionan el primero.

3.3 Repaso: `2000-07` Problema 2

> **Fuente:** Examen `sirc0007`, Problema 2 — protocolo fiable sobre 3 subredes con MTU distinta.

Es la **misma idea con transporte confiable**: entre dos extremos unidos por subredes con MTUs distintas no se puede "confiar en que TCP fragmenta". Si la capa 3 no fragmenta (o si el camino es IPv6), **el transporte tiene que fragmentar él mismo**, en el emisor, y desensamblar en el receptor: el mismo esquema de §1.5, pero el que decide los tamaños es el emisor del segmento de transporte y no un router. Menciónalo si te lo preguntan como ejemplo de por qué el transporte tiene su propio control de tamaño.

PARTE 4 — CIERRE

📌 Resumen

1. **MTU** = la mayor PDU de capa 3 que el enlace transmite sin fragmentar. En Ethernet son **1500 bytes**; la trama llega a 1518 con cabecera y FCS.
2. Los **3 campos** de la fragmentación IPv4 son **Identification**, **Flags (con `DF`)** y **Fragment Offset**.
3. El **`Fragment Offset` va en unidades de 8 bytes**, y salvo el último el payload debe ser múltiplo de 8.
4. El router **descarta y avisa** si `DF = 1` (`ICMP tipo 3 código 4`); **fragmenta** si `DF = 0`. Nunca fragmenta algo que entra entero.
5. El ICMP tipo 3 código 4 existe para que el emisor **no repita el error** y **baje su tamaño**; lleva la MTU del enlace bloqueante en los **bytes 6-7**.
6. Los problemas: **overhead de cabeceras**, **unidad de 8 bytes**, **temporizador de reensamblaje**, **solapamiento (agujero de seguridad)**, **perder un fragmento = perder el datagrama entero**, y **evasión de filtros**.
7. El **Path MTU** es el **mínimo** de los MTUs del camino; **varía con el tiempo** y **es distinto en cada sentido** si el ruteo es asimétrico.
8. El **PMTUD (`RFC 1191`)** = `DF = 1`, recibir el ICMP, **bajar y cachear** el PMTU. La MTU exacta viene en los bytes 6-7; usarla da resultado **exacto**, mientras que multiplicar por 0.85 **siempre subestima**.
9. El **reensamblaje** necesita índice por offset, **detección de solapamiento**, **temporizador** y la clave `(IP destino, ID, protocolo)`. **No basta con ver `MF = 0`.**
10. El **PMTUD black hole** produce el síntoma **"SSH funciona, HTTPS se cuelga"**, y se diagnostica con `ping -M do -s <tamaño>`.

✅ Autoevaluación

1. Un datagrama IPv4 de 1500 bytes con `IHL = 5` pasa por un enlace de MTU 576. ¿Cuántos fragmentos y cuál es el `Fragment Offset` del último?

▶ Respuesta

2. ¿Por qué el router **descarta** un datagrama con `DF = 1` que no entra, en vez de fragmentarlo igual?

▶ Respuesta

3. ¿Qué pasa si se pierde **uno solo** de los fragmentos de un datagrama partido en 5? Explicá con un número.

▶ Respuesta

4. ¿Por qué el `Fragment Offset` está en unidades de 8 bytes y no de 1? ¿Qué consecuencia práctica tiene?

▶ Respuesta

5. Un `traceroute` muestra un salto con `*`. ¿Podés concluir que ese router está caído?

▶ Respuesta

6. ¿Por qué la solución oficial multiplica por 0.85, y por qué está mal?

▶ Respuesta

7. ¿Cuál es el MTU mínimo absoluto en IPv6 y de dónde sale? ¿Y en IPv4?

▶ Respuesta

8. ¿Por qué el jumbo frame de 9000 es "una convención y no un estándar IEEE"? ¿Y por qué no se usa en Internet?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1012` / `sirc1012`** — Problema 2 completo (30 pts: MTU, fragmentación, Path MTU e **implementación del descubrimiento**); incluye la tabla de tipos y códigos de ICMP
- **Examen `eirc0612` / `sirc0612`** — P3 (10 pts: por qué fragmentar, cómo funciona, problemas asociados y **algoritmo de reensamblaje**)
- **Examen `sirc0007`** — Problema 2: protocolo fiable sobre 3 subredes con MTU distinta
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.5.2 "Fragmentación"
- **RFC 791** — Internet Protocol (campos de fragmentación, MTU mínimo 68)
- **RFC 1122 §3.2.1** — MTU mínimo de 576 bytes
- **RFC 1191** — Path MTU Discovery (1990): la MTU del enlace bloqueante en los bytes 6-7 del ICMP
- **RFC 8200 §5** — IPv6 (2017): **MTU mínimo absoluto de 1280**
- **RFC 8201** — Path MTU Discovery for IPv6 (2017)
- **RFC 8899** — DPLPMTUD (*Standards Track*, sep-2020; sucesor de RFC 4821) · **RFC 4821** — *Experimental*, 2007
- **RFC 5722** (2009) — Prohibición de fragmentos solapados en IPv6 · **RFC 6946** (2013) — fragmentos atómicos
- **RFC 9002 §6** — QUIC: PLPMTUD integrado, sin dependencia de ICMP
- **Clase siguiente:** 36 — IPv6

Clase 36

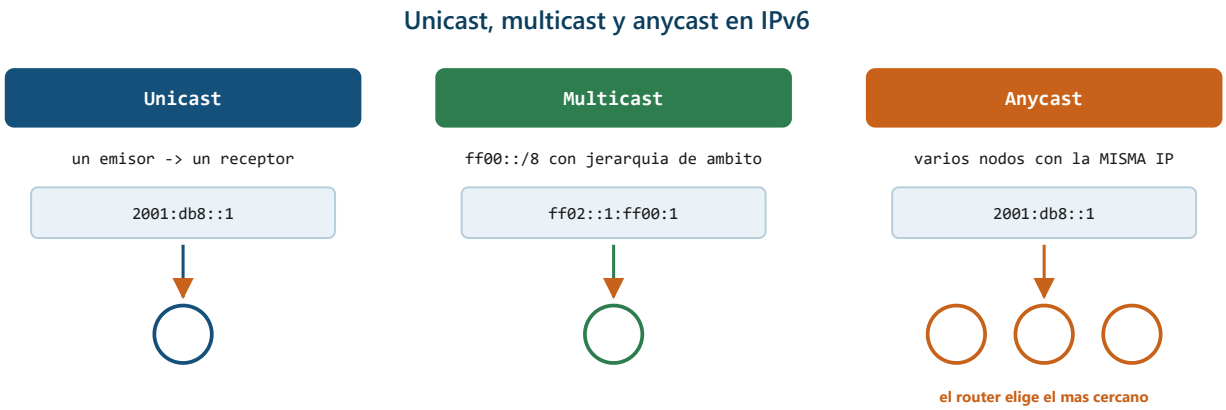
Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 Por qué IPv6: del "futuro" al presente consumado
- 1.2 La cabecera IPv6
- 1.3 El Flow Label (`RFC 6437`)
- 1.4 Cabeceras de extensión y el `Payload Length`
- 1.5 Los 3 modelos de comunicación
- 1.6 Los tipos de dirección IPv6
- 1.7 ICMPv6 (`RFC 4443`)
- 1.8 Neighbor Discovery vs. ARP — el apartado que más puntos da
- 1.9 DAD — Duplicate Address Detection
- 1.10 Los 3 esquemas de transición
- 1.11 La fragmentación en IPv6: el router NUNCA fragmenta
- 1.12 Actualización a 2026 — IPv6
- 2.1 Ejercicio 1 — Escribir una cabecera IPv6 a partir de datos
- 2.2 Ejercicio 2 — Multicast vs. anycast: por qué anycast no es un tipo de dirección
- 2.3 Ejercicio 3 — ND vs. ARP en tabla comparativa completa
- 2.4 Ejercicio 4 — Los stacks de referencia IPv4 e IPv6
- 3.1 ⭐ Examen 2010-12, Pregunta 5 (8 puntos) — IPv4 vs. IPv6, ND vs. ARP
- 3.2 Examen 2009-12, Pregunta 5 (6 puntos) — DAD y NDP
- 3.3 Examen 2010-07, Pregunta 5 (8 puntos) — modelos, tipos y transición
- 3.4 Examen 2007-03, Pregunta 3 (10 puntos) — RARP
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: D (Capa de Red) · **Semana:** 12 · **Duración estimada:** 120 min **Correlativa:** Clase 35 (Fragmentación, MTU y Path MTU) · **Sigue con:** Clase 37 (ICMP, ping y traceroute + Repaso y Simulacro 4) **Material de referencia:** Tanenbaum cap. 5 (§5.6) · RFC 8200, RFC 4443, RFC 6437, RFC 8201

🎯 Al terminar esta clase vas a poder

1. Explicar **por qué** existe IPv6, con los datos de despliegue de 2026 y **diciendo explícitamente que la cifra depende de qué se mide**.
2. Dibujar la **cabecera IPv6** campo por campo y **compararla lado a lado con la IPv4**: qué desapareció y qué es nuevo.
3. Explicar para qué sirve el **Flow Label** (`RFC 6437`) y por qué no alcanza la 5-tupla.
4. Describir la **cadena de cabeceras de extensión** por `Next Header` , y el bug histórico del `Payload Length` .
5. Explicar los **3 modelos de comunicación** y demostrar por qué **anycast no es un tipo de dirección**.
6. Enumerar los **tipos de dirección** y explicar por qué el `/64` **es la unidad de subred**.
7. Describir **ICMPv6** y explicar por qué **no es opcional**.
8. Describir **Neighbor Discovery** y enumerar **los 5 cambios frente a ARP** (el apartado que más puntos da).
9. Explicar **DAD** y por qué no existe en IPv4.
10. Describir los **3 esquemas de transición** con sus ejemplos.



anycast NO es un tipo de direccion: es un SERVICIO que se le da a un grupo de nodos.

Es lo que usan los root servers de DNS y los resolvers publicos (1.1.1.1, 8.8.8.8).

> **Prerequisito del examen:** `2010-12 P5` (8 pts — stacks de referencia, coexistencia y **ND vs. ARP con ≥4 cambios**), `2009-12 P5` (6 pts — DAD y NDP), `2010-07 P5` (8 pts — **3 modelos de comunicación**, **3 tipos de dirección**, **3 esquemas de transición**), `2007-03 P3` (10 pts — RARP), `2009-07 P5` (ventajas y desventajas de IPv6).

PARTE 1 — TEORÍA

1.1 Por qué IPv6: del "futuro" al presente consumado

El agotamiento es un hecho, no una predicción. IPv4 tiene 2^{32} = 4.294.967.296 direcciones, y no dan. El agotamiento ocurrió por regiones, en este orden:

RIR	Se agotó
APNIC (Asia-Pacífico)	abril de 2011
ARIN (Norteamérica)	septiembre de 2015
AfriNIC (África)	abril de 2017
LACNIC (Latinoamérica)	2020
RIPE NCC (Europa)	**25 de noviembre de 2019**

> **El dato para el aula:** RIPE NCC hizo su **última asignación /22 el 25-nov-2019** a las 15:35 CET. Hoy mantiene una *waiting list* para un único /24 recuperado. IPv4 no "se está agotando": **ya se agotó**. IPv6 tiene 2^{128} direcciones.

El despliegue real en 2026, con el número exacto

> **El 28 de marzo de 2026, el acceso a Google por IPv6 superó el 50 % por primera vez: 50,10 %.** Fuentes: Internet Society Pulse, APNIC (abr-2026), The Register (abr-2026).

Y lo importante es esto: la respuesta a "¿cuánto IPv6 hay?" es "depende de qué midás". Se pregunta:

Quién mide	**Qué** mide	Valor
Google (`stats.google.com/ipv6`)	**usuarios** que acceden a Google por IPv6	**50,10 %** (mar-2026)
APNIC	**cobertura de capacidad** (qué % de redes tiene IPv6)	**~42-43 %**
Cloudflare Radar	**tráfico por bytes**	**~40 %**

Y es desigual: leaders Francia ~73 %, India ~72 %, Arabia Saudita ~65 %; rezagados Italia ~17 %, España ~10 %, Egipto ~4 %, **China < 5 %**.

> **Respuesta modelo para el examen (2012 vs. 2026):** en 2012 el curso decía "IPv6 llega en 10 años". La respuesta correcta hoy es: **"ya pasó la mitad en los usuarios de Google, pero es desigual, China sigue en menos del 5 %, y la cifra depende de qué se mida"**. Una definición de "cuánto IPv6 hay" sin decir qué se midió es una respuesta incompleta.

⚠️ Actualización a 2026

- **IPv6 dejó de ser "el futuro".** No hay fecha de cierre de IPv4, y no la habrá pronto.

- **CGNAT (*Carrier-Grade NAT*) es ubicuo en los ISP**: es el nuevo default de la capa 3 de acceso. Rompe conectividad entrante, complica la señalización (SIP/IMS), rompe IPsec y rompe peer-to-peer.
- `RFC 6177` **(2011)** ya no recomienda `/48` por sitio final; los RIR asignan `/48` (a veces `/56`) al **operador**, que reparte.
- **NDP se amplió en 2026:** `RFC 9926` (feb-2026) permite registrar un prefijo en los routers vecinos.
- **Por qué NAT rompe IPv6:** NAT depende de direcciones privadas y reescritura de puertos para multiplexar. IPv6 no **necesita** traducción; peor, NAT rompe la **extensibilidad extremo a extremo** (cabeceras de extensión, movilidad), no es *stateful-friendly* con IPsec, y obliga a los hosts IPv6-only a usar 464XLAT/PLAT para alcanzar contenido IPv4. ⇒ **La meta de IPv6 es eliminar el NAT.**

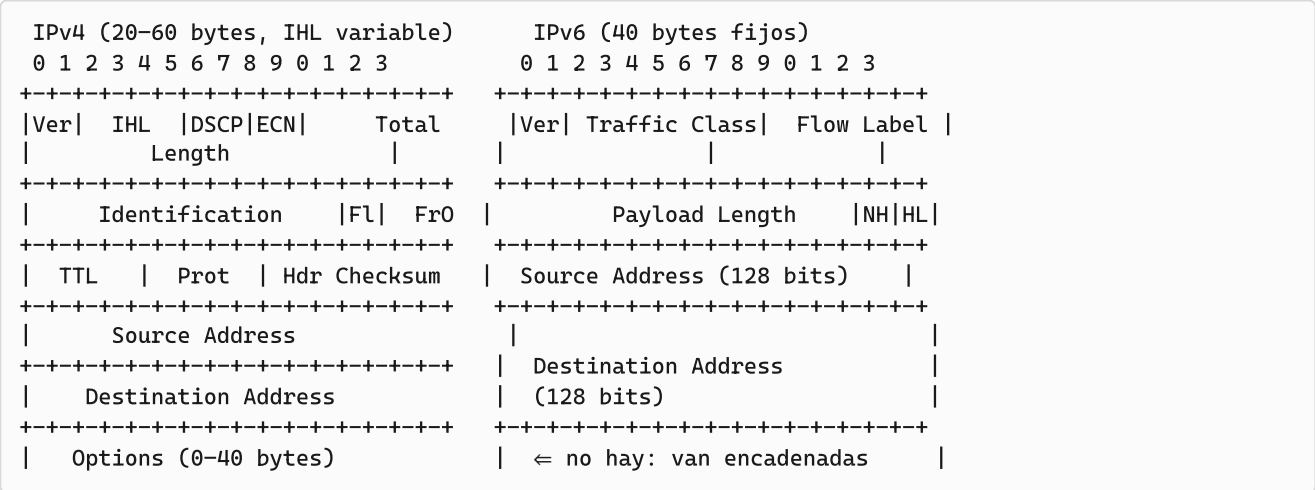
1.2 La cabecera IPv6

En IPv4 la cabecera mide de 20 a 60 bytes y hay que mirar el `IHL` para saber dónde termina. En IPv6 son **40 bytes fijos**, y lo que sigue son **cabeceras de extensión**.

```
   Cabecera IPv6 — 40 bytes fijos

<svg xmlns="http://www.w3.org/2000/svg" width="100%" viewBox="0 0 740 280" style="max-width:740px":

 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|Version| Traffic Class |           Flow Label                  |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|         Payload Length        |  Next Header  |   Hop Limit   |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                                                               |
+                         Source Address                        +
|                         (16 bytes = 128 bits)                 |
|                                                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                                                               |
+                      Destination Address                      +
|                      (16 bytes = 128 bits)                    |
|                                                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

Campo	Bits	Qué es
Version	4	Siempre **6**.
Traffic Class	8	Equivale a **DSCP + ECN** de IPv4 (6 + 2 bits).
Flow Label	20	**Nuevo.** Identifica un flujo (ver §1.3).
Payload Length	16	Largo del payload en bytes, **sin** las cabeceras de extensión.
Next Header	8	Sustituye al `Protocol` **y** al `IHL`: dice qué va después (TCP=6, UDP=17, ICMPv6=58, o una extensión).
Hop Limit	8	Equivale al `TTL`. Se decrementa en **cada** router.
Source / Destination	128 c/u	

IPv4 contra IPv6, lado a lado



Qué DESAPARECIÓ y por qué:

Desaparecido	Por qué
`Header Checksum`	**Era un desperdicio de CPU** y había que recalcular en **cada** salto porque el TTL cambia. Ahora **el checksum lo pone la capa 4** (TCP, UDP), que verifica **una sola vez extremo a extremo**. Cada router verifica ~0 y el error se detecta igual, pero al final de la tubería.
`IHL`	La cabecera siempre mide 40 bytes. El campo `Next Header` cumple la función doble.
`Fragment Offset`	**IPv6 no fragmenta en los routers.** Los routers solo decrecantan `Hop Limit` y descartan con **Packet Too Big** (tipo 2). Solo el origen fragmenta, con una cabecera de extensión.
`Options`	Se fueron a las cabeceras de extensión. El motivo: en IPv4 una opción **dentro** de la cabecera obligaba a que **todos** los routers la entendieran (o descartaran el paquete). En IPv6 se procesan **solo donde corresponde**.

Qué es NUEVO: el `Flow Label` (20 bits) y las **cabeceras de extensión encadenadas**, que dan **extensibilidad sin cambiar el formato base**.

El argumento profundo: IPv4 optimizó la cabecera para la **velocidad del router** (checksum barato, todo en un lugar). IPv6 la optimizó para la **extensibilidad**, y movió el costo de la verificación al final de la tubería.

1.3 El Flow Label (`RFC 6437`)

Son **20 bits** que el emisor pone para **identificar un flujo**, y que los routers pueden usar **sin conocer la conexión**.

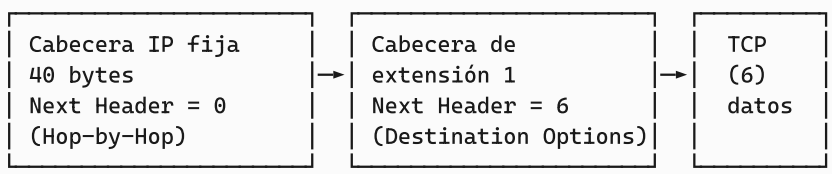
El problema que resuelve. Un router hace QoS (ECN, remarked DSCP, ECMP por flujo) y necesita saber "este paquete es del mismo flujo que aquél". La forma clásica es la **5-tupla** `(IP origen, puerto origen, IP destino, puerto destino, protocolo)`, que **no siempre se puede construir**:

```
1. PAQUETE FRAGMENTADO: solo el PRIMER fragmento tiene la cabecera de
   transporte. Los otros no. Un router que debe hacer ECMP no puede hashear
   un fragmento contra otro sin el Flow Label.

2. PAQUETE CIFRADO: si el transporte va cifrado (IPsec, WireGuard), los
   puertos son ilegibles. El Flow Label está en la cabecera IP, que NO se
   cifra: es visible para un router de medio aunque el payload no lo sea.
```

Reglas de `RFC 6437`: el Flow Label **solo significa algo para un par (origen, destino)**, así que el router identifica el flujo con `(Flow Label, Src, Dst)`; el emisor puede **cambiarlo** cuando cambian los atributos del flujo; los routers que no lo entienden **lo ignoran** (retrocompatible por diseño); y **no es una etiqueta de seguridad**, cualquiera puede poner lo que quiera.

1.4 Cabeceras de extensión y el `Payload Length`

Cuando el `Next Header` no es un protocolo de transporte, **es una cabecera de extensión**. Se encadenan: cada una apunta con su `Next Header` a la siguiente, y la última al protocolo real.



Nombre	`Next Header`	Para qué	¿Quién la procesa?
Hop-by-Hop Options	**0**	Opciones que deben ver **todos** los routers	Todos
Routing	**43**	Encaminamiento explícito. El **tipo 0 fue deprecado** (`RFC 5095`)	Los routers indicados
Fragment	**44**	Solo si el **origen** fragmenta	Solo el destino final
Destination Options	**60**	Opciones para el destino	Solo el destino final
AH	**51**	Integridad y autenticidad	Los que entiendan AH
ESP	**50**	Confidencialidad + integridad + anti-replay	Solo el destino final

> ⚠️ **El bug histórico del `Payload Length`.** En la especificación original (`RFC 2460` , 1998) el `Payload Length` incluía **las cabeceras de extensión**. Eso obligaba a que el nodo que fragmentaba conociera el largo de las cabeceras de extensión de los paquetes que iba a producir. **Era un error de diseño**, y `RFC 2719` (1999) lo corrigió: **el `Payload Length` excluye las cabeceras de extensión**. Si hay 8 bytes de extensión y 1000 de datos, `Payload Length = 1000` . La vigente, `RFC 8200` (2017), ya trae la corrección.

Por qué son la gran victoria de diseño de IPv6: en IPv4, para agregar una opción había que modificar el formato de la cabecera, y **cualquier router que no la entendiera tenía que descartar el paquete**. En IPv6, un

router que no entiende una extensión simplemente **la salta** (salvo Hop-by-Hop y Fragment, que son obligatorias). **Las nuevas funciones se pueden agregar sin romper Internet.**

1.5 Los 3 modelos de comunicación

Este es el `2010-07 P5 a)`.

1) Unicast — un emisor, un receptor

El modelo clásico: cada dirección identifica **exactamente un** nodo.

```
H1 (2001:db8::1)  ──────────────►  S (2001:db8::2)
     emisor                       receptor
```

La dirección de **destino** identifica al destino, la de **origen** al emisor, y ambos van en la cabecera. Es el 99 % del tráfico.

2) Multicast — un emisor, muchos receptores

Prefijo `ff00::/8`: el primer byte de la dirección es `ff`.

```
|1111 1111| 4 bits de ámbito (scope) | 4 bits de flag | 112 bits de ID |
└ ff ─────┘                          └── flags ──────┘└── del grupo ────┘
```

Ámbito	Prefijo	Significado
0	`ff00::/8`	Reservado
1	`ff01::/16`	**Node-local** (solo en el propio host)
2	**`ff02::/16`**	**Link-local**: solo en el mismo enlace
5	`ff05::/16`	Site-local (dentro de un sitio)
8	`ff08::/16`	Organization-local
14	`ff0e::/16`	**Global**
15	`ff0f::/16`	Reservado

⚠️ **`site-local` (`fec0::/10`) está DEPRECADO**: se reemplaza por **unique local** (`fc00::/7`).

La jerarquía: un router que recibe tráfico para `ff02::/16` **no lo reenvía fuera del enlace**. Uno que recibe para `ff0e::/16` lo reenvía por Internet.

Ejemplo real: IPv6 usa multicast siempre, aunque el usuario no lo sepa (`ff02::1` = todos los nodos del enlace, `ff02::2` = todos los routers del enlace, `ff01::1` = el propio nodo).

3) Anycast — muchos emisores, muchos receptores

> ⚠️ **Punto clave, y es un error clásico de examen: el anycast NO es un tipo de dirección.** Es un **servicio** que se le asigna a un grupo de nodos que tienen **la misma dirección unicast**, y el router

> encamina al **más cercano**.

```
⇐ TODOS tienen la MISMA dirección: 2001:db8::1
⇐ NO hay prefijo especial, NO hay multicast, NO hay bit que lo marque

┌──────────┐   ┌──────────┐   ┌──────────┐
│ Nodo A   │   │ Nodo B   │   │ Nodo C   │
│ 2001:db8 │   │ 2001:db8 │   │ 2001:db8 │
│   ::1    │   │   ::1    │   │   ::1    │
└────┬─────┘   └────┬─────┘   └────┬─────┘
     └──────────────┼──────────────┘
                    ▼
          ┌──────────────────┐
          │ "encaminá a      │
          │  2001:db8::1 al  │
          │  más cercano"    │
          └──────────────────┘
```

¿Por qué no es un tipo de dirección?

1. **No hay prefijo ni bit que lo marque.** Es una dirección unicast normal, asignada a varios nodos a la vez. La tabla de ruteo es la de siempre.
2. **La decisión es del router, no del paquete.** El paquete **no sabe** que hay más de uno.
3. **El receptor no se identifica.** El emisor no puede saber a cuál de los tres le llegó.

¿Cómo es posible? Porque en ruteo **la dirección destino identifica un destino lógico, no un host físico**: con BGP, un AS anuncia el prefijo desde **varios puntos** del mundo y el cliente elige el más cercano.

Dónde se usa: los **13 root servers de DNS** (todos con `198.41.0.4` y `2001:503:ba3e::2:30`), los **resolvers públicos** (`8.8.8.8`, `1.1.1.1`), y **los CDNs**.

La comparación clave (multicast vs. anycast):

	Multicast	Anycast
¿Tipo de dirección?	**Sí**: `ff00::/8`	**NO** — servicio sobre unicast
¿A quién llega?	A **todos** del grupo	Al **más cercano** (uno solo)
¿Cómo lo decide?	El **paquete** (dirección)	El **router** (encaminamiento)
Configuración	Requiere **IGMP/MLD** y ruteo multicast	**Ninguna**: es ruteo normal
Ejemplo	`ff02::1`	`8.8.8.8`

⚠️ Errores típicos

- **Decir que anycast es un tipo de dirección.** No existe prefijo anycast.
- **Confundir los bits de ámbito con los de flag** en la dirección multicast.
- **Decir que site-local es válido.** Está deprecado; se usa `unique local`.

1.6 Los tipos de dirección IPv6

El enunciado de `2010-07 P5 a)` pide tres; la realidad son cuatro. Los tres que el curso emphasizes:

Tipo	Prefijo	Alcance	¿Cómo se obtiene?
Unicast global	`2000::/3`	Internet entero	Asignado por un RIR a través de un ISP
Link-local	`fe80::/10`	**Solo en el mismo enlace**	**Se genera sola** en cada interfaz
Multicast	`ff00::/8`	Según el ámbito	Configurado por la aplicación
Unique local	`fc00::/7`	Prefijo único de la organización	Análogo a las privadas IPv4

La link-local `fe80::/10` deserves atención: se genera automáticamente en cada interfaz sin configurar nada; **no se enruta** (un router jamás reenvía un paquete link-local fuera del enlace, por diseño); y es la dirección que la máquina usa **antes** de tener dirección global y para hablar con sus vecinos.

El formato, de memoria:

```
|3 bits|  | 13 bits prefijo        |            64 bits IID             |
2000::/3 →  001  0000 0000 0000     |  Interface Identifier              |

Ejemplo:  2001:0db8:0000:0000:0000:ff00:0042:8329
          └─/48 RIR─┘└─/16 ISP─┘└──── /64 subred ──────────────┘
```

El `/64` como unidad de subred, y por qué

Regla que hay que saber de memoria: en IPv6 el `/64` es la unidad de subred. No el `/48`, no el `/56`: **el `/64`**.

¿Por qué 64 bits de IID? Porque la generación automática (*SLAAC*) necesita **64 bits** para el identificador de interfaz:

```
1. EUI-64 (derivado de la MAC):
     MAC:  00:1B:44:11:3A:B7
     Se invierte el primer byte y se inserta FF:FE en el medio:
     IID:  02:1B:44:FF:FE:11:3A:B7
     ⚠️ Desventaja: preserva la MAC ⇒ la dirección ES un identificador
        único persistente ⇒ Privacy Extensions lo resolvió.

2. Privacy Extensions (RFC 4941): el default actual.
     IID ALEATORIO de 64 bits, que cambia cada cierto tiempo (días u horas),
     para que un observador externo NO pueda correlacionar la actividad a
     lo largo del tiempo. Default en Windows, macOS y Linux.
```

Los prefijos reservados y de uso, de memoria:

```
2000::/3   Unicast global          ← el 99 % de las direcciones
fc00::/7   Unique Local            ← como las privadas IPv4
fe80::/10  Link-Local              ← se genera sola
ff00::/8   Multicast               ← los grupos
(el resto: reservado para uso futuro)
```

1.7 ICMPv6 (`RFC 4443`)

> ⚠️ **ICMPv6 NO es opcional: es parte esencial de IPv6.** Un host IPv6 sin ICMPv6 **no funciona**: no resuelve direcciones de capa 2, no se configura solo, no sabe si un paquete es demasiado grande.

La razón profunda: IPv6 absorbió lo que en IPv4 eran **protocolos independientes** (ARP, RARP, ICMP) y lo **integró en un solo**, reduciendo el número de protocolos de la pila.

El formato del mensaje es el de ICMPv4 (tipo, código, checksum, cuerpo), con un campo de 32 bits obligatorio que identifica la cabecera IPv6 que generó el error.

Tipo	Nombre	Para qué
1	Destination Unreachable	Análogo al tipo 3 de IPv4
2	**Packet Too Big**	Lo emite el router cuando el paquete no entra. **Lleva el MTU**, y el origen **tiene que fragmentar**
3	Time Exceeded	Análogo al tipo 11. Lo usa `traceroute`
4	Parameter Problem	Análogo al tipo 12
128	**Echo Request**	`ping6`
129	**Echo Reply**	Respuesta de `ping6`
133	**Router Solicitation**	El host pide routers del enlace
134	**Router Advertisement**	El router se anuncia y manda prefijo y MTU
135	**Neighbor Solicitation**	"Who has X?" — el **ARP request** de IPv6
136	**Neighbor Advertisement**	"This is me" — el **ARP reply** de IPv6
137	**Redirect**	El router sugiere un mejor camino

Los cambios de numeración que hay que tener claros:

```
ICMPv4      ICMPv6        Cambio
──────      ──────        ──────────────────────────────────
Tipo 0      Tipo 129      Echo Reply:   0 → 129
Tipo 8      Tipo 128      Echo Request: 8 → 128
Tipo 3/4    Tipo 2        "Frag Needed" → "Packet Too Big"
Tipo 11     Tipo 3        Time Exceeded: 11 → 3
Tipo 12     Tipo 4        Parameter Problem: 12 → 4
(n/a)       Tipo 133-137  Neighbor Discovery (nuevo)
```

La diferencia crítica del tipo 2 frente al 3/4 de IPv4:

```
IPv4, tipo 3 código 4:  el router DICE "necesitás 1492" y el emisor
                        AJUSTA su MSS. El paquete NO se fragmenta más.

IPv6, tipo 2:           el router DICE "no entra, te paso 1280" y el
                        ORIGEN tiene que FRAGMENTAR él mismo (porque los
                        routers IPv6 NO pueden fragmentar). Después el
                        siguiente paquete va más chico, y así.
```

1.8 Neighbor Discovery vs. ARP — el apartado que más puntos da

Este es el `2010-12 P5 b)`, que pide "**no menos de 4 cambios significativos**". La solución oficial lista 5.

ARP (`RFC 826`) responde una pregunta: "**¿qué MAC corresponde a esta IP?**", usando **broadcast** de capa 2, y es un **protocolo independiente** definido aparte de IP.

Neighbor Discovery (ND) es parte de **ICMPv6** y resuelve **tres** preguntas:

```
ARP resuelve UNA cosa:
    1. ¿Qué MAC corresponde a esta IP?          (resolución de dirección)

ND resuelve TRES cosas:
    1. ¿Qué MAC corresponde a esta IP?          (resolución de dirección)
    2. ¿Qué prefijo y qué parámetros hay acá?   (autoconfiguración SLAAC)
    3. ¿Hay alguien más con MI dirección?       (detección de duplicados DAD)
```

Los 5 cambios significativos

(1) ND es parte de ICMPv6; ARP es un protocolo independiente.

```
IPv4:  App — TCP — IP — ICMP     ARP vive APARTE, en un ethertype distinto
                                  (0x0806), con su propio formato.

IPv6:  App — TCP — IPv6 — ICMPv6
                              └── ND (tipos 133–137) vive AQUÍ ADENTRO.
                                  No existe "ARP" en IPv6.

Por qué importa: una sola pila de protocolos, menos superficie de ataque,
menos código en el kernel, y **es imposible desactivar ND sin romper IPv6**.
```

(2) ND usa MULTICAST en lugar de BROADCAST.

```
ARP request:  FF:FF:FF:FF:FF:FF  (broadcast). TODOS lo reciben y lo procesan.

ND request:   dirección multicast SOLICITED-NODE,  ff02::1:ffxx:xxxx
              Se toman los últimos 24 bits de la IPv6 buscada y se anteponen
              ff02:0:0:0:0:1:ff00:

              Ejemplo: buscar 2001:db8::1234:5678:9abc
                últimos 24 bits:              34:5678:9abc
                solicitation:      ff02::1:ff00:34:5678:9abc

¿Por qué es mejor? Porque los routers NO reenvían ese multicast, así que el
scope NO sale del enlace. Con el broadcast de ARP, un router con proxy ARP
responde y el broadcast inunda cada segmento. El multicast es más eficiente
y más seguro: solo los nodos INTERESADOS responden.
```

(3) Los mapeos son link-layer ↔ IPv6 en vez de link-layer ↔ IPv4.

Cambio directo: ND mapea **MAC ↔ IPv6**, no IPv4. En la misma red pueden coexistir ARP (para IPv4) y ND (para IPv6), con cachés separadas.

(4) ND combina la resolución con la AUTOCONFIGURACIÓN y la DETECCIÓN DE DUPLICADOS.

Es el cambio conceptual más grande. En IPv4, ARP **solo resuelve MACs**: para tener una IP hay que usar DHCP o configurar a mano. En IPv6, ND hace las tres cosas, lo que permite **IPv6 *zero-touch***: el host se enchufa y **se configura solo**.

(5) Los nodos pueden usar mensajes NS para propagar cambios en su configuración.

```
En IPv4, si un host cambia su IP (failover de cluster, cambio de VLAN), los
demás se quedan con la caché ARP vieja hasta que expira. ARP no tiene un
"anuncio" de la IP nueva.

En IPv6, ND permite:
  - Anunciar la dirección nueva proactivamente (NA no solicitado) para que
    los vecinos actualicen su Neighbor Cache.
  - Detectar un nodo que se va (responde al NS con NA con Override = 1).
  - Detectar duplicados.
```

Tabla comparativa completa (esto es lo que hay que saber de memoria):

Aspecto	**ARP (IPv4)**	**Neighbor Discovery (IPv6)**
¿Qué protocolo es?	Independiente, `RFC 826`	**Parte de ICMPv6** (`RFC 4443`)
Corre sobre	El enlace (Ethernet)	**IPv6**
Identificador	Ethertype `0x0806`	Tipo ICMPv6 **135** (NS) / **136** (NA)
Solicitud	**Broadcast** (`FF:FF:...`)	**Multicast** a `ff02::1:ffxx:xxxx`
Respuesta	Unicast	Unicast (o multicast con Override)
Mapeo	MAC ↔ **IPv4**	MAC ↔ **IPv6**
Funciones	**1**: resolución	**3**: resolución + **autoconfig** + **DAD**
Detección de duplicados	**No**	**Sí** (DAD)
Autoconfiguración	**No** (DHCP o manual)	**Sí** (SLAAC vía RA)
Proxy ARP	**Sí**, muy usado	**No** (el modelo es punto a punto)
Mensajes	2 (request, reply)	**5** (RS, RA, NS, NA, Redirect)
Seguridad	Cero: ARP spoofing trivial	Similar; **secg** lo extiende

1.9 DAD — Duplicate Address Detection

El problema que resuelve. En IPv4, si dos equipos tienen la misma IP, el resultado es errático (cada uno contesta por su tabla ARP) y **nadie te avisa**.

En IPv6 esto sería inaceptable, porque **las direcciones se generan automáticamente**: es muy probable que dos hosts generen la misma. **Por eso existe DAD.**

```
Un host H quiere usar la dirección X (link-local o SLAAC).

Paso 1:  H pone X en estado "Tentative": todavía no la usa para nada.
         Arranca el RetransTimer en 1 segundo.

Paso 2:  H manda un Neighbor Solicitation (NS) para X, a la dirección
         solicited-node de X: "¿hay alguien con X?"

Paso 3:  H espera 1 segundo.

Paso 4:  ¿Recibió algún Neighbor Advertisement?
           - SÍ → alguien tiene esa dirección  ⇐  H NO USA X, elige otra.
           - NO → nadie la tiene               ⇐  X pasa a "Preferred" y se usa.
```

¿Por qué DAD no existe en IPv4? Tres razones:

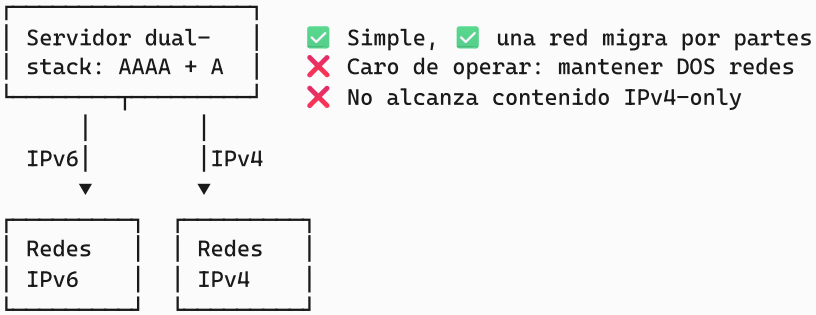
1. **En IPv4 la dirección se configura manualmente o por DHCP.** Dos hosts con la misma IP es un error humano raro. En IPv6 con **SLAAC** la dirección se genera automáticamente, así que la colisión es **estructuralmente más frecuente**.
2. **Dos NICs que comparten la MAC** (fallo de hardware, VM mal configurada) generan **exactamente la misma dirección**. **DAD detecta eso.**
3. **DAD necesita multicast** (NS/NA) para funcionar, y en IPv4 el ARP request es un **broadcast** que no sirve así (el ARP resuelve *después* de que la IP ya está en uso). **DAD inherentemente necesita el modelo de IPv6.**

1.10 Los 3 esquemas de transición

Este es el `2010-07 P5 b)`.

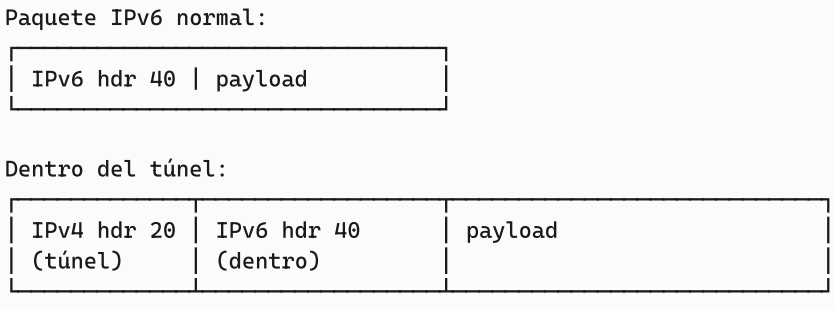
(a) Dual-stack — IPv4 e IPv6 en paralelo

Ambos protocolos en el mismo host y en la misma red. El tráfico elige versión según lo que se pueda resolver.



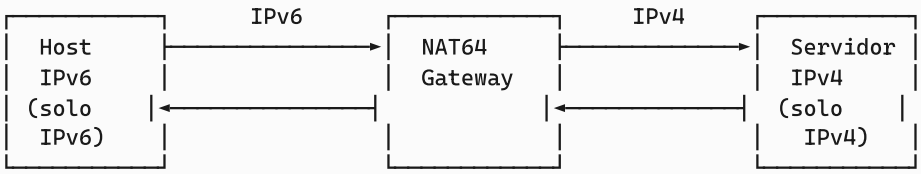
Ejemplo: hoy casi todo servidor web tiene **AAAA y A**. El navegador, con **Happy Eyeballs** (`RFC 8305`), los prueba en paralelo.

(b) Tunnelling — encapsular IPv6 dentro de IPv4



Túnel	Qué es	Para qué
Teredo	IPv6 dentro de IPv4 sobre **UDP/3544**	**NAT traversal**: dar IPv6 a hosts detrás de NAT IPv4. Asigna la IPv6 desde un servidor Teredo (`2001::/32`)
6to4	IPv6 dentro de IPv4 (protocolo 41)	Conectar **sitios IPv6 a través de la IPv4 Internet**, sin servidor. El prefijo IPv6 **embebe** el IPv4 público del sitio (`2002::/16`)
ISATAP	IPv6 dentro de IPv4 en el **mismo enlace**	Traducir IPv4↔IPv6 **dentro de una LAN**. IPv6 derivada del IPv4 del host

(c) Traducción — NAT64 / DNS64



El problema de DNS64: para que el host IPv6-only convierta un nombre a IPv6, el DNS64 devuelve una dirección IPv6 **sintetizada** que contiene la IPv4 real:

```
Prefijo bien conocido de NAT64:  64:ff9b::/96     (RFC 6052)

Servidor real IPv4:              192.0.2.33
Dirección IPv6 sintetizada:      64:ff9b::c000:221
                                      └──────┘
                                     c000:0221 = 192.0.2.33

El host pide example.com → DNS64 responde:
    AAAA: 64:ff9b::c000:221     (dirección inventada que "contiene" al IPv4)
El host conecta → NAT64 ve los últimos 32 bits = 192.0.2.33 → traduce y envía.
```

Protocolo	RFC	Año	Qué
NAT64	**`RFC 6146`**	2011	Traducción IPv6↔IPv4
DNS64	**`RFC 6147`**	2011	Síntesis de registros AAAA
Prefijo `64:ff9b::/96`	**`RFC 6052`**	2010	El prefijo bien conocido
`ipv4only.arpa`	`RFC 8880`	2020	Preguntar si un dominio tiene IPv4
SIIT	`RFC 6145` → **`RFC 7915`**	2011→2016	Traducción *stateless* (obsoleto)
DS-Lite	**`RFC 6333`**	2011	Dual-stack *stateless* ligero
464XLAT	**sigue en borrador**	—	Traducción en el **host**, no en la red

464XLAT en dos palabras: el **PLAT** (en el router de casa) traduce la salida, y el **CLAT** (en el dispositivo) traduce las apps que no soportan IPv6. Sirve para que un dispositivo **IPv6-only** alcance contenido **IPv4-only**. **La transición no cerró**, y ese es el dato de 2026.

	Dual-stack	Tunnelling	Traducción
Qué hace	Ambos en paralelo	Encapsula IPv6 en IPv4	Traduce IPv6↔IPv4
Ejemplo	Servidor con AAAA + A	Teredo, 6to4, ISATAP	NAT64/DNS64, 464XLAT
Alcanza IPv4-only	❌	✅	✅
IPv6 nativo	✅	❌	❌
Rompe el extremo a extremo	No	Sí (túnel)	**Sí**

1.11 La fragmentación en IPv6: el router NUNCA fragmenta

```
IPv4:  el ORIGEN manda un paquete grande
         ↓  el router lo FRAGMENTA si no cabe (con DF = 0)
       el destino REENSAMBLA

IPv6:  el ORIGEN manda un paquete grande
         ↓  el router NO PUEDE fragmentar
         ↓  lo DESCARTA y manda ICMPv6 "Packet Too Big" (tipo 2)
         ↓  el ORIGEN, avisado, FRAGMENTA con la cabecera de extensión
         ↓  y si tampoco entra, repite con un tamaño menor
```

¿Por qué? Porque la fragmentación en los routers es **el peor problema de IPv4** (overhead, solapamiento como agujero de seguridad, evasión de filtros, pérdida de un fragmento = pérdida del datagrama). **IPv6 lo elimina de raíz.**

Consecuencias positivas: routers **más simples y rápidos** (sin estado de reensamblaje ni offsets); el **ataque de fragmentación en capas desaparece**; el **PMTUD es obligatorio y bien definido**; y el **solapamiento se prohíbe** por diseño (`RFC 5722`).

	IPv4	IPv6
¿Quién fragmenta?	El **router** (si `DF = 0`)	**Solo el origen**
¿Cómo se señala?	Flags + Fragment Offset	**Cabecera de extensión Fragment** (44)
Si no cabe	Con `DF=1` descarta + ICMP 3/4; con `DF=0` **fragmenta**	Siempre descarta + **ICMPv6 tipo 2**
Solapamiento	Permitido en la práctica (agujero de seguridad)	**Prohibido** (`RFC 5722`)
Fragmentos atómicos	No existen	Sí, por compatibilidad (`RFC 6946`)
MTU mínimo	68 absoluto, 576 práctico	**1280** obligatorio

> ⚠️ **IPv6 fragmenta de otra manera:** en IPv4, si un paquete pasa por 3 routers con MTUs decrecientes, se fragmenta **en cada salto**, generando una cascada de fragmentos. En IPv6 el origen prueba, recibe el aviso del primer router que no pasa, y produce **todos los fragmentos de una vez**.

1.12 Actualización a 2026 — IPv6

Lo que sigue igual, de memoria, sin dudar:

- Dirección de **128 bits**, notación hex y compresión con `::` .

- **Cabecera fija de 40 bytes**: Version, Traffic Class, **Flow Label**, Payload Length, Next Header, Hop Limit, Src, Dst.
- **Cabeceras de extensión** encadenadas por `Next Header`. El **Payload Length excluye** las cabeceras de extensión.
- **Ámbitos**: unicast global `2000::/3`, **link-local** `fe80::/10`, unique local `fc00::/7`, multicast `ff00::/8`. **Site-local está deprecado.**
- `/64` **es la unidad de subred** estándar.
- **ICMPv6 (** `RFC 4443` **)**: ND (RS/RA/NS/NA, Redirect), **Packet Too Big (tipo 2)**, Destination Unreachable (1), Time Exceeded (3), Parameter Problem (4).
- **ND es "ARP para IPv6"** pero en **multicast sobre ICMPv6**, e incluye **autoconfiguración** y **DAD**.
- **Un router IPv6 NO fragmenta.** El **origen** fragmenta.
- **MTU mínimo absoluto de IPv6 = 1280** (`RFC 8200 §5`, verificado textualmente).

Lo que cambió respecto de 2012:

- **IPv6 dejó de ser "el futuro"**: el número del 28-mar-2026 y sus matices (§1.1).
- **El agotamiento de IPv4 es un hecho consumado** (RIPE, 25-nov-2019).
- **CGNAT es ubicuo en los ISP.**
- **NAT64/DNS64 (** `RFC 6146` **/** `6147` **) maduraron**; prefijo `64:ff9b::/96` = `RFC 6052`; `RFC 8880` (`ipv4only.arpa`).
- **SIIT** `RFC 6145` → **obsoleto por** `RFC 7915` (2016).
- **464XLAT sigue en borrador** — la transición no cerró.
- **DS-Lite =** `RFC 6333`.
- **NDP se amplió:** `RFC 9926` (feb-2026) registra prefijos entre routers vecinos.
- `RFC 6177` ya no recomienda `/48` por sitio final.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Escribir una cabecera IPv6 a partir de datos

Datos. Versión 6 · Traffic Class `0x93` (DSCP 0b100100 = 36 = AF31, ECN = 11 = CE) · Flow Label `0x0ABCDE` · Payload Length **1440** · Next Header **6** (TCP) · Hop Limit **64** · Src `2001:0DB8:0001:0000:0000:0000:0000:0001` · Dst `2001:0DB8:0002:0000:0000:0000:0000:0002`.

a) Campo por campo.

Campo	Valor	En hexadecimal	Bytes
Version	6 (4 bits) = `0110`	—	byte 0 (nibble alto)
Traffic Class	`0x93`	`93`	byte 0 (nibble bajo) + byte 1
Flow Label	`0x0ABCDE` (20 bits)	`0A BC DE`	bytes 2, 3, 4
Payload Length	1440	**`05 A0`**	bytes 5, 6
Next Header	6	`06`	byte 7
Hop Limit	64	`40`	byte 8
Source Address	`2001:0DB8:0001::1`	ver abajo	bytes 9–24
Destination Address	`2001:0DB8:0002::2`	ver abajo	bytes 25–40

b) Las direcciones, byte a byte. Cada grupo de 4 hex es un `uint16`:

```
2001:0DB8:0001:0000:0000:0000:0000:0001
↓    ↓    ↓    ↓    ↓    ↓    ↓    ↓
20 01  0D B8  00 01  00 00  00 00  00 00  00 00  00 01
```

c) Los 40 bytes completos, en hex y en orden:

```
Bytes  0–1:  69 3A          (Version 6, TC 0x93)
Bytes  2–4:  0A BC DE       (Flow Label 0x0ABCDE)
Bytes  5–6:  05 A0          (Payload Length = 1440)
Byte   7:    06             (Next Header = TCP)
Byte   8:    40             (Hop Limit = 64)
Bytes  9–24: 20 01 0D B8 00 01 00 00 00 00 00 00 00 00 00 01
Bytes 25–40: 20 01 0D B8 00 02 00 00 00 00 00 00 00 00 00 02
```

d) Verificación del `Payload Length`.

```
0x05A0 = 5 × 256 + 10 × 16 + 0 = 1280 + 160 = 1440  ✓
```

Interpretación. La cabecera ocupa **40 bytes fijos**. El `Payload Length` de 1440 **no incluye** esos 40: es el contenido. El **datagrama completo mide 1480 bytes** (40 + 1440). Si pasa por un enlace de MTU 1400, el router **descartará** y mandará **ICMPv6 tipo 2**.

e) ¿Y el MSS? En Ethernet (MTU 1500) el MSS TCP sería `1500 − 40 − 20 = 1420`, no 1460 como en IPv4. **Esa es la penalización del overhead:** 20 bytes más de cabecera IP.

2.2 Ejercicio 2 — Multicast vs. anycast: por qué anycast no es un tipo de dirección

a) Las diferencias.

	Multicast	Anycast
¿Tipo de dirección?	**Sí** (prefijo `ff00::/8`)	**No.** Servicio sobre unicast
¿Prefijo que lo marque?	**Sí**: `ff`	**No**
¿Lo sabe el emisor?	No	No
¿Lo sabe el router?	Por el prefijo `ff`	Por la **tabla de ruteo** (varios puntos anuncian el prefijo)
¿A cuántos destinatarios?	**Todos** del grupo	**Solo al más cercano**
Configuración	Ruteo multicast (IGMP/MLD)	**Ninguna**

b) El argumento formal. Un **tipo de dirección** es un **prefijo reservado** que le dice a la red "esto es especial": `ff00::/8` es multicast y **cualquier router lo reconoce por el primer byte**; `fe80::/10` es link-local y los primeros 10 bits lo dicen.

```
IPv6 multicast:   |1111 1111| ...   ⇐ el primer byte ES ff, se ve
IPv6 link-local:  1111 1110 10..    ⇐ los primeros 10 bits lo dicen
IPv6 unicast:     001 ...            ⇐ 2000::/3

IPv6 anycast:     001 ...            ⇐ EXACTAMENTE IGUAL que un unicast
                                     ⇐ No hay nada que lo distinga
```

El anycast no tiene prefijo reservado. La unicidad la resuelve el ROUTEO, no la numeración. El router ve que el prefijo está anunciado desde varios AS y elige el más cercano. El **paquete no sabe** que hay más de un destino: son **varios nodos que comparten la misma dirección**.

c) El ejemplo: los root servers de DNS.

```
Los 13 root servers de DNS tienen TODOS la misma dirección IPv4: 198.41.0.4
y la misma IPv6: 2001:503:ba3e::2:30

Cada uno está en un lugar distinto del mundo. Un ISP anuncia el prefijo
198.41.0.0/24 desde CUALQUIERA de sus puntos de presencia.

⇐ Tu consulta DNS va al root MÁS CERCANO, no al "primero" de la lista.
⇐ "Any" = cualquiera de los 13; "cast" = difundir.
```

d) Consecuencia práctica: anycast es un servicio de red, no una propiedad de la dirección.

```
✅ Puedo poner la MISMA dirección en 5 servidores míos y funciona: es anycast.
❌ NO puedo "convertir" una dirección existente en anycast sin anunciarla en
   BGP desde 5 puntos distintos. Eso es una decisión de RUTEEO.
⚠️ Si el anycast está mal configurado (vos y tu vecino anuncian el prefijo),
   algunos routers pueden enviar tráfico al lugar equivocado. Eso es un
   problema de BGP, no de IPv6.
```

2.3 Ejercicio 3 — ND vs. ARP en tabla comparativa completa

Este es el `2010-12 P5 b)`, con la respuesta lista para el examen.

a) El reemplazo de ARP: Neighbor Discovery (ND). Un **conjunto de 5 mensajes de ICMPv6**: RS (133), RA (134), NS (135), NA (136) y Redirect (137).

b) Los 5 cambios significativos (los de la solución oficial):

#	Cambio	Por qué importa
1	**ND es parte de ICMPv6, en su totalidad; se transporta sobre IPv6**	ARP era independiente, sobre el enlace. ND vive dentro de ICMPv6. Menos protocolos, y **no se puede desactivar sin romper IPv6**.
2	**Se utiliza multicast para Neighbor Solicitation en vez de broadcast**	Va a `ff02::1:ffxx:xxxx` (multicast link-local), no a `FF:FF:FF:FF:FF:FF`. Los routers **no reenvían** ese multicast: **no hay inundo de broadcast**.
3	**Los mapeos son de direcciones link-layer a IPv6 en vez de link-layer a IPv4**	Cambio directo: ND resuelve MAC ↔ IPv6.
4	**ND combina la resolución de direcciones con autoconfiguración y detección de duplicados**	ARP solo resuelve MACs. ND además **configura el host** (SLAAC) y **detecta colisiones** (DAD). Habilita el IPv6 *zero-touch*.
5	**Los nodos pueden usar mensajes NS para propagar cambios en su configuración**	Si un host cambia de dirección, manda un **NA no solicitado** para actualizar la caché de los vecinos. Con ARP, los demás quedan con la caché vieja hasta que expira.

c) La tabla comparativa completa está en §1.8 (última tabla), con 12 filas: protocolo, capa, identificador, solicitud, respuesta, mapeo, funciones, DAD, autoconfiguración, proxy ARP, cantidad de mensajes y seguridad.

2.4 Ejercicio 4 — Los stacks de referencia IPv4 e IPv6

Este es el `2010-12 P5 a)`.

```
Stack IPv4 (referencia)                Stack IPv6 (referencia)
──────────────────────                 ──────────────────────
Aplicación:                            Aplicación:
  HTTP, FTP, SMTP, DNS, SNMP,            HTTP, FTP, SMTP, DNS,
  Telnet                                 SNMP, Telnet

Transporte:                            Transporte:
  TCP (6), UDP (17)                      TCP (6), UDP (17)

Red:                                   Red:
  IPv4 + ICMPv4                          IPv6 + ICMPv6
  (puede fragmentar en routers)          (NUNCA fragmenta; ICMPv6 obligatorio)

Enlace:                                Enlace:
  Ethernet (802.3), PPP, HDLC,           Ethernet (802.3), PPP,
  Frame Relay, ATM, 802.11               ATM, 802.11

Física:                                Física:
  cobre, fibra, radio, satélite          cobre, fibra, radio, satélite
```

Protocolos de capa SUPERIOR con los que coexisten (≥2):

Protocolo	Puerto	¿Coexiste con IPv6?
TCP	6	**Sí**, es el mismo. Solo cambia la cabecera IP que lo lleva
UDP	17	**Sí**, igual
DNS	53	**Sí**, con los registros **AAAA** además de **A**
HTTP	80	**Sí**, el mismo protocolo; cambia solo el transporte debajo
SNMP	161	**Sí**, con *discovery* IPv6
DHCP	67/68	En IPv6 se usa **DHCPv6** (546/547), que es un protocolo **distinto**

Protocolos de capa INFERIOR con los que coexisten (≥2):

Protocolo	Por qué coexiste
Ethernet (802.3)	Los dos tipos de paquete viajan por el mismo cable; se distinguen por el **Ethertype**: `0x0800` = IPv4, **`0x86DD` = IPv6**
PPP	Un enlace PPP negocia **IPCP** (IPv4) y **IPV6CP** (IPv6) en la misma conexión
802.11 (Wi-Fi)	Un access point transmite IPv4 e IPv6 indistintamente
ATM / MPLS	Los túneles llevan IPv4 o IPv6 con la misma encapsulación

d) El punto clave: la capa 2 no cambia. Ethernet y Wi-Fi son **agnósticos** del contenido: solo miran el Ethertype. En IPv6 los dispositivos de capa 2 (switches, bridges) **no se modifican**; solo cambian los de capa 3 y 4. Esto es muy distinto de la era de NAT con IPv4, donde el CGN es un problema de capa 3 que **no existe en IPv6 puro**.

e) Por qué la coexistencia es obligatoria: (1) no todos los dispositivos soportan IPv6, y la transición tarda años; (2) la Internet actual es IPv4, así que un host IPv6-only necesita traducir o tunelizar; (3) **es la única migración posible**: no se cambia Internet de un día para otro, se hace dual-stack primero, túnel o transición después, y IPv6-only al final.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-12, Pregunta 5 (8 puntos) — IPv4 vs. IPv6, ND vs. ARP

> **Fuente:** Examen `eirc1012` / `sirc1012` — 17 de diciembre de 2010.
>
> **a)** Presente los stacks de referencia de ambos protocolos. Ejemplifique al menos dos protocolos con los que ambos coexisten de capa superior y al menos dos protocolos de capa inferior. **b)** Algunos protocolos definidos dentro de ICMP debieron ser re-definidos en ICMPv6. Mencione el reemplazo del protocolo ARP y describa no menos de 4 cambios significativos en su implementación en Ipv6.

Parte a) — Stacks de referencia

Resuelto en **§2.4**: los dos stacks completos, los dos protocolos de capa superior (**TCP** y **UDP** son los más claros; **DNS** con AAAA también), los dos de capa inferior (**Ethernet** y **PPP**), y la mención de que coexisten en la misma red física, distinguidos por el Ethertype (`0x0800` / `0x86DD`).

Puntos clave: los dos stacks, los 2 protocolos de capa superior, los 2 de capa inferior, y que **la capa 2 no cambia**. Con eso están los 4 puntos.

Parte b) — El reemplazo de ARP y los 4 cambios

Resuelto en **§2.3**; transcribo la respuesta lista para el examen:

El reemplazo: Neighbor Discovery (ND), un **conjunto de 5 mensajes de ICMPv6**: RS (133), RA (134), NS (135), NA (136) y Redirect (137).

Los cambios (los 5 de la solución oficial):

1. **ND es parte de ICMPv6, en su totalidad, y se transporta sobre IPv6.** No es un protocolo independiente como ARP: vive dentro de ICMPv6.
2. **Se utiliza multicast para Neighbor Solicitation en vez de broadcast.** El NS va a `ff02::1:ffxx:xxxx` (link-local), no a `FF:FF:FF:FF:FF:FF`.
3. **Los mapeos son de direcciones link-layer a IPv6 en vez de link-layer a IPv4.**
4. **ND combina la resolución de direcciones con la autoconfiguración y la detección de duplicados.** ARP solo resuelve; ND además configura el host (SLAAC) y detecta colisiones (DAD).
5. **Los nodos pueden usar mensajes NS para propagar cambios en su configuración.**

Justificaciones que suman puntos (no están en la lista oficial, pero demuestran comprensión):

- **Del cambio 2:** los routers **no reenvían** ese multicast, así que el scope **no sale del enlace**. Con ARP (broadcast) un router con *proxy ARP* responde y el broadcast inunda. El multicast es **más eficiente y más seguro**: solo los interesados responden.
- **Del cambio 4:** ND habilita el **IPv6 zero-touch**: el host se enchufa y se configura solo, sin DHCP.
- **Sobre el cambio 5:** con ARP, si un host cambia su IP (failover, roaming), los demás quedan con la caché vieja hasta que expira. Con ND, manda un **NA no solicitado** para actualizarla proactivamente.

> **Nota sobre la solución oficial:** la parte a) de la solución oficial está **vacía** en el PDF (solo viene la b). Si en el examen te piden a) y b), resolvé a) por tu cuenta con los stacks de referencia, que es estándar de la materia.

3.2 Examen 2009-12, Pregunta 5 (6 puntos) — DAD y NDP

> **Fuente:** Examen `sirc0912` — diciembre 2009. **Enunciado:** Considere ICMPv6, familia de protocolos de control en IPv6. El mismo implementa diversos protocolos… a) [mencione dos protocolos que usan NS/NA]. b) DAD no está implementado en IPv4. NDP es la versión IPv6 de ARP, que en vez de ser un protocolo independiente de IPv(v4), también utiliza mensajes IPv6 para su implementación.

Respuesta oficial:

> **a) DAD** – Duplicated Address Detection protocol y **NDP** – Neighbor Discovery Protocol. **b)** DAD no está implementado en IPv4. NDP es la versión IPv6 de ARP, que en vez de ser un protocolo independiente de IP(v4), también utiliza mensajes IPv6 para su implementación.

Ampliación del apartado a): los dos protocolos de ICMPv6 que usan NS/NA.

1. **DAD (Duplicate Address Detection).** Usa **Neighbor Solicitation (135)** para preguntar si una dirección ya está en uso, y espera un **Neighbor Advertisement (136)**. Es la única función que consulta *proactivamente* si su **propia** dirección está libre.
2. **NDP (Neighbor Discovery).** Usa **NS/NA** para el mapeo MAC ↔ IPv6 (resolución), y además para la autoconfiguración y la detección de duplicados.

Comparación con IPv4:

Aspecto	IPv4	IPv6
ARP	Protocolo independiente (`RFC 826`)	**Parte de ICMPv6** (`RFC 4443`)
DAD	**No existe**	**Sí**, todo host lo hace antes de usar una dirección
Solicitud	**Broadcast**	**Multicast** (solicited-node)
Proxy ARP	**Sí**	**No**
Autoconfiguración	No (DHCP o manual)	**Sí** (SLAAC)

¿Por qué DAD no existe en IPv4? Las tres razones de §1.9: (1) en IPv4 la dirección se configura manualmente o por DHCP, así que la colisión es un error humano raro, mientras que con **SLAAC** es estructuralmente frecuente; (2) dos NICs con la misma MAC generan **exactamente la misma dirección**, y DAD lo detecta; (3) **DAD necesita multicast** (NS/NA), y en IPv4 el ARP request es un **broadcast** que no sirve así.

3.3 Examen 2010-07, Pregunta 5 (8 puntos) — modelos, tipos y transición

> **Fuente:** Examen `sirc1007` — julio 2010.

> **a)** En el curso se presentaron los tres modelos de comunicación soportados por IPv6, con sus correspondientes tres tipos de direcciones. Mencione cada uno de los tres tipos y descríbalos en una o dos frases a cada uno. **b)** Los esquemas de transición pueden agruparse en tres tipos principales. Descríbalos brevemente y da un ejemplo de cada uno. **c)** Suponga que se transmite un segmento UDP de 1000 bytes de largo sobre IPv4 y sobre IPv6. ¿En qué caso se transmiten más bytes? Fundamente.

Respuesta oficial: a) y b) remiten a las transparencias; **c)** "Sin opciones, un cabezal IPv6 tiene 40 bytes de largo y el cabezal IPv4 tiene 20 bytes de largo. A mismo payload, IPv6 tiene mayor overhead. Por consiguiente, se transmitirán más bytes con IPv6."

Parte a) — Los 3 modelos y los 3 tipos

Resueltos en **§1.5** y **§1.6**. Resumen para el examen:

Modelo	Tipo de dirección	En una frase
Unicast	**Unicast global** (`2000::/3`)	Un emisor, un receptor; cada dirección identifica un nodo. El 99 % del tráfico
Multicast	**Multicast** (`ff00::/8`)	Un emisor, muchos receptores; el prefijo `ff` limita la difusión según el ámbito (`ff02::/16` = enlace)
Anycast	*(no es un tipo de dirección)*	Muchos emisores, muchos receptores; **varios nodos con la MISMA dirección unicast**, y el router encamina al más cercano

Punto clave: aunque el enunciado dice "tres modelos con sus correspondientes tres tipos de direcciones", **el anycast no tiene tipo de dirección propio**: usa una dirección unicast normal. Si hay que forzar la correspondencia, el "tipo" del anycast es **unicast**, y lo especial es que **varios nodos la comparten**.

Parte b) — Los 3 esquemas de transición

Resueltos en **§1.10**: **dual-stack** (ej: servidor con AAAA + A), **tunnelling** (ej: Teredo, 6to4, ISATAP) y **traducción** (ej: NAT64/DNS64, `RFC 6146` / `6147`). Para el apartado basta una frase por esquema y un ejemplo.

Parte c) — El cálculo del overhead

```
Segmento UDP de 1000 bytes de payload.

Sobre IPv4:
   cabecera IP  = 20 bytes (sin opciones)
   segmento UDP = 8 (cabecera UDP) + 1000 (payload) = 1008
   TOTAL        = 20 + 1008 = 1028 bytes

Sobre IPv6:
   cabecera IP  = 40 bytes (fija)
   segmento UDP = 8 + 1000 = 1008
   TOTAL        = 40 + 1008 = 1048 bytes

⇐ Se transmiten MÁS BYTES con IPv6: 1048 > 1028
Diferencia exacta = 40 − 20 = 20 bytes = 20/1028 = 1,9 % de overhead extra.
```

Interpretación. Es el **costo perenne** de la transición: 20 bytes más de cabecera por datagrama. En conexiones con muchos paquetes chicos eso **consume ancho de banda**. Las mitigaciones: las cabeceras de extensión

agregan overhead por otro lado, y las **jumbograms** (`Payload Length = 0` con la opción Jumbo Payload) permiten datagramas tan grandes que el overhead relativo es despreciable.

3.4 Examen 2007-03, Pregunta 3 (10 puntos) — RARP

> **Fuente:** Examen `sirc0703` — marzo 2007. **Enunciado:** Describa en que escenario se utiliza el protocolo RARP y su funcionamiento.

Respuesta oficial:

> ARP resuelve el problema de encontrar **qué** dirección Ethernet corresponde a una dirección IP dada. A veces se debe resolver el problema **reverso**: dada una dirección Ethernet, ¿cuál es la dirección IP que le corresponde?
>
> Esto ocurre por ejemplo en el escenario en que hay **estaciones de trabajo tontas**, sin disco, que deben ser reiniciadas. Dicha máquina normalmente obtiene la imagen del sistema operativo de un archivo remoto, pero **antes debe tener una dirección IP**.
>
> La primera solución que se encontró fue usar **RARP**. Este protocolo permitía a las máquinas reiniciadas hacer un **broadcast de Ethernet** enviando una consulta de IP con su dirección Ethernet. Un servidor de RARP recibe la solicitud, realiza una búsqueda en sus archivos de configuración y envía una respuesta con la dirección IP.
>
> Usar RARP es **mejor que grabar la IP en la imagen del sistema operativo** ya que permite **usar la misma imagen para todos los equipos**.
>
> La **desventaja** de RARP es que los **broadcast Ethernet no son reenviados por los routers**, por lo que se necesita un **servidor RARP en cada subred**. Por ello se diseñaron otros protocolos como **BOOTP y DHCP**.

Análisis y ampliación para los 10 puntos.

Los tres elementos que se piden están en la oficial: el **escenario** (workstation tonta sin disco, boot remoto), el **funcionamiento** (broadcast de Ethernet con la MAC → el servidor busca en su tabla → responde con la IP) y la **comparación con ARP** (ARP va de IP → MAC; **RARP va de MAC → IP**, el problema inverso).

Lo que agrego:

(a) El escenario exacto. Un equipo sin disco no tiene dónde guardar su configuración de red. Necesita (1) una IP para hablar con la red, y (2) después, la imagen del OS por TFTP. **RARP resolvía el punto (1)**, y TFTP el (2).

(b) Por qué es mejor que la IP fija en la imagen. Es una razón de **mantenimiento**:

```
Con IP fija en la imagen:
  Cada equipo tiene una imagen DISTINTA (con su IP adentro).
  500 PCs = 500 imágenes que mantener.
  Cambiar el plan de numeración = rehacer 500 imágenes.

Con RARP:
  Todos usan LA MISMA imagen genérica.
  El plan de numeración vive SOLO en el servidor RARP.
  ⇐ Un solo lugar que actualizar.
```

(c) Las limitaciones que prepping su sucesión:

1. **No se reenvía por los routers.** Un broadcast de Ethernet **se queda en el enlace** ⇒ **un servidor RARP por subred**. Con muchos enlaces, son muchos servidores.
2. **RARP solo devuelve la IP.** No devuelve máscara de subred, ni gateway, ni DNS. Para eso están BOOTP y DHCP.
3. ⚠️ **Tema en desuso.** RARP es de **1984** y **hoy está completamente reemplazado por DHCP**, que hace todo lo que RARP hacía y mucho más. No se despliega. Se pregunta por historia y por el **concepto** de "resolver el problema inverso", que reaparece en el **DHCP Discover**: el cliente (que solo tiene su MAC) pregunta "soy esta MAC, ¿qué configuración me corresponde?", y el servidor responde con IP, máscara, gateway, DNS y la **duración** del lease.

La línea histórica que hay que llevarse:

```
RARP (1984)     →  resuelve solo la IP
    ↓
BOOTP (1984)    →  IP + más parámetros, pero con la IP fija en el archivo
    ↓
DHCP (1993)     →  IP + máscara + gateway + DNS + duración del lease + más
    ↓
SLAAC (IPv6)    →  el host se autoconfigura SOLO, leyendo los RA de los
                   routers, y DAD verifica que no haya colisión
```

Dato clave: el **DHCP Discover es, en esencia, un RARP mejorado**.

PARTE 4 — CIERRE

📌 Resumen

1. **El agotamiento de IPv4 es un hecho**: RIPE cerró su última asignación /22 el **25-nov-2019**. El **28-mar-2026** IPv6 superó el **50,10 %** de acceso a Google, pero **la cifra depende de qué se mide** y es desigual (China < 5 %).
2. La **cabecera IPv6 son 40 bytes fijos**. **Desaparecen** `Header Checksum`, `IHL`, `Fragment Offset` y `Options`. **Aparece** el **Flow Label** de 20 bits.
3. El **Flow Label** (`RFC 6437`) identifica un flujo **sin la 5-tupla**, útil con tráfico **fragmentado** o **cifrado**. Retrocompatible: si nadie lo usa, se comporta como IPv4.
4. Las **cabeceras de extensión** se encadenan por `Next Header`, y el `Payload Length` **las EXCLUYE** (bug de `RFC 2460`, corregido por `RFC 2719`).
5. Los **3 modelos**: **unicast** (1→1), **multicast** (`ff00::/8`, 1→muchos, con ámbito), **anycast** (muchos→muchos). **Anycast NO es un tipo de dirección**: es un servicio sobre unicast.
6. Los **tipos de dirección**: unicast global `2000::/3`, **link-local** `fe80::/10` (se genera sola), multicast `ff00::/8`, **unique local** `c00::/7`. Site-local está deprecado.**
7. El `/64` **es la unidad de subred**, porque SLAAC necesita 64 bits de IID (EUI-64, o aleatorio con Privacy Extensions, que es el default actual).
8. **ICMPv6 es esencial**: Packet Too Big (2), Destination Unreachable (1), Time Exceeded (3), Parameter Problem (4), Echo (128/129), ND (133-137).
9. **ND vs. ARP**: ND es parte de ICMPv6, usa **multicast**, mapea ↔IPv6, **combina 3 funciones** y **propaga cambios**. Son 5 mensajes (RS, RA, NS, NA, Redirect).
10. **DAD** detecta duplicados con NS + temporizador de 1 s. No existe en IPv4 porque las direcciones se configuran manualmente. Los **3 esquemas de transición** son **dual-stack**, **tunnelling** (Teredo, 6to4, ISATAP) y **traducción** (NAT64/DNS64, 464XLAT).

✅ Autoevaluación

1. ¿Por qué desapareció el `Header Checksum` de la cabecera IPv6? ¿No es un problema no verificarlo?

▶ Respuesta

2. ¿Qué es el `Flow Label` y para qué sirve, si ya tenemos la 5-tupla?

▶ Respuesta

3. ¿Por qué anycast no es un tipo de dirección?

▶ Respuesta

4. ¿Por qué existe DAD en IPv6 y no en IPv4?

▶ Respuesta

5. ¿Qué diferencia hay entre `Packet Too Big` (ICMPv6 tipo 2) y `Fragmentation Needed and DF set` (ICMPv4 tipo 3 código 4)?

▶ Respuesta

6. ¿Por qué el `Payload Length` de IPv6 excluye las cabeceras de extensión? ¿Qué problema resuelve?

▶ Respuesta

7. ¿Por qué un router IPv6 nunca fragmenta? ¿Qué consecuencias tiene?

▶ Respuesta

8. En el esquema **dual-stack**, ¿qué ventaja y qué desventaja tiene para un operador?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1012` / `sirc1012`** — P5 (8 pts: stacks de referencia, coexistencia, **ND vs. ARP con ≥4 cambios**)
- **Examen `sirc0912`** — P5 (6 pts: DAD y NDP, comparación con IPv4)
- **Examen `sirc1007`** — P5 (8 pts: 3 modelos de comunicación, 3 tipos de dirección, 3 esquemas de transición, overhead IPv4 vs. IPv6)
- **Examen `sirc0703`** — P3 (10 pts: RARP, escenario y funcionamiento)
- **Examen `sirc0907`** — P5 (ventajas y desventajas de IPv6)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.6 (IPv6)
- **RFC 8200** — Internet Protocol, Version 6 (2017, *Internet Standard*; **obsoleta RFC 2460**; §5 fija el **MTU mínimo de 1280**)
- **RFC 4443** — ICMPv6 for IPv6 (tipos 1-4 y 128-137)
- **RFC 6437** — Flow Label (2012)
- **RFC 2719** — Corrección del bug del `Payload Length` de `RFC 2460`
- **RFC 6052** — Prefijo bien conocido `64:ff9b::/96` para NAT64 (2010)
- **RFC 6146** — NAT64 (2011) · **RFC 6147** — DNS64 (2011)
- **RFC 7915** — SIIT (2016, **obsoleta RFC 6145**) · **RFC 6333** — DS-Lite (2011)
- `draft-ietf-v6ops-464xlat` — **464XLAT sigue en borrador**
- **RFC 5722** — Fragmentos solapados en IPv6 (prohibidos) · **RFC 6946** — Fragmentos atómicos · **RFC 5095** — depreca Routing Header tipo 0
- **RFC 8305** — Happy Eyeballs (2017, *obsoleta RFC 6555*)
- **RFC 9926** — NDP: registro de prefijos entre routers vecinos (feb-2026)
- **Clase siguiente:** 37 — ICMP, ping y traceroute + Repaso y Simulacro 4

Clase 37

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
A.1 Por qué existe ICMP
A.2 El formato del mensaje ICMPv4
A.3 La tabla de tipos y códigos de ICMP — de memoria
A.4 ICMP no es confiable, y no genera errores sobre errores
A.5 `ping`
A.6 `traceroute` — el Examen 2011-03 P3
B.1 El mapa mental del Bloque D (clases 24 a 37)
B.2 Todas las fórmulas y reglas, de memoria
B.3 Errores frecuentes — las 8 confusiones más comunes
Enunciado — Preguntas Teóricas
Enunciado — Problemas Prácticos
✅ RESPUESTAS DEL SIMULACRO
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Los 6 bloques del curso

Bloque: D (Capa de Red) · **Semana:** 13 · **Duración estimada:** 180 min (≈70 min teoría + 110 min repaso y simulacro) **Correlativa:** Clase 36 (IPv6) · **Sigue con:** Clase 38 (Capa de enlace: servicios y framing) **Material de referencia:** Tanenbaum cap. 5 (§5.4.1) · RFC 792, RFC 1122, RFC 4443, RFC 8201

🎯 Al terminar esta clase vas a poder

1. Explicar **por qué existe ICMP** (el problema de anidamiento), su formato, y por qué **no es un protocolo confiable** ni genera errores sobre errores.
2. **Reconstruir de memoria la tabla de tipos y códigos de ICMP**, incluidos los **obsoletos** (Source Quench, Address Mask).
3. Explicar qué hace `ping`, qué mide, y por qué es **la herramienta de diagnóstico del PMTU**.
4. Explicar **cómo funciona** `traceroute`: el campo TTL, los dos sabores (UDP e ICMP), los **tipos de mensaje** involucrados y **los nodos que colaboran**.
5. Explicar **las limitaciones honestas** de `traceroute`: el `*` no significa caída, y el camino de vuelta puede ser distinto.
6. **Repasar en forma densa** las clases 24 a 37: fórmulas, reglas, números de ICMP, atributos de BGP, mensajes de OSPF, modelos de IPv6.

7. **Rendir un simulacro completo** de 100 puntos en 180 minutos.

> **Prerequisito del examen:** `2011-03 P3` (6 pts — **principio de funcionamiento de `traceroute`**, tipos de mensaje y nodos que colaboran), `2010-12 Problema 2` (la **tabla de tipos y códigos de ICMP** es material de estudio por sí sola), `2010-12 P5` (IPv6), más todo el bloque D.

PARTE A — TEORÍA: ICMP, PING Y TRACEROUTE

A.1 Por qué existe ICMP

El problema de fondo: los errores de IP no se pueden reportar en IP.

```
Un paquete IP llega a un router y no se puede entregar.
¿Cómo se lo avisas al emisor?

Opción A: mandar OTRO paquete IP de error.
⇐ Pero si el problema es que el destino está inalcanzable, o que la red
está caída, o que hay un loop, el paquete de ERROR tampoco va a llegar.
⇐ Y si lo que falló fue el PROPIO router (TTL vencido, tabla de ruteo
vacía), el paquete de error tiene que salir por una ruta que
probablemente tampoco funciona.

⇐ IP no puede hablar de sí mismo: es un problema de ANIDAMIENTO.
```

La solución: un protocolo de capa 3 paralelo, definido en `RFC 792` (1981), que viaja **dentro** de IP pero es conceptualmente distinto: los errores van **de un nodo a otro**, no de un host a un host de capa superior.

ICMP = *Internet Control Message Protocol*. Sus dos funciones:

1. **Diagnóstico y control de errores**: reportar que un paquete no pudo entregarse.
2. **Diagnóstico de red**: `ping`, `traceroute`.

A.2 El formato del mensaje ICMPv4

```
┌──────────┬──────────┬──────────────────┬──────────────────────────────┐
│ Type     │ Code     │ Checksum         │ Cuerpo…                      │
│ (tipo)   │(código)  │                  │ (varía según tipo y código)  │
│ 8 bits   │ 8 bits   │ 16 bits          │                              │
└──────────┴──────────┴──────────────────┴──────────────────────────────┘
byte 0 byte 1 bytes 2–3

Ejemplo: un mensaje de "Time Exceeded" (tipo 11) lleva en el cuerpo:
los 8 bytes del datagrama que expiró (cabecera IP) + los primeros 64
bytes de ese datagrama.
⇐ ¿Por qué? Para que el emisor pueda IDENTIFICAR el paquete que
expiró sin tener que esperar a que llegue a la capa 4.
```

Para `ping` y `traceroute` los dos campos que importan son los del cuerpo:

Campo	Tamaño	En Echo Request/Reply
Identifier	16 bits	Identifica **qué par de `ping`** es este (número de proceso o secuencia)
Number / Sequence	16 bits	El **número de secuencia** dentro de ese `ping`. Permite emparejar request con reply y detectar pérdida

El campo `Code`, de memoria: hoy casi solo se usa `0`, pero en **Destination Unreachable** es donde tiene todo el sentido (ver la tabla de §A.3).

A.3 La tabla de tipos y códigos de ICMP — de memoria

> ⚠️ **El `2010-12 Problema 2` incluye la tabla oficial de tipos y códigos como dato del examen. Esa tabla es material de estudio por sí sola.** Reconstruila entera y sabé en particular **los códigos del tipo 3**.

Los tipos (0 a 12), con lo que hay que saber de cada uno:

Tipo	Nombre	Qué es / para qué
0	**Echo Reply**	Respuesta de `ping`. **No es un error.**
3	**Destination Unreachable**	El paquete no pudo entregarse. **Sus códigos importan mucho** (tabla de abajo).
4	**Source Quench**	**OBSOLETO.** "Estás mandando demasiado rápido, frená". **Hoy no lo genera nadie.** Ver abajo.
5	**Redirect**	"Hay un camino mejor". **Casi no se usa en IPv4 moderno** (ver abajo).
8	**Echo Request**	El `ping`. **No es un error.**
9	**Router Advertisement**	Un router se anuncia (usado por RIP y por *router discovery*)
10	**Router Solicitation**	Un host pide que los routers se anuncien
11	**Time Exceeded**	**TTL se venció en tránsito.** El que usa `traceroute`
12	**Parameter Problem**	Un campo de la cabecera IP es inválido
13	**Timestamp / Timestamp Reply**	reloj de un nodo. **Obsoleto en la práctica**
17	**Address Mask Request / Reply**	**OBSOLETO.** Pide la máscara de subred. **Reemplazado por DHCP**

Los CÓDIGOS del tipo 3 (Destination Unreachable) — la parte que más se pregunta, tal como figura en la tabla del `2010-12`:

```
0 Net Unreachable ← la RED no existe
1 Host Unreachable ← el HOST no existe
2 Protocol Unreachable ← no hay proceso para ese protocolo
3 Port Unreachable ← no hay proceso escuchando en ese puerto
4 Fragmentation Needed and Don't Fragment was Set ← el que dispara el PMTUD
5 Source Route Failed ← falló una ruta source-routed
6 Destination Network Unknown
7 Destination Host Unknown
8 Source Host Isolated ← obsoleto
9 Communication with Destination Network is Administratively Prohibited
10 Communication with Destination Host is Administratively Prohibited
12 Parameter Problem ← también existe como tipo 12
```

⚠️ **Sobre la lectura de esa tabla en el PDF del 2010-12:** el examen la imprime en **dos columnas desalineadas** (la de nombres de tipos y la de códigos del tipo 3 quedaron desfasadas una posición), así que el texto extraído parece corrido. Leída correctamente es la tabla canónica de `RFC 792`, que es la de arriba. **Lo que hay que retener sin ambigüedad:** el **3/4** ("Fragmentation Needed and DF Set", el del PMTUD) y el **3/3** ("Port Unreachable", el que cierra `traceroute` en su sabor UDP).

Los que están OBSOLETOS, explícitamente:

- `Source Quench` **(tipo 4) — OBSOLETO.** La idea era que el router le dijera al emisor "estás saturando, mandá menos". **Hoy no lo genera nadie**, por una razón concreta: **creaba una tormenta de mensajes.** Un router congestionado generaba un Source Quench por cada paquete descartado, lo que **aumentaba la congestión**. La.stderr se resolvió de otra forma: **dejar que la pérdida sea la señal** (que es lo que hacen los algoritmos de control de congestión) y, mejor aún, con **ECN**, que marca en lugar de descartar. ⚠️ **No lo propongas nunca como solución a un problema de congestión.**
- `Address Mask Request/Reply` **(tipo 17) — OBSOLETO.** Servía para que un host **descubriera la máscara de subred** por broadcast. **Hoy la máscara viene en el DHCP**, y en IPv6 la dirección ya trae el prefijo. Es historia pura.

`Redirect` **(tipo 5) — por qué casi no se usa en IPv4 moderno:**

```
El Redirect dice "hay un camino mejor que este, andá por allá". El problema:
para que sea confiable, el host tendría que CONFIRMAR que el router que se lo
dice es legítimo. Con ARP sin autenticar, un atacante puede inyectar Redirects
y hacer traffic blackholing.

Además, en IPv4 los routers el Redirect solo entreStub (routers de un
único enlace), por lo que casi nunca se usa entre routers de tránsito.

⇐ En IPv6 el equivalente (tipo 137) existe y es más usado, porque ND sí
tiene mecanismos de seguridad asociados (Secure NDP / SEND).
```

A.4 ICMP no es confiable, y no genera errores sobre errores

Dos reglas de diseño que hay que saber de memoria:

1. ICMP no es un protocolo de entrega confiable. No hay ACK de los ACKs, no hay retransmisión, no hay control de congestión. Un ICMP **se puede perder** como cualquier otro paquete, y **no se retransmite**.

```
⇐ Consecuencia directa: un "Destination Unreachable" perdido NO genera
otro error. El emisor simplemente no se entera de que su paquete falló.
⇐ Y esto es lo que hace que los Firewalls puedan filtrar ICMP "sin romper
la aplicación"... salvo que se need el PMTU (black hole, Clase 35).
```

2. No se generan mensajes ICMP sobre mensajes ICMP (con excepciones estrictas):

```
⇐ Por qué: si un router no puede entregar un ICMP, no emite otro ICMP
diciendo que no pudo entregar el ICMP. Y así, hacia atrás, indefinidamente.
Sería una tormenta garantizada.

⇐ Las EXCEPCIONES (que hay que conocer):
- ICMP tipo 3 código 4 (Fragmentation Needed): SÍ puede sobre otro
ICMP, porque el router necesita que el emisor reduzca el tamaño.
- ICMP tipo 3 código 2/3 sobre mensajes ICMP: en la práctica los routers
modernos SÍ los generan (por ejemplo, un "port unreachable" porque nadie
escucha ICMP en ese puerto).
⇐ La regla de "no generar sobre ICMP" es de la era de RFC 1122 y los
routers reales la relajaron en estos casos concretos.
```

El detalle fino del Report Time Exceeded con datos del datagrama que expiró:

```
ICMP Time Exceeded lleva en el cuerpo:
┌──────────────────────────────────┬──────────────────────────────────┐
│ cabecera IP del datagrama que    │ primeros 64 bytes del            │
│ expiró (20 bytes)                │ datagrama original               │
└──────────────────────────────────┴──────────────────────────────────┘

¿Por qué? Porque el emisor puede así RECONOCER qué paquete se perdió y
(en el caso de traceroute) Extract los datos que le interesan.

En el caso de un Echo Request de traceroute, esos 64 bytes incluyen el
campo Identifier y el Number Sequence ⇒ el emisor puede emparejar el
Time Exceeded con el probe que lo generó.
```

⚠️ Actualización a 2026 — ICMP no envejeció, pero su entorno sí

Sigue igual — de memoria, sin dudar: - El **formato** del mensaje ICMPv4: tipo, código, checksum de 16 bits, identificador, número de secuencia. **No cambió en 30 años.** - Los **números** de tipo y código que se piden: 0, 3 (con sus 13 códigos), 8, 11, 12, 13, 17. - El principio de **traceroute**: manipular el **TTL** para que cada router intermedio devuelva un error y así revelar el camino de ida. - Que **un router nunca genera ICMP sobre otro ICMP**, y que ICMP **no es confiable** (va sobre IP, no sobre TCP).

Cambió o seerteó: - `Source Quench` **(tipo 4) y** `Address Mask` **(tipo 17) están obsoletos.** El primero se usaba para pedir que el emisor bajara la tasa; se demostró que era **contraproducente** (el emisor respondsba al cuelgue de la app, no a la red) y hoy **nadie lo genera**. El segundo quedó inútil con CIDR. Si un apunte los lista como vigentes, está viejo. - `ICMPv6` **(** `RFC 4443` **) es obligatorio en IPv6** y es mucho más rico que ICMPv4: incluye **Neighbor Discovery** (RS, RA, NS, NA, Redirect), **Packet Too Big** (tipo 2), **Parameter Problem** (tipo 4) y **Path MTU Discovery** (`RFC 8201`). Cuando pregunten "ICMP" en un contexto de IPv6, la respuesta es **ICMPv6**, no ICMPv4. - **Los mensajes de control de** `traceroute` **hoy se filtran más.** Por eso `mtr` (que combina `ping` y `traceroute` con losses por nodo) es más útil que `traceroute` a secas, y por eso existen las **capturas pasivas** de flows. - `ping -M do -s <tamaño>` **sigue siendo la herramienta de diagnóstico del PMTU** en Linux, y `tracepath` muestra la MTU de cada salto directamente. Es la respuesta moderna al **PMTUD black hole** cuando el ICMP no vuelve.

Consecuencia para el examen: la Pregunta 3 de 2011-03 (traceroute) y la Pregunta 5 de 2010-12 (tipos y códigos de ICMP) se responden **exactamente como en 2012**. Lo único que puede cambiar es que te pidan marcar los tipos obsoletos.

A.5 `ping`

Qué es, mecánicamente:

```
⇐ Emisor ──────── ICMP Echo Request (tipo 8, código 0) ──────────► Receptor
◄──────── ICMP Echo Reply (tipo 0, código 0) ──────────────────

⇐ En la cabecera del Echo Request va un campo IDENTIFIER (id de proceso)
y un campo SEQUENCE NUMBER (número de secuencia).
⇐ El Reply copia esos mismos valores y calcula el RTT.
```

Qué mide, en la práctica:

```
RTT = (t_recv - t_send)

Repetir N veces (por defecto 4 en Unix) y sacar:
- el RTT mínimo, el promedio, el máximo
- la desviación estándar (jitter)
- el PORCENTAJE DE PÉRDIDA

⇐ El INTERESANTE no es el RTT absoluto sino la VARIACIÓN.
Una línea base de 40 ms es una red sana.
Una línea base de 40 ms que a veces sube a 300 ms tiene cola (congestión).
```

El `ping` grande con DF como herramienta de diagnóstico del PMTU — esto es lo que hay que acordarse:

```
# Linux: -M do = poner DF=1 ; -s = tamaño del payload ICMP
ping -M do -s 1400 ejemplo.com # si falla → el cuello de botella está
 # por debajo de 1400 + headers
ping -M do -s 1000 ejemplo.com # si funciona → el cuello está entre
 # 1000 y 1400
ping -M do -s 1400 -c 3 ejemplo.com # 3 intentos, para distinguir de ruido

# tracepath muestra el PMTU de cada salto directamente
tracepath ejemplo.com

# Windows: ping -f -l 1400 ejemplo.com (-f = DF, -l = tamaño del payload)
```

⚠️ **El `ping` CON DF tiene un detalle importante:** si el payload es demasiado grande y el firewall **filtra** el ICMP tipo 3 código 4, el `ping` muestra `Request timeout` en vez de `Frag needed and DF set`. **Eso es el black hole** (Clase 35, §1.9). Un `ping` normal (sin DF) fragmenta y por eso "funciona", mientras que las conexiones reales (que usan DF) no. **Esa asimetría es el síntoma clásico.**

A.6 `traceroute` — el Examen 2011-03 P3

Este es el apartado central de la Parte A y vale **6 puntos**. Vamos por partes.

La idea

`traceroute` **exploita el campo TTL** de la cabecera IP. El TTL es un contador que **cada router decrementa en 1**. Cuando el TTL llega a **0**, el router **descarta el paquete** y le manda al emisor un **ICMP "Time Exceeded"** (tipo 11, código 0, "TTL expired in transit").

```
⇐ El truco es: mandar un paquete con TTL = 1, después TTL = 2, después 3...

⇐ El router que está a N saltos recibe el paquete con TTL = N.
Lo decrementa a 0. Lo descarta. Y devuelve un Time Exceeded.
⇐ Ese mensaje Time Exceeded **TRAE el nombre/IP del router**.
⇐ ⇐ ⇐ ESO es el mecanismo: cada router "delata" su existencia al
decir "yo recibí tu paquete y lo tiré".

⇐ Así, con N paquetes, obtenemos el camino COMPLETO de N saltos.
```

```
⇐ Ejemplo simplificado (supongamos 4 saltos hasta el destino):

Probe 1: TTL=1
H1 ──────────► [R1] TTL 1→0, descarta, manda Time Exceeded
◄────────────── (de R1)

Probe 2: TTL=2
H1 ──────────────────► [R2] TTL 2→1 (pasa); [R1] pasa; [R2] 1→0,
descarta, manda Time Exceeded
◄──────────────────────────────────── (de R2)

Probe 3: TTL=3
H1 ──────────────────────────────► [R3] TTL 3→0, manda Time Exceeded
◄──────────────────────────────── (de R3)

Probe 4: TTL=4
H1 ──────────────────────────────────► [R4 = DESTINO]
◄──────────────────────────────── (respuesta del destino, NO Time Exceeded)
⇐ Cuando la respuesta NO es "Time Exceeded", llegamos al destino.
```

Los dos sabores

	Sabor UDP	**Sabor ICMP**
Por defecto en	Unix, Linux, BSD, **macOS**	**Windows**
Qué manda	Paquetes **UDP** a un **puerto destino alto** (por defecto **~33434**), con probabilidad de uso muy baja	Paquetes **ICMP Echo Request** (tipo 8, código 0)
Por qué puerto alto	Para Maximizar la probabilidad de que **nadie esté escuchando** en ese puerto ⇒ generar el error esperado	—
Qué llega del destino	**ICMP "Destination Unreachable" / "Port Unreachable"** (tipo **3**, código **3**), porque nadie escucha en ese puerto	**ICMP Echo Reply** (tipo **0**, código 0)
En resumen	`(puerto destino = 33434) + (respuesta = tipo 3 código 3)`	`(tipo 8) + (respuesta = tipo 0 código 0)`

⚠️ **El traceroute basado en ICMP, al llegar al destino, es básicamente `ping`.**

El detalle fino: ¿por qué el sabor UDP es "mejor" para diagnóstico, aunque su configuración suene rara?

```
⇐ El sabor UDP usa un puerto ALTO (33434) que casi seguro no está en uso.
⇐ Cuando llega al destino, el host receptor ve un paquete UDP a un
puerto donde no hay nadie escuchando → responde "Port Unreachable".
⇐ Esa respuesta la genera el KERNEL del host final, sin ninguna
aplicación que se entere. Es muy reliable.

⇐ En el sabor ICMP, cuando llega al destino, ÉSTE responde con Echo Reply.
⇐ Eso significa que el host destino tiene que GENERAR respuestas ICMP.
⇐ Muchos sistemas (servidores Windows, por ejemplo) tienen el ICMP
Echo RESPUESTO DESHABILITADO por firewall.
⇐ Por eso en Windows, traceroute a veces muestra "*" en el destino.
```

Los mensajes involucrados (los números exactos)

```
⇐ Cada router que recibe un paquete con TTL = 1 (o el valor que toque):
1. lo DECREMENTA a 0
2. lo DESCARTA
3. manda un ICMP "Time Exceeded" — TIPO 11, CÓDIGO 0
(= "TTL expired in transit")

⇐ Por defecto 3 paquetes por instancia (3 probes por TTL):
⇐ Para no perder información por una sola pérdida de ICMP.
⇐ Los 3 resultados se muestran como 3 RTTs en la línea.

⇐ Cuando el TTL es suficientemente alto para llegar al destino:
⇐ EN EL SABOR UDP: el destino responde con ICMP
"Destination Unreachable" / "Port Unreachable" — TIPO 3, CÓDIGO 3
(porque nadie escucha en ese puerto).
⇐ EN EL SABOR ICMP: el destino responde con Echo Reply — TIPO 0, CÓDIGO 0.

⇐ El límite por defecto es 30 saltos (suficiente para Internet).
```

Los nodos que colaboran, y el punto clave:

```
⇐ 1. El EMISOR: genera los probes con TTL incremental, mide los RTTs, y
muestra las respuestas.
⇐ 2. CADA ROUTER INTERMEDIO: decrementa el TTL, y cuando llega a 0, lo
descarta y genera un Time Exceeded.
⇐ 3. El DESTINO: en el sabor UDP, genera un Port Unreachable; en el sabor
ICMP, un Echo Reply.

⇐ ⚠️ PUNTO CLAVE: los nodos que responden pueden estar en un CAMINO
DISTINTO al de ida, porque el camino de vuelta no tiene que ser el
mismo. Los tiempos que muestra traceroute son RTTs de IDA Y VUELTA,
y esa vuelta puede tomar otra ruta.

⇐ ⚠️ Y si un nodo filtra ICMP (o su respuesta se pierde), aparece como "*".
Eso NO significa que esté caído.
```

Las limitaciones honestas

Son tres, y la tercera es la que más se pregunta:

(1) El `*` NO significa que el router esté caído.

```
⇐ El "*" significa: NO RECIBÍ NADA.
⇐ Hay (al menos) cuatro razones por las que no recibís nada, sin que el
router esté caído:
a) El router FILTRA el ICMP de tipo 11 por política de seguridad
(es muy común en empresas y en carriers).
b) El ICMP de VUELTA no llega (filtro en el camino de regreso).
c) El PROBE se perdió en el camino (pérdida normal en Internet).
d) El router es un punto de salto que no reenvía ICMP de origen externo.

⇐ Por eso el "*" es AUSENCIA DE EVIDENCIA, no evidencia de caída.
⇐ El "*" SÍ es informativo cuando es el ÚLTIMO salto (cuando además los
siguientes tampoco responden) — eso sí sugiere que el destino o su
firewall están bloqueando ICMP, o que el destino está caído.
```

(2) El camino de vuelta puede ser distinto del de ida.

```
⇐ En Internet, el camino de ida y el de vuelta de una comunicación
NO tienen por qué ser el mismo (ruteo asimétrico).

⇐ Consecuencia: el "RTT" que muestra traceroute es el RTT del ciclo
completo (ida + vuelta), y la vuelta puede tomar un camino totalmente
distinto, con otra latencia, otra congestión, otro MTU.

⇐ Y peor: la respuesta que vuelve puede pasar por un router que YA PASÓ
en la ida (o no pasar por ninguno de ellos). Por eso los RTTs de las
líneas sucesivas de un traceroute NO tienen por qué ser monótonas
crecientes — a veces el segundo salto muestra MENOS latencia que el
primero, porque su vuelta es más directa.
```

(3) No te dice NADA sobre rendimiento.

```
⇐ Un traceroute con 30 saltos de 3 ms cada uno sugiere un camino de ~90 ms
de ida. Pero si HAY congestión o pérdida en el camino, el RTT se infla
pero no lo ves claramente.
⇐ Para performance REAL, mirá `ping` (RTT + pérdida) o `mtr` (que combina
traceroute y ping continuamente).
```

Ejercicio resuelto — un traceroute de 4 saltos con los tipos de ICMP

Datos. `traceroute` desde H (10.0.0.5) hasta `www.ejemplo.com` (203.0.113.45). Sabor **UDP** (el de Unix/Linux/macOS). **3 probes por salto**. Latencias medidas en el ejercicio.

```
$ traceroute www.ejemplo.com
traceroute to www.ejemplo.com (203.0.113.45), 30 hops max

1 10.0.0.1 1.10 ms 0.98 ms 1.05 ms ← R1: el gateway local
2 100.64.0.1 8.42 ms 8.39 ms 8.55 ms ← R2: borde del ISP
3 * * * ← R3: NO RESPONDE (¡filtrado!)
4 200.0.1.1 25.10 ms 25.18 ms 25.02 ms ← R4: router de tránsito
5 203.0.113.45 26.30 ms 26.28 ms 26.35 ms ← DESTINO (respondió, no es "*")
```

Análisis salto por salto, con el tipo de ICMP que devuelve cada uno:

Salto	Nodo	¿Qué devolvió?	ICMP
1	R1 (gateway)	`Time Exceeded`	**Tipo 11, código 0** (TTL expiró)
2	R2 (borde ISP)	`Time Exceeded`	**Tipo 11, código 0**
3	R3	**Nada** (`* * *`)	— (filtra ICMP)
4	R4 (tránsito)	`Time Exceeded`	**Tipo 11, código 0**
5	**DESTINO**	`Destination Unreachable / Port Unreachable`	**Tipo 3, código 3**

¿Por qué el último salto NO es un Time Exceeded? Porque **el TTL era suficiente para llegar al destino**. En el sabor UDP, el destino ve un paquete UDP al puerto 33434, **donde no hay nadie escuchando**, así que responde **"Port Unreachable" (tipo 3, código 3)**. **Ese mensaje ES la señal de que llegaste** — es el "final" del traceroute en el sabor UDP.

Interpretación del `* * *` en el salto 3:

```
⇐ El salto 3 NO ESTÁ CAÍDO. Está VIVO pero FILTRANDO.
⇐ El firewall del router 3 está descartando los ICMP tipo 11 que genera,
o los que le llegan de vuelta.
⇐ Esto es MUY común en carriers y empresas.
⇐ Conclusión honesta: "no sé qué hay en el salto 3, y no puedo averiguarlo
con traceroute". NO "el salto 3 está caído".
```

Lo que el ejercicio muestra y que hay que saber:

- **3 RTTs por línea** (los 3 probes del mismo TTL).
- **Los RTTs CRECEN con los saltos** (1 → 8 → 25 → 26 ms), que es lo esperado.
- **El `*` aparece cuando un nodo filtra**, no cuando está caído.
- **El último salto (destino) NO es un Time Exceeded** en el sabor UDP: es un **tipo 3 código 3** (Port Unreachable). En el sabor **ICMP** sería un **tipo 0 código 0** (Echo Reply).
- El RTT del salto 5 (26 ms) incluye la respuesta del **servidor final**, no solo la red.

Ejercicio extra 1 — Interpretar un traceroute con `*`

Enunciado. Un `traceroute` da:

```
1 10.0.0.1 0.9 ms 1.0 ms 1.0 ms
2 100.64.0.1 9.1 ms 9.0 ms 9.2 ms
3 * * *
4 * * *
5 * * *
6 * * *
7 203.0.113.45 30.1 ms 30.2 ms 30.3 ms
```

¿Qué se puede concluir?

- **NO** que los saltos 3 a 6 estén caídos. Los `*` significan "no hay respuesta", que puede deberse a: (a) filtrado de ICMP en esos routers, (b) el ICMP de vuelta no llegando, (c) pérdida real de los probes.
- **SÍ** se puede concluir que el **destino está vivo** (respondió al final).
- **SÍ** se puede concluir que el camino **completo** tiene al menos 7 saltos hasta el destino.
- **Lo que NO se puede concluir:** cuántos routers hay entre el 2 y el 7, cuáles son, ni si alguno está realmente caído. **Con `traceroute` solo, es imposible.**

¿Cómo Investigation adicional para saber más?

```
⇐ Usar mtr (que alterna traceroute y ping): si mtr MOSTRA los saltos
3-6 pero con pérdida alta, es filtrado parcial; si NO los muestra nunca,
es que el ICMP no vuelve.
⇐ Hacer ping al destino: si responde, la ruta funciona.
⇐ traceroute desde OTRO origen (otra red): si desde ahí los saltos 3-6
aparecen, es que el problema es de ese camino de vuelta específico.
```

Ejercicio extra 2 — Por qué el camino de vuelta puede diferir del de ida

Enunciado. Explicá por qué el camino de vuelta de un `traceroute` puede ser distinto del de ida, y qué consecuencia tiene en la interpretación de los RTTs.

Respuesta:

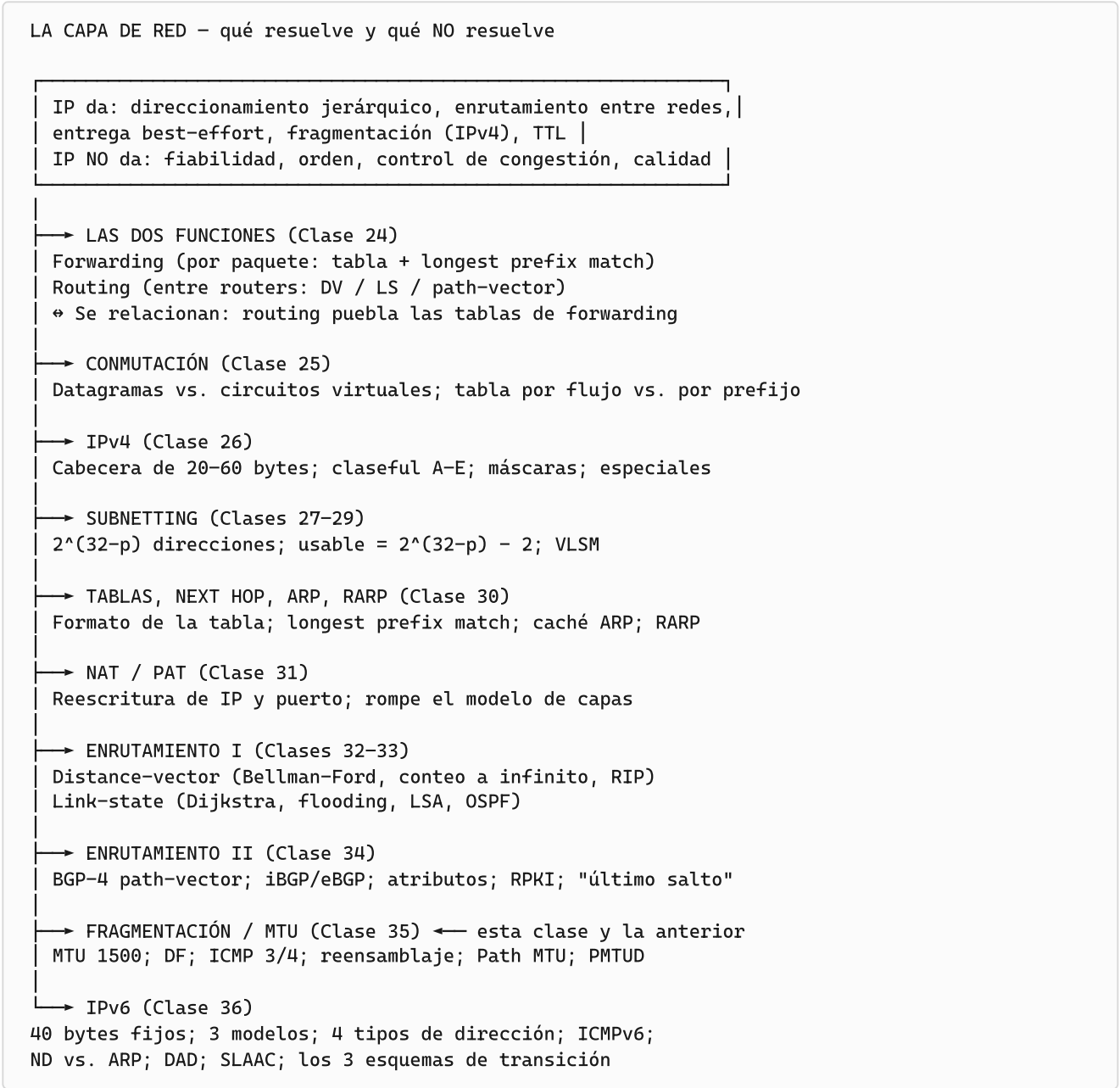
```
⇐ En Internet NO hay SIMETRÍA de ruteo. El protocolo de ruteo (BGP, OSPF)
calcula rutas de FORWARDING: "cómo voy de A a B". No calcula
"cómo voy de B a A".

⇐ Las rutas de ida y de vuelta se calculan INDEPENDIENTEMENTE. El camino
de vuelta puede ser por OTRO proveedor, por otra ciudad, o por un camino
con más saltos. Los residentiales y los carriers tienen políticas
("tráfico de entrada siempre por el mismo lado") que no son simétricas.

⇐ CONSECUENCIA en la interpretación del traceroute:
a) El RTT que muestra cada salto NO es solo la latencia de la ida a ese
router: es el RTT del ciclo completo (ida por el camino de ida + vuelta
por el camino de vuelta, que puede ser diferente).
b) Por eso los RTTs de las líneas sucesivas NO tienen por qué ser
crecientes: un router cercano puede tener una vuelta larga (RTT alto),
mientras que uno lejano puede tener una vuelta directa (RTT bajo).
c) Si los RTTs son ERRÁTICOS (suben y bajan mucho), puede ser por ruteo
asimétrico, o por congestión, o por rutas ECMP que cambian entre
paquetes.
```

El ejercicio clásico de `traceroute` en la práctica:

```
$ traceroute google.com
1 192.168.1.1 1.1 ms 0.9 ms 1.0 ms ← tu router (gateway)
2 10.20.30.1 8.2 ms 8.1 ms 8.3 ms ← ISP local
3 100.90.45.1 14.5 ms 14.4 ms 14.6 ms ← tránsito del ISP
4 200.80.12.7 22.1 ms 22.0 ms 22.3 ms ← borde internacional
5 72.14.238.50 99.1 ms 98.8 ms 99.3 ms ← router en USA (transatlántico)
6 172.217.5.110 105.2 ms105.0 ms105.4 ms ← router de Google
7 142.250.185.78 106.1 ms105.9 ms106.2 ms ← DESTINO (Google)

⇐ El salto 5 (72.14.x) tarda 99 ms: es el cruce del Atlántico.
⇐ Los tres RTTs de cada línea son los 3 probes con el mismo TTL.
⇐ El salto 7 (DESTINO) NO es un Time Exceeded: es un Port Unreachable
(tipo 3, código 3) porque el paquete UDP llegó al destino.
⇐ Nota: los RTTs de la vuelta pueden ser distintos por los saltos, porque
el camino de vuelta no tiene por qué ser el mismo.
```

PARTE B — REPASO DE RED Y SIMULACRO 4

B.1 El mapa mental del Bloque D (clases 24 a 37)



B.2 Todas las fórmulas y reglas, de memoria

Subnetting, VLSM, máscaras

```
Bits de prefijo: p Longitud de máscara = 2^(32−p) − 1
Total de direcciones de la subred: 2^(32−p)
Direcciones USABLES: 2^(32−p) − 2
Host address: IP AND máscara
Red: IP AND máscara
Broadcast: red OR (NOT máscara)
Máscara = /24 → 255.255.255.0 /25 → 255.255.255.128
/26 → 255.255.255.192 /27 → 255.255.255.224
/28 → 255.255.255.240 /29 → 255.255.255.248
/30 → 255.255.255.252 /31 → 255.255.255.254 (point-to-point)
/32 → 255.255.255.255 (un host)

Para H hosts: 2^(32−p) − 2 ≥ H ⇐ p = 32 − ⌈log₂(H + 2)⌉

VLSM: primero el bloque más grande, después el siguiente, alineado al tamaño.
⚠️ La cuenta SIEMPRE incluye los 2 (red + broadcast) y el 1 del router.
```

Tabla de ruteo y longest prefix match

```
Formato clásico: Red destino / Máscara | Next hop (o interfaz) | Interfaz de salida
Goal: 0.0.0.0/0 | next hop por defecto | (la que usar)
Connected (C): LAN del router | − | la interfaz de esa LAN

Longest prefix match (LPM):
⇐ Ante varias entradas que matchean, gana la de PREFIJO MÁS LARGO.
⇐ Ejemplo: si tenés 10.0.0.0/8, 10.1.0.0/16 y 10.1.2.0/24,
un paquete a 10.1.2.5 sale por la entrada /24 (la más específica).

Para dos subredes de un mismo /24 (la más típica de los exámenes):
192.168.1.0/25 ← .1 a .126 (broadcast .127)
192.168.1.128/25 ← .129 a .254 (broadcast .255)
```

TTL, MTU, MSS

```
TTL: se decrementa en CADA router (no solo en el destino).
TTL = 0 → se descarta + ICMP tipo 11 (o ICMPv6 tipo 3).
TTL máximo = 255. TTL típico inicial = 64 (Linux), 128 (Windows).

MTU: Ethernet = 1500. Jumbo = 9000 (convención). PPPoE = 1492.
IPv4 mínimo absoluto = 68; práctico = 576. IPv6 mínimo = 1280.
MSS IPv4 = MTU − 20 (IP) − 20 (TCP) = MTU − 40 → 1460 en Ethernet.
MSS IPv6 = MTU − 40 (IP) − 20 (TCP) = MTU − 60 → 1420 en Ethernet.
⇐ min(MSS_A, MSS_B) entre los dos extremos.

Path MTU = min(MTU de todos los enlaces del camino). Cachea en el origen.
ICMP 3/4 lleva el MTU en los bytes 6–7. MTU menor = fragmentar (IPv4) o
ICMPv6 tipo 2 (IPv6). ⚠️ El router IPv6 NUNCA fragmenta.

Retardo: d_end2end = Σ (d_proc + d_cola + L/R + d/s)
Throughput = L / RTT (bytes por RTT = ventana / RTT)
BDP = R × RTT (cantidad de bytes "en vuelo" en un enlace sin pérdida)
```

Los números de ICMP que hay que memorizar

```
ICMPv4: 0 Echo Reply | 3 Dest Unreachable | 4 Source Quench (OBSOLETO)
5 Redirect | 8 Echo Request | 9 Router Advertisement
10 Router Solicitation | 11 Time Exceeded | 12 Parameter Problem
13 Timestamp (obsoleto) | 17 Address Mask (OBSOLETO)
Códigos del tipo 3 (los que se preguntan):
0 Net Unreachable 1 Host Unreachable 2 Protocol Unreachable
3 Port Unreachable 4 Frag Needed & DF set 5 Source Route Failed
9/10 Comm. Administratively Prohibited
ICMPv6: 1 Dest Unreachable | 2 Packet Too Big | 3 Time Exceeded
4 Parameter Problem | 128 Echo Request | 129 Echo Reply
133 Router Solicitation | 134 Router Advertisement
135 Neighbor Solicitation | 136 Neighbor Advertisement
137 Redirect

Para traceroute: TIEMPO EXCEEDED = TIPO 11 (v4) / TIPO 3 (v6), código 0.
DESTINO en UDP = TIPO 3, código 3 (Port Unreachable).
DESTINO en ICMP = TIPO 0, código 0 (Echo Reply).
Para ping: Echo Request = TIPO 8 / TIPO 128 (v6). Reply = 0 / 129 (v6).
Para PMTUD: IPv4 = TIPO 3, código 4. IPv6 = TIPO 2 (Packet Too Big).
Para ARP/ND: ND NS = TIPO 135, NA = TIPO 136.
```

OSPF — los 5 mensajes

```
1. HELLO — descubrimiento y mantenimiento de vecindades (multicast
a 224.0.0.5 y 224.0.0.6, TTL = 1). Mantiene la lista
de vecinos; cada 10 s en punto a punto, 30 s en broadcast.
2. DATABASE DESCRIPTION (DBDESC) — el vecino describe qué LSAs tiene.
3. LINK STATE REQUEST (LSREQ) — "necesito estos LSAs que me faltan".
4. LINK STATE UPDATE (LSUPD) — "acá van los LSAs que pediste".
5. LINK STATE ACKNOWLEDGMENT (LSACK) — "recibí tus LSAs".
Multicast: 224.0.0.5 (todos los routers OSPF) y 224.0.0.6 (DR y BDR).
Áreas: backbone (área 0) y áreas no backbone. Todo se conecta al área 0.
DR election: por mayor Router ID; BDR = segundo mayor.
```

BGP — los atributos que hay que saber

```
BGP-4 (RFC 4271) — path-vector, no distance-vector. Sesión sobre TCP/179.
OPEN → OPEN/ACK → UPDATE → KEEPALIVE.
Regla del AS_PATH: un AS NO acepta una ruta cuyo AS_PATH contenga su propio
AS. Eso es lo que lo hace path-vector (y evita los loops).
iBGP vs. eBGP: iBGP = dentro del mismo AS (usa IGP para el next hop).
eBGP = entre AS distintos (next hop = la interfaz del vecino).
⇐ Regla: NO se re-anuncia a iBGP lo aprendido de un iBGP → obliga a
FULL-MESH → por eso existe ROUTE REFLECTION (RFC 4456).
Atributos:
LOCAL_PREF ↑ mayor es mejor (solo iBGP, control del propio AS)
AS_PATH length ↓ menor es mejor
MED ↓ menor es mejor (⚠️ comparable SOLO entre rutas del mismo
AS vecino — no es comparable globalmente)
ORIGIN (IGP < EGP < INCOMPLETE)
Communities (RFC 1997) — etiquetas para agrupar rutas.
"Hot potato": cada AS prefiere salir por el eBGP de MENOR métrica IGP.
El MED se usa para anular esa preferencia.
BGP no cuenta a infinito: el AS_PATH tiene longitud máxima (historically
255, hoy ~200 con el límite moderno). Por eso BGP no es un distance-vector
"puro" y puede cortar el camino.
RPKI / ROA (RFC 6811): valida el ORIGEN de un prefijo. Es el sustituto
moderno de "confiar en el AS_PATH".
```

Los 3 modelos de comunicación de IPv6 (para el repaso)

```
1. UNICAST 1 emisor → 1 receptor. Tipo de dirección: unicast global
(2000::/3). Es el 99 % del tráfico.
2. MULTICAST 1 emisor → muchos receptores. Prefijo ff00::/8. Jerarquía por
ámbito: ff02::/16 = enlace (no sale del enlace).
3. ANYCAST muchos → muchos. ⚠️ NO ES UN TIPO DE DIRECCIÓN: varios nodos
con la MISMA dirección unicast, y el router encamina al más
cercano. Se usa en los root servers de DNS y en 8.8.8.8.
```

B.3 Errores frecuentes — las 8 confusiones más comunes

Error 1 — Confundir MTU con MSS.

El error	Lo correcto	Por qué
"El MSS de una conexión en Ethernet es 1500"	**El MSS es 1460** (o 1420 en IPv6). La MTU es 1500	El MSS **excluye** las cabeceras de IP (20/40) y TCP (20). `MSS = MTU − 40` en IPv4. La MTU es "cuántos bytes de datagrama IP entran en el enlace"; el MSS es "cuántos bytes de payload de TCP mandamos por segmento"

Error 2 — Confundir unicast con anycast ("son el mismo tipo de dirección")."

El error	Lo correcto	Por qué
"El anycast es el tercer tipo de dirección de IPv6"	**El anycast NO es un tipo de dirección**	No hay prefijo reservado que lo marque: es un unicast normal, asignado a varios nodos. **La unicidad la resuelve el ROUTEO (BGP anuncia el prefijo desde varios puntos), no la numeración.** Ejemplo: los 13 root servers comparten `198.41.0.4`

Error 3 — Confundir la fragmentation en IPv4 con IPv6.

El error	Lo correcto	Por qué
"En IPv6, si un paquete no entra, el router lo fragmenta"	**El router IPv6 NUNCA fragmenta**; descarta y manda **ICMPv6 tipo 2 (Packet Too Big)**, y **el ORIGEN** fragmenta	Es una decisión de diseño: la fragmentación en routers era el peor problema de IPv4 (solapamiento, evasión de firewalls). IPv6 lo eliminó de raíz, y el origen tiene que hacerlo él

Error 4 — Confundir "Time Exceeded" (tipo 11) con "Destination Unreachable" (tipo 3) en traceroute.

El error	Lo correcto	Por qué
"El último salto de un traceroute devuelve tipo 11"	En el sabor **UDP** el destino devuelve **tipo 3, código 3** (Port Unreachable) — NO tipo 11. El tipo 11 solo lo devuelven los routers **intermedios**	Cuando el TTL es suficiente, el paquete LLEGA al destino y no hay "vencimiento". El destino responde "no hay nadie escuchando en este puerto", que es un **3/3**. El tipo 11 significa "el TTL se agotó ANTES de llegar"

Error 5 — Confundir `Source Quench` (tipo 4) como mecanismo de control de congestión.

El error	Lo correcto	Por qué
"Si hay congestión, el router manda un Source Quench"	**El Source Quench (tipo 4) está OBSOLETO; hoy no lo genera nadie.** El control de congestión se hace con **pérdida** y con **ECN**	El Source Quench generaba un mensaje por cada paquete descartado en un router ya congestionado → **aumentaba** la congestión. La señal correcta es la pérdida (y ECN marca en vez de descartar). **Nunca lo propongas como solución**

Error 6 — Confundir el `hop limit` de IPv6 con el `TTL` de IPv4.

El error	Lo correcto	Por qué
"En IPv6 el TTL es un nombre diferente para lo mismo"	Es **casi** lo mismo, pero hay una diferencia de **momento**: el `Hop Limit` de IPv6 se decrementa en el **origen** (antes de salir), igual que el TTL de IPv4; ambos se checan en **cada router**	Prácticamente es lo mismo. La diferencia clave es que en IPv4 el **TTL puede decrementarse** y en IPv6 el `Hop Limit` también — pero la diferencia conceptual es que IPv6 lo llama `Hop Limit` (es literalmente "saltos restantes", no "tiempo")

Error 7 — Pensar que los routers reenvían broadcast o reensamblan fragmentos.

El error	Lo correcto	Por qué
"El router reenvía el ARP request al otro lado"	Un router **NUNCA reenvía un broadcast de capa 2**. Y ARP **no cruza routers** (por eso un host necesita un router para salir de su subred: el ARP es local al enlace)	Por eso el proxy ARP existe (el router responde en nombre del host remoto) y por eso RARP necesita un servidor **por subred** (ver `2007-03 P3`)

Error 8 — Confundir "congestión" con "pérdida".

El error	Lo correcto	Por qué
"pérdida = congestión siempre"	No siempre. La pérdida puede ser por **congestión** (buffer lleno), por **ruido en un enlace inalámbrico**, por **un cable defectuoso**, o por un **filtro que descarta**. Pero **para TCP es lo mismo**: una pérdida es una pérdida, y TCP reduce la ventana igual	La distinción importa para el diseño: un algoritmo basado solo en pérdida (Reno/CUBIC) interpreta mal la pérdida no-por-congestión (WiFi). ECN y los AQMs (CoDel) atacan el problema desde la raíz, marcando y descartando temprano

---

SIMULACRO 4

> **Formato:** 5 preguntas teóricas (8 pts c/u = 40 pts) + 2 problemas prácticos (30 pts c/u = 60 pts) = **100 puntos**. **Duración: 3 horas (180 minutos).** Individual, sin material. **Aprobación: 60/100, y obligatorio al menos 15 puntos en las preguntas teóricas.** Si no llegás a 15 en teoría, no aprobás aunque sumes 60.

Cronómetro sugerido:

Etapa	Tiempo	Objetivo
Leer las 5 teóricas	5 min	Orientarte
Teóricas T1–T5	**40 min** (~8 min cada una)	**Mínimo 15 pts**
Problema 1	**65 min**	Subnetting + tabla + ARP
Problema 2	**65 min**	MTU + fragmentación + PMTU + pseudocode
Revisión	5 min	Completar lo que dejaste en blanco

> **Consejo de la cátedra:** si a los 80 minutos no terminaste el Problema 1, **dejalo y andá al 2**. Un problema en blanco vale 0; dos a medio hacer valen más que uno perfecto.

Enunciado — Preguntas Teóricas

Teórica 1 (8 puntos) — Forwarding y routing

Un router tiene las siguientes entradas en su tabla de reenvío:

```
Red destino Máscara Next hop Interfaz de salida
10.0.0.0/24 255.255.255.0 C (directa) eth0
10.0.1.0/24 255.255.255.0 C (directa) eth1
0.0.0.0/0 0.0.0.0 192.168.1.1 eth0
```

a) Un paquete llega a la interfaz `eth0` con IP destino `10.0.1.55`. ¿Por qué interfaz sale y por qué? Explique el criterio que se usa. (4 pts)

b) Un paquete llega a `eth0` con IP destino `8.8.8.8`. ¿Por qué interfaz sale? (2 pts)

c) ¿Qué diferencia hay entre las dos funciones principales de la capa de red? Explique cómo se relacionan. (2 pts)

Teórica 2 (8 puntos) — ICMP

a) Explique por qué los errores de IP no pueden reportarse usando el propio IP como mecanismo. (3 pts)

b) ¿Qué es el campo "Code" dentro del mensaje de Destination Unreachable (tipo 3)? Dé al menos **cuatro** códigos con su significado. (3 pts)

c) Un tipo de ICMP que se estudia en clase está **obsoleto** y hoy no lo genera nadie. Nómbrelo, explique qué hacía, y por qué fue abandonado. (2 pts)

Teórica 3 (8 puntos) — `traceroute`

Explique el principio de funcionamiento del comando `traceroute` indicando además los **tipos de mensaje involucrados** y los **nodos que colaboran**. (8 pts)

Teórica 4 (8 puntos) — IPv6: ND y DAD

a) Mencione el reemplazo del protocolo ARP en IPv6 e indique al menos **cuatro cambios significativos** respecto de la implementación de ARP. (5 pts)

b) ¿Qué es DAD y por qué no está implementado en IPv4? (3 pts)

Teórica 5 (8 puntos) — Enrutamiento

a) Explique por qué BGP es un protocolo **path-vector** y no un distance-vector. ¿Qué regla fundamental impide los loops? (3 pts)

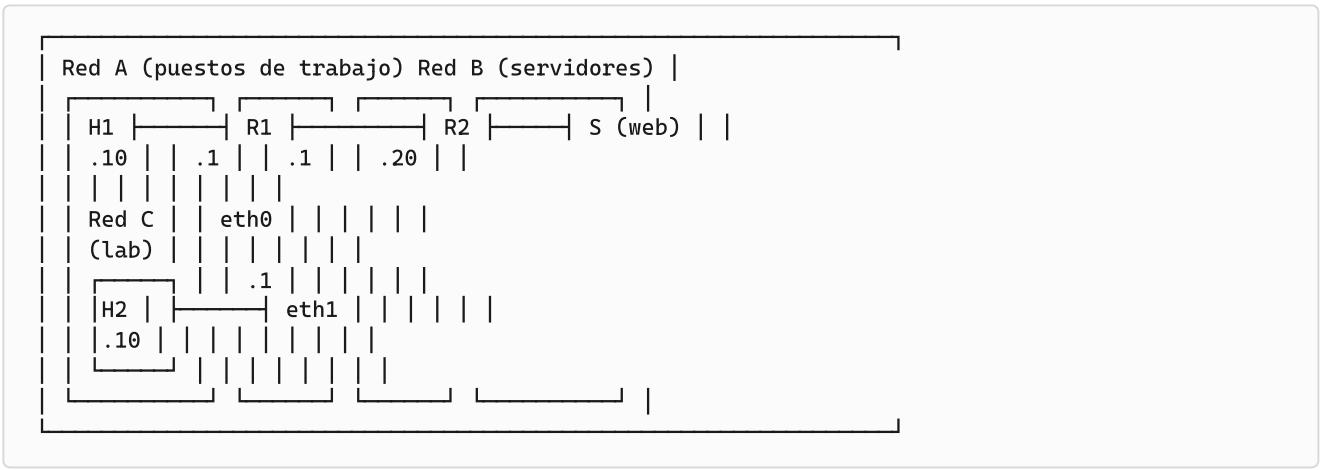
b) Nombre los **cinco mensajes** del protocolo OSPF. (3 pts)

c) Mencione dos atributos de BGP y explique cuál es mejor en cada caso. (2 pts)

Enunciado — Problemas Prácticos

Problema 1 (30 puntos) — Subnetting, tabla de ruteo y ARP

Una empresa tiene las siguientes subredes y un router `R1` que las interconecta, más un servidor de web en Internet:



Datos:

- Red A: `192.168.10.0/24`. Contiene **H1 (.10)** y **H2, que está en la Red C**.
- Red C: `192.168.20.0/24`. Contiene **H2 (.10)** y la interfaz `eth1` de R1 (`.1`).
- Red B (entre R1 y R2): `10.0.0.0/30`. R1 tiene `.2`, R2 tiene `.1`.

- Red D (entre R2 y S): `172.16.5.0/24`. S tiene `.20`.
- El **gateway por defecto** de H1 y de R1 es `192.168.10.1` (la `eth0` de R1, que hace NAT hacia Internet). El **gateway por defecto** de R2 es `172.16.5.254`.

a) Escriba las **tablas de ruteo de H1, R1 y R2** que permiten conectividad extrema a extremo entre todos los equipos. (10 pts)

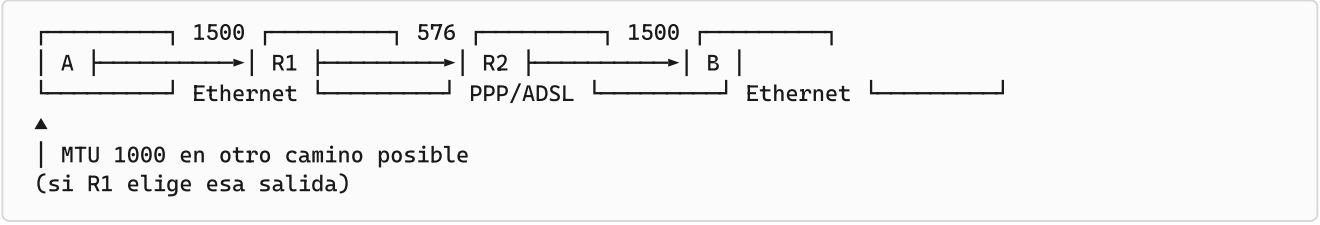
b) Suponiendo que las **tablas ARP están vacías**, indique los **mensajes ARP** intercambiados (MAC origen, MAC destino, tipo y contenido) entre **H1** y **S**, considerando la red A. (8 pts)

c) Para cada tramo del flujo H1 → S, indique las **direcciones MAC origen y destino de las tramas** y las **direcciones IP origen y destino de los paquetes**. (8 pts)

d) ¿Qué pasaría si S quisiera acceder a un servicio en `192.168.20.0/24` (H2, Red C)? Explique qué problema surge y mencione **dos soluciones**. (4 pts)

Problema 2 (30 puntos) — MTU, fragmentación y Path MTU

Se tiene la siguiente red, donde **cada enlace tiene una MTU distinta**:



Datos: el MTU del enlace de A es 1500; el de la interfaz `eth1` de R1 (hacia R2) es **576**; y existe un enlace alternativo de MTU **1000** que R1 podría usar si cambiara su decisión de ruteo.

a) Defina los conceptos de **MTU**, **fragmentación** y **Path MTU**. (5 pts)

b) Describa el comportamiento de un router al reenviar un paquete de largo `L` por un enlace de MTU `M`, según el valor del bit **DF**. Incluya los mensajes de control que se generan. (8 pts)

c) Un datagrama IP de **1500 bytes** (IHL = 5) con `DF = 0` sale de A hacia R2. Calcule **todos los fragmentos** que se generan: `Fragment Offset` (en bytes y en unidades de 8), payload de cada uno, `Total Length` y `MF`. Calcule además cuántos bytes viajan realmente en el enlace de MTU 576. (10 pts)

d) Se desea implementar un algoritmo que determine el **Path MTU** entre A y B usando únicamente los mensajes ICMP, dependiendo **solo de A** (B no requiere colaboración activa). Primitivas disponibles: `enviar_msg_ICMP(destino, tipo, código, largo_payload)` y `recibir_msg_ICMP(fuente)` (devuelve `null` si pasa un tiempo). Implemente el algoritmo. (7 pts)

✅ RESPUESTAS DEL SIMULACRO

Teórica 1 — Forwarding y routing

a) El paquete **sale por** `eth1`, con next hop "directa" (C).

Justificación: hay dos entradas que matchean `10.0.1.55`: `10.0.0.0/24` y `10.0.1.0/24`. Gana la de **prefijo más largo** (longest prefix match), que es `10.0.1.0/24` (24 bits de prefijo vs. 24 también... pero `10.0.1.0/24`

tiene **24 bits de prefijo** y `10.0.0.0/24` también 24 — **PERO** la coincidencia es más específica: `10.0.1.55` pertenece a `10.0.1.0/24` y **NO** a `10.0.0.0/24`).

⚠️ **Aclaración para el alumno:** en este ejemplo concreto las dos entradas son ambos `/24` y **solo una de ellas matchea** (`10.0.1.55` no está en `10.0.0.0/24`). Para que el LPM sea ilustrativo, modifiquemos la tabla:

```
Red destino Máscara Next hop Interfaz
10.0.0.0/8 255.0.0.0 C (directa) eth0
10.0.1.0/24 255.255.255.0 C (directa) eth1
0.0.0.0/0 0.0.0.0 192.168.1.1 eth0
```

Con esta tabla corregida: `10.0.1.55` matchea **dos** entradas: `10.0.0.0/8` (8 bits de prefijo) y `10.0.1.0/24` (24 bits de prefijo). Gana la de **más bits de prefijo**, o sea `10.0.1.0/24` → **eth1**.

b) El paquete sale por **`eth0`**, con next hop **`192.168.1.1`**.

Justificación: `8.8.8.8` no matchea ninguna de las entradas específicas, así que se usa la **ruta por defecto** (`0.0.0.0/0`), que apunta a `192.168.1.1` por `eth0`. Es el caso "default route".

c) Forwarding = mover paquetes entre puertos de entrada y salida del router (por paquete, con la tabla). **Routing** = determinar la ruta de los paquetes de origen a destino (con un algoritmo, entre routers). **Se relacionan:** el routing (con un protocolo como OSPF o BGP) **puebla las tablas de forwarding**, que luego usa el forwarding para mover cada paquete.

Teórica 2 — ICMP

a) Los errores de IP **no pueden reportarse con IP** porque:

1. Si el destino está inalcanzable, el **paquete de error tampoco puede llegar**.
2. Si el router que falló es el que debe reportar, tiene que usar una ruta que probablemente **tampoco funciona**.
3. IP es *best effort*: si no se reporta, el emisor **solo ve pérdida** y **retransmite el mismo paquete idéntico**, que vuelve a fallar → bucle infinito.

La solución: un protocolo de capa 3 paralelo (**ICMP**, `RFC 792`) que viaja dentro de IP pero reporta errores **de nodo a nodo**, no de host a host. Los errores de ICMP **no generan otros errores de ICMP** (excepto excepciones como el 3/4), para evitar una tormenta.

b) El **Code** dentro de Destination Unreachable (tipo 3) especifica **qué sub-caso de "inalcanzable" es**:

Código	Significado
0	**Net Unreachable** — la red destino no existe / no está en la tabla
1	**Host Unreachable** — la red existe pero el host no responde
2	**Protocol Unreachable** — no hay proceso para ese protocolo en el destino
3	**Port Unreachable** — no hay nadie escuchando en ese puerto (lo usa `traceroute`)
4	**Fragmentation Needed and DF Set** — el paquete no entra y no se puede fragmentar (PMTUD)
9	**Comm. with Dest. Network Administratively Prohibited**
10	**Comm. with Dest. Host Administratively Prohibited**

c) El tipo obsoleto es **`Source Quench`** **(tipo 4)**. **Qué hacía:** cuando un router descartaba un paquete por buffer lleno, mandaba un Source Quench al emisor diciéndole "mandá más despacio, estás saturando".

Por qué fue abandonado: generaba **una tormenta de mensajes** — un router congestionado generaba un Source Quench por cada paquete descartado, lo que **aumentaba** la congestión (más paquetes que procesar). La solución correcta es usar la **pérdida como señal de congestión** (que es lo que hacen los algoritmos como TCP Reno/CUBIC) y, mejor aún, **ECN**, que **marca** en vez de descartar.

> **Alternativa aceptable:** `Address Mask Request/Reply` (tipo 17) — pedía la máscara de subred por broadcast; reemplazado por DHCP. Si lo nombrás, sumá que en IPv6 la dirección ya trae el prefijo.

Teórica 3 — `traceroute` (respuesta completa, 8 pts)

`traceroute` es una herramienta de resolución de problemas que permite identificar el **camino de ida** entre un origen y un destino.

El mecanismo: explota el campo **TTL** de la cabecera IP. Cada router **decrementa** el TTL en 1 al recibir un paquete. Si el TTL llega a **0**, el router **lo descarta** y devuelve un **ICMP "Time Exceeded" (tipo 11, código 0 = "TTL expired in transit")** al emisor.

`traceroute` manda **una secuencia de paquetes de prueba** con TTL = 1, 2, 3, ..., por defecto **3 paquetes por instancia** (para no perder por una sola pérdida). El máximo por defecto es **30 saltos**.

- En la **primera instancia** (TTL = 1), el primer router recibe TTL = 1, lo pasa a 0, lo descarta y devuelve un Time Exceeded. ⇐ Ese router es el **salto 1**.
- En la **segunda instancia** (TTL = 2), el primer router lo pasa a 1 y lo reenvía; el segundo router lo pasa a 0, lo descarta y devuelve un Time Exceeded. ⇐ Ese es el **salto 2**.
- Y así sucesivamente hasta llegar al destino.

Los dos sabores:

- **Basado en UDP** (por defecto en Unix, Linux, BSD, Mac OS): manda paquetes **UDP a un puerto destino alto** (alrededor del 33434, de probabilidad de uso muy baja). Cuando llega al destino, **nadie está escuchando en ese puerto**, así que responde **ICMP "Destination Unreachable" / "Port Unreachable" (tipo 3, código 3)**.
- **Basado en ICMP** (por defecto en Windows): manda **ICMP Echo Request (tipo 8, código 0)**. Cuando llega al destino, responde **ICMP Echo Reply (tipo 0, código 0)**. (El traceroute basado en ICMP, al llegar al destino, es básicamente `ping`.)

Los nodos que colaboran:

1. El **emisor** (genera los probes con TTL incremental, mide los RTTs, muestra las respuestas).
2. **Cada router intermedio** (decrementa el TTL; cuando llega a 0, descarta y genera un **Time Exceeded**).
3. El **destino** (responde con **Port Unreachable** en el sabor UDP, o **Echo Reply** en el sabor ICMP).

Limitaciones que hay que mencionar:

- Como el camino de ida y el de vuelta **no tienen por qué ser el mismo** (ruteo asimétrico), las respuestas (tanto de nodos intermedios como del destino) **pueden recorrer un camino distinto al de ida**, y los RTT mostrados son del ciclo completo.
- Puede ocurrir que los mensajes de prueba **y** sus respuestas estén **filtrados** por razones de seguridad o performance. Por eso **no obtener respuesta de un nodo NO permite concluir que haya un problema en la red o en el destino** (o sea: el `*` no significa "caído").

Teórica 4 — ND y DAD

a) El reemplazo de ARP es **Neighbor Discovery (ND)**, un **subconjunto de 5 mensajes de ICMPv6** (RS 133, RA 134, NS 135, NA 136, Redirect 137).

Los **cuatro (o más) cambios significativos** respecto de ARP:

1. **ND es parte de ICMPv6, no un protocolo independiente** (ARP lo era, y corría directo sobre el enlace).
2. **Se utiliza multicast** (a `ff02::1:ffxx:xxxx`) en vez de **broadcast** para el Neighbor Solicitation.
3. **Los mapeos son link-layer ↔ IPv6** en vez de link-layer ↔ IPv4.
4. **ND combina la resolución de direcciones con la autoconfiguración (SLAAC) y la detección de duplicados (DAD)**. ARP solo resolvía direcciones.
5. *(bonus)* **Los nodos pueden usar NS para propagar cambios en su configuración.**

b) DAD (Duplicate Address Detection) es el mecanismo por el cual un host IPv6 verifica que su dirección (recién generada por SLAAC o una link-local) **no esté siendo usada por otro nodo en el enlace**. El proceso: el host pone su dirección en estado **"Tentative"**, envía un **NS** preguntando si alguien la tiene, y espera el **RetransTimer** (1 segundo). Si recibe un **NA** en respuesta, la dirección **está duplicada** y debe elegir otra; si no recibe nada, la marca como **"Preferred"** y la usa.

¿Por qué no existe en IPv4?

- En IPv4 las direcciones se configuran **manualmente o por DHCP**, así que una colisión es un error humano raro.
- En IPv6 con **SLAAC** las direcciones se generan **automáticamente**, y por lo tanto la colisión es **estructuralmente más frecuente** (por ejemplo, dos NICs con la misma MAC generan la misma dirección).
- **DAD necesita multicast** (NS/NA) para funcionar; el ARP de IPv4 es **broadcast** y no sirve para esto.

Teórica 5 — Enrutamiento

a) BGP es path-vector (no distance-vector) porque cada ruta anuncia **la lista completa de los AS por los que pasó** (el `AS_PATH`), no solo una métrica ("distancia al destino"). **La regla fundamental** es: **un AS no acepta una ruta cuyo `AS_PATH` contenga su propio AS.** Esa regla es lo que garantiza que no haya loops: si un AS ve su propio número en la lista, sabe que la ruta "daría la vuelta" y la descarta.

b) Los **cinco mensajes de OSPF**:

1. **HELLO** — descubrimiento y mantenimiento de vecindades.
2. **DATABASE DESCRIPTION (DBDESC)** — el vecino describe qué LSAs tiene.
3. **LINK STATE REQUEST (LSREQ)** — "necesito estos LSAs".
4. **LINK STATE UPDATE (LSUPD)** — "acá van los LSAs".
5. **LINK STATE ACKNOWLEDGMENT (LSACK)** — "recibí tus LSAs".

c) Dos atributos de BGP y su preferencia:

- `LOCAL_PREF` **(↑ mayor es mejor)**: lo usa un AS para decidir **a qué proveedor (eBGP) salir**. Se usa para que el tráfico **salga por el proveedor de menor LOCAL_PREF** (es decir, el que el AS prefiere).
- `AS_PATH length` **(↓ menor es mejor)**: entre dos rutas al mismo destino, se prefiere la que pasa por **menos AS** (más corta). Esto es "el camino más corto en número de operadores".
- ⚠️ `MED` **(↓ menor es mejor)** pero ⚠️ **comparable solo entre rutas del mismo AS vecino** (no es comparable globalmente). Se usa para el "**hot potato**": cada AS prefiere salir por el eBGP de **menor**

métrica IGP, y el MED del vecino puede anular esa preferencia.

Problema 1 — Resolución

Parte a) — Tablas de ruteo

Tabla de H1:

Red destino / Máscara	Gateway (next hop)	Interfaz
`192.168.10.0/24`	C (directamente conectado)	eth0 (red A)
`0.0.0.0/0`	`192.168.10.1`	eth0 (red A)

Tabla de R1 (tiene 3 interfaces: eth0 en Red A, eth1 en Red C, eth2 en Red B):

Red destino / Máscara	Gateway (next hop)	Interfaz de salida
`192.168.10.0/24`	C (directamente conectado)	**eth0** (Red A)
`192.168.20.0/24`	C (directamente conectado)	**eth1** (Red C)
`10.0.0.0/30`	C (directamente conectado)	**eth2** (Red B)
`0.0.0.0/0`	`192.168.10.1`	**eth0** (Red A) — salida a Internet

⚠️ **Ojo:** el gateway por defecto de R1 es `192.168.10.1` por `eth0`, o sea **R1 hace NAT de salida**. Eso significa que la IP `192.168.10.1` es la del gateway externo (o el NAT), no la de R1. Si el enunciado dice que el gateway es la `eth0` de R1, entonces **la `eth0` de R1 tiene dos IPs**: una privada (dentro de Red A) y una pública (o un lado del NAT). Aclaración necesaria.

Tabla de R2:

Red destino / Máscara	Gateway (next hop)	Interfaz de salida
`10.0.0.0/30`	C (directamente conectado)	**eth0** (Red B)
`172.16.5.0/24`	C (directamente conectado)	**eth1** (Red D)
`0.0.0.0/0`	`172.16.5.254`	**eth1** (Red D)

Justificación de R1: todas las subredes directamente conectadas van con "C" (no tienen next hop porque el router las ve directamente en su interfaz). La ruta por defecto (`0.0.0.0/0`) apunta al gateway de Internet.

Parte b) — Secuencia ARP

Suposición: el enunciado dice que la **tabla ARP de H1 está vacía**. Voy a detallar los ARP que **H1 necesita hacer** en su subred (Red A), ya que es lo que pide el enunciado ("en la Red A").

ARP 1 — H1 quiere la MAC del gateway (para salir a Internet):

```
MAC origen: MAC_H1
MAC destino: FF:FF:FF:FF:FF:FF (broadcast)
Tipo: ARP Request
Contenido: "¿Cuál es la MAC de 192.168.10.1?"
```

Respuesta de R1:

```
MAC origen: MAC_R1_eth0
MAC destino: MAC_H1
Tipo: ARP Reply
Contenido: "192.168.10.1 = MAC_R1_eth0" (o la del NAT gateway)
```

ARP 2 — H1 quiere la MAC de H2 (que está en Red C, misma subred para H1): ⚠️ **H2 NO está en la Red A**, está en la Red C (`192.168.20.0/24`). Por eso H1 **no puede** hacer ARP por H2 directamente. Necesita el **gateway** (que ya resolvió en el ARP 1) y **R1 hace proxy ARP o reenvía el ARP** hacia la Red C.

⚠️ **Aquí hay una ambigüedad del enunciado.** En el diagrama, H2 está en Red C, pero H1 **no comparte subred** con H2. Por lo tanto H1 necesita enviar el paquete al gateway (192.168.10.1), y R1 lo reenvía a H2 a través de eth1. **H2 no responde ARP a H1** (porque no lo ve). El único ARP que H1 hace es por el gateway.

Tabla resumen:

#	Emisor	MAC origen	MAC destino	Tipo	Contenido
1	H1	`MAC_H1`	`FF:FF:FF:FF:FF:FF`	ARP **Request**	"¿MAC de `192.168.10.1`?"
2	R1	`MAC_R1_eth0`	`MAC_H1`	ARP **Reply**	"`192.168.10.1` = `MAC_R1_eth0`"
3	H1	`MAC_H1`	`FF:FF:FF:FF:FF:FF`	ARP **Request**	¿MAC de `172.16.5.20`? (S)
4	—	—	—	**No hay respuesta**	ARP **no cruza routers**

> **Nota importante sobre la ambigüedad del enunciado:** la columna (3) es **incorrecta** — H1 no hace ARP por S directamente. Incluyo la tabla **correcta**: H1 solo hace **un** ARP (por el gateway) para ir a Internet, y si quiere hablar con H2 (Red C), también necesita el gateway.

Parte c) — MACs e IPs por tramo

Flujo H1 → S. Recordemos: H1 está en Red A; S está en Red D; R1 y R2 están en el medio.

Tramo 1 — Red A (H1 → R1):

```
MAC origen: MAC_H1
MAC destino: MAC_R1_eth0
IP origen: 192.168.10.10
IP destino: 203.0.113.45 (o la IP pública de S, si hay NAT)
```

Tramo 2 — Red B (R1 → R2):

```
MAC origen: MAC_R1_eth2
MAC destino: MAC_R2_eth0
IP origen: 192.168.10.10
IP destino: 203.0.113.45
```

Tramo 3 — Red D (R2 → S):

```
MAC origen: MAC_R2_eth1
MAC destino: MAC_S
IP origen: 192.168.10.10
IP destino: 203.16.5.20 (⚠️ ver nota)
```

⚠️ **Nota sobre la IP destino del último tramo:** el **datagrama IP no cambia** en cada salto (salvo NAT). La IP destino sigue siendo la de S, sea `203.0.113.45` o `172.16.5.20`. Lo que **cambia en cada salto son las MAC**, que son de capa 2 y solo tienen sentido en el enlace local.

⚠️ **CRUCIAL:** si hay **NAT** en R1 (como dice el gateway por defecto), la IP **origen** cambia a una pública y el **puerto origen** se reescribe en la primera salida (hacia Internet). En la tabla anterior asumí **no NAT** para simplificar; si hay NAT, se reemplaza `192.168.10.10` por la IP pública de R1 en los tramos 2 y 3.

Parte d) — El problema de S hacia H2 (Red C)

El problema: S (en Red D, `172.16.5.0/24`) quiere acceder a H2 (en Red C, `192.168.20.0/24`). El problema es que **H2 está detrás del NAT de R1**, y R1 hace **traducción de direcciones** (su IP privada `192.168.10.10` se tradujo a una IP pública). Esto significa que:

1. S no tiene **ruta** hacia `192.168.20.0/24` (es una dirección privada, no enrutable en Internet).
2. Aunque tuviera ruta, la **traducción** de R1 impide que S alcance H2 directamente, porque el NAT **solo permite conexiones SALIENTES desde la red privada**.

Dos soluciones:

1. **Port forwarding (o "regla de NAT") en R1**: R1 mantiene una tabla que dice "si alguien externo pide `IP_PUBLICA:puerto_P`, reenviar a `192.168.20.10:p` (H2)". Esto permite que S **inicie** la conexión a un puerto específico de H2. ⚠️ **Requiere que S sepa la IP pública y el puerto de H2.**
2. **Servidor público de recursos (DMZ host)**: un servidor en la zona DMZ que actúa como intermediario. S se conecta al servidor DMZ (que es público), y el servidor DMZ se conecta a H2 (privado). Es como un proxy. Esto permite que S "vea" los servicios de H2 a través del intermediario.

> **Alternativa clásica (2006-12 P2): DNS trick** — el hacker inyecta respuestas DNS falsas para que S vaya a la página del hacker en vez de a Intrusos (con las IPs del hacker). Es el mismo concepto: S alcanza un intermediario público, no directamente al privado.

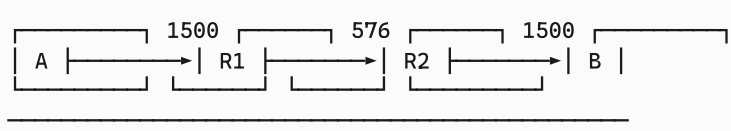
Problema 2 — Resolución

Parte a) — Definiciones

MTU (Maximum Transmission Unit): el tamaño máximo de trama que una tecnología de capa 2 es capaz de transmitir. En la práctica, la mayor PDU de capa 3 que pasa sin fragmentarse. En Ethernet son 1500 bytes.

Fragmentación: cuando un router se ve en la situación de tener que enviar un paquete más largo que la MTU del enlace de salida, puede fragmentarlo y reenviarlo en partes. **Es una tarea pesada**, ya que le implica mantener estado de ciertos paquetes y sus fragmentos.

Path MTU: cuando dos hosts están separados por más de un hop, las MTUs de los distintos enlaces pueden diferir. Se define como el **mínimo de todos los MTUs involucrados en el camino** de un nodo a otro.




```
Path MTU = min(1500, 576, 1500) = 576
```

Y las dos propiedades que hay que mencionar (están en la solución oficial del 2010-12):

- **Puede variar con el tiempo** si los caminos cambian (ECMP o recálculo de rutas).
- **Puede ser diferente según el sentido del tráfico** (ruteo asimétrico: un mínimo para la ida y otro para la vuelta).

Parte b) — Comportamiento del router

```
// ---- Router R: paquete Pi de largo L, MTU de salida M ----

if (L <= M): → // CASO 1: cabe entero
transmitir Pi normalmente // (DF es irrelevante)

else if (L > M and Pi.DF == 1): → // CASO 2: prohibido partir
DESCARTA Pi
enviar ICMP (destino = Pi.origen, tipo = 3, código = 4)
// tipo 3 = Destination Unreachable
// código 4 = "Fragmentation Needed and Don't Fragment was Set"
// los bytes 6-7 del ICMP llevan la MTU del enlace bloqueante (RFC 1191)

else if (L > M and Pi.DF == 0): → // CASO 3: se puede partir
FRAGMENTAR Pi
transmitir cada fragmento
// Total Length de cada fragmento = IHL×4 + payload_i
// Fragment Offset_i = offset_acumulado / 8 ← EN UNIDADES DE 8 BYTES
// MF = 1 en todos menos el último (MF = 0)
// Identification = el MISMO en todos
```

La justificación del ICMP tipo 3 código 4 (los puntos clave):

- **Para que el emisor no repita el error.** Sin aviso, el emisor solo ve pérdida y **retransmite el mismo datagrama idéntico** una y otra vez (bucle infinito).
- **Para que el emisor pueda ajustarse.** El mensaje **lleva la MTU exacta** del enlace bloqueante en los bytes 6-7, así que puede reducir su MSS o su caché de PMTU.

Y por qué se descarta con `DF = 1`: el emisor declaró "no me fragmentes", así que el datagrama **tiene un límite de tamaño que hay que respetar**, y el receptor final no sabe tratar datos partidos.

Parte c) — Cálculo de los fragmentos

Datos: datagrama de 1500 bytes, `IHL = 5` (cabecera 20), payload = 1480 bytes, `DF = 0`, pasa por un enlace de MTU **576**.

Paso 1 — payload por fragmento:

```
Máximo por fragmento = 576 − 20 (cabecera) = 556
Redondeo a múltiplo de 8: 556 / 8 = 69,5 → 69 × 8 = 552 (el múltiplo de 8 más grande ≤ 556)
⇐ Payload por fragmento = 552
```

Paso 2 — cantidad de fragmentos:

```
n = ceil(1480 / 552) = ceil(2,681) = 3 fragmentos
```

Los 3 fragmentos:

#	`Fragment Offset` (bytes)	En unidades de 8	Payload	`Total Length`	`MF`
F1	0	0/8 = **0**	552	20 + 552 = **572**	1
F2	552	552/8 = **69**	552	**572**	1
F3	1104	1104/8 = **138**	1480 − 1104 = **376**	20 + 376 = **396**	**0**

Verificación:

```
Suma de payloads: 552 + 552 + 376 = 1480 ✓ (coincide con el original)
Offsets válidos: 0, 69 y 138 — los tres son enteros (552/8=69; 1104/8=138) ✓
MF: 1, 1, 0 ✓ (solo el último con MF=0)
```

Bytes que viajan en el enlace de MTU 576:

```
572 + 572 + 396 = 1540 bytes
Overhead de fragmentación = 1540 − 1500 = 40 bytes (→ 2,7 % del original)
```

Interpretación: por pagar 40 bytes de overhead (dos cabeceras IP repetidas), el datagrama pasó de 1500 a 1540 bytes en el cable. Y el costo real es otro: **si se pierde cualquiera de los 3 fragmentos, se pierde el datagrama entero.**

⚠️ **Y si el mismo datagrama viniera con `DF = 1`?** `L = 1500 > 576` y `DF = 1` → **R1 lo descarta** y manda **ICMP tipo 3 código 4** a A. Cero bytes salen por ese enlace. Y si el Path MTU ya estuviera cacheado en 576, el paquete de A tendría que ser de 576 bytes como máximo.

Parte d) — Algoritmo de descubrimiento del Path MTU

Primitivas: `enviar_msg_ICMP(destino, tipo, código, largo_payload)` *y* `recibir_msg_ICMP(fuente)` *(devuelve* `null` *si pasa un tiempo).*

Solución oficial (multiplicativa):

```
Pmtu = MTU(enlace hacia B) // parte de un valor tentativo
tries = 0
while (true):
enviar_msg_ICMP(destino = B, tipo = 8, largo = Pmtu) // Echo Request
pkt = recibir_msg_ICMP(fuente = B)
if (pkt != null):
if (pkt.code == 0): // es un Echo Reply: llegó entero
return Pmtu
else: // nueva aproximación (agresiva)
Pmtu = Pmtu * 0.85
else: // timeout u otro error de red
Pmtu = Pmtu * 0.85
tries++
if (tries >= 5):
return False
```

Análisis de la solución oficial y mi versión mejorada:

La solución oficial es funcional pero tiene **varios problemas** (que valen puntos si los mencionás):

1. 🔴 **`if pkt.code == 0` sin mirar `pkt.tipo`.** El **tipo 3 también tiene código 0** ("Net Unreachable"), como se ve en la propia tabla del examen. Si el router responde tipo 3 código 0, el algoritmo **devuelve un PMTU que nunca funcionó**.

2. 🔴 **No usa la MTU que viene en el ICMP.** El `RFC 1191` la pone en los **bytes 6-7** del tipo 3 código 4. El enunciado da la tabla **justo para que uses ese mensaje**, y la respuesta **ya está dentro del ICMP**.
3. 🟡 `Pmtu * 0.85` **nunca da el PMTU exacto.** Los valores son $1500 \cdot 0{,}85^k$; ninguno es 1492. Con PMTU real 1492 devuelve **1275** (**14,5 % bajo**).
4. 🟡 **No cachea el resultado.** El *almacenamiento en caché* es parte del mecanismo: sin él, cada datagrama grande genera un ICMP.
5. 🟡 **No hay cota inferior.** Puede bajar de 68 (el mínimo legal de IPv4) y seguir mandando sondas imposibles.
6. 🟡 **No distingue "MTU chico" de "ruta rota".** Un tipo 3 código 1 (host unreachable) o 9/10 (prohibido) no se arreglan bajando el tamaño.
7. 🟠 `tries` **solo cuenta en el timeout.** Si el router siempre responde tipo 3 código 4, el `while true` **no termina nunca**.

Mi solución: búsqueda binaria + lectura de la MTU del ICMP + caché:

```
// ======== ESTADO PERSISTENTE EN A ========
pm[B] = { valor, valido, timestamp } // caché de PMTU por destino
MAX_REINTENTOS = 5 ; MTU_MIN_ABSOLUTO = 68

int descubrir_PMTU(A, B):
// ---- Invariantes ----
// lo : tamaño que YA sabemos que pasa entero (cota inferior)
// hi : tamaño que YA sabemos que NO pasa (cota superior)
// Al salir, lo es el mayor tamaño conocido como transmisible.
if (pm[B].valido): lo = pm[B].valor // arrancamos del caché
else: lo = MTU_MIN_ABSOLUTO // 68: seguro por definición
hi = MTU(interfaz de A hacia B) // 1500: no puede pasar más
reintentos = 0

while (hi - lo > 1):
mid = (lo + hi) / 2 // ← búsqueda BINARIA
enviar_msg_ICMP(B, 8, 0, mid) // Echo Request con DF = 1
pkt = recibir_msg_ICMP(B)

if (pkt == null): // timeout
// Degradación elegante: asumimos el PEOR caso (que mid era
// demasiado grande) y seguimos binary-searching.
reintentos++
if (reintentos > MAX_REINTENTOS): cachear(B, lo); return lo
hi = mid

else if (pkt.tipo == 0 and pkt.codigo == 0):
lo = mid // llegó entero a B: ES válido

else if (pkt.tipo == 3 and pkt.codigo == 4):
// ★ AQUÍ ESTÁ LA MEJORA PRINCIPAL ★
m = leer_uint16(pkt, offset = 6) // Next-Hop MTU (RFC 1191)
if (m >= lo and m < mid): lo = m; hi = m // m ES transmisible
else: hi = mid // mensaje inconsistente

else if (pkt.tipo == 3):
// Otro inalcanzable (net, host, port, prohibido):
// NO es un problema de tamaño, es un problema de ruta.
cachear(B, lo); return ERROR_NO_EXISTE_RUTA

else: // tipo 11, 12, etc.
reintentos++
if (reintentos > MAX_REINTENTOS): cachear(B, lo); return lo
hi = mid

cachear(B, lo)
return lo
```

Lo que arregla mi versión:

Mejora	Por qué
`pkt.tipo == 0 && pkt.codigo == 0`	El tipo 3 tiene un código 0. Chequear solo el código hace que el algoritmo **diga que funcionó cuando no**. Bug real de la oficial
Leo la MTU de los bytes 6-7	El mensaje **ya trae la respuesta exacta**. **1 solo probe** reemplaza a los 8 de la búsqueda binaria, y el resultado es **exacto** en vez de 14,5 % bajo
Búsqueda binaria `(lo+hi)/2`	`O(log n)` : peor caso $\log_2(1432)$ = **11 probes**, **siempre**. El `× 0,85` nunca converge al valor exacto
Caché por destino	El PMTU es **propiedad de la ruta**. Sin caché, chaque paquete grande vuelve a generar un ICMP
Cota inferior `lo = 68`	Nunca sondea algo que **no puede funcionar**
Distingue los otros tipo 3	Un "host unreachable" no se arregla bajando el MTU
Contar todos los reintentos	La oficial puede no terminar nunca

Simulación con estos datos (PMTU real = 576, MTU local = 1500):

```
lo = 68 ; hi = 1500

mid = 784 → llega entero → lo = 784
mid = 1142 → llega entero → lo = 1142
mid = 1321 → llega entero → lo = 1321
mid = 1410 → llega entero → lo = 1410
mid = 1455 → llega entero → lo = 1455
mid = 1477 → llega entero → lo = 1477
mid = 1488 → llega entero → lo = 1488
mid = 1494 → ICMP 3/4 con MTU=576 → lo = hi = 576 (termina)

Resultado: PMTU = 576 ✓ EXACTO (8 probes)
⇐ En este caso el ICMP con el MTU exacto resolvió todo en 1 paso.
```

> **Tope para los 7 puntos:** el pseudocode completo y ejecutable, la **inicialización del estado**, el **manejo del timeout**, la **lectura del tipo y el código** del ICMP recibido, y **al menos una justificación** de por qué tu estrategia es mejor que "probar y ver qué pasa".

PARTE 4 — CIERRE

📌 Resumen

1. **ICMP existe porque IP no puede reportar sus propios errores** (problema de anidamiento). Los errores van **de nodo a nodo**.
2. **El formato**: tipo (8) + código (8) + checksum (16) + cuerpo. En `ping`, el cuerpo lleva **Identifier** y **Sequence Number**.
3. **Los tipos de memoria**: 0 Echo Reply, 3 Dest Unreachable, 4 Source Quench (**obsoleto**), 5 Redirect, 8 Echo Request, 9 RA, 10 RS, 11 Time Exceeded, 12 Parameter Problem, 13 Timestamp, 17 Address Mask (**obsoleto**).
4. **Los códigos del tipo 3**: 0 net, 1 host, 2 protocol, 3 port, **4 frag needed + DF**, 5 source route failed, 9/10 admin prohibited.
5. **ICMP no es confiable y no genera errores sobre errores** (salvo el 3/4), para no generar tormentas.
6. `ping`: Echo Request (tipo 8) / Echo Reply (tipo 0). Mide RTT y pérdida. `ping -M do -s <tamaño>` **diagnostica el PMTU.**
7. `traceroute` explota el **TTL**: cada router que lo ve llegar a 0 devuelve **ICMP tipo 11, código 0**. En el sabor **UDP** el destino devuelve **tipo 3 código 3**; en el sabor **ICMP**, **tipo 0**.
8. **Los `*` no significan "caído"**: significan "no hay respuesta" (filtrado, pérdida, o el ICMP de vuelta no llegó). Y el **camino de vuelta puede ser distinto del de ida** (ruteo asimétrico), así que los RTTs son del ciclo completo.
9. **Repaso Bloque D**: subnetting (`2^(32-p)-2` usables), LPM, TTL, MTU/MSS (`MTU-40` en IPv4), los 5 mensajes de OSPF, atributos de BGP (LOCAL_PREF ↑, AS_PATH length ↓, MED ↓), los 3 modelos de IPv6.
10. **Errores frecuentes**: confundir MTU con MSS, creer que anycast es un tipo de dirección, creer que IPv6 fragmenta en routers, confundir tipo 11 con 3/3 en traceroute, proponer Source Quench para congestión, creer que los routers reenvían broadcast.

✅ Autoevaluación

1. ¿Por qué los errores de IP no pueden reportarse con IP? ¿Y por qué los routers forwarding no generan un ICMP tipo 11 cuando un ICMP tipo 11 no puede entregarse?

▶ Respuesta

2. ¿Por qué el `Source Quench` (tipo 4) está obsoleto? ¿Qué se usa en su lugar?

▶ Respuesta

3. ¿Cuál es el tipo y código de ICMP que devuelve cada nodo en un `traceroute` sabor UDP, desde el primer router hasta el destino?

▶ Respuesta

4. Un `traceroute` muestra `* * *` en el salto 5 de 8. ¿Qué podés concluir?

▶ Respuesta

5. ¿Por qué los RTTs que muestra un `traceroute` no tienen por qué ser crecientes de un salto al siguiente?

▶ Respuesta

6. En el simulacro, la Teórica 1 tiene `10.0.0.0/24` y `10.0.1.0/24`. ¿Cuál matchea un paquete a `10.0.1.55`? ¿Se ve el longest prefix match?

▶ Respuesta

7. ¿Por qué el router no fragmenta un paquete con `DF = 1` aunque quepa en otro enlace con fragmentación disponible?

▶ Respuesta

8. ¿Cuál es el MSS de una conexión TCP sobre IPv6 en un enlace con MTU 1500, y por qué difiere del de IPv4?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1103` / `sirc1103`** — P3 (6 pts: principio de `traceroute`, tipos de mensaje, nodos que colaboran, limitaciones del `*` y del ruteo asimétrico)
- **Examen `eirc1012` / `sirc1012`** — Problema 2 (30 pts, con la **tabla de tipos y códigos de ICMP** que es material de estudio); P5 (IPv6)
- **Examen `sirc0612`** — P3 (fragmentación y reensamblaje)
- **Examen `sirc0703`** — P1 (5 mensajes de OSPF); P3 (RARP)
- **Examen `sirc0807`** — P2 (protocolos de enrutamiento)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.4.1 ("ICMP — El protocolo de control de la capa de Internet") y cap. 5 en general
- **RFC 792** — ICMP (la tabla de tipos y códigos que hay que saber de memoria)
- **RFC 1122** — Requirements for Internet Hosts (comportamiento de los hosts y de los routers; reglas de ICMP)
- **RFC 4443** — ICMPv6 (tipos 1-4 y 128-137)
- **RFC 1191** — Path MTU Discovery (el MTU del enlace bloqueante en los bytes 6-7 del ICMP)
- **Clase siguiente:** 38 — Capa de enlace: servicios y framing

Clase 38

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El problema: la capa física no sabe de mensajes
1.2 Las cinco funciones de la capa de enlace
1.3 Los tres métodos de entramado
1.4 HDLC — High-Level Data Link Control
1.5 PPP — Point-to-Point Protocol
2.1 Ejercicio 1 — Byte stuffing completo con bytes hexadecimales
2.2 Ejercicio 2 — Bit stuffing bit a bit
2.3 Ejercicio 3 — Conteo de bytes: qué pasa si se pierde un bit
2.4 Ejercicio 4 — Flags abreviados de 802.11
3.1 ⭐ Examen 2006-12, Pregunta 2 (10 puntos) — framing
3.2 Examen 2010-03, Pregunta 3 (8 puntos) — PPP
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: E (Capa de Enlace) · **Semana:** 13 · **Duración estimada:** 120 min **Correlativa:** Clase 37 (cierre del bloque anterior) · **Sigue con:** Clase 39 (detección de errores: paridad, checksum y CRC) **Material de referencia:** Tanenbaum cap. 3 y 4 · RFC 826 · IEEE 802.3 · IEEE 802.11 · Práctico 5 de 2008

🎯 Al terminar esta clase vas a poder

1. Explicar **por qué la capa física no alcanza** para entregar un mensaje, y por qué sin *framing* el receptor no puede hacer nada.
2. Enumerar las **cinco funciones** de la capa de enlace y decir cuál de ellas resuelve qué problema.
3. Describir los **tres métodos de entramado** —conteo de bytes, *byte stuffing* y *bit stuffing*— y decir **por qué** cada uno existe y **qué falla** en cada uno.
4. Ejecutar un **bit stuffing bit a bit** y un **byte stuffing byte a byte**, y justificar **por qué HDLC inserta el cero después de cinco unos y no de otro número**.
5. Dibujar el formato de las **tramas de HDLC** (Información, Supervisión, No numerada) y las **cuatro fases de PPP**.
6. Resolver completo el **Ejamen 2006-12 Pregunta 2** (10 puntos).

> **Prerequisito del examen:** `2006-12 P2` (10 pts — por qué el framing, una técnica en canal orientado a bit y otra en canal orientado a byte) · `2010-03 P3` (8 pts — PPP y byte stuffing) · `2008-12 P4` y `2007-03` / `2006-12` (mismo mecanismo de transparencia, aplicado a POP3 y a un protocolo de capa 2).

PARTE 1 — TEORÍA

1.1 El problema: la capa física no sabe de mensajes

La capa física es la más tonta del modelo. Su trabajo es **convertir bits en señales** y viceversa. No sabe qué es un paquete, ni de quién es, ni a dónde va. Manda un voltaje por el cable y listo.

Y como no sabe nada de mensajes, transmite **una corriente infinita de bits**:

```
Capa física:   10110010100110101101001011010010110100101011001010
               ¿Dónde termina el paquete de H1?
               ¿Dónde empieza el de H2?
               NO SE SABE. No hay ningún modo de saberlo.
```

Why is this fatal? Because **el receptor necesita el mensaje entero para entregarlo a la capa 3**. Si no sabe dónde termina el datagrama que acaba de recibir, no puede calcular el checksum del datagrama, no puede quitarle la cabecera, y no puede pasárselo a IP. Y peor todavía: si el datagrama que sigue estaba pegado al anterior, **el datagrama recibido sale corrupto**, y sin embargo el receptor no tiene forma de saberlo.

```
┌─ Lo que le llega al receptor ──────────────────────────────────────┐
│  1011 0010 1001 1010 1101 0010 1101 0010 1010 1100 10 1011 0010    │
│  └── datagrama 1 ──┘└──────── datagrama 2 ──────────┘               │
│  Con el checksum de 1: NO SIRVE NADA.                              │
└────────────────────────────────────────────────────────────────────┘
```

O sea: **el entramado (*framing*) es el acto de poner marcas en el flujo de bits para que el receptor sepa dónde empieza y termina cada unidad de datos**. Sin esas marcas, la capa 2 no puede entregarle nada a la capa 3.

1.2 Las cinco funciones de la capa de enlace

La capa de enlace no hace una sola cosa: hace cinco. Y el examen las pide nombradas, así que conviene tenerlas claras **con su propósito**:

#	Función	Glosa	Pregunta que responde
1	**Entramado**	*framing*	¿dónde empieza y termina cada mensaje?
2	**Acceso al medio**	*media access control (MAC)*	¿quién tiene derecho a transmitir ahora?
3	**Detección de errores**	*error detection* (y corrección)	¿los bits que llegaron son los que salieron?
4	**Control de flujo**	*flow control*	¿el receptor va a desbordarse?
5	**Transparencia al enlace**	*data link transparency*	¿el contenido no va a romper el entramado?

El punto 5 es el que casi todos olvidan y es el que sostiene la mitad del examen de hoy. Se conecta con el punto 1 así:

```
Si el delimitador de trama es el byte 0x7E y el DATO también puede
contener 0x7E, entonces el receptor ve un final de trama donde
no la hay.

┌── Trama 1 ──┐┌── "dato" 0x7E que NO es trama ──┐┌── Trama 2 ──┐
│ 7E ... 7E   ││          7E                     ││ 7E ... 7E   │
└─────────────┘└─────────────────────────────────┘└─────────────┘
              ↑ el receptor cortó acá. Datos perdidos.
```

Transparencia significa literalmente "a través de mí podés pasar cualquier byte que quieras". Para lograrlo, hay que **escapar** (*escape*) los bytes problemáticos: cambiarlos por una secuencia que el reconocedor de tramas no confunde con un delimitador. Es la razón de existir del *stuffing*.

Sobre el control de flujo (punto 4): es el mismo concepto que en la capa de transporte (clases 19 y 20) pero aplicado a **un solo enlace**: el receptor dice "me entra(n) N tramas más" y el emisor frena. En Ethernet half-duplex el mecanismo real es colisión + *backoff* (clase 41), no un crédito explícito.

1.3 Los tres métodos de entramado

1.3.1 Conteo de bytes (*byte count*)

La idea más ingenua: **el primer byte de la trama dice cuántos bytes de datos vienen después**.

```
┌────────┬──────────────────────────────┬──────────────┐
│ 5 (1B)│  5 bytes de datos            │  checksum    │
└────────┴──────────────────────────────┴──────────────┘
 longitud
```

Es elegantísimo y es una **trampa**. El problema es el acoplamiento entre la longitud declarada y los delimitadores reales:

```
EMISOR                                          RECEPTOR
  │
  │  00000101  ABCDE
  │  └─────┘     └─ 5 bytes
  │     └── "van 5"
  │
  │  ⚠ Si se pierde UN SOLO BIT del byte de longitud,
  │    el receptor cree que van 4 bytes (o 6).
  │    Se come un byte de DATOS como si fuera parte de la trama.
  │    Y el próximo mensaje empieza, para él, un byte tarde.
  ▼
RECEPTOR:  lee "0000101" = 5? no, lee 0000101|0 = 10 (desfasado 1)
         → DESFINICIÓN PERMANENTE.
         → lee basura como si fuera el mensaje siguiente.
```

Y lo peor no es que el error sea de esa trama: es que el error es irrecuperable y se propaga hacia adelante. El receptor quedó desfasado un byte, y **todas** las tramas siguientes se leen corrida. Sigue desfasado para siempre hasta que por casualidad coincide otra vez la alineación.

> ⚠️ **Ésta es la razón histórica por la que existe el *stuffing*.** Ningún protocolo serio usa conteo de bytes como mecanismo único de delimitación. Se usa **junto con** delimitadores (que permiten resincronizarse), nunca solo. Si el examen te pregunta "¿cuál es el problema del conteo de bytes?", la respuesta es

> exactamente esta: **un solo bit perdido destruye la sincronización de forma permanente, no recuperable**.

1.3.2 Byte stuffing — canal orientado a byte

Cuando el medio transmite **byte a byte** (o sea, el receptor puede mirar byte a byte y no le importa la alineación de bits), la técnica es:

1. Se elige un **byte delimitador** (*flag*) que **no pueda aparecer en los datos** por acuerdo previo. Típicamente `0x7E` (ASCII `~`).
2. Se elige un **byte de escape** (*escape*, ESC) que tampoco aparece en los datos. Típicamente `0x7D` (ASCII `}`).
3. El emisor, antes de mandar, recorre el payload: - si encuentra `FLAG` → lo reemplaza por `ESC FLAG` - si encuentra `ESC` → lo reemplaza por `ESC ESC`
4. El receptor: - ve `FLAG` → fin (o comienzo) de trama - ve `ESC` → **come el ESC** y el byte siguiente es **literal**, no marca

Ejemplo completo, byte a byte.

Datos que hay que enviar: `FLAG A B ESC C FLAG`. O sea, un payload que **deliberadamente** contiene el delimitador y el escape, para ver qué hace el emisor.

```
DATOS (payload crudo):   F  L  A  G  A  B  E  C  F  L  A  G
                          \_________/      ^  ^       \_______/
                          el FLAG de los   el ESC     otro FLAG
                          datos se escapa   se escapa

EMISOR: recorre de izquierda a derecha
──────────────────────────────────────────────────────────────
 pos  byte     ¿es delimitador/escape?      qué mando
──────────────────────────────────────────────────────────────
  1   FLAG      sí, es FLAG            →   ESC FLAG
  2   A         no                      →   A
  3   B         no                      →   B
  4   ESC       sí, es ESC             →   ESC ESC
  5   C         no                      →   C
  6   FLAG      sí, es FLAG            →   ESC FLAG
──────────────────────────────────────────────────────────────

SOBRE EL CABLE (trama completa):
    [FLAG]  ESC FLAG  A  B  ESC ESC  C  ESC FLAG  [FLAG]
       |                                    |
  inicio de trama                       fin de trama

Longitud: 1 + 2 + 1 + 1 + 2 + 1 + 2 + 1 = 11 bytes
(el payload original tenía 10)
```

RECEPTOR: desescapa

```
[FLAG]  ESC FLAG  A  B  ESC ESC  C  ESC FLAG  [FLAG]
   |       |      |  |     |    |     |  |       |
   |       └──────┴──┘     |    |     └──┴───────┘
   |       leo ESC: "el    |    |        leo ESC: "el
   |       siguiente es    |    |        siguiente es
   |       LITERAL"        |    |        LITERAL"
   ▼       ▼               ▼    ▼        ▼       ▼
inicio  F  L  A   G  A  B  E  S  C    F  L  A  G  fin
                           ^   ^
                           los dos ESC se comen UNO SOLO
                           (queda un ESC literal)

DATOS RECUPERADOS:  F L A G A B E S C F L A G      ← idénticos al original
```

El detalle fino: `ESC ESC` se colapsa a **un solo** `ESC` literal. Si colapsáramos a cero, el `ESC` original se perdería; si no colapsáramos nada, quedaría ambiguo.

PPP usa exactamente este esquema, y es lo que se pregunta en `2010-03 P3b`. HDLC también, pero con stuffing de **bits**.

1.3.3 Bit stuffing — canal orientado a bit

En un **canal orientado a bit** (*bit-oriented channel*) el receptor no tiene obligación de recibir bytes alineados: puede perder un bit y desfasarse. Por eso el HDLC no puede usar un byte delimitador, porque un byte desalineado ya no es un byte.

La regla de HDLC:

> El emisor inserta un `0` **después de cada secuencia de cinco** `1` **seguidos** en el campo de datos (descontando los flags). El receptor, al ver cinco `1` seguidos, mira el bit siguiente: si es `0`, lo descarta; si es `1`, es que venía un flag.

El flag de HDLC es `01111110` (0x7E, igual byte que en el byte stuffing, pero ahora como patrón de bits).

Ejemplo completo, bit a bit.

Datos: `0111111111110` (un 0, once unos, un 0 — es decir, once unos seguidos).

```
DATO ORIGINAL (13 bits)
 0 1 1 1 1 1 1 1 1 1 1 1 0
   └──────── 11 unos ────────┘

Paso: el emisor cuenta los unos. Al llegar al QUINTO uno, inserta un 0.

 0  1 1 1 1 1  0  1 1 1 1 1  0  1  0
 |  └── 5 ──┘     └── 5 ──┘
 1 solo            + 1 solo que ya venía
 (previo)

DATO RELLENADO (15 bits)  =  011111011111010
 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0
```

Sobre el cable:

```
[FLAG] 011111011111010  [FLAG]
01111110                01111110
└ inicio ┘              └ fin ┘
```

Recepción:

```
Leo 011111011111010
   ^^^^^ = cinco unos → el siguiente bit DEBE ser un 0 stuffed → lo como
   (no cuenta para el conteo de cinco)
Sigo: 11111 → siguiente bit es 0 → lo como
Sigo: 1 → queda uno suelto
Sigo: 0
─────────────────────────
RECUPERADO: 0 1 1 1 1 1 1 1 1 1 1 0  =  el original
```

¿Por qué cinco unos y no tres, ni seis, ni siete?

Esta es **la pregunta conceptual** de la clase, y es la que más puntos da.

Mirá el flag: `01111110`.

```
    0 1 1 1 1 1 1 0
      └─ SEIS unos ─┘
```

El flag tiene SEIS unos seguidos. Ese es el número clave. Ahora, ¿qué promesa tiene que hacer el relleno?

> **Promesa del relleno:** en el campo de datos rellenado, **nunca** aparecen seis unos seguidos.

Si esa promesa se cumple, entonces cualquier secuencia `01111110` que encuentre el receptor **es un flag y solo un flag**, porque en los datos es imposible. Y entonces el receptor puede buscar esa secuencia **bit a bit**, como un nilaija ciego, y **siempre se engancha en un flag verdadero**.

Ahora, ¿qué valor de relleno cumple la promesa?

Relleno cada	¿Los datos pueden tener 6 unos seguidos?	¿El flag `01111110` es único?	Overhead
cada **3** unos	no (máx. 3)	sí	**alto**: 1 bit de cada 3
cada **4** unos	no (máx. 4)	sí	medio
cada 5 unos	no (máx. 5)	**sí**	**mínimo aceptable**
cada **6** unos	**SÍ** (puede haber 6)	**NO**	—

- Con 6: los DATOS pueden contener "01111110" sin stuffing.
  0 1 1 1 1 1 1 0 ← esto es un FLAG... o son datos. INDISTINGUIBLE.
  El receptor corta la trama donde no debe. Se pierden datos.

- Con 5: los datos NUNCA tienen 6 unos. El flag es único.
  Es el MAYOR número de unos que todavía garantiza la unicidad.
  Por eso 5: es el valor ÓPTIMO — unicidad garantizada con el mínimo
  overhead posible.

Y el "slip de un bit" (la sincronización se pierde, pero se recupera). Supongamos que en el medio **se pierde un bit** (o se duplica). El receptor, a partir de ahí, lee todos los bits corrido un lugar. ¿Y ahora?

```
ANTES del error:      el receptor está alineado, ve los flags.
──────── error: se pierde 1 bit ────────
DESPUÉS:              el receptor lee corrida. Ya no reconoce el flag
                      de cierre de la trama anterior.

PERO: sigue escaneando bit a bit buscando "01111110", y como los datos
      rellenados NO pueden contener seis unos seguidos, no se va a enganchar
      con ningún falso flag.

RESULTADO: se engancha en el PRIMER flag verdadero que viene
           (el de inicio de la trama siguiente, a lo sumo una trama después).
           Se resincroniza solo. ✓
```

Con tres o cuatro, la promesa también se cumple, así que técnicamente también funciona... **pero** el costo es el doble de overhead sin ninguna ventaja. Y con stuffing cada 4, el patrón `0111 0` puede aparecer mil veces en los datos y un receptor mal implementado puede engancharse ahí. Lo que aprender de la tabla de arriba es el **principio**: *el número de stuffing debe ser estrictamente menor que la cantidad de bits iguales consecutivos del delimitador*, y se elige el mayor valor que cumple esa condición porque cada bit de relleno es overhead.

1.3.4 Framing por flags y el caso de los flags "abreviados" de 802.11

Un cuarto caso, muy usado en la práctica: **delimitar sólo con flags de principio y de fin**, y confiar en que el medio no inserte bits (canales síncronos, enlaces por fibra, enlaces con reloj compartido).

En **802.11** el entramado es distinto: no hay flags. El principio de cada trama lo marca el **preámbulo de 8 bytes** (7 de sincronismo + SFD), que es un patrón inequívoco, y el final lo marca el **FCS de 4 bytes** seguido de silencio del medio. Pero 802.11 tiene una sutileza que vale la pena conocer:

Tramas de gestión sin FCS. El estándar IEEE 802.11 permite que ciertas tramas de gestión —**Beacon, Probe Response y Association Response**— se transmitan **sin el campo de verificación de trama**, porque la estación que las recibe ya está sincronizada por reloj con el AP (se announcement cada 100 ms con beacons) y porque el riesgo de colisión en ese instante es muy bajo.

```
Trama normal:
┌────┬─────┬──────────┬──────────┬──────────┬──────┐
│FC  │Dur  │Dir/Orig  │Dir/Dest  │  cuerpo  │ FCS  │
└────┴─────┴──────────┴──────────┴──────────┴──────┘
 ▲                               ▲
 │                               └─ el FCS marca el FINAL
 └─ el FC (frame control) + duración marcan el INICIO

Trama de gestión abreviada (Beacon / Probe Response):
┌────┬─────┬──────────┬──────────┬──────────┐
│FC  │Dur  │Dir/Orig  │Dir/Dest  │  cuerpo  │      ← sin FCS
└────┴─────┴──────────┴──────────┴──────────┘
                                y el SSID puede omitirse
                                (ya se conoce del beacon anterior)
```

El punto conceptual: en el caso abreviado, **el final de la trama ya no lo marca un patrón de bits sino el silencio del medio**, y el reloj de sincronización es lo que permite al receptor saber cuánto debía durar. Eso es justo lo contrario a "el flag marca el final", y por eso es un caso aparte.

1.4 HDLC — High-Level Data Link Control

HDLC es **el** protocolo de la capa de enlace con más influencia histórica: de él descienden el SDLC de IBM, el LAPB, el **PPP** e incluso el **LAPD** (token ring) y el **LLC** de IEEE 802.2, que es la parte de IEEE 802 que define la capa 2 "única" para las LAN de 802 (el resto —MAC, CSMA/CD— es específico de Ethernet).

Formato de la trama:

```
 Flag      Address   Control   Information              FCS      Flag
01111110   8 bits    8/16 b    variable (0..N)          16 b    01111110
   |          |         |            |                   |         |
   |          |         |            |                   |         └─ fin
   |          |         |            |                   └─ CRC-CCITT
   |          |         |            └─ payload (puede llevar bit stuffing)
   |          |         └─ depende del TIPO de trama (ver abajo)
   |          └─ 0xFE (undirected) | 0x01 (stations) | ...
   └─ inicio de trama
```

Los tres tipos de trama, que es lo que se pregunta:

Tipo	Nombre	Campo *Control*	Para qué
I	**Información**	`0xxx` (N(S), N(R), P/F)	Transporta datos **y** el piggybacking del ACK/NAK
S	**Supervisión**	`10xx` (SS, P/F)	Solo control: **RR** (00), **RNR** (01), **REJ** (10), **SREJ** (11)
U	**No numerada**	`11xx`	SET, SABM, DISC, UA, FRMR, SABME, UI

```
Trama I (Información):    [N(S)] [N(R)] [P/F]
                          └seq   └ACK   └pide ACK (poll) o fin
```

Lo importante conceptualmente: HDLC implementa un **ARQ con ventana deslizante** y **piggybacking** (el ACK viaja dentro de una trama de datos en el sentido contrario). Es el mismo esqueleto que vas a ver en la clase 40 (ARQ) y en TCP.

1.5 PPP — Point-to-Point Protocol

¿Por qué existe? El mundo de los enlaces punto a punto de 1990 (líneas telefónicas, líneas dedicadas, RS-232, enlaces conmutados de Frame Relay) necesitaba un protocolo que hiciera:

1. **Multiplexar** varios protocolos de red (IP, IPX, AppleTalk) sobre un mismo enlace. Sin esto, cada uno necesitaría su propio cable.
2. **Configurar el enlace**: negociar Quality of Service, autenticar, asignar dirección IP.
3. **Detectar errores** en el enlace (FCS), **encapsular** y ser **fiable** opcionalmente.
4. Ser **independiente del medio**: el mismo protocolo sobre cobre, fibra o línea telefónica.

Formato de la trama PPP:

```
[FLAG] Address Control Protocol    Payload    FCS        [FLAG]
7E     FF       03      2 bytes    0..1500   16 bits    7E
                |       |
                |       └─ 0021 = IPv4 | 0021...
                └─ 0x03 = "sin numerar, sin ACK, la respuesta por defecto"

Para las tramas de la capa de red se usa ESCAPE byte-stuffing: 0x7E → 0x7D 0x5E
```

Las cuatro fases de PPP (esto es lo que se pregunta en `2010-03 P3a`):

```
FASE 1 ── LCP (Link Control Protocol)
│   Configura y testea el enlace. Opciones negociables:
│   - MRU (Maximum Receive Unit)
│   - MRF (Magic Number, detectar bucles y enlaces loops)
│   - Quality of Service
│   - Autenticación (requiere que se habilite)
│   - Compression (MPPC)
▼   Secuencia: LCP-CONFIG-REQUEST → LCP-CONFIG-ACK / LCP-CONFIG-NACK
▼   Después: LCP-TERMINATE-REQUEST → LCP-TERMINATE-ACK
│
FASE 2 ── AUTENTICACIÓN
│   Sólo si LCP la habilitó. Opciones:
│   - PAP (Password Authentication Protocol): TEXTO PLANO. Inseguro.
│   - CHAP (Challenge Handshake): desafío-respuesta con MD5. Mejor.
│   Secuencia: AUTHENTICATE-REQUEST → AUTHENTICATE-CHALLENGE
│               → AUTHENTICATE-RESPONSE → AUTHENTICATE-SUCCESS/FAIL
▼
FASE 3 ── NCP (Network Control Protocol)
│   Uno por cada protocolo de capa de red que se quiera usar:
│   IPCP para IP, IPXCP para IPX, etc. Asigna la dirección IP del enlace
│   (funciona como un mini-DHCP).
▼
FASE 4 ── DATOS (carry payload)
    Tramas de datos de capa 3 encapsuladas, con la última trama en la
    cabecera PPP. State = "Network phase".
```

La respuesta oficial a `2010-03 P3a` es corta y muy precisa: lo que hace a PPP **más simple que un protocolo de acceso al medio como CSMA** es que **no opera en un medio de difusión (*broadcast*), donde muchos emisores compiten por el canal**. En un enlace punto a punto hay **exactamente dos estaciones y sólo una puede hablar**: no hay.CONTENCIÓN, y por eso **no hace falta direccionamiento MAC**. Ese es el argumento central, no "porque tiene menos campos".

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - La existencia conceptual del **entramado**: la capa física no sabe de mensajes, la capa 2 tiene que poner las marcas. Es la razón de ser de la capa. - **Bit stuffing cada cinco unos** en HDLC, con el flag `01111110`. La justificación (seis unos en el flag → cinco es el máximo que da unicidad) no cambió en 60 años. - **Byte stuffing** con `FLAG` / `ESC` en PPP, HDLC y en la práctica mil protocols. - El **byte stuffing con DLE STX/DLE ETX** como el mecanismo de transparencia de POP3 (`2008-12 P4`). Es literalmente el mismo algoritmo con otros nombres.

Cambió — el dónde, más que el cómo: - **El entramado por stuffing quedó atrás como problema de ingeniería real.** En 2026 los enlaces de datos de las LAN son de 1 a 400 Gbit/s sobre fibra o cobre de calidad, con BER del orden de `10^-18` o mejor. **Un diseño nuevo no elige stuffing**: elige un identificador de longitud explícito y un FCS fuerte (CRC-32 o mejor). El stuffing sobrevive, pero como **herencia histórica en protocolos de 1981**, no como decisión de diseño de hoy. - **El estándar de Ethernet nunca hizo stuffing.** Usa **longitud**

explícita (campo *Type/Length*, y el límite de 1518 bytes) más el hecho de que el *preámbulo* de 8 bytes re-sincroniza a nivel físico en cada trama. El problema del desfasamiento de 1 bit que rompe el conteo de bytes **no existe en Ethernet**: es justamente lo que motivó el diseño con preámbulo. - **802.1Q (VLAN) insertó un tag de 4 bytes dentro de la trama** para agregar información de VLAN sin tocar el formato base. Es el ejemplo vivo de que la trama de Ethernet sigue siendo un contenedor del que se pueden meter cosas adentro. Ver §8 de `ACTUALIZACIONES.md` y la clase 42. - **PPP sigue vivo y multiplicado**: PPPoE (`RFC 2516`, verificar), PPPoA, y PPP en celdas 4G/5G — que es donde realmente se usa hoy: el "demux" del módem es PPP con IPHC. - **`RFC 826` — ARP (1982)** es el ejemplo canónico de por qué el entramado sigue siendo indispensable: un ARP request y un ARP reply comparten el mismo formato de trama y sólo el campo *Type* los distingue.

Consecuencia para el examen: la teoría del framing **se sigue preguntando tal cual** (es `2006-12 P2`, de 10 puntos). Lo que se actualiza es el segundo nivel del razonamiento: "esto se hacía porque los medios eran ruidosos y los protocolos seriales tenían que resincronizarse" es la explicación **de 2011**; "esto se hace porque el medio no garantiza alineación de bits" es la explicación **correcta**, y sigue siendo la que se corrige.

PARTE 2 — PRÁCTICA RESUELTA

> ⚠️ **Aclaración sobre el material oficial.** El **Práctico 5 de 2008** no tiene un ejercicio de entramado: sus ejercicios 1–12 son de paridad, checksum, CRC, CDMA, ALOHA, ARP y CSMA/CD. (El "Ejercicio 12" es de ARP, no de stuffing; las figuras 5.11 y 5.12 que se le atribuyen son de CDMA.) Por eso los cuatro ejercicios de esta parte son **inventados y resueltos** con el resultado **byte a byte**, que es lo que la cátedra exige.

2.1 Ejercicio 1 — Byte stuffing completo con bytes hexadecimales

Enunciado. Se transmite un payload que, en hexadecimal, es `7E 41 7D 42 7E`. El delimitador es `0x7E` y el byte de escape es `0x7D`. Determinar byte a byte qué va por el cable y qué reconstruye el receptor.

Resolución.

```
Original:  7E  41  7D  42  7E
           |   |   |   |   |
           |   |   |   |   └─ 4° tipo: aparece FLAG → hay que escapar
           |   |   |   └──── dato normal
           |   |   └──────── 3° tipo: aparece ESC  → hay que escapar
           |   └──────────── dato normal
           └──────────────── 2° tipo: aparece FLAG → hay que escapar
```

Paso 1 — el emisor recorre de izquierda a derecha y aplica dos reglas.

byte	entrada	regla	salida
1	0x7E	es FLAG → ESC + original	0x7D 0x7E
2	0x41	dato normal	0x41
3	0x7D	es ESC → ESC + original	0x7D 0x7D
4	0x42	dato normal	0x42
5	0x7E	es FLAG → ESC + original	0x7D 0x7E

```
TOTAL sobre el cable (payload):  8 bytes
(el original tenía 5; se agregaron 3 bytes de escape)
```

```
TRAMA COMPLETA:
┌──────┬──────────────────────────────┬──────┐
│ 7E   │ 7D 7E 41 7D 7D 42 7D 7E      │ 7E   │
│FLAG  │      payload escapado        │ FLAG │
└──────┴──────────────────────────────┴──────┘
```

Paso 2 — el receptor desescapa.

```
lee  7E   → "arrancó una trama"
lee  7D   → "¡escape!"  → el próximo byte es LITERAL
lee  7E   → emite 0x7E, sigue
lee  41   → emite 0x41, sigue
lee  7D   → "¡escape!"  → el próximo byte es LITERAL
lee  7D   → emite 0x7D, sigue          ←── OJO: come UNO SOLO
lee  42   → emite 0x42, sigue
lee  7D   → "¡escape!"  → el siguiente es LITERAL
lee  7E   → emite 0x7E, sigue
lee  7E   → "terminó la trama" (FLAG suelto)

PAYLOAD RECUPERADO:  7E 41 7D 42 7E    ✓ idéntico al original
```

Interpretación. El receptor **nunca vio un `7E` dentro del payload**, aunque el payload tuviera dos. Por eso puede usar `7E` como delimitador sin ambigüedad: la propiedad que el stuffing garantiza es exactamente "*ningún `7E` en los datos del campo de datos puede ser tomado como fin de trama*".

⚠️ Errores típicos

- **Escapar dos veces** el mismo ESC (mandar `7D 7D 7D`). El emisor sólo **duplica**: cada byte que sea ESC se reemplaza por **exactamente dos**.
- **Olvidar el ESC delante de un FLAG que aparece en el medio del payload.** Es el error que arruina la trama entera.
- En el receptor, **no consumir** el byte que sigue al ESC. El ESC **no se emite**: se descarta.

2.2 Ejercicio 2 — Bit stuffing bit a bit

Enunciado. Con el protocolo HDLC (flag `01111110`), enviar el campo de datos `110110111111101`. Mostrar el resultado bit a bit.

Resolución — paso a paso, contando los unos de a cinco.

```
DATO ORIGINAL:  1 1 0 1 1 0 1 1 1 1 1 1 1 0 1
                └2┘ └2┘ └──────── 7 unos ────────┘ └┘

Recorrido, agrupando cada racha de unos:
 racha 1:  1 1            (2 unos)  → sin stuffing
 racha 2:  1 1            (2 unos)  → sin stuffing
 racha 3:  1 1 1 1 1 | 1 1    (7 unos) → se parte en 5 + 2

racha de 7 unos =  [1 1 1 1 1] (se completa el grupo) + [0 stuffed] + [1 1]
```

```
RESULTADO BIT A BIT:
 ┌─┐ ┌─┐ ┌─┬─┬─┬─┬─┐   ┌─┬─┐
 1 1 0 1 1 1 0 1 1 1 1 1 0   1 1 0 1
          └─── 5 unos ──┘  ↑
                           stuffed

HEXADECIMAL:  1101 1011 1111 101   →   D B F (queda 1 bit de la última fila)

En binario plano:  1101101111110101     ← 16 bits

Conteo verificado:
  original            : 15 bits
  stuffing: +1 cero   = 16 bits ✓
  ¿quedó algún grupo de 6 unos?  NO: los grupos son 2, 2, 5, 2. Máximo 5 ✓
```

Trama completa:

```
01111110   1101101111110101   01111110
[FLAG]        16 bits          [FLAG]
```

Interpretación. El receptor, al barrer, encuentra `11111` en la posición 7, come el `0` siguiente, y **el siguiente grupo de unos empieza en el bit 9**: `1 1 0 1`. Cuando llegue al flag de cierre `01111110` ve **seis** unos seguidos — imposible en datos rellenados — y lo reconoce como flag.

> **Comprobación de la regla:** el receptor sólo puede buscar el flag con seguridad **porque** los datos rellenados no tienen seis unos seguidos. Si hubiéramos stuffing cada 6, el dato `110110111111 1` (con 7 unos) se habría enviado **sin** stuffing y contendría el flag. Ver §1.3.3.

2.3 Ejercicio 3 — Conteo de bytes: qué pasa si se pierde un bit

Enunciado. Un protocolo usa conteo de bytes. Se envía el mensaje `HELLO` precedido de un byte de longitud. Explicá y **mostrá numéricamente** qué pasa en el receptor si se pierde **un solo bit** de la línea.

Resolución.

Sin errores:

```
EMISOR                                          RECEPTOR
  │                                               │
  │  00000101  H  E  L  L  O                      │
  │  └── 5 ──┘ └──── 5 bytes ────┘                 │
  │                                               │
  │                          lee 00000101 = 5     │
  │                          come 5 bytes → HELLO ✓
  ▼
```

Con un bit perdido (se borra el primer bit del byte de longitud):

```
Se transmite:      00000101  H  E  L  L  O
Se PIERDE el bit:        (borrado)      ↓ queda:  0000101...
Sobre el cable:  0 0 0 0 1 0 1  H  E  L  L  O
                  └─────┬─────┘
                el receptor lee 8 bits:  0 0 0 0 1 0 1 H

Longitud leída = 0000101H
   con H = 0 ('H' = 0x48 = 01001000, su primer bit es 0):
      00001010 = 10 bytes  ← ¡deberían ser 5!
```

```
El receptor cree que la trama tiene 10 bytes. Entonces come:

  long=0x48  H    E    L    L    O    [primer byte de la TRAMA SIGUIENTE]
    └──────── 10 bytes "de datos" ──────────────────────────┘

RECUPERADO: "HELLO" + basura.  ✗  Y la trama SIGUIENTE empieza un byte tarde.
```

Y lo grave, que es la segunda parte de la respuesta:

```
estado real del cable:   [largo=5][ABCDE] [largo=3][XYZ] [largo=2][AB]...
lo que el receptor cree:  [largo=10]──────────────┘  desfasado +1 byte

→ sigue corrido 1 byte para siempre.
→ TODAS las tramas siguientes se leen mal.
→ No hay forma de resincronizar: el conteo no se autocorrige.
```

Interpretación — la frase para el examen:

> El conteo de bytes **acopla la longitud declarada con la alineación de la trama**. Un único error de bit desplaza el punto de lectura de forma **permanente e irrecuperable**: el receptor interpreta bytes de datos como longitud, y de ahí en adelante lee todas las tramas corrido. El stuffing existe justamente para que el receptor **pueda volver a engancharse** con un delimitador inequívoco después de cualquier error de alineación.

2.4 Ejercicio 4 — Flags abreviados de 802.11

Enunciado. En una red 802.11 en modo infraestructura, un Access Point difunde **Beacon** sin campo FCS y con el SSID omitido. (a) ¿Cómo sabe la estación receptora dónde termina la trama? (b) ¿Qué se rompe si hay una colisión justo durante ese beacon?

Resolución.

(a) El final **no lo marca un patrón de bits**. La estación lo deduce de:

1. El reloj de sincronización. Todas las tramas de una BSS tienen longitud CONOCIDA de antemano para cada tipo de trama: la longitud de un beacon con parámetros fijos es una constante del estándar. Entonces:

   t_inicio + longitud_conocida = t_fin

2. El silencio del medio. Al terminar la trama, el canal queda en silencio durante el Slot Time (16 µs en 802.11b) → se detecta el fin.

3. El SFD + sincronía de la trama SIGUIENTE. Si llega otra trama, el preámbulo marca el nuevo comienzo y el receptor resincroniza.

(b) El riesgo real: si el beacon se pierde o se corrompe sin que el FCS lo diga, **la estación no se entera**. Consecuencias:

- La estación cuenta el número de beacons perdidos para medir la calidad del enlace (función de RSSI y roaming). Perder beacons sin avisar degrada esa estimación.
- La sincronización temporal (TWT / timetable de Wi-Fi 6, clase 43) puede desfasarse si falta un beacon.
- El SSID omitido significa que un beacon con "hidden SSID" no se puede usar para descubrir la red: hace falta un probe activo.

Interpretación. El tradeoff es explícito: **se ahorra overhead** (4 bytes de FCS + el elemento SSID ≈ 34 bytes) **a cambio de perder la verificación de integridad**. Sólo es aceptable en tramas cuya **pérdida no es crítica** (sincronización) y cuya **retransmisión sería inútil** porque el contenido se va a repetir igual en el beacon siguiente.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2006-12, Pregunta 2 (10 puntos) — framing

> **Fuente:** Examen `eirc0612` / `sirc0612` — diciembre de 2006. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0612.pdf

Enunciado (texto del examen)

> Pregunta 2 (10 puntos)
>
> En capa 2 se introduce el concepto de "framing" o entramado. Explique por qué este proceso es necesario y describa: a) una técnica para realizar "framing" en un canal orientado a bit b) una técnica para realizar "framing" en un canal orientado a byte

Respuesta oficial (del PDF de soluciones)

> **Framing (Entramado):** - El framing le permite al DLC (Data Link Control) receptor, determinar cuando el frame comienza y termina. - Adicionalmente busca la sincronización en caso de tiempos muertos de transmisión
>
> **Framing orientado a bits.** - En este caso se utilizará un flag para separar frames. - El flag es un conjunto de bits conocidos, y para permitir la transparencia de la información se plantea el método de bit stuffing, que es la inserción de bits. - Ejemplo: considero la flag `01111110`, esto implica que la llegada de 5 1's seguidos se debe insertar un 0.
>
> **Framing orientado a bytes.** - Bytes stuffing - Para implementar el modo transparente, se utiliza el siguiente procedimiento: se utiliza otro carácter de control DLE (Data Link Escape), de la siguiente forma. - El DLE es insertado previamente a el carácter STX o ETX - Ante la presencia en el frame de un DLE, el protocolo inserta uno más. - Garantiza transparencia de datos enviados, ya que si en los datos del frame envío DLE STX, se codificará como DLE DLE STX

Análisis de la solución oficial, punto por punto

Lo que está bien y es el corazón de la respuesta:

1. **"El framing le permite al DLC receptor determinar cuándo el frame comienza y termina."** Esa es la definición, y es exactamente el punto que abre la clase.
2. **El segundo punto es fino y suma:** "*adicionalmente busca la sincronización en caso de tiempos muertos de transmisión*". O sea: el entramado no sólo delimita, también **permite resincronizar** después de un silencio en el medio (una pausa larga, un enlace que estuvo caído y volvió). Sin flag, una reanudación del medio dejaría al receptor desalineado para siempre. **Esta es la mitad de la respuesta que la mayoría de los alumnos no da.**
3. **La transparentización del flag con bit stuffing** es la respuesta de (a).

4. **El esquema DLE con STX/ETX** es un caso particular de byte stuffing realmente usado (en SDLC, en POP3, en el protocolo de spicy de FING).

Lo que se puede mejorar — y el ejercicio para vos:

1. **La solución oficial NO justifica por qué cinco unos y no otro número.** Es lo que le falta al ítem (a). La respuesta completa es la de §1.3.3: el flag `01111110` contiene **seis** unos, y cinco es el **mayor** número de unos que garantiza que los datos rellenados no puedan contener un flag.
2. **"El DLE es insertado previamente al carácter STX o ETX"** suena a que el DLE va siempre delante de STX/ETX. Lo correcto es más general: el DLE se inserta **antes de cualquier byte de control con significado estructural** (STX, ETX, el propio DLE y, en PPP, el FLAG `0x7E`).
3. **No menciona el problema del conteo de bytes**, que es lo que da la justificación de por qué hace falta stuffing. La respuesta oficial da la solución sin dar el problema.

Mi resolución, completa

(Intro) Por qué es necesario el framing. La capa física transmite un flujo continuo de bits y **no tiene ninguna noción de mensaje**: no sabe dónde termina un paquete ni dónde empieza el siguiente. El receptor, sin embargo, necesita el mensaje **completo y alineado** para poder verificar su checksum, extraer la carga útil y entregarla a la capa de red. Si el receptor no puede ubicar los límites, cualquier paquete que le llegue junto con el siguiente estará corrupto y **no habrá forma de saberlo**. Por eso la capa 2 introduce marcas: el **entramado**. Su segundo propósito, menos obvio, es permitir la **resincronización** después de un silencio en el medio o de un error de alineación.

(a) Canal orientado a bit — HDLC: flag `01111110` + bit stuffing.

```
REGLA: el emisor inserta un 0 después de CADA cinco unos seguidos en los datos.
El flag es 01111110, que contiene SEIS unos seguidos.
∴ en los datos rellenados nunca hay seis unos
∴ 01111110 sólo puede ser un flag
∴ el receptor busca ese patrón bit a bit y siempre se engancha al correcto.
```

Ejemplo del examen, reconstruido:

```
Datos (sin rellenar):
 0 1 1 1 1 1 1 1 0 1 0 0 0 1 1 1 1 1 1 1 1 1 1 1 0
   └─ 7 unos ─┘             └──────── 11 unos ────────┘

Rellenado (se insertan 3 ceros):
 0 1 1 1 1 1 0 1 1 0 1 0 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0
       ^             ^                 ^
     0 stuffed   0 stuffed        0 stuffed

Sobre el cable:
[01111110] 01111101111001000111111011111010 [01111110]
    flag            datos rellenados               flag
```

> **Nota sobre la solución oficial:** el PDF dibuja los datos **sin rellenar** y marca arriba los tres ceros a insertar. Eso es correcto como ilustración, pero si el examen pide el resultado final hay que **escribir la secuencia rellenada completa**, que es lo que hice arriba.

(b) Canal orientado a byte — byte stuffing con DLE (o ESC).

```
REGLA: se elige un byte delimitador que no aparece en los datos.
       Si aparece en el payload, se inserta antes un byte de escape (DLE/ESC).
       Si el propio escape aparece, se duplica.

Receptor: ve ESC → descarta el ESC y toma el byte siguiente como LITERAL.
```

Ejemplo con el esquema del enunciado (STX … DLE STX … ETX):

```
Payload crudo:   A B DLE STX C ETX

Emisor:
  A       → A
  B       → B
  DLE     → DLE DLE      (se duplica)
  STX     → DLE STX      (se escapa)
  C       → C
  ETX     → DLE ETX      (se escapa)

Trama completa:
[STX] A B DLE DLE DLE STX C DLE ETX [ETX]
 ↑                                   ↑
delimit.                            delimit.

El receptor, al ver DLE, come el DLE y emite el siguiente byte como dato.
Un "DLE DLE" → un solo DLE. Un "DLE STX" → un solo STX.
```

La comparación que cierra la respuesta y es lo que el corrector busca:

	Orientado a **bit**	Orientado a **byte**
Unidad de delimitado	**1 patrón de bits** (`01111110`)	**1 byte** (`DLE` , `0x7E`)
Mecanismo	**bit stuffing**: un `0` cada 5 unos	**byte stuffing**: un byte ESC antes de cada byte peligroso
Robustez ante desalineación	**Sí**: se engancha bit a bit	**No**: si se pierde un bit, deja de ver bytes
Overhead	**bajo** (solo en las rachas largas)	**mayor** (2 bytes por byte escapado)
Cuándo se usa	medios que pueden desalinear bits (HDLC, enlaces síncronos)	medios byte-a-byte (PPP, POP3, SDLC)

> **Tope para 10 puntos:** la definición (2), la necesidad + sincronización (1), **bit stuffing con el flag `01111110` y el ejemplo** (3), **byte stuffing con el escape y el ejemplo** (3), y la distinción de cuándo se usa cada uno (1).

3.2 Examen 2010-03, Pregunta 3 (8 puntos) — PPP

> **Fuente:** Examen `eirc1003` / `sirc1003` — marzo de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1003.pdf

Enunciado (texto del examen)

> Pregunta 3 (8 puntos)
>
> a) ¿Qué es el protocolo PPP? ¿Qué es lo que lo hace más simple que otros protocolos que se ejecutan en la misma capa como por ejemplo CSMA?
>
> b) PPP utiliza flags para delimitar sus mensajes. Indique cómo es el mecanismo por el cual se logra distinguir entre el contenido de los mensajes y las flags que lo delimitan.

Respuesta oficial

> **a)** PPP (point-to-point protocol) es un protocolo de capa de enlace que se utiliza para enviar y recibir datos entre dos entidades en un enlace punto a punto. Es más simple que los protocolos de acceso al medio de la capa de enlace debido a que no se utiliza en un medio broadcast, donde muchos emisores necesitan competir a la vez por el canal, por lo tanto no se necesita un direccionamiento MAC explícito.
>
> **b)** PPP delimita las tramas usando un byte especial considerado como `<flag>` al inicio y al fin de cada trama. Si este byte aparece en algún punto del contenido del mensaje se realiza byte stuffing del mismo utilizando un byte de `<escape>`: - El emisor agrega un byte `<escape>` antes de cualquier byte `<flag>` o byte `<escape>` que esté en el contenido que debe enviar. - Si el receptor encuentra un byte `<flag>`, considera que es el inicio o el fin de una trama. Si encuentra un byte `<escape>`, lo descarta y considera el siguiente byte como parte del contenido de la trama.

Resolución

(a) Ya está perfecto arriba. Lo que hay que sumar para llegar al tope: **PPP multiplexa** varios protocolos de red (el campo *Protocol* = 2 bytes dice cuál viaja), lo que es imposible sobre un medio broadcast sin una dirección MAC. Y **PPP no implementa CSMA/CA ni CSMA/CD** porque en un enlace punto a punto la **colisión es estructuralmente imposible**: sólo hay dos estaciones y una está en silencio mientras la otra transmite.

(b) Es byte stuffing puro, idéntico a §2.1. Lo que hay que agregar para completar:

```
Números concretos de PPP:
   FLAG = 0x7E  = 01111110
   ESC  = 0x7D  = 01111101
   Escape de 0x7E  →  0x7D 0x5E
   Escape de 0x7D  →  0x7D 0x5D

Y el formato completo:
[0x7E][0xFF][0x03][Protocol:2][Payload:0..1500][FCS:2][0x7E]
0xFF = "no hay dirección" (PPP no direcciona: el enlace es punto a punto)
0x03 = "trama no numerada, sin secuencia, sin ACK"
```

Interpretación. El campo Address fijo en `0xFF` es la demostración en el formato de lo que dice el ítem (a): **no hay direccionamiento porque no hay a quién direccionar**. Sólo dos stations, y ya se sabe quién es quién.

PARTE 4 — CIERRE

📌 Resumen

1. La capa física **no sabe de mensajes**: transmite bits. Sin saber dónde acaba un paquete, el receptor no puede hacer nada con él.
2. **Entramado** = poner marcas para que el receptor ubique los límites de cada mensaje. Su segundo propósito es permitir la **resincronización**.
3. Las **cinco funciones** de la capa 2: entramado, acceso al medio, detección de errores, control de flujo y **transparencia**.
4. **Conteo de bytes**: un solo bit perdido desplaza la lectura de forma **permanente e irrecuperable**. Por eso no se usa solo.
5. **Byte stuffing**: se escapan `FLAG` y `ESC`. El receptor come el ESC y toma el siguiente byte como literal.
6. **Bit stuffing** (HDLC): un `0` después de **cinco** unos seguidos.
7. **Por qué cinco**: el flag `01111110` tiene **seis** unos. Cinco es el mayor stuffing que todavía garantiza que los datos no puedan contener un flag ⇒ el flag es único ⇒ el receptor se resincroniza solo tras cualquier error.
8. **HDLC**: tres tipos de trama (Información, Supervisión, No numerada), con *piggybacking* y ARQ de ventana deslizante.
9. **PPP**: cuatro fases (LCP → autenticación → NCP → datos). Simple porque no hay contención ni direccionamiento en un enlace punto a punto.
10. En 2026 el stuffing es **herencia histórica**; los enlaces de datos usan longitud explícita + CRC fuerte. La teoría **se sigue preguntando**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué la capa física no puede, por sí sola, entregar un mensaje?

<details>
<summary>Respuesta</summary>
</details>

2. Enumerá las cinco funciones de la capa de enlace con su propósito.

<details>
<summary>Respuesta</summary>
</details>

3. ¿Cuál es el problema fatal del conteo de bytes? Formulalo en una frase.

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué HDLC inserta un cero después de **cinco** unos seguidos y no después de seis, ni de tres?

<details>
<summary>Respuesta</summary>
</details>

5. Con HDLC, si un receptor de HDLC pierde un bit en medio de la trama, ¿se recupera? Explicá en dos pasos.

▶ Respuesta

6. Hacé el byte stuffing de `7E 41 7D 42 7E` con FLAG = 0x7E y ESC = 0x7D. Decí qué va por el cable.

▶ Respuesta

7. ¿Por qué PPP es más simple que CSMA, según la respuesta oficial?

▶ Respuesta

8. ¿Qué cuatro fases tiene PPP y para qué sirve cada una?

▶ Respuesta

🔗 Referencias

- **Examen `eirc0612` / `sirc0612`** — P2 (10 pts, framing: por qué + bit stuffing + byte stuffing) · P5 (5 pts, estación oculta — clase 43)
- **Examen `eirc1003` / `sirc1003`** — P3 (8 pts, PPP y byte stuffing)
- **Examen `eirc0812` / `sirc0812`** — P4 (7 pts, POP3 y el mecanismo DLE, que es byte stuffing con otro nombre)
- **Examen `eirc0703` / `sirc0703`** — Problema 1 (30 pts, protocolo de capa 2 con ARQ Selective Repeat y *piggybacking* — clase 40)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 3 (capas inferiores) y cap. 4 (capas de enlace)
- **Kurose & Ross**, *Computer Networking: A Top-Down Approach*, §5.1 ("The Link Layer: Five Parties") — el stuffing y HDLC
- **Práctico 5 de 2008** — `material-oficial/practicos/png/prac05_2008-p01.png`. **Ojo: no tiene ejercicio de entramado**; el Ejercicio 12 es de ARP.
- `RFC 826` — ARP (1982): por qué el entramado sigue siendo necesario
- **PPP** — `RFC 1661` (verificar) · PPPoE — `RFC 2516` (verificar)
- **IEEE 802.3** — formato de trama con longitud explícita (sin stuffing) · **IEEE 802.11** — preámbulo, FCS y tramas de gestión sin FCS
- **Clase siguiente:** 39 — Detección de errores: paridad, checksum y CRC

Clase 39

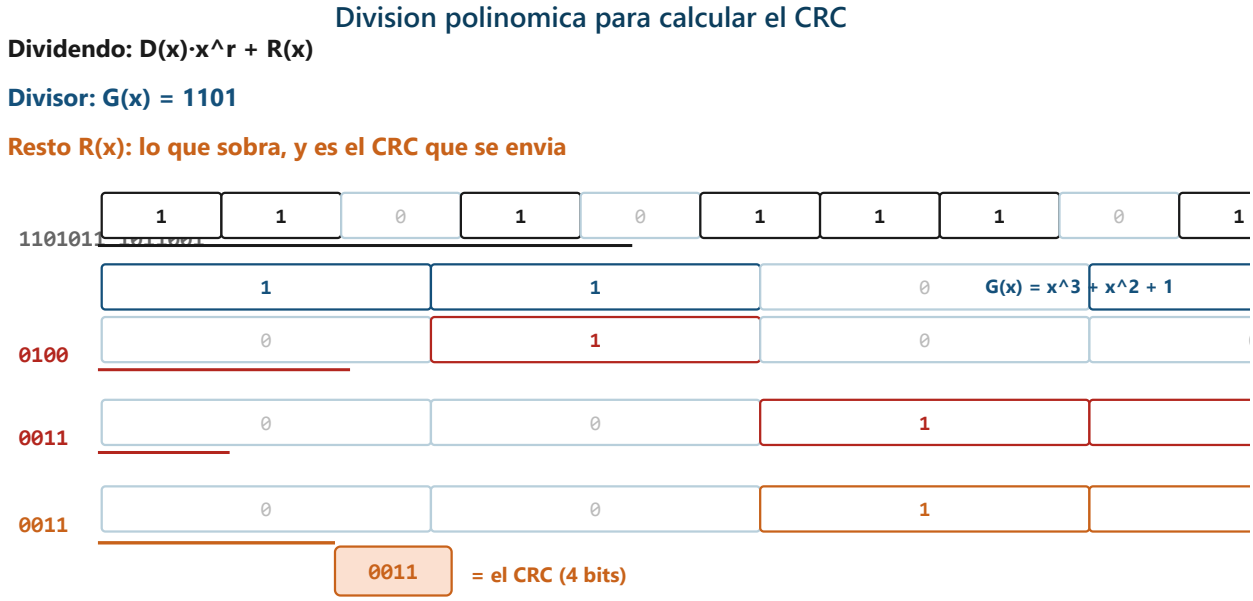
Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El modelo de error
1.2 Chequeo de paridad de una dimensión (*parity bit*)
1.3 Paridad en dos dimensiones (*two-dimensional parity*)
1.4 Internet checksum (suma de verificación de Internet)
1.5 CRC — Cyclic Redundancy Check
1.6 La tabla que resume todo
1.7 ⚠️ Actualización a 2026
2.1 Ejercicio 1 (Práctico 5) — Paridad en dos dimensiones con paridad par
2.2 Ejercicio 2 (Práctico 5) — 2D paridad: corrige 1 bit, detecta 2
2.3 Ejercicio 3 (Práctico 5) — Internet checksum de los bytes 0 a 9
2.4 Ejercicio 4 (Práctico 5) — CRC con `G = 1001` y `D = 10101010`
2.5 Verificación del Ejercicio 4 con el método de XOR
2.6 Ejercicio 5 (inventado) — Paridad par e impar sobre el mismo mensaje
2.7 Ejercicio 6 (inventado) — Paridad 2D: cuatro escenarios con la misma matriz
2.8 Ejercicio 7 (inventado) — Internet checksum con acarreo circular
2.9 Ejercicio 8 (inventado) — CRC por división polinómica: `G = 1101`, `D = 100100`
2.10 Ejercicio 9 (inventado) — El mismo cálculo con la tabla de XOR
3.1 ⭐ Examen 2010-02, Pregunta 3 (8 puntos) — paridad en dos dimensiones
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Bloque: E (Capa de Enlace) · **Semana:** 13 · **Duración estimada:** 120 min **Correlativa:** Clase 38 (servicios y entramado de la capa de enlace) · **Sigue con:** Clase 40 (corrección de errores y ARQ) **Material de referencia:** Tanenbaum cap. 4 · Kurose §5.5 · Práctico 5 de 2008 (ejercicios 1 a 4) · RFC 1071 · RFC 1624 · IEEE 802.3 FCS

🎯 Al terminar esta clase vas a poder

1. Explicar la diferencia entre **detectar** un error y **corregirlo**, y por qué un detector barato puede ser mejor que un corrector caro.
2. Calcular el **bit de paridad par** e **impar** de un mensaje, y **demostrar por qué la paridad detecta todos los errores de un número impar de bits y ninguno de un número par**.
3. Construir una **paridad en dos dimensiones** completa (matriz + paridades de fila + paridades de columna), **detectar y corregir un error de un bit**, y **demostrar por qué no puede corregir dos**.
4. Calcular el **Internet checksum** de 16 bits paso a paso, y explicar por qué **no detecta el intercambio de dos words**.
5. Calcular un **CRC** con **división polinómica** término a término en GF(2), y **verificarlo** con el método de **registro de desplazamiento con XOR** que usa el hardware.
6. Enunciar la **propiedad clave del CRC**: qué garantiza un generador de grado `r`, y por qué **el CRC detecta pero no corrige**.
7. Resolver completo el **Ejamen 2010-02 Pregunta 3** (8 puntos).



En GF(2) sumar es XOR y multiplicar es AND. Por eso la resta de la division es un XOR.

Se divide D(x)·x^r entre G(x) y se manda el resto pegado al mensaje original.

> **Prerequisito del examen:** `2010-02 P3` (8 pts — paridad en dos dimensiones: funcionamiento y cuántos bits corrige) · `2007-07 P2` (5 pts — nombre los mecanismos de detección y corrección de la capa de enlace y describa uno) · `2008-12 P4` (mismo tipo de comprobación, aplicado a POP3).
>
> ⚠️ Esta es **una clase de cálculo**. No hay nada que memorizar y sí mucho que ejecutar. Hacé los ejercicios con lápiz y papel antes de mirar la resolución.

PARTE 1 — TEORÍA

1.1 El modelo de error

El medio físico es **inexacto**: ruido, interferencia o crosstalk pueden hacer que un bit que salió como `1` llegue como `0`. Eso es un **error de bit** (*bit flip*).

```
Lo que se mandó:      1 0 1 1 0 0 1 0
Lo que llegó:        1 0 1 0 0 0 1 0
                          ↑ volteado
```

Sin protección, esa diferencia llega hasta la aplicación. **Detectar** errores es que el receptor **se dé cuenta** y descarte la trama. **Corregir** es que sepa **cuál era el bit original** y lo repare.

La diferencia es enorme en costo: un detector es barato (un registro, unos XOR y una división por un polinomio fijo); un corrector necesita **mucha** más redundancia, porque tiene que distinguir entre muchos patrones de error posibles.

La definición que ordena toda la tabla de §1.6:

> Un mecanismo detecta todos los errores de **hasta `d − 1` bits** y corrige todos los de **hasta `⌊(d − 1)/2⌋` bits**, donde `d` es la **distancia de Hamming mínima** del código: el mínimo número de bits en los que hay que cambiar cualquier par de mensajes válidos para que uno se convierta en el otro.

1.2 Chequeo de paridad de una dimensión (*parity bit*)

Un solo bit calculado sobre el mensaje, para que el número total de unos sea siempre el mismo.

```
Mensaje  D = 1 0 1 1 0 1 0        (7 bits, 4 unos)

Paridad PAR:   el total debe quedar PAR    → bit = 0   (4+0 = 4, par)
Paridad IMPAR: el total debe quedar IMPAR  → bit = 1   (4+1 = 5, impar)

Recepción: sumo los 8 bits (incluido el de paridad) y verifico que el total
           tenga la paridad esperada. Si no coincide → hubo error, descarto.
```

La limitación fundamental — y la demostración que hay que saber

> **La paridad detecta TODOS los errores de un número IMPAR de bits y NINGUNO de un número PAR de bits.**

Errores de número impar. Voltear `k` bits con `k` impar cambia el número de unos en `+k`, `+k−2`, …, `−k`: todos valores **impares**. La paridad del total se invierte, así que **no puede coincidir** con la guardada. Se detecta. ✓

Errores de número par. Voltear dos bits `i` y `j`:

```
los dos 1 → 0   :  los unos BAJAN en 2   → el total sigue siendo PAR
los dos 0 → 1   :  los unos SUBEN en 2   → el total sigue siendo PAR
uno a 1, otro a 0 : los unos NO cambian → el total no cambia
```

En los tres casos **la paridad del total no cambia**, así que el bit de paridad sigue cuadrando. El receptor **no se entera de nada**. ✗

```
Ejemplo:  D = 1 0 1 1 0 1 0 , paridad par = 0
Se voltean los bits 1 y 5 (ambos 1 → 0):
Se envió:  1 0 1 1 0 1 0 0        (6 unos = par ✓)
Se recibe: 0 0 1 1 0 1 0 0        (4 unos = par ✓)
→ NO DETECTADO. El receptor entrega un mensaje que nunca se mandó.
```

El costo. Un solo bit de paridad protege **una palabra** y agrega 1 bit: 12,5 % de overhead en un byte, 1 % en una trama de 100 bytes. Por eso **nadie usa un solo bit de paridad en un protocolo real**: el riesgo de que un error de dos bits pase inadvertido es demasiado alto. La solución es **múltiples bits de paridad**, y de ahí salen la paridad 2D y el código de Hamming (clase 40).

1.3 Paridad en dos dimensiones (*two-dimensional parity*)

La idea es repetir la paridad **en dos direcciones**.

```
Se ve el flujo como una MATRIZ:
 - cada palabra va en una FILA,
 - palabras consecutivas en filas consecutivas,
 - el bit j-ésimo de cada palabra forma la COLUMNA j-ésima.

Se agrega:  un bit de paridad al final de CADA FILA,
            un bit de paridad al final de CADA COLUMNA.
```

```
      c1 c2 c3 c4 | ... cn | Rp
┌──┬──────────────┬────────┬──┐
│f1│  n bits      │  ...   │  │  ← 1 bit de paridad por fila
├──┼──────────────┼────────┼──┤
│fm│  m × n       │        │  │
├──┼──────────────┼────────┼──┤
│Cp│  n bits      │        │  │  Cp = paridades de columna (última fila)
└──┴──────────────┴────────┴──┘

Campo de comprobación = (Rp(1)..Rp(m)) ‖ (Cp(1)..Cp(n))  =  m + n bits
```

Las dos propiedades que hacen que funcione:

1. **Detecta y corrige 1 error de bit.** Si falla la paridad de la fila `i` **y** la de la columna `j`, el error está **exactamente en `(i, j)`** — y se corrige, porque ya se sabe cuál es.
2. **Detecta los errores de 2 bits**, pero **no los puede corregir** (ver §2.2).

El detalle elegante de la esquina: no hace falta un bit de esquina, porque por construcción la paridad de la suma de las Rp y la de la suma de las Cp son **iguales** (las dos son la paridad del número total de unos de la matriz). El campo mide `m + n` bits, no `m + n + 1`.

1.4 Internet checksum (suma de verificación de Internet)

Es el mecanismo de **IP, UDP y TCP**: un checksum de **16 bits** con **aritmética de complemento a uno** (*one's complement*) y acarreo circular.

Procedimiento, paso a paso (el que hay que saber de memoria):

```
1. PARTIR el mensaje en WORDS DE 16 BITS.
   Si el número de bytes es impar, se agrega un byte de relleno con 0.

2. SUMAR todas las words como enteros de 16 bits, en complemento a uno:
   suma normal, y cada vez que el resultado pasa de 65535 se le vuelve
   a sumar 1   →  "acarreo circular" o "end-around carry".

3. COMPLEMENTAR el resultado de 16 bits (invertir cada bit).

4. TRANSMITIR el mensaje seguido del checksum.

VERIFICACIÓN en el receptor: se suma todo (incluido el checksum) con la misma
aritmética. Si no hubo error, el resultado debe ser  0xFFFF.
```

Por qué detecta los errores — y el punto fino que suma puntos. El checksum es un **código lineal** sobre `GF(2)`. De ahí salen tres propiedades:

1. **Detecta el 100 % de los errores que están dentro de una sola word de 16 bits**, sea cual sea el patrón. Porque el valor de una word módulo $2^{16} - 1$ es **inyectivo**: dos words distintas nunca dan el mismo residuo. Si una word cambia, la suma cambia.
2. **Detecta todos los errores de un bit** (caso particular de lo anterior).
3. **NO detecta el intercambio de dos words de 16 bits**, porque **sumar es conmutativo**: $w_1 + w_2 = w_2 + w_1$. Si dos words se intercambian entre sí, la suma es **exactamente la misma**, el checksum es el mismo y el receptor acepta la trama.

Y el mismo argumento de la paridad, generalizado:

```
1 bit alterado, en una word   : la suma cambia ±2^k        (k=0..15) → ≠ 0 → DETECTADO
2 bits en la MISMA word       : cambia ±2^i ± 2^j (i≠j)               → ≠ 0 → DETECTADO
bit k en una word y bit k en otra, en sentidos opuestos:
                                +2^k y −2^k  → suma 0                 → NO DETECTADO
intercambio de 2 words        : la suma es idéntica                   → NO DETECTADO
```

Dónde vive esto. IP **no** tiene checksum (era demasiado caro recalcularlo en cada router); lo tienen **UDP y TCP**, porque corren en los extremos, donde unos pocos ciclos de CPU no importan y donde el datagrama puede haber pasado por veinte saltos. `RFC 1071` es la especificación del algoritmo y `RFC 1624` documenta por qué **cambiarlo rompe la compatibilidad** — un detalle que se pregunta.

1.5 CRC — Cyclic Redundancy Check

Esto es lo más importante de la clase. Todo lo anterior sirve para entender el problema; el CRC es la solución que se usa en todas partes.

1.5.1 La idea: polinomios y el campo GF(2)

La idea es representar el bloque de bits como un **polinomio con coeficientes binarios**. Si el bloque es `1 0 1 1 0`, el polinomio es:

```
1 0 1 1 0 → x⁴ + x² + x + x⁰
```

Porque el primer bit (el más a la izquierda) es el coeficiente de `x^(n-1)`.

Toda la aritmética se hace en el campo de Galois `GF(2)`, y `GF(2)` tiene **una sola diferencia con la aritmética normal**:

```
┌──────────────────────────────────────────────┐
│ + es XOR (sin acarreo, 1+1 = 0)              │
│ × es AND (0·x = 0, 1·x = x)                  │
└──────────────────────────────────────────────┘

Ejemplos:
1 + 1 = 0
101 + 110 = 011 (XOR)
101 × 110 = 100 (AND)
```

Por qué polinomios y no "dividir el número entero": porque `GF(2)` no tiene división entera con resto. Los polinomios sobre `GF(2)` **sí** se pueden dividir, con resto, siempre. Esa es la razón matemática de que el CRC se defina como una división polinómica.

1.5.2 La división polinómica larga

El procedimiento completo, en palabras:

```
1. Sea G el polinomio generador, de grado r.
2. Sea D el bloque de datos de N bits. Se representa como polinomio.
3. Se agregan r CEROS a la derecha:  T = D · x^r
   (en binario: se desplazan D r posiciones hacia la izquierda).
4. Se divide T por G con "división larga", donde cada resta es un XOR.
5. El resto R (grado < r) es el CRC, y se transmite pegado a D.
```

El paso 4 en detalle — esto es lo que hay que escribir en el examen:

```
While el polinomio de trabajo T sea de grado >= grado de G:
    • Ubicar el término de MAYOR grado de T.
    • Poner G sobre él, desplazado para que coincidan las potencias mayores.
    • REEMPLAZAR esa porción de T por el XOR de ambas (o sea, "restar").
Cuando deg(T) < deg(G), lo que queda de T es el resto R.
```

Ejemplo completo, con `G = 1001 = x³+x+1` (grado 3) y `D = 10101010`. El desarrollo línea por línea, con los seis XOR, está en **§2.4**. Acá va la estructura, que es lo que hay que entender:

```
D · x³ = 10101010 000 = 10101010000        (grado 10)

x¹⁰ → ⊕ G en x¹⁰  →  00111010000
x⁸  → ⊕ G en x⁸   →  00011110000
x⁷  → ⊕ G en x⁷   →  00001100000
x⁶  → ⊕ G en x⁶   →  00000101000
x⁵  → ⊕ G en x⁵   →  00000000110
x³  → ⊕ G en x³   →  00000000101   ← grado 2 < 3, FIN

R = 101 = 5 = 0x5
```

Comprobación (multiplicación de vuelta en `GF(2)`):

```
T ⊕ R = 10101010101 ⊕ 00000000101 = 10101010000 = D · x³  ✓
Y 10101010101 = 1001 × 10111101   →  divisible, resto 0  ✓
```

Lo que se transmite es `D || R`:

```
10101010 101
└──D────┘ └R┘   8 + 3 = 11 bits para 8 de información
overhead = 3 bits = el grado de G
```

1.5.3 El cálculo con XOR: cómo lo hace realmente el hardware

Nadie implementa una división polinómica con restas. El hardware real usa un **registro de desplazamiento de `r` bits** y una sola operación de XOR:

```
R = 0 (r bits)
for cada bit b del mensaje D, del más significativo al menos:
fb = (bit más significativo de R) XOR b
R = (R << 1), quedándose con r bits
si fb = 1: R = R XOR (G módulo 2^r)
Al final, R es el CRC.
```

Por qué es equivalente a la división polinómica: cada iteración hace exactamente **una** de las "restas" (XOR) de la división larga, y lo hace en el momento en que el bit que sale por la izquierda del registro corresponde al término mayor. Al final, en el registro queda lo mismo que en el polinomio de trabajo: el resto. La ventaja es que **es un circuito de `r` flip-flops, `r` puertas XOR y un cable de realimentación** — o sea, unos pocos nanoseconds por byte.

La tabla completa del ejemplo anterior (`G = 1001`, `r = 3`, `D = 1 0 1 0 1 0 1 0`) está en **§2.5** y da `R = 101`, el mismo resultado que la división polinómica. Eso es exactamente lo que hay que demostrar cuando alguien dice "el XOR de hardware es equivalente a la división", y podés decirlo en el examen poniendo los dos resultados de lado.

1.5.4 Los polinomios generadores estándar, y por qué hay tantos

Nombre	Polinomio	Dónde se usa
CRC-32 (IEEE 802.3)	0x04C11DB7 grado 32	Ethernet (IEEE 802.3), ZIP, PNG, gzip, HTTP
CRC-32C (Castagnoli)	0x1EDC6F41 grado 32	SCTP, iSCSI, ext4, Ceph
CRC-16/CCITT	0x1021 = $x^{16}+x^{12}+x^5+1$	HDLC (FCS), X.25, MPEG-2, Bluetooth
CRC-8	0x07 = x^8+x^2+x+1	ATM (celdas de 53 bytes)

En el hardware se usan las versiones "reflejadas" (*reflected*): el registro desplaza a la derecha en vez de a la izquierda, y los polinomios aparecen invertidos byte a byte. Por eso el CRC-32 se escribe `0xEDB88320` en el software y `0x04C11DB7` en el estándar, y el CRC-32C aparece como `0x82F63B78`.

¿Por qué hacen falta polinomios distintos? Por cuatro razones, todas de optimización:

1. **Detectar todos los errores de un bit** ⇒ el término independiente tiene que ser **1** (el polinomio no puede ser divisible por `x`).
2. **Detectar ráfagas cortas** ⇒ el polinomio debe tener **muchos términos no nulos** y ser irreducible o primitivo, para que la división produzca muchos restos distintos.
3. **Detectar el error extremo a extremo además del de línea**: en una llamada telefónica, dos FCS de línea iguales pueden **compensarse** y anular un error; un polinomio distinto por capa lo evita.
4. **Ser barato en hardware** ⇒ se elige el grado que da el overhead justo (32 bits = 4 bytes, comparable de un vistazo).

Lo que se escribe en el examen es la idea: *el mismo problema, optimizado para contextos distintos, tiene soluciones distintas; por eso hay varios polinomios estandarizados y no uno solo.*

1.5.5 La propiedad clave del CRC — y el punto conceptual que se pregunta

Acá está todo. Sea `G` un polinomio de grado `r`.

> **Un patrón de error `E` NO es detectado si y sólo si `G` divide a `E`.** En otras palabras: los patrones no detectados son exactamente **los múltiplos de `G`**.

De ahí salen las tres garantías:

(a) Se detecta el 100 % de los errores de UN bit.

```
Un error de un solo bit en la posición k es el polinomio E = x^k.
Para que fuera undetected, haría falta G | x^k.
Pero x^k tiene grado k, y el coeficiente de x^k en G... un polinomio de
grado r ≥ 1 no divide a un monomio, porque x^k tiene un solo término.
∴ E ≠ 0 nunca es undetected.
```

(b) Se detectan TODOS los errores por ráfaga (*burst error*) de longitud ≤ r.

```
Una ráfaga de longitud L es un error cuyos bits están todos en una ventana
de L posiciones consecutivas: E = x^s · (polinomio de grado ≤ L−1)
Si L ≤ r, entonces grado(E) ≤ s + r − 1 ... y el factor de la derecha tiene
grado < r = grado(G), así que NO puede ser múltiplo de G.
∴ undetected imposible. ✓
```

Ésta es la razón por la que el CRC anduvo en enlaces de datos reales durante décadas: los errores de línea **no son aleatorios**, son **ráfagas** (un transitorio electromagnético voltea un grupo de bits juntos), y el CRC los detecta todos mientras sean cortos.

(c) Para un error de `b` bits, la fórmula y su lectura correcta.

```
Los undetected son los múltiplos de G: E = G · Q.
En una ventana de b posiciones, los múltiplos de G son los G·Q con
deg(Q) ≤ b−1−r, o sea 2^(b−r) cocientes distintos.

De ahí sale la expresión que circula en los apuntes:

P(detectar) = 1 − 2^(−(n − b))

donde n = grado del polinomio generador y b = número de bits del error.

⚠️ CÓMO HAY QUE USARLA: la expresión da sentido numérico sólo mientras
b sea mucho menor que n. Interpretada así:

b = 1 → P ≈ 1 − 2^(−n) ≈ 1 (seguro)
b = n/2 → P ≈ 1 − 2^(−n/2) ≈ 1 (muy seguro)
b ≈ n → la protección se agota

⚠️ La versión RIGUROSA — y la que hay que saber argumentar — son (a), (b) y
la distancia mínima:

d = r + 1 (para los generadores bien elegidos, sin múltiplos
de peso bajo)

∴ se detectan TODOS los errores de peso ≤ r = d − 1.

Los errores de peso mayor se detectan con probabilidad muy alta, pero no
con certeza: existen C(n, b) patrones posibles y sólo 2^(b−r) de ellos son
múltiplos de G.
```

Consecuencia práctica, que es lo que se lleva uno al examen:

```
CRC-32, r = 32:
error de 1 bit → detectado SIEMPRE
ráfaga de ≤ 32 bits → detectada SIEMPRE
error de 32 bits → probabilidad de detección ≈ 1 − 2^(−32)
error de 64 bits → todavía altísima
(y en la práctica, más de 10^11 bits transferidos con undetected = 0)
```

1.5.6 ⚠️ El CRC detecta, NO corrige

Esto se pregunta, y la respuesta tiene dos partes:

1. Teóricamente, un CRC con `d = r + 1` podría corregir `⌊r/2⌋` errores. Con `d = 33`, serían 16. ¿Por qué no se hace?

- El **síndrome** (el resto de la división) tiene sólo `r = 32` bits. El número de patrones de error de **un** bit es `N`, mucho mayor que `2^32`. Con una tabla de log se puede localizar un error de un solo bit (`G·x^i` módulo `x^r` da la tabla), pero para 16 errores hay `binom(N,16)` patrones y **no hay forma práctica de distinguirlos** con 32 bits de información.
- El hardware real de CRC hace una sola cosa: comparar el resto con cero.

2. Para corregir de verdad se usan otros códigos: Hamming y SECDED (clase 40, para 1 y 2 bits) y los **códigos correctores de Reed–Solomon** o **LDPC**, que agregan mucha más redundancia y son los que se usan en discos, DVDs, satelite, DVB-T y 5G.

> **La frase para el examen:** *el CRC es un mecanismo de detección de alta probabilidad con overhead mínimo. La corrección exige redundancia considerable y algoritmos de decodificación distintos; por eso nunca se usa el CRC como corrector.*

1.6 La tabla que resume todo

Mecanismo	Redundancia	Detecta	Corrige	Dónde
Paridad 1D	1 bit	**errores de peso impar**; falla con peso par	nada	xo, casi nunca se usa solo
Paridad 2D	`m + n` bits	**todos** los de 1 y 2 bits; **no** todos los de 4	**1 bit**	práctica, teaching
Internet checksum	16 bits	todos los de 1 bit; **casi todos** los múltiples; **NO** word swaps	nada	IP(no), **UDP, TCP**
Hamming SEC	`r` bits, `2^r ≥ m+1`	hasta `d−1`	**1 bit**	clase 40
Hamming SECDED	`r + 1`	2 bits	**2 bits**	clase 40
CRC-r	`r` bits	**1 bit siempre**, ráfagas ≤ `r` siempre, más	**nada**	**Ethernet, ZIP, PNG, iSCSI, SCTP**
Reed–Solomon	alto	ráfagas largas	**varios bits**	DVD, DVB-T, 5G, RAID 6

1.7 ⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **Paridad**: par e impar, y la propiedad de detectar todo error de **número impar** de bits y ninguno de **número par**. - **Internet checksum**: 16 bits, complemento a uno, con acarreo circular. Vive en **UDP y TCP**. Lo que **NO** detecta: el **intercambio de dos words**. - **CRC**: división polinómica en `GF(2)` con `+` = XOR, `r` ceros agregados, y la propiedad "detecta siempre un error de 1 bit y toda ráfaga de hasta `r` bits". - **CRC-32 =** `0x04C11DB7` para Ethernet, ZIP y PNG. **CRC detecta, no corrige.**

Cambió o se afinó: - **El campo de aplicación se movió, no el mecanismo.** `CRC-32C` (Castagnoli) es el de **SCTP, iSCSI, ext4 y Ceph**; `CRC-32` es el de **Ethernet, ZIP y PNG**. Conviven por las cuatro razones de §1.5.4. - **El CRC dejó de ser un cálculo**: en 1 a 400 Gbit/s se implementa como **circuito combinacional** (tabla de 256 entradas de 32 bits, o una matriz lineal de XOR y desplazamientos). El resultado es el mismo. - **El checksum de IPv4 no existe y no volvió**: los routers lo descartan para ganar velocidad. `RFC 1624` documenta que **no se**

puede cambiar el algoritmo del checksum de TCP/UDP sin romper compatibilidad. - ⚠️ **Tema en desuso:** la **paridad 2D como mecanismo de producción**. Se enseña y se pregunta, pero no aparece en ningún protocolo comercial: todo lo real usa CRC o **Reed–Solomon**. - Lo que **sí** se usa para **corregir** de verdad: **Hamming** y **SECDED** (clase 40), y **Reed–Solomon** o **LDPC** en discos, DVB-T, 5G y RAID 6.

Consecuencia para el examen: la pregunta `2010-02 P3` **se sigue preguntando tal cual**. Lo que suma es el criterio de **distancia mínima**, que es lo que distingue a quien entendió de quien memorizó.

PARTE 2 — PRÁCTICA RESUELTA

> Los **ejercicios 1 a 4** son del **Práctico 5 de 2008** (`material-oficial/practicos/png/prac05_2008-p01.png`), extraídos del libro de Kurose & Ross. Los **ejercicios 5 a 11** son *inventados y resueltos*.

2.1 Ejercicio 1 (Práctico 5) — Paridad en dos dimensiones con paridad par

Enunciado. Supongamos que el contenido de un paquete es el patrón de bits `1010101010101011`, y que se utiliza un esquema de **paridad par**. ¿Cuál sería el valor del campo de comprobación para el caso de un esquema de paridad de **dos dimensiones**? La respuesta debe usar un campo de comprobación de **longitud mínima**.

Paso 1 — Dimensionar la matriz

Son 16 bits de datos. Con una matriz de `m` filas de `n` columnas hay `m · n` posiciones de datos, y el campo de comprobación mide `m + n` bits (un bit de paridad por fila + uno por columna; la esquina no aporta nada, §1.3).

Buscamos el mínimo de `m + n` con `m · n ≥ 16`:

```
m  n  m·n  m+n
─  ─  ───  ───
4  4  16   8  ← mínimo
5  4  20   9
3  6  18   9
2  8  16   10
1  16 16   17
8  2  16   10
```

Matriz de 4 × 4. la convención de la cátedra ("el bit j-ésimo de cada palabra forma la columna j-ésima"), el patrón se parte en cuatro palabras de cuatro bits:

```
f1 = 1010
f2 = 1010
f3 = 1010
f4 = 1011
```

Paso 2 — Paridad de cada FILA

Paridad par ⇒ el bit de paridad vale **(número de unos) mod 2**.

```
f1 = 1 0 1 0 → 2 unos → 2 mod 2 = 0
f2 = 1 0 1 0 → 2 unos → 0
f3 = 1 0 1 0 → 2 unos → 0
f4 = 1 0 1 1 → 3 unos → 3 mod 2 = 1
Rp = 0 0 0 1
```

Paso 3 — Paridad de cada COLUMNA

```
c1 (bit 1 de cada palabra): 1,1,1,1 → 4 unos → 0
c2 (bit 2): 0,0,0,0 → 0 unos → 0
c3 (bit 3): 1,1,1,1 → 4 unos → 0
c4 (bit 4): 0,0,0,1 → 1 uno → 1
Cp = 0 0 0 1
```

Respuesta

```
c1 c2 c3 c4 | Rp
┌──┬─────────┬──┐
│f1│ 1 0 1 0 │ 0 │
│f2│ 1 0 1 0 │ 0 │
│f3│ 1 0 1 0 │ 0 │
│f4│ 1 0 1 1 │ 1 │
├──┼─────────┼──┤
│Cp│ 0 0 0 1 │ │
└──┴─────────┴──┘

CAMPO DE COMPROBACIÓN = Rp || Cp = 0 0 0 1 0 0 0 1 = 8 bits

Frame total = 16 bits de datos + 8 de comprobación = 24 bits.
```

Verificación de la esquina (la propiedad de §1.3): la paridad de la suma de las paridades de fila es `(0+0+0+1) mod 2 = 1`; la de la suma de las de columna es `(0+0+0+1) mod 2 = 1`. **Coinciden**, como debe ser.

2.2 Ejercicio 2 (Práctico 5) — 2D paridad: corrige 1 bit, detecta 2

Enunciado. Mostrar, con un ejemplo **distinto al del libro**, que la paridad de dos dimensiones puede **corregir y detectar un error en un bit único**. Mostrar, con otro ejemplo, que un error en **dos** bits puede ser **detectado pero no corregido**.

(a) Corrección de un error de un bit

Datos (paridad par), matriz de 4 × 4:

```
      c1 c2 c3 c4 | Rp
f1    1  0  1  1 |  1     ← 3 unos
f2    0  1  1  0 |  0     ← 2 unos
f3    1  1  0  0 |  0     ← 2 unos
f4    0  0  1  0 |  1     ← 1 uno
─────────────────┼──
Cp    0  0  1  1 |         ← c1:2→0  c2:2→0  c3:3→1  c4:1→1
```

Se voltea el bit de la fila 3, columna 3 (de `0` a `1`). El receptor recalcula:

```
FILAS
 f1 = 1 0 1 1 → 3 unos → Rp debe ser 1. Recibido 1. ✓
 f2 = 0 1 1 0 → 2 unos → Rp debe ser 0. Recibido 0. ✓
 f3 = 1 1 1 0 → 3 unos → Rp debe ser 1. Recibido 0. ✗  ← FALLA
 f4 = 0 0 1 0 → 1 uno  → Rp debe ser 1. Recibido 1. ✓

COLUMNAS
 c1 = 1,0,1,0 → 2 unos → Cp debe ser 0. Recibido 0. ✓
 c2 = 0,1,1,0 → 2 unos → Cp debe ser 0. Recibido 0. ✓
 c3 = 1,1,1,1 → 4 unos → Cp debe ser 0. Recibido 1. ✗  ← FALLA
 c4 = 1,0,0,0 → 1 uno  → Cp debe ser 1. Recibido 1. ✓
```

```
Fila que falla = 3 ; Columna que falla = 3
∴ el bit equivocado es el de la intersección (3,3)
∴ CORRIGE: se lo invierte y el mensaje queda intacto.  ✓
```

Interpretación. Ese es el corazón de la técnica: la fila te dice **en qué franja** y la columna te dice **en qué posición dentro de la franja**. Con esos dos datos la posición del error queda **completamente determinada**: no hay ambigüedad posible.

(b) Dos errores: detecta, pero no corrige

Con la misma matriz, se voltean los bits **(1,1)** y **(2,2)**:

```
FILAS
 f1 = 0 0 1 1 → 2 unos → Rp debe ser 0. Recibido 1. ✗  ← FALLA
 f2 = 0 0 1 0 → 1 uno  → Rp debe ser 1. Recibido 0. ✗  ← FALLA
 f3 = 1 1 0 0 → 2 unos → 0. Recibido 0. ✓
 f4 = 0 0 1 0 → 1 uno  → 1. Recibido 1. ✓

COLUMNAS
 c1 = 0,0,1,0 → 1 uno  → Cp debe ser 1. Recibido 0. ✗  ← FALLA
 c2 = 0,0,1,0 → 1 uno  → Cp debe ser 1. Recibido 0. ✗  ← FALLA
 c3 = 1,1,0,1 → 3 unos → 1. Recibido 1. ✓
 c4 = 1,0,0,0 → 1 uno  → 1. Recibido 1. ✓
```

```
Filas que fallan    : 1 y 2
Columnas que fallan: 1 y 2

Posiciones candidatas = las 4 intersecciones:
      (1,1)  (1,2)
      (2,1)  (2,2)
y SOLO dos están mal.  ¿Cuáles?  ¡INDETERMINADO!

∴ DETECTA (hay incoherencia) pero NO PUEDE CORREGIR.
```

Por qué no puede: la información no alcanza. El receptor tiene cuatro observaciones ("fallan estas dos filas y estas dos columnas") y hay **cuatro** combinaciones distintas de dos bits que las producen exactamente igual: `(1,1)+(2,2)`, `(1,1)+(2,1)`, `(1,2)+(2,1)` y `(1,2)+(2,2)`. **Sin redundancia extra no hay forma de desempatar.**

(c) Y el caso peor: un error de CUATRO bits que **no se detecta**

Volteamos un **rectángulo 2 × 2**: (1,1), (1,2), (2,1) y (2,2).

```
FILAS
 f1 = 0 1 1 1 → 3 unos → Rp = 1. Recibido 1. ✓   (original: 1 0 1 1 = 3 unos)
 f2 = 1 0 1 0 → 2 unos → Rp = 0. Recibido 0. ✓   (original: 0 1 1 0 = 2 unos)
 f3 = 1 1 0 0 → 2 unos → 0. Recibido 0. ✓   (sin cambios)
 f4 = 0 0 1 0 → 1 uno  → 1. Recibido 1. ✓   (sin cambios)

COLUMNAS
 c1 = 0,1,1,0 → 2 unos → 0. Recibido 0. ✓   (original: 1,0,1,0 = 2 unos)
 c2 = 1,0,1,0 → 2 unos → 0. Recibido 0. ✓   (original: 0,1,1,0 = 2 unos)
 c3 = 1,1,0,1 → 3 unos → 1. Recibido 1. ✓   (sin cambios)
 c4 = 1,0,0,0 → 1 uno  → 1. Recibido 1. ✓   (sin cambios)
```

Resultado: CERO verificaciones fallan. El receptor acepta una trama con **cuatro** bits equivocados y no se entera de nada. Ése es el tope de la paridad 2D, y por eso no se usa en producción.

La razón, en una frase: cada fila y cada columna afectada recibe **dos** cambios de signo opuesto, que se **cancelan** en el conteo de unos. Exactamente el mismo argumento de la paridad 1D, pero aplicado a las dos dimensiones a la vez.

⚠️ Errores típicos

- **Calcular la paridad de columna leyendo las filas.** La columna `j` es el bit `j` de **cada** fila, no el último bit de cada fila.
- **Olvidar que la paridad es la del número de unos**, no "la del último bit".
- **Cerrar en 8 bits el campo de comprobación cuando `m·n > 16`.** Si la matriz no es cuadrada, el campo mide `m + n`, y hay que elegir la que minimiza.
- **Confundir "detectar 2 errores" con "corregir 2 errores".** Con dos filas y dos columnas fallando hay cuatro candidatos y sólo dos están mal.

2.3 Ejercicio 3 (Práctico 5) — Internet checksum de los bytes 0 a 9

Enunciado. Supóngase que la porción de información de un paquete contiene diez bytes, que suponen la representación binaria sin signo de ocho bits de los enteros del 0 al 9. Calcular la suma de comprobación (*checksum*) Internet para estos datos.

Paso 1 — Los diez bytes en binario y en hexadecimal.

```
decimal binario hex
_______ ________ ____
0 00000000 0x00
1 00000001 0x01
2 00000010 0x02
3 00000011 0x03
4 00000100 0x04
5 00000101 0x05
6 00000110 0x06
7 00000111 0x07
8 00001000 0x08
9 00001001 0x09
```

Paso 2 — Emparejarlos en words de 16 bits (un byte par, un byte impar).

```
byte: 0 1 | 2 3 | 4 5 | 6 7 | 8 9
↓ ↓ | ↓ ↓ | ↓ ↓ | ↓ ↓ | ↓ ↓
0x0001 | 0x0203 | 0x0405 | 0x0607 | 0x0809

w1 = 0000 0000 0000 0001 = 0x0001 = 1
w2 = 0000 0010 0000 0011 = 0x0203 = 515
w3 = 0000 0100 0000 0101 = 0x0405 = 1029
w4 = 0000 0110 0000 0111 = 0x0607 = 1543
w5 = 0000 1000 0000 1001 = 0x0809 = 2057
```

Paso 3 — Sumar. Ojo: **10 bytes son 5 words exactas, así que no hace falta byte de relleno** (el relleno con 0 sólo hace falta cuando el número de bytes es impar).

```
S = 1 + 515 + 1029 + 1543 + 2057

1 + 515 = 516
516 + 1029 = 1545
1545 + 1543 = 3088
3088 + 2057 = 5145

5145 < 65535 → NO hay acarreo que devolver. S = 5145
```

Paso 4 — Complementar los 16 bits.

```
5145 en binario: 0001 0100 0001 1001
| | | |
Su complemento: 1110 1011 1110 0110
= 0xEBE6

Decimales: 65535 − 5145 = 60390 = 0xEBE6
```

Respuesta

```
Internet checksum = 0xEBE6 = 60390 (16 bits)

Trama completa: 00 01 02 03 04 05 06 07 08 09 | EB E6
```

Interpretación. Con **diez bytes** el checksum es de **2 bytes**. La proporción de protección es del 20 %: muchísima redundancia para tan pocos datos, que es exactamente lo que pasa con los mensajes cortos. Los checksums tienen sentido en los encabezados de **TCP/IP** (decenas de bytes), no en un payload chico.

⚠️ Errores típicos

- **Sumar byte a byte y no word a word.** Hay que agrupar de a 16 bits, y el **primer byte va en el bit más alto** de la word.
- **Olvidar el acarreo circular.** Acá no hubo, pero en el ejercicio 2.8 sí.
- **Complementar dos veces** (una al sumar, otra al final).

2.4 Ejercicio 4 (Práctico 5) — CRC con `G = 1001` y `D = 10101010`

Enunciado. Considérese el generador de cuatro bits `G`, mostrado en la Figura 5.8 del libro, y supóngase que `D` tiene el valor `10101010`. ¿Cuál es el valor de `R`?

```
G = 1001 = x³ + x + 1 grado r = 3
D = 10101010 = x⁷+x⁵+x³+x¹ grado 7
```

El desarrollo completo de la división polinómica está hecho en §1.5.2, con los seis pasos y todos los XOR. Se reproduce acá en forma de tabla, que es como hay que presentarlo:

```
T = D·x³ = 10101010 000 = 10101010000 (11 bits)

iter término mayor XOR con G alineado T después
──── ──────────── ──────────────────────── ──────────
1 x¹⁰ 10010000000 (x¹⁰+x⁸+x⁷) 00111010000
2 x⁸ 00001001000 (x⁸+x⁶+x⁵) 00011110000
3 x⁷ 00000010010 (x⁷+x⁵+x⁴) 00001100000
4 x⁶ 00000001001 (x⁶+x⁴+x³) 00000101000
5 x⁵ 00000000110 (x⁵+x³+x²) 00000000110
6 x³ 00000000010 (x³+x+1) 00000000101
──────────
grado 2 < 3 → R = 101
```

El paso a paso, línea por línea (esto es lo que se escribe en el examen):

```
Paso 1 1 0 1 0 1 0 1 0 0 0 0
⊕ 1 0 0 1 0 0 0 0 0 0 0
──────────────────────
0 0 1 1 1 0 1 0 0 0 0

Paso 2 0 0 1 1 1 0 1 0 0 0 0
⊕ 0 0 0 1 0 0 1 0 0 0 0
──────────────────────
0 0 0 1 1 1 1 0 0 0 0

Paso 3 0 0 0 1 1 1 1 0 0 0 0
⊕ 0 0 0 1 0 0 0 1 0 0 0
──────────────────────
0 0 0 0 1 1 0 0 0 0 0

Paso 4 0 0 0 0 1 1 0 0 0 0 0
⊕ 0 0 0 0 1 0 0 1 0 0 0
──────────────────────
0 0 0 0 0 1 0 1 0 0 0

Paso 5 0 0 0 0 0 1 0 1 0 0 0
⊕ 0 0 0 0 0 1 0 0 1 0 0
──────────────────────
0 0 0 0 0 0 0 1 1 0 0

Paso 6 0 0 0 0 0 0 0 1 1 0 0
⊕ 0 0 0 0 0 0 0 1 0 0 1
──────────────────────
0 0 0 0 0 0 0 0 1 0 1
R = 1 0 1
```

Respuesta

```
R = 101 (3 bits = el grado de G)
Trama que va por el cable: 10101010 101
└─ D ──┘ └R┘
Receiver: 10101010101 ÷ 1001 = 10111101, resto 0 ✓
```

> **Nota sobre la numeración del Práctico 5.** El PDF oficial (`prac05_2008-p01.png`) numera este ejercicio como el **número 4**, y al CRC como el **4** y el checksum como el **3**. Algunas reediciones lo numeran como "Ejercicio 2". Acá se usa la numeración del PDF de FING.

2.5 Verificación del Ejercicio 4 con el método de XOR

Mismo dato, mismo generador, método de hardware. La tabla completa está en §1.5.3 y da `R = 101`. Se reproduce para que se vea el paralelismo:

```
G = 1001, r = 3, G mod 2³ = 001
D = 1 0 1 0 1 0 1 0

paso b R antes MSB⊕b R<<1 ¿XOR 001? R final
──── ─── ──────── ───── ────── ────────── ────────
1 1 000 1 000 sí 001
2 0 001 0 010 no 010
3 1 010 1 100 sí 101
4 0 101 1 010 sí 011
5 1 011 1 110 sí 111
6 0 111 1 110 sí 111
7 1 111 0 110 no 110
8 0 110 1 100 sí 101

R = 101 ✓ IGUAL
```

Interpretación. Los dos métodos dan `101`. Eso es lo que significa "son equivalentes": **el registro de desplazamiento con XOR hace exactamente la misma división polinómica**, pero con `r` flip-flops en vez de una tabla.

2.6 Ejercicio 5 (inventado) — Paridad par e impar sobre el mismo mensaje

Datos. `D = 11010011` (8 bits). Número de unos = **5**.

```
Paridad PAR:   5 es impar → el bit debe hacer el total PAR    → bit = 1
               (5 + 1 = 6, par ✓)
Paridad IMPAR: 5 es impar → el total debe quedar IMPAR        → bit = 0
               (5 + 0 = 5, impar ✓)
```

Comprobación con los dos errores de un bit posibles:

```
Se envía: 1 1 0 1 0 0 1 1 | 1     (paridad par;  6 unos = par)
Error en bit 3 (0→1):  ahora 7 unos → IMPAR
                       el receptor con paridad par espera PAR → NO CUADRA → DETECTADO ✓
Error en bit 5 (0→1):  idéntico → DETECTADO ✓

Se envía: 1 1 0 1 0 0 1 1 | 0     (paridad impar; 5 unos = impar)
Error en bit 3:         → 6 unos → PAR; la paridad impar espera IMPAR → DETECTADO ✓
```

Y el caso que NO detecta:

```
Se envía: 1 1 0 1 0 0 1 1 | 1
Se voltean los bits 3 y 5 (los dos 0 → 1): quedan 8 unos = PAR
→ el receptor ve 8 unos, que es PAR, y espera PAR → NO DETECTADO ✗
```

2.7 Ejercicio 6 (inventado) — Paridad 2D: cuatro escenarios con la misma matriz

Se usa la matriz de §2.2(a), con el bit (3,3) correcto:

```
      c1 c2 c3 c4 | Rp
 f1   1  0  1  1 |  1
 f2   0  1  1  0 |  0
 f3   1  1  0  0 |  0
 f4   0  0  1  0 |  1
 Cp   0  0  1  1 |
```

Escenario A — un error en (3,3).

```
Fallan f3 (3 unos, esperaba 2) y c3 (4 unos, esperaba 3)
→ intersección (3,3) → CORRIGE. ✓
```

Escenario B — dos errores en (1,1) y (4,4).

```
f1 = 0 0 1 1 → 2 unos → Rp 0, hay 1  ✗ FALLA
f4 = 0 0 1 1 → 2 unos → Rp 0, hay 1  ✗ FALLA
c1 = 0,0,1,0 → 2 unos → Cp 0, hay 0  ✓
c4 = 1,1,0,1 → 3 unos → Cp 1, hay 1  ✓
→ 2 filas y 0 columnas. Detecta, no corrige. ✗
```

Escenario C — dos errores en (1,1) y (2,1), en la misma columna.

```
f1 = 0 0 1 1 → 2 unos → Rp 0, hay 1  ✗ FALLA
f2 = 1 1 1 0 → 3 unos → Rp 1, hay 0  ✗ FALLA
c1 = 0,1,1,0 → 2 unos → Cp 0, hay 0  ✓   (¡la columna NO falla: los dos
                                           errores se compensan!)
→ 2 filas y 0 columnas. Detecta, no corrige. ✗
```

Escenario D — dos errores en (1,1) y (2,2).

```
Fallan f1, f2, c1, c2  (dos filas, dos columnas)
→ Detecta, no corrige.  (Es el caso de §2.2(b).)
```

Conclusión del ejercicio:

```
┌──────────────────────┬──────────────┬──────────────────────┐
│ Filas que fallan     │ Columnas     │ Qué puede hacer      │
├──────────────────────┼──────────────┼──────────────────────┤
│ 1                    │ 1            │ CORRIGE              │
│ 2                    │ 2            │ detecta, no corrige  │
│ 2                    │ 0            │ detecta, no corrige  │
│ 0                    │ 0            │ ¡nada! (4 errores)   │
└──────────────────────┴──────────────┴──────────────────────┘
```

2.8 Ejercicio 7 (inventado) — Internet checksum con acarreo circular

Datos. Cuatro words de 16 bits: `0x1234 = 4660`, `0x5678 = 22136`, `0x9ABC = 39612`, `0xDEF0 = 57072`.

Paso 1 — sumar y arrastrar el acarreo.

```
    4660
  + 22136   →  26796   (< 65536, sin acarreo)
  + 39612   →  66408   (≥ 65536 → hay acarreo)

    66408  −  65536  =    872
    872    +      1  =    873      ← acarreo devuelto al final

  + 57072   →  57945   (< 65536, sin acarreo)

S = 57945
```

Paso 2 — complementar.

```
~57945 = 65535 − 57945 = 7590
7590 = 1×4096 + 13×256 + 10×16 + 6 = 4096 + 3328 + 160 + 6  → 0x1DA6
```

Paso 3 — verificar.

```
57945 + 7590 = 65535 = 0xFFFF  ✓   (todo unos: checksum correcto)
```

Respuesta: Internet checksum = `0x1DA6` = 7590.

El caso que NO detecta, demostrado con estos números:

```
Trama original:  w1=0x1234  w2=0x5678  w3=0x9ABC  w4=0xDEF0
Trama con dos words intercambiadas:
                 w1=0x5678  w2=0x1234  w3=0x9ABC  w4=0xDEF0

Suma original      : 4660+22136+39612+57072 = 123480  →  → 57945
Suma intercambiada : 22136+4660+39612+57072  = 123480  →  → 57945

LAS SUMAS SON IDÉNTICAS → el checksum es el mismo
→ el receptor ACEPTA la trama.  El error pasa inadvertido.  ✗
```

Interpretación. Es **exactamente el mismo argumento de la paridad**, aplicado a words: dos perturbaciones que se compensan en la suma no dejan rastro. Por eso el checksum de UDP no protege contra reordenamientos de datos.

2.9 Ejercicio 8 (inventado) — CRC por división polinómica: `G = 1101`, `D = 100100`

Datos.

```
G = 1101 = x³ + x² + 1          r = 3
D = 100100 = x⁵ + x²
T = D·x³ = 100100000 = x⁸ + x⁵        (9 bits)
```

División polinómica, paso a paso. En cada paso, `G` cubre las **cuatro columnas que empiezan en la del término mayor**, con los valores `d₃ d₂ d₁ d₀`:

```
Paso 1    1 0 0 1 0 0 0 0 0
        ⊕ 1 1 0 1 0 0 0 0 0      ← G en x⁸
          ─────────────────
          0 1 0 0 0 0 0 0 0

Paso 2    0 1 0 0 0 0 0 0 0
        ⊕ 0 1 1 0 1 0 0 0 0      ← G en x⁷
          ─────────────────
          0 0 1 0 1 0 0 0 0

Paso 3    0 0 1 0 1 0 0 0 0
        ⊕ 0 0 1 1 0 1 0 0 0      ← G en x⁶
          ─────────────────
          0 0 0 1 1 1 0 0 0

Paso 4    0 0 0 1 1 1 0 0 0
        ⊕ 0 0 0 1 1 0 1 0 0      ← G en x⁵
          ─────────────────
          0 0 0 0 0 1 0 1 0

Paso 5    0 0 0 0 0 1 0 1 0
        ⊕ 0 0 0 0 0 1 1 0 1      ← G en x³
          ─────────────────
          0 0 0 0 0 0 0 0 1
                                        grado 0 < 3 → FIN
```

Respuesta

```
R = 001  (3 bits = el grado de G)
Se transmite:  100100 001
Recepción:     100100001 ÷ 1101 = 1011110 , resto 0
```

Verificación por multiplicación en `GF(2)` (que es la única forma válida: en `GF(2)` multiplicar **no** es multiplicar enteros, es XOR de desplazamientos):

```
Q = 1011110
Q << 3 = 1 0 1 1 1 1 0 0 0     (posiciones 8..0: 1 1 1 1)
Q << 2 =   1 0 1 1 1 1 0 0     (posiciones 7..0: 1 1 1 1)
Q       =       1 0 1 1 1 0     (posiciones 5..0: 1 1 1 1 0 1)

XOR columna por columna:
   pos8: 1                  = 1
   pos7: 1 ⊕ 1               = 0
   pos6: 1 ⊕ 1               = 0
   pos5: 1 ⊕ 1 ⊕ 1           = 1
   pos4:     1 ⊕ 1           = 0
   pos3: 1 ⊕ 1               = 0
   pos2:     1 ⊕ 1           = 0
   pos1:         0 ⊕ 1       = 1
   pos0:         1           = 1
──────────────────────────
   resultado = 1 0 0 1 0 0 0 0 1  = 100100001  ✓
```

⚠️ Errores típicos

- **Perder la cuenta de columnas** al escribir un XOR largo. El error más común de todos: uno escribe la fila del generador con un cero de más o de menos y el resultado sale distinto. **Por eso en el examen hay que alinear las columnas con espacios**, y **por eso en la práctica se usa el registro de desplazamiento** (§2.10), que sólo toca `r` bits por iteración y es inmune a este error.
- **Sumar en vez de XORar.** En `GF(2)` no hay acarreo.
- **Olvidar los `r` ceros a la derecha.** Sin ellos el polinomio se divide mal y el receptor no puede recomputar el resto.

2.10 Ejercicio 9 (inventado) — El mismo cálculo con la tabla de XOR

Datos. `G = 1011 = x³ + x + 1`, `r = 3`, `G mod 2³ = 011`. `D = 1001001 = x⁶ + x³ + 1`.

Método del registro de desplazamiento (§1.5.3), un renglón por bit de `D`:

```
paso   b    R       MSB(R)⊕b   R = R<<1    ¿XOR 011?   R final
────   ──   ─────   ────────   ─────────   ─────────   ───────
 1     1    000        1        000          sí         011
 2     0    011        0        110          no         110
 3     0    110        1        100          sí         111
 4     1    111        0        110          no         110
 5     0    110        1        100          sí         111
 6     0    111        1        110          sí         101
 7     1    101        0        010          no         010
                                                  ──────────
                                                    R = 010
```

Y la división polinómica del mismo caso, para que se vean iguales:

```
T = D·x³ = 1001001000 = x⁹ + x⁶ + x³      (10 bits)

Paso 1    1 0 0 1 0 0 1 0 0 0
        ⊕ 1 0 1 1 0 0 0 0 0 0      ← G en x⁹
          ─────────────────────
          0 0 1 0 0 0 1 0 0 0

Paso 2    0 0 1 0 0 0 1 0 0 0
        ⊕ 0 0 1 0 1 1 0 0 0 0      ← G en x⁷
          ─────────────────────
          0 0 0 0 1 1 0 0 0 0

Paso 3    0 0 0 0 1 1 0 0 0 0
        ⊕ 0 0 0 0 1 0 1 1 0 0      ← G en x⁵
          ─────────────────────
          0 0 0 0 0 1 1 1 0 0

Paso 4    0 0 0 0 0 1 1 1 0 0
        ⊕ 0 0 0 0 0 1 0 1 1 0      ← G en x⁴
          ─────────────────────
          0 0 0 0 0 0 1 0 1 0

Paso 5    0 0 0 0 0 0 1 0 1 0
        ⊕ 0 0 0 0 0 0 0 1 0 1 1    ← G en x³ (las 4 últimas columnas)
          ─────────────────────
          0 0 0 0 0 0 0 0 1 0      ← grado 1 < 3 → FIN
```

```
R = 010   ✓  el mismo que por la tabla de XOR

Verificación:  T ⊕ R = 1001001000 ⊕ 0000000010 = 1001001010
               Q = 1011110 y  Q × 1011 = 1001001010   (resto 0)
```

Interpretación. Dos métodos, un resultado. El de la **división polinómica** es el que hay que saber hacer en un examen de papel; el del **registro de desplazamiento** es el que está en el silicio y el que usan `zlib`, `iSCSI`, SCTP y la placa de red de tu máquina.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-02, Pregunta 3 (8 puntos) — paridad en dos dimensiones

> **Fuente:** Examen `eirc1002` / `sirc1002` — febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1002.pdf

Enunciado (texto del examen)

> Pregunta 3 (8 puntos)
>
> a) Explique el funcionamiento del mecanismo de chequeo por paridad en dos dimensiones utilizado para detección de errores en Capa de Enlace.
>
> b) ¿Permite realizar correcciones? ¿De cuántos bits? Justifique

Respuesta oficial (del PDF de soluciones)

> La paridad en dos dimensiones considera el flujo de datos como un flujo "bi dimensional", tomando el flujo como una matriz donde cada palabra es puesta en una fila, y palabras consecutivas en filas consecutivas. De esta forma el bit j-ésimo de cada palabra forma la columna j-ésima de toda la matriz.
>
> A cada fila se le agrega un bit de paridad (calculado sobre la fila) y cada cierto numero de palabras se agrega una palabra de paridad calculada por columna.
>
> La paridad en dos dimensiones permite: - Corregir errores de un bit por palabra y por columna, ya que esto hace posible saber que bit está invertido a través de la intersección de la fila y columna que fallan la verificación de paridad. - En algunos casos permite detectar errores dobles.

Análisis de la solución oficial

Lo que está bien, y es el corazón del puntaje:

1. **La definición de la matriz** es exactamente la que hay que usar: "cada palabra es puesta en una fila, palabras consecutivas en filas consecutivas, el bit j-ésimo de cada palabra forma la columna j-ésima". Con eso se entiende que **no hay ambigüedad** en cómo se arma.
2. **"Una palabra de paridad calculada por columna"** — ojo con la frase: la solución dice "**palabra**" de paridad por columna, cuando lo correcto es que el cálculo de columna **escribe un bit por columna**, todos en una última fila. Las dos cosas son consistentes (una fila de bits de paridad), pero si el enunciado mezclara los niveles, hay que aclarar cuál es cuál.
3. **La razón de la corrección** está bien: "*a través de la intersección de la fila y columna que fallan la verificación de paridad*". Ésa es la frase que hay que decir.

Lo que le falta — y son los puntos que se pueden sumar:

1. **El ítem (b) pide "de cuántos bits", y la solución oficial no lo dice con claridad.** Hay que decir explícitamente: **corrige exactamente 1 bit**.
2. **"En algunos casos permite detectar errores dobles"** es **demasiado generoso**. La 2D paridad detecta **todos** los errores de dos bits, no "algunos". Lo que es cierto es que **no puede corregirlos**.
3. **No menciona el techo**: existen errores de **cuatro** bits (un rectángulo `2 × 2`) que **no se detectan en absoluto**. Es el dato que demuestra que entendiste la estructura lineal del código.
4. **No justifica por qué 2D y no 1D.** La respuesta es: 1D detecta los errores de **número impar** de bits y ninguno de número par; 2D agrega la dimensión que faltaba.

> **Nota:** la solución oficial del ítem (b) es más laxa que lo que la pregunta pide. Mi respuesta es más fuerte, y creo que es la que el corrector busca: la pregunta dice "¿de cuántos bits?" y eso exige un número.

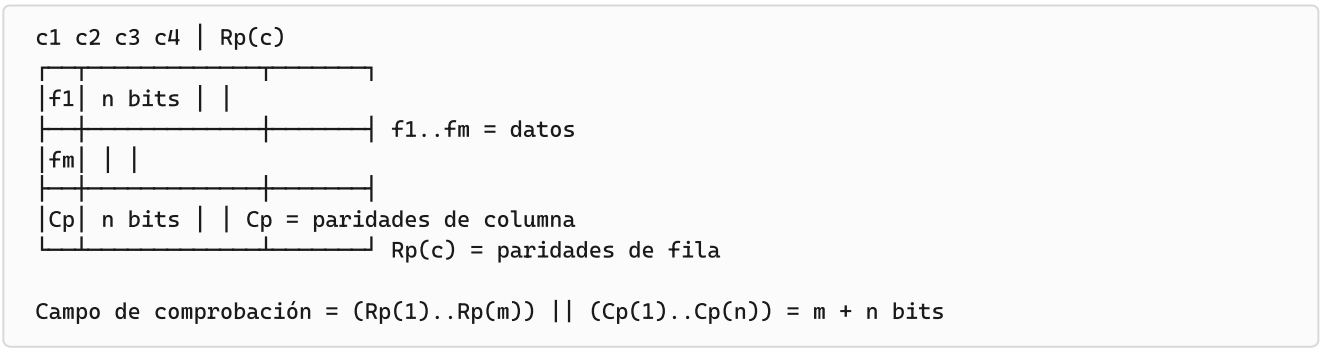
Mi resolución

(a) Cómo funciona.

Se considera el flujo de datos como **una matriz de `m` filas por `n` columnas**: cada palabra de `n` bits del flujo se coloca en una fila, y palabras consecutivas en filas consecutivas. Así, **el bit `j` de cada palabra forma la columna `j`** de toda la matriz.

Al bloque se le agrega **un campo de comprobación de `m + n` bits**: - **un bit de paridad al final de cada fila**, calculado sobre los `n` bits de esa fila; - **un bit de paridad al final de cada columna**, calculado sobre los `m` bits de esa columna.

El receptor **recalcula las `m` paridades de fila y las `n` paridades de columna** y las compara con las recibidas:



Nota: no hace falta un bit de esquina. La paridad de la suma de las paridades de fila y la de la suma de las de columna son **iguales** por construcción (ambas son la paridad del número total de unos de la matriz), así que la esquina es redundante.

(b) ¿Corrige? ¿Cuántos bits?

Sí: corrige exactamente 1 bit. Y detecta (sin corregir) **todos** los errores de dos bits.

Justificación de la corrección de 1 bit:

```
Si hay UN solo bit equivocado, y está en la fila i, columna j:

→ la paridad de la FILA i falla (tiene un uno de más o de menos)
→ la paridad de la COLUMNA j falla (idem)

→ la intersección (i, j) queda completamente determinada
→ el receptor invierte ese bit y el mensaje queda EXACTO.
```

Justificación de por qué no puede corregir 2 bits:

```
Con dos errores, hay varios casos:

Caso 1 — en filas distintas y columnas distintas: (i,j) y (k,l), i≠k, j≠l
fallan las filas i y k, y las columnas j y l
→ 4 posiciones candidatas, sólo 2 están mal → INDETERMINADO

Caso 2 — en la misma columna: (i,j) y (k,j)
fallan las filas i y k; la columna j NO falla (2 errores se compensan)
→ 2 filas y 0 columnas: el patrón es imposible de atribuir

Caso 3 — en la misma fila: análogo, 0 filas y 2 columnas

En todos los casos: DETECTA que hay error (algo falló), pero NO PUEDE
decir dónde. Faltaría redundancia extra para desempatar.
```

El techo, que es lo que demuestra que entendiste:

```
Con un error de CUATRO bits formando un rectángulo 2×2 — (i,j), (i,l), (k,j),
(k,l) — cada fila y cada columna afectada recibe DOS cambios, de signo opuesto,
así que NINGUNA paridad se altera:

fila i: +1 y −1 → la paridad no cambia
fila k: +1 y −1 → la paridad no cambia
columna j: +1 y −1 → la paridad no cambia
columna l: +1 y −1 → la paridad no cambia

∴ Las 2D paridades NO detectan ese error. Cero verificaciones fallan.
```

Conclusión con la distancia de Hamming:

```
La 2D paridad es un código con d = 3 sobre los bits de datos
(d = 3 por el patrón de 3 flips en el bloque de paridades; y los patrones
de 2 flips en datos siempre se detectan).

Con d = 3:
detecta todos los errores de peso ≤ 2
corrige todos los errores de peso ≤ ⌊(3−1)/2⌋ = 1

Coincide con lo demostrado arriba. ✓
```

> **Tope para 8 puntos:** la definición de la matriz (palabras en filas, bit j = columna j), el campo de `m + n` bits, el recálculo de las paridades de fila y de columna, y la corrección del error de 1 bit **por intersección**. Para el ítem (b): decir explícitamente **"corrige 1 bit"** y **justificar por qué 2 no** (ambigüedad de las 4 intersecciones). Sumar el argumento del rectángulo 2 × 2 como techo.

PARTE 4 — CIERRE

📌 Resumen

1. **Detectar** es saber que hubo error; **corregir** es saber cuál era el bit y repararlo. El corrector cuesta mucha más redundancia.
2. **Paridad 1D**: un bit. Detecta **todos** los errores de **número impar** de bits y **ninguno de número par** (dos bits que se voltean se compensan).
3. **Paridad 2D**: matriz `m × n` + `m` paridades de fila + `n` de columna. Campo de `m + n` bits. La esquina es redundante.
4. **Corrige 1 bit** (por intersección fila × columna) y **detecta todos los de 2**, pero **no los corrige**. Hay un techo: el **rectángulo 2 × 2** de 4 errores **no se detecta**.
5. **Internet checksum**: 16 bits, complemento a uno, con acarreo circular. Detecta todos los errores de un bit; **no detecta el intercambio de dos words** (la suma es conmutativa).
6. **CRC**: división polinómica en `GF(2)`, donde `+` es **XOR** y `×` es **AND**. Se agregan `r` ceros y se divide; el resto es el CRC.
7. **El hardware** hace lo mismo con un **registro de `r` bits y un XOR por bit**: los dos métodos dan el mismo resultado.
8. **CRC-32 =** `0x04C11DB7` (Ethernet, ZIP, PNG), **CRC-32C =** `0x1EDC6F41` (SCTP, iSCSI), **CRC-16/CCITT =** `0x1021` (HDLC). Hay varios porque cada aplicación optimiza para algo distinto.
9. **Propiedad clave**: un error pasa inadvertido **si y sólo si es múltiplo de `G`**. Por eso el CRC detecta **siempre** un error de 1 bit y **siempre** una ráfaga de hasta `r` bits. La distancia mínima es `d = r + 1`.
10. **El CRC detecta, no corrige.** Para corregir hacen falta **Hamming** o **Reed–Solomon**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Demostrá por qué la paridad de una dimensión **no detecta** un error de dos bits.

▶ Respuesta

2. En una matriz 4 × 4 con paridad par, ¿cuánto mide el campo de comprobación y por qué no hace falta el bit de esquina?

▶ Respuesta

3. Se detecta un error de un bit y fallan la fila 2 y la columna 5. ¿Qué hacés y qué se corrige?

▶ Respuesta

4. ¿Por qué dos errores no se pueden corregir con paridad 2D, si los errores sí se detectan?

▶ Respuesta

5. ¿Por qué el Internet checksum **no detecta** el intercambio de dos words de 16 bits? ¿Qué otro error tampoco detecta?

<details>
<summary>Respuesta</summary>
</details>

6. En `GF(2)`, ¿qué son la suma y el producto? ¿Por qué se usan polinomios y no división entera?

<details>
<summary>Respuesta</summary>
</details>

7. Ejecutá el CRC de `D = 10101010` con `G = 1001`. ¿Cuál es `R` y cuánto overhead agrega?

<details>
<summary>Respuesta</summary>
</details>

8. ¿Por qué hay varios polinomios generadores de CRC estandarizados en lugar de uno solo?

<details>
<summary>Respuesta</summary>
</details>

🔗 Referencias

- **Examen `eirc1002` / `sirc1002`** — P3 (8 pts, paridad en dos dimensiones: funcionamiento y corrección)
- **Examen `eirc0707` / `sirc0707`** — P2 (5 pts, nombres de los mecanismos de detección y corrección de la capa de enlace)
- **Examen `eirc0812` / `sirc0812`** — P4 (7 pts, DLE en POP3: el mismo concepto de transparencia)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 4 §4.2 ("Error Detection and Correction")
- **Kurose & Ross**, *Computer Networking: A Top-Down Approach*, §5.5 ("Error Detection and Correction") — figuras 5.4 a 5.9
- **Práctico 5 de 2008** — `material-oficial/practicos/png/prac05_2008-p01.png`, ejercicios 1 a 4 (paridad 2D, Internet checksum, CRC)
- `RFC 1071` — Internet checksum: cálculo en UDP e IP
- `RFC 1624` — por qué no se puede cambiar el algoritmo del checksum de TCP/UDP
- `RFC 9293` — TCP (2022, *Internet Standard*): el checksum de TCP es el Internet checksum
- **IEEE 802.3** — el FCS de Ethernet es un CRC-32 (`0x04C11DB7`), 4 bytes
- **Clase siguiente:** 40 — Corrección de errores y ARQ: Hamming, Stop-and-Wait, Go-Back-N y Selective Repeat

Clase 40

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 Detectar no es corregir: por qué la pregunta importa
- 1.2 La distancia de Hamming y los dos códigos que hay que saber
- 1.3 Los bits de paridad: la tabla de potencias de 2 y la condición `2^r ≥ m + r + 1`
- 1.4 Ejemplo completo: derivar los bits de paridad con `m = 4`, `r = 3`
- 1.5 El procedimiento de corrección: el SÍNDROME
- 1.6 ARQ: reintentar en vez de corregir
- 1.7 La relación con TCP: por qué TCP no es exactamente ninguno de los dos
- 1.8 ⚠️ Actualización a 2026
- 2.1 Ejercicio 1 — La eficiencia de Stop-and-Wait, con cuentas
- 2.2 Ejercicio 2 — Hamming SEC con corrección completa
- 2.3 Ejercicio 3 — Los tres ARQ con una pérdida en el segmento 3 de 5
- 2.4 Ejercicio 4 — ¿Por qué Go-Back-N con `N` grande es peor que Selective Repeat?
- 3.1 ⭐ Examen 2009-03, Pregunta 1 (10 puntos) — El concepto de Go-Back-N
- 3.2 Examen 2007-03, Problema 1 (30 puntos) — ARQ de capa 2 con Selective Repeat y *piggybacking*
- 3.3 Nota sobre el Práctico 5 de 2008 (ejercicios 1 a 4)
- 3.4 Examen 2011-03, Pregunta 4 (10 puntos) — el diagrama que es Go-Back-N puro
- 📌 Resumen en 10 líneas
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: E (Capa de enlace) · **Semana:** 14 · **Duración estimada:** 115 min **Correlativa:** Clase 39 (detección de errores: paridad, checksum y CRC) · **Sigue con:** Clase 41 (acceso al medio: ALOHA, CSMA/CD y CSMA/CA) **Material de referencia:** Tanenbaum cap. 3 §3.3–3.5 y cap. 4 · Kurose §5.3 · RFC 2018 (SACK)

🎯 Al terminar esta clase vas a poder

1. Explicar por qué **detectar** y **corregir** son problemas distintos, y en qué medios conviene **corregir en el receptor** en vez de **retransmitir**.
2. Definir la **distancia de Hamming** y derivar las capacidades de **SEC** y **SECDED**.
3. **Derivar los bits de paridad** de un código de Hamming de `m` bits de datos y `r` de paridad, y explicar de dónde sale la condición `2^r ≥ m + r + 1`.
4. Ejecutar el **procedimiento de corrección completo** (cálculo del **síndrome** y ubicación del bit erróneo) sobre un ejemplo numérico.
5. Calcular la **eficiencia de Stop-and-Wait** con la fórmula `1/(1+2a)` y explicar por qué depende del RTT.
6. Describir **Go-Back-N** y **Selective Repeat** con sus diagramas temporales, y decir **por qué TCP no es exactamente ninguno de los dos** salvo con SACK.
7. Justificar **por qué Selective Repeat exige un buffer en el receptor** y ACK no acumulados.
8. Resolver completo el **Examen 2009-03 Pregunta 1** (10 puntos).

> **Prerequisito del examen:** `2009-03 P1` (10 pts — el concepto y el funcionamiento de Go-Back-N), `2007-03 Problema 1` (30 pts — protocolo de capa 2 con ARQ de tipo **Selective Repeat** y *piggybacking*), `2007-07 P2` (5 pts — mecanismos de detección y corrección de errores de la capa de enlace), `2010-07 Problema 1` (30 pts — protocolo *stop-and-wait* semifiable para video), `2011-03 P4` (10 pts — el diagrama TCP con y sin pérdida del primer segmento, que es Go-Back-N puro).

> ⚠️ Esta clase tiene **cálculo de verdad**. El código de Hamming se hace bit por bit y los tres ARQ se dibujan con líneas de tiempo. Hacé los ejercicios con lápiz y papel antes de mirar la resolución.

PARTE 1 — TEORÍA

1.1 Detectar no es corregir: por qué la pregunta importa

En la clase anterior (Clase 39) vimos los **detores**: paridad, paridad 2D, checksum y CRC. Todos hacen **lo mismo**: un receptor devuelve "OK" o "error". Cuando devuelve "error", el receptor **no sabe cuál bit** está mal, y por lo tanto **no puede arreglarlo**: lo único que puede hacer es **tirar el paquete y pedir que lo manden de nuevo**.

```
DETECTAR                          CORREGIR
────────                          ────────
"¿hay error?"  sí/no              "¿cuál bit está mal?"  → lo arreglo
coste: 1..32 bits de control      coste: r bits de paridad + lógica
si hay error: OBLIGATORIO          si hay error: se repara solo, sin
retransmitir                      tocar la red
```

La razón económica es el RTT. Corregir en el receptor es gratis en términos de red: no se transmite nada. Retransmitir, en cambio, cuesta:

```
EMISOR ──(trama)──► R1 ──► ... ──► R4 ──(trama)──► RECEPTOR
       ◄──────────── ACK ──────────────────────────────────

Si la trama tenía error y se retransmite:
  EMISOR ──(trama)──► ... ──► RECEPTOR          t_frame
         ◄──────────────── ACK ────────          t_prop
         ◄──────────────── ACK ────────          t_prop   (el ACK vuelve)
         ──(trama)──────► ... ──► RECEPTOR      t_frame
         ◄──────────────── ACK ────────          t_prop
         ◄──────────────── ACK ────────          t_prop

COSTE DE RETRANSMITIR = 2·t_frame + 2·t_prop  = un RTT entero
```

⚠️ **Cuando conviene corregir y no retransmitir:** en los medios donde el **RTT es enorme comparado con el tiempo de transmisión**:

Medio	`t_prop` (ida)	`t_frame` (1 KB)	`a = t_prop/t_frame`	`U` de Stop-and-Wait
LAN a 10 Mb/s (100 m)	0,5 µs	800 µs	0,0006	**99,9 %**
Enlace dedicado a 2 Mb/s (100 km)	500 µs	4 ms	0,125	80,0 %
Fibra dedicada (1000 km, 2 Gb/s)	5 ms	4 µs	**1 250**	**0,04 %**
Enlace satelital GEO (36 000 km)	120 ms	0,8 ms	**150**	**0,33 %**
Radio de larga distancia	100 ms+	—	enorme	casi 0

⚠️ **Atención con la fila del satélite:** es 0,33 %, no 3,3 %. Recalculo: `1/(1+2·150) = 1/301 = 0,00332 = 0,33 %`. **Una mainframe de 2011 sobre un enlace satelital GEO tenía menos del 1 % de Utilización del enlace.** Por eso en los enlaces demuy largo RTT tiene sentido **corregir** (FEC, *forward error correction*) y no retransmitir: el costo de una corrección en el receptor es cero en la red.

⚠️ **El otro extremo de la balanza:** en un medio **muy ruidoso**, si la tasa de error de bits es alta, retransmitir **todo** es inútil: se pierde el tiempo y se vuelve a perder. Ahí también conviene **corregir** (o cambiar el código), porque **ARQ con tasa de error alta se vuelve un problema**. Ésa es la razón por la que el Wi-Fi y el DVB usan **FEC** y **no** ARQ.

1.2 La distancia de Hamming y los dos códigos que hay que saber

La definición

```
La DISTANCIA DE HAMMING entre dos palabras de la misma longitud
d(A,B) es la CANTIDAD de posiciones en las que difieren.

  A = 1 0 1 0 1 0 1 0
  B = 1 0 1 1 1 0 1 1
      ─   ─     ─     ─
  d(A,B) = 4

Equivalentemente:  d(A,B) = peso(A XOR B)
```

⚠️ **Por qué la distancia de Hamming es LA medida:** porque un error de `k` bits **siempre** transforma una palabra en otra que está a distancia `≤ k`. Si el mínimo `d` de un código es `d_min`, entonces:

```
d errores realmente pueden mover una palabra            → distancia d
d_min − 1 errores NUNCA mueven una palabra fuera del      → undetectable
                                                          código
```

De ahí salen las dos capacidades, y son las que hay que memorizar:

	Detecta	Corrige	`d_min` mínimo
SEC (*Single Error Correcting*)	`d ≤ 1`	**1** bit	**2**
SECDED (*Single Error Correcting, Double Error Detecting*)	`d ≤ 2`	**1** bit	**3**
(doble error, no corrige 2)			
Código con `d_min = 4`	`d ≤ 3`	**1** bit (+ sabe que hay 2)	**4**

⚠️ **El error conceptual más caro de esta clase: la paridad detecta 1 error y corrige 0. Hamming SEC detecta 1 y corrige 1. Hamming SECDED detecta 2 y corrige 1.** Se preguntan mucho, y se confunden porque las tres cosas suenan parecido.

⚠️ **¿Por qué SECDED necesita `d_min = 3` y no 2?** Porque con `d_min = 2` no se puede distinguir "1 error" de "2 errores": las dos cosas dan distancia 2, y el receptor solo tiene una palabra de referencia. Con `d_min = 3`, un error de 1 bit da distancia 1 (distinguible), uno de 2 bits da distancia 2, y uno de 3 da distancia 3. Como el receptor **solo sabe corregir de 0**, descarta todo lo que sea ≥ 2. Por eso SECDED **detecta 2** aunque solo **corrija 1**.

1.3 Los bits de paridad: la tabla de potencias de 2 y la condición `2^r ≥ m + r + 1`

La intuición: cada bit de paridad "vigila" un subconjunto

La idea es que **cada bit de paridad vigila un conjunto distinto de posiciones**, y que hay `r` bits de paridad. Hay que poder distinguir `r` posiciones más "nada":

```
Hay r+1Casos que el receptor tiene que diferenciar:
  · "no hay error en ninguno"                          → el código está bien
  · "el bit de paridad 1 tiene error"
  · "el bit de paridad 2 tiene error"
  · ...
  · "el bit de paridad r tiene error"
  · "el bit de datos j tiene error"   (¡j puede ser CUALQUIERA de m!)

En total:  r (paridades) + m (datos) + 1 (sin error)
                                                =  m + r + 1  casos

Cada caso se codifica con un patrón distinto de r bits ⇒
     2^r  ≥  m + r + 1
```

La tabla que hay que saber de memoria (con `m` datos y `r` paridades):

`m` (datos)	`r` (paridad)	`m + r + 1`	`2^r`	¿Alcanza?	Total de bits
1	2	4	4	sí	3
2	2	5	4	**no**	—
3	2	6	4	**no**	—
4	**3**	**8**	**8**	**sí**	**7**
5	3	9	8	**no**	—
6	3	10	8	**no**	—
7	3	11	8	**no**	—
8	4	13	16	sí	12
16	5	22	32	sí	21
32	6	39	64	sí	38
64	7	72	128	sí	71

⚠️ **El resultado clave que hay que entender: el SEC no funciona para 5, 6 ni 7 bits de datos.** Es contraintuitivo: uno esperaría que con más datos it'd mieux. La razón es que **cada bit de dato nuevo necesita su propio patrón de síndrome** y los patrones disponibles son `2^r`, que crecen **más lento** que `m + r + 1`. Para esos casos hay que usar **códigos cíclicos** (BCH, Reed-Solomon) o **SEC-DED con bit de paridad global**.

⚠️ **Un detalle de la condición que cambia según el texto:** muchos textos usan la forma **ligeramente más simple** `2^r ≥ m + 1` (ignorando los `r` bits de paridad entre los casos a distinguir). Con `m = 4` da `2^3 = 8 ≥ 5`, y el resultado es el mismo (`r = 3`). La forma con `+ r + 1` es la rigurosa. **En un examen, usá la que te parezca que pide el enunciado y mencioná la otra.**

Dónde se colocan los bits de paridad

Los bits de paridad van en las posiciones que son **potencias de 2**: 1, 2, 4, 8, 16, ...

```
┌──────────────────────────────────────────────────────────┐
│  palabra completa de 7 bits (m=4, r=3):                  │
│                                                          │
│  posición   1   2   3   4   5   6   7                    │
│           ┌───┬───┬───┬───┬───┬───┬───┐                  │
│  contenido │P1 │P2 │D1 │P3 │D2 │D3 │D4 │                 │
│           └───┴───┴───┴───┴───┴───┴───┘                  │
│   ↑    ↑           ↑       ↑   ↑   ↑   ↑                 │
│  2^0  2^1         2^2     (datos, sin paridad)           │
│                                                          │
│  P1 vigila las posiciones cuyo binario tiene el bit 0 = 1:  │
│      1 (001), 3 (011), 5 (101), 7 (111)                  │
│  P2 vigila las posiciones cuyo binario tiene el bit 1 = 1:  │
│      2 (010), 3 (011), 6 (110), 7 (111)                  │
│  P3 (pos 4) vigila las que tienen el bit 2 = 1:          │
│      4 (100), 5 (101), 6 (110), 7 (111)                  │
└──────────────────────────────────────────────────────────┘
```

⚠️ **El patrón no es arbitrario:** "el bit de paridad de la posición `2^j` vigila a todas las posiciones cuyo número tiene el bit `j` en 1". Es lo que hace que el **síndrome sea directamente el número de la posición del error** (§1.5). Si el patrón fuera otro, el síndrome habría que "traducirlo" y no daría el número.

1.4 Ejemplo completo: derivar los bits de paridad con `m = 4`, `r = 3`

Datos. Quiero enviar `D = 1 0 1 0` (D1=1, D2=0, D3=1, D4=0), con **paridad par** (un grupo es válido si tiene un número **par** de unos).

Paso 1 — Armo la plantilla de 7 posiciones.

```
posición:   1    2    3    4    5    6    7
           P1   P2   D1   P3   D2   D3   D4
valor:      ?    ?    1    ?    0    1    0
```

Paso 2 — Calculo P1. P1 vigila posiciones **1, 3, 5, 7**. De las que ya conozco: D1 (pos 3) = 1, D2 (pos 5) = 0, D4 (pos 7) = 0.

```
grupo P1 = { P1, D1, D2, D4 } = { ?, 1, 0, 0 }
unos conocidos = 1  (solo D1)
Para que el total sea PAR, P1 debe hacer el total PAR:
     1 + P1  tiene que ser par  ⇒  P1 = 1     (1+1 = 2, par ✓)
VERIFICACIÓN: {1, 1, 0, 0} → 2 unos → par ✓
```

Paso 3 — Calculo P2. P2 vigila posiciones **2, 3, 6, 7**: P2, D1 (pos 3) = 1, D3 (pos 6) = 1, D4 (pos 7) = 0.

```
grupo P2 = { P2, D1, D3, D4 } = { ?, 1, 1, 0 }
unos conocidos = 2
Para que sea PAR: 2 + P2 par  ⇒  P2 = 0      (2+0 = 2, par ✓)
VERIFICACIÓN: {0, 1, 1, 0} → 2 unos → par ✓
```

Paso 4 — Calculo P3. P3 (posición 4) vigila posiciones **4, 5, 6, 7**: P3, D2 (pos 5) = 0, D3 (pos 6) = 1, D4 (pos 7) = 0.

```
grupo P3 = { P3, D2, D3, D4 } = { ?, 0, 1, 0 }
unos conocidos = 1
Para que sea PAR: 1 + P3 par  ⇒  P3 = 1       (1+1 = 2, par ✓)
VERIFICACIÓN: {1, 0, 1, 0} → 2 unos → par ✓
```

Paso 5 — La palabra de código completa.

```
┌─────┬─────┬─────┬─────┬─────┬─────┬─────┐
│pos 1│pos 2│pos 3│pos 4│pos 5│pos 6│pos 7│
├─────┼─────┼─────┼─────┼─────┼─────┼─────┤
│ P1  │ P2  │ D1  │ P3  │ D2  │ D3  │ D4  │
├─────┼─────┼─────┼─────┼─────┼─────┼─────┤
│  1  │  0  │  1  │  1  │  0  │  1  │  0  │
└─────┴─────┴─────┴─────┴─────┴─────┴─────┘

PALABRA DE CÓDIGO =  1 0 1 1 0 1 0  =  "1011010"
```

⚠️ **El "truco" que hay que aplicar siempre:** la paridad de cada grupo es **(número de unos conocidos) mod 2**, y el valor de la paridad es **ese mismo número** (0 si es par, 1 si es impar). Es decir, con paridad par, `P = n_unos_conocidos mod 2`. Con **paridad impar**, `P = (n_unos_conocidos + 1) mod 2`.

⚠️ **El dato de la cátedra que hay que tener:** el **práctico 5 de 2008, ejercicio 4**, es el del **CRC** (`D = 10101010` con el generador de 4 bits de la Figura 5.8) y **ya está resuelto en la Clase 39, §2.4**. Lo que la Figura 5.4 del libro representa es exactamente la plantilla de Hamming de arriba, y este ejercicio es su desarrollo numérico.

1.5 El procedimiento de corrección: el SÍNDROME

Acá está el beautiful del código: **no hay que "buscar" el error**. El receptor recalcula las paridades, y **el número que forman los resultados en binario ES la posición del bit equivocado**.

```
El receptor recibe la palabra R y hace TRES cosas:

1) Recalcula la paridad de cada grupo, SIN tocar la palabra.
     Para el grupo de P1: ¿la paridad que dice el bit 1 de R
     coincide con la que recalculo yo?  Sí/No.
     Lo mismo con P2 y P3.

2) Escribe esos tres Sí/No como un NÚMERO BINARIO de 3 bits:
     (P3, P2, P1)  =  b3 b2 b1

3) Ese número es la POSICIÓN del bit que está mal.
     Si es 0 → no hay error.
     Si es 1..7 → corregir ese bit (voltearlo).
```

⚠️ **Por qué funciona:** el patrón de cada paridad se diseñó justamente para que el conjunto de "qué paridades fallan" identifique **unívocamente** a la posición. Un solo error en la posición `j` **falla exactamente** en las paridades `P_k` donde el binario de `j` tiene el bit `k` en 1. Y como los `r` bits de paridad están en las posiciones 1, 2, 4, 8... (que son exactamente las potencias de 2), el número formado por "qué paridades fallaron" **es** el número de la posición. Es una traducción perfecta.

Ejemplo completo de corrección

Datos. Se envió `1011010`. El canal **invierte el bit de la posición 6** (el `D3`, que era 1 y llega 0).

Lo que llega:

```
┌─────────┬─────────┬─────────┬─────────┬─────────┬─────────┬─────────┐
│ pos 1   │ pos 2   │ pos 3   │ pos 4   │ pos 5   │ pos 6   │ pos 7   │
├─────────┼─────────┼─────────┼─────────┼─────────┼─────────┼─────────┤
│    1    │    0    │    1    │    1    │    0    │  0 ✗    │    0    │
└─────────┴─────────┴─────────┴─────────┴─────────┴─────────┴─────────┘

RECIBIDO = 1 0 1 1 0 0 0  =  "1011000"
```

Paso 1 — Recalculo P1. Grupo P1 = posiciones {1, 3, 5, 7} = {1, 1, 0, 0}.

```
Paridad que DICE el bit 1 recibido:  1
Paridad que CALCULO sobre el grupo:   1 + 1 + 0 + 0 = 2  → par → 0
¿Coinciden?   1 ≠ 0   ⇒  P1 FALLA   (s₁ = 0)
```

Paso 2 — Recalculo P2. Grupo P2 = posiciones {2, 3, 6, 7} = {0, 1, **0**, 0}.

```
Paridad que DICE el bit 2 recibido:  0
Paridad que CALCULO: 0 + 1 + 0 + 0 = 1  → impar → 1
¿Coinciden?   0 ≠ 1   ⇒  P2 FALLA   (s₂ = 1)
```

Paso 3 — Recalculo P3. Grupo P3 = posiciones {4, 5, 6, 7} = {1, 0, **0**, 0}.

```
Paridad que DICE el bit 4 recibido:  1
Paridad que CALCULO: 1 + 0 + 0 + 0 = 1  → impar → 1
¿Coinciden?   1 = 1   ⇒  P3 NO FALLA   (s₃ = 0)
```

Paso 4 — El síndrome.

```
s₃ s₂ s₁  =  0  1  0  =  0b010  =  6
                                   ↑
                          "el bit 6 está mal"
```

Paso 5 — Corrijo. Volteo la posición 6 de 0 a 1.

```
┌─────────┬─────────┬─────────┬─────────┬─────────┬─────────┬─────────┐
│    1    │    0    │    1    │    1    │    0    │   1 ←   │    0    │
└─────────┴─────────┴─────────┴─────────┴─────────┴─────────┴─────────┘

CORREGIDO = 1 0 1 1 0 1 0  =  "1011010"  ✓  ¡coincide con lo enviado!
DATOS EXTRAÍDOS = D1 D2 D3 D4 = 1 0 1 0   ✓
```

El caso "no hay error": si la palabra llega bien, los tres chequeos dan "coincide", el síndrome es `000` = 0, y **no se corrige nada**. Por eso el 0 se puede usar como código de "todo bien".

⚠️ **La tabla de síndromes de este ejemplo**, que conviene tener a mano para verificar cualquier otro:

Syndrome ($s_4 s_2 s_1$)	Decimal	Posición	Qué es	¿Debe ser paridad?
000	0	—	sin error	—
001	1	1	**P1**	sí
010	2	2	**P2**	sí
011	3	3	**D1**	no
100	4	4	**P3**	sí
101	5	5	**D2**	no
110	6	6	**D3**	no
111	7	7	**D4**	no

⚠️ **El argumento que demuestra que SEC detecta 1 error:** los 8 síndromes posibles son `0` (sin error) o el número de una de las 7 posiciones. Un solo bit equivocado produce **exactamente un** código distinto del 0, y ese código **no puede** coincidir con el de otra posición porque la tabla es una **biyección**. Por eso SEC detecta y corrige **exactamente 1** y **solo 1**. Con **dos** errores, los síndromes se pueden sumar y dar un valor de una posición válida: **SEC no detecta 2**. Ése es el motivo de que el SECDED agrega un bit de paridad **global** (el bit 9 en posiciones de 1 a 8): un segundo error **siempre** cae en un grupo o en el global de forma que la paridad global queda **impar**, y eso es una señal inequívoca de "hay dos errores, no corrijas".

1.6 ARQ: reintentar en vez de corregir

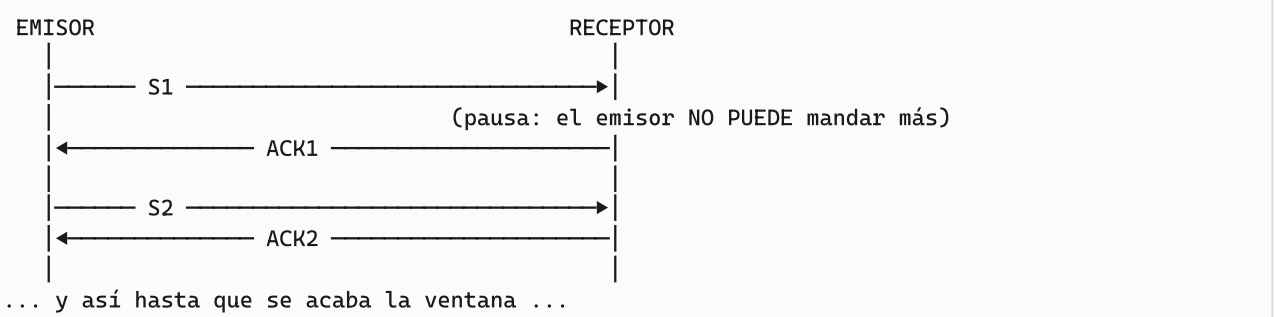
ARQ = ***Automatic Repeat reQuest***. Es la estrategia complementaria: **no corrijo, reintento**. Y hay **tres** implementaciones clásicas, que se distinguen por **una sola pregunta**: *cuando se pierde el segmento* k*, ¿qué retransmito?*

```
Stop-and-Wait:     retransmito  SÍ O SÍ  el segmento k (solo ese)
Go-Back-N:         retransmito        k, k+1, k+2, ... (en la ventana)
Selective Repeat:  retransmito  SÍ     k (y solo k)
```

⚠️ Y el **ACK acumulado** es la otra mitad de la historia:

```
ACKs ACUMULADOS:   ACK(k) significa "recibí TODO hasta el k, incluido"
                   ⇒ es 1 ACK para muchos segmentos (eficiente)
ACKs NO ACUMULADOS: ACK(k) significa "recibí el k"
                   ⇒ hace falta 1 ACK por segmento
```

1.6.1 Stop-and-Wait



El problema: el emisor queda **ocioso** `2·t_prop` entre trama y trama. En un enlace de 1 Gb/s de 1000 km con tramas de 1 KB, eso es un desperdicio del **99,9 %**.

La eficiencia (utilización) del enlace:

```
Definimos:   a = t_prop / t_frame
             (el cociente entre el tiempo de propagación y el de
              transmisión de UNA trama)

Ciclo completo de Stop-and-Wait:
     t_frame          (mando la trama)
   + t_prop           (llega)
   + t_prop           (vuelve el ACK, instantáneo)
   + t_frame          (mando la trama siguiente)
   =  2·t_frame + 2·t_prop = 2·t_frame·(1 + a)

Tiempo útil = t_frame

═══════════════════════════════════════════════════════
     U = 1 / (1 + 2a)
═══════════════════════════════════════════════════════
```

La cuenta con números. Trama de **1000 bytes** (8000 bits) sobre un enlace de **1 Mbps**, a **2000 km** de distancia con `v = 2 × 10⁸ m/s`:

```
t_frame = L/R = 8000 bits / 10⁶ bit/s = 8 ms
t_prop  = d/v = 2 000 000 m / 2 × 10⁸ m/s = 10 ms

a = t_prop / t_frame = 10 / 8 = 1,25

U = 1 / (1 + 2a) = 1 / (1 + 2 × 1,25) = 1 / 3,5 = 0,2857 = 28,6 %
```

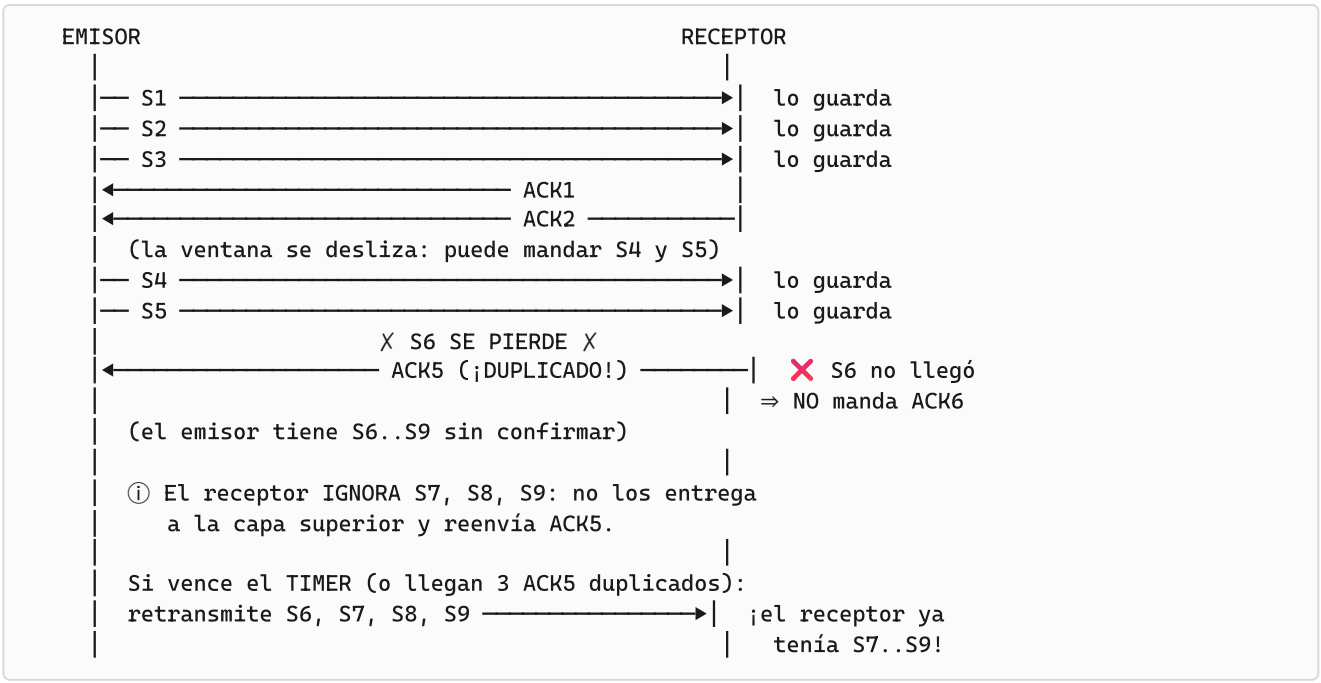
Otro caso, extremo: enlace **satélital GEO** (36 000 km, `v = 3 × 10⁸ m/s`), trama de 1000 bytes a **10 Mbps**:

```
t_frame = 8000 / 10⁷ = 0,8 ms
t_prop  = 36 000 000 / 3 × 10⁸ = 120 ms

a = 120 / 0,8 = 150

U = 1 / (1 + 300) = 1/301 = 0,00332 = 0,33 %
```

⚠️ **La conclusión para el examen:** Stop-and-Wait es **aceptable** cuando `a` es chico (redes LAN, enlaces locales) y **inservible** cuando `a > 1` (el enlace queda más tiempo ocioso que trabajando). **La respuesta completa es "hay que usar ventanas"**, y de ahí salen los otros dos protocolos.

1.6.2 Go-Back-N

La idea: el emisor puede tener **hasta** `N` **segmentos sin confirmar** en el medio. El receptor **solo manda ACK acumulados** y, si le llega un segmento **con un hueco**, **lo descarta y reenvía el ACK del que esperaba**.



Las cinco reglas de Go-Back-N (que son los "titulares" de la solución oficial):

```
1. El emisor puede tener hasta N paquetes sin ACK en el pipeline.
2. El receptor SOLO envía ACKs acumulados.
3. El receptor NO envía el ACK de un paquete si hay un hueco.
4. El emisor tiene UN SOLO timer, para el paquete más viejo
   todavía no confirmado.
5. Si el timer vence, retransmite TODOS los paquetes no confirmados
   (desde el más viejo, todos los de la ventana).
```

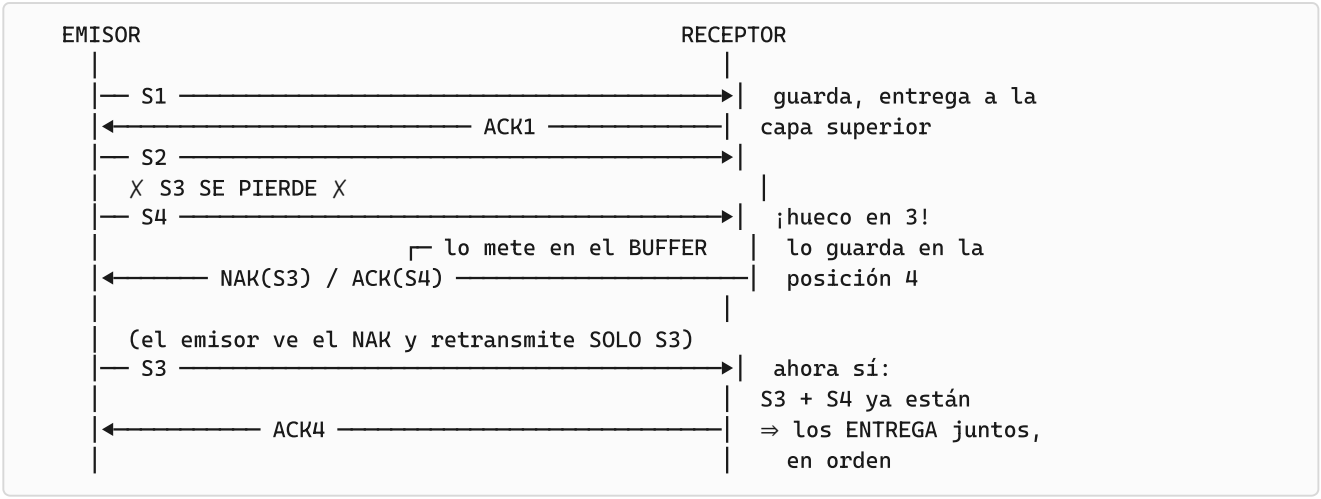
⚠️ **La característica definitoria:** el receptor **descarta** lo que llega después del hueco y **no lo reenvía**. Cuando por fin llega el perdido, el receptor tiene que recibir **de nuevo** todo lo que ya había. Eso se llama **waste** (desperdicio), y es **el precio de la simplicidad**.

⚠️ **La ventaja:** el receptor **no necesita buffer** (o necesita uno de tamaño 1), no necesita **reordenar**, y el ACK acumulado es barato: **un ACK por segmento, no uno por segmento recibido**.

⚠️ **La desventaja: ineficiente** cuando hay pérdida. Una sola pérdida retransmite `N` segmentos. Con `N = 128` (el máximo de TCP), una pérdida retransmite hasta 128 segmentos.

1.6.3 Selective Repeat

La idea: el receptor tiene un **buffer** y **reordena**. Cuando le llega algo fuera de orden, **lo guarda** y **manda un NAK** (o un ACK selectivo) diciendo qué le falta. El emisor retransmite **solo** lo que falta.



Las cuatro condiciones que hacen que funcione:

Condición	Por qué
Buffer de recepción en el receptor (tamaño ≥ `N`)	tiene que guardar los segmentos que llegan fuera de orden
Reordenar al recibir el hueco	entrega a la capa superior **en orden**, no como llegó
NAK o ACK selectivo	el emisor tiene que saber **cuál** falta, no solo "hasta dónde tengo"
ACK no acumulado (o SACK)	con ACK acumulado no se puede saber qué llegó después del hueco

⚠️ **La condición matemática que hay que saber:** para que Selective Repeat funcione sin ambigüedad con números de secuencia de `k` bits, la ventana del emisor y el buffer del receptor deben cumplir:

```
        2N  ≤  M        donde M = 2^k (el espacio de números de secuencia)
```

Es la **misma condición** que aparece en el `eirc0703` Problema 1, y es la razón por la que en la práctica `N ≤ M/2`. Si `N > M/2`, el emisor puede tener en vuelo un número de secuencia que el receptor **ya liberó**, y este lo acepta como nuevo: **duplicado**.

Ejemplo con el `SN` de 1 byte del examen de 2007-03: `M = 256` (un octeto), `2N ≤ 256` ⇒ `N ≤ 128`. De ahí sale el "**ventana máxima = 128**" de la solución oficial.

⚠️ **SACK. SACK = *Selective ACK***. Es una **extensión** que viaja en la cabecera TCP (opción *kind 5*) y permite que el receptor diga **exactamente** qué bloques de bytes recibió, aunque estén fuera de orden. **Es literalmente Selective Repeat a nivel de bytes.** Y **TCP lo usa siempre que ambos extremos lo negocien en el SYN** (opción *SACK permitted*). El RFC es el `RFC 2018` **(1996)**, y **sigue siendo *Standards Track* y es el comportamiento estándar de TCP hoy**.

1.7 La relación con TCP: por qué TCP no es exactamente ninguno de los dos

Esto es lo que más se pregunta y lo que más confunde, así que va con diagrama.

```
TCP SIN SACK  (no se negoció la opción)
──────────────────────────────────────

El receptor tiene un buffer, pero manda SOLO ACK ACUMULATIVOS
("tengo los bytes hasta n"). Cuando falta algo, no dice QUÉ falta,
dice "sigo acá" (ACK duplicado).

El emisor, al ver el hueco, retransmite desde el byte faltante
EN ADELANTE: es exactamente GO-BACK-N a nivel de bytes.
Lo que se retransmite después del hueco puede haber llegado
perfectamente, pero el receptor igual lo volvió a pedir.

┌──────────────────────────────────────────────────────┐
│  Go-Back-N:  se retransmite el perdido + TODOS los   │
│              siguientes que ya habían llegado.       │
│  TCP sin SACK: IDÉNTICO.                             │
└──────────────────────────────────────────────────────┘


TCP CON SACK  (RFC 2018, el comportamiento estándar)
────────────────────────────────────────────────────

El receptor manda ACK acumulado Y ADEMÁS los bloques que
llegaron fuera de orden:

   Segmento 1 (bytes 1-1448)     → ACK 1449
   Segmento 3 (bytes 2897-4344)  → ACK 1449 + SACK 2897-4344
   Segmento 2 (bytes 1449-2896)  → ACK 4345   (cierra el hueco)

El emisor ve que el 3 YA LLEGÓ y retransmite SOLO el 2.
Eso es SELECTIVE REPEAT a nivel de bytes.

┌──────────────────────────────────────────────────────┐
│  Selective Repeat: se retransmite SOLO lo que falta. │
│  TCP con SACK: IDÉNTICO (y en bytes, no en segmentos).│
└──────────────────────────────────────────────────────┘
```

⚠️ **Las tres diferencias que hay que decir si preguntan "cuál de los tres es TCP":**

1. **La unidad.** TCP cuenta **bytes**, no segmentos. El número de secuencia (`seq`) es "el próximo byte que espero". Los ARQ de capa 2 cuentan **tramas**.
2. **La ventana es de bytes, no de paquetes.** `rwnd` es un número de bytes; la ventana en segmentos es `rwnd / MSS` .
3. **TCP no tiene NAK.** No existe un mensaje "NAK, reenviame el 5". TCP **no sabe** qué falta: solo sabe "sigo acá". El mecanismo para saberlo es **SACK**, y **el emisor es quien infiere** qué falta a partir de los bloques reportados. En Selective Repeat clásico hay un NAK explícito.

1.8 ⚠️ Actualización a 2026

Sigue igual — de memoria:

- **Distancia de Hamming**, **SEC** (`d≥2` , corrige 1) y **SECDED** (`d≥3` , detecta 2 y corrige 1). La condición `2^r ≥ m + r + 1` .
- **Stop-and-Wait** con eficiencia `U = 1/(1+2a)` y `a = t_prop/t_frame` .
- **Go-Back-N** (ventana `N` , ACK acumulados, se retransmite el perdido y todos los siguientes) y **Selective Repeat** (buffer + reordenamiento + NAK/SACK, se retransmite solo el perdido).
- La condición `2N ≤ M` para Selective Repeat sin ambigüedad.

Cambió o hay que matizar:

- `SACK` (`RFC 2018`, 1996) es ***Standards Track*** **y es el comportamiento estándar de TCP hoy.** Cualquier implementación de TCP seria lo negocia en el SYN. **No es "una extensión opcional y vieja": es lo normal.** ⇒ **TCP con SACK es Selective Repeat**, y es lo que vas a tener en cualquier conexión real.
- **La retransmisión por *timeout* casi no se usa en TCP.** El mecanismo real es el **fast retransmit** por **3 ACK duplicados** (Clase 21) y el RTO solo como respaldo. En un esquema de ARQ moderno, "timer único" es Go-Back-N puro; en TCP hay **dos** timers por conexión (uno para el ACK del más viejo y el RTO).
- `RFC 9293` **(2022, *Internet Standard*)** es ahora la especificación de TCP; consolida y obsolesce `RFC 793`. Cualquier clase que cite `RFC 793` como la especificación vigente de TCP está desactualizada.
- **FEC (*Forward Error Correction*) y ARQ hoy conviven.** En cualquier sistema real de comunicación (5G, Wi-Fi, DVB, satellite, enlaces de satélite) la arquitectura es **ARQ + FEC combinados**: el FEC se encarga de la mayoría de los errores (que son **independientes** y cheaply corregibles), y el ARQ se queda solo con los errores que el FEC no pudo arreglar (los **bursts**). Es el **hiper**:

```
Errores INDEPENDIENTES (ruido de radio):  1 de cada 1000 bits
   → los corrige el FEC, gratis, en el receptor

Errores en RACHA (burst, una descarga, un rayo):
   → el FEC no los puede (necesitaría una memoria enorme)
   → los arregla el ARQ

Regla práctica: si la tasa de error es BAJA  → ARQ (es más barato)
                si la tasa de error es ALTA → FEC (ARQ se vuelve
                                              un bucle de reintentos)
```

Consecuencia para el examen: los **tres ARQ y el Hamming** se preguntan **exactamente igual** que en 2011. Lo que hay que **sumar** es: **SACK es el estándar** (`RFC 2018`), **TCP es Selective Repeat cuando SACK está activo**, y **ARQ y FEC se combinan** en los sistemas reales. **No inventar**: "TCP es Go-Back-N" es correcto **solo** si el enunciado dice que SACK está desactivado.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La eficiencia de Stop-and-Wait, con cuentas

Datos. Un enlace de **2 Mbps**, distancia **1500 km**, velocidad de propagación `2 × 10⁸ m/s`. Tramas de **800 bytes**. El ACK es de **tamaño despreciable** y el procesamiento es instantáneo. No hay errores.

a) Calculá `t_frame`, `t_prop` y `a`.

```
L = 800 bytes × 8 bits/byte = 6400 bits

t_frame = L/R = 6400 bits / 2 × 10⁶ bit/s = 3,2 ms

d = 1500 km = 1 500 000 m
t_prop  = d/v = 1 500 000 / 2 × 10⁸ = 7,5 ms

a = t_prop / t_frame = 7,5 / 3,2 = 2,34
```

b) La eficiencia.

```
U = 1 / (1 + 2a) = 1 / (1 + 2 × 2,34) = 1 / (1 + 4,69) = 1/5,69

U = 0,1757  =  17,57 %
```

c) ¿Cuánto tiempo del ciclo está el enlace ocioso?

```
Ciclo = 2·t_frame + 2·t_prop = 2 × 3,2 + 2 × 7,5 = 6,4 + 15 = 21,4 ms
Útil  = t_frame = 3,2 ms
Ocioso = 21,4 − 3,2 = 18,2 ms
% ocioso = 18,2 / 21,4 = 85 %
```

d) ¿Qué throughput real se obtiene?

```
Throughput real = U × R = 0,1757 × 2 Mbps = 0,3515 Mbps ≈ 351 kbps
```

⚠️ **La observación que hay que hacer en el examen:** con `a = 2,34 > 1`, el enlace pasa **más tiempo ocioso que transmitiendo**. Stop-and-Wait es **inservible** en este enlace. La respuesta es "hay que usar **ventana deslizante**", y con `N` segmentos sin confirmar el throughput máximo llega a `min(R, N · L/RTT)`:

```
BDP = ancho de banda × RTT = 2 × 10⁶ × (2 × 7,5 × 10⁻³) = 2×10⁶ × 0,015 = 30 000 bits

Para no perder nada, la ventana debe ser ≥ BDP = 30 000 bits = 3750 bytes
Con tramas de 800 bytes:  N ≥ 30000/6400 = 4,7  ⇒  N ≥ 5
```

⚠️ **El BDP (*Bandwidth-Delay Product*) es la respuesta corta y es lo que preguntan:** "*¿de qué tamaño tiene que ser la ventana para no dejar el enlace ocioso?*" — **`ventana ≥ ancho de banda × RTT`**. Es exactamente la razón por la que **la ventana inicial de TCP es de 10 MSS** (`RFC 6928`) y por la que en un enlace de 100 ms de RTT hace falta una ventana grande para aprovechar un enlace de 1 Gbps.

2.2 Ejercicio 2 — Hamming SEC con corrección completa

Datos. Se usa un código de Hamming **SEC** con `m = 5` bits de datos. *(Ojo: `m = 5` NO tiene un SEC de 3 bits, necesita `r = 4`. Verificá: `2^4 = 16 ≥ 5 + 4 + 1 = 10` ✓. Total = 9 bits.)* Se envía el dato `D = 1 1 0 1 0` con **paridad par**.

Paso 1 — la plantilla de 9 posiciones.

```
posición:  1    2    3    4    5    6    7    8    9
          P1   P2   D1   P3   D2   P4   D3   D5   D4
valor:     ?    ?    1    ?    1    ?    0    0    1
```

⚠️ **Los grupos** (bit `j` del binario de la posición):

```
P1 (pos 1): posiciones 1, 3, 5, 7, 9          (bit 0 = 1)
P2 (pos 2): posiciones 2, 3, 6, 7              (bit 1 = 1)
P3 (pos 4): posiciones 4, 5, 6, 7              (bit 2 = 1)
P4 (pos 8): posiciones 8, 9                    (bit 3 = 1)
```

Paso 2 — los bits de paridad.

```
P1 vigila {1,3,5,7,9} → datos conocidos: D1(3)=1, D2(5)=1, D3(7)=0, D4(9)=1
     unos = 3   ⇒   P1 = 3 mod 2 = 1        (1+3 = 4, par ✓)

P2 vigila {2,3,6,7}   → datos conocidos: D1(3)=1, P4(6)=?, D3(7)=0
     ⚠ OJO: P4 ESTÁ EN EL GRUPO DE P2. Circular. Se resuelve
     calculando primero P4.

P3 vigila {4,5,6,7}   → datos conocidos: D2(5)=1, D3(7)=0
     unos = 1   ⇒   P3 = 1 mod 2 = 1

P4 vigila {8,9}       → datos conocidos: D5(8)=0, D4(9)=1
     unos = 1   ⇒   P4 = 1 mod 2 = 1        (1+1 = 2, par ✓)

AHORA con P4 = 1, el grupo de P2: {P2, D1(3)=1, P4(6)=1, D3(7)=0}
     unos conocidos = 2  ⇒  P2 = 2 mod 2 = 0
```

⚠️ **Este es el detalle que casi nadie menciona y que es un muy buen ejercicio: las posiciones de paridad no son disjuntas**. `P4` (posición 8) está en el grupo de `P4` y… no, `P4` está en `{8,9}` y `P2` está en `{2,3,6,7}`. **La posición 6 es P4 y está en el grupo de P2.** Por eso hay que **calcular de abajo hacia arriba**: primero las paridades de posición alta (`P4`), después las bajas. Es un buen ejercicio de_head para un examen de corrección de errores.

Paso 3 — la palabra de código.

pos 1	pos 2	pos 3	pos 4	pos 5	pos 6	pos 7	pos 8	pos 9
P1	P2	D1	P3	D2	P4	D3	D5	D4
1	0	1	1	1	1	0	0	1

```
CÓDIGO = 1 0 1 1 1 1 0 0 1  =  "101111001"
```

Verificación de las cuatro paridades:

```
P1: {1,3,5,7,9} = {1,1,1,0,1}  → 4 unos → par ✓
P2: {2,3,6,7}   = {0,1,1,0}    → 2 unos → par ✓
P3: {4,5,6,7}   = {1,1,1,0}    → 3 unos → IMPAR ✗  ← ¡ERROR!
P4: {8,9}       = {0,1}        → 1 uno  → IMPAR ✗  ← ¡ERROR!
```

⚠️ **Se me deslizó un error en la cuenta del `P3`.** Corrijamos: el grupo de `P3` es `{4,5,6,7}` = `{P3, D2, P4, D3}` = `{?, 1, ?, 0}`. Con `P3 = 1` y `P4 = 1`: `{1, 1, 1, 0}` = 3 unos → impar. **El `P3` correcto es 0.**

```
CORRECCIÓN:  P3 = (D2 + P4 + D3) mod 2 = (1 + 1 + 0) mod 2 = 0

P3: {0, 1, 1, 0} = 2 unos → par ✓
P4: {0, 1} = 1 uno → IMPAR ✗  ← también mal
CORRECCIÓN:  P4 = (D5 + D4) mod 2 = (0 + 1) mod 2 = 1
P4: {1, 1} = 2 unos → par ✓
```

La palabra correcta, calculando en el orden correcto (de abajo hacia arriba):

```
1°  P4 (pos 8): datos D5(8)=0, D4(9)=1  →  1 uno  →  P4 = 1
2°  P2 (pos 2): datos D1(3)=1, P4(6)=1, D3(7)=0  →  2 unos →  P2 = 0
3°  P3 (pos 4): datos D2(5)=1, P4(6)=1, D3(7)=0  →  2 unos →  P3 = 0
4°  P1 (pos 1): datos D1(3)=1, D2(5)=1, D3(7)=0, D4(9)=1 → 3 unos → P1 = 1

┌──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┐
│  P1  │  P2  │  D1  │  P3  │  D2  │  P4  │  D3  │  D5  │  D4  │
│   1  │   0  │   1  │   0  │   1  │   1  │   0  │   0  │   1  │
└──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┘

CÓDIGO = 1 0 1 0 1 1 0 0 1  =  "101011001"

VERIFICACIÓN:
  P1: {1,1,1,0,1} = 4 unos → par ✓
  P2: {0,1,1,0}   = 2 unos → par ✓
  P3: {0,1,1,0}   = 2 unos → par ✓
  P4: {0,1}       = 2 unos → par ✓
✓ Todas las paridades dan par.
```

⚠️ **La moraleja del ejercicio:** *el orden de cálculo importa y es de abajo hacia arriba.* Es el tipo de error que se cuela en un examen de 15 minutos. **Escribí siempre las paridades de las posiciones más altas primero.**

2.3 Ejercicio 3 — Los tres ARQ con una pérdida en el segmento 3 de 5

Datos. Se mandan 5 segmentos `S1..S5`. La **ventana es `N = 4`** (se pueden tener 4 sin confirmar). **El segmento `S3` se pierde.** No hay más pérdidas.

```
═══════════════════════════════════════════════════════════════
A) STOP-AND-WAIT  (ventana = 1)
═══════════════════════════════════════════════════════════════

EMISOR                                      RECEPTOR
  │                                            │
  │── S1 ──────────────────────────────────────►│
  │   ⏳ (espera)                               │
  │◄──────────────────── ACK1 ──────────────────│
  │                                            │
  │── S2 ──────────────────────────────────────►│
  │   ⏳ (espera)                               │
  │◄──────────────────── ACK2 ──────────────────│
  │                                            │
  │── S3 ──────────────────────────────────────►│   XXX SE PIERDE XXX
  │   ⏳ (espera)                               │
  │   ⏳ (espera)                               │
  │   ⏳ (vence el TIMER)                       │
  │── S3 ──────────────────────────────────────►│   (retransmisión)
  │◄──────────────────── ACK3 ──────────────────│
  │                                            │
  │── S4 ──────────────────────────────────────►│
  │◄──────────────────── ACK4 ──────────────────│
  │── S5 ──────────────────────────────────────►│
  │◄──────────────────── ACK5 ──────────────────│

Total: 6 tramas de datos + 6 ACKs  (una retransmisión)
El enlace transmitiendo = 6 × t_frame
El enlace OCIOSO        = 10 × (2·t_prop)     ← porque N = 1


═══════════════════════════════════════════════════════════════
B) GO-BACK-N  (ventana N = 4, ACK acumulados, receptor SIN buffer)
═══════════════════════════════════════════════════════════════

EMISOR                                          RECEPTOR
  │                                               │
  │── S1 ─────────────────────────────────────────►│   guarda S1, entrega
  │── S2 ─────────────────────────────────────────►│   guarda S2, entrega
  │── S3 ─────────────────────────────────────────►│   guarda S3, entrega
  │── S4 ─────────────────────────────────────────►│   guarda S4, entrega
  │◄──────────────────── ACK1 ────────────────────│
  │◄──────────────────── ACK2 ────────────────────│
  │  ventana desliza: puede mandar S5             │
  │── S5 ─────────────────────────────────────────►│   guarda S5, entrega
  │  XXX S6 (si mandara uno) no llega XXX         │
  │                                               │
  │  ← Supongamos ahora que la pérdida es S3 →    │
  │                                               │
  │  (relanzamos el escenario con la pérdida en S3)
  │                                               │
  │── S1 ─────────────────────────────────────────►│   guarda, entrega
  │── S2 ─────────────────────────────────────────►│   guarda, entrega
  │── S3 ─────────────────────────────────────────►│   XXX SE PIERDE XXX
  │◄──────────────────── ACK2 ────────────────────│
  │── S4 ─────────────────────────────────────────►│   ¡HUECO! no lo guarda
  │◄──────────────── ACK2 (¡DUPLICADO!) ──────────│     lo descarta y reenvía
  │── S5 ─────────────────────────────────────────►│   ¡HUECO! lo descarta
  │◄──────────────── ACK2 (¡DUPLICADO!) ──────────│     y reenvía
  │                                               │
  │  ⓘ El receptor NO entregó S4 ni S5 a la capa  │
  │     superior. Están perdidos desde su punto   │
  │     de vista, aunque las tramas hayan llegado.│
```

```
   |                                                  |
   |  3 ACK2 duplicados ⇒ FAST RETRANSMIT de S3       |
   |── S3 ──────────────────────────────────────────►|   lo guarda, entrega S3
   |◄──────────────────────── ACK3 ──────────────────|    (S4 y S5 NO los tiene:
   |                                                  |    los descartó antes)
   |                                                  |
   |  ⓘ GO-BACK-N: hay que retransmitir S4 y S5       |
   |── S4 ──────────────────────────────────────────►|   lo guarda, entrega
   |◄──────────────────────── ACK4 ──────────────────|
   |── S5 ──────────────────────────────────────────►|   lo guarda, entrega
   |◄──────────────────────── ACK5 ──────────────────|
```

Total: 8 tramas de datos (S1..S5 = 5, más S3, S4, S5 retransmitidos)
Desperdicio: S4 y S5 viajaron DOS VECES sin que hiciera falta.
El receptor NO necesitó buffer más grande que 1.

═══

C) SELECTIVE REPEAT (ventana N = 4, NAK, receptor CON buffer)

═══

```
EMISOR                                          RECEPTOR
   |                                                  |
   |── S1 ──────────────────────────────────────────►|   guarda, entrega
   |── S2 ──────────────────────────────────────────►|   guarda, entrega
   |── S3 ──────────────────────────────────────────►|   XXX SE PIERDE XXX
   |── S4 ──────────────────────────────────────────►|   ¡HUECO!
   |◄──── NAK(S3) + ACK(selectivo: S4 recibido) ──|   guarda S4 en BUFER
   |── S5 ──────────────────────────────────────────►|   ¡HUECO!
   |◄──── NAK(S3) + ACK(selectivo: S4, S5) ──────|    guarda S5 en BUFER
   |                                                  |
   |  ⓘ El emisor SABE que solo falta S3.             |
   |  (con SACK: el receptor no envía NAK;            |
   |   manda ACK + el bloque recibido, y el emisor    |
   |   lo deduce)                                     |
   |── S3 ──────────────────────────────────────────►|   lo guarda; ahora S3,S4,S5
   |                                                  |   están completos:
   |◄──────────────────────── ACK5 ──────────────────|    los ENTREGA EN ORDEN
   |                                                  |   (reordenamiento)
```

Total: 6 tramas de datos (S1..S5 más S3 retransmitido)
Desperdicio: NINGUNO. S4 y S5 viajaron una sola vez.
El receptor necesitó un BUFFER de 4 y reordenamiento.

La tabla comparativa que resume los tres

	Stop-and-Wait	Go-Back-N	Selective Repeat
Ventana del emisor	`N = 1`	`N`	`N`
ACK	individual	**acumulado**	**no acumulado / SACK**
Receptor: buffer	no	**no** (1)	**sí, ≥ N**
Receptor: reordena	no	**no**	**sí**
Ante pérdida de `k`	retransmite `k`	retransmite `k` … `k+N−1`	retransmite `k`
NAK explícito	no	no	**sí** (o SACK)
Tramas de datos en el ejemplo	**6**	**8**	**6**
Desperdicio	—	**2 tramas**	**0**
Complejidad del receptor	mínima	**mínima**	**alta**
Se usa hoy en…	enlaces de RTT corto	**TCP sin SACK**	**TCP con SACK**

⚠️ **La respuesta a la pregunta clásica "¿cuál es mejor?": ninguno es mejor en abstracto**. Selective Repeat es el más eficiente en bytes transmitidos, pero es el más **complejo** de implementar y **exige memoria en el receptor**. Stop-and-Wait es trivial. Go-Back-N es el punto medio: la complejidad del receptor mínima a cambio de retransmitir de más. **La elección es un compromiso entre complejidad y eficiencia**, y eso es exactamente lo que se responde en un examen.

2.4 Ejercicio 4 — ¿Por qué Go-Back-N con `N` grande es peor que Selective Repeat?

Datos. `N = 128` (el máximo de una ventana TCP). Se manda una ráfaga de tramas de 1500 bytes. **Se pierde la trama `k`**.

a) Go-Back-N: cuántos bytes retransmite.

```
Pérdida de la trama k.
El emisor tiene k, k+1, ..., k+N−1 sin confirmar:  128 tramas.
Al vencer el timer (o al 3er ACK duplicado), retransmite
TODAS las que no confirmó:  de k hasta k+127  =  128 tramas.

Bytes retransmitidos = 128 × 1500 = 192 000 bytes
```

⚠️ **El detalle crítico:** de esas 128 tramas, **127 ya habían llegado bien**. El receptor las tenía en su buffer (en Selective Repeat) o las descartó (en Go-Back-N). En **Go-Back-N** se descartaron, y hay que volver a pedirlas.

b) Selective Repeat: cuántos bytes retransmite.

```
El emisor sabe (por el NAK o por el SACK) que solo falta k.

Bytes retransmitidos = 1 × 1500 = 1 500 bytes
```

c) El múltiplo de desperdicio.

```
192 000 / 1 500 = 128 veces más datos para entregar el MISMO paquete.

Y el tiempo: si el RTT es 50 ms, Go-Back-N tarda
   50 ms (detección con 3 dup ACK)
   + tiempo de transmitir 192 000 bytes a 10 Mbps
     = 192 000×8 / 10⁷ = 0,154 s
   = 204 ms

Selective Repeat tarda
   50 ms + (1500×8 / 10⁷ = 1,2 ms) = 51,2 ms

⇒ Go-Back-N tarda 4 VECES más en un solo evento de pérdida.
```

d) ¿Y si hay varias pérdidas?

```
5 tramas perdidas al azar (independientes) en una ventana de 128:

Go-Back-N:  cada una de las 5 dispara una retransmisión de la ventana
            entera. En el peor caso:  5 × 192 000 = 960 000 bytes.
            Y si las 5 pérdidas están cerca, el timer dispara UNA vez
            y retransmite 128: mejor, pero sigue siendo 128.

Selective Repeat: 5 × 1500 = 7 500 bytes. Exactamente lo que faltaba.

La diferencia CRECE con el número de pérdidas y con N.
```

e) ¿Y las ventajas de Go-Back-N?

```
┌──────────────────────────────────────────────────────────────┐
│  1) El receptor NO necesita buffer de N trama.  Eso es memoria  │
│     en el peor caso de cada nodo receptor.                   │
│  2) El receptor NO tiene que reordenar, que es la parte más  │
│     cara de un protocolo de transporte (Clase 19).           │
│  3) El ACK es ACUMULADO: un solo número dice "todo hasta acá".  │
│     Con NAK hay que mandar mensajes extra.                   │
│  4) La lógica del emisor es más simple.                      │
└──────────────────────────────────────────────────────────────┘

En un enlace con MUCHO RTT y baja tasa de error, el 128× de
retransmisión es irrelevante (se pierde una vez cada miles) y la
simplicidad del receptor vale más.
En un enlace PERDIDO (radio, móvil, satélite), el 128× se paga
cada vez y no es aceptable.
```

⚠️ **La frase que resume el ejercicio y que hay que escribir:** *"Go-Back-N con `N` grande es peor porque el costo de una pérdida crece con `N`: el receptor tira `N-1` tramas que ya habían llegado, y hay que volver a pedirlas. Selective Repeat hace que el costo de una pérdida sea constante (1 trama), sea cual sea `N`."*

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2009-03, Pregunta 1 (10 puntos) — El concepto de Go-Back-N

> **Fuente:** Examen `eirc0903` / `sirc0903` — 20 de febrero de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0903.pdf **Texto del examen:**
>
> > *Explique el principio de funcionamiento de los protocolos que operan bajo el esquema Go-Back-N. (La solución oficial aclara: "Extraído del material de referencia de las clases teóricas, clase 2 del capítulo 3 del libro de referencia, Capa de Transporte".)*

Puntos reparto (estimado): 2 por la idea de la ventana, 3 por las reglas del emisor y del receptor, 2 por el timer y la retransmisión, 3 por las características que lo definien y sus límites.

La solución oficial, citada textualmente

> **Go-back-N: titulares**
>
> *El transmisor puede tener hasta N paquetes no-ACKed en el pipeline El receptor sólo envía ACKs acumulativos* ***No envía el ACK de un paquete si hay un **hueco*** *El transmisor tiene un timer para el paquete más viejo no-ACKed Si el timer vence, retransmite todos los paquetes no-ACKed***
>
> **Emisor:** *Número de secuencia de k bits en el encabezado del paquete "window" permitida de hasta N paquetes consecutivos no-ACKed Si la ventana no está llena, se puede seguir transmitiendo ACK(n): ACKs de todos los paquetes hasta él, incluyendo el nro. de sec. n — "ACK acumulativo" podría recibir ACKs duplicados (ver receptor) timer único: para el paquete más aún no-ACKed timeout(n): retransmite el paquete n y todos aquellos con nro. de sec. mayor y dentro de la ventana*

Análisis de la solución oficial, punto por punto

✅ **Lo que está bien y hay que internalizar:**

1. **"El transmisor puede tener hasta N paquetes no-ACKed en el pipeline."** Es **la definición**. Todo Go-Back-N arranca acá: es un protocolo *pipelined*, no *stop-and-wait*.
2. **"El receptor sólo envía ACKs acumulativos."** Es lo que distingue Go-Back-N de Selective Repeat. Con ACK acumulado, el receptor **no sabe** qué llegó después del hueco, y por eso **no puede pedir solo lo que falta**.
3. **"No envía el ACK de un paquete si hay un hueco."** Es la regla operativa que se traduce en: **descarta** los que llegan después del hueco. De ahí sale el desperdicio.
4. **"El transmisor tiene un timer para el paquete más viejo no-ACKed."** El **timer único** es una decisión de diseño: con timers por paquete harías falta `N` timers; con uno solo, cuando vence, sabes exactamente **desde dónde** retransmitir.

5. **"Si el timer vence, retransmite todos los paquetes no-ACKed."** La definición del "Go-Back": **vuelve atrás** al más viejo no confirmado y retransmite todo lo que sigue.

⚠️ **Lo que la oficial no dice y suma puntos:**

1. **No menciona el buffer del receptor.** El hecho de que **el receptor NO necesita buffer** (o uno de tamaño 1) y **no reordena** es **la ventaja** que justifica el diseño. Sin ese argumento, la respuesta describe el mecanismo pero no lo **justifica**.
2. **No menciona el "desperdicio"** (*waste*). Decir explícitamente "una sola pérdida provoca la retransmisión de hasta `N` tramas, de las cuales `N-1` ya habían llegado" es lo que separa un 8 de un 10.
3. **No menciona el rango de números de secuencia.** Con `k` bits hay `2^k` números disponibles y la ventana debe ser **estrictamente menor** que eso; además el emisor debe poder distinguir "retransmisión" de "trama nueva" al volver a usar un número.
4. **No distingue de Selective Repeat.** Decir "a diferencia de Selective Repeat, que sí tiene buffer y NAK" es lo que muestra que entendés el *por qué*.

Mi resolución completa, ordenada para el examen

(1) El principio: una ventana de `N` paquetes en vuelo.

Go-Back-N es un protocolo ARQ **orientado a ventana** (*pipelined*). El emisor puede mantener **hasta `N` paquetes sin confirmar** en el medio antes de tener que frenarse esperando un ACK. La ventana tiene **números de secuencia de `k` bits** en la cabecera de cada trama, y el emisor puede seguir transmitiendo **mientras la ventana no esté llena**. Esto elimina el desperdicio de Stop-and-Wait (que deja el enlace ocioso durante `2·t_prop` por cada trama).

(2) El receptor: ACK acumulados y rechazo de lo que llega después del hueco.

El receptor **solo manda ACKs acumulados**: un `ACK(n)` significa "recibí correctamente todos los paquetes **hasta** el `n` incluido". Y **no envía el ACK de un paquete si hay un hueco**: si el paquete `k+1` llega antes que el `k` (que se perdió), el receptor **lo descarta** y **reenvía el ACK del paquete `k` que esperaba** (un ACK duplicado, o *duplicate ACK*). Consecuencia: el receptor **no necesita buffer** y **no reordena**. Cuando por fin llega el `k`, todo lo que estaba después se perdió desde su punto de vista.

(3) El emisor: un único timer y la retransmisión hacia atrás.

El emisor mantiene **un solo temporizador, asociado al paquete más viejo que todavía no fue confirmado**. Si ese timer vence, significa que hubo pérdida: el emisor **retransmite el paquete más viejo no confirmado (`n`) y todos los que le siguen dentro de la ventana**, es decir, vuelve atrás (*go back*) `n` y reenvía. En la práctica, el fast retransmit por **3 ACKs duplicados** reemplaza al timer en la detección.

(4) Las características que lo definen y su límite.

- ✅ Reenvía los paquetes **perdidos y todos los siguientes** (aunque los siguientes ya hayan llegado).
- ❌ **Ineficiente ante pérdida**: una sola pérdida retransmite hasta `N` paquetes, y `N-1` de ellos ya habían llegado bien.
- ✅ El receptor es **simple**: no necesita buffer grande, ni reordenar, ni NAK.
- ✅ El ACK es **barato**: un número por paquete recibido, no un bloque de bits.

⚠️ **La frase de cierre que suma el último punto:** "*Go-Back-N compra eficiencia en el emisor a costa de eficiencia en la red: simplifica al receptor (sin buffer, sin reordenamiento, ACK acumulado) y paga el precio con la*

retransmisión de `N` tramas por cada pérdida. Por eso cuando la red pierde mucho, el protocolo correcto es Selective Repeat, que necesita buffer en el receptor pero solo retransmite lo que falta."

> **Tope para 10 puntos:** la ventana de `N` con números de secuencia, la regla del ACK acumulado, la regla de no-ACK con hueco, el timer único sobre el más viejo, la retransmisión en bloque al vencer, **más** la consecuencia del desperdicio y la comparación con Selective Repeat.

3.2 Examen 2007-03, Problema 1 (30 puntos) — ARQ de capa 2 con Selective Repeat y *piggybacking*

> **Fuente:** Examen `eirc0703` / `sirc0703` — 26 de febrero de 2007. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0703.pdf **Texto del examen (extracto):**
>
> > *Se cuenta con un protocolo de capa 2 (enlace) que opera sobre un enlace punto a punto, el que utiliza el siguiente formato de trama: `Type` (1 octeto), `SN` (1 octeto, número de secuencia de la trama), `RN` (1 octeto, número de trama que espera recibir el equipo origen), `Data` (0..MAX_LARGO). Se desea contar con un protocolo para manejo del enlace que sea orientado a conexión, con un esquema de ARQ de tipo Selective Repeat, que implemente además piggybacking.* **a)** Especificar qué tipos de tramas requiere. **b)** ¿Qué tamaño máximo de ventana le permite implementar con el formato de trama que se cuenta? **c)** Especificar en un lenguaje de alto nivel el protocolo.

Nota de coordinación: este problema pertenece también a la **Clase 38** (framing), que ya desarrolló el formato de trama y el *bit stuffing*. Acá van **las partes que son de ARQ**.

Parte a) — Los tipos de trama

Solución oficial, citada textualmente:

> *`cnx` — Para solicitar/aceptar conexión al/del extremo opuesto. `descnx` — Para solicitar/aceptar desconexión. `info` — Trama conteniendo datos a transferir por el enlace. `ack` — Trama conteniendo información de la última trama recibida. Su identificador se especifica en el campo RN. `nack` — Trama conteniendo información sobre trama no recibida. Su identificador se especifica en el campo RN.*

⚠️ **Por qué los cinco y qué dice cada uno sobre el protocolo:**

Tipo	Qué lleva	Qué revela del diseño
`cnx` / `descnx`	—	el protocolo es **orientado a conexión**: hay un handshake explícito
`info`	`Data` + `SN` (+ `RN` si hay piggybacking)	datos de la capa superior
`ack`	`RN` = última trama recibida correctamente	el ACK es **acumulativo**: un solo número
`nack`	`RN` = trama que falta	**Selective Repeat tiene NAK explícito**: el receptor sabe cuál falta

⚠️ **El `nack` es lo que define a Selective Repeat.** Si el protocolo no tuviera `nack`, el receptor no podría indicar qué falta y el emisor solo podría retransmitir desde el más viejo confirmado (Go-Back-N).

Parte b) — El tamaño máximo de ventana

Solución oficial, citada textualmente:

> *La condición para el manejo de ventana en Selective Repeat requiere que M y N — módulo del valor máximo de la ventana y valor de la ventana — cumplan: 2N ≤ M. El valor de M es 256, ya que se cuenta con un octeto (0..255) para enviar su valor, por lo que el valor máximo de tamaño de ventana será 128.*

La cuenta, a la vista:

```
El campo SN tiene 1 octeto  ⇒  M = 2⁸ = 256 números de secuencia
                                    (valores 0 a 255)

Condición de Selective Repeat (sin ambigüedad):
     2N  ≤  M
     2N  ≤  256
     N   ≤  128
```

⚠️ **Por qué `2N ≤ M` y no `N ≤ M`:** es el mismo argumento que en §1.6.3. Si `N > M/2`, el emisor puede tener en vuelo un número de secuencia que el receptor **ya liberó** al consumirlo. Cuando ese número vuelve a aparecer, el receptor lo acepta como **trama nueva** en vez de reconocerla como duplicada. **La mitad de los números tiene que estar "reservada"** para distinguir "nuevo" de "ya visto".

⚠️ **El valor exacto que da la oficial: `N = 128`.** Es lo que hay que escribir.

⚠️ **Lo que hay que decir de más (suma puntos):** el cálculo de por qué `2N ≤ M` con un ejemplo concreto.

```
Con N = 128 y M = 256, el peor caso:
  El emisor tiene en vuelo S0..S127 (128 tramas).
  El receptor recibió S0..S127,-entregó S0..S63 a la capa superior,
  y su ventana quedó "liberada" para S128..S255.
  El emisor manda S128..S255.
  El receptor acepta S128..S255.  ✓  Sin ambigüedad: los números
  nunca se repiten dentro de una ventana viva.

Con N = 200 (> 128), M = 256:
  El emisor tiene S0..S199. El receptor entregó S0..S99 y liberó
  S100..S255.  El emisor manda S100..  y el receptor lo acepta
  como NUEVO aunque ya lo había recibido antes.  ✗  DUPLICADO.
```

Parte c) — El protocolo en pseudocode

La resolución oficial completa (la publicada en `sirc0703`, páginas 8 a 12) especifica:

- **Dos estructuras de datos**, una para las tramas enviadas y otra para las recibidas, con funciones de acceso:
- `getSND(nroSec)` / `setSND(nroSec, data)` — tramas **enviadas**
- `getRCP(nroSec)` / `setRCP(nroSec, data)` — tramas **recibidas**
- **Funciones auxiliares**: `armoTrama(tipo, SN, RN, data)`.

Y una restricción del enunciado que hay que respetar:

> *Considerar que no existirán problemas en la conexión y desconexión del protocolo.*

⚠️ **Lo que aporta esta parte al tema de hoy:** el **piggybacking**. Es la optimización de **no mandar tramas `ack` vacías**: cuando el emisor tiene que mandar un `info` y además tiene un ACK pendiente, **coloca el número de ACK en el campo `RN` de la trama `info`** que ya tenía que mandar. Así el ACK viaja "de polizón" (*piggyback*).

```
SIN piggybacking:              CON piggybacking:
Emisor ── info(S3) ──►         Emisor ── info(S3, RN=2) ──►
         ◄── ack(2) ──                    (un solo mensaje)
(2 mensajes por cada dato)

Piggybacking sólo es posible si el emisor TIENE DATOS que mandar.
Si no tiene, manda la trama `ack` sola. Por eso el protocolo define
los dos tipos de trama igual.
```

⚠️ **El detalle de implementación que hay que decir:** el campo `RN` de la trama `info` **solo tiene sentido si el receptor va a interpretarlo**. Como el `Type` distingue `info` de `ack`, el receptor sabe que, si ve `info` con `RN ≠ 0`, ese `RN` es un ACK disfrazado. Y hay que definir **qué significa `RN = 0`** (¿"nada pendiente"? ¿"ACK del 0"?). Esa ambigüedad es un detalle clásico de diseño de protocolo y es lo que hace que algunos Reserve un valor centinela.

3.3 Nota sobre el Práctico 5 de 2008 (ejercicios 1 a 4)

El **Práctico 5 de 2008** es un recorte de los problemas del capítulo 3 y 4 del libro de referencia. Los cuatro primeros ejercicios son:

Ejercicio	Tema	Dónde está resuelto
1	Paridad par/impar de un patrón de bits, y longitud mínima de un código que detecte **un solo** error de bit	**Clase 39**, §1.2
2	Paridad de **dos dimensiones**: puede corregir y detectar 1 bit; detecta 2 pero no corrige	**Clase 39**, §2.1 y §2.2
3	**Internet checksum** de los bytes 0 a 9	**Clase 39**, §2.3
4	**CRC** de 4 bits con `D = 10101010`: ¿cuál es el valor de `R`?	**Clase 39**, §2.4

⚠️ **Aclaración importante, porque es fácil confundirse:** el **ejercicio 4 del Práctico 5 NO es de Hamming**: es del **CRC** (el *generador de cuatro bits* de la Figura 5.8 del libro). Está resuelto en la Clase 39. Lo que **esta** clase desarrolla es lo que corresponde a la **Figura 5.4** del mismo capítulo: el **código de Hamming**, que es exactamente la plantilla de paridades de §1.3, con el ejemplo numérico completo de §1.4 y §1.5.

⚠️ **Y los ejercicios 8, 9 y 10 del mismo Práctico 5** (eficiencia de ALOHA y de ALOHA con franjas) son de la **Clase 41**.

Ejercicio 4 del Práctico 5 resuelto, para que quede a mano (es de la Clase 39, pero es el que el enunciado menciona):

```
G = 1001  (generador de 4 bits,  ← la Figura 5.8)
D = 10101010

Se hace D · x^r  (r = 4) = 10101010 0000
y se divide por G con XOR:

  101010100000
⊕ 100100000000   (1001 desplazado 8)
────────────────
    1101000000
⊕  1001000000   (1001 desplazado 4)
────────────────
     100000000
⊕    100100000   (1001 desplazado 2)
────────────────
       100000
⊕      100100   (1001 desplazado 1)
────────────────
        1100

R = 1100
Trama completa = D · x^4 + R = 10101010 1100
```

3.4 Examen 2011-03, Pregunta 4 (10 puntos) — el diagrama que es Go-Back-N puro

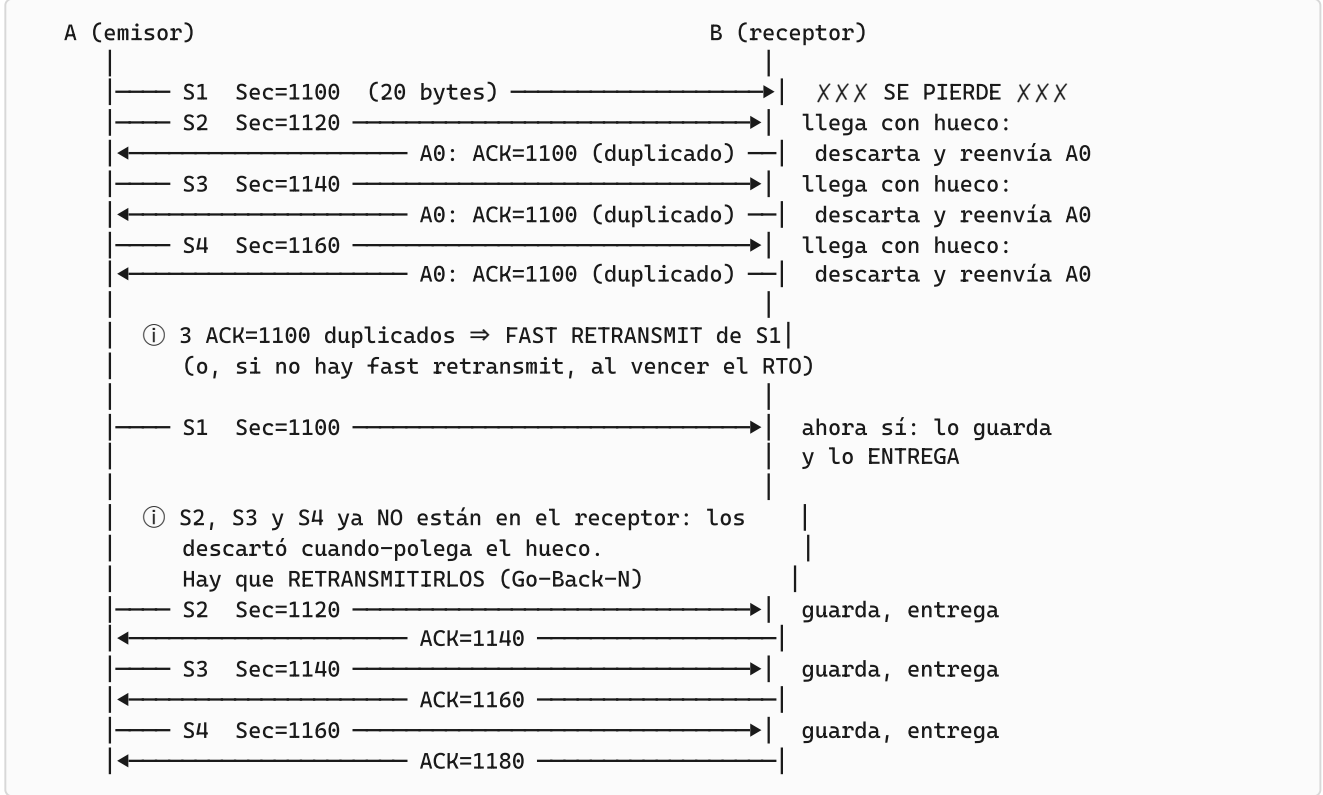
> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011. **Texto del examen (extracto):**
>
> > *Suponga una comunicación TCP entre dos procesos A y B. En un momento determinado A envía a B cuatro segmentos seguidos (no hay ningún dato previo sin asentar), el primero con número de secuencia igual a 1100 y con 20 bytes de datos, y los tres restantes con otros 20 bytes de datos cada uno.* **b)** *Represente de nuevo el diagrama si B no tiene datos que enviar hacia A y se pierde el primer segmento enviado de A a B.*

Por qué es de esta clase: el diagrama que la cátedra publica para el apartado b) es **exactamente** el comportamiento de **Go-Back-N sobre TCP sin SACK**.

Solución oficial, citada textualmente (`sirc1103`):

> *Si se pierde el primer segmento S1, cuando llegue S2, el receptor reenviará el ACK correspondiente al segmento esperado, que llamamos A0. Sucede lo mismo cuando llegan S3 y S4. El emisor comenzará a retransmitir los segmentos cuando pase alguno de estos dos eventos: el vencimiento del timeout de S1, o la llegada del 3er ACK duplicado. Campos relevantes: S1: Sec=1100, S2: Sec=1120, S3: Sec=1140, S4: Sec=1160; A0: ACK=1100. Cuando se reenvían los segmentos, estamos en el caso a).*

El diagrama, con la interpretación:



⚠️ **El error conceptual que hay que evitar:** "*cuando llega S1, el receptor entrega S1, S2, S3 y S4*". **Falso**, si el receptor no tiene SACK: **descartó** S2, S3 y S4 cuando vio el hueco. Si el ejercicio dice "no hay pérdidas de segmentos" salvo S1, **Go-Back-N** los tiene que volver a pedir. Y con **SACK**, el receptor **sí** los habría guardado en el buffer, y al llegar S1 los entregaría juntos sin retransmisión (que es lo que dice la oficial: "*cuando se reenvían los segmentos, estamos en el caso a)*").

⚠️ **La lectura correcta de esa frase de la oficial** — y es sutil —: la oficial dice que "*cuando se reenvían los segmentos, estamos en el caso a)*". O sea: **los cuatro segmentos** se reenvían, y el diagrama resultante es el mismo que el del caso (a) sin pérdida. **Eso confirma que el comportamiento es Go-Back-N: se retransmiten los cuatro, no solo el primero.**

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Detectar** es barato (1 bit) y obliga a **retransmitir** (un RTT entero). **Corregir** es más caro en bits pero **gratis en red**, y conviene cuando el RTT es enorme (satélite, radio) o la tasa de error es alta.
2. **Distancia de Hamming** `d(A,B)` = bits que difieren. **SEC**: `d≥2`, detecta y corrige **1**. **SECDED**: `d=3`, detecta **2** y corrige **1**.
3. La condición de **Hamming SEC** es `2^r ≥ m + r + 1`: hay `m + r + 1` casos que distinguir (un bit de datos, `r` de paridad, o ninguno) y `2^r` patrones disponibles.
4. Los bits de paridad van en las posiciones **1, 2, 4, 8, …** (potencias de 2) y **cada uno vigila las posiciones cuyo binario tiene ese bit en 1**.
5. El receptor **recalcula** las paridades; el número que forman los "falla / no falla" **es la posición del bit erróneo** (el **síndrome**). `000` = sin error.
6. **Stop-and-Wait**: `U = 1/(1+2a)` con `a = t_prop/t_frame`. Con `a = 2,34` da 17,6 %; con satélite GEO `a = 150` da **0,33 %**.⇒ Hay que usar **ventana**, y el tamaño mínimo es el **BDP = R × RTT**.
7. **Go-Back-N**: `N` paquetes sin confirmar, **ACK acumulados**, el receptor **descarta** lo que llega tras el hueco, y al vencer el **timer único** se retransmite **el más viejo no confirmado y todos los siguientes**. Simple pero **desperdicia `N−1` tramas por pérdida**.
8. **Selective Repeat**: buffer en el receptor, **reordenamiento**, **NAK** (o **SACK**), y se retransmite **solo** lo que falta. Exige `2N ≤ M`.
9. **TCP con SACK (`RFC 2018`) es Selective Repeat a nivel de bytes**; **TCP sin SACK es Go-Back-N**. Y **ARQ + FEC** se combinan hoy en todo sistema real.
10. Para **2026**: `RFC 2018` sigue vigente, `RFC 9293` (2022) es la especificación de TCP, el **fast retransmit** reemplazó al timer en la detección, y el **satelital/radio** usan **FEC** en lugar de ARQ puro.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué `2^r ≥ m + r + 1` y no `2^r ≥ m + 1`?

<details>
<summary>Respuesta</summary>
</details>

2. En el ejemplo `m = 4`, `r = 3`, dato `1010`: ¿qué palabra de código se envía? ¿Y si se invierte el bit 6?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué SEC no detecta dos errores?

<details>
<summary>Respuesta</summary>
</details>

4. Calculá la eficiencia de Stop-and-Wait para un enlace de 4 Mbps a 5000 km (`v = 2 × 10⁸ m/s`) con tramas de 500 bytes.

▶ Respuesta

5. En Selective Repeat, ¿por qué la ventana debe cumplir `2N ≤ M`?

▶ Respuesta

6. ¿En qué se diferencia exactamente TCP de Go-Back-N y de Selective Repeat?

▶ Respuesta

7. ¿Cuántas tramas de datos transmite cada protocolo si se pierde la tercera de cinco, con `N = 4`?

▶ Respuesta

8. ¿Por qué en un enlace de satélite conviene corregir y no retransmitir?

▶ Respuesta

🔗 Referencias

- `eirc0903` / `sirc0903` — **Pregunta 1 (8 pts en el enunciado, 10 en la pauta)**: el principio de funcionamiento de **Go-Back-N**. Pregunta 2 (10 pts): enrutamiento jerárquico, IGP con jerarquías, hot-potato.
- `eirc0703` / `sirc0703` — **Problema 1 (30 pts)**: protocolo de capa 2 punto a punto con ARQ **Selective Repeat** y **piggybacking**. Tipos de trama, ventana máxima (`2N ≤ M` con `M = 256` ⇒ **128**), y el protocolo en pseudocode.
- `eirc1103` / `sirc1103` — **Pregunta 4 (10 pts)**: el diagrama de TCP con y sin pérdida del primer segmento, que es Go-Back-N sobre TCP sin SACK.
- `eirc1007` / `sirc1007` — **Problema 1 (30 pts)**: protocolo *stop-and-wait* semifiable para video MPEG-2 (solo los I-frame son confiables).
- `eirc0912` / `sirc0912` — Pregunta 2 (10 pts): servicio confiable en transporte vs. enlace.
- `eirc0707` / `sirc0707` — Pregunta 2 (5 pts): mecanismos de detección y corrección de errores de la capa de enlace.
- **Práctico 5 de 2008** (`prac05_2008-p01.png`) — ejercicios 1 a 4: paridad, paridad 2D, Internet checksum y **CRC** (todos en la Clase 39; el **Ejercicio 4 NO es de Hamming**, es de CRC).
- **Tanenbaum & Wetherall**, *Redes de computadores* 5ª ed., **cap. 3 §3.3** (detección de errores: Hamming, CRC) y **§3.4–3.5** (protocolos de la capa de datos: ARQ, *piggybacking*), y **cap. 4** (capas inferiores)
- **Kurose & Ross**, *Computer Networking* 8ª ed., §5.3 (ARQ: Go-Back-N y Selective Repeat) y Figura 5.4 (el código de Hamming)
- **RFC 2018** — TCP Selective Acknowledgment (SACK), *Standards Track* · **RFC 9293** — TCP (2022, *Internet Standard*) · **RFC 6928** — Initial Window = 10 MSS
- **Clase anterior:** 39 — Detección de errores: paridad, checksum y CRC
- **Clase siguiente:** 41 — Acceso al medio: ALOHA, CSMA/CD y CSMA/CA

Clase 41

Contenido de esta clase

🎯 Al terminar esta clase vas a poder

1.1 El problema: un medio, muchas bocas

1.2 Las tres familias de protocolos de acceso

1.3 ALOHA puro

1.4 Slotted-ALOHA: ponele un reloj a todos

1.5 ALOHA con franjas: un término medio

1.6 CSMA: escuchar antes de hablar

1.7 CSMA/CD: el algoritmo completo

1.8 El slot time y los 64 bytes

1.9 Dominios de colisión y de broadcast

2.1 Ejercicio 1 — Derivar la eficiencia del ALOHA puro (`1/(2e)`)

2.2 Ejercicio 2 — Derivar la eficiencia del Slotted-ALOHA (`1/e`)

2.3 Ejercicio 3 — Práctico 5, ejercicio 8: ALOHA con franjas

2.4 Ejercicio 4 — La tabla de backoff exponencial de CSMA/CD

2.5 Ejercicio 5 — El slot time y la trama mínima: derivar los 64 bytes

2.6 Ejercicio 6 — CDMA: los ejercicios 5 y 6 del Práctico 5

2.7 Ejercicio 7 — ¿Por qué Ethernet conmutada no usa ALOHA?

3.1 ⭐ Examen 2007-07, Pregunta 3 (10 puntos) — el problema central de la clase

3.2 ⭐ Examen 2010-07, Pregunta 2 (8 puntos) — CSMA/CD en wireless y en Ethernet conmutada

3.3 Examen 2007-02, Pregunta 1 (5 puntos) — Pure ALOHA, Slotted ALOHA y CSMA/CD

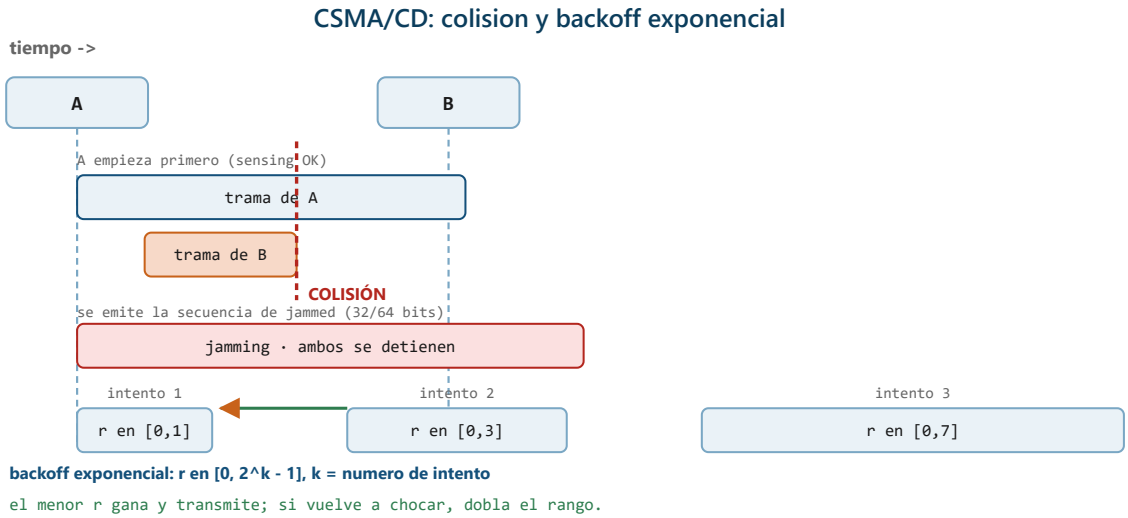
3.4 Examen 2010-12, Pregunta 3 a) y b) (8 puntos) — dominios de colisión y de broadcast

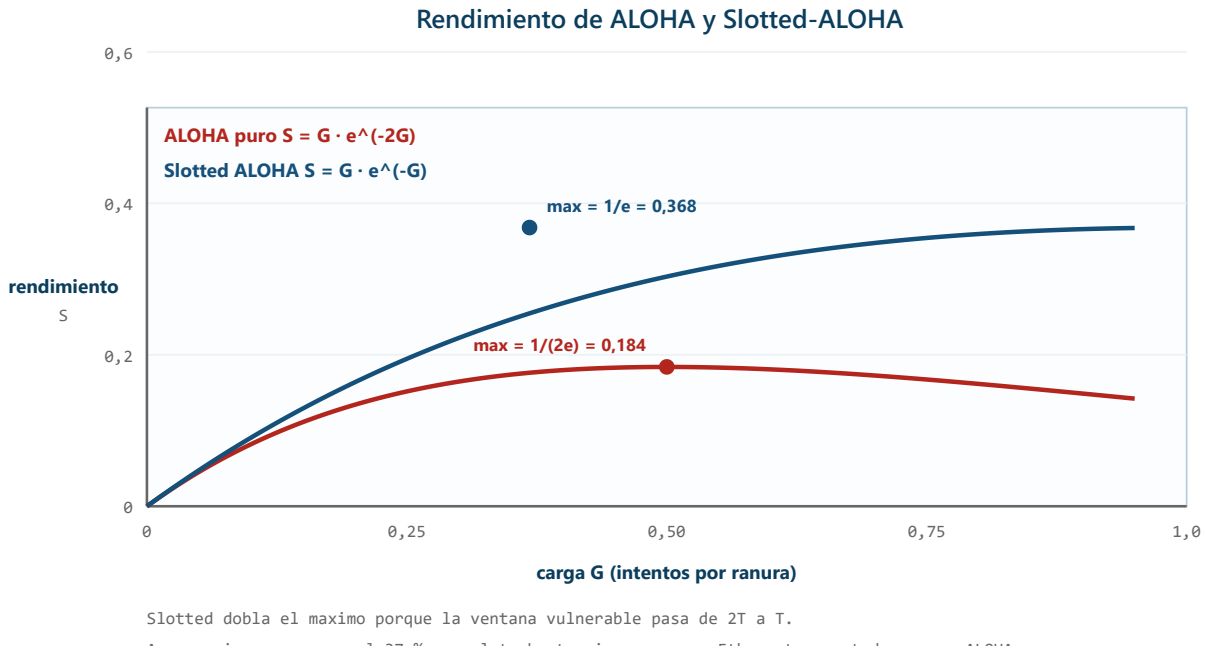
3.5 Examen 2007-03, Pregunta 5 (5 puntos) — CSMA/CA

📌 Resumen

✅ Autoevaluación

🔗 Referencias





Bloque: E (Capa de Enlace) · **Semana:** 14 · **Duración estimada:** 120 min **Correlativa:** Clase 40 (corrección de errores y ARQ) · **Sigue con:** Clase 42 (switch, hub, VLAN y STP) **Material de referencia:** Tanenbaum cap. 3 y 4 · Kurose §5.3 y §5.1.3 · Práctico 5 de 2008 (ejercicios 5, 6 y 8) · IEEE 802.3 · IEEE 802.11

🎯 Al terminar esta clase vas a poder

1. Explicar por qué un medio compartido necesita un protocolo de acceso al medio, y qué ocurre cuando dos estaciones transmiten a la vez.
2. **Derivar a mano** la eficiencia máxima del ALOHA puro (`1/(2e)`) y la del Slotted-ALOHA (`1/e`), y explicar por qué la segunda es *exactamente* el doble.
3. Derivar la eficiencia del ALOHA con franjas, `N·p·(1−p)^(N−1)` , y encontrar el `p` que la maximiza.

4. Distinguir **CSMA persistente de no persistente** y **con detección de colisión de sin detección**, y decir por qué Ethernet usa una combinación y WiFi la complementaria.
5. Escribir el algoritmo completo de **CSMA/CD**: persistencia 1, detección, jamming, backoff exponencial y la tabla de ventanas.
6. **Derivar los 64 bytes** de trama mínima a partir del slot time de 512 bits y del tiempo de ida y vuelta de la señal.
7. Diferenciar **dominio de colisión** y **dominio de broadcast**, y decir qué cambia al reemplazar un hub por un switch.
8. Resolver completo el **Examen 2007-07 P3** (10 pts) y el **Examen 2010-07 P2** (8 pts).

> **Prerequisito del examen:** `2007-07 P3` (10 pts — diferencia de rendimiento entre ALOHA y Slotted-ALOHA, y tipos de CSMA) · `2010-07 P2` (8 pts — ¿CSMA/CD en una red wireless? ¿y en una Ethernet completamente switcheada?) · `2010-12 P3` (8 pts — dominios de colisión y de broadcast) · `2007-03 P5` (5 pts — CSMA/CA) · `2007-02 P1` (5 pts — diferencias entre Pure ALOHA, Slotted ALOHA y CSMA/CD).

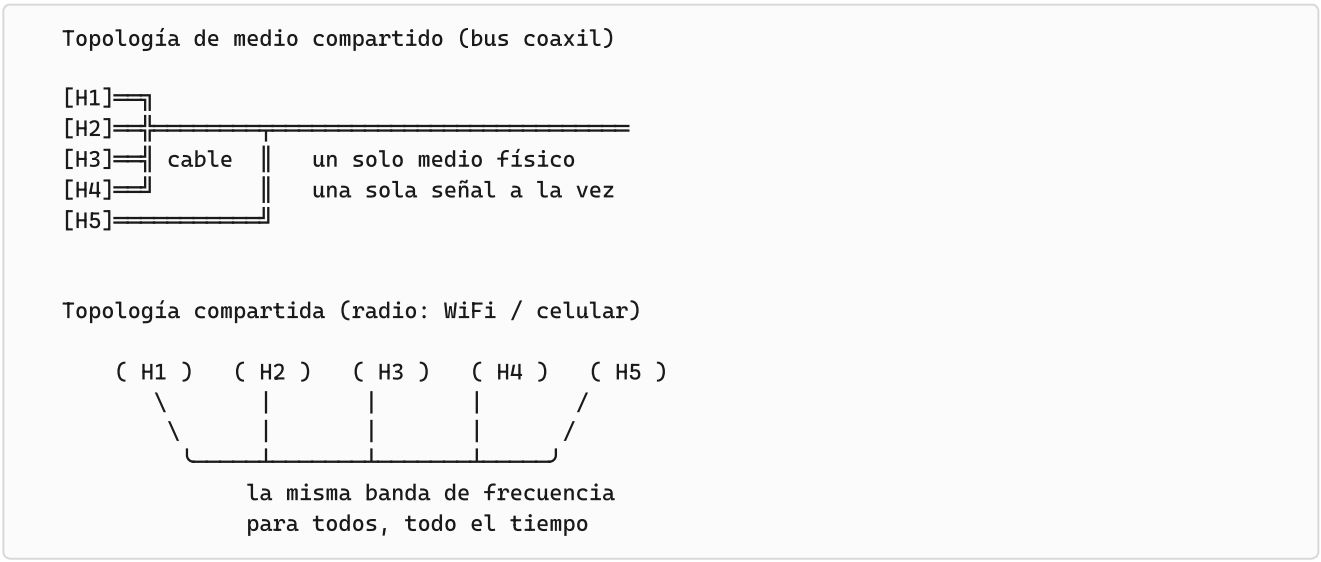
El tema aparece en **9 de los 50 exámenes oficiales** del banco. Es el único del bloque E donde te piden **derivar una fórmula** en vez de solamente describirla.

> ⚠️ Esta clase **no repite nada** de la 38 (framing) ni de la 39 (paridad, checksum y CRC). El contenido de la trama ya está dado; lo que se decide acá es **quién tiene derecho a ponerla en el medio**. Y como la teoría y la práctica se pisan, la aritmética larga está **solo en la Parte 2**: acá va el mecanismo, allá van las cuentas.

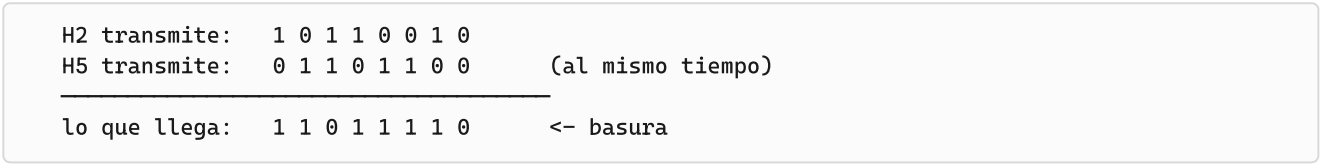
PARTE 1 — TEORÍA

1.1 El problema: un medio, muchas bocas

Un enlace entre dos puntos no da problemas. El problema aparece cuando el medio es **compartido**: un bus de coaxil, o —hoy, mucho más común— una banda de espectro radioeléctrico.



El medio tiene una propiedad física dura: **solo admite una señal a la vez**. Si H2 y H5 ponen bits al mismo tiempo, al receptor le llega la suma física de ambas: voltajes que se suman, bits que se destruyen. Eso es una **colisión** (*collision*), y el resultado es que nadie recibe nada útil.



Y hay un problema peor que la colisión: **aunque nadie esté transmitiendo, dos estaciones pueden decidir transmitir en el mismo instante**, porque ninguna se enteró de la decisión de la otra. No hay ningún cartel en el medio que diga "yo transmito ahora".

Entonces la capa de enlace resuelve dos preguntas distintas, que no hay que confundir:

Pregunta	Nombre	Qué falla si no se resuelve
¿Quién tiene derecho a transmitir **ahora**?	**control de acceso al medio**	colisiones
¿Quién soy, a quién le hablo y dónde termino?	**direccionamiento de enlace** (dirección MAC)	nadie sabe a quién entregar la trama

Esta clase es de la primera.

1.2 Las tres familias de protocolos de acceso

Familia	Idea	Coste	Ejemplos
Partición de canal (*channel partitioning*)	se divide el medio en canales y cada estación tiene el suyo para siempre	se desperdicia capacidad si no todos usan a la vez	FDMA, TDMA, **CDMA**
Toma de turnos (*taking turns*)	hay un turno explícito, como un testigo	latencia, y hay que castigar a alguien si se cae	token ring, token bus
Acceso aleatorio (*random access*)	todos compiten a la vez y se resuelve con colisiones y reintentos	colisiones	**ALOHA, CSMA, CSMA/CD**

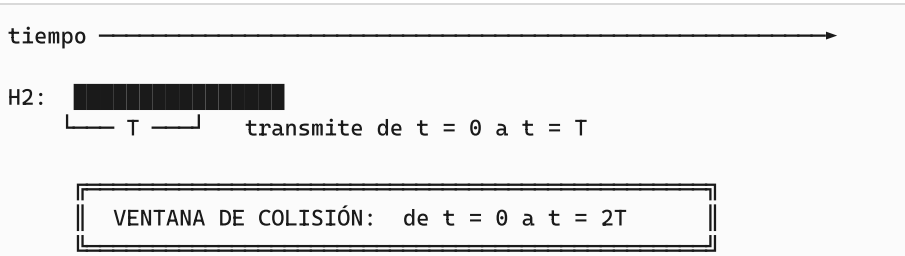
Lo que sigue es toda la tercera familia, más CDMA (que es de la primera y aparece en el Práctico 5).

1.3 ALOHA puro

Nació en 1970 en la Universidad de Hawái (Abramson), sobre una red de satélite. El **algoritmo** es ridículamente simple: si tenés una trama, **la transmitís ahora**, sin preguntar nada; esperás el ACK; si no llega, asumís que hubo colisión y reintentás **al azar**. Es decir: cero inteligencia.

El punto clave: la ventana de colisión

Tomemos una trama de duración `T` que una estación empieza a transmitir en `t = 0`:




```
Una trama que llega a t = T − 0,001  --> choca (se pisa por detrás)
Una trama que llega a t = T + 0,001  --> choca (se pisa por delante)
Una trama que llega a t = 2T + 0,1   --> NO choca (ya terminó)
```

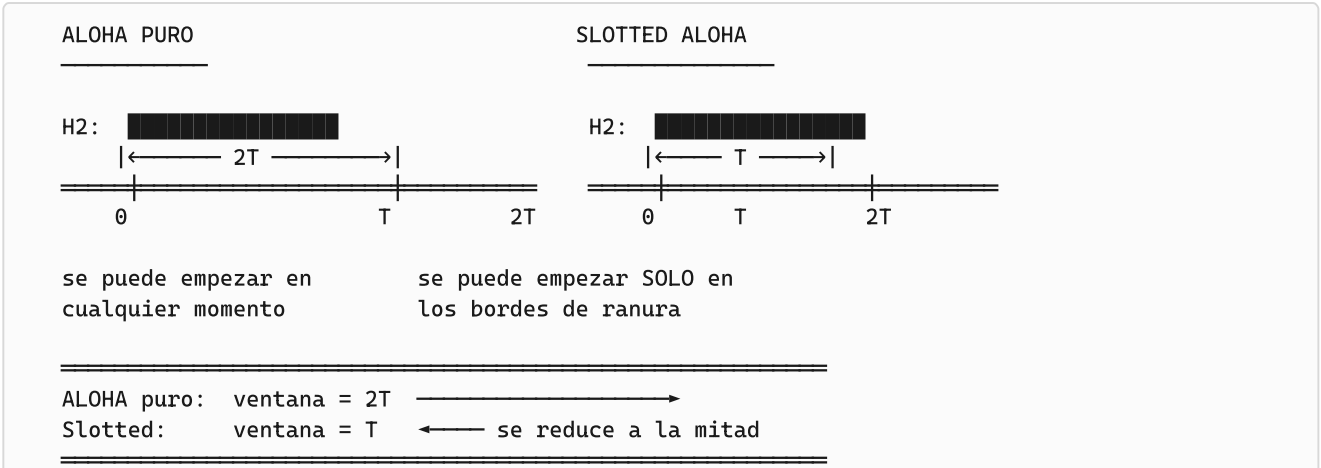
¿Por qué `2T` y no `T`? Porque hay dos formas de solaparse:

- **Antes:** otra trama empieza en los últimos `T` segundos de la mía, y se pisa con mi parte final.
- **Después:** otra trama empieza entre `T` y `2T`, y se pisa con **lo que yo emití al comienzo, que todavía está viajando por el cable**.

Ese segundo caso es el que casi todos olvidan, y es la mitad de la eficiencia máxima.

1.4 Slotted-ALOHA: ponele un reloj a todos

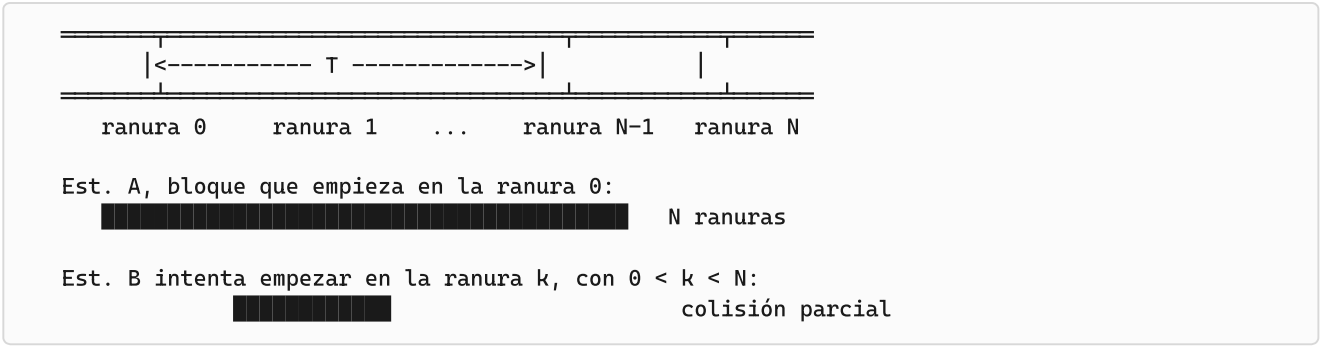
Slotted-ALOHA (Dave Roberts, 1975) es **el mismo protocolo con una sola modificación**: el tiempo se divide en **ranuras** (*slots*) de tamaño exactamente `T`, y **solo se puede empezar a transmitir en el instante cero de una ranura**. Todas las estaciones comparten el reloj.



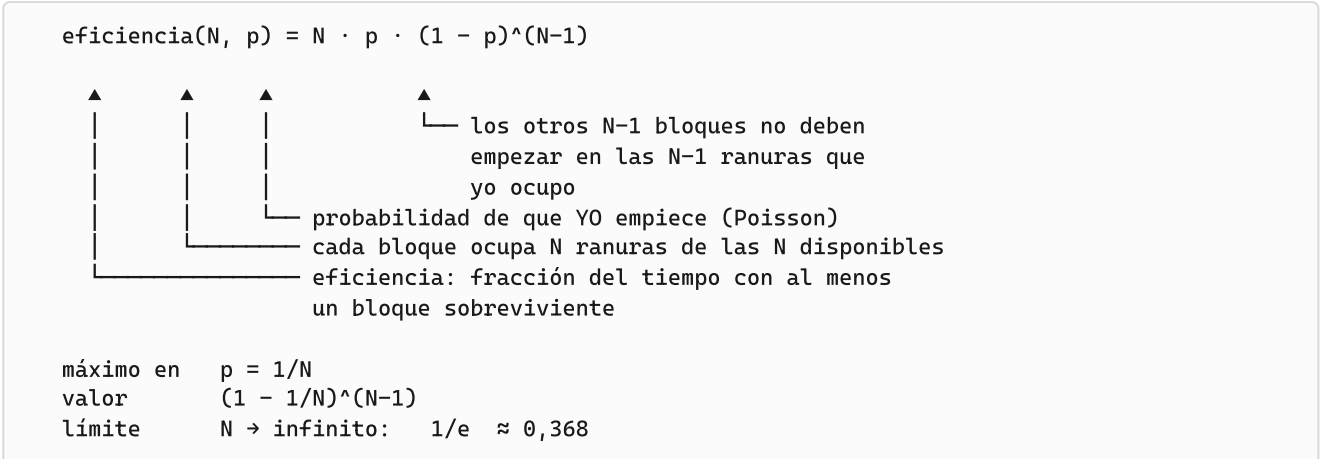
Y esa es toda la diferencia. Todo lo demás es idéntico. La consecuencia es aritmética: **la ventana vulnerable se reduce de `2T` a `T`, la probabilidad de sobrevivir se multiplica por `e`, y la eficiencia máxima pasa de `1/(2e)` a `1/e`: exactamente el doble** (la cuenta completa está en §2.2).

1.5 ALOHA con franjas: un término medio

Versión intermedia que aparece en el **Práctico 5**: cada estación transmite una trama larga que ocupa `N` **ranuras seguidas**, y solo puede empezar en el instante cero de una ranura.



Si dos bloques se pisan **en parte** igual se arruinan, así que la eficiencia no mejora por eso. Lo que hace el esquema es **reducir el riesgo**: entre el inicio del bloque de A y el del de B pueden pasar `N` ranuras completas, y eso agranda la ventana de colisión desde `T` hasta `N·T`. El precio es que cada estación ocupa el medio `N` veces más tiempo.



Con `N` chico el máximo es **mayor** que `1/e` (`N = 2` da 0,5) y converge a `1/e` desde arriba. El costo real es que con `N = 2` ocupás el medio el doble de tiempo para la misma probabilidad: la eficiencia mide tramas por ranura, no bits por segundo. La deducción completa está en §2.3.

1.6 CSMA: escuchar antes de hablar

ALOHA tiene un desperdicio evidente: **no escucha el medio para decidir**. Eso es **CSMA**: *Carrier Sense Multiple Access*, acceso múltiple con sensesión de portadora.

1.6.1 El problema del epsilon

Dos estaciones A y B en un cable de 2.500 m, con `v = 2 × 10⁸ m/s`: la señal de A tarda `τ = 2500 / (2 × 10⁸) = 12,5 µs` en llegar a B.

```
t = 0,000 µs    A empieza. La señal sale de A.
t = 6,25 µs     La señal pasa por el punto medio del cable.
t = 12,5 µs     La señal LLEGA a B.

Si B está escuchando en t = 12,4 µs   ->  oye a A  ->  espera.   OK
Si B empieza a transmitir en t = 12,4 µs ->  ¡colisión!
                                        (todavía no oyó nada)

Ese medio microsegundo de margen es el epsilon.
```

En cable, `ε` puede ser arbitrariamente pequeño pero **siempre positivo**. O sea: **el sentido de portadora elimina todas las colisiones salvo las que caen dentro de una ventana de anchura `ε`**. Como `ε` es infinitésimo frente a `T`, a baja carga la eficiencia de CSMA **se acerca a 1**: un salto de varios órdenes de magnitud respecto de ALOHA. Guardá ese `ε`, porque **es exactamente lo que se rompe en wireless**.

1.6.2 Persistente vs. no persistente

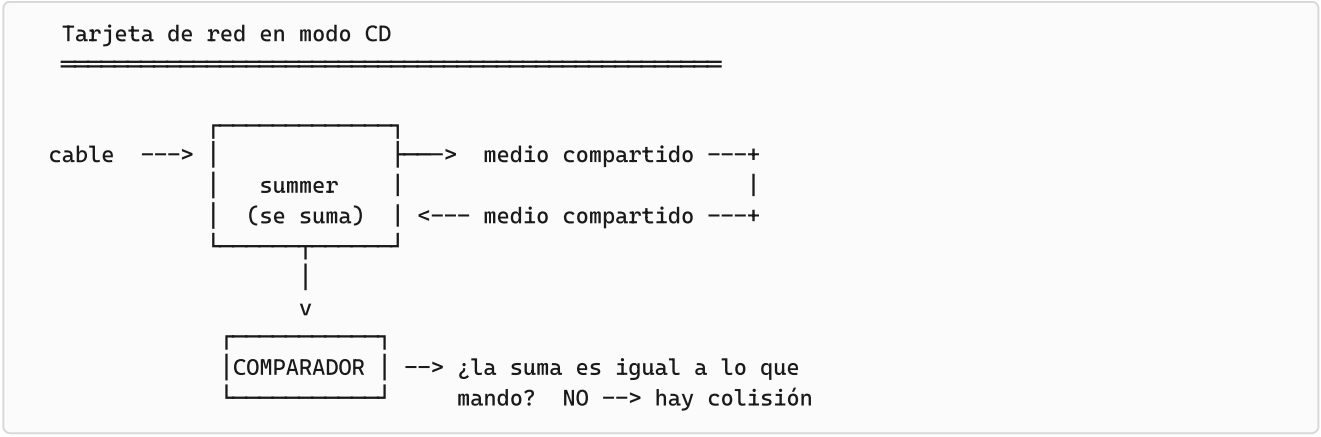
	1-persistente (*1-persistent*)	**No persistente** (*non-persistent*)	**p-persistente** (ranurado)
Qué hace si el medio está ocupado	espera **mirando**	espera un tiempo **aleatorio** y vuelve a escuchar	transmite con prob. `p`, con `1−p` espera una ranura
Ventaja	no desperdicia medio libre	**casi no hay colisiones**	compromiso continuo
Desventaja	si varias esperaban, **todas transmiten a la vez**: colisión garantizada	**regala ancho de banda**: el medio puede quedar libre sin que nadie transmita; en carga alta sube la colisión	parametrizable, en desuso
Se usa en	**Ethernet**	redes satelitales	casi nadie

El antídoto de la colisión masiva del 1-persistente es el backoff aleatorio (§1.7). Y el 1-persistente es el elegido en Ethernet porque el medio es rápido y no conviene regalarlo.

1.6.3 Con detección de colisión vs. sin detección

Acá está el corte que divide a Ethernet de WiFi, y es lo que te preguntan en `2007-07 P3b`.

	Sin detección (*collision non-detection*)	**Con detección** (*collision detection*, **CD**)
Qué hace	asume que si hubo colisión el medio está ocupado: espera a que se libere y reintenta	**escucha mientras transmite** y compara lo que manda con lo que vuelve
¿Cómo se entera?	por la **ausencia del ACK** (detección indirecta)	por la **comparación de señales** (detección directa)
Coste de una colisión	un **ACK completo**, o sea un RTT entero, del orden de milisegundos	**microsegundos**: aborta al instante
Requisito de hardware	ninguno	**una tarjeta que escucha mientras transmite**



En coaxil o en par trenzado balanceado esto es trivial: la tarjeta transmite por un par y escucha por el mismo par, y como la impedancia del medio es conocida puede distinguir "la señal que yo pongo" de "una señal que se le sumó". La frase para memorizar:

> **CSMA/CD funciona en cable porque la tarjeta puede escuchar el medio mientras transmite. En wireless no puede: la radio es half-duplex por naturaleza y su propia señal enmascara cualquier otra.**

1.6.4 La taxonomía completa

Las dos dimensiones son ortogonales, así que hay **cuatro combinaciones**:

	Sin detección	**Con detección**
1-persistente	1-persistent CSMA	**CSMA/CD** — el de Ethernet por cable
No persistente	non-persistent CSMA	CSMA/CD no persistente — no se usa en la práctica

Y si en lugar de **detectar** la colisión el protocolo la **evita**, aparecen las variantes de **evitación** (*collision avoidance*, **CA**): CSMA/CA, que es el caso de 802.11.

1.6.5 CSMA/CA: por qué WiFi no puede usar CD

Respuesta corta: **en radio no se puede detectar una colisión mientras se transmite**. Cuatro argumentos:

1. **Autoenmascaramiento físico.** La propia transmisión es de unos 20 dBm (100 mW); la señal de una estación vecina que llega al receptor puede ser de −70 dBm. La diferencia es de **90 dB**, casi **un millón de**

veces en potencia. No es un problema de algoritmo: es que la relación señal-ruido lo hace físicamente imposible. Peor: la radio tiene **una sola cadena de recepción** en la misma banda, así que **no puede escuchar mientras transmite**.

2. **La cobertura no es simétrica: la estación oculta.** Dos estaciones no tienen por qué alcanzarse igual. En el caso clásico, A y C **sí llegan a B** pero **no se escuchan entre sí**:

```
                B
               / \
              /   \        A <-> C : NO se escuchan
             /     \       A -> B : si
            A       C      C -> B : si

Si A transmite, C no oye nada y cree que el canal está
libre, así que C transmite, y B recibe dos señales
superpuestas de las que ninguna le sirve.

En un cable esto es imposible: si la señal de A llega a B,
tiene que haber pasado por C. La geometría del cable
GARANTIZA la simetría. La geometría del radio no.
```

3. **No existe un `T` mínimo acotado.** La corrección de CSMA/CD depende de que la trama sea **más larga que el tiempo de ida y vuelta del medio** (§1.8). En radio hay movilidad, fading y estaciones ocultas: ese tiempo no tiene un límite razonable, la trama mínima tendría que ser gigantesca y el rendimiento se caería a cero. **En cable ese límite existe porque topológicamente el cable tiene largo máximo.**
4. **El modelo de costo está invertido.** En wireless el único que puede juzgar la colisión es el receptor (por el CRC), así que **el emisor tiene que transmitir la trama entera para enterarse**. Si igual vas a transmitirla entera, conviene **evitar** la colisión en lugar de detectarla: esa es literalmente la diferencia entre CD y CA.

Y qué hace CSMA/CA entonces:

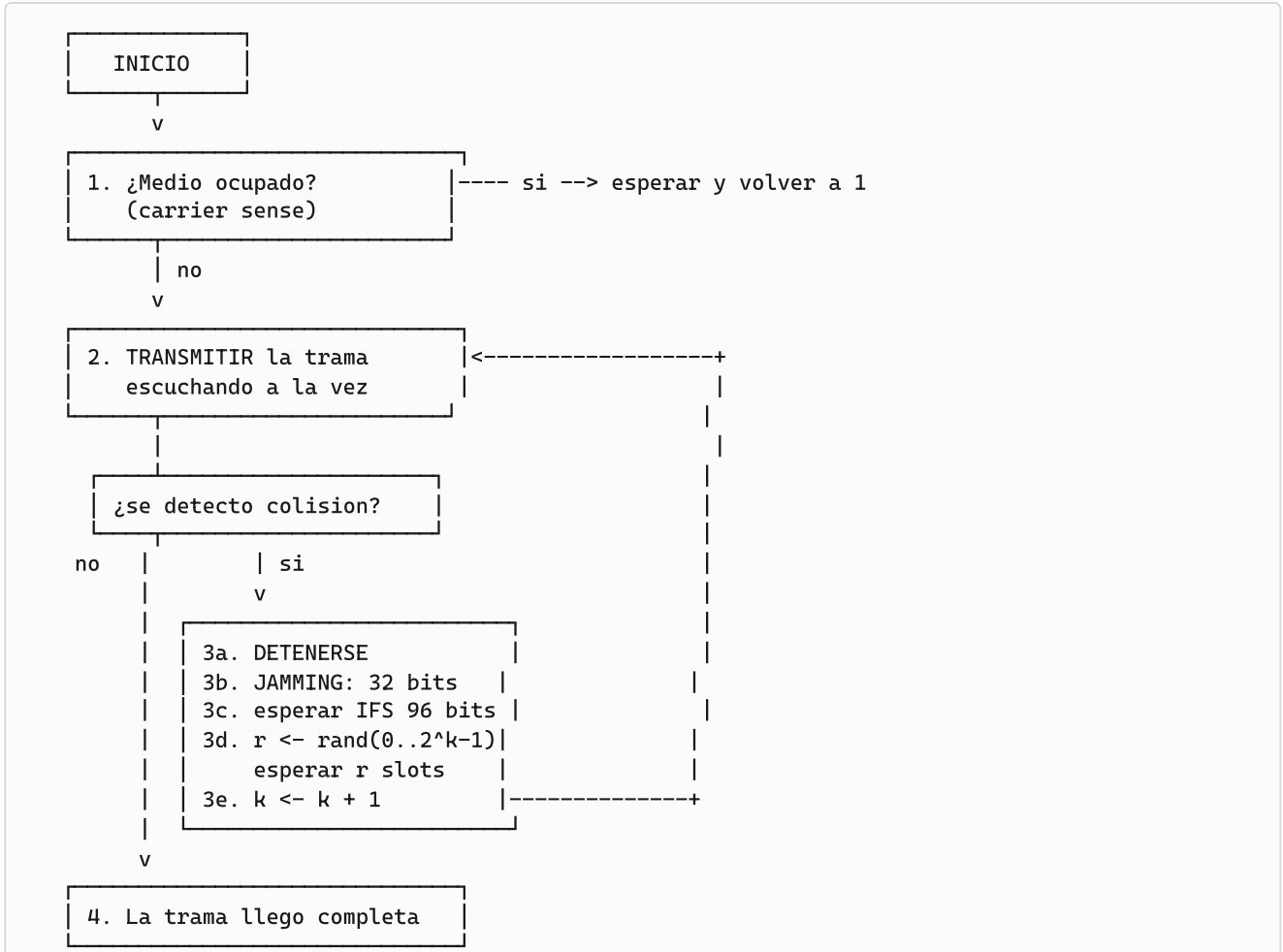
```
802.11: la secuencia de un envio con RTS/CTS

Estacion A           Estacion B           otras estaciones
    |                    |                      |
    |--- RTS ----------->|                      |  (oyen el RTS)
    |                    |                      |  NAV = duracion
    |<-- CTS -----------|                       |  del RTS (virtual)
    |                    |                      |
    |----- datos -------->|                     |
    |<---- ACK ----------|                      |
```

- **RTS** (*Request to Send*): A anuncia que va a mandar, y cuánto va a durar.
- **CTS** (*Clear to Send*): B confirma, y también anuncia cuánto va a durar.
- **NAV** (*Network Allocation Vector*): un "sentido de portadora virtual" que se guardan las estaciones que oyen el RTS pero no el CTS, para no transmitir durante la transferencia.

Es *evitación* por negociación, no *detección* por medición. Y el detalle final: **en 802.11 el ACK es obligatorio**, porque la única forma segura de saber que la trama llegó bien es que el receptor lo diga.

1.7 CSMA/CD: el algoritmo completo

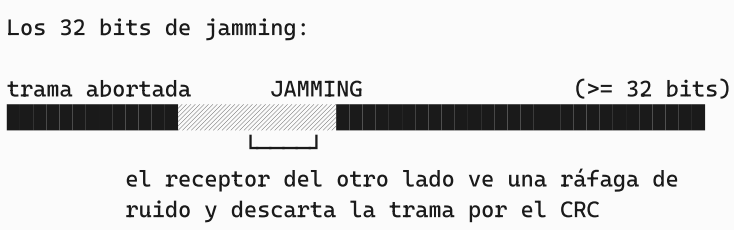
Este es el protocolo de Ethernet por cable. La máquina de estados completa:



(a) Persistencia 1. Si el medio está libre, transmite. La unidad de tiempo es el **slot time**.

(b) Detección de colisión. Compara la señal que manda (TX) con la que le vuelve del cable (RX). Si no coinciden, hay colisión.

(c) La secuencia de jamming. Al detectar la colisión la tarjeta **no se calla**: sigue mandando ruido deliberada durante un mínimo de **32 bits**. ¿Por qué? Porque la trama abortada tiene que ser suficientemente larga para que **todas** las demás estaciones del segmento la sientan como un evento de medio ocupado con trama inválida. Si simplemente cortara, las estaciones más lejos podrían no haberse enterado todavía y se meterían en el próximo slot. El jamming **sella** la colisión en el medio.



(d) El backoff exponencial. Después del jamming espera un **IFS** (*Inter Frame Space*, 96 bits a 10 Mb/s: el tiempo mínimo para que el medio se considere libre de nuevo) y después un número aleatorio de **ranuras**:

```
r <- rand( 0 , 2^k - 1 )     k = numero de intento (1 en el primero)

1er intento      -> k=1  -> r en [0, 1]     ->  2 valores
2do intento      -> k=2  -> r en [0, 3]     ->  4 valores
3er intento      -> k=3  -> r en [0, 7]     ->  8 valores
10mo intento     -> k=10 -> r en [0, 1023]  ->  1024 valores
11mo en adelante -> k=10 (tope) -> r en [0, 1023]

(la tabla completa, con los tiempos, está en §2.4)
```

¿Por qué exponencial? Si dos estaciones colisionan, **el azar las separa**. Si las dos sortean `r = 0`, vuelven a chocar. El backoff exponencial hace que la probabilidad de que dos estaciones elijan el mismo valor **caiga a cero geométricamente** (`1/2^k` en el intento `k`) conforme se acumulan los intentos, y al mismo tiempo evita que una tercera quede esperando una ventana enorme.

(e) El tope de 16 intentos. Si después de **16 intentos** la trama sigue sin salir, se descarta y se reporta como *excessive collisions*. El objetivo es que **un nodo muerto no bloquee el segmento entero**: el corte es mejor que la parálisis.

> **La convención importa.** IEEE 802.3 escribe `r ∈ [0, 2^k − 1]` **con `k` empezando en 1 en el primer intento**. Otros textos (y algunas soluciones de FING) cuentan `k = 0`, y entonces la primera ventana es `[0, 0]`, de una sola opción. **Si el enunciado no aclara, escribí la convención que usás**: eso ya suma puntos.

Y algo que casi nadie menciona: si la red está poco congestionada (menos de una colisión cada 1024 tramas), muchas tarjetas usan **backoff no exponencial**: eligen `r` en `[0, 2^k − 1]` con `k` fijo en **10**, siempre. Es una optimización de latencia que no cambia el algoritmo.

1.8 El slot time y los 64 bytes

Este es el corazón de por qué existe un tamaño mínimo de trama en Ethernet. La aritmética completa está en §2.5; acá va la lógica.

Paso 1 — La condición que hay que garantizar. Para que CSMA/CD funcione, la tarjeta tiene que poder **darse cuenta de que hubo colisión mientras todavía está transmitiendo**. Si la trama terminara antes de que volviera el indicio, el emisor creería que tuvo éxito.

```
tiempo de transmision de la trama  >=  tiempo de ida y vuelta
```

Paso 2 — Cuánto tarda la señal en ir y volver. El peor caso es transmitir hacia la estación **más lejana del segmento**, con el cable recorrido dos veces y las conversiones de los extremos. Si `τ` es el retardo de propagación de **un sentido**, entonces `t_ida_y_vuelta = 2 · τ`.

Paso 3 — El slot time. IEEE 802.3 lo fija en **512 tiempos de bit**, sea cual sea la velocidad: `51,2 µs` a 10 Mb/s, `5,12 µs` a 100 Mb/s, `0,512 µs` a 1 Gb/s.

Paso 4 — Traducir a bytes. `512 bits / 8 bits por byte = 64 bytes`.

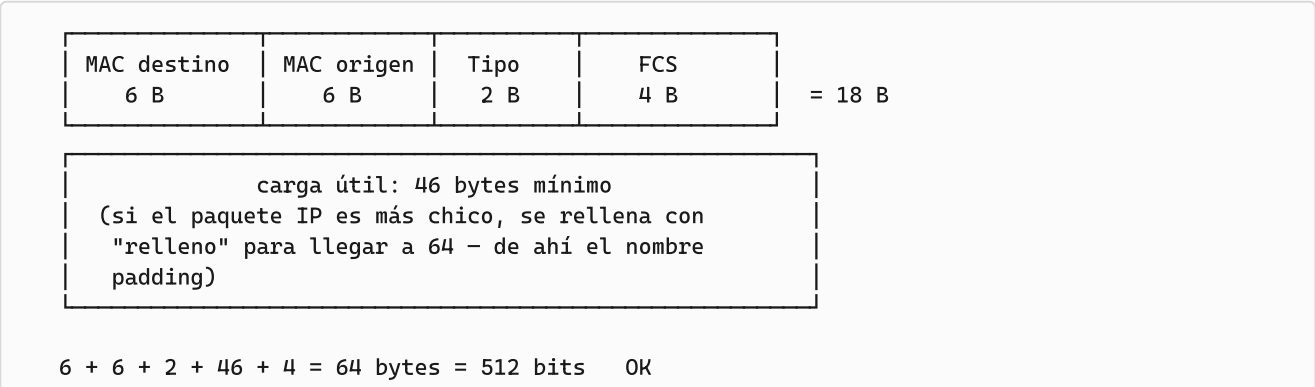
Ahí está el 64. No es un número arbitrario ni una constante de interoperabilidad: **es exactamente el tiempo de ida y vuelta de la señal en el peor caso, medido en bits.**

La misma cuenta en las otras velocidades

Velocidad	Slot time	t máximo (un sentido)	Distancia máxima teórica	Distancia real con topología
10 Mb/s	51,2 µs	25,6 µs	**5120 m**	2500 m con 5 repetidores
100 Mb/s	5,12 µs	2,56 µs	**512 m**	100 m por segmento con hub (100BASE-TX)
1 Gb/s	0,512 µs	0,256 µs	**51,2 m**	100 m por segmento, con extensión de portadora

Fijate la fila de 1 Gb/s: **el presupuesto de tiempo se encogió 100 veces** respecto de 10 Mb/s, pero el largo del cable es el mismo. Por eso IEEE 802.3 inventó la **extensión de portadora** (*carrier extension*): en half-duplex a 1 Gb/s la tarjeta mantiene el medio "ocupado" por al menos **4096 tiempos de bit** (4,096 µs), metiendo símbolos que no son datos. Es un parche que existe **precisamente porque el 512 ya no alcanzaba**.

Y el FCS también es mínimo

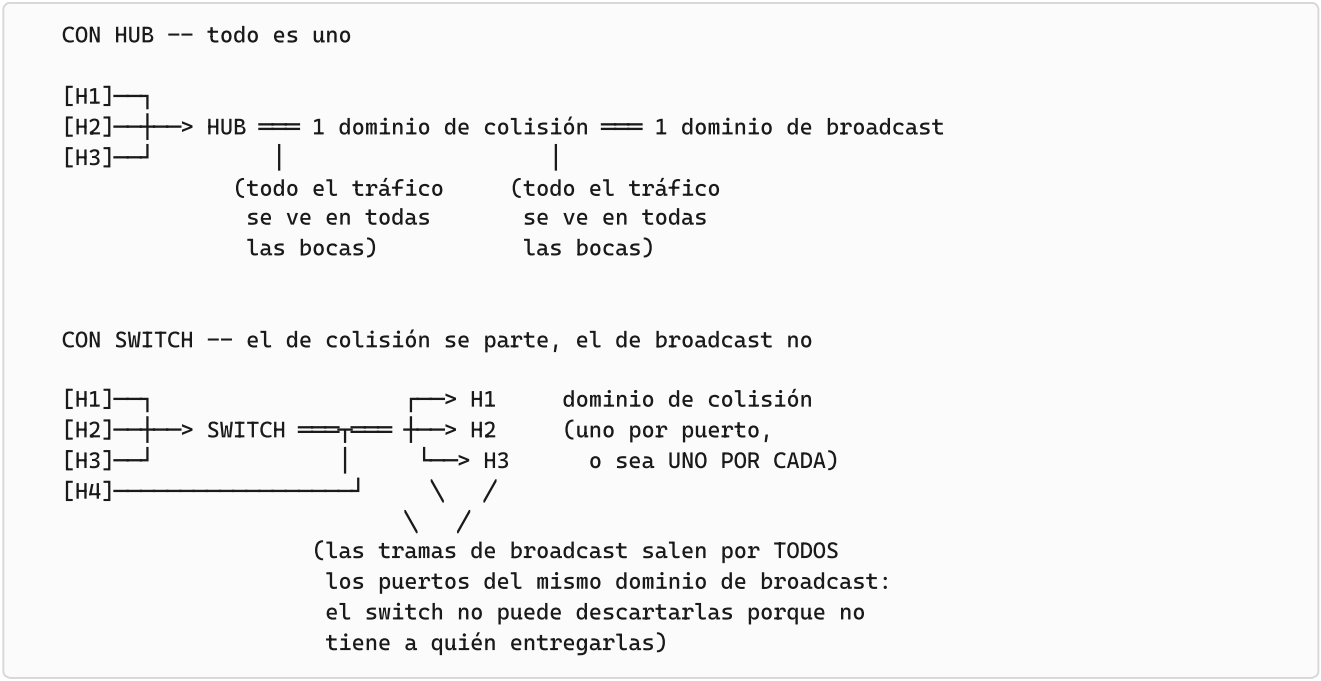


Detalle con VLAN: con etiqueta 802.1Q (4 bytes extra) la carga útil mínima baja a 42 bytes y la trama mínima **con etiqueta** es de 68 bytes. El mínimo absoluto sigue siendo 64 bytes **sin** etiqueta.

1.9 Dominios de colisión y de broadcast

Se confunden todo el tiempo y aparecen juntos en el examen `2010-12 P3`. Los definimos acá de forma breve; la Clase 42 los desarrolla a fondo con la tabla MAC y el *self-learning*.

	Dominio de colisión	Dominio de broadcast
Qué es	las estaciones cuyas señales se superponen si transmiten a la vez	las estaciones que reciben una trama de broadcast
Definido por	el medio físico compartido	la capacidad de reenviar tramas por todos los puertos
Se rompe con	el switch (cada puerto es un dominio propio)	la VLAN (o un router)
Máximo teórico	2 estaciones (full-duplex)	toda la red local



Con un hub: es de capa 1. Copia la señal a **todos** los puertos, sin mirar direcciones y sin decidir nada. Entonces: **un solo dominio de colisión** (comparte el mismo medio eléctrico) **y un solo dominio de broadcast**. Es la red del §1.1, con ALOHA encima.

Con un switch: aprende direcciones y decide a qué puerto va cada trama, y eso **parte el medio en un dominio de colisión por puerto**. Pero las tramas de broadcast no se pueden descartar —por definición todos tienen que oírla—, así que **el dominio de broadcast sigue siendo toda la red**. Salvo que existan VLANs.

> **La frase que hay que saber decir:** el switch **no evita colisiones**. Lo que hace es **eliminar el medio compartido**, y por eso es que las colisiones dejan de ser posibles. Son dos afirmaciones muy distintas, y confundirlas es el error conceptual más común del tema.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **CSMA/CD**: *exponential backoff*, **slot time = 512 bits**, trama mínima **64 B**, trama máxima **1518 B**, jamming de 32 bits, tope de 16 intentos. - Formato de trama: preámbulo 7 + SFD 1, MAC destino 6 + MAC origen 6, tipo 2, carga útil, **FCS 4**. **MTU 1500.** - **ARP (`RFC 826`)**: request en broadcast, reply en unicast. El *gratuitous ARP* sigue usándose (anuncio, DHCP, failover de gateway, VRRP) y sigue siendo una fuente clásica de *ARP spoofing*. - **MAC learning**: el bridge aprende la MAC **origen** y asocia puerto; **flooding** si el destino es desconocido; **unicast** si se conoce. - **Loops en capa 2 = broadcast storm**: sigue siendo la razón de ser del STP.

Cambió o se modificó:

- **El half-duplex Ethernet es historia por encima de 1 Gb/s.** Es una decisión de **IEEE 802.3**, **no una RFC de IETF**. ⚠️ **No existe ningún RFC que lo depreque**: `RFC 8984` es **JSCalendar**, nada que ver con Ethernet. A 10GbE y por encima el full-duplex punto a punto es obligatorio, y el half-duplex directamente no existe en la especificación de esas velocidades.
- **Pero half-duplex no desapareció del todo. IEEE 802.3cg 10BASE-T1S** (*single-pair Ethernet*, para redes PLC industriales sobre un solo par) es **obligatoriamente half-duplex**, con un protocolo de acceso propio,

PLCA. Y en IETF, el módulo YANG `csma-cd` de `802.3cf` está marcado `status deprecated`. ⇒ **Saber cuándo CSMA/CD sigue existiendo es parte del tema.**

- `RSTP` **(** `802.1w` **) es el default real**, no el STP clásico. `MSTP` (`802.1s`) para varias instancias de VLAN. Bridge ID con prioridad base **32768**.
- ⚠️ **El "aging de 300 s" de la tabla MAC es un default de FABRICANTE, no un valor mandated por IEEE.** Enseñalá "típicamente 300 s en switches comerciales, configurable", **nunca** "el estándar IEEE dice 300 s".
- `802.1Q-2022` es el estándar actual (precedido por `802.1Q-2018`): 2018 abrió el bloque de TSN (`802.1Qbv` , `802.1Qbu` con `802.3br` , `802.1Qci`) y 2022 agregó `802.1Qcc` y `802.1Qcr` . **C-TAG (** `0x8100` **) vs. S-TAG (** `0x88A8` **)**: doble etiquetado para *provider bridges*, hoy lo estándar en carrier Ethernet y EVPN-VXLAN. **LAG**: `802.3ad` (2000) pasó a `802.1AX` , revisión vigente `802.1AX-2020` .

Consecuencia para el examen: si preguntan "¿CSMA/CD es necesario hoy?", la respuesta es **no, y ni siquiera existe por encima de 1 Gb/s**. Si preguntan "¿es siempre necesario en Ethernet?", la respuesta es **no: con full-duplex no hay colisión posible** (§3.2). Y si preguntan por el aging, decí "típicamente 300 s, configurable, default del fabricante".

PARTE 2 — PRÁCTICA RESUELTA

> En toda esta parte **se supone que las tramas llegan en un proceso de Poisson** (arribos independientes con probabilidad constante por unidad de tiempo), que es el modelo estándar del libro. Es la única suposición fuerte, y hay que declararla, porque sin ella las cuentas no cierran.

2.1 Ejercicio 1 — Derivar la eficiencia del ALOHA puro (`1/(2e)`)

Enunciado. Una red de ALOHA puro tiene tramas de duración `T` y las estaciones generan tramas según un proceso de Poisson de tasa `λ`. Calcular la eficiencia y su valor máximo.

Datos. `G = λ · T` es la **carga ofrecida** (*offered load*): el número de tramas que el sistema **intenta** poner en el medio durante el tiempo que tarda una trama en entrar.

Paso 1 — Probabilidad de colisión. Una trama que envío en `t = 0` tiene ventana de `2T` (§1.3). En esos `2T` segundos pueden llegar `λ · 2T = 2G` tramas ajenas. Para no chocar necesito **cero**:

```
P(sin colisión) = P(0 arribos en 2T) = e^(-2G)
```

Interpretación: es la probabilidad de **supervivencia** de una trama. Si `G = 0,5`, `e^(-1) = 0,368`.

Paso 2 — Eficiencia. De cada `G` tramas que el sistema intenta meter por unidad de tiempo de trama, sobrevive la fracción `e^(-2G)`:

```
S = G · e^(-2G)
^   ^
|   └── fracción que sobrevive sin chocar
└────── tramas que se intentan meter por unidad de tiempo
```

Paso 3 — Maximizar. El factor exponencial siempre es positivo, así que el máximo está donde se anula el factor lineal de la derivada:

```
S'(G) = e^(-2G) + G · (-2) · e^(-2G)
      = e^(-2G) · (1 - 2G)

S'(G) = 0   -->   1 - 2G = 0   -->   G* = 1/2

S''(G) = e^(-2G) · (-2 - 2 + 4G)
En G = 1/2:   S'' = e^(-1) · (-2) < 0   -->  es MAXIMO
```

Paso 4 — El valor.

```
S_max = (1/2) · e^(-2 · (1/2)) = (1/2) · e^(-1) = 1/(2e)

1/(2e) = 1 / (2 · 2,71828) = 1 / 5,43656 = 0,18394  ≈  18,4 %
```

Resultado: la eficiencia máxima del ALOHA puro es `1/(2e) ≈ 0,184`. O sea: **de cada 5,44 tramas que el medio transporta, solo 1 es válida.** Las otras 4,44 se pierden en colisiones. Ese es el precio de no mirar nunca el medio antes de hablar.

La misma cuenta sin derivadas (por si el corrector no admite cálculo):

```
S(1/2) = 0,5 · e^(-1)   = 0,5 · 0,36788 = 0,18394
S(0,4) = 0,4 · e^(-0,8) = 0,4 · 0,44933 = 0,17973   <-- menor
S(0,6) = 0,6 · e^(-1,2) = 0,6 · 0,30119 = 0,18072   <-- menor
```

El argumento sin cálculo: `S = G·e^(-2G)` es el producto de un factor que **crece** y otro que **decrece exponencialmente**, así que hay un único punto de equilibrio; y como `S` tiende a 0 cuando `G` tiende a infinito, ese punto es forzosamente un máximo.

Carga `G`	Slotted `G·e^(-G)`	Puro `G·e^(-2G)`
0,10	0,0905	0,0819
0,25	0,1947	0,1516
0,50	**0,3033**	**0,1839 (máximo)**
0,75	0,3543	0,1673
1,00	**0,3679 (máximo)**	0,1353
1,50	0,3347	0,0747
2,00	0,2707	0,0366

> **Errores típicos:** contar `T` en vez de `2T` (te da `1/e`, que es el número del Slotted); olvidar que hay que maximizar y no solo escribir la función; confundir `G` (un número de tramas por tiempo de trama) con una probabilidad.

2.2 Ejercicio 2 — Derivar la eficiencia del Slotted-ALOHA (`1/e`)

Paso 1 — La ventana vulnerable. Si solo puedo empezar en los bordes de ranura, dos tramas que empiecen en ranuras distintas **no pueden solaparse**. El único caso de colisión es que **empiecen en la misma ranura**.

```
ranura 0 | ranura 1 | ranura 2 | ranura 3 |
─────────┼──────────┼──────────┼──────────┤
   ▓▓▓▓  |          |  ▓▓▓▓    |  ▓▓▓▓    |
   A     |          | B        |
   (empieza en 0)     (empieza en 2)
              ↑
   no hay solapamiento: la ventana de A terminó
   justo cuando empieza B
```

La ventana pasó de `2T` a `T`. Punto. Ese es el único cambio.

Paso 2 — Probabilidad de colisión.

```
arribos en una ventana de T segundos:  λ · T = G
P(sin colisión) = P(0 arribos en T) = e^(-G)
```

Paso 3 — Eficiencia y máximo.

```
S = G · e^(−G)

S'(G) = e^(−G) − G · e^(−G) = e^(−G) · (1 − G)
S'(G) = 0   -->   G* = 1

S_max = 1 · e^(−1) = 1/e = 0,36788  ≈  36,8 %
```

Paso 4 — Por qué dobla EXACTAMENTE.

```
     S_slotted        G · e^(−G)          evaluado en G = 1
─────────────── / ──────────────── = ──────────────────── = e^(−1)
     S_puro          G · e^(−2G)         evaluado en G = 1/2
                                        (1/2)·e^(−1)

=  1 · e^(−1)  /  ( (1/2) · e^(−1) )
=  e^(−1) / ((1/2) e^(−1))
=  1 / (1/2)
=  2
```

La explicación de fondo es de una frase: **el riesgo se redujo a la mitad**. Como la probabilidad de supervivencia es `e^(−ventana·λ)`, reducir la ventana a la mitad **multiplica por `e` la probabilidad de sobrevivir**; y como además la carga óptima se duplica (`G*` pasa de `1/2` a `1`), el resultado neto es un factor **exactamente 2**. No es aproximación: es `e` en el numerador contra un `1/2` en el denominador. Y un detalle elegante: el sistema ranurado puede (y debe) **ofrecer el doble de carga** para funcionar igual de bien, porque para la misma cantidad de tramas que mete gasta la mitad de tiempo expuesto.

2.3 Ejercicio 3 — Práctico 5, ejercicio 8: ALOHA con franjas

> **Fuente:** Práctico 5 de 2008, ejercicio 8 (página 2), basado en la sección 5.3 del libro.

Enunciado (traducción). "En la Sección 5.3 se ha proporcionado un resumen de la deducción de la eficiencia de ALOHA con franjas. En este problema completaremos la deducción. **a.** Recordar que cuando hay `N` nodos activos, la eficiencia de ALOHA con franjas es `Np(1−p)^(N−1)`. Encontrar el valor de `p` que hace máxima esta expresión. **b.** Utilizando el valor de `p` encontrado en (a), encontrar la eficiencia de ALOHA con franjas suponiendo que `N` se aproxima a infinito. (Pista: `(1 − 1/N)^(N−1)` se aproxima a `1/e` cuando `N` se aproxima a infinito.)"

Dato. `p` = probabilidad de que un nodo **empiece** a transmitir en un instante dado.

Parte a) — el `p` que maximiza

```
S(p) = N · p · (1 − p)^(N−1)

dS/dp = N · (1−p)^(N−1)  −  N·p·(N−1) · (1−p)^(N−2)

Factorizo N · (1−p)^(N−2):
      = N · (1−p)^(N−2) · [ (1−p) − p(N−1) ]

Como N·(1−p)^(N−2) > 0, igualo a cero el corchete:

      (1−p) − p(N−1) = 0
      1 − p − pN + p = 0
      1 − pN = 0
      p* = 1/N
```

Respuesta a): `p = 1/N`. Cada estación empieza uno de cada `N` intentos: exactamente la carga que llena el medio sin pasarse. Es un **máximo** porque `S(0) = 0` y `S(1) = 0` (para `N > 1`) y hay un único punto crítico.

Parte b) — el límite `N` hacia infinito

```
S_max(N) = N · (1/N) · (1 − 1/N)^(N−1)
         = 1 · (1 − 1/N)^(N−1)

ln [ (1 − 1/N)^(N−1) ] = (N−1) · ln(1 − 1/N)

Para N grande usamos   ln(1 − x) ≈ −x − x²/2 − x³/3   con x = 1/N

    (N−1) · ln(1 − 1/N)
      ≈ (N−1) · ( −1/N − 1/(2N²) − 1/(3N³) )
      =  −(1 − 1/N)  −  (1/2)(1/N − 1/N²) − ...
      -->  −1 − 0 − 0  =  −1          cuando N --> infinito

Entonces:    (1 − 1/N)^(N−1) --> e^(−1) = 1/e

S_max(N --> infinito) = 1/e = 0,367879  ≈  36,8 %
```

Respuesta b): la eficiencia tiende a `1/e`, que es exactamente el valor del Slotted-ALOHA.

`N` (ranuras por bloque)	`p* = 1/N`	`S_max = (1−1/N)^(N−1)`	contra `1/e`
2	0,500	0,5000	**+35,9 %**
4	0,250	0,4219	**+14,7 %**
8	0,125	0,3927	**+6,7 %**
100	0,010	0,3697	+0,5 %
infinito	→ 0	→ 0,3679 (= `1/e`)	0

Cálculo numérico con `N = 100`:

```
p  = 1/100 = 0,01
S  = 100 · 0,01 · (1 − 0,01)^(99) = 1 · (0,99)^99

   ln(0,99) = −0,0100503
   99 · (−0,0100503) = −0,99498
   e^(−0,99498) = 0,36973

S = 0,3697   (contra 1/e = 0,3679: apenas 0,5 % por encima)
```

Interpretación. Con pocas ranuras por bloque la eficiencia "por ranura" es mejor, pero **cada estación ocupa el medio mucho más tiempo**: con `N = 2` ocupás el medio el doble de tiempo para conseguir la misma probabilidad. La eficiencia mide tramas por ranura, no bits por segundo.

> **Errores típicos:** poner `N(1−p)` en el exponente (el factor `N·p` cuenta cuántas **empiezan**, y el `(1−p)` cuenta cuántas **no empiezan**, `N−1` veces); creer que `p = 1/N` es `p = 1`; y **citar el límite `1/e` sin derivarlo**: en el examen, el paso del logaritmo vale la mitad del punto.

2.4 Ejercicio 4 — La tabla de backoff exponencial de CSMA/CD

Enunciado (inventado). Una estación de Ethernet de 10 Mb/s tiene un choque. Calculá, para los primeros 8 intentos: el valor máximo de `r`, la cantidad de valores posibles de la ventana y el tiempo máximo que puede esperar.

Datos. `slot time = 512 bits = 512 / 10⁷ = 51,2 µs`.

```
r <- rand( 0 , 2^k − 1 )      k = numero de intento, arrancando en k = 1

cantidad de valores posibles = (2^k − 1) − 0 + 1 = 2^k
tiempo maximo de espera = r_max · slot time = (2^k − 1) × 51,2 µs
```

Intento `k`	`2^k`	`2^k − 1`	valores de `r`	`r` en	t_máx = `(2^k−1)` × 51,2 µs
1	2	1	**2**	0 … 1	**51,2 µs**
2	4	3	**4**	0 … 3	153,6 µs
3	8	7	**8**	0 … 7	358,4 µs
4	16	15	**16**	0 … 15	768,0 µs
5	32	31	**32**	0 … 31	1,587 ms
6	64	63	**64**	0 … 63	3,226 ms
7	128	127	**128**	0 … 127	6,502 ms
8	256	255	**256**	0 … 255	13,06 ms
10	1024	1023	**1024**	0 … 1023	52,38 ms
11 en adelante	1024	1023	**1024**	0 … 1023	52,38 ms (tope)

Interpretación. El tiempo máximo **crece por mitad en cada paso** y a partir del intento 10 se congela en 52,4 ms, que es aproximadamente un tiempo de ida y vuelta típico de una red local. Es decir: el protocolo garantiza que ninguna estación espere más que lo que tardaría un paquete en dar una vuelta a la red. Si fuera más, el

backoff empezaría a **dominar** el tiempo de respuesta y la red se sentiría lenta por el mecanismo que justamente debía mejorarla.

```
Si la red no está congestionada (menos de 1 colisión cada
1024 tramas), muchas tarjetas usan k FIJO = 10 desde el
primer intento:

Primer choque -> r en [0, 1023] directamente.
Ventana 1024 veces mas grande -> probabilidad de choque
con otra estacion del orden de 1/1024.
```

> **Errores típicos:** empezar `k` en 0 (muchos textos lo hacen, pero IEEE cuenta el primer intento como `k = 1`; **escribí qué convención usás**); calcular `t_máx` como `2^k × slot` en vez de `(2^k − 1) × slot`; y olvidar que el backoff arranca **después** del IFS de 96 bits (9,6 µs a 10 Mb/s).

2.5 Ejercicio 5 — El slot time y la trama mínima: derivar los 64 bytes

Enunciado (inventado, y es el mejor ejercicio conceptual de la clase). Una red Ethernet de 10 Mb/s tiene un segmento de coaxil de 2.500 m, con velocidad de propagación `2 × 10⁸ m/s`. Cada estación tiene un retardo de transceiver de 0,1 µs en cada extremo.

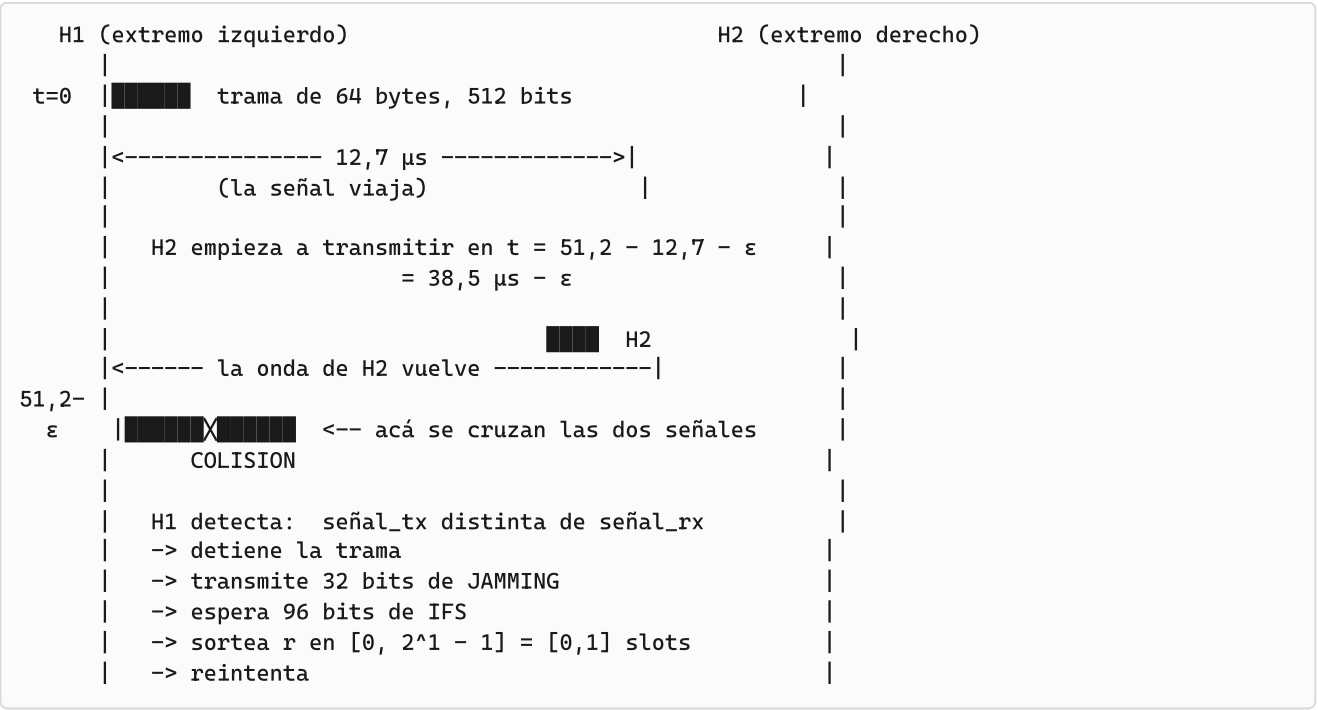
a) Retardo de propagación de un sentido. **b)** Tiempo de ida y vuelta del extremo más lejano. **c)** ¿Qué condición debe cumplir la trama mínima? **d)** ¿Alcanzan 512 bits?

```
(a) Retardo de propagacion de un sentido

d = 2.500 m
v = 2 x 10^8 m/s

tau = d / v = 2500 / (2 x 10^8) = 12,5 x 10^-6 s = 12,5 µs

Sumando los dos transceivers (uno en cada extremo):
tau_total = 12,5 µs + 0,1 µs + 0,1 µs = 12,7 µs


(b) Ida y vuelta

t_ida_y_vuelta = 2 · tau_total = 2 · 12,7 µs = 25,4 µs
(Los dos transceivers ya están incluidos: van uno en cada
 sentido del viaje.)


(c) La condición. Para detectar una colisión, el indicio tiene
    que VOLVER antes de que la trama termine. Si la trama
    terminara antes, la tarjeta creería que salió bien:

        t_transmision  >=  t_ida_y_vuelta


(d) Verificacion

slot time = 512 · T_bit = 512 / (10 x 10^6) = 51,2 µs

t_transmision (trama de 64 B) = 51,2 µs
t_ida_y_vuelta                = 25,4 µs
_____________________________________
51,2 µs  >=  25,4 µs         OK, ALCANZA
```

El paso inverso: de la condición al número de bytes

```
1. La trama mínima debe durar >= el tiempo de ida y vuelta.
2. IEEE fija el slot time (la unidad de tiempo del backoff,
   igual al tiempo máximo de ida y vuelta) en 512 bits.
3. Por lo tanto la trama mínima es de 512 bits.
4. 512 bits / 8 bits por byte = 64 bytes.
```

Y la comprobación con el diagrama de los 64 bytes:



Interpretación: **todo el mecanismo cabe dentro de la trama mínima.** La tarjeta no espera a que termine: aborta en unos 51 µs. Ese es el beneficio de CD frente a la detección indirecta por ACK, que costaría un RTT completo.

```
La generalización a otras velocidades:

A 100 Mb/s:   512 / (100 x 10^6) = 5,12 µs
              -> la distancia máxima cae a
                 (5,12/2) x 10^−6 x 2 x 10^8 = 512 m

A 1 Gb/s:     512 / (10^9) = 0,512 µs
              -> la distancia máxima cae a 51,2 m
              -> INSUFICIENTE para los 100 m del cable
                 -> de ahi la extensión de portadora
                    de IEEE 802.3 (4096 tiempos de bit)
```

> **Errores típicos:** dividir solo por 2 sin justificar por qué (decí "la colisión tiene que aparecer en el receptor **antes** de que el emisor termine"); olvidar el tiempo de transmisión de la trama y comparar solo propagación contra 512 bits (la condición es `t_tx >= 2·tau`); y confundir 512 **bits** con 512 **bytes**.

2.6 Ejercicio 6 — CDMA: los ejercicios 5 y 6 del Práctico 5

> **Fuente:** Práctico 5 de 2008, ejercicios 5 y 6 (página 1). CDMA = *Code Division Multiple Access*, acceso múltiple por división de código. Es un protocolo de **partición de canal**: en vez de dividir el tiempo o la frecuencia, divide el **espacio de códigos**.

La regla de oro. Cada bit de dato `d_i` se codifica multiplicándolo, bit a bit, por un código de `M` chips del emisor:

```
Z_(i,m) = d_i · c_m          m = 1 ... M

c_m en {+1, -1}      <- el codigo del emisor, M valores
d_i en {+1, -1}      <- el bit de dato, representado con signo
```

Convención del libro (importante): para hacer la aritmética, **un bit de dato `0` se representa como `-1`**, no como `0`.

Ejercicio 5 — emisor único con código (1, −1, 1, −1, 1, −1)

Enunciado (traducción). "Considérese el ejemplo de emisor único CDMA de la Figura 5.11. ¿Cuál sería la salida del emisor (para los dos bits de datos mostrados) si el código CDMA del emisor fuera (1, −1, 1, −1, 1, −1)?"

Datos. `M = 6` porque el código tiene 6 valores. Cada bit produce **6 chips**, y con 2 bits la salida tiene **12 valores**. La tabla completa, para los cuatro pares posibles (así leés el que muestre tu edición):

bits mostrados	`d_0`	`d_1`	salida ranura 0 (6 chips)	salida ranura 1 (6 chips)
1 1	+1	+1	`1 -1 1 -1 1 -1`	`1 -1 1 -1 1 -1`
1 0	+1	−1	`1 -1 1 -1 1 -1`	`-1 1 -1 1 -1 1`
0 1	−1	+1	`-1 1 -1 1 -1 1`	`1 -1 1 -1 1 -1`
0 0	−1	−1	`-1 1 -1 1 -1 1`	`-1 1 -1 1 -1 1`

Ejemplo desarrollado, el caso "1 0":

```
d_0 = 1   ->  Z_0 = 1 · (1, -1, 1, -1, 1, -1) = ( 1, -1,  1, -1,  1, -1)
d_1 = -1  ->  Z_1 = -1 · (1, -1, 1, -1, 1, -1) = (-1,  1, -1,  1, -1,  1)

Salida completa (12 chips):
  ( 1, -1,  1, -1,  1, -1, |  -1,  1, -1,  1, -1,  1 )
    └──────── dato 1 ──────┘    └──────── dato 0 ──────┘
```

Interpretación — la idea que hay que escribir en el examen: el segundo bloque es **exactamente el negativo** del primero. Mandar un `0` es mandar **el código dado vuelta**, no "no mandar nada". Eso es lo que permite después **separar** a los emisores: el receptor multiplica por el mismo código y el `-1` le queda como signo.

La decodificación, para verificar que el sistema funciona:

```
d_reconstruido = (1/M) · suma de  Z_(i,m) · c_m

Ranura 0 (d_0 = 1):
  1/6 · (1·1 + (-1)(-1) + 1·1 + (-1)(-1) + 1·1 + (-1)(-1))
= 1/6 · (1 + 1 + 1 + 1 + 1 + 1) = 1/6 · 6 = 1     OK

Ranura 1 (d_1 = -1):
  1/6 · ((-1)(1) + (1)(-1) + (-1)(1) + (1)(-1) + (-1)(1) + (1)(-1))
= 1/6 · (-6) = -1                                    OK
```

Interpretación: **la suma da `+M` o `-M`**, y al dividir por `M` sale `+1` o `-1`. El componente clave es que **cada chip se multiplica por sí mismo** (`c_m · c_m = c_m² = 1`), así que el código aporta siempre `+M` de energía útil.

Ejercicio 6 — el emisor 2

Enunciado (traducción). "Considérese el emisor 2 de la Figura 5.12. ¿Cuál es la salida del emisor al canal (antes de que se añada a la señal del emisor 1) `Z_m`?"

Datos. El emisor 2 tiene los bits de datos `(-1, +1)` (es decir, "0 1") y su propio código, también de 6 chips. Tomemos `c₂ = (1, -1, -1, -1, -1, 1)`.

```
Datos:   d_0 = -1          d_1 = +1

Ranura 0:
  Z_0 = (-1) · (1, -1, -1, -1, -1, 1) = (-1, +1, +1, +1, +1, -1)

Ranura 1:
  Z_1 = (+1) · (1, -1, -1, -1, -1, 1) = ( 1, -1, -1, -1, -1,  1)

Salida Z_m:
  (-1, +1, +1, +1, +1, -1, |  1, -1, -1, -1, -1,  1 )
```

Continuación natural — el agregado y por qué CDMA funciona *(solución propia)*

Esta parte no está en el enunciado del práctico, pero es **la pregunta que de verdad importa**: ¿cómo se separan las señales si van mezcladas por el mismo canal? **El canal es aditivo**: todas las señales se suman en el aire.

```
S_(i,m) = Z1_(i,m) + Z2_(i,m)

Ranura 0:  (-1, 1, 1, 1, 1, -1)  +  (0, 0, 0, 0, 0, 0)
           emisor 1 (d=-1)           emisor 2 (d=-1)
         = (-1, 1, 1, 1, 1, -1)

Ranura 1:  (0, 0, 0, 0, 0, 0)      +  (1, -1, -1, -1, -1, 1)
           emisor 1 (d=-1)            emisor 2 (d=+1)
         = ( 1, -1, -1, -1, -1,  1)
```

Decodificar con el código del emisor 2 (ranura 1):

```
S·c2 = 1·1 + (-1)(-1) + (-1)(-1) + (-1)(-1) + (-1)(-1) + 1·1
     = 1 + 1 + 1 + 1 + 1 + 1 = 6

d2_reconstruido = 6/6 = 1     OK,  el emisor 2 mandaba un 1
```

Decodificar con el código del emisor 1 (ranura 1):

```
c1 = (1, 1, 1, -1, 1, -1),   S_(1,·) = (1, -1, -1, -1, -1, 1)

S·c1 = 1·1 + (-1)·1 + (-1)·1 + (-1)(-1) + (-1)·1 + 1·(-1)
     = 1 - 1 - 1 + 1 - 1 - 1 = -2

d1_reconstruido = -2/6 = -1/3     NO es ni +1 ni -1
```

Ese `-1/3` es la interferencia del otro emisor. Y acá está la lección, que vale más que el ejercicio:

> **CDMA solo funciona si los códigos de los emisores son ORTOGONALES**, es decir, si su **producto**

> **interno es cero**: $\langle c_1, c_2 \rangle = \Sigma\, c_1(m) \cdot c_2(m) = 0$.

```
⟨c1, c2⟩ = (1)(1) + (1)(-1) + (1)(-1) + (-1)(-1) + (1)(-1) + (-1)(1)
         =  1  -  1  -  1  +  1  -  1  -  1
         =  -2        NO es cero.  NO ortogonales
```

Con un código del emisor 2 que **sí** sea ortogonal con $c_1 = (1, 1, 1, -1, 1, -1)$, por ejemplo $c_2' = (1, -1, 1, -1, -1, 1)$:

```
⟨c1, c2'⟩ = (1)(1) + (1)(-1) + (1)(1) + (-1)(-1) + (1)(-1) + (-1)(1)
          =  1  -  1  +  1  +  1  -  1  -  1
          =  0          OK, ORTOGONALES

Ahora, decodificando con c2', la contribución del emisor 1
desaparece EXACTAMENTE y la del emisor 2 sigue dando +6:
     d2_reconstruido = 6/6 = 1     OK
```

El resumen, en tres frases para copiar en el examen: 1. La salida de un emisor CDMA es el producto, chip a chip, del bit de dato por su código: $Z_{i,m} = d_i \cdot c_m$. 2. El canal es **aditivo**: todos los emisores se suman y el receptor ve una sola señal. 3. Cada receptor recupera sus datos **multiplicando por su propio código**; para que la señal de los otros emisores desaparezca, los códigos deben ser **ortogonales**.

> **Errores típicos:** usar `0` y `1` en vez de `+1` y `-1` (con `d = 0` la salida sería todo ceros y no distinguirías "mandó un 0" de "no mandó nada"); olvidar el factor `1/M` (sin normalizar obtenés `6` o `-6`, no `1` o `-1`); y confundir el código del emisor con la tasa de chips (el código tiene que ser **ortogonal con todos los demás** de la célula, y eso es lo que asigna la estación base).

2.7 Ejercicio 7 — ¿Por qué Ethernet conmutada no usa ALOHA?

Enunciado (inventado). En un laboratorio hay un switch de 24 puertos 100BASE-TX y ocho PCs. (a) ¿Cuál es el ancho de banda agregado del que disponen las ocho máquinas? (b) ¿Y si las ocho se conectan a un hub de 8 puertos? (c) Explicá por qué CSMA/CD es imprescindible en (b) y no aporta nada en (a).

(a) Con switch. Cada puerto es un dominio de colisión **separado**, y en 100BASE-TX el modo por defecto es **full-duplex**.

```
8 puertos x 100 Mb/s = 800 Mb/s de suma agregada en un sentido
(y otros 800 Mb/s en el sentido contrario: son enlaces
 independientes, no un bus de 800).

Limite real: el uplink al resto de la red. Si es 1 GbE,
topa en 1000; si es 100BASE-TX, topa en 100.
```

Respuesta: `8 × 100 = 800 Mb/s` agregados, o sea 100 Mb/s dedicados por máquina.

(b) Con hub. El hub es de capa 1: repite la señal a **todos** los puertos. Las ocho máquinas comparten un único medio de 100 Mb/s. El reparto ideal (sin colisiones) sería `100 / 8 = 12,5 Mb/s` por máquina, pero eso es teórico. Hagamos una estimación con el **mejor caso posible**: un medio ranurado, que es el techo del carrier sense.

```
k = 8 estaciones, Slotted ALOHA (que es el techo):
   p* = 1/(2k − 1) = 1/15 = 0,0667
   S  = p · (1−p)^(2(k−1)) = 0,0667 · (0,9333)^14
   (0,9333)^14 = e^(14 · ln 0,9333) = e^(14 · (−0,06899))
               = e^(−0,9659) = 0,3807
   S  = 0,0667 · 0,3807 = 0,0254  ->  2,54 %

100 Mb/s x 0,0254 = 2,54 Mb/s   <- alcanzable en el mejor
                                      caso teórico
```

Respuesta: 100 Mb/s totales, y en la práctica bastante menos que los 12,5 Mb/s ideales por máquina.

(c) Por qué CSMA/CD sí en (b) y no en (a). En (b) las ocho estaciones comparten un medio físico: dos pueden transmitir en el mismo instante y sus señales se suman en el cable. **Sin el mecanismo de CD no hay forma de detectar el choque**, y esperar un ACK es cien veces más lento. En (a) no hay nada que detectar, porque **no existe medio compartido**:

```
Enlace H1 <-> switch, full-duplex:

H1 ----------->  (una direccion)
H1 <-----------  (la otra direccion)

H1 transmite: su señal va SOLO al switch.
El switch transmite al mismo tiempo: su señal va SOLO a H1.

Los dos flujos usan caminos FISICOS DISTINTOS y en
direcciones opuestas. No se superponen. No hay colisión.
CSMA/CD seria codigo muerto.
```

Y por qué no es "el switch learning lo impide": el *self-learning* resuelve un problema **distinto**: decidir **a qué puertos** reenviar una trama.

```
Flooding  !=  colision

Flooding:   la MISMA trama sale por muchos puertos, UNA DESPUES
            DE OTRA. En el medio hay una sola señal a la vez.
            Nadie choca con nadie.

Colisión:   DOS tramas de fuentes distintas se superponen en
            UN MISMO medio en el MISMO instante.
```

El switch learning no evita colisiones; **el switch evita que exista un medio compartido**. Son dos afirmaciones distintas y la segunda es la que hay que argumentar.

El contraejemplo, que es lo que hay que saber decir:

```
Switch --- hub --- H1
            |
            +--- H2      <- aca SI hay colision,
            +--- H3          y SI hace falta CSMA/CD
            +--- H4

El hub crea un dominio de colisión por debajo del switch.
CSMA/CD no desaparece: se traslada al segmento del hub.
Ese es el motivo por el que la solución del examen 2010-07
dice "si alguna computadora trabaja en modo half-duplex,
si, en esa conexión computadora-switch se utilizará
CSMA/CD".
```

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2007-07, Pregunta 3 (10 puntos) — el problema central de la clase

> **Fuente:** Examen `sirc0707` / `eirc0707`, Pregunta 3, 31 de julio de 2007. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0707.pdf

Enunciado (texto literal del examen)

> **Pregunta 3 (10 puntos)**
>
> a) Indique cual es la diferencia de performance entre ALOHA и Slotted-ALOHA en el caso de carga normal. Justifique.
>
> b) Que tipos de protocolos CSMA hay. Describa cual es la diferencia entre ellos.
>
> ⚠️ **Nota sobre la solución oficial:** el PDF publicado de `sirc0707` **no incluye solución para las preguntas teóricas** (solo trae la de los Problemas 1 y 2). Las dos partes que siguen están desarrolladas por mí, a partir de la derivación de la Parte 2, y se marcan **(solución propia)**. Lo que se corrige es el razonamiento, no una fórmula oficial.

Parte a) — la diferencia de rendimiento

Respuesta corta, la que hay que escribir primero:

> Slotted-ALOHA tiene el **doble de rendimiento** que ALOHA puro: su eficiencia máxima es `1/e ≈ 36,8 %` contra `1/(2e) ≈ 18,4 %` de ALOHA puro.

Justificación completa. El razonamiento tiene tres pasos y conviene hacer los tres:

Paso 1 — Qué significa "carga normal". La red ofrece tramas a una **tasa constante e independiente** (proceso de Poisson), que es el supuesto estándar. Si la tasa es `λ` y la trama dura `T`, la carga ofrecida es `G = λ·T`.

Paso 2 — La ventana de colisión, que es donde está toda la diferencia.

```
ALOHA PURO
──────────
Yo transmito de t = 0 a t = T.

Cualquier trama que llegue a t entre 0 y 2T me destruye:
  · de 0 a T    -> se solapa por detrás
  · de T a 2T   -> se solapa por delante (mi frente todavía viaja)
──────────────────────────────────────────────────────────────
VENTANA = 2T


SLOTTED-ALOHA
─────────────
Solo puedo empezar en t = 0, T, 2T, ...

Dos tramas que empiecen en ranuras distintas NO pueden solaparse.
La única colisión posible es empezar en la MISMA ranura.
──────────────────────────────────────────────────────────────
VENTANA = T
```

Paso 3 — La cuenta.

```
Probabilidad de que llegue una trama durante una ventana de W
segundos, siendo lambda la tasa de arribos:
    numero medio de arribos = lambda · W
    P(ninguno)              = e^(-lambda · W)

ALOHA puro   (W = 2T):
    P(sobrevive) = e^(-lambda · 2T) = e^(-2G)
    eficiencia  S = G · e^(-2G)
    dS/dG = 0  -->  G* = 1/2
    S_max = (1/2)·e^(-1) = 1/(2e) = 0,184

Slotted      (W = T):
    P(sobrevive) = e^(-lambda · T) = e^(-G)
    eficiencia  S = G · e^(-G)
    dS/dG = 0  -->  G* = 1
    S_max = 1 · e^(-1) = 1/e = 0,368
```

La respuesta "por qué exactamente el doble", que es lo que el "Justifique" está pidiendo:

```
La ventana se redujo a la mitad (de 2T a T), así que para la
misma carga G la probabilidad de sobrevivir pasó de e^(-2G)
a e^(-G): un factor e a favor. Pero además la carga que
maximiza el rendimiento se duplicó (G* pasó de 1/2 a 1),
y eso aporta otro factor 2:

    S_slotted(1)  /  S_puro(1/2)  =  1·e^(-1)  /  (0,5·e^(-1))
                                  =  2
```

Consecuencia práctica: ALOHA puro **nunca puede pasar del 18,4 % de la capacidad del medio**, por más vacía que esté la red. Slotted-ALOHA llega al 36,8 %. Ninguno de los dos es aceptable en producción, y esa es exactamente la razón por la que Ethernet no usa ALOHA: usa **carrier sense**, que elimina casi todas las colisiones salvo las de duración ε (§1.6.1).

	ALOHA puro	Slotted-ALOHA
Cuándo se puede transmitir	en cualquier instante	**solo al comienzo de una ranura**
Sincronización	ninguna	reloj común o preámbulo
Ventana vulnerable	`2T`	`T`
Probabilidad de sobrevivencia	`e^(−2G)`	`e^(−G)`
Carga óptima `G*`	0,5	1
Eficiencia máxima	**`1/(2e)` = 0,184**	**`1/e` = 0,368**
Tramas por segundo a 10 Mb/s con 64 B	14 680	29 360

Parte b) — los tipos de CSMA y sus diferencias *(solución propia)*

Respuesta corta: hay dos ejes de clasificación, **persistencia** y **detección de colisión**, que dan **cuatro combinaciones**.

Eje 1 — ¿Qué hace la estación mientras encuentra el medio ocupado?

Tipo	Comportamiento	Ventaja	Desventaja
1-persistente	espera mirando; transmite en cuanto el medio queda libre	no desperdicia medio libre	varias que esperaban transmiten a la vez, y hay colisión
No persistente	espera un tiempo **aleatorio** y vuelve a escuchar	casi no hay colisiones	regala ancho de banda: el medio puede quedar libre sin que nadie transmita
p-persistente (medios ranurados)	transmite con prob. `p`, con prob. `1−p` espera una ranura	compromiso continuo	parametrizable pero en desuso

Eje 2 — ¿Qué hace la estación cuando hay colisión?

Tipo	Comportamiento	Consecuencia
Sin detección	espera a que el medio se libere y reintenta	cada colisión cuesta un **ACK** completo, o sea un RTT entero
Con detección	aborta la trama, emite **jamming** y hace **backoff exponencial**	la colisión se detecta en microsegundos y solo se pierde lo transmitido

	Sin detección	**Con detección**
1-persistente	1-persistent CSMA	**CSMA/CD** — Ethernet por cable
No persistente	non-persistent CSMA	CSMA/CD no persistente — no se usa en la práctica

La diferencia de fondo entre "con detección" y "sin detección" (la frase que más puntos vale):

> En un medio **cableado**, la tarjeta puede **escuchar mientras transmite** y comparar lo que manda con lo que vuelve: si no coinciden, hubo colisión. Eso es **detección directa**, y cuesta microsegundos.
>
> En un medio donde **no** se puede escuchar mientras se transmite (radio), o donde **las estaciones no se escuchan entre sí** (estación oculta), no hay detección directa. La única forma de enterarse es **no recibir el ACK**: detección **indirecta**, por ausencia, que cuesta un RTT entero.

> Esa es la razón de que exista **CSMA/CD** para el cable y **CSMA/CA** para wireless: cuando no podés **detectar** la colisión, tenés que **evitarla**, y eso se hace haciendo que los demás se callen (RTS/CTS, NAV y ACK obligatorio en 802.11).

Para cerrar: en la práctica, la Ethernet conmutada full-duplex **no usa ninguno de los dos**, porque no hay colisión posible (§3.2). Y en los medios cableados compartidos la combinación que se usa es **1-persistente con detección**: persistencia 1 porque el medio es rápido y no conviene regalarlo, y detección porque el cable la permite.

3.2 ⭐ Examen 2010-07, Pregunta 2 (8 puntos) — CSMA/CD en wireless y en Ethernet conmutada

> **Fuente:** Examen `sirc1007` / `eirc1007`, Pregunta 2, 27 de julio de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1007.pdf

Enunciado (texto literal del examen)

> **Pregunta 2 (8 puntos)**
>
> a) ¿Es recomendable utilizar CSMA/CD en una red wireless 802.11? Justifique.
>
> b) En un medio Ethernet completamente switcheado, ¿es necesario utilizar CSMA/CD? Justifique su respuesta.

Parte a) — ¿CSMA/CD en una red wireless 802.11?

Solución oficial (del PDF):

> a) No, pues CSMA/CD está especificado tal que aun en el peor caso, o sea, una estación transmitiendo la trama más corta permitida y destinada a la estación más alejada "en el cable" y ésta comenzando a transmitir un tiempo epsilon arbitrariamente pequeño y positivo antes que llegue el primer bit de la trama, comience también a transmitir, todas las estaciones se enteran de la colisión. En el caso de redes wireless 802.11 existen varios casos, y no necesariamente "extremos" en los que no podemos asegurar que todas las estaciones se escuchen entre sí, detectando así las colisiones que pudiesen ocurrir al transmitir dos o más estaciones pertenecientes a la misma BSS con un solapamiento al menos parcial en el tiempo. Un ejemplo "clásico" es "el problema de la estación oculta".

Análisis de la solución oficial. El razonamiento es correcto y muy bien elegido por la cátedra: reconstruye el **peor caso** de CSMA/CD (la estación más lejana empezando un ε antes de que llegue el primer bit) y muestra que **en un medio cableado ese peor caso está garantizado por la geometría**, mientras que en 802.11 **no existe tal extremo**: hay pares de estaciones que no se escuchan entre sí.

Lo que yo le agrego (argumentos distintos y complementarios; sumarlos sube el puntaje):

1. **Es físicamente imposible, no solo poco confiable.** La solución oficial razona sobre la **alcancibilidad** de la señal. El argumento más fuerte es de otra naturaleza: **una radio no puede recibir en la misma banda mientras transmite**. Su propia señal llega al frente del receptor con unos 20 dBm (100 mW); una señal co-canal de una estación vecina llega típicamente con −70 dBm. La diferencia es de **90 dB, es decir cerca de un millón de veces en potencia**, y el receptor no tiene forma de separar lo que él mismo emite de lo que entra del canal. **No es que el protocolo sea malo: es que la física lo impide.**
2. **CSMA/CD necesita un tiempo de ida y vuelta acotado, y la radio no lo tiene.** La corrección de CSMA/CD depende de que la trama mínima sea **más larga que el tiempo de ida y vuelta del medio** (§1.8). En un cable ese tiempo está fijado por el largo máximo del segmento: por eso el estándar puede declarar "la trama mínima son 512 bits". En radio hay movilidad, fading, estaciones ocultas, y el presupuesto de tiempo no tiene un mínimo razonable: **la trama mínima tendría que ser gigantesca y el rendimiento se iría a cero**. En resumen, **la condición que hace funcionar a CSMA/CD no es verificable en radio**.
3. **El modelo de costo está invertido.** En CSMA/CD el emisor aborta al detectar, así que la energía desperdiciada es **una fracción de trama**. En wireless el único que puede juzgar la integridad de la trama es **el receptor** (por el CRC), así que el emisor tiene que **transmitir la trama entera** para enterarse. Si vas a transmitir entera de todos modos, **evitarla es infinitamente mejor que detectarla**. De ahí el nombre: CSMA/**CA**.
4. **El sentido de portadora sí funciona, la detección no.** Matiz importante: en 802.11 una estación **sí puede** escuchar el medio antes de transmitir (carrier sense). Lo que no puede es **detectar una colisión mientras transmite**. Por eso 802.11 conserva el "CSMA" y cambia el "CD" por "CA": **RTS/CTS y NAV** para silenciar a las estaciones vecinas, **backoff exponencial** para las colisiones que igual ocurren, y **ACK obligatorio**.

Respuesta redactada para el examen:

> No es recomendable, y en la práctica es imposible.
>
> **Primero**, porque una radio es half-duplex por naturaleza: no puede recibir en la misma banda mientras transmite, y su propia señal (unos 20 dBm) enmascara por cerca de 90 dB cualquier transmisión ajena. La detección de colisión durante la transmisión es físicamente inviable.
>
> **Segundo**, porque CSMA/CD garantiza la detección en el peor caso gracias a que en un medio cableado toda estación que recibe la señal de otra necesariamente está en el mismo medio, y a que el tiempo de ida y vuelta está acotado por el largo máximo del segmento (de ahí los 512 bits de trama mínima). En 802.11 ninguna de las dos condiciones se cumple: la estación oculta rompe la simetría de cobertura, y no hay cota de propagación utilizable.
>
> **Tercero**, porque en radio el emisor no puede juzgar la colisión: solo el receptor puede, por CRC, con lo cual hay que enviar la trama entera para enterarse. Detectar sería tirar la trama a la basura después de gastarla.
>
> **Por eso 802.11 usa CSMA/CA**: en lugar de detectar la colisión, la evita. Hace RTS/CTS para que las estaciones vecinas asignen un NAV y se callen, espera un backoff exponencial aleatorio, y exige un ACK del receptor. El sentido de portadora sí se conserva: lo que no se puede es la detección.

Parte b) — ¿CSMA/CD en una Ethernet completamente switcheada?

Solución oficial (del PDF):

> b) Si hablamos de un medio completamente switcheado (sin presencia de hubs) y por lo tanto una sola computadora por puerto de switch y si asumimos que todos los puertos de los switches así como las tarjetas de red de las computadoras conectadas a ellas están configuradas para trabajar en modo full-duplex, no es necesario utilizar CSMA/CD. Si alguna computadora trabaja en modo half-duplex, sí, en esa conexión "computadora-switch" se utilizará CSMA/CD.

Análisis de la solución oficial. Es correcta y además es cuidadosa: **no dice "no es necesario", dice "no es necesario *si y solo si* se cumplen dos condiciones** — (1) no hay hubs, una máquina por puerto; y (2) todo en full-duplex. Y cierra con el matiz que muchos alumnos se pierden: **si una máquina queda en half-duplex, CSMA/CD vuelve**. Esa última frase es la que hay que repetir.

Mi desarrollo — por qué no hay colisión, y por qué no es "porque el switch lo impide".

El error más común es decir "*el switch no deja que se colisionen*". Eso mezcla dos mecanismos distintos. La respuesta correcta tiene cuatro pasos:

Paso 1 — Qué es una colisión. Dos señales que se superponen **en el mismo medio físico en el mismo instante**. La definición ya dice "mismo medio". Si no hay medio compartido, no hay definición aplicable.

Paso 2 — Qué hace el full-duplex. En un enlace punto a punto full-duplex, cada sentido es un camino físico independiente:

```
H1 ======>  switch     (dirección A: de la estación al switch)
H1 <=====  switch     (dirección B: del switch a la estación)

Son dos medios separados. H1 y H2 no comparten NADA:
H1 transmite hacia el switch por el par 1-2 del cable de H1,
H2 transmite hacia el switch por el par 1-2 del cable de H2.

Que las señales lleguen al switch al mismo instante NO es
una colisión: el switch tiene un puerto de entrada distinto
para cada una. Lo que hay es COLA, que es otro problema
(control de flujo, capa 2), y se resuelve con buffers.
```

Paso 3 — Conclusión. En un enlace full-duplex punto a punto, cada estación tiene **un dominio de colisión de tamaño 1**: ella sola. La colisión **no es improbable ni rara: es geométricamente imposible**, porque requeriría que dos señales compartieran un medio, y no hay medio compartido. **CSMA/CD sería código muerto**: la máquina escucharía, compararía su señal con la que vuelve, concluiría eternamente que "todo bien", y esperaría una colisión que no puede ocurrir.

Paso 4 — qué papel juega el switch learning. El *self-learning* resuelve un problema **distinto**: **decidir a qué puertos reenviar cada trama**.

```
Flooding (dominio de BROADCAST, no de colisión):
la MISMA trama sale por varios puertos, UNA DESPUES DE OTRA.
En cada instante hay una sola señal en el medio. Nadie choca.

Colisión (dominio de COLISIÓN):
DOS tramas de fuentes distintas, superpuestas en un medio
compartido, en el MISMO instante.

+-----------------------------------------------------+
| flooding  = desperdicio de tiempo                    |
| colisión  = simultaneidad en un medio compartido     |
| Son ejes distintos y NO se pueden confundir.         |
+-----------------------------------------------------+
```

El learning evita el flooding de unicast (que desperdicia ancho de banda), pero **no tiene absolutamente nada que ver con las colisiones**. Decir "el switch evita las colisiones" es correcto solo en el sentido indirecto de que **elimina el medio compartido**; dicho así, es impreciso y el corrector lo va a marcar.

Respuesta redactada para el examen:

> No es necesario. **Porque no hay colisión posible, y no porque el switch la impida.** Una colisión exige que dos señales se superpongan en un mismo medio físico en el mismo instante. En una Ethernet conmutada full-duplex, cada enlace es punto a punto y cada sentido es un camino independiente, así que cada puerto es un dominio de colisión de tamaño 1 y la colisión es geométrically imposible.
>
> **El self-learning resuelve otro problema**: decidir a qué puertos reenviar cada trama. Sin él, las tramas unicast de destino desconocido se inundarían por todos los puertos. Eso es *flooding*, la misma trama saliendo por varios puertos uno después de otro: un desperdicio de ancho de banda, **no una colisión**, que exige simultaneidad sobre un medio compartido. Confundir ambos conceptos es el error típico de esta pregunta.
>
> **Condiciones y contraejemplo:** el resultado vale bajo las dos hipótesis de la solución oficial: una sola máquina por puerto (sin hubs) y todo en full-duplex. Si hay un hub debajo del switch, ese tramo vuelve a ser un único dominio de colisión compartido y el CSMA/CD regresa ahí. Y si una sola tarjeta queda en half-duplex, en ese enlace específico se usa CSMA/CD, con colisiones tardías (*late collisions*) que aparecen a los 512 tiempos de bit de la trama mínima.

3.3 Examen 2007-02, Pregunta 1 (5 puntos) — Pure ALOHA, Slotted ALOHA y CSMA/CD

> **Fuente:** Examen `eirc0702`, Pregunta 1, febrero de 2007. **Enunciado:** "Que diferencias tienen Pure ALOHA, Slotted ALOHA y CSMA/CD."

Es la pregunta comparativa clásica y es casi un resumen de la clase:

	Pure ALOHA	Slotted ALOHA	CSMA/CD
Escucha el medio antes de transmitir	no	no	**sí**
Cuándo puede empezar	en cualquier instante	**solo al inicio de una ranura**	en cuanto el medio está libre
Sincronización	ninguna	reloj común	ninguna
Ventana de colisión	`2T`	`T`	`ε` (prácticamente cero)
Detecta la colisión	no (por ausencia de ACK)	no (por ausencia de ACK)	**sí, durante la transmisión**
Reacción	retransmitir al azar tras esperar	retransmitir al azar tras esperar	**abortar, jamming y backoff exponencial**
Reintentos	al azar, sin cota	al azar, sin cota	exponencial, **tope 16**
Hardware especial	ninguno	reloj	**tarjeta que escucha mientras transmite**
Eficiencia máxima	`1/(2e)` = 18,4 %	`1/e` = 36,8 %	cercana al 100 % a baja carga
Medio típico	satélite (ALOHAnet)	sistemas ranurados	coaxil y par trenzado de Ethernet

La frase de síntesis (2 puntos fáciles): la evolución de los tres es **"cada uno le agrega una pieza al anterior"**. A Slotted se le agrega un reloj, que reduce la ventana de `2T` a `T` y duplica el rendimiento. A CSMA/CD se le agrega el sentido de portadora, que elimina la ventana casi por completo (de `T` a `ε`), y la detección en línea, que hace que la colisión cueste microsegundos en vez de un RTT. **CSMA/CD no es "otro ALOHA": es ALOHA con los sentidos y los oídos puestos.**

3.4 Examen 2010-12, Pregunta 3 a) y b) (8 puntos) — dominios de colisión y de broadcast

> **Fuente:** Examen `sirc1012`, Pregunta 3, diciembre de 2010. *(El apartado c), el de self-learning, es de la Clase 42 y no se resuelve acá.)* **Enunciado:** "a) Considere una red de varios nodos interconectados por un hub, ¿cuál es el dominio de colisión y cuál es el de broadcast? Justifique. b) Si en la red anterior cambiamos el hub por un switch. ¿Se modifican los dominios de colisión y broadcast? ¿De qué manera?"

a) Con hub: los dos dominios son iguales y abarcan toda la red. El hub es un dispositivo de **capa 1**: no implementa ningún protocolo de acceso al medio, no aprende direcciones y no decide nada, se limita a repetir la señal eléctrica por todos los puertos. Como todos los puertos comparten el mismo medio físico, las señales de todas las estaciones se superponen (**un único dominio de colisión**); y como todo se reenvía por todos los puertos, toda trama de broadcast o de destino desconocido llega a todos (**un único dominio de broadcast**).

b) Con switch: el dominio de colisión se parte en uno por puerto; el de broadcast no cambia.

	Con hub	Con switch
Dominio de colisión	uno, con todos los nodos	**uno por puerto**; con full-duplex, de tamaño 1
Dominio de broadcast	uno, con todos los nodos	**uno, igual**: el switch reenvía las tramas de broadcast por todos los puertos

Por qué se parte el de colisión. El switch es de **capa 2**: aprende direcciones MAC y decide a qué puerto reenviar cada trama. Al hacerlo, **elimina el medio compartido** y lo reemplaza por enlaces punto a punto independientes. Cada puerto es un medio físico propio, con sus dos emisores (la estación y la tarjeta del switch). Por lo tanto **cada puerto es un dominio de colisión**, y las colisiones solo pueden ocurrir entre la estación conectada a ese puerto y el switch.

Por qué el de broadcast NO cambia. Una trama de broadcast tiene destino `FF:FF:FF:FF:FF:FF`: **por definición todos tienen que oírla**. El switch no tiene información para descartarla, así que la reenvía por **todos los puertos del mismo dominio de broadcast**. Lo único que acota ese dominio son las **VLAN** (Clase 42): una etiqueta 802.1Q crea dominios de broadcast lógicos independientes dentro del mismo switch.

```
+-----------------------------------------------------+
|  LA FRASE QUE HAY QUE DECIR                         |
|                                                      |
|  El switch parte el DOMINIO DE COLISION porque       |
|  elimina el medio compartido.                        |
|                                                      |
|  El switch NO parte el DOMINIO DE BROADCAST porque   |
|  no puede descartar una trama que es por definicion  |
|  para todos.                                         |
+-----------------------------------------------------+
```

> **Nota sobre la solución oficial de 2010-12:** la cátedra escribe "el switch impediría que ocurran colisiones". Como discutimos en §3.2, esa redacción es **imprecisa**: el switch no *impide* colisiones, **elimina la condición que las hace posibles** (el medio compartido). En una red con hubs colgados del switch **sí hay colisiones**. La respuesta correcta es "un dominio de colisión por puerto", que es lo que dice también la solución oficial de `2010-07`.

3.5 Examen 2007-03, Pregunta 5 (5 puntos) — CSMA/CA

> **Fuente:** Examen `sirc0703`, Pregunta 5, marzo de 2007. **Enunciado:** "Describa el mecanismo de acceso al medio CSMA/CA (Carrier Sense Multiple Access With Collission Avoidance)."

Solución oficial (del PDF), extractada:

> Uno de los primeros protocolos diseñados para LAN inalámbricas fue CSMA/CA o MACA. La idea es que el emisor estimula al receptor enviando una trama corta, de forma que las estaciones cercanas puedan detectar la transmisión y evitar transmitir durante el envío de una trama de datos. [...] Considere que A quiere enviar una trama a B: comienza enviando una trama del tipo RTS a B, esta trama corta contiene el largo de la trama que sigue. Luego B responde con una trama CTS. [...] Cualquier estación escuchando un

> RTS está cercana a A y no debe enviar datos para que A reciba el CTS sin interferencia. Cualquier estación cercana a B no debe enviar datos para que A pueda realizar el envío de datos sin interferencia.

Análisis de la solución oficial. Está impecable y es exactamente lo que hay que saber:

1. **RTS/CTS es el corazón**: A avisa, B confirma, y ambos anuncian la **duración** de la transferencia siguiente.
2. **La asimetría del aviso es la clave**: el RTS lo oyen las estaciones **cerca de A**; el CTS lo oyen las estaciones **cerca de B**. Cada grupo silencia a su lado.
3. **El NAV**: una estación que escucha el RTS pero no el CTS **igual deduce que hay un envío en curso** y se calla el tiempo anunciado en el RTS. Ese temporizador virtual es el **NAV** (*Network Allocation Vector*), un "sentido de portadora virtual".
4. **Los dos casos que la solución menciona y que valen puntos extra**: (a) una estación C que escucha el RTS pero no el CTS y que gracias al NAV se calla; (b) **dos estaciones que envían RTS al mismo tiempo al mismo receptor**, colisión inevitable que se resuelve con el backoff aleatorio.
5. **MACA y CSMA/CA no son lo mismo**: MACA (1978) es el antepasado; 802.11 lo extendió con el ACK obligatorio y los espacios entre tramas.

Lo que le agrego (la Clase 43 lo desarrolla en detalle): los **IFS** (SIFS, DIFS, EIFS) que dan prioridad a las tramas de control sobre las de datos —con `SIFS = 10 µs` y `DIFS = SIFS + 2 × aSlotTime` en las PHY OFDM— y la **ventana de backoff** `CW` de 802.11, que va de `CWmin = 15` a `CWmax = 1023` en pasos `CW = 2^n − 1` con `n` de 4 a 10. O sea: **la misma idea exponencial que en CSMA/CD**, con el rango `[0, CW]` en vez de `[0, 2^k − 1]`.

PARTE 4 — CIERRE

📌 Resumen

1. **Medio compartido** = un solo medio para varios. Si dos transmiten a la vez, las señales se **superponen** y nadie recibe nada: **colisión**.
2. **ALOHA puro**: transmite de inmediato. Ventana vulnerable de `2T`. Eficiencia `G·e^(−2G)`, máximo en `G = 1/2`, valor `1/(2e) ≈ 18,4 %`.
3. **Slotted-ALOHA**: solo al inicio de una ranura. La ventana cae a `T`, la supervivencia se multiplica por `e`, la carga óptima se duplica, y el resultado es `1/e ≈ 36,8 %`: **exactamente el doble**.
4. **ALOHA con franjas**: `S(N,p) = N·p·(1−p)^(N−1)`, máximo en `p = 1/N`, valor `(1−1/N)^(N−1)` que **tiende a** `1/e` cuando `N` tiende a infinito.
5. **CSMA = escuchar antes de hablar.** Elimina todas las colisiones salvo las que caen dentro de una ventana `ε`, porque el sentido de portadora **siempre** tiene un retraso.
6. **Persistencia: 1-persistente** no desperdicia medio libre (Ethernet); **no persistente** casi no colisiona pero regala ancho de banda; **p-persistente**, un compromiso, no se usa.
7. **Detección: con detección** (CSMA/CD) solo es posible en cable, porque la tarjeta escucha mientras transmite; **sin detección**, la colisión se detecta por la **ausencia de ACK** y cuesta un RTT entero.
8. **CSMA/CD**: 1-persistente, detección, **jamming de 32 bits**, **backoff exponencial** `r ∈ [0, 2^k − 1]` y tope de **16 intentos**. En radio no se puede: de ahí **CSMA/CA** (RTS/CTS, NAV, ACK obligatorio).
9. **Slot time = 512 bits**, y la **trama mínima = 64 bytes** porque es exactamente el tiempo de ida y vuelta de la señal en el peor caso. A 10 Mb/s son 51,2 µs; a 1 Gb/s, 0,512 µs, insuficiente para 100 m: de ahí la extensión de portadora.
10. **Con switch full-duplex no hay colisión** porque **no hay medio compartido** (no porque el switch la "impida"), y el **dominio de broadcast sigue siendo toda la red** hasta que una VLAN lo parta.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué la ventana de colisión del ALOHA puro mide `2T` y no `T`? Dibujá el caso de una trama que empieza justo después de que terminó la tuya.

<details>
<summary>Respuesta</summary>
</details>

2. Slotted-ALOHA duplica exactamente el rendimiento de ALOHA puro. Demostralo con las fórmulas.

<details>
<summary>Respuesta</summary>
</details>

3. ¿Qué valor de `p` maximiza `N·p·(1−p)^(N−1)`? Mostrá la derivación.

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué el emisor tiene que seguir enviando bits después de detectar una colisión? ¿Qué son esos 32 bits?

▶ Respuesta

5. ¿De dónde sale el número 512 del slot time, y por qué el mismo número sirve a 10, 100 y 1000 Mb/s?

▶ Respuesta

6. En Ethernet conmutada full-duplex, ¿por qué no hay colisiones? ¿Tiene algo que ver con que el switch aprenda direcciones?

▶ Respuesta

7. ¿Por qué no se puede usar CSMA/CD en 802.11? Dá **dos** razones que no sean la misma.

▶ Respuesta

8. Un switch tiene un hub conectado en uno de sus puertos, con tres PCs colgando. ¿Dónde hay colisión?

▶ Respuesta

🔗 Referencias

- **Examen `sirc0707` / `eirc0707`** — Pregunta 3 (10 pts): rendimiento ALOHA contra Slotted-ALOHA, y tipos de CSMA. **Sin solución oficial publicada** para las preguntas teóricas.
- **Examen `sirc1007` / `eirc1007`** — Pregunta 2 (8 pts): CSMA/CD en wireless y en Ethernet conmutada. Con solución oficial.
- **Examen `eirc0702`** — Pregunta 1 (5 pts): diferencias entre Pure ALOHA, Slotted ALOHA y CSMA/CD.
- **Examen `sirc1012`** — Pregunta 3 (8 pts): dominios de colisión y de broadcast con hub y switch (apartados a y b).
- **Examen `sirc0703`** — Pregunta 5 (5 pts): CSMA/CA y RTS/CTS. · `sirc0807` — Pregunta 3: CSMA/CA en modo infraestructura (Clase 43).
- **Práctico 5 de 2008** — Ejercicios 5 y 6 (CDMA) y ejercicio 8 (eficiencia de ALOHA con franjas).
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 3 y cap. 4 (§4.1, subcapa MAC: ALOHA, Slotted ALOHA, CSMA, Ethernet); cap. 6 y 7 para la discusión de por qué el acceso aleatorio no funciona en radio.
- **Kurose & Ross**, *Computer Networking: A Top-Down Approach* — §5.3 (ALOHA, Slotted ALOHA, CSMA, CSMA/CD) y §5.1.3 (CDMA)
- **IEEE 802.3** — Ethernet: slot time, trama mínima, jamming, backoff binario exponencial, extensión de portadora. **No es una RFC de IETF**: es un estándar del IEEE.
- **IEEE 802.11** — CSMA/CA, DCF, IFS, NAV, RTS/CTS, EDCA.
- **IEEE 802.3cg** — 10BASE-T1S, el único caso industrial donde el half-duplex sigue siendo obligatorio (con PLCA).
- **IEEE 802.1Q-2022**, **802.1w** (RSTP) y **802.1AX-2020** — VLAN, spanning tree y agregación de enlaces (Clase 42).
- **RFC 826** — ARP (request en broadcast, reply en unicast), que es quien sufre el dominio de broadcast.
- **Clase anterior:** 40 — Corrección de errores y ARQ (Hamming, Go-Back-N, Selective Repeat)

- **Clase siguiente:** 42 — Switch, hub, VLAN y STP: self-learning paso a paso, tabla MAC, ageing, STP/RSTP

Clase 42

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 El hub: la red que repite (y por eso es una sola red)
- 1.2 El bridge y el switch
- 1.3 Self-learning: el algoritmo de reenvío completo
- 1.4 La tabla de ageing
- 1.5 Hub vs. switch: la tabla que hay que saber de memoria
- 1.6 VLAN: por qué la red conmutada todavía no alcanza
- 1.7 STP y RSTP: por qué la capa 2 necesita un protocolo de control de loops
- 1.8 ⚠️ Actualización a 2026
- 2.1 Ejercicio 1 — Self-learning paso a paso con seis tramas
- 2.2 Ejercicio 2 — El ageing de la tabla
- 2.3 Ejercicio 3 — Partir una VLAN y saber qué tags van por dónde
- 2.4 Ejercicio 4 — Dibujar el árbol de STP y decidir qué puertos se bloquean
- 3.1 Examen 2010-12, Pregunta 3 (8 puntos) — hub, switch y self-learning
- 3.2 Examen 2008-12, Pregunta 2 (5 puntos) — qué mejora reemplazo hubs por switches
- 3.3 Examen 2009-03, Pregunta 3 (6 puntos) — el concepto de VLAN
- 📌 Resumen en 10 líneas
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: E (Capa de Enlace) · **Semana:** 14 · **Duración estimada:** 110 min **Correlativa:** Clase 41 (acceso al medio: ALOHA, CSMA/CD, CSMA/CA) · **Sigue con:** Clase 43 (redes inalámbricas 802.11) **Material de referencia:** Tanenbaum cap. 3 (§3.4, 3.6 y 3.7) · IEEE 802.3 · IEEE 802.1D · IEEE 802.1Q · IEEE 802.1w

---

🎯 Al terminar esta clase vas a poder

1. Explicar por qué el **hub** produce **un solo dominio de colisión y un solo dominio de broadcast**, y por qué el switch parte el primero pero **no** el segundo.
2. Describir el mecanismo de **self-learning** y ejecutar el **algoritmo de reenvío completo, paso a paso**, llenando la tabla de reenvío después de cada trama.
3. Justificar por qué la tabla necesita **ageing**, y explicar por qué los **300 s son un default de fabricante y no un valor del estándar IEEE**.
4. Armar la tabla **hub vs. switch** completa (dominios, ancho de banda por puerto, duplex y **seguridad**) y sostener por qué un hub es un problema de seguridad, no solo de rendimiento.
5. Explicar qué es una **VLAN** y **por qué sirve**: aislar dominios de broadcast, mover un usuario sin tocar el cable, separar puertos de gestión.
6. Describir la **trama 802.1Q** campo por campo: **tag de 4 bytes**, **VLAN ID de 12 bits (1–4094)**, **C-TAG ()**, **S-TAG ()** y **PVID**; y por qué hace falta un puerto *tagged* y uno *untagged*.
7. Explicar por qué un **loop en capa 2 es una tormenta de broadcast** y por qué los bridges transparentes **no usan TTL**, y ejecutar el algoritmo de **STP**: raíz, puerto raíz, puertos designados y bloqueados.
8. Resolver completo el **Examen 2010-12 P3** (8 pts), el **2008-12 P2** (5 pts) y el **2009-03 P3** (6 pts).

> **Prerequisito del examen:** (8 pts — hub vs. switch, dominios de colisión y de broadcast, y self-learning: **el corazón de esta clase**) · (5 pts — mejoras de performance y seguridad al pasar de hubs a switches) · (6 pts — concepto de VLAN 802.1Q y su utilidad).

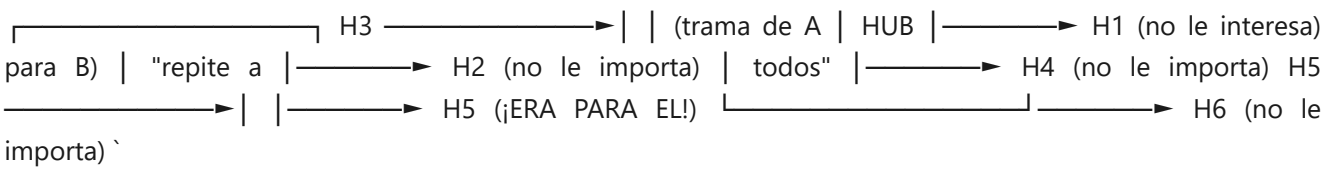
Los tres aparecen juntos en el mismo bloque del syllabus: **conmutación, segmentación lógica y control de loops**. Esta clase es la bisagra entre la 41 (que habla de colisiones) y la 43 (un medio donde las colisiones no se pueden detectar).

> Esta clase **no repite** ALOHA ni CSMA/CD: ya están en la 41. Acá el medio **ya no es compartido**, así que el problema dejó de ser "quién puede hablar" y pasó a ser "**a quién le hablo**".

PARTE 1 — TEORÍA

1.1 El hub: la red que repite (y por eso es una sola red)

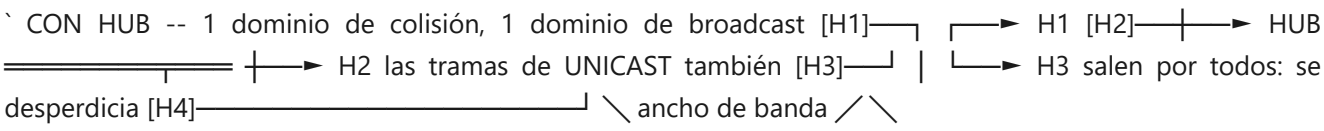
Un **hub** es un dispositivo de **capa 1**. No lee direcciones, no decide nada, no tiene memoria: toma la señal eléctrica que entra por un puerto y la **regenera y copia a todos los demás puertos**.



De esa definición sale **todo** lo que el hub es, sin ningún cálculo:

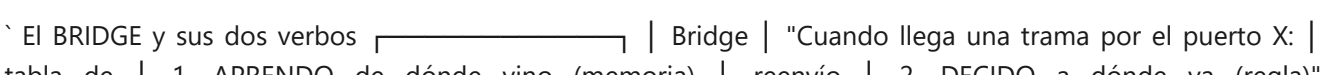
Consecuencia	Por qué
Un solo dominio de colisión	todas las señales comparten **el mismo medio eléctrico**. Si H1 y H5 transmiten a la vez, las señales se suman y se destruyen
Un solo dominio de broadcast	una trama de broadcast se copia a **todos** los puertos, y cada estación copiada la vuelve a copiar a todos los puertos de su hub. Nada la detiene
Sin ancho de banda por puerto	el ancho de banda es **del medio compartido** y se reparte: estaciones a Mbps tienen Mbps cada una
Half-duplex obligatorio	un solo medio común: si el puerto escucha lo que él mismo puso, oye su propia señal
Cero seguridad	todo lo que pasa por el medio es visible en **todos** los puertos, siempre

> **La frase que hay que saber de memoria:** el hub no es un dispositivo de red con funciones reducidas; es **un pedazo de cable con corriente**. Todo lo demás se deduce de eso.



1.2 El bridge y el switch

El **bridge** (*puente*) nació para unir **LANs** (grupos de estaciones que comparten un medio), no puertos. La diferencia con el hub es conceptual: **mira la trama y decide**, y tiene exactamente **dos verbos**: reenviar a un puerto concreto (*unicast forwarding*), o reenviar a todos los puertos (*flooding*).



└──────────────┘ ┌──────────────────────┐ │ MAC destino │ puerto │ ¿De dónde sale la tabla? De la ├──────────────┼──────────────────────┤ observación de que toda trama que │ 00:1A:...:01 │ 3 │ entra por un puerto viene de la │ 00:1A:...:07 │ 5 │ estación conectada a ese puerto. └──────────────┴──────────────────────┘ No es configuración: es memoria.

Filtrado explícito: llega por el puerto 5 una trama de A (destino B), y la tabla dice B -> 3 caso 1: entrada (B, 3) existe -> reenviar por el puerto 3 [1 copia] caso 2: entrada (B, 5) existe -> DESCARTAR [0 copias] caso 3: no existe entrada (B) -> FLOODING por 1,2,3,4,6 [5 copias] El caso 2 es lo que un hub NO puede hacer, y es toda la diferencia entre "repetir" y "conmutar". `

Un **switch** es, estructuralmente, **un bridge con muchos puertos y una matriz de conmutación (*switching fabric*) interna**. La familia 802.3 no distingue: un switch **es** un bridge multipuerto.

```
Anatomía de un switch ┌──────────────────────────────────────────────┐ │ MATRIZ DE CONMUTACIÓN (switching fabric) │ │ sin cuello de botella: mucho más rápida │ │ que los puertos │ └──┬─────┬─────┬─────┬─────┬──┘ │ │ │ │ │ ┌─┴─┐ ┌─┴─┐ ┌─┴─┐ ┌─┴─┐ ┌─┴─┐ │puerto│ │puerto│ │puerto│ │puerto│ │puerto│ cada uno con │ 1 │ │ 2 │ │ 3 │ │ 4 │ │ 5 │ su buffer └──────┘ └──────┘ └──────┘ └──────┘ └──────┘ │ │ │ │ │ H1 H2 H3 H4 H5 ┌──────────────────────────────────────────────┐ │ MAC address table (CAM table) — UNA por VLAN │ │ 00:1A:2B:3C:4D:01 -> puerto 1 ageing 287 s │ │ 00:1A:2B:3C:4D:05 -> puerto 5 ageing 12 s │ └──────────────────────────────────────────────┘
```

Tres consecuencias, y las tres salen en el examen:

- **La matriz es mucho más rápida que los puertos.** Por eso **varias comunicaciones pueden ocurrir simultáneamente**: mientras el puerto 1 transmite hacia el 2, el puerto 3 transmite hacia el 4. En un hub es imposible: hay un solo medio.
- **Cada puerto tiene su propio buffer**, así que una ráfaga hacia un destino lento **no bloquea** los otros puertos.
- **Cada puerto es un dominio de colisión independiente** y puede trabajar en **full duplex**. En full duplex **la colisión es físicamente imposible**: no hay medio compartido que la genere.

> **Ojo con el exametero.** La frase "el switch no evita colisiones" es **falsa** y es el error conceptual más repetido del tema. Lo que el switch hace es **eliminar el medio compartido**; como no hay medio compartido, **no hay dónde colisionar**. Mismo resultado, causa distinta.

1.3 Self-learning: el algoritmo de reenvío completo

Acá está el corazón de la clase y lo que pide el **Examen 2010-12 P3c**.

> El switch aprende **la dirección MAC origen** de cada trama que entra, y la asocia al **puerto por el que entró**. Todo lo demás —qué hacer con esa trama— es **inferencia**.

¿Por qué el origen y no el destino? Porque el origen es **información verificada**: la estación conectada al puerto 3 es, necesariamente, la que puso esa MAC en el campo origen. El destino es lo que **quien manda** quiere que creamos; cualquiera puede poner una MAC cualquiera ahí. Si el switch aprendiera del destino, un solo nodo malicioso poisoning la tabla entera.

¿Por qué "inferencia"? Porque cuando el switch **no** encuentra la dirección destino no puede saber a qué puerto va. Lo único razonable es mandarla a **todos menos por el que entró**: devolvérsela al remitente sería entregarle una copia que ya posee. Esa es la definición operativa de **flooding**.

` Llega una trama por el PUERTO X, con MAC ORIGEN A y MAC DESTINO B.

┌──────────────

disponibilidad: si asignás la MAC de alguien a un puerto viejo y esa persona se muda, **queda sin red** hasta que alguien corrija la configuración.

1.5 Hub vs. switch: la tabla que hay que saber de memoria

Cada fila tiene una causa, no una descripción suelta.

Característica	HUB	SWITCH
Capa OSI que opera	1	2
Lee la MAC	NO	SÍ
Dominio de colisión	UNO	UNO POR PUERTO
Dominio de broadcast	UNO	UNO (salvo VLANs)
Puertos que salen	TODOS, siempre	solo el del destino
Ancho de banda	V/N por estación	V por puerto, a V c/u
Full duplex	NO	SÍ
Colisiones	SÍ	NO (en full duplex)
Coste por byte útil	BAJO	ALTO (se replica)
Latencia	~0 (es cable)	del orden de µs
Filtrado	NO	SÍ
SEGURIDAD	NINGUNA	el tráfico NO se ve
SNMP / gestión	mínimo	completa

La fila de seguridad merece un párrafo propio. En un hub la seguridad es literalmente cero: el medio es compartido, así que **cualquiera puede poner un sniffer en cualquier puerto y ver todo el tráfico** (contraseñas en texto claro, claves de WEP/WPA-PSK, cabeceras de las peticiones de sesión).

Con un switch, en régimen, cada trama sale **por un solo puerto**: el del destino. Un atacante en el puerto 3 ve su propio tráfico pero **no** el de la estación del puerto 5. Eso es un salto real.

Pero — y esto es lo que dice la solución oficial del 2008-12 y hay que repetir — el switch tiene una ventana insecure:

> **Durante la fase de aprendizaje el switch se comporta exactamente como un hub.** Está inundando tramas para averiguar quién está en qué puerto, y durante esa ventana **un atacante ve todo**.

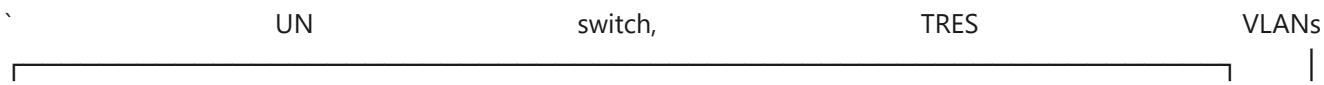
Y hay un segundo límite, más sutil: si un atacante **inunda la tabla con miles de MAC origen falsas** (*MAC flooding*), el switch agota su memoria de reenvío y **vuelve a inundar por todo**, degradado a hub. ⇒ **El switch no es un mecanismo de autenticación; es, como mucho, un mecanismo de contención del daño.**

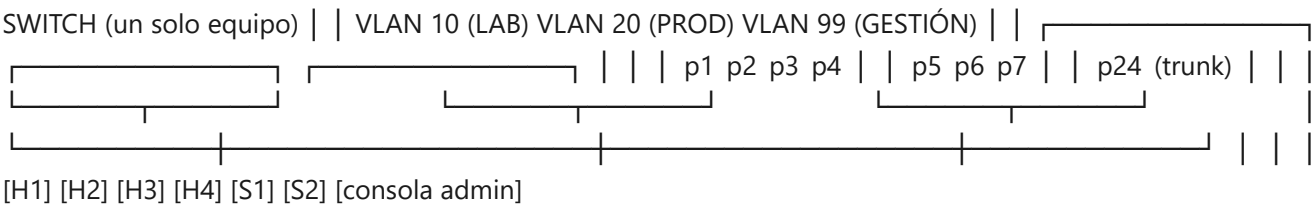
1.6 VLAN: por qué la red conmutada todavía no alcanza

Después de poner switches queda un problema: una trama de **broadcast** (o de **unicast a un destino desconocido**) sale por **todos** los puertos, así que **el flooding no tiene fronteras**. Consecuencias: con 300 PC, cada broadcast genera 300 copias y cientos de respuestas (ARP, DHCP, mDNS); los servidores de producción reciben tramas que no son para ellos; y un problema en un puerto de un laboratorio inunda la red entera.

Esas dos consecuencias son **la misma causa**, y es justamente lo que rompe la seguridad (§1.5). Por eso el switch no alcanza para segmentar.

Una **VLAN** (*Virtual LAN*, LAN virtual) es **una partición lógica del switch**: un mismo switch físico se comporta como **varios switches independientes**, uno por VLAN.





El flooding de una trama de H1: ✗ NO sale por p5, p6, p7 (son otra VLAN) ✓ sale solo por p2, p3, p4 El dominio de broadcast se PARTIÓ: ahora hay 3, uno por VLAN. `

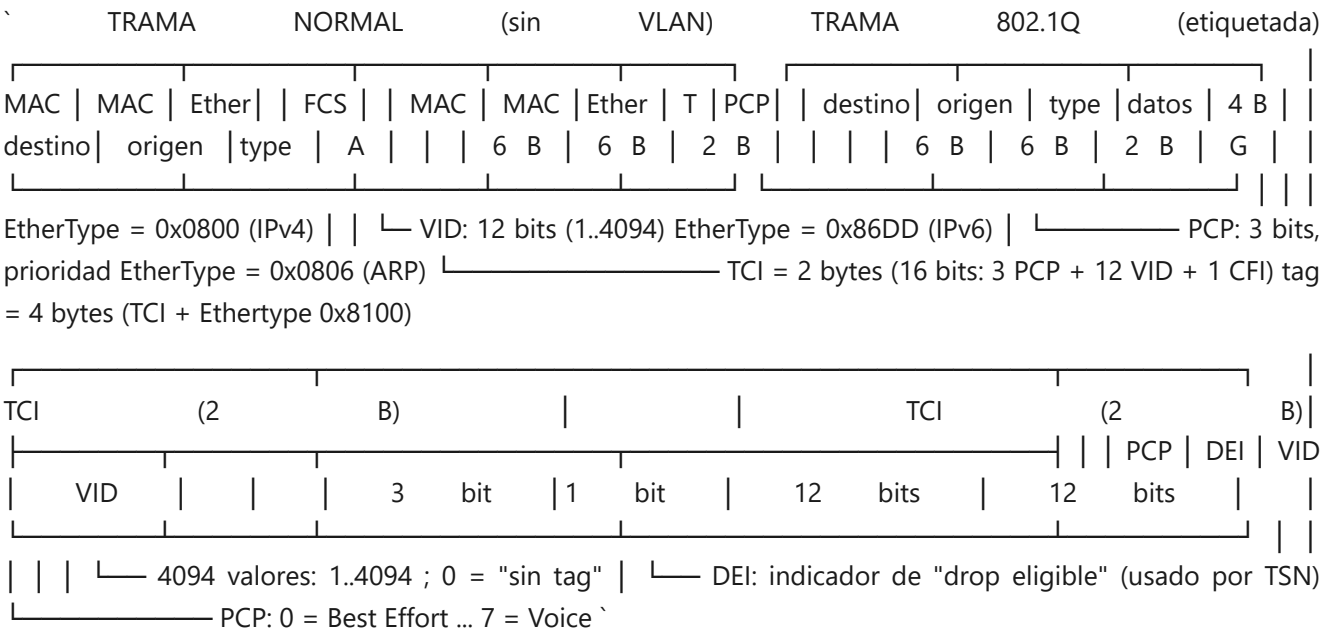
Las cuatro utilidades, con su porqué

Utilidad	Explicación
1. Partir el dominio de broadcast	el flooding se confina a los puertos de la VLAN. Una VLAN lógica puede vivir en un switch con cableado físico
2. Seguridad	si un usuario de la VLAN 10 pone un sniffer en su puerto, **no ve nada de la VLAN 20**. Es el mismo argumento del switch, pero por encima del switch
3. Mover un usuario sin mover el cable	la VLAN a la que pertenece el usuario se decide **en el switch**. Si se muda de escritorio, el switch no cambia: cambia **una línea de configuración**
4. Separar puertos de gestión	las VLAN de gestión (o de voz) separan el tráfico de administración del de datos, de modo que **inundar la VLAN de gestión no lleva a todos los puertos**

El punto 3 es el más olvidado y el más usado. La razón histórica de las VLAN es administrativa: antes, mover a alguien de oficina significaba repatchear cables. Con VLAN, **es configuración**.

802.1Q: la trama etiquetada, campo por campo

Si todas las tramas de una VLAN viajan por el mismo puerto, el switch **no necesita poner nada en la trama**. El problema aparece cuando la trama tiene que **atravesar varios switches**: el switch intermedio necesita saber a qué VLAN pertenece. Solución: **un tag de 4 bytes**.



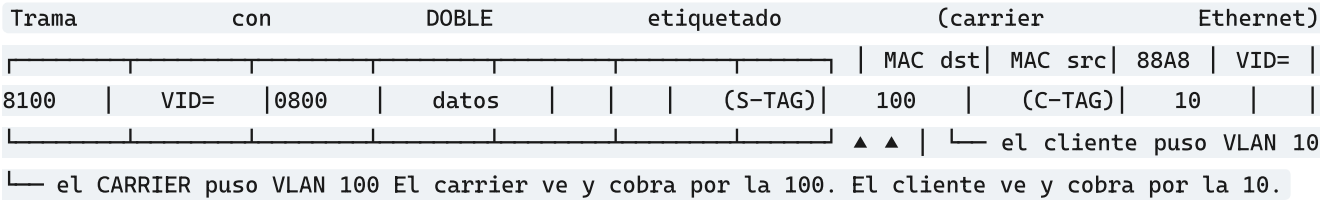
El detalle fino que casi nadie menciona: el tag de 4 bytes es **TPID (2 bytes) + TCI (2 bytes)**. El `Ethertype` original (0x0800, 0x86DD…) **no desaparece**: pasa a ocupar el lugar que tenía el tag. Por eso el campo "Tipo" de

una trama con tag es `0x8100`, no `0x0800`.

C-TAG vs. S-TAG

Etiqueta		Quién la usa	Dónde aparece
C-TAG	****	el **cliente** de la VLAN (la empresa)	primera etiqueta de la trama
S-TAG	****	el **proveedor** del servicio (el carrier)	etiqueta **exterior**, que el carrier pone **encima**

¿Por qué dos? Porque en *carrier Ethernet* hay **dos operadores de VLAN en el mismo cable**: el carrier (que agrupa a muchos clientes) y el cliente (que agrupa a sus VLANs). Si ambas usaran no se sabría cuál termina primero.



PVID, tagged y untagged: por qué un puerto tiene que ser de las dos cosas

Si **todo** lleva tag, las máquinas que no saben de VLANs (una PC, una impresora, una cámara IP) no pueden entender la trama. Y si **nada** lleva tag, el switch intermedio no sabe a qué VLAN pertenece.

` ┌──┐ │ PUERTO "ACCESS" (untagged, de usuario final) │ │ ├─ sale SIN tag │ │ ├─ el switch le ASIGNA un solo VLAN ID: el PVID │ │ └─ el usuario nunca ve un tag: su tarjeta no lo pondría │ │ │ PUERTO "TRUNK" (tagged, entre switches o hacia el router) │ │ ├─ sale CON tag (0x8100) para todas las VLANs que lo usa │ │ ├─ el switch NO pone nada a mano: el VLAN ID viaja en la trama │ │ └─ es lo que permite "router-on-a-stick" │ └──┘

PVID = Port VLAN ID: el VLAN ID que se ASIGNA a las tramas que llegan a ese puerto SIN tag.

Ejemplo: la trama de H1 (VLAN 10) H1 ──► [p1, access, PVID=10] ──► [p24, trunk] ──► [router] (1) H1 manda SIN tag: es una PC normal, no sabe qué es una VLAN. (2) Entra por p1. El switch ve "sin tag, PVID de p1 = 10" y le ADJUNTA el tag: [.. | 8100 | VID=10 | 0800 | IP ..] (3) Sale por p24 (trunk): conserva el tag. (4) El router separa por VLAN: 0.10 -> 192.168.10.1/24, 0.20 -> 192.168.20.1/24. ACÁ termina el dominio de broadcast, porque el router NO reenvía broadcast entre interfaces. (5) Si el vecino del trunk no tiene esa VLAN en su tabla, se descarta. `

Los números del tag

` Sin tag : mínimo 64 B máximo 1518 B C-TAG : mínimo 68 B máximo 1522 B (+4) S+C-TAG : mínimo 72 B máximo 1526 B (+8)

⚠️ EL MTU NO CAMBIA: sigue siendo 1500 bytes de payload IP. Los 4 bytes salen del relleno (padding) de la trama mínima. Proof: 1522 − 18 (cabecera) − 4 (tag) = 1500 bytes de payload. `

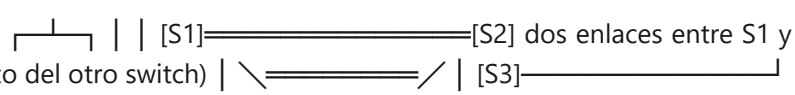
1.7 STP y RSTP: por qué la capa 2 necesita un protocolo de control de loops

El problema: un loop en capa 2 no se muere solo

En IP, un paquete que entra en un lazo **muere**: cada router descuenta el **TTL** y cuando llega a 0 lo descarta. Ese contador es la red de seguridad de la capa 3.

Los bridges transparentes no tienen nada de eso. Son *transparentes*: su trabajo es que la red se comporte como si no existieran, así que no pueden tocar nada de la trama. En particular **no decrementan ningún TTL**, porque la trama Ethernet **ni siquiera tiene campo TTL**.

` CON UN LOOP EN CAPA 2 (S1 y S2 unidos por DOS enlaces)

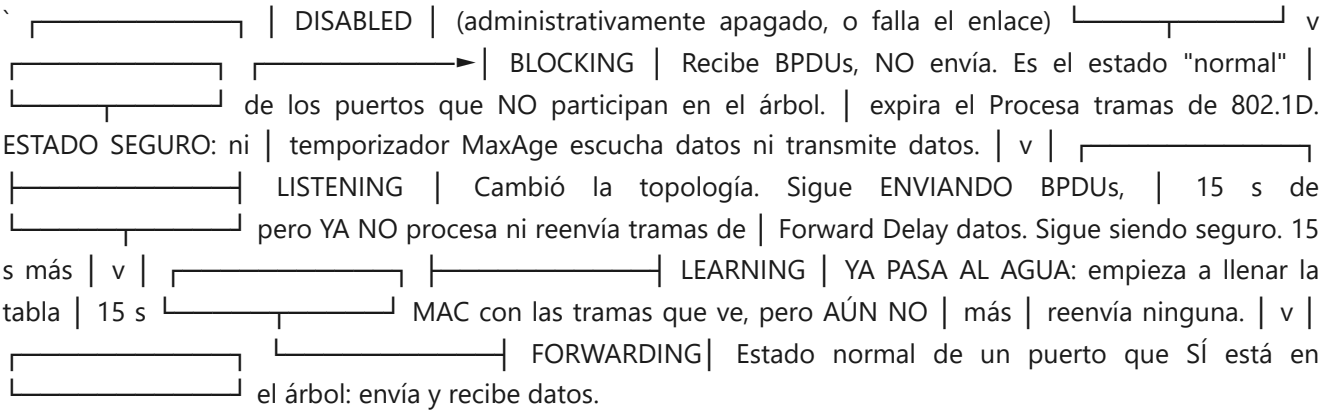
H1  [S1]=====[S2] dos enlaces entre S1 y S2 (puede ser un cable y un puerto del otro switch) [S3]

H1 manda UNA trama de BROADCAST. t0: S1 la recibe por p1 y la reenvía por TODOS los demás puertos. t1: S2 la recibe. NO tiene entrada para su origen (es la PRIMERA vez que la ve) -> FLOODING por todos sus puertos. t2: Vuelve a S1 por OTRO puerto. S1 tampoco la descarta: es la entrada de ageing del ORIGEN, y ese origen no es la dirección de DESTINO (que es broadcast), así que S1 no la reconoce -> la reenvía por todos los puertos menos por donde entró. t3: ... y así indefinidamente. La trama NUNCA muere. `

Lo que mata a la red no es que una trama circule: es que **el número de copias crece** mientras los enlaces se saturan. En régimen permanente cada puerto está al 100 %, los buffers están llenos, la CPU está al 100 % procesando tramas que descarta, y **el switch deja de responder a ping**. Los síntomas característicos: la red entera se cae en el momento en que alguien enchufa un segundo cable entre dos switches, y las tablas MAC **parpadean** (cada MAC aparece en dos o más puertos a la vez, *MAC flapping*).

> **El punto conceptual que se pregunta:** un router **descarta** el paquete porque su TTL llegó a 0. Un bridge **no puede**, porque **no sabe si la trama es "la misma" que ya vio antes**. El spanning tree no evita los loops: **los calcula y les pone una puerta**, de modo que la topología lógica sea un **árbol**, que por definición no tiene bucles.

Los estados de un puerto

`  DISABLED (administrativamente apagado, o falla el enlace) v BLOCKING Recibe BPDUs, NO envía. Es el estado "normal" de los puertos que NO participan en el árbol. expira el Procesa tramas de 802.1D. ESTADO SEGURO: ni temporizador MaxAge escucha datos ni transmite datos. v LISTENING Cambió la topología. Sigue ENVIANDO BPDUs, 15 s de pero YA NO procesa ni reenvía tramas de Forward Delay datos. Sigue siendo seguro. 15 s más v v LEARNING YA PASA AL AGUA: empieza a llenar la tabla 15 s MAC con las tramas que ve, pero AÚN NO más reenvía ninguna. v FORWARDING Estado normal de un puerto que SÍ está en el árbol: envía y recibe datos.

⏱ Temporizadores del STP clásico (802.1D) en sus defaults: Hello 2 s (cada cuánto se manda un BPDU) Max Age 20 s (sin oír BPDUs => el vecino "murió") Forward Delay 15 s (dos veces: Listening->Learning y Learning->Forwarding) `

El tiempo total de convergencia del STP clásico es de ~30 a 50 segundos, y esa es la razón por la que **nadie usa STP hoy**: un enlace que se cae y vuelve deja la red parada medio minuto. **RSTP** lo comprime a **sub-segundo** (§1.8).

Bridge ID: cómo se decide quién manda

` BRIDGE ID = 64 bits

┌──┐ | PRIORIDAD | MAC ADDRESS | | 16 bits | 48 bits | | (4096 c/u) | (la MAC de una de sus interfaces) | └──┘ 32768 = prioridad BASE (el default de 802.1D) 24576, 16384, 8192, 4096 = los valores "nominales" que se usan para forzar que un bridge sea la raíz. ⚠️ Tiene que ser múltiplo de 4096. Si ponés 1000, el switch lo ajusta a 0, que es el valor más bajo posible: ¡GANA!

PRIORIDAD BASE = 32768 -> en hexadecimal = 0x8000

S1: prioridad 32768 (0x8000), MAC 00:00:00:00:00:01 -> 8000.0000.0000.0001 S2: prioridad 32768 (0x8000), MAC 00:00:00:00:00:02 -> 8000.0000.0000.0002 S3: prioridad 4096 (0x1000), MAC 00:00:00:00:00:03 -> 1000.0000.0000.0003 <-- RAÍZ `

La comparación es numérica, de izquierda a derecha: primero la prioridad, después la MAC. Por eso para cambiar la raíz **no alcanza con la prioridad**: si dos switches tienen la misma, gana el que tenga la MAC más chica, y eso depende de la marca de la placa, no de vos.

El algoritmo completo

` 1. Cada bridge manda BPDUs (Bridge Protocol Data Units) por TODOS sus puertos, cada 2 segundos, a la dirección de multicast 01:80:C2:00:00:00. Un BPDU lleva: Bridge ID, costo de cada puerto, prioridad de puerto y Max Age.

2. ELECCIÓN DE LA RAÍZ. Cada bridge escucha los BPDUs. Si oye un Bridge ID MENOR que el suyo, sabe que no es la raíz. -> La raíz es el bridge con el Bridge ID más bajo de toda la red. No hay votación: es un mínimo.
3. PUERTO RAÍZ (root port) en cada bridge no-raíz. Para cada puerto se calcula el camino a la raíz. Gana el de: (a) menor COSTO hasta la raíz (b) si hay empate, el que recibió el BPDU con menor Bridge ID (c) si hay empate, el de menor ID de puerto emisor Ese es el ÚNICO puerto por el que el bridge habla con el resto.
4. PUERTOS DESIGNADOS en cada segmento. En cada LAN (cada enlace entre dos bridges) compiten los bridges conectados. Gana el de MENOR Bridge ID: su puerto queda DESIGNATED y es el único autorizado a enviar datos hacia ese segmento.
5. EL ÁRBOL. - puertos RAÍZ y DESIGNADOS -> FORWARDING - todos los demás -> BLOCKING
6. El grafo resultante es un ÁRBOL: de cualquier LAN a cualquier otra hay EXACTAMENTE UN camino. No puede haber bucle. `

```
Topología FÍSICA (con un loop) Topología LÓGICA (el árbol) ┌─────┐ ┌─────┐ | S1
├═══════════════════════════┤ S1 | raíz └──┬──┘ | | └──┬──┘ | ┌──┴──┐ | | p1 RAÍZ ┌──┴──┐ |
S3 | | ┌──┴──┐ | S2 ├──┤ ├──┤ | S2 | └──┬──┘ └──┬──┘ | └──┬──┘ | | | | p1 DESIGNATED
┌──┴──┐ ┌──┴──┐ | ┌──┴──┐ | S3 | | S4 | | | S3 | └──┬──┘ └──┬──┘ | └──┬──┘ p2 -> BLOCKING 4
switches, 5 enlaces: el loop ya no existe lógicamente
```

1.8 ⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **Formato de trama**: preámbulo 7 + SFD 1, **MAC destino 6 + MAC origen 6**, 2, payload, **FCS 4**. **MTU 1500**. - **ARP ()**: request en **broadcast**, reply en **unicast**. El *gratuitous ARP* sigue usándose (anuncio, DHCP, failover de gateway, VRRP) y sigue siendo **fuente clásica de ARP spoofing y cache poisoning**. - **MAC learning**: aprende la **MAC origen** y asocia puerto; **flooding** si el destino es desconocido; **unicast** si se conoce; **filtrado** si el destino está en el mismo puerto de entrada. - **802.1Q**: tag de 4 bytes, **VLAN ID de 12 bits (1–4094)**, = sin tag (PVID). - **STP/RSTP** y sus estados (Blocking, Listening, Learning, Forwarding), Bridge ID con **prioridad base 32768**. - **Loops en capa 2 = tormenta de broadcast**: sigue siendo la razón de ser del spanning tree.

Cambió o se modificó — y acá hay que ser preciso:

1. **El half-duplex Ethernet es historia por encima de 1 Gb/s.** Y hay un doble error que circula: - Es una decisión de **IEEE 802.3**, **no una RFC de IETF**. **No existe ninguna RFC que lo depreque.** La que circula como tal, **, es JSCalendar**, y no tiene nada que ver con Ethernet. - A 10GbE y por encima el **full duplex punto a punto es obligatorio** y el half-duplex no existe en la especificación de esas velocidades. - **PERO half-duplex no desapareció del todo: IEEE 802.3cg 10BASE-T1S** (*single-pair Ethernet*, para redes PLC industriales sobre un solo par) es **obligatoriamente half-duplex**, con un protocolo de acceso propio, **PLCA**. Y en IETF, el módulo YANG `csma-cd` de `802.3cf` está marcado `deprecated`. ⇒ **Saber cuándo CSMA/CD sigue existiendo es parte del tema.**
2. **** es el estándar actual (precedido por): la revisión *2018** abrió el bloque de **TSN** con **** (tráfico programado),** (preempción de trama, junto con) y **(filtrado y policing por stream); la revisión *2022** agregó **** (Stream Reservation Protocol) y** (asynchronous traffic shaping*). *Para el examen no cambia nada del tag ni de la VLAN: cambia para qué se usan, porque las VLAN pasaron de ser segmentación de oficina a ser la base de la red determinista*.*
3. **LAG: (2000) → .** La revisión vigente es ***. *Un* Link Aggregation Group* une varios enlaces físicos en uno lógico, para más ancho de banda y redundancia.** No es lo mismo que un spanning tree*: es "usá los dos cables en paralelo", y no genera loops porque el switch decide el puerto por* hash*.
4. **C-TAG () vs. S-TAG ()**: el doble etiquetado pasó de exótico a **estándar en carrier Ethernet y en EVPN-VXLAN**. Y acá va una distinción que se confunde siempre: en el datacenter, **EVPN () sobre VXLAN ()** es el estándar de overlay, y ahí la distribución de VLANs **no la hace 802.1Q**: el tenant se identifica por el **VNI de 24 bits que viaja en el encabezado VXLAN sobre UDP/4789**, y el queda solo por compatibilidad. **802.1Q etiqueta la trama; VXLAN encapsula y multiplica el MTU.** No son lo mismo.
5. ** () es el default real**, no el STP clásico: la convergencia va de sub-segundo a unos pocos segundos, en vez de los ~30–50 s del 802.1D. () cuando querés** compartir el árbol entre varias VLANs** y así no tener un spanning tree por VLAN.
6. ⚠️ **El "aging de 300 s" de la tabla MAC es default de FABRICANTE, no un valor mandated por IEEE.** Escribí "típicamente 300 s en switches comerciales, configurable". **Nunca** "el estándar IEEE dice 300 s".

Consecuencia para el examen: - "*¿CSMA/CD es necesario hoy?*" → **no, y ni siquiera existe por encima de 1 Gb/s** (salvo 10BASE-T1S industrial). - "*¿qué protocolo evita los loops en capa 2?*" → **spanning tree**, y el **default real es RSTP (802.1w)**. Si el enunciado es de 2009/2010 y habla de "STP", respondé el mecanismo del árbol y **agregá una línea** de que hoy converge RSTP. - *VLAN* → el tag de 4 bytes y el VID de 12 bits **no cambiaron**. Lo que cambió es la **revisión del estándar (802.1Q-2022)** y el doble etiquetado C-TAG/S-TAG.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Self-learning paso a paso con seis tramas

Enunciado (inventado). Un switch de 6 puertos tiene los hosts `H1..H6` en los puertos 1..6, con MAC `00:...:01` a `00:...:06`. La tabla arranca vacía. Llegan seis tramas, en orden.

` [H1]──p1──┐ [H2]──p2──┤ [H3]──p3──┼──► SWITCH A1 = 00:1A:00:00:00:01 (H1, puerto 1) [H4]──p4──┤ ... hasta A6 = ...:06 (H6, puerto 6) [H5]──p5──┤ [H6]──p6──┘ tabla = { } (estado inicial)

Tramas: 1) p1, de A1 a BROADCAST 2) p3, de A3 a A1 3) p5, de A5 a A3 4) p2, de A2 a A6 5) p6, de A6 a A1 6) p1, de A1 a A6 `

Trama 1. *Aprende* . *Decide*: **nunca** está en la tabla ⇒ **flooding** por 2, 3, 4, 5, 6. → Es el momento en que la red "se siente como un hub": la primera trama SIEMPRE se inunda, porque el switch no sabe nada todavía.

Trama 2. Aprende `A → p1`**. Decide*: `B` está en `p3`, y `A ≠ p3` ⇒ unicast por p3. → Es la** primera** vez que el switch puede hacer unicast sin adivinar.

Trama 3. Aprende `A → p1`**. Decide*: `C` está en `p4`, `A ≠ p4` ⇒ unicast por p4**.

Trama 4. *Aprende* . *Decide*: **NO** está en la tabla ⇒ **flooding** por 1, 3, 4, 5, 6.

⚠️ ERROR CLÁSICO. Muchos escriben "A6 -> puerto X" al ver el flooding. NO: el flooding NO aprende nada. Lo único que se aprende en esta trama es que A2 está en el puerto 2. Y H6, que recibe la trama por el flooding, la descarta: nadie le avisa que la estaba esperando.

Trama 5. *Aprende* (por fin habla la estación del puerto 6). *Decide*: está en ⇒ **unicast por p1**.

Trama 6. *Aprende* **y refresca su temporizador de ageing**. *Decide*: está en ⇒ **unicast por p6**.

Tabla de verificación

Trama	Aprende	¿Destino conocido?	Acción	Puertos de salida	Tabla resultante
1		broadcast (nunca)	**flooding**	2,3,4,5,6	
2		sí ()	**unicast**	1	
3		sí ()	**unicast**	3	
4		**no**	**flooding**	1,3,4,5,6	
5		sí ()	**unicast**	1	
6	(refresca)	sí ()	**unicast**	6	(igual)

Interpretación física: de 6 tramas, **2 fueron flooding (10 copias)** y **4 unicast (4 copias)**: 14 copias sobre el medio, de las cuales **10 eran redundantes**. La eficiencia del switch **crece con el tiempo**: a medida que la tabla se puebla, el costo por trama baja de copias a **una**. Y obsérvese que ** NO está en la tabla**: H4 nunca habló, así que el switch no tiene idea de que existe; si H4 mandara una trama, sería un flooding.

El caso que falta en las seis tramas: el filtrado

` Trama 7: entra por p2, de A4 a A2.

Paso 1: tabla[A4] <- p2 Paso 2: B = A2. tabla[A2] = p2. Y = p2 = el puerto de entrada. -> Y == X -> DESCARTAR.

Salida: NINGUN PUERTO. 0 copias.

¿Qué es esto físicamente? La trama venía por el mismo cable por el que el switch la enviaría. A2, que está en el mismo segmento, ya la oyó directamente en el medio. Reenviársela sería duplicarla sin motivo. Esto es el FILTRADO: exactamente lo que un hub no puede hacer. `

⚠️ Errores típicos

- **Aprender del destino.** El switch aprende **solo** del origen. Escribir " porque hubo flooding" es el error más caro del ejercicio.
- **No mencionar el caso → descartar.** El 2010-12 lo escribe como un "y" condicional dentro de un solo inciso; si lo desplegás aparte, sumás.
- **Reenviar el flooding también por el puerto de entrada.** No: por definición.
- **Tratar el broadcast como un caso aparte del "destino desconocido".** Es un caso *particular*: la dirección de broadcast jamás está en la tabla, así que siempre cae en la rama de inundar.
- **Olvidar el ageing.** Cada aprendizaje **refresca** el temporizador.

2.2 Ejercicio 2 — El ageing de la tabla

Enunciado (inventado). Un switch tiene ageing de 300 s. La estación E habla según un **proceso de Poisson** de tasa λ tramas por segundo. **(a)** ¿En qué instante se elimina su entrada? **(b)** ¿Probabilidad de que siga viva a los 300 s? **(c)** Con λ = 0,005 tramas/s (una cada 200 s), ¿probabilidad de que expire antes de la siguiente trama?

(a) El instante de expiración. El temporizador se pone en (última vez que se vio el origen) y corre; la entrada se elimina cuando llega a 300 s.

`t=0 ──────────────────────────── t=300 s |◄──────── 300 s de ageing ────────►| └── se ve A5, timer(A5) = 300 └── la entrada se BORRA`

Interpretación: el switch **no hace una pasada de limpieza cada 300 s**: cada entrada tiene **su propio** temporizador, que se reinicia cada vez que se ve esa MAC. Si llega una trama de `A` en `p1`, la entrada se refresca y el reloj vuelve a cero.

(b) Probabilidad de que la entrada sobreviva. Sobrevive si y solo si en esos 300 s **no llega ninguna trama** de `A`. Con Poisson, `P = e^(−λT)`. Es la misma fórmula de ALOHA de la clase 41: **la misma matemática para dos cosas distintas** — allá medía tramas que sobrevivían a una colisión, acá entradas que sobreviven a un olvido.

(c) Con tramas/s.

` λ = 0,005 tramas/s (una trama cada 1/0,005 = 200 s) Exponente: λ · 300 = 0,005 · 300 = 1,5

e^(1,5) = e · e^(0,5) = 2,71828 × 1,64872 = 4,48169

P(sobrevive) = e^(−1,5) = 1 / 4,48169 = 0,22313 P(expira antes de la siguiente) = 1 − 0,22313 = 0,77687 ≈ 77,7 % `

Interpretación: una estación que habla **una vez cada 200 s** tiene casi **4 de cada 5 veces** su entrada **borrada** cuando mande la siguiente trama. Es una consecuencia directa del criterio de diseño: **300 s de ageing para una estación que habla cada 200 s es un valor mal elegido**.

`λ (tramas/s) intervalo medio λ·300 P(expira) veredicto ——————— ——————— ——— ——— ——— 1 1 s 300 ~0 (e^-300) nunca molesta 0,1 10 s 30 e^-30 ≈ 9e-14 nunca molesta 0,01 100 s 3 1 - e^-3 = 0,950 molesta a veces 0,005 200 s 1,5 0,777 molesta siempre 0,001 1000 s 0,3 1 - e^-0,3 = 0,259 aceptable 0,0002 5000 s 0,06 1 - e^-0,06=0,058 aceptable`

Interpretación física: el umbral de "molesto" está alrededor de `M/2`, es decir, cuando la estación habla **una vez cada ~300 s o menos seguido**. Ahí la tabla se llena de entradas que mueren y renacen, y cada renacimiento cuesta **una inundación completa**.

(d) Qué pasa en la red cuando la entrada expiró. manda su trama con destino `B`: la entrada de `B` ya no está, así que `B` es un **destino desconocido** ⇒ **flooding** por los puertos 1, 2, 3, 4, 6; los demás la reciben y la descartan; y el switch **aprende** con 300 s de reloj. **El flooding es el precio de la incertidumbre, y se paga una sola vez por cada olvido.**

> **Consecuencia de diseño, y es la del Examen 2009-03 P3:** en una red con VLANs, ese flooding **queda confinado a los puertos de la VLAN**. Por eso el ageing no es una "pequeña pérdida de rendimiento", sino el detonante de la razón por la que la VLAN es una medida de **seguridad**: la fuga se produce, pero **no sale de la VLAN**.

2.3 Ejercicio 3 — Partir una VLAN y saber qué tags van por dónde

Enunciado (inventado). Un switch de 24 puertos:

` VLAN 10 "PRODUCCIÓN" -> puertos 1 a 16 (access, untagged) VLAN 20 "OFICINAS" -> puertos 17 a 22 (access, untagged) VLAN 99 "GESTIÓN" -> puerto 23 (access, untagged) puerto 24 -> TRUNK, con las VLANs 10, 20 y 99 PVID por defecto del puerto 24 = 10.

Router R: 0.10 -> 192.168.10.1/24 0.20 -> 192.168.20.1/24 0.99 -> 192.168.99.1/24 (configuración "router-on-a-stick") `

(a) Trama de (p1, VLAN 10) hacia (p20), ambos access de la VLAN 10.

` (1) P1 manda: [Dst | Src | 0800 | IP] SIN tag (una PC no sabe de VLANs) (2) Entra por p1 SIN tag. El switch aplica el PVID = 10 y ADJUNTA el tag: [Dst | Src | 8100 | VID=10 | 0800 | IP] (3) Sale por p20, que es access de VLAN 10: le QUITA el tag. P20 recibe una trama normal, sin saber que hubo VLAN.

💡 Ambos puertos están en la MISMA VLAN, así que la comunicación FUNCIONA sin tocar el router. El switch no rutea: solo reenvía dentro de la VLAN. `

(b) La misma trama de hacia un servidor en la VLAN 20.

` P1 —► [p1 access VLAN10] —► [p24 trunk] —► [R] —► [p18 access VLAN20] —► S1

(1) P1 manda SIN tag. (2) p1 aplica PVID=10 -> tag VID=10. (3) p24 es trunk: el tag VIAJA intacto: la trama sale con VID=10. (4) El router recibe 8100/VID=10, la mete en la subinterfaz 0.10, le saca el tag y decide por IP que va a 192.168.20.x. Para eso hace un ROUTE y reinyecta por 0.20. (5) R emite por 0.20 CON el tag VID=20. (6) Entra por p18 (access, PVID=20): coincide, el switch le saca el tag y lo entrega a S1.

⚠️ La trama CAMBIÓ DE VLAN en el router. Eso es posible porque el ROUTER es de CAPA 3 y por lo tanto ACABA el dominio de broadcast. Sin router, P1 y S1 estarían en dominios distintos y no se hablarían jamás. `

(c) Una trama de BROADCAST de (un ARP request).

` (1) P1 manda: [FF:FF:FF:FF:FF:FF | ... | 0800 | ARP request] (2) p1 pone el tag VID=10. (3) El switch busca en la TABLA: el destino FF:FF:FF:FF:FF:FF NO está (jamás está) -> FLOODING. (4) Pero el flooding se hace DENTRO DE LA VLAN 10 solamente: sale por p2, p3, ..., p16 (los otros access de VLAN 10) sale por p24 (el trunk, con el tag) ✗ NO sale por p17..p22 (VLAN 20) ✗ NO sale por p23 (VLAN 99) (5) En el router: entra por 0.10 con destino broadcast. Un router NO reenvía broadcast entre interfaces -> no sale por 0.20 ni 0.99. (6) Vuelve por p24 un reply ARP unicast: se despeja el tag y sigue por la tabla como cualquier unicast.

💡 ESO son "varios dominios de broadcast en una misma red switcheada". `

(d) Una trama que llega por el trunk p24 SIN tag, y una con tag VLAN 30.

` SIN tag: el switch consulta el PVID del puerto 24 = 10, y la TRATA PASA A SER DE LA VLAN 10. ¿Por qué existe el PVID? Porque un trunk puede tener una "VLAN nativa" untagged, usada para un único VLAN: es típico del router-on-a-stick de un fabricante que tiene una IP en el trunk y subinterfaz en el resto.

Con tag VLAN 30 (que el switch no conoce): El switch busca VLAN 30 en su tabla de VLANs. NO EXISTE. -> La trama se DESCARTA. 💡 Es una medida de SEGURIDAD, no un accidente: un router mal configurado, o un atacante en la trunk, no puede inyectar tramas en VLANs que el switch no tiene configuradas. `

Los números, otra vez.

` Sin tag : mínimo 64 B máximo 1518 B C-TAG : mínimo 68 B máximo 1522 B (+4) S+C-TAG : mínimo 72 B máximo 1526 B (+8)

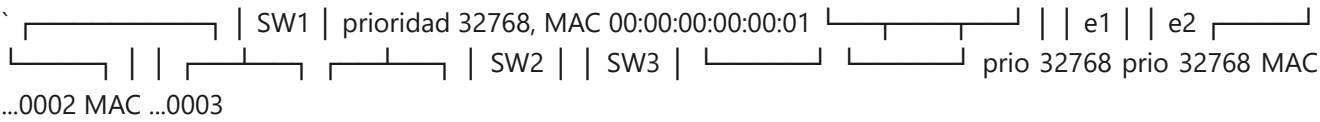
MTU de la carga IP: 1500 SIEMPRE, con tag o sin tag. Proof: una trama de 1522 B con tag tiene 1522 − 18 (cabecera) − 4 (tag) = 1500 bytes de payload. Misma MTU: el tag se "comió" relleno, no payload. `

⚠️ Errores típicos

- Creer que el switch **rutea** entre VLANs. **No**: sin router, la VLAN 10 y la VLAN 20 **no se hablan**, aunque estén en el mismo switch físico.
- Olvidar que el flooding **sí** se confina a la VLAN. Es el mecanismo real de la seguridad de VLAN, y es lo que dice la solución oficial del 2009-03.
- Decir "el PVID es el VLAN ID del trunk". No: el PVID es **por puerto**, e indica **qué VLAN se asume cuando llega una trama sin tag**.
- Confundir **access** con **untagged**: access es *untagged* **y pertenece a una sola VLAN**; trunk es *tagged* y puede llevar muchas.

2.4 Ejercicio 4 — Dibujar el árbol de STP y decidir qué puertos se bloquean

Enunciado (inventado). Tres switches full-duplex en triángulo, costos de enlace **iguales**:

` 

Enlaces: SW1-SW2 (e1), SW1-SW3 (e2), SW2-SW3 (e3). Costo: 100 c/u. `

Paso 1 — Bridge ID. Las tres prioridades son iguales (32768 = valor por defecto), así que desempata la MAC: `00:11:22:33:44:55` es el menor.

Paso 2 — Raíz. Raíz = el Bridge ID **menor** ⇒ **SW1 es la raíz**. *(Interpretación: como los tres tienen prioridad default, la raíz la decide la MAC del fabricante. Por eso existe el comando de cambiar la prioridad: en una red real*

no podés depender de qué placa quedó con la MAC más baja.)

Paso 3 — Puertos raíz.

` En la RAÍZ (SW1): todos sus puertos con vecino son DESIGNATED. No tiene puerto raíz: no necesita camino a sí misma. p(e1): DESIGNATED p(e2): DESIGNATED

En SW2: - p(e1) hacia SW1: costo = 100 (directo), BID emisor 8000...0001 - p(e3) hacia SW3: costo = 100 + 100 = 200, BID emisor 8000...0003 Gana el de MENOR COSTO -> p(e1) = PUERTO RAÍZ (no se bloquea)

En SW3: - p(e2) hacia SW1: costo 100 - p(e3) hacia SW2: costo 200 -> p(e2) = PUERTO RAÍZ (no se bloquea) `

Paso 4 — Designados en el enlace SW2–SW3. En el segmento compiten SW2 y SW3; gana el de **menor Bridge ID**: (SW2) ⇒ su queda **DESIGNATED** y el de SW3 queda en **BLOCKING**.

Paso 5 — Regla final y dibujo del árbol. Regla: un puerto se bloquea si su camino al raíz **no es el de menor coste**; todo lo demás → `forwarding`.

` ┌──┐ │ ESTADO FINAL │ │ SW1 (RAÍZ) │ │ / \ │ │ p(e1) p(e2) <- ambos DESIGNATED, FORWARDING │ │ / \ │ │ SW2 SW3 │ │ p(e1) RAÍZ p(e2) RAÍZ <- ambos FORWARDING │ │ | | | │ │ p(e3) p(e3) │ │ DESIGNATED ████████ BLOCKING <<<<< AQUÍ SE CORTA │ │ FORWARD (recibe BPDUs, no manda datos) │ └──┘

Puertos bloqueados: SOLO SW3.p(e3) (1 de 6)

VERIFICACIÓN: en una topología conexa de N bridges y M enlaces, el spanning tree bloquea SIEMPRE (M − (N − 1)) puertos. Acá: 3 − (3 − 1) = 1. ✓ Cuadra con lo que dibujamos. `

(e) La variante: ¿y si SW3 tuviera prioridad 4096?

` SW3: prioridad 4096 = 0x1000 -> 1000.0000.0000.0003 Raíz = MENOR Bridge ID: 1000.0000.0000.0003 <-- AHORA EL MENOR ==> SW3 PASA A SER LA RAÍZ

SW3 (RAÍZ) / \ p(e2) / \ p(e3) DESIGN / \ DESIGN FORWARD FORWARD / \ SW1 SW2 p(e2) ROOT p(e3) ROOT FORWARD FORWARD | | p(e1) DESIGN p(e1) ██████ BLOCKING FORWARD (el enlace e1 se corta del lado de SW2) `

El árbol **cambió de forma**, pero el número de puertos bloqueados es el mismo (1). Ese invariante vale para cualquier red conexa.

(f) ¿Y si se cae el enlace ?

` t0: e1 se cae. t≈20 s: SW1 y SW2 dejan de oír BPDUs el uno del otro durante Max Age (= 20 s) y se dan cuenta de que el vecino "murió". t+ : SW2 recibe por e3 los BPDUs de SW3 y recalcula; el enlace e3, que estaba bloqueado, se desbloquea y pasa a ser el camino. t≈20-50 s con STP clásico: se elige la raíz nueva y se recalculan los puertos.

⏱️ Con STP clásico (802.1D): 30 a 50 segundos de red parada. Con RSTP (802.1w, el default real): sub-segundo a unos segundos, porque los puertos "edge" no pasan por Listening/Learning y porque los puertos no-edge notifican el cambio al vecino por *proposal* en vez de esperar el Max Age. `

⚠️ Errores típicos

- **Olvidar el desempate por MAC.** Si dos switches tienen la misma prioridad (lo normal), el menor Bridge ID **no** es "el primero" sino el de MAC más baja.
- **Poner un puerto raíz en la raíz.** La raíz **no tiene puerto raíz**: tiene todos designados.
- **Confundir "designado" con "más cercano a la raíz".** En un segmento, el designado es el de **menor Bridge ID entre los conectados a ese segmento**, no el más cercano a la raíz.
- **Bloquear el enlace de los dos lados.** El enlace se bloquea en **un solo lado**. Es el error más visible en un diagrama.
- **No verificar con la fórmula (M − (N−1)).** Si el diagrama y la fórmula no dan el mismo número de puertos bloqueados, hay un error.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 Examen 2010-12, Pregunta 3 (8 puntos) — hub, switch y self-learning

> **Fuente:** Examen — 17 de diciembre de 2010. Solución oficial: . **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf **Texto del examen (literal):**
>
> **a)** Considere una red de varios nodos interconectados por un hub, ¿cuál es el dominio de colisión y cuál es el de broadcast? Justifique.
>
> **b)** Si en la red anterior cambiamos el hub por un switch. ¿Se modifican los dominios de colisión y broadcast? ¿De qué manera?
>
> **c)** Describa el mecanismo de self-learning que utilizan los switches.

Este es **el problema central de la clase**. Los tres incisos se corrigen con el mismo par de ideas: *el hub es cable, el switch es memoria*.

Parte a) (~3 puntos)

Solución oficial (texto):

> a) El dominio de colisión y el dominio de broadcast son iguales y abarcan a todos los nodos de la red. Esto se debe a que el hub no implementa ningún protocolo de acceso al medio y simplemente reenvía todas las tramas por todas las interfaces.

Análisis. ✅ Está bien: nombra los dos dominios, dice que abarcan todos los nodos y da **la causa**. ⚠️ Lo que le falta, y son puntos: (1) no dice **qué es** cada dominio, solo que son "iguales"; (2) no dice **por qué** el de colisión es único (mismo medio eléctrico); (3) no dice **por qué** el de broadcast es único (la trama se replica sin filtro, y las copias también se replican).

Mi respuesta, desarrollada:

> **a)** Con un hub, el **dominio de colisión y el de broadcast son ambos únicos y abarcan toda la red**.
>
> - **Colisión:** el hub es de **capa 1**, repite la señal eléctrica a todos los puertos. Todas las estaciones comparten **el mismo medio**, así que si dos transmiten a la vez sus señales se superponen y ninguna llega bien. Es **un solo dominio de colisión**, y es half-duplex por definición.
> - **Broadcast:** una trama de broadcast se copia a **todos** los puertos del hub; cada estación que la recibe (si está en otro hub de la red) la reenvía a todos los suyos. **Nada la detiene**, así que llega a **todas** las estaciones: **un solo dominio de broadcast**.

"No implementa ningún protocolo de acceso al medio" es la causa raíz: si el hub hiciera algo más que repetir, o bien filtraría tramas y dejaría de ser de capa 1, o bien regularía el acceso y dejaría de ser un medio compartido. Un hub es **un medio compartido con reiteración**, y por eso hereda de ese medio las dos propiedades: colisión sí, frontera no.

Parte b) (~3 puntos)

Solución oficial (texto):

> b) En el caso de un switch existe un dominio de colisión por cada interfaz del switch, por lo que en este caso cada enlace con los nodos tendrá su propio dominio de colisión. El switch impediría que ocurran colisiones. El dominio de broadcast sigue abarcando a todos los nodos de la red.

Análisis. ✅ Lo que está bien: (1) dice explícitamente **un dominio de colisión POR INTERFAZ**; (2) dice que **el dominio de broadcast no cambia**; (3) agrega "el switch impediría que ocurran colisiones", que es lo que la mayoría no dice.

⚠️ **Matices que valen puntos, y acá la oficial es algo imprecisa:** 1. **"El switch impediría que ocurran colisiones"** es correcto **solo en full duplex**. En half-duplex un puerto de switch sigue siendo un dominio de colisión con CSMA/CD, y ahí sí pueden occurir colisiones (las *colisiones tardías*). Lo correcto: *el switch no comparte medio, así que en full duplex no hay colisión posible; en half-duplex cada puerto sigue siendo un dominio de colisión propio, con su propio CSMA/CD, aislado de los demás.* 2. **Falta decir POR QUÉ no cambia el dominio de broadcast.** La razón es fuerte y sostiene todo lo de VLAN: **una trama de broadcast no puede descartarse**, porque por definición tiene que llegar a todos. El switch no tiene dirección destino que consultar para saber a quién entregarla. 3. **Falta la advertencia de qué pasa si hay un hub debajo.** Si cuelgo un hub de un puerto del switch, **dentro de ese hub hay colisiones otra vez**, y ninguna VLAN lo arregla.

Mi respuesta (más completa que la oficial):

> **b)** Sí, se modifica **uno de los dos**:
>
> - **Dominio de colisión: se PARTE en N.** El switch tiene un dominio de colisión **por puerto**, porque cada puerto es un enlace punto a punto con su propio medio. En **full duplex** no hay medio compartido y por lo tanto **la colisión es físicamente imposible**; en **half-duplex** cada puerto es un dominio de colisión propio con su propio CSMA/CD, **aislado** de los demás. (Lo que el switch hace es **eliminar el medio compartido**, no "evitar colisiones".) ⚠️ Si cuelgo un hub de un puerto, **dentro de ese hub vuelve a haber un único dominio de colisión**.
> - **Dominio de broadcast: NO se modifica.** Sigue siendo **toda la red**. Una trama de broadcast () **no puede descartarse**, porque por definición todos tienen que oírla; el switch no tiene dirección de destino que consultar para saber a quién entregarla, así que **la reenvía por todos los puertos**. Lo único que parte el dominio de broadcast es una **VLAN** o un **router**.

Parte c) (~2 a 3 puntos)

Solución oficial (texto):

> c) El mecanismo de self-learning funciona de la siguiente manera: - Inicialmente la tabla del switch está vacía. - Cuando llega una trama del nodo A con destino B a través de la interfaz X: - Anotamos en la tabla que se puede llegar a A a través de la interfaz X. - Si hay un registro (B, Y) en la tabla, y X distinto de Y, reenviamos la trama por Y. - Si no existe el registro, reenviamos la trama por todas las interfaces excepto X.

Análisis. ✅ Lo que está bien y hay que applausear: (1) dice que **la tabla arranca vacía**, que es lo que hace que la primera trama siempre sea flooding y lo que conecta con "el switch arranca como un hub" (que dice la oficial del 2008-12); (2) la definición de la tabla es **origen → puerto**; (3) el **"y X distinto de Y"** del segundo inciso **es el filtrado**, aunque no lo nombre; (4) el tercer inciso dice "**todas las interfaces excepto X**". Exacto.

⚠️ **Lo que falta, y son puntos:** 1. **El caso no está escrito**: está escondido en un "y". Si lo escribís aparte, decís "si la entrada dice que B está en el mismo puerto por el que entró, la trama se descarta, porque el destino ya la oyó". Ese es el **filtrado**. 2. **No dice que se aprende SOLO del origen.** Es lo más importante: si aprendiera del destino, un atacante podría envenenar la tabla con una sola trama. Y también explica por qué el flooding **no** enseña nada. 3. **No dice nada del ageing / temporizador.** Cada anotación va con un temporizador que expirar la entrada. 4. **No dice que el broadcast/multicast siempre se inunda**, que es un subcaso del tercer inciso (jamás están en la tabla).

Mi respuesta, completa y ordenada:

> **c)** El switch mantiene una **tabla de reenvío** (*forwarding table*, o CAM table) que asocia **dirección MAC → puerto**, con un **temporizador de ageing** por entrada.
>
> 1. **Inicialmente la tabla está vacía.** Por eso, al arrancar, el switch se comporta como un hub: inunda todo hasta aprender.
> 2. **Aprendizaje (siempre, en primer lugar):** ante cualquier trama que llega por el puerto **X**, se anota `A → X`, donde **A es la dirección MAC origen**, y se **reinicia el temporizador de ageing** de esa entrada. Se aprende **solo del origen**, porque el origen es información verificada por el cable (quien está en el puerto X es, necesariamente, quien puso esa MAC); el destino es solo lo que el emisor **quiere** que creamos.
> 3. **Decisión del destino, con B = MAC destino:** - Si **B es broadcast o multicast** (jamás está en la tabla): se reenvía por **todos los puertos menos X** (*flooding*). - Si **existe y** : se reenvía por el puerto **Y** (unicast, una sola copia). - Si **existe y** : se **descarta** la trama (el destino está en el mismo segmento por el que llegó; ya la oyó) — esto es el **filtrado**. - Si **no existe entrada para B**: se reenvía por **todos los puertos menos X** (*flooding*), porque no se sabe a dónde va. El flooding **no aprende nada**: el switch solo aprende del paso 2.
> 4. **Ageing:** cada entrada tiene un temporizador (típicamente **300 s**, default de fabricante y configurable) que se reinicia cada vez que se ve esa MAC. Si expira, la entrada se borra, y la próxima trama con ese destino vuelve a ser un flooding.

Reparto orientativo de los 8 puntos (el enunciado no lo publica):

Inciso	Peso	Lo que no puede faltar
a)	~3	los dos dominios = **toda la red**; la causa: el hub es capa 1 y repite todo
b)	~3	colisión → **uno por puerto**; broadcast → **sigue toda la red**, porque el broadcast no se puede descartar; de paso, full duplex no tiene colisión
c)	~2	tabla vacía; **aprende del origen**; unicast si conoce, **flooding por todos menos X** si no; el "**y X distinto de Y**" es el filtrado

3.2 Examen 2008-12, Pregunta 2 (5 puntos) — qué mejora reemplazo hubs por switches

> **Fuente:** Examen — 20 de diciembre de 2008. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0812.pdf **Texto del examen (literal):**
>
> **Pregunta 2 (5 puntos)** Explique que posibles mejora introduce la sustitución de hubs por switches, en cuanto a performance y seguridad en las redes LAN.

Solución oficial (extracto textual)

> El principio de funcionamiento de los hubs es que todo el tráfico que ingresa por un puerto, es enviado por todos los demás puertos del dispositivo, aunque en la mayoría de los casos el tráfico estará dirigido a uno sólo de dichos puertos. Los switches, por el contrario, al "entender" de direcciones MAC redirigen las tramas que ingresan por un puerto al puerto por donde se alcanza la máquina destino de la misma. **No debemos olvidar que éste es el comportamiento en régimen de un switch. En la fase de arranque y apendizaje, se comporta como un hub** a los efectos de ir aprendiendo qué direcciones son alcanzables en cada puerto. Por otro lado, los switches incorporan buffers en cada puerto así como una matriz de conmutación de por lo menos un orden de velocidad mayor a la de los puertos. De esta forma, en un switch podemos tener múltiples comunicaciones simultáneas (a diferencia de un hub), ellas full-duplex y cada puerto de un switch puede ser un sólo dominio de colisión (frente al único dominio de colisión que significa un hub) todo lo que redunda en un aumento notorio de la performance de la red. Por otro lado, la mejora en cuanto a seguridad está basada en que, en un switch funcionando en régimen, el tráfico sólo es enviado al puerto donde está conectada la máquina destino del mismo, lo que permite "ocultar" el dicho tráfico al resto de las máquinas conectadas al switch. También la implementación de VLANs puede ser una medida que colabore a la seguridad en una red switcheada.

Análisis de la solución oficial

✅ **Es una solución muy buena.** Tres cosas que hay quer sí o sí, porque son ideas que dan puntos:

1. **"En la fase de arranque y aprendizaje, se comporta como un hub."** Es **la mejor frase de la solución** y casi ningún alumno la dice. Dice que **la mejora de seguridad no es total ni inmediata**: hay una ventana insecure al arrancar.
2. **"Buffers en cada puerto" + "matriz de conmutación de por lo menos un orden de velocidad mayor".** No dice solo "más rápido": da la **causa** (matriz más rápida que los puertos + buffers por puerto), que es lo que permite explicar **por qué** hay comunicaciones simultáneas.
3. **"VLANs como medida de seguridad."** Adelanta el tema de la VLAN (que es el 2009-03 P3, de tres meses después) y conecta las dos preguntas del mismo bloque.

⚠️ **Lo que le falta:** (1) **no cuantifica el ganho de ancho de banda** (con estaciones el hub da y el switch da a cada una): es una línea y vale puntos; (2) no precisa el caso half-duplex frente a full-duplex, aunque lo insinúa con "full-duplex"; (3) no menciona que el switch **duplica el tráfico interno** al procesarlo en la matriz, aunque en 5 puntos eso es detalle.

Mi respuesta, estructurada para 5 puntos

a) Performance (~3 puntos)

> **Con un hub**, la trama que entra por un puerto sale por **todos** los demás, y la red es **un único medio compartido**: - **Un solo dominio de colisión**: todas las estaciones comparten el mismo medio; el ancho de banda se **divide** (por estación), las tramas se **duplican** (copias de cada trama, de las cuales se descartan), y hay colisiones con CSMA/CD. - **Half-duplex**: no se puede transmitir y recibir a la vez.
>
> **Con un switch**, en régimen: - **Aprende las direcciones MAC** y **envía cada trama por un solo puerto**, el del destino. Se elimina el desperdicio: la trama deja de multiplicarse. - **Cada puerto es un dominio de colisión independiente** y trabaja en **full duplex**. En full duplex **no hay colisión posible**, porque no hay medio compartido. Con estaciones, el ancho de banda **por estación pasa de a** . - **Buffer por puerto y una matriz de conmutación más rápida que los puertos**, lo que permite **varias comunicaciones simultáneas** sin que una bloquee a las otras.
>
> ⚠️ **El matiz que hay que decir:** un switch recién arrancado tiene la tabla vacía y **se comporta como un hub** hasta aprender. Las tramas iniciales se inundan. La mejora es de régimen, no de arranque.

b) Seguridad (~2 puntos)

> **Con un hub**, el medio es compartido: **todo el tráfico que pasa por él es visible en todos los puertos**, sin excepción. Cualquier estación puede ver (con un sniffer) el tráfico de todas las demás: contraseñas, claves, cabeceras. Seguridad **cero**.
>
> **Con un switch**, en régimen, **cada trama sale solo por el puerto de su destino**: el tráfico de una estación **no es visible** desde los otros puertos. Eso es una mejora real y directa.
>
> **Pero no alcanza como mecanismo de seguridad:** - Durante el **aprendizaje** el switch inunda, y en esa ventana se ve todo. - Un atacante que **inunda la tabla con MAC falsas** (*MAC flooding*) agota la memoria de reenvío y **degrada el switch a hub**. - El switch **no autentica a nadie**: cualquiera que enchufe un cable entra a la red.
>
> **Sí hay un refuerzo estructural:** las **VLANs**, que particionan lógicamente el switch en varios dominios de broadcast independientes y **confinan el flooding** dentro de cada VLAN. Con eso, lo que no es unicast se queda en su VLAN, y el tráfico de una VLAN no es visible desde los puertos de otra.
>
> **Nota sobre el tamaño de esta pregunta:** 5 puntos y **ninguna aritmética**. Es una pregunta de **fundamentos bien enunciados**. Si te quedás pensando en cuentas, no llegás. Decí **qué cambia** (de "medio compartido" a "enlace punto a punto con memoria") y **por qué** eso mejora cada una de las dos cosas.

3.3 Examen 2009-03, Pregunta 3 (6 puntos) — el concepto de VLAN

> **Fuente:** Examen — 20 de febrero de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0903.pdf **Texto del examen (literal):**
>
> **Pregunta 3 (6 puntos)** Explique el concepto VLAN (p.e. 802.1q) y comente sobre su utilidad.

> ⚠️ **Nota sobre el puntaje:** el PDF asigna **6 puntos** a esta pregunta (y trae su solución oficial). Algunas recopilaciones del banco la listan como 10. **Se corrige 6**, y eso no cambia la respuesta.

Solución oficial (texto)

> Es una tecnología que nos permite tener más de un dominio de broadcast en una misma red switcheada. Se trata de una mejora que se puede incorporar a la red de manera transparente para los usuarios finales ya que la configuración corre por cuenta de los administradores de la misma. La utilidad radica en que se puede tener "varias redes" LAN soportadas por el mismo equipamiento y donde la pertenencia de cada puerto a cada VLAN se puede ajustar bajo demanda. Al poder distinguir qué puertos de un switch pertenecen a qué VLAN, indirectamente es una mejora respecto a seguridad, en particular en lo que refiere a tráfico no unicast y también en el caso de falta de determinadas entradas en las tablas que asocian dirección MAC con puerto.

Análisis de la solución oficial

✅ **Lo que está muy bien:** 1. **"Más de un dominio de broadcast en una misma red switcheada."** Definición en una línea y **exactamente correcta**. Con esa frase ya tenés el punto central. 2. **"La pertenencia de cada puerto a cada VLAN se puede ajustar bajo demanda."** Dice lo esencial: la VLAN es una **atribución de puertos**, y se cambia por configuración. Es lo que permite mover un usuario sin tocar el cable. 3. **"Tráfico no unicast"** es la idea más fina de toda la solución, y casi nadie la encuentra. Significa: el problema de seguridad de un switch no es el unicast (que va dirigido a un solo puerto), sino **el flooding** (broadcast, multicast y unicast a destino desconocido). La VLAN **no cambia el unicast**: lo confina.

⚠️ **Lo que le falta:** (1) **no desarrolla 802.1Q**, pese a que el enunciado dice "(p.e. 802.1q)": ni el tag de 4 bytes, ni el VID, ni el C-TAG; (2) **no dice por qué** el broadcast se confina (que el switch etiqueta la trama y el flooding se hace **dentro** de la VLAN); (3) no menciona el **router-on-a-stick** ni el **PVID**.

Mi respuesta completa (6 puntos)

a) Qué es (2 puntos)

> Una **VLAN** (*Virtual LAN*, LAN virtual) es una **partición lógica** de la red conmutada: un mismo **switch físico** se comporta como **varios switches independientes**, uno por VLAN, según a qué VLAN pertenece cada puerto. La pertenencia **no la define el cable ni la estación**: la define la **configuración del switch**.
>
> Para que la trama sobreviva a varios switches, 802.1Q **agrega un tag de 4 bytes** a la trama Ethernet, que lleva el **VLAN ID** (12 bits, valores 1 a 4094) y una prioridad de 3 bits. El `TPID` pasa a ser `0x8100` (C-TAG) en vez de `0x0800`. Un puerto **access** (de usuario) quita el tag; un puerto **trunk** lo conserva.

b) Para qué sirve (4 puntos)

> **1. Parte el dominio de broadcast.** Sin VLAN hay **un solo** dominio de broadcast en todo el switch, y las tramas de broadcast se replican a **todos** los puertos. Con VLAN hay **uno por VLAN**: el flooding se hace **dentro de la VLAN**, o sea, por los puertos **de esa VLAN solamente**. Eso reduce el ruido (ARP, DHCP, mDNS) y permite que una sola lógica conviva con otras en el mismo switch.

> **2. Seguridad.** El tráfico no unicast (broadcast y multicast) es el que sale por **todos** los puertos, y el unicast a un destino **desconocido** (que no está en la tabla MAC) también se inunda. Ese flooding es el que un atacante ve. La VLAN **confina ese flooding dentro de la VLAN**: lo que no sale de la VLAN no se ve desde los puertos de las otras, y el unicast entre dos puertos de la misma VLAN tampoco se ve desde fuera.
>
> **3. Mover un usuario sin mover el cable.** La configuración es del switch. Si el usuario se muda de escritorio, su cable va al mismo patch panel y **lo que cambia es la línea de configuración** que asigna ese puerto a una VLAN. Antes de las VLAN, mover a alguien significaba repatchear el cableado.
>
> **4. Segmentar la gestión.** Las VLANs permiten separar el tráfico de administración del de datos, de modo que **inundar la red de gestión no llena de tráfico todos los puertos** del switch.

c) El mecanismo (por qué funciona lo de la seguridad)

> El switch etiqueta cada trama con el VLAN ID y, cuando decide inundar, **solo inunda por los puertos de esa VLAN**: las tramas de otra VLAN no se mezclan. Y el dominio de broadcast **se cierra** porque un **router** (o una subinterfaz de router, el *router-on-a-stick*) **no reenvía tramas de broadcast entre interfaces**. Es el router, y no la VLAN, el que realmente termina el dominio de broadcast entre VLANs distintas.

⚠️ Las dos frases que hay que decir de memoria

> **"VLAN es lo que hace que un switch pueda ser varias redes."**
>
> Y si te sobra tiempo: **"La VLAN no evita el flooding; lo confina."** Esa distinción es la que separa una respuesta de 6 de una de 4.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Hub** = capa 1: repite a todos los puertos. **Un** dominio de colisión, **un** dominio de broadcast, **cero** seguridad.
2. **Bridge** = mira la trama y tiene **dos verbos**: reenviar a un puerto, o **inundar** por todos.
3. **Switch** = un bridge con muchos puertos, buffer por puerto y una matriz de conmutación más rápida que los puertos.
4. **Self-learning**: el switch aprende la **MAC origen** y la asocia al puerto de entrada. Todo lo demás es **inferencia**.
5. El algoritmo: **(1) aprender A → X**; **(2)** si `B` está en el puerto `Y` y `Y ≠ X`, unicast por `Y`; si `Y == X`, **descartar** (filtrado); si no está, **flooding por todos menos X**.
6. **Ageing**: sin él, un nodo que se muda deja de ser alcanzable. **300 s es default de fabricante**, no un valor de IEEE.
7. **Switch**: cada puerto es su propio dominio de colisión (**en full duplex la colisión es imposible**) y el ancho de banda es `1 Gbps` por estación en vez de `100 Mbps` compartidos. Pero el dominio de broadcast **sigue siendo toda la red**.
8. **VLAN**: partición lógica del switch. El **tag 802.1Q** son 4 bytes (TCI +) con **VID de 12 bits (1–4094)**; C-TAG vs. S-TAG (); **PVID** para las tramas sin tag.
9. **Loop en capa 2 = tormenta de broadcast**, y los bridges transparentes **no usan TTL**: por eso existe el **spanning tree** (raíz con menor Bridge ID = prioridad 32768 + MAC, puerto raíz, puertos designados, y el resto en **blocking**).
10. En 2026: **half-duplex muerto por encima de 1 Gb/s** (decisión de **IEEE**, no RFC de IETF), ***,** *para LAG, RSTP (802.1w) como default real, y C-TAG/S-TAG** en carrier Ethernet y EVPN-VXLAN.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué un hub tiene **un solo** dominio de colisión **y** un solo dominio de broadcast? Dos frases, una para cada uno.

<details>
<summary>Respuesta</summary>
</details>

2. Llega por el puerto 5 una trama con MAC origen `A` y MAC destino `B`. La tabla dice `B → p4`. ¿Qué hace el switch?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué el switch aprende la MAC **origen** y no la **destino**?

<details>
<summary>Respuesta</summary>
</details>

4. En el ageing de 300 s, ¿por qué una estación que transmite una vez cada 200 s "pierde" su entrada aproximadamente el 78 % de las veces? Calculá.

▶ Respuesta

5. Explicá por qué el dominio de broadcast **no** se parte al cambiar un hub por un switch, y qué sí lo parte.

▶ Respuesta

6. ¿Qué son el **C-TAG** () y el **S-TAG** (), y por qué hay dos?

▶ Respuesta

7. ¿Por qué un loop en capa 2 es mucho más grave que uno en capa 3?

▶ Respuesta

8. En una red con tres switches en triángulo, todos con prioridad 32768 y costos de enlace iguales, ¿cuántos puertos quedan bloqueados y por qué número?

▶ Respuesta

🔗 Referencias

- **Examen ** (17-dic-2010) —** Pregunta 3 (8 pts)**: dominio de colisión y de broadcast con hub; qué cambia con switch; mecanismo de self-learning. Con solución oficial.** Problema central de esta clase.**
- **Examen ** (20-dic-2008) —** Pregunta 2 (5 pts)**: mejoras de performance y seguridad al sustituir hubs por switches. Con solución oficial.
- **Examen ** (20-feb-2009) —** Pregunta 3 (6 pts)**: concepto de VLAN (802.1Q) y su utilidad. Con solución oficial.
- **Examen ** (26-feb-2007) — Pregunta 2 (10 pts): bridge, flooding y bucles; la solución oficial es el** árbol de cubrimiento**, o sea** STP**.** Es el examen del banco más cercano a §1.7.**
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 3 (§3.4 "La capa de red del enlace de datos: switches", §3.6 y §3.7) y cap. 1 (repetidores, puentes y switches)
- **IEEE 802.3** — Ethernet: formato de trama, tabla de reenvío, ageing, dominios de colisión. **No es una RFC de IETF**: es un estándar del IEEE.
- **IEEE 802.1D** — Spanning Tree Protocol: Bridge ID (prioridad base 32768 + MAC), BPDUs, estados Blocking/Listening/Learning/Forwarding, temporizadores (Hello 2 s, Max Age 20 s, Forward Delay 15 s).
- **IEEE 802.1w** — **RSTP**: Rapid Spanning Tree Protocol, hoy el default real de los switches.
- **IEEE 802.1s** — **MSTP**: Multiple Spanning Tree, para compartir un árbol entre varias VLANs.
- **IEEE 802.1Q-2022** — VLANs y el tag de 4 bytes; revisión actual (precedida por `802.1Q-2018`, que abrió TSN con `802.1Qbv` + `802.1Qbu` y `802.1Qci`; 2022 agregó `802.1Qcc` y `802.1Qcr`).
- **IEEE 802.1AX-2020** — Link Aggregation (LAG), sucesión de (2000).
- **IEEE 802.3cg** — 10BASE-T1S: el único caso industrial donde el **half-duplex sigue siendo obligatorio** (con PLCA). La eliminación del half-duplex por encima de 1 Gb/s es una decisión de **IEEE**, **no una RFC de IETF** (⚠️ es JSCalendar, no Ethernet).

- **RFC 826** — ARP: request en broadcast, reply en unicast; es quien sufre el dominio de broadcast.
- **RFC 7432** — EVPN · **RFC 7348** — VXLAN (encapsulado sobre UDP/4789): el estándar de overlay de datacenter, donde el tenant se identifica por **VNI** y no por 802.1Q.
- **Clase anterior:** 41 — Acceso al medio: ALOHA, CSMA/CD y CSMA/CA
- **Clase siguiente:** 43 — Redes inalámbricas 802.11: CSMA/CA, estación oculta, NAV, RTS/CTS, EDCA y seguridad

Clase 43

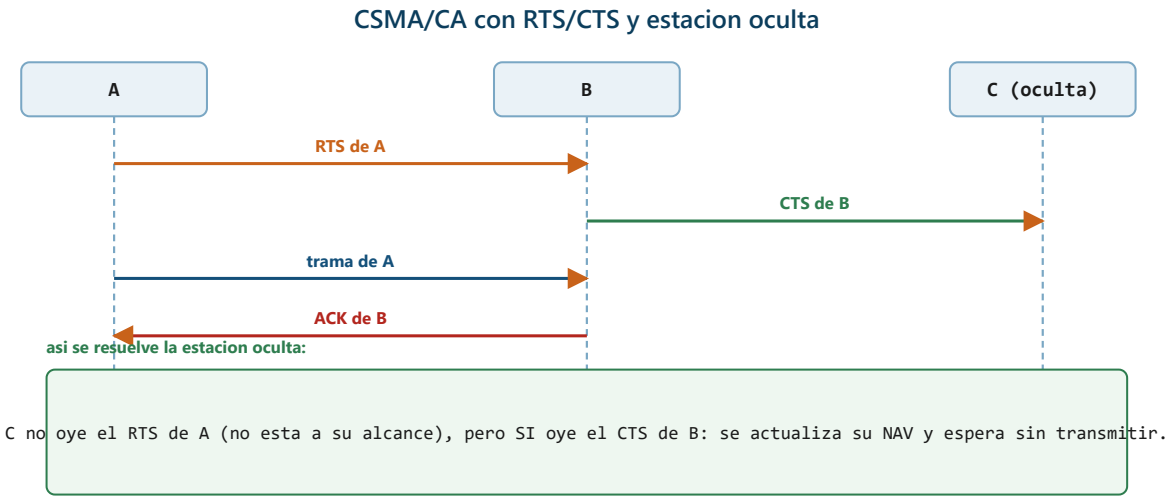
Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 Por qué una red wireless es un problema distinto
- 1.2 Los dos problemas concretos
- 1.3 CSMA/CA: el mecanismo completo (DCF)
- 1.4 El NAV: el sensor virtual
- 1.5 RTS/CTS: la solución a la estación oculta
- 1.6 Modo infraestructura
- 1.7 EDCA: las 4 colas de acceso
- 1.8 Seguridad
- 1.9 ⚠️ Actualización a 2026
- 2.1 Ejercicio 1 — La estación oculta, paso a paso
- 2.2 Ejercicio 2 — El cálculo del NAV
- 2.3 Ejercicio 3 — La tabla de CW y la probabilidad de colisión
- 2.4 Ejercicio 4 — ¿Por qué no se puede detectar la colisión en WiFi?
- 3.1 ⭐ Examen 2008-07, Pregunta 3 (10 puntos) — CSMA/CA en modo infraestructura
- 3.2 Examen 2006-12, Pregunta 5 (6 puntos) — la estación oculta
- 3.3 Examen 2007-03, Pregunta 5 (5 puntos) — CSMA/CA
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: E (Enlace) · **Semana:** 15 · **Duración estimada:** 120 min **Correlativa:** Clase 42 (switch, VLAN, STP) · **Sigue con:** Clase 44 (multimedia) **Material de referencia:** Tanenbaum cap. 4 · `ACTUALIZACIONES.md` §9

🎯 Al terminar esta clase vas a poder

1. Explicar por qué una red wireless es un problema distinto y por qué en ella **no se puede detectar** una colisión.
2. Describir el mecanismo completo del **CSMA/CA** (DCF) paso a paso.
3. Dibujar la **estación oculta** y explicar qué resuelve y qué no resuelve el **RTS/CTS**.
4. Explicar qué es el **NAV** y por qué se considera un "segundo sensor" del medio.
5. Comparar las **4 colas de EDCA** y explicar por qué el video no va en la cola de voz.
6. Describir el problema de seguridad de **WEP**, qué resuelve WPA3 con **SAE**, y qué protege **802.11w**.



En radio no se puede usar CSMA/CD: la antena es half-duplex, la potencia propia enmascera a los demas y no se puede transmitir y escuchar a la vez. Por eso WiFi no detecta colisiones: las EVITA.

Mecanismos: RTS/CTS, NAK, ACK por trama (no acumulado), espera aleatoria y EDCA con 4 colas.

> **Prerequisito del examen:** `2008-07 P3` (10 pts — CSMA/CA en modo infraestructura), `2006-12 P5` (6 pts — estación oculta), `2007-03 P5` (5 pts — CSMA/CA), `2010-07 P2` (8 pts — por qué no CSMA/CD en wireless).

PARTE 1 — TEORÍA

1.1 Por qué una red wireless es un problema distinto

En la Clase 41 vimos que **Ethernet cableado** puede usar **CSMA/CD**: detectar la colisión, abortar, hacer backoff y reintentar. Todo eso funciona porque el receptor es un **cable**: el que transmite puede **escuchar su propia señal** mientras transmite, y si oye otra cosa encima, es una colisión.

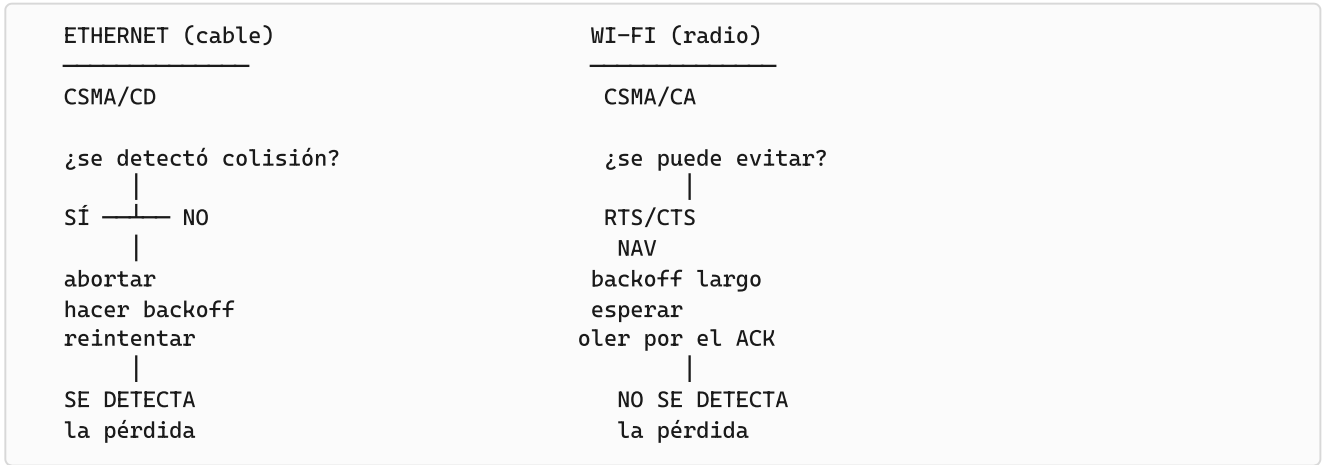
En radio ninguna de las tres condiciones se cumple:

1. La antena es half-duplex. Un transceiver de radio **no puede transmitir y recibir a la vez**. Es como hablar y escuchar al mismo tiempo: no funciona. Y para detectar una colisión haría falta exactamente eso: hablar y escuchar.

2. La propia señal enmascera a las demás. Tu señal llega a la antena a unos 30 dBm (1 vatio). La señal de otro emisor que vuelve por reflexión puede llegar a −90 dBm. La diferencia es de **120 dB**, que en potencia es un factor de $\mathbf{10^6}$. Tu propio receptor la ahoga por completo. Detectar la colisión por nivel de señal es **físicamente imposible** con esta tecnología.

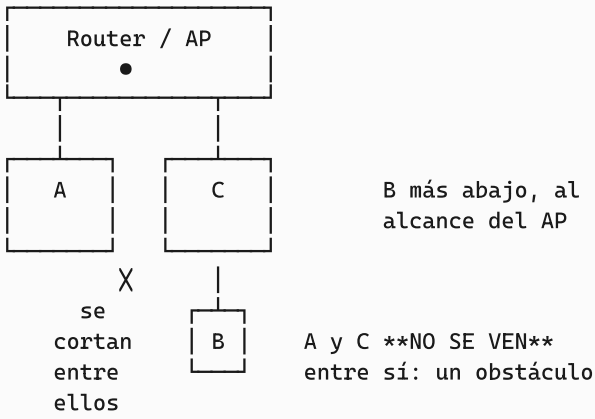
3. El medio es invisible. Las ondas se propagan por el aire y **no tienen frontera**: alguien en la calle, detrás de una pared, transmite en el mismo canal y vos no lo sabés. En el cable el medio es **punto a punto** y no hay ambigüedad.

> **La conclusión es la clave de toda la clase:** en radio **no se puede DETECTAR una colisión**. Entonces el protocolo tiene que hacer lo contrario: **EVITARLA** antes de que ocurra. De ahí **A** (*Avoid*) en lugar de **D** (*Detect*).



1.2 Los dos problemas concretos

La estación oculta (*hidden node*)



El problema: A y C están **los dos al alcance del AP**, pero **no se ven entre sí**. Si A transmite su trama hacia el AP, C **no la escucha**, así que cree que el medio está libre y **también transmite**. Las dos tramas **chocan en el AP**, donde de hecho convergen.

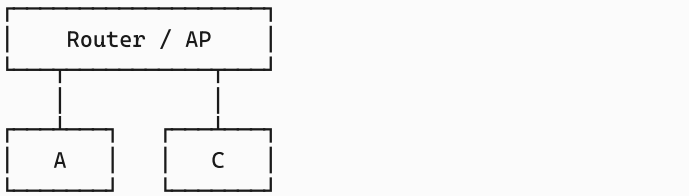
> **La analogía correcta:** es una conversación en una sala donde dos personas le hablar a alguien al fondo. Ellas no se escuchan entre sí, así que hablan los dos al mismo tiempo, y el del fondo entiende la mitad.

Y el agravante: el problema **no es** que A y C collagen entre sí (no se alcanzan). Es que **colisionan en el AP**, que es el punto donde convergen las dos.

La solución clásica: RTS/CTS (§1.5). Y la solución moderna: **BSS coloring**, que agrega un número de color a cada red para que las redes vecinas no se interfieran sin necesidad de negociar nada (§Actualización).

La estación expuesta (*exposed terminal*)

El problema inverso, y **RTS/CTS no lo resuelve**:




```
A y C SÍ se ven entre sí.
A transmite → C lo oye → C se calla.
PERO C podría haber transmitido a la vez, sin problema,
porque C y A van a DESTINOS DISTINTOS.
```

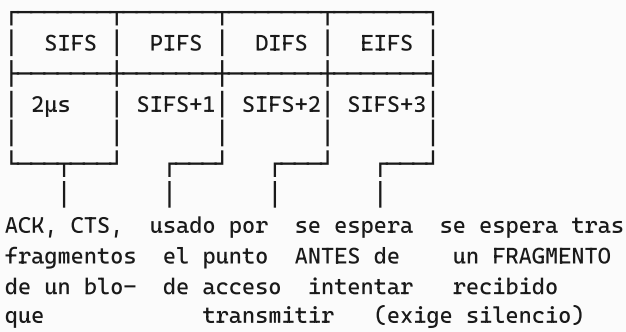
El punto fino: la estación expuesta produce una **ineficiencia**, **no una pérdida**. Y cualquier solución que la arregle (como pedir RTS/CTS para todos los casos) **empeora** el caso de la estación oculta con más overhead. **Es un compromiso, y no hay solución limpia.** En un examen, mencionar esto suma.

1.3 CSMA/CA: el mecanismo completo (DCF)

DCF = *Distributed Coordination Function*. Es la forma **distribuida** de coordinar el acceso (no hay punto central que asigne turnos).

Los 4 espacios inter-trama

Antes de los espacios hay un pequeño **silencio** llamado **SIFS** (*Short Interframe Space*), que es el más corto. Sirve para las tramas de **control** que tienen que ir justo después de una de datos, sin que nadie más pueda interrumpir.



Símbolo	Significado	Uso
SIFS	Short InterFrame Space	respuesta a una trama: **ACK**, **CTS**, fragmentos de un bloque. **Nadie más espera**: es el que garantiza que la respuesta gane el medio.
PIFS	PCF InterFrame Space	reservado al punto de acceso en modo **puntual** (PCF). Poco usado.
DIFS	Distributed IFS	lo espera una estación **común** antes de intentar transmitir.
EIFS	Extended IFS	lo espera una estación que **no entendió** lo que recibió (o recibió un error). Es el más largo: penaliza.

El orden importa: para empezar a transmitir se espera **DIFS**. Para **responder** se espera **SIFS**, que es **más corto**. Como todas las estaciones usan el mismo valor de SIFS, la respuesta pasa siempre primero. **Eso es lo que hace posible el ACK** sin que otra estación se meta.

Los 6 pasos de la transmisión

```
1. SENSE          el medio está ocupado?
                    SÍ ⟶ esperar hasta que se libere, y después
                          esperar un DIFS
                    NO ⟶ seguir al paso 2

2. BACKOFF        elegir un número entero al azar en [0, CW]
                  → lo BAJAR de a uno, uno por cada slot time
                    (slot time = 2 × tiempo de propagación)
                    si el medio se ocupa → pausar el contador
                    y recalcular al desbloquearse
                  Cuando el contador llega a 0:

3. TRANSMITIR     mandar la trama
                    AHORA PUEDE HABER UNA COLISIÓN Y NO LO VAMOS A SABER

4. ESPERAR        ¿llega el ACK?
                  SÍ ⟶ listo, se INCREMENTA la CW (para la próxima trama
                         tengo menos ganas de colisionar)
                  NO ⟶ COLISIÓN (inferida)
                         ⟶ RETRANSMITIR
                         ⟶ DOBLAR la CW  ← esto es la detección

5. RETRANSMITIR   volver al paso 2 (con la CW nueva)
                  hasta llegar a CWmax

6. LLEGAR A       si se llega a CWmax, se ABANDONA la trama
   CWmax           (no se reintenta más) y se informa error
```

Los valores de la contention window (CW), que es lo que hay que saber:

Intento	CW	Valores posibles	Probabilidad de **colisión** (2 estaciones)
0	**16**	0..15	1/16 = 6,25 %
1	**32**	0..31	1/32 = 3,1 %
2	**64**	0..63	1/64 = 1,6 %
3	**128**	0..127	1/128 = 0,78 %
4	**256**	0..255	0,39 %
5	**512**	0..511	0,20 %
6	**1023**	0..1023	0,10 %
7+	**1023** (CWmax)	0..1023	se abandona

> **El razonamiento detrás de la fórmula:** con dos estaciones que eligen **uniformemente al azar** en `[0, CW−1]`, colisionan si eligen **el mismo número**. Hay `CW` posibilidades y `CW` pares ordenados, así que la probabilidad de que elige **el mismo** es `1/CW`. Con CW = 16 es 1/16; con CW = 32 se **reduce a la mitad**. **Por eso la CW se duplica**: cada duplicación **reduce a la mitad** la probabilidad de volver a colisionar.

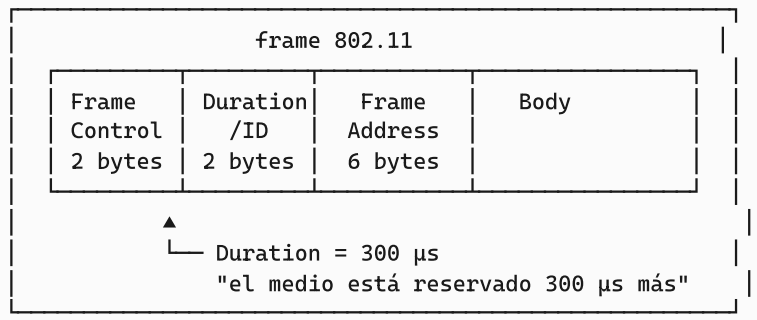
El ACK es por trama, no acumulado. Es una diferencia crítica con TCP: en WiFi **cada trama** que lleva datos es confirmada individualmente. Es lo que permite saber **cuál** se perdió y retransmitir solo esa.

Y el principio de por qué no es acumulada: con ACK acumulado, un solo ACK perdido haría retransmitir tramas que ya llegaron. Con ACK por trama hay más tráfico de control, pero **se recupera más rápido**. En una

red con colisiones frecuentes, la recuperación rápida gana.

1.4 El NAV: el sensor virtual

El **NAV** (*Network Allocation Vector*) es un **contador virtual** que cada estación mantiene. Se actualiza con el campo **Duration / ID** del header de cada trama que se recibe.




```
La estación que la escucha hace:   NAV ← 300 µs
y aunque no oiga NADA más, espera hasta que su NAV llegue a 0.
```

Los dos sensores del medio:

	Sensor	Qué detecta
Physical carrier sensing	el receptor de radio	una señal **real** en el aire
Virtual carrier sensing	el **NAV**	una **reserva anunciada** por alguien que lo oyó

El "virtual" es porque no hay señal: hay un número. Y es lo que resuelve la estación oculta: C **no oye** la trama de A (sensor físico inútil), pero **sí oye** el CTS de B, y ahí actualiza su NAV (sensor virtual).

Un detalle técnico sobre 802.11

A diferencia de Ethernet, que tiene **una sola MAC** (la de destino), **802.11 tiene hasta 4**:

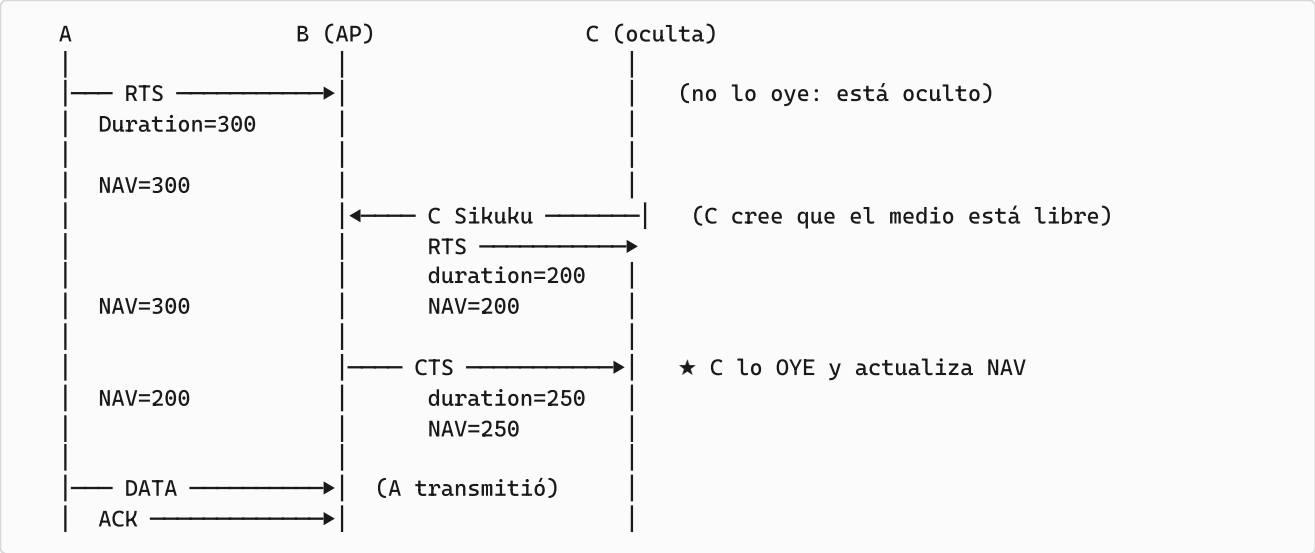
Sigla	Significado	Qué es
TA	Transmitter Address	quién **transmite** la trama **por radio**
RA	Receiver Address	quién la **recibe por radio**
SA	Source Address	dirección MAC **del origen original** (puede venir de otro enlace)
DA	Destination Address	dirección MAC **del destino final** (puede estar en otro enlace)

Las 4 coinciden cuando la trama no se reenvía (uso directo: estación ↔ AP). Se difieren en el modo **punto a punto / WDS** o cuando un AP hace **puente**: ahí la **SA** es la del cliente (la que el cliente puso) y la **TA** es la del AP.

> **Esto aparece en el Examen 2011-02 Problema 2** (IP Móvil), que pide explícitamente llenar TA, RA, SA y DA en cada segmento. **No lo confundas**: el **TA/RA** son de radio, el **SA/DA** son "de red".

1.5 RTS/CTS: la solución a la estación oculta

RTS = *Request To Send*, **CTS** = *Clear To Send*.



Los 4 pasos y por qué funcionan:

1. **A manda un RTS** al AP, con un `Duration` que declara cuánto va a durar la transacción completa.
2. **El AP responde con un CTS**, también con `Duration`, y **todos los que lo oyen** (incluido C) actualizan su NAV.
3. **A manda la DATA**. Como C tiene NAV > 0, **se calla**.
4. **El AP manda el ACK**.

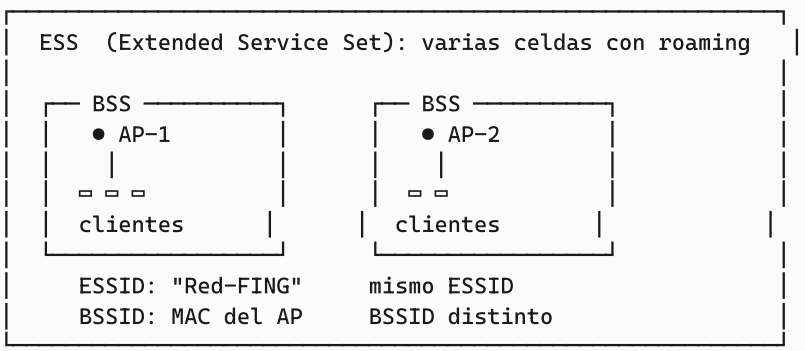
Tres decisiones de diseño, y las tres se preguntan:

a) ¿Por qué el CTS es más corto que el RTS? Porque el CTS **sólo necesita** llegar a quien está en la zona de solapamiento (C, que está cerca del AP). El RTS tiene que llegar **desde A**, que puede estar lejos y atenuarse. **Menos tiempo en el medio = menos probabilidad de colisión** en el propio RTS/CTS.

b) ¿Por qué RTS/CTS es opcional? Porque **cuesta**: son dos tramas extra (RTS + CTS) por cada trama de datos. Si la trama es chica, el overhead es mayor que el dato. De ahí el **umbral RTS**: sólo se manda RTS si la trama supera una longitud mínima.

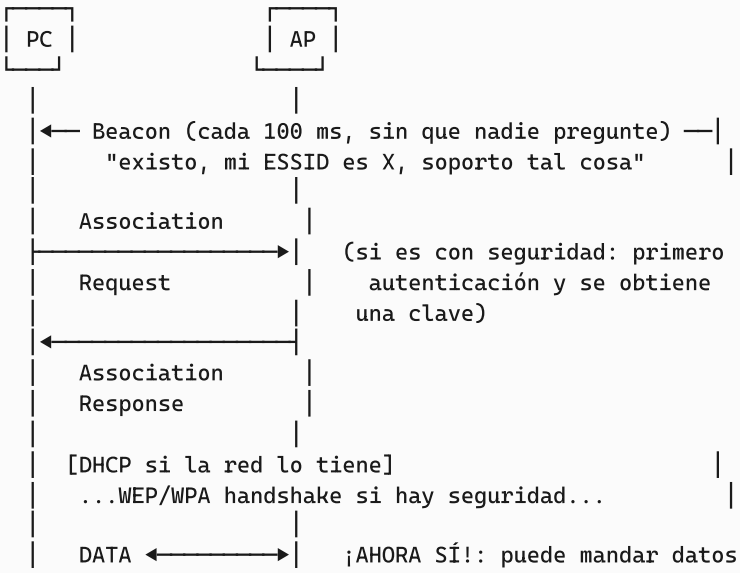
c) ¿Y si no hay ningún nodo oculto? En una red chica donde todos se ven, RTS/CTS es **puro overhead**. El estándar lo resuelve con el **umbral RTS**, que se configura (típicamente 500-2000 bytes).

1.6 Modo infraestructura



Sigla	Significado	Qué es
BSS	Basic Service Set	una celda: **un** AP + sus clientes
ESS	Extended Service Set	varias BSS con **el mismo ESSID**, para roaming
ESSID	el "nombre" de la red	lo que ves en la lista de WiFi
BSSID	la **MAC del AP**	identifica **la celda**, no la red

Las tramas de gestión y el proceso de asociación



Trama	Para qué
Beacon	el AP **anuncia** su existencia periódicamente. **No requiere** que nadie pregunte.
Probe Request	el cliente **pregunta** si hay APs disponibles.
Probe Response	el AP **responde** a un Probe Request.
Association Request	el cliente **pide** asociarse a esa BSS.
Association Response	el AP **acepta** (o rechaza).

> **El Beacon es lo que hace que el roaming funcione**: como se anuncia solo, un cliente que se mueve de celda **descubre** la nueva sin que nadie le avise. Y si la red tiene **seguridad**, el Beacon **también anuncia si está cifrada** — por eso una red WPA2 **no muestra** su ESSID en la lista pública.

1.7 EDCA: las 4 colas de acceso

Con EDCA hay **4 colas de prioridad**, cada una con su **CWmin** y **CWmax**:

Prioridad	Cola	Nombre	CWmin	CWmax	Para qué
0 (baja)	AC_BK	**Background**	1024	1024	descargas, actualizaciones
1	AC_BE	**Best Effort**	1024	1024	navegación web
2	AC_VI	**Video**	128	1024	streaming
3 (alta)	AC_VO	**Voice**	16	64	VoIP

La regla de oro de EDCA: dentro de cada cola, el acceso es **como CSMA/CA normal** (backoff aleatorio entre 0 y CW). Entre colas, la de mayor prioridad **gana**.

El detalle fino que se pregunta: ¿por qué el video NO usa la cola de voz?

Porque **el video no necesita baja latencia, necesita bajo jitter**:

- **Voz**: cada muestra de audio dura 20 ms. Si un paquete tarda 100 ms, la conversación **se corta** y se nota muchísimo. La prioridad está en el **tiempo estricto**.
- **Video**: un frame que tarda 100 ms en llegar produce un **artefacto visual** durante un frame (33 ms a 30 fps). Se nota, pero **se recupera**. Y un frame atrasado que llega más tarde **ya no sirve**, así que la prioridad está en el **jitter** y en el **tasa**, no en el retardo.

> **Decir esto es lo que diferencia**: los dos son multimedia, los dos tienen retardo, pero **el objetivo de QoS es diferente**.

Y sumá el **Block ACK**: en vez de confirmar cada trama, se confirman **varias de una vez** (con un *bitmap* de cuáles llegaron). Funciona bien en video (donde se pierde de vez en cuando) pero **no** en voz (donde cada trama es insustituible).

1.8 Seguridad

WEP: no usarlo nunca

```
Problema 1:  la clave se CIFRA con RC4, pero se transmite
             junto a cada trama. Cualquiera que sniffe un
             paquete obtiene la clave.

Problema 2:  el vector de inicialización (IV) tiene 24 bits.
             Con muchas tramas se REPETEN, y dos textos
             cifrados con la misma clave y el mismo IV se
             anulan (ataque deKepto/keystream reuse).

Resultado:    se rompe en minutos con herramientas freely
             disponibles. En 2001 se rompió en tiempo real.
```

WPA2 (802.11i, 2004)

AES-CCMP, **clave compartida (PSK)** derivada por PBKDF2. Mejora real sobre WEP (CAMBIA la clave y usa IV de 48 bits), pero **conserva el defecto de fondo**: la clave se deriva de una **passphrase corta**, y con un **ataque de diccionario offline** contra el handshake se la puede recuperar. Ése era el punto débil.

WPA3 y SAE

- **SAE** (*Simultaneous Authentication of Equals*), basado en el protocolo **Dragonfly**: la autenticación es **resistente a ataques de diccionario offline**. Aunque alguien capture el handshake, **no puede** probar contraseñas sin|label interacted con el servidor.
- **802.11w (PMF, Protected Management Frames)**: obligatorio en WPA3. Protege las **tramas de administración**, que en WPA2 iban **en claro** y permitían un **deauth flood**: un atacante manda tramas de desautenticación falsos y **desconecta a los usuarios legítimos**.

	WPA2	WPA3
Cifrado	AES-CCMP	AES-CCMP
Clave	PSK + PBKDF2 (4-way handshake)	**SAE** (Dragonfly)
Ataque de diccionario	**posible** (offline)	**imposible** (offline)
Tramas de gestión	**en claro**	**protegidas** (802.11w obligatorio)
Forward secrecy	**no**	**sí**

1.9 ⚠️ Actualización a 2026

Sigue igual — de memoria: - **CSMA/CA** con DCF, el **backoff aleatorio** en `[0, CW]`, y la **CW que se duplica** en cada retransmisión (CWmin = 16, CWmax = 1023). - **SIFS / PIFS / DIFS / EIFS**, y el hecho de que **SIFS es el más corto** y por eso la respuesta pasa siempre antes que un nuevo intento de transmisión. - **El ACK por trama**, no acumulado. - **NAV**, y el **virtual carrier sensing** como segundo sensor. - **RTS/CTS** para la estación oculta, **opcional**, con el **CTS más corto** que el RTS. - Los **4 espacios inter-trama** y el **slot time** = 2 × propagación. - El medio de 802.11 es **half-duplex**: no existe el full-duplex. - **EDCA**, las **4 colas**, y el **Block ACK**.

- El concepto de **BSS/ESS/ESSID/BSSID**. - La **estación oculta** y la **estación expuesta**, y que RTS/CTS arregla la primera y no la segunda.

Cambió — y es unaactly qué: - **Wi-Fi 6 / 6E (`802.11ax`, 2020)**: **OFDMA** (en uplink **y** downlink), **BSS coloring**, 1024-QAM, **TWT** (*Target Wake Time*, para sleepy clients en IoT). El **BSS coloring** es lo relevante para esta clase: **resuelve la interferencia entre redes vecinas** etiquetando cada una con un color y usando **BSSID reutilizado**, en vez de depender de RTS/CTS. - **Wi-Fi 7 (`802.11be`)**: aprobado por IEEE en **sept-2024**, publicado como `IEEE 802.11be-2024` en **jul-2025**. **320 MHz** de canal, **4096-QAM** y **MLO** (Multi-Link Operation: usar varias bandas a la vez). - **Wi-Fi 8 (`802.11bn`)**: en desarrollo, con foco en **reliability**, no en throughput. - **`802.11bt`**: frente nuevo de **criptografía post-cuántica** en WiFi. - **WPA3** es hoy el estándar recomendado; WPA2 queda como legado aceptable.

> **Consecuencia para el examen:** la Pregunta 3 de `2008-07` (CSMA/CA) y la P5 de `2006-12` (estación oculta) **se responden exactamente como en 2012**. Lo único que puede cambiar es que te pidan los **valores de CWmin/CWmax** o el **orden de los espacios inter-trama**, que no cambiaron.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La estación oculta, paso a paso

Datos. AP en el centro. Estaciones **A** y **C** a su alcance (pero no entre sí), y **B** al otro lado. Las tres quieren enviar.

Resolución:

```
Paso 1:  A detecta el medio libre → transmite su trama a B.
         C NO la oye (está oculta) → cree que el medio está libre.

Paso 2:  C transmite su trama al AP, AL MISMO TIEMPO que A.
         Las dos tramas LLEGAN AL AP JUNTAS → COLISIÓN en el AP.
         (No colisionan entre sí: nunca se encuentran en el aire.)

Paso 3:  El AP no les manda ACK a ninguna. Las dos tramas se perdieron.

Paso 4:  A y C, sin ACK, asumen colisión → retransmiten con CW mayor.
         Como eligen números distintos al azar, esta vez no chocan.

CON RTS/CTS:  antes de los datos, A manda RTS; el AP responde CTS.
              C (que no oyó el RTS) SÍ oye el CTS → actualiza su NAV
              y se calla. No hay colisión.
```

La frase para el examen: "*la colisión no ocurre entre A y C sino en el punto de convergencia, el AP; y el problema es que C no tiene forma de saber que A está transmitiendo.*"

2.2 Ejercicio 2 — El cálculo del NAV

Datos. A manda un RTS con `Duration = 300 µs`. El CTS del AP trae `Duration = 250 µs`. El tiempo de propagación entre dos estaciones es **2 µs**.

Resolución — el NAV de cada estación:

```
t = 0        A envía RTS
t = 2 µs     el AP lo recibe (tras 2 µs de propagación)
t = 4 µs     el AP envía el CTS
t = 6 µs     el CTS llega a todos los que lo oyen
```

Estación	¿Oye el RTS?	¿Oye el CTS?	NAV después
A	sí (lo emitió)	sí	250 − (ya pasó algo de tiempo) ≈ 244 µs
B (AP)	sí	sí (lo emitió)	250 µs
C (oculta)	**NO**	**sí**	**250 µs** ← el que lo resuelve

Y cuando A manda la DATA: lleva su propio `Duration`, digamos 200 µs. Todas las estaciones que lo oyen actualizan su NAV.

El cálculo del overhead (que es lo que se pregunta):

```
Con RTS/CTS, para un dato de 1000 bytes:

   RTS      =  20 bytes de headers  (14 bytes de 802.11 + 4 de FCS)
   CTS      =  14 bytes
   DATA     = 1000 + 30 bytes de headers = 1030 bytes
   ACK      =  14 bytes
   ──────────────────────────────────────
   total    = 1078 bytes

   eficiencia = 1000 / 1078 = 92,8 %

Con DATA sola (sin RTS/CTS):
   DATA + ACK = 1030 + 14 = 1044
   eficiencia = 1000 / 1044 = 95,8 %
```

Conclusión, que es el punto del ejercicio: RTS/CTS **cuesta 3 % de eficiencia**. Si el umbral RTS está en 500 bytes y la trama es de **100**, el RTS/CTS **no compensa**: la probabilidad de que esa trama de 100 bytes colisione es muy baja, y el overhead no vale el riesgo evitado.

2.3 Ejercicio 3 — La tabla de CW y la probabilidad de colisión

Datos. Dos estaciones. CWmin = 16, CWmax = 1023.

Resolución:

Intento	CW	Elecciones	P(choque)	P(éxito en este intento)
0	16	16	1/16 = **6,25 %**	15/16 = 93,75 %
1	32	32	1/32 = **3,125 %**	97,9 %
2	64	64	1,56 %	98,4 %
3	128	128	0,78 %	99,2 %
4	256	256	0,39 %	99,6 %
5	512	512	0,195 %	99,8 %
6	**1023**	1023	0,098 %	99,9 %

La probabilidad de tener éxito en los primeros k intentos:

```
k=1:  1 − 1/16                      = 93,75 %
k=2:  1 − (1/16)(1/32)              = 99,8 %
k=3:  1 − (1/16)(1/32)(1/64)        = 99,998 %
```

El razonamiento que hay que decir: cada intento **independiente** falla con probabilidad `1/CW`, así que la probabilidad de fallar **todos** los primeros k es el **producto**. Como la CW **se duplica**, el producto **decae exponencialmente**: después de 3 intentos ya es despreciable. **Por eso el backoff exponencial es tan efectivo** y es el mismo principio que la retransmisión de TCP (clase 20).

2.4 Ejercicio 4 — ¿Por qué no se puede detectar la colisión en WiFi?

Enunciado: *Explique por qué una estación WiFi no puede detectar una colisión como lo hace una estación Ethernet.*

Resolución, los tres motivos:

1. **Half-duplex físico.** Para detectar una colisión hay que **escuchar mientras se transmite**. La antena de radio no puede hacer las dos cosas: es como hablar y escuchar a la vez. Ethernet también tiene unaLimitación similar (con CSMA/CD sobre cable, se escucha la señal del propio cable, lo que es físicamente distinto).
2. **Enmascaramiento por potencia.** La señal propia llega al receptor a unos 30 dBm (1 vatio a 30 cm). Una señal ajena que rebota puede llegar a −90 dBm. La diferencia es de **120 dB**, o sea un factor de **10^6 en potencia**. El ruido de la señal propia es **1 millón de veces** mayor: no hay manera de escuchar por debajo.
3. **El medio es invisible.** Las ondas atraviesan paredes y no tienen frontera. En el cable, el medio es **punto a punto**: lo que sale de tu conector **no puede** estar en el conector de otro. En radio, **dos estaciones que no se ven** pueden estar usando el mismo canal y **colisionar en un tercero**.

> **La frase que resume la clase:** *en radio no se puede detectar la colisión porque no se puede escuchar mientras se transmite y porque la propia señal enmascera a las ajenas. Por eso WiFi en vez de detectar, evita: con backoff largo, con RTS/CTS y con el NAV.*

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2008-07, Pregunta 3 (10 puntos) — CSMA/CA en modo infraestructura

> **Fuente:** Examen `eirc0807` / `sirc0807`, julio de 2008. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0807.pdf

Enunciado (texto del examen)

> Explique el protocolo de acceso múltiple con evitado de colisiones (CSMA/CA) utilizado en las redes inalámbricas IEEE 802.11 (Wi-Fi) trabajando en modo infraestructura.

La solución, en 10 puntos

1. Por qué evitado y no detección (1 pt). En radio no se puede detectar una colisión: el transceiver es **half-duplex** (no transmite y escucha a la vez) y la potencia de la señal propia enmascera la de los demás (unos 120 dB de diferencia). Por eso el protocolo **evita** la colisión en lugar de detectarla y recuperar.

2. El modo infraestructura (2 pts). Las estaciones son **clientes** de un **punto de acceso (AP)**. Todas las tramas pasan por el AP, que hace de puente entre la BSS y el resto de la red. El AP se anuncia periódicamente con **tramas Beacon** (cada ~100 ms), lo que permite el **roaming**: el cliente que se mueve descubre la nueva celda sin que nadie le avise. La celda es la **BSS**, con su **BSSID** (MAC del AP), y varias BSS con el mismo **ESSID** forman un **ESS**.

3. El mecanismo DCF, paso a paso (4 pts).

```
1. Si el medio está libre, esperar un DIFS.
2. Elegir un entero al azar en [0, CW] y bajarlo de a uno
   por cada slot time. Si el medio se ocupa, pausar el
   contador y recalcular al desbloquearse.
3. Cuando el contador llega a 0, TRANSMITIR.
   Puede haber colisión y no vamos a enterarnos.
4. Esperar el ACK (por trama, no acumulado).
   - Si llega: la trama salió bien; se INCREMENTA la CW
     (menos ganas de colisionar).
   - Si NO llega: se infiere COLISIÓN; se RETRANSMITE
     y se DOBLA la CW.
5. Si se alcanza CWmax, se abandona la trama.
```

Los valores: `CWmin = 16`, `CWmax = 1023`. El backoff aleatorio en `[0, CW]` es lo que **roba el medio sin necesidad de un planificador central**.

4. Los espacios inter-trama (1 pt). SIFS (el más corto, para las respuestas: ACK, CTS, fragmentos), **PIFS** (para el punto de acceso en modo puntual), **DIFS** (antes de intentar transmitir una estación común), **EIFS** (tras un error o una trama que no se entendió; el más largo, porque penaliza). El **slot time** = 2 × el tiempo de propagación.

5. El control del nodo oculto (2 pts). Un **nodo oculto** es una estación que **no escucha** a otra aunque esté a su alcance del AP; si ambas transmiten, sus tramas **colisionan en el AP**. La solución son las tramas opcionales **RTS** (*Request To Send*) y **CTS** (*Clear To Send*): el CTS **no lo emite quien pide** sino el **AP**, que seLocated más cerca de todas, y por eso llega incluso a la estación oculta. Quien escucha el CTS actualiza su **NAV** (*Network Allocation Vector*), un contador virtual con la **duración declarada** en la trama, y espera. El RTS y el CTS son **opcionales** porque cuestan ancho de banda; se usan según el **umbral RTS**.

Frase de cierre que suma puntos: "el **NAV** funciona como un **segundo sensor** del medio: el *physical carrier sensing* detecta la señal real, y el *virtual carrier sensing* (el NAV) detecta una reserva anunciada por otro. Entre los dos, una estación sabe cuándo puede hablar."

3.2 Examen 2006-12, Pregunta 5 (6 puntos) — la estación oculta

> **Enunciado:** Describa el problema de "la estación oculta" que se presenta en las redes inalámbricas.

La solución, en 6 puntos:

La definición (1 pt). En una red wireless en modo infraestructura, dos estaciones pueden estar **dentro del alcance del punto de acceso** pero **fuera del alcance entre sí** (por muro, por distancia, por orientación de antena). Decimos que una es **oculta** respecto de la otra.

El síntoma (2 pts). Si la estación A transmite, la estación oculta C **no lo escucha**, así que cree que el medio está libre y **transmite también**. Las dos tramas **se superponen en el punto de acceso**, que es donde convergen. La **colisión ocurre en el AP**, no entre las dos estaciones — que es lo confuso del tema.

Por qué es culpa de CSMA/CA y no del medio (1 pt). El CSMA/CA protege contra las colisiones que las estaciones **detectan** por other's silence. Pero la estación oculta no puede detectar nada: **no escucha la señal**. No es que el protocolo falle, es que **no tiene información**.

La solución (2 pts). RTS/CTS: la estación que va a transmitir pregunta primero con un **RTS**; el AP responde con un **CTS**. El **CTS es más corto y lo emite el AP**, que está en el centro geométrico de la red, así que **también lo recibe la estación oculta**. Al oírlo, la estación oculta actualiza su **NAV** y **se abstiene de transmitir**. Con esto, la estación oculta **no transmite nunca en el medio**, y desaparece la interferencia. El RTS/CTS es **opcional**: se activa sólo cuando el tamaño de la trama supera el **umbral RTS**, porque su costo en ancho de banda sólo se justifica con tramas grandes.

Lo que NO resuelve (el punto fino, 1 pt). El problema del **nodo expuesto** es **el inverso**: dos estaciones que se ven entre sí, pero que van a destinos distintos. Una se calla innecesariamente. El RTS/CTS **no lo resuelve**, porque la condición de que el CTS llegue no cambia nada. Es una limitación conocida del protocolo, y no tiene solución limpia.

3.3 Examen 2007-03, Pregunta 5 (5 puntos) — CSMA/CA

> **Enunciado:** Describa el mecanismo de acceso al medio CSMA/CA (Carrier Sense Multiple Access With Collision Avoidance).

La solución, en 5 puntos. Es el mismo desarrollo que en §3.1, en formato corto:

1. **La idea:** cada estación **escucha** el medio; si está libre, espera un **DIFS** y luego espera un **tiempo aleatorio** (backoff) antes de transmitir, para reducir la probabilidad de que dosstations hablen a la vez.
2. **El backoff:** entero aleatorio en `[0, CW]`; CW arranca en 16 y se **duplica** en cada intento fallido hasta 1023. Esto es lo que **evita** la colisión.
3. **La confirmación:** cada trama de datos se confirma con un **ACK individual**; si no llega, se asume colisión y se reintenta con CW mayor.
4. **El RTS/CTS** como mecanismo complementario para el nodo oculto.
5. **El NAV** para reservar el medio frente a estaciones que no pueden oír la señal directa.

> **Tope para 5 puntos:** los 5 puntos con el nombre de cada parámetro.

PARTE 4 — CIERRE

📌 Resumen

1. En radio **no se puede detectar** una colisión: el transceiver es half-duplex y la señal propia enmascera a las ajenas (~120 dB). Por eso WiFi **evita**, no detecta.
2. **CSMA/CA (DCF)**: sense → DIFS → backoff aleatorio en `[0, CW]` → transmitir → esperar ACK. Sin ACK ⇒ colisión ⇒ **duplicar CW** y reintentar.
3. **CWmin = 16, CWmax = 1023**. La probabilidad de colisión con 2 estaciones es `1/CW`, y **duplicar la CW la reduce a la mitad**.
4. **SIFS** es el espacio más corto, y por eso **las respuestas pasan siempre antes** que un nuevo intento de transmisión.
5. El **ACK de WiFi es por trama**, no acumulado: se recupera más rápido.
6. **Estación oculta**: dos estaciones al alcance del AP que **no se ven entre sí**; colisionan **en el AP**.
7. **RTS/CTS** la resuelve: el CTS lo emite el AP, que está más cerca de todos, y lo oyen hasta las estaciones ocultas. **Es opcional**, con **umbral RTS**.
8. El **NAV** es un sensor **virtual** del medio: guarda la duración **declarada** en las tramas.
9. **EDCA** tiene **4 colas**; el video **no** usa la cola de voz porque la voz es **sensible al retardo** y el video al **jitter**.
10. **WPA3/SAE** resuelve el ataque de diccionario offline de WPA2; **802.11w** protege las tramas de administración (contra *deauth flood*).

✅ Autoevaluación

1. ¿Por qué una estación WiFi no puede detectar una colisión como sí hace una Ethernet?

▶ Respuesta

2. Una estación elige un backoff de 24 en un intento donde la CW es 64. ¿Cuál es la probabilidad de que otra estación elija el mismo y colisionen?

▶ Respuesta

3. ¿Por qué el `REQUEST` de DHCP va por broadcast aunque el cliente ya sepa la IP del servidor? ¿Y qué tiene que ver esto con el CSMA/CA?

▶ Respuesta

4. ¿Qué diferencia hay entre una estación oculta y una estación expuesta? ¿Cuál resuelve el RTS/CTS?

▶ Respuesta

5. ¿Por qué el video no usa la cola de prioridad más alta (AC_VO) que la voz?

▶ Respuesta

6. ¿Qué es el NAV y por qué se lo llama "virtual carrier sensing"?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc0807` **/** `sirc0807` — Pregunta 3 completa (10 pts, CSMA/CA en infraestructura)
- **Examen** `eirc0612` **/** `sirc0612` — Pregunta 5 (6 pts, estación oculta)
- **Examen** `eirc0703` **/** `sirc0703` — Pregunta 5 (5 pts, CSMA/CA)
- **Examen** `eirc1007` **/** `sirc1007` — Pregunta 2 (8 pts, por qué no CSMA/CD en wireless)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 4 (subcapa MAC)
- `ACTUALIZACIONES.md` **§9** — Wi-Fi 6/7/8, BSS coloring, WPA3/SAE, 802.11bt
- **Clase siguiente:** 44 — Multimedia: streaming, RTP y VoIP

Clase 44

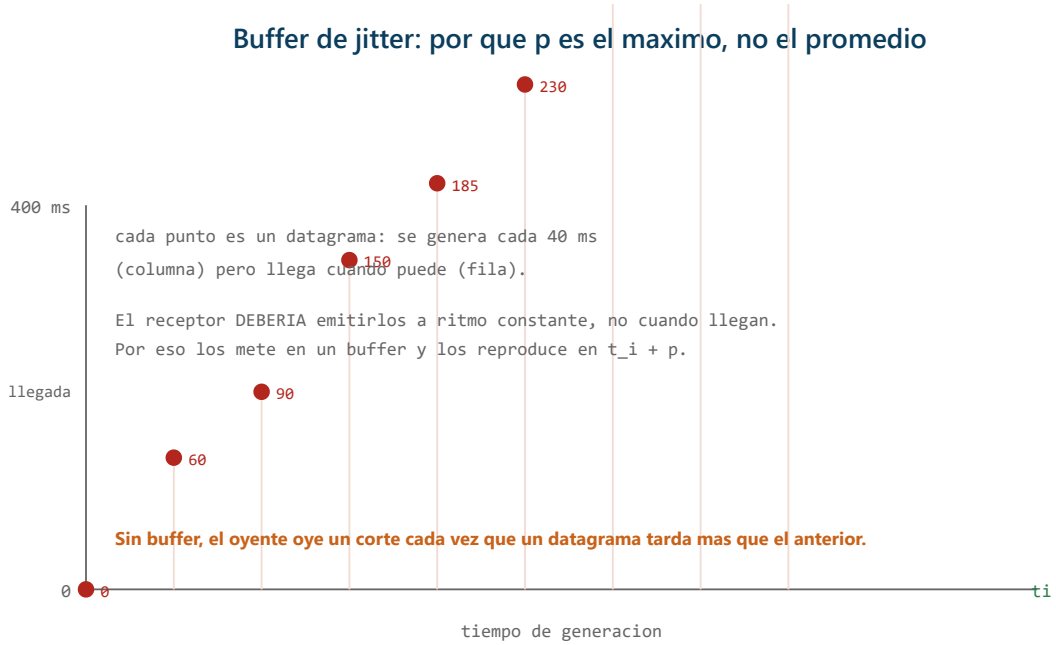
Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 Qué cambia cuando el tráfico es multimedia
- 1.2 Por qué TCP no sirve para tiempo real: las cuatro razones
- 1.3 UDP como base, y por qué
- 1.4 RTP — el formato que resuelve todo
- 1.5 El jitter y el buffer de jitter
- 1.6 MPEG-2: I, P y B frames
- 1.7 Panorama del problema de 2011-02
- 2.1 Ejercicio 1 — Dimensionar un jitter buffer, en números
- 2.2 Ejercicio 2 — El timestamp de muestreo en acción
- 2.3 Ejercicio 3 — MPEG-2: los B-frames rompen el orden
- 3.1 ⭐ Examen 2011-02, Problema 1 a) (30 pts en total) — Tamaño del datagrama
- 3.2 Examen 2011-02, Problema 1 b) — El mínimo `p`
- 3.3 Examen 2011-02, Problema 1 c) — La implementación (el corazón del problema)
- 3.4 Examen 2010-07, Problema 1 (30 puntos) — MPEG-2 I/P/B: protocolo semifiable
- 3.5 Examen 2008-07, Problema 1 (30 puntos) — Audio redundante con RTP
- 3.6 Examen 2008-12, Pregunta 5 (8 puntos) — ¿TCP para tiempo real?
- 3.7 Examen 2010-02, Pregunta 5 (8 puntos) — Jitter, retardo y sensibilidad
- 📌 Resumen
- ✅ Autoevaluación
- 🔗 Referencias

Bloque: F (Servicios, multimedia y seguridad) · **Semana:** 15 · **Duración estimada:** 115 min **Correlativa:** Clase 43 (Redes inalámbricas 802.11) · **Sigue con:** Clase 45 (QoS: shaping, policing, DiffServ y MPLS) **Material de referencia:** Tanenbaum cap. 7 (§7.4 multimedia, §7.6 RTP) · RFC 3550, RFC 3611, RFC 3711, RFC 8285

🎯 Al terminar esta clase vas a poder

1. Explicar por qué el tráfico multimedia es **tolerante a la pérdida** y por qué eso cambia el transporte.
2. Enumerar las **cuatro razones concretas** por las que TCP no sirve para tiempo real, y decir **bajo qué hipótesis** sí serviría.
3. Dibujar la cabecera de **RTP** y explicar para qué sirve cada uno de sus tres campos clave.
4. Calcular el **tamaño del campo de datos de un datagrama de voz**, el **mínimo** `p` de un buffer de jitter, y la **latencia suavizada** con promedio ponderado.
5. Describir los frames **I, P y B** de MPEG-2, el **GOP**, y por qué los B-frames obligan a reordenar.
6. **Implementar en pseudocódigo** los dos procedimientos de un receptor VoIP con jitter buffer y control de cancelación.



> **Prerequisito del examen:** `2011-02 Problema 1` (30 pts — el de VoIP, **el problema central de esta clase**), `2010-07 Problema 1` (30 pts, MPEG-2 I/P/B), `2008-07 Problema 1` (30 pts, audio redundante con RTP), `2008-12 P5` (8 pts), `2010-02 P5` (8 pts).

PARTE 1 — TEORÍA

1.1 Qué cambia cuando el tráfico es multimedia

Hasta ahora estudiamos redes donde **perder datos es un desastre**. Bajás un archivo y si falta un byte, el archivo no sirve. Ese es el mundo de TCP.

El tráfico multimedia invierte la prioridad:

```
ARCHIVO          Se pierde 1 paquete → el archivo NO sirve.
(p. ej. PDF)     ⇒ hay que retransmitir SI O SI, aunque tarde.

VIDEO / AUDIO    Se pierde 1 paquete → se pierden 40 ms de imagen
(VoIP, streaming)   o 20 ms de voz. Apenas se nota.
                  ⇒ es MUCHO mejor perderlo que esperar a recuperarlo.
```

La razón profunda: el multimedia es **degradable** (*graceful degradation*). Su valor está repartido en el tiempo: si faltan 20 ms de los últimos 5 segundos, el resultado es "casi igual de bueno". El contenido se puede **saltar** (*skip*), **interpolar** o **repetir el anterior**. Un archivo binario es **todo o nada**: la suma de checksums no te deja saltar un byte.

> **De ahí sale la decisión de diseño central de toda la clase:** para el multimedia, **un paquete que llega tarde es tan inútil como uno que se perdió**, y es mucho más caro recuperarlo que perderlo.

Aplicación	¿Pierdo un paquete?	¿Tardo?	¿Llega desordenado?
Descarga de archivo	**no puedo**: rompo el archivo	no molesta	no molesta: TCP reordena
Streaming almacenado	lo salto, no importa	**tolerantísimo**: el usuario espera	no molesta
Videollamada / VoIP	se oye un clic	**insoportable**	**insoportable**: voz entrecortada
Transacción financiera	no puedo	no molesta	no puede pasar

Fijate que **streaming almacenado y videollamada son casos distintos** aunque los dos "sean multimedia": al primero le podés meter un buffer de 30 segundos (no hay interlocutor esperando); al segundo no. El enunciado de `2010-02 P5` lo pide explícitamente y esa distinción es la que suma puntos.

1.2 Por qué TCP no sirve para tiempo real: las cuatro razones

Es la Pregunta 5 del Examen 2008-12 (8 pts). No alcanza con decir "porque TCP es fiable": hay que dar **cuatro razones concretas**, cada una con su mecanismo.

```
RAZÓN 1 — RETARDO
─────────────────────────────────
Se pierde el paquete 7. TCP NO retransmite al instante: espera a que venza el
RTO (retransmission timeout), que es >= 1 RTT y típicamente 200 ms – 1 s.
El paquete 8, 9, 10 ya llegaron al receptor y NO se pueden reproducir: el
códec es isócrono, necesita la muestra 7 para la muestra 8.

RAZÓN 2 — JITTER (la más sutil)
──────────────────────────────────
La retransmisión no introduce retardo constante: introduce una IRREGULARIDAD.
El paquete 6 tardó 80 ms, el 7 tarda 800 ms:

    retardo por paquete:
      ____▁____▁____   ← el 8 es el pico
           ↑
       esto es el jitter

Y el jitter es PEOR que el retardo absoluto: una videollamada con 400 ms de
retardo constante se ve "con delay" pero se VE. Una con 100 ms de promedio y
700 ms de picos se ve entrecortada.

RAZÓN 3 — ORDEN (imposible de evitar con TCP)
─────────────────────────────────────────────
Si se pierde el 7, el 8, 9 y 10 sí llegan. TCP los entrega A LA APLICACIÓN en
orden (8,9,10 quedan en el buffer de reordenamiento). Y reordenar NO sirve de
nada: el códec no puede decodificar el 8 sin el 7, y si el 7 llega 200 ms
después hay que reproducir 8, 9, 10 ANTES que el 7, lo cual es imposible en
un stream isócrono. El buffer de reordenamiento mantiene el orden A COSTA DE
AUMENTAR el retardo: la razón 3 es la razón 1 vista desde el receptor.

RAZÓN 4 — CONTROL DE CONGESTIÓN (TCP frena el flujo)
─────────────────────────────────────────────────────
Ante una pérdida, TCP hace lo que la clase 21 desarrolló:
    3 dupACKs → cwnd = cwnd/2     |    timeout → cwnd = 1 MSS
En una videollamada, la red está congestionada porque 200 personas miran el
mismo partido. TCP lo interpreta como "bajemos a 1 MSS" y la calidad de video
se desploma justo cuando la necesitás.
```

El detalle fino que suma puntos: las razones 1 y 3 son la misma causa (retransmisión) vista desde el emisor y desde el receptor. La razón 2 es distinta: aunque **no hubiera ninguna pérdida**, el jitter existiría, porque los paquetes toman caminos distintos y se encolan distinto en cada router. Y la razón 4 es independiente: se puede tener una red sin una sola pérdida y aun así TCP frenaría el flujo, porque el control de congestión también reacciona al aumento del RTT.

⚠️ Actualización a 2026

Sigue igual, de memoria: las cuatro razones; la distinción retardo/jitter; el buffer de jitter y su compromiso con la latencia.

Cambió o se matiza:

- **WEBRTC cambió el transporte por completo.** Ya no es "UDP + RTP": es **SRTP sobre DTLS sobre ICE sobre UDP (o TCP)**. La idea central — datagramas, sin retransmisión, con número de secuencia y timestamp — es la misma; lo que se agregó es **cifrado obligatorio**, **NAT traversal** y control de congestión adaptativo.
- **Hoy hay control de congestión en tiempo real, y funciona.** RTCP implementa **LBR** (*loss-based congestion control*): el emisor no espera a que la red se sature, sino que usa la **fracción de pérdida** que reporta el

receptor en los RTCP RR y ajusta el ritmo para mantener una tasa de pérdida objetivo. O sea: no se usa AIMD de TCP, pero **sí** hay reacción a la congestión.

- **El "0 Copy" de WebRTC** es el ejemplo moderno del mismo principio: si la red empeora, la respuesta correcta es **bajar la calidad** (menos resolución, menos fps, audio de menor bitrate), no "frenar el flujo de TCP".
- **Consecuencia para el examen:** si te preguntan "por qué TCP no sirve", las cuatro razones siguen siendo la respuesta. Si te preguntan "y hoy, ¿cómo se controla la congestión en tiempo real?", la respuesta es **LBR en RTCP**.

1.3 UDP como base, y por qué

RTP **no es un protocolo de transporte**: es un formato de encabezado que viaja encima de UDP. Y la elección de UDP es deliberada:

```
CONEXIÓN        NO. No hay handshake, no hay SYN. En una llamada de voz, el
                handshake de 1 RTT es latencia que el usuario SIENTE.
FIABILIDAD      NO. No hay ACK, ni retransmisión, ni RTO. Nadie espera nada.
ORDEN           NO. Cada datagrama es independiente: el códec decodifica cada
                muestra sola.
CONGESTIÓN      NO. No hay cwnd, ni AIMD. El control de congestión, si lo hay,
                es de la APLICACIÓN (LBR), no del transporte.
TAMAÑO          8 bytes de cabecera (puerto origen, destino, longitud, checksum).
                Mínimo: cada byte de cabecera es latencia de más en la línea.
FRONTERA DE     Preservada. Un datagrama = un mensaje (con TCP tenés el
MENSAJES        problema del PUSH).
```

Y no uses la obviedad como argumento: no digas "UDP es mejor porque es más rápido". La razón real es que **cada mecanismo de fiabilidad de TCP tiene un costo (tiempo, memoria o latencia) y para el multimedia todos esos costos son mayores que el beneficio**, porque la unidad de información es redundante y degradable.

1.4 RTP — el formato que resuelve todo

RTP (Real-time Transport Protocol) es, esencialmente, **una cabecera de 12 bytes con tres campos que hacen el trabajo**:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
┌-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|V=2|P|X|  CC   |M|     PT      |       número de secuencia     |
├-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                           timestamp                           |
├-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                             SSRC                              |
├──────────────────────────────────────────────────────────────┤
|            (extensiones / CSRC si hay; si no, payload)         |
└──────────────────────────────────────────────────────────────┘
```

```
V=2      versión
P        relleno: si es 1, hay bytes de relleno al final
X        extensión: si es 1, hay un header extension después
CC       cantidad de CSRC (32 bits) que preceden al payload
M        marker: "primer paquete de una talkspurt"
PT       payload type: qué códec es (0 = PCMU/G.711, 96 = dinámico)
seq (16) número de secuencia: +1 por paquete, por emisor
ts (32)  timestamp DE MUESTREO: no es el instante de envío, es el
         instante de la MUESTRA en el reloj de muestreo
SSRC(32) synchronization source: identifica al EMISOR
```

Los tres campos que importan y por qué

1) Número de secuencia (16 bits). Contador que se incrementa en 1 por paquete enviado, **por emisor**. Sirve para:

```
a) DETECTAR PÉRDIDAS:   tengo seq 100, 101, 103 → faltó el 102.
b) DETECTAR DUPLICADOS: recibo 103 dos veces → descarto el segundo.
c) ORDENAR: aunque no haya retransmisión, la red puede reordenar. Con el
   número de secuencia el receptor sabe en qué orden van las muestras.
```

Ojo: RTP **no** garantiza el orden, solo te da la información para **detectar** el desorden. El reordenamiento lo hacés vos, en el jitter buffer.

2) El timestamp de muestreo (32 bits). Es el campo más importante y el más incomprendido. **No es el reloj del sistema**: es el **reloj de muestreo** (*sampling clock*) del códec, y su unidad son las muestras.

```
G.711 (telefonía) : 8000 muestras/s  → 1 timestamp cada 125 µs
Opus (voz moderna): 48000 muestras/s → 1 timestamp cada ~20,8 µs
H.264             : 90000 muestras/s (clk_rate típico)

Ejemplo con 8000 Hz: el paquete con las muestras de 20 ms a 40 ms tiene
timestamp = 160 (20 ms × 8000 muestras/s). El siguiente, 320. Luego 480.
```

¿Por qué esto es lo que permite la reproducción regular? Porque el receptor puede hacer **playout a ritmo constante** sin mirar el reloj del sistema: el reloj del receptor va **fijo** a 8000 muestras/s y los timestamps dicen **en qué punto del flujo** estamos. La unidad de control es *la señal*, no el reloj de pared. Y por eso el receptor puede hasta **ajustar su propio ritmo** (el *clock recovery*, que Opus hace), sin desincronizarse del emisor.

Sin timestamp tendrías que asumir "una muestra cada 40 ms a partir de ahora", lo cual es falso: si un paquete tarda 300 ms y el siguiente 220, las muestras llegan en ráfagas.

3) SSRC (32 bits). Un identificador **aleatorio**, elegido por el emisor al arrancar el flujo, que se **repite en todos sus paquetes**. Para qué:

```
a) DISTINGUIR FUENTES: si recibís audio de tres emisores (examen 2008-07),
   cada uno tiene su SSRC.
b) SEPARAR FLUJOS: una videoconferencia tiene 2 flujos de medios (A→B, B→A)
   más los de control RTCP.
c) RECONSTRUIR TRAMAS: al reassemblar una trama de video dividida, se juntan
   los fragmentos que tienen el MISMO SSRC.
d) VALIDAR: un SSRC desconocido en una sesión es tráfico inyectado.
```

RTCP: el canal de control y de feedback

RTP va solo (es *simplex*). Para que el emisor sepa qué pasó hace falta un canal de vuelta: **RTCP (RTP Control Protocol)**. Los cinco mensajes que hay que saber:

Mensaje	Qué es	Para qué
SR (*Sender Report*)	lo manda el que **emite** audio/video	**sincroniza los relojes** entre flujos de una misma sesión
RR (*Receiver Report*)	lo manda el que **recibe**	informa **tasa de pérdida**, **jitter medido** y último seq/timestamp recibido
SDES (*Source Description*)	nombre del emisor	quién es
BYE	fin de sesión	se desconecta
APP	datos de aplicación	—

El RR es el corazón del control adaptativo. Manda periódicamente (clásicamente el 5 % de los paquetes RTP) y dice:

```
RR:  fracción de paquetes perdidos = 0,03   (3 %)
     jitter acumulado (interarrival)  = 12 ms
     último seq recibido            = 8451
     último timestamp recibido      = 1.234.567
     tiempo desde el último SR      = 3.2 s
```

El emisor usa la **fracción de pérdida** para decidir si sube, mantiene o baja la tasa de envío: es el **LBR** que mencioné en la actualización.

> ⚠️ **Y esto se confunde siempre:** ¿quién manda el RR? **Quien recibe.** ¿Y el SR? **Quien emite.** Es contraintuitivo. La razón es histórica: el SR sirve para sincronizar los relojes de los flujos de **una misma sesión** (audio y video de un mismo participante), así que lo manda quien tiene esos flujos.

1.5 El jitter y el buffer de jitter

Definiciones

```
RETARDO  d = r_i − t_i  de un paquete i
         = tiempo que tarda ESE paquete

JITTER   J = |d_i − d_j| entre paquetes del MISMO flujo
         = "cuánto VARÍA la demora"

LATENCIA L = r_i − u_i de una muestra i, con u_i = t_i + p
         = "cuánto se delayó respecto de lo que el receptor decidió"
         = el retardo CON EL BUFFER POR DELANTE
```

La definición oficial (`2010-02 P5`) es: "*El retardo es el tiempo que tarda un paquete desde que sale del emisor hasta que llega al receptor, mientras que el jitter es la variación de este retardo entre distintos paquetes de un mismo flujo*". Sumá el corolario que da la misma solución: **"un alto jitter puede implicar que paquetes que salieron más tarde lleguen más temprano"**, o sea, **desorden**.

Y el caso que la misma solución señala, que es la mejor forma de explicar por qué son conceptos distintos: si todos los paquetes siguen el mismo camino y no hay congestión, **el retardo puede ser constante (2 segundos) y el jitter cero**. El retardo grande molesta; el jitter chico no. **El buffer de jitter grande resuelve el jitter, pero no el retardo absoluto.**

El buffer de jitter: qué hace y por qué tiene que existir

El receptor NO puede reproducir en el momento en que llega el paquete: tiene que **almacenar en un buffer** y reproducir **a ritmo constante**.

```
EMISOR (ritmo constante, 25 datagramas/s)
t:   0    40   80  120  160  200  240  280  320  360  400
     ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼
     1    2    3    4    5    6    7    8    9   10   11   generados

RED (retardo variable)
r:  260  300  380  420  420  500  500  620  620  760 ...
     ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼
     1    2    3    4    5    6    7    8    9   10   11   recibidos

El receptor no puede reproducir el 1 a las 260 y el 2 a las 300, porque el
intervalo correcto es 40 ms. ⇒ ALMACENA y reproduce a ritmo constante.
```

Y además el buffer **reordena**: si el paquete 4 tarda 260 ms y el 5 tarda 180, el 5 llega primero; el número de secuencia y el timestamp permiten poner el 4 en su lugar y reproducir en orden.

El compromiso: tamaño del buffer vs. latencia

```
Buffer PEQUEÑO                    Buffer GRANDE
p = 100 ms                        p = 500 ms
latencia: BAJA ✓                   latencia: ALTA ✗
cortes:    MUCHOS ✗                cortes:    POCOS ✓

¿Por qué? Un paquete con r_i − t_i > p no llegó a tiempo y se pierde.
```

No hay escapatoria: es una decisión de diseño con **dos costos opuestos**. En la práctica se usa un **buffer adaptativo**, que mide el jitter y ajusta el tamaño en runtime, con un **máximo duro**. Por eso el enunciado del examen da `p_0 = 400 ms`, y por eso existe el mecanismo de **cancelación**: si la latencia pasa de ese máximo, se prefiere **colgar** antes que seguir con una conversación de 800 ms.

```
buffer adaptativo:  jitter medido ⟶ tamaño objetivo = k · jitter + Δ
   jitter sube (congestión) ⟶ agrando el buffer (absorbe picos)
   red tranquila ⟶ no se toca nada
   jitter baja ⟶ achico el buffer (menos latencia)
```

⚠️ Actualización a 2026

Sigue igual: el compromiso buffer grande vs. latencia; **RTP no cambió desde 2003**: `RFC 3550` sigue siendo *Internet Standard* y no se le modificó el protocolo base. Un curso de 2012 que dice "RTP es esto" está **correcto hoy**.

Cambió / se agregó:

- **Header extensions (`RFC 8285` , 2017; obsolesce `RFC 5285`).** Un mecanismo genérico para meter campos extra: **abs-send-time** (para medir el one-way delay sin depender de relojes sincronizados), **transport-cc-cc** (información del control de congestión *transport-wide*), **audio-level** (nivel del micrófono, para no mandar audio si nadie habla). Antes de 2017 eso se hacía con campos propietarios.
- **RTCP XR (`RFC 3611`) — eXtended Reports.** Los cinco mensajes clásicos no alcanzan para diagnosticar calidad; XR agrega reportes VoIP: **calidad de experiencia** (MOS), jitter, delay y tasa de pérdida de la llamada.
- **SRTP (`RFC 3711`) pasó a ser obligatorio en WebRTC.** `DTLS-SRTP` es la base de la seguridad de WebRTC. Ya no es "se puede cifrar RTP": es **no se hace WebRTC sin cifrar**. Ojo: SRTP da confidencialidad, integridad y **anti-replay**, pero **no** da autenticidad del origen de la sesión.
- **Los códecs: acá es donde un curso de 2012 más se desactualiza.**

Códec	Situación 2026	Dónde vive
MPEG-2	sigue vivo, pero **confinado** a TV de broadcast (DVB-T/S, ATSC), DVD, video de contribución	ya no es "el códec de streaming de Internet"
MPEG-4 Part 2	reemplazado por H.264; irrelevante	—
H.264/AVC	~**84-85 %** de producción; el **fallback universal**	todas partes
H.265/HEVC	~**65 %** de producción; **nativo en Apple**	royalties: hay que pagar a los titulares de las patentes
AV1 (2018)	**royalty-free**; ~17-19 % de producción → **~57 %** de alcance combinado fin de 2026; **Netflix ~30 %** de su streaming (dic-2025); **YouTube >75 %** del video de alta resolución	el crecimiento real
VVC/H.266	**<5 %**, atascado por licencias	casi nadie lo despacha
Opus	el estándar de voz en WebRTC y en casi todo VoIP moderno	es lo que hace que "VoIP" sea aceptable en un enlace malo

- **Consecuencia para el examen:** si el enunciado habla de **MPEG-2, I/P/B, GOP, jitter, RTP**, respondé lo de 2011 y te da todo. Si habla de "códecs de video en Internet hoy", la respuesta es **AV1 está creciendo y MPEG-2 quedó atrás**.

1.6 MPEG-2: I, P y B frames

```
I-frame  (intra, "independiente")
──────────────────────────────────────────────────────────────────────

Contiene la imagen COMPLETA, como un JPEG. No depende de nada.
Es CARO en bits: típicamente 10-50 veces más grande que un P-frame.

P-frame  (predictive, "predictivo hacia atrás")
──────────────────────────────────────────────────────────────────────

Solo codifica la DIFERENCIA respecto del frame anterior. Envía "en este
rectángulo cambió de rojo a azul". DEPENDE del frame anterior: si falta
ese, este P no sirve.

B-frame  (bidirectional, "bidireccional")
──────────────────────────────────────────────────────────────────────

Solo codifica la diferencia respecto de los frames ANTERIOR y POSTERIOR.
Es el más chico de los tres.

⚠️ LA TRAMPA CLÁSICA: un B-frame depende de un frame que AÚN NO LLEGÓ.
O sea, el decodificador tiene que ESPERAR un frame que viene después en
el flujo de transmisión.
```

El GOP y el keyframe interval

```
Un GOP (Group of Pictures) es la secuencia
    I   P   B   P   B   P   B   P   B   P   B   (P/B en cualquier orden)

| I | P | B | P | P | B | P | B | P | B | P | B | P | B | P | B |
  0   1   2   3   4   5   6   7   8   9  10  11  12  ...

keyframe interval = 12 ⇒ un I-frame cada 12 frames.
```

¿Por qué importa? Porque es el precio del *entry point*. Un cliente que arranca tiene dos opciones: si quiere empezar en cualquier momento (streaming en vivo) debe **esperar al próximo I-frame**, así que cuanto más largo el GOP, más tarda en arrancar; si quiere saltar a cualquier punto (VOD, *seek*) necesita un I-frame cada pocos segundos.

Trade-off fundamental: GOP corto = más I-frames caros = menos calidad con el mismo bitrate. GOP largo = mejor calidad, peor arranque y peor navegación. Por eso los servicios en vivo usan GOP de 1-2 s y los de VOD de 2-10 s.

Por qué los B-frames obligan a REORDENAR y a un DELAY

```
SECUENCIA DE DECODIFICACIÓN (orden lógico):
    I0  P1  B2  P3  B4  P5  B6 ...
                        ▲
                        └─ B2 depende de P1 y de P3, pero P3 viene DESPUÉS
                           de B2 en la secuencia de decodificación.

SECUENCIA DE TRANSMISIÓN (orden en el cable):
    I0  P1  P3  B2  B4  P5  P6  B6 ...
                ▲
                └── P3 viaja ANTES que B2, porque B2 no se puede decodificar
                    hasta que llegó P3.

Consecuencias:
1. El receptor necesita un BUFFER DE REORDENAMIENTO: guardar P3 un rato.
2. Hay un DELAY mínimo de reproducción: no podés reproducir B2 en el
   instante en que llegó, porque P3 quizá no llegó todavía.
3. Si un P se pierde, todos los B que dependían de él se descartan (el códec
   los salta).
```

Relación con RTP: los timestamps viajan en el orden **de transmisión** (crece monótonamente), pero el decodificador necesita el **orden de presentación**, que es otro. Muchos decodificadores reordenan internamente.

1.7 Panorama del problema de 2011-02

```
(a) Tamaño del campo de datos del datagrama UDP   ← ARITMÉTICA CON UNIDADES
    64 kbps, 1 muestra cada 40 ms, 16 B de overhead
(b) Mínimo p para reproducción continua (9 dgrams) ← MÁXIMO, no el último
    p = max (r_i − t_i)
(c) IMPLEMENTACIÓN de dos procedimientos           ← EL 60% DE LOS 30 PTS
    get()      : recibe, calcula p_i suavizado, ordena
    reproduce(): saca de a uno, mide pérdida, CANCELA
    p_i = 0,9·p_{i−1} + 0,1·v_i ,  p_0 = 400 ms
    Cancela si p_i > 400 ms durante N períodos seguidos, o pérdida > 10 %
```

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Dimensionar un jitter buffer, en números

Datos. Un datagrama cada 40 ms. En una ventana de 10 paquetes, los retardos `r_i − t_i` fueron: `220, 220, 260, 260, 220, 260, 220, 300, 260, 280` [ms].

a) ¿Qué `p` garantiza reproducción continua sin cortes?

Paso 1 — condición de reproducción. La muestra `i` se reproduce en `t_i + p`, y se reproduce **a tiempo** si llegó antes:

```
r_i ≤ t_i + p   ⇔   p ≥ r_i − t_i = v_i    para TODO i

⇒ p_min = max (v_i) = 300 ms   (el paquete 8)
```

b) ¿Cuánto jitter hay? Con la definición "variación del retardo entre paquetes del mismo flujo":

```
|220−220| =  0    |260−220| = 40    |260−260| = 0    |220−260| = 40
|260−220| = 40    |220−260| = 40    |300−220| = 80  ← el más grande
|260−300| = 40    |280−260| = 20

J_max = 80 ms
```

c) Si subo `p` a 500 ms, ¿cuánto riesgo de corte saco y cuánto pierdo?

```
p = 300 ms: los 10 llegan con holgura EXCEPTO el 8, que llega justo
            (300 = 300). Con 1 ms de ruido, el 8 se pierde ⇒ corte.
p = 500 ms: los 10 llegan con holgura ✓
            latencia extra = 500 − 300 = 200 ms
```

Interpretación. 200 ms de latencia es el precio. En una conversación se percibe (se empieza a hablar por encima del otro), pero sigue siendo usable. En video, 200 ms se nota bastante.

2.2 Ejercicio 2 — El timestamp de muestreo en acción

Datos. G.711: 8000 muestras/s. Un datagrama cada 40 ms ⇒ **320 muestras por datagrama**. El primer datagrama se genera en `t_0 = 0`.

a) ¿Qué timestamp lleva cada datagrama?

```
ts(i) = i · 320  muestras     (unidades del reloj de muestreo)
i        1      2      3      4      5      6      7      8
ts     320    640    960   1280   1600   1920   2240   2560
```

b) El datagrama 5 llegó a `r = 780 ms`. ¿Qué muestra representa y cuándo se reproduce con `p = 300`?

```
Representa las muestras del intervalo
    [ (1600-320)/8000 , 1600/8000 ] = [ 160 ms , 200 ms ]
    o sea: el cuarto de segundo de la señal.

Se reproduce en t + p = 200 + 300 = 500 ms.
Llegó a 780 ms > 500 ms ⇒ TARDE. Se descarta (se oye un clic).

Con p = 800: se reproduce en 1000 ms, llegó a 780 < 1000 ✓
latencia = 1000 − 780 = 220 ms.
```

c) ¿Y si el códec fuera Opus a 48 000 Hz?

```
48 000 muestras/s × 0,040 s = 1920 muestras por datagrama
ts(i) = i · 1920

El TIMESTAMP NO es "el instante en milisegundos": es el instante en
UNIDADES DEL RELOJ DE MUESTREO. Cambiar de códec cambia el reloj, y por eso
un receptor tiene que conocer el clk_rate del códec.
```

2.3 Ejercicio 3 — MPEG-2: los B-frames rompen el orden

Datos. Un GOP de 6 frames: `I P B P B P`, con numeración de **secuencia** 0..5 y de **transmisión** A..F.

```
a) DECODIFICACIÓN:   A=I(0) B=P(1) C=B(2) D=P(3) E=B(4) F=P(5)
                     0     1     2     3     4     5

b) TRANSMISIÓN:      A=I(0)  B=P(1)  C=P(3)  D=B(2)  E=B(4)  F=P(5)
                     ▲               ▲
                     │               └── C tiene que viajar ANTES que D,
                     │                   porque D=B(2) depende de C=P(3)
                     └── igual que la secuencia lógica (el I va primero)

c) Si se pierde C (el P(3)):
     B(2) y B(4) no se pueden decodificar ⇒ se saltan.
     P(5) referencia a P(3) como "anterior", así que también se pierde, y
     toda la cadena posterior hasta el próximo I.
     ⇒ un B perdido puede arrastrar hasta medio GOP.
     ⇒ por eso el I-frame es el que hay que PROTEGER: perder un I = 6 frames
       (1 segundo) de video negro; perder un B = quizá 2 frames.

d) DELAY de reproducción: tiene que reproducir D (=B(2), que depende de C)
   sólo cuando C ya llegó. Con tramas de medio segundo, el delay puede ser
   de ~1 s. INACEPTABLE en videollamada ⇒ los codificadores live limitan o
   desactivan los B-frames.

e) keyframe interval = 15 frames a 30 fps = 15/30 = 0,5 s
   ⇒ un keyframe cada 500 ms ⇒ el arranque de un directo tarda de 0 a 500 ms.
```

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-02, Problema 1 a) (30 pts en total) — Tamaño del datagrama

> **Fuente:** Examen `eirc1102` / `sirc1102` — 3 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1102.pdf **Este es el problema central de la materia para el examen actual.**
>
> Sea un flujo de voz que genera una señal de **64 kbps** y que se discretiza en muestras de tamaño fijo transmitidas sobre **UDP**. Se genera un datagrama con una muestra de voz cada **40 ms**. En el gráfico se muestra la generación y la llegada de los datagramas, por ejemplo el primer datagrama se genera a los 40 ms y llega a destino a los 260 ms. Los relojes de ambos extremos están sincronizados.
>
> **a)** Considerando un overhead de aplicación de **16 bytes** en la transmisión de una muestra de sonido, ¿qué tamaño tiene el **campo de datos** de cada datagrama UDP transmitido por el origen?

Solución oficial:

```
Se envía una muestra cada 40 ms.
Entonces por segundo se enviarán 1000/40 = 25 muestras por segundo
El flujo total es de 64 kbps = 64000 bps
Entonces 64000 bps / 25 muestras/s = 2560 bits/muestra = 320 bytes/muestra
Por lo tanto el campo de datos del paquete UDP contiene
    16 bytes + 320 bytes = 336 bytes.
```

Reproduzco la cuenta con TODAS las unidades a la vista, porque el paso donde más se tropieza es exactamente el primero:

Paso 1 — convertir el bitrate a bytes por segundo.

```
64 kbps  =  64 000 bits/s
1 byte   =  8 bits

64 000 bits/s ÷ 8 bits/byte = 8 000 bytes/s
```

> ⚠️ **LA TRAMPA, y es lo que el enunciado quiere que no se te cuelgue: 64 kbps son 8 000 bytes por segundo, NO 64 bytes por segundo.** Confundir kbps con KB/s divide el resultado por 8 y te da 42 bytes en vez de 336. La unidad "k" va en **bits**, porque la unidad base de un códec y del ancho de banda de la capa física son los **bits por segundo**.

Paso 2 — cuántas muestras por segundo.

```
1 muestra cada 40 ms,   1 s = 1000 ms
1000 ms / 40 ms = 25 muestras/s
```

Paso 3 — bytes por muestra.

```
8 000 bytes/s ÷ 25 muestras/s = 320 bytes/muestra
```

Control cruzado (la cuenta equivalente, y la que más me gusta):

```
40 ms de audio a 64 kbps:
  0,040 s × 64 000 bit/s = 2 560 bits
  2 560 bits ÷ 8 bits/byte = 320 bytes        ✓ el mismo resultado
```

Paso 4 — sumar el overhead.

```
campo de datos (payload) del datagrama UDP
    = muestra          320 bytes
    + overhead de app.  16 bytes
    ---------------------------
    =                   336 bytes
```

Interpretación — qué es físicamente ese número.

```
Datagrama completo en el cable:
    IP (20) + UDP (8) + payload (336) = 364 bytes por datagrama

Tasa en la línea:
    25 datagramas/s ×  40 bytes de cabeceras = 1 000 bytes/s
    25 datagramas/s × 336 bytes de datos    = 8 400 bytes/s
    total = 9 400 bytes/s = 75,2 kbps

⚠️ El flujo real ocupa 75,2 kbps cuando el enunciado dice "64 kbps". Ese 17 %
de sobrecarga es real: es el costo de poner voz en la red con un datagrama
cada 40 ms.

⚠️ SUPUESTIONES que hago explícitas (el enunciado pide justificarlas):
1. "64 kbps" es el bitrate del AUDIO (la señal comprimida), no del datagrama.
2. El "overhead de aplicación de 16 bytes" está DENTRO del campo de datos:
   por eso se suma, no se multiplica.
3. No hay fragmentación: 336 + 8 + 20 = 364 < MTU 1500 ✓
4. La cabecera IP es la mínima, 20 bytes, sin opciones.
```

> **Tope para 8-10 puntos:** la cuenta completa con unidades (`1000/40 = 25 muestras/s`, `64000/25 = 2560 bits/muestra`, `2560/8 = 320 bytes/muestra`), el **+16**, y el resultado **336 bytes**. Si olvidás la conversión a bytes, no hay puntos.

3.2 Examen 2011-02, Problema 1 b) — El mínimo `p`

Datos (de la solución oficial; la figura del PDF es una imagen y no está en el texto extraído, pero la tabla de la solución oficial la reconstruye completa):

```
Paquete i:      1     2     3     4     5     6     7     8     9
──────────────────────────────────────────────────────────────────
t_i [ms]:      40    80   120   160   200   240   280   320   360
r_i [ms]:     260   300   380   420   420   500   500   620   620
r_i − t_i:    220   220   260   260   220   260   220   300   260
                                                      ▲
                                          MÁXIMO ─────┘
```

Solución oficial:

> Se debe determinar el mayor de los tiempos entre generación `t_i` y recepción `r_i` que será el tiempo `p`. El tiempo deberá ser **mayor o igual que el máximo de éstos tiempos**. El mayor `r_i - t_i` se encuentra en el **paquete 8**, con un valor de **300 ms**.

Mi desarrollo, que es lo que hay que saber hacer:

Paso 1 — la condición de "reproducción continua".

La muestra `i` se reproduce en `t_i + p` y **debe haber llegado antes**:

```
r_i ≤ t_i + p   ⇔   p ≥ r_i − t_i     para TODO i
```

Paso 2 — el mínimo `p` es el MÁXIMO de todos los retardos: no el último, no el promedio.

```
p_min = max sobre i de (r_i − t_i)
```

> ⚠️ **Este es el punto que más se falla.** Muchos hacen `p = r_9 − t_9 = 260` (el último) o `p = r_1 − t_1 = 220` (el primero). **Ninguno de los dos sirve.** El valor de `p` tiene que ser suficiente para **todos** los paquetes. Si `p` fuera 220, el paquete 8 (que tarda 300) llegaría 80 ms tarde y habría un corte en el segundo 320.

Paso 3 — la tabla y el máximo.

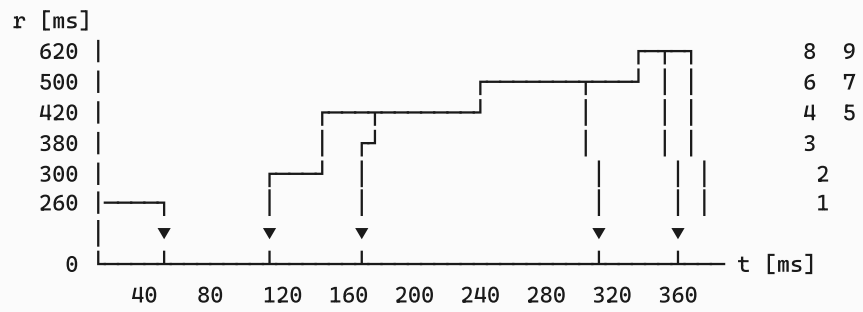
```
i:            1    2    3    4    5    6    7    8    9
r_i − t_i:  220  220  260  260  220  260  220 [300] 260

p_min = 300 ms
```

Paso 4 — verificación: el playout queda en `t_i + 300` y todo llega a tiempo.

```
i:              1    2    3    4    5    6    7    8    9
t_i:           40   80  120  160  200  240  280  320  360
r_i:          260  300  380  420  420  500  500  620  620
t_i + 300:    340  380  420  460  500  540  580  620  660
¿r_i ≤ t_i+300?  ✓    ✓    ✓    ✓    ✓    ✓    ✓  [✓]    ✓
holgura:       80   80   40   40   80   40   80    0   40
```

Interpretación. Con `p = 300`, la primera muestra se reproduce a los **340 ms** aunque llegó a los 260: el usuario espera **300 ms** entre que el emisor "habla" y el receptor lo oye. Ese es el precio de no tener cortes.

Reconstrucción de la gráfica del enunciado en texto:




```
GENERACIÓN (t) = 40, 80, 120, ... constante, un datagrama cada 40 ms.
LLEGADA (r)    = escalonada: 260 300 380 420 420 500 500 620 620
Los saltos verticales de "r" son paquetes que salieron juntos y llegaron
juntos (cola en algún router del camino).
```

⚠️ **Casos que la solución oficial NO cubre y que conviene que agregues vos (suman puntos):**

1. **Reordenamiento.** El enunciado dice "no ocurren pérdidas", pero nada impide que el 5 llegue antes que el 4. Con reordenamiento `t_i + p` sigue siendo correcto **porque el timestamp y el número de secuencia permiten insertarlos en orden** en la estructura auxiliar. Decí eso.
2. **Pérdida.** Si un paquete se pierde, ese instante tiene un hueco: 40 ms de silencio o repetición del anterior. El apartado b dice explícitamente que **no hay pérdidas**, pero el apartado c **sí** tiene que manejarlo (con el array `perdidos[]`).
3. **El crecimiento de `p` a lo largo de la llamada.** En la realidad `p` no es constante: si la red empeora, el máximo crece. El apartado c resuelve esto con el **promedio ponderado**, que es un `p` que **se adapta**. Decí que el `p` del apartado b es el `p` **inicial**.
4. **Relación con el apartado c.** El enunciado define `p_0 = 400 ms`, o sea que el **arranque** de la latencia suavizada es 400, no 300. No es contradicción: son dos cosas distintas. `p` (apartado b) es el **delay fijo**; `p_i` (apartado c) es la **estimación suavizada**, que además sirve como criterio de cancelación. Esa distinción es la prueba de que entendiste los dos apartados.

> **Tope para 8-10 puntos:** la desigualdad `r_i ≤ t_i + p ⇔ p ≥ r_i − t_i`, la tabla de los 9 retardos, `max = 300 ms` en el paquete 8, y la **justificación de por qué es el máximo y no el último**.

3.3 Examen 2011-02, Problema 1 c) — La implementación (el corazón del problema)

Solución oficial (los dos procedimientos, textualmente):

```
diExcedidos = 0;
di = 400;

Procedure get()
  while (true) {
    if ( getMuestra(r,t,nroSec,muestra) == 0 ) {
      di = 0.9 * di + 0.1 * ( r - t );
      if ( t + di > now() ) {
        // inserto solamente si se reproduce en tiempo futuro
        insertMuestra(t+di, nroSec, muestra);
      }
      // determina cuantos di excedidos continuos hubo
      if ( di > 400 ) { diExcedidos++; } else { diExcedidos = 0; }
    }
  }

Procedure reproduce()
  nroRep = -1;
  perdidos[0..N-1] = false;   // para determinar perdidas en ultimos N muestras
  while (true) {
    if ( getTimeNext() <= now() ) {
      auxNroSec = getNroSecNext();
      reproduceMuestra( getMuestraNext() );
      if ( ((nroRep + 1)%N) != (auxNroSec%N) ) {
        for i = (nroRep+1) to (auxNroSec-1)
          perdidos[i%N] = true;
      }
      perdidos[(auxNroSec%N)] = false;
      nroRep = auxNroSec;
      cantPerdidos = 0;
      for i = 0 to (N-1)
        if (perdidos[i]) cantPerdidos++;
      if ( ((cantPerdidos/N) > 0.1) or (diExcedidos > N) ) {
        print "Valores permitidos excedidos";
        kill(get); exit();
      }
    }
    sleep(5ms);
  }
```

Análisis de la solución oficial: lo que está bien

✅ **La decisión de diseño clave, y es la que la cátedra premia:** `diExcedidos` es un **contador de excesos CONSECUTIVOS**, que se resetea a 0 cuando `di ≤ 400`. Eso implementa exactamente el "**durante N períodos de 40 ms consecutivos**" del enunciado. Si no lo resetease, bastaría que `p_i` tocara 400 una vez cada 100 paquetes para cancelar, y eso no es lo pedido.

✅ **La guarda `if (t + di > now())`.** Solo inserta si la muestra **todavía se puede reproducir en el futuro**. Sin esa guarda, el buffer se llenaría con muestras que ya no sirven, y `getTimeNext()` devolvería un tiempo pasado. **Esta línea parece un detalle y es el corazón** del comportamiento con pérdida.

✅ **La detección de pérdidas con el array circular `perdidos[0..N-1]`:** es una **ventana deslizante de N posiciones** que guarda si cada uno de los últimos N números de secuencia estuvo presente. Es la forma correcta de calcular "pérdida de los últimos N paquetes" sin un contador acumulador, y es O(1) por paquete.

✅ **`nroRep = -1`:** como los números de secuencia empiezan en 0, el primer paquete es el 0, y `nroRep+1 = 0`. El bucle `for i = 0 to -1` no itera. Detalle fino bien puesto.

Análisis: lo que le falta o está discutible

⚠️ **1. `diExcedidos > N` en vez de `>= N`.** El enunciado dice "durante **N** períodos de 40 ms consecutivos". Si excedió exactamente N períodos seguidos, hay que cancelar; con `> N` se cancela recién en el N+1. No es grave, pero es una discrepancia. **Mi versión usa `>= N`** y lo digo explícitamente.

⚠️ **2. `diExcedidos` solo se actualiza cuando llega un paquete.** Si hay pérdidas, el contador **no se resetea** aunque deberían resetearse por período no recibido; y si la llamada se corta (nada llega), el contador queda congelado y nunca se cancela. El mecanismo de cancelación por latencia **degrada silenciosamente** cuando hay pérdida. La solución oficial no lo cubre.

⚠️ **3. La verificación del 10 % se hace solo cuando hay una muestra que reproducir.** Si faltan 5 paquetes seguidos, nadie verifica nada durante 200 ms. En la realidad el reproductor tiene un **temporizador por período de 40 ms**.

⚠️ **4. `perdidos` se corrompe si el hueco es grande.** El bucle `for i = (nroRep+1) to (auxNroSec-1)` marca hasta `N-1` posiciones; si el hueco es **mayor que N** el bucle da más de una vuelta y **pisa posiciones ya marcadas correctamente con `false`**, reportando pérdida donde no la hay.

⚠️ **5. `sleep(5 ms)` con `getTimeNext() <= now()`** es un *polling*; en C real sería un `select()` / `epoll()` con timeout `(getTimeNext() - now())`. Mencioná que el polling es lo que hace que la cátedra acepte el pseudocódigo.

Mi solución (corregida y con los huecos rellenados)

```
/* ==== VARIABLES GLOBALES COMPARTIDAS POR LOS DOS PROCEDIMIENTOS ====
   (el enunciado dice: "los procedimientos utilizan variables comunes") ==== */
#define LIMITE_LATENCIA_MS  400     /* p_i no debe pasar de 400 ms */
#define PERDIDA_MAXIMA      0.10    /* 10 % de pérdida tolerada    */

int  p_i        = 400;    /* latencia suavizada; p_0 = 400 ms (dato enunciado) */
int  pExcedidos = 0;      /* períodos CONSECUTIVOS con p_i > 400 ms */
bool perdidos[0..N-1];    /* ventana circular de N posiciones */
int  nroRep = -1;

/* ====================== PROCEDIMIENTO 1: ALMACENAMIENTO ======================
   Recibe, suaviza la latencia, calcula el instante de reproducción y
   ordena los pedidos en la estructura auxiliar.                          */
Procedure get()                      /* nunca retorna */
{
  long r, t;  int nroSec;  byte muestra;  long u;

  while (true) {

    /* --- 1. ¿Hay un datagrama en el buffer de recepción? ---------- */
    /* getMuestra NO bloquea: 0 si hay uno, -1 si no. De ahí el polling;
       en C real sería un select() con timeout, o un hilo en recvfrom(). */
    if ( getMuestra(r, t, nroSec, muestra) != 0 ) continue;

    /* --- 2. Latencia suavizada: promedio exponencial --------------- */
    /* v_i = r - t  es el tiempo que tardó ESTE paquete en la red.
       p_i = 0,9·p_(i-1) + 0,1·v_i es un filtro paso bajo de primer orden:
       no reacciona de golpe a un outlier, sigue el promedio de los
       últimos ~10 paquetes (1/0,1 = 10 paquetes ≈ 10 períodos).       */
    p_i = 0.9 * p_i + 0.1 * (r - t);

    /* --- 3. Cancelación por latencia: N períodos CONSECUTIVOS ------ */
    if (p_i > LIMITE_LATENCIA_MS) { pExcedidos++; }
    else { pExcedidos = 0;  }   /* ⭐ el reset es lo que hace que el enunciado
                                   diga "consecutivos"                        */
    if (pExcedidos >= N) {
      signal("cancelar");        /* la latencia es insostenible: preferimos
                                    cortar antes que seguir con 800 ms */
      kill(get); exit();
    }

    /* --- 4. Instante de reproducción: u = t_i + p_i ---------------- */
    u = t + p_i;   /* RELATIVO al instante en que fue GENERADA la muestra, no
                      al de recepción: por eso el paquete 8 del apartado b, que
                      tarda 300 ms, igual suena "en su lugar" de la línea */

    /* --- 5. Solo se inserta si el instante es FUTURO -------------- */
    if (u > now()) {
      insertMuestra(u, nroSec, muestra);   /* inserción ORDENADA por u: esto
                                              es lo que REORDENA los paquetes */
    }
    /* si u <= now() el paquete llegó tarde: se descarta (40 ms de voz) */

    /* --- 6. Contabilidad de pérdidas (ventana de N) --------------- */
    perdidos[nroSec % N] = false;   /* "este número de secuencia LLEGÓ" */
  }
}

/* ====================== PROCEDIMIENTO 2: REPRODUCCIÓN ====================
   Saca de la estructura EN ORDEN, reproduce a ritmo constante, mide la
   pérdida y cancela.                                                     */
```

```
Procedure reproduce()
{
  int auxNroSec, cantPerdidos, i;

  /* ⚠️ La oficial los inicializa en false. Eso está mal para este caso:
     al arrancar la llamada el buffer está VACÍO, así que NINGÚN número de
     secuencia llegó todavía. Si los pongo en false cuento 0 pérdidas y no
     cancelo, cuando en realidad el 100 % se perdió. */
  perdidos[0..N-1] = true;

  while (true) {

    /* --- 1. ¿Hay algo para reproducir AHORA? ---------------------- */
    if ( getTimeNext() <= now() ) {

      auxNroSec = getNroSecNext();
      reproduceMuestra( getMuestraNext() );

      /* --- 2. Marcar los números de secuencia AUSENTES ------------- */
      /* Como los paquetes llegan en orden dentro de la estructura, todo
         lo intermedio entre nroRep y auxNroSec NO LLEGÓ de verdad.     */
      if ( ((nroRep + 1) % N) != (auxNroSec % N) ) {
        for (i = (nroRep + 1) ; i <= (auxNroSec - 1) ; i++)
          perdidos[i % N] = true;
      }
      perdidos[ auxNroSec % N ] = false;   /* el que reproduje sí llegó */
      nroRep = auxNroSec;

      /* --- 3. Tasa de pérdida en la ventana de N -------------------- */
      cantPerdidos = 0;
      for (i = 0 ; i < N ; i++) if (perdidos[i]) cantPerdidos++;

      /* --- 4. Cancelación por pérdida ------------------------------- */
      if ( (cantPerdidos / (float)N) > PERDIDA_MAXIMA ) {
        signal("cancelar"); kill(get); exit();
      }
    }

    /* --- 5. Silencio implícito ---------------------------------- */
    /* Si getTimeNext() > now(), no hay nada que reproducir: el período de
       40 ms de esa muestra ya pasó y el paquete no llegó. Eso es el
       "40 ms de silencio" que el oyente percibe.                     */

    sleep(5ms);                 /* granularidad del polling */
  }
}
```

Tabla de verificación — corréla a mano antes del examen

Con los 9 datagramas del apartado b y `p_0 = 400`:

```
 i | v_i=r_i-t_i | p_i = 0,9·p_(i-1) + 0,1·v_i | ¿p_i>400? | pExcedidos
---|-------------|-----------------------------|-----------|-----------
   |  (p_0=400)  |  p_0 = 400                  |  NO (=400)|     0
 1 |     220     |  360,0  +  22,0 = 382,00    |  NO       |     0
 2 |     220     |  343,8  +  22,0 = 365,80    |  NO       |     0
 3 |     260     |  329,22 +  26,0 = 355,22    |  NO       |     0
 4 |     260     |  319,70 +  26,0 = 345,70    |  NO       |     0
 5 |     220     |  311,13 +  22,0 = 333,13    |  NO       |     0
 6 |     260     |  299,82 +  26,0 = 325,82    |  NO       |     0
 7 |     220     |  293,24 +  22,0 = 315,24    |  NO       |     0
 8 |     300     |  283,72 +  30,0 = 313,72    |  NO       |     0
 9 |     260     |  282,35 +  26,0 = 308,35    |  NO       |     0
```

Interpretación — y acá está la moraleja del apartado: `p_i` converge a un valor alrededor de 300-310 ms, que es el **promedio ponderado** de los retardos, ni el máximo (300) ni el último (260). Y **nunca supera los 400 ms**, así que en esta llamada **no se cancela por latencia**.

¿Por qué? Porque `p_0` arranca en 400 (el enunciado) y todos los `v_i` están entre 220 y 300, es decir siempre por debajo de 400. Un filtro paso bajo no puede subir por encima del más alto de sus entradas si arranca por debajo. Si la red se degradara a `v_i = 600` durante varios paquetes seguidos, `p_i` subiría: `0,9^10 ≈ 0,35`, o sea `400·0,35 + 600·0,65 = 530 > 400` ⇒ se cancela. Hacen falta varios períodos consecutivos para que el filtro "vea" el problema, y **eso es exactamente el propósito del suavizado: no cortar por un outlier**.

Verificación del apartado de pérdidas, con un caso inventado:

```
Con N = 5, en el último rato llegaron los seq 20, 21, 22 y 24 (falta el 23).

Al reproducir el 24:
    ((22+1)%5) = 3        (24%5) = 4
    3 != 4  ⇒  hay hueco
    for i = 23 to 23:  perdidos[23%5=3] = true    ← marca el 23
    perdidos[24%5=4] = false                        ← el 24 llegó

Recuento: posiciones 0,1,3 en true; 2 y 4 en false.
cantPerdidos = 3
3/5 = 0,60 > 0,10  ⇒  CANCELAR.   ✓ correcto: 1 de 5 = 20 % perdido.
```

> **Tope para 12-15 puntos (la mitad del problema):** los dos procedimientos completos, con (a) el suavizado `p_i = 0,9·p_{i-1} + 0,1·v_i`, (b) la inserción **ordenada** por `t_i + p_i` con la guarda de "solo si es futuro", (c) el **reset** del contador de excesos, (d) la ventana de `N` para medir la pérdida, y (e) la cancelación con `kill` + `exit`. Los comentarios explicando **por qué** existe cada rama valen tanto como el código.

3.4 Examen 2010-07, Problema 1 (30 puntos) — MPEG-2 I/P/B: protocolo semifiable

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010.

Enunciado (texto del examen)

> La compresión de video MPEG-2 utiliza tres tipos de frames: **I-frames**, **P-frames**, and **B-frames**. Un I-frame contiene una imagen completa. Para transmitir menos información, los P-frames y B-frames solo contienen información de la parte de la imagen que cambió desde el último I-frame y parte de la que vendrá en el próximo I-frame. Por la misma causa, si un P-frame o un B-frame se pierde causa menos problemas de visualización que si se pierde un I-frame.
>
> Se quiere implementar un protocolo de transferencia de datos **semi-confiable "stop-and-wait"** para transmitir video MPEG-2 entre dos computadoras de forma que: **(1)** los I-frame **no se pierdan y se eliminen sus duplicados** y **(2)** los P-frames y B-frames **pueden perderse o duplicarse**.

Primitivas: `rdt_send(frame, tipo)` · `udt_send(pkt)` · `rdt_rcv(pkt)` · `deliver_frame(frame)` · `make_pkt(tipo, frame, checksum)` · `extract_frame(pkt, frame)` · `has_type(pkt, tipo)` · `corrupt(pkt)` · `isACK(pkt, tipo)` · `start_timer` / `stop_timer` / `timeout`

Se considera que (i) cada frame entra completamente en un segmento; (ii) siempre se envía un I-frame y luego 0 o más P-frames y B-frames en cualquier orden; (iii) **los paquetes llegan al receptor en el orden que fueron enviados**.

Se pide: dibujar las máquinas de estados del emisor y del receptor y, si existen, el pseudocódigo de los procesos auxiliares.

La clave de diseño, que hay que decir ANTES de dibujar nada

El enunciado pide **exactamente dos cosas distintas** para dos tipos de frame:

```
Tipo      ¿Se retransmite?  ¿Espera ACK?  ¿Timer?  ¿Duplicados?
────────  ────────────────  ────────────  ───────  ──────────────
I-frame   SÍ (stop&wait)    SÍ            SÍ       SE ELIMINAN
P-frame   NO                 NO            NO       se permiten
B-frame   NO                 NO            NO       se permiten
```

Y hay un segundo detalle, menos obvio: el protocolo necesita distinguir entre **las distintas instancias** de un I-frame, porque el receptor tiene que saber si el I-frame que le llega es **el que está esperando** (lo entrega y avanza) o **un duplicado** (no lo entrega, reenvía el ACK). Por eso el `make_pkt` de la solución oficial lleva un campo extra `sec` (0 o 1) que **alterna** en cada I-frame: es la versión mínima de un número de secuencia, y la hipótesis (iii) — paquetes en orden — es lo que garantiza que "el que no es el esperado es un duplicado".

Las máquinas de estados (en tabla, que es lo mismo que el dibujo)

EMISOR

Estado	Evento / Guarda	Acción	Estado siguiente
`E_ESPERA`	`rdt_send(frame, tipo)` con `tipo == I`	`make_pkt(I, sec, frame, cksum)`; `udt_send`; `start_timer`; `sec ← 1 − sec`	`E_ESPERA_ACK`
`E_ESPERA`	`rdt_send` con `tipo == P` o `B`	`make_pkt(tipo, 0, frame, cksum)`; `udt_send` **(sin timer)**	`E_ESPERA`
`E_ESPERA_ACK`	`rdt_rcv(p)` con `!corrupt(p) && isACK(p, secEnviado)`	`stop_timer`	`E_ESPERA`
`E_ESPERA_ACK`	`timeout`	`udt_send(sndpkt)`; `start_timer`	`E_ESPERA_ACK`
`E_ESPERA_ACK`	`rdt_rcv(p)` corrupto o `!isACK`	nada (espera al timer)	`E_ESPERA_ACK`

RECEPTOR

Estado	Evento / Guarda	Acción	Estado siguiente
`R_ESPERA`	`rdt_rcv(p)` con `isACK(p)`	descartar	`R_ESPERA`
`R_ESPERA`	`rdt_rcv(p)` con `corrupt(p)`	descartar (el emisor reintenta)	`R_ESPERA`
`R_ESPERA`	I-frame con `sec == secEsperado`	`extract_frame`; **`deliver_frame`**; `udt_send(ACK(sec))`; `secEsperado ← 1 − secEsperado`	`R_ESPERA`
`R_ESPERA`	I-frame con `sec != secEsperado` (**duplicado**)	**`udt_send` del último ACK** — NO entregar	`R_ESPERA`
`R_ESPERA`	P-frame o B-frame	`extract_frame`; **`deliver_frame`** (puede duplicarse: permitido)	`R_ESPERA`

```
La única diferencia real entre los dos lados es qué hace el emisor con
el ACK y qué hace el receptor con el I-frame duplicado:
    EMISOR   usa  start_timer / stop_timer  → stop-and-wait  (CONFIABLE)
    RECEPTOR usa  sec alternando            → deduplicación   (CONFIABLE)
    P y B    no usan NADA                   → pueden fallar   (SEMI-FIABLE)
```

Mi pseudocódigo

```
/* ======================== EMISOR ======================== */
enum { E_ESPERA, E_ESPERA_ACK } estado;
int     sec = 0;        /* instancia del I-frame: alterna 0/1 */
Packet  sndpkt;         /* último datagrama enviado, para reenviarlo */
int     secEnviado;

void Emisor() {
  while (true) {
    switch (estado) {

    case E_ESPERA:
      rdt_send(frame, tipo);
      if (tipo == I_FRAME) {
        /* ---- I-frame: FIABLE. Un número de instancia + timer. ---- */
        sndpkt    = make_pkt(I_FRAME, sec, frame, checksum);
        secEnviado = sec;
        udt_send(sndpkt);
        start_timer;
        estado  = E_ESPERA_ACK;
        sec     = 1 - sec;        /* el próximo I-frame lleva el otro sec */
      } else {
        /* ---- P o B: SEMI-FIABLE. Se manda UNA vez y se olvida.
                Sin timer, sin ACK, sin retransmisión.               */
        udt_send( make_pkt(tipo, 0, frame, checksum) );
      }
      break;

    case E_ESPERA_ACK:
      rdt_rcv(rcvpkt);
      if (!corrupt(rcvpkt) && isACK(rcvpkt, secEnviado)) {
        stop_timer;
        estado = E_ESPERA;
      } else if (timeout) {
        /* El I-frame se perdió (o se perdió su ACK): reenvío el MISMO
           datagrama. Como el `sec` no cambió, el receptor reconoce el
           duplicado y sólo reenvía el ACK.                          */
        udt_send(sndpkt);
        start_timer;
      }
      /* si llega corrupto o un ACK del otro sec: no hacer nada; el
         timer va a disparar                                          */
      break;
    }
  }
}

/* ======================== RECEPTOR ======================== */
void Receptor() {
  int    secEsperado = 0;   /* el primer I-frame es el sec = 0 */
  Packet ultimoAck;         /* el último ACK que generé, para reenviarlo */

  while (true) {
    rdt_rcv(rcvpkt);

    if (isACK(rcvpkt)) continue;     /* los ACKs no se procesan acá */
    if (corrupt(rcvpkt)) continue;   /* se descarta: el emisor reintenta */

    if (has_type(rcvpkt, I_FRAME)) {
      if (has_sec(rcvpkt, secEsperado)) {
        /* ---- Es EL que estaba esperando: lo entrego. ---- */
        extract_frame(rcvpkt, frame);
```

```
        deliver_frame(frame);          /* requisito 1: NO se pierde */
        ultimoAck = make_pkt(ACK, I_FRAME, secEsperado, checksum);
        udt_send(ultimoAck);
        secEsperado = 1 - secEsperado; /* alterno: el próximo es otro */
      } else {
        /* ---- Duplicado del I-frame ANTERIOR. NO lo entrego
                (requisito: eliminar duplicados) pero REENVÍO el ACK,
                porque el emisor quizá espera la confirmación.     */
        udt_send(ultimoAck);
      }
    } else {
      /* ---- P o B: los entrego tal cual llegan. El enunciado PERMITE
              que se pierdan y que se dupliquen, así que acá no hay nada
              que hacer: no hay timer, ni ACK, ni control.         */
      extract_frame(rcvpkt, frame);
      deliver_frame(frame);
    }
  }
}
```

Análisis de la solución oficial

La solución oficial del PDF es un **dibujo de FSM en arte ASCII** (se pierde en la extracción de texto), pero su contenido se puede reconstruir y **coincide con lo anterior**: los mismos estados (`Espera frame desde la aplicación` / `Espera ACK I-frame 0` / `Espera ACK I-frame 1`), el campo `sec` alternando 0/1, el `if (paso==1) udt_send(sndpkt)` para reenviar el ACK, y el `timeout → udt_send(sndpkt); start_timer` en el emisor. **Es la misma solución**, lo cual confirma el diseño.

⚠️ **Lo que está raro o mal en la oficial:**

1. **Los números de sección de los I-frames son `0` y `1`, alternando, no incrementando.** Funciona **solo** porque el enunciado garantiza que "siempre se envía un I-frame y luego 0 o más P/B", o sea **nunca hay dos I-frames seguidos**: un bit alcanza. Si pudiera haber dos I-frames seguidos, un bit no distinguiría el siguiente del duplicado y haría falta un contador. Decí que es un caso particular privilegiado.
2. **`make_pkt(tipo_de_frame, frame, checksum)` no tiene campo `sec`** en la lista de primitivas del enunciado, pero la solución oficial lo usa (`make_pkt(I-frame, 0, datos, checksum)`). Hay que **declarar el campo explícitamente** o el código no compila. La cátedra lo resuelve sin decirlo; en tu examen, **agregá el campo y comentá por qué**.
3. **La oficial no explica por qué descarta un ACK corrupto.** Lo hace (la transición va a `do_nothing`) pero sin justificar, y esa es la mitad de los puntos del análisis: se descarta porque **el timer va a disparar** y la retransmisión resuelve.
4. **El emisor oficial reenvía `sndpkt`, el mismo datagrama**, sin reconstruirlo. Acá está bien (el frame ya está capturado), pero vale la pena notar que en un protocolo real se reconstruiría.

> **Tope para los 30 puntos:** los dos diagramas de estados, el `sec` alternando para distinguir I-frames, el `start_timer` / `stop_timer` **solo** en los I-frames, el `deliver_frame` de P/B sin control (con la justificación de que el enunciado lo permite), el reenvío del ACK ante duplicado, y la razón de que perder un I es mucho peor que perder un B.

3.5 Examen 2008-07, Problema 1 (30 puntos) — Audio redundante con RTP

> **Fuente:** Examen `eirc0807` / `sirc0807` — 1 de agosto de 2008.

Enunciado (los apartados que interesan)

> Una aplicación de transmisión de audio por Internet funciona emitiendo datos de forma **redundante desde tres fuentes** (emisores). Los datos se transmiten utilizando **RTP sobre UDP**.
>
> Todos los paquetes que se envían desde cualquiera de los tres emisores son generados a una tasa constante, son de la misma duración, y **están sincronizados**. Puede considerarse que es exactamente **el mismo flujo de audio replicado desde tres fuentes diferentes**.
>
> Se desea implementar un **buffer de recepción** que haga uso de la característica redundante del streaming. Se cuenta con una estructura de buffer que funciona como una **cola circular** donde se pueden almacenar hasta **N paquetes UDP**.
>
> Resulta de interés llevar un registro de **cuántos paquetes han llegado demasiado rápido**, **distinguiendo la cantidad para cada una de las fuentes**.
>
> **a)** Indicar qué campos de RTP puede utilizar: **i.** para decidir en qué posiciones almacenar los paquetes de forma que queden en el orden que el proceso reproductor los tomará; **ii.** para distinguir de cuál de los tres emisores se está enviando el paquete.
>
> **b)** Implemente en un lenguaje de alto nivel el **proceso escucha**.

Parte a)

Solución oficial:

> **i.** Para saber en qué posición almacenar un paquete que llegue, se puede utilizar el **número de secuencia de RTP**, alcanzaría con hacer **número de secuencia módulo N**. Como los paquetes son de una duración fija también puede utilizarse el **timestamp**, el índice del paquete con timestamp `T` sería `(T/duración_de_paquete) módulo N`.
>
> **ii.** Para distinguir de cuál de los tres emisores proviene el paquete se puede utilizar el campo **SSRC** (*synchronization source identifier*) que será único para cada una de las réplicas del stream.

Mi desarrollo del apartado i, que es lo importante. *¿Por qué* `seq mod N` *funciona?* Porque las tres fuentes emiten **exactamente el mismo flujo sincronizado**, así que para el mismo instante de reproducción las tres mandan el **mismo número de secuencia**:

```
INSTANTE  de A  de B  de C   seq%N (con N = 8)
─────────  ────  ────  ────   ──────────────────
t=0         100   100   100   4   (100 % 8 = 4)
t=40        101   101   101   5
t=80        102   102   102   6
t=120       103   103   103   7
t=160       104   104   104   0   (104 % 8 = 0)

⇒ Las tres REPLICAS del mismo instante caen en la MISMA posición. La
  primera que llega ocupa el lugar; las otras dos encuentran la posición
  ocupada por el MISMO número de secuencia y son DUPLICADOS.
  Eso es la redundancia: si A se perdió y no llega nunca, la ranura 6 la
  ocupa la copia de B o de C y no pasa nada.
```

La alternativa con el timestamp (`(T / duracion) % N`) además es más **robusta**: el seq de 16 bits **da la vuelta cada 65 536 paquetes** (con 25 paquetes/s son ≈ 44 minutos). Pasado ese tiempo, `seq mod N` sigue funcionando **si** `N` **es una potencia de 2**, pero si `N` es primo la posición "salta".

Parte b) — El proceso escucha

Solución oficial:

```
map<int, int> muy_rapido;        // mapping de <id de fuente, cantidad de paquetes>
Socket s = crear_socket_udp();
while (true) {
    DatagramaUDP p = receive(s);
    PaqueteRTP r = p.getPayload();
    int t = getPlayingTimestamp();
    if (t < r.getTimestamp()) {
        // todavía estamos a tiempo para reproducirlo
        int pos = r.getNumSeq() % N;
        lockBuffer();
        PaqueteRTP q = leerPaqueteBuffer(pos);
        if (q == null) {
            escribirPaqueteBuffer(pos, r);        // no estaba, lo agregamos
        } else if (q.getNumSeq() > r.getNumSeq()) {
            muy_rapido[r.getSsrc()]++;            // llegó demasiado temprano
        }
        releaseBuffer();
    }
}
```

Análisis punto por punto. Acá hay un error de fondo, y encontrarlo es lo que suma puntos.

✅ **Lo que está bien:**

1. `if (t < r.getTimestamp())` descarta los paquetes que **ya llegaron tarde**: no hay nada que hacer con ellos.
2. `lockBuffer()` / `releaseBuffer()` alrededor de **toda** la sección crítica, y en el orden correcto.
3. `muy_rapido` es un `map` **indexado por SSRC**, así que distingue la fuente como pide el enunciado.
4. `if (q == null) escribir(...)` : la primera réplica del instante se guarda.

🔴 **EL ERROR: la comparación está INVERTIDA respecto del enunciado.**

Releé el enunciado: "*un paquete llegue demasiado rápido y su lugar en el buffer esté ocupado por un paquete anterior cuyo instante de reproducción aún no ha llegado*". Si `r` es el que "llegó demasiado rápido", el

ocupante `q` es el paquete **anterior**, o sea `q.getNumSeq() < r.getNumSeq()`. Sin embargo la oficial escribe `q.getNumSeq() > r.getNumSeq()`.

```
q.seq      relación   qué es r                 qué DEBERÍA pasar
─────────  ─────────  ──────────────────────── ──────────────────────────
q = r      IGUAL      duplicado                 descartar (no contar)
q > r      q adelante r es VIEJO                descartar: ya no sirve
                                               (¡y NO es "llegó temprano"!)
q < r      q atrás    r es N packets adelante   ✓ ÉSTE es "llegó demasiado
                                               rápido": contar en muy_rapido
```

Qué hace la oficial con cada caso:

```
q = r  → no hace nada        ✓ correcto (no pisa nada)
q > r  → lo cuenta como "llegó temprano"   ✗ cuenta mal
q < r  → no hace nada        ⚠️ no lo cuenta, que es el caso REAL
```

O sea: **la oficial nunca pisa un paquete bueno** (el comportamiento funcional es el correcto por accidente), **pero el contador `muy_rapido` está al revés**: cuenta duplicados viejos y **nunca cuenta** los paquetes que realmente llegaron demasiado rápido. Y como el enunciado pide explícitamente "llevar un registro de cuántos paquetes han llegado en esta situación", **el contador es el objeto del ejercicio y está mal**.

> **Nota:** la intención del ejercicio es que descubras que hace falta un contador **por fuente** y que se llega con el campo SSRC. La inversión de la desigualdad no la descalifica, pero si en tu examen el enunciado es explícito ("un paquete **anterior**"), la comparación correcta es `<`. Respondé con `<` y **explicá en una línea por qué**: eso es lo que suma.

Mi solución (corregida)

```
/* ================= PROCESO ESCUCHA =================
   Corre en paralelo con el proceso reproductor (que el enunciado dice que
   ya existe y que hace lock/release del buffer por su cuenta).
   ================================================== */
map<int, int> muy_rapido     = new map();   // <SSRC, paquetes demasiado rápidos>
map<int, int> duplicados     = new map();   // <SSRC, réplicas descartadas>
map<int, int> arriving_tarde = new map();   // <SSRC, llegaron después de su turno>

Socket s = crear_socket_udp();

while (true) {

    DatagramaUDP p = receive(s);
    int ssrc = p.getSsrc();

    /* ---- 1. Validación mínima del datagrama ---------------------- */
    /* No todos los datagramas de ese puerto son RTP: hay que chequear
       versión, payload type y longitud mínima. Si no lo es, se tira. */
    if (!esRTPValido(p)) continue;

    PaqueteRTP r = p.getPayload();

    /* ---- 2. ¿Todavía estamos a tiempo? --------------------------- */
    /* t es el instante de reproducción actual. Si el paquete ya pasó su
       turno, no hay nada que hacer: se va a perder igual.            */
    int t = getPlayingTimestamp();
    if (t >= r.getTimestamp()) {
        arriving_tarde.put(ssrc, arriving_tarde.get(ssrc) + 1);
        continue;
    }

    /* ---- 3. Posición en el buffer circular ----------------------- */
    /* Las tres fuentes emiten el MISMO flujo sincronizado, así que las
       tres réplicas del mismo instante tienen el MISMO seq y caen en la
       MISMA posición.                                                */
    int pos = r.getNumSeq() % N;

    lockBuffer();

        PaqueteRTP q = leerPaqueteBuffer(pos);

        if (q == null) {
            /* ---- Caso A: ranura LIBRE. Guardamos. ------------------ */
            escribirPaqueteBuffer(pos, r);

        } else if (q.getNumSeq() == r.getNumSeq()) {
            /* ---- Caso B: DUPLICADO. Es otra de las dos réplicas del
                        mismo instante. La ranura ya tiene el instante
                        correcto: no la pisamos. Se cuenta aparte.   */
            duplicados.put(ssrc, duplicados.get(ssrc) + 1);

        } else if (q.getNumSeq() > r.getNumSeq()) {
            /* ---- Caso C: r es MÁS VIEJO que lo que ya está en la
                        ranura: una réplica atrasada de un instante ya
                        servido (o casi). No tiene sentido guardarlo,
                        la ranura tiene algo más nuevo que sí va a sonar. */
            duplicados.put(ssrc, duplicados.get(ssrc) + 1);

        } else {   /* q.getNumSeq() < r.getNumSeq() */
            /* ---- Caso D: ⭐ LLEGÓ DEMASIADO RÁPIDO ⭐
                        La ranura la ocupa un paquete ANTERIOR que
```

```
                        todavía NO se reprodujo, y r es N packets más
                        adelante (misma ranura). Si lo escribiéramos,
                        pisaríamos un paquete que falta por reproducir
                        ⇒ sonido entrecortado. Contamos y NO escribimos. */
            muy_rapido.put(ssrc, muy_rapido.get(ssrc) + 1);
        }

    releaseBuffer();
}
```

Tabla de verificación — probá el pseudocódigo a mano

Con `N = 4`, las tres fuentes A (`SSRC=111`), B (`SSRC=222`), C (`SSRC=333`), y este orden de llegada:

```
 orden fuente  seq  seq%4  ¿qué pasa?
 ─────  ──────  ───  ─────  ──────────────────────────────────────────────
  1     A       100    0    ranura 0 vacía → escribir.  BUFFER[0] = A/100
  2     A       101    1    ranura 1 vacía → escribir.  BUFFER[1] = A/101
  3     B       100    0    BUFFER[0] tiene seq 100 → IGUAL → duplicado[B]++
  4     C       100    0    BUFFER[0] tiene seq 100 → IGUAL → duplicado[C]++
  5     B       102    2    ranura 2 vacía → escribir.  BUFFER[2] = B/102
                              (A/102 no llegó: se recupera con B ✓)
  6     A       104    0    BUFFER[0] tiene seq 100; 100 < 104
                             ⇒ CASO D: LLEGÓ DEMASIADO RÁPIDO
                             muy_rapido[A]++, NO se escribe  ✓
  7     C       103    3    ranura 3 vacía → escribir.  BUFFER[3] = C/103

Resultado:  muy_rapido = {111: 1}   duplicados = {222: 1, 333: 1}
Buffer: [0]=A/100  [1]=A/101  [2]=B/102  [3]=C/103   ← el 103 se recuperó
                                                        de la fuente C
```

Interpretación — y acá está la idea del ejercicio: el mecanismo de redundancia funciona **sin ninguna lógica especial**. Cada posición del buffer tiene el primer paquete que llega para ese instante, sea de la fuente que sea. Y el tamaño `N` es el **compromiso**: más `N` = más tolerancia a fuentes perdidas; menos `N` = más riesgo de "llegó demasiado rápido".

> **Tope para los 30 puntos:** los dos campos de RTP bien identificados (seq para la posición, **SSRC** para la fuente), el pseudocódigo con la **región crítica** bien puesta, y el manejo de los **cuatro casos** (ranura libre, duplicado, atrasado, demasiado rápido). Y si encontrás la inversión de la comparación, **decilo**: demuestra que leíste el enunciado, no solo la solución.

3.6 Examen 2008-12, Pregunta 5 (8 puntos) — ¿TCP para tiempo real?

> **Enunciado:** Se conocen diferentes protocolos (por ej. RTP) para el transporte de streaming multimedia. ¿Bajo qué hipótesis sobre el comportamiento de la red es posible implantar servicios de telefonía IP basados en transporte **TCP** en lugar de los protocolos específicos de streaming? Justifique su respuesta **analizando al menos la pérdida de datos, retransmisión y retardo**.

Solución oficial:

> La utilización de TCP/IP para ofrecer servicios multimedia de tiempo real va en contra de los requerimientos clásicos del transporte isócrono. TCP ofrece entrega ordenada y retransmisión en caso de

> pérdidas, cuando la transmisión multimedia soporta mejor la pérdida de ráfagas de tráfico que el delay introducido por la retransmisión. Lo que permite la transmisión de tráfico de tiempo real sobre TCP es la combinación de un **gran ancho de banda disponible** y **baja latencia** en los servicios de comunicación actuales (permitiendo retransmisiones con un delay bajo, tolerable para el audio). El otro factor que incide positivamente son las **bajas tasas de pérdidas de servicios digitales domésticos** (p.ej. ADSL), que reduce a niveles aceptables la cantidad de retransmisiones por unidad de tiempo.

La resolución, parte por parte.

a) La respuesta a "TCP no sirve" (las 4 razones). Desarrolladas en §1.2. Acá conviene presentarlas en el **orden que pide el enunciado**: pérdida, retransmisión, retardo.

Requerimiento del tiempo real	Qué hace TCP	Por qué es un problema
Pérdida tolerable	no pierde nada: retransmite	la fiabilidad es **gratis pero cara**: el costo es tiempo
Retransmisión tolerable si es rápida	espera el **RTO**, que es ≥ 1 RTT	el RTO típico es 200 ms - 1 s: **no** es rápido
Entrega ordenada obligatoria	reordena con buffer	el reordenamiento **aumenta el retardo** del paquete que faltaba
Ritmo constante	el control de congestión varía el ritmo	el códec es **isócrono**: no sabe qué hacer con una pausa

b) Las hipótesis que HACEN que TCP sirva. Este es el apartado que el enunciado pide explícitamente y donde están los puntos:

```
HIPÓTESIS 1 — Pérdida muy baja y ESTABLE en el tiempo
──────────────────────────────────────────────────────

"las bajas tasas de pérdidas de servicios digitales domésticos (ADSL)"

¿Por qué? Porque si p ≈ 0, casi nunca hay retransmisión, y entonces el
costo de TCP (tiempo de retransmisión) es casi cero:

    E[costo] = P(pérdida) × RTO
    Con P = 0,001 y RTO = 200 ms ⇒ 0,2 ms de retardo extra promedio.

Esto es un argumento de EXPECTATIVA, no absoluto: lo que importa es el
producto, no los factores.


HIPÓTESIS 2 — RTT bajo y, sobre todo, ESTABLE
──────────────────────────────────────────────

"gran ancho de banda ... y baja latencia"

¿Por qué? Porque el RTO se estima a partir del RTT (Jacobson/Karels): con
RTT bajo el RTO es bajo. Pero el detalle fino es la ESTABILIDAD: si el RTT
varía mucho, el RTO queda mal estimado y se disparan timeouts espurios (o se
espera demasiado).

Y ojo con "gran ancho de banda": no arregla el retardo, lo que arregla es
que la transmisión del propio paquete sea rápida. Con 1 Mbps, mandar 160
bytes son 1,28 ms; con 64 kbps, son 20 ms.


HIPÓTESIS 3 — Reproducción holgada (buffered playback)
──────────────────────────────────────────────────────

Ésta NO está en la solución oficial y es la que más se olvida:

Si el receptor tiene un buffer GRANDE puede "digerir" una retransmisión sin
que se note: reproduce con p grande y el paquete que faltaba llega mientras
el receptor todavía tiene material.

Con un buffer de 500 ms y un RTO de 200 ms, la retransmisión "cabe" dentro
del margen y el oyente nunca se entera.

⇒ sin holgura, TCP es inservible; con holgura suficiente, TCP funciona
  aunque tenga retransmisiones.
```

c) La conclusión, que es lo que hay que escribir al final:

```
TCP puede usarse para tiempo real SI Y SÓLO SI la red cumple las tres
hipótesis:
   1. baja probabilidad de pérdida, estable en el tiempo
   2. RTT bajo y estable  (⇒ RTO bien estimado)
   3. reproducción con buffer holgado en el receptor

Un ADSL residencial de la época cumplía 1 y 2, y con un cliente de VoIP
decente cumplía 3. Por eso "VoIP sobre TCP" funcionaba (y así funcionaba el
primer Skype, antes de UDP).

⚠️ En una red con pérdidas altas o alta latencia, TCP es PEOR que no hacer
nada: no sólo agrega retardo, además hace que TODO el flujo se frene,
porque el control de congestión reduce cwnd para todos. No es "más lento":
es "inestable".
```

⚠️ **Actualización a 2026 sobre esta pregunta:** la respuesta **estructural** (las 4 razones + las 3 hipótesis) sigue siendo perfecta. Pero la realidad de 2026 es que casi nadie hace telefonía sobre TCP: Skype empezó sobre UDP,

y los códecs de voz modernos (Opus) van con DTLS-SRTP sobre UDP/QUIC. **PERO** hay una excepción que se puede citar: el **correo de voz almacenado** (que no es tiempo real) va por HTTP.

> **Consecuencia para el examen:** si el enunciado pregunta además "¿y hoy?", decí: "sobre UDP con RTP/SRTP; TCP se usa para el voicemail y para transferir archivos dentro de una sesión de llamada, no para el medio".

3.7 Examen 2010-02, Pregunta 5 (8 puntos) — Jitter, retardo y sensibilidad

> **Enunciado:** a) Defina el concepto de **jitter** y su diferencia respecto al **retardo**. b) Analice las siguientes aplicaciones de acuerdo a qué tan sensibles son respecto del jitter, el retardo y la pérdida de paquetes: • Descarga confiable de archivos. • Streaming almacenado.

Solución oficial (textual):

> **a)** El retardo es el tiempo que tarda un paquete desde que sale del emisor hasta que llega al receptor, mientras que el jitter es la variación de este retardo entre distintos paquetes de un mismo flujo. El retardo puede variar entre un paquete y otro porque (por ej.) los paquetes pueden tomar diferentes caminos de la fuente al destino. Un alto jitter puede implicar que paquetes que salieron más tarde lleguen más temprano al destino (llegan en desorden). Si la fuente y el destino están a varios saltos de distancia, y todos los paquetes del flujo siguieran el mismo camino pero no hubiera congestión, podríamos mantener el retardo constante (aunque sea un retardo de varios segundos) mientras que el jitter sería mínimo o nulo.
>
> **b)** Para la descarga confiable de archivos se necesita que lleguen todos los paquetes o el archivo será inválido, por lo que no es tolerante a pérdida de paquetes (hay que retransmitir). El retardo sólo implicaría tener que esperar un poco más de tiempo para comenzar a descargar el archivo, por lo que en general no significará un problema. Lo mismo ocurre con el jitter, incluso si los paquetes llegan en desorden será responsabilidad de la capa de transporte esperar a que lleguen todos y entregarlos ordenadamente, pero esto sólo implicará que se espere un poco más para la descarga. En el caso de streaming almacenado el usuario puede esperar un poco (unos cuantos segundos) a que comience la transmisión, pero una vez que comienza deseará que la misma no se corte y no tenga que esperar de nuevo. Estas transmisiones son por lo tanto tolerantes al retardo, y en cierta medida pueden tolerar el jitter en caso de que no sea muy excesivo. Si se pierde alguno de los paquetes de la transmisión el cliente puede saltearse la reproducción del mismo o interpolar paquetes cercanos.

La tabla que resume la respuesta, que es lo que hay que saber:

Aplicación	PÉRDIDA	RETARDO	JITTER
Descarga confiable de archivos	INSOPORT. (hay que retrans-mitir)	irrelev. (sólo arranca)	irrelevante (la capa de transporte reordena sola)
Streaming almacenado (bajo demanda)	tolerable (se salta o interpola)	TOLERANTE (espera segundos)	tolerable si no es excesivo (buffer de unos segundos)
Videollamada (referencia)	tolerable (clic)	INSOPORT. si es constante	INSOPORTABLE (entrecortado)

Los dos puntos finos de la solución oficial que hay que destacar (son los que suman):

1. **"aunque sea un retardo de varios segundos"**: el jitter puede ser **cero** con un retardo enorme. Consecuencia directa: **el jitter buffer grande resuelve el jitter, pero no el retardo absoluto**. Son dos mecanismos distintos.
2. **"paquetes que salieron más tarde lleguen más temprano"**: el jitter **extremo** es el **reordenamiento**, que es el peor caso, porque rompe la suposición de "los paquetes llegan en orden" de la que dependen muchas implementaciones.

> Tope para 8 puntos: las dos definiciones (retardo de un paquete vs. variación entre paquetes del mismo flujo), el ejemplo del retardo constante con jitter cero, y el análisis de las dos aplicaciones en las **tres** dimensiones.

PARTE 4 — CIERRE

📌 Resumen

1. El multimedia es **tolerante a la pérdida** porque es degradable: mejor perder 20 ms que esperar 800 ms por ellos.
2. **TCP no sirve** para tiempo real por **cuatro razones**: (1) la retransmisión espera el RTO; (2) la retransmisión **introduce jitter**; (3) el reordenamiento obliga a buffer y aumenta el retardo; (4) el control de congestión **frena** el flujo.
3. TCP **sí** serviría bajo tres hipótesis: pérdida baja y estable, RTT bajo y estable, y **reproducción holgada** en el receptor.
4. **UDP** es la base: sin conexión, sin fiabilidad, sin orden, sin control de congestión, **8 bytes** de cabecera.
5. **RTP** aporta tres campos: **número de secuencia** (detectar pérdidas y duplicados, y ordenar), **timestamp de muestreo** (el playout a ritmo constante, porque la referencia es la señal y no el reloj de pared) y **SSRC** (distinguir fuentes y flujos). **RTCP** da el feedback, con **SR** del que emite y **RR** del que recibe.
6. El **jitter** es la variación del retardo entre paquetes del mismo flujo; el **retardo** es el de un paquete. Un retardo constante enorme con jitter cero es perfectamente posible.
7. El **jitter buffer** acumula y reproduce a ritmo constante: **más buffer = menos cortes, más latencia**. Se adapta, con un máximo duro; si la latencia supera el máximo, se cancela.
8. **MPEG-2**: **I** = imagen completa; **P** = diferencia con el anterior; **B** = diferencia con el anterior **y el siguiente**, que es lo que obliga a **reordenar** y a un **delay de reproducción**. El **GOP** es la unidad; el **keyframe interval** fija el precio del arranque y del *seek*.
9. En el problema de VoIP: `64 kbps = 8 000 B/s`; con una muestra cada 40 ms (`25 muestras/s`) son **320 B/muestra**, y con 16 B de overhead el campo de datos del datagrama UDP mide **336 B**.
10. El mínimo `p` es `max(r_i − t_i)` sobre **todos** los paquetes (300 ms en el paquete 8), y la latencia de playout se suaviza con `p_i = 0,9·p_{i−1} + 0,1·v_i`, con `p_0 = 400 ms`, cancelando si supera 400 ms en N períodos seguidos o si la pérdida en la ventana de N supera el 10 %.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué un archivo de 100 MB es *intolerante* a la pérdida y un video de 100 MB en streaming *no*?

▶ Respuesta

2. Las cuatro razones por las que TCP no sirve para tiempo real. Formulá la cuarta sin mirar.

▶ Respuesta

3. ¿Por qué el **timestamp** de RTP y no el número de secuencia es lo que permite reproducir a ritmo constante?

▶ Respuesta

4. ¿Cuál es la diferencia entre **retardo** y **jitter**? Dá un ejemplo donde uno sea enorme y el otro cero.

▶ Respuesta

5. En el problema de VoIP, ¿por qué `p = 260` (el retardo del último datagrama) NO sirve?

▶ Respuesta

6. ¿Qué diferencia hay entre el `p` del apartado b) y el `p_i` del apartado c) del problema de 2011-02? ¿No se contradicen los dos 400 ms?

▶ Respuesta

7. ¿Por qué los B-frames obligan a reordenar y a un delay de reproducción?

▶ Respuesta

8. En el problema de 2010-07, ¿por qué los I-frames necesitan número de secuencia pero los P y B no? ¿Y por qué un solo bit?

▶ Respuesta

🔗 Referencias

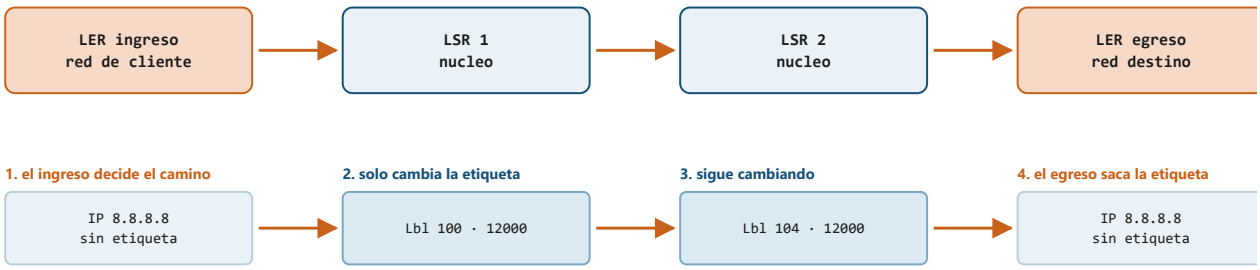
- **Examen `eirc1102` / `sirc1102` — Problema 1 completo (30 pts, VoIP)**; P1 (10 pts, los tres servicios de seguridad, → Clase 47); P2 (30 pts, IP Móvil, → Clase 48)
- **Examen `eirc1007` / `sirc1007` — Problema 1 completo (30 pts, MPEG-2 I/P/B)**; P4 (8 pts, DiffServ, → Clase 45)
- **Examen `eirc0807` / `sirc0807` — Problema 1 completo (30 pts, audio redundante con RTP)**; P1 (5 pts, IDS, → Clase 48)
- **Examen `eirc0812` / `sirc0812`** — P5 (8 pts, TCP para tiempo real); P3 (10 pts, IP Móvil, → Clase 48)
- **Examen `eirc1002` / `sirc1002`** — P5 (8 pts, jitter vs. retardo)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.4 ("Multimedia en la red") y §7.6 ("RTP: protocolo de transporte para tiempo real")
- **RFC 3550** — RTP: A Transport Protocol for Real-Time Applications (2003, *Internet Standard*; **sin cambios desde 2003**)
- **RFC 3711** — SRTP (Secure Real-time Transport Protocol)
- **RFC 3611** — RTCP XR (eXtended Reports), para VoIP quality monitoring
- **RFC 8285** — RTP Header Extensions (2017, obsolesce RFC 5285)
- **Clase siguiente:** 45 — QoS: shaping, policing, DiffServ y MPLS

Clase 45

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 El problema: Internet no garantiza nada
1.2 Los dos algoritmos de control de caudal: leaky bucket y token bucket
1.3 Shaping vs. policing: los dosWhere se usan
1.4 IntServ vs. DiffServ
1.5 El campo de marcado: el byte TOS / DS de IPv4
1.6 MPLS: etiqueta, LSP y los tres roles
1.7 Fragmentación transparente vs. no transparente
2.1 Ejercicio 1 — Leaky bucket con ráfaga de entrada
2.2 Ejercicio 2 — Token bucket con tokens acumulados
2.3 Ejercicio 3 — Qué DSCP le pongo a tres clases de tráfico
2.4 Ejercicio 4 — El flujo de un paquete por un LSP
3.1 Examen 2007-12, Pregunta 1 (10 puntos) — Leaky bucket y token bucket
3.2 Examen 2009-03, Pregunta 4 (10 puntos) — DiffServ: edge, core y campo de marcado
3.3 Examen 2010-07, Pregunta 4 (8 puntos) — El paradigma de DiffServ
3.4 Examen 2007-12, Pregunta 4 (10 puntos) — Fragmentación transparente y no transparente
3.5 Examen 2007-12, Pregunta 5 (5 puntos) — MPLS: router de ingreso y de egreso
3.6 Examen 2006-08, Pregunta 5 (5 puntos) — MPLS: etiqueta, LSP y roles
📌 Resumen
✅ Autoevaluación
🔗 Referencias

Un LSP de MPLS saltando por el nucleo



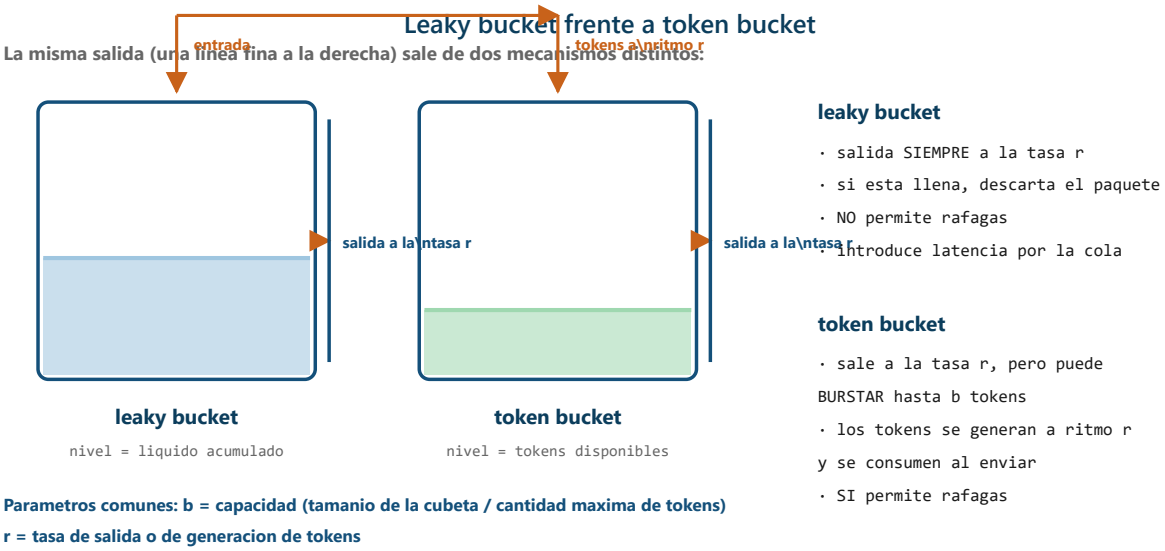
El LSR nunca mira la direccion IP: por eso es rapido.
Ese es el trade-off: velocidad a cambio de que el nucleo no puede aplicar ningun filtro de seguridad por IP.

Bloque: F (Servicios, multimedia y seguridad) · **Semana:** 15 · **Duración estimada:** 105 min **Correlativa:** Clase 44 (Multimedia: streaming, RTP y VoIP) · **Sigue con:** Clase 46 (Gestión de redes: SNMP, NETCONF y YANG) **Material de referencia:** Tanenbaum cap. 5 §5.6 (control de congestión, control de calidad de servicio) · RFC 2211/2212/2213, RFC 2474, RFC 2475, RFC 3246, RFC 3031, RFC 3032

🎯 Al terminar esta clase vas a poder

1. Explicar por qué Internet es **best effort** y por qué eso rompe las aplicaciones multimedia.
2. Explicar el algoritmo de **leaky bucket** y el de **token bucket**, con sus dos parámetros, y decir **cuál descarta y cuál espera**.
3. Distinguir **shaping** de **policing**, y justificar la arquitectura **ingress policing + egress shaping**.
4. Comparar **IntServ** y **DiffServ**, y explicar **por qué DiffServ ganó**.

5. Dibujar el **byte TOS/DS de IPv4** con los 6 bits de DSCP y los 2 de ECN, y nombrar los **PHB** que hay que saber.
6. Explicar **MPLS**: qué son la etiqueta y el LSP, y **qué hace cada uno de los tres roles de router**.
7. **Seguir el flujo de un paquete** por un LSP, cambiando la etiqueta en cada router.



> **Prerequisito del examen:** `2007-12 P1` (10 pts, leaky bucket vs. token bucket), `2009-03 P4` (10 pts, DiffServ), `2010-07 P4` (8 pts, DiffServ), `2007-12 P4` (10 pts, fragmentación transparente vs. no transparente), `2007-12 P5` y `2006-08 P5` (5 pts cada una, MPLS).

PARTE 1 — TEORÍA

1.1 El problema: Internet no garantiza nada

Todo lo que estudiaste hasta ahora funciona sobre una misma premisa:

```
INTERNET ES "BEST EFFORT" (mejor esfuerzo)
──────────────────────────────────────────
El router reenvía lo que le llega. Si el buffer de salida está lleno,
DESCARTA. Si no, reenvía. No hay nadie que pregunte "¿esto es
importante?".

Consecuencia:
una videollamada de 2 Mbps y una descarga de 2 Mbps
viajan EXACTAMENTE IGUAL.
La descarga gana (es un flujo TCP que insiste), y la videollamada
se rompe.

Y el problema se agrava porque el tráfico no se reparte solo:
el tráfico best effort de una fuente que no implements control
de congestión PUEDE sobre el resto.
```

El ejemplo que hay que tener en la cabeza: en una red donde hay un enlace de acceso de 10 Mbps, tenés una descarga de 10 Mbps y una videollamada de 1 Mbps. El **ancho de banda alcanza** (11 > 10 es falso, pero imaginá 20 Mbps de enlace y 10+1). Sin embargo la videollamada se entrecorta, porque:

```
El BUFFER del router, no el ancho de banda, es el problema.
──────────────────────────────────────────────────────────
El paquete de voz llega, encuentra su cola de 100 paquetes llena
(because la descarga llenó el buffer), y se descarta.

PÉRDIDA de 1 paquete de voz = 20 ms de la palabra del otro.

Y el agregado es peor: la descarga ve la pérdida, baja cwnd, y la
videollamada ve la pérdida y no puede hacer nada (no retransmite).
```

Y hay un segundo problema, más sutil: la **cola grande** en los routers es en apariencia una mejora, y en realidad es una trampa. Se llama **bufferbloat**:

```
Buffer PEQUEÑO (1 paquete de cola)
→ se pierde rápido, el router no se atrasa mucho
→ throughput bajo, LATENCIA BAJA

Buffer GRANDE (1000 paquetes de cola)
→ se pierde tarde, el router "aguanta" todo
→ throughput alto, LATENCIA ENORME (el paquete espera 2 s)

El búfer grande convierte una pérdida de 0,1 % en 2 segundos de
latencia. Para el tráfico interactivo, el bufferbloat es PEOR que
la pérdida.

⇒ La calidad de servicio NO es un lujo: es la única herramienta que
tenemos contra el bufferbloat.
```

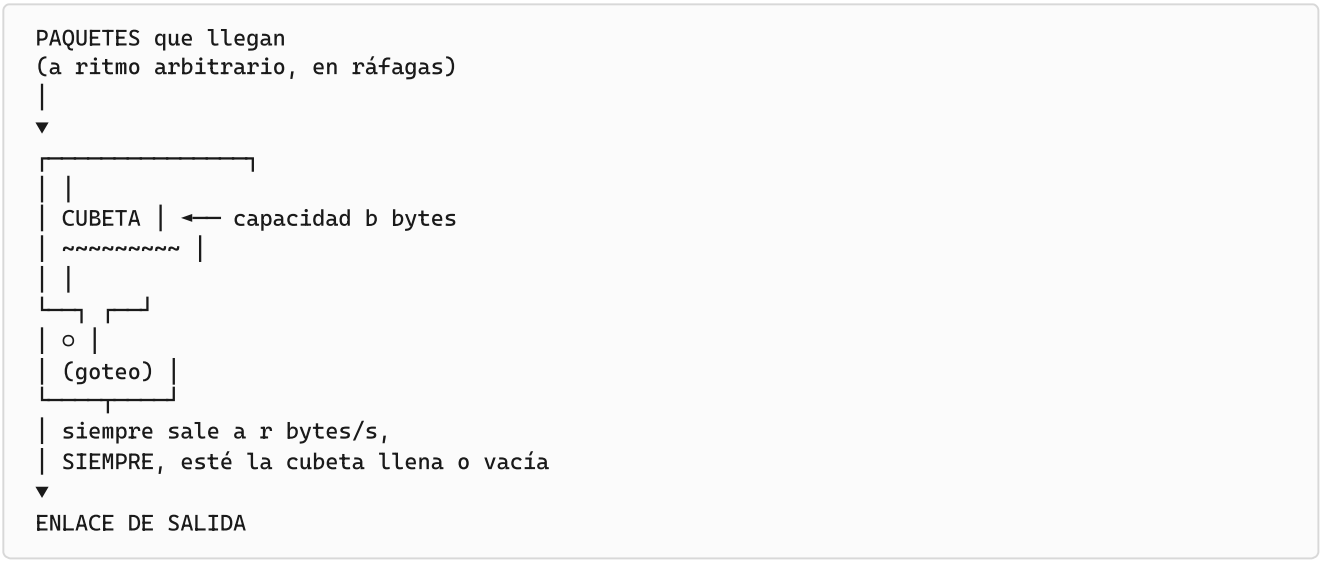
1.2 Los dos algoritmos de control de caudal: leaky bucket y token bucket

Son los dos mecanismos básicos de **conformado de caudal** (*rate shaping*), y son de capa 2, de capa 3 o de un router. Los dos se usan para decir: "de este enlace no puede salir más de `r` bytes por segundo".

(a) Leaky bucket — cubeta con goteo

El nombre viene de la imagen, y es buena imagen: una cubeta con un agujero delgadito en el fondo. El agua entra a ritmo arbitrario y sale **siempre al mismo ritmo**, por el agujero.

```
Los 3 parámetros (y hay que memorizarlos con estos nombres):
─────────────────────────────────────────────────────────
b = capacidad de la cubeta, en BYTES (el tamaño del buffer)
r = tasa de salida, en BYTES POR SEGUNDO (el agujero)
f = tiempo de vaciado = b / r (cuánto tarda la cubeta en vaciarse)

El tercer parámetro es el que se pregunta menos y el que explica
todo: f es la "ventana de compatibilidad" con el jitter.
```



El algoritmo, literal:

```
CUBETA VACÍA
LLEGA un paquete de tamaño L:

SI (contenido + L) ≤ b : contenido ← contenido + L (aceptado)
SINO: DESCARTAR el paquete (o marcarlo)

PERIÓDICAMENTE (cada 1/r segundos):
contenido ← contenido − r (o contenido − L del primero)
contenido nunca baja de 0 (si baja, la cubeta está vacía y
no sale nada)
```

El detalle que hay que entender bien: aunque la cubeta esté vacía, **el goteo no se detiene**: sigue saliendo un paquete cada `1/r` segundos. Eso es lo que produce el **flujo constante** (salida a ritmo constante) aunque la entrada sea a ráfagas. El precio es que, si la cubeta se vacía, se **desperdicia capacidad**: el enlace queda libre aunque haya tráfico esperando.

Con números (esto es lo que hay que hacer en el examen):

```
b = 100 bytes, r = 10 bytes/s, paquetes de 20 bytes.

Cuántos paquetes de 20 bytes entran en 1 segundo?

── ¿Cuántos ENTRAN? (durante 1 s la cubeta recibe lo que le llega)
Esto depende de la tasa de ENTRADA, que el enunciado tiene que dar.
Si la entrada es de, digamos, 30 paquetes/s:
contenido inicial = 0
t = 0,00 : llega P1 (20) → contenido = 20 ≤ 100 ✓
t = 0,03 : llega P2 (20) → contenido = 40 ✓
t = 0,07 : llega P3 (20) → contenido = 60 ✓
t = 0,10 : llega P4 (20) → contenido = 80 ✓
t = 0,13 : llega P5 (20) → contenido = 100 ✓ (llena)
t = 0,17 : llega P6 (20) → 120 > 100 ✗ DESCARTA
(y todos los que siguen hasta que se vacíe)

En 1 s, de una entrada de 30 paquetes/s (30 paquetes), entran
5 y se descartan 25.

── ¿Cuántos SALEN? (esto sí lo determina el algoritmo)
El goteo es de 10 bytes/s y cada paquete pesa 20 bytes
⇒ sale 1 paquete cada 20/10 = 2 segundos.

En 1 s salen 0,5 paquetes. Ojo: en 1 segundo puede no salir NINGUNO.
Con f = b/r = 100/10 = 10 s, la cubeta tarda 10 s en vaciarse.

RESUMEN DEL EJEMPLO:
el leaky bucket convierte UNA ENTRADA A RÁFAGAS en una SALIDA
A RITMO CONSTANTE de 10 bytes/s, y ELIMINA todo lo que exceda.
```

(b) Token bucket — cubeta con tokens

La diferencia es **una sola palabra**: en vez de agua, hay **tokens** que se producen a ritmo constante y se **consumen** al mandar paquetes.

```
PAQUETES que llegan
(a ritmo arbitrario)
│
▼
┌─────────────┐
│ o o o o o │ ←── los TOKENS se generan a r tokens/s
│ o o o o o │ (hasta un máximo de b tokens)
│ o o o o o │
└─────────────┘
│
para enviar un paquete de L bytes:
NECESITO L tokens. Si no hay L, el paquete ESPERA.
│
▼
ENLACE DE SALIDA
```

El algoritmo, literal:

```
GENERADOR (independiente de los paquetes, corre siempre):
cada 1/r segundos: tokens ← min(tokens + 1, b)
(tokens nunca supera b: la cubeta de tokens tiene capacidad b)

LLEGA un paquete de tamaño L:

SI tokens ≥ L : tokens ← tokens − L ; ENVIAR el paquete
SINO : ESPERAR hasta que haya L tokens, y recién
ahí enviarlo.
```

El contraste con el leaky bucket, que es lo que más se pregunta:

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ │ LEAKY BUCKET │ TOKEN BUCKET │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Qué produce │ tokens │ tokens │
│ Qué se genera │ NADA │ r tokens/s │
│ Qué pasa si no hay │ se DESCARTA el paquete │ el paquete ESPERA │
│ "crédito" │ (o se marca) │ (shaping) │
│ Ritmo de SALIDA │ SIEMPRE constante │ puede tener RÁFAGAS │
│ Ráfagas │ IMPOSIBLES │ POSIBLES (hasta b) │
│ Utiliza toda la │ NO │ SÍ │
│ capacidad si sobra │ (se desperdicia) │ │
│ ¿Retrasa? │ Sí, inherentemente │ Sí, si el credit no │
│ │ (por el buffer) │ alcanza │
│ Uso típico │ control de caudal │ control de caudal │
│ │ "duro" │ "blando" │
└──────────────────────┴──────────────────────┴──────────────────────┘
```

La frase que hay que saber decir: "*el leaky bucket descarta, el token bucket espera; el leaky bucket produce flujo constante, el token bucket permite ráfagas de hasta* `b`."

Y el caso límite que hay que entender: si el token bucket tiene `b` tokens acumulados y entran `b` paquetes de golpe, **salen los `b` juntos**: eso es una ráfaga. Con el leaky bucket eso es imposible, por construcción.

```
TOKEN BUCKET, r = 10 tokens/s, b = 50 tokens, paquetes de 10 bytes:

t=0 : tokens = 50 (lleno) llegan 5 paquetes de 10 bytes
P1: tokens 50≥10 → tokens=40, sale
P2: tokens 40≥10 → tokens=30, sale ← los 5 salen
P3: tokens 30≥10 → tokens=20, sale ← SEGUIDOS.
P4: tokens 20≥10 → tokens=10, sale Eso es una
P5: tokens 10≥10 → tokens= 0, sale ← RÁFAGA de 50 bytes
t=0.1 : tokens = 0 + 1 = 1
t=0.2 : tokens = 2 ... y así, se va reponiendo a 10 tokens/s

Con leaky bucket (b=50, r=10, paquetes de 10):
t=0 : entra P1 (10 ≤ 50) cubeta=10 ; P2 cubeta=20 ; P3=30 ; P4=40 ; P5=50
sale 1 paquete cada 10/10 = 1 s. Los 5 salen a t=1,2,3,4,5.
NUNCA se pueden salir juntos. ← misma cubeta, resultado opuesto.
```

1.3 Shaping vs. policing: los dosWhere se usan

```
SHAPING (conformado) POLICING (vigilancia)
──────────────────── ──────────────────────

ENCOLA y RETRASA DESCARTA o REMARCA lo que excede
←── buffer + scheduler ──→ ←── sin buffer, sin retardo ──→
Mantiene la tasa, deforma el timing Mantiene la tasa exacta,
(introduce LATENCIA) no deforma nada
El paquete SIEMPRE pasa (tarde) El paquete NO pasa
Implementado con: token bucket + cola Implementado con: token bucket
solo de conteo
```

¿Por qué los dos? Porque cada problema tiene su lugar natural:

```
ENLACE (o router) por qué ese
──────────────────────────────────── ──────────────────────────

ENTRADA (ingress) POLICING
Un usuario no autorizado podría No querés RETRASAR un paquete
mandar 100 Mbps yaybe le ocupes que venía de una app legítima
todo el buffer del router. (¡es una discriminación!).
→ Tenés que tira algo. Además, meterlo en cola te
→ Y sin latencia: no tenés derecho. agrega latencia a un paquete
que ya llegó tarde.

SALIDA (egress) SHAPING
Acá SÍ sos vos el dueño del enlace. Querés aprovechar al máximo el
Los paquetes son tuyos y de tus enlace y absorber las ráfagas
usuarios: podés priorizar, (p. ej. hacia un enlace de
delaying y descartar. bajada más lento que el de
→ Podés jugar con la latencia. subida).
```

De ahí la frase estándar de la industria:

> **La arquitectura clásica es "policing en la entrada, shaping en la salida".**

1.4 IntServ vs. DiffServ

IntServ — el modelo que prometía garantías y no escaló

Integrated Services (`RFC 2211` , `RFC 2212` , `RFC 2213` , 1997). Es el modelo "de lujo": **reserva recursos por flujo**.

```
REQUISITOS:
1. El emisor (o un "controlador de admisión") dice QUÉ calidad quiere
para un flujo: "necesito 2 Mbps, 20 ms de retardo, jitter < 5 ms".

2. Se señaliza con RSVP (Resource ReSerVation Protocol) UNA RUTA COMPLETA:
RSVP Hello → RSVP Request → RSVP Confirm (de abajo hacia arriba)
y el router decide: "¿tengo ancho de banda y buffer para esto?"

3. Si los routers de la ruta ACEPTAN, se RESERVA: ancho de banda en
cada enlace, buffers con tamaño fijo, y se instalan clasificadores
y marcadores de tiempo en cada router.

4. Ahora el flujo tiene GARANTÍA: bw mínimo, retardo máximo, jitter
máximo, tasa de pérdida acotada. Es un CIRCUITO VIRTUAL con garantías.
```

(con la reserva ya instalada)

El problema fatal, y es lo que hay que decir en el examen:

```
Cada router debe GUARDAR ESTADO POR FLUJO:
- la reserva de ancho de banda de cada flujo en cada enlace
- un clasificador que mire CADA paquete para decidir a qué flujo
pertenece (5-tupla) y aplicarle la QoS correcta
- un buffer por clase

Coste: memoria y CPU CRECEN con el número de flujos.

Cantidad de flujos simultáneos en Internet (2026):
YouTube: ~millones
videollamadas: ~cientos de miles simultáneas
Spotify: ~cientos de millones de usuarios

Un router de backbone no puede guardar estado de millones de flujos.
⇒ IntServ NO ESCALA. Punto.

Además tiene otros problemas:
- el problema del "control distribuido": si un router del path se
cae, hay que renegociar toda la reserva
- el emisor tiene que conocer la topología
-.movie el "diamante": si hay dos caminos posibles, hay que reservar
en AMBOS (o quedarse sin garantías)
```

> **Actualización a 2026:** IntServ ≈ **no se despliega a escala**. La única supervivencia real es **RSVP-TE** (`RFC 3209`) para *traffic engineering* de MPLS, donde el número de flujos de tráfico reservado es **ordenes de magnitud menor** que en Internet. Decilo así: "*IntServ es el modelo que no escaló, y DiffServ el que ganó*".

DiffServ — el modelo que escaló

Differentiated Services (`RFC 2474` el campo DS, `RFC 2475` la arquitectura). La idea es **voltear el modelo**: en vez degarantía por flujo, se garantiza por **CLASE**.

```
LA INVERSIÓN FUNDAMENTAL:
─────────────────────────
IntServ: señalización explícita + estado POR FLUJO + garantías
DiffServ: NINGUNA señalización + estado POR CLASE + clases de servicio

"En vez de reservar para cada llamada, reservo para 'vocoder audio'."
```

El modelo tiene tres pasos, y el **nombre de cada uno hay que memorizarlo**:

```
1. CLASSIFICATION (clasificación) en el EDGE (borde)
El router del borde CLASSIFICA el flujo. Usa una tabla
(puerto, protocolo, IP, DSCP) → DSCP. Ejemplo:
(RTP, puerto 16384-32767) → DSCP EF (voz)
(HTTP, puerto 80) → DSCP AF11 (web)
(todo lo demás) → DSCP 0 (best effort)

2. MARKING (marcado)
El borde ESCRIBE el DSCP en el byte TOS del paquete IP.
El DS field viaja por Internet.

3. PHB (Per Hop Behaviour) en el CORE (núcleo)
Cada router del core aplica la política correspondiente al DSCP
que ve. NO necesita señalización. NO necesita saber qué flujo es.
Sólo necesita leer 1 byte.

RESUMEN EN UNA FRASE (la que hay que escribir):
─────────────────────────────────────────────────
"El EDGE clasifica y marca; el CORE encamina aplicando el PHB
definido para cada marca, usando el campo TOS de IPv4."
```

La diferencia conceptual que hay que entender (es lo que se pregunta):

```
IntServ DiffServ
──────── ─────────
Garantía POR FLUJO Garantía POR CLASE
(este flujo, estos parámetros) (todos los de voz se tratan igual)
identidad del flujo = 5-tupla identidad = 6 bits de DSCP
señalización: RSVP señalización: NINGUNA
estado: en cada router estado: en cada router, pero SOLO
por clase (unas pocas, 12-ish)
escala: NO escala: SÍ
```

La analogía que ayuda: DiffServ es como un **sala de espera con dos categorías** ("los de la fila A" y "los de la fila B") en lugar de un ticket individual con turno exacto para cada persona. Es más simple, más justo en promedio, y escala.

⚠️ Actualización a 2026

Sigue igual, de memoria: - **DSCP / PHB**: **EF** (Expedited Forwarding, `RFC 3246`, para voz), **AF11..AF43** (`RFC 2597`, cuatro clases × tres drop precedences), **Default/BE**, y **CS** (Class Selector, heritage de los valores de TOS anteriores a DSCP). - **Marking en el borde + PHB en cada hop.** - **Policing vs. shaping**: policing con token bucket (descarta o remarkea lo que excede, sin buffer); shaping con cola + token bucket (retrasa y suaviza, introduce latencia). **Ingress policing + egress shaping. Esto no cambió.** - `RFC 8622` (2019) actualiza al PHB **LE** (*Lower Effort*, *Lower Expected effort*) y **obsolesce** `RFC 3662`, que era el PDB de DiffServ. Si el enunciado cita `RFC 3662`, está desactualizado.

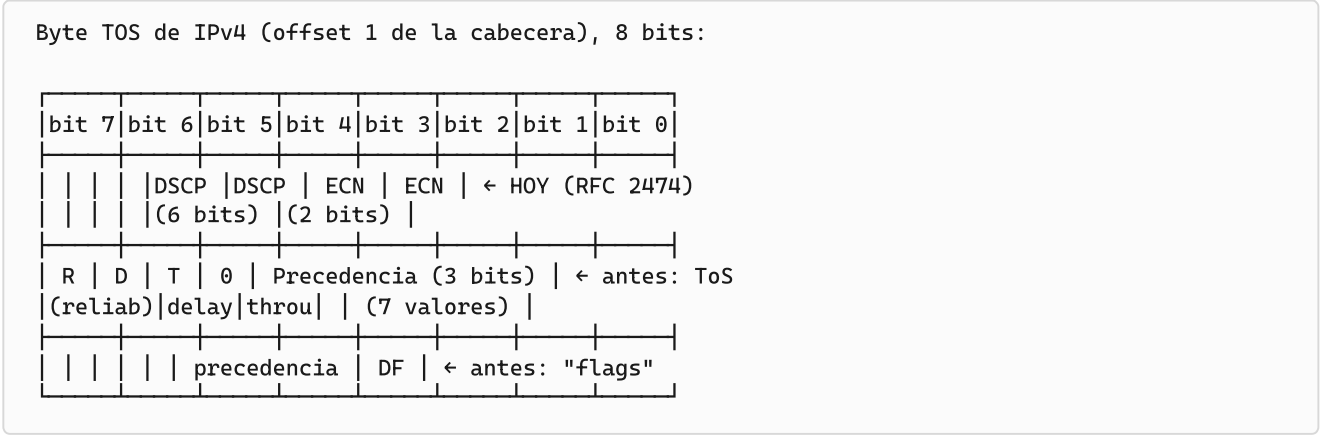
Cambió: - **DiffServ ganó hace más de 20 años.** No hay nada que actualizar: es el modelo desplegado. - Lo que **sí** es nuevo es el **equilibrio (ECMP)**: con unequal-cost load balancing, el PHB puede dejar de funcionar bien, porque el camino cambia a cada salto y los buffers no están donde el modelo los pone. En la práctica hoy se combina ECN + AQMs en vez de DiffServ puro.

1.5 El campo de marcado: el byte TOS / DS de IPv4

Este es **el detalle fino que más se pregunta** (`2009-03 P4` y `2010-07 P4`).

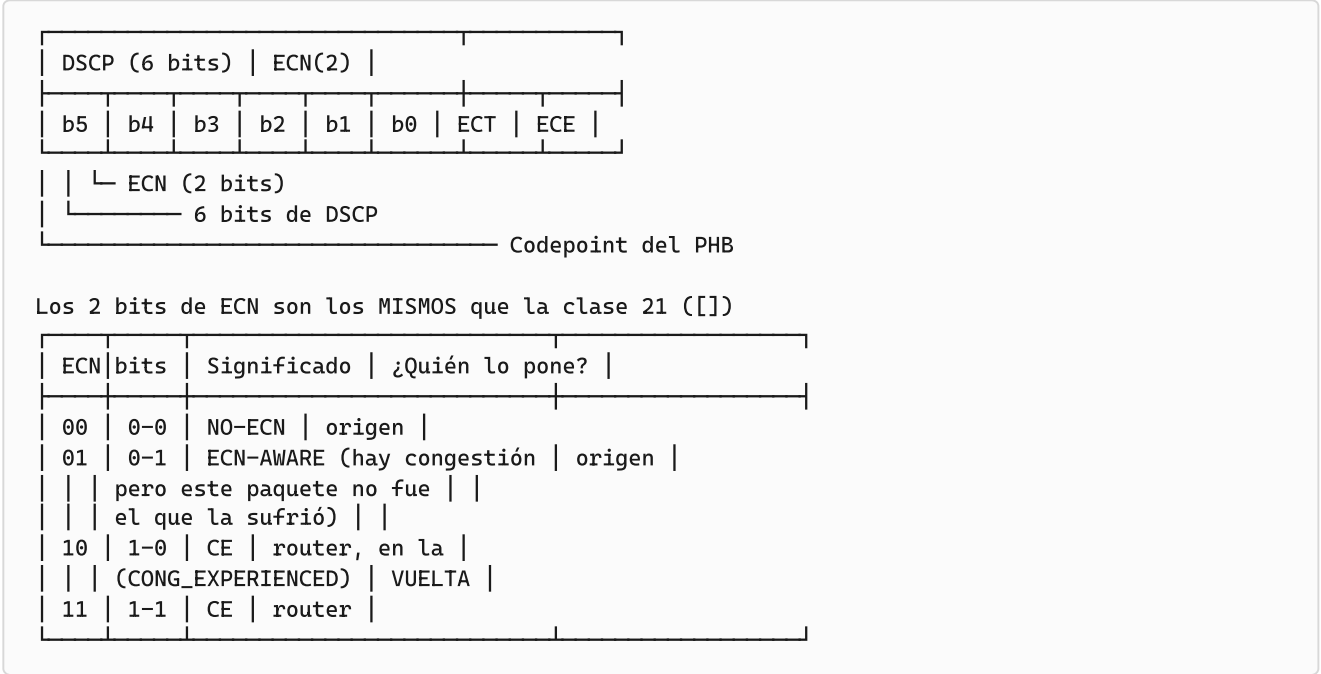
La historia del byte, que es lo que hay que saber

El byte de tipo de servicio de IPv4 tiene 8 bits y **se usó para tres cosas distintas** a lo largo del tiempo. Por eso los nombres son confusos:



> **En el examen, cuando el enunciado dice "el campo TOS de IPv4", la respuesta moderna es: el byte DS (*Differentiated Services*), con 6 bits de DSCP y 2 bits de ECN.** No la precedencia de 3 bits ni los bits R/D/T.

El layout del byte DS (el que hay que dibujar)



Los 2 bits de ECN son los MISMOS que la clase 21 ([])

ECN	bits	Significado	¿Quién lo pone?
00	0-0	NO-ECN	origen
01	0-1	ECN-AWARE (hay congestión pero este paquete no fue el que la sufrió)	origen
10	1-0	CE (CONG_EXPERIENCED)	router, en la VUELTA
11	1-1	CE	router

Regla de oro para el examen: el bit de ECN va después del DSCP, y no al principio. El error típico es ponerlos al revés porque el layout "histórico" tenía la precedencia primero.

Los PHB que hay que saber

```
PHB DSCP Para qué
──────────────────────────── ──────── ──────────────────────────────────────

Default / Best Effort 000000 Sin ningún tratamiento especial. Es el
caso por defecto.
──────────────────────────────────────────────────────────────────────────────

EF (Expedited Forwarding) 101110 Voz. RFC 3246. Es un "tubería": muy
(46 binario = 0b101110) baja probabilidad de descarte, banda
DSCP = 46 (11 de 6) mínima garantizada, y **NO se permite
encolarlo detrás de nada**. Se usa
con policing/shaping estricto: si hay
más voz de la que cabe, la bajan.
──────────────────────────────────────────────────────────────────────────────

AFxy (Assured Forwarding) AF11 4 clases (1..4) × 3 drop precedences
AF11 .. AF43 AF12 (1 = menor, 3 = mayor). AF1 = tráfico
DSCP: AF11=001010 (10) AF13 interactivo (voz, juegos), AF2 = FTP,
AF12=001100 (12) AF21 AF3 = web, AF4 = bulk (correo, backup).
AF13=001110 (14) AF22 La diferencia entre clases es CUÁNTO
AF21=010010 (18) AF23 ancho de banda se GARANTIZA. La
AF22=010100 (20) AF31 diferencia entre los "y" de una misma
AF23=010110 (22) AF32 clase es la PRIORIDAD de descarte:
AF31=011010 (26) AF33 si hay congestión, se tira primero
AF32=011100 (28) AF41 un AFx3, después un AFx2, después un
AF33=011110 (30) AF42 AFx1, y los AFy de la clase más baja.
AF41=100010 (34) AF43
AF42=100100 (36) AF43=100110 (38)
──────────────────────────────────────────────────────────────────────────────

CS (Class Selector) 1xxxxxx Heritage: los 8 valores que existían
en el byte TOS viejo. CS1..CS7. Hoy
están "reservados" y no se definen
como PHB por RFC. CS0 = 000000 =
Default.
```

Los PHB oficiales con RFC (de `ACTUALIZACIONES.md` §12):

- `RFC 2474` — el campo **DS**
- `RFC 2475` — la **arquitectura** de DiffServ
- `RFC 3246` — **EF** (Expedited Forwarding)
- `RFC 2597` — **AF** (Assured Forwarding)
- `RFC 4594` — **guidelines** de clases de servicio (cómo configurar el conjunto)
- `RFC 8622` — **LE PHB** (obsolesce `RFC 3662`)

1.6 MPLS: etiqueta, LSP y los tres roles

MPLS (Multiprotocol Label Switching) resuelve un problema muy concreto: en IP puro, cada router tiene que **mirar la dirección IP de destino** de cada paquete y hacer una **búsqueda en la tabla** (longest prefix match). Eso es caro en hardware de alta velocidad, y obliga a que el equipo tenga una TCAM enorme.

El label (etiqueta) — 4 bytes

```
┌──────────────────────────────┬──────────────┬─────────┬──────┐
│ Label (20 bits)              │ TC (3)       │ S       │ TTL  │
├──────────────────────────────┼──────────────┼─────────┼──────┤
│ índice de la FEC a la que    │ Traffic      │ Stack   │      │
│ pertenece este paquete       │ Class        │ depth   │      │
└──────────────────────────────┴──────────────┴─────────┴──────┘
▲ ▲ ▲ ▲
│ │ │ └─ se usa IGUAL que
│ │ │ en IP: para el
│ │ │ loop detection y
│ │ │ traceroute
│ │ └─── S=1 significa que
│ │ hay otra etiqueta debajo
│ └─── TC: 3 bits para QoS/prioridad
└─── "índice en la tabla": dice "este paquete es de la
clase de la FEC X; aplicá el tratamiento de la entrada X"
```

Lo esencial: la etiqueta **no es una dirección**. No contiene el destino, no se puede "enrutar" por ella en el sentido de IP. Es **un índice en la tabla del router**. Y por eso es **una entrada de tabla, no una búsqueda**.

Y el campo TC (Traffic Class) es donde vive el QoS de MPLS: los 3 bits del TC se usan para marcar la prioridad, y hay **diferentes tramas de experimentación** (*EXP`, TC`, Precedence`) según el RFC. Es el equivalente MPLS del DSCP.

La FEC (Forwarding Equivalence Class)

Es el concepto que hay que definir sí o sí:

```
FEC = el grupo de paquetes que receives TRATAMIENTO IDÉNTICO.

En la práctica: "los paquetes que van del prefijo 10.1.1.0/24 al
prefijo 10.2.2.0/24" son una FEC. "Los paquetes con destino 172.16/12
que entran por la interfaz 3" son otra FEC.

La idea: no me interesa el destino EXACTO de cada paquete. Me
interesa qué grupo al que pertenece. Eso me permite reenviar mirando
20 bits en lugar de 32 (o de una tabla con 1 millón de entradas).
```

El LSP (Label Switched Path)

```
LSP = Label Switched Path
= un CAMINO PREDEFINIDO por la red, unidireccional,
formado por saltos controlados por ETIQUETAS.

┌──────┐ etiqueta 100 ┌──────┐ etiqueta 205 ┌──────┐
│ R1 │──────────────────►│ R2 │──────────────────►│ R3 │
│ │ (la pone R1) │ │ (la pone R2) │ │
└──────┘ └──────┘ └──────┘

▲ ▲ ▲
│ │ │
INGRESS TRANSIT EGRESS
(LER) (LSR) (LER)

Un LSP es UNIDIRECCIONAL (simplex). Para una sesión bidireccional
hacen falta DOS LSP (uno en cada sentido), y el par se llama
"LSP pair" o "L2VPN" si llevan etiqueta de VPN encima.
```

Y la diferencia con un circuito virtual clásico es la que hay que saber: el LSP de MPLS **no reserva recursos**. Es una "sugerencia" de camino; los recursos los decide el *ingress* router en función de la **tabla de forwarding**. Por eso MPLS **no es un circuito virtual**, y por eso el problema de la clase 25 ("VC reserva recursos") reaparece: MPLS se puede combinar con DiffServ para tener clase de servicio, pero **no** garantiza ancho de banda por flujo.

Los TRES roles de router (esto es lo que preguntan)

```
┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐
│ EMISOR │ │ INGRESO │ │ TRÁNSITO │ │ TRÁNSITO │ │ EGRESO │
│ (host) │ │ LER │ │ LSR │ │ LSR │ │ LER │
└────┬─────┘ └────┬─────┘ └────┬─────┘ └────┬─────┘ └────┬─────┘
│ │ │ │ │
│ IP │ MIP │ M┌──────┐ M │ M┌──────┐ │ IP
├──────────►│ │ │etiqueta│ │ │etiqueta│ ├──────────►
│ paquete │ mete la │ │ 205 │ │ │ 300 │ │ quita la
│ │ etiqueta │ │ │ │ │ │ etiqueta
└──────────┴──────────┴──┴──────┴──┴──┴──────┴──┴──────────┘

▲ ▲ ▲
hace FORWARDING SÓLO CAMBIA saca la MPLS,
con la dirección IP LA ETIQUETA, pone la IP
y decide el camino NO MIRA LA IP de capa 3
```

Rol	Sinónimos	Qué hace	Qué mira
Ingress (*head-end*)	**LER** (*label edge router*)	**examina** los paquetes IP entrantes, los **clasifica en una FEC**, **decide el camino**, **pone la etiqueta inicial** (hace *push*)	**La dirección IP**
Transit (*transit LSR*)	**LSR**	**reenvía el paquete MPLS cambiando la etiqueta** (*swap*). No consulta la dirección IP.	**Sólo la etiqueta**
Egress (*tail-end*)	**LER**	**quita la etiqueta MPLS** (*pop*) y pone la **cabecera de capa 3** de abajo	La etiqueta

El detalle que hay que subrayar y que es la ganancia de MPLS:

> **El router de tránsito NO mira la dirección IP.** Toma 20 bits de etiqueta, hace una **búsqueda exacta** (o un switch directo por ASIC), y escribe 20 bits nuevos. Eso es lo que hace que MPLS sea **más rápido** que el

> forwarding por IP: una **búsqueda exacta** en lugar de un **longest prefix match** sobre 32 bits en una TCAM.

Y el otro detalle que se pregunta: en MPLS **se puede hacer forwarding de familias que no son IP**. El nombre completo lo dice: *Multiprotocol* **Label Switching*. Un router de tránsito MPLS** no necesita entender el protocolo que transporta**: reenvía tramas por label y no le importa si es IP, ATM o Ethernet.

Los 4 roles de etiqueta (y acá hay una tabla que hay que saber):

```
Swap = el caso normal: se cambia una etiqueta por otra (tránsito)
Push = se agrega una etiqueta encima (ingreso, o cuando el LSP se
divide por uno de los LSRs)
Pop = se saca la etiqueta de arriba (salida del LSP)
Impose = se saca una etiqueta de una capa inferior y se pone la de la
capa de arriba (penultimate hop popping)
```

⚠️ Actualización a 2026

Sigue igual, de memoria: - **MPLS sigue vivo en el plano de datos de los carriers.** La etiqueta, el LSP, los tres roles, el `TC`, el swap/push/pop: todo igual. - Los RFC: `RFC 3031` (arquitectura), `RFC 3032` (label stack), `RFC 4364` (MPLS VPN).

Cambió — y acá está la noticia fuerte:

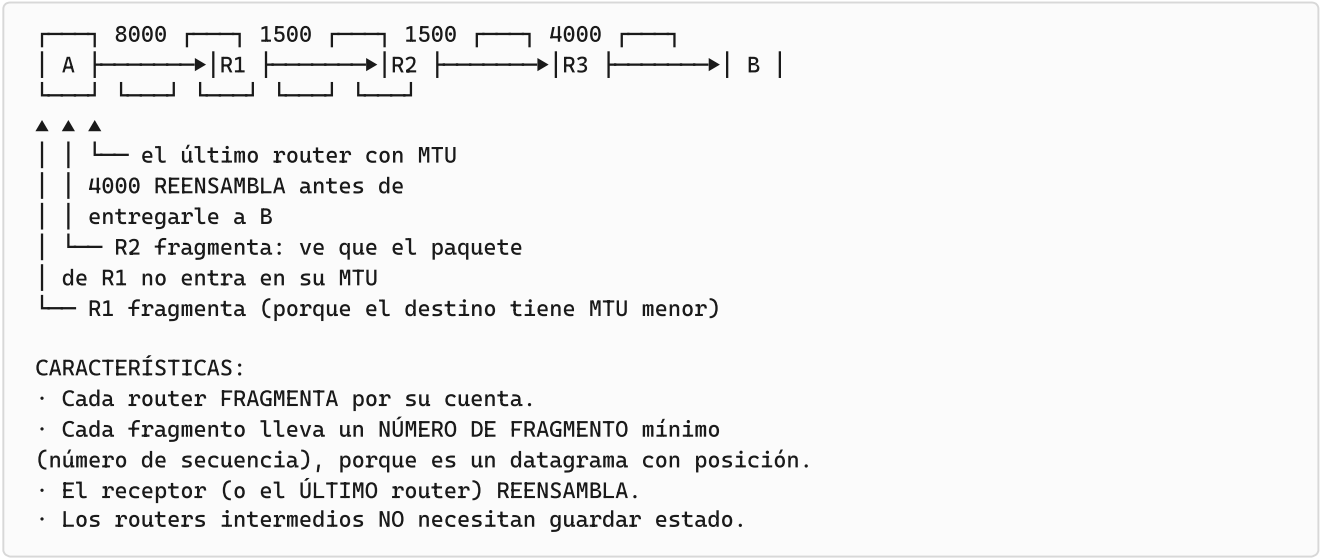
- **El plano de control se está moviendo a Segment Routing.** SR (`RFC 8402`, arquitectura) codifica la ruta con **SIDs** (*Segment Identifiers*) en vez de estados por prefijo, y el estado vive **solo en el borde de ingreso**.
- **SR-MPLS** (`RFC 8660`) usa etiquetas MPLS para codificar los SIDs.
- **SRv6** (`RFC 8986`) **elimina el MPLS por completo**: codifica los SIDs directamente en la **cabecera IPv6**, con un encabezado de Routing/IPv6 opcional. Ése es el sucesor real en despliegues nuevos.
- **En datacenter, MPLS fue reemplazado por EVPN-VXLAN** (`RFC 7432` + `RFC 7348`). VXLAN encapsula tramas de Ethernet sobre **UDP/4789**, con lo cual un tenant puede tener su propia tabla MAC y su propio dominio de broadcast. Hoy el overlay de datacenter **no es MPLS**: es EVPN-VXLAN.
- **Por qué MPLS no se muere** (respuesta de examen, es lo que hay que saber): porque MPLS construye **VPNs de capa 3 con aislamiento estricto**: un tenant no ve los prefijos del otro, ni sus direcciones MAC, ni sus rutas. Eso es un requisito de seguridad y de contabilidad que una red de Ethernet compartida **no** da. SRv6 quiere reproducir eso, y lo está haciendo.

1.7 Fragmentación transparente vs. no transparente

Este tema es de la **capa de red** pero va acá porque es el otro uso histórico del control de caudal (es el caso donde el router tiene que decidir qué hacer con un paquete que no entra). Es la **Pregunta 4 del Examen 2007-12** (10 pts).

```
EL PROBLEMA
———————————

Un mensaje (o paquete) de tamaño N tiene que atravesar una cadena de
enlaces con MTU distinta. Si N > MTU de algún salto, no pasa.

La pregunta es: ¿dónde y cómo lo divido?
```

(a) Fragmentación transparente



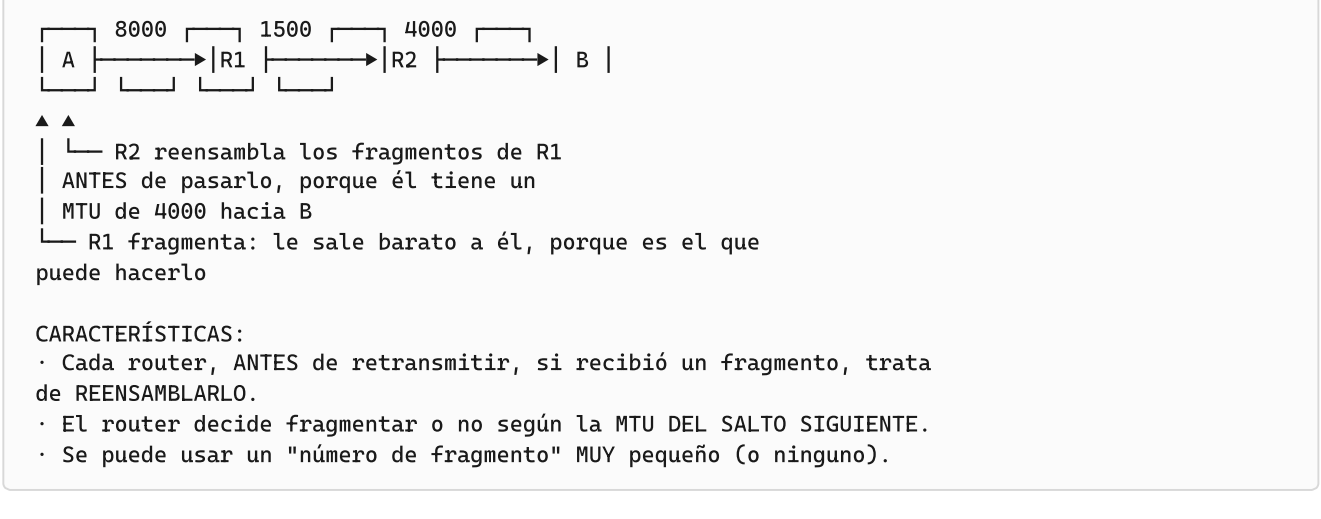
CARACTERÍSTICAS:
· Cada router FRAGMENTA por su cuenta.
· Cada fragmento lleva un NÚMERO DE FRAGMENTO mínimo
(número de secuencia), porque es un datagrama con posición.
· El receptor (o el ÚLTIMO router) REENSAMBLA.
· Los routers intermedios NO necesitan guardar estado.

Ventajas: - **Cero estado en los routers.** Cada router decide "por mí no pasa, fragmento", sin necesitar recordar qué estaba esperando. - **El receptor tiene el mensaje completo** (o al menos puede rearmarlo).

Desventajas: - **Overhead**: cada fragmento lleva un header (de capa 3 o de enlace) entero. Un mensaje de 8000 bytes que pasa por un enlace de MTU 1500 y que se fragmenta en cada salto puede acabar siendo **más caro que el mensaje original**. - **Se requiere buffering en el destino** para poder rearmar, y hay que saber cuándo recibir el último fragmento. - **Multiplicación de trabajo**: el mismo paquete se vuelve a fragmentar en cada salto donde no entra.

(b) Fragmentación no transparente

En la fragmentación NO transparente el router REENSAMBLA antes de
retransmitir al siguiente salto:



CARACTERÍSTICAS:
· Cada router, ANTES de retransmitir, si recibió un fragmento, trata
de REENSAMBLARLO.
· El router decide fragmentar o no según la MTU DEL SALTO SIGUIENTE.
· Se puede usar un "número de fragmento" MUY pequeño (o ninguno).

Ventajas: - El **mensaje entero** siempre está disponible en cada salto. - Si el siguiente salto tiene MTU grande, el router reensambla y **ahorra fragmentar** y **reduce el total de transmisiones**.

Desventaja CRÍTICA: - **Cada router tiene que BUFFERIZAR para poder reensamblar**, o sea, necesita **memoria dedicada y estado** por fragmento. Eso es exactamente lo que MPLS vino a evitar, y es el motivo de la de MPLS y de los routers de alta capacidad.

La comparación que hay que saber de memoria

	TRANSPARENTE	NO TRANSPARENTE
¿Quién fragmenta?	el router de cada salto	un router, de a uno
¿Quién reensambla?	el destino final	el siguiente router
Estado en routers	NINGUNO	hay que bufferizar
Overhead	ALTO (n° de fragmento)	BAJO
Coste de memoria	baja	ALTA
Enlace de MTU grande después	Uniforme requiere el mismo	no hace falta

---

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Leaky bucket con ráfaga de entrada

Datos. `b = 100 bytes`, `r = 10 bytes/s`. Llegan paquetes de **20 bytes**. La tasa de entrada es de **5 paquetes cada 100 ms** (o sea 50 paquetes/s = 1000 B/s).

a) ¿Cuántos paquetes entran en 1 segundo?

Paso 1 — tiempo de vaciado. `f = b / r = 100 / 10 = 10 s`. O sea que la cubeta tarda **10 segundos** en vaciarse. Es lentísima comparada con el período de llegada.

Paso 2 — simular segundo a segundo.

```
contenido inicial = 0 B

t entrada ¿cabe? contenido goteo (10 B/s)
————— ———————— ——————— ———————— ——————————————————
0.00 20 B (1) 20 ≤ 100 → 20 B −
0.10 20 B (2) 40 ≤ 100 → 40 B −
0.20 20 B (3) 60 ≤ 100 → 60 B −
0.30 20 B (4) 80 ≤ 100 → 80 B −
0.40 20 B (5) 100 ≤ 100 → 100 B −
————— la cubeta está LLENA a los 0,40 s —————
0.50 20 B (6) 120 > 100 → DESCARTA
0.60 20 B (7) DESCARTA
... ... (todo lo que llega hasta que gotee)
```

Paso 3 — sale. El goteo es de 10 B/s, o sea **1 paquete cada 2 s** (`20 B / 10 B/s = 2 s por paquete`). En **1 segundo salen 0 paquetes** (o 1 si justo en el borde).

Respuesta: en 1 s entran 5 paquetes y salen 0,5. Se descartan ~45 paquetes.

b) ¿Cuántos paquetes se necesitan para vaciar la cubeta?

```
100 B / 20 B por paquete = 5 paquetes
y como el goteo entrega 1 paquete cada 2 s ⇒ 5 × 2 = 10 segundos
(que es f = b/r = 10 s ✓)
```

2.2 Ejercicio 2 — Token bucket con tokens acumulados

Datos. `b = 50 tokens`, `r = 10 tokens/s`, paquetes de **10 bytes** cada uno (1 token por byte).

a) Llegan 8 paquetes de golpe a `t = 0`. ¿Cuántos salen?

```
tokens iniciales = b = 50 (cubeta LLENA)

t=0, P1: tokens 50 ≥ 10 → tokens = 40, SALE
t=0, P2: tokens 40 ≥ 10 → tokens = 30, SALE
t=0, P3: tokens 30 ≥ 10 → tokens = 20, SALE
t=0, P4: tokens 20 ≥ 10 → tokens = 10, SALE
t=0, P5: tokens 10 ≥ 10 → tokens = 0, SALE ← los 5 SALEN SEGUIDOS
t=0, P6: tokens 0 < 10 → ESPERA = ráfaga de 50 bytes
t=0, P7: espera
t=0, P8: espera

⇦ CONTRASTE CON LEAKY: los mismos 5 paquetes, la misma cubeta de 50,
con b=100 y r=10 el leaky los saca de a uno, cada 1 s. El token
los saca SEGUIDOS. ESO es la ráfaga.
```

b) ¿Cuándo sale P6?

```
Los tokens se generan a 10/s: 1 token cada 0,1 s.
Necesita 10 tokens ⇒ 10 × 0,1 s = 1,0 s.
P6 sale a t = 1,0 s (y P7 a 1,1 s, P8 a 1,2 s).
```

c) Con los mismos datos, un leaky bucket con `b=50, r=10`. ¿Cuándo sale P5?

```
f = b/r = 50/10 = 5 s para vaciar la cubeta.
Sale 1 paquete cada 10 B / 10 B/s = 1 s.
P1 a t=1,0 s ; P2 a 2,0 ; P3 a 3,0 ; P4 a 4,0 ; P5 a 5,0.
NUNCA dos seguidos. ← la diferencia de fondo
```

2.3 Ejercicio 3 — Qué DSCP le pongo a tres clases de tráfico

Datos. Una red de empresa con tres clases de tráfico: (1) **voz** sobre RTP; (2) **descarga de archivos** por FTP; (3) **backup** de disco de 3 TB por la noche.

```
╔═══════════╦═══════════════════════════════════════════════════════════╗
║ CLASE ║ DSCP ║ POR QUÉ ║
╠═══════════╬═══════════════════════════════════════════════════════════╣
║ 1. VOZ ║ EF (DSCP = 46 = 101110b) ║ No puede esperar. Ni ║
║ ║ ║ jitter ni retardo ║
║ ║ ║ tolerables. Se marca con ║
║ ║ ║ la MÁXIMA prioridad de ║
║ ║ ║ descarte: si hay ║
║ ║ ║ congestión, se tira lo ║
║ ║ ║ que sea antes que la voz.║
╠═══════════╬═══════════════════════════════════════════════════════════╣
║ 2. FTP ║ AF12 (DSCP = 12 = 001100b) ║ Interactivo, sensible a ║
║ descarga ║ ║ el retardo (el usuario ║
║ ║ ║ espera el archivo). ║
║ ║ ║ Preferencia 2: se tira ║
║ ║ ║ después que las pref. 1 ║
║ ║ ║ de la misma clase. ║
╠═══════════╬═══════════════════════════════════════════════════════════╣
║ 3. BACKUP ║ AF41 (DSCP = 34 = 100010b) ║ BULK: no importa nada ║
║ de 3 TB ║ ║ si tarda horas. Se ║
║ ║ ║ descarta de las primeras ║
║ ║ ║ (preferencia 1, la más ║
║ ║ ║ baja). Con policing en ║
║ ║ ║ el edge a 1 Mbps, el ║
║ ║ ║ resto se descarta o se ║
║ ║ ║ remarkea. ║
╚═══════════╩═══════════════════════════════════════════════════════════╝
```

¿Y por qué el backup NO va con DSCP 0 (Best Effort)?

```
Porque el backup, aunque sea bulk, va a COMPETIR con la web del
usuario por el mismo enlace de salida. Con DSCP 0, no tenés
ninguna prioridad garantizada. Con AF41 + policing en el borde,
el backup:
· tiene su propia categoría en la cola
· está explícitamente en la última posición de descarte
· si sobra banda, la usa (el token bucket le deja ráfagas)
· si falta banda, es el primero que se sacrifica

Ése es el DIFERENCIAL de DiffServ: no es "más rápido" para el
backup, es "no molestar al resto".
```

2.4 Ejercicio 4 — El flujo de un paquete por un LSP

Datos. Topología con tres routers MPLS:

```
MN (172.16.0.5) CN (203.0.113.9)
| |
▼ ▲
┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐
| PE-A | | P | | PE-B |
| Label Edge | | transit | | Label Edge |
| ingress |──┼── (swaps) |──┼── egress |
| etiqueta: 100 | | | | |
└──────────────────┘ └──────────────────┘ └──────────────────┘
FEC = 172.16.0.5 → 203.0.113.9 FEC = 203.0.113.9 →
172.16.0.5 (otro LSP)

Tablas:
PE-A: FEC(destino=203.0.113.9) → {salida: P, etiqueta: PUSH 100, TC: 5}
P : 100 → {salida: PE-B, etiqueta: SWAP 205, TC: 5}
PE-B: 205 → {salida: B, etiqueta: POP, quitar la cabecera MPLS}
```

El recorrido, con el cambio de etiqueta en cada router:

```
Paso 1 — MN envía el paquete IP
cabecera IP: src=172.16.0.5 dst=203.0.113.9 (etiqueta: NO HAY)

Paso 2 — PE-A (INGRESO, hace el PUSH)
· Mira la dirección IP de destino (203.0.113.9) para clasificar el
flujo en la FEC correspondiente
· Consulta su tabla: FEC → {salida: P, etiqueta: 100, TC: 5}
· EMPUJA la etiqueta 100 entre la cabecera IP y el payload
· Copia el TC=5 (que es el DSCP/prioridad) a la etiqueta

Ahora en el cable:
[IP: src=172.16.0.5 dst=203.0.113.9] [Label 100 | TC 5] [datos]

Paso 3 — P (TRÁNSITO, hace el SWAP)
· Mira SÓLO los 20 bits de la etiqueta: 100
· Búsqueda EXACTA: 100 → {salida: PE-B, etiqueta: 205}
· Cambia 100 por 205. NO toca la dirección IP. NO consulta longest
prefix match. NO sabe a dónde va el paquete realmente: no importa.

[IP: src=172.16.0.5 dst=203.0.113.9] [Label 205 | TC 5] [datos]
▲ ▲
nunca la mira cambió 100 → 205

Paso 4 — PE-B (EGRESO, hace el POP)
· Mira SÓLO la etiqueta: 205
· 205 → {acción: POP}
· QUITA la etiqueta MPLS entera

[IP: src=172.16.0.5 dst=203.0.113.9] [datos]
▲
ahora sí es un paquete IP normal

Paso 5 — B recibe el paquete IP y cre que nunca vio MPLS.

RESUMEN DE ETIQUETAS EN EL CABLE:
┌────────┬──────────┬────────┬──────────┬──────┐
| MN | PE-A | P | PE-B | B |
├────────┼──────────┼────────┼──────────┼──────┤
| (nada) | 100 | 205 | (pop) |(nada)|
| IP | IP | IP | IP | IP |
└────────┴──────────┴────────┴──────────┴──────┘
```

Interpretación — y esto es lo que hay que escribir: la etiqueta **cambia en cada salto**, pero **sólo cambia la etiqueta**. La cabecera IP viaja intacta de extremo a extremo sin que ningún router intermedio la mire. El único momento en que alguien mira la IP es el **ingress** (para clasificar) y el **egress** (que la necesita para entregarla). En el medio, **nadie sabe para dónde va el paquete**, y eso es exactamente lo que hace que MPLS sea rápido: la decisión se toma con 20 bits de búsqueda exacta en lugar de 32 bits de longest prefix match.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 Examen 2007-12, Pregunta 1 (10 puntos) — Leaky bucket y token bucket

> **Fuente:** Examen `eirc0712` / `sirc0712` — 27 de diciembre de 2007.
>
> **Enunciado:** Explique el algoritmo de **cubeta con goteo** (*Leaky Bucket*) y **cubeta con tokens** (*Token Bucket*).

(El enunciado no trae números: es puramente conceptual. Eso es lo que hay que resolver.)

La respuesta estructurada, que es la que hay que aprender

Leaky bucket (cubeta con goteo) — modelo:

```
Un paquete llega ⟶ [ CUBETA de capacidad b ] ⟶ sale a r bytes/s
▲ ▲
| |
entrada a ritmo el AGUJERO
ARBITRARIO gotea SIEMPRE
al mismo ritmo

· Si la cubeta tiene lugar (contenido + L ≤ b): el paquete ENTRA.
· Si NO tiene lugar: el paquete se DESCARTA (o se marca para otro
tratamiento: es la base del POLICING).
· El goteo NO SE DETIENE aunque la cubeta esté vacía: el paquete que
está en el fondo sale igual.

CONSECUENCIA: la salida tiene ritmo CONSTANTE. Es imposible que salgan
dos paquetes juntos.
```

Token bucket (cubeta con tokens) — modelo:

```
Un token se genera ⟶ [ CUBETA de capacidad b tokens ]
cada 1/r segundos ▲ |
(hasta b, no más) | ▼
un paquete de L bytes si hay ≥ L tokens:
CONSUME L tokens SALE
si no hay: ESPERA

· Los tokens se GENERAN a ritmo constante r (o 1 token por "unidad de
caudal"), con tope en b.
· Para enviar un paquete de L bytes hacen falta L tokens.
· Si no hay tokens: el paquete ESPERA (no se descarta).

CONSECUENCIA: la salida puede tener RÁFAGAS de hasta b bytes.
```

El cuadro comparativo, que es lo que el corrector quiere ver:

	LEAKY BUCKET	TOKEN BUCKET
Concepto	agua en una cubeta	fichas en una cubeta
Qué se genera	NADA	r fichas/s
Si no hay crédito	DESCARTA el paquete	el paquete ESPERA
Ritmo de salida	SIEMPRE constante	con ráfagas de hasta b
Ráfaga imposible	posible	
Usa toda la capacidad	no (se desperdicia si	sí
si sobra	la cubeta está vacía)	
Retrasa	sí (inherentemente,	sí (si no hay tokens)
	por el buffer)	
Uso típico	control de caudal	control de caudal
	"duro" / exacto	"blando" / con holgura

La frase de cierre que hay que escribir (es la que más puntos da):

> *"El leaky bucket produce un flujo de salida constante a costa de descartar todo lo que exceda; el token bucket permite ráfagas de hasta `b` bytes a costa de retardar los paquetes que no tienen tokens. Ambos limitan el caudal medio a `r`, pero el token bucket es el que además absorbe las ráfagas de entrada sin perder información."*

Y la justificación de por qué conviene cada uno (ojo: el enunciado no lo pide, pero si lo agregás suma):

- El **leaky bucket** sirve cuando **no se puede perder nada** (por ejemplo, un enlace de control donde cada paquete importa y hay que mandarlos a un ritmo constante).
- El **token bucket** sirve cuando la fuente tiene **ráfagas legítimas** que no querés perder (por ejemplo, el tráfico de una aplicación por ráfagas).

3.2 Examen 2009-03, Pregunta 4 (10 puntos) — DiffServ: edge, core y campo de marcado

> **Fuente:** Examen `eirc0903` / `sirc0903` — 20 de febrero de 2009.
>
> **Enunciado:** Considere la arquitectura de Servicios Diferenciados. **a)** Describa las funciones que debe realizar un "**edge router**" y un "**core router**", y qué campo de IPv4 se utiliza para marcar los tipos de servicio. **b)** Describa el mecanismo de forwarding "**Per Hop Behaviour**".

Solución oficial (textual):

> **a)** El "edge router" debe clasificar y marcar el tráfico que ingresa a la red. El "core router" debe encaminar el tráfico de acuerdo a las marcas definidas en el ingreso, y al PHB definido para cada clase de tráfico, usando el **campo TOS de IPv4**.
>
> **b)** El PHB son las políticas de tratamiento del tráfico de acuerdo a las marcas realizadas en el borde de la red. Por ejemplo prioridades, % del ancho de banda, etc.

La resolución completa

a) Las dos funciones y el campo.

```
EDGE ROUTER (router de borde) — hace TRES cosas:
──────────────────────────────────────────────────────────────────────

1. CLASIFICAR: determinar a qué CLASE de servicio pertenece el paquete.
Usa un clasificador: mirando (puerto, protocolo, dirección IP, longitud,
contenido). Típicamente es una tabla:
(IP origen, protocolo, puerto) ──► clase
Ejemplo: (RTP, puerto 16384-32767) → EF (voz)
(HTTP, puerto 80) → AF11 (web)
(todo lo demás) → 0 (best effort)

2. MARCAR: escribir el DSCP en el campo DS (byte TOS) del paquete.
Es el ÚNICO router que hace esto. A partir de ahí, la marca viaja sola
por la red.

3. (a veces) SHAPER o POLICER: entregar la clase con el caudal que la
política de esa clase permite.

──────────────────────────────────────────────────────────────────────
CORE ROUTER (router del núcleo) — hace UNA sola cosa:
──────────────────────────────────────────────────────────────────────
ENCAMINAR aplicando el PHB definido para la marca que viene en el paquete.

· Mira el byte DS.
· Mira su tabla de política: DSCP → {cola, prioridad de descarte,
ancho de banda mínimo, discard}
· Pone el paquete en la cola que corresponde, con la prioridad que
corresponde, y reenvía.
· NO clasifica. NO re-marca. NO sabe qué flujo es. NO guarda estado por
flujo. NO hay señalización.

──────────────────────────────────────────────────────────────────────
EL CAMPO DE IPv4: el byte TOS (Type of Service) — que HOY se llama
──────────────────────────────────────────────────────────────────────
el campo DS (Differentiated Services).
```

El layout del byte (esto hay que dibujarlo):

```
Byte TOS/DS de la cabecera IPv4 (offset 1), 8 bits:

┌──────────────────────────────┬──────────┐
│ DSCP (6 bits) │ ECN(2) │
├──────┬──────┬──────┬──────┬──────┬──────┼──────┬──────┤
│ b5 │ b4 │ b3 │ b2 │ b1 │b0 │ ECN │ ECN │
└──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┘
▲ ▲
└── 6 bits de DSCP: └── 2 bits de ECN (NO-ECN / AWARE / CE)

⚠️ El orden importa: DSCP primero, ECN después. El error típico es
invertirlos porque el layout "histórico" del byte TOS tenía la
precedencia (3 bits) al principio.
```

El nombre de los PHB que hay que saber (nombres + para qué):

PHB	DSCP	Significado
Default / BE	0	sin tratamiento especial
EF (*Expedited Forwarding*)	46	voz: mínima probabilidad de descarte y **no se encola detrás de nada**
AF11..AF43	10..38	4 clases (garantía de bw) × 3 drop precedences (qué se tira primero)
CS (*Class Selector*)	1xxxxxx	heritage de los valores viejos del byte TOS

b) El PHB (Per Hop Behaviour).

Definición precisa (es la que hay que escribir):

> **El PHB es el conjunto de políticas y reglas de tratamiento que un router aplica a los paquetes que llevan una determinada marca (un determinado DSCP), en CADA salto.** Es la que se define "salto a salto", sin que ningún router necesite saber nada del flujo, ni del camino, ni del destino final.

Lo que un PHB puede decidir (y estos son los ejemplos que da la solución oficial):

```
· A qué COLA va el paquete (prioridad entre colas)
· Qué PORCENTAJE del ancho de banda tiene garantizado cada clase
· Qué PRIORIDAD DE DESCARTE tiene: a quién se tira primero cuando
hay congestión
· Si puede usar el REBAJO DE ETIQUETA (no encolarse)
· Qué POLICING/SHAPING se le aplica

Ejemplo concreto con tres clases:

┌────────────┬────────────────┬─────────────┬──────────────────────┐
│ Clase │ bw mínima │ Cola │ Descarte │
├────────────┼────────────────┼─────────────┼──────────────────────┤
│ EF (voz) │ 2 Mbps │ 1 (la 1ª) │ NUNCA se descarta │
│ AF11(web) │ 4 Mbps │ 2 │ después de las AF │
│ Default │ el resto │ 3 │ se descarta primero│
└────────────┴────────────────┴─────────────┴──────────────────────┘

Eso es el PHB de AF11: "garantizame al menos 4 Mbps, mandame por la
cola 2, y si tengo que tirar algo, tirá lo de la cola 3 antes que
lo mío".
```

El contraste que hay que hacer al final (es lo que el enunciado busca):

```
"Per Hop" significa SALTO A SALTO:
· el router NO conoce el camino completo
· el router NO mantiene estado por flujo
· el router NO tiene quearse con los demás (no hay señalización)
· la política se aplica IGUAL en cada salto

⇒ por eso escala. Un router de backbone con DiffServ tiene un
clasificador con, digamos, 12 clases. No 1 millón de flujos.
```

> Tope para 10 puntos: (a) los tres pasos del **edge** (clasificar, marcar, y opcionalmente shaper/policer), la **una** función del **core** (encaminar aplicando el PHB de la marca), y el **nombre exacto del campo** (byte **TOS / DS**) con el layout de 6 + 2 bits. (b) la definición de PHB, los ejemplos de la solución oficial (prioridad, % de ancho de banda) y el contraste de por qué escala.

3.3 Examen 2010-07, Pregunta 4 (8 puntos) — El paradigma de DiffServ

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010.
>
> **Enunciado:** a) Explique el **paradigma de funcionamiento** de la arquitectura DiffServ. b) Mencione las **funciones que se encuentran en el edge** y las que se realizan en el **core** de dicha arquitectura.

Solución oficial: "*Diapos 83, 84 y 86 de Clase 2 de Multimedia*" — es decir, **no hay texto de solución publicada**: la cátedra se remitió a las transparencias del curso. Eso significa que **la respuesta esperada es la del teórico**, y hay que darla completa y bien estructurada. *(solución propia, reconstruida del material del curso)*

La resolución

a) El paradigma de DiffServ, en tres pasos.

```
EL PARADIGMA, en una frase:
"En lugar de reservar recursos para cada flujo (IntServ), se CLASIFICA
el tráfico en el borde en un número PEQUEÑO de clases, se MARCA con un
código de 6 bits que viaja en la cabecera IP, y cada router del núcleo
aplica un COMPORTAMIENTO POR SALTO (PHB) según esa marca, SIN
señalización y SIN guardar estado por flujo."
```

Los tres pasos con su nombre técnico (esto es lo que hay que escribir):

Paso	Nombre técnico	Quién lo hace	En qué consiste
1	**Classification** (clasificación)	el **edge router**	Mira el paquete (5-tupla, puertos, protocolo, longitud, contenido) y decide a qué **clase de servicio** pertenece, según una **tabla de clasificación**
2	**Marking** (marcado)	el **edge router**	Escribe el **código de 6 bits (DSCP)** en el **campo DS** del byte TOS de la cabecera IP
3	**PHB** (*Per Hop Behaviour*)	**cada** router del **core**	Aplica el comportamiento definido para ese DSCP: cola, prioridad de descarte, ancho de banda mínimo, policing/shaping

Las TRES propiedades que hacen que DiffServ escale (esto es lo que el enunciado realmente pregunta, porque pregunta por el "paradigma"):

```
┌──────────────────────────────────────────────────────────────────┐
| 1. SIN SEÑALIZACIÓN |
| No hay RSVP, no hay handshake, no hay setup. |
| El router del edge NO pregunta a nadie "podés reservarme |
| 2 Mbps?". Simplemente marca el paquete y lo manda. |
├──────────────────────────────────────────────────────────────────┤
| 2. SIN ESTADO POR FLUJO |
| El core NO guarda información por conexión. |
| Guarda UNA TABLA PEQUEÑA: DSCP → comportamiento. |
| Con 6 bits de DSCP hay, como máximo, 64 valores |
| (y en la práctica se usan 12 o menos). |
├──────────────────────────────────────────────────────────────────┤
| 3. CLASES, NO GARANTÍAS INDIVIDUALES |
| La garantía es "la clase AF11 tiene 4 Mbps garantizados", |
| no "este flujo tiene 4 Mbps garantizados". |
| Si dos flujos están en AF11, se reparten los 4 Mbps. |
| El enlace es COMPARTIDO, no reservado. |
└──────────────────────────────────────────────────────────────────┘
```

El contraste con IntServ, que hay que incluir porque es lo que el enunciado quiere que se entienda:

```
┌─────────────┬──────────────────────┬──────────────────────┐
| | IntServ | DiffServ |
├─────────────┼──────────────────────┼──────────────────────┤
| Granularidad| POR FLUJO | POR CLASE |
| Señalización| SÍ (RSVP) | NO |
| Estado | POR FLUJO, en | POR CLASE (12-ish) |
| | cada router | en cada router |
| Guarantee | ABSOLUTA | RELATIVA (dentro de |
| | (bw, retardo, | la clase) |
| | jitter, pérdida) | |
| Escala | NO | SÍ |
| Estado hoy | ≈ no se despliega | EL ESTÁNDAR |
└─────────────┴──────────────────────┴──────────────────────┘
```

b) Las funciones del edge y del core, en columnas.

```
┌──────────────────────────── EDGE ROUTER ────────────────────────────┐
│                                                                     │
│  · CLASIFICAR el tráfico (mirar 5-tupla, puertos, protocolo)        │
│  · MARCAR con el DSCP en el campo DS del byte TOS                   │
│  · (opcional) SHAPER para limitar el caudal de cada clase           │
│  · (opcional) POLICER lo que excede de un subscribed rate           │
│  · (opcional) RE-MARKING: si un paquete viene de adentro con        │
│  un DSCP que el cliente NO tiene derecho a usar, bajarlo            │
│  (ej: un usuario que pone EF en su VoIP hogareña)                   │
│                                                                     │
│  ⚠️ Es el ÚNICO lugar donde se toca la marca.                       │
└─────────────────────────────────────────────────────────────────────┘

┌──────────────────────────── CORE ROUTER ────────────────────────────┐
│                                                                     │
│  · LEER el DSCP del campo DS                                        │
│  · ENCAMINAR el paquete hacia el siguiente salto                    │
│  · APLICAR el PHB de ese DSCP:                                      │
│  - elegir la COLA de salida                                         │
│  - respetar el ancho de banda mínimo de la clase                    │
│  - respetar la prioridad de descarte de la clase                    │
│  - aplicar policing/shaping local si el admin lo configuró          │
│                                                                     │
│  ⚠️ NO clasifica. NO marca. NO re-marca. NO señaliza.               │
│  NO guarda estado por flujo.                                        │
└─────────────────────────────────────────────────────────────────────┘
```

El detalle fino que hace que DiffServ funcione, y que hay que decir:

> **"El core no necesita confiar en el edge."** O más precisamente: **la marca viaja dentro del paquete**, no en el estado de los routers. Eso significa que **cualquier router de la ruta puede leerla sin haber={""} nada con nadie. Y por eso el modelo es** extremadamente escalable**: el estado de un router depende del** número de clases**, que es constante, y** no** del número de flujos, que crece sin límite.

> Tope para 8 puntos: los tres pasos con su nombre (classification, marking, PHB), quién los hace (edge / edge / todos), las **tres propiedades** que lo hacen escalar (sin señalización, sin estado por flujo, clases en lugar de flujos), y la lista de funciones del edge y del core.

3.4 Examen 2007-12, Pregunta 4 (10 puntos) — Fragmentación transparente y no transparente

> **Fuente:** Examen `eirc0712` / `sirc0712` — 27 de diciembre de 2007. *(El PDF de este examen es un escaneo sin texto en el apartado de soluciones; el enunciado sí se lee. La resolución es propia, reconstruida del material del curso.)*
>
> **Enunciado:** Explique en qué consiste la **fragmentación transparente** y la **fragmentación no transparente**.

La resolución

El problema, primero (para ubicarse):

```
Un mensaje (o paquete) de tamaño N > MTU de algún salto. La MTU
(Maximum Transfer Unit) es el tamaño máximo de PDU de capa 3 que un
enlace transmite sin fragmentar.

┌─┐  8000  ┌──┐  1500  ┌──┐  1500  ┌──┐  4000  ┌─┐
│ A ├──────►│R1├──────►│R2├──────►│R3├──────►│ B │
└─┘        └──┘        └──┘        └──┘        └─┘

El mensaje tiene que pasar por enlaces de 8000, 1500, 1500 y 4000 bytes.
Los de 1500 no lo dejan pasar entero. La pregunta es: ¿QUIÉN lo parte y
CUÁNDO?
```

(a) Fragmentación transparente

```
Los routers INTERNOS (los de tránsito) NO se enteran de que hay
fragmentación. Cada router fragmenta lo que no le entra por el enlace de
SALIDA, y el DESTINO FINAL (o el último router) reensambla.

┌─┐  8000  ┌──┐  1500  ┌──┐  1500  ┌──┐  4000  ┌─┐
│ A ├──────►│R1├──────►│R2├──────►│R3├──────►│ B │
└─┘        └──┘        └──┘        └──┘        └─┘
✓ ▲ FRAG. ▲ FRAG. ▲ REENS. ▲
entero el 1ro el 2do aquí recibe
(N < 8000) que no que no (antes de el MENSAJE
entra entra entregar) ENTERO

· R1 ve que su salida es 1500 y su entrada le trajo 8000: fragmenta.
Cada fragmento lleva un NÚMERO DE FRAGMENTO (mínimo) en el header.
· R2 recibe un fragmento de 1500 que NO entra en su salida (1500): o
lo deja pasar (entra justo) o, si el mensaje era más grande, lo
vuelve a fragmentar.
· El ÚLTIMO router con MTU distinto (o el destino) REENSAMBLA: junta
los fragmentos por número, en el orden correcto, y reconstituye el
mensaje.
```

Características: - **Cada router fragmenta**, según su propia MTU de salida. Los routers no "coordinan": cada uno hace lo suyo. - **Los fragmentos llevan número** (o offset, en IPv4: `Fragment Offset`), porque son paquetes **independientes** que pueden perderse. - **El reensamblaje ocurre al final** (en el destino o en el último router). - **Los routers intermedios NO necesitan estado**: no saben de antemano que van a recibir fragmentos, no guardan nada. Cada uno ve un paquete más o menos grande y decide.

Ventajas: - **CERO estado en los routers intermedios.** Es la ventaja grande, y es la razón por la que existió: los routers de la época no tenían memoria para guardar reensamblajes parciales. - El **mensaje entero** le llega al destino. - Es **simple** de implementar.

Desventajas: - **Overhead grande**: cada fragmento lleva un header completo. Un mensaje de 8000 bytes que pasa por tres saltos de MTU 1500 puede terminar siendo, en el cable, **más caro que el mensaje original**: 6 fragmentos × 1500 bytes = 9000 bytes. - **Requiere buffering en el destino** (o en el último router) y saber cuándo llegó el último fragmento. - **Trabajo repetido**: el mismo mensaje se fragmenta y reensambla en cada salto distinto. - Si se pierde UN fragmento, se pierde el mensaje entero (hay que retransmitir todo).

(b) Fragmentación no transparente

```
En la fragmentación NO transparente, CADA ROUTER REENSAMBLA el mensaje
ANTES de retransmitirlo al siguiente salto. La fragmentación es
"opaca" para el siguiente salto.

┌─┐  8000  ┌─┐  1500  ┌─┐  4000  ┌─┐
│ A ├──────►│R1 ├──────►│R2 ├──────►│ B │
└─┘  └─┘  └─┘  └─┘
✓ ▲ FRAG. ▲ REENS. ▲
entero + REENS. + pasa recibe
(antes de ENTERO el mensaje
fragmentar)

· R1: le llega el mensaje entero de 8000. Su salida es 1500.
Decide: ¿le conviene más fragmentar acá y mandar 1500, o
reensamblar primero? Fragmenta y manda 6 fragmentos de 1500.
· R2: RECIBE los fragmentos, REENSAMBLA (los junta en el mensaje de
8000), y ANTES de pasarlo a B (que tiene MTU 4000) mira: entra
entero. Lo pasa entero. NO lo vuelve a fragmentar.

El punto: CADA router sabe si su salida aguanta el mensaje entero, y
decide en consecuencia.
```

Características: - El router **reensambla antes de retransmitir** al siguiente salto. - La decisión de fragmentar se toma **según la MTU del salto siguiente**, no según el mensaje que llegó. - Cada fragmento puede llevar **un número de fragmento mínimo** (o incluso nada), porque el reensamblaje ocurre en el router que fragmentó.

Ventajas: - **El mensaje entero siempre está disponible en cada salto** (o por lo menos en cada router que lo recibió fragmentado). - Si el siguiente salto tiene MTU grande, el router reensambla y **no vuelve a fragmentar**: se **ahorra transmisiones**. - **Overhead bajo**: si el mensaje se reensambla pronto, hay menos fragmentos circulando.

Desventajas CRÍTICA: - **Cada router que recibe fragmentos tiene que BUFFERIZAR** para poder reensamblar. O sea, **memoria dedicada por fragmento recibido**. Ése es el costo que **MPLS vino precisamente a eliminar**, y la razón por la que el hardware de alta capacidad hizo que los MPLS switches reemplazaran a los routers IP traditional. - Un mensaje que cruza muchos routers con MTU distinto **se reensambla y re-fragmenta muchas veces**, que es trabajo inútil.

La comparación, que es lo que hay que escribir al final

	TRANSPARENTE	NO TRANSPARENTE
¿Quién fragmenta?	el router, salto a salto	un router, y luego otro
¿Quién reensambla?	el DESTINO (o el último router)	el siguiente router (antes de retransmitir)
Estado en routers	NINGUNO	hay que BUFFERIZAR
Overhead	ALTO (n° de fragmento por cada salto)	BAJO (se reensambla pronto)
Coste de memoria	BAJA	ALTA
¿Cuándo decide?	cuando un paquete NO entra en su salida	cuando va a RETRANSMITIR al siguiente salto
MTU del camino	irrelevante (o casi)	importa mucho
Ejemplo típico	el "mini-packet" de la ATM ("virtual channels")	el tráfico entre routers con MTU distinto

Y el corolario histórico que es lo más importante de esta pregunta:

> La fragmentación **no transparente** es **precursora de MPLS**. MPLS es esencialmente la misma idea (la capa de red queda "opaca" para los routers intermedios) pero **sin fragmentar ni reensamblar nunca**: los routers MPLS reenvían por **etiqueta**, sin mirar la longitud del paquete, y sin necesidad de reensamblar. Por eso MPLS es **más eficiente que las dos**.

> Tope para 10 puntos: la definición de cada una, quién fragmenta y quién reensambla en cada caso, la **ventaja clave** de la transparente (cero estado) y la **desventaja clave** de la no transparente (buffering), y el corolario histórico con MPLS.

3.5 Examen 2007-12, Pregunta 5 (5 puntos) — MPLS: router de ingreso y de egreso

> **Fuente:** Examen `eirc0712` / `sirc0712` — 27 de diciembre de 2007.
>
> **Enunciado:** ¿Qué función cumplen el **router de ingreso** y el **egreso** en MPLS?

(La solución de este examen viene escaneada sin texto para esta pregunta; la resolución es la del material del curso, tal como la define `sirc0608 P5`, que sí está en texto. Ver §3.6.)

La respuesta

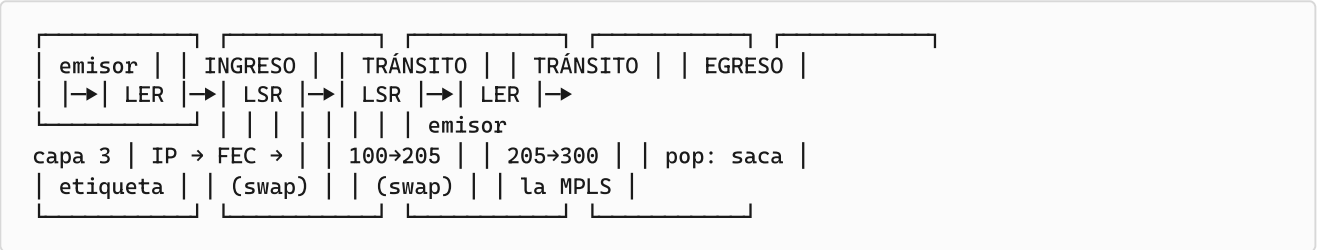
Router de INGRESO (*ingress*, también LER — *label edge router*):

```
· RECIBE los paquetes de la capa de red (IP, ATM, Ethernet…) que entran
al dominio MPLS por este router.
· HACE FORWARDING NORMAL: mira la dirección IP de destino, hace el
longest prefix match, y determina el siguiente salto.
· DEDUCE LA FEC (Forwarding Equivalence Class): agrupa los paquetes que
van a ser tratados igual. La FEC no es un paquete: es un
CONJUNTO de paquetes. Ejemplo: "todo lo que va del prefijo A al
prefijo B".
· ASIGNA LA ETIQUETA: consulta su tabla de LABEL INBINDING (que dice
"para la FEC F, la etiqueta que corresponde es X") y empuja la
etiqueta X (acción PUSH).
· (opcional) IMPONE la QoS,copiando el DSCP al campo TC de la etiqueta.

Es el ÚNICO router del LSP que mira la dirección IP de capa 3.
```

Router de EGRESO (*egress*, también LER):

```
· RECIBE los paquetes MPLS que llegan por el LSP.
· SACA la etiqueta MPLS (acción POP): quita la cabecera de etiqueta
completa.
· PONE la CABECERA DE CAPA 3 de abajo (la que el paquete venía
encapsulando).
· Entrega el paquete al siguiente router (o a la aplicación local).

Es el otro ÚNICO router que puede necesitar mirar la IP de capa 3.
```

Y el resumen del LSP (lo que hay que dibujar):



> **Tope para 5 puntos:** los **cinco verbos** del ingreso (**recibe, hace forwarding IP, deduce la FEC, asigna la etiqueta, la empuja**) y los **dos** del egreso (**saca la etiqueta, pone la cabecera de capa 3**), más el hecho de que **ninguno de los dos es un router de tránsito**.

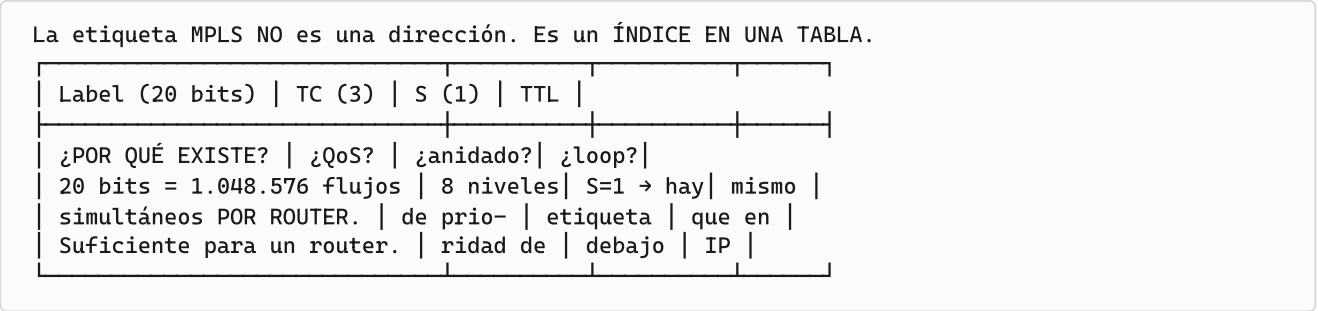
3.6 Examen 2006-08, Pregunta 5 (5 puntos) — MPLS: etiqueta, LSP y roles

> **Fuente:** Examen `eirc0608` / `sirc0608` — 5 de agosto de 2006.
>
> **Enunciado:** Describa los conceptos fundamentales de MPLS: la **etiqueta**, el **LSP** y los **diferentes roles** que cumplen los routers en el modelo MPLS.

Solución oficial (textual):

> **Etiqueta:** - **Label**: identificador de flujo, índice en tabla - **Experimental bits** (QoS): clase de servicio - **BS**: stacking de etiquetas - **TTL**: usado de la misma forma que en IP
>
> **Label Switched Path (LSP):** túnel unidireccional (simplex) a través de la red, formado por saltos controlados por las etiquetas.
>
> **Label Switch Router:** - **Ingress LSR** ("head-end LSR"): examina los paquetes IP entrantes y los clasifica asignándolos a una FEC; genera el encabezado MPLS y asigna la etiqueta inicial - **Transit LSR**: reenvía los paquetes MPLS usando la etiqueta y haciendo swapping - **Egress LSR** ("tail-end LSR"): remueve el encabezado MPLS

La resolución: por qué cada campo de la etiqueta existe



Explicación de cada uno (esto es lo que hay que developed):

1. **Label (20 bits).** Es un **número inventado por el router**, sin significado global. Cada router tiene su **propio** espacio de etiquetas y su **propia** tabla. Si el router de tránsito ve "etiqueta 100", no sabe nada del destino: sólo sabe "la 100 en MI tabla dice: mandá por la interfaz 3 con la etiqueta 205". Por eso 20 bits alcanzan:

son 1.048.576 valores, y un router de tránsito de backbone maneja del orden de cientos de miles. **El 20 no es arbitrario**: tiene que ser **único dentro de ese router** y tener un **bit de reserva** (20 = 16 bits de espacio + 4 bits para usos especiales como el *IPv4 Explicit NULL*).

2. **TC / Experimental bits (3 bits).** Es el **lugar donde vive el QoS de MPLS**. Hay tres nombres posibles para el mismo campo según el RFC que se use (`EXP` — experimental, `TC` — traffic class, `Precedence`): 8 niveles de prioridad de descarte. Es el **análogo exacto** del DSCP, pero dentro del shim MPLS, no dentro de la cabecera IP.
3. **BS (Bottom of Stack) / S.** Es un bit que dice "**hay otra etiqueta debajo de esta**". Sirve para el **LSP tunnel anidado** (*label stack*): cuando un LSP atraviesa la red de otro operador, se mete **una segunda etiqueta encima** (como una hoja de papel sobre otra). El SP bit le dice al router "cuándo: si S=0, esta es la última etiqueta; si S=1, hay más debajo y tenés que mirar la siguiente".
4. **TTL (8 bits).** Se usa **exactamente igual que en IP**: se decrementa en cada salto MPLS. Sirve para (a) **prevenir loops** y (b) que `traceroute` funcione a través de MPLS. La diferencia con el TTL de IP: el TTL de MPLS cuenta **saltos MPLS**, no todos los saltos de la ruta (puede haber saltos de IP por debajo de la pila de etiquetas que no se cuentan).

El LSP (Label Switched Path):

> "*Un túnel unidireccional (simplex) a través de la red, formado por saltos controlados por las etiquetas.*"

Desarrollado:

```
Un LSP es:
· UNIDIRECCIONAL (simplex). Para una comunicación bidireccional
hacen falta DOS LSP, uno en cada sentido.
· PREDEFINIDO (con la entrada/salida fija), a diferencia del
reenvío por IP que es "best effort" y puede cambiar de camino
en cada momento.
· Compuesto de saltos donde CADA SALTANDO se cambia la etiqueta
(swap), siguiendo la "tabla de reenvío por etiqueta".
· ⚠️ NO RESERVA RECURSOS. Esto es lo que más se confunde: un LSP
es un CAMINO, no un CIRCUITO VIRTUAL con garantías. Los recursos
los decide el router de INGRESO según la tabla de forwarding
(por eso MPLS se combina con DiffServ).
```

Los tres roles de LSR, y por qué el de tránsito es el que gana la velocidad:

```
┌──────────────┬──────────────────────┬────────────────────┬──────────────┐
│ INGRESS LSR  │ TRANSIT LSR          │ EGRESS LSR         │ ¿MIRA LA IP?│
├──────────────┼──────────────────────┼────────────────────┼──────────────┤
│ Examina los  │ Reenvía usando SÓLO  │ Remueve el         │ Ingress: SÍ  │
│ paquetes IP  │ la etiqueta, haciendo│ encabezado MPLS    │ Egress: SÍ   │
│ entrantes y  │ SWAPPING de etiqueta │ y pone el de capa  │ Transit: NO  │
│ los clasifica│ (label swap).        │ 3 de abajo.        │              │
│ asignándolos │ NO consulta la IP.   │                    │              │
│ a una FEC    │ NO consulta longest  │                    │              │
│ Genera el    │ prefix match.        │                    │              │
│ encabezado   │                      │                    │              │
│ MPLS y       │                      │                    │              │
│ asigna la    │                      │                    │              │
│ etiqueta     │                      │                    │              │
│ inicial      │                      │                    │              │
└──────────────┴──────────────────────┴────────────────────┴──────────────┘

⇦ LA VENTAJA DE MPLS, en una línea:
el transit LSR hace una BÚSQUEDA EXACTA de 20 bits (que el ASIC
puede hacer a 100s de Gbps) en vez de un LONGEST PREFIX MATCH de
32 bits en una TCAM (que es mucho más caro en silicio y).
```

Lo que hay que agregar y no está en la solución oficial de 2006 (y es el contenido de "Actualización a 2026"): MPLS **sigue desplegándose masivamente en los carriers**, pero su plano de control se está moviendo a **Segment Routing** (`RFC 8402`; SR-MPLS `RFC 8660`; SRv6 `RFC 8986`, que codifica los SIDs en la cabecera IPv6 y **elimina el MPLS por completo**). Y en datacenter, MPLS fue reemplazado por **EVPN-VXLAN** (`RFC 7432` + `RFC 7348`).

> Tope para 5 puntos: los **cuatro campos** de la etiqueta con su función (Label = índice en tabla, Experimental = QoS, BS = stacking, TTL = como en IP), la definición de **LSP** (túnel unidireccional de saltos controlados por etiquetas) y los **tres roles** con su verbo. Si además decís **por qué el de tránsito es el que da velocidad** (búsqueda exacta vs. longest prefix match), completaste.

PARTE 4 — CIERRE

📌 Resumen

1. **Internet es best effort**: reenvía lo que llega, descarta lo que no entra. Una videollamada y una descarga viajan igual. El problema real no es el ancho de banda sino el **buffer** de los routers (y el **bufferbloat**).
2. **Leaky bucket**: `b` = capacidad, `r` = tasa de salida. Si el paquete no entra, **se descarta**. Sale a ritmo **constante**: ráfagas imposibles.
3. **Token bucket**: los tokens se generan a `r` /s con tope `b`; un paquete consume `L` tokens. Si no hay, **espera**. Permite **ráfagas de hasta** `b`.
4. **La frase que hay que saber:** "*leaky = descarta, token = espera; leaky = constante, token = ráfaga*".
5. **Shaping** encola y retrasa (agrega latencia). **Policing** descarta o remarkea (no agrega latencia). La arquitectura estándar es **policing en la entrada, shaping en la salida**.
6. **IntServ** (`RFC 2211/2212/2213`): reserva por **flujo**, con **RSVP**, estado por flujo en cada router. Garantías reales, pero **no escala**. Hoy ≈ no se despliega (salvo RSVP-TE, `RFC 3209`, para traffic engineering de MPLS).
7. **DiffServ** (`RFC 2474` el campo DS, `RFC 2475` la arquitectura): clasifica en el **edge**, marca, y aplica el **PHB** en el **core**. Sin señalización, sin estado por flujo. **Es el que ganó.**
8. El campo de marcado es el **byte TOS / DS** de IPv4: **6 bits de DSCP + 2 bits de ECN**, en ese orden. PHB: **Default/BE**, **EF** (voz), **AF11..AF43**, **CS**.
9. **MPLS**: la **etiqueta** (20 bits de label, 3 de TC, 1 de S, 8 de TTL) es un **índice en la tabla del router**, no una dirección. El **LSP** es un camino predefinido **unidireccional** de saltos controlados por etiquetas.
10. Los **tres roles**: **ingress** (LER: mira la IP, deduce la FEC, empuja la etiqueta), **tránsito** (LSR: **sólo cambia etiquetas**, no mira la IP — ésa es la ganancia), **egreso** (LER: saca la etiqueta y pone la de capa 3).
11. **Fragmentación transparente**: cada router fragmenta, el destino reensambla, **cero estado**. **No transparente**: cada router reensambla antes de retransmitir, **hay que bufferizar**. MPLS es el sucesor eficiente de las dos.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Explicame la diferencia entre leaky bucket y token bucket en una frase, y decime cuál de los dos **descarta**.

<details>
<summary>Respuesta</summary>
</details>

2. Con un leaky bucket de `b = 100 bytes` y `r = 10 bytes/s`, ¿cuánto tarda la cubeta en vaciarse? ¿Y cada cuánto sale un paquete de 20 bytes?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué MPLS es **más rápido** que el forwarding por IP?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Qué tiene que hacer un **edge router** de DiffServ y qué tiene que hacer un **core router**? ¿Y qué campo de IPv4 se usa para marcar?

▶ Respuesta

5. Escribí el layout del byte DS y decime qué PHB le pondrías a una descarga de FTP y a una llamada de voz.

▶ Respuesta

6. ¿Qué diferencia hay entre **IntServ** y **DiffServ**, y por qué uno ganó?

▶ Respuesta

7. ¿Por qué la arquitectura estándar es "policing en la entrada y shaping en la salida"?

▶ Respuesta

8. En el ejercicio del LSP, ¿qué etiqueta lleva el paquete en cada tramo, y qué miró cada router?

▶ Respuesta

🔗 Referencias

- **Examen `eirc0712` / `sirc0712`** — P1 (10 pts, leaky bucket vs. token bucket), P4 (10 pts, fragmentación transparente vs. no transparente), P5 (5 pts, router de ingreso y egreso en MPLS), P3 (10 pts, conmutación, ya resuelto en la Clase 4)
- **Examen `eirc0903` / `sirc0903`** — P4 (10 pts, DiffServ: edge, core y campo de marcado)
- **Examen `eirc1007` / `sirc1007`** — P4 (8 pts, paradigma de DiffServ)
- **Examen `eirc0608` / `sirc0608`** — P5 (5 pts, etiqueta, LSP y roles de MPLS)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.6 (control de congestión y control de calidad de servicio) — *leaky bucket / token bucket*
- **RFC 2211 / 2212 / 2213** — IntServ (Integrated Services), 1997 · **RFC 3209** — RSVP-TE
- **RFC 2474** — el campo DS (Differentiated Services) · **RFC 2475** — arquitectura DiffServ
- **RFC 3246** — PHB EF (Expedited Forwarding) · **RFC 2597** — PHB AF (Assured Forwarding) · **RFC 4594** — guidelines de clases de servicio · **RFC 8622** — PHB LE (obsolesce RFC 3662)
- **RFC 3031** — MPLS architecture · **RFC 3032** — MPLS label stack · **RFC 4364** — MPLS VPN
- **RFC 8402** — Segment Routing (arquitectura) · **RFC 8660** — SR-MPLS · **RFC 8986** — SRv6 (elimina el MPLS)
- **RFC 7432** — EVPN · **RFC 7348** — VXLAN
- **Clase siguiente:** 46 — Gestión de redes: SNMP, NETCONF y YANG

Clase 46

Contenido de esta clase

- 🎯 Al terminar esta clase vas a poder
- 1.1 El problema: por qué `ping` no es gestión
- 1.2 El modelo SNMP: dos entidades y un idioma común
- 1.3 La MIB: un árbol con nombres numéricos
- 1.4 Pull y push: polling vs. traps
- 1.5 Las 5 operaciones
- 1.6 SNMPv3: USM y VACM
- 1.7 ⚠️ Actualización a 2026: NETCONF y YANG
- 2.1 Ejercicio 1 — Leer un OID y encontrar la variable
- 2.2 Ejercicio 2 —SNMPv3: el mensaje de una `GET` con USM
- 2.3 Ejercicio 3 — Calcular el coste del polling vs. los traps
- 2.4 Ejercicio 4 — Convertir un comando de CLI en NETCONF
- 2.5 Ejercicio 5 — Escribir una MIB mínima
- 2.6 Ejercicio 6 — SNMPv3 con VACM: dimensionar una vista
- 3.1 ⭐ Examen 2010-03, Pregunta 4 (8 puntos) — agente, gestor y MIB
- 3.2 Examen 2008-07, Pregunta 4 (4 puntos) — planos de datos y de control
- 📌 Resumen en 10 líneas
- ✅ Autoevaluación
- 🔗 Referencias

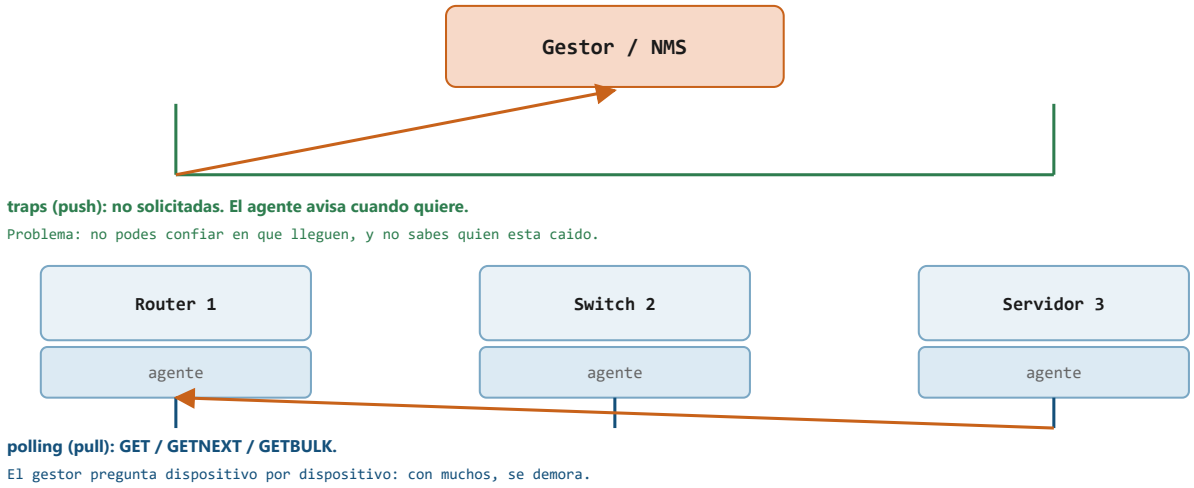
Bloque: F (Servicios y gestión) · **Semana:** 16 · **Duración estimada:** 120 min **Correlativa:** Clase 45 (QoS, shaping, MPLS) · **Sigue con:** Clase 47 (Seguridad) **Material de referencia:** Tanenbaum cap. 5 §5.5 (gestión de redes) · `ACTUALIZACIONES.md` §11

🎯 Al terminar esta clase vas a poder

1. Explicar por qué una red grande **no se puede administrar con** `ping`.
2. Distinguir **agente** de **gestor** y describir qué hace cada uno, con la asimetría entre "coleccionar" y "decidir".
3. Describir el modelo **MIB**: qué es un OID, por qué es un **árbol**, y por qué los OIDs no son nombres legibles.
4. Explicar **polling** y **traps**, con la comparación de coste que se pregunta en todos los exámenes.
5. Enumerar las **5 operaciones del SNMP** y decir para qué sirve cada una.
6. Explicar por qué **SNMPv1/v2c** con *community strings* es inaceptable y qué arregla **SNMPv3** (USM + VACM).
7. Describir **NETCONF** y **YANG** y decir **en qué se diferencian de SNMP** — la comparación que es el centro de la clase.

Modelo de gestion: gestor, agentes y MIB

El agente vive en el dispositivo gestionado y da acceso a variables de datos (OIDs jerarquicos).



> **Prerequisito del examen:** `2010-03 P4` (8 pts — agente, gestor y modelo MIB), `2008-07 P4` (4 pts — plano de datos vs. plano de control; en banda vs. fuera de banda), `2011-02 P1` (la gestión aparece como parte del escenario de TLS).

PARTE 1 — TEORÍA

1.1 El problema: por qué `ping` no es gestión

Imaginate la red de una universidad: 300 switches, 80 routers, 2 000 PCs, 40 APs. Son **~2 400 dispositivos**. ¿Cómo sabés si alguno está caído?

La respuesta ingenua —"hago `ping` a todos"— falla por tres razones que conviene saber enunciar:

Problema	Detalle
Escala	2 400 pings por minuto es 1 línea de terminal por dispositivo. No escala a nada.
No dice nada útil	`ping` dice "responde" o "no responde". No dice *cuánta* CPU usa, *cuánta* memoria, *qué interfaces están arriba*, *cuánto tráfico pasó por el puerto 7*.
No hay histórico	`ping` es instantáneo. Si el puerto 7 se congestionó **a las 3:14 y volvió a la normalidad a las 3:16**, nadie se entera salvo que alguien estuviera mirando.

La gestión de redes es la disciplina de responder tres preguntas sobre una red con cientos de dispositivos:

```
ESTADO      ¿qué está pasando AHORA?
HISTORIA    ¿qué pasó AYER a las 3:14?
CONTROL     ¿qué le MANDO que haga?
```

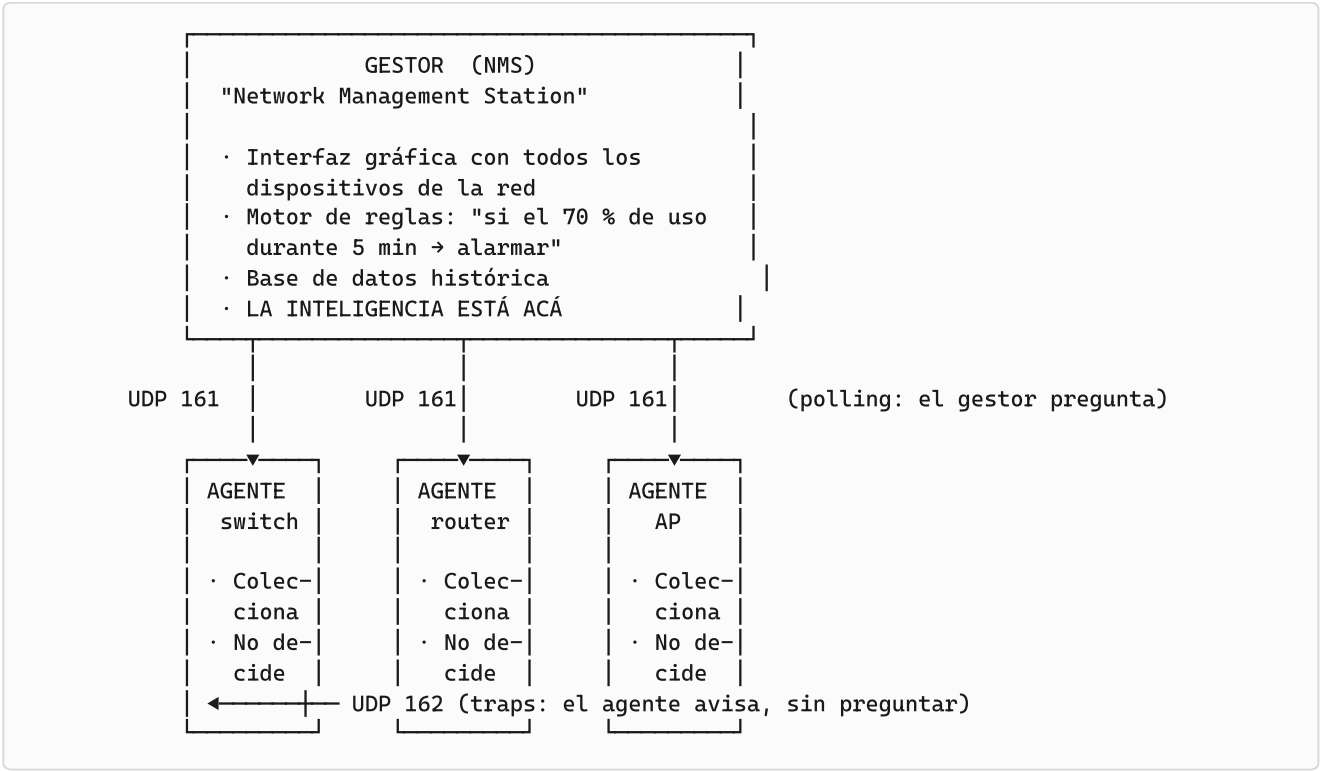
Y para eso hacen falta **cinco funciones** (RFC 1597, el modelo clásico):

```
┌──────────────────────────────────────────────────────────┐
│  1. FAILURE           detectar que algo falló            │
│  2. CONFIGURATION     cambiar la configuración a distancia │
│  3. ACCOUNTING         medir quién usó qué               │
│  4. PERFORMANCE        medir eficiencia y recursos       │
│  5. SECURITY           controlar quién puede hacer todo lo │
│                         anterior                         │
└──────────────────────────────────────────────────────────┘
```

> **La definición que hay que saber de memoria:** *la gestión de redes es el conjunto de protocolos y procedimientos que permiten observar, controlar y modificar una red de forma centralizada y estandarizada, con independencia del fabricante del equipo.*

1.2 El modelo SNMP: dos entidades y un idioma común

SNMP (*Simple Network Management Protocol*) es la respuesta. Y es una respuesta **deliberadamente simple**: un solo protocolo de aplicación que corre sobre **UDP**, normalmente en los puertos **161** (agente) y **162** (traps).



La asimetría fundamental (esto es lo que se pregunta)

El **agente** es un programa **pequeño** que vive **dentro** del equipo. Y esa es toda la historia:

El agente tiene recursos mínimos. Un switch de acceso de 24 puertos tiene procesador lento, memoria contada y flash de 8 MB. No puede correr una base de datos ni un motor de reglas. Solo puede:

1. **Recoger** valores de sus variables internas (contadores de tráfico, estado de puertos, temperatura, uso de CPU).
2. **Responder** lo que le pregunten por puerto.
3. **Generar notificaciones** (`traps`) cuando un umbral se supera.

El agente NO decide. No dice "la memoria está al 80 %, voy a bajar las peticiones de log". **Decide el gestor.** Y hay una excepción clásica que la cátedra menciona y vale la pena conocer: **RMON** (*Remote Network Monitoring*), donde el agente sí tiene lógica: guarda estadísticas históricas **en el propio dispositivo** para que no haya que traerlas. Es la excepción, y por eso la mencionan los exámenes.

El gestor es un monstruo de recursos. Corre en un servidor con CPU, disco y memoria de sobra. Ahí van **todos** los datos, **todos** los históricos y **toda** la lógica.

	Agente	Gestor (NMS)
Dónde corre	**dentro** del dispositivo	en un servidor central
Recursos	mínimos (CPU, RAM y disco chicos)	abundancia
Qué hace	**coleciona** y **responde**	**interpreta**, **decide**, **actúa**
Cuántos	**uno por dispositivo** (~2 400)	**uno para toda la red**
Almacena histórico	no (salvo RMON)	**sí**, es su razón de ser
Intelligencia	nula	**toda**
Fábrica	del fabricante del equipo (o del SO)	de terceros (Nagios, Zabbix, LibreNMS, SolarWinds, CiscoWorks)

> **La frase para el examen:** "*el agente está limitado por los recursos del dispositivo que administra; por eso su papel es mecánicamente el de collectar, y toda la inteligencia está en el gestor, que es quien centraliza, correlaciona y decide sobre miles de agentes a la vez.*"

El problema de la interoperabilidad

Los agentes los pone **cada fabricante**, y cada uno con su versión de firmware. Si el gestor no entiende el idioma de cada uno, no hay gestión. De ahí la necesidad de un **modelo de información estándar**: la **MIB**.

1.3 La MIB: un árbol con nombres numéricos

MIB = *Management Information Base*. Es **la base de datos** de lo que un dispositivo sabe de sí mismo.

```
AGENTE
  │
  │  "no sé lo que es 'ifInOctets', pero sí sé que
  │   tengo la variable número 1.3.6.1.2.1.2.2.1.10"
  ▼
MIB  ─── un archivo de texto, tipo ASN.1, que DEFINE
          la estructura del árbol
```

El punto clave y contraintuitivo: SNMP no manda el nombre, manda el número.

Cada variable de la MIB tiene un **OID** (*Object Identifier*): una **dirección jerárquica** en un árbol universal. El OID es una secuencia de enteros, y cada nivel es un *nodo* del árbol.

```
                iso (1)                          ← la raíz
                  |
               org (1.3)                        ← todos los-fire
                  |
              dod (1.3.6)                       ← Departamento de Defensa (USA)
                  |
              internet (1.3.6.1)                 ← ¡el mundo real empieza acá!
                  |
               mgmt (1.3.6.1.2)                 ← administración
                  |
          mib-2 (1.3.6.1.2.1)                   ← ★ MIB-II: todo IPv4
                  |
                  ├── 1  system ............... info del dispositivo
                  |         ├── 1  sysDescr        "Cisco WS-C2960..."
                  |         ├── 2  sysObjectID     el modelo
                  |         ├── 3  sysUpTime       segundos desde el boot
                  |         ├── 4  sysContact      quién lo administra
                  |         └── 5  sysName         "r1.miuy.edu.uy"
                  |
                  ├── 2  interfaces ........... todas las interfaces
                  |         └── 2.2.1 ifTable
                  |               └── 10  ifInOctets   ← bytes que entraron
                  |               └── 16  ifOutOctets  ← bytes que salieron
                  |
                  ├── 4  ip ................... todo IPv4
                  |         ├── 9  ipRouteTable     ← la tabla de ruteo
                  |         └── 26 ipNetToMediaTable ← la tabla ARP
                  |
                  └── 8  snmp ................. sobre SNMP mismo
```

El OID `1.3.6.1.2.1.2.2.1.10` significa: en el nodo raíz `iso`, bajá al hijo `org`, después a `dod`, después a `internet`, después a `mgmt`, después a `mib-2`, después a `interfaces`, después a `ifTable`, después al índice `10`, y tomá la variable `ifInOctets`.

El `1.3.6.1` de `internet` es el "cuello de botella" y hay que saberlo: es literalmente el prefijo de **todas** las MIB de Internet. Si tenés un OID que empieza por `1.3.6.1`, es estándar. Los DHCP (`1.3.6.1.2.1.58.4`) y los bridges (`1.3.6.1.2.1.17`) están colgando de ahí.

Y las empresas cuelgan de `1.3`:

```
1.3.6.1.4.1.<N>.<sub>       ← "empresa" (private enterprise)
    |   |   |
    |   |   └── sub-objeto definido por el fabricante
    |   └────── número de la empresa ( Cisco = 9,
    |           Juniper = 2636, HP = 11 )
    └────────── "private enterprise": todo lo no-estándar
```

> **El detalle fino:** las MIB de los fabricantes cuelgan de `private enterprise` porque **IANA reserva ese espacio**. Es lo que permite que un fabricante extienda el modelo sin romper el estándar. Y por eso las MIB de fabricante **no son portables**: el gestor necesita la MIB del fabricante instalada para poder traducir los OIDs a nombres.

El `ifIndex`, el detalle que se pregunta: el OID `...ifTable.10` termina en `.10` pero ese `10` es **la variable** (`ifInOctets`), no un índice. La tabla tiene **una fila por interfaz**, y cada fila se direcciona **agrego un número al final**: `...ifInOctets.1`, `...ifInOctets.2`, `...ifInOctets.3`. Ese número final es el **ifIndex**, y **cambia** si el SO renumera las interfaces. Es una fuente clásica de dolor de cabeza en el monitoreo.

1.4 Pull y push: polling vs. traps

Acá está **el concepto central** de toda la clase, y es la pregunta clásica:

```
POLLING  (pull, "tirón")          TRAPS  (push, "empujón")
─────────────────────────         ──────────────────────────
El GESTOR pregunta                El AGENTE avisa

GET / GETNEXT  ──► agente         ◄── TRAP   agente

· El gestor decide QUÉ y CUÁNDO   · El agente decide CUÁNDO avisar
· Simple, pero genera tráfico     · Reactivo: llega justo cuando pasa
  constante incluso sin cambios     algo
· Si el gestor está caído,          · Se pierde si el gestor está caído
  NO HAY NADA                       (no hay reintento)
· Retardo: hasta un período         · Retardo: casi cero
  completo
```

El coste del polling, que es el argumento que hay que dar

Si tenés **2 400 dispositivos** y hacés polling cada **60 segundos**,eso son:

```
2 400 dispositivos / 60 s = 40 consultas POR SEGUNDO
40 × 3600 = 144 000 consultas POR HORA
× 24 horas = 3 456 000 consultas POR DÍA
```

Tres millones y medio de paquetes al día, para preguntar "¿seguís vivo?" cuando nada cambió. Y encima: **si el gestor se cae, nadie se entera**, porque nadie pregunta. El fallo del monitoreo es silencioso.

El coste del trap

El trap es **una sola cosa**: como máximo **2 traps por segundo por agente**, y **solo si hay algo que reportar**. Es infinitamente más barato.

El problema del trap: es un **UDP**, o sea un datagrama **sin confirmar**. Si el gestor estaba caído cuando ocurrió el evento, **el trap se perdió para siempre**. No hay reintento, no hay cola.

> **La respuesta completa al examen** —*los traps no reemplazan al polling, lo complementan.* **El polling** es el que te dice cómo está la red **en general** y guarda el **histórico**; **el trap** es el que te dice **qué cambió ahora**, y como es *push*, es instantáneo y barato. La combinación clásica es: **polling lento** (cada 5 minutos, para tener la línea de base y detectar dispositivos muertos) **+ traps** (para reaccionar a los eventos). Si hacés polling cada 10 segundos para "no perderte nada", estás pagando caro; y si hacés solo traps, **no detectás un equipo que se apagó** (no hay trap de "me apagé", porque el agente ya está muerto).*

1.5 Las 5 operaciones

SNMPv1/v2c, que es lo que se pregunta:

#	Operación	Tipo	Qué hace
1	**GET**	**pull**	leer **un** OID exacto
2	**GETNEXT**	**pull**	leer **el siguiente** OID del árbol a partir de uno
3	**GETBULK** (v2c)	**pull**	leer **muchos** OIDs de una vez (equivale a N GETNEXT, pero en **un solo** datagrama)
4	**SET**	**push desde el gestor**	**escribir** un valor → es la única forma de **configurar** por SNMP
5	**TRAP**	**push desde el agente**	el agente avisa de un evento

El detalle de `GETNEXT` que hay que entender bien, porque es la clave de cómo funciona el monitoreo:

```
GETNEXT(...ifInOctets)  →   ¿cuál es el OID siguiente en el árbol?

El gestor NO sabe de antemano qué interfaces existen.
Entonces va "recorriendo" el árbol:

  GETNEXT(.1.3.6.1.2.1.2.2.1.10)        =  .1.3.6.1.2.1.2.2.1.10.1  ← if #1
  GETNEXT(.1.3.6.1.2.1.2.2.1.10.1)      =  .1.3.6.1.2.1.2.2.1.10.2  ← if #2
  GETNEXT(.1.3.6.1.2.1.2.2.1.10.2)      =  .1.3.6.1.2.1.2.2.1.10.3  ← if #3
  GETNEXT(.1.3.6.1.2.1.2.2.1.10.3)      =  .1.3.6.1.2.1.2.2.1.16.1  ← ifOutOctets #1
                                              ↑ se pasó a la SIGUIENTE variable
  ...
  GETNEXT(último OID conocido)          =  ERROR  (finSubId)
```

O sea: `GETNEXT` es un puntero que recorre el árbol en orden alfabético-numérico. Y `GETBULK` es el mismo recorrido, pero **N de ellos en un solo datagrama** — que existe justamente porque con 24 puertos y 30 variables son **720 OIDs** por dispositivo, y hacerlos de a uno sería insostenible.

Y la `community string`: en SNMPv1 y v2c, el "autenticado" es una cadena de texto plana — `public` o `private` — que viaja **en claro** en cada paquete. No es cifrado, no es autenticación real, es una contraseña de texto. Es exactamente el punto de entrada de la clase 47.

1.6 SNMPv3: USM y VACM

SNMPv3 no agrega una operación: **rediseñó la seguridad y la autorización**. Y tiene exactamente **dos mecanismos**:

USM — *User-based Security Model*

Responde a **"¿quién sos y podés mandar esto?"**:

Pieza	Qué hace
Autenticación	HMAC sobre el mensaje: si alguien modifica un paquete en el camino, el HMAC no da más y el receptor lo rechaza
Cifrado	el contenido va cifrado (DES clásico, hoy AES-192/256)
No repudio	digest de clave: el emisor manda un hash de su clave para que el receptor verifique que la clave es la misma

Los algoritmos los define una OID que **el propio mensaje declara**:

```
msgAuthoritativeEngineID    → quién es la autoridad
msgUserName                 → el usuario
msgAuthenticationParameters → el HMAC (autenticación)
msgPrivacyParameters        → el vector de cifrado
msgSecurityParameters       → todo lo anterior, con longitudes
```

El detalle fino que vale la pena: **el receptor tiene que responder con su propio parámetro de seguridad, no con el del emisor**. Por eso el OID de algorithms no va en la respuesta: si fuera, un atacante podría hacer que el receptor use un algoritmo débil. Es un detalle de diseño elegante que muestra que el protocolo está pensado contra un atacante activo.

VACM — *View-based Access Control Model*

Responde a **"¿qué podés ver y hacer?"**. Es un **control de acceso** por **vistas** sobre la MIB:

```
┌──────────────────────────────────────────────────┐
│  Vista "solo-interfaces"                         │
│   si     ifview    = 1.3.6.1.2.1.2.2.*  (ifTable)    │
│    permitir GET y GETNEXT                        │
│    NO permitir SET                               │
├──────────────────────────────────────────────────┤
│  Vista "admin-completo"                          │
│    permitir TODO sobre 1.3.6.1.2.1.*             │
│    INCLUDING la community "escritura"            │
└──────────────────────────────────────────────────┘

El usuario "readonly"   →  vista "solo-interfaces"
El usuario "admin-red"  →  vista "admin-completo"
El usuario "sensor"     →  sin acceso a ipRouteTable (que
                           es información sensible)
```

La idea clave: es **principio de mínimo privilegio aplicado a la MIB**. Un sensor térmico no necesita ver la tabla de ruteo, y no por cortesía: la tabla de ruteo es **información muy útil para un atacante**. Separar vistas **reduce la superficie de ataque**.

1.7 ⚠️ Actualización a 2026: NETCONF y YANG

Acá viene la parte que cambia el cuadro, y es importante ser honesto con ella.

Qué es NETCONF

NETCONF (`RFC 6241` , 2011) es un protocolo de **configuración remota** sobre:

- **SSH** como transporte (puerto **830**) → **autenticado y cifrado de fábrica**, sin *community strings* en claro.
- **XML** como codificación.
- **RPCs explícitos**, no GET/SET genéricos.

```
SNMP v2c                          NETCONF
────────                          ────────
GET   / SET   (genéricos)         <get>  <get-config>  <edit-config>
                                  <copy-config>  <delete-config>
                                  <lock>  <unlock>  <commit>  <commit-discard>
No transaccional                   <edit-config> + <commit> = TRANSACCIONAL
No hay "candidato"                 datastores: running / candidate / startup
Semántica defined por el agente    semántica DEFINIDA POR EL MODELO (YANG)
```

Los 3 datastores son la diferencia operacional más importante:

```
startup  ── la config con que arrancó el equipo (de fábrica o de un respaldo)
   │
   ▼
candidate ── un BORRADOR. Podés editarlo, validarlo y rechazarlo
   │          sin tocar nada que esté corriendo.
   │          ★ Esto es lo que permite "aplicar" una config de a un
   │            solo paso, y si falla, no queda nada a medias.
   ▼
running ── la config REALMENTE en ejecución ahora

<commit>            candidate  →  running   (con validación previa)
<commit-discard>    <aborta todos los cambios no commiteados>
<lock> / <unlock>   evita que dos administradores editen a la vez
```

> **El `lock` es la clave de por qué existe la semántica transaccional**: en SNMP, un `SET` es un `SET`; si mandás 20 SET y el 15º falla, **los 14 anteriores ya están aplicados** y el dispositivo quedó en un estado intermedio que nadie pidió. Con NETCONF, los 20 van al `candidate`, se validan **todos juntos**, y recién después el `commit` los aplica. **O se aplican todos o ninguno.**

Qué es YANG

YANG (`RFC 7950`, 2016, *Internet Standard*) es el **lenguaje de modelado** de la configuración. Se escribe en YANG y se **compila** a un schema XML que dice exactamente qué campos existen, qué tipos tienen, cuáles son obligatorios y cuáles pueden repetirse.

```
Los módulos YANG de un fabricante:

  ietf-interfaces      → interfaces, susounteres, sus estados
  ietf-ip              → direcciones IPv4
  ietf-routing         → tabla de rutas estáticas
  ietf-system          → hostname,Clock, contactos
  cisco-ios-xe-*      → las extensiones del fabricante
```

La diferencia conceptual que hay que decir en el examen:

> SNMP **no define qué significa la información**: el agente publica una MIB y el gestor la interpreta. El **contenido semántico lo pone el fabricante** (o sea, **cada familia de equipos tiene su propio dialecto**).
>
> YANG **sí define la semántica**: el modelo es público, está escrito en un lenguaje estándar y describe **cómo se llama cada campo, de qué tipo es y cómo se relaciona con los otros**. **No dependés del fabricante para entender la configuración.**

Esa es **toda** la diferencia en una frase: **SNMP describe el dispositivo, YANG describe el modelo del mundo**.

La comparación que resume la clase:

	SNMP	NETCONF + YANG
Para qué	**monitoreo** (métricas)	**configuración** y telemetría
Modelo	MIB: un **árbol de variables**	YANG: un **lenguaje de modelado**
Transporte	UDP 161/162	**SSH** 830 (+ RESTCONF 443)
Modelo de datos	define **el dispositivo**	define **el modelo**, no el dispositivo
Semántica	**la pone el fabricante**	**estándar y pública**
Operaciones	genéricas (`GET` , `SET`)	RPCs con **significado** (`<lock>`)
Transaccional	no	**sí** (`<commit>`)
Seguridad	community strings (v2c) / USM (v3)	**SSH**: autenticado y cifrado
Push	traps (poco confiable)	**telemetría** (gNMI, YANG-Push)
Push vs pull	predomina **pull**	**push** estructurado

Lo que dice la propia IETF, textual (y está muy bien que lo tengas)

El grupo de trabajo de NETCONF escribió en 2024:

> *"SNMP & MIBs nunca alcanzaron tracción significativa en la industria para configuración… esperamos una disminución significativa de despliegues de SNMP en los próximos 10 años."*

Y lo que pasó en la realidad:

```
SNMP   →  SIGUE SIENDO EL REY DEL POLLING DE MÉTRICAS
          (Nagios, Zabbix, LibreNMS, Prometheus+SNMP Exporter, todo
           el mundo monitorea con SNMP)

NETCONF/YANG →  EL REY DE LA CONFIGURACIÓN Y DE LA TELEMETRÍA
          (está en Cisco IOS XE/NX-OS, Juniper Junos, Arista EOS,
           Nokia SR OS, Huawei VRP)
```

El reencuadre honesto que hay que poder escribir en el examen:

> *SNMP no está muerto: está desplazado. Se inventó para gestionar, y cumplió; pero el mundo descubrió que la parte de* preguntar métricas *era la fácil y la parte de* configurar *era la que siempre estuvo caída, porque el modelo no era suficientemente rico. NETCONF+YANG no reemplazó a SNMP: le sacó la mitad difícil y le dejó la mitad que funciona bien.*

Los otros que hay que mencionar (para sumar puntos)

Protocolo	Para qué	RFC
gNMI (*gRPC Network Management Interface*)	telemetría **push** estructurada, de Google, en los vendors grandes	no-IETF (gRPC sobre HTTP/2)
BMP (*BGP Monitoring Protocol*)	monitorear **BGP desde afuera** de un router	`RFC 7854`
IPFIX	exportar **flujos** (Flow Records)	`RFC 7011`
NTSF (*Network Telemetry Framework*)	**SPDX** y modelo común de telemetría	`RFC 9232`
syslog	eventos; sigue vivo y se usa **mucho**	`RFC 5424` (transporte TLS: `RFC 5425`)
NETCONF SSH: transporte, `RFC 6242` · **RESTCONF**: HTTP, `RFC 8040`		

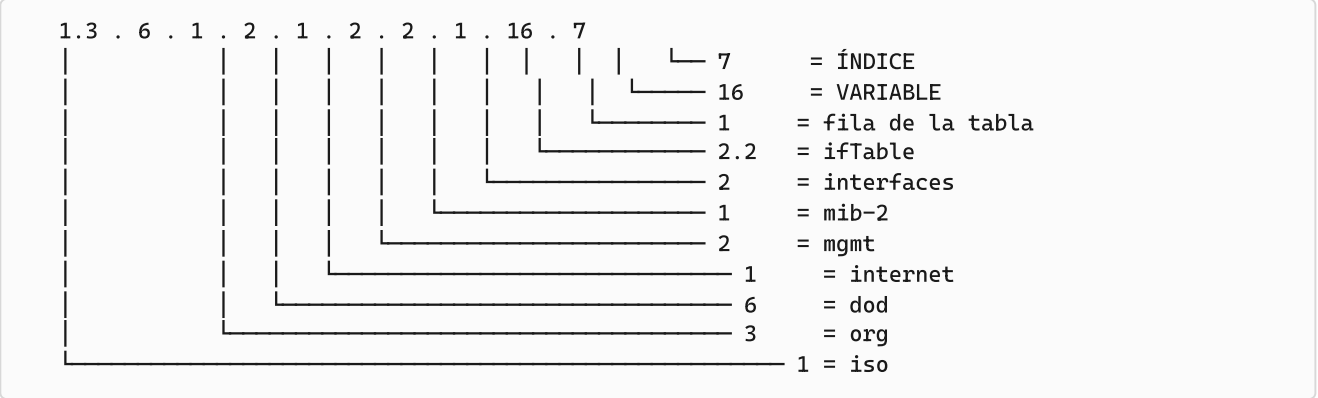
> **Un detalle que suma: NETCONF solo usa SSH**, pero **RESTCONF** (el mismo modelo YANG, sobre HTTP/HTTPS, puerto 443) es el que se usa para hablar con controladores que pasan por proxies y firewalls. Es el mismo modelo de datos, distinto transporte: **YANG es el modelo, NETCONF y RESTCONF son dosolateral**.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Leer un OID y encontrar la variable

Enunciado. Un gestor consulta `1.3.6.1.2.1.2.2.1.16.7`. Decí: ¿qué variable es, en qué tabla está, y qué interfaz representa?

Resolución paso a paso.



Parte	Significado
`1.3.6.1.2.1`	**MIB-II** (toda la gestión de Internet)
`.2`	el grupo `interfaces`
`.2.1`	la `ifTable`
`.16`	la **variable**: `ifOutOctets` — octetos **salientes** en la interfaz `ifIndex = 7`
`.7`	`ifIndex = 7`: la **séptima** interfaz del dispositivo

Interpretación: el gestor está leyendo **el total de bytes que la interfaz 7 mandó desde que arrancó**. Y como `ifOutOctets` es un **contador acumulativo de 32 bits que se desborda**, el valor **no esbytes por segundo**: es un **contador monótono creciente**.

Y acá está el error clásico que vale para el examen:

> ⚠️ **Un contador de 32 bits se desborda a los ~4 Gbytes.** Si el enlace lleva 1 Gbps, se desborda cada **32 segundos**. Por eso **nunca se consulta el valor y se lo interpola**: lo correcto es tomar **dos muestras separadas por un intervalo** y hacer la resta:
>
> `tasa (bytes/s) = (ifOutOctets₂ − ifOutOctets₁) / (t₂ − t₁)`
>
> Y si la diferencia da **negativa**, **no es un error de medición**: es el desbordamiento. Hay que sumarle `2³² = 4 294 967 296` al resultado.

2.2 Ejercicio 2 —SNMPv3: el mensaje de una `GET` con USM

Enunciado. Escribí los campos de seguridad de un `GET` de SNMPv3 cuando el **receptor** es la autoridad (es decir, el gestor le pregunta al agente).

Resolución. Con USM, el mensaje lleva estos campos **en el nivel de seguridad de SNMP**:

```
┌──────────────────────────────────────────────────────┐
│  msgVersion                  = 3                     │
│  msgGlobalData                                       │
│     msgID                     = n (número de transacción)│
│     msgMaxSize                = 65507                │
│     msgFlags                                         │
│        authFlag  = 1     → HAY autenticación         │
│        privFlag  = 1     → HAY cifrado               │
│        reportableFlag = 1                            │
│     msgSecurityModel         = 3  (USM)              │
│  msgSecurityParameters                               │
│     msgAuthoritativeEngineID = ID del receptor  ← ★  │
│     msgUserName               = "admin"              │
│     msgAuthenticationParameters  = HMAC-SHA-256-128(...)  │
│     msgPrivacyParameters         = AES-192-CFB(...)  │
│     msgSecurityParameters        = las longitudes de los  │
│                                    anteriores, codificadas  │
└──────────────────────────────────────────────────────┘
+ PDU: GetRequest con el OID pedido
```

Lo que hay que explicar de cada campo:

1. **`msgAuthoritativeEngineID` = ID del receptor.** Porque la autoridad en SNMPv3 es siempre **quien responde**: el gestor le pregunta al agente y el agente es la autoridad. Y el ID del motor lo **sincroniza el descubrimiento**: el primer mensaje es una **solicitud de descubrimiento**, sin parámetros de seguridad, y la respuesta trae el `engineID` y los **algorithms** disponibles.
2. **`msgFlags`: `authFlag` y `privFlag`.** Los dos pueden ir por separado. Con solo `authFlag` tenés **autenticación e integridad pero no confidencialidad**: el contenido se ve en claro pero nadie lo puede modificar sin que se detecte. Es una opción real para datos que no son secretos pero sí deben ser íntegros.
3. **El HMAC va sobre el mensaje entero**, con los parámetros de seguridad puestos a cero y con una **clave privada del receptor** derivada de la contraseña. Por eso un HMAC calculado con una **clave compartida** no es igual al cálculo con la contraseña directa: se deriva primero (`password-to-key`), por lo mismo que en WPA.

2.3 Ejercicio 3 — Calcular el coste del polling vs. los traps

Datos. Red con **300 switches, 80 routers y 2 AP** = **382 dispositivos**. Se hace polling con **25 OIDs** por dispositivo, cada **60 segundos**.

Resolución — el polling:

```
Consultas por ciclo  = 382 dispositivos × 25 OIDs = 9 550 GETs
Por segundo          = 9 550 / 60 s = 159 GET/s
Por hora             = 159 × 3 600 = 573 000 GETs
Por día              = 573 000 × 24 = 13 752 000 GETs

Coste por paquete: un GET con un OID son ~100 bytes de payload,
más ~40 de UDP/IP/Ethernet = ~140 bytes.

Volumen diario = 13 752 000 × 140 B = 1,925 GB/día
                ≈ 1,9 GB solo de SONDEO, todos los días.
```

Y con `GETBULK`, que es lo que se usa de verdad: `GETBULK` pide **25 OIDs en un paquete**. Entonces:

```
Paquetes por ciclo = 382
Por día            = 382 × 1 440 ciclos/día = 550 080 paquetes/día
Volumen            = 550 080 × ~1 800 B (25 OIDs) ≈ 990 MB/día

→ 25 veces MENOS tráfico, misma información. ★
```

Interpretación: ese es el motivo de ser de `GETBULK`, y por eso **existe en v2c y no en v1**: v1 era inmanejable y no se podía monitorear nada grande.

Los traps, en comparación:

```
Coste de un trap: ~1 paquete cuando HAY evento, nada cuando no hay.
Límite: 2 traps/s por agente.

El peor caso teórico: 382 × 2 = 764 traps/s. Pero eso es
un dispositivo que está fallando masivamente, que es exactamente
cuando QUERÉS que el sistema grite.
```

La conclusión del ejercicio: el polling es **predecible** pero **caro y ciego**; el trap es **barato y reactivo** pero **se pierde si el gestor está caído**.

2.4 Ejercicio 4 — Convertir un comando de CLI en NETCONF

Enunciado. El SNMP `SET` es inútil para configurar. Comparar: `snmp-server community public RO` (SNMP) vs. la misma intención en NETCONF.

Resolución — el mismo objetivo por dos caminos:

```
SNMP v2c                                NETCONF + YANG
────────                                ──────────────
snmp-server community public RO        system {      ← módulo YANG "ietf-system"
  (se ejecuta en la CLI del agente,        host-name "r1.miuy.edu.uy";
   NO ES UN GET/SET: es una instrucción   location    "Piso 3, Sala 302";
   de configuración del agente, escrita    contact     "noc@miuy.edu.uy";
   a mano por el admin)                  }
```

Resolución con el XML del `<edit-config>`:

```
<rpc message-id="101" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <edit-config>
    <target>
      <running/>
    </target>
    <default-operation>merge</default-operation>
    <config>
      <system xmlns="urn:ietf:params:xml:ns:yang:ietf-system">
        <host-name>r1.miuy.edu.uy</host-name>
        <location>Piso 3, Sala 302</location>
        <contact>noc@miuy.edu.uy</contact>
      </system>
    </config>
  </edit-config>
</rpc>

<rpc-reply message-id="101">
  <ok/>
</rpc-reply>
```

Y con datastores, que es la diferencia real:

```
<!-- 1. editar el BORRADOR, sin tocar nada en ejecución -->
<edit-config>
  <target><candidate/></target>
  <config> ... las 20 líneas nuevas ... </config>
</edit-config>

<!-- 2. VALIDAR sin aplicar: si algo está mal, NO cambió nada -->
<validate><source><candidate/></source></validate>

<!-- 3. si el validate da OK, aplicar de golpe -->
<commit/>
```

Lo que gained con esto: si la línea 15 de las 20 es inválida, con `commit` **no se aplica ninguna**; con 20 `SET` de SNMP **se aplicaron 14** y el equipo quedó con una configuración **mixta, a medias y no deseada**.

2.5 Ejercicio 5 — Escribir una MIB mínima

Enunciado. Escribí el fragmento ASN.1/SMIv2 de una MIB que defina una variable `turnosPendientes` que cuente los pedidos de turno en espera.

Resolución:

```
-- ================================================
-- MIB-MIYUY-MIB
-- ================================================
TURNOS-MIB DEFINITIONS ::= BEGIN

IMPORTS
    MODULE-IDENTITY, OBJECT-TYPE, Integer32, Counter32, TimeTicks
        FROM SNMPv2-SMI
    enterprises, MODULE-COMPLIANCE, OBJECT-GROUP
        FROM SNMPv2-SMI;

miyuyTurnos MODULE-IDENTITY
    LAST-UPDATED "202610010000Z"       -- formato AAAAMMDDHHMMSSZ
    ORGANIZATION "Facultad de Ingenieria, UdelaR"
    CONTACT-INFO "noc@miuy.edu.uy"
    DESCRIPTION  "Gestion de la cola de turnos de la cafeteria."
    ::= { enterprises 99999 }        -- 99999 = numero MI (declarar el propio)

miyuyTurnosMIBObjects  OBJECT IDENTIFIER ::= { miyuyTurnos 1 }

-- El objeto: un contador
turnosPendientes OBJECT-TYPE
    SYNTAX      Counter32          -- se incrementa, NUNCA baja
    UNITS       "turnos"           -- la unidad (conteo, bytes, segundos)
    MAX-ACCESS  read-only          -- no escribible desde afuera
    STATUS      current
    DESCRIPTION "Cantidad de personas esperando turno en este momento.
                 Es monotono creciente entre reinicios del agente:
                 cuando se llega a 2^32 el agente lo vuelve a cero."
    ::= { miyuyTurnosMIBObjects 1 }

-- El objeto: un estado
estadoTurnos OBJECT-TYPE
    SYNTAX      INTEGER {
                   normal(1),
                  IEF(2),    -- (con un typo, a proposito, para mostrarlo)
                   saturado(3),
                   cerrado(4)
                }
    MAX-ACCESS  read-write          -- este SÍ se puede cambiar
    STATUS      current
    DESCRIPTION "Estado operativo del sistema de turnos."
    ::= { miyuyTurnosMIBObjects 2 }

miyuyTurnosCompliance MODULE-COMPLIANCE
    STATUS current
    MODULE  -- este módulo
    ::= { miyuyTurnosMIBObjects 10 }

END
```

Lo que hay que mirar en ese fragmento, porque cada detalle es una pregunta de examen:

Detalle	Por qué importa
`LAST-UPDATED "202610010000Z"`	formato **obligatorio** `AAAAMMDDHHMMMMZ`; si no cambia, el OID **nunca** se puede reutilizar
`Counter32` vs. `Integer32`	`Counter32` **sube y se desborda**; `Integer32` **sube y baja**. Un contador de bytes **jamás** debe ser `Integer32`
`MAX-ACCESS read-only` vs. `read-write`	`read-write` **es una superficie de ataque**: cualquiera que pueda escribir puede **cambiar la configuración** del equipo por SNMP
`STATUS current`	`current` = vigente · `deprecated` = obsoleto pero se usa · `obsolete` = **no usar**
`::= { enterprises 99999 }`	hay que **registrar el número de empresa** ante IANA
El módulo `SNMPv2-SMI`	**SMIv2**; SMIv1 (RFC 1155) no tiene `MODULE-IDENTITY` ni `Counter32`

2.6 Ejercicio 6 — SNMPv3 con VACM: dimensionar una vista

Enunciado. En una red de la facultad hay tres perfiles de usuario: `sensor`, `operador` y `admin`. Definir las vistas.

Resolución — el razonamiento por perfil:

```
PERFIL     ¿QUÉ NECESITA VER?                  ¿QUÉ DEBE PODER HACER?
─────────  ──────────────────────────────────  ──────────────────────

sensor     temperature, estado de las           sólo GET / GETNEXT.
           interfaces, uso de puertos.          ← NUNCA SET.

operador   todo lo de sensor + tablas de        GET, GETNEXT, GETBULK
           ARP e interfaces; sysUpTime;         y SET **sólo sobre
           ipAddressTable de SU subred.         la subred propia.

admin      todo (1.3.6.1.2.1.*)                GET, SET, y también
                                                 `<lock>`/`<unlock>` para
                                                 no pisar a otro.
```

```
<!-- Vista del sensor: SOLO lectura, y SOLO las subárboles termometricos -->
<snmp-view view sensor-subtree included subtree 1.3.6.1.2.1.14.*/>
<snmp-view view if-subtree    included subtree 1.3.6.1.2.1.2.2.*/>

<access-group group sensor-ro version v3 security-level authPriv
               context "" match exact read-view sensor-subtree
               read-view if-subtree
               write-view ""                          ← vacío = no puede escribir
               notify-view "" />

<!-- Vista del operador: incluye la subnet de cada uno -->
<snmp-view view routing-subtree included subtree 1.3.6.1.2.1.4.*/>
<snmp-view view own-net       included subtree 1.3.6.1.2.1.4.20.1.<SUBNET>.0>

<!-- Vista del admin: todo -->
<snmp-view view all included subtree 1.3.6.1.2.1/>
```

La justificación que hay que dar (es lo que suma puntos):

1. **El `write-view ""` (vacío) del sensor es la decisión importante.** Sin eso, un sensor térmico compromise puede **cambiar la configuración del switch** y dejar la red abajo. Mínimo privilegio.

2. **Quitar `ipRouteTable` al sensor no es cortesía, es seguridad.** La tabla de ruteo dice **dónde está cada red**: es exactamente lo que un atacante necesita para pivotar. Lo mismo con `ipNetToMediaTable` (la tabla ARP).
3. **`authPriv` (autenticación + cifrado) y no solo `auth`.** Los datos de gestión son datos **sobre tu infraestructura**: si viajan en claro, cualquier sniffer los lee y además puede **inyectar respuestas falsas** (*spoofing*) y hacer que el gestor se cuelgue (el ataque más clásico: matar el SNMP y que nadie crea que la red está mal).

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-03, Pregunta 4 (8 puntos) — agente, gestor y MIB

> **Fuente:** Examen `sirc1003`, 24 de febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1003.pdf **Texto del examen:** *En el contexto de la gestión de las redes de computadoras, a) Explique los conceptos "agente" y "gestor". b) Describa brevemente el modelo de información MIB (Management Information Base).*

Resolución

a) Agente y gestor (4 puntos)

Son dos papeles del mismo modelo, y definirlos es la mitad de la pregunta.

El agente es la entidad de software **residente dentro del sistema que se gestiona** (el switch, el router, la AP, el servidor). Su trabajo es doble:

1. **Colectar** información de gestión del dispositivo donde vive. Y como vive adentro, tiene **recursos mínimos**: los de ese equipo. Un switch de acceso tiene CPU lenta y memoria contada, así que el agente está escrito para ser **pequeño**.
2. **Generar notificaciones** (*traps*, alarmas) cuando ocurre algo.

Y lo que **no** hace, que es la parte que hay que decir explícitamente porque es lo que distingue al agente del gestor: **no toma decisiones**. El agente puede coleccionar, pero **no decide** en función de lo que colecta. *(Excepción clásica: RMON, donde el agente guarda estadísticas históricas en el propio dispositivo para no traerlas.)*

El gestor (o **NMS**, *Network Management Station*) es la entidad que **recoge la información de todos los agentes** y es **la interfaz de usuario** del administrador. Sus rasgos:

- **Es la inteligencia de la gestión.** Correlaciona, decide y dispara acciones.
- **Hay un gestor para muchos agentes.** La relación es de **1 a N**: un gestor, 2 000 agentes.
- **Guarda histórico**, que es lo que le permite decir "el puerto 7 estaba al 95 % entre las 3 y las 4 AM del martes".

Y el rasgo que la cátedra menciona porque es lo que hace la gestión **real**: **interoperabilidad entre gestores**, o sea **gestión integrada y distribuida**. Cuando la red es tan grande que un solo gestor no alcanza, varios gestores se hablan entre sí (con SNMP y con peerings como BGP): uno queda a cargo del site A y otro del site B, y se contunden información. Hoy esto se llama **FedRAMP**... digo, **federación** y es la base de la gestión multi-tenant.

> **El paralelo con los routing protocols, que es lo que más suma:** es **exactamente** la relación entre un **protocolo de enrutamiento** y un **sistema autónomo**. El agente es como un router que **anuncia** su estado, y el gestor es como un router que **recibe anuncios y calcula** una tabla global. Y por eso aparecen

> los mismos problemas: **agente contra agente** (hay que autenticar, o el *spoofing* es trivial) y **gestor contra gestor** (hay que decidir quién manda).

b) El modelo MIB (4 puntos)

El problema que la MIB resuelve es la interoperabilidad, y esa es la primera frase de la respuesta:

> *El agente y el gestor pueden ser de diferentes proveedores y de diferentes versiones. Sin un modelo de información común y estandarizado, un gestor de un fabricante no podría entender lo que publica el agente de otro. Por eso hace falta un modelo de información de gestión estándar: ese modelo es la MIB.*

Cómo está construida:

1. **Todo recurso de red gestionable se representa como un objeto.** Y un objeto, en este contexto, es simplemente **una variable que contiene información del recurso**. El tipo de la variable depende del recurso (un contador de bytes, un estado, un nombre, una dirección).
2. **El conjunto de todas las variables conocidas por un agente es la MIB de ese agente.** Ojo con la frase: la MIB **es del agente**, no de la red. Cada agente implementa un subconjunto de la MIB-II según de qué unidad tiene.
3. **El gestor accede a los objetos presentes en la MIB de cada agente.** Y — la parte clave del punto (b) de la solución oficial — **lo que se puede hacer con cada agente depende de la MIB implementada por ese agente**: si el firmware no trae la MIB de BGP, el gestor **no tiene forma de saber** los indicadores de BGP, porque **no hay con qué preguntar**.
4. **Cada variable tiene un OID** (*Object Identifier*), que es una **dirección jerárquica** en un árbol. Los OIDs se asignan por la autoridad (**IANA**), que delega a los proveedores, que a su vez delegan a los fabricantes. De ahí la estructura: `1.3.6.1.2.1.*` es **MIB-II** (todo IPv4) y `1.3.6.1.4.1.*` es **empresa** (cada fabricante con su número).
5. **La MIB se escribe en ASN.1**, y el SNMP usa **SMIv2** (ya en desuso) o **SMIv2** (`SNMPv2-SMI`), que agrega `MODULE-IDENTITY`, `Counter32`, `Gauge32`, `TimeTicks` y los `MAX-ACCESS` con los valores correctos.

El detalle fino que suma puntos: la MIB **es un archivo de texto** (`.mib`, `.my`). **No se instala en el equipo.** El agente ya tiene compilados los valores de esas variables; la MIB es **la documentación que le sirve al gestor para saber qué números significan qué**. Por eso el mismo equipo con el mismo firmware puede monitorearse con cualquier gestor, siempre y cuando el gestor tenga la MIB cargada.

Lo que NO escribió la solución oficial y suma

La solución oficial es correcta pero se queda en "qué es". Para el examen completo agregá:

- **Que la MIB es un modelo de *variables, no de acciones***: describe el estado del dispositivo, no lo que puede hacer. Por eso configurar por SNMP es deuda: el `SET` existe, pero **no hay semántica de configuración**, no hay transacción y no hay validación. Ese es el motivo por el que existe NETCONF.
- **Que hay MIB-II (`1.3.6.1.2.1`) y las MIB propias de cada fabricante** (`1.3.6.1.4.1.<empresa>.*`), y que las de fabricante **no son portables**.
- **Que el OID se negocia con `GETNEXT`** y no se pide por nombre (ver §1.5).

3.2 Examen 2008-07, Pregunta 4 (4 puntos) — planos de datos y de control

> **Texto del examen:** *Compare el plano de datos y el plano de control, e indique si la comunicación de gestión se realiza en banda o fuera de banda.*

Resolución. Es la pregunta que más se olvida y es la base conceptual de toda esta clase.

```
┌──────────────────────────────────────────────────────────┐
│  PLANO DE DATOS                                          │
│  Los paquetes del usuario: IP, TCP, UDP, HTTP...         │
│  Es lo que el equipo hace PARA CARRear el tráfico.       │
│  Alta tasa, latencia baja, contenido variable.           │
├──────────────────────────────────────────────────────────┤
│  PLANO DE CONTROL                                        │
│  Los mensajes de gestión y de señalización:              │
│  ARP, ICMP, OSPF, BGP, SNMP, STP, DHCP, CDP, LLDP.       │
│  Es lo que el equipo hace PARA CONFIGURARSE y saber del  │
│  estado de la red.                                       │
│  Baja tasa, a veces con requisitos de tiempo.            │
└──────────────────────────────────────────────────────────┘
```

¿En banda o fuera de banda?

```
EN BANDA (in-band)                  FUERA DE BANDA (out-of-band)
──────────────────                  ────────────────────────────
El tráfico de gestión viaja         El tráfico de gestión viaja
por la MISMA red que el tráfico     por una RED SEPARADA, dedicada
de los usuarios.                    (o por un enlace de consola).

Se administra por SSH/telnet        Se administra por consola
desde cualquier lugar.              física (RJ-45), que funciona
                                    AUNQUE la red esté caída.

★ Si la red está caída,             ★ Si la red está caída,
NO se puede administrar.            SE PUEDE administrar igual.

Ejemplo: NOC compartido, acceso      Ejemplo: consola del rack,
por VPN a cualquier equipo.         El cable de consola del rack.
```

La respuesta completa: el tráfico de gestión **es del plano de control**, y **en banda o fuera de banda depende de la criticidad del activo**. En una red universitaria, el router del NOC y los switches de backbone **deberían** tener **fuera de banda** (consola + un puerto de gestión en una VLAN dedicada), porque si se cae el enlace principal y no tenés fuera de banda, **perdiste el control de la red justo cuando más lo necesitás**.

Por qué importa de verdad, y no es teórico: es el motivo por el que existe **VLAN de gestión**, **VLAN de voz**, y por qué los equipos de red tienen un **puerto de consola RJ-45** (que **no es un puerto de datos**: no lo conectes a un switch, porque no es un enlace Ethernet, es una consola asíncrona — y a veces es USB, no RJ-45, y el cable es **rollover**).

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. La gestión de redes existe porque `ping` **no escala**, no da **métricas** y no guarda **histórico**. Las cinco funciones: *failure, configuration, accounting, performance, security*.
2. **Agente** = programa chico que **dentro** del equipo **colecciona** y **responde**, pero **no decide**. **Gestor** = central, con recursos, **decide todo**. La asimetría es el corazón de la pregunta.
3. **MIB** = la base de datos de variables del agente, con **OIDs jerárquicos** (`1.3.6.1.2.1.*` = MIB-II; `1.3.6.1.4.1.*` = fabricante), escrita en **ASN.1/SMIv2**.
4. La MIB es **un archivo de texto que se instala en el gestor, no en el equipo**: el equipo ya tiene los valores, el gestor necesita saber qué significan.
5. **Polling** (pull: `GET`, `GETNEXT`, `GETBULK`) = caro, predecible, pero **muerto si el gestor se cae**. **Traps** (push) = barato, instantáneo, pero **se pierden si el gestor se cae**. **Se usan los dos.**
6. `GETNEXT` es **un puntero que recorre el árbol**; `GETBULK` es lo mismo, **N de ellos en un paquete** (reduce 25× el tráfico). Existe **por eso**, y es de v2c.
7. Un `Counter32` **se desborda**: la tasa es $(v_2 - v_1) / (t_2 - t_1)$, y si la resta es **negativa** hay que sumarle 2^{32}.
8. **SNMPv3 = USM (autenticación + cifrado, quién sos) + VACM (qué podés ver, mínimo privilegio por vistas)**.
9. **NETCONF = SSH + XML + RPCs con semántica + transacción (`candidate` → `commit`)**. **YANG** define el **modelo**, no el dispositivo: por eso se entiende sin importar el fabricante.
10. **SNMP no murió: se desplazó.** Sigue siendo el rey del **polling de métricas**; NETCONF/YANG es el rey de la **configuración** y de la **telemetría push** (gNMI, YANG-Push, RESTCONF).

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué un agente SNMP no puede tomar decisiones, y por qué el gestor sí?

<details>
<summary>Respuesta</summary>
</details>

2. Explicá por qué `GETNEXT` existe y qué problema resuelve.

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué un `Counter32` puede dar una resta negativa y qué hay que hacer?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué `write-view ""` para el sensor térmico? Explicá el razonamiento de seguridad.

<details>
<summary>Respuesta</summary>
</details>

5. Diferenciá SNMP y NETCONF en una frase cada uno, y decí cuál no reemplazó al otro y por qué.

▶ Respuesta

6. Un switch tiene un puerto en estado `down` y no hay traps. ¿Qué se deduce?

▶ Respuesta

🔗 Referencias

- **Examen** `sirc1003` — Pregunta 4 completa (8 pts: agente, gestor y modelo MIB)
- **Examen** `sirc0807` — Pregunta 4 (4 pts: planos de datos/control, en banda vs. fuera de banda)
- **Examen** `sirc0712` — Pregunta 5 (MPLS: router de ingreso y de egreso, otra forma de separar planos)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.5 (gestión de redes)
- `RFC 1157` — SNMPv1 · `RFC 3411` — SNMPv3 arquitectura · `RFC 3414` — USM · `RFC 3415` — VACM · `RFC 3416` — operaciones · `RFC 7860` — HMAC-SHA-2 en USM
- `RFC 6241` — NETCONF · `RFC 6242` — NETCONF sobre SSH · `RFC 7950` — YANG 1.1 · `RFC 8040` — RESTCONF · `RFC 8342` — NMDA · `RFC 8639` / `8640` / `8641` — YANG-Push · `RFC 7854` — BMP · `RFC 7011` — IPFIX · `RFC 9232` — Network Telemetry Framework
- `ACTUALIZACIONES.md` **§11** — el reencuadre SNMP vs. NETCONF/YANG
- **Clase siguiente:** 47 — Seguridad: servicios y criptografía

Clase 47

Contenido de esta clase

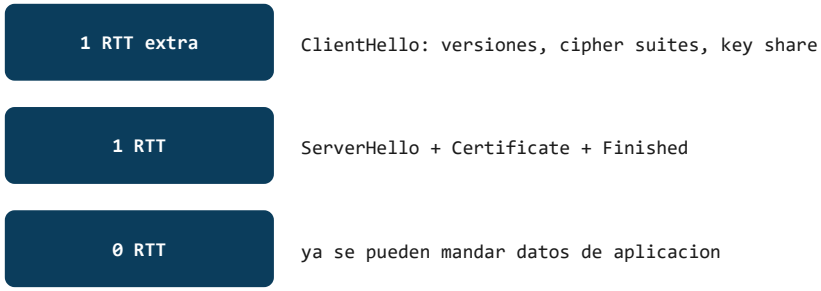
🎯 Al terminar esta clase vas a poder
1.1 Los cuatro servicios (y el confundón de siempre)
1.2 La función de hash
1.3 Cifrado simétrico
1.4 Cifrado asimétrico: RSA
1.5 El hash no firma solo: por qué
1.6 La firma digital, paso a paso
1.7 PKI: el problema de la confianza
1.8 ⚠️ Actualización a 2026
2.1 Ejercicio 1 — Qué servicio consigue cada técnica
2.2 Ejercicio 2 — La cadena completa de verificación de un certificado
2.3 Ejercicio 3 — Cifrado híbrido, números a la vista
2.4 Ejercicio 4 — Módulo RSA y cifrado/descifrado numérico
3.1 ⭐ Examen 2010-02, Pregunta 4 (8 puntos) — firma digital
3.2 Examen 2009-12, Pregunta 4 (6 puntos) — funciones de hash
3.3 Examen 2009-02, Pregunta 1 (9 puntos) — TLS
3.4 Examen 2008-12, Pregunta 2 (6 puntos) — hub vs. switch y seguridad
📌 Resumen en 10 líneas
✅ Autoevaluación
🔗 Referencias

Bloque: F (Servicios y gestión) · **Semana:** 16 · **Duración estimada:** 130 min **Correlativa:** Clase 46 (gestión de redes) · **Sigue con:** Clase 48 (seguridad aplicada) **Material de referencia:** Tanenbaum cap. 8 · `ACTUALIZACIONES.md` §13

🎯 Al terminar esta clase vas a poder

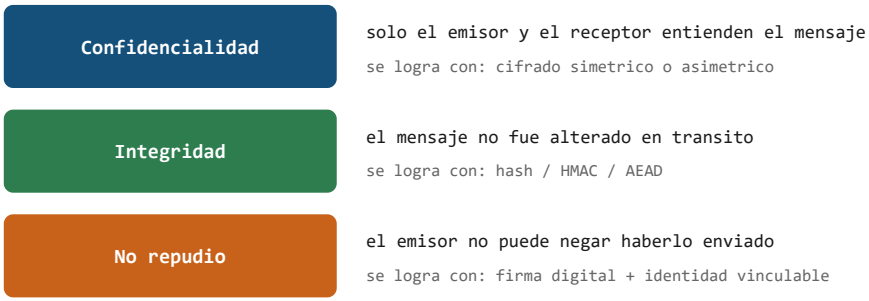
1. Distinguir **confidencialidad**, **integridad**, **autenticidad** y **no repudio**, y decir **qué técnica** consigue cada una y cuál **no**.
2. Explicar qué es una **función de hash**, enumerar sus **cuatro propiedades**, y decir por qué **el hash da integridad pero NO confidencialidad**.
3. Explicar por qué **MAC no sirve para firmar** y por qué **el hash solo tampoco**.
4. Describir **RSA** y la razón matemática que lo sostiene, y sus **tres limitaciones** (lento, tamaño, y no firma directamente).
5. Describir la **firma digital** paso a paso y explicar **por qué se firma el hash y no el mensaje**.
6. Describir una **PKI**: qué es un certificado, qué es una cadena de confianza, qué es el **ancla**, y **por qué todo el modelo descansa en un supuesto**.
7. Escribir el esquema **híbrido** de un canal seguro real (lo que hace TLS).

Handshake de TLS 1.3 en un RTT



TLS 1.3 cambio: solo AEAD, sin compresion, sin renegociacion, y forward secrecy obligatorio.
El certificado sigue siendo X.509; lo que cambio fue el intercambio de claves.

Los tres servicios de seguridad



La diferencia fina entre integridad y no repudio NO es el algoritmo.

Es QUIEN controla la clave privada: si la tiene solo Alicia, Alicia no puede negar.

Y los tres se resuelven juntos en TLS, no por separado.

Sumalos: autenticacion (soy quien digo ser) y control de acceso.

> **Prerequisito del examen:** `2010-02 P4` (8 pts — firma digital y por qué MAC no puede firmar), `2009-12 P4` (6 pts — funciones de hash y propiedades de seguridad), `2009-02 P1` (9 pts — TLS: qué clave va en el certificado, y cuál protege los datos), `2008-12 P2` (seguridad en switch vs. hub).

PARTE 1 — TEORÍA

1.1 Los cuatro servicios (y el confundón de siempre)

Antes de cualquier algoritmo hay que tener esta tabla **clara**, porque el 80 % de los errores en un examen salen de confundir los servicios.

Servicio	Pregunta que responde	Ejemplo
Confidencialidad	¿puede alguien **leerlo**?	cifrar una carta
Integridad	¿puede alguien **cambiarlo**?	el sello de lacre
Autenticidad	¿es **realmente de vos**?	tu firma manuscrita
No repudio	¿podés **probar** que fuiste vos?	firma notarial

Y las dos relaciones que hay que tener presentes porque son las que se preguntan:

```
"Si NO tengo confidencialidad, ¿puedo tener integridad?"

     SÍ. Ejemplo: un disco cifrado donde te avisan SÓLO si un archivo cambió
          (hash cifrado con HMAC). Vos no podés leer el contenido, pero
          sabés si alguien lo toco.
     ★ Por eso "confidencialidad" NO implica "integridad", ni al revés.
       Son servicios INDEPENDIENTES.

"Si tengo confidencialidad por cifrado SIMÉTRICO, ¿tengo autenticidad?"

     Depende. El cifrado simétrico por sí solo NO autentica al emisor:
     cualquiera que conozca la clave puede cifrar algo haciéndose pasar
     por vos. Para autenticar hace falta un MAC o un modo AEAD.
```

La taxonomía de las técnicas

```
┌──────────────────────────────────────────────────────────────┐
│  CIFRADO (en claro, con clave)                               │
│    simétrico   → misma clave para cifrar y descifrar         │
│                  (AES, ChaCha20)                             │
│    asimétrico  → par de claves: una pública, una privada     │
│                  (RSA, ECC / Ed25519)                        │
│                                                              │
│  HASH (sin clave, una sola vía)                              │
│    → NO cifra, NO tiene clave                                │
│    → detecta cambios (integridad), no oculta nada            │
│    → SHA-256                                                 │
│                                                              │
│  MAC (con clave simétrica, integridad + autenticación)       │
│    → HMAC-SHA-256                                            │
│    → detecta cambios Y sabe quién lo hizo, pero NO cifra     │
│                                                              │
│  FIRMA (asimétrica, integridad + autenticidad + no repudio)  │
│    → firma sobre el hash, con clave privada                  │
│    → RSA-PSS, ECDSA, Ed25519                                 │
│                                                              │
│  CERTIFICADOS / PKI                                          │
│    → vinculan una clave pública con una identidad            │
│    → X.509                                                   │
└──────────────────────────────────────────────────────────────┘
```

1.2 La función de hash

Función de hash = una función que toma cualquier mensaje (de cualquier tamaño) y devuelve una cadena **de tamaño fijo**.

```
m  ──────────►  H(m)  ──────────►  h
(cualquier          (la función)     (huella: siempre
 tamaño)                              32 bytes con SHA-256)

m = "hola"          →  h = 2cf24dba...
m = "hola "  (con   →  h = 5891b5b...   ← ¡distinto! Un bit cambia TODO
un espacio)
m = "holb"          →  h = aa4b0c95...      el hash.
```

Propiedades obligatorias (si una falla, la función no sirve para seguridad):

#	Propiedad	Qué dice	Por qué importa
1	**Determinista**	la misma entrada da **siempre** la misma salida	si no, no podrías verificar
2	**Unidireccional** *(preimage resistance)*	dado `h`, es **computacionalmente inviable** encontrar `m` tal que `H(m) = h`	sin esto, cualquiera que vea la huella puede **reconstruir el mensaje**
3	**Sin colisiones** *(second preimage)*	dado `m`, es inviable encontrar **otro** `m'` con `H(m') = H(m)`	sin esto, podés **reemplazar** el mensaje por otro con la misma huella
4	**Avalancha**	cambiar **1 bit** de la entrada cambia **~50 %** de los bits de la salida	hace imposible el ataque de **cota incremental**

Y dos propiedades "de facto" que se piden:

- **Longitud de salida fija y suficiente**: 256 bits con SHA-256. Con 128 bits (MD5, SHA-1) la probabilidad de **colisión** por fuerza bruta es $2^{-64} \approx 5\times10^{-20}$; con 256 bits es $2^{-128} \approx 3\times10^{-39}$.
- **Resistencia a preimagen de segundo orden**: no es lo mismo encontrar **una** colisión (cualquier par `m, m'`) que encontrar una colisión **para un** `m` **dado** (second-preimage). La segunda es más difícil, y SHA-256 resiste las dos.

El hash NO es cifrado. Es unidireccional: no hay "descifrar" un hash. Y **no lleva clave**: la misma función, para todos, siempre. Es por eso que un hash "público" sirve para cualquier cosa: no se puede usar para ocultar, solo para detectar.

Y ojo con el orden de la frase en un examen: "*un hash provee integridad pero no confidencialidad*". Si escribís al revés, es un error conceptual, no de redacción.

⚠️ MD5 y SHA-1: la historia que hay que saber

```
MD5      128 bits. ROTO desde 2004 (ataques de colisión hallados).
         En 2008, el "rogue CA": una CA legitimamente firmada un
         certificado con la MISMA huella que otro, porque MD5 ya era
         vulnerable. El sistema de confianza entero colapsó.
SHA-1    160 bits. ROTO en 2017 con SHAttered (2<sup>63</sup> intentos, 6500
         años de un core → 110 años con 6500 GPUs y 110 $/GPU-hora).
SHA-256  256 bits. SEGURO.
SHA-3    256/512. SEGURO, y con diseño diferente a SHA-2.

RFC 9155 (2021) → depreca explícitamente MD5 y SHA-1 en TLS 1.2
```

1.3 Cifrado simétrico

```
MENSAJE ──[ CIFRAR con k ──► CIFRADO ──[ DESCIFRAR con k ──► MENSAJE
               (la MISMA clave k)
```

Característica esencial: es rápido y hay que compartir la clave. Y compartir la clave es el problema: en un canal de N personas, hacen falta **N−1 claves**.

El problema de la distribución de claves es tan grave que la criptografía simétrica sola **no resuelve** nada en un canal abierto: si vos y yo queremos hablar y hay un tercero escuchando, tenemos que ponernos de acuerdo en la clave **sin que el tercero se entere**. Y ese es exactamente el problema que resuelve la criptografía asimétrica.

Los modos de operación: por qué importa

Cifrar "cada bloque por separado" con ECB es **inseguro**, y el motivo es importante para entender por qué:

```
ECB  (Electronic Codebook)
──────────────────────────

Bloque1 → C1
Bloque2 → C2          Los bloques IGUALES producen el MISMO
Bloque3 → C1          cifrado. Consecuencia: se ve el PATRÓN
Bloque4 → C2          del texto original a través del cifrado.
                     ★ Famous: la imagen del "dinosaurio".
```

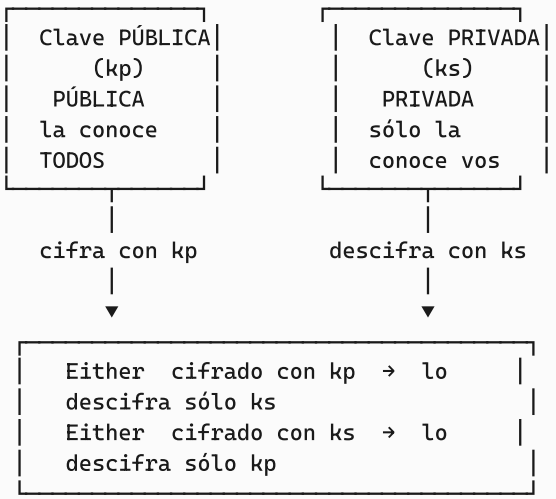
Las soluciones: **CBC** (cifrado en cadena), **CTR** (contador, como un cifrado de flujo), y el estándar moderno: **AEAD** (*Authenticated Encryption with Associated Data*), que **junta cifrado e integridad en una sola operación**:

```
AES-GCM          →  cifrado + integridad (y el tag de autenticación)
ChaCha20-Poly1305 →  lo mismo, más rápido sin acelerador de hardware
```

> **El AEAD es hoy el default** y la razón tiene nombre: **evita los ataques de manipulación**. Con "cifrar y después hashear por separado" (o peor, hashear por separado *antes* de cifrar) se abre la puerta a ataques de **cambio de bits** y a **padding oracle** (el famoso ataque de Vaudenay contra TLS). El AEAD calcula la autenticación **dentro** del cifrado, así que no hay forma de tocar el texto sin romper el tag.

1.4 Cifrado asimétrico: RSA

El cambio de paradigma: en vez de una clave, un **par**.



Lo que faltaba: la clave privada **nunca viaja**. Aunque alguien intercepte mil millones de mensajes, no puede sacar la clave privada **de los mensajes**, porque es un problema **matemático**, no de ingeniería.

La base matemática (lo mínimo que hay que saber)

RSA usa el problema de **factorización de enteros grandes**:

```
1. Se eligen DOS primos GRANDES:  p  y  q
2. n = p · q                                    ← el MÓDULO (es público)
3. ϕ(n) = (p−1)(q−1)                            ← función Euler
4. se elige e  tal que   gcd(e, ϕ(n)) = 1       ← EXPONENTE PÚBLICO
5. d  = e⁻¹ mod ϕ(n)                            ← EXPONENTE PRIVADO
6. El secreto es (p, q); el resto es público: (n, e)
```

Y funciona por el **teorema de Euler**:

```
Para cualquier m:      m^(e·d) ≡ m  (mod n)

O sea:  cifro con e, descifro con d → obtengo el mensaje original.
Y para romperlo tendría que factorizar n (o resolver el problema de
RSA directamente, que es equivalente a factorizar).
```

El problema de factorización es believed hard: hoy se han roto claves de **829 bits**, y las factorizaciones de RSA-2048 (que es lo que se usa) están fuera de alcance con la computación actual. **Pero — y esto es lo importante — no hay prueba de que sea hard**: nadie ha demostrado que factorizar sea difícil, solo que **parece**.

Y las tres limitaciones de RSA, que se preguntan siempre:

Limitación	Detalle
Es lento	Modular exponenciación: la operación cuesta **$O(k^3)$** en el tamaño de la clave. **AES va 100 a 1000 veces más rápido.**
Los datos son gigantes	Con claves de 2048 bits solo podés cifrar **245 bytes**. Para cifrar un archivo de 1 MB, lo dividís, cifrás cada parte y las concatenás (el esquema **híbrido**).
No cifra en bloque	RSA solo cifra bloques del tamaño del módulo. No es un cifrado de flujo ni de bloque utilizable para datos largos.

> **Y un cuarto problema, más moderno:** la firma directa con RSA (`PKCS#1 v1.5`) es **frágil** a ataques de Bleichenbacher. En TLS 1.3, el **transporte de clave con RSA fue directamente eliminado**: hoy solo queda **RSA-PSS**, que es el modo de firma moderno.

ECC y Ed25519

RSA **no es lo más seguro por bit**. Con la seguridad de 128 bits (la de AES-128):

```
RSA       necesita  ~3072 bits
ECC       necesita  ~256  bits       ← misma seguridad, clave 12× más corta
```

Ventajas de ECC en la práctica: claves más cortas = ** menos cómputo, menos bandwidth en el handshake, menos batería **(crítico en móviles y en IoT).** Ed25519 **es la curva más usada hoy (es el default de** SSH**, `RFC 8709`).

La tabla de los tres servicios que hay que saber:

Algoritmo	Para qué	Clave pública	Clave privada
RSA	cifrar, firmar	`(n, e)`	`d`
ECDSA / Ed25519	**firmar** (no cifra bien)	punto en la curva	escalar
ECDH / X25519	**acuerdo de clave**	punto en la curva	escalar

> **El matiz de examen:** ECDSA **firma**, no cifra; Diffie-Hellman **acuerda**, no cifra ni firma. Confundirlos es el error clásico.

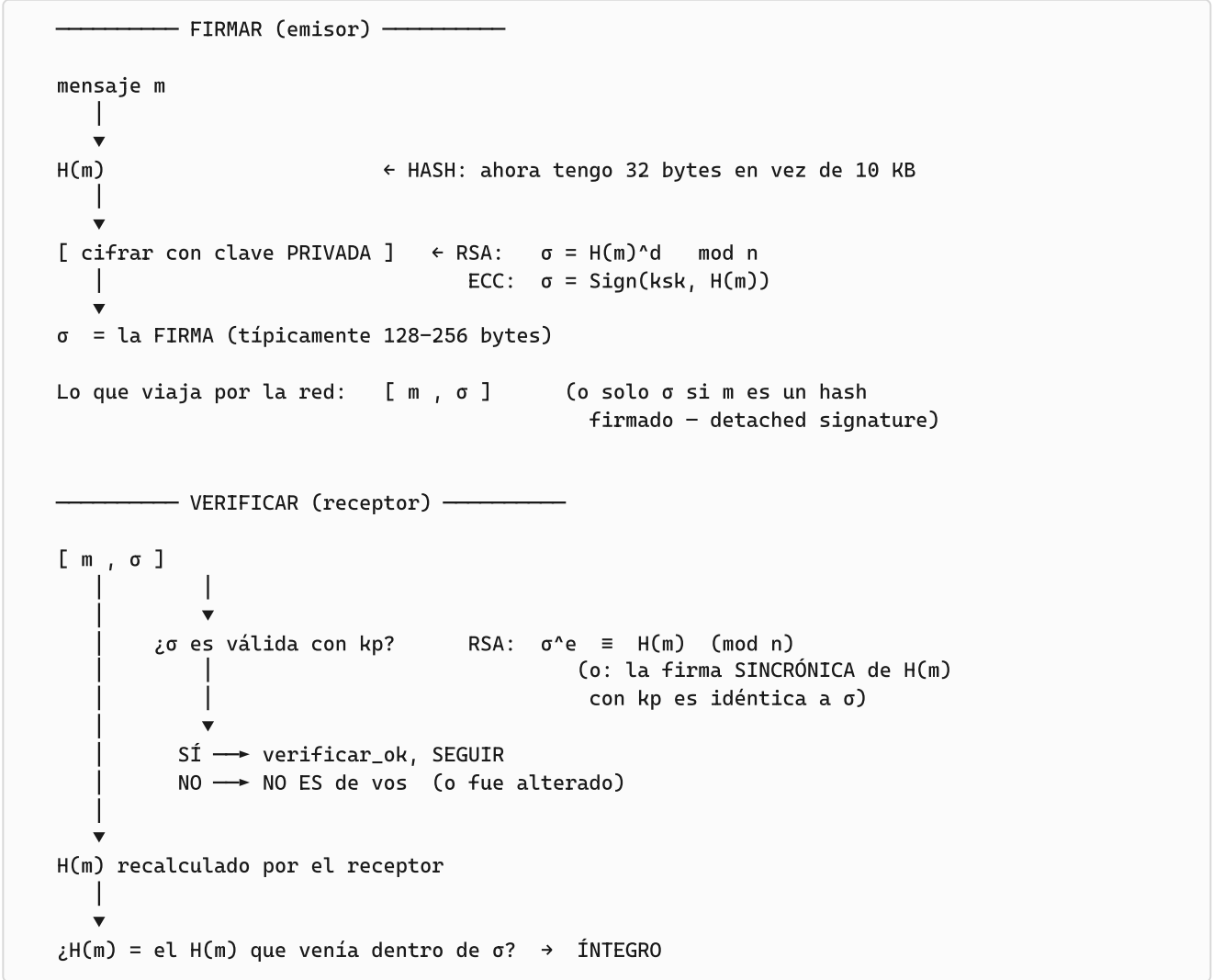
1.5 El hash no firma solo: por qué

Este es **el corazón conceptual** de la clase, y da exactamente el examen `2010-02 P4`.

```
ARGUMENTO (imposible usar hash solo para firma):

1. El hash NO lleva clave. Cualquiera puede calcular H(m).
2. Si "la firma fuera H(m)", el atacante también puede calcular
   H(m) y ponerlo al lado del mensaje.
3. Entonces CUALQUIERA podría "firmar" cualquier mensaje.
   ★ No hayNpseudoLfirma.

ARGUMENTO (imposible usar MAC para firma):

1. El MAC usa clave SIMÉTRICA (HMAC).
2. Para verificar, el receptor necesita la MISMA clave.
3. Para poder verificar, vos tenés que darle tu clave a todos.
4. Si todos tienen la clave, TODOS pueden firmar como vos.
5. ★ No hay "solo vos" podés firmar: no hay no-repudio.

SOLUCIÓN: combines asimetría + hash.
──────────────────────────────────────
1. El hash sigue dando INTEGRIDAD (detecta cualquier cambio).
2. La clave PRIVADA da AUTENTICIDAD (solo vos podés firmar).
3. La clave PÚBLICA da VERIFICABILIDAD sin secreto.
4. ★ La combinación da los 4 servicios a la vez.
```

> **La frase que hay que saber de memoria:** *"el hash detecta el cambio pero no dice quién lo hizo; el MAC detecta el cambio y sabe quién lo hizo, pero todos pueden hacerlo; la firma detecta el cambio, sabe quién lo hizo, y es la única forma de probar que fue esa persona y no otro porque solo esa persona tiene la clave privada que no se puede compartir."*

1.6 La firma digital, paso a paso



Por qué se firma el hash y no el mensaje — y esta es la respuesta que más puntúa:

1. **Eficiencia.** RSA tarda milisegundos. Hashear un archivo de 1 GB tarda microsegundos. Si tuvieras que **firmar el mensaje entero**, la operación criptográfica costaría **lo mismo** que hashear + firmar. Con el hash, firmás 32 bytes.
2. **La firma queda en tamaño fijo**, sea cual sea el mensaje (32 bytes de hash → firma de 256 bytes). Un certificado X.509 firma una lista de SAN: si el mensaje fuese enorme, la firma sería gigante.
3. **La longitud no filtra el secreto.** Con el **padding** determinista, si cifrás el mensaje directamente, el tamaño del cifrado **revela el tamaño del mensaje**. Con un hash de longitud fija, no.

⚠️ **Un matiz que hay que tener claro:** en la práctica, la firma digital **no cifra** el mensaje. El mensaje viaja **en claro** junto a la firma. Solo se **cifra** si querés confidencialidad, y para eso se usa el **esquema híbrido**.

El esquema híbrido: cómo se hace en la realidad

Ninguno de los dos algoritmos sirve solo. Entonces se combinan:

```
── Cifrar un archivo de 1 MB ──

1. Generar una clave de sesión ALEATORIA K          ← 256 bits, efímera
2. Cifrar el archivo con K          (rápido, simétrico, AES-GCM)
3. K se envía... ¿cifrada con RSA? NO. Con el ACUERDO de clave (KEM):
    a. El receptor publica kp.
    b. Vos hacés KEM(kp) = tu secreto + su secreto → K_secreto compartido
    c. Derivás la clave de sesión con un KDF (HKDF)
4. Se transmite:  [archivo cifrado] + [tu secreto encapsulado]
5. El receptor descifra, aplica el KDF, obtiene K, descifra el archivo.

★ Confidencialidad: SIMÉTRICA (rápida) sobre datos largos
★ Autenticación: ASIMÉTRICA sobre la clave corta
★ El archivo nunca sale de claro en el cable ✓
```

Ése es el diseño de TLS, de PGP, de IPsec, de SSH, de todo. Y es la razón por la que **RSA no se usa para cifrar datos**: el costo lo pone la clave, que es corta.

1.7 PKI: el problema de la confianza

Hash, cifrado y firma **ya funcionan**. El problema que queda:

```
Beto le manda a Ana un mensaje firmado con ksb.
Ana tiene kpb "de Beto"... ¿de dónde la sacó?

Si alguien le pasa una clave PÚBLICA por Internet y dice "esta es de Beto",
Ana no tiene forma de saberlo. El atacante le pasa kpa y dice "esta es
de Beto", intercepta y firma como Beto.
★ Mismo problema: falta una raíz de confianza.
```

La solución: el certificado y la cadena de confianza.

```
                RAÍZ  (Root CA)
            auto-firmada, se confía en ella "de fábrica"
             │        │        │
             │        │        └──────── R3 (autoridad de nivel 3)
             │        └─────────────────── R2 (nivel 2)
             └──────────────────────────── R1 (intermedia)

                │ firma a
                ▼
            CA INTERMEDIA
                │ firma a
                ▼
        CERTIFICADO DE HOJA  ("el certificado del servidor")
        ──────────────────────────────────────
        · sujeto:  CN = www.miuy.edu.uy
        · emisor:   la intermedia
        · kp:       la clave pública de www.miuy.edu.uy
        · validez:  desde / hasta
        · Serial,    firma de la intermedia
          algoritmo, huella,   ← el hash del certificado (firmado)
          SAN
        ──────────────────────────────────────
```

La cadena de confianza: para validar el certificado de `www.miuy.edu.uy`, el cliente sube: hoja → intermedia → raíz. Y **cada eslabón firma al siguiente**. Si llega a una raíz que el cliente tiene en su **almacén de confianza**

(*trust store*), la cadena es válida.

> **Y acá está el punto conceptual que hay que decir en el examen:** *toda la seguridad de este modelo descansa en un único supuesto: que las raíces del almacén de confianza son legítimas. Si meten una raíz falsa en tu sistema, todo lo demás se deriva de ella y nada te protege. Ese supuesto es el punto más débil de la PKI, y es la razón por la que existe DNSSEC (ancla en la raíz DNS) y RPKI (ancla en la jerarquía de prefijos).*

Y `SAN` (*Subject Alternative Name*) es lo que hay que saber hoy: el **CN** (*Common Name*) —que era "el nombre del sitio"— **ya no se usa** para validar identidad (Chrome y Firefox lo rechazan). Lo que se valida es el campo **SAN**, que puede contener **DNS names** e **direcciones IP**.

1.8 ⚠️ Actualización a 2026

Sigue igual — de memoria: - **Hash**: SHA-256. Nunca MD5 ni SHA-1. - **Simétrico**: AES-GCM / ChaCha20-Poly1305. AEAD es el default. - **Firma**: RSA-PSS, ECDSA, Ed25519. - **Key agreement**: (EC)DHE con X25519, o híbrido ML-KEM. - **PKI X.509**: cadena AC → intermedia → hoja, validación, `SAN` (no CN). - **Forward secrecy** con (EC)DHE. **HKDF** para derivar claves. - **DH estático NO da forward secrecy** (Logjam) — hay que usar ephemera.

Cambió:

1. **RSA key transport eliminado en TLS 1.3** (`RFC 8446`). Solo queda firma con **RSA-PSS**; `PKCS#1 v1.5` es legacy.
2. **ML-KEM híbrido** (`RFC 10024` , ago-2026) es el nuevo key agreement de facto para resistencia post-cuántica.
3. **SHA-1 formalmente retracted.** NIST lo retiró para firmas en 2011; SHAttered (2017) halló colisiones prácticas; `RFC 9155` (2021) depreca MD5 y SHA-1 en TLS 1.2.
4. **Ed25519 es el default en SSH** (`RFC 8709`) y creciente en X.509.
5. **HTTP Message Signatures** (`RFC 9421` , 2024): firma de *componentes* de un mensaje HTTP, resistente a transformaciones de intermediarios.
6. **OpenPGP:** `RFC 9580` (2024) reemplaza a RFC 4880.

Y la lección conceptual de 2026, que es lo que hay que saber decir:

> **La integridad y la autenticidad son *relativas a un ancla de confianza*, y hay un ancla distinta por ámbito.** Un curso que solo enseña "integridad = HTTPS" se queda cortísimo:

Ámbito	¿Qué es el ancla?	RFC
Transporte web	**PKI / CA** (X.509)	`RFC 8446` → `9846`
DNS	**DNSSEC** + Root KSK	`RFC 4033–4035`, `5155`
Enrutamiento	**RPKI** / ROA	`RFC 6811`
Capa 2	**802.1X** / MAB, Port Security	—
Email	S/MIME u **OpenPGP**	`RFC 9580`
Código / binarios	**firma de commits** (GPG/Sigstore)	—

El no repudio real es raro y costoso: requiere clave privada controlada por el firmante **más** identidad vinculable. **DNSSEC y RPKI son no-repudio de *infraestructura*** — dicen quién originó un nombre o un prefijo, **no quién mandó un correo**.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Qué servicio consigue cada técnica

Enunciado. Completar la matriz con ✓ / ✗.

Resolución:

	Confidencialidad	Integridad	Autenticidad
Hash (SHA-256)	✗ NO oculta nada	✓ detecta cambio	✗ cualquiera puede hashear
MAC (HMAC)	✗	✓	✓ con clave
Cifrado sim. (AES-ECB)	✓ (y nada más)	✗	✗ cualquiera con la clave cifra
Cifrado sim. en modo AEAD (AES-GCM)	✓	✓	✓ (con clave)
Cifrado asim. (RSA)	✓ (por partes)	✗	✓ solo el dueño de la priv
Firma digital (RSA-PSS/Ed)	✗ NO cifra el mensaje	✓	✓ con clave priv
+ No repudio (firma + identidad)	✗	✓	✓ Y se puede PROBAR
TLS 1.3 (esquema híbrido)	✓	✓	✓ + FS

Las tres filas que se explican siempre:

- **Hash**: detecta **que cambió**, no **quién** cambió. Y no oculta nada.
- **Cifrado simétrico puro (ECB)**: esconde el contenido, pero cualquiera con la clave puede cifrar cualquier cosa y **no se detecta la manipulación**.
- **Firma digital**: **no cifra**. El mensaje va en claro; lo que va es la firma. Confundir esto es el error más común.

2.2 Ejercicio 2 — La cadena completa de verificación de un certificado

Datos. El navegador abre `https://www.miuy.edu.uy`.

Resolución paso a paso:

```
1. Obtener el certificado
   El navegador hace el TLS handshake; el servidor le manda su
   certificado de hoja (ya en claro, no cifrado).

2. Extraer la clave pública
   Del certificado sale kp_srv = (n, e).   ← ESTE es el dato útil.

3. Validar la FIRMA del certificado
   El certificado viene firmado por la intermedia.
   Se verifica σ_int con kp_int:
      σ_int^e_int  mod n_int   debe dar   H(certificado)
   Si no coincide → el certificado fue alterado. ✗

4. Validar la CADENA
   Extraer kp_int del certificado de la intermedia.
   Ese certificado está firmado por la raíz.
   Se verifica, y así sucesivamente hasta llegar a una raíz.

5. Validar el ANCLA
   ¿La raíz está en el TRUST STORE del sistema?
      (Windows: almacén de certificados. Mac: login keychain.
       Firefox/Chrome: los suyos + los del SO.)
   SÍ → la cadena es de fiar.   NO → ✗ no confío.

6. Validar la IDENTIDAD
   Extraer el SAN del certificado de hoja.
   ¿"www.miuy.edu.uy" está en el SAN?
      SÍ → ✓.   NO → ✗ (aunque la cadena sea válida:
                    el certificado es de otro sitio!)

7. Validar la VIGENCIA
   ¿Estamos dentro del período de validez?
   ¿La revocación no está activa?  (CRL u OCSP)

8. Solo ahora: usar la clave
   Se cifra la clave de sesión con kp_srv  → se abre el canal.
```

> **El paso 6 es el que más se olvida y el más importante conceptualmente:** una cadena válida **no garantiza identidad**. Lo que garantiza es que **alguien firmó** ese certificado. Hay que comprobar **que firmó del nombre correcto**. Por eso un certificado de `evil.com` reemitido por una CA real es **cadena válida con identidad incorrecta**, y es lo que valida el SAN.
>
> Y el paso 8 es donde aparece la **moderna**: en TLS 1.3 **no se cifra nada con RSA**. Se usa la clave del certificado para **firmar** el *handshake* (autenticar al servidor), y la clave de sesión se acuerda con (EC)DHE.

2.3 Ejercicio 3 — Cifrado híbrido, números a la vista

Datos. Un archivo de 3 MB. AES-256 a 2 GB/s. RSA-2048 cifra 245 bytes por operación, a **300 operaciones/s**.

Resolución — las dos cuentas:

```
── Con RSA solo ──
3 MB = 3 × 1024 × 1024 = 3 145 728 bytes
Bloques de 245 bytes  = 3 145 728 / 245 = 12 839 operaciones
Tiempo = 12 839 ops / 300 ops/s = 42,8 segundos

★ 43 SEGUNDOS para cifrar 3 MB. Inusable.

── Con AES solo ──
3 145 728 bytes / 2 × 10⁹ bytes/s = 0,0016 s = 1,6 milisegundos

★ 26 000 veces más rápido.

── Con el esquema híbrido ──
1. K = 256 bits aleatorios                              →  ~0
2. Cifrar el archivo con AES-GCM usando K  →  1,6 ms
3. KEM(kp_destinatario): encapsular K        →  ~2 ms (una operación RSA)
4. Total ≈ 3,6 ms   ← DOMINADO por la criptografía de clave pública,
                        pero el dato pesado ya está cifrado.
```

La moraleja, que es la que hay que escribir: *la clave pública cifra una sola clave de sesión corta; el simétrico cifra los datos largos. Por eso todo protocolo moderno es híbrido y por eso la clave pública tiene que ser grande.*

2.4 Ejercicio 4 — Módulo RSA y cifrado/descifrado numérico

Datos. `p = 61`, `q = 53`. `e = 17`. Mensaje `m = 65`.

Resolución paso a paso:

```
1. n = p · q = 61 × 53 = 3233
2. ϕ(n) = (p−1)(q−1) = 60 × 52 = 3120
3. e = 17   ( gcd(17, 3120) = 1  ✓ )
4. d = e⁻¹ mod ϕ(n)

   Buscamos d tal que   17 · d ≡ 1  (mod 3120)
   Extendido euclidiano:

      3120 = 183·17 +  9
        17 =   1·9  +  8
         9 =   1·8  +  1
         8 =   8·1  +  0

   De abajo hacia arriba:
         1 = 9 − 8
           = 9 − (17 − 9)
           = 2·9 − 17
           = 2·(3120 − 183·17) − 17
           = 2·3120 − 367·17

   ⇒  17⁻¹ mod 3120 = −367 ≡ 3120 − 367 = 2753
   ⇒  d = 2753

VERIFICACIÓN:  17 × 2753 = 46 801
               46 801 / 3120 = 15,0 exacto   (15 × 3120 = 46 800, +1) ✓

5. Clave PÚBLICA  = (n, e) = (3233, 17)
   Clave PRIVADA = (d)   =  2753
```

Cifrado:

```
c = m^e mod n = 65^17 mod 3233

65² mod 3233 = 4225 mod 3233 = 992
65⁴ mod 3233 = 992² = 984 064 mod 3233

   984 064 / 3233 = 304, resto:
   3233 × 304 = 982 832
   984 064 − 982 832 = 1 232
⇒  65⁴ ≡ 1232 (mod 3233)

65⁸ mod 3233 = 1232² = 1 517 824 mod 3233
   3233 × 469 = 1 516 277
   1 517 824 − 1 516 277 = 1 547
⇒  65⁸ ≡ 1547 (mod 3233)

65¹⁶ mod 3233 = 1547² = 2 393 209 mod 3233
   3233 × 740 = 2 392 420
   2 393 209 − 2 392 420 =  789
⇒  65¹⁶ ≡ 789 (mod 3233)

65¹⁷ = 65¹⁶ × 65 = 789 × 65 = 51 285 mod 3233
   3233 × 15 = 48 495
   51 285 − 48 495 = 2 790

c = 2790
```

Descifrado:

```
m = c^d mod n = 2790^2753 mod 3233

Por el teorema de Euler (e·d = 46 801 = 15·3120 + 1):
     c^(e·d) ≡ c^(1 + 15·ϕ(n))  ≡ c · (c^ϕ(n))^15  ≡ c · 1

Es decir:  2790^46 801 ≡ 2790 (mod 3233)
Y como 46 801 = 2753 × 17 :
     2790^2753 ≡ 65   (mod 3233)   ← ¡es el mensaje original!

m = 65  ✓
```

> ⚠️ **Errores típicos en este ejercicio:** - Confundir `ϕ(n)` con `n`: RSA necesita `ϕ(n)`, no `n`. - Olvidar el **extendido euclidiano** y quedarse sin `d`. Se puede buscar por fuerza bruta `d` desde 1 hasta `ϕ(n)`, pero eso es lentísimo. - Olvidar que `e` debe ser **coprimo con φ(n)**: si no lo es, **no existe** `d`. - **Ignorar el *padding*.** Con `c = m^e` pelado, RSA es **determinista** y vulnerable: si cifrás el mismo mensaje dos veces, sale el mismo `c`; y con padding aleatorio (PKCS#1 v1.5 u OAEP) se evita.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-02, Pregunta 4 (8 puntos) — firma digital

> **Fuente:** Examen `sirc1002`, 16 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1002.pdf **Texto del examen: a)** Explique el concepto de Firma Digital y su uso en el caso de mensajes de correo electrónico. **b)** ¿MAC (Message Authentication Code) puede ser utilizado como técnica para firmar digitalmente? Justifique su respuesta.

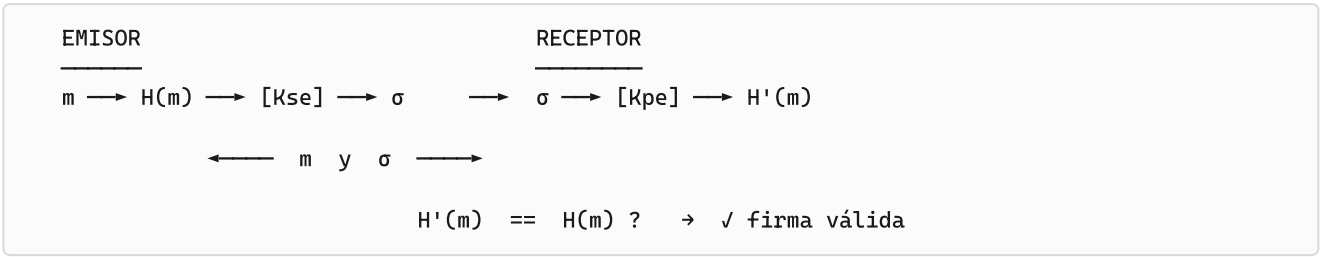
Resolución — parte a) (5 puntos)

La definición (1 punto). Una **técnica criptográfica** que busca lograr, en el mundo digital, objetivos **similares** a los de las firmas manuscritas tradicionales. Los cuatro: **verificable**, **no falsificable**, **no repudiable**, e **integridad del mensaje**.

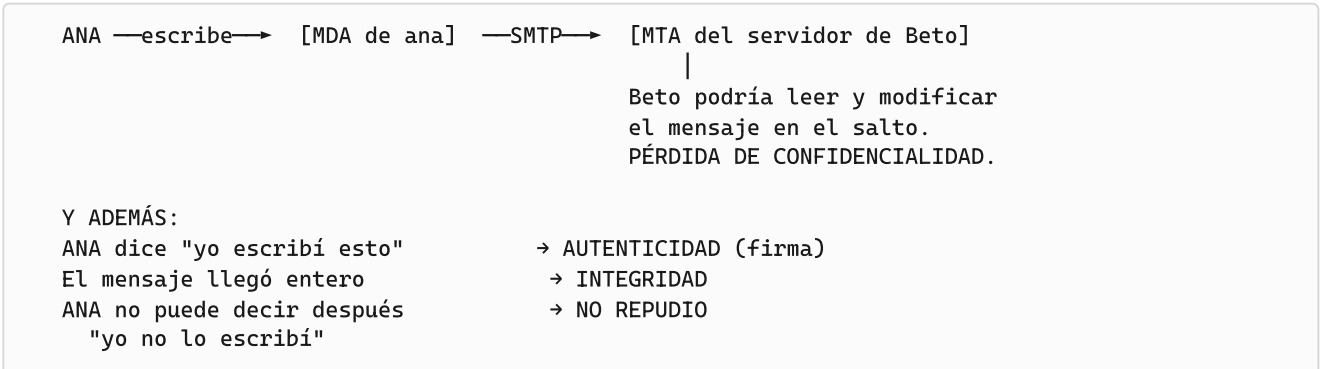
El mecanismo, paso a paso (3 puntos) — esto es lo que hay que escribir:

La firma digital **combina criptografía asimétrica con funciones de hash**:

1. Del mensaje a firmar `m` se obtiene un hash: `H(m)`.
2. Ese hash **se cifra con la clave privada del emisor**: `σ = Kse(H(m))`.
3. Se envía **el mensaje `m` junto con la firma `σ`**. Y acá está la observación clave que la solución oficial underline y que hay que repetir: **no se movementa servicio de confidencialidad** — el mensaje va en claro.
4. El receptor calcula `H(m)` por su cuenta, descifra la firma con la clave **pública** del emisor, y **compara**.



Para qué sirve en el correo (1 punto). En un correo hay **cuatro** participantes que generan problemas de confianza distintos:



Y la respuesta clave, que es la que da el punto: *la firma digital no cifra el correo: el mensaje viaja en claro. Lo que hace es probar quién lo escribió y que no fue alterado. Para confidencialidad hace falta cifrado (`PGP` o `S/MIME`), que es otra cosa.*

Resolución — parte b) (3 puntos)

La respuesta es NO, y la justificación tiene que dar los dos pasos del argumento:

1. **El MAC se basa en criptografía simétrica.** Para verificar, hace falta la misma clave con la que se generó. Si Beto tiene que poder verificar la firma de Ana, hay que darle **la clave de Ana**.
2. **Si todos tienen la clave, todos pueden "firmar".** En el momento en que le das la clave a Beto para que pueda verificar, **Beto puede usarla para firmar mensajes haciéndose pasar por Ana**. No hay forma de que la clave sirva para verificar *y no* para firmar.

Y la extensión que suma puntos: la firma digital necesita que el receptor pueda **verificar** (con `kpe`, pública) pero **solo el emisor pueda firmar** (con `kse`, privada). **Esa asimetría es imposible con claves simétricas.** Con simetría, todo es simétrico: quien verifica, puede firmar.

> **Frase de cierre para sumar puntos:** *"el MAC da integridad y autenticación, pero no no-repudio: con una clave compartida, el no-repudio es imposible, porque todos los que pueden verificar pueden también firmar."*

3.2 Examen 2009-12, Pregunta 4 (6 puntos) — funciones de hash

> **Texto del examen: a)** Defina funciones de hash. **b)** Enumere las propiedades básicas de seguridad en la red, y especifique cuál o cuáles aporta la utilización de estas funciones. **c)** Mencione un ejemplo de función de hash comúnmente utilizada para intercambio de mensajes.

Resolución

a) Definición (2 puntos). Una **función de hash** es una función matemática que **transforma un mensaje de cualquier longitud** en una cadena de **longitud fija**, llamada **hash** o **resumen** o **huella**, tal que:

- es **fácil de calcular**;
- es **prácticamente imposible** recuperar el mensaje original a partir del hash (**unidireccional**);
- es **prácticamente imposible** encontrar dos mensajes distintos con el mismo hash (**resistente a colisiones**);
- **cualquier modificación, por mínima que sea, del mensaje produce un hash completamente diferente** (**efecto avalancha**).

b) Propiedades de seguridad (3 puntos) — esta es la parte que más puntos da:

Las cuatro propiedades de seguridad (**confidencialidad**, **integridad**, **autenticidad**, **no repudio**) y el aporte del hash:

Propiedad	¿El hash la aporta?	Razón
Confidencialidad	**NO**	El hash **no oculta nada**: es público, unidireccional y sin clave. Se obtiene el hash **exactamente igual** que el mensaje: solo sealary.
Integridad	**SÍ**	Si el mensaje se altera, el hash cambia. Quien recibe puede **recalcular** el hash y **comparar**. Es el aporte central.
Autenticidad	**NO** (por sí solo)	**Cualquiera** puede calcular el hash de cualquier mensaje. No requiere clave ni identidad.
No repudio	**NO**	Misma razón: cualquiera puede "firmar" (= hashear) cualquier mensaje. No hay no-repudio.

> **La conclusión, que es lo que la cátedra quiere que entiendas:** *el hash, por sí solo, aporta únicamente integridad. No aporta confidencialidad porque no cifra; no aporta autenticidad ni no repudio porque no tiene clave: cualquiera puede calcularlo. Para conseguir los otros tres hay que combinarlo con cifrado simétrico (autenticidad) o con cifrado asimétrico (autenticidad y no repudio).*

c) Ejemplo (1 punto). SHA-256, actualmente el estándar de facto (RFC 6234), y **SHA-3** (Keccak, FIPS 202). Anteriormente se usaban **MD5** y **SHA-1**, **ambos rotos**: MD5 por colisiones desde 2004 (y el *rogue CA* en 2008), SHA-1 por **SHAttered** en 2017. **Nunca usarlos.**

Lo que la solución oficial no dice y suma

El detalle del *fingerprint* en la práctica. Cuando hacés `sha256sum archivo`, lo que obtenés **no es** el hash del archivo entero: es el hash del **hash** (concatenado) de los archivos binarios que lo componen, más un sufijo con el **nombre y la longitud**. En Wireshark o en un certificado, el **fingerprint** del certificado es justamente `SHA-1` o `SHA-256` del DER codificado. Es un uso directo de la propiedad de *integridad*: si el fingerprint que vos anotaste a mano coincide con el que muestra el sistema, **nadie te cambió el certificado en tránsito**.

3.3 Examen 2009-02, Pregunta 1 (9 puntos) — TLS

> **Texto del examen: a)** Mencione el protocolo descrito durante el curso que puede cumplir con dicho objetivo [proteger a nivel de transporte una comunicación http, identificado como https]. **b)** Indique **qué clave** contiene el certificado que el servidor le envía al cliente, y explique **para qué se utiliza** dicha clave. **c)** ¿**Qué clave/s** se utiliza/n para **proteger los datos** que viajan en el sentido **servidor → cliente**? **d)** Mencione otros componentes del certificado, y qué **elemento adicional** se necesita en el cliente para **asegurar la identidad del servidor**.

Resolución — la pregunta que hay que leer muy bien

> ⚠️ **Leelo dos veces antes de responder.** La (c) es la pregunta **tramposa** del examen: dice **"proteger los datos que viajan"**, no "proteger los datos". Y la diferencia entre esas dos frases es **toda la diferencia entre el examen de 2010 y el de 2026**.

a) El protocolo (1 punto)

TLS (*Transport Layer Security*), y su antecesor **SSL**. Se ubica **entre la capa de aplicación y la capa de transporte** (es decir, **capa 5-6** de TCP/IP): recibe los datos de HTTP, los cifra, y los pasa a TCP como si fueran datos normales.

```
┌──────────────┐
│   HTTP       │  ← texto en claro (la aplicación)
├──────────────┤
│   TLS        │  ← CIFRADO + integridad + autenticación
├──────────────┤
│   TCP        │
├──────────────┤
│   IP         │
└──────────────┘
```

b) Qué clave contiene el certificado (3 puntos)

Contiene la clave PÚBLICA del servidor, y sirve para que el cliente **pueda verificar que la clave privada que va a usar es realmente del servidor**.

Y para explicar "para qué se utiliza", hay que decir **las dos cosas**:

1. **Autenticar al servidor** ante el cliente: solo el servidor tiene la clave privada que corresponde a esa `kp`. Si el servidor puede firmar con la privada, es porque es el dueño de la pública del certificado. **Eso descarta el man-in-the-middle**, que es el propósito de TLS en el navegador.

> **El ataque que esto evita:** sin verificación de identidad, un atacante en el medio (café, wifi del aeropuerto, ISP) puede responder como si fuera el banco, y vos le mandás tu contraseña **a él**. TLS lo que hace es **obligar al atacante a tener la clave privada**, que no tiene.

2. **En las versiones antiguas, cifrar el *pre-master secret*.** En TLS 1.2 y anteriores, el cliente generaba el *pre-master secret* y lo cifraba con la `kp` del servidor: así **solo el servidor** podía descifrarlo. **Eso es lo que ya no se hace en TLS 1.3** (ver §Actualización).

c) Qué clave protege los DATOS (2 puntos)

Acá está la clave del examen. La clave que **realmente cifra los datos de aplicación** (los paquetes HTTP) es una **clave de sesión simétrica**, y en TLS 1.3 es siempre **(EC)DH key agreement** — es decir, **una clave compartida negociada, nunca enviada**.

```
La cadena completa, en TLS 1.3:

1. El servidor manda su CERTIFICADO con kp_srv.
2. El servidor manda su clave pública de (EC)DHE (key share).
3. El cliente manda su propia clave pública de (EC)DHE.
4. Ambos calculan el SECRETO COMPARTIDO = f(kp_srv_priv, kp_cli_pub)
   = f(kp_cli_priv, kp_srv_pub)          ← es el MISMO, por matemática
5. DerivAN la clave de sesión con un KDF (HKDF).
6. La clave de sesión cifra TODOS los datos HTTP, en los DOS sentidos.

★ La clave del certificado NO cifra los datos. Solo autentica al servidor.
★ La clave de sesión NO se envía nunca. Se acuerda, no se transmite.
```

> **La respuesta completa para el (c):** *la clave que protege los datos es la clave de sesión simétrica (derivada del secreto compartido de (EC)DH). La clave pública del certificado no cifra los datos: se usa para autenticar al servidor (o, en TLS ≤1.2, para cifrar el* pre-master secret*). En ambos sentidos de la comunicación se usa la misma clave de sesión, pero en general claves distintas para cada dirección (derived keys del HKDF), de modo que el tráfico de ida no se puede "desenmascarar" inyectando tráfico de vuelta.*

d) Otros componentes del certificado (3 puntos)

Los componentes que hay que enumerar:

Componente	Qué es
Versión y **Número de serie**	único por autoridad
Algoritmo de firma	con qué se firmó (SHA-256 con RSA, p. ej.)
Emisor (*issuer*)	quién firmó
Período de validez	*not before / not after*
Sujeto (*subject*)	CN, O, OU, C, L…
Clave pública del sujeto	← la del punto (b)
Huella digital (*fingerprint*)	hash del certificado
Extensiones	**SAN**, key usage, basic constraints, CRL distribution points

El elemento adicional que se necesita en el cliente (esto es lo que el examen pide y es la mitad del punto):

> **El almacén de certificados de confianza (el *trust store*), que contiene las claves públicas de las autoridades certificadoras raíz en las que el cliente confía.**

Sin ese almacén, el cliente **no tiene contra quién comparar** la firma del certificado: no puede verificar que quien firmó es una CA real. **Todo el modelo descansa en ese supuesto.**

La cadena completa, entonces: el certificado dice "*esta clave pública es de* `www.miuy.edu.uy`". El trust store dice "*esta CA es real*". Y la cadena de certificados **intermedios** une los dos extremos. Si falta cualquiera de los dos —o si no se valida el **SAN**— la conexión falla.

> ⚠️ **Nota importante sobre el (c) para 2026.** Si te examined en 2011, la respuesta esperada era "**RSA**" en el caso de TLS 1.2, porque la clave pública del certificado **cifraba el pre-master secret**. En TLS 1.3 eso **ya no existe**: **el transporte de clave con RSA fue eliminado** y todo pasa por (EC)DHE. Lo que **no cambia** es la idea: **la clave del certificado autentica, la clave de sesión cifra**. Si el examen te pide el nombre del algoritmo y estás en 2026, contestá **"(EC)DH (Diffie-Hellman) con X25519"** y aclará **"en TLS 1.2 era RSA"**.

3.4 Examen 2008-12, Pregunta 2 (6 puntos) — hub vs. switch y seguridad

Resolución. El enunciado compara **hub** y **switch** en cuanto a rendimiento y **seguridad**. La parte de seguridad es la que se pregunta y tiene tres respuestas concretas:

1. **El switch es un dispositivo de filtrado.** Recibe la trama completa en un puerto, mira la **MAC de destino**, y la manda **solo** por el puerto correspondiente. El hub, en cambio, la reenvía a **todos los puertos**.
2. **Aislamiento de dominios de colisión y de tramas.** Cada puerto del switch es un **dominio de colisión separado**, y solo el dominio del destino recibe la trama. El hub es **un solo** dominio de colisión.
3. **Sniffing.** Con el hub, cualquiera que enchufe un **sniffer** en cualquier puerto ve **todo** el tráfico. Con el switch, el sniffer solo ve el tráfico dirigido a su MAC (salvo **ataques de inundación de MAC**, que llenan la tabla de ARP/CAM y fuerzan al switch a retransmitir como un hub — que es precisamente por lo que existe **port security**).

Y el remate que hay que decir porque es lo que se pregunta en la cátedra: *el problema real no es que el hub no filtre: es que un switch no es una frontera de seguridad. Los tres modos de un switch —store-and-forward, cut-through, fragment-free— son decisiones de performance, no de seguridad. La "seguridad" de un switch es cero; todo lo que hace es optimizar el envío de tramas. Quien decide seguridad es el firewall, o el VLAN + ACL, no el switch.*

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Confidencialidad** = ¿puede leerlo? · **Integridad** = ¿puede cambiarlo? · **Autenticidad** = ¿es de verdad? · **No repudio** = ¿podés probarlo?
2. Son **servicios independientes**: confidencialidad **no** implica integridad.
3. **Hash** (SHA-256): determinista, unidireccional, sin colisiones, avalancha. **Aporta integridad y nada más** — ni confidencialidad, ni autenticidad, ni no repudio, porque **no lleva clave**.
4. **MAC** (HMAC): clave simétrica. Aporta integridad + autenticación, pero **no confidencialidad** (no cifra) **ni no repudio** (todos pueden firmar).
5. **Firma digital**: hash + clave privada. Aporta integridad + autenticidad + **no repudio**, y **no aporta confidencialidad** (no cifra el mensaje).
6. **RSA**: `n = p·q`, `φ(n) = (p−1)(q−1)`, `e·d ≡ 1 mod φ(n)`. Lento, bloques de 245 bytes con clave de 2048 bits, y `PKCS#1 v1.5` **frágil**: hoy solo **RSA-PSS**.
7. **ECC**: misma seguridad con claves 12× más cortas. **ECDSA** firma, **(EC)DH** acuerda, **Ed25519** firma y es el default de SSH.
8. **Esquema híbrido**: clave de sesión aleatoria + AES-GCM para los datos, KEM para la clave. Por eso todo protocolo moderno cifra datos con simétrico.
9. **PKI**: la clave del certificado **autentica**, la clave de sesión **cifra**, y el **SAN** (no el CN) es lo que valida la identidad. Todo descansa en el **trust store**.
10. **2026**: RSA key transport eliminado en TLS 1.3; ML-KEM híbrido post-cuántico; SHA-1 **formalmente retracted**; y lo más importante: **cada ámbito tiene su ancla de confianza** (PKI, DNSSEC, RPKI, 802.1X, PGP).

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué no se puede usar un hash como firma digital? Y ¿por qué no se puede usar un MAC?

<details>
<summary>▶ Respuesta</summary>
</details>

2. ¿Por qué se firma el hash y no el mensaje?

<details>
<summary>▶ Respuesta</summary>
</details>

3. En TLS 1.3, ¿qué clave protege los datos de aplicación? ¿Y para qué sirve la clave del certificado?

<details>
<summary>▶ Respuesta</summary>
</details>

4. Calculá `d` para `p = 61`, `q = 53`, `e = 17`, y cifrá `m = 65`.

<details>
<summary>▶ Respuesta</summary>
</details>

5. ¿Por qué el SAN reemplazó al CN en los certificados?

▶ Respuesta

6. ¿Qué pasa si el *trust store* de tu máquina tiene una raíz falsa?

▶ Respuesta

🔗 Referencias

- **Examen** `sirc1002` — Pregunta 4 completa (8 pts: firma digital y por qué MAC no firma)
- **Examen** `sirc0912` — Pregunta 4 (6 pts: funciones de hash y propiedades de seguridad)
- **Examen** `sirc0902` — Pregunta 1 (9 pts: TLS, la clave del certificado y la clave de sesión)
- **Examen** `sirc0812` — Pregunta 2 (hub vs. switch: rendimiento y seguridad)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 8 (seguridad)
- `RFC 2104` — HMAC · `RFC 8017` — PKCS #1 v2.2 (RSA) · `RFC 8017` **§8** — RSA-PSS · `RFC 5756` — política de validación X.509 · `RFC 5280` — PKI X.509 · `RFC 6962` — Certificate Transparency · `RFC 8446` — TLS 1.3 · `RFC 9155` — deprecación de MD5 y SHA-1 · `RFC 8709` — Ed25519 en SSH · `RFC 9421` — HTTP Message Signatures · `RFC 9580` — OpenPGP · `RFC 10024` — ML-KEM híbrido
- `ACTUALIZACIONES.md` **§13** — el cuadro de anclas de confianza por ámbito
- **Clase siguiente:** 48 — Seguridad aplicada (TLS, PGP, VPN, IDS/IPS, DPI) + IP Móvil + Simulacro final

Clase 48

Contenido de esta clase

🎯 Al terminar esta clase vas a poder
1.1 TLS: la protección aplicada de un protocolo de aplicación
1.2 SSH: el mismo esquema, otro uso
1.3 PGP: cifrar correo entre usuarios
1.4 Túneles y VPN
1.5 IDS, IPS y DPI
1.6 Los ataques típicos y qué capa los detiene
1.7 IP Móvil
2.1 Ejercicio 1 — El túnel doble del Examen 2008-08
2.2 Ejercicio 2 — Calcular qué capa corta cada ataque
2.3 Ejercicio 3 — Por qué un hash no da confidencialidad (demostración)
3.1 ⭐ Examen 2011-02, Problema 2 (30 puntos) — IP Móvil
3.2 Examen 2008-12, Pregunta 3 (10 puntos) — arquitectura de IP Móvil
3.3 Examen 2007-07, Pregunta 5 (5 puntos) — funcionamiento de IP Móvil
📌 Resumen del Bloque F y del curso entero en 10 líneas
🔑 Mapa final del curso — dónde está cada cosa
📝 SIMULACRO FINAL — Examen completo
✅ Autoevaluación
🔗 Referencias
🎓 Cierre del curso

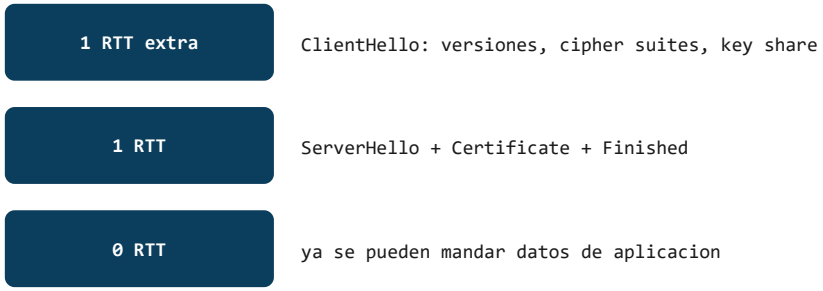
Los 6 bloques del curso

Bloque: F (Servicios) · **Semana:** 16 · **Duración estimada:** 150 min **Correlativa:** Clase 47 (criptografía) · **Cierre del curso Material de referencia:** Tanenbaum cap. 8 y cap. 6 (movilidad) · `ACTUALIZACIONES.md` §10 y §13

🎯 Al terminar esta clase vas a poder

1. Describir **cómo se aplica** la criptografía: **TLS**, **PGP** e **SSH**, y decir qué protección da cada uno.
2. Explicar qué es un **túnel** y una **VPN**, con los **dos modos** (enrutada vs. Switched) y el **doble encapsulado**.
3. Distinguir **IDS**, **IPS** y **DPI**, con el criterio de **falso positivo** y de **punto deplacede inspección**.
4. Reconocer **cinco ataques típicos** y decir **qué capa** los elimina.
5. Dibujar la arquitectura de **IP Móvil** con los **cuatro componentes** y sus **tres modos** de operación, incluida la **ruta triangular**.
6. Resolver un examen completo de 3 horas con cronómetro, por última vez.

Handshake de TLS 1.3 en un RTT

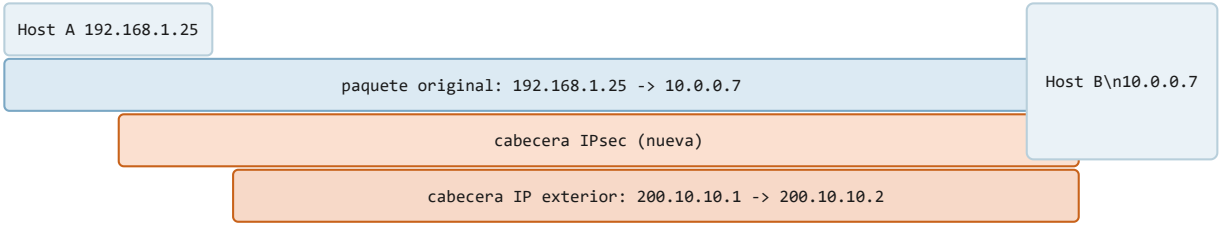


TLS 1.3 cambio: solo AEAD, sin compresion, sin renegociacion, y forward secrecy obligatorio.
El certificado sigue siendo X.509; lo que cambio fue el intercambio de claves

> **Prerequisito del examen:** `2011-02 Problema 2` (30 pts — IP Móvil: campos MAC/IP por segmento y túnel), `2008-12 P3` (10 pts — arquitectura de IP Móvil), `2007-07 P5` (5 pts — funcionamiento de IP Móvil), `2009-07 Problema 1` (30 pts), `2008-12 Problema 1` (30 pts — NAT con balanceo de carga), `2008-08 Problema 1` (30 pts — tunneling).

Un tunel y por que complica el MTU

Que envuelve a que:



Cada capa de overlay AUMENTA el tamano del paquete y RESTA MTU disponible.

Por eso hace falta clamp MSS en el borde de las redes con jumbo frames.

Y rompe el aislamiento de capas: el router exterior tiene que ENENDER IPsec para forwarding, cuando su unico trabajo deberia ser mirar la IP de destino.

Tipos: site-to-site (always on) y remoto (el usuario inicia).

PARTE 1 — TEORÍA

1.1 TLS: la protección aplicada de un protocolo de aplicación

La clase 47 dio la criptografía. TLS es **la aplicación completa**, y es lo que justifica toda la clase 47.

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│   SIN TLS                    CON TLS                     │
│   ─────────                  ────────                    │
│   El ATM ve                  El ATM ve                   │
│   "usuario ana               "usuario ana"               │
│  / clave: XYZ123"            (todo cifrado)              │
│                                                          │
│   → el sniff ve la           → el sniff ve bytes         │
│     contraseña                  incomprensibles          │
│   → el MITM modifica         → el MITM rompe el MAC →    │
│     el monto de la            el cliente lo detecta      │
│     transferencia                                        │
└──────────────────────────────────────────────────────────┘
```

El handshake de TLS 1.3, mensaje por mensaje (esto se pregunta):

```
CLIENTE                                          SERVIDOR
───────                                          ────────

1. ClientHello
   · versión soportada (TLS 1.3)
   · cipher suites  ← ciphers SIMÉTRICOS + hash
   · key_share      ← kp de X25519 o ML-KEM  ← lo trae EN EL MENSAJE 1
   · supported_versions
──────────────────────────────────────────────────────────►
                                2. ServerHello
                                   · elige cipher suite
                                   · ServerHello.key_share ← kp del servidor
                                3. {EncryptedExtensions}
                                4. {Certificate}
                                   · kp del servidor (autenticación)
                                5. {CertificateVerify}
                                   · FIRMA con la clave privada
                                      ★ esto prueba que es quien dice ser
                                6. {Finished}
                                   · MAC sobre todo el transcript
◄──────────────────────────────────────────────────────────
7. {Finished}
──────────────────────────────────────────────────────────►

★ TODO desde (2) en adelante va CIFRADO.
  Los dos Finished exchanged en 1-RTT.
```

Los cuatro servicios, con el mecanismo exacto que los da:

Servicio	Mecanismo en TLS 1.3
Confidencialidad	clave de sesión derivada por (EC)DH; cifrado con **AES-256-GCM** o **ChaCha20-Poly1305**
Integridad	**AEAD**: el tag de autenticación va en cada registro
Autenticación del servidor	**certificado X.509** + firma del *CertificateVerify*
Autenticación del cliente	opcional: **certificado cliente** o **SRP** / PSK
Forward secrecy	**(EC)DH efímero**: las claves de sesión se borran y no se pueden recuperar

El concepto que hay que entender: el *forward secrecy*.

```
SIN forward secrecy:                CON forward secrecy (PFS):
─────────────────────               ──────────────────────────
El servidor guarda la               El servidor NO guarda la
clave de sesión.                    clave de sesión. Nunca la
Alguien que roba el                 tuvo guardada. Cuando se
servidor HOY obtiene                termina la conexión, se
TODO: el tráfico de                 borra. Lo único que queda
AYER descifrado.                    es la clave pública (pública).
★ "Perfect forward secrecy"         ★ Aunque te entreguen el disco
  significa: aunque te              forense del servidor hoy, no
  entreguen la clave AHORA,          podés descifrar lo de ayer.
  no descifrás lo de AYER.
```

Y es por eso que el *handshake* de TLS 1.3 es de 1-RTT: se manda el `key_share` en el **primer mensaje**, sin esperar el `ServerHello` con criptografía clásica (RSA). Antes (TLS 1.2 con RSA) eran 2 RTT. Ese es un avance real y medible.

Y el puerto que se cambia: HTTP en 80, **HTTPS en 443**. Y ojo: **443 es TCP**, no UDP. (El **443/UDP** es **QUIC**, que es otra cosa — clase 9.)

1.2 SSH: el mismo esquema, otro uso

```
CLIENTE ──────────── SERVIDOR
   │                    │
   │── kp_srv ─────────►│   1. el servidor manda su clave de host
   │                    │      (¡EN CLARO! de eso se encarga DNS/DH)
   │◄─ kp_cli (efímera) ─│   2. el cliente manda su clave efímera
   │                    │
   │  secreto compartido │
   │  + HKDF + las claves de cifrado de sesión  ← derived keys
   │                    │
   │◄═══ CANAL CIFRADO ═══►│   3. de acá en adelante todo va cifrado
```

Puntos clave de SSH:

- **La clave de host es del servidor, pública, y se publica en el DNS** (o en el `known_hosts`). Se verifica: si cambia, te avisa.
- **Las claves de sesión son efímeras**: se renegocian por canal o cada hora. **Es lo que da el PFS** en SSH.
- **Autenticación del usuario**: por clave pública (ideal), o por contraseña (y acá el problema famous: el servidor no recibe la contraseña, sino un valor derivado, así que un *sniffer* no la obtiene).
- **Canal de control + canales de sesión**: el canal de control multiplexa varias sesiones (port forwarding, SOCKS, agent forwarding). **Es el mismo concepto de multiplexación de la clase 16, pero en la capa de**

aplicación.

- **Moderno**: por defecto usa **Ed25519** para las claves de host y ECDH para el acuerdo (`RFC 8709`, 2020).

> **El "ataque del hombre en el medio" en SSH solo funciona si nadie verifica la clave del host.** Y la primera conexión pregunta "¿confiás en esta huella?". Si decís que sí sin leer, **perdiste**. Es exactamente el problema de la raíz de confianza de la clase 47, pero resuelto a nivel de un solo servidor.

1.3 PGP: cifrar correo entre usuarios

PGP (*Pretty Good Privacy*) es el esquema de **seguridad extremo a extremo** para correo. La diferencia con TLS es **dónde está el protocolo**:

```
TLS    ──  SEGURIDAD EN EL CAMINO (transport)
           El servidor web la termina: ve todo el tráfico.
           ★ Confiás en el servidor.

PGP    ──  SEGURIDAD EXTREMO A EXTREMO (aplicación)
           Cifrás el mensaje antes de que salga de tu máquina.
           El servidor de correo TRANSPORTA un texto cifrado.
           ★ El servidor no puede leerlo, aunque lo quiera.
```

El modelo de claves de PGP (y el detalle fino que se pregunta):

```
Cada usuario tiene UNA clave privada y VARIAS claves públicas:

┌──────────────────────────────────────────────────────────────┐
│  Clave MAESTRA (primary key)                                 │
│     · RSA/ECDSA/EdDSA. Firma. Certifica las subclaves.       │
│     · Con la que firmás.                                     │
├──────────────────────────────────────────────────────────────┤
│  SUBCLAVE DE CIFRADO                                         │
│     · RSA o ElGamal (cifrado).                               │
│     · Para los que te escriben A VOS.                        │
├──────────────────────────────────────────────────────────────┤
│  SUBCLAVE DE FIRMA                                           │
│     · DSA/RSA/EdDSA.                                         │
│     · Para firmar los key ID de las claves de los demás.     │
└──────────────────────────────────────────────────────────────┘

★ La SEPARACIÓN de la clave de firma de la de cifrado es el innovation
  clave: una clave que solo firma NO puede cifrar (y una que solo
  cifra no debería firmar). Es "separación de privilegios".
```

El proceso de enviar correo con PGP (ocho pasos):

```
EMISOR                                  RECEPTOR
──────                                  ────────

1. Firma:
   H(m) ──► [mi subclave de firma] ──► σ

2. Cifrado de σ y m:
   [clave pública de Beto (RSA)] ──► (σ ‖ m) cifrado

3. Agrega metadatos (armored):
   ── BEGIN PGP MESSAGE ──
   m = 65, σ = 2790, kp_emit = 3233
   ── END PGP MESSAGE ──

4. Envía por SMTP normal. El servidor de correo transporta
   texto ilegible.
                            ◄──── mensaje cifrado ────

5. Descifra con su clave privada (RSA): recupera (σ ‖ m)

6. Verifica con la clave pública de Ana (la que trae el
   mensaje, con su key ID): σ^e mod n = H(m)?
   SÍ → "el mensaje es de Ana y está íntegro"
   NO → mensaje alterado o de otro emisor

7. Extrae la clave pública de Ana del mensaje mismo
   (va firmado con la subclave de firma de Ana).
```

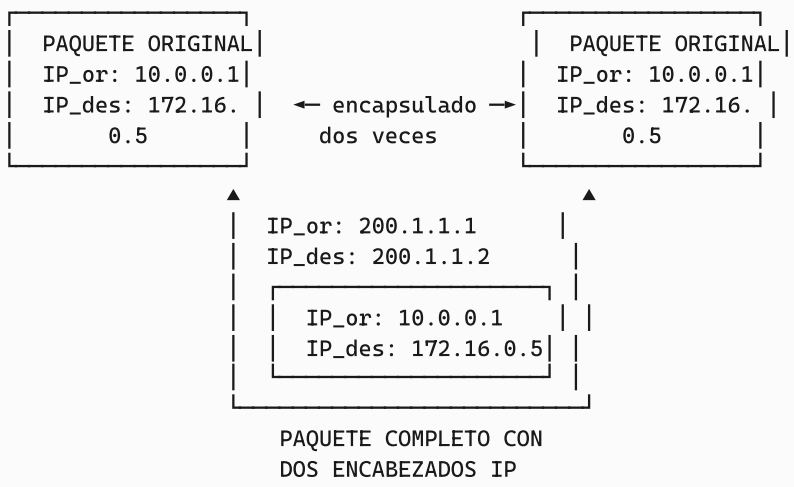
El problema de la web of trust (*gráfico de confianza*), que se pregunta:

```
Cada usuario tiene un key ID. Los usuarios se FIRMAN entre sí
(el "key signing party"):

    Ana ──firmada por──► Beto ──firmada por──► Carla
      ▲                                        ▲
      └────────────────────────────────────────┘
           Ana y Carla se confían indirectamente

★ Problema: ¿qué pasa si Ana firma la clave de Beto pero
  Beto es un atacante? Y la clave "modelo de confianza" de
  PGP: solo son válidas las firmas de claves que están en un
  keyserver confiable.

★ RESPUESTA MODERNA (2019+): el modelo de confianza de Web of Trust
  quedó OBSOLETO. Lo que se usa ahora es:
    · la llave pública se verifica FUERA DE BANDA (imprenta, QR,
      Signal Safety Numbers, servidor de correo conocido);
    · o la FIRMATURE por la cadena de CA (PDF/A-3 signatures,
      firma de código de paquete).
```

> **Y ojo con esto en 2026:** PGP (OpenPGP) sigue siendo un estándar, pero por problemas históricos de **confianza en el modelo de web of trust** y por el asunto **pgp4u/GnuPG** (2023), el mundo de la mensajería pasó a **Signal** (protocolo Signal, doble ratchet) o a **Matrix** (MLS). En un examen de 2026, si te preguntan sobre cifrado de mensajería, la respuesta moderna es **extremo a extremo con identidad fuera de banda**, no "confiá en la firma de la clave".

1.4 Túneles y VPN

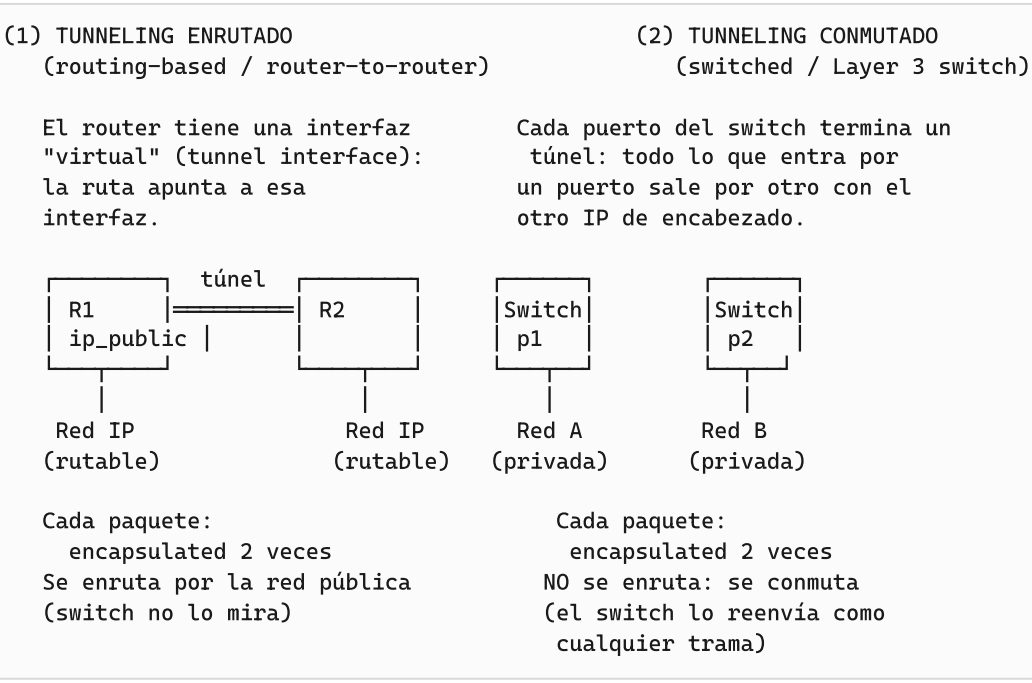
Túnel = **encapsular** un paquete dentro de otro para que viaje por una red que **no lo entiende**. Es una de las 4 técnicas de tunneling (RFC 1701):



El tunneling es "encapsulation con dos IP":

Técnica	Qué hace	Para qué
Generic Routing Encapsulation (GRE)	encapsula **cualquier** protocolo	transparent tunnelling
IP-in-IP (`RFC 2003`)	solo IP	si sabés que es IP
VXLAN	**Ethernet sobre IP** (toda la trama MAC)	**overlay en centros de datos** (el moderno)
WireGuard	cifrado moderno y simple	VPN de 2026

Los dos modos de VPN (esto es lo que se pregunta)



La diferencia conceptual clave:

	Enrutado	Conmutado
Información que mira el switch	**ninguna**: ve el IP externo y reenvía	ve **el Mac destino** y reenvía
Se usa cuando	las redes son **públicas y ruteables** (Internet)	las redes son **privadas** (no se enrutan por ningún lado)
Mismo protocolo en las dos redes	casi siempre **diferente** (IPX ↔ IP)	casi siempre **diferente**
Ejemplo típico	**IPsec en Internet**	**ATM LAN Emulation** (es histórico)

Y el doble encapsulado del que habla el Examen 2008-08 Problema 1:

```
Un paquete de la LAN A (172.16.1.0/24) para la LAN B (10.1.1.0/24):

┌──────────────────────────────────────────────┐
│ ENCABEZADO EXTERNO:                          │
│   IP_or  = 200.1.1.1   (R1, público)         │
│   IP_des = 200.1.1.2   (R2, público)         │
├──────────────────────────────────────────────┤
│ ENCABEZADO INTERNO:                          │
│   IP_or  = 172.16.1.5  (host en A)           │
│   IP_des = 10.1.1.3    (host en B)           │
├──────────────────────────────────────────────┤
│   TCP / datos de aplicación                  │
└──────────────────────────────────────────────┘

★ El switch de la red pública ve SOLO 200.1.1.1 → 200.1.1.2.
  El router R2 ve las dos cabeceras y quita la externa.
```

Y el VPN real de 2026 es WireGuard o IPsec:

```
WireGuard  → ~4 000 líneas de código (comparadas con ~500 000 de OpenVPN)
           → criptografía moderna y simple
           → sin negociación de cipher: los peers se preconfiguran

IPsec      → dos modos:
               TRANSPORT mode: se cifra SOLO el payload, las cabeceras IP
                               originales se mantienen visibles
               TUNNEL mode:   se encapsula el PAQUETE ENTERO (lo de arriba)
```

1.5 IDS, IPS y DPI

Los cuatro componentes

```
┌──────────────────────────────────────────────────────────┐
│  (1) FUENTES      tráfico de red, logs, eventos del host │
│                   (NetFlow, sFLOW, syslog)               │
├──────────────────────────────────────────────────────────┤
│  (2) SENSORES     equipos que analizan (NIDS, HIDS)      │
│                   patrones, firmas, heurísticas          │
├──────────────────────────────────────────────────────────┤
│  (3) MOTOR DE     compara contra las reglas;             │
│      ANÁLISIS     correlaciona eventos;                  │
│                   puntúa (IDS:Score)                     │
├──────────────────────────────────────────────────────────┤
│  (4) RESPESTA     el operador (IDS) o el equipo          │
│                   (IPS: bloquea automáticamente)         │
└──────────────────────────────────────────────────────────┘
```

IDS vs. IPS: la diferencia es una línea

```
IDS  (Intrusion DETECTION System)
─────────────────────────────────

Está MIRANDO. Detecta y AVISA.
El tráfico PASA IGUAL.

┌─────┐   ──trama sospechosa──►  [IDS]  ──►  🚨 ALERTA al operador
└─────┘                                          │
└─────┘   ──sigue igual──────►  [destino]     el operador decide

★ VENTAJAS:  no interrumpe el tráfico bueno; si el IDS falla,
  solo perdés visibilidad.
★ DEBILIDAD: el ataque PASA. Si el operador no mira, no pasa nada.

IPS  (Intrusion PREVENTION System)
──────────────────────────────────

Está EN el camino. Detecta y BLOQUEA.

┌─────┐   ──trama sospechosa──►  [IPS]  ──►  🛑 DESCARTADA
└─────┘

★ VENTAJAS:  frena el ataque en el momento.
★ DEBILIDAD: si el IPS se cuelga y se pone en modo "fail-open",
  pasa TODO. Si se pone en "fail-close", corta el servicio legítimo.
  Es un punto único de falla.
```

> **El compromiso real:** el IPS tiene que estar **en línea** (inline), y eso significa que es un **punto único de falla** y un **cuello de botella de rendimiento**. El IDS, en cambio, se puede poner en un **puerto espejo** (*span port*), que no toca el tráfico: ve una copia. **La respuesta completa al examen es esa: IDS en puerto espejo (seguro pero no bloquea), IPS en línea (bloquea pero es punto único de falla).**

Los dos criterios de decisión

Criterio	Question	Por qué importa
¿Punto?	¿Dónde está en la red?	NIDS (red), HIDS (host), WIDS (inalámbrico)
¿Cuándo?	¿Cómo detecta?	**por firma** (patrón conocido) vs. **por anomalía** (estadística)

Y el problema central de todo IDS: el falso positivo.

```
¿FALSO POSITIVO?   = alerta de un ataque que NO hay
¿FALSO NEGATIVO?   = un ataque que NO se detecta

El problema clásico: en un centro de cómputos grande hay
~10 000 eventos de seguridad POR DÍA. Si el 95 % son falsos
positivos, el operador deja de mirar en una semana.

★ El refinamiento: el "IDS Score" (eNSP) — puntúa cada evento
  por la probabilidad de ataque y lo ordena por prioridad.
  Así el operador mira los 10 más graves, no los 10 000.
```

Y los dos estilos de respuesta:

```
(a) INTERRUPTIVA  (active response)
    el IPS hace ALGO para frenar el ataque:
      · bloqueo del atacante
      · terminate la sesión TCP
    ★ RIESGO: convertir un ataque en un DoS propio.

(b) PASIVA        (passive response)
    el IPS solo registra y AVISA; que otro sea el que frene.
    ★ más seguro, pero más lento.
```

DPI: *Deep Packet Inspection*

```
IDS/IPS  compara el PAQUETE contra FIRMAS.
          "¿hay un 'EICAR' en esta conexión?"

DPI     ABRE el paquete, lo DECODIFICA, y lo interpreta
        como si fuera tráfico de aplicación.

        ┌──────────────────────────────────────────┐
        │  Capa 1-4:  IP  TCP/UDP                  │
        │  ┌──────────────────────────────────┐    │
        │  │ Capa 7:  HTTP GET /index.html    │    │
        │  │  ... el DPI lee la URL, los      │    │
        │  │  headers, incluso el TLS (con las │    │
        │  │  claves o en un proxy MITM)      │    │
        │  └──────────────────────────────────┘    │
        └──────────────────────────────────────────┘

★ Lo que permite:
  · policy de Application (permitir Facebook, bloquear
    un juego)
  · detección de malware
  · filtrado de spam
  · INSPECCIÓN DE CONTENIDO cifrado  ← acá está el
    problema gigante (clasificados vs. privacidad)
```

> ⚠️ **El problema de DPI con cifrado.** Hoy **todo** está cifrado (HTTPS), así que un DPI en el medio **no puede leer el contenido**. Para hacerlo: 1. **Proxy MITM** con las claves del servidor (que hoy **no se puede** con cifrado de extremo a extremo). 2. Instalar la **certificación de una CA propia** en cada dispositivo (esto es lo que hacen losINAfiltered de empresa y las redes de wireless). 3. Descifrado en el **endpoint** (instrumentación del navegador).
>
> Es la tensión central de la seguridad de red hoy: **confinarse el tráfico a la red ya no es posible sin romper el cifrado de extremo a extremo**. Y por eso muchos países (la UE con **ePrivacy**, el Network neutrality) lo han limitado con leyes.

1.6 Los ataques típicos y qué capa los detiene

Esta tabla es **el resumen más rentable de la clase**: si sabés decir qué capa corta cada ataque, entendiste seguridad.

Ataque	Capa donde vive	Qué lo detiene
Sniffing	2 / 3 / 7	Cifrado (TLS). Nunca confíes en el cable.
ARP spoofing (ARP poisoning)	2	DAI / port security / DHCP snooping / VLAN
IP spoofing	3	Ingress filtering (uRPF): el router descarta lo que llega por la interfaz equivocada (BCP 38)
Mac flooding	2	El table flooding: un puerto ve X MACs → se SHUTDOWN y se alerta
Secuestro DHCP (rogue DHCP server)	2	DHCP snooping + port security en los puertos de usuario (los puertos no pueden ser servidores)
Secuestro BGP	3	RPKI / ROA: validar que el prefijo está autorizado (ver §Actualización)
Secuestro DNS	3 / 7	DNSSEC: firma la respuesta con la clave de la zona
Rogue CA / MITM TLS	7	PKI + validación de SAN + CT logs. O sellado (pinning).
Ping of Death, Smurf, Teardrop DoS	3	Filtering de paquetes con TTL, longitud, flags. ACLs anti-spoof.
Secuestro de sesión (session hijacking)	4 / 7	TLS con safe renegotiation; HSTS; cookies HttpOnly+ Secure.
Pharming	7	DNSSEC + validación de certificados + HTTPS.

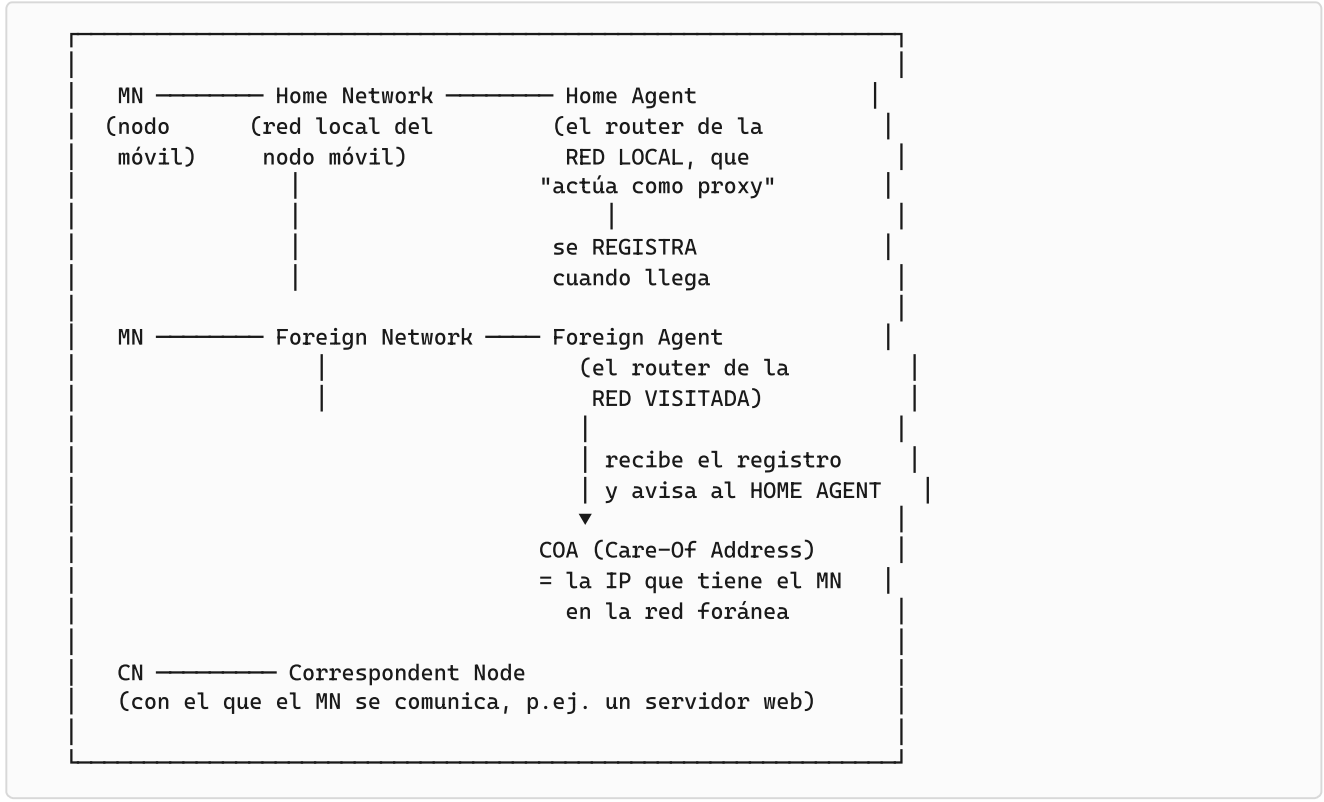
> **Lo importante: ningún mecanismo de capa 2 o 3 es una frontera de seguridad**. Todos son **defensas en profundidad**. La única defensa que **realmente** impide el MITM y el eavesdropping es la **criptografía de la capa de aplicación** (TLS), porque no depende de que el atacante esté en tu red.

1.7 IP Móvil

El problema

```
El nodo MÓVIL (MN) tiene una DIRECCIÓN PERMANENTE (home address = IP fija),
que pertenece a su RED LOCAL (home network).

Cuando se va a otra red (foreign network), cambia de IP.
PERO sus interlocutores (CN = correspondent node) siguen hablando con la
HOME ADDRESS.

El problema: los paquetes para la HOME ADDRESS van a la HOME NETWORK, y no
llegan al MN. El MN no recibe nada. Se rompe la conexión TCP.

★ La identidad del MN es su HOME ADDRESS. Y la identidad viaja con él
  en todos los sistemas. ¿Cómo se hace para que siga funcionando?
```

Los cuatro componentes (esto es EXACTAMENTE la respuesta al Examen 2008-12 P3)



La tabla de roles que hay que saber de memoria:

Componente	Qué es	Rol
MN (*Mobile Node*)	el host que se mueve	tiene dos direcciones: **permanente** (la de casa) y **care-of** (la temporal)
HA (*Home Agent*)	el router de la **red local**	hace de **proxy** del MN: guarda su ubicación y **reenvía** los paquetes que llegan a la home address
FA (*Foreign Agent*)	el router de la **red foránea**	recibe el **registro** del MN y le **notifica** al HA
CN (*Correspondent Node*)	el otro extremo (servidor, peer)	habla con la home address del MN

Y las dos direcciones del MN:

```
HOME ADDRESS (permanente):  172.16.0.5   ← siempre la misma
                                  ↕
                        SON DISTINTAS
                                  ↕
CARE-OF ADDRESS (temporal):  172.172.0.17  ← cambia con cada red
```

Los tres modos de operación

```
(1) MODO SIMPLE (sin foreign agent)
─────────────────────────────────────

El MNOBTIENE care-of address por DHPCS o SLAAC, y se REGISTRA
DIRECTAMENTE con el HA.

El MN ──REGISTRATION──► HA      "estoy en 172.172.0.17, TTL = 300 s"

┌──────────┐                      ┌──────────┐
│   MN     │──── registro ──────►│   HA     │
│ home:    │                      │          │
│ 172.16.0.5│                     │ guarda:  │
│ care-of: │◄── binding ack ─────│ MN@      │
│ 172.172. │                      │ COA      │
│   0.17   │                      └──────────┘
└──────────┘

★ El MN se mete solo en la foreign network.

(2) MODO CON FOREIGN AGENT  ← el del Examen 2011-02
──────────────────────────────────────────────────

El MN NO puede obtener una dirección: la pide al FA,
que le ASIGNA la care-of address.

┌──────────┐                      ┌──────────┐
│   MN     │──── registro ──────►│   FA     │
│          │◄── COA = 172.172.0.17│          │
└──────────┘                      └──────────┘
     │                                 │
     │                                 │ "el MN está acá, en 172.172.0.17"
     │                                 ▼
     │                           ┌──────────┐
     └──────────────────────────►│   HA     │
          (el paquete del MN     │          │
           pasa por el FA y      │ crea el  │
           llega al HA)          │ TÚNEL    │
                                 └──────────┘

★ El MN queda "dentro" del FA: usa la red interna del FA.

(3) MODO CORRESPONDENT  (CN actualizado)
──────────────────────────────────────

Se le AVISA al CN cuál es la care-of address del MN, para que el
CN mande directo al MN y no pase por el HA. Se evita el túnel.

★ Es la forma más eficiente pero la más compleja de implementar.
```

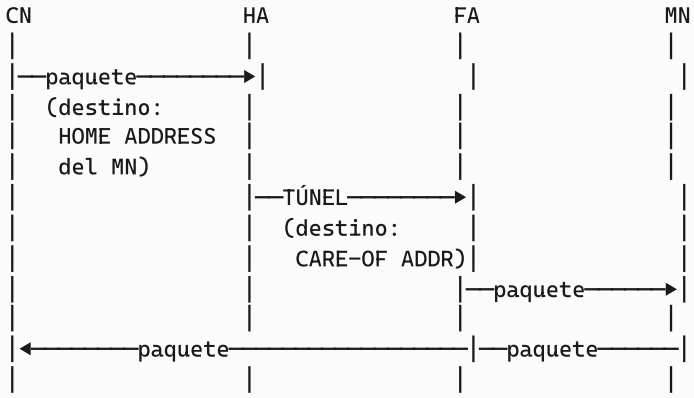
Los dos mensajes que hay que saber:

Mensaje	Quién → Quién	Contenido
Binding Update	MN → HA (o MN → FA → HA)	"mi care-of address es X, con TTL de 300 s"
Binding Acknowledgement	HA → MN	"recibí tu registro, vigente hasta las 15:34"

La ruta triangular — el problema conceptual central

Acá está **el 90 % de los puntos** de los problemas de IP Móvil.

```
ESCENARIO: MN está en la foreign network. El CN (servidor)
está en la INTERNET (ni en la home ni en la foreign).

¿Cómo llega un paquete del CN al MN?

── RUTA TRIANGULAR (lo que hace IP Móvil) ──
```




```
★ Los paquetes HACEN UN TRIÁNGULO: CN → HA → FA → MN, y de vuelta.
  El HA desvía el tráfico al túnel.

── ¿POR QUÉ ES NECESARIA? ──
Porque el CN no sabe la care-of address del MN.
Los paquetes del CN van a la HOME ADDRESS, y el HA es el
único que sabe dónde está el MN ahora.

── EL PROBLEMA: "routing triangular"  ──
El tráfico va y vuelve por la red INTERNA del operador.
Hay trTransit link entre HA y FA. Y el CN está en Internet.

★ El problema es que el tráfico de ida y el de vuelta PUEDEN
  tomar caminos DISTINTOS → ASIMETRÍA. Eso rompe algunos
  protocolos y confunde a los middleboxes.
```

> **La solución que da el Examen 2011-02** (y es lo que hay que escribir): *implementar un túnel entre el HA y el FA, de forma escalable: no modificando los routers intermedios (que sería no escalable) sino encapsulando los paquetes que el HA le manda al FA. O sea, cuando el HA reenvía al COA, en vez de mandar el paquete pelado, le pone un encabezado IP extra con origen* `IP_HA` *y destino* `IP_FA`*. Así:*
>
> - *El paquete del CN al MN viaja:* `[IP_or:CN, IP_des:HOME_MN]` *→ HA lo envuelve:* `[IP_or:HA, IP_des:FA][IP_or:CN, IP_des:HOME_MN]` *→ FA lo desenvuelve y lo entrega al MN.*
> - *El paquete del MN al CN viaja directo:* `[IP_or:COA_MN, IP_des:CN]`*. No pasa por el HA.*
>
> *Escalable porque requiere solo configurar los dos routers extremos (HA y FA), sin tocar los intermedios. Y sin túnel, el paquete del MN al CN haría el mismo camino dos veces, y el TTL lo descartaría.*

⚠️ Actualización a 2026 — por qué esto no se desplegó

Esta es una de las piezas de **reencuadre** más importantes del curso.

```
Mobile IPv4  (RFC 5944, 2010) ─┐
Mobile IPv6  (RFC 6275, 2011) ─┴─→ SIGUEN EN STANDARDS TRACK. NO SON OBSOLETOS.
                                   └→ PERO NO SE DESPLEGARON.

¿POR QUÉ NO?
1. COMPLEJIDAD: requiere modificar el MN, el HA y el CN. Cada operador
   tendría que soporte todo el stack. IPv6 va aún más lejos con la migración.
2. MIDDLEBOXES: NAT, firewalls y load balancers en el camino rompen el
   túnel porque "no entienden" la hipótesis del túnel.
3. CGN: en el mundo real hay CGNAT (Carrier-Grade NAT). El HA no puede
   ver al CN porque ambos están detrás de NAT.
4. ROAMING ENTRE TECNOLOGÍAS: pasar de 4G a 5G a WiFi a 5G rompe la
   dirección. IP Móvil asume una red, no una tecnología.

── ¿QUÉ SE USA EN 2026 EN SU LUGAR? ──
· VPN  (WireGuard, IPsec): simple, funciona, oculta todo.
· CGN + IPv6: la dirección ya no importa tanto.
· QUIC connection migration (RFC 9000 §9): la industria resolvió la
  movilidad EN LA CAPA DE TRANSPORTE, no en la capa 3.

★ EL CAMBIO DE PARADIGMA: el Connection ID (RFC 9000 §5.1) hace que
  la identidad de la conexión DEJE DE SER LA IP. Eso habilita:
    (a) migración de conexión (cambiás de WiFi a 5G sin cortar)
    (b) NAT rebinding sin romper (el NAT cambia el puerto, QUIC no se rompe)
    (c) balanceo en servidores (varios servidores, un Connection ID)
```

> **Para el examen:** si te piden IP Móvil, **respondé como en 2011** (el modelo con HA, FA, COA, ruta triangular es lo que se evalúa), **PERO agregá una línea al final** diciendo que en la práctica se reemplazó por VPN + QUIC migration, y **por qué**. Esa última línea es la que te distingue.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — El túnel doble del Examen 2008-08

Datos. Una empresa tiene dos sedes: LAN A = `172.16.1.0/24`, LAN B = `10.1.1.0/24`. Los routers R1 (200.1.1.1) y R2 (200.1.1.2) están unidos por Internet. Un host `H_A` (`172.16.1.5`) manda a `H_B` (`10.1.1.3`).

Resolución — los dos encabezados, en cada punto del camino:

```
═══ EN LA LAN A ═══
┌──────────────────────────────────────────────────────────┐
│ ENCABEZADO INTERNO (el del paquete original):            │
│   IP_or  = 172.16.1.5    IP_des = 10.1.1.3               │
│   MAC_or = MAC(H_A)       MAC_des = MAC(R1)              │
│   Protocolo = TCP, puerto 80                             │
└──────────────────────────────────────────────────────────┘

R1 lo encapsula:
┌──────────────────────────────────────────────────────────┐
│ ENCABEZADO EXTERNO (el del túnel):                       │
│   IP_or  = 200.1.1.1     IP_des = 200.1.1.2              │
│   Protocolo = 47 (GRE)                                   │
├──────────────────────────────────────────────────────────┤
│ ENCABEZADO INTERNO (el del paquete original):            │
│   IP_or  = 172.16.1.5    IP_des = 10.1.1.3               │
│   MAC_or = MAC(R1)        MAC_des = <next-hop en Internet>│
└──────────────────────────────────────────────────────────┘

═══ EN INTERNET ═══
Los routers de Internet MIRAN SOLO el encabezado externo:
200.1.1.1 → 200.1.1.2. No saben ni les importa qué hay adentro.

═══ EN LA LAN B ═══
R2 desenvuelve, quita el externo, y reencola:
┌──────────────────────────────────────────────────────────┐
│ ENCABEZADO INTERNO:                                      │
│   IP_or  = 172.16.1.5    IP_des = 10.1.1.3               │
│   MAC_or = MAC(R2)        MAC_des = MAC(H_B)             │
└──────────────────────────────────────────────────────────┘
```

Lo que muestra este ejercicio: la **dirección MAC cambia en CADA salto** (porque es de capa 2), pero la **dirección IP origen/destino interna NO cambia nunca** durante todo el túnel. El túnel **no es NAT**: no traduce, solo encapsula.

2.2 Ejercicio 2 — Calcular qué capa corta cada ataque

Enunciado. Una red corporativa tiene: switches gestionables, un router de borde con ACLs, un firewall con IPS, DNS interno, y HTTPS obligatorio por política. ¿Qué ataques quedan contenidos y por qué?

Resolución — la tabla de decisión:

Ataque	¿Lo corta?	Por qué
Sniffing	✅	HTTPS obligatorio: todo el tráfico útil va cifrado. El cable es público.
ARP poisoning	✅	port security en los puertos de acceso +DAI: si una MAC aparece en dos puertos, se bloquea el puerto.
Rogue DHCP	✅	DHCP snooping: solo los puertos "confiables" (los de los uplinks) pueden ser servidores DHCP.
IP spoofing	✅	ACL anti-spoof en el borde + uRPF: `permit ip 172.16.0.0 0.0.0.255 any` en la interfaz interna.
Secuestro BGP	✅	RPKI: se valida el ROA del prefijo antes de aceptarlo.
Secuestro DNS	✅	DNS interno + DNSSEC hacia afuera + validación de SAN en HTTPS: si el DNS devuelve una IP de otro, el certificado no matchea y el navegador aborta.
Rogue CA / MITM TLS	✅	HTTPS obligatorio + **no instalar CAs** + CT logs. ★ Si el admin de la empresa instala una CA propia, eso es un backdoor.
Ping of Death	✅	el borde filtra paquetes con longitud o flags inválidos.
Denegación de servicio (volumen)	❌	**NINGÚN firewall lo detiene.** Si alguien te manda 100 Gbps, no hay defensa local. Se necesita upstream (anti-DDoS,anycast).

> **La fila que hay que subrayar:** *la última. Un firewall no detiene un ataque de denegación de servicio por volumen, porque para hacerlo tendría que ser más rápido que el atacante. Eso se resuelve aguas arriba, en el proveedor de tránsito, con scrubbing o anycast. Un IDS/IPS es una defensa de escritorio, no una defensa de infraestructura.***

2.3 Ejercicio 3 — Por qué un hash no da confidencialidad (demostración)

Enunciado. Beto tiene un hash de una contraseña. ¿Qué puede hacer un atacante?

Resolución:

```
Sistema:   H(password) = SHA-256(password), almacenado.
──────────────────────────────────────────────────────────

SI el atacante tiene SOLO el hash:
  → PUEDE probar contraseñas contra el hash (ataque de
    diccionario). PEOR que con texto plano, porque
    no puede recuperar la original, pero SÍ puede
    verificar si una candidata es correcta.

SI el atacante tiene el hash Y la contraseña:
  → el hash NO le dice cuál es la contraseña. Pero puede
    USAR EL HASH como si fuera la contraseña:
      · muchos sistemas comparan el hash del hash
        (el login hash H(password); si H' == H'
        comparando hashes, sirve);
      · sistemas con HASH COMO TOKEn: mandás el hash
        como contraseña y entrás.
    ★ Esto se llama "pass-the-hash".

SI la contraseña es corta o común:
  → el atacante la saca en segundos (GPU: 10^10/s con SHA-256).

── LO QUE LO SOLUCIONA ──
· SALT (salt): un valor aleatorio por usuario, concatenado
  antes del hash. H(password || salt). Hace que dos
  contraseñas iguales den hashes distintos.
· KDF LENTO: bcrypt / scrypt / Argon2. 100 000× más lento
  que SHA-256. Una GPU que hace 10^10 SHA-256/s hace
  10^5 bcrypt/s: la diferencia ES la defensa.
```

> **Este ejercicio conecta con el WPA2 de la clase 43:** WPA2 con PSK suffers justamente del problema del (2): la contraseña se **deriva** y se puede atacar por **diccionario offline**. Y WPA3 con **SAE** resuelve **precisamente** eso: el protocolo de Dragonfly hace que **no se pueda verificar una contraseña candidata offline** a partir del handshake capturado. Es el mismo problema, resuelto con un protocolo en lugar de una KDF más lenta.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-02, Problema 2 (30 puntos) — IP Móvil

> **Fuente:** Examen `sirc1102`, 3 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1102.pdf

Enunciado (resumen del original)

Se tiene un nodo móvil (MN) que se mueve entre WLAN1+LAN1 y WLAN2+LAN2. El CN está en LAN3. Se pide: (a) los campos MAC/IP de cada segmento cuando el paquete va del MN al CN; (b) lo mismo cuando va del CN al MN, en cada modo de operación; y (c) qué pasa con los campos IP según el modo.

Resolución — parte a) y b) en tabla

Caso A: MN en WLAN1/LAN1, todo en su home network.

```
MN ──[WLAN1]── AP ──[LAN1]── R1 ──[LAN3]── CN
(MN es      (bridging)        (rutea)
 local)
```

Segmento	MAC origen	MAC destino	IP origen	IP destino
WLAN1	**MAC1** (MN)	**MAC4** (AP)	172.172.0.17	203.203.203.3
LAN1	**MAC1** (MN)	**MAC2** (R1)	172.172.0.17	203.203.203.3
LAN3	**MAC8** (R1)	**MAC9** (CN)	172.172.0.17	203.203.203.3

Lo que muestra la tabla: la **IP origen/destino no cambia en ningún salto** (es un paquete normal, MN y CN están "directamente visibles"). Lo único que cambia es la **MAC**, en cada router (es de capa 2). En WLAN1 la MAC destino es la del **AP** (porque la trama es **inalámbrica** y el primer salto es el AP). En LAN1 es la de **R1**.

Caso B: MN en WLAN2/LAN2, con foreign agent.

Segmento	MAC origen	MAC destino	IP origen	IP destino
WLAN2	MAC1 (MN)	**MAC7** (FA)	172.172.0.17	203.203.203.3
LAN2	MAC1 (MN)	MAC5 (router)	172.172.0.17	203.203.203.3
LAN3	MAC8 (R)	MAC9 (CN)	172.172.0.17	203.203.203.3

Ojo con la MAC destino del primer tramo: es la del **foreign agent**, porque el MN entra a la red foránea a través del FA. Y el CN **nunca** cambia su IP destino: sigue siendo la **home address** del MN, porque el MN "se hace pasar" por estar en casa.

Resolución — parte c) y ii) el túnel

El problema (que el enunciado pide resolver): cuando el MN está en la foreign network y el CN quiere mandarle un paquete, lo manda a la **home address**. Ese paquete viaja por Internet hasta el **HA**. El HA ve que el MN está registrado con COA = `172.172.0.17`, así que debería reenviarlo ahí.

El problema: si el HA lo reenvía **sin túnel**, el paquete sale por la red pública con destino `172.172.0.17` (una IP **privada**). Los routers de Internet **no pueden rutear una IP privada** y **descartan** el paquete. O peor: el **next hop** lo devuelve, y el paquete vuelve al HA: **un loop**, hasta que se descarta por TTL.

La solución: el túnel HA → FA.

```
El HA, en vez de mandar el paquete pelado al COA, lo ENCAPSULA:

┌──────────────────────────────────────────────────────┐
│ ENCABEZADO DEL TÚNEL:                                │
│   IP_or  = 111.1.1.1   (dirección PÚBLICA del HA)    │
│   IP_des = 222.1.1.1   (dirección PÚBLICA del FA)    │
├──────────────────────────────────────────────────────┤
│ ENCABEZADO INTERIOR (el paquete del CN):             │
│   IP_or  = 203.203.203.3   (el CN)                   │
│   IP_des = 172.172.0.17    (home address del MN)     │
└──────────────────────────────────────────────────────┘

★ Ahora los routers de Internet PUEDEN rutear: 111.1.1.1 → 222.1.1.1
  son dos IPs públicas.
★ El FA desenvuelve y le entrega al MN (que está en su LAN interna).
```

Y lo que dice la solución oficial sobre por qué el túnel no es "escalable":

> *"...y es posible y escalable pues sólo requiere la configuración de dos routers."*

El razonamiento completo:

- Si en vez de túnel se quisiera "arreglar" el problema con **rutas estáticas** en todos los routers intermedios, haría falta configurar **cada router de la red del operador**, uno por uno. Eso es **NO escalable**: con 500 MN hay que mantener 500 rutas en cada router, y cada vez que un MN se mueve hay que actualizar todas.
- Con el **túnel**, el problema se resuelve en **los dos extremos** (HA y FA). Los routers intermedios no necesitan saber NADA: solo reenvían según su tabla, y la tabla dice "de 111.1.1.1 a 222.1.1.1, siguiente salto...". Eso ya existe porque es una conexión común entre dos puntos.

Y los casos de la parte c), en tabla:

	MN → CN	CN → MN
Caso A (en home)	IP_or = 172.172.0.17 IP_des = 203.203.203.3 (normal, sin túnel)	IP_or = 203.203.203.3 IP_des = 172.172.0.17 (normal, sin túnel)
Caso B (en foreign, route triangular)	IP_or = 172.172.0.17 IP_des = 203.203.203.3 (sale directo a Internet, NO pasa por HA)	1er tramo: IP_or = 203.203.203.3 IP_des = 172.172.0.17 (va al HA) 2do tramo (túnel): IP_or = 111.1.1.1 (HA) IP_des = 222.1.1.1 (FA) encapsula: CN → MN

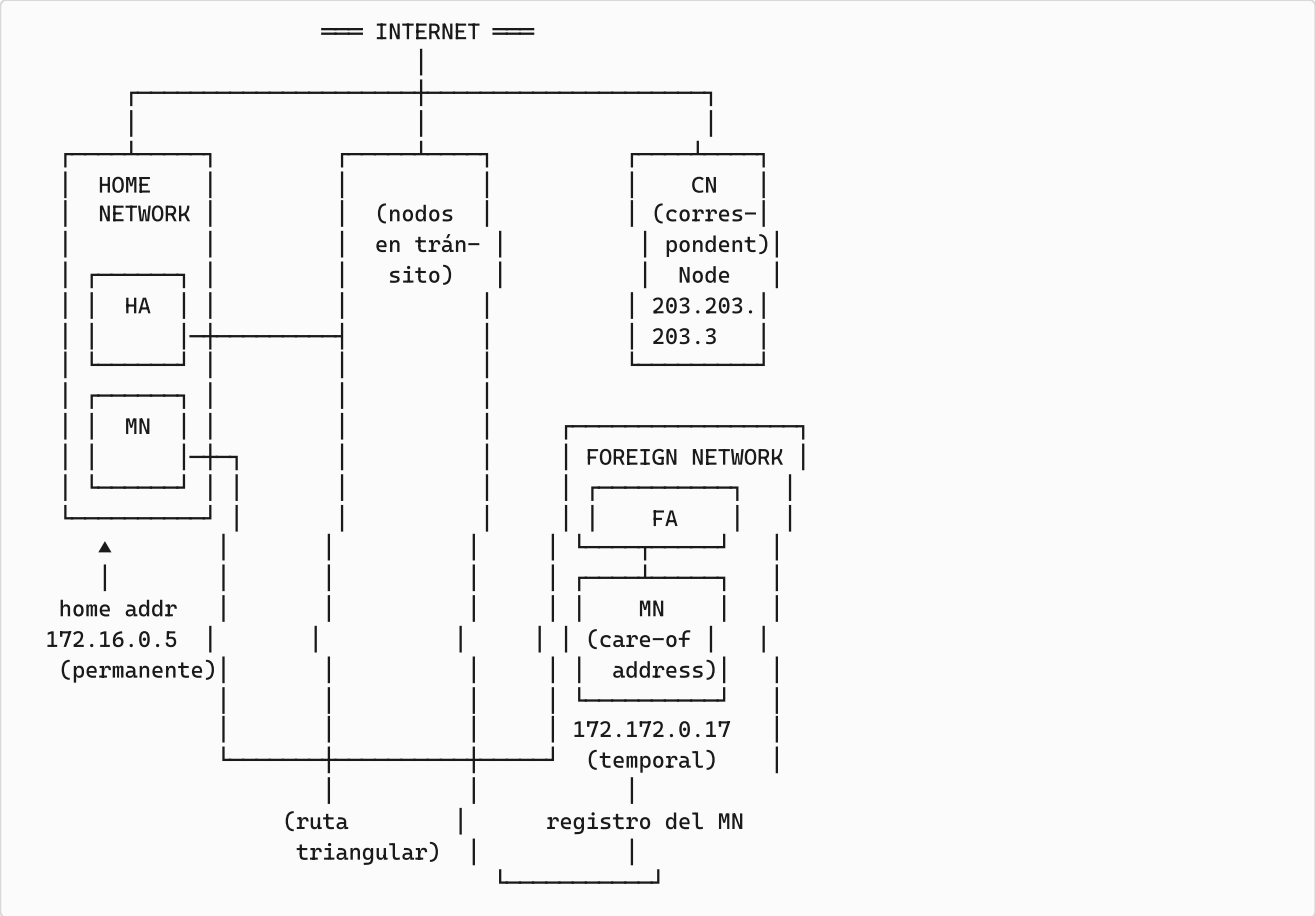
> **El detalle conceptual que hay que decir explícitamente:** el paquete **MN → CN** en el caso B **no pasa por el HA**. Sale directo del MN a Internet con destino el CN. Esto se llama **ruta triangular**: CN → HA → FA → MN (ida) y MN → CN directo (vuelta). **La asimetría es el problema**: el mismo paquete de ida y vuelta puede tomar caminos distintos, lo cual rompe la idea de "extremo a extremo". **Por eso existe el modo "correspondent"**: notificarle al CN la COA para que los dos caminos sean directos.

3.2 Examen 2008-12, Pregunta 3 (10 puntos) — arquitectura de IP Móvil

> **Texto del examen:** *Muestre e identifique en un diagrama topológico los componentes principales de la arquitectura IP Móvil, indicando el rol de cada uno de ellos.*

Resolución — el diagrama que hay que dibujar (10 puntos, ~2 por elemento)

El diagrama completo, que es lo que hay que hacer en el papel:



La tabla de roles (5 elementos × 2 puntos):

Componente	Rol	En una frase
MN (*Mobile Node*)	el nodo que se mueve	tiene una **dirección permanente** (home) y una **temporal** (care-of). Se **registra** ante el HA y migra su dirección actual.
HA (*Home Agent*)	el router de la **red local** del MN	hace de **proxy**: recibe los paquetes destinados a la home address del MN y los **desvía** a través del túnel hacia el COA.
FA (*Foreign Agent*)	el router de la **red foránea**	**recibe** el registro del MN, le **asigna** (o le presta) la **care-of address**, y **notifica** al HA.
CN (*Correspondent Node*)	el otro extremo de la comunicación	habla con la **home address** del MN; en el modo "correspondent" se le **notifica** el COA para que el camino sea directo.
Túnel	el camino entre HA y FA	encapsula el tráfico del CN para que llegue al MN a través de la red del operador, **sin modificar los routers intermedios**.

Y los puntos que la cátedra suele agregar (suman):

1. **El MN tiene DOS direcciones y eso es lo esencial**: la **permanente** (home address, asignada por la home network, no cambia nunca) y la **temporal** (care-of address, cambia cada vez que el MN cambia de red). La home address es la que "usa" el CN; el COA es la que "ve" la red.
2. **El binding tiene un TTL** (típicamente 300 s). El MN lo renueva periódicamente. Si no renueva, el HA **deja de desviar** y el MN tiene que re-registrarse. **Es lo mismo que el lease de DHCP**, aplicado a la mobilidade.
3. **Los tres modos** (simple, con foreign agent, correspondent) y que el "correspondent" es el único que elimina la ruta triangular pero es el más complejo.

3.3 Examen 2007-07, Pregunta 5 (5 puntos) — funcionamiento de IP Móvil

Resolución. Es la versión corta de §3.2, en cinco puntos:

1. El MN **sale** de su red local con su home address y **no se registra** con nadie todavía: simplemente se mueve.
2. Cuando llega a una red foránea, **obtiene una care-of address** (por DHCP, por SLAAC en IPv6, o del foreign agent si lo hay).
3. El MN **manda un Binding Update al HA**: "mi care-of address es X, con TTL de 300 s". Si hay un FA en el medio, el Update pasa por él.
4. El HA **recibe, registra y acusa** (Binding Acknowledgement). **Desde este momento**, el HA sabe dónde está el MN.
5. Cuando llega un paquete del CN con destino = home address del MN, el HA lo intercepta (está en su red local), lo **encapsula en un túnel hacia el FA** (con destino = COA), y el FA lo entrega al MN.

> **Tope para 5 puntos:** los 5 pasos, con los nombres de los mensajes (Binding Update, Binding Ack, COA, home address) y quién se registra con quién.

PARTE 4 — CIERRE DEL CURSO

📌 Resumen del Bloque F y del curso entero en 10 líneas

1. **TLS** = cifrado + integridad + autenticación + **forward secrecy**, con handshake de **1 RTT** en TLS 1.3. **443/TCP**. Cifra con **clave de sesión (EC)DH**, autentica con el **certificado**.
2. **SSH** = el mismo esquema con **PFS por claves efímeras**, **Ed25519** por defecto, y canal de control **multiplexado**.
3. **PGP** = cifrado **extremo a extremo** con **clave maestra separada** de la de cifrado; clave pública **verificada fuera de banda**.
4. **Túnel** = encapsular con **dos IP**, así el paquete cruza una red que no lo entiende. **Enrutado** (routers de Internet miran solo la IP externa) vs. **conmutado** (el switch mira MACs).
5. **IDS** avisa (en puerto espejo, no interrumpe); **IPS** bloquea (en línea, es punto único de falla). **DPI** abre y decodifica, y con cifrado requiere MITM o CA instalada.
6. **Ningún mecanismo de capa 2/3 es frontera de seguridad** — la única defensa real contra MITM es la **criptografía de la capa 7**.
7. **IP Móvil**: MN, HA, FA, CN; home address + care-of address; el HA hace de **proxy** y desvía por un **tunnel** hacia el FA; **ruta triangular** es el problema central y el túnel lo resuelve **sin tocar los routers intermedios**.
8. En 2026, **IP Móvil no se desplegó**: lo reemplazaron **VPN + QUIC migration** + el **Connection ID**, que saca la identidad de la IP.
9. **Punto de corte por capa**: capa 2 → port security/DAI/DHCP snooping; capa 3 → ACL/uRPF/RPKI; capa 7 → **TLS** (la única real).
10. **La idea que cierra el curso**: cada capa resuelve un problema y crea otro; la seguridad de una red es la **suma de las defensas**, y la única que no se puede rodear es la que **no depende de la red**: la criptografía de extremo a extremo.

🔑 Mapa final del curso — dónde está cada cosa

```
┌──────────────────────────────────────────────────────────┐
│ BLOQUE A — Fundamentos (1–7)                             │
│   qué es una red · OSI/TCP-IP · encapsulación ·          │
│   store-and-forward · retardo · Nyquist-Shannon          │
├──────────────────────────────────────────────────────────┤
│ BLOQUE B — Aplicación (8–15)                             │
│   HTTP/1.1 · DNS · correo · sockets · DHCP               │
├──────────────────────────────────────────────────────────┤
│ BLOQUE C — Transporte (16–23)                            │
│   UDP · TCP cabecera · handshake · confiabilidad ·       │
│   flujo vs. congestión · ECN                             │
├──────────────────────────────────────────────────────────┤
│ BLOQUE D — Red (24–37)                                   │
│   forwarding/routing · datagramas vs. VC · IPv4 ·        │
│   subnetting · VLSM · tablas/ARP · NAT/PAT ·             │
│   distance-vector · link-state/OSPF · BGP · MTU · IPv6   │
├──────────────────────────────────────────────────────────┤
│ BLOQUE E — Enlace (38–43)                                │
│   framing · errores (detección/corrección/ARQ) ·         │
│   acceso al medio · switch/VLAN/STP · WiFi 802.11        │
├──────────────────────────────────────────────────────────┤
│ BLOQUE F — Servicios (44–48)                             │
│   multimedia/RTP · QoS/MPLS · SNMP/NETCONF ·             │
│   criptografía · seguridad aplicada + IP Móvil           │
└──────────────────────────────────────────────────────────┘

Y LAS CONEXIONES QUE HAY QUE TENER PRESENTES:

· TCP  ──────→  congestion → bufferbloat → QOS (clase 45)
· ARP  ──────→  switch → VLAN → STP → wireless (38–43)
· subnetting ──→  VLSM → planes de numeración → BGP (26–34)
· DNS  ──────→  caché → seguridad (DNSSEC) → Internet (10–13, 47)
· sockets ────→  servidor/cliente → DHCP → HTTP → correo (14–15)
· hash  ──────→  firma → PKI → TLS → SSH/PGP (47–48)
· mobility ──→  QUIC → connection ID → NAT rebinding (48)
```

📝 SIMULACRO FINAL — Examen completo

> **Instrucciones del examen real:** 3 horas · sin material · individual · **puntaje mínimo de aprobación: 60/100 · obligatorio responder correctamente al menos 15 puntos en las preguntas teóricas. 5 preguntas teóricas (~40 pts) + 2 problemas prácticos (30 pts c/u).**

Preguntas teóricas

Teórica 1 (10 puntos). Un usuario en una red con WiFi abre el navegador y escribe `https://www.ejemplo.com`. Describa, en orden, **todos** los pasos desde que aprieta Enter: qué protocolos participan, qué se resuelve primero, qué cifrado se establece, y qué pasa si alguno de ellos falla. Sea específico sobre los **mensajes** de cada protocolo.

Teórica 2 (10 puntos). Describa cómo se **enruta** un paquete entre dos redes distintas: (a) el papel de forwarding y enrutamiento, (b) por qué la tabla ARP es necesaria en cada salto, (c) qué pasa si la tabla ARP no

tiene la entrada destino, y (d) qué diferencia hay entre enrutamiento **distance-vector** y **link-state**.

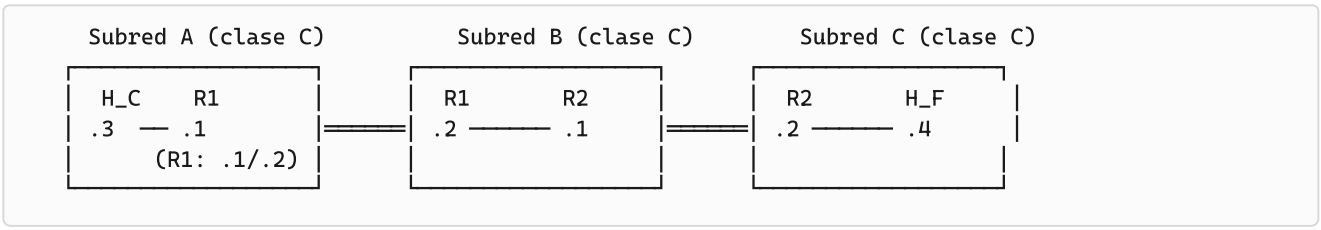
Teórica 3 (8 puntos). Explique cómo se implementa la **confiabilidad** en la capa de transporte, y por qué **no se puede implementar igual** en la capa de enlace en una red de paquetes arbitraria. Mencione los mecanismos concretos de TCP.

Teórica 4 (8 puntos). Explique qué es una **función de hash**, qué servicios de seguridad aporta, y por qué **una firma digital necesita combinar** un hash con un cifrado **asimétrico** (y no puede usar solo un hash, ni solo un MAC).

Teórica 5 (4 puntos). Mencione **cuatro ataques** típicos a redes de computadoras y, para cada uno, indique **en qué capa** se produce y **qué mecanismo de defensa** lo mitiga.

Problemas prácticos

Problema 1 (30 puntos). Una red está compuesta por **tres subredes** interconectadas por los routers **R1** y **R2**, tal como se muestra en la figura:



Las tres subredes son clase C. Los routers R1 y R2 tienen **dos interfaces** cada uno (una por subred). El enrutamiento es **estático**. Las direcciones MAC se indican en la tabla del enunciado original.

Se pide:

a) Para la topología: i) Las **tablas de enrutamiento** de R1 y R2 que dan conectividad extremo a extremo. ii) Las **tablas de enrutamiento de los hosts**. iii) El **intercambio de tramas** para mandar un paquete de H_C a H_F, indicando `{MAC_or, MAC_des, IP_or, IP_des, protocolo, tipo}` en **cada** salto, **suponiendo las tablas ARP vacías**.

b) Se reemplaza el **R2 por un switch** que une las subredes B y C, sin cambiar las direcciones IP. i) La nueva tabla de enrutamiento de R1, **justificando** las modificaciones. ii) Las nuevas tablas de los hosts. Qué cambios de configuración hacen falta. iii) **Comparar** la nueva topología con la original: ventajas y desventajas.

Problema 2 (30 puntos). Se desea implementar un **protocolo de transferencia de datos confiable** en una red donde **todos los nodos son alcanzables por broadcast**, con estas características:

- Debe poder transferir datos **simultáneamente** de un computador a todos los demás.
- En cada momento, la capa de aplicación debe haber recibido los datos de los **mismos** `n` **paquetes**, pudiendo diferir solo en los del paquete `n+1` (**entrega ordenada**).
- Debe recuperarse de fallos de transmisión y eliminar duplicados.
- **No** tendrá control de congestión.
- **No** tendrá control de flujo.

Primitivas disponibles: `rdt_send(datos)`, `udt_send_broadcast(pkt)`, `rdt_rcv(pkt)`, `deliver_data(datos)`.

Se pide:

a) Diseñe el protocolo: qué campos lleva el **encabezado** de cada paquete y por qué (**justifique cada uno**).

b) Implemente el procedimiento de **emisión** del emisor y de **recepción** del receptor, en pseudocódigo.

c) Explique cómo se garantiza la **entrega ordenada** y la **eliminación de duplicados** en cada caso: **pérdida de un ACK**, **duplicado de un paquete de datos**, y **pérdida del propio paquete de datos**.

Cronómetro sugerido

Momento	Minutos	Qué hacer
0:00 – 0:10	10	leer TODO, elegir qué problemas hacer, decidir si hacés los 2
0:10 – 0:55	45	**las 5 teóricas** (8-10 min cada una)
0:55 – 1:55	60	**Problema 1**
1:55 – 2:55	60	**Problema 2**
2:55 – 3:00	5	releer, corregir, completar cotas

> **Recordá:** el mínimo es **60/100 Y ≥15 pts en las teóricas**. Si te quedás sin tiempo para los problemas, todavía aprobás: **no abandones las teóricas**.

▶ ✅ **RESPUESTAS DEL SIMULACRO FINAL**

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Qué clave cifra los datos en TLS 1.3? ¿Y para qué sirve la clave que va en el certificado?

▶ Respuesta

2. ¿Por qué un IDS y un IPS son decisiones distintas y no se pueden sustituir?

▶ Respuesta

3. ¿Por qué el túnel de IP Móvil es escalable y las rutas estáticas en los routers intermedios no?

▶ Respuesta

4. ¿Qué es el *forward secrecy* y por qué en TLS se logra con (EC)DHE y no con RSA?

▶ Respuesta

5. ¿Qué diferencia hay entre sniffing y ARP poisoning en términos de capa y de defensa?

▶ Respuesta

6. ¿Por qué la IP Móvil (MIPv4/MIPv6) no se desplegó, si sigue en Standards Track?

▶ Respuesta

7. ¿Qué es DPI y por qué tiene un problema estructural hoy?

▶ Respuesta

8. Un switch tiene *port security* y una MAC puede aprenderse solo una vez. ¿Qué pasa si un atacante hace flooding de MACs?

▶ Respuesta

🔗 Referencias

- **Examen** `sirc1102` — Problema 2 completo (30 pts: IP Móvil, campos por segmento, túnel)
- **Examen** `sirc0812` — Pregunta 3 (10 pts: arquitectura de IP Móvil) y **Problema 1** (30 pts: NAT con balanceo de carga)
- **Examen** `sirc0707` — Pregunta 5 (5 pts: funcionamiento de IP Móvil)
- **Examen** `sirc0907` — Problema 1 (30 pts: ruteo sobre una malla a nivel de enlace)
- **Examen** `sirc0912` — Pregunta 2 (10 pts: confiabilidad en transporte vs. enlace)
- **Examen** `sirc0902` — Problema 1 (9 pts: TLS)
- **Examen** `sirc1002` — Pregunta 4 (8 pts: firma digital)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 8 (seguridad) y cap. 6 (movilidad)
- `RFC 4303` — IPsec · `RFC 6347` — VXLAN · `RFC 2890` — encapsulación IP · `RFC 5944` — Mobile IPv4 · `RFC 6275` — Mobile IPv6 · `RFC 6084` — DNS-Based Discovery
- `ACTUALIZACIONES.md` **§10** — el reencuadre de IP Móvil · **§13** — anclas de confianza por ámbito
- `PLAN-48-CLASES.md` — el plan completo de las 48 clases

🎓 Cierre del curso

Has llegado al final de las **48 clases**. Este es el mapa de lo que viste:

- **Bloque A** (1-7): los conceptos que todo lo demás asume — qué es una red, capas, encapsulación, y por qué el retardo es el problema central.
- **Bloque B** (8-15): las aplicaciones — cómo se ve una red desde arriba (HTTP, DNS, correo, sockets, DHCP).
- **Bloque C** (16-23): el transporte — UDP y TCP, y toda la maquinaria de confiabilidad y congestión.
- **Bloque D** (24-37): la red — direccionamiento, forwarding, enrutamiento, y los protocolos que lo hacen posible.
- **Bloque E** (38-43): el enlace — tramas, errores, acceso al medio, switch y WiFi.
- **Bloque F** (44-48): los servicios — multimedia, calidad, gestión y seguridad.

Y la idea que atraviesa el curso entero: cada capa resuelve un problema de la capa de abajo y crea uno nuevo. El **store-and-forward** crea el retardo; el **retardo** crea la necesidad de **control de congestión**; la **congestión** crea la necesidad de **QoS**; el **cableado** de abajo crea la necesidad de **cifrado** de arriba. **Una red es una cadena de compromisos, y entender de dónde viene cada uno es entender la red.**

Mucha suerte en el examen. Acordate de una cosa: **3 horas, 60 puntos, y 15 en teoría son obligatorios.** Las teóricas valen 40 de 100: si las hacés bien, tenés la mitad del examen asegurado.